# Tcl/Tk 9.0

Handbuchseiten (deutsch)

2026-10-26

# Inhalt

**Tcl 8**
Tcl 8
abstract 12
after 13
append 15
apply 15
array 17
bgerror 20
binary 21
break 34
buildinfo 35
callback 36
catch 38
cd 39
chan 40
class 59
classvariable 62
clock 63
close 75
concat 76
configurable 77
const 82
continue 83
cookiejar 84
copy 87
coroutine 88
dde 93
define 95
dict 108
encoding 113
eof 116
error 116
eval 117
exec 118
exit 125
expr 126
fblocked 132
fconfigure 132
fcopy 133
file 136
fileevent 144
filename 144
flush 147
for 147

foreach 148
format 149
fpclassify 153
gets 154
glob 154
global 157
history 157
http 159
idna 183
if 184
incr 185
info 186
interp 196
join 208
lappend 209
lassign 209
ledit 210
library 211
lindex 218
link 220
linsert 222
list 223
llength 223
lmap 224
load 225
lpop 228
lrange 229
lremove 230
lrepeat 231
lreplace 231
lreverse 233
lsearch 233
lseq 236
lset 238
lsort 240
mathfunc 243
mathop 248
memory 253
msgcat 254
my 266
namespace 268
next 280
object 284
open 286
package 294
packagens 298

pid 299
pkgMkIndex 299
platform 302
platform_shell 303
prefix 304
proc 305
process 307
puts 309
pwd 310
re_syntax 310
read 321
refchan 321
regexp 328
registry 330
regsub 333
rename 336
return 337
safe 340
scan 348
seek 351
self 352
set 354
singleton 355
socket 356
source 360
split 361
string 362
subst 367
switch 369
tailcall 370
tclsh 371
tcltest 374
tclvars 390
tell 396
throw 397
time 397
timerate 398
tm 400
trace 404
transchan 409
try 412
unknown 413
unload 415
unset 417
update 418
uplevel 419

upvar 420
variable 422
vwait 424
while 429
zipfs 430
zlib 435
**Tk 441**
bell 441
bind 442
bindtags 454
bitmap 455
busy 456
button 460
canvas 464
checkbutton 487
chooseColor 491
chooseDirectory 492
clipboard 493
colors 495
console 514
cursors 517
destroy 520
dialog 521
entry 522
event 530
focus 539
focusNext 541
font 541
fontchooser 547
frame 550
getOpenFile 552
grab 555
grid 557
image 563
keysyms 565
label 600
labelframe 602
listbox 605
loadTk 613
lower 614
menu 615
menubutton 627
message 629
messageBox 631
nsimage 632
option 634

optionMenu 636
options 637
pack 643
palette 648
panedwindow 649
photo 654
place 664
popup 667
print 668
radiobutton 669
raise 673
scale 673
scrollbar 677
selection 682
send 685
spinbox 687
sysnotify 696
systray 697
text 698
tk 720
tk_mac 722
tkerror 726
tkvars 726
tkwait 728
toplevel 729
winfo 731
wish 736
wm 740
**Ttk 752**
ttk_button 752
ttk_checkbutton 754
ttk_combobox 755
ttk_entry 758
ttk_frame 765
ttk_image 766
ttk_intro 767
ttk_label 769
ttk_labelframe 770
ttk_menubutton 772
ttk_notebook 773
ttk_panedwindow 778
ttk_progressbar 780
ttk_radiobutton 782
ttk_scale 783
ttk_scrollbar 785
ttk_separator 788

ttk_sizegrip 788
ttk_spinbox 789
ttk_style 792
ttk_treeview 794
ttk_vsapi 804
ttk_widget 806
**Lizenz 811**
Lizenzbedingungen Tcl 811
Lizenzbedingungen Tk 812

# Tcl

## Tcl

*Stichwörter:* Tcl, backslash, command, comment, script, substitution, variable

### Name

Tcl - Tool Command Language

### Übersicht

Zusammenfassung der Tcl-Sprachsyntax.

### Beschreibung

#### Syntax

Die folgenden Regeln definieren Syntax und Semantik der Tcl-Sprache:

**[1] Befehle.**

Ein Tcl-Skript ist eine Zeichenkette mit einem oder mehreren Befehlen. Semikolons und Zeilenumbrüche sind Befehlstrenner, sofern sie nicht wie unten beschrieben in Anführungszeichen stehen. Schließende eckige Klammern sind während der Befehlssubstitution (siehe unten) Befehlsablaufmarker, sofern sie nicht in Anführungszeichen stehen.

**[2] Auswertung.**

Ein Befehl wird in zwei Schritten ausgewertet. Zuerst zerlegt der Tcl-Interpreter den Befehl in *Wörter* und führt die unten beschriebenen Substitutionen durch. Diese Substitutionen erfolgen für alle Befehle auf gleiche Weise. Zweitens wird das erste Wort verwendet, um eine Routine zur Ausführung des Befehls zu finden, und die übrigen Wörter des Befehls werden an diese Routine übergeben. Die Routine kann jedes ihrer Wörter beliebig interpretieren, etwa als Ganzzahl, Variablenname, Liste oder Tcl-Skript. Verschiedene Befehle interpretieren ihre Wörter unterschiedlich.

**[3] Wörter.**

Die Wörter eines Befehls werden durch Leerzeichen getrennt (ausgenommen Zeilenumbrüche, die Befehlstrenner sind).

**[4] Doppelte Anführungszeichen.**

Steht das erste Zeichen eines Wortes in doppelten Anführungszeichen ("""), wird das Wort durch das nächste doppelte Anführungszeichen abgeschlossen. Stehen Semikolons, schließende eckige Klammern oder Leerzeichen (einschließlich Zeilenumbrüche) zwischen den Anführungszeichen, werden sie als gewöhnliche Zeichen behandelt und zum Wort gezählt. Auf die Zeichen zwischen den Anführungszeichen werden Befehls-, Variablen- und Backslash-Substitutionen wie unten beschrieben angewendet. Die doppelten Anführungszeichen gehören nicht zum Wort.

**[5] Argument-Erweiterung.**

Beginnt ein Wort mit der Zeichenkette "{*}", gefolgt von einem Nicht-Leerzeichen, wird das führende "{*}" entfernt und der Rest des Wortes wie jedes andere Wort geparst und substituiert. Nach der Substitution wird das Wort als Liste geparst (ohne Befehls- oder Variablensubstitutionen; Backslash-Substitutionen erfolgen wie üblich für eine Liste, und einzelne innere Wörter können in geschweiften Klammern oder doppelten Anführungszeichen stehen), und seine Wörter werden dem substituierten Befehl hinzugefügt. Beispielsweise ist "cmd a {*}{b [c]} d {*}{$e f {g h}}" gleichwertig zu "cmd a b {[c]} d {$e} f {g h}".

**[6] Geschweifte Klammern.**

Steht das erste Zeichen eines Wortes in einer öffnenden geschweiften Klammer ("{") und gilt Regel [5] nicht, wird das Wort durch die passende schließende geschweifte Klammer ("}") abgeschlossen. Geschweifte Klammern können im Wort verschachtelt sein: Für jede zusätzliche öffnende geschweifte Klammer muss eine zusätzliche schließende vorhanden sein (steht jedoch eine öffnende oder schließende geschweifte Klammer im Wort in Backslash-Anführung, zählt sie nicht bei der Suche nach der passenden schließenden Klammer). Auf die Zeichen zwischen den geschweiften Klammern werden keine Substitutionen angewendet, außer den unten beschriebenen Backslash-Zeilenumbruch-Substitutionen; Semikolons, Zeilenumbrüche, schließende eckige Klammern oder Leerzeichen erhalten keine besondere Bedeutung. Das Wort besteht genau aus den Zeichen zwischen den äußeren geschweiften Klammern, ohne die Klammern selbst.

**[7] Befehlssubstitution.**

Enthält ein Wort eine öffnende eckige Klammer ("["), führt Tcl *Befehlssubstitution* durch. Dazu ruft es den Tcl-Interpreter rekursiv auf, um die Zeichen nach der öffnenden eckigen Klammer als Tcl-Skript zu verarbeiten. Das Skript kann beliebig viele Befehle enthalten und muss mit einer schließenden eckigen Klammer ("]") abgeschlossen werden. Das Ergebnis des Skripts (d. h. das Ergebnis seines letzten Befehls) wird an Stelle der eckigen Klammern und aller Zeichen dazwischen in das Wort substituiert. In einem einzelnen Wort können beliebig viele Befehlssubstitutionen vorkommen. Befehlssubstitution wird bei Wörtern in geschweiften Klammern nicht durchgeführt.

**[8] Variablensubstitution.**

Enthält ein Wort ein Dollarzeichen ("$"), gefolgt von einer der unten beschriebenen Formen, führt Tcl *Variablensubstitution* durch: Das Dollarzeichen und die folgenden Zeichen werden im Wort durch den Wert einer Variable ersetzt. Variablensubstitution kann eine der folgenden Formen annehmen:

**$*name***

*Name* ist der Name einer Skalarvariablen; der Name ist eine Folge aus einem oder mehreren Zeichen, die ein Buchstabe, eine Ziffer, ein Unterstrich oder Namespace-Trenner (zwei oder mehr Doppelpunkte) sind. Buchstaben und Ziffern sind *ausschließlich* die standardmäßigen ASCII-Zeichen (**0**–**9**, **A**–**Z** und **a**–**z**).

**$*name*(*index*)**

*Name* gibt den Namen einer Array-Variable an und *index* den Namen eines Elements in diesem Array. *Name* darf nur Buchstaben, Ziffern, Unterstriche und Namespace-Trenner enthalten und kann eine leere Zeichenkette sein. Buchstaben und Ziffern sind *ausschließlich* die standardmäßigen ASCII-Zeichen (**0**–**9**, **A**–**Z** und **a**–**z**). Auf die Zeichen von *index* werden Befehls-, Variablen- und Backslash-Substitutionen angewendet.

**${*name*}**

*Name* ist der Name einer Skalarvariablen oder eines Array-Elements. Er darf beliebige Zeichen enthalten, außer schließenden geschweiften Klammern. Er bezeichnet ein Array-Element, wenn *name* die Form "*arrayName***(***index***)**" hat, wobei *arrayName* keine öffnenden Klammerzeichen "**(**" oder schließenden geschweiften Klammern "**}**" enthält und *index* eine beliebige Zeichenfolge außer schließenden geschweiften Klammern sein kann.

Beim Parsen von *name* werden keine weiteren Substitutionen durchgeführt.

In einem einzelnen Wort können beliebig viele Variablensubstitutionen vorkommen. Variablensubstitution wird bei Wörtern in geschweiften Klammern nicht durchgeführt.

Beachten Sie, dass Variablen Zeichenfolgen enthalten können, die nicht in den obigen Formen aufgeführt sind; in diesem Fall müssen andere Mechanismen zum Zugriff verwendet werden (z. B. die Ein-Argument-Form des Befehls set).

**[9] Backslash-Substitution.**

Erscheint ein Backslash ("\") innerhalb eines Wortes, tritt *Backslash-Substitution* auf. In allen Fällen außer den unten beschriebenen wird der Backslash entfernt und das folgende Zeichen als gewöhnliches Zeichen behandelt und zum Wort gezählt. So können Zeichen wie doppelte Anführungszeichen, schließende eckige Klammern und Dollarzeichen in Wörter aufgenommen werden, ohne besondere Verarbeitung auszulösen. Die folgende Tabelle listet die Backslash-Sequenzen auf, die besonders behandelt werden, zusammen mit dem Wert, der jede Sequenz ersetzt.

**\a**

Hörbare Warnung (Glocke) (Unicode U+000007).

**\b**

Rückwärtsblättern (Backspace) (Unicode U+000008).

**\f**

Seitenvorschub (Form Feed) (Unicode U+00000C).

**\n**

Zeilenumbruch (Newline) (Unicode U+00000A).

**\r**

Wagenrücklauf (Carriage Return) (Unicode U+00000D).

**\t**

Tabulator (Tab) (Unicode U+000009).

**\v**

Vertikaler Tabulator (Vertical Tab) (Unicode U+00000B).

**\*whiteSpace***

Ein einzelnes Leerzeichen ersetzt den Backslash, den Zeilenumbruch und alle Leerzeichen und Tabulatorzeichen nach dem Zeilenumbruch. Diese Backslash-Sequenz ist einzigartig, weil sie in einem separaten Pre-Pass ersetzt wird, bevor der Befehl tatsächlich geparst wird. Das bedeutet, dass sie auch ersetzt wird, wenn sie zwischen geschweiften Klammern steht, und das resultierende Leerzeichen als Worttrenner behandelt wird, wenn es nicht in geschweiften Klammern oder Anführungszeichen steht.

**\\**

Backslash ("\").

**\*ooo***

Die Ziffern *ooo* (eine, zwei oder drei) geben einen acht Bit breiten Oktalwert für das Unicode-Zeichen an, das eingefügt wird, im Bereich *000–377* (d. h. U+000000–U+0000FF). Der Parser stoppt kurz bevor dieser Bereich überläuft oder wenn das Maximum von drei Ziffern erreicht ist. Die oberen Bits des Unicode-Zeichens sind 0.

**\x*hh***

Die hexadezimalen Ziffern *hh* (eine oder zwei) geben einen acht Bit breiten hexadezimalen

Wert für das Unicode-Zeichen an, das eingefügt wird. Die oberen Bits des Unicode-Zeichens sind 0 (d. h. das Zeichen liegt im Bereich U+000000–U+0000FF).

**\u*hhhh***

Die hexadezimalen Ziffern *hhhh* (eine, zwei, drei oder vier) geben einen sechzehn Bit breiten hexadezimalen Wert für das Unicode-Zeichen an, das eingefügt wird. Die oberen Bits des Unicode-Zeichens sind 0 (d. h. das Zeichen liegt im Bereich U+000000–U+00FFFF).

**\U*hhhhhhhh***

Die hexadezimalen Ziffern *hhhhhhhh* (eine bis acht) geben einen einundzwanzig Bit breiten hexadezimalen Wert für das Unicode-Zeichen an, das eingefügt wird, im Bereich U+000000–U+10FFFF. Der Parser stoppt kurz bevor dieser Bereich überläuft oder wenn das Maximum von acht Ziffern erreicht ist. Die oberen Bits des Unicode-Zeichens sind 0.

Backslash-Substitution wird bei Wörtern in geschweiften Klammern nicht durchgeführt, außer für Backslash-Zeilenumbruch wie oben beschrieben.

**[10] Kommentare.**

Erscheint ein Rautezeichen ("#") an einer Stelle, an der Tcl das erste Zeichen des ersten Wortes eines Befehls erwartet, werden das Rautezeichen und die folgenden Zeichen bis zum nächsten Zeilenumbruch als Kommentar behandelt und ignoriert. Das Kommentarzeichen ist nur bedeutsam, wenn es am Anfang eines Befehls steht.

**[11] Reihenfolge der Substitution.**

Jedes Zeichen wird vom Tcl-Interpreter beim Erzeugen der Wörter eines Befehls genau einmal verarbeitet. Tritt beispielsweise Variablensubstitution auf, werden auf den Wert der Variable keine weiteren Substitutionen angewendet; der Wert wird wörtlich in das Wort eingefügt. Tritt Befehlssubstitution auf, wird der verschachtelte Befehl vollständig durch den rekursiven Aufruf des Tcl-Interpreters verarbeitet; vor dem rekursiven Aufruf werden keine Substitutionen durchgeführt, und auf das Ergebnis des verschachtelten Skripts werden keine zusätzlichen Substitutionen angewendet.

Substitutionen erfolgen von links nach rechts, und jede Substitution wird vollständig ausgewertet, bevor die nächste versucht wird. Eine Sequenz wie

```
set y [set x 0][incr x][incr x]
```

wird die Variable *y* immer auf den Wert *012* setzen.

**[12] Substitution und Wortgrenzen.**

Substitutionen beeinflussen die Wortgrenzen eines Befehls nicht, außer bei Argument-Erweiterung gemäß Regel [5]. Wird beispielsweise während der Variablensubstitution der gesamte Wert der Variable Teil eines einzelnen Wortes, auch wenn der Variablenwert Leerzeichen enthält.

## Tcl-Werte

Begrifflich sind alle Werte in Tcl Zeichenketten, genauer: Folgen von Unicode-Codepunkten. Dass alle Werte Zeichenketten sind, heißt üblicherweise *Everything Is A String (EIAS)*.

Meist kann man Tcl-Zeichenketten als Folgen von Zeichen statt von Codepunkten auffassen; dem üblichen Sprachgebrauch folgend sprechen Befehlsbeschreibungen so. Der Unterschied zählt, weil manche Zeichen mehrere Darstellungen als Folgen von Unicode-Codepunkten haben und Tcl bei Vergleichen einzelne Codepunkte vergleicht, nicht abstrakte Zeichen.

Obwohl alle Werte Zeichenketten sind, können einzelne Befehle ihre Argumente als Darstellung eines bestimmten Typs behandeln oder erwarten. incr erwartet zum Beispiel Ganzzahlen.

Die wichtigste Folge des EIAS-Prinzips: Befehle müssen Argumente mit derselben Zeichendarstellung gleich behandeln, unabhängig davon, wie der Wert entstanden ist. incr muss **100**, entstanden durch Anhängen von **0** an **10**, genauso behandeln wie **100**, entstanden durch Multiplikation von 10 mit sich selbst.

Das EIAS-Prinzip gilt nur auf der begrifflichen Ebene. Aus Effizienzgründen kann die Implementierung je nach Verwendung verschiedene interne Darstellungen wählen. Arithmetik mit Ganzzahlen kann den Wert intern als Maschinenwort speichern. Eine *binäre Zeichenkette* ist eine Folge von Unicode-Codepunkten im Bereich U+0000 bis U+00FF und kann als Bytefeld gespeichert werden. Diese interne Darstellung ist auf der Tcl-Sprachebene belanglos und in der Tcl-C-API sichtbar.

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# abstract

*Stichwörter:* abstract, abstract class, class, metaclass, object

## Name

oo::abstract - Eine Klasse, die keine direkten Instanzen von sich selbst erlaubt

## Übersicht

**package require tcl::oo**

**oo::abstract** *method* ?*arg* ...?

## Klassenhierarchie

```
oo::object
   → oo::class
       → oo::abstract
```

## Beschreibung

Abstrakte Klassen sind Klassen, die Definitionen enthalten können, aber nicht direkt instanziiert werden können. Sie sind nur dazu gedacht, vererbt und indirekt instanziiert zu werden. Die charakteristischen Methoden von oo::class (**create** und **new**) werden von einer Instanz von **oo::abstract** nicht exportiert.

Beachten Sie, dass **oo::abstract** selbst keine Instanz von **oo::abstract** ist.

## Konstruktor

Die Klasse **oo::abstract** definiert keinen expliziten Konstruktor; das bedeutet, dass sie effektiv identisch mit dem Konstruktor der Klasse oo::class ist.

## Destruktor

Die Klasse **oo::abstract** definiert keinen expliziten Destruktor; das Zerstören einer Instanz ist genauso wie das Zerstören einer gewöhnlichen Klasse (und zerstört alle ihre Unterklassen).

### Exportierte Methoden

Die Klasse **oo::abstract** definiert keine neuen exportierten Methoden.

### Nicht-exportierte Methoden

Die Klasse **oo::abstract** erklärt explizit, dass **create**, **createWithNamespace** und **new** nicht exportiert sind.

## Beispiele

Dieses Beispiel definiert eine einfache Klassenhierarchie, erzeugt eine neue Instanz und ruft eine Methode des Objekts auf, bevor die Hierarchie zerstört wird und gezeigt wird, dass die Zerstörung transitiv ist.

```
oo::abstract create fruit {
    method eat {} {
        puts "yummy!"
    }
}
oo::class create banana {
    superclass fruit
    method peel {} {
        puts "skin now off"
    }
}
set b [banana new]
$b peel              → prints 'skin now off'
$b eat               → prints 'yummy!'
set f [fruit new]    → error 'unknown method "new"...'
```

*Copyright (c) 2018 Donal K. Fellows*

# after

*Stichwörter:* after, cancel, delay, idle callback, sleep, time

## Name

after - Führt einen Befehl nach einer Zeitverzögerung aus

## Übersicht

**after** *ms*
**after** *ms* ?*script script script* ...?
**after cancel** *id*
**after cancel** *script script script ...*
**after idle** ?*script script script* ...?
**after info** ?*id*?

## Beschreibung

Dieser Befehl verzögert die Programmausführung oder führt einen Befehl zu einem späteren Zeitpunkt im Hintergrund aus. Die Form hängt vom ersten Argument ab:

**after** ***ms***

*ms* muss eine Ganzzahl in Millisekunden sein; negative Werte gelten als 0. Der Befehl schläft *ms* Millisekunden und kehrt dann zurück. Während des Schlafens reagiert die Anwendung nicht auf Events.

**after *ms* ?*script script script* ...?**

In dieser Form kehrt der Befehl sofort zurück, plant aber die Ausführung eines Tcl-Befehls *ms* Millisekunden später als Event-Handler. Der Befehl wird genau einmal zur angegebenen Zeit ausgeführt. Der verzögerte Befehl entsteht durch Verkettung aller *script*-Argumente wie bei concat. Die Ausführung erfolgt auf globaler Ebene (außerhalb jeder Tcl-Prozedur). Tritt beim Ausführen des verzögerten Befehls ein Fehler auf, meldet ihn der mit interp bgerror registrierte Handler. **after** gibt eine Kennung zurück, mit der der verzögerte Befehl per **after cancel** abgebrochen werden kann. *ms* von 0 (oder negativ) reiht das Event sofort mit Vorrang vor anderen Event-Typen ein (sofern nicht mit einem Event-Proc installiert, der auf die nächste Event-Runde wartet).

**after cancel *id***

Bricht die Ausführung eines zuvor geplanten verzögerten Befehls ab. *id* muss der Rückgabewert eines früheren **after**-Befehls sein. Wurde der Befehl bereits ausgeführt, hat **after cancel** keine Wirkung.

**after cancel *script script* ...**

Bricht ebenfalls einen verzögerten Befehl ab. Die *script*-Argumente werden mit Leerzeichen verkettet (wie concat). Gibt es einen passenden ausstehenden Befehl, wird er abgebrochen und nie ausgeführt; sonst keine Wirkung.

**after idle *script* ?*script script* ...?**

Verkettet die *script*-Argumente mit Leerzeichen (wie concat) und plant die Auswertung als Idle-Callback. Das Skript läuft genau einmal, wenn die Event-Schleife das nächste Mal ohne zu verarbeitende Events betreten wird. Der Befehl gibt eine Kennung für **after cancel** zurück. Fehler werden über interp bgerror gemeldet.

**after info ?*id*?**

Liefert Informationen über vorhandene Event-Handler. Ohne *id*: Liste der Kennungen aller von **after** für diesen Interpreter erzeugten Handler. Mit *id*: ein bestehender Handler (Rückgabewert eines früheren **after**, noch nicht ausgelöst oder abgebrochen). Dann eine Liste mit zwei Elementen: das zu *id* gehörige Skript und entweder **idle** oder **timer** als Handler-Typ.

Die Formen **after** *ms* und **after idle** setzen eine event-getriebene Anwendung voraus: verzögerte Befehle laufen nur, wenn die Event-Schleife betreten wird. In nicht event-getriebenen Anwendungen wie tclsh kann das mit vwait und update geschehen.

## Beispiele

Befehl, der Tcl *N* Sekunden lang nichts tun lässt:

```
proc sleep {N} {
    after [expr {int($N * 1000)}]
}
```

Plant die Ausführung von *wake_up* in acht Stunden (wenn die Event-Schleife dann aktiv ist):

```
after [expr {1000 * 60 * 60 * 8}] wake_up
```

Schrittweise lange Berechnungen (*::my_calc::one_step* liefert, ob ein weiterer Schritt nötig ist), ohne die Event-Schleife zu verhungern — nützlich, damit eine Tk-GUI bei langsamen Aufgaben reagiert:

```
proc doOneStep {} {
    if {[::my_calc::one_step]} {
        after idle [list after 0 doOneStep]
    }
}
doOneStep
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# append

*Stichwörter:* append, variable

## Name

append - Hängt an eine Variable an

## Übersicht

**append** *varName* ?*value value value* ...?

## Beschreibung

Hängt alle *value*-Argumente an den aktuellen Wert der Variable *varName* an. Existiert *varName* noch nicht, erhält sie einen Wert, der der Verkettung aller *value*-Argumente entspricht. Verweist varName auf ein nicht existierendes Element eines Arrays mit gesetztem Standardwert, wird die Verkettung des Standardwerts und aller value-Argumente im Array-Element gespeichert. Das Ergebnis des Befehls ist der neue in *varName* gespeicherte Wert. Der Befehl eignet sich effizient zum schrittweisen Aufbau langer Variablen. Beispielsweise ist "**append a $b**" deutlich effizienter als "**set a $a$b**", wenn **$a** lang ist.

## Beispiel

Baut in einer Schleife schrittweise eine Zeichenkette kommagetrennter Zahlen auf:

```
set var 0
for {set i 1} {$i<=10} {incr i} {
    append var "," $i
}
puts $var
# Gibt aus: 0,1,2,3,4,5,6,7,8,9,10
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# apply

*Stichwörter:* apply, anonymous function, argument, lambda, procedure

## Name

apply - Wendet eine anonyme Funktion an

## Übersicht

**apply** *func* ?*arg1 arg2* ...?

## Beschreibung

Der Befehl **apply** wendet die Funktion *func* auf die Argumente *arg1 arg2 ...* an und gibt das Ergebnis zurück.

Die Funktion *func* ist eine Liste mit zwei Elementen *{args body}* oder mit drei Elementen *{args body namespace}* (als wäre der Befehl list verwendet worden). Das erste Element *args* legt die formalen Argumente von *func* fest. Die Spezifikation der formalen Argumente *args* ist mit dem Befehl proc identisch und wird auf der entsprechenden Manpage im Detail beschrieben.

Der Inhalt von *body* wird vom Tcl-Interpreter ausgeführt, nachdem den lokalen Variablen, die den formalen Argumenten entsprechen, die Werte der tatsächlichen Parameter *arg1 arg2 ...* zugewiesen wurden. Während *body* ausgeführt wird, beziehen sich Variablennamen normalerweise auf lokale Variablen, die beim ersten Zugriff automatisch erzeugt und beim Zurückkehren von **apply** gelöscht werden. Für jedes Argument der Funktion wird automatisch eine lokale Variable erzeugt. Auf globale Variablen kann nur über den Befehl global oder upvar zugegriffen werden. Auf Namespace-Variablen nur über variable oder upvar.

Der Aufruf von **apply** legt einen Aufrufframe auf Tcls Auswertungsstack (den Stack der Frames, auf die uplevel zugreift). Die Ausführung von *body* erfolgt in diesem Aufrufframe im durch *namespace* angegebenen Namespace oder im globalen Namespace, wenn keiner angegeben wurde. Ist *namespace* gesetzt, wird es relativ zum globalen Namespace interpretiert, auch wenn der Name nicht mit "::" beginnt.

Die Semantik von **apply** lässt sich auch ungefähr so beschreiben:

```
proc apply {fun args} {
    set len [llength $fun]
    if {($len < 2) || ($len > 3)} {
        error "can't interpret \"$fun\" as anonymous function"
    }
    lassign $fun argList body ns
    set name ::$ns::[getGloballyUniqueName]
    set body0 {
        rename [lindex [info level 0] 0] {}
    }
    proc $name $argList ${body0}$body
    set code [catch {uplevel 1 $name $args} res opt]
    return -options $opt $res
}
```

## Beispiele

Dies zeigt, wie man einen einfachen allgemeinen Befehl erstellt, der eine Transformation auf jedes Element einer Liste anwendet.

```
proc map {lambda list} {
    set result {}
    foreach item $list {
        lappend result [apply $lambda $item]
    }
    return $result
}
map {x {return [string length $x]:$x}} {a bb ccc dddd}
      → 1:a 2:bb 3:ccc 4:dddd
map {x {expr {$x**2 + 3*$x - 2}}} {-4 -3 -2 -1 0 1 2 3 4}
      → 2 -2 -4 -4 -2 2 8 16 26
```

Der Befehl **apply** eignet sich auch zum Definieren von Callbacks für den Befehl trace:

```
set vbl "123abc"
trace add variable vbl write {apply {{v1 v2 op} {
    upvar 1 $v1 v
    puts "updated variable to \"$v\""
}}}
set vbl 123
set vbl abc
```

*Copyright (c) 2006 Miguel Sofer*
*Copyright (c) 2006 Donal K. Fellows*

# array

*Stichwörter:* array, element names, search

## Name

array - Array-Variablen bearbeiten

## Übersicht

**array** *option arrayName* ?*arg arg* ...?

## Beschreibung

Der Befehl führt eine von mehreren Operationen an der Variable *arrayName* aus. Sofern unten nicht anders angegeben, muss *arrayName* eine bestehende Array-Variable sein. *option* bestimmt die Aktion (Abkürzungen erlaubt):

**array anymore *arrayName searchId***

Gibt 1 zurück, wenn in einer Arraysuche noch Elemente übrig sind, sonst 0. *searchId* bezeichnet die Suche (Rückgabewert von **array startsearch**). Nützlich bei leerem Elementnamen, da **array nextelement** allein nicht anzeigt, ob die Suche abgeschlossen ist.

**array default *subcommand arrayName args...***

Verwaltet den Standardwert des Arrays. Arrays haben zunächst keinen Standardwert; nach dem Setzen wird er beim Lesen eines Elements geliefert, wenn das Lesen sonst einen Fehler ergeben würde. Das kann das Verhalten von append, dict, incr und lappend bei nicht existierenden Elementen ändern.

*subcommand* steuert die Operation am Standardwert von *arrayName*. Unterstützte Unterbefehle:

**array default exists *arrayName***

Gibt einen booleschen Wert zurück, ob ein Standardwert gesetzt ist. Falsch, wenn arrayName nicht existiert. Fehler, wenn arrayName existiert, aber kein Array ist.

**array default get *arrayName***

Gibt den aktuellen Standardwert zurück. Fehler, wenn arrayName kein Array ist oder keinen Standardwert hat.

**array default set *arrayName value***

Setzt den Standardwert auf value, Rückgabe leerer String. Fehler bei ungültigem Arraynamen oder wenn arrayName keine Array-Variable ist. Existiert arrayName nicht, wird ein leeres Array mit Standardwert angelegt.

**array default unset *arrayName***

Entfernt den Standardwert, Rückgabe leerer String. Ohne Wirkung, wenn kein Standardwert gesetzt ist. Fehler, wenn arrayName keine Array-Variable ist.

**array donesearch *arrayName searchId***

Beendet eine Arraysuche und gibt den zugehörigen Zustand frei. *searchId* stammt von **array startsearch**. Rückgabe leerer String.

**array exists *arrayName***

1, wenn *arrayName* ein Array ist; 0, wenn die Variable fehlt oder skalar ist.

**array for *{keyVariable valueVariable} arrayName body***

Erstes Argument: zweielementige Liste mit Variablennamen für Schlüssel und Wert jedes Eintrags. Zweites: Arrayname. Drittes: auszuführender Code. Schlüsselreihenfolge undefiniert. Wird während *array for* ein Element gelöscht oder neu eingefügt, bricht der Befehl mit Fehler ab.

**array get *arrayName* ?*pattern*?**

Liste aus Paaren: ungerade Position = Elementname, gerade = Wert. Paarreihenfolge undefiniert. Ohne *pattern* alle Elemente. Mit *pattern* nur passende Namen (string match). Kein Array oder leeres Array: leere Liste. Ändern Traces die Elementliste während des Aufrufs, werden nur Elemente zurückgegeben, die vor und nach **array get** existieren.

**array names *arrayName* ?*mode*? ?*pattern*?**

Liste der Elementnamen, die *pattern* entsprechen. *mode*: **-exact**, **-glob** oder **-regexp** (Standard **-glob**). Siehe string match und regexp. Ohne *pattern* alle Namen. Keine Treffer oder kein Array: leerer String.

**array nextelement *arrayName searchId***

Name des nächsten Elements oder leerer String, wenn die Suche erschöpft ist. *searchId* von **array startsearch**. Warnung: werden Elemente während der Suche hinzugefügt oder gelöscht, werden alle Suchen beendet wie bei **array donesearch**; **array nextelement** schlägt dann für diese Suchen fehl.

**array set *arrayName list***

Setzt ein oder mehrere Elemente. *list* wie von **array get**: gerade Anzahl von Elementen, ungerade = Name, folgendes = Wert. Existiert *arrayName* nicht und ist *list* leer, entsteht ein leeres Array.

**array size *arrayName***

Dezimalstring mit der Elementanzahl. Kein Array: 0.

**array startsearch *arrayName***

Startet elementweise Suche; **array nextelement** liefert die Namen. Abschluss mit **array donesearch**. Rückgabe: Such-ID für parallele Suchen im selben Array. Für die meisten Arrays sind **array get** oder **array names** mit foreach effizienter (siehe Beispiele).

**array statistics *arrayName***

Statistik zur Datenverteilung in der Hashtable des Arrays: Einträge, Buckets, Auslastung.

**array unset *arrayName* ?*pattern*?**

Entfernt alle Elemente, deren Namen *pattern* entsprechen (string match). Kein Array oder keine Treffer: kein Fehler. Ohne *pattern* bei Array-Variable: gesamtes Array löschen. Rückgabe immer leerer String.

## Beispiele

```
array set colorcount {
    red   1
    green 5
    blue  4
    white 9
}

foreach {color count} [array get colorcount] {
    puts "Color: $color Count: $count"
}
  → Color: blue Count: 4
    Color: white Count: 9
    Color: green Count: 5
    Color: red Count: 1

foreach color [array names colorcount] {
    puts "Color: $color Count: $colorcount($color)"
}
  → Color: blue Count: 4
    Color: white Count: 9
    Color: green Count: 5
    Color: red Count: 1

foreach color [lsort [array names colorcount]] {
    puts "Color: $color Count: $colorcount($color)"
}
  → Color: blue Count: 4
    Color: green Count: 5
    Color: red Count: 1
    Color: white Count: 9

array statistics colorcount
```

```
 → 4 entries in table, 4 buckets
   number of buckets with 0 entries: 1
   number of buckets with 1 entries: 2
   number of buckets with 2 entries: 1
   number of buckets with 3 entries: 0
   number of buckets with 4 entries: 0
   number of buckets with 5 entries: 0
   number of buckets with 6 entries: 0
   number of buckets with 7 entries: 0
   number of buckets with 8 entries: 0
   number of buckets with 9 entries: 0
   number of buckets with 10 or more entries: 0
   average search distance for entry: 1.2
```

*Copyright (c) 1993-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# bgerror

*Stichwörter:* bgerror, background error, reporting

## Name

bgerror - Befehl zur Behandlung von Hintergrundfehlern

## Übersicht

**bgerror** *message*

## Beschreibung

Tcl 8.5 unterstützt den Befehl interp bgerror, mit dem Anwendungen in einem Interpreter den Befehl registrieren können, der Hintergrundfehler in diesem Interpreter behandelt. In älteren Tcl-Versionen war diese Steuerung nicht verfügbar; Anwendungen konnten Hintergrundfehler nur steuern, indem sie im globalen Namespace eines Interpreters einen Befehl mit genau dem Namen **bgerror** definierten. Die folgende Dokumentation beschreibt die Schnittstellenanforderungen des Befehls **bgerror**, den eine Anwendung definieren kann, um mit Tcl-Versionen vor 8.5 kompatibel zu bleiben. Anwendungen, die nur Tcl 8.5 und neuer unterstützen wollen, sollten interp bgerror verwenden.

Der Befehl **bgerror** ist kein eingebauter Teil von Tcl. Stattdessen können einzelne Anwendungen oder Benutzer einen Befehl **bgerror** definieren (z. B. als Tcl-Prozedur), wenn sie Hintergrundfehler behandeln möchten.

Ein Hintergrundfehler tritt auf, wenn ein Fehler in einem Ereignishandler oder einem anderen Befehl auftritt, der nicht von der Anwendung stammt. Tritt beispielsweise ein Fehler beim Ausführen eines mit after angegebenen Befehls auf, handelt es sich um einen Hintergrundfehler. Bei einem Nicht-Hintergrundfehler kann der Fehler einfach durch verschachtelte Tcl-Befehlsauswertungen nach oben gereicht werden, bis er den Anwendungscode auf oberster Ebene erreicht; dann kann die Anwendung den Fehler melden, wie sie es wünscht. Bei einem Hintergrundfehler endet das Entwinden in der Tcl-Bibliothek, und Tcl hat keinen offensichtlichen Weg, den Fehler zu melden.

Wenn Tcl einen Hintergrundfehler erkennt, speichert es Informationen über den Fehler und ruft später einen mit interp bgerror registrierten Handler-Befehl als Idle-Ereignishandler auf. Der Standard-Handler ruft wiederum den Befehl **bgerror** auf. Vor dem Aufruf von **bgerror** stellt Tcl errorInfo und errorCode auf die Werte zum Zeitpunkt des Fehlers wieder her und ruft dann **bgerror** mit der Fehlermeldung als einzigem Argument auf. Tcl geht davon aus, dass die Anwendung den Befehl **bgerror** implementiert hat und dass der Befehl den Fehler sinnvoll für die Anwendung meldet. Tcl ignoriert jedes von **bgerror** zurückgegebene Ergebnis, solange kein Fehler erzeugt wird.

Tritt innerhalb von **bgerror** ein weiterer Tcl-Fehler auf (z. B. weil kein **bgerror** definiert wurde), meldet Tcl den Fehler selbst, indem es eine Meldung nach stderr schreibt.

Häufen sich mehrere Hintergrundfehler an, bevor **bgerror** sie verarbeitet, wird **bgerror** einmal pro Fehler in der Reihenfolge ihres Auftretens aufgerufen. Gibt **bgerror** jedoch mit einer **break** -Ausnahme zurück, werden verbleibende Fehler übersprungen, ohne **bgerror** aufzurufen.

Schreiben Sie Code, der von anderen als Teil eines Pakets oder einer Bibliothek genutzt wird, sollten Sie **bgerror** vermeiden. Der Programmierer der Anwendung möchte möglicherweise ebenfalls **bgerror** definieren oder anderen Code verwenden, der das tut — die Integration Ihres Codes wird dann schwierig.

## Beispiele

Diese **bgerror**-Prozedur hängt Fehler mit Zeitstempel an eine Datei an.

```
proc bgerror {message} {
    set timestamp [clock format [clock seconds]]
    set fl [open mylog.txt {WRONLY CREAT APPEND}]
    puts $fl "$timestamp: bgerror in $::argv '$message'"
    close $fl
}
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# binary

*Stichwörter:* binary, format, scan

## Name

binary - Felder in Binärzeichenketten einfügen und extrahieren

## Übersicht

**binary decode** *format ?-option value ...? data*
**binary encode** *format ?-option value ...? data*
**binary format** *formatString ?arg arg ...?*
**binary scan** *string formatString ?varName varName ...?*

## Beschreibung

Dieser Befehl stellt Hilfsmittel zur Verarbeitung binärer Daten bereit. Der Unterbefehl **binary format** erzeugt aus normalen Tcl-Werten eine Binärzeichenkette. Gegeben beispielsweise die Werte 16 und 22, kann er auf einer 32-Bit-Architektur eine 8-Byte-Binärzeichenkette aus zwei

4-Byte-Ganzzahlen erzeugen — je eine für jede Zahl. Der Unterbefehl **binary scan** macht das Gegenteil: Er extrahiert Daten aus einer Binärzeichenkette und gibt sie als gewöhnliche Tcl-Zeichenkettenwerte zurück. Die Unterbefehle **binary encode** und **binary decode** wandeln binäre Daten in Zeichenkettenkodierungen um bzw. umgekehrt, z. B. base64 (wie in MIME-Nachrichten).

Andere Operationen auf binären Daten — etwa eine Teilfolge bilden, die Länge ermitteln oder die Daten mit einer bestimmten Kodierung als Zeichenkette interpretieren — erledigen andere Tcl-Befehle (in den genannten Fällen string range, string length und encoding convertfrom). Eine Binärzeichenkette in Tcl ist schlicht eine Zeichenkette, deren Zeichen alle im Bereich \u0000-\u00FF liegen.

## Kodierung und Dekodierung

Zum Kodieren binärer Daten als lesbare Zeichenkette werden die Ausgangsdaten zusammen mit dem Namen der zu verwendenden Kodierung und gewünschten kodierungsspezifischen Optionen an **binary encode** übergeben. Kodierte Daten lassen sich mit **binary decode** wieder in binäre Form zurückwandeln. **binary encode** löst einen Fehler aus, wenn das Argument *data* keine Binärdaten sind. Folgende Formate und Optionen werden unterstützt.

**base64**

Die Binärkodierung **base64** wird häufig in E-Mail-Nachrichten und XML-Dokumenten verwendet und nutzt vor allem Groß- und Kleinbuchstaben sowie Ziffern. Sie kann beliebig umbrochen werden, ohne Information zu verlieren.

Beim Kodieren werden folgende Optionen unterstützt:

**-maxlen *length***

Die Ausgabe soll in Zeilen von höchstens *length* Zeichen aufgeteilt werden. Standardmäßig erfolgt kein Zeilenumbruch.

**-wrapchar *character***

Gibt an, welches Zeichen beim Umbrechen wegen **-maxlen** die Zeilen trennt. Standard ist das Zeilenumbruchzeichen "\n".

Beim Dekodieren werden folgende Optionen unterstützt:

**-strict**

Der Dekodierer soll einen Fehler auslösen, wenn Zeichen angetroffen werden, die nicht strikt Teil der Kodierung sind. Andernfalls werden sie ignoriert. RFC 2045 verlangt nicht-strikte base64-Dekodierer.

**hex**

Die Binärkodierung **hex** wandelt jedes Byte in ein Paar hexadezimaler Ziffern um, die den Byte-Wert als hexadezimale Ganzzahl darstellen. Beim Kodieren werden Kleinbuchstaben verwendet. Beim Dekodieren werden Groß- und Kleinbuchstaben akzeptiert.

Beim Kodieren werden keine Optionen unterstützt. Beim Dekodieren:

**-strict**

Der Dekodierer soll einen Fehler auslösen, wenn Leerzeichen angetroffen werden. Andernfalls werden sie ignoriert.

**uuencode**

Die Binärkodierung **uuencode** war früher beim Datentransfer zwischen Unix-Systemen und in USENET verbreitet, ist heute seltener und wurde weitgehend von **base64** abgelöst.

Beim Kodieren werden folgende Optionen unterstützt (Änderungen können Dateien erzeugen, die andere Dekodierer nicht verarbeiten):

**-maxlen *length***

Maximale Zeichenanzahl pro kodierte Zeile. Gültiger Bereich: 5 bis 85. Außerhalb dieses Bereichs ist das Format nicht darstellbar. Standard: 61.

**-wrapchar *character***

Zeichen, die das Ende jeder kodierten Zeile markieren. Zulässig: eine Folge von null oder mehr Zeichen aus { \x09 (TAB), \x0B (VT), \x0C (FF), \x0D (CR) } gefolgt von null oder einem Zeilenumbruch \x0A (LF). Andere Werte werden abgelehnt, weil die kodierte Ausgabe nicht dekodierbar wäre. Standard: ein einzelner Zeilenumbruch.

Beim Dekodieren:

**-strict**

Der Dekodierer soll einen Fehler auslösen, wenn etwas außerhalb des Standardformats angetroffen wird. Ohne diese Option toleriert er einige Abweichungen, vor allem um Zeilenumbrüche zwischen Kodierer und Dekodierer zu verzeihen.

Weder Kodierer noch Dekodierer verarbeiten Kopf- und Fußzeile des uuencode-Formats.

## Binärformat

**binary format** erzeugt eine Binärzeichenkette, deren Layout durch *formatString* festgelegt ist und deren Inhalt aus den weiteren Argumenten stammt. Rückgabe: die erzeugte Binärzeichenkette.

*formatString* besteht aus einer Folge von null oder mehr Feldspezifizierern, getrennt durch null oder mehr Leerzeichen. Jeder Feldspezifizierer ist ein Typzeichen, optional gefolgt von einem Flag-Zeichen und einer optionalen numerischen *count*. Die meisten Feldspezifizierer verbrauchen ein Argument für den zu formatierenden Wert. Das Typzeichen legt die Formatierung fest. *count* gibt in der Regel an, wie viele Elemente des Typs aus dem Wert übernommen werden. Ist *count* gesetzt, ist es eine nicht-negative Dezimalzahl oder "*****"; letzteres bedeutet normalerweise, dass alle Elemente des Werts verwendet werden. Stimmt die Argumentanzahl nicht mit der Zahl der argumentverbrauchenden Felder in der Formatzeichenkette überein, entsteht ein Fehler. Das Flag-Zeichen wird bei **binary format** ignoriert.

Ein kleines Beispiel verdeutlicht den Zusammenhang zwischen Feldspezifizierern und Argumenten:

```
binary format d3d {1.0 2.0 3.0 4.0} 0.1
```

Das erste Argument ist eine Liste mit vier Zahlen; wegen *count* 3 beim zugehörigen Feldspezifizierer werden nur die ersten drei verwendet. Das zweite Argument gehört zum zweiten Feldspezifizierer. Die resultierende Binärzeichenkette enthält die vier Zahlen 1.0, 2.0, 3.0 und 0.1.

Jedes Paar Typ-*count* bewegt einen imaginären Cursor durch die Binärdaten, speichert Bytes an der aktuellen Position und setzt den Cursor unmittelbar hinter das zuletzt gespeicherte Byte.

Anfangs steht der Cursor bei Position 0. Der Typ kann eines der folgenden Zeichen sein:

**a**

Speichert eine Byte-Zeichenkette der Länge *count* in der Ausgabezeichenkette. Jedes Zeichen wird modulo 256 genommen (nur das niedrige Byte; das hohe Byte entfällt). Für Zeichenketten, die nicht vollständig mit \u0000-\u00ff darstellbar sind, sollte vorher encoding convertto die externe Darstellung erzeugen, wenn diese Kürzung unerwünscht ist (d. h. wenn die Zeichen nicht zu ISO 8859-1 gehören). Hat *arg* weniger als *count* Bytes, wird mit Null-Bytes aufgefüllt. Ist *arg* länger, werden überzählige Zeichen ignoriert. Bei *count* "*****" werden alle Bytes von *arg* formatiert. Fehlt *count*, wird ein Zeichen formatiert. Beispiel:

```
binary format a7a*a alpha bravo charlie
```

liefert eine Binärzeichenkette äquivalent zu:

```
alpha\000\000bravoc
```

der Befehl:

```
binary format a* [encoding convertto utf-8 \u20ac]
```

liefert eine Binärzeichenkette äquivalent zu:

```
\342\202\254
```

(UTF-8-Bytefolge für das Euro-Zeichen), und:

```
binary format a* [encoding convertto iso8859-15 \u20ac]
```

liefert eine Binärzeichenkette äquivalent zu:

```
\244
```

(ISO 8859-15-Bytefolge für das Euro-Zeichen). Im Gegensatz dazu:

```
binary format a* \u20ac
```

liefert eine Binärzeichenkette äquivalent zu:

```
\254
```

(d. h. **\xAC**) durch Abschneiden der hohen Bits — vermutlich nicht gewünscht.

**A**

Wie **a**, aber Auffüllung mit Leerzeichen statt Nullen. Beispiel:

```
binary format A6A*A alpha bravo charlie
```

liefert

```
alpha bravoc
```

**b**

Speichert eine Folge von *count* Binärziffern in niedrig-nach-hoch-Reihenfolge innerhalb jedes Bytes. *Arg* muss **1** und **0** enthalten. Die Bytes werden in Reihenfolge erster bis letzter ausgegeben, die Bits niedrig-nach-hoch innerhalb jedes Bytes. Hat *arg* weniger als *count* Ziffern, werden die restlichen Bits mit Nullen aufgefüllt. Mehr Ziffern als *count* werden ignoriert. Bei *count* "*****" werden alle Ziffern von *arg* formatiert. Fehlt *count*, eine Ziffer. Endet die Bitanzahl nicht an einer Byte-Grenze, sind die restlichen Bits des letzten Bytes Null. Beispiel:

```
binary format b5b* 11100 111000011010
```

liefert eine Binärzeichenkette äquivalent zu:

```
\x07\x87\x05
```

**B**

Wie **b**, aber Bits in hoch-nach-niedrig-Reihenfolge innerhalb jedes Bytes. Beispiel:

```
binary format B5B* 11100 111000011010
```

liefert eine Binärzeichenkette äquivalent zu:

```
\xE0\xE1\xA0
```

**H**

Speichert *count* hexadezimale Ziffern in hoch-nach-niedrig-Reihenfolge innerhalb jedes Bytes. *Arg* muss Zeichen aus "0123456789abcdefABCDEF" sein. Bytes erster bis letzter, Ziffern hoch-nach-niedrig pro Byte. Weniger als *count* Ziffern: Rest mit Nullen. Mehr Ziffern: ignoriert. *count* "*****": alle Ziffern von *arg*. Fehlt *count*: eine Ziffer. Endet die Ziffernanzahl nicht an einer Byte-Grenze, sind die restlichen Bits des letzten Bytes Null. Beispiel:

```
binary format H3H*H2 ab DEF 987
```

liefert eine Binärzeichenkette äquivalent zu:

```
\xAB\x00\xDE\xF0\x98
```

**h**

Wie **H**, aber Ziffern niedrig-nach-hoch innerhalb jedes Bytes. Selten benötigt. Beispiel:

```
binary format h3h*h2 AB def 987
```

liefert eine Binärzeichenkette äquivalent zu:

```
\xBA\x00\xED\x0F\x89
```

**c**

Speichert einen oder mehrere 8-Bit-Ganzzahlwerte. Ohne *count* muss *arg* eine Ganzzahl sein. Mit *count* eine Liste mit mindestens so vielen Ganzzahlen. Die unteren 8 Bits jeder Ganzzahl werden als ein Byte an der Cursorposition gespeichert. *count* "*****": alle Ganzzahlen der Liste.

Mehr Listenelemente als *count*: überzählige ignoriert. Beispiel:

```
binary format c3cc* {3 -3 128 1} 260 {2 5}
```

liefert eine Binärzeichenkette äquivalent zu:

```
\x03\xFD\x80\x04\x02\x05
```

während:

```
binary format c {2 5}
```

einen Fehler erzeugt.

**s**

Wie **c**, speichert aber einen oder mehrere 16-Bit-Ganzzahlen in Little-Endian-Byte-Reihenfolge. Die unteren 16 Bits jeder Ganzzahl als Zwei-Byte-Wert, niederwertiges Byte zuerst. Beispiel:

```
binary format s3 {3 -3 258 1}
```

liefert eine Binärzeichenkette äquivalent zu:

```
\x03\x00\xFD\xFF\x02\x01
```

**S**

Wie **s**, aber 16-Bit-Ganzzahlen in Big-Endian-Byte-Reihenfolge. Beispiel:

```
binary format S3 {3 -3 258 1}
```

liefert eine Binärzeichenkette äquivalent zu:

```
\x00\x03\xFF\xFD\x01\x02
```

**t**

(mnemonisch *tiny*) Wie **s** und **S**, speichert 16-Bit-Ganzzahlen in der nativen Byte-Reihenfolge des Rechners, auf dem das Tcl-Skript läuft. Die native Byte-Reihenfolge steht im Element **byteOrder** des Arrays tcl_platform.

**i**

Wie **c**, speichert aber einen oder mehrere 32-Bit-Ganzzahlen in Little-Endian-Byte-Reihenfolge. Die unteren 32 Bits jeder Ganzzahl als Vier-Byte-Wert, niederwertiges Byte zuerst. Beispiel:

```
binary format i3 {3 -3 65536 1}
```

liefert eine Binärzeichenkette äquivalent zu:

```
\x03\x00\x00\x00\xFD\xFF\xFF\xFF\x00\x00\x01\x00
```

**I**

Wie **i**, speichert aber einen oder mehrere 32-Bit-Ganzzahlen in Big-Endian-Byte-Reihenfolge.

Beispiel:

```
binary format I3 {3 -3 65536 1}
```

liefert eine Binärzeichenkette äquivalent zu:

```
\x00\x00\x00\x03\xFF\xFF\xFF\xFD\x00\x01\x00\x00
```

**n**

(mnemonisch *number* oder *normal*) Wie **i** und **I**, speichert 32-Bit-Ganzzahlen in der nativen Byte-Reihenfolge des Rechners. Siehe **byteOrder** in tcl_platform.

**w**

Wie **c**, speichert aber einen oder mehrere 64-Bit-Ganzzahlen in Little-Endian-Byte-Reihenfolge. Die unteren 64 Bits jeder Ganzzahl als Acht-Byte-Wert, niederwertiges Byte zuerst. Beispiel:

```
binary format w 7810179016327718216
```

liefert die Binärzeichenkette **HelloTcl**.

**W**

Wie **w**, speichert aber einen oder mehrere 64-Bit-Ganzzahlen in Big-Endian-Byte-Reihenfolge. Beispiel:

```
binary format Wc 4785469626960341345 110
```

liefert die Binärzeichenkette **BigEndian**

**m**

(mnemonisch Spiegel von **w**) Wie **w** und **W**, speichert 64-Bit-Ganzzahlen in der nativen Byte-Reihenfolge. Siehe **byteOrder** in tcl_platform.

**f**

Wie **c**, speichert aber einen oder mehrere Gleitkommazahlen einfacher Genauigkeit in der nativen Darstellung des Rechners. Diese Darstellung ist architekturabhängig und sollte nicht zum Netzwerktransport von Gleitkommazahlen dienen. Die Byteanzahl kann variieren. Bei Überlauf wird FLT_MAX des Systems verwendet. Tcl arbeitet intern mit doppelter Genauigkeit; beim Umwandeln in einfache Genauigkeit kann Genauigkeit verloren gehen. Beispiel auf Windows mit Intel Pentium:

```
binary format f2 {1.6 3.4}
```

liefert eine Binärzeichenkette äquivalent zu:

```
\xCD\xCC\xCC\x3F\x9A\x99\x59\x40
```

**r**

(mnemonisch *real*) Wie **f**, speichert Gleitkommazahlen einfacher Genauigkeit in Little-Endian-Reihenfolge. Sinnvolle Ausgabe nur auf Rechnern mit IEEE-Gleitkommadarstellung (sehr verbreitet, nicht universell).

**R**

Wie **r**, aber Big-Endian-Reihenfolge.

**d**

Wie **f**, speichert aber einen oder mehrere Gleitkommazahlen doppelter Genauigkeit in der nativen Darstellung. Beispiel auf Windows mit Intel Pentium:

```
binary format d1 {1.6}
```

liefert eine Binärzeichenkette äquivalent zu:

```
\x9A\x99\x99\x99\x99\x99\xF9\x3F
```

**q**

(mnemonisch Spiegel von **d**) Wie **d**, speichert Gleitkommazahlen doppelter Genauigkeit in Little-Endian-Reihenfolge. Sinnvolle Ausgabe nur bei IEEE-Gleitkommadarstellung.

**Q**

Wie **q**, aber Big-Endian-Reihenfolge.

**x**

Speichert *count* Null-Bytes. Fehlt *count*, ein Null-Byte. Bei *count* "*****": Fehler. Verbraucht kein Argument. Beispiel:

```
binary format a3xa3x2a3 abc def ghi
```

liefert eine Binärzeichenkette äquivalent zu:

```
abc\000def\000\000ghi
```

**X**

Setzt den Cursor *count* Bytes in der Ausgabezeichenkette zurück. Bei *count* "*****" oder größer als die aktuelle Cursorposition: Position 0, das nächste Byte ist das erste der Ergebniszeichenkette. Fehlt *count*, ein Byte zurück. Verbraucht kein Argument. Beispiel:

```
binary format a3X*a3X2a3 abc def ghi
```

liefert **dghi**.

**@**

Setzt den Cursor auf die absolute Position *count* in der Ausgabezeichenkette. Position 0 ist das erste Byte. Liegt *count* hinter dem bisher letzten gespeicherten Byte, werden Zwischenpositionen mit Null-Bytes gefüllt. Bei *count* "*****": Cursor ans Ende der bisherigen Ausgabe. Fehlt *count*: Fehler. Verbraucht kein Argument. Beispiel:

```
binary format a5@2a1@*a3@10a1 abcde f ghi j
```

liefert

```
abfdeghi\000\000j
```

## Binärscan

**binary scan** parst Felder aus einer Binärzeichenkette und gibt die Anzahl der durchgeführten

Konvertierungen zurück. *String* liefert die einzulesenden Bytes, *formatString* legt das Parsen fest. Ist *string* keine gültige Binärdaten-Zeichenkette, entsteht ein Fehler. Jedes *varName* ist ein Variablenname; gescannte Werte werden der zugehörigen Variablen zugewiesen.

Wie bei **binary format** besteht *formatString* aus Feldspezifizierern (Typzeichen, optionales Flag, optionale *count*), getrennt durch Leerzeichen. Die meisten Feldspezifizierer verbrauchen ein Argument: die Variable für die gescannten Werte. Das Typzeichen legt die Interpretation fest. *count* gibt in der Regel an, wie viele Elemente des Typs aus den Daten genommen werden. Ist *count* gesetzt, Dezimalzahl oder "*" (normalerweise alle verbleibenden Elemente). Reichen die Bytes ab der Cursorposition nicht für den aktuellen Feldspezifizierer, bleibt die Variable unverändert und **binary scan** kehrt sofort mit der Anzahl bereits gesetzter Variablen zurück. Fehlen Argumente für alle argumentverbrauchenden Felder, entsteht ein Fehler. Das Flag "u" kann einige Typen als vorzeichenlos lesen lassen; es wird für alle Feldtypen akzeptiert, bei Nicht-Ganzzahltypen ignoriert.

Analoges Beispiel zu **binary format** für den Zusammenhang von Feldspezifizierern und Argumenten:

```
binary scan $bytes s3s first second
```

Dieser Befehl (sofern die Binärzeichenkette in *bytes* lang genug ist) weist *first* eine Liste aus drei Ganzzahlen und *second* einen einzelnen Wert zu. Enthält *bytes* weniger als 8 Bytes (vier 2-Byte-Ganzzahlen), erfolgt keine Zuweisung an *second*; bei weniger als 6 Bytes (drei 2-Byte-Ganzzahlen) keine an *first*. Daher:

```
puts [binary scan abcdefg s3s first second]
puts $first
puts $second
```

gibt aus (wenn die Variablen vorher nicht gesetzt waren):

```
1
25185 25699 26213
can't read "second": no such variable
```

**Wichtig: c**, **s** und **S** (sowie **i** und **I** auf 64-Bit-Systemen) werden in Werte der Größe *long* gescannt. Werte mit gesetztem Hochbit (0x80 für Zeichen, 0x8000 für shorts, 0x80000000 für ints) werden vorzeichenerweitert. Beispiel:

```
set signShort [binary format s1 0x8000]
binary scan $signShort s1 val; # val == 0xFFFF8000
```

Benötigen Sie vorzeichenlose Werte, kann das Flag-Zeichen "u" nach dem Feldtyp stehen. Beispiel für einen vorzeichenlosen short-Wert:

```
set signShort [binary format s1 0x8000]
binary scan $signShort su1 val; # val == 0x00008000
```

Jedes Paar Typ-*count* bewegt einen imaginären Cursor und liest ab der aktuellen Position. Anfangsposition: 0. Mögliche Typzeichen:

**a**

Die Daten sind eine Byte-Zeichenkette der Länge *count*. Bei *count* "*" alle verbleibenden

Bytes von *string* in die Variable. Fehlt *count*, ein Byte. Alle Bytes gelten als Zeichen \u0000-\u00ff; für andere Kodierungen ist encoding convertfrom nötig. Beispiel:

```
binary scan abcde\000fghi a6a10 var1 var2
```

gibt **1** zurück, *var1* enthält die Zeichenkette äquivalent zu **abcde\000**, *var2* unverändert, und

```
binary scan \342\202\254 a* var1
set var2 [encoding convertfrom utf-8 $var1]
```

speichert ein Euro-Zeichen in *var2*.

**A**

Wie **a**, entfernt aber nachgestellte Leerzeichen und Nullen vor dem Speichern. Beispiel:

```
binary scan "abc efghi  \000" A* var1
```

gibt **1** zurück, in *var1* steht **abc efghi**.

**b**

Die Daten werden in eine Folge von *count* Binärziffern in niedrig-nach-hoch-Reihenfolge als "1" und "0" umgewandelt. Bytes erster bis letzter, Bits niedrig-nach-hoch pro Byte; überzählige Bits im letzten Byte werden ignoriert. *count* "*": alle verbleibenden Bits. Fehlt *count*: ein Bit. Beispiel:

```
binary scan \x07\x87\x05 b5b* var1 var2
```

gibt **2** zurück, *var1* **11100**, *var2* **1110000110100000**.

**B**

Wie **b**, Bits hoch-nach-niedrig pro Byte. Beispiel:

```
binary scan \x70\x87\x05 B5B* var1 var2
```

gibt **2** zurück, *var1* **01110**, *var2* **1000011100000101**.

**C**

Ähnlich **A**, scannt von Anfang an und endet beim ersten Null-Byte (C-String-Semantik). Beispiel:

```
binary scan "abc\000efghi" C* var1
```

gibt **1** zurück, in *var1* steht **abc**.

**H**

Die Daten werden in *count* hexadezimale Ziffern in hoch-nach-niedrig-Reihenfolge als Zeichen aus "0123456789abcdef" umgewandelt. Bytes erster bis letzter, Ziffern hoch-nach-niedrig; überzählige Bits im letzten Byte ignoriert. *count* "*": alle verbleibenden Hex-Ziffern. Fehlt *count*: eine Ziffer. Beispiel:

```
binary scan \x07\xC6\x05\x1F\x34 H3H* var1 var2
```

gibt **2** zurück, *var1* **07c**, *var2* **051f34**.

**h**

Wie **H**, Ziffern in umgekehrter (niedrig-nach-hoch) Reihenfolge pro Byte. Beispiel:

```
binary scan \x07\x86\x05\x12\x34 h3h* var1 var2
```

gibt **2** zurück, *var1* **706**, *var2* **502143**.

Code, der Hex-Ziffern mehrerer Bytes in Reihenfolge parsen soll, sollte meist **H** verwenden.

**c**

Die Daten werden in *count* vorzeichenbehaftete 8-Bit-Ganzzahlen umgewandelt und als Liste in der Variable gespeichert, oder vorzeichenlos, wenn unmittelbar nach **c** **u** steht. *count* "*****": alle verbleibenden Bytes. Fehlt *count*: eine 8-Bit-Ganzzahl. Beispiel:

```
binary scan \x07\x86\x05 c2c* var1 var2
```

gibt **2** zurück, *var1* **7 -122**, *var2* **5**. Die Ganzzahlen sind vorzeichenbehaftet, außer bei **cu** statt **c**.

**s**

Die Daten werden als *count* vorzeichenbehaftete 16-Bit-Ganzzahlen in Little-Endian-Byte-Reihenfolge interpretiert, oder vorzeichenlos bei **u** unmittelbar nach **s**. Speicherung als Liste. *count* "*****": alle verbleibenden Bytes. Fehlt *count*: eine 16-Bit-Ganzzahl. Beispiel:

```
binary scan \x05\x00\x07\x00\xF0\xFF s2s* var1 var2
```

gibt **2** zurück, *var1* **5 7**, *var2* **-16**. Vorzeichenbehaftet, außer bei **su** statt **s**.

**S**

Wie **s**, aber *count* 16-Bit-Ganzzahlen in Big-Endian-Byte-Reihenfolge. Beispiel:

```
binary scan \x00\x05\x00\x07\xFF\xF0 S2S* var1 var2
```

gibt **2** zurück, *var1* **5 7**, *var2* **-16**.

**t**

Die Daten werden als *count* vorzeichenbehaftete 16-Bit-Ganzzahlen in der nativen Byte-Reihenfolge des Tcl-Rechners interpretiert, oder vorzeichenlos bei **u** nach **t**. Ansonsten wie **s** und **S**. Siehe **byteOrder** in tcl_platform.

**i**

Die Daten werden als *count* vorzeichenbehaftete 32-Bit-Ganzzahlen in Little-Endian-Byte-Reihenfolge interpretiert, oder vorzeichenlos bei **u** nach **i**. Speicherung als Liste. *count* "*****": alle verbleibenden Bytes. Fehlt *count*: eine 32-Bit-Ganzzahl. Beispiel:

```
set str \x05\x00\x00\x00\x07\x00\x00\x00\xF0\xFF\xFF\xFF
binary scan $str i2i* var1 var2
```

gibt **2** zurück, *var1* **5 7**, *var2* **-16**. Vorzeichenbehaftet, außer bei **iu** statt **i**.

**I**

Wie **i**, aber *count* 32-Bit-Ganzzahlen in Big-Endian-Byte-Reihenfolge, oder vorzeichenlos bei **u** nach **I**. Beispiel:

```
set str \x00\x00\x00\x05\x00\x00\x00\x07\xFF\xFF\xFF\xF0
binary scan $str I2I* var1 var2
```

gibt **2** zurück, *var1* **5 7**, *var2* **-16**.

**n**

Die Daten werden als *count* vorzeichenbehaftete 32-Bit-Ganzzahlen in der nativen Byte-Reihenfolge interpretiert, oder vorzeichenlos bei **u** nach **n**. Ansonsten wie **i** und **I**. Siehe **byteOrder** in tcl_platform.

**w**

Die Daten werden als *count* vorzeichenbehaftete 64-Bit-Ganzzahlen in Little-Endian-Byte-Reihenfolge interpretiert, oder vorzeichenlos bei **u** nach **w**. Speicherung als Liste. *count* "*****": alle verbleibenden Bytes. Fehlt *count*: eine 64-Bit-Ganzzahl. Beispiel:

```
set str \x05\x00\x00\x00\x07\x00\x00\x00\xF0\xFF\xFF\xFF
binary scan $str wi* var1 var2
```

gibt **2** zurück, *var1* **30064771077**, *var2* **-16**.

**W**

Wie **w**, aber Big-Endian-Byte-Reihenfolge, oder vorzeichenlos bei **u** nach **W**. Beispiel:

```
set str \x00\x00\x00\x05\x00\x00\x00\x07\xFF\xFF\xFF\xF0
binary scan $str WI* var1 var2
```

gibt **2** zurück, *var1* **21474836487**, *var2* **-16**.

**m**

Die Daten werden als *count* vorzeichenbehaftete 64-Bit-Ganzzahlen in der nativen Byte-Reihenfolge interpretiert, oder vorzeichenlos bei **u** nach **m**. Ansonsten wie **w** und **W**. Siehe **byteOrder** in tcl_platform.

**f**

Die Daten werden als *count* Gleitkommazahlen einfacher Genauigkeit in der nativen Darstellung interpretiert und als Liste gespeichert. *count* "*****": alle verbleibenden Bytes. Fehlt *count*: eine Zahl einfacher Genauigkeit. Die Byteanzahl kann variieren. Ungültige Gleitkommadaten: Ergebnis undefiniert und compilerabhängig. Beispiel auf MS-Windows mit einem x86/x64-Prozessor:

```
binary scan \xCD\xCC\xCC\x3F f var1
```

gibt **1** zurück, in *var1* steht **1.6000000238418579**.

**r**

Wie **f**, aber Gleitkommazahlen einfacher Genauigkeit in Little-Endian-Reihenfolge. Nicht portabel auf Systeme ohne IEEE-Gleitkomma.

**R**

Wie **f**, aber Big-Endian-Reihenfolge. Nicht portabel ohne IEEE-Gleitkomma.

**d**

Wie **f**, aber Gleitkommazahlen doppelter Genauigkeit in der nativen Darstellung. Beispiel auf Windows mit Intel Pentium:

```
binary scan \x9A\x99\x99\x99\x99\x99\xF9\x3F d var1
```

gibt **1** zurück, in *var1* steht **1.6000000000000001**.

**q**

Wie **d**, aber doppelte Genauigkeit in Little-Endian-Reihenfolge. Nicht portabel ohne IEEE-Gleitkomma.

**Q**

Wie **d**, aber Big-Endian-Reihenfolge. Nicht portabel ohne IEEE-Gleitkomma.

**x**

Bewegt den Cursor *count* Bytes vorwärts in *string*. Bei *count* "*" oder größer als die verbleibenden Bytes: Cursor hinter das letzte Byte. Fehlt *count*: ein Byte vor. Verbraucht kein Argument. Beispiel:

```
binary scan \x01\x02\x03\x04 x2H* var1
```

gibt **1** zurück, in *var1* steht **0304**.

**X**

Setzt den Cursor *count* Bytes in *string* zurück. Bei *count* "*" oder größer als die aktuelle Position: Position 0. Fehlt *count*: ein Byte zurück. Verbraucht kein Argument. Beispiel:

```
binary scan \x01\x02\x03\x04 c2XH* var1 var2
```

gibt **2** zurück, *var1* **1 2**, *var2* **020304**.

**@**

Setzt den Cursor auf die absolute Position *count* in der Datenzeichenkette. Position 0 ist das erste Byte von *string*. Liegt *count* hinter dem Ende, steht der Cursor hinter dem letzten Byte. Fehlt *count*: Fehler. Beispiel:

```
binary scan \x01\x02\x03\x04 c2@1H* var1 var2
```

gibt **2** zurück, *var1* **1 2**, *var2* **020304**.

## Portabilitätsfragen

Die Konvertierungen **r**, **R**, **q** und **Q** funktionieren beim Datentransfer zwischen Rechnern zuverlässig nur, wenn alle IEEE-Gleitkommadarstellung verwenden — sehr verbreitet, aber nicht universell. Für portablen Gleitkommatransport zwischen allen Architekturen die Textdarstellung (wie von format erzeugt) verwenden.

## Beispiele

Prozedur zum Schreiben einer Tcl-Zeichenkette als UTF-8-Daten mit vorangestelltem Längenwort auf einen binär kodierten Kanal:

```
proc writeString {channel string} {
    set data [encoding convertto utf-8 $string]
    puts -nonewline [binary format Ia* \
            [string length $data] $data]
}
```

Prozedur zum Lesen einer Zeichenkette von einem Kanal, der von der obigen *writeString* -Prozedur geschrieben wurde:

```
proc readString {channel} {
    if {![binary scan [read $channel 4] I length]} {
        error "missing length"
    }
    set data [read $channel $length]
    return [encoding convertfrom utf-8 $data]
}
```

Wandelt den Inhalt einer Datei (Name in *filename*) in base64 um und gibt ihn aus:

```
set f [open $filename rb]
set data [read $f]
close $f
puts [binary encode base64 -maxlen 64 $data]
```

*Copyright (c) 1997 Sun Microsystems, Inc.*
*Copyright (c) 2008 Donal K. Fellows*

# break

*Stichwörter:* break, abort, loop

## Name

break - Bricht eine Schleife ab

## Übersicht

**break**

## Beschreibung

Dieser Befehl wird typischerweise im Rumpf eines Schleifenbefehls wie for, foreach oder while aufgerufen. Er liefert den Ergebniscode 3 (TCL_BREAK), wodurch eine Break-Ausnahme ausgelöst wird. Die Ausnahme bricht die aktuelle Skriptausführung bis zum innersten enthaltenden Schleifenbefehl ab, der seine Ausführung dann beendet und normal zurückkehrt. Break-Ausnahmen werden auch in einigen anderen Situationen behandelt, etwa beim Befehl catch, in Tk-Event-Bindings und in den äußersten Skripten von Prozedurrümpfen.

## Beispiele

Gibt für jede Ganzzahl von 0 bis 5 eine Zeile aus:

```
for {set x 0} {$x<10} {incr x} {
    if {$x > 5} {
        break
    }
    puts "x is $x"
}
```

*Copyright (c) 1993-1994 The Regents of the University of California.*

*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# buildinfo

*Stichwörter:* buildinfo, build-info

## Name

buildinfo - Build-Informationen

## Übersicht

**::tcl::build-info** ?*field*?

## Beschreibung

Dieser Befehl liefert Informationen darüber, wie Tcl gebaut wurde. Ohne Argumente: Tcl-Patchlevel, dann "+", dann die Fossil-Commit-ID, gefolgt von einer Liste punktgetrennter Tags. Ist *field* angegeben, wird das entsprechende Feld extrahiert (siehe unten). Jeder andere *field*-Wert, der unten nicht genannt ist, liefert "0". Bei offiziellen Tcl-Releases gelten folgende *field*-Werte:

**::tcl::build-info clang**
Clang-Versionsnummer (4 Ziffern), wenn Tcl mit Clang kompiliert wurde, sonst 0.

**::tcl::build-info commit**
Fossil-Commit-ID, aus der Tcl gebaut wurde.

**::tcl::build-info compiledebug**
1, wenn Tcl mit **-DTCL_COMPILE_DEBUG** kompiliert wurde, sonst 0.

**::tcl::build-info compiler**
Compilername (clang, gcc, icc oder msvc), Bindestrich, (4-stellige) Versionsnummer.

**::tcl::build-info compilestats**
1, wenn Tcl mit **-DTCL_COMPILE_STATS** kompiliert wurde, sonst 0.

**::tcl::build-info cplusplus**
1, wenn Tcl mit einem C++-Compiler kompiliert wurde, sonst 0.

**::tcl::build-info debug**
1, wenn Tcl nicht mit **-DNDEBUG** kompiliert wurde, sonst 0.

**::tcl::build-info gcc**
GCC-Versionsnummer (4 Ziffern) bei GCC-Build, sonst 0.

**::tcl::build-info icc**
ICC-Versionsnummer (4 Ziffern) bei ICC-Build, sonst 0.

**::tcl::build-info ilp32**
1, wenn Integer, Long und Zeiger jeweils 32 Bit sind, sonst 0.

**::tcl::build-info memdebug**
1, wenn Tcl mit **-DTCL_MEM_DEBUG** kompiliert wurde, sonst 0.

**::tcl::build-info msvc**
MSVC-Versionsnummer (4 Ziffern) bei MSVC-Build, sonst 0.

**::tcl::build-info nmake**
1, wenn Tcl mit nmake gebaut wurde, sonst 0.

**::tcl::build-info no-deprecate**
1, wenn Tcl mit **-DTCL_NO_DEPRECATED** kompiliert wurde, sonst 0.

**::tcl::build-info no-thread**
1, wenn Tcl mit **-DTCL_THREADS=0** kompiliert wurde, sonst 0.

**::tcl::build-info no-optimize**
1, wenn Tcl nicht mit **-DTCL_CFG_OPTIMIZED** kompiliert wurde, sonst 0.

**::tcl::build-info objective-c**
1, wenn Tcl mit einem Objective-C-Compiler kompiliert wurde, sonst 0.

**::tcl::build-info objective-cplusplus**
1, wenn Tcl mit einem Objective-C++-Compiler kompiliert wurde, sonst 0.

**::tcl::build-info patchlevel**
Tcl-Patchlevel, wie info patchlevel.

**::tcl::build-info profile**
1, wenn Tcl mit **-DTCL_CFG_PROFILED** kompiliert wurde, sonst 0.

**::tcl::build-info purify**
1, wenn Tcl mit **-DPURIFY** kompiliert wurde, sonst 0.

**::tcl::build-info static**
1, wenn Tcl als statische Bibliothek kompiliert wurde, sonst 0.

**::tcl::build-info tommath**
libtommath-Versionsnummer (4 Ziffern), wenn libtommath in Tcl eingebaut ist, sonst 0.

**::tcl::build-info version**
Tcl-Version, wie info tclversion.

**::tcl::build-info zlib**
zlib-Versionsnummer (4 Ziffern), wenn zlib in Tcl eingebaut ist, sonst 0.

### Beispiele

Verwendung von **::tcl::build-info**:

```
::tcl::build-info
     → 9.0.2+af16c07b81655fabde8028374161ad54b84ef9956843c63f49976b4ef6
    01b611.gcc-1204
::tcl::build-info commit
     → af16c07b81655fabde8028374161ad54b84ef9956843c63f49976b4ef601b611
::tcl::build-info compiler
     → gcc-1204
::tcl::build-info gcc
     → 1204
::tcl::build-info version
     → 9.0
::tcl::build-info patchlevel
     → 9.0.2
```

*Copyright (c) 2025 Jan Nijtmans.*

## callback

*Stichwörter:* callback, object

### Name

callback, mymethod - Erzeugt Rückrufe auf Methoden

## Übersicht

**package require tcl::oo**

**callback** *methodName* ?*arg* ...?
**mymethod** *methodName* ?*arg* ...?

## Beschreibung

Der Befehl **callback**, auch **mymethod** genannt (Kompatibilität mit den ooutil- und snit-Paketen von Tcllib), darf nur im Kontext eines Methodenaufrufs verwendet werden (also im Körper einer Methode, eines Konstruktors oder Destruktors). Er erzeugt ein Skriptfragment, das beim Ausführen die Methode *methodName* auf dem aktuellen Objekt (gemeldet durch self) aufruft. Zusätzliche Argumente werden als führende Argumente des Rückrufs übergeben. Das resultierende Skriptfragment ist eine gültige Liste.

Der Aufrufer muss sicherstellen, dass das aktuelle Objekt *methodName* verarbeiten kann; dieser Befehl prüft das nicht. *methodName* kann jede exportierte oder nicht exportierte Methode bezeichnen, aber keine private Methode, da diese nur direkt aus Methoden aufgerufen werden können. Existiert beim Aufruf des Rückrufs keine solche Methode, wird der Standard-**unknown** -Methoden-Handler aufgerufen.

## Beispiele

Eine einfache Echo-Server-Klasse. **callback** wird zweimal verwendet: für eingehende Socket-Verbindungen (**Accept**) und für eingehende Bytes auf diesen Verbindungen (**Receive**).

```
oo::class create EchoServer {
    variable server clients
    constructor {port} {
        set server [socket -server [callback Accept] $port]
        set clients {}
    }
    destructor {
        chan close $server
        foreach client [dict keys $clients] {
            chan close $client
        }
    }

    method Accept {channel clientAddress clientPort} {
        dict set clients $channel [dict create \
                address $clientAddress port $clientPort]
        chan event $channel readable [callback Receive $channel]
    }
    method Receive {channel} {
        if {[chan gets $channel line] >= 0} {
            my echo $channel $line
        } else {
            chan close $channel
            dict unset clients $channel
        }
    }

    method echo {channel line} {
        dict with clients $channel {
            chan puts $channel \
                    [format {[%s:%d] %s} $address $port $line]
        }
    }
}
```

*Copyright (c) 2018 Donal K. Fellows*

# catch

*Stichwörter:* catch, error, exception

## Name

catch - Wertet ein Skript aus und fängt außergewöhnliche Rückgaben ab

## Übersicht

**catch** *script* ?*resultVarName*? ?*optionsVarName*?

## Beschreibung

Der Befehl **catch** verhindert, dass Fehler die Befehlsauswertung abbrechen. **catch** ruft den Tcl-Interpreter rekursiv auf, um *script* auszuführen, und kehrt stets ohne Fehler zurück — unabhängig davon, ob bei der Ausführung von *script* Fehler auftreten.

Löst *script* einen Fehler aus, gibt **catch** eine von Null verschiedene Ganzzahl zurück, die dem außergewöhnlichen Rückgabecode der Skriptauswertung entspricht. Tcl definiert den normalen Rückgabecode als Null (0) bzw. **TCL_OK**. Vier außergewöhnliche Rückgabecodes sind: 1 ( **TCL_ERROR**), 2 (**TCL_RETURN**), 3 (**TCL_BREAK**) und 4 (**TCL_CONTINUE**). Fehler während der Skriptauswertung entsprechen **TCL_ERROR**. Die anderen außergewöhnlichen Codes liefern return, break und continue sowie in der Dokumentation beschriebene Sonderfälle. Neue Befehle aus Tcl-Paketen und Skripte mit return -code können andere Ganzzahlen zurückgeben; diese müssen außerhalb des für Tcl reservierten Bereichs liegen (siehe return).

Ist *resultVarName* angegeben, wird die benannte Variable auf das Ergebnis der Skriptauswertung gesetzt. Ist der Rückgabecode 1 (**TCL_ERROR**), enthält *resultVarName* eine Fehlermeldung. Ist der Rückgabecode 0 (**TCL_OK**), enthält *resultVarName* den von *script* zurückgegebenen Wert.

Ist *optionsVarName* angegeben, wird die benannte Variable auf ein Dictionary der Rückgabeoptionen gesetzt. Tcl definiert stets die Einträge **-code** und **-level**. Ist der Rückgabecode nicht **TCL_RETURN**, ist **-level** 0 und **-code** entspricht dem Rückgabecode. Nur bei **TCL_RETURN** können **-level** und **-code** andere Werte annehmen (siehe return).

Bei **TCL_ERROR** enthält das Dictionary vier weitere Einträge: **-errorinfo**, **-errorcode**, **-errorline** und **-errorstack**. **-errorinfo** ist ein formatierter Stack-Trace für Menschen. **-errorcode** ist zusätzliche Fehlerinformation als Liste für Programme. **-errorline** ist die

Zeilennummer in *script*, in der der Fehler auftrat. **-errorstack** ist eine Liste gerader Länge aus Token-Parameter-Paaren beim Stack-Unwinden: Token **CALL** mit Prozedurname und Argumenten der Ebene, oder **UP** mit relativer Ebene (wie bei uplevel) zur vorherigen **CALL**. Unterschiede zu **-errorinfo**:

1. maschinenlesbare Form, geeignet für [foreach {tok prm} ...],
2. echte (substituierte) Argumentwerte statt statischem Aufruftext,
3. gröbere Granularität: ein Element pro Stack-Frame (keine separaten Elemente für foreach -Konstrukte).

Die Werte von **-errorinfo** und **-errorcode** des letzten Fehlers stehen auch in **::errorInfo** und **::errorCode**. **-errorstack** entspricht info errorstack.

Tcl-Pakete können weitere Dictionary-Einträge setzen; Skripte können mit return zusätzliche Rückgabeoptionen setzen.

## Beispiele

**catch** eignet sich in if, um nach Erfolg eines Skripts zu verzweigen:

```
if { [catch {open $someFile w} fid] } {
    puts stderr "Could not open $someFile for writing\n$fid"
    exit 1
}
```

Weitere **catch**-Beispiele stehen in der Dokumentation zu return.

*Copyright (c) 1993-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# cd

*Stichwörter:* cd, working directory

## Name

cd - Wechselt das Arbeitsverzeichnis

## Übersicht

**cd** ?*dirName*?

## Beschreibung

Wechselt das aktuelle Arbeitsverzeichnis zu *dirName* oder, wenn *dirName* nicht angegeben ist, zum Home-Verzeichnis (wie in der Umgebungsvariable HOME festgelegt). Gibt eine leere Zeichenkette zurück.

Das aktuelle Arbeitsverzeichnis ist eine ressourcenbezogene Größe pro Prozess; der Befehl **cd** ändert das Arbeitsverzeichnis für alle Interpreter und alle Threads.

## Beispiele

Wechselt ins Home-Verzeichnis des Benutzers **fred**:

```
cd [file home fred]
```

Wechselt in das Verzeichnis **lib**, das ein Geschwisterverzeichnis des aktuellen ist:

```
cd ../lib
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# chan

*Stichwörter:* chan, blocking, channel, end of file, events, input, non-blocking, offset, output, readable, seek, stdio, tell, writable

## Name

chan - Lesen, Schreiben und Manipulieren von Kanälen

## Übersicht

**chan** *option* ?*arg arg* ...?

## Beschreibung

Dieser Befehl bietet mehrere Operationen zum Lesen von, Schreiben auf und sonstigen Manipulieren geöffneter Kanäle (wie sie mit den Befehlen open und socket erzeugt wurden oder wie die Standardkanäle **stdin**, **stdout** und **stderr**, die den Standard-Eingabe-, Ausgabe- und Fehlerströmen des Prozesses entsprechen). *Option* gibt an, was mit dem Kanal geschehen soll; jede eindeutige Abkürzung von *option* ist zulässig. Gültige Optionen sind:

**chan blocked *channel***

Prüft, ob die letzte Eingabeoperation auf dem Kanal *channel* fehlgeschlagen ist, weil sie den Prozess sonst hätte blockieren lassen, und gibt 1 zurück, wenn das der Fall war. Andernfalls gibt sie 0 zurück. Beachten Sie, dass hier nur dann 1 zurückgegeben wird, wenn der Kanal für den nicht blockierenden Modus konfiguriert ist; alle Tcl-Kanäle haben standardmäßig Blockierung eingeschaltet.

**chan close *channel* ?*direction*?**

Schließt und zerstört den Kanal *channel*. Beachten Sie, dass dabei alle auf dem Kanal registrierten Datei-Ereignisse gelöscht werden. Ist das Argument *direction* (das **read** oder **write** oder eine eindeutige Abkürzung davon sein muss) angegeben, wird der Kanal nur halb geschlossen, sodass er von Lese-Schreib- auf nur-Schreib- bzw. nur-Lese-Modus wechseln kann. Wird ein nur-Lese-Kanal zum Lesen geschlossen, entspricht das einem vollständigen Schließen des Kanals; analog für nur-Schreib-Kanäle. Ohne das Argument *direction* wird der Kanal sowohl zum Lesen als auch zum Schreiben geschlossen (aber nur, wenn diese Richtungen derzeit geöffnet sind). Es ist ein Fehler, einen nur-Lese-Kanal zum Schreiben oder einen nur-Schreib-Kanal zum Lesen zu schließen.

Beim Schließen des Kanals wird die gesamte gepufferte Ausgabe auf das Ausgabegerät des Kanals geschrieben (nur wenn der Kanal nicht mehr beschreibbar wird), gepufferte Eingabe verworfen (nur wenn der Kanal nicht mehr lesbar wird), die zugrunde liegende Betriebssystemressource geschlossen und *channel* für künftige Verwendung ungültig (jeweils nur wenn der Kanal vollständig geschlossen wird).

Ist der Kanal blockierend und wird er nicht mehr beschreibbar, kehrt der Befehl erst zurück, wenn die gesamte Ausgabe geleert wurde. Ist der Kanal nicht blockierend und gibt es noch nicht geleerte Ausgabe, bleibt der Kanal geöffnet und der Befehl kehrt sofort zurück; die Ausgabe wird im Hintergrund geleert und der Kanal wird geschlossen, wenn das Leeren abgeschlossen ist.

Ist *channel* ein blockierender Kanal einer Befehlspipeline, wartet **chan close**, bis die Kindprozesse beendet sind.

Wird der Kanal zwischen Interpretern geteilt, macht **chan close** *channel* im aufrufenden Interpreter unverfügbar, hat aber keine weiteren Auswirkungen, bis alle teilenden Interpreter den Kanal geschlossen haben. Wenn der letzte Interpreter, in dem der Kanal registriert ist, **chan close** (oder close) aufruft, erfolgen die oben beschriebenen Aufräumarbeiten. Beim Halbschließen gilt das Halbschließen des Kanals nur für die Sicht des aktuellen Interpreters, bis alle Interpreter ihn in dieser Richtung (oder vollständig) geschlossen haben. Siehe den Befehl interp für eine Beschreibung der Kanalfreigabe.

Kanäle werden automatisch vollständig geschlossen, wenn ein Interpreter zerstört wird und wenn der Prozess beendet wird. Kanäle werden in den blockierenden Modus geschaltet, um sicherzustellen, dass die gesamte Ausgabe vor Prozessende korrekt geleert wird.

Der Befehl gibt eine leere Zeichenkette zurück und kann einen Fehler erzeugen, wenn beim Leeren der Ausgabe ein Fehler auftritt. Gibt ein Befehl in einer mit open erzeugten Befehlspipeline einen Fehler zurück, erzeugt **chan close** einen Fehler (ähnlich dem Befehl exec).

Beachten Sie, dass Halbschließen von Sockets und Befehlspipelines wichtige Nebenwirkungen haben kann, weil dabei **shutdown()** oder **close()** der zugrunde liegenden Systemressource ausgelöst wird, was das Verhalten anderer Prozesse oder Systeme gegenüber dem Tcl-Programm ändern kann.

Kanäle werden automatisch geschlossen, wenn ein Interpreter zerstört wird und wenn der Prozess beendet wird. Nicht blockierende Kanäle werden beim Beenden nicht in den blockierenden Modus geschaltet; das garantiert ein zeitnahes Beenden auch dann, wenn der Peer oder ein Kommunikationskanal hängt. Um das ordnungsgemäße Leeren hängender nicht blockierender Kanäle beim Beenden sicherzustellen, muss man sie entweder (a) aktiv wieder in den blockierenden Modus schalten oder (b) die Umgebungsvariable **TCL_FLUSH_NONBLOCKING_ON_EXIT** verwenden.

**chan configure *channel* ?*optionName*? ?*value*? ?*optionName value* ...?**

Abfragen oder Setzen der Konfigurationsoptionen des Kanals *channel*.

Werden keine Argumente *optionName* oder *value* angegeben, gibt der Befehl eine Liste mit abwechselnden Optionsnamen und Werten für den Kanal zurück. Ist *optionName* angegeben, aber kein *value*, gibt der Befehl den aktuellen Wert der angegebenen Option zurück. Werden ein oder mehrere Paare *optionName* und *value* angegeben, setzt der Befehl jede benannte Option auf den entsprechenden *value*; in diesem Fall ist der Rückgabewert eine leere Zeichenkette.

Die unten beschriebenen Optionen werden für alle Kanäle unterstützt. Zusätzlich kann jeder Kanaltyp eigene Optionen hinzufügen. Siehe den Manpage-Eintrag des Befehls, der den jeweiligen Kanaltyp erzeugt, für die von diesem Typ unterstützten Optionen. Beispielsweise enthält der Manpage-Eintrag zum Befehl socket zusätzliche Optionen für Sockets und der Befehl open zusätzliche Optionen für serielle Geräte.

**-blocking *boolean***

Die Option **-blocking** legt fest, ob E/A-Operationen auf dem Kanal den Prozess unbegrenzt blockieren können. Der Wert der Option muss ein gültiger boolescher Wert sein. Kanäle sind normalerweise im blockierenden Modus; wird ein Kanal in den nicht blockierenden Modus versetzt, beeinflusst das die Operationen der Befehle **chan gets**, **chan read**, **chan puts**, **chan flush** und **chan close**; Einzelheiten finden sich in deren Dokumentation. Damit der nicht blockierende Modus korrekt funktioniert, muss die Anwendung die Tcl-Ereignisschleife verwenden (z. B. durch Aufruf von Tcl_DoOneEvent oder den Befehl vwait).

**-buffering *newValue***

Ist *newValue* **full**, puffert das E/A-System die Ausgabe, bis der interne Puffer voll ist oder der Befehl **chan flush** aufgerufen wird. Ist *newValue* **line**, leert das E/A-System die Ausgabe für den Kanal automatisch, sobald ein Zeilenumbruch ausgegeben wird. Ist *newValue* **none**, leert das E/A-System nach jeder Ausgabeoperation automatisch. Standardmäßig ist **-buffering** auf **full** gesetzt, außer bei Kanälen, die mit terminalähnlichen Geräten verbunden sind; für diese Kanäle ist die Anfangseinstellung **line**. Zusätzlich sind **stdin** und **stdout** anfänglich auf **line** gesetzt, **stderr** auf **none**.

**-buffersize *newSize***

*newSize* muss eine Ganzzahl sein; ihr Wert wird verwendet, um die Größe der für diesen Kanal anschließend zugewiesenen Ein- oder Ausgabepuffer in Bytes festzulegen. *newSize* darf höchstens eine Million sein, sodass Puffer bis zu einer Million Bytes groß sein können.

**-encoding *name***

Diese Option legt die Kodierung des Kanals als eine der von encoding names zurückgegebenen benannten Kodierungen fest, damit die Daten für die Verwendung in Tcl nach Unicode konvertiert werden können. Damit Tcl beispielsweise Zeichen aus einer japanischen Datei in **shiftjis** liest und den Inhalt korrekt verarbeitet und anzeigt, würde die Kodierung auf **shiftjis** gesetzt. Beim Lesen vom Kanal werden die Bytes in der japanischen Datei beim Lesen nach Unicode konvertiert. Schreiben wird ebenfalls unterstützt — Tcl-Zeichenketten werden beim Schreiben auf den Kanal automatisch in die angegebene Kodierung konvertiert.

Enthält eine Datei reine Binärdaten (z. B. ein JPEG-Bild), sollte die Kodierung des Kanals auf **iso8859-1** gesetzt werden. Tcl weist den Daten in der Datei dann keine Interpretation zu und liest oder schreibt einfach rohe Bytes. Der Tcl-Befehl binary kann zur Bearbeitung dieser byteorientierten Daten verwendet werden. In der Regel ist es besser, die Option **-translation** auf binary zu setzen, wenn Binärdaten übertragen werden sollen, da damit auch die anderen automatischen Interpretationen der Bytes im Strom abgeschaltet werden.

Die Standardkodierung für neu geöffnete Kanäle ist dieselbe plattform- und localeabhängige Systemkodierung, die für die Schnittstelle zum Betriebssystem verwendet wird, wie von encoding system zurückgegeben.

**-eofchar *char***

Diese Option unterstützt DOS-Dateisysteme, die Control-z (\x1A) als Dateiende-Markierung verwenden. Ist *char* keine leere Zeichenkette, signalisiert dieses Zeichen beim Lesen das Dateiende. Andernfalls (Standard) gibt es keine spezielle Dateiende-Zeichenmarkierung. Der zulässige Bereich für **-eofchar**-Werte ist \x01 - \x7f; der Versuch, **-eofchar** auf einen Wert außerhalb dieses Bereichs zu setzen, erzeugt einen Fehler.

**-profile *profile***

Legt das für den Kanal zu verwendende Kodierungsprofil fest. Die für Ein- und Ausgabe des Kanals verwendeten Kodierungstransformationen unterliegen dann den Regeln dieses Profils. Fehler führen zu einem Kanalfehler. Siehe PROFILE in der Dokumentation zu encoding(n) für Einzelheiten zu Kodierungsprofilen.

**-translation *translation***

...siehe nächstes...

**-translation *{inTranslation outTranslation}***

In Tcl-Skripten wird das Ende einer Zeile immer durch ein einzelnes Newline-Zeichen (\n) dargestellt. In tatsächlichen Dateien und Geräten kann das Zeilenende jedoch auf verschiedenen Plattformen oder sogar für verschiedene Geräte auf derselben Plattform unterschiedlich dargestellt sein. Unter UNIX werden in Dateien beispielsweise Newlines verwendet, während in Netzwerkverbindungen normalerweise Wagenrücklauf-Zeilenvorschub-Sequenzen verwendet werden. Bei der Eingabe (d. h. mit **chan gets** und **chan read**) übersetzt das Tcl-E/A-System die externe Zeilenende-Darstellung automatisch in Newline-Zeichen. Bei der Ausgabe (d. h. mit **chan puts**) übersetzt das E/A-System Newlines in die externe Zeilenende-Darstellung. Der Standardübersetzungsmodus **auto** behandelt alle gängigen Fälle automatisch, aber die Option **-translation** bietet explizite Kontrolle über die Zeilenende-Übersetzungen.

Der mit **-translation** verknüpfte Wert ist ein einzelnes Element für Nur-Lese- und Nur-Schreib-Kanäle. Für Lese-Schreib-Kanäle ist der Wert eine Liste mit zwei Elementen; der Leseübersetzungsmodus ist das erste Element der Liste, der Schreibübersetzungsmodus das zweite. Der Einfachheit halber kann beim Setzen des Übersetzungsmodus für einen Lese-Schreib-Kanal ein einzelner Wert angegeben werden, der für Lesen und Schreiben gilt. Beim Abfragen des Übersetzungsmodus eines Lese-Schreib-Kanals wird immer eine Liste mit zwei Elementen zurückgegeben. Derzeit werden folgende Werte unterstützt:

**auto**

Als Eingabeübersetzungsmodus behandelt **auto** jedes von Newline (**lf**), Wagenrücklauf (**cr**) oder Wagenrücklauf gefolgt von einem Zeilenvorschub (**crlf**) als Zeilenende-Darstellung. Die Zeilenende-Darstellung kann sogar von Zeile zu Zeile wechseln; alle Fälle werden in ein Newline übersetzt. Als Ausgabeübersetzungsmodus wählt **auto** eine plattformspezifische Darstellung; für Sockets auf allen Plattformen wählt Tcl **crlf**, für alle Unix-Varianten **lf** und für die verschiedenen Windows-Varianten **crlf**. Die Standardeinstellung für **-translation** ist **auto** sowohl für Eingabe als auch Ausgabe.

binary

Wie **lf** wird keine Zeilenende-Übersetzung durchgeführt, setzt aber zusätzlich **-eofchar** auf die leere Zeichenkette, um es zu deaktivieren, und **-encoding** auf **iso8859-1**. Mit dieser einen Einstellung ist ein Kanal vollständig für binäre Ein- und Ausgabe konfiguriert: Jedes vom Kanal gelesene Byte wird zum Unicode-Zeichen mit demselben Wert wie dieses Byte, und jedes in den Kanal geschriebene Zeichen wird zu einem einzelnen Byte in der

Ausgabe. So kann nahtlos mit Binärdaten gearbeitet werden, solange jedes Zeichen in den Daten im Bereich 0 bis 255 bleibt, sodass kein Unterschied zwischen Binärdaten und Text besteht. Beispielsweise kann ein JPEG-Bild von einem solchen Kanal gelesen, bearbeitet und wieder auf einen solchen Kanal geschrieben werden.

**cr**

Das Ende einer Zeile in der zugrunde liegenden Datei oder dem Gerät wird durch ein einzelnes Wagenrücklauf-Zeichen dargestellt. Als Eingabeübersetzungsmodus konvertiert der Modus **cr** Wagenrückläufe in Newline-Zeichen. Als Ausgabeübersetzungsmodus übersetzt **cr** Newline-Zeichen in Wagenrückläufe.

**crlf**

Das Ende einer Zeile in der zugrunde liegenden Datei oder dem Gerät wird durch ein Wagenrücklauf-Zeichen gefolgt von einem Zeilenvorschub dargestellt. Als Eingabeübersetzungsmodus konvertiert der Modus **crlf** Wagenrücklauf-Zeilenvorschub-Sequenzen in Newline-Zeichen. Als Ausgabeübersetzungsmodus übersetzt **crlf** Newline-Zeichen in Wagenrücklauf-Zeilenvorschub-Sequenzen. Dieser Modus wird typischerweise auf Windows-Plattformen und für Netzwerkverbindungen verwendet.

**lf**

Das Ende einer Zeile in der zugrunde liegenden Datei oder dem Gerät wird durch ein einzelnes Newline- (Zeilenvorschub-)Zeichen dargestellt. In diesem Modus erfolgen weder bei der Eingabe noch bei der Ausgabe Übersetzungen. Dieser Modus wird typischerweise auf UNIX-Plattformen verwendet.

**chan copy *inputChan outputChan* ?*-size size*? ?*-command callback*?**

Liest Zeichen von *inputChan* und schreibt sie nach *outputChan*, bis alle Zeichen kopiert sind, blockiert bis der Kopiervorgang abgeschlossen ist und gibt die Anzahl der kopierten Zeichen zurück. Nutzt interne Puffer, um zusätzliche Kopien zu vermeiden und zu verhindern, dass beim Kopieren großer Dateien auf langsame Ziele wie Netzwerk-Sockets zu viele Daten im Hauptspeicher gepuffert werden.

**-size** begrenzt die Anzahl der zu kopierenden Zeichen.

Ist **-command** angegeben, kehrt **chan copy** sofort zurück, arbeitet im Hintergrund und ruft *callback* nach Abschluss des Kopiervorgangs auf, wobei als zusätzliches Argument die Anzahl der nach *outputChan* geschriebenen Zeichen übergeben wird. Tritt während des Hintergrundkopierens ein Fehler auf, liefert ein weiteres Argument die Fehlermeldung. *inputChan* und *outputChan* werden bei Bedarf automatisch für den nicht blockierenden Modus konfiguriert. Hintergrundkopieren funktioniert nur korrekt, wenn Ereignisse verarbeitet werden, z. B. über vwait oder Tk.

Während eines Hintergrundkopierens darf auf *inputChan* keine andere Leseoperation und auf *outputChan* keine Schreiboperation ausgeführt werden. Schreiboperationen auf *inputChan* und Leseoperationen auf *outputChan* sind jedoch möglich, wie im bidirektionalen Kopierbeispiel unten gezeigt.

Wird entweder *inputChan* oder *outputChan* während des Kopiervorgangs geschlossen, bricht das Kopieren ab und es wird **kein** Callback aufgerufen. Wird *inputChan* geschlossen, wird die gesamte bereits in der Warteschlange befindliche Ausgabe nach *outputChan* geschrieben.

Für *inputChan* sollte kein Ereignishandler eingerichtet sein, da er während eines Hintergrundkopierens lesbar werden kann. Ein Lese- oder Schreibversuch aus einem Ereignishandler führt zum Fehler "channel busy". Jede falsch gerichtete E/A (durch einen **chan event**-Handler oder anderweitig) führt zu einem Fehler "channel busy".

**chan create *mode cmdPrefix***

Dieser Unterbefehl erzeugt einen neuen Kanal auf Skriptebene, der das Befehlspräfix *cmdPrefix* als Handler verwendet. Ein solcher Kanal heißt **reflektierter** Kanal. Das angegebene Befehlspräfix **cmdPrefix** muss eine nicht leere Liste sein und sollte die in der Manpage refchan beschriebene API bereitstellen. Der Bezeichner des neuen Kanals wird als Ergebnis des Befehls **chan create** zurückgegeben, und der Kanal ist geöffnet. Verwenden Sie close oder **chan close**, um den Kanal zu entfernen.

Das Argument *mode* gibt an, ob der neue Kanal zum Lesen, Schreiben oder beidem geöffnet ist. Es muss eine Liste sein, die beliebige der Zeichenketten "**read**" oder "**write**" enthält. Die Liste muss mindestens ein Element haben, da ein Kanal, auf den man weder schreiben noch lesen kann, keinen Sinn ergibt. Der Handler-Befehl für den neuen Kanal muss den gewählten Modus unterstützen, sonst wird ein Fehler ausgelöst.

Das Befehlspräfix wird im globalen Namespace am oberen Ende des Aufrufstapels ausgeführt, nach Anhängen der Argumente wie in der Manpage refchan beschrieben. Die Befehlsauflösung erfolgt zum Zeitpunkt des Aufrufs. Das Umbenennen oder Zerstören des Befehls bedeutet, dass der nächste Aufruf einer Handler-Methode fehlschlagen kann und damit auch der den Handler aufrufende Kanalbefehl; je nach aufgerufenem Unterbefehl kann die Fehlermeldung den Grund dafür nicht erklären.

Jeder mit diesem Unterbefehl erzeugte Kanal kennt den Interpreter, in dem er erzeugt wurde, und führt seinen Handler-Befehl nur in diesem Interpreter aus, auch wenn der Kanal geteilt und/oder in einen anderen Interpreter verschoben wurde. Jeder reflektierte Kanal kennt auch den Thread, in dem er erzeugt wurde, und führt seinen Handler-Befehl nur in diesem Thread aus, auch wenn der Kanal in einen anderen Thread verschoben wurde. Zu diesem Zweck werden alle Handler-Aufrufe durch Posten spezieller Ereignisse an den ursprünglichen Thread weitergeleitet. Das bedeutet, dass der ursprüngliche Thread (d. h. der Thread, der **chan create** ausgeführt hat) eine aktive Ereignisschleife haben muss, d. h. er muss solche Ereignisse verarbeiten können. Andernfalls blockiert der sendende Thread *unbegrenzt*. Es kann zu einer Deadlock-Situation kommen.

Beachten Sie, dass damit ein Kanal erzeugt werden kann, dessen beiden Endpunkte in zwei verschiedenen Threads leben und eine streamorientierte Brücke zwischen diesen Threads bildet. Mit anderen Worten: reguläre Stream-Kommunikation zwischen Threads statt Befehle senden zu müssen.

Wird ein Thread oder Interpreter gelöscht, werden alle mit diesem Unterbefehl erzeugten Kanäle, die diesen Thread/Interpreter als Rechenbasis verwenden, ebenfalls in allen

Interpretern gelöscht, mit denen sie geteilt oder in die sie verschoben wurden, und in jedem Thread, in den sie übertragen wurden. Obwohl das den anderen Thread(s) und/oder Interpreter(n) den Boden entzieht, lässt sich das nicht vermeiden. Der Versuch, einen solchen Kanal zu verwenden, führt zu einem regulären Fehler über unbekannte Kanalbezeichner.

Dieser Unterbefehl ist **safe** und für sichere Interpreter zugänglich. Obwohl er die Ausführung beliebigen Tcl-Codes veranlasst, stellt das System sicher, dass der Code immer innerhalb des sicheren Interpreters ausgeführt wird.

**chan eof *channel***

Prüft, ob die letzte Eingabeoperation auf dem Kanal *channel* fehlgeschlagen ist, weil das Ende des Datenstroms erreicht wurde; gibt 1 zurück, wenn Dateiende erreicht wurde, und 0 andernfalls.

**chan event *channel event* ?*script*?**

Richtet ein, dass das Tcl-Skript *script* als *Datei-Ereignishandler* installiert wird, der aufgerufen wird, sobald der Kanal *channel* in den von *event* beschriebenen Zustand wechselt (*event* muss **readable** oder **writable** sein); pro Ereignis und Kanal kann jeweils nur ein solcher Handler installiert sein. Ist *script* die leere Zeichenkette, wird der aktuelle Handler gelöscht (das geschieht auch beim Schließen des Kanals oder Löschen des Interpreters). Wird *script* weggelassen, wird das aktuell installierte Skript zurückgegeben (oder eine leere Zeichenkette, wenn kein solcher Handler installiert ist). Der Callback wird nur ausgeführt, wenn die Ereignisschleife bedient wird (z. B. über vwait oder update).

Ein Datei-Ereignishandler ist eine Bindung zwischen einem Kanal und einem Skript, sodass das Skript ausgewertet wird, sobald der Kanal lesbar oder beschreibbar wird. Datei-Ereignishandler werden am häufigsten verwendet, um Daten ereignisgesteuert von einem anderen Prozess zu empfangen, sodass der Empfänger weiter mit dem Benutzer oder anderen Kanälen interagieren kann, während auf die Daten gewartet wird. Ruft eine Anwendung **chan gets** oder **chan read** auf einem blockierenden Kanal auf, wenn keine Eingabedaten verfügbar sind, blockiert der Prozess; bis die Eingabedaten eintreffen, kann er keine anderen Ereignisse bedienen, sodass er für den Benutzer "einfriert" wirkt. Mit **chan event** kann der Prozess erkennen, wann Daten vorhanden sind, und **chan gets** oder **chan read** nur aufrufen, wenn sie nicht blockieren.

Ein Kanal gilt als lesbar, wenn auf dem zugrunde liegenden Gerät ungelesene Daten verfügbar sind. Ein Kanal gilt auch als lesbar, wenn ungelesene Daten in einem Eingabepuffer liegen, außer im Sonderfall, dass der letzte Leseversuch vom Kanal ein **chan gets**-Aufruf war, der keine vollständige Zeile im Eingabepuffer finden konnte. Diese Eigenschaft ermöglicht das zeilenweise Lesen einer Datei im nicht blockierenden Modus mit Ereignissen. Ein Kanal gilt auch als lesbar, wenn auf der zugrunde liegenden Datei oder dem Gerät ein Dateiende- oder Fehlerzustand vorliegt. Es ist wichtig, dass *script* diese Zustände prüft und angemessen behandelt; fehlt eine spezielle Prüfung auf Dateiende, kann eine Endlosschleife entstehen, in der *script* keine Daten liest, zurückkehrt und sofort erneut aufgerufen wird.

Ein Kanal gilt als beschreibbar, wenn mindestens ein Byte auf die zugrunde liegende Datei oder das Gerät ohne Blockieren geschrieben werden kann oder ein Fehlerzustand auf der zugrunde liegenden Datei oder dem Gerät vorliegt. Beachten Sie, dass Client-Sockets im asynchronen Modus beschreibbar werden, wenn die Verbindung hergestellt ist oder die Verbindung fehlschlägt.

Ereignisgesteuerte E/A funktioniert am besten für Kanäle, die mit **chan configure** in den nicht blockierenden Modus versetzt wurden. Im blockierenden Modus kann **chan puts** blockieren, wenn mehr Daten übergeben werden, als die zugrunde liegende Datei oder das Gerät aufnehmen kann, und **chan gets** oder **chan read** blockieren, wenn mehr Daten gelesen werden sollen als bereitstehen; während die Befehle blockieren, werden keine Ereignisse verarbeitet. Im nicht blockierenden Modus blockieren **chan puts**, **chan read** und **chan gets** nie.

Das Skript für ein Datei-Ereignis wird auf globaler Ebene (außerhalb des Kontexts einer Tcl-Prozedur) in dem Interpreter ausgeführt, in dem **chan event** aufgerufen wurde. Tritt beim Ausführen des Skripts ein Fehler auf, wird der mit interp bgerror registrierte Befehl zur Fehlermeldung verwendet. Zusätzlich wird der Datei-Ereignishandler gelöscht, wenn er jemals einen Fehler zurückgibt; das verhindert Endlosschleifen durch fehlerhafte Handler.

**chan flush *channel***

Stellt sicher, dass die gesamte ausstehende Ausgabe für den Kanal *channel* geschrieben wird.

Ist der Kanal im blockierenden Modus, kehrt der Befehl erst zurück, wenn die gesamte gepufferte Ausgabe an den Kanal geleert wurde. Ist der Kanal im nicht blockierenden Modus, kann der Befehl zurückkehren, bevor die gesamte gepufferte Ausgabe geleert wurde; der Rest wird im Hintergrund so schnell geleert, wie die zugrunde liegende Datei oder das Gerät Daten aufnehmen kann.

**chan gets *channel* ?*varName*?**

Liest eine Zeile vom Kanal, bestehend aus allen Zeichen bis zur nächsten Zeilenende-Sequenz oder bis Dateiende erreicht ist. Das zur Zeilenende-Sequenz gehörende Zeilenvorschub-Zeichen ist nicht Teil der Zeile. Ist das Argument *varName* angegeben, wird die Zeile in der Variable dieses Namens gespeichert und der Befehl gibt die Länge der Zeile zurück. Ist *varName* nicht angegeben, gibt der Befehl die Zeile selbst als Ergebnis zurück.

Ist keine vollständige Zeile verfügbar und der Kanal nicht am Dateiende, blockiert der Befehl bei einem blockierenden Kanal. Bei nicht blockierenden Kanälen gibt der Befehl bei nicht angegebenem *varName* die leere Zeichenkette zurück und -1, wenn *varName* angegeben ist.

Ist ein blockierender Kanal bereits am Dateiende, gibt der Befehl bei nicht angegebenem *varName* eine leere Zeichenkette zurück. Beachten Sie, dass eine leere Zeichenkette auch zurückgegeben werden kann, wenn eine Leerzeile gelesen wurde (keine Zeichen vor der nächsten Zeilenende-Sequenz). Die beiden Fälle lassen sich durch Aufruf von **chan eof** zur Prüfung auf Dateiende unterscheiden. Ist *varName* angegeben, gibt der Befehl bei Dateiende -1 zurück. In diesem Fall gibt es keine Mehrdeutigkeit, da Leerzeilen 0 zurückgeben.

Ist ein nicht blockierender Kanal bereits am Dateiende, gibt der Befehl bei nicht angegebenem *varName* eine leere Zeile zurück. Das lässt sich von einer leeren Zeile als Rückgabewert unterscheiden, die entweder durch eine Leerzeile oder durch eine nicht verfügbare vollständige Zeile verursacht wurde, mittels **chan eof** und **chan blocked**. Gibt

**chan eof** wahr zurück, ist der Kanal am Dateiende. Gibt **chan blocked** wahr zurück, war keine vollständige Zeile verfügbar. Geben beide falsch zurück, wurde eine Leerzeile gelesen. War *varName* für einen nicht blockierenden Kanal am Dateiende angegeben, gibt der Befehl -1 zurück. Das lässt sich von einer nicht verfügbaren vollständigen Zeile wie oben durch **chan eof** oder **chan blocked** unterscheiden. Beachten Sie, dass bei angegebenem *varName* keine Unterscheidung zwischen Dateiende und Leerzeilen nötig ist, da Letztere 0 zurückgeben.

Ist das Kodierungsprofil **strict** für den Kanal aktiv, löst der Befehl eine Ausnahme mit dem POSIX-Fehlercode **EILSEQ** aus, wenn in den Kanaleingabedaten Kodierungsfehler auftreten. Der Dateizeiger bleibt unverändert, und es ist möglich, durch Änderung der verwendeten Kodierung zu untersuchen und in manchen Fällen wiederherzustellen. Siehe Beispiele für Kodierungsfehler weiter unten.

**chan isbinary *channel***

Prüft, ob der Kanal *channel* ein Binärkanal ist; gibt 1 zurück, wenn ja, und 0 andernfalls. Ein Binärkanal ist ein Kanal mit iso8859-1-Kodierung, **-eofchar** auf {} und **-translation** auf lf.

**chan names ?*pattern*?**

Erzeugt eine Liste aller Kanalnamen. Ist *pattern* angegeben, werden nur die Kanalnamen zurückgegeben, die ihm entsprechen (gemäß den Regeln von string match).

**chan pending *mode channel***

Gibt je nachdem, ob *mode* **input** oder **output** ist, die Anzahl der Bytes der Eingabe bzw. Ausgabe zurück, die derzeit intern für *channel* gepuffert sind (besonders nützlich in einem lesbaren Ereignis-Callback, um anwendungsspezifische Grenzen für Eingabezeilenlängen zu setzen und einen potenziellen Denial-of-Service-Angriff zu vermeiden, bei dem ein feindlicher Benutzer eine extrem lange Zeile erzeugt, die den verfügbaren Speicher für den Puffer übersteigt). Gibt -1 zurück, wenn der Kanal nicht für den betreffenden Modus geöffnet wurde.

**chan pipe**

Erzeugt eine eigenständige Pipe, deren Lese- und Schreibseiten-Kanäle als 2-Element-Liste zurückgegeben werden; das erste Element ist die Leseseite, das zweite die Schreibseite. Kann z. B. nützlich sein, um **stderr** und **stdout** eines Unterprozesses getrennt umzuleiten. Dazu den Unterprozess mit den Umleitungsoperatoren "2>@" oder ">@" auf die Schreibseite einer Pipe starten und sie im Elternprozess sofort schließen. Das ist nötig, um auf der Leseseite ein EOF zu erhalten, sobald der Kindprozess beendet ist oder seine Ausgabe sonst geschlossen hat.

Beachten Sie, dass die Puffersemantik auf Betriebssystemebene erheblich variieren kann; man darf nicht annehmen, dass ein Schreibvorgang auf der Ausgabeseite der Pipe sofort auf der Eingabeseite erscheint. Das ist ein grundlegender Unterschied, den Tcl nicht verbergen kann. Die Gesamt-Stream-Semantik *ist* kompatibel, sodass blockierende Lese- und Schreibvorgänge die meisten Unterschiede nicht sehen, aber die Details, was genau wann geschrieben wird, sind es nicht. Das zeigt sich am ehesten bei Pipelines zum Testen; es ist Vorsicht geboten, Deadlocks zu vermeiden und kurze Leseoperationen zuzulassen.

**chan pop *channel***

Entfernt die oberste Transformation vom Kanal *channel*, falls vorhanden. Sind dem Kanal *channel* keine Transformationen hinzugefügt, entspricht das **chan close** dieses Kanals. Das Ergebnis ist normalerweise die leere Zeichenkette, kann in manchen Situationen jedoch ein Fehler sein (z. B. wenn der zugrunde liegende Systemstrom geschlossen wird und das einen

Fehler verursacht).

**chan postevent *channel eventSpec***

Dieser Unterbefehl wird von mit **chan create** angegebenen Befehlshandlern verwendet. Er meldet dem durch den Bezeichner *channel* dargestellten Kanal, dass die in *eventSpec* aufgeführten Ereignisse eingetreten sind. Das Argument muss eine Liste sein, die beliebige der Zeichenketten **read** und **write** enthält. Die Liste muss mindestens ein Element enthalten, da der Aufruf ohne zu meldende Ereignisse keinen Sinn ergibt.

Beachten Sie, dass dieser Unterbefehl nur mit Kanalbezeichnern verwendet werden kann, die mit **chan create** erzeugt/geöffnet wurden. Alle anderen Kanäle führen zu einem Fehler dieses Unterbefehls.

Da nur die Tcl-Ebene eines Kanals, d. h. sein Befehlshandler, Ereignisse an ihn melden sollte, ist die Verwendung auf den Interpreter beschränkt, der den Kanal erzeugt hat. Ereignisse an einen reflektierten Kanal aus einem Interpreter ohne dessen Implementierung zu melden, ist nicht erlaubt. Der Versuch, ein Ereignis aus einem anderen Interpreter zu melden, führt zu einem Fehler dieses Unterbefehls.

Eine weitere Einschränkung: Es ist nicht möglich, Ereignisse zu melden, für die der E/A-Kern kein Interesse registriert hat. Der Versuch führt zu einer Ausnahme der Methode. Siehe die Handler-Methode **watch** in **refchan**, dem Dokument zur API von Befehlshandlern für reflektierte Kanäle.

Dieser Befehl ist **safe** und für sichere Interpreter zugänglich. Er kann die Ausführung von **chan event**-Handlern auslösen, ob im aktuellen Interpreter oder in anderen Interpretern oder Threads, auch wenn das Ereignis aus einem sicheren Interpreter gemeldet und von einem vertrauenswürdigen Interpreter abgehört wird. **Chan event**-Handler werden *immer* in dem Interpreter ausgeführt, der sie eingerichtet hat.

**chan push *channel cmdPrefix***

Fügt dem Kanal *channel* eine neue Transformation hinzu. Das Argument *cmdPrefix* beschreibt eine Liste aus einem oder mehreren Wörtern, die einen Handler darstellen, der die Transformation implementiert. Das Befehlspräfix muss die in der Manpage transchan beschriebene API bereitstellen. Das Ergebnis dieses Unterbefehls ist ein Bezeichner für die Transformation. Es ist wichtig sicherzustellen, dass die Transformation den Kanalmodus unterstützt, mit dem sie verwendet wird, sonst kann der Kanal weder lesbar noch beschreibbar werden.

**chan puts ?-nonewline? ?*channel*? *string***

Schreibt *string* auf den Kanal *channel* gefolgt von einem Newline-Zeichen. Ein abschließendes Newline wird geschrieben, sofern die optionale Flagge **-nonewline** nicht angegeben ist. Wird *channel* weggelassen, wird die Zeichenkette auf den Standardausgabekanal **stdout** geschrieben.

Newline-Zeichen in der Ausgabe werden von **chan puts** gemäß der aktuell konfigurierten Option **-translation** des Kanals in plattformspezifische Zeilenende-Sequenzen übersetzt (auf PCs werden Newlines normalerweise durch Wagenrücklauf-Zeilenvorschub-Sequenzen ersetzt; siehe **chan configure** oben für Einzelheiten).

Tcl puffert Ausgabe intern, sodass mit **chan puts** geschriebene Zeichen möglicherweise nicht sofort auf der Ausgabedatei oder dem Gerät erscheinen; Tcl verzögert die Ausgabe normalerweise, bis der Puffer voll ist oder der Kanal geschlossen wird. Mit **chan flush** kann die Ausgabe sofort erzwungen werden.

Wenn der Ausgabepuffer voll ist, blockiert **chan puts** normalerweise, bis das Betriebssystem alle gepufferten Daten zur Ausgabe angenommen hat. Ist *channel* im nicht blockierenden Modus, blockiert **chan puts** nicht, auch wenn das Betriebssystem die Daten nicht annehmen kann. Stattdessen puffert Tcl weiter und schreibt im Hintergrund so schnell, wie die zugrunde liegende Datei oder das Gerät Daten aufnehmen kann. Die Anwendung muss die Tcl-Ereignisschleife verwenden, damit nicht blockierende Ausgabe funktioniert; sonst erfährt Tcl nie, dass die Datei oder das Gerät bereit für weitere Ausgabedaten ist. Für einen Kanal im nicht blockierenden Modus kann beliebig viel Daten gepuffert werden, was viel Speicher verbrauchen kann. Um Speicherverschwendung zu vermeiden, sollte nicht blockierende E/A normalerweise ereignisgesteuert mit **chan event** verwendet werden (rufen Sie **chan puts** nicht auf, es sei denn, Sie wurden kürzlich per Datei-Ereignis benachrichtigt, dass der Kanal bereit für weitere Ausgabedaten ist).

Der Befehl löst eine Fehlerausnahme mit dem POSIX-Fehlercode **EILSEQ** aus, wenn das Kodierungsprofil **strict** für den Kanal aktiv ist und die Ausgabedaten nicht in der für den Kanal konfigurierten Kodierung kodiert werden können. In diesem Fall können Daten teilweise in den Kanal geschrieben worden sein.

**chan read *channel* ?*numChars*?**

...siehe nächstes...

**chan read ?-nonewline? *channel***

In der ersten Form besteht das Ergebnis aus den nächsten *numChars* Zeichen, die vom Kanal *channel* gelesen werden; wird *numChars* weggelassen, werden alle Zeichen bis zu dem Punkt gelesen, an dem der Kanal einen Fehler signalisieren würde (ob Dateiende, Blockierung oder anderer Fehlerzustand). In der zweiten Form (d. h. wenn *numChars* weggelassen wurde) kann die Flagge **-nonewline** angegeben werden, um anzuzeigen, dass ein abschließendes Newline in der gelesenen Zeichenkette entfernt werden soll.

Ist *channel* im nicht blockierenden Modus, liest **chan read** möglicherweise nicht so viele Zeichen wie angefordert: sobald alle verfügbare Eingabe gelesen ist, gibt der Befehl die verfügbaren Daten zurück, statt auf weitere Eingabe zu blockieren. Ist der Kanal für eine Mehrbyte-Kodierung konfiguriert, können im internen Puffer Bytes verbleiben, die kein vollständiges Zeichen bilden. Diese Bytes werden erst zurückgegeben, wenn ein vollständiges Zeichen verfügbar ist oder Dateiende erreicht ist. Der Schalter **-nonewline** wird ignoriert, wenn der Befehl zurückkehrt, bevor das Dateiende erreicht wird.

**Chan read** übersetzt Zeilenende-Sequenzen in der Eingabe gemäß der Option **-translation** des Kanals in Newline-Zeichen (siehe **chan configure** oben für eine Diskussion, wie **chan configure** die Eingabe verändert).

Beim Lesen von einem seriellen Port sollten die meisten Anwendungen den seriellen Port-Kanal auf nicht blockierend setzen, etwa so:

```
chan configure channel -blocking 0.
```

Dann verhält sich **chan read** wie oben beschrieben. Beachten Sie, dass die meisten seriellen Ports vergleichsweise langsam sind; es ist durchaus möglich, für jedes vom Port gelesene Zeichen ein **readable**-Ereignis zu erhalten. Vorsicht ist geboten bei **chan read** auf blockierenden seriellen Ports:

**chan read *channel numChars***

In dieser Form blockiert **chan read**, bis *numChars* vom seriellen Port empfangen wurden.

**chan read *channel***

In dieser Form blockiert **chan read**, bis das Dateiende-Zeichen empfangen wurde; siehe **chan configure -eofchar**. Ist für den Kanal kein Dateiende-Zeichen konfiguriert, blockiert **chan read** für immer.

Ist das Kodierungsprofil **strict** für den Kanal aktiv, löst der Befehl eine Ausnahme mit dem POSIX-Fehlercode **EILSEQ** aus, wenn in den Kanaleingabedaten Kodierungsfehler auftreten. Ist der Kanal im blockierenden Modus, wird der Fehler ausgelöst, nachdem der Dateizeiger auf den Anfang der ungültigen Daten gesetzt wurde. Der erfolgreich dekodierte führende Teil der Daten vor der Fehlerposition wird als Wert des Schlüssels **-data** im Fehleroptions-Wörterbuch zurückgegeben. Ist der Kanal im nicht blockierenden Modus, gibt der Befehl den erfolgreich dekodierten Datenteil ohne Fehlerausnahme zurück. Ein nachfolgendes Lesen beginnt bei den ungültigen Daten und löst sofort eine **EILSEQ**-POSIX-Fehlerausnahme aus. Im Gegensatz zum blockierenden Kanal ist der Schlüssel **-data** im Fehleroptions-Wörterbuch nicht vorhanden. Bei einer Ausnahme wegen Kodierungsfehlern ist Untersuchung und in manchen Fällen Wiederherstellung durch Änderung der verwendeten Kodierung möglich. Siehe Beispiele für Kodierungsfehler weiter unten.

**chan seek *channel offset* ?*origin*?**

Setzt die aktuelle Zugriffsposition im zugrunde liegenden Datenstrom für den Kanal *channel* auf *offset* Bytes relativ zu *origin*. *Offset* muss eine Ganzzahl sein (kann negativ sein) und *origin* muss eines der folgenden sein:

**start**

Die neue Zugriffsposition liegt *offset* Bytes vom Anfang der zugrunde liegenden Datei oder des Geräts entfernt.

**current**

Die neue Zugriffsposition liegt *offset* Bytes von der aktuellen Zugriffsposition entfernt; ein negatives *offset* verschiebt die Zugriffsposition rückwärts in der zugrunde liegenden Datei oder dem Gerät.

**end**

Die neue Zugriffsposition liegt *offset* Bytes vom Ende der Datei oder des Geräts entfernt. Ein negatives *offset* setzt die Zugriffsposition vor das Dateiende, ein positives *offset* danach.

Das Argument *origin* ist standardmäßig **start**.

**Chan seek** leert die gesamte gepufferte Ausgabe für den Kanal, bevor der Befehl zurückkehrt, auch wenn der Kanal im nicht blockierenden Modus ist. Er verwirft auch gepufferte und ungelesene Eingabe. Dieser Befehl gibt eine leere Zeichenkette zurück. Ein

Fehler tritt auf, wenn der Befehl auf Kanälen angewendet wird, deren zugrunde liegende Datei oder Gerät kein Suchen unterstützt.

Beachten Sie, dass *offset*-Werte Byte-Offsets sind, keine Zeichen-Offsets. Sowohl **chan seek** als auch **chan tell** arbeiten in Bytes, nicht in Zeichen, im Gegensatz zu **chan read**.

**chan tell *channel***

Gibt eine Zahl zurück, die die aktuelle Zugriffsposition im zugrunde liegenden Datenstrom für den Kanal *channel* angibt. Der zurückgegebene Wert ist ein Byte-Offset, der an **chan seek** übergeben werden kann, um den Kanal auf eine bestimmte Position zu setzen. Beachten Sie, dass dieser Wert in Bytes angegeben ist, nicht in Zeichen wie bei **chan read**. Der zurückgegebene Wert ist -1 für Kanäle, die kein Suchen unterstützen.

**chan truncate *channel* ?*length*?**

Setzt die Bytelänge des zugrunde liegenden Datenstroms für den Kanal *channel* auf *length* (oder auf den aktuellen Byte-Offset im zugrunde liegenden Datenstrom, wenn *length* weggelassen ist). Der Kanal wird vor dem Kürzen geleert.

## Beispiele

### Einfache Kanaloperations-Beispiele

Weisen Sie Tcl an, Ausgabe auf **stdout** sofort zu senden, unabhängig davon, ob es an ein Terminal geht:

```
chan configure stdout -buffering none
```

Im folgenden Beispiel wird eine Datei mit der unter Windows üblichen Kodierung CP1252 geöffnet, nach einer Zeichenkette gesucht, dieser Teil neu geschrieben und die Datei zwei Zeilen später gekürzt.

```
set f [open somefile.txt r+]
chan configure $f -encoding cp1252
set offset 0

# Nach der Zeichenkette "FOOBAR" in der Datei suchen
while {[chan gets $f line] >= 0} {
    set idx [string first FOOBAR $line]
    if {$idx >= 0} {
        # Gefunden; Zeile umschreiben

        chan seek $f [expr {$offset + $idx}]
        chan puts -nonewline $f BARFOO

        # Zur nächsten Zeile springen und kürzen
        chan gets $f
        chan gets $f
        chan truncate $f

        # Suche in der Datei jetzt beenden
        break
    }

    # Offset des Zeilenanfangs für später merken
    set offset [chan tell $f]
}
chan close $f
```

Dieses Beispiel veranschaulicht das Leeren eines Kanals. Der Benutzer wird nach Informationen gefragt. Da der Standardeingabekanal zeilenweise gepuffert ist, muss er geleert werden, damit der Benutzer die Eingabeaufforderung sieht.

```
chan puts -nonewline "Please type your name: "
chan flush stdout
chan gets stdin name
chan puts "Hello there, $name!"
```

Dieses Beispiel liest eine Datei zeilenweise und gibt sie mit der aktuellen Zeilennummer am Zeilenanfang aus.

```
set chan [open "some.file.txt"]
set lineNumber 0
while {[chan gets $chan line] >= 0} {
    chan puts "[incr lineNumber]: $line"
}
chan close $chan
```

In diesem Beispiel für ereignisgesteuertes Lesen wird **GetData** mit dem Kanal als Argument aufgerufen, sobald $chan lesbar wird. Der **read**-Aufruf liest alle derzeit verfügbaren Binärdaten ohne Blockieren. Hier wird das Datei-Ereignis bei Dateiende vom Kanal entfernt, um wiederholte Aufrufe zu vermeiden (siehe oben). Alternativ kann der Kanal unter dieser Bedingung geschlossen werden.

```
proc GetData {chan} {
    set data [chan read $chan]
    chan puts "[string length $data] $data"
    if {[chan eof $chan]} {
        chan event $chan readable {}
    }
}

chan configure $chan -blocking 0 -translation binary
chan event $chan readable [list GetData $chan]
```

Das nächste Beispiel ist ähnlich, verwendet aber **chan gets** zum Lesen zeilenorientierter Daten.

```
proc GetData {chan} {
    if {[chan gets $chan line] >= 0} {
        chan puts $line
    }
    if {[chan eof $chan]} {
        chan close $chan
    }
}

chan configure $chan -blocking 0 -buffering line -translation crlf
chan event $chan readable [list GetData $chan]
```

Ein Netzwerkserver, der seine Eingabe zeilenweise zurückgibt, ohne die gleichzeitige Bedienung anderer Verbindungen zu blockieren:

```
# Sehr einfacher Logger...
proc log {message} {
    chan puts stdout $message
}

# Wird aufgerufen, wenn ein neuer Client sich mit dem Server verbindet
proc connect {chan host port} {
    set clientName [format <%s:%d> $host $port]
    log "connection from $clientName"
    chan configure $chan -blocking 0 -buffering line
    chan event $chan readable [list echoLine $chan $clientName]
}

# Wird aufgerufen, sobald mindestens ein Byte Eingabedaten
# verfügbar ist oder der Kanal vom Client geschlossen wurde.
proc echoLine {chan clientName} {
    chan gets $chan line
    if {[chan eof $chan]} {
        log "finishing connection from $clientName"
        chan close $chan
    } elseif {![chan blocked $chan]} {
        # Hat nicht auf Zeilenende blockiert
        log "$clientName - $line"
        chan puts $chan $line
    }
}

# Server-Socket erzeugen und Ereignisschleife für
# eingehende Verbindungen starten...
socket -server connect 12345
vwait forever
```

Das folgende Beispiel liest ein PPM-Bild aus einer Datei mit gemischtem ASCII- und Binärinhalt.

```
# Datei öffnen und in Unix-ASCII-Modus versetzen
set f [open teapot.ppm]
chan configure $f -encoding ascii -translation lf

# Header lesen
if {[chan gets $f] ne "P6"} {
    error "not a raw-bits PPM"
}

# Zeilen lesen, bis Nicht-Kommentarzeilen
# drei Dezimalwerte liefern.
set words {}
while {[llength $words] < 3} {
    chan gets $f line
    if {[string match "#*" $line]} continue
    lappend words {*}[join [scan $line %d%d%d]]
}

# Diese Wörter liefern Bildgröße und
# Gesamttiefe pro Kanal. Variablen zuweisen.
lassign $words xSize ySize depth

# Jetzt in den Binärmodus wechseln, um die Daten
# ein Byte pro Kanal (rot, grün, blau) pro Pixel zu lesen.
chan configure $f -translation binary
set numDataBytes [expr {3 * $xSize * $ySize}]
set data [chan read $f $numDataBytes]

close $f
```

### Beispiele für Datei-Suchoperationen

Eine Datei zweimal lesen:

```
set f [open file.txt]
set data1 [chan read $f]
chan seek $f 0
set data2 [chan read $f]
chan close $f
# $data1 eq $data2, wenn die Datei nicht aktualisiert wurde
```

Die letzten 10 Bytes einer Datei lesen:

```
set f [open file.data]
# Garantiert für Binärdaten, kann bei
# anderen Kodierungen fehlschlagen...
chan configure $f -translation binary
chan seek $f -10 end
set data [chan read $f 10]
chan close $f
```

Eine Zeile von einem Dateikanal nur lesen, wenn sie mit **foobar** beginnt:

```
# Offset speichern für den Fall, dass das Lesen rückgängig gemacht
    werden muss...
set offset [tell $chan]
if {[read $chan 6] eq "foobar"} {
    gets $chan line
} else {
    set line {}
    # Lesen rückgängig machen...
    seek $chan $offset
}
```

### Beispiele für Kodierungsfehler

Das Beispiel unten zeigt den Umgang mit einem beim Kanaleingang aufgetretenen Kodierungsfehler. Zuerst Erzeugung einer Testdatei mit der ungültigen UTF-8-Sequenz (**A \xC3 B** ):

```
% set f [open test_A_195_B.txt wb]; chan puts -nonewline $f A\xC3B;
    chan close $f
```

Ein Leseversuch der Datei führt zu einem Kodierungsfehler, der dann durch Umschalten des Kanals in den Binärmodus untersucht wird. Beachten Sie im Beispiel, dass bei der Fehlermeldung der Dateizeiger unverändert bleibt, sodass **chan gets** bei der Wiederherstellung die vollständige Zeile zurückgibt.

```
% set f [open test_A_195_B.txt r]
file384b6a8
% chan configure $f -encoding utf-8
% catch {chan gets $f} e d
1
% set d
-code 1 -level 0
-errorstack {INNER {invokeStk1 gets file384b6a8}}
-errorcode {POSIX EILSEQ {invalid or incomplete multibyte or wide
    character}}
-errorinfo {...} -errorline 1
% chan tell $f
0
% chan configure $f -translation binary
% chan gets $f
AÃB
```

Das folgende Beispiel ist ähnlich, zeigt aber die Wiederherstellung nach einem blockierenden Lesevorgang. Die erfolgreich dekodierten Daten "A" werden im Fehleroptions-Wörterbuchschlüssel **-data** zurückgegeben. Der Dateizeiger wird bei der Kodierungsfehlerposition 1 fortgeschrieben. Die Daten an der Fehlerposition werden daher vom nächsten **chan read**-Befehl gelesen.

```
% set f [open test_A_195_B.txt r]
file35a65a0
% chan configure $f -encoding utf-8 -blocking 1
% catch {chan read $f} e d
1
% set d
-data A -code 1 -level 0
-errorstack {INNER {invokeStk1 read file35a65a0}}
-errorcode {POSIX EILSEQ {invalid or incomplete multibyte or wide
    character}}
-errorinfo {...} -errorline 1
% chan tell $f
1
% chan configure $f -translation binary
% chan read $f
ÃB
% chan close $f
```

Schließlich dasselbe Beispiel, diesmal mit einem nicht blockierenden Kanal.

```
% set f [open test_A_195_B.txt r]
file35a65a0
% chan configure $f -encoding utf-8 -blocking 0
% chan read $f
A
```

```
% chan tell $f
1
% catch {chan read $f} e d
1
% set d
-code 1 -level 0
-errorstack {INNER {invokeStk1 read file384b228}}
-errorcode {POSIX EILSEQ {invalid or incomplete multibyte or wide
    character}}
-errorinfo {...} -errorline 1
```

### Beispiele für Kanalkopie

Das erste Beispiel überträgt den Inhalt eines Kanals exakt auf einen anderen. Beim Kopieren einer Datei auf eine andere ist file copy besser geeignet, da dabei auch Dateimetadaten (z. B. Zugriffsrechte) kopiert werden, wo möglich.

```
chan configure $in -translation binary
chan configure $out -translation binary
chan copy $in $out
```

Das zweite Beispiel zeigt, wie dem Callback die Anzahl der übertragenen Bytes übergeben wird. Es verwendet auch vwait, um die Anwendung in die Ereignisschleife zu bringen. Dieses vereinfachte Beispiel ließe sich natürlich auch ohne Befehls-Callback durchführen.

```
proc Cleanup {in out bytes {error {}}} {
    global total
    set total $bytes
    chan close $in
    chan close $out
    if {$error ne ""} {
        # Fehler während des Kopiervorgangs
    }
}

set in [open $file1]
set out [socket $server $port]
chan copy $in $out -command [list Cleanup $in $out]
vwait total
```

Das dritte Beispiel kopiert in Stücken und prüft im Befehlsrückruf auf Dateiende.

```
proc CopyMore {in out chunk bytes {error {}}} {
    global total done
    incr total $bytes
    if {($error ne "") || [chan eof $in]} {
        set done $total
        chan close $in
        chan close $out
    } else {
        chan copy $in $out -size $chunk \
            -command [list CopyMore $in $out $chunk]
    }
}

set in [open $file1]
set out [socket $server $port]
set chunk 1024
set total 0
chan copy $in $out -size $chunk \
    -command [list CopyMore $in $out $chunk]
vwait done
```

Das vierte Beispiel startet ein asynchrones, bidirektionales Kopieren zwischen zwei Sockets. Das können auch Pipes von zwei bidirektionalen Pipelines sein (z. B. *[open* "|*hal 9000*" *r+]*); die Kommunikation bleibt für das Skript im Wesentlichen unsichtbar, da alle vier **chan event**-Plätze belegt sind, obwohl alle Transformationen, die mit **chan push** auf die Kanäle gelegt werden, den Datenverkehr beobachten können.

```
proc Done {dir args} {
    global flows done
    chan puts "$dir is over."
    incr flows -1
    if {$flows <= 0} {
        set done 1
    }
}

set flows 2
chan copy $sok1 $sok2 -command [list Done UP]
chan copy $sok2 $sok1 -command [list Done DOWN]
vwait done
```

*Copyright (c) 2005-2006 Donal K. Fellows*

# class

*Stichwörter:* class, metaclass, object

## Name

oo::class - Klasse aller Klassen

## Übersicht

**package require tcl::oo**

**oo::class** *method* ?*arg* ...?

## Klassenhierarchie

```
oo::object
   → oo::class
```

## Beschreibung

Klassen sind Objekte, die nach einem in dem Fabrikobjekt (der Klasse) gespeicherten Muster andere Objekte erzeugen können. Eine Instanz der Klasse wird erzeugt, indem eine der Fabrikmethoden der Klasse aufgerufen wird, typischerweise **create**, wenn ein expliziter Name vergeben wird, oder **new**, wenn automatisch ein beliebiger eindeutiger Name gewählt werden soll.

Die Klasse **oo::class** ist die Klasse aller Klassen; jede Klasse ist eine Instanz dieser Klasse, die folglich eine Instanz von sich selbst ist. Diese Klasse ist eine Unterklasse von oo::object, sodass jede Klasse auch ein Objekt ist. Weitere Metaklassen (Klassen von Klassen) können bei Bedarf durch Subklassierung von **oo::class** definiert werden. Beachten Sie, dass das Objekt **oo::class** die Methode **new** auf sich selbst verbirgt; neue Klassen sollten daher immer mit der Methode **create** erzeugt werden.

## Konstruktor

Der Konstruktor der Klasse **oo::class** nimmt ein optionales Argument entgegen, das — falls vorhanden — zusammen mit dem Namen der neu erzeugten Klasse an oo::define übergeben wird, damit die Klasse bei der Erstellung bequem konfiguriert werden kann.

## Destruktor

Die Klasse **oo::class** definiert keinen expliziten Destruktor. Wird jedoch eine Klasse zerstört, werden auch alle ihre Unterklassen und Instanzen zerstört, ebenso alle Objekte, in die sie gemischt wurde.

## Exportierte Methoden

***cls* create *name* ?*arg* ...?**

Erzeugt eine neue Instanz der Klasse *cls* mit dem Namen *name* (wird im Namespace des Aufrufkontexts aufgelöst, wenn nicht voll qualifiziert), übergibt die Argumente *arg ...* an den Konstruktor und gibt (bei erfolgreichem Ergebnis) den voll qualifizierten Namen des erzeugten Objekts zurück (das Ergebnis des Konstruktors wird ignoriert). Schlägt der Konstruktor fehl (liefert kein OK-Ergebnis), wird das Objekt zerstört und die Fehlermeldung ist das Ergebnis dieses Methodenaufrufs.

***cls* new ?*arg* ...?**

Erzeugt eine neue Instanz der Klasse *cls* mit einem neuen eindeutigen Namen, übergibt die Argumente *arg ...* an den Konstruktor und gibt (bei erfolgreichem Ergebnis) den voll qualifizierten Namen des erzeugten Objekts zurück (das Ergebnis des Konstruktors wird ignoriert). Schlägt der Konstruktor fehl, wird das Objekt zerstört und die Fehlermeldung ist das Ergebnis dieses Methodenaufrufs.

Beachten Sie, dass diese Methode vom Objekt **oo::class** selbst nicht exportiert wird; Klassen sollten daher nicht mit dieser Methode erzeugt werden.

## Nicht exportierte Methoden

Die Klasse **oo::class** unterstützt folgende nicht exportierte Methoden:

***cls* createWithNamespace *name nsName* ?*arg* ...?**

Erzeugt eine neue Instanz der Klasse *cls* mit dem Namen *name* (wird im Namespace des Aufrufkontexts aufgelöst, wenn nicht voll qualifiziert), übergibt die Argumente *arg ...* an den Konstruktor und gibt (bei erfolgreichem Ergebnis) den voll qualifizierten Namen des erzeugten Objekts zurück (das Ergebnis des Konstruktors wird ignoriert). Der Name des Instanz-Namespace des Objekts ist *nsName*; es ist ein Fehler, wenn dieser Namespace nicht erzeugt werden kann. Schlägt der Konstruktor fehl, wird das Objekt zerstört und die Fehlermeldung ist das Ergebnis dieses Methodenaufrufs.

## Beispiele

Dieses Beispiel definiert eine einfache Klassenhierarchie, erzeugt eine neue Instanz und ruft eine Methode des Objekts auf, bevor die Hierarchie zerstört wird und gezeigt wird, dass die Zerstörung transitiv ist.

```
oo::class create fruit {
    method eat {} {
        puts "yummy!"
    }
}
oo::class create banana {
    superclass fruit
    constructor {} {
        my variable peeled
        set peeled 0
    }
    method peel {} {
        my variable peeled
        set peeled 1
        puts "skin now off"
    }
    method edible? {} {
        my variable peeled
        return $peeled
    }
    method eat {} {
        if {![my edible?]} {
            my peel
        }
        next
    }
}
set b [banana new]
$b eat               → prints "skin now off" and "yummy!"
fruit destroy
$b eat               → error "unknown command"
```

*Copyright (c) 2007 Donal K. Fellows*

# classvariable

*Stichwörter:* classvariable, class, class variable, variable

## Name

classvariable - Verknüpft lokale Variable mit Variable in der Klasse

## Übersicht

**package require tcl::oo**

**classvariable** *variableName* ?...?

## Beschreibung

Der Befehl **classvariable** ist innerhalb von Methoden verfügbar. Er nimmt eine Reihe von einem oder mehr Variablennamen und macht sie im Gültigkeitsbereich der Methode verfügbar; diese Namen dürfen nicht qualifiziert sein und dürfen sich nicht auf Array-Elemente beziehen. Der Ursprungs-Gültigkeitsbereich für die Variablen ist der Namespace der Klasse, in der die Methode definiert wurde. Die referenzierten Variablen werden also von allen Instanzen dieser Klasse geteilt.

Beachten Sie, dass dieser Befehl dem Befehl **typevariable** aus dem snit-Paket in tcllib für denselben Zweck entspricht. Wird er in einer direkt auf einer Klasseninstanz definierten Methode verwendet (z. B. über oo::objdefine **method**), entspricht er im Wesentlichen:

```
namespace upvar [namespace current] $var $var
```

für jede an **classvariable** übergebene Variable.

## Beispiele

Diese Klasse zählt, wie viele Instanzen erzeugt wurden.

```
oo::class create Counted {
    initialise {
        variable count 0
    }

    variable number
    constructor {} {
        classvariable count
        set number [incr count]
    }

    method report {} {
        classvariable count
        puts "This is instance $number of $count"
    }
}

set a [Counted new]
set b [Counted new]
```

```
$a report
        → This is instance 1 of 2
set c [Counted new]
$b report
        → This is instance 2 of 3
$c report
        → This is instance 3 of 3
```

*Copyright (c) 2011-2015 Andreas Kupries*
*Copyright (c) 2018 Donal K. Fellows*

# clock

*Stichwörter:* clock, date, time

## Name

clock - Datum und Uhrzeit ermitteln und manipulieren

## Übersicht

**package require Tcl 8.5-**

**clock add** *timeVal* ?*count unit* ...? ?-*option value*?
**clock clicks** ?-*option*?
**clock format** *timeVal* ?-*option value* ...?
**clock microseconds**
**clock milliseconds**
**clock scan** *inputString* ?-*option value* ...?
**clock seconds**

## Beschreibung

Der Befehl **clock** führt mehrere Operationen aus, die Werte ermitteln und manipulieren, die Zeitpunkte repräsentieren. Der Befehl unterstützt mehrere Unterbefehle, die festlegen, welche Aktion ausgeführt wird.

**clock add *timeVal* ?*count unit...*? ?-*option value*?**

Addiert einen (möglicherweise negativen) Offset zu einer Zeit, die als ganzzahlige Sekundenanzahl ausgedrückt ist. Siehe Uhrzeit-Arithmetik für eine vollständige Beschreibung.

**clock clicks ?-*option*?**

Liefert ohne -*option*-Argument einen hochauflösenden Zeitwert als systemabhängigen Ganzzahlwert. Die Einheit des Werts ist systemabhängig, sollte aber die höchste verfügbare Auflösung des System-Takts sein, z. B. ein CPU-Zyklenzähler. Siehe Hochauflösende Zeitmesser für eine vollständige Beschreibung.

Ist das -*option*-Argument **-milliseconds**, ist der Befehl synonym mit **clock milliseconds** (siehe unten). Diese Verwendung ist veraltet; **clock milliseconds** gilt als bevorzugte Methode, eine Millisekundenanzahl zu erhalten.

Ist das -*option*-Argument **-microseconds**, ist der Befehl synonym mit **clock microseconds** (siehe unten). Diese Verwendung ist veraltet; **clock microseconds** gilt als bevorzugte

Methode, eine Mikrosekundenanzahl zu erhalten.

**clock format *timeVal* ?*-option value* ...?**

Formatiert eine als ganzzahlige Sekundenanzahl ausgedrückte Zeit in ein für Benutzer oder externe Programme bestimmtes Format. Siehe Zeiten formatieren für eine vollständige Beschreibung.

**clock microseconds**

Gibt die aktuelle Zeit als ganzzahlige Mikrosekundenanzahl zurück. Siehe Hochauflösende Zeitmesser für eine vollständige Beschreibung.

**clock milliseconds**

Gibt die aktuelle Zeit als ganzzahlige Millisekundenanzahl zurück. Siehe Hochauflösende Zeitmesser für eine vollständige Beschreibung.

**clock scan *inputString* ?*-option value* ...?**

Liest eine als Zeichenkette ausgedrückte Zeit ein und erzeugt eine ganzzahlige Sekundenanzahl. Siehe Zeiten einlesen für eine vollständige Beschreibung.

**clock seconds**

Gibt die aktuelle Zeit als ganzzahlige Sekundenanzahl zurück.

## Parameter

***count***

Eine Ganzzahl, die eine Anzahl einer Zeiteinheit darstellt. Siehe Uhrzeit-Arithmetik für die Details.

***timeVal***

Ein an den Befehl **clock** übergebener Ganzzahlwert, der einen absoluten Zeitpunkt als Sekundenanzahl ab der *Epoche* vom 1. Januar 1970, 00:00 UTC darstellt. Die Sekundenanzahl enthält keine Schaltsekunden; Sekunden werden gezählt, als ob jeder UTC-Tag genau 86400 Sekunden hätte. Tcl reagiert auf Schaltsekunden, indem es die Uhr einige Minuten lang minimal beschleunigt oder verlangsamt, bis sie wieder mit UTC übereinstimmt; das Datenmodell stellt keine Minuten mit 59 oder 61 Sekunden dar.

***now***

Statt *timeVal* kann der nicht-ganzzahlige Wert **now** als Ersatz für "heute" verwendet werden; er wird zur Laufzeit durch den Wert von **clock seconds** ersetzt. Beispiel: **clock format now -f %a; # aktueller Wochentag clock add now 1 month; # nächster Monat**

***unit***

Eines der Wörter **seconds**, **minutes**, **hours**, **days**, **weekdays**, **weeks**, **months** oder **years**. Zusammen mit *count* zur Angabe eines Zeitintervalls, z. B. *3 seconds* oder *1 year*.

## Optionen

**-base time**

Legt fest, dass relative Zeiten in einem **clock scan**-Befehl relativ zu *time* angegeben werden. *time* muss als nominale Sekundenanzahl ab der Epoche vom 1. Januar 1970, 00:00 UTC ausgedrückt sein.

**-format format**

Legt das gewünschte Ausgabeformat für **clock format** oder das erwartete Eingabeformat für **clock scan** fest. Die Zeichenkette *format* besteht aus beliebig vielen Zeichen außer dem Prozentzeichen **%**, durchsetzt mit beliebig vielen *Formatgruppen*, also zwei- oder dreistelligen

Folgen, die mit dem Prozentzeichen beginnen. Die zulässigen Formatgruppen und ihre Interpretation sind unter Formatgruppen beschrieben.

Bei **clock format** ist das Standardformat

```
%a %b %d %H:%M:%S %Z %Y
```

Bei **clock scan** bedeutet das Fehlen von **-format**, dass ein *Freiform-Scan* angefordert wird; siehe Freiform-Scan für eine Beschreibung des Verhaltens.

**-gmt boolean**

Ist *boolean* wahr, wird festgelegt, dass eine an **clock add**, **clock format** oder **clock scan** übergebene Zeit in UTC verarbeitet wird. Ist *boolean* falsch, gilt standardmäßig die lokale Zeitzone. Diese Verwendung ist veraltet; korrekt ist die Angabe der UTC-Zeitzone mit **-timezone** *:UTC* oder einem gleichwertigen Weg.

**-locale localeName**

Legt fest, dass locale-abhängiges Einlesen und Formatieren (sowie Datumsarithmetik für Daten vor Einführung des Gregorianischen Kalenders) in der von *localeName* bezeichneten Locale erfolgt. Der Locale-Name kann jede von msgcat akzeptierte Locale sein, der Sondername *system* (aktuelle Locale des Prozesses) oder die leere Zeichenkette (Tcls Standard-Locale).

Der Einfluss der Locale auf Einlesen und Formatieren wird bei den einzelnen Formatgruppen unter Formatgruppen erläutert. Der Einfluss auf die Uhrzeit-Arithmetik unter Uhrzeit-Arithmetik.

**-timezone zoneName**

Legt fest, dass Uhrzeit-Arithmetik, Formatierung und Einlesen nach den Regeln der von *zoneName* bezeichneten Zeitzone erfolgen. Zulässige Werte und ihre Interpretation unter Zeitzonen. Bei Unterbefehlen mit **-timezone** ist standardmäßig die *aktuelle Zeitzone* gemeint. Sie wird in dieser Reihenfolge ermittelt:

1. der Umgebungsvariable **TCL_TZ**.
2. der Umgebungsvariable **TZ**.
3. unter Windows den Zeitzoneneinstellungen der Systemsteuerung.

**-validate boolean**

Ist *boolean* wahr (Standard), meldet **clock scan** einen Fehler bei ungültigen Werten, z. B. Tag des Monats größer als die Monatslänge. Bei false passt der Befehl Werte in den zulässigen Bereich an. Siehe Zeiten einlesen.

Sind keine dieser Quellen vorhanden, versuchen die C-Funktionen **localtime** und **mktime**, Zeiten zwischen lokaler Zeit und Greenwich umzurechnen. Auf 32-Bit-Systemen ist dieser Ansatz fehleranfällig, besonders für Zeiten außerhalb des in einer 32-Bit-Ganzzahl darstellbaren Bereichs (etwa 1902 bis 2037).

## Uhrzeit-Arithmetik

Der Befehl **clock add** führt Uhrzeit-Arithmetik auf einem Wert aus (ausgedrückt als nominale Sekunden ab der Epoche vom 1. Januar 1970, 00:00 UTC), der als erstes Argument übergeben wird. Die übrigen Argumente (außer den möglichen Optionen **-timezone**, **-locale** und **-gmt**) sind abwechselnd Ganzzahlen und Schlüsselwörter aus **seconds**, **minutes**, **hours**, **days**,

**weekdays**, **weeks**, **months** oder **years**.

Das Addieren von Sekunden, Minuten und Stunden ist unmittelbar: Der gegebene Zeitzuwachs (×60 für Minuten, ×3600 für Stunden) wird zum an **clock add** übergebenen *timeVal* addiert. Das Ergebnis gilt als nominale Sekundenanzahl ab der Epoche.

Beim Überschreiten eines Schaltsekunden-Einfüge- oder -Entfernungspunkts können überraschende Ergebnisse entstehen; **clock add** ignoriert Schaltsekunden und nimmt eine fortlaufende Folge 23:59:58, 23:59:59, 00:00:00 an. Tcl gleicht Schaltsekunden durch minimale Beschleunigung oder Verlangsamung der Uhr aus, bis die Zeit wieder mit der Welt übereinstimmt.

Weil Stundenaddition in absoluter Zeit definiert ist, werden in Zonen mit Sommerzeit feste Zeitbeträge addiert. Das folgende Beispiel setzt **x** auf **04:00:00**, weil die Uhr im betrachteten Intervall umgestellt wurde.

```
set s [clock scan {2004-10-30 05:00:00} \
           -format {%Y-%m-%d %H:%M:%S} \
           -timezone :America/New_York]
set a [clock add $s 24 hours -timezone :America/New_York]
set x [clock format $a \
           -format {%H:%M:%S} -timezone :America/New_York]
```

Tage und Wochen werden addiert/subtrahiert, indem die Zeit in Kalendertag und Tageszeit der passenden Zeitzone und Locale umgewandelt, die nötige Tagesanzahl (Wochen ×7) zum Kalendertag addiert und Datum und Zeit wieder in Sekunden ab der Epoche zurückgerechnet werden. Das Schlüsselwort **weekdays** entspricht **days**, überspringt aber Wochenenden (Samstag und Sonntag).

Addiert/subtrahiert man über den Sommerzeit-Umstellungspunkt hinweg eine gegebene Tagesanzahl, ergibt sich am betreffenden Tag dieselbe *lokale Uhrzeit*. Das folgende Beispiel setzt **x** auf **05:00:00**.

```
set s [clock scan {2004-10-30 05:00:00} \
           -format {%Y-%m-%d %H:%M:%S} \
           -timezone :America/New_York]
set a [clock add $s 1 day -timezone :America/New_York]
set x [clock format $a \
           -format {%H:%M:%S} -timezone :America/New_York]
```

Bei Mehrdeutigkeit, wenn dieselbe lokale Uhrzeit am selben Tag zweimal vorkommt, gilt die frühere. Liefert die Umrechnung eine unmögliche Zeit (z. B. 02:30 bei der Frühjahrs-Sommerzeitumstellung nach US-Regeln), wird gerechnet, als ob die Uhr nicht umgestellt worden wäre. **x** wird dann **03:30:00**.

```
set s [clock scan {2004-04-03 02:30:00} \
           -format {%Y-%m-%d %H:%M:%S} \
           -timezone :America/New_York]
set a [clock add $s 1 day -timezone :America/New_York]
set x [clock format $a \
           -format {%H:%M:%S} -timezone :America/New_York]
```

Eine gegebene Tages- oder Wochenanzahl funktioniert auch über den julianisch-gregorianischen Kalenderwechsel; ausgelassene Tage werden übersprungen. **z** wird **1752-09-14**.

```
set x [clock scan 1752-09-02 -format %Y-%m-%d -locale en_US]
set y [clock add $x 1 day -locale en_US]
set z [clock format $y -format %Y-%m-%d -locale en_US]
```

Führt das Addieren der Tage zu einem nicht existierenden Datum in den ausgelassenen Tagen der julianisch-gregorianischen Umstellung, wird gerechnet, als läge das Datum im julianischen Kalender.

Monate oder Jahre werden ähnlich addiert: Umwandlung in Kalenderdatum und Tageszeit, Addition der Monats- oder Jahresanzahl, Rückumwandlung in absolute Zeit.

Ist das Ergebnis unmöglich, weil der Monat zu wenige Tage hat (z. B. 1 Monat zu 31. Januar), wird der letzte Tag des Monats genommen: 1 Monat zu 31. Januar ergibt 28. bzw. 29. Februar.

Die Regeln für Sommerzeit und den Gregorianischen Kalender gelten bei Monaten und Jahren wie bei Tagen und Wochen.

Stehen mehrere *count unit*-Paare auf dem Befehl, werden sie nacheinander von links nach rechts ausgewertet.

## Hochauflösende Zeitmesser

Die meisten Unterbefehle von **clock** arbeiten mit Zeiten als Sekundenanzahl ab der Epoche; so liefert **clock seconds**. Drei Ausnahmen dienen höherer Auflösung: **clock milliseconds** und **clock microseconds** liefern Milli- bzw. Mikrosekunden ab der Epoche. **clock clicks** liefert einen plattformabhängigen Hochauflösungs-Timer. Im Gegensatz zu **clock seconds** und **clock milliseconds** ist **clock clicks** nicht an eine feste Epoche gebunden, sondern der präziseste verfügbare Intervall-Timer für relative Messungen wie Benchmarks.

## Zeiten formatieren

Der Befehl **clock format** erzeugt Zeiten zur Anzeige oder für externe Medien. Er akzeptiert Zeiten als Sekunden ab der Epoche vom 1. Januar 1970, 00:00 UTC, wie von **clock seconds**, **clock scan**, **clock add**, file atime oder file mtime.

Ist **-format** gesetzt, legt das folgende Argument fest, wie Datum und Uhrzeit formatiert werden. Die Zeichenkette besteht aus Zeichen außer **%**, durchsetzt mit *Formatgruppen* (zweistellige Folgen ab **%**). Sie sind unter Formatgruppen beschrieben.

Ist **-timezone** gesetzt, legt das folgende Argument die Zeitzone für die Formatierung fest. Statt **-timezone** *:UTC* ist die veraltete Verwendung **-gmt** *true* möglich. Zulässige Varianten unter Zeitzonen.

Ist **-locale** gesetzt, legt das folgende Argument die Locale im Format von msgcat fest. Ohne **-locale** gilt die Root-Locale **{}**, nicht die aktuelle; die aktuelle erhält man mit **-locale current**. Einige Plattformen bieten die Locale **system** (Benutzerauswahl). Unter Windows entspricht **system** den Datums-/Zeitformaten der Systemsteuerung. Wo keine Benutzerauswahl getrennt von **LC_TIME** existiert, ist **-locale system** gleichbedeutend mit **-locale current**.

## Zeiten einlesen

Der Befehl **clock scan** akzeptiert formatierte Zeichenketten und wandelt sie in Sekunden ab der Epoche vom 1. Januar 1970, 00:00 UTC um. Normalerweise folgt auf **-format** eine Zeichenkette mit dem erwarteten Eingabeformat. (Siehe Freiform-Scan ohne dieses Argument.) Die Zeichenkette besteht aus Zeichen außer **%**, durchsetzt mit *Formatgruppen* (zweistellige Folgen ab **%**), beschrieben unter Formatgruppen.

Ist **-timezone** gesetzt, legt das folgende Argument die Zeitzone für die Interpretation fest. Statt **-timezone** *:UTC* ist **-gmt** *true* veraltet möglich. Varianten unter Zeitzonen.

Ist **-locale** gesetzt, legt das folgende Argument die Locale (wie bei msgcat) fest. Standard ohne **-locale** ist **{}**; aktuelle Locale mit **-locale current**. **system** spiegelt die Benutzerauswahl (unter Windows die Systemsteuerung). Ohne getrennte Benutzerauswahl von **LC_TIME** ist **-locale system** gleich **-locale current**.

Ist **-base** gesetzt, ist das folgende Argument die *Basiszeit* (Sekunden ab der Epoche) für relative Zeiten; sonst die aktuelle Zeit.

Einlesen im festen Format ermittelt Datum, Tageszeit und Zeitzone, kombiniert sie zu einem Zeitpunkt und gibt Sekunden ab der Epoche zurück.

Vor dem Einlesen werden **%c**, **%Ec**, **%x**, **%Ex**, **%X**, **%Ex**, **%r**, **%R**, **%T**, **%D**, **%EY** und **%+** durch locale-gerechte Entsprechungen ohne diese Gruppen ersetzt. **%D** wird z. B. in **en_US** zu **%m/%d/%Y**.

Das Datum wird nach den Feldern der vorverarbeiteten Formatzeichenkette bestimmt, in dieser Priorität:

1. Enthält die Zeichenkette **%s** (Sekunden ab der Epoche), bestimmt diese Gruppe das Datum.
2. Enthält sie **%J**, **%EJ** oder **%Ej** (Kalender- oder astronomische Julian Day Number), bestimmen diese Gruppen das Datum; bei **%EJ** oder **%Ej** mit Zeitanteil auch die Uhrzeit.
3. Enthält sie ein vollständiges Set aus Jahrhundert, Jahr, Monat und Tag; Jahrhundert, Jahr und Tag des Jahres; oder ISO8601-Geschäftsjahr, Woche und Wochentag, werden diese kombiniert. Bei mehreren Sets gilt das rechteste.
4. Fehlt das Jahrhundert, aber es gibt Jahr-des-Jahrhunderts mit Monat/Tag, Jahr und Tag des Jahres, oder zweistelliges ISO8601-Geschäftsjahr mit Woche und Wochentag, werden diese kombiniert (rechtestes Set). Das Jahr liegt vermutlich zwischen 1938 und 2037.
5. Fehlt jede Jahresangabe (oder nur im alternativen Kalender der Locale) und es gibt Monat/Tag, Tag des Jahres oder Woche und Wochentag, werden diese kombiniert (rechtestes Set). Das Jahr folgt aus der Basiszeit in der gegebenen Zeitzone.
6. Enthält die Zeichenkette keines der obigen Sets, aber Tag des Monats oder Wochentag, wird anhand der Basiszeit in der Zeitzone der passende Tag der aktuellen Woche oder des Monats gewählt (Woche Mo–So nach ISO8601). Bei beiden hat der Monatstag Vorrang.
7. Liefert keine Regel ein brauchbares Datum, gilt das Datum der Basiszeit in der gegebenen Zeitzone.

Die Tageszeit wird nach den Feldern der vorverarbeiteten Zeichenkette bestimmt, in dieser Priorität:

1. Enthält die Zeichenkette **%s**, bestimmt diese Gruppe die Tageszeit.
2. Enthält sie eine Stunde im 24- oder 12-Stunden-Format mit AM/PM, bestimmt diese die Stunde. Mit Minuten- und Sekundengruppe werden diese kombiniert.
3. Enthält sie weder **%s** noch eine Stundenangabe, gilt Mitternacht (**00:00**, Beginn des Datums). Die Zeitzone folgt aus **-timezone**, **-gmt** oder der aktuellen Zeitzone.

Fehlen **%z** und **%Z**, kann die Uhrzeit am selben Tag zweimal vorkommen (mit und ohne Sommerzeit); dann gilt die erste. Bei lokalen Zeiten sollte die Eingabe die Zeitzone enthalten (nicht bei UTC).

Liefert die Interpretation eine unmögliche Zeit wegen eines Feldes außerhalb des Bereichs, wird bei **-validate** true (Standard) eine Ausnahme ausgelöst; bei false wird genug der Einheit addiert oder subtrahiert, bis der Wert passt. Tag 0 des Monats wird z. B. zum letzten Tag des Vormonats.

Bei unmöglicher oder mehrdeutiger Zeit wegen Sommerzeitumstellung ohne **%z** oder **%Z** wird gerechnet, als ob die Uhr nicht umgestellt worden wäre.

## Formatgruppen

Die folgenden Formatgruppen werden von **clock scan** und **clock format** erkannt.

**%a**

Bei der Ausgabe eine Abkürzung (*z. B.* **Mon**) des Wochentags in der gegebenen Locale. Bei der Eingabe der Wochentagsname (kurz oder voll oder ein eindeutiges Präfix).

**%A**

Bei der Ausgabe der vollständige Name (*z. B.* **Monday**) des Wochentags in der gegebenen Locale. Bei der Eingabe wie bei **%a**.

**%b**

Bei der Ausgabe eine Abkürzung (*z. B.* **Jan**) des Monatsnamens in der gegebenen Locale. Bei der Eingabe der Monatsname (kurz oder voll oder ein eindeutiges Präfix).

**%B**

Bei der Ausgabe der vollständige Monatsname (*z. B.* **January**) in der gegebenen Locale. Bei der Eingabe wie bei **%b**.

**%c**

Bei der Ausgabe eine locale-abhängige Darstellung von Datum und Tageszeit (Gregorianischer Kalender erwartet). Bei der Eingabe das, was **%c** erzeugt.

**%C**

Bei der Ausgabe die Jahrhundertzahl in indo-arabischen Ziffern. Bei der Eingabe ein oder zwei Ziffern (ggf. mit führendem Leerzeichen) als Jahrhundertzahl.

**%d**

Bei der Ausgabe die Tageszahl des Monats mit zwei Dezimalziffern. Bei der Eingabe ein oder zwei Ziffern (ggf. mit führendem Leerzeichen) als Tageszahl.

**%D**

Synonym mit **%m/%d/%Y**. Nur zum Datenaustausch in der Locale **en_US** verwenden; andere Locales ordnen die Datumsfelder meist anders.

**%e**

Bei der Ausgabe die Tageszahl mit ein oder zwei Ziffern (einstellig mit führendem Leerzeichen). Bei der Eingabe wie bei **%d**.

**%Ec**

Bei der Ausgabe eine locale-abhängige Darstellung von Datum und Tageszeit im alternativen Kalender der Locale. Bei der Eingabe das, was **%Ec** erzeugt. Der alternative Kalender muss nicht gregorianisch sein.

**%EC**

Bei der Ausgabe ein locale-abhängiger Epochenname im alternativen Kalender. Bei der Eingabe der Epochenname oder ein eindeutiges Präfix.

**%EE**

Bei der Ausgabe **B.C.E.** oder **C.E.** (oder locale-gleichwertig), ob **%Y** Jahre vor oder nach Jahr 1 der Common Era meint. Bei der Eingabe **B.C.E.**, **B.C.**, **C.E.**, **A.D.** oder die passende Abkürzung der Locale zur Festlegung für **%Y**.

**%Ej**

Bei der Ausgabe die astronomische Julian Date bzw. Julian Day Number (JDN/JD); im Gegensatz zu **%J** beginnt der Tag um Mittag. Bei der Eingabe Ziffern (oder Gleitkomma mit Zeitanteil). Zählt Kalendertage seit dem 1. Januar 4713 v. Chr. des proleptischen julianischen Kalenders inkl. Zeitanteil. Der 1. Januar 1970 entspricht astronomischem JDN 2440587.5 (wie in SQLite **select julianday(:seconds, 'unixepoch')**).

**%EJ**

Bei der Ausgabe die Calendar Julian Date als Gleitkommazahl; im Gegensatz zu **%J** mit Nachkommastellen, im Gegensatz zu **%Ej** ab Mitternacht. Bei der Eingabe wie **%Ej**. Zählt Kalendertage seit dem 1. Januar 4713 v. Chr. inkl. Zeitanteil. Der 1. Januar 1970 entspricht JDN 2440588.

**%Es**

Wirkt wie **%s**, formatiert oder parst aber lokale Sekunden (nicht POSIX-Sekunden). Hat dieselbe Priorität wie **%s** und setzt alle anderen Eingabeformate außer Kraft.

**%Ex**

Bei der Ausgabe eine locale-abhängige Datumsdarstellung im alternativen Kalender. Bei der Eingabe das, was **%Ex** erzeugt. Der alternative Kalender muss nicht gregorianisch sein.

**%EX**

Bei der Ausgabe eine locale-abhängige Tageszeitdarstellung in alternativen Ziffern. Bei der Eingabe das, was **%EX** erzeugt.

**%Ey**

Bei der Ausgabe eine locale-abhängige Jahreszahl der Epoche im alternativen Kalender und in alternativen Ziffern. Bei der Eingabe eine solche Zahl.

**%EY**

Bei der Ausgabe eine Jahresdarstellung im alternativen Kalender und in alternativen Ziffern. Bei der Eingabe das, was **%EY** erzeugt. Oft synonym mit **%EC%Ey**.

**%g**

Bei der Ausgabe eine zweistellige Jahreszahl für den wochenbasierten ISO8601-Kalender, passend zur von **%V** erzeugten Wochennummer. Bei der Eingabe eine solche zweistellige Jahreszahl (ggf. mit führendem Leerzeichen).

**%G**

Bei der Ausgabe eine vierstellige Jahreszahl für den wochenbasierten ISO8601-Kalender, passend zur von **%V** erzeugten Wochennummer. Bei der Eingabe eine solche vierstellige Jahreszahl (ggf. mit führendem Leerzeichen).

**%h**

Synonym mit **%b**.

**%H**

Bei der Ausgabe die Stunde (00-23) im 24-Stunden-Format mit zwei Ziffern. Bei der Eingabe eine solche Zahl.

**%I**

Bei der Ausgabe die Stunde (12-11) im 12-Stunden-Format mit zwei Ziffern. Bei der Eingabe eine solche Zahl.

**%j**
Bei der Ausgabe den Tag des Jahres (001-366) mit drei Ziffern. Bei der Eingabe eine solche Zahl.

**%J**
Bei der Ausgabe die Julian Day Number als Ziffernfolge. Bei der Eingabe eine Ziffernfolge als Julian Day Number — die Anzahl Kalendertage seit dem 1. Januar 4713 v. Chr. des proleptischen julianischen Kalenders. Die Epoche vom 1. Januar 1970 entspricht Julian Day Number 2440588.

**%k**
Bei der Ausgabe die Stunde (0-23) im 24-Stunden-Format mit ein oder zwei Ziffern. Bei der Eingabe eine solche Zahl.

**%l**
Bei der Ausgabe die Stunde (12-11) im 12-Stunden-Format mit ein oder zwei Ziffern. Bei der Eingabe eine solche Zahl.

**%m**
Bei der Ausgabe die Monatszahl (01-12) mit genau zwei Ziffern. Bei der Eingabe zwei Ziffern als Monatszahl.

**%M**
Bei der Ausgabe die Minute (00-59) mit genau zwei Ziffern. Bei der Eingabe zwei Ziffern als Minute.

**%N**
Bei der Ausgabe die Monatszahl (1-12) mit ein oder zwei Ziffern (einstellig mit führendem Leerzeichen). Bei der Eingabe ein oder zwei Ziffern (ggf. mit führendem Leerzeichen) als Monatszahl.

**%Od, %Oe, %OH, %OI, %Ok, %Ol, %Om, %OM, %OS, %Ou, %Ow, %Oy**
Synonym mit den Entsprechungen ohne **O**, jedoch Ausgabe und Eingabe in den locale-abhängigen alternativen Ziffern.

**%p**
Bei der Ausgabe **AM** oder **PM** passend zur Locale (bei mehreren Schriftformen bevorzugt Kleinbuchstaben). Bei der Eingabe **AM** oder **PM** der Locale in beliebiger Schreibweise.

**%P**
Bei der Ausgabe **am** oder **pm** passend zur Locale (bei mehreren Schriftformen bevorzugt Großbuchstaben). Bei der Eingabe **AM** oder **PM** der Locale in beliebiger Schreibweise.

**%Q**
Diese Formatgruppe ist für den internen Gebrauch in der Tcl-Bibliothek reserviert.

**%r**
Bei der Ausgabe eine locale-abhängige Tageszeit im 12-Stunden-Format. Bei der Eingabe das, was **%r** erzeugt.

**%R**
Bei der Ausgabe die Zeit in 24-Stunden-Notation (%H:%M). Mit Sekunden siehe **%T**. Bei der Eingabe das, was **%R** erzeugt.

**%s**
Bei der Ausgabe wird *timeVal* als Dezimalzahl in die Ausgabe eingefügt. Bei der Eingabe eine Dezimalzahl als Zeitwert ohne weitere Verarbeitung. Da **%s** einen Zeitpunkt eindeutig

festlegt, hat es Vorrang vor allen anderen Eingabeformaten.

**%S**

Bei der Ausgabe die Sekunde (00-59) mit zwei Ziffern. Bei der Eingabe zwei Ziffern als Sekunde.

**%t**

Bei der Ausgabe ein TAB-Zeichen. Bei der Eingabe ein TAB-Zeichen.

**%T**

Synonym mit **%H:%M:%S**.

**%u**

Bei der Ausgabe die Wochentagsnummer (**1**→Montag, **7**→Sonntag). Bei der Eingabe eine Ziffer als Wochentag; Sonntag kann **0** oder **7** sein.

**%U**

Bei der Ausgabe die Ordinalzahl der Jahrwoche (00-53); der erste Sonntag des Jahres ist Tag 1 der Woche 01. Bei der Eingabe zwei Ziffern, die sonst ignoriert werden. Diese Gruppe bestimmt kein Eingabedatum. Die Deutung war früher im US-Bankwesen üblich, ist heute weitgehend veraltet. ISO8601-Wochennummer: **%V**.

**%V**

Bei der Ausgabe die ISO8601-Wochennummer (01-53). Woche 01 enthält den 4. Januar, oder ist die erste Woche mit mindestens 4 Tagen, oder enthält den ersten Donnerstag des Jahres (äquivalent). Jede Woche beginnt montags. Bei der Eingabe die ISO8601-Wochennummer.

**%w**

Bei der Ausgabe die Ordinalzahl des Wochentags (Sonntag==0; Samstag==6). Bei der Eingabe eine Ziffer; Sonntag kann 0 oder 7 sein. **%w** ist nicht die ISO8601-Wochentagsnummer — die liefert **%u**.

**%W**

Bei der Ausgabe eine Wochennummer (00-53) im Jahr; Woche 01 beginnt am ersten Montag. Bei der Eingabe zwei ignorierte Ziffern. Bestimmt kein Eingabedatum. Nicht die ISO8601-Wochennummer — die liefert **%V**.

**%x**

Bei der Ausgabe das Datum in locale-abhängiger Darstellung. Bei der Eingabe das, was **%x** erzeugt, zur Bestimmung des Kalenderdatums.

**%X**

Bei der Ausgabe die Tageszeit in locale-abhängiger Darstellung. Bei der Eingabe das, was **%X** erzeugt, zur Bestimmung der Tageszeit.

**%y**

Bei der Ausgabe das zweistellige Jahr des Jahrhunderts. Bei der Eingabe zwei Ziffern; das Datum liegt vermutlich zwischen 1938 und 2037. **%y** passt nicht zur ISO8601-Wochennummer **%V** — dafür **%g** verwenden.

**%Y**

Bei der Ausgabe das vierstellige Kalenderjahr. Bei der Eingabe vier Ziffern zur Datumsbestimmung. **%Y** passt nicht zur ISO8601-Wochennummer **%V** — dafür **%G** verwenden.

**%z**

Bei der Ausgabe die aktuelle Zeitzone als Offset östlich (+hhmm) oder westlich (-hhmm) von

Greenwich. Bei der Eingabe ein Zeitzonen-Spezifizierer (siehe Zeitzonen) zur Bestimmung der Zeitzone.

**%Z**

Bei der Ausgabe den Namen der aktuellen Zeitzone, ggf. in der Locale übersetzt. Bei der Eingabe ein Zeitzonen-Spezifizierer (siehe Zeitzonen). Bei der Eingabe im Allgemeinen nur beim Parsen von RFC822-Daten verwenden; sonst mehrdeutig (**BST** z. B. British Summer Time oder Brazilian Standard Time). Für maschinelle Datums-/Zeitstrings numerische Zeitzonen empfohlen.

**%%**

Bei der Ausgabe ein wörtliches **%**-Zeichen. Bei der Eingabe ein wörtliches **%**-Zeichen.

**%+**

Synonym mit **%a %b %e %H:%M:%S %Z %Y**.

## Zeitzonen

Verarbeitet der Befehl **clock** eine lokale Zeit, stehen mehrere Quellen für die Zeitzone zur Verfügung, in dieser Reihenfolge:

1. Eine in der zu parsenden Zeichenkette angegebene Zeitzone, erkannt über **%z** oder **%Z**.
2. Eine mit **-timezone** am **clock**-Befehl angegebene Zeitzone (oder gleichwertig **-gmt 1**).
3. Eine in der Umgebungsvariable **TCL_TZ** angegebene Zeitzone.
4. Eine in der Umgebungsvariable **TZ** angegebene Zeitzone.
5. Die lokale Zeitzone aus der Systemsteuerung unter Windows.
6. Die Vorstellung der C-Bibliothek über die lokale Zeitzone gemäß **mktime** und **localtime**.

Nur in Fall [1] wird geprüft, ob die Zeichenkette eine der folgenden ist:

```
gmt     ut      utc     bst     wet     wat     at
nft     nst     ndt     ast     adt     est     edt
cst     cdt     mst     mdt     pst     pdt     yst
ydt     hst     hdt     cat     ahst    nt      idlw
cet     cest    met     mewt    mest    swt     sst
eet     eest    bt      it      zp4     zp5     ist
zp6     wast    wadt    jt      cct     jst     cast
cadt    east    eadt    gst     nzt     nzst    nzdt
idle
```

Steht die Zeichenkette in dieser Liste, bezeichnet sie eine bekannte Zeitzone.

Für Zeitzonen in Fall [1], die nicht in der Liste stehen, und stets in den Fällen [2]–[6] gelten folgende Regeln.

Beginnt die Zeitzone mit einem Doppelpunkt, ist sie ein standardisierter Name wie **:America/New_York** mit Regeln für verschiedene Orte. Eine vollständige Liste der Ortsnamen ist hier zu lang. Auf den meisten Tcl-Installationen liegen die Definitionen in benannten Dateien im Verzeichnis "*/no_backup/tools/lib/tcl9.0/clock/tzdata*". Auf einigen Unix-Systemen fehlen diese Dateien; die Definitionen stammen dann aus Systemdateien in "*/usr/share/zoneinfo*", "*/usr/share/lib/zoneinfo*" oder "*/usr/local/etc/zoneinfo*". Als Sonderfall bezeichnet **:localtime** die von der C-Bibliothek definierte lokale Zeitzone.

Eine Zeitzone aus Plus- oder Minuszeichen und vier oder sechs Dezimalziffern wird als Offset in

Stunden, Minuten und (bei sechs Ziffern) Sekunden von UTC gelesen. Plus bedeutet östlich von Greenwich, Minus westlich.

Eine der Posix-Spezifikation der Umgebungsvariable **TZ** entsprechende Zeichenkette wird erkannt. Die Spezifikation steht unter https://pubs.opengroup.org/onlinepubs/009695399/basedefs/xbd_chap08.html.

Enthält die Posix-Zeitzone einen DST- (Daylight Saving Time-) Teil ohne Regel für Beginn und Ende der Sommerzeit, gelten Standardregeln. Bei Offsets zwischen 0 und +12 die europäisch/russischen Regeln, sonst die US-Regeln. In Europa (+0 bis +2) Umstellung auf Sommerzeit am letzten Sonntag im März um 1:00 GMT, zurück am letzten Sonntag im Oktober um 2:00 GMT. In Russland (+3 bis +12) gleiche Daten, Umstellung auf Sommerzeit um 2:00 Ortszeit, zurück um 3:00 Ortszeit. In den USA Umstellung auf Sommerzeit am zweiten Sonntag im März um 2:00 Ortszeit, zurück am ersten Sonntag im November um 3:00 Ortszeit. Damit sind DST-Berechnungen in europäischen, russischen und US-(kompatible) Zonen für Daten ab 2007 korrekt, sofern die Regeln sich nicht erneut ändern.

Jede andere Zeitzone wird mit vorangestelltem Doppelpunkt als Ortsname wie oben verarbeitet.

## Lokalisierung

Entwickler, die Datums- und Zeitein-/ausgabe lokalisieren wollen, finden eine Spezifikation unter https://tip.tcl-lang.org/173.

## Freiform-Scan

Wird **clock scan** ohne **-format** aufgerufen, wird ein *Freiform-Scan* angefordert. Diese Form ist veraltet, weil zu viele Mehrdeutigkeiten bestehen (bedeutet die Zeichenkette **2000** ein Jahr, eine Tageszeit oder eine Menge?). Keine Regelmenge für Freiform-Daten und -Zeiten liefert in allen Fällen erwartbare Ergebnisse.

Beim Freiform-Scan sind nur **-base** und **-gmt** zulässig. **-timezone** und **-locale** führen ohne **-format** zu einem Fehler.

Für Nutzer von Legacy-Code mit Freiform-Scan folgt die Dokumentation der Interpretation:

Ist nur eine Uhrzeit angegeben, gilt das aktuelle Datum. Enthält *inputString* keine Zeitzonen-Abkürzung, gilt die lokale Zeitzone, außer **-gmt** ist wahr — dann wird die angegebene Zeit relativ zur Greenwich Mean Time gerechnet. **-gmt** beeinflusst nur den berechneten Zeitwert, nicht die Deutung von **-base**.

Ist **-base** gesetzt, muss das nächste Argument ein ganzzahliger Clock-Wert sein; nur das Datum wird verwendet, nicht die Uhrzeit — nützlich für Zeiten an einem bestimmten Tag oder andere datumsrelative Umrechnungen.

*inputString* besteht aus null oder mehr Spezifikationen folgender Form:

***time***

Eine Tageszeit der Form **hh?:mm?:ss?? ?meridian? ?zone?** oder **hhmm ?meridian? ?zone?**. Ohne Meridian wird **hh** im 24-Stunden-Format interpretiert. Die Formate "24:00" und "24:00:00" (mit oder ohne Doppelpunkt) sind nur ohne *meridian* zulässig.

***date***

Ein bestimmter Monat und Tag mit optionalem Jahr. Zulässige Formate: **mm/dd?/yy?**, **monthname dd?, yy?**, **day, dd monthname ?yy?**, **dd monthname yy**,**?CC?yymmdd** und **dd-monthname-?CC?yy**. Standardjahr ist das aktuelle. Jahre unter 100: 00–68 als

2000–2068, 69–99 als 1969–1999. Nicht alle Plattformen können 38–70 darstellen — Fehler möglich.

***ISO 8601 point-in-time***

Ein ISO-8601-Zeitpunkt, z. B. **CCyymmddThhmmss**, wobei **T** das wörtliche **T** ist, oder **CCyymmdd hhmmss**, **CCyymmddThh:mm:ss** oder **CCyy-mm-ddThh:mm:ss**. Nur diese vier Formate werden akzeptiert. Der Befehl akzeptiert *nicht* den vollen ISO8601-Umfang. Schaltsekunden werden beispielsweise nicht unterstützt. Die Formate "24:00" und "24:00:00" (mit oder ohne Doppelpunkt) werden unterstützt. Andere Formate erkennt **clock scan** mit explizitem **-format**.

***relative time***

Relativ zur aktuellen Zeit: **number** unit. Einheiten: **year**, **fortnight**, **month**, **week**, **day**, **hour**, **minute** (oder **min**), **second** (oder **sec**), Singular oder Plural wie **3 weeks**. Zusätzlich: **tomorrow**, **yesterday**, **today**, **now**, **last**, **this**, **next**, **ago**.

Das Datum wird in folgenden Schritten berechnet.

Zuerst absolute Datum-/Zeit- Angaben; darauf Wochentags-Spezifikationen; dann relative Angaben. Ist nur ein Datum oder Tag ohne absolute oder relative Zeit angegeben, gilt Mitternacht. Zuletzt Korrektur für Sommerzeit und für den Übergang vom Ende eines langen Monats zu einem kurzen.

Die Rangfolge einzelner Token bzw. die Reihenfolge bei der Berechnung der Zeit ist komplex und hängt stark von der Genauigkeit des Tokentyps ab. Im folgenden Beispiel enthält die zweite Datumszeichenkette "next January", daher ergibt sich das nächste Jahr, aber im Januar. Die dritte Datumszeichenkette enthält neben "January" zusätzlich "Fri", sodass sich der nächste Freitag ergibt. Beide gewinnen daher vor "385 days" bzw. machen die Angabe genauer, weil diese Tokentypen eine höhere Genauigkeit haben.

```
% clock format [clock scan "5 years 18 months 385 days" -base 0 -gmt
    1] -gmt 1
Thu Jul 21 00:00:00 GMT 1977
% clock format [clock scan "5 years 18 months 385 days next January"
    -base 0 -gmt 1] -gmt 1
Sat Jan 21 00:00:00 GMT 1978
% clock format [clock scan "5 years 18 months 385 days next January
    Fri" -base 0 -gmt 1] -gmt 1
Fri Jan 27 00:00:00 GMT 1978
```

## Copyright

Copyright © 2004 Kevin B. Kenny kennykb@acm.org. All rights reserved.

*Copyright (c) 2004 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*

# close

*Stichwörter:* close

## Name

close - Schließt einen offenen Kanal

## Übersicht

**close** *channel* ?*r(ead)*|*w(rite)*?

## Beschreibung

Der Befehl **close** wurde durch den Befehl chan close ersetzt, der die gleiche Syntax und die gleichen Optionen unterstützt.

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# concat

*Stichwörter:* concat, concatenate, join, list

## Name

concat - Verbindet Listen miteinander

## Übersicht

**concat** ?*arg arg* ...?

## Beschreibung

Der Befehl verbindet die Argumente mit Leerzeichen, nachdem führende und nachfolgende Leerraumzeichen an jedem Argument entfernt wurden. Sind alle Argumente Listen, entspricht das dem Zusammenfügen zu einer Liste. Leere Argumente (nach dem Trimmen) werden ignoriert. Beliebig viele Argumente; ohne *arg*s: leere Zeichenkette.

## Beispiele

**concat** verkettet Listen und flacht sie dabei ab:

```
% concat a b {c d e} {f {g h}}
a b c d e f {g h}
```

Es verkettet auch Nicht-Listen:

```
% concat " a b {c   " d "  e} f"
a b {c d e} f
```

Die Verkettung entfernt keine Leerzeichen in der Mitte von Werten:

```
% concat "a   b   c" { d e f }
a   b   c d e f
```

(drei Leerzeichen zwischen **a**, **b** und **c**).

Für echte Listenverkettung list mit Expansion:

```
% list {*}"a   b   c" {*}{ d e f }
a b c d e f
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# configurable

*Stichwörter:* configurable, class, object, properties, configuration

## Name

oo::configurable, configure, property - Klasse für konfigurierbare Klassen und Objekte sowie Unterstützung konfigurierbarer Properties

## Übersicht

**package require TclOO**

**oo::configurable create** *class* ?*definitionScript*?

**oo::define** *class* **{**
**property** *propName* ?*options...*? ?*propName* ?*options...*?*...*?
**}**

**oo::objdefine** *object* **{**
**property** *propName* ?*options...*? ?*propName* ?*options...*?*...*?
**}**

*objectName* **configure**
*objectName* **configure** -*prop*
*objectName* **configure** -*prop value* ?-*prop value...*?

## Klassenhierarchie

```
oo::object
   → oo::class
       → oo::configurable

oo::object
   → oo::class
       → oo::configurablesupport::configurable
```

## Beschreibung

Konfigurierbare Objekte sind Objekte, die über eine **configure**-Methode konfiguriert werden können. Jede konfigurierbare Entität eines Objekts heißt Property des Objekts. Properties können auf Klassen oder Instanzen definiert werden; beim Konfigurieren eines Objekts können alle von seinen Klassen (direkt oder indirekt) oder von der Instanz selbst definierten Properties konfiguriert werden.

Die Metaklasse **oo::configurable** installiert grundlegende Unterstützung für konfigurierbare Objekte in einer Klasse. Dazu gehört ein **property**-Definitionsbefehl in Definitionsskripten für Klasse und Instanzen (z. B. aus dem Konstruktor der Klasse, innerhalb von oo::define und oo::objdefine) sowie eine **configure**-Methode in den Instanzen.

### Configure-Methode

Das Verhalten der **configure**-Methode orientiert sich am Befehl fconfigure/chan configure.

Ohne weitere Argumente gibt die **configure**-Methode ein alphabetisch sortiertes Dictionary aller lesbaren und les- und schreibbaren Properties mit ihren aktuellen Werten zurück.

Mit genau einem weiteren Argument muss dieses der Name einer auszulesenden Property sein (oder ein eindeutiges Präfix davon); der Wert wird zurückgegeben.

Andernfalls: Bei einer geraden Anzahl weiterer Argumente gibt jedes Argumentpaar einen Property-Namen (oder ein eindeutiges Präfix) und den zu setzenden Wert an. Die Properties werden in der angegebenen Reihenfolge gesetzt, einschließlich Duplikaten. Schlägt das Setzen einer Property fehl, schlägt die **configure**-Methode insgesamt fehl; vorherige Paare (falls vorhanden) bleiben angewendet, nachfolgende Paare (falls vorhanden) werden nicht angewendet. Bei Erfolg ist das Ergebnis der **configure**-Methode in diesem Modus eine leere Zeichenkette.

## Property-Definitionen

Wurde eine Klasse von der Metaklasse **oo::configurable** (oder einer ihrer Unterklassen) erzeugt, erhält sie eine zusätzliche Definition **property**. Die Definition **property** legt eine oder mehrere Properties fest, die von Instanzen der Klasse bereitgestellt werden.

Der Befehl **property** erwartet zuerst den Namen einer zu definierenden Property *ohne führenden Bindestrich*, gefolgt von Option-Wert-Paaren, die das Grundverhalten der Property ändern. Anschließend können beliebig viele weitere Property-Definitionen folgen. Unterstützte Optionen:

**-get *getterScript***

Definiert die Implementierung des Lesens der Property; *getterScript* wird zum Rumpf einer Methode (ohne Argumente) auf der Klasse, wenn die Property-Art das Lesen erlaubt. Die Methode heißt und ist standardmäßig ein einfaches Lesen der Instanzvariablen gleichen Namens wie die Property (z. B. **property** xyz ergibt eine Methode "").

**-kind *propertyKind***

Legt die Art der Property fest. *propertyKind* muss genau eines von **readable**, **writable** oder **readwrite** (Standard) sein; die Property ist dann nur lesbar, nur schreibbar bzw. les- und schreibbar. Nur-lesbare Properties können nur gelesen, nur-schreibbare nur geschrieben werden; les-schreibbare Properties können beides.

Beachten Sie, dass nur-schreibbare Properties schwer auffindbar sind: Die **configure** -Methode meldet sie nicht, außer in Fehlermeldungen beim Schreiben auf eine nicht existierende Property.

**-set *setterScript***

Definiert die Implementierung des Schreibens der Property; *setterScript* wird zum Rumpf einer Methode mit einem Argument *value* auf der Klasse, wenn die Property-Art das Schreiben erlaubt. Die Methode heißt und ist standardmäßig ein einfaches Schreiben der Instanzvariablen gleichen Namens wie die Property (z. B. **property** xyz ergibt eine Methode "").

Instanzen einer von **oo::configurable** erzeugten Klasse unterstützen ebenfalls **property** -Definitionen; die Semantik ist wie oben, die Properties werden jedoch nur auf der Instanz definiert.

Die von **property** definierten Implementierungsmethoden sollten nicht privat sein, da sie sonst für die **configure**-Implementierung unzugänglich sind (absichtlich; der Property-Konfigurationsmechanismus ist vor allem für die Nutzung außerhalb einer Klasse gedacht, während eine Klasse Variablen direkt zugreifen kann). Die von den

Standardimplementierungen der Properties genutzten Variablen *dürfen* privat sein, wenn so deklariert.

## Erweiterte Verwendung

Das System konfigurierbarer Klassen besteht aus mehreren Teilen. Die Metaklasse **oo::configurable** mischt eine Klasse ein und setzt bei der Objekterzeugung Definitions-Namespaces, die die übrige Mechanik bereitstellen. Die zentralen Implementierungsteile sind hier aufgeführt, damit anderer Code sie nutzen kann:

**oo::configuresupport::configurable**
Klasse, die die **configure**-Methode bereitstellt (beschrieben unter Configure-Methode).

**oo::configuresupport::configurableclass**
Namespace mit dem Definitionsdialekt, der die **property**-Deklaration für Klassen bereitstellt (über oo::define und Klassenkonstruktoren), wie unter Property-Definitionen beschrieben. Er namespace exportiert den Befehl **property** für einfache Nutzung in benutzerdefinierten Definitionsdialekten.

**oo::configuresupport::configurableobject**
Namespace mit dem Definitionsdialekt, der die **property**-Deklaration für Instanzobjekte bereitstellt (über oo::objdefine und die self-Deklaration in oo::define), wie unter Property-Definitionen beschrieben. Er namespace exportiert den Befehl **property** für einfache Nutzung in benutzerdefinierten Definitionsdialekten.

Der zugrunde liegende Property-Erkennungsmechanismus nutzt vier Slots (siehe oo::define für die Bedeutung), die konfigurierbare Properties auflisten. Diese Slots legen selbst keine Semantik fest außer eindeutigen Namen, fehlender wichtiger Reihenfolge, Vererbbarkeit und Auffindbarkeit auf Klassen und Instanzen.

Diese Slots und ihre beabsichtigte Semantik:

**oo::configuresupport::readableproperties**
Menge der Properties einer Klasse (ohne die der Oberklassen), die beim Konfigurieren einer Instanz der Klasse gelesen werden dürfen. Dieser Slot kann auch mit **info class properties** gelesen werden.

**oo::configuresupport::writableproperties**
Menge der Properties einer Klasse (ohne die der Oberklassen), die beim Konfigurieren einer Instanz der Klasse geschrieben werden dürfen. Dieser Slot kann auch mit **info class properties** gelesen werden.

**oo::configuresupport::objreadableproperties**
Menge der Properties einer Objektinstanz (ohne die ihrer Klassen), die beim Konfigurieren des Objekts gelesen werden dürfen. Dieser Slot kann auch mit **info object properties** gelesen werden.

**oo::configuresupport::objwritableproperties**
Menge der Properties einer Objektinstanz (ohne die ihrer Klassen), die beim Konfigurieren des Objekts geschrieben werden dürfen. Dieser Slot kann auch mit **info object properties** gelesen werden.

Obwohl es sich um Slots handelt, liegen sie *nicht* in den Standard-Namespaces oo::define oder oo::objdefine; in Definitionsskripten müssen sie voll qualifiziert referenziert werden. Sie sind Bausteine des konfigurierbaren Property-Systems und nicht für direkte Nutzung durch normalen

Anwendercode gedacht.

### Implementierungshinweis

Die **configure**-Methode nutzt **info object properties** mit der Option **-all**, um zu ermitteln, welche Properties sie bearbeiten darf.

## Beispiele

Hier wird eine einfache konfigurierbare Klasse erzeugt und ihre Konfiguration gezeigt:

```
oo::configurable create Point {
    property x y
    constructor args {
        my configure -x 0 -y 0 {*}$args
    }
    variable x y
    method print {} {
        puts "x=$x, y=$y"
    }
}

set pt [Point new -x 27]
$pt print;   # x=27, y=0
$pt configure -y 42
$pt print;   # x=27, y=42
puts "distance from origin: [expr {
    hypot([$pt configure -x], [$pt configure -y])
}]";         # distance from origin: 49.92995093127971
puts [$pt configure]
             # -x 27 -y 42
```

Eine solche konfigurierbare Klasse kann durch Subklassierung erweitert werden; die Unterklasse muss ebenfalls mit **oo::configurable** erzeugt werden, wenn sie die **property**-Definition nutzt:

```
oo::configurable create Point3D {
    superclass Point
    property z
    constructor args {
        next -z 0 {*}$args
    }
}

set pt2 [Point3D new -x 2 -y 3 -z 4]
puts [$pt2 configure]
             # -x 2 -y 3 -z 4
```

Hat man eine konfigurierbare Klasse, können auch Instanz-Properties hinzugefügt werden. (Die hinterlegten Variablen aller Properties starten unset.) Unten wird ein eindeutiges Präfix eines Property-Namens beim Setzen verwendet; das ist für alle Properties möglich, vollständige

Namen werden jedoch empfohlen, weil Unterklassen ein eindeutiges Präfix nicht mehr eindeutig machen.

```
oo::objdefine $pt {
    property color
}
$pt configure -c bisque
puts [$pt configure]
             # -color bisque -x 27 -y 42
```

Abgeleitete Properties sind möglich, indem sie nur lesbar gemacht und mit einem berechnenden Skript versehen werden.

```
oo::configurable create PointMk2 {
    property x y
    property distance -kind readable -get {
        return [expr {hypot($x, $y)}]
    }
    variable x y
    constructor args {
        my configure -x 0 -y 0 {*}$args
    }
}

set pt3 [PointMk2 new -x 3 -y 4]
puts [$pt3 configure -distance]
             # 5.0
$pt3 configure -distance 10
             # ERROR: bad property "-distance": must be -x or -y
```

Setter dienen zur Validierung des Typs einer Property:

```
oo::configurable create PointMk3 {
    property x -set {
        if {![string is double -strict $value]} {
            error "-x property must be a number"
        }
        set x $value
    }
    property y -set {
        if {![string is double -strict $value]} {
            error "-y property must be a number"
        }
        set y $value
    }
    variable x y
    constructor args {
        my configure -x 0 -y 0 {*}$args
    }
}

set pt4 [PointMk3 new]
puts [$pt4 configure]
             # -x 0 -y 0
$pt4 configure -x 3 -y 4
puts [$pt4 configure]
             # -x 3 -y 4
$pt4 configure -x "obviously not a number"
             # ERROR: -x property must be a number
```

*Copyright (c) 2019 Donal K. Fellows*

# const

*Stichwörter:* const, namespace, procedure, variable, constant

## Name

const - Konstante erzeugen und initialisieren

## Übersicht

**const** *varName value*

## Beschreibung

Der Befehl wird üblicherweise im Körper einer Prozedur (oder Methode, oder in einem Lambda-Term) verwendet, um dort eine Konstante anzulegen, oder im Körper von namespace eval, um eine Konstante im Namespace zu erzeugen. Die Konstante ist eine nicht veränderbare Variable *varName*, initialisiert mit *value*. Bei Erfolg liefert **const** stets einen leeren String.

Existiert *varName* noch nicht, wird sie mit Wert *value* erzeugt und als Konstante markiert: andere Befehle (set, append, incr, unset) dürfen die Variable weder ändern noch entfernen; vor Traces wird geprüft, ob eine Variable Konstante ist. Existiert *varName* bereits, ist das ein Fehler — außer die Variable ist bereits als Konstante markiert (dann ist **const** eine No-Op).

*varName* darf kein qualifizierter Name sein und darf kein Array-Element bezeichnen. Existiert die Variable bereits als Array, ist das ein Fehler.

Konstanten werden normalerweise erst entfernt, wenn die umgebende Prozedur endet oder der Namespace gelöscht wird.

## Beispiele

Konstante in einer Prozedur:

```
proc foo {a b} {
    const BAR 12345
    return [expr {$a + $b + $BAR}]
}
```

Konstante in einem Namespace für einen regulären Ausdruck:

```
namespace eval someNS {
    const FOO_MATCHER {(?i)}\mfoo\M}

    proc findFoos str {
        variable FOO_MATCHER
        regexp -all $FOO_MATCHER $str
    }

    proc findFooIndices str {
        variable FOO_MATCHER
        regexp -all -indices $FOO_MATCHER $str
    }
}
```

Konstante in einer Schleife erzeugt keinen Fehler:

```
proc foo {n} {
    set result {}
    for {set i 0} {$i < $n} {incr i} {
        const X 123
    lappend result [expr {$X + $i**2}]
    }
}
```

*Copyright (c) 2023 Donal K. Fellows*

## continue

*Stichwörter:* continue, iteration, loop

### Name

continue - Springt zur nächsten Schleifeniteration

### Übersicht

**continue**

### Beschreibung

Dieser Befehl wird typischerweise im Rumpf eines Schleifenbefehls wie for, foreach oder while aufgerufen. Er liefert den Ergebniscode 4 (TCL_CONTINUE), wodurch eine Continue-Ausnahme ausgelöst wird. Die Ausnahme bricht die aktuelle Skriptausführung bis zum innersten enthaltenden Schleifenbefehl ab, der dann mit der nächsten Schleifeniteration fortfährt. Continue-Ausnahmen werden auch in einigen anderen Situationen behandelt, etwa beim Befehl catch und in den äußersten Skripten von Prozedurrümpfen.

### Beispiele

Gibt für jede Ganzzahl von 0 bis 10 *außer* 5 eine Zeile aus:

```
for {set x 0} {$x<10} {incr x} {
    if {$x == 5} {
        continue
    }
    puts "x is $x"
}
```

*Copyright (c) 1993-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# cookiejar

*Stichwörter:* cookiejar, cookie, internet, security policy, www

## Name

cookiejar - Implementierung des Cookie-Jar-Protokolls des Tcl-http-Pakets

## Übersicht

**package require cookiejar** ?**0.1**?

**::http::cookiejar configure** ?*optionName*? ?*optionValue*?
**::http::cookiejar create** *name* ?*filename*?
**::http::cookiejar new** ?*filename*?

*cookiejar* **destroy**
*cookiejar* **forceLoadDomainData**
*cookiejar* **getCookies** *protocol host path*
*cookiejar* **storeCookie** *options*
*cookiejar* **lookup** ?*host*? ?*key*?

## Beschreibung

Das Paket **cookiejar** stellt eine Implementierung des Cookie-Jar-Protokolls des http-Pakets unter Verwendung einer SQLite-Datenbank bereit. Es bietet einen Hauptbefehl, **::http::cookiejar**, eine TclOO-Klasse, die instanziiert werden sollte, um einen Cookie-Jar zu erzeugen, der eine bestimmte HTTP-Sitzung verwaltet.

Die Datenbankverwaltungsrichtlinie kann auf Paketebene über die Methode **configure** des Klassenobjekts **::http::cookiejar** gesteuert werden:

**::http::cookiejar configure ?*optionName*? ?*optionValue*?**

Wenn weder *optionName* noch *optionValue* angegeben sind, gibt dieser Befehl eine Kopie der Konfiguration als Tcl-Dictionary zurück. Wenn nur *optionName* angegeben ist, wird nur der Wert der benannten Option zurückgegeben. Wenn sowohl *optionName* als auch *optionValue* angegeben sind, wird die benannte Option auf den angegebenen Wert geändert.

Unterstützte Optionen sind:

**-domainfile *filename***

Eine Datei (standardmäßig innerhalb des cookiejar-Pakets), die eine Beschreibung der Liste der Top-Level-Domains enthält (z. B. **.com** oder **.co.jp**). Solche Domains dürfen *keine* auf ihnen gesetzten Cookies akzeptieren. Beachten Sie, dass die Liste solcher Domains sowohl sicherheitsrelevant als auch *nicht* konstant ist und regelmäßig neu abgerufen werden sollte. Cookie-Jars führen einen eigenen Cache der Domain-Liste.

**-domainlist *url***

Eine URL, von der die Liste der Top-Level-Domains (z. B. **.com** oder **.co.jp**) abgerufen wird. Solche Domains dürfen *keine* auf ihnen gesetzten Cookies akzeptieren. Beachten Sie, dass die Liste solcher Domains sowohl sicherheitsrelevant als auch *nicht* konstant ist und regelmäßig neu abgerufen werden sollte. Cookie-Jars führen einen eigenen Cache der Domain-Liste.

**-domainrefresh *intervalMilliseconds***

Die Anzahl Millisekunden zwischen Prüfungen der **-domainlist** auf neue Domains.

**-loglevel *level***

Die Protokollierungsstufe dieses Pakets. Die Protokollierungsstufe muss (in absteigender Ausführlichkeit) eine von **debug**, **info**, **warn** oder **error** sein.

**-offline *flag***

Ermöglicht, die Cookie-Verwaltungsengine in den Offline-Modus zu versetzen. Im Offline-Modus wird die Domain-Liste sofort aus der in der Option **-domainfile** konfigurierten Datei gelesen, und die Option **-domainlist** wird nicht verwendet; außerdem wird die Option **-domainrefresh** faktisch ignoriert.

**-purgeold *intervalMilliseconds***

Die Anzahl Millisekunden zwischen Prüfungen der Datenbank auf abgelaufene Cookies; abgelaufene Cookies werden gelöscht.

**-retain *cookieCount***

Die maximale Anzahl Cookies, die in der Datenbank aufbewahrt werden.

**-vacuumtrigger *deletionCount***

Anzahl persistenter Cookie-Löschungen zwischen zwei **VACUUM**-Durchläufen der Datenbank.

Cookie-Jar-Instanzen können mit allen üblichen TclOO-Instanzerzeugungsmethoden (**create** oder **new**) erzeugt werden.

**::http::cookiejar new ?*filename*?**

Wenn ein Argument *filename* angegeben wird, ist es der Name einer Datei mit einer SQLite-Datenbank, die die persistenten Cookies des Cookie-Jars enthält; die Datenbank wird erzeugt, wenn die Datei noch nicht existiert. Wenn *filename* nicht angegeben wird, liegt die Datenbank vollständig im Speicher, wodurch alle darin enthaltenen Cookies faktisch Sitzungs-Cookies sind.

## Instanzmethoden

Die folgenden Methoden werden von Instanzen unterstützt:

***cookiejar* destroy**

Dies ist die Standard-TclOO-Zerstörungsmethode. Sie löscht die SQLite-Datenbank *nicht*, wenn sie auf die Platte geschrieben wurde. Aufrufer sind dafür verantwortlich, dass der Cookie-Jar zum Zeitpunkt der Zerstörung nicht mehr vom http-Paket verwendet wird.

***cookiejar* forceLoadDomainData**

Diese Methode veranlasst den Cookie-Jar, die Domain-Listendaten sofort zu laden (und zu cachen). Die Domain-Liste wird aus der auf Paketebene konfigurierten **-domainlist** geladen, sofern diese aktiviert ist, andernfalls aus der auf Paketebene konfigurierten **-domainfile**.

***cookiejar* getCookies *protocol host path***

Diese Methode ermittelt die Cookies für eine bestimmte HTTP-Anfrage. *Sie implementiert das http-Cookie-Jar-Protokoll.*

***cookiejar* policyAllow *operation domain path***

Diese Methode wird von der Methode **storeCookie** aufgerufen, um zu entscheiden, ob *operation* für *domain* und *path* ausgeführt werden darf. Die Prüfung erfolgt unmittelbar vor der Aktualisierung der Datenbank, nachdem die eingebauten Sicherheitsprüfungen durchgeführt wurden, und soll einen booleschen Wert zurückgeben; ist der Wert falsch, wird die Operation abgelehnt und die Datenbank nicht geändert. Unterstützte *operation*en sind:

**delete**

Die *domain* möchte ein Cookie löschen.

**session**

Die *domain* möchte ein Sitzungs-Cookie anlegen oder aktualisieren.

**set**

Die *domain* möchte ein persistentes Cookie (mit definierter Lebensdauer) anlegen oder aktualisieren.

Die Standardimplementierung dieser Methode gibt einfach true zurück; Unterklassen dieser Klasse können eigene Regeln festlegen.

***cookiejar* storeCookie *options***

Diese Methode speichert ein einzelnes Cookie aus einer bestimmten HTTP-Antwort. Cookies, die Sicherheitsprüfungen nicht bestehen, werden ignoriert. *Sie implementiert das http-Cookie-Jar-Protokoll.*

***cookiejar* lookup ?*host*? ?*key*?**

Diese Methode sucht ein Cookie durch exakte Host- (oder Domain-)Übereinstimmung. Wenn weder *host* noch *key* angegeben sind, wird die Liste der Hosts zurückgegeben, für die ein Cookie gespeichert ist. Wenn nur *host* (Hostname oder Domainname) angegeben ist, wird die Liste der gespeicherten Cookie-Schlüssel für diesen Host zurückgegeben. Wenn sowohl *host* als auch *key* angegeben sind, wird der Wert für diesen Schlüssel zurückgegeben; es ist ein Fehler, wenn Host oder Schlüssel nicht exakt übereinstimmen.

## Beispiele

Der einfachste Weg, einen Cookie-Jar zu verwenden, besteht darin, ihn zu Beginn der Anwendung dauerhaft zu konfigurieren.

```
package require http
package require cookiejar

set cookiedb [file join [file home] cookiejar]
http::config -cookiejar [http::cookiejar new $cookiedb]

# Keine weiteren expliziten Schritte zum Verwenden von Cookies nötig
set tok [http::geturl http://core.tcl-lang.org/]
```

Um nur einer bestimmten Domain Cookies zu erlauben, etwa weil nur ein bestimmter Host Sitzungen anlegen und verwalten soll, legen Sie eine Unterklasse an, die diese Richtlinie durchsetzt.

```
package require http
package require cookiejar

oo::class create MyCookieJar {
    superclass http::cookiejar

    method policyAllow {operation domain path} {
        return [expr {$domain eq "my.example.com"}]
    }
}

set cookiedb [file join [file home] cookiejar]
http::configure -cookiejar [MyCookieJar new $cookiedb]

# Keine weiteren expliziten Schritte zum Verwenden von Cookies nötig
set tok [http::geturl http://core.tcl-lang.org/]
```

*Copyright (c) 2014-2018 Donal K. Fellows.*

## copy

*Stichwörter:* copy, clone, duplication, object

### Name

oo::copy - Erzeugt Kopien von Objekten und Klassen

### Übersicht

**package require tcl::oo**

**oo::copy** *sourceObject* ?*targetObject*? ?*targetNamespace*?

### Beschreibung

Der Befehl **oo::copy** erzeugt eine Kopie eines Objekts oder einer Klasse. Er nimmt den Namen des zu kopierenden Objekts oder der Klasse *sourceObject* und optional den Namen des zu erzeugenden Objekts oder der Klasse *targetObject*, der relativ zum aktuellen Namespace aufgelöst wird, wenn er kein absolut qualifizierter Name ist, und *targetNamespace*, den Namen des Namespace, der den internen Zustand des Objekts (my-Befehl usw.) hält; er *darf nicht* auf einen existierenden Namespace verweisen. Ist *targetObject* oder *targetNamespace* weggelassen oder als leere Zeichenkette angegeben, wird ein neuer Name gewählt. Namen werden, sofern nicht angegeben, mit demselben Algorithmus wie die Methode **new** von oo::class gewählt. Das kopierte Objekt hat dieselbe Klasse wie das Quellobjekt und alle seine objektspezifischen Methoden werden kopiert. Ist es eine Klasse, werden auch alle Klassenmethoden der Klasse kopiert, aber keine ihrer Instanzen.

Nachdem targetObject erzeugt und alle Konfigurationsdefinitionen (z. B. Methoden, Filter, Mixins) kopiert wurden, wird die Methode von targetObject aufgerufen, um das erzeugte Objekt anzupassen, z. B. durch Installation zugehöriger Variablen-Spuren. Das einzige Argument ist sourceObject. Die Standardimplementierung dieser Methode (in oo::object) kopiert nur die Prozeduren und Variablen im Namespace von sourceObject in den Namespace von targetObject.

Liefert dieser Methodenaufruf kein erfolgreiches Ergebnis (d. h. einen Fehler oder eine andere Ausnahme), wird targetObject gelöscht und ein Fehler zurückgegeben.

Das Ergebnis von **oo::copy** ist der voll qualifizierte Name des neuen Objekts oder der Klasse.

### Beispiele

Dieses Beispiel erzeugt ein Objekt, kopiert es, ändert das Quellobjekt und zeigt, dass das kopierte Objekt tatsächlich eine Kopie ist.

```
oo::object create src
oo::objdefine src method msg {} {puts foo}
oo::copy src dst
oo::objdefine src method msg {} {puts bar}
src msg             → prints "bar"
dst msg             → prints "foo"
```

*Copyright (c) 2007 Donal K. Fellows*

## coroutine

*Stichwörter:* coroutine, generator

### Name

coroutine, yield, yieldto, coroinject, coroprobe - Coroutinen erzeugen und Werte aus ihnen liefern

### Übersicht

**coroutine** *name command* ?*arg...*?

**yield** ?*value*?
**yieldto** *command* ?*arg...*?
*name* ?*value...*?

**coroinject** *coroName command* ?*arg...*?
**coroprobe** *coroName command* ?*arg...*?

### Beschreibung

Der Befehl **coroutine** erzeugt einen neuen Coroutine-Kontext (mit zugehörigem Befehl) namens *name* und führt diesen Kontext aus, indem er *command* aufruft und die übrigen Argumente ohne weitere Interpretation übergibt. Kehrt *command* normal oder mit einer Ausnahme zurück (z. B. einem Fehler), wird der Coroutine-Kontext *name* gelöscht.

Innerhalb des Kontexts können mit **yield** Werte als Ergebnisse erzeugt werden; fehlt *value*, wird die leere Zeichenkette verwendet. Beim Aufruf von **yield** wird die Ausführung des Kontexts ausgesetzt und der Befehl **coroutine** gibt das Argument von **yield** zurück. Die Ausführung des Kontexts kann fortgesetzt werden, indem der Kontextbefehl aufgerufen wird — optional mit dem *einen* Wert, der als Ergebnis des **yield**-Aufrufs gelten soll, der den Kontext ausgesetzt hat. Liefert der Coroutine-Kontext nie **yield** und kehrt stattdessen konventionell zurück, ist das Ergebnis von **coroutine** das Ergebnis der Auswertung des Kontexts.

Die Coroutine kann die Ausführung auch mit **yieldto** aussetzen, der statt zurückzukehren die Ausführung an einen Befehl *command* (im Kontext der Coroutine aufgelöst) übergibt, dem *beliebig viele* Argumente übergeben werden können. Da jede Coroutine einen Kontextbefehl hat, kann **yieldto** die Kontrolle direkt von einer Coroutine zur anderen übergeben (sinnvoll nur, wenn beide Coroutinen das erwarten), aber *jeder* Befehl kann Ziel sein. Wurde eine Coroutine auf diese Weise ausgesetzt, kann die Coroutine-Verarbeitung fortgesetzt werden, indem der Kontextbefehl aufgerufen wird — optional mit beliebig vielen Argumenten. Der Rückgabewert des **yieldto**-Aufrufs ist die Liste der Argumente, die an den Kontextbefehl übergeben wurden; der Aufrufer entscheidet, was damit geschieht.

Die empfohlene Art, eine Variante von **yield** zu schreiben, die die Fortsetzung mit mehreren Argumenten erlaubt, nutzt **yieldto** und den Befehl return, etwa so:

```
proc yieldMultiple {value} {
    tailcall yieldto string cat $value
}
```

Die Coroutine kann auch gelöscht werden, indem der Befehl *name* zerstört wird; der Name der aktuellen Coroutine lässt sich mit info coroutine abfragen. Gibt es Lösch-Spuren auf Variablen in der Coroutine-Implementierung, werden sie ausgelöst, wenn die Coroutine explizit gelöscht wird (oder natürlich, wenn der Befehl konventionell zurückkehrt).

Zum Zeitpunkt des Aufrufs von *command* ist der aktuelle Namespace der globale Namespace, und es gibt keine Stackframes darüber (im Sinne von upvar und uplevel). Welcher Befehl aufgerufen wird, wird jedoch im Namespace bestimmt, aus dem **coroutine** aufgerufen wurde.

Eine ausgesetzte Coroutine (also eine, die yield oder yieldto ausgeführt hat) kann an diesem Punkt mit coroprobe inspiziert oder verändert werden, indem an der aktuellen Position der Coroutine ein Befehl ausgeführt wird. Der Befehl nimmt den Namen der Coroutine coroName, in der ausgeführt werden soll, und den Namen eines Befehls (sowie alle benötigten Argumente), der sofort an dieser Stelle ausgeführt wird. Das Ergebnis dieses Befehls ist das Ergebnis von coroprobe; der Gesamtzustand der Coroutine bleibt danach gleich (die Coroutine erwartet weiterhin die Ergebnisse eines yield oder yieldto wie zuvor), Variablen können sich jedoch geändert haben.

Entsprechend kann **coroinject** einen Befehl in eine ausgesetzte Coroutine einfügen (der bei Fortsetzung ausgeführt wird), um Argumente zu verarbeiten — mit großer Ähnlichkeit zu **coroprobe**. Bei **coroinject** gibt es jedoch mehrere wesentliche Unterschiede:

- Die Coroutine wird nach der Injektion nicht sofort fortgesetzt. Dadurch sind mehrere Injektionen möglich, bevor die Coroutine fortgesetzt wird. Die injizierten Befehle werden in umgekehrter Definitionsreihenfolge ausgeführt (intern auf einem Stack gespeichert).
- Der Liste der auszuführenden Argumente (also command und seinen args) werden zwei weitere Elemente angehängt. Das erste ist der Name des Befehls, der die Coroutine ausgesetzt hat (yield oder yieldto), das zweite das Argument (bzw. bei yieldto die Liste der Argumente), das der aktuelle Fortsetzungswert ist.
- Das Ergebnis des injizierten Befehls wird als Ergebnis des yield oder yieldto verwendet, das die Coroutine ausgesetzt hat. Bei mehreren Injektionen wird das Ergebnis eines zum Fortsetzungswert des nächsten.

Die Injektion ist einmalig. Sie bleibt nach der Ausführung nicht erhalten. Der injizierte Befehl darf im Verlauf **yield** oder **yieldto** ausführen.

Beachten Sie, dass laufende Coroutinen weder untersucht noch injiziert werden können; die Operationen gelten nur für ausgesetzte Coroutinen. (Läuft eine Coroutine, würde

Introspektionscode nur den Zustand der aktuellen Ausführung betrachten; **coroinject**/ **coroprobe** sind dann unnötig.)

## Beispiele

Dieses Beispiel zeigt eine Coroutine, die eine unendliche Folge gerader Werte erzeugt, und eine Schleife, die die ersten zehn davon verbraucht.

```
proc allNumbers {} {
    yield
    set i 0
    while 1 {
        yield $i
        incr i 2
    }
}
coroutine nextNumber allNumbers
for {set i 0} {$i < 10} {incr i} {
    puts "received [nextNumber]"
}
rename nextNumber {}
```

In diesem Beispiel addiert die Coroutine die an sie übergebenen Argumente.

```
coroutine accumulator apply {{} {
    set x 0
    while 1 {
        incr x [yield $x]
    }
}}
for {set i 0} {$i < 10} {incr i} {
    puts "$i -> [accumulator $i]"
}
```

Dieses Beispiel zeigt Coroutinen zur Implementierung des klassischen Sieb des Eratosthenes zur Primzahlsuche. Beachten Sie die Erzeugung von Coroutinen innerhalb einer Coroutine.

```
proc filterByFactor {source n} {
    yield [info coroutine]
    while 1 {
        set x [$source]
        if {$x % $n} {
            yield $x
        }
    }
}
coroutine allNumbers apply {{} {while 1 {yield [incr x]}}}
coroutine eratosthenes apply {c {
    yield
    while 1 {
        set n [$c]
        yield $n
        set c [coroutine prime$n filterByFactor $c $n]
    }
}} allNumbers
for {set i 1} {$i <= 20} {incr i} {
    puts "prime#$i = [eratosthenes]"
}
```

Dieses Beispiel zeigt, wie ein Wert zwischen drei Coroutinen weitergegeben wird, die sich gegenseitig **yieldto** aufrufen:

```
proc juggler {name target {value ""}} {
    if {$value eq ""} {
        set value [yield [info coroutine]]
    }
    while {$value ne ""} {
        puts "$name : $value"
        set value [string range $value 0 end-1]
        lassign [yieldto $target $value] value
    }
}
coroutine j1 juggler Larry [
    coroutine j2 juggler Curly [
        coroutine j3 juggler Moe j1]] "Nyuck!Nyuck!Nyuck!"
```

Dieses Beispiel zeigt eine einfache Coroutine, die nicht-leere Werte sammelt und ohne Argument eine Liste davon zurückgibt. Es zeigt auch, wie man in die Coroutine hineinschauen und die Eingabe einmalig ändern kann.

```
proc collectorImpl {} {
    set me [info coroutine]
    set accumulator {}
    for {set val [yield $me]} {$val ne ""} {set val [yield]} {
        lappend accumulator $val
    }
    return $accumulator
}

coroutine collect collectorImpl
collect 123
collect "abc def"
collect 456

puts [coroprobe collect set accumulator]
# ==> 123 {abc def} 456
```

```
collect "pqr"

coroinject collect apply {{type value} {
    puts "Received '$value' at a $type in [info coroutine]"
    return [string toupper $value]
}}

collect rst
# ==> Received 'rst' at a yield in ::collect
collect xyz

puts [collect]
# ==> 123 {abc def} 456 pqr RST xyz
```

### Detaillierte Semantik

Dieses Beispiel zeigt, dass Coroutinen im globalen Namespace starten und dass die Auflösung von *command* erfolgt, bevor der Coroutine-Stack erzeugt wird.

```
proc report {where level} {
    # Wo wurde der Aufrufer aufgerufen?
    set ns [uplevel 2 {namespace current}]
    yield "made $where $level context=$ns name=[info coroutine]"
}
proc example {} {
    report outer [info level]
}
namespace eval demo {
    proc example {} {
        report inner [info level]
    }
    proc makeExample {} {
        puts "making from [info level]"
        puts [coroutine coroEg example]
    }
    makeExample
}
```

Das erzeugt die unten stehende Ausgabe. Insbesondere ist Stack-Manipulation erfolgt (Vergleich der Ebenen der ersten und zweiten Zeile), und die übergeordnete Ebene in der Coroutine ist der globale Namespace. Coroutine-Namen sind ohne Qualifizierung lokal zum aktuellen Namespace, und Coroutinen können in der Tiefe **yield**en (z. B. in aufgerufenen Prozeduren).

```
making from 2
made inner 1 context=:: name=::demo::coroEg
```

*Copyright (c) 2009 Donal K. Fellows.*

# dde

*Stichwörter:* dde, application, name, remote execution

## Name

dde - Dynamic-Data-Exchange-Befehl ausführen

## Übersicht

**package require dde 1.4**

**dde servername** ?**-force**? ?*-handler proc*? ?**--**? ?*topic*?
**dde execute** ?**-async**? ?**-binary**? *service topic data*
**dde poke** ?**-binary**? *service topic item data*
**dde request** ?**-binary**? *service topic item*
**dde services** *service topic*
**dde eval** ?**-async**? *topic cmd* ?*arg arg* ...?

## Beschreibung

Dieser Befehl lässt eine Anwendung unter Microsoft Windows einen Dynamic-Data-Exchange-Befehl (DDE) senden. Dynamic Data Exchange ist ein Verfahren, mit dem Anwendungen Rohdaten austauschen. Jede DDE-Transaktion braucht einen *service name* und ein *topic*. Beides legt die Anwendung fest. Tcl verwendet den Dienstnamen **TclEval**, der Themenname ist der Name des Interpreters aus **dde servername**. Andere Anwendungen haben eigene Dienstnamen und Themen. Microsoft Excel verwendet zum Beispiel den Dienstnamen **Excel**.

## DDE-Befehle

Die folgenden Befehle sind eine Teilmenge des vollständigen Dynamic-Data-Exchange-Befehlssatzes.

**dde servername ?-force? ?*-handler proc*? ?--? ?*topic*?**

**dde servername** registriert den Interpreter als DDE-Server mit dem Dienstnamen **TclEval** und dem durch *topic* angegebenen Themennamen. Fehlt *topic*, gibt **dde servername** den Namen des aktuellen Themas zurück oder die leere Zeichenkette, wenn der Interpreter nicht als Dienst registriert ist. Ist der angegebene Themenname schon vergeben, wird ein Suffix der Form " #2" oder " #3" angehängt, damit der Name eindeutig ist. Das Ergebnis des Befehls ist der tatsächlich verwendete Name. **-force** erzwingt die Registrierung genau des angegebenen Themennamens.

Die Option **-handler** legt eine Tcl-Prozedur fest, die Aufrufe an den DDE-Server verarbeitet. Wurde das Paket in einen sicheren Interpreter geladen, muss eine **-handler**-Prozedur definiert sein. Die Prozedur erhält alle Argumente des entfernten Aufrufs.

**dde execute ?-async? ?-binary? *service topic data***

**dde execute** nimmt *data* und sendet sie an den durch *service* bezeichneten Server mit dem Thema *topic*. Typischerweise ist *service* der Name einer Anwendung und *topic* eine zu bearbeitende Datei. Das Feld *data* geht an die entfernte Anwendung. Typischerweise behandelt die Anwendung *data* als Skript und führt es aus. **-async** verlangt einen asynchronen Aufruf. Der Befehl gibt eine Fehlermeldung zurück, wenn das Skript nicht lief, außer **-async** ist gesetzt; dann kehrt der Befehl sofort ohne Fehler zurück.

Ohne **-binary** werden alle Daten in Unicode gesendet. DDE-Clients ohne das Zwischenablageformat CF_UNICODE übersetzen das automatisch in die Systemkodierung. Mit **-binary** und dem Ergebnis von encoding convertto lassen sich Daten in jeder anderen Kodierung senden.

**dde poke ?-binary? *service topic item data***

**dde poke** übergibt *data* an den durch *service* bezeichneten Server, mit den angegebenen *topic* und *item*. Typischerweise ist *service* der Name einer Anwendung. *topic* ist anwendungsspezifisch, kann ein Befehl an den Server oder der Name einer zu bearbeitenden Datei sein. *item* ist ebenfalls anwendungsspezifisch und oft unbenutzt, darf aber nie leer sein. Das Feld *data* geht an die entfernte Anwendung.

Ohne **-binary** werden alle Daten in Unicode gesendet. DDE-Clients ohne das Zwischenablageformat CF_UNICODE übersetzen das automatisch in die Systemkodierung. Mit **-binary** und dem Ergebnis von encoding convertto lassen sich Daten in jeder anderen Kodierung senden.

**dde request ?-binary? *service topic item***

**dde request** holt typischerweise den Wert von etwas, etwa den Wert einer Zelle in Microsoft Excel oder den Text einer Auswahl in Microsoft Word. *service* ist typischerweise der Name einer Anwendung, *topic* typischerweise der Dateiname, *item* anwendungsspezifisch. Der Befehl gibt den in der Anwendung definierten Wert von *item* zurück. Normalerweise gilt das als Zeichenkette mit abschließendem Nullbyte. Mit **-binary** kommt das Ergebnis als Bytefeld zurück.

**dde services *service topic***

**dde services** gibt eine Liste der Dienst-Thema-Paare zurück, die auf dem Rechner gerade existieren. Sind *service* und *topic* beide leere Zeichenketten ({}), kommen alle derzeit verfügbaren Paare zurück. Ist *service* leer und *topic* nicht, kommen alle Dienste mit diesem Thema zurück. Ist *service* nicht leer und *topic* leer, kommen alle Themen dieses Dienstes zurück. Sind beide nicht leer und das Paar existiert, wird es zurückgegeben, sonst eine leere Zeichenkette.

**dde eval ?-async? *topic cmd ?arg arg ...?***

**dde eval** wertet einen Befehl und seine Argumente im durch *topic* bezeichneten Interpreter aus. Der DDE-Dienst muss **TclEval** sein. **-async** verlangt einen asynchronen Aufruf. Der Befehl gibt eine Fehlermeldung zurück, wenn das Skript nicht lief, außer **-async** ist gesetzt; dann kehrt der Befehl sofort ohne Fehler zurück. Dieser Befehl kann unter Windows **send** ersetzen.

## DDE und Tcl

Ein Tcl-Interpreter hat immer den Dienstnamen **TclEval**. Jeder Interpreter aller laufenden Tcl-Anwendungen braucht einen eindeutigen Namen aus **dde servername**. Ein Interpreter ist nur dann ein DDE-Thema, wenn **dde servername** den Themennamen gesetzt hat. **dde services TclEval {}** gibt daher eine Liste von Dienst-Thema-Paaren zurück, in der jeder gerade laufende Interpreter ein Thema ist.

Verarbeitet Tcl einen Befehl **dde execute**, läuft die Execute-Datenmenge als Skript in dem Interpreter, den das Thema des Befehls **dde execute** nennt.

Verarbeitet Tcl einen Befehl **dde request**, gibt er den Wert der im DDE-Befehl genannten Variable im Interpreter des DDE-Themas zurück. Tcl reserviert die Variable **$TCLEVAL$EXECUTE$RESULT** für den internen Gebrauch; **dde request** auf diese Variable liefert unvorhersehbare Ergebnisse.

Eine fremde Anwendung, die ein Skript in Tcl ausführen will, soll das Skript sein Ergebnis in einer Variable speichern lassen, **dde execute** ausführen und danach mit **dde request** den Wert der Variable holen.

Bei DDE muss die Ereignisschleife mit update oder vwait geleert werden. Das geschieht bei wish von selbst, außer ein blockierender Befehl wird aufgerufen (etwa exec ohne **&**, das den Prozess in den Hintergrund legt). Wird die Ereignisschleife nicht geleert, können DDE-Befehle hängen, bis sie geleert wird. Das kann eine Verklemmung erzeugen.

### Beispiel

Dies bittet Internet Explorer (der schon laufen muss), eine bestimmte Website zu öffnen:

```
package require dde
dde execute -async iexplore WWW_OpenURL http://www.tcl-lang.org/
```

*Copyright (c) 1997 Sun Microsystems, Inc.*
*Copyright (c) 2001 ActiveState Corporation.*

## define

*Stichwörter:* define, class, definition, method, object, slot

### Name

oo::define, oo::objdefine, oo::Slot - Klassen und Objekte definieren und konfigurieren

### Übersicht

**package require tcl::oo**

**oo::define** *class defScript*
**oo::define** *class subcommand arg* ?*arg* ...?
**oo::objdefine** *object defScript*
**oo::objdefine** *object subcommand arg* ?*arg* ...?

**oo::Slot** *arg...*

### Klassenhierarchie

```
oo::object
   → oo::Slot
```

### Beschreibung

Der Befehl **oo::define** steuert die Konfiguration von Klassen, und der Befehl **oo::objdefine** steuert die Konfiguration von Objekten (einschließlich Klassen als Instanzobjekte). Die Konfiguration wird auf die Entität angewendet, die im Argument *class* bzw. *object* benannt ist. Die Konfiguration einer Klasse aktualisiert auch die Konfiguration aller Unterklassen der Klasse und aller Objekte, die Instanzen dieser Klasse sind oder sie eingemischt haben (modifiziert durch jede instanzspezifische Konfiguration). Die Art der Konfiguration wird entweder durch das

Argument *defScript* oder durch *subcommand* und die folgenden *arg*-Argumente gesteuert; ist Letzteres vorhanden, entspricht es genau dem Fall, dass alle Argumente ab *subcommand* zu einer Liste zusammengefasst und diese Liste als *defScript*-Argument verwendet wird.

Beachten Sie, dass der Konstruktor von oo::class **oo::define** mit dem übergebenen Skriptargument aufruft. Das ist ein bequemer Weg, eine Klasse in einem Schritt zu erzeugen und zu definieren.

## Konfigurieren von Klassen

Die folgenden Befehle werden im *defScript* für **oo::define** unterstützt; jeder kann auch in der *subcommand*-Form verwendet werden:

**classmethod *name* ?*argList bodyScrip*?**

Erzeugt eine Klassenmethode oder (wenn argList und bodyScript weggelassen werden) befördert eine vorhandene Methode des Klassenobjekts zur Klassenmethode. Die Argumente name, argList und bodyScript entsprechen der method-Definition unten.

Klassenmethoden können sowohl auf der Klasse selbst als auch auf Instanzen dieser Klasse aufgerufen werden. Beim Aufruf ist das aktuelle Objekt (siehe die Befehle self und my) die Klasse, auf der sie aufgerufen werden, bzw. die Klasse der Instanz, auf der sie aufgerufen werden — je nachdem, ob der Aufruf auf der Klasse oder einer Instanz der Klasse erfolgt. Wird auf einer Unterklasse oder Instanz der Unterklasse aufgerufen, ist das aktuelle Objekt die Unterklasse.

In einem privaten Definitionskontext sind die Methoden bei Aufruf auf Klassen *nicht* privat, die Methoden bei Aufruf auf Instanzen von Klassen jedoch schon.

**constructor *argList bodyScript***

Erzeugt oder aktualisiert den Konstruktor einer Klasse. Die formalen Argumente des Konstruktors (definiert im gleichen Format wie beim Tcl-Befehl proc) sind *argList*, und der Rumpf des Konstruktors ist *bodyScript*. Wird der Rumpf des Konstruktors ausgewertet, ist der aktuelle Namespace des Konstruktors ein Namespace, der für das zu konstruierende Objekt eindeutig ist. Innerhalb des Konstruktors sollte der Befehl next verwendet werden, um die Konstruktoren der Superklassen aufzurufen. Ist *bodyScript* die leere Zeichenkette, wird der Konstruktor gelöscht.

Klassen müssen keinen Konstruktor definieren. Ist keiner angegeben, wird stattdessen der Konstruktor der Superklasse verwendet.

**destructor *bodyScript***

Erzeugt oder aktualisiert den Destruktor einer Klasse. Destruktoren nehmen keine Argumente entgegen, und der Rumpf des Destruktors ist *bodyScript*. Der Destruktor wird aufgerufen, wenn Objekte der Klasse gelöscht werden; beim Aufruf ist der eindeutige Namespace des Objekts der aktuelle Namespace. Destruktoren sollten den Befehl next verwenden, um die Destruktoren der Superklassen aufzurufen. Beachten Sie, dass Destruktoren nicht in allen Situationen aufgerufen werden (z. B. wenn der Interpreter zerstört wird). Ist *bodyScript* die leere Zeichenkette, wird der Destruktor gelöscht. Beachten Sie, dass Fehler während der Auswertung eines Destruktors *nicht* an den Code zurückgegeben werden, der die Zerstörung eines Objekts verursacht. Stattdessen werden sie an den aktuell definierten bgerror-Handler weitergegeben.

**export *name* ?*name* ...?**

Legt fest, dass jede der benannten Methoden *name* von der definierten Klasse exportiert wird (d. h. außerhalb einer Instanz über den Befehl des Instanzobjekts nutzbar). Beachten Sie, dass die Methoden selbst tatsächlich von einer Superklasse definiert sein können; Exporte von Unterklassen überschreiben die Sichtbarkeit der Superklasse und können wiederum von Instanzen überschrieben werden.

**forward *name cmdName* ?*arg* ...?**

Erzeugt oder aktualisiert eine weitergeleitete Methode namens *name*. Die Methode wird an den Befehl *cmdName* weitergeleitet; zusätzliche Argumente *arg* usw. werden vor den vom Aufrufer der Methode angegebenen Argumenten eingefügt. *cmdName* wird stets nach den Regeln der Namespaces der aufrufenden Objekte aufgelöst, d. h. wenn *cmdName* nicht voll qualifiziert ist, wird der Befehl im Namespace jedes Objekts gesucht — über den Namespace-Pfad der Instanz oder im globalen Namespace. Die Methode wird exportiert, wenn *name* mit einem Kleinbuchstaben beginnt, andernfalls nicht exportiert.

In einem privaten Definitionskontext (siehe den Definitionsbefehl private unten) erzeugt dieser Befehl private weitergeleitete Methoden.

**initialise *script***

...siehe nächstes...

**initialize *script***

Wertet script in einem Kontext aus, der lokale Variablen unterstützt und in dem der aktuelle Namespace der Instanz-Namespace des Klassenobjekts selbst ist. Das ist nützlich zum Einrichten von z. B. klassenweiten Variablen.

**method *name* ?*option*? *argList bodyScript***

Erzeugt oder aktualisiert eine Methode, die als prozedurähnliches Skript implementiert ist. Der Name der Methode ist *name*, die formalen Argumente der Methode (definiert im gleichen Format wie beim Tcl-Befehl proc) sind *argList*, und der Rumpf der Methode ist *bodyScript*. Wird der Rumpf der Methode ausgewertet, ist der aktuelle Namespace der Methode ein Namespace, der für das aktuelle Objekt eindeutig ist. Die Methode wird exportiert, wenn *name* mit einem Kleinbuchstaben beginnt, andernfalls nicht exportiert; dieses Verhalten kann über **export** und **unexport** oder durch Angabe von -export, -private oder -unexport im optionalen Parameter option überschrieben werden.

In einem privaten Definitionskontext (siehe den Definitionsbefehl private unten) oder wenn für option das Flag -private gesetzt ist, erzeugt dieser Befehl private prozedurähnliche Methoden.

**private *cmd arg...***

...siehe nächstes...

**private *script***

Wertet das script (oder die Liste aus Befehl und Argumenten, gegeben durch cmd und args) in einem Kontext aus, in dem die am aktuellen Objekt vorgenommenen Definitionen private Definitionen sind.

Die folgenden Klassendefinitionsbefehle sind von **private** betroffen: **forward**, **method**, **self** und **variable**. Verschachteltes **private** innerhalb von **private** hat keinen kumulativen Effekt;

der innerste Definitionskontext ist lediglich ein privater Definitionskontext. Alle anderen Definitionsbefehle verhalten sich in einem privaten Definitionskontext nicht anders.

**self *subcommand arg …***

…siehe nächstes…

**self *script***

…siehe nächstes…

**self**

Dieser Befehl entspricht dem Aufruf von **oo::objdefine** auf der gerade definierten Klasse (siehe Konfigurieren von Objekten unten für eine Beschreibung der unterstützten Werte von *subcommand*). Er folgt dem gleichen allgemeinen Muster der Argumentverarbeitung wie die Befehle **oo::define** und **oo::objdefine**, und "**oo::define** *cls* self *subcommand …*" wirkt identisch zu "**oo::objdefine** *cls subcommand …*".

Werden überhaupt keine Argumente verwendet, liefert er den Namen der gerade konfigurierten Klasse. In einem privaten Definitionskontext (siehe den Definitionsbefehl private unten) werden die Definitionen am Klassenobjekt ebenfalls in einem privaten Definitionskontext vorgenommen.

**superclass ?*-slotOperation*? ?*className* …?**

Dieser Slot (siehe Slot-Definitionen unten) ermöglicht die Änderung der Superklassen der definierten Klasse. Jedes Argument *className* benennt eine Klasse, die Superklasse der definierten Klasse sein soll. Beachten Sie, dass Objekte nicht von Klassen zu Nicht-Klassen oder umgekehrt geändert werden dürfen, dass eine leere Elternklasse oo::object entspricht und dass die Elternklassen von oo::object und oo::class nicht geändert werden dürfen. Standardmäßig arbeitet dieser Slot durch Ersetzung.

**unexport *name* ?*name* …?**

Legt fest, dass jede der benannten Methoden *name* von der definierten Klasse nicht exportiert wird (d. h. nicht außerhalb der Instanz über den Befehl des Instanzobjekts nutzbar, sondern nur über den in jedem Objektkontext sichtbaren Befehl my). Beachten Sie, dass die Methoden selbst tatsächlich von einer Superklasse definiert sein können; Unexporte von Unterklassen überschreiben die Sichtbarkeit der Superklasse und können von Instanz-Unexporten überschrieben werden.

**variable ?*-slotOperation*? ?*name* …?**

Dieser Slot (siehe Slot-Definitionen unten) sorgt dafür, dass jede der benannten Variablen in den von der definierten Klasse deklarierten Methoden, im Konstruktor und im Destruktor automatisch verfügbar gemacht wird. Jeder Variablenname darf keine Namespace-Trenner enthalten und darf nicht wie ein Array-Zugriff aussehen. Alle Variablen sind tatsächlich im Namespace des Instanzobjekts vorhanden, auf dem die Methode ausgeführt wird. Beachten Sie, dass die von einer Super- oder Unterklasse deklarierten Variablenlisten vollständig disjunkt sind, ebenso wie von Instanzen deklarierte Variablenlisten; die Liste der Variablennamen gilt nur für von dieser Klasse deklarierte Methoden (sowie Konstruktoren und Destruktoren). Standardmäßig arbeitet dieser Slot durch Anhängen.

In einem privaten Definitionskontext (siehe den Definitionsbefehl private unten) verwaltet dieser Slot die Liste der privaten Variablenbindungen für diese Klasse. Bei einer privaten Variablenbindung unterscheidet sich der Name der Variable im Instanzobjekt vom in der

Definition angegebenen Namen; der in der Definition verwendete Name ist der Name, mit dem Sie in den Methoden dieser Klasse auf die Variable zugreifen, und der Name der Variable im Instanz-Namespace hat ein eindeutiges Präfix, das versehentliche Verwendung aus anderen Klassen extrem unwahrscheinlich macht.

## Erweiterte Konfigurationsoptionen für Klassen

Die folgenden Definitionen werden ebenfalls unterstützt, sind in einfachen Programmen aber nicht erforderlich:

**definitionnamespace ?*kind*? *namespaceName***

Ermöglicht die Steuerung, welcher Namespace von den Befehlen oo::define und oo::objdefine zum Nachschlagen der verwendeten Definitionsbefehle genutzt wird. Wird an einem Objekt eine Definitionsoperation ausgeführt, wird die Klasse, deren Instanz es ist (und deren Superklassen und Mixins) konsultiert, welcher Definitions-Namespace zu verwenden ist. oo::define erhält den Klassen-Definitions-Namespace, und ::oo::objdefine den Instanz-Definitions-Namespace; ansonsten ist die Nachschlageoperation identisch.

Setzt den Definitions-Namespace der Art *kind*, bereitgestellt von der aktuellen Klasse, auf *namespaceName*. *namespaceName* muss auf einen existierenden Namespace verweisen oder die leere Zeichenkette sein (damit die aktuelle Klasse keinen solchen Namespace mehr verbunden hat). *kind*, falls angegeben, muss entweder **-class** (Standard) oder **-instance** sein, um festzulegen, ob der Namespace für **oo::define** bzw. **oo::objdefine** gesetzt wird.

Die Klasse oo::object hat ihren Instanz-Namespace auf **::oo::objdefine** festgelegt, und die Klasse oo::class hat ihren Klassen-Namespace auf **::oo::define** festgelegt. Eine Folge davon ist, dass die effektive Nutzung dieses Merkmals für Klassen die Definition einer Metaklasse erfordert.

**deletemethod *name* ?*name* ...?**

Löscht jede der Methoden namens *name* aus einer Klasse. Die Methoden müssen zuvor in dieser Klasse existiert haben. Beeinflusst nicht die Superklassen der Klasse, noch Unterklassen oder Instanzen der Klasse (außer wenn sie eine Aufrufkette durch die geänderte Klasse haben) oder das Klassenobjekt selbst.

**filter ?*-slotOperation*? ?*methodName* ...?**

Dieser Slot (siehe Slot-Definitionen unten) setzt oder aktualisiert die Liste von Methodennamen, die steuern, ob Methodenaufrufe an Instanzen der Klasse durchgeführt werden dürfen und welche Ergebnisse die Methoden liefern. Jedes *methodName* benennt eine einzelne Filtermethode (exportiert oder nicht exportiert); es ist kein Fehler, eine nicht existierende Methode zu benennen, da sie von Unterklassen definiert sein kann. Standardmäßig arbeitet dieser Slot durch Anhängen.

**mixin ?*-slotOperation*? ?*className* ...?**

Dieser Slot (siehe Slot-Definitionen unten) setzt oder aktualisiert die Liste zusätzlicher Klassen, die in alle Instanzen der definierten Klasse eingemischt werden sollen. Jedes Argument *className* benennt eine einzelne einzumischende Klasse. Standardmäßig arbeitet dieser Slot durch Ersetzung.

**renamemethod *fromName toName***

Benennt die in einer Klasse *fromName* genannte Methode in *toName* um. Die Methode muss zuvor in der Klasse existiert haben, und *toName* darf zuvor nicht auf eine Methode in dieser

Klasse verweisen. Beeinflusst nicht die Superklassen der Klasse, noch Unterklassen oder Instanzen der Klasse (außer wenn sie eine Aufrufkette durch die geänderte Klasse haben) oder das Klassenobjekt selbst. Ändert nicht den Exportstatus der Methode; war sie zuvor exportiert, ist sie es danach auch.

## Konfigurieren von Objekten

Die folgenden Befehle werden im *defScript* für **oo::objdefine** unterstützt; jeder kann auch in der *subcommand*-Form verwendet werden:

**export *name* ?*name* ...?**

Legt fest, dass jede der benannten Methoden *name* von dem definierten Objekt exportiert wird (d. h. außerhalb des Objekts über den Befehl des Objekts nutzbar). Beachten Sie, dass die Methoden selbst tatsächlich von einer Klasse oder Superklasse definiert sein können; Objekt-Exporte überschreiben die Klassensichtbarkeit.

**forward *name cmdName* ?*arg* ...?**

Erzeugt oder aktualisiert eine weitergeleitete Objekt-Methode namens *name*. Die Methode wird an den Befehl *cmdName* weitergeleitet; zusätzliche Argumente *arg* usw. werden vor den vom Aufrufer der Methode angegebenen Argumenten eingefügt. Weitergeleitete Methoden sollten mit dem Unterbefehl **method** gelöscht werden. Die Methode wird exportiert, wenn *name* mit einem Kleinbuchstaben beginnt, andernfalls nicht exportiert.

In einem privaten Definitionskontext (siehe den Definitionsbefehl private unten) erzeugt dieser Befehl private weitergeleitete Methoden.

**method *name* ?*option*? *argList bodyScript***

Erzeugt, aktualisiert oder löscht eine Objekt-Methode. Der Name der Methode ist *name*, die formalen Argumente der Methode (definiert im gleichen Format wie beim Tcl-Befehl proc) sind *argList*, und der Rumpf der Methode ist *bodyScript*. Wird der Rumpf der Methode ausgewertet, ist der aktuelle Namespace der Methode ein Namespace, der für das Objekt eindeutig ist. Die Methode wird exportiert, wenn *name* mit einem Kleinbuchstaben beginnt, andernfalls nicht exportiert; dies kann durch Angabe von -export, -private oder -unexport im optionalen Parameter option oder über die Definitionen export und unexport überschrieben werden.

In einem privaten Definitionskontext (siehe den Definitionsbefehl private unten) oder wenn für option das Flag -private gesetzt ist, erzeugt dieser Befehl private prozedurähnliche Methoden.

**mixin ?*-slotOperation*? ?*className* ...?**

Dieser Slot (siehe Slot-Definitionen unten) setzt oder aktualisiert eine pro Objekt geltende Liste zusätzlicher Klassen, die in das Objekt eingemischt werden sollen. Jedes Argument *className* benennt eine einzelne einzumischende Klasse. Standardmäßig arbeitet dieser Slot durch Ersetzung.

**private *cmd arg...***

...siehe nächstes...

**private *script***

Wertet das script (oder die Liste aus Befehl und Argumenten, gegeben durch cmd und args) in einem Kontext aus, in dem die am aktuellen Objekt vorgenommenen Definitionen private Definitionen sind.

Die folgenden Klassendefinitionsbefehle sind von **private** betroffen: **forward**, **method** und **variable**. Verschachteltes **private** innerhalb von **private** hat keinen kumulativen Effekt; der innerste Definitionskontext ist lediglich ein privater Definitionskontext. Alle anderen Definitionsbefehle verhalten sich in einem privaten Definitionskontext nicht anders.

**unexport *name* ?*name* ...?**

Legt fest, dass jede der benannten Methoden *name* von dem definierten Objekt nicht exportiert wird (d. h. nicht außerhalb des Objekts über den Befehl des Objekts nutzbar, sondern nur über den im Objektkontext sichtbaren Befehl my). Beachten Sie, dass die Methoden selbst tatsächlich von einer Klasse definiert sein können; Instanz-Unexporte überschreiben die Klassensichtbarkeit.

**variable ?-*slotOperation*? ?*name* ...?**

Dieser Slot (siehe Slot-Definitionen unten) sorgt dafür, dass jede der benannten Variablen in den von dem definierten Objekt deklarierten Methoden automatisch verfügbar gemacht wird. Jeder Variablenname darf keine Namespace-Trenner enthalten und darf nicht wie ein Array-Zugriff aussehen. Alle Variablen sind tatsächlich im Namespace des Objekts vorhanden, auf dem die Methode ausgeführt wird. Beachten Sie, dass die von den Klassen und Mixins, deren Instanz das Objekt ist, deklarierten Variablenlisten vollständig disjunkt sind; die Liste der Variablennamen gilt nur für von diesem Objekt deklarierte Methoden. Standardmäßig arbeitet dieser Slot durch Anhängen.

In einem privaten Definitionskontext (siehe den Definitionsbefehl private unten) verwaltet dieser Slot die Liste der privaten Variablenbindungen für dieses Objekt. Bei einer privaten Variablenbindung unterscheidet sich der Name der Variable im Instanzobjekt vom in der Definition angegebenen Namen; der in der Definition verwendete Name ist der Name, mit dem Sie in den Methoden dieses Instanzobjekts auf die Variable zugreifen, und der Name der Variable im Instanz-Namespace hat ein eindeutiges Präfix, das versehentliche Verwendung aus Superklassen-Methoden extrem unwahrscheinlich macht.

## Erweiterte Konfigurationsoptionen für Objekte

Die folgenden Definitionen werden ebenfalls unterstützt, sind in einfachen Programmen aber nicht erforderlich:

**class *className***

Ermöglicht die Änderung der Klasse eines Objekts nach der Erstellung. Beachten Sie, dass die Konstruktoren der Klasse dabei nicht aufgerufen werden; das Objekt kann daher in einem inkonsistenten Zustand sein, sofern keine zusätzliche Konfigurationsarbeit erfolgt.

**deletemethod *name* ?*name* ...?**

Löscht jede der Methoden namens *name* aus einem Objekt. Die Methoden müssen zuvor in diesem Objekt existiert haben (z. B. weil sie durch **oo::objdefine method** erzeugt wurden). Beeinflusst nicht die Klassen, deren Instanz das Objekt ist, und entfernt nicht die Sichtbarkeit der von diesen Klassen bereitgestellten Methoden in der Instanz dieser Klasse.

**filter ?-*slotOperation*? ?*methodName* ...?**

Dieser Slot (siehe Slot-Definitionen unten) setzt oder aktualisiert die Liste von Methodennamen, die steuern, ob ein Methodenaufruf an das Objekt durchgeführt werden darf und welche Ergebnisse die Methode liefert. Jedes *methodName* benennt eine einzelne Filtermethode (exportiert oder nicht exportiert); es ist kein Fehler, eine nicht existierende

Methode zu benennen. Beachten Sie, dass die tatsächliche Filterliste auch von den Filtern abhängt, die für Klassen gesetzt sind, deren Instanz das Objekt ist. Standardmäßig arbeitet dieser Slot durch Anhängen.

**renamemethod *fromName toName***

Benennt die in einem Objekt *fromName* genannte Methode in *toName* um. Die Methode muss zuvor im Objekt existiert haben, und *toName* darf zuvor nicht auf eine Methode in diesem Objekt verweisen. Beeinflusst nicht die Klassen, deren Instanz das Objekt ist, und kann in einem Instanzobjekt nicht die von diesen Klassen bereitgestellten Methoden umbenennen (eine per **oo::objdefine forward** weitergeleitete Methode kann jedoch eine gleichwertige Fähigkeit bieten). Ändert nicht den Exportstatus der Methode; war sie zuvor exportiert, ist sie es danach auch.

**self**

Liefert den Namen des gerade konfigurierten Objekts.

## Private Methoden

Hat eine Klasse oder Instanz eine private Methode, kann diese private Methode nur innerhalb von Methoden dieser Klasse bzw. Instanz aufgerufen werden. Andere Aufrufer der Methoden des Objekts können private Methoden nicht aufrufen; es ist, als ob die privaten Methoden nicht existieren. Eine private Methode einer Klasse kann jedoch aus den Methoden der Klasse aufgerufen werden, wenn diese Methoden auf einem anderen Instanzobjekt verwendet werden; das bedeutet, dass eine Klasse sie nutzen kann, um das Verhalten mehrerer Instanzen von sich selbst zu koordinieren, ohne zu beeinträchtigen, wie andere Klassen (insbesondere Unter- oder Superklassen) interagieren. Private Methoden stehen in der Methodenaufrufreihenfolge (wie von self call gemeldet) vor allen eingemischten Klassen.

## Slot-Definitionen

Einige der konfigurierbaren Definitionen einer Klasse oder eines Objekts sind *Slot-Definitionen*. Das bedeutet, die Konfiguration wird durch ein Slot-Objekt implementiert — eine Instanz der Klasse **oo::Slot** —, das eine Liste von Werten (Klassennamen, Variablennamen usw.) verwaltet, die den Inhalt des Slots bilden.

Die Klasse **oo::Slot** definiert sechs Operationen (als Methoden), die am Slot ausgeführt werden können:

***slot* -append ?*member* ...?**

Hängt die angegebenen *member*-Elemente an die Slot-Definition an.

***slot* -appendifnew ?*member* ...?**

Hängt die angegebenen member-Elemente an die Slot-Definition an, sofern sie noch nicht existieren.

***slot* -clear**

Setzt die Slot-Definition auf die leere Liste.

***slot* -prepend ?*member* ...?**

Fügt die angegebenen member-Elemente der Slot-Definition voran.

***slot* -remove ?*member* ...?**

Entfernt die angegebenen member-Elemente aus der Slot-Definition.

***slot* -set ?*member* ...?**

Ersetzt die Slot-Definition durch die angegebenen *member*-Elemente.

Eine Folge davon ist, dass jede Verwendung der Standardoperation eines Slots, bei der das erste *member*-Argument mit einem Bindestrich beginnt, ein Fehler ist. In diesen Fällen sollte eine der obigen Operationen explizit verwendet werden.

Eine Instanz von **oo::Slot** muss nur erzeugt werden, wenn ein eigener Slot definiert wird, der sich wie ein Standard-Slot verhält.

### Slot-Implementierung

Intern definieren Slot-Objekte auch eine Methode **--default-operation**, die an die Standardoperation des Slots weitergeleitet wird (für den Klassen-"**variable**"-Slot also an "my -append"), sowie diese Methoden, die die Implementierungsschnittstelle bereitstellen:

***slot* Get**

Gibt eine Liste zurück, die den aktuellen Inhalt des Slots ist, modifiziert den Slot aber nicht. Diese Methode muss stets aus einem Stack-Frame aufgerufen werden, der durch einen Aufruf von **oo::define** oder **oo::objdefine** erzeugt wurde. Diese Methode *sollte* keinen Fehler zurückgeben, außer wenn sie außerhalb eines Definitionskontexts oder mit falscher Argumentanzahl aufgerufen wird.

Die Elemente der Liste sollten vollständig aufgelöst sein, sofern das für den Slot ein sinnvoller Begriff ist.

***slot* Resolve *slotElement***

Gibt slotElement mit einer Auflösungsoperation angewendet zurück, modifiziert den Slot aber nicht. Bei Slots einfacher Zeichenketten ist dies eine Operation ohne Wirkung, bei Slots von Klassen wird ein Klassenname auf seinen voll qualifizierten Klassennamen abgebildet. Diese Methode muss stets aus einem Stack-Frame aufgerufen werden, der durch einen Aufruf von oo::define oder oo::objdefine erzeugt wurde. Diese Methode sollte keinen Fehler zurückgeben, außer wenn sie außerhalb eines Definitionskontexts oder mit falscher Argumentanzahl aufgerufen wird; nicht auflösbare Argumente sollten unverändert zurückgegeben werden (da nicht alle Slot-Operationen zwingend auflösbare Werte erfordern).

Implementierungen *sollten* in dieser Methode keine Eindeutigkeits- und Sortierungsbeschränkungen erzwingen; das ist Aufgabe der Methode **Set**. Eine Standardimplementierung dieser Methode (sie gibt nur ihr Argument *slotElement* zurück) stellt die Klasse **oo::Slot** bereit.

***slot* Set *elementList***

Setzt den Inhalt des Slots auf die Liste *elementList* und gibt die leere Zeichenkette zurück. Diese Methode muss stets aus einem Stack-Frame aufgerufen werden, der durch einen Aufruf von **oo::define** oder **oo::objdefine** erzeugt wurde. Diese Methode kann einen Fehler zurückgeben, wenn sie die Änderung des Slot-Inhalts ablehnt (z. B. wegen ungültiger Werte), sowie wenn sie außerhalb eines Definitionskontexts oder mit falscher Argumentanzahl aufgerufen wird.

Diese Methode *kann* die Elemente neu sortieren und filtern, wenn das nötig ist, um die zugrunde liegenden Beschränkungen des Slots zu erfüllen. (Slots von Klassen erzwingen z. B. eine Eindeutigkeitsbeschränkung, die jedes Element an der frühestmöglichen Position im Slot platziert.)

Die Implementierung dieser Methoden hängt vom Slot ab (und ist dafür verantwortlich, auf den richtigen Teil der Klassen- oder Objektdefinition zuzugreifen). Slots haben auch einen Unknown-Methoden-Handler, der all diese Teile zusammenführt, und verbergen ihre Methode **destroy**, damit sie nicht versehentlich aufgerufen wird. Es wird *empfohlen*, Benutzeränderungen am Slot-Mechanismus selbst auf die Definition neuer Operationen zu beschränken, deren Namen mit einem Bindestrich beginnen.

Beachten Sie, dass Slot-Instanzen nicht erwartungsgemäß den Speicher für den von ihnen verwalteten Slot enthalten; der befindet sich in oder ist an die verwaltete Klasse bzw. das verwaltete Objekt angehängt. Diese Instanzen sollten eigene Implementierungen der Methoden **Get** und **Set** bereitstellen (optional **Resolve**; standardmäßig eine durchreichende No-Op-Implementierung).

Die meisten Slot-Operationen wenden zunächst Resolve auf ihre Argumentliste an, kombinieren das Ergebnis mit den Ergebnissen der Methode Get und rufen anschließend Set auf. Manche Operationen lassen einen oder beide der ersten Schritte weg; das Weglassen des dritten würde zu einer idempotenten schreibgeschützten Operation führen (der Standardmechanismus zum Lesen von Slots erfolgt über info class und info object).

## Beispiele

Dieses Beispiel zeigt die Verwendung beider Formen der Befehle **oo::define** und **oo::objdefine** (sie funktionieren gleich) und veranschaulicht vier ihrer Unterbefehle.

```
oo::class create c
c create o
oo::define c method foo {} {
    puts "world"
}
oo::objdefine o {
    method bar {} {
        my Foo "hello "
        my foo
    }
    forward Foo ::puts -nonewline
    unexport foo
}
o bar                → prints "hello world"
o foo                → error "unknown method foo"
o Foo Bar            → error "unknown method Foo"
oo::objdefine o renamemethod bar lollipop
o lollipop           → prints "hello world"
```

Dieses Beispiel zeigt, wie zusätzliche Klassen in ein Objekt eingemischt werden können. Es zeigt auch, dass **mixin** ein Slot ist, der Anhängen unterstützt:

```
oo::object create inst
inst m1              → error "unknown method m1"
inst m2              → error "unknown method m2"
```

```
oo::class create A {
    method m1 {} {
        puts "red brick"
    }
}
oo::objdefine inst {
    mixin A
}
inst m1              → prints "red brick"
inst m2              → error "unknown method m2"

oo::class create B {
    method m2 {} {
        puts "blue brick"
    }
}
oo::objdefine inst {
    mixin -append B
}
inst m1              → prints "red brick"
inst m2              → prints "blue brick"
```

Dieses Beispiel zeigt, wie Klassenvariablen erzeugt und verwendet werden. Es ist eine Klasse, die zählt, wie viele Instanzen von sich selbst erzeugt wurden.

```
oo::class create Counted
oo::define Counted {
    initialise {
        variable count 0
    }

    variable number
    constructor {} {
        classvariable count
        set number [incr count]
    }

    method report {} {
        classvariable count
        puts "This is instance $number of $count"
    }
}

set a [Counted new]
set b [Counted new]
$a report
        → This is instance 1 of 2
```

```
set c [Counted new]
$b report
        → This is instance 2 of 3
$c report
        → This is instance 3 of 3
```

Dieses Beispiel zeigt die Verwendung von Klassenmethoden. (Beachten Sie, dass der Konstruktor von oo::class **oo::define** auf der Klasse aufruft.)

```
oo::class create DBTable {
    classmethod find {description} {
        puts "DB: locate row from [self] matching $description"
        return [my new]
    }
    classmethod insert {description} {
        puts "DB: create row in [self] matching $description"
        return [my new]
    }
    method update {description} {
        puts "DB: update row [self] with $description"
    }
    method delete {} {
        puts "DB: delete row [self]"
        my destroy; # Nur das Objekt löschen, nicht die DB-Zeile
    }
}

oo::class create Users {
    superclass DBTable
}
oo::class create Groups {
    superclass DBTable
}

set u1 [Users insert "username=abc"]
        → DB: create row from ::Users matching username=abc
set u2 [Users insert "username=def"]
        → DB: create row from ::Users matching username=def
$u2 update "group=NULL"
        → DB: update row ::oo::Obj124 with group=NULL
$u1 delete
        → DB: delete row ::oo::Obj123
set g [Group find "groupname=webadmins"]
        → DB: locate row ::Group with groupname=webadmins
$g update "emailaddress=admins"
        → DB: update row ::oo::Obj125 with emailaddress=admins
```

Dieses Beispiel zeigt, wie eine benutzerdefinierte Definition für eine Klasse erstellt wird. Beachten Sie, dass es ausdrücklich die Delegation an die vorhandenen Definitionsbefehle über namespace path einschließt.

```
namespace eval myDefinitions {
    # An vorhandene Definitionen delegieren, wo nicht überschrieben
    namespace path ::oo::define

    # Ein benutzerdefinierter Methodentyp
    proc exprmethod {name arguments body} {
        tailcall method $name $arguments [list expr $body]
    }

    # Ein benutzerdefinierter Weg, einen Konstruktor zu bauen
    proc parameters args {
        uplevel 1 [list variable {*}$args]
        set body [join [lmap a $args {
            string map [list VAR $a] {
                set [my varname VAR] [expr {double($VAR)}]
            }
        }] ";"]
        tailcall constructor $args $body
    }
}

# Den Namespace in eine (sehr einfache) Metaklasse zur Nutzung
    einbinden
oo::class create exprclass {
    superclass oo::class
    definitionnamespace myDefinitions
}

# Die benutzerdefinierten Definitionen verwenden
exprclass create quadratic {
    parameters a b c
    exprmethod evaluate {x} {
        ($a * $x**2) + ($b * $x) + $c
    }
}

# Die resultierende Klasse und das Objekt in Aktion zeigen
quadratic create quad 1 2 3
for {set x 0} {$x <= 4} {incr x} {
    puts [format "quad(%d) = %.2f" $x [quad evaluate $x]]
}
        → quad(0) = 3.00
        → quad(1) = 6.00
        → quad(2) = 11.00
        → quad(3) = 18.00
        → quad(4) = 27.00
```

*Copyright (c) 2007-2018 Donal K. Fellows*

# dict

*Stichwörter:* dict, dictionary, create, update, lookup, iterate, filter, map

## Name

dict - Verarbeitet Dictionaries

## Übersicht

**dict** *option arg* ?*arg* ...?

## Beschreibung

Führt je nach *option* eine von mehreren Operationen auf Dictionary-Werten oder Variablen mit Dictionary-Werten aus (siehe Abschnitt Dictionary-Werte). Zulässige *option*s (abkürzbar):

**dict append *dictionaryVariable key* ?*string* ...?**

Hängt die angegebenen Zeichenketten an den Wert an, den *key* im Dictionary in *dictionaryVariable* abbildet, und schreibt das resultierende Dictionary zurück in die Variable. Fehlende Schlüssel gelten als leere Zeichenkette. Gibt das aktualisierte Dictionary zurück. Verweist dictionaryVariable auf ein nicht existierendes Array-Element mit Standardwert, wird der Standardwert vor dem Anhängen als Dictionary-Wert verwendet.

**dict create ?*key value* ...?**

Gibt ein neues Dictionary mit allen als Argumente angegebenen Schlüssel/Wert-Paaren zurück (Schlüssel und Werte abwechselnd).

**dict exists *dictionaryValue key* ?*key* ...?**

Gibt einen booleschen Wert zurück, ob der Schlüssel (oder Pfad durch verschachtelte Dictionaries) in *dictionaryValue* existiert — genau dann, wenn **dict get** auf diesem Pfad erfolgreich wäre.

**dict filter *dictionaryValue filterType arg* ?*arg* ...?**

Nimmt ein Dictionary und gibt ein neues zurück, das nur die passenden Schlüssel/Wert-Paare enthält. Unterstützte *filterType* (abkürzbar):

**dict filter *dictionaryValue* key ?*globPattern* ...?**

Die key-Regel behält Paare, deren Schlüssel einem der Muster entsprechen (Stil von string match).

**dict filter *dictionaryValue* script *{keyVariable valueVariable} script***

Die script-Regel weist Schlüssel und Wert den Variablen zu und wertet *script* aus (boolesches Ergebnis). Das erste Argument nach der Regel ist eine zweielementige Liste. TCL_BREAK beendet die weitere Prüfung; TCL_CONTINUE entspricht "falsch". Paare werden in Einfügereihenfolge getestet.

**dict filter *dictionaryValue* value ?*globPattern* ...?**

Die value-Regel behält Paare, deren Werte einem der Muster entsprechen (string match).

**dict for *{keyVariable valueVariable} dictionaryValue body***

Drei Argumente: zweielementige Liste von Variablennamen, Dictionary-Wert, Skript (wie foreach). Ergebnis: leere Zeichenkette. TCL_BREAK beendet die Iteration sofort erfolgreich; TCL_CONTINUE wie TCL_OK. Iterationsreihenfolge = Einfügereihenfolge der Schlüssel.

**dict get *dictionaryValue ?key …?***

Liest den Wert zu *key* aus *dictionaryValue*. Mehrere *keys*: wie **dict get [dict get $dictVal $key] …** mit den restlichen Schlüsseln — Nachschlagen in verschachtelten Dictionaries. Beispiel, äquivalent:

```
dict get $dict foo bar spong
dict get [dict get [dict get $dict foo] bar] spong
```

Ohne Schlüssel: Liste von Paaren wie array get (Schlüssel, Wert, …).

Fehler, wenn ein Schlüssel fehlt.

**dict getdef *dictionaryValue ?key …? key default***

siehe unten …

**dict getwithdefault *dictionaryValue ?key …? key default***

Wie dict get (mindestens ein key), aber bei fehlendem Schlüssel (oder fehlendem Schlüssel auf dem Pfad) wird default zurückgegeben statt eines Fehlers.

Mindestens ein *key* ist erforderlich. **dict getdef** und **dict getwithdefault** sind Aliase.

**dict incr *dictionaryVariable key ?increment?***

Addiert *increment* (Ganzzahl, Standard 1) zum Wert von *key* in *dictionaryVariable* und schreibt zurück. Fehlende Schlüssel gelten als 0. Fehler, wenn der bestehende Wert keine Ganzzahl ist. Gibt das aktualisierte Dictionary zurück. Array-Standardwert wie bei dict append.

**dict info *dictionaryValue***

Informationen zum Dictionary (für Menschen); Format implementationsabhängig. Bei Hash-Tabellen oft Tcl_HashStats, analog array statistics.

**dict keys *dictionaryValue ?globPattern?***

Liste aller Schlüssel; mit Muster nur passende (string match). Reihenfolge = Einfügereihenfolge.

**dict lappend *dictionaryVariable key ?value …?***

Hängt *value* an die Listen-Variable an, die *key* abbildet; fehlende Schlüssel = leere Liste. Fehler, wenn der Wert keine Liste ist. Gibt das aktualisierte Dictionary zurück. Array-Standardwert wie oben.

**dict map *{keyVariable valueVariable} dictionaryValue body***

Transformation je Mapping; neues Dictionary (wie lmap). Bei normalem TCL_OK (im Gegensatz zu error usw.) landet das Skriptergebnis im Akkumulator unter dem aktuellen *keyVariable* -Inhalt.

break beendet sofort; continue überspringt die aktuelle Iteration ohne Änderung am Akkumulator. Reihenfolge wie **dict for**.

**dict merge *?dictionaryValue …?***

Dictionary mit allen Inhalten der Argumente; bei gleichem Schlüssel gewinnt der Wert aus dem letzten Dictionary mit diesem Schlüssel.

**dict remove *dictionaryValue* ?*key* ...?**

Kopie ohne die genannten Schlüssel; kein Schlüssel nötig; fehlende Schlüssel sind unkritisch.

**dict replace *dictionaryValue* ?*key value* ...?**

Kopie mit geänderten oder neuen Paaren; ohne Paare erlaubt; Schlüssel ohne Wert ist Fehler.

**dict set *dictionaryVariable key* ?*key* ...? *value***

Setzt in *dictionaryVariable* die Abbildung von *key* (bzw. verschachtelter Pfad) auf *value*; erzeugt bei Bedarf verschachtelte Dictionaries. Gibt das aktualisierte Dictionary zurück. Array-Standardwert wie oben.

**dict size *dictionaryValue***

Anzahl der Schlüssel/Wert-Paare.

**dict unset *dictionaryVariable key* ?*key* ...?**

Entfernt die Abbildung für *key* (Pfad durch verschachtelte Dictionaries); mindestens ein *key*; der letzte auf dem Pfad darf fehlen, alle anderen müssen existieren. Gibt das aktualisierte Dictionary zurück. Array-Standardwert wie oben.

**dict update *dictionaryVariable key varName* ?*key varName* ...? *body***

Führt *body* aus, wobei jeder *key* (aus *dictionaryVariable*) der Variable *varName* zugeordnet ist. Fehlender Schlüssel = unset *varName*. Nach *body* (auch bei Fehler oder Ausnahme) werden Änderungen an den *varName*s zurück ins Dictionary geschrieben — außer *dictionaryVariable* wird unlesbar, dann werden Updates verworfen. Ergebnis von **dict update** ist (ohne Fehler) das Ergebnis von *body*. Array-Standardwert wie oben.

Jede *varName* liegt im umschließenden Gültigkeitsbereich; empfohlen nur lokal (proc, Lambda für apply, Methode). Die Variablen existieren nach dem Befehl weiter (sofern nicht unset).

Keine Traces: Änderungen am Dictionary nur beim Beenden von *body*.

**dict values *dictionaryValue* ?*globPattern*?**

Liste aller Werte; mit Muster nur passende Werte (string match). Reihenfolge der Werte = Einfügereihenfolge der zugehörigen Schlüssel.

**dict with *dictionaryVariable* ?*key* ...? *body***

Führt *body* aus; jeder Schlüssel in *dictionaryVariable* wird einer gleichnamigen Variable zugeordnet (ähnlich **dict update**). Mit *key*s: Pfad durch verschachtelte Dictionaries; innerstes Dictionary wird geöffnet. Wie bei **dict update**: unlesbare Variable oder zerstörte Pfadkette verwirft Updates. Ergebnis ist (ohne Fehler) das Ergebnis von *body*. Array-Standardwert wie oben.

Die Variablen liegen im umschließenden Gültigkeitsbereich von **dict with**; empfohlen nur lokal (proc, Lambda für apply, Methode). Die Variablen bleiben nach dem Befehl bestehen (sofern nicht unset).

Keine Traces: Änderungen am Dictionary nur beim Beenden von *body*.

Ist der Inhalt von *dictionaryVariable* beim Beenden von *body* kein Dictionary (z. B. wenn der Name mit einem Schlüssel kollidiert), tritt ein Fehler auf — nicht empfohlen, wenn Schlüssel mit *dictionaryVariable* kollidieren können. Mappt der Schlüssel auf ein Dictionary, wird das innere in das äußere eingefügt; siehe Beispiele.

## Dictionary-Werte

Dictionaries sind Werte mit effizienter, reihenfolgetreuer Abbildung beliebiger Schlüssel auf beliebige Werte; jeder Schlüssel mappt auf genau einen Wert. Textuell entsprechen sie einer Liste mit gerader Elementanzahl (je Mapping zwei Elemente). Erzeugt ein Befehl aus einem Dictionary ein neues (Rückgabe oder Zurückschreiben in die Ausgangsvariable), bleibt die Schlüsselreihenfolge erhalten — gelöschte Schlüssel entfallen, neue kommen ans Ende. Bei doppelten Schlüsseln in der Zeichenkettendarstellung zählt nur der letzte Wert; "apple banana" und "apple carrot apple banana" sind äquivalente Dictionaries (unterschiedliche Darstellung).

Operationen, die ein neues Dictionary aus einem alten ableiten (**dict set**, **dict unset**, ...), erhalten die Schlüsselreihenfolge; neue Schlüssel werden angehängt, entfernte gestrichen.

## Beispiele

Grundlegende Dictionary-Nutzung:

```
# Dictionary: Endungen -> Beschreibungen
set filetypes [dict create .txt "Text File" .tcl "Tcl File"]

# Ergänzen/Aktualisieren
dict set filetypes .tcl "Tcl Script"
dict set filetypes .tm  "Tcl Module"
dict set filetypes .gif "GIF Image"
dict set filetypes .png "PNG Image"

# Einfaches Lesen
set ext ".tcl"
set desc [dict get $filetypes $ext]
puts "$ext is for a $desc"

# Mit Existenztest
foreach filename [glob *] {
    set ext [file extension $filename]
    if {[dict exists $filetypes $ext]} {
        puts "$filename is a [dict get $filetypes $ext]"
    }
}
```

Verschachtelte Dictionaries:

```
# Data for one employee
dict set employeeInfo 12345-A forenames "Joe"
dict set employeeInfo 12345-A surname   "Schmoe"
dict set employeeInfo 12345-A street "147 Short Street"
dict set employeeInfo 12345-A city   "Springfield"
dict set employeeInfo 12345-A phone  "555-1234"
# Data for another employee
dict set employeeInfo 98372-J forenames "Anne"
dict set employeeInfo 98372-J surname   "Other"
dict set employeeInfo 98372-J street "32995 Oakdale Way"
dict set employeeInfo 98372-J city   "Springfield"
dict set employeeInfo 98372-J phone  "555-8765"
# The above data probably ought to come from a database...

# Print out some employee info
set i 0
puts "There are [dict size $employeeInfo] employees"
dict for {id info} $employeeInfo {
    puts "Employee #[incr i]: $id"
    dict with info {
        puts "   Name: $forenames $surname"
        puts "   Address: $street, $city"
        puts "   Telephone: $phone"
    }
}
# Another way to iterate and pick out names...
foreach id [dict keys $employeeInfo] {
    puts "Hello, [dict get $employeeInfo $id forenames]!"
}
```

Lokalisierbare Variante von string toupper:

```
# Set up the basic C locale
set capital [dict create C [dict create]]
foreach c [split {abcdefghijklmnopqrstuvwxyz} ""] {
    dict set capital C $c [string toupper $c]
}

# English locales can luckily share the "C" locale
dict set capital en [dict get $capital C]
dict set capital en_US [dict get $capital C]
dict set capital en_GB [dict get $capital C]

# ... and so on for other supported languages ...

# Now get the mapping for the current locale and use it.
set upperCaseMap [dict get $capital $env(LANG)]
set upperCase [string map $upperCaseMap $string]
```

Detail von **dict with**:

```
proc sumDictionary {varName} {
    upvar 1 $varName vbl
    foreach key [dict keys $vbl] {
        # Manufacture an entry in the subdictionary
        dict set vbl $key total 0
        # Add the values and remove the old
        dict with vbl $key {
            set total [expr {$x + $y + $z}]
            unset x y z
        }
    }
    puts "last total was $total, for key $key"
}

set myDict {
    a {x 1 y 2 z 3}
    b {x 6 y 5 z 4}
}

sumDictionary myDict
#    prints: last total was 15, for key b

puts "dictionary is now \"$myDict\""
#    prints: dictionary is now "a {total 6} b {total 15}"
```

**dict with** bei kollidierendem Schlüssel und Variablennamen:

```
set foo {foo {a b} bar 2 baz 3}
dict with foo {}
puts $foo
#    prints: a b foo {a b} bar 2 baz 3
```

*Copyright (c) 2003 Donal K. Fellows*

# encoding

*Stichwörter:* encoding, unicode

## Name

encoding - Verwaltet Kodierungen

## Übersicht

**encoding** *option* ?*arg arg* ...?

## Einleitung

Zeichenketten in Tcl sind logisch eine Folge von Unicode-Zeichen. Im Speicher werden sie als Bytefolge in einer von mehreren Kodierungen dargestellt: modifiziertes UTF-8 (1 bis 4 Bytes pro Zeichen) oder eine benutzerdefinierte Kodierung, die als 8-Bit-Binärdaten beginnt.

Betriebssystem-Schnittstellen oder Anwendungen können Zeichenketten in anderen Kodierungen wie Shift-JIS erzeugen. Der Befehl **encoding** überbrückt die Lücke zwischen Unicode und diesen anderen Formaten.

## Beschreibung

Führt je nach *option* eine von mehreren kodierungsbezogenen Operationen aus. Zulässige *option* en sind:

**encoding convertfrom ?*encoding*? *data***

siehe unten ...

**encoding convertfrom ?*-profile profile*? ?-failindex var? *encoding data***

Wandelt *data*, das als Binärstring gemäß *encoding* kodiert sein sollte, in eine Tcl-Zeichenkette um. Fehlt *encoding*, wird die aktuelle Systemkodierung verwendet.

Die Option -profile bestimmt das Verhalten des Befehls bei Konvertierungsfehlern. Details siehe Abschnitt Profile unten. Jeder vorzeitige Abbruch der Verarbeitung wegen Fehlern wird durch eine Exception gemeldet, sofern die Option -failindex nicht angegeben ist.

Ist **-failindex** angegeben, wird bei vorzeitigem Abbruch statt einer Exception das Konvertierungsergebnis bis zum Fehlerpunkt als Befehlsergebnis zurückgegeben. Zusätzlich wird der Index des auslösenden Quellbytes in **var** gespeichert. Tritt kein Fehler auf, wird das vollständige Konvertierungsergebnis zurückgegeben und **-1** in **var** gespeichert.

**encoding convertto ?*encoding*? *data***

siehe unten ...

**encoding convertto ?*-profile profile*? ?-failindex var? *encoding data***

Wandelt *string* in die angegebene *encoding* um. Das Ergebnis ist ein Tcl-Binärstring mit der Bytefolge der konvertierten Zeichenkette in der angegebenen Kodierung. Fehlt *encoding*, wird die aktuelle Systemkodierung verwendet.

Die Optionen -profile und -failindex wirken wie beim Befehl encoding convertfrom beschrieben.

**encoding dirs ?*directoryList*?**

Tcl kann Kodierungsdatendateien vom Dateisystem laden, die zusätzliche Kodierungen beschreiben. Dieser Befehl setzt den Suchpfad für ***.enc**-Kodierungsdatendateien auf die Verzeichnisliste *directoryList*. Fehlt *directoryList*, gibt der Befehl die aktuelle Suchpfadliste zurück. *directoryList* muss eine gültige Liste sein, sonst Fehler. Beim Suchen einer Kodierungsdatendatei werden Einträge in *directoryList*, die nicht auf ein lesbares, durchsuchbares Verzeichnis verweisen, ignoriert.

**encoding names**

Gibt eine Liste mit den Namen aller derzeit verfügbaren Kodierungen zurück. Die Kodierungen "utf-8" und "iso8859-1" sind in der Liste garantiert enthalten.

**encoding profiles**

Gibt eine Liste der Namen von Kodierungsprofilen zurück. Siehe Profile unten.

**encoding system ?*encoding*?**

Setzt die Systemkodierung auf *encoding*. Fehlt *encoding*, gibt der Befehl die aktuelle Systemkodierung zurück. Die Systemkodierung wird verwendet, wenn Tcl Zeichenketten an Systemaufrufe übergibt.

**encoding user**

Gibt den Namen der Kodierung gemäß den Benutzereinstellungen zurück. Unter Windows basiert dies auf den Codepage-Einstellungen des Benutzers in der Registry. Auf anderen Plattformen entspricht der Wert dem von encoding system.

## Profile

Bei Kodierungstransformationen können verschiedene Fehler auftreten, etwa ungültige Sequenzen in den Quelldaten oder Zeichen, die in der Zielkodierung nicht darstellbar sind. Ein profile legt die Strategie zum Umgang mit solchen Fehlern fest — auf eine von zwei Arten:

- Weitere Verarbeitung der Quelldaten beenden. Das Profil bestimmt nicht, wie dieser vorzeitige Abbruch dem Aufrufer mitgeteilt wird. Standardmäßig erfolgt dies durch eine Exception. Ist die Option -failindex angegeben, werden Fehler über diesen Mechanismus gemeldet.
- Weitere Verarbeitung der Quelldaten mit einer Fallback-Strategie fortsetzen, z. B. Ersetzen oder Verwerfen der fehlerhaften Bytes profilabhängig.

Folgende Profile sind derzeit implementiert; Standard ist **strict**, wenn **-profile** nicht angegeben ist.

**strict**

Das Profil strict stoppt die Verarbeitung stets bei einem Konvertierungsfehler. Der Fehler wird per Exception oder über -failindex gemeldet. strict entspricht dem Unicode-Standard.

**tcl8**

Das Profil tcl8 folgt stets der ersten Strategie oben und entspricht dem Verhalten von Kodierungstransformationen in Tcl 8.6. Beim Konvertieren von einer externen Kodierung außer utf-8 in Tcl-Zeichenketten mit encoding convertfrom werden ungültige Bytes ihren numerisch äquivalenten Codepunkten zugeordnet. Das Byte 0x80, in ASCII ungültig, wird z. B. auf Codepunkt U+0080 abgebildet. Beim Konvertieren von utf-8 werden ungültige Bytes, die in CP1252 definiert sind, ihren Unicode-Äquivalenten zugeordnet, andere fallen auf numerische Äquivalente zurück. Byte 0x80 ist in CP1252 definiert und wird daher U+20AC zugeordnet; Byte 0x81 ist in CP1252 nicht definiert und wird U+0081 zugeordnet. Als Sonderfall wird die Sequenz 0xC0 0x80 auf U+0000 abgebildet. Beim Konvertieren von Tcl-Zeichenketten in ein externes Format mit encoding convertto werden nicht darstellbare Zeichen durch ein kodierungsabhängiges Zeichen ersetzt, meist ?.

**replace**

Wie tcl8 setzt replace bei Konvertierungsfehlern die Verarbeitung fort, verwendet aber eine Unicode-konforme Ersetzungsmethode für ungültige Quelldaten. Bei encoding convertfrom werden ungültige Bytes durch den Codepunkt U+FFFD REPLACEMENT CHARACTER ersetzt. Bei encoding convertto werden nicht darstellbare Codepunkte in ein kodierungsspezifisches Ersatzzeichen umgewandelt: U+FFFD REPLACEMENT CHARACTER für UTF-Ziele, sonst meist ?.

## Beispiele

Diese Beispiele verwenden die unten stehende Hilfsprozedur, die die Unicode-Codepunkte einer Tcl-Zeichenkette ausgibt.

```
proc codepoints s {join [lmap c [split $s {}] {
    string cat U+ [format %.6X [scan $c %c]]}]
}
```

Beispiel 1: Bytefolge in japanischer euc-jp-Kodierung in eine Tcl-Zeichenkette umwandeln:

```
% codepoints [encoding convertfrom euc-jp "\xA4\xCF"]
U+00306F
```

Das Ergebnis ist der Unicode-Codepunkt "\u306F", der Hiragana-Buchstabe HA.

Beispiel 2: Fehlerbehandlung anhand von Profilen:

Der Buchstabe **A** ist das Unicode-Zeichen U+0041, das Byte "\x80" ist in ASCII-Kodierung ungültig.

```
% codepoints [encoding convertfrom -profile tcl8 ascii A\x80]
U+000041 U+000080
% codepoints [encoding convertfrom -profile replace ascii A\x80]
U+000041 U+00FFFD
% codepoints [encoding convertfrom -profile strict ascii A\x80]
unexpected byte sequence starting at index 1: '\x80'
```

Beispiel 3: Teilweise Daten und Fehlerposition:

```
% codepoints [encoding convertfrom -profile strict -failindex idx
    ascii AB\x80]
U+000041 U+000042
% set idx
2
```

Beispiel 4: Zeichen kodieren, das in ISO8859-1 nicht darstellbar ist:

```
% encoding convertto iso8859-1 A\u0141
A?
% encoding convertto -profile strict iso8859-1 A\u0141
unexpected character at index 1: 'U+000141'
% encoding convertto -profile strict -failindex idx iso8859-1 A\u0141
A
% set idx
1
```

*Copyright (c) 1998 Scriptics Corporation.*

# eof

*Stichwörter:* eof

## Name

eof - Prüft auf Dateiende-Bedingung eines Kanals

## Übersicht

**eof** *channel*

## Beschreibung

Der Befehl **eof** wurde durch den Befehl chan eof ersetzt, der die gleiche Syntax und die gleichen Optionen unterstützt.

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# error

*Stichwörter:* error, exception

## Name

error - Löst einen Fehler aus

## Übersicht

**error** *message* ?*info*? ?*code*?

## Beschreibung

Gibt den Rückgabecode TCL_ERROR zurück und bricht die Befehlsauswertung ab. *message* ist eine Zeichenkette für die Anwendung, die den Fehler beschreibt.

Die Rückgabeoption **-errorinfo** des Interpreters sammelt einen Stack-Trace des laufenden Kontexts; beim Unwinden verschachtelter Befehle ergänzt der Interpreter **-errorinfo**. Ist *info* angegeben, initialisiert es **-errorinfo**; der erste Unwind-Schritt des Interpreters entfällt — der Befehl mit **error** erscheint nicht im Stack-Trace, stattdessen steht dort *info*. Historisch besonders mit catch: lässt sich ein gefangener Fehler nicht beheben, kann *info* den Stack-Trace am ursprünglichen Fehlerort wiederherstellen:

```
catch {...} errMsg
set savedInfo $::errorInfo
...
error $errMsg $savedInfo
```

Ab Tcl 8.5 stattdessen:

```
catch {...} errMsg options
...
return -options $options $errMsg
```

Ist *code* angegeben, wird sein Wert in **-errorcode** gespeichert — maschinenlesbare Fehlerbeschreibung; Format siehe return.

## Beispiele

Fehler, wenn eine einfache Rechenoperation scheitert:

```
if {1+2 != 3} {
    error "something is very wrong with addition"
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# eval

*Stichwörter:* eval, concatenate, evaluate, script

## Name

eval - Wertet ein Tcl-Skript aus

## Übersicht

**eval** *arg* ?*arg* ...?

## Beschreibung

**eval** nimmt ein oder mehrere Argumente, die zusammen ein Tcl-Skript mit einem oder mehreren Befehlen bilden. **eval** verkettet alle Argumente wie concat, übergibt die Zeichenkette rekursiv an den Tcl-Interpreter und gibt das Auswertungsergebnis (oder einen dabei erzeugten Fehler) zurück.

Der Befehl list quotiert Wortfolgen so, dass **eval** sie nicht weiter aufsplittet; für beliebige Werte $a, $b und $c sind diese Zeilen wirkungsgleich:

```
eval [list $a $b $c]
$a $b $c
```

## Beispiele

Oft speichert man ein Skriptfragment in einer Variable und führt es später mit angehängten Werten aus (z. B. in fcopy, lsort, trace-Callbacks):

```
set script {
    puts "logging now"
    lappend $myCurrentLogVar
}
set myCurrentLogVar log1
# Set up a switch of logging variable part way through!
after 20000 set myCurrentLogVar log2

for {set i 0} {$i<10} {incr i} {
    # Introduce a random delay
    after [expr {int(5000 * rand())}]
    update    ;# Check for the asynch log switch
    eval $script $i [clock clicks]
}
```

Ist das Fragment nur ein Befehlspräfix (Liste von Wörtern), ist **{*}$script** meist robuster als **eval**. Eine Prozedur analog zu lappend, die Argumente am Listenanfang einfügt:

```
proc lprepend {varName args} {
    upvar 1 $varName var
    # Ensure that the variable exists and contains a list
    lappend var
    # Now we insert all the arguments in one go
    set var [eval [list linsert $var 0] $args]
}
```

Heute üblicher ohne **eval**:

```
set var [linsert $var 0 {*}$args]
```

oder:

```
set var [list {*}$args {*}$var]
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# exec

*Stichwörter:* exec, execute, pipeline, redirection, subprocess

## Name

exec - Startet Unterprozesse

## Übersicht

**exec** ?*switches*? *arg* ?*arg* ...? ?**&**?

## Beschreibung

Dieser Befehl behandelt seine Argumente als Spezifikation eines oder mehrerer auszuführender Unterprozesse. Die Argumente haben die Form einer Standard-Shell-Pipeline, bei der jedes *arg* ein Wort eines Befehls wird und jeder eigenständige Befehl ein Unterprozess wird. Das Ergebnis des Befehls ist die Standardausgabe des letzten Unterprozesses in der Pipeline, dekodiert gemäß der Option **-encoding**.

Beginnen die anfänglichen Argumente von **exec** mit **-**, werden sie als Befehlszeilenschalter behandelt und gehören nicht zur Pipeline-Spezifikation. Derzeit werden folgende Schalter unterstützt:

**-encoding** ***encodingName***

Legt den Namen der Kodierung fest, mit der das Ergebnis des Befehls dekodiert wird, wenn er nicht im Hintergrund läuft. Standard ist die von encoding system zurückgegebene Kodierung.

**-ignorestderr**

Verhindert, dass der Befehl **exec** die Ausgabe von Meldungen auf dem Standardfehlerkanal der Pipeline als Fehlerfall behandelt.

**-keepnewline**

Behält ein abschließendes Zeilenende in der Pipeline-Ausgabe bei. Normalerweise wird ein abschließendes Zeilenende entfernt.

**--**

Markiert das Ende der Schalter. Das folgende Argument wird als erstes *arg* behandelt, auch wenn es mit **-** beginnt.

Hat ein *arg* (oder ein Paar von *arg*s) eine der unten beschriebenen Formen, verwendet **exec** es zur Steuerung des Ein- und Ausgabeflusses zwischen den Unterprozessen. Solche Argumente werden nicht an die Unterprozesse übergeben. Bei Formen wie "**<** *fileName*" kann *fileName* entweder in einem separaten Argument von "**<**" stehen oder im selben Argument ohne Leerzeichen dazwischen (d. h. "**<***fileName*").

**|**

Trennt eigenständige Befehle in der Pipeline. Die Standardausgabe des vorhergehenden Befehls wird in die Standardeingabe des nächsten Befehls geleitet.

**|&**

Trennt eigenständige Befehle in der Pipeline. Sowohl Standardausgabe als auch Standardfehler des vorhergehenden Befehls werden in die Standardeingabe des nächsten Befehls geleitet. Diese Umleitungsform setzt Formen wie 2> und >& außer Kraft.

**<** ***fileName***

Die von *fileName* benannte Datei wird geöffnet und als Standardeingabe für den ersten Befehl in der Pipeline verwendet.

**<@** ***fileId***

*FileId* muss der Bezeichner einer geöffneten Datei sein, etwa der Rückgabewert eines früheren Aufrufs von open. Er wird als Standardeingabe für den ersten Befehl in der Pipeline verwendet. *FileId* muss zum Lesen geöffnet worden sein.

**<< *value***

*Value* wird mit der Systemkodierung kodiert, Zeilenenden durch plattformspezifische Zeilenendesequenzen ersetzt und dann dem ersten Befehl als Standardeingabe übergeben.

**> *fileName***

Die Standardausgabe des letzten Befehls wird in die von *fileName* benannte Datei umgeleitet und überschreibt deren bisherigen Inhalt.

**2> *fileName***

Der Standardfehler aller Befehle in der Pipeline wird in die von *fileName* benannte Datei umgeleitet und überschreibt deren bisherigen Inhalt.

**>& *fileName***

Sowohl die Standardausgabe des letzten Befehls als auch der Standardfehler aller Befehle werden in die von *fileName* benannte Datei umgeleitet und überschreiben deren bisherigen Inhalt.

**>> *fileName***

Die Standardausgabe des letzten Befehls wird in die von *fileName* benannte Datei umgeleitet und an diese angehängt, statt sie zu überschreiben.

**2>> *fileName***

Der Standardfehler aller Befehle in der Pipeline wird in die von *fileName* benannte Datei umgeleitet und an diese angehängt, statt sie zu überschreiben.

**>>& *fileName***

Sowohl die Standardausgabe des letzten Befehls als auch der Standardfehler aller Befehle werden in die von *fileName* benannte Datei umgeleitet und an diese angehängt, statt sie zu überschreiben.

**>@ *fileId***

*FileId* muss der Bezeichner einer geöffneten Datei sein, etwa der Rückgabewert eines früheren Aufrufs von open. Die Standardausgabe des letzten Befehls wird in die Datei von *fileId* umgeleitet, die zum Schreiben geöffnet worden sein muss.

**2>@ *fileId***

*FileId* muss der Bezeichner einer geöffneten Datei sein, etwa der Rückgabewert eines früheren Aufrufs von open. Der Standardfehler aller Befehle in der Pipeline wird in die Datei von *fileId* umgeleitet. Die Datei muss zum Schreiben geöffnet worden sein.

**2>@1**

Der Standardfehler aller Befehle in der Pipeline wird in das Befehlsergebnis umgeleitet. Dieser Operator ist nur am Ende der Befehlspipeline gültig.

**>&@ *fileId***

*FileId* muss der Bezeichner einer geöffneten Datei sein, etwa der Rückgabewert eines früheren Aufrufs von open. Sowohl die Standardausgabe des letzten Befehls als auch der Standardfehler aller Befehle werden in die Datei von *fileId* umgeleitet. Die Datei muss zum Schreiben geöffnet worden sein.

Wurde die Standardausgabe nicht umgeleitet, gibt der Befehl **exec** die Standardausgabe des letzten Befehls in der Pipeline zurück, sofern nicht "2>@1" angegeben wurde — in diesem Fall ist auch der Standardfehler enthalten. Beendet ein Befehl in der Pipeline abnormal, wird beendet oder angehalten, gibt **exec** einen Fehler zurück; die Fehlermeldung enthält die Ausgabe der Pipeline, gefolgt von Fehlermeldungen zu den abnormalen Beendigungen; die Rückgabeoption **-errorcode** enthält zusätzliche Informationen zur zuletzt aufgetretenen abnormalen Beendigung. Schreibt ein Befehl auf seine Standardfehlerdatei und ist der Standardfehler nicht umgeleitet und **-ignorestderr** nicht angegeben, gibt **exec** einen Fehler zurück; die Fehlermeldung enthält die Standardausgabe der Pipeline, gefolgt von Meldungen zu abnormalen Beendigungen (falls vorhanden), gefolgt von der Standardfehlerausgabe.

Ist das letzte Zeichen des Ergebnisses oder der Fehlermeldung ein Zeilenende, wird dieses Zeichen normalerweise aus dem Ergebnis oder der Fehlermeldung entfernt. Das entspricht anderen Tcl-Rückgabewerten, die normalerweise nicht mit Zeilenenden enden. Ist jedoch **-keepnewline** angegeben, bleibt das abschließende Zeilenende erhalten.

Wird die Standardeingabe nicht mit "<", "<<" oder "<@" umgeleitet, stammt die Standardeingabe für den ersten Befehl in der Pipeline von der aktuellen Standardeingabe der Anwendung.

Ist das letzte *arg* "&", wird die Pipeline im Hintergrund ausgeführt. In diesem Fall gibt **exec** eine Liste zurück, deren Elemente die Prozesskennungen aller Unterprozesse in der Pipeline sind. Die Standardausgabe des letzten Befehls in der Pipeline geht auf die Standardausgabe der Anwendung, sofern nicht umgeleitet; die Fehlerausgabe aller Befehle in der Pipeline geht auf die Standardfehlerdatei der Anwendung, sofern nicht umgeleitet.

Das erste Wort jedes Befehls gilt als Befehlsname; enthält das Ergebnis keinen Schrägstrich, werden die Verzeichnisse in der PATH-Umgebungsvariable nach einem ausführbaren Programm mit diesem Namen durchsucht. Enthält der Name einen Schrägstrich, muss er auf ein aus der aktuellen Verzeichnisstruktur erreichbares ausführbares Programm verweisen. Auf die Befehlsargumente werden keine "glob"-Erweiterungen oder andere shell-ähnliche Substitutionen angewendet.

## Portabilitätsfragen

### Windows (alle Versionen)

Lesen von oder Schreiben auf einen Socket mit der Notation "**@** *fileId*" funktioniert nicht. Beim Lesen von einem Socket hängt eine 16-Bit-DOS-Anwendung, eine 32-Bit-Anwendung kehrt sofort mit Dateiende zurück. Schreibt eine der beiden Anwendungstypen auf einen Socket, werden die Informationen stattdessen an die Konsole gesendet, falls vorhanden, oder verworfen.

Beachten Sie, dass die derzeitige Escape- bzw. Quoting-Behandlung von Argumenten unter Windows nur mit ausführbaren Programmen funktioniert, die CommandLineToArgv, die CRT-Bibliothek oder Ähnliches verwenden, sowie mit Windows-Batchdateien (außer dem Zeilenende, siehe unten). Obwohl es der übliche Escape-Algorithmus ist, ist in Wirklichkeit entscheidend, wie das ausführbare Programm die Befehlszeile parst (bzw. in einzelne Argumente aufteilt).

Leider gibt es derzeit keine Möglichkeit, ein Zeilenendezeichen innerhalb eines Arguments an Batchdateien (**.cmd** oder **.bat**) oder an den Befehlsprozessor (**cmd.exe /c**) zu übergeben, weil die Befehlszeile (auch die Argumentkette) beim ersten Zeilenendezeichen abgeschnitten wird. Mit einem ausführbaren Programm (CommandLineToArgv usw.) funktioniert es jedoch korrekt.

**Argument-Quoting**

Die Argumente des Befehls **exec** werden den Argumenten des aufgerufenen Programms zugeordnet. Zusätzliche Anführungszeichen (**"**) werden bei Bedarf automatisch um Argumente gesetzt. Sonderzeichen werden durch Einfügen von Backslash-Zeichen escaped.

Die MS-Windows-Umgebung führt in den Argumenten genannte Programme aus, und aufgerufene Batchdateien (conspec) ersetzen Umgebungsvariablen, was Nebenwirkungen (Sicherheitslücken) haben oder bestehendes Quoting zerstören kann (z. B. wenn die Umgebungsvariable ein Sonderzeichen wie **"** enthält). Beispiele:

```
% exec my-echo.cmd {test&whoami}
  test
  mylogin
% exec my-echo.cmd "ENV X:%X%"
  ENV X: CONTENT OF X
```

Folgende Formatierung wird automatisch auf jedes Argumentelement angewendet, um die Ausführung von Unterprogrammen zu vermeiden: Jedes Argument, das ein Sonderzeichen (**&**, **|**, **^**, **<**, **>**,**!**, **(**,**)**, **(**, **%**) enthält, wird automatisch in Anführungszeichen (**"**) gesetzt. Jedes Daten-Anführungszeichen wird durch Einfügen von Backslash-Zeichen escaped.

Die automatische Auflösung von Umgebungsvariablen mit "**%var%**" ist kritisch, hat aber mehr Nutzen als Gefahr und wird nicht escaped.

Das Tk-Konsolen-Text-Widget bietet keine echten Standard-E/A-Fähigkeiten. Unter Tk sehen alle Anwendungen bei Umleitung von der Standardeingabe sofort Dateiende; auf Standardausgabe oder Standardfehler umgeleitete Informationen werden verworfen.

Sowohl vorwärts als auch rückwärts gerichtete Schrägstriche werden als Pfadtrenner in Argumenten von Tcl-Befehlen akzeptiert. Beim Ausführen einer Anwendung kann der angegebene Pfadname ebenfalls vorwärts oder rückwärts gerichtete Schrägstriche als Pfadtrenner enthalten. Beachten Sie jedoch, dass die meisten Windows-Anwendungen Argumente mit vorwärts gerichteten Schrägstrichen nur als Optionstrenner und rückwärts gerichtete Schrägstriche nur in Pfaden akzeptieren. Argumente mit vorwärts gerichteten Schrägstrichen als Pfadtrenner werden nicht automatisch in Backslashes umgewandelt. Ob ein solches Argument als Pfadname erkannt wird, hängt vom Programm ab.

Zwei oder mehr aufeinanderfolgende vorwärts oder rückwärts gerichtete Schrägstriche in einem Pfad bezeichnen einen Netzwerkpfad. Eine einfache Verkettung des Stammverzeichnisses **c:/** mit einem Unterverzeichnis **/windows/system** ergibt z. B. **c://windows/system** (zwei Schrägstriche zusammen), was auf den Einhängepunkt **system** auf dem Rechner **windows** verweist (und **c:/** ignoriert wird), und ist nicht gleichwertig mit **c:/windows/system**, das ein Verzeichnis auf dem aktuellen Rechner beschreibt. Zum Verketten von Pfadkomponenten sollte file join verwendet werden.

Es gibt zwei allgemeine Typen von Win32-Konsolenanwendungen:
1. CLI — CommandLine Interface, einfacher stdio-Austausch. Beispiel: **netstat.exe**.
2. TUI — Textmode User Interface, jede Anwendung, die die Konsolen-API für Cursorbewegung,

Textfarbe, Tastendrücke, Mausbewegung usw. nutzt. Beispiel: **telnet.exe** unter Windows 2000. Solche Anwendungen sind unter Windows selten, existieren aber.

**exec** funktioniert mit TUI-Anwendungen schlecht, wenn keine Konsole vorhanden ist, wie beim Start unter wish. Konsolenanwendungen sollen verborgen und abgetrennt laufen. Das ist eine bewusste Einschränkung, weil **exec** über Pipes kommunizieren will. Die Expect-Erweiterung adressiert dieses Problem bei der Kommunikation mit TUI-Anwendungen.

Beim Versuch, eine Anwendung auszuführen, sucht **exec** zuerst nach dem angegebenen Namen. Anschließend werden der Reihe nach **.com**, **.exe**, **.bat** und **.cmd** an den angegebenen Namen angehängt und nach dem längeren Namen gesucht. War kein Verzeichnisname Teil des Anwendungsnamens, werden folgende Verzeichnisse der Reihe nach automatisch durchsucht:

- Das Verzeichnis, aus dem die Tcl-Ausführungsdatei geladen wurde.
- Das aktuelle Verzeichnis.
- Das Windows-32-Bit-Systemverzeichnis.
- Das Windows-Home-Verzeichnis.
- Die in PATH aufgeführten Verzeichnisse.

Um Shell-eingebaute Befehle wie **dir** und **copy** auszuführen, muss der Aufrufer den gewünschten Befehl mit "**cmd.exe /c** " voranstellen, weil eingebaute Befehle nicht als ausführbare Programme implementiert sind.

**Unix (einschließlich macOS)**

Der Befehl **exec** ist voll funktionsfähig und arbeitet wie beschrieben.

## Unix-Beispiele

Hier einige Beispiele für **exec** unter Unix. Ein einfaches Programm ausführen und das Ergebnis erhalten:

```
exec uname -a
```

### Mit von Null verschiedenen Ergebnissen arbeiten

Um ein Programm auszuführen, das einen von Null verschiedenen Rückgabewert liefern kann, sollte der Aufruf von **exec** in catch eingebettet und bei Fehler der Inhalt der Rückgabeoption **-errorcode** geprüft werden:

```
set status 0
if {[catch {exec grep foo bar.txt} results options]} {
    set details [dict get $options -errorcode]
    if {[lindex $details 0] eq "CHILDSTATUS"} {
        set status [lindex $details 2]
    } else {
        # Anderer Fehler; Fehler an Aufrufer weitergeben
        return -options $options -level 0 $results
    }
}
```

Einfacher schreibt sich das mit try, weil sich bestimmte Fehlertypen leichter abfangen lassen:

```
try {
    set results [exec grep foo bar.txt]
    set status 0
} trap CHILDSTATUS {results options} {
    set status [lindex [dict get $options -errorcode] 2]
}
```

### Mit in Anführungszeichen gesetzten Argumenten arbeiten

Beim Übersetzen eines Befehls aus einer Unix-Shell-Aufrufzeile ist zu beachten, dass einfache Anführungszeichen in Tcl keine besondere Bedeutung haben. Also:

```
awk '{sum += $1} END {print sum}' numbers.list
```

wird etwa so übersetzt:

```
exec awk {{sum += $1} END {print sum}} numbers.list
```

### Mit Globbing arbeiten

Bei Aufrufen mit Shell-Globbing ist zu bedenken, dass Tcl standardmäßig kein Globbing durchführt und Dinge nicht in mehrere Argumente expandiert. Stattdessen schreibt man z. B.:

```
exec ls -l {*}[glob *.tcl]
```

### Mit vom Benutzer gelieferten Shell-Skriptfragmenten arbeiten

Eine nützliche Technik ist, Benutzern eines Skripts ein Shell-Skriptfragment ausführen zu lassen, dem auf der Standardeingabe vom Tcl-Programm erzeugte Daten übergeben werden. Das ist bei *lpr* zum Drucken üblich. Am einfachsten übergibt man das Benutzerskript der Shell des Benutzers zur Verarbeitung — das vermeidet viel Komplexität beim Parsen anderer Sprachen.

```
set lprScript [get from user...]
set postscriptData [generate somehow...]

exec $env(SHELL) -c $lprScript << $postscriptData
```

## Windows-Beispiele

Hier einige Beispiele für **exec** unter Windows. Eine *notepad*-Instanz starten, die eine Datei bearbeitet, ohne auf das Ende der Bearbeitung zu warten:

```
exec notepad myfile.txt &
```

Eine Textdatei mit *notepad* drucken:

```
exec notepad /p myfile.txt
```

### Mit Konsolenprogrammen arbeiten

Ruft ein Programm andere Programme auf, wie bei Compilern üblich, kann es nötig sein, Batchdateien zu verwenden, um Konsolenfenster zu verbergen, die manchmal aufpoppen:

```
exec cmp.bat somefile.c -o somefile
```

Mit einer *cmp.bat* etwa wie:

```
@gcc %*
```

oder einer Variante mit einzelnen Parametern:

```
@gcc %1 %2 %3 %4 %5 %6 %7 %8 %9
```

### Mit eingebauten Befehlen arbeiten

Manchmal ist Vorsicht nötig: Verschiedene Programme können denselben Namen haben und in PATH liegen. Dann kann derselbe Befehl an der DOS-Eingabeaufforderung *ein anderes Programm* finden als über **exec**. Das liegt an den (dokumentierten) Verhaltensunterschieden zwischen **exec** und DOS-Batchdateien.

Im Zweifel auto_execok verwenden: Es liefert den vollständigen Pfad zum Programm, wie **exec** ihn sieht. Das gilt besonders für "interne" Befehle wie *dir* aus einem Tcl-Skript (zum bloßen Auflisten von Dateinamen glob verwenden). So geht es:

```
exec {*}[auto_execok dir] *.tcl
```

### Mit nativen Dateinamen arbeiten

Viele Windows-Programme verlangen Dateinamen mit Backslashes als Pfadtrenner. Dazu dient file nativename. Um ein Verzeichnis (auf NTFS) zu verschlüsseln, sodass nur der aktuelle Benutzer darauf zugreifen kann, braucht man *CIPHER*, etwa so:

```
set secureDir [file join [file home] Desktop/SecureDirectory]
file mkdir $secureDir
exec CIPHER /e /s:[file nativename $secureDir]
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2006 Donal K. Fellows.*

## exit

*Stichwörter:* exit, abort, process

### Name

exit - Beendet die Anwendung

### Übersicht

**exit** ?*returnCode*?

### Beschreibung

Beendet den Prozess und übergibt *returnCode* dem System als Exit-Status. Ist *returnCode* nicht angegeben, ist der Standardwert 0.

### Beispiele

Da der aufrufende Prozess Exit-Codes ungleich null meist als Fehler interpretiert, ist **exit** wichtig, um schwerwiegende Probleme zu signalisieren. Dieses Codefragment eignet sich in Skripten als allgemeine Fehlerfalle:

```
proc main {} {
    # ... den eigentlichen Hauptcode hier einfügen ...
}

if {[catch {main} msg options]} {
    puts stderr "unexpected script error: $msg"
    if {[info exists env(DEBUG)]} {
        puts stderr "---- BEGIN TRACE ----"
        puts stderr [dict get $options -errorinfo]
        puts stderr "---- END TRACE ----"
    }

    # Code 1 für "erwartete" Fehler-Exits reservieren...
    exit 2
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# expr

*Stichwörter:* expr, arithmetic, boolean, compare, expression, fuzzy comparison, integer value

## Name

expr - Einen Ausdruck auswerten

## Übersicht

**expr** *arg* ?*arg arg* ...?

## Beschreibung

Fügt *arg*s durch Leerzeichen getrennt zu einem Ausdruck zusammen, wertet diesen aus und gibt seinen Wert zurück. In einem Ausdruck sind eine Teilmenge der in C-Ausdrücken erlaubten Operatoren zulässig. Für Operatoren, die sowohl in Tcl als auch in C vorkommen, gelten dieselbe Bedeutung und dieselbe Priorität wie bei den entsprechenden C-Operatoren. Der Wert eines Ausdrucks ist oft ein numerisches Ergebnis, entweder eine Ganzzahl oder ein Gleitkommawert, kann aber auch ein nicht-numerischer Wert sein. Beispielsweise liefert der Ausdruck

```
expr 8.2 + 6
```

den Wert 14.2. Ausdrücke unterscheiden sich von C-Ausdrücken in der Art, wie Operanden angegeben werden. Ausdrücke unterstützen außerdem nicht-numerische Operanden, Zeichenkettenvergleiche und einige zusätzliche Operatoren, die in C nicht vorkommen.

Ist das Ergebnis eines Ausdrucks eine Ganzzahl, liegt es in Dezimalform vor; ist es eine Gleitkommazahl, in der Form, die der **%g**-Konvertierungsspezifizierer von format erzeugt.

An jeder Stelle im Ausdruck außer innerhalb von doppelten Anführungszeichen oder geschweiften Klammern markiert # den Beginn eines Kommentars, der bis zum Zeilenende oder bis zum Ende des Ausdrucks reicht, je nachdem, was zuerst eintritt.

## Operanden

Ein Ausdruck besteht aus einer Kombination von Operanden, Operatoren, Klammern und Kommas, ggf. mit Leerraum zwischen diesen Elementen, der ignoriert wird. Jeder Operand wird nach Möglichkeit als numerischer Wert interpretiert.

Jeder Operand hat eine der folgenden Formen:

**Numerischer Wert**

Ganzzahl oder Gleitkommazahl. Die ersten zwei Zeichen einer Ganzzahl können auch **0d** für dezimal, **0b** für binär, **0o** für oktal oder **0x** für hexadezimal sein.

Eine Gleitkommazahl kann in mehreren gängigen Dezimalformaten angegeben werden und darf den Dezimalpunkt**.**, **e** oder **E** für die wissenschaftliche Notation sowie die Vorzeichen **+** und **-** verwenden. Folgende Werte sind gültige Gleitkommazahlen: 2.1, 3., 6e4, 7.91e+16. Die Zeichenketten **Inf** und **NaN** in beliebiger Groß-/Kleinschreibung werden ebenfalls als Gleitkommawerte erkannt. Ein Operand ohne numerische Interpretation muss in geschweifte Klammern oder doppelte Anführungszeichen gesetzt werden.

Ziffern in einem numerischen Wert dürfen durch ein oder mehrere Unterstriche "**_**" getrennt sein. Ein Trennzeichen darf nur zwischen Ziffern stehen, nicht am Anfang eines numerischen Werts, nicht zwischen der führenden 0 und dem Radix-Präfix und nicht am Ende. Beispiele:

```
expr 100_000_000        100000000
expr 0xffff_ffff        4294967295
format 0x%x 0b1111_1110_1101_1011       0xfedb
expr 3_141_592_653_589e-1_2     3.141592653589
```

**Boolescher Wert**

In jeder von string is **boolean** verstandenen Form.

**Variable**

Mit der üblichen **$**-Notation. Der Wert der Variable ist der Wert des Operanden.

**Zeichenkette in doppelten Anführungszeichen**

Backslash-, Variablen- und Befehlssubstitution erfolgen gemäß den Regeln für Tcl.

**Zeichenkette in geschweiften Klammern**

Der Operand wird als Wert in geschweiften Klammern gemäß der Klammerregel in Tcl behandelt.

**Tcl-Befehl in eckigen Klammern**

Befehlssubstitution erfolgt gemäß der Substitutionsregel für Befehle in Tcl.

**Funktionsaufruf**

Mathematische Funktion wie **sin($x)**, deren Argumente eine der obigen Operandenformen haben können. Siehe Mathematische Funktionen unten.

Da **expr** Werte parst und Substitutionen vornimmt, die Tcl bereits geparst und substituiert hat, empfiehlt es sich in der Regel, Ausdrücke in geschweifte Klammern zu setzen, damit Tcl die erste Substitutionsrunde vermeidet.

Unten einige einfache Ausdrücke, wenn **a** den Wert 3 und **b** den Wert 6 hat. Der Befehl links erzeugt den Wert rechts.

```
expr {3.1 + $a} 6.1
expr {2 + "$a.$b"}  5.6
expr {4*[llength {6 2}]}    8
expr {{word one} < "word $a"}   0
```

## Operatoren

Bei Operatoren mit numerischem und Zeichenkettenmodus wird der numerische Modus gewählt, wenn alle Operanden numerisch interpretierbar sind. Die Ganzzahl-Interpretation hat Vorrang vor der Gleitkomma-Interpretation. Für Zeichenkettenoperationen auf beliebigen Werten empfiehlt es sich in der Regel, **eq**, **ne** oder den Befehl string statt vielseitigerer Operatoren wie **==** zu verwenden.

Sofern nicht anders angegeben, akzeptieren Operatoren auch nicht-numerische Operanden. Der Wert einer booleschen Operation ist 1 bei wahr, sonst 0. Siehe auch string is **boolean**. Die gültigen Operatoren, von denen die meisten auch als Befehle im Namespace **tcl::mathop** verfügbar sind (siehe mathop(n)), sind unten in abnehmender Priorität aufgeführt:

**- + ~!**

Unäres Minus, unäres Plus, bitweises NOT, logisches NOT. Diese Operatoren dürfen nur auf numerische Operanden angewendet werden; bitweises NOT nur auf Ganzzahlen.

`**`

Potenzierung. Gültig für numerische Operanden. Der maximale Exponent, den Tcl verarbeiten kann, wenn die erste Zahl eine Ganzzahl > 1 ist, beträgt 268435455.

*** / %**

Multiplikation und Division (numerische Operanden) sowie Rest (Ganzzahlen). Der Rest hat einen absoluten Betrag kleiner als der des Divisors und dasselbe Vorzeichen wie der Divisor.

Bei Ganzzahlen kann man Division und Rest als Aufteilung der Zahlengeraden in aneinandergrenzende Intervalle der Größe des Divisors auffassen; der Quotient gibt an, in welchem Intervall der Dividend liegt, der Rest, wo innerhalb dieses Intervalls. Folglich ist das Ergebnis von "-57 **/** 10" immer -6 und das von "-57 **%** 10" immer 3.

**+ -**

Addition und Subtraktion. Gültig für numerische Operanden.

**<< >>**

Links- und Rechtsshift. Gültig für Ganzzahlen. Ein Rechtsshift überträgt stets das Vorzeichenbit.

**< > <= >=**

Boolesche, numerisch bevorzugte Vergleiche: kleiner, größer, kleiner gleich, größer gleich. Ist ein Argument nicht numerisch, erfolgt der Vergleich als UNICODE-Zeichenkettenvergleich wie bei den unten genannten Zeichenkettenoperatoren gleicher Priorität.

**lt gt le ge**

Boolesche Zeichenkettenvergleiche: kleiner, größer, kleiner gleich, größer gleich. Vergleichen Werte stets als UNICODE-Zeichenketten (siehe auch string compare), im Gegensatz zu den obigen numerisch bevorzugten Vergleichen gleicher Priorität.

**==!=**

Boolesches gleich und ungleich.

**eq ne**
Boolesches Zeichenketten-gleich und Zeichenketten-ungleich.

**in ni**
Listenelement und negiertes Listenelement. Das erste Argument wird als Zeichenkette, das zweite als Liste interpretiert. **in** prüft die Zugehörigkeit zur Liste, **ni** ist die Negation.

**&**
Bitweises AND. Gültig für Ganzzahl-Operanden.

**^**
Bitweises exklusives OR. Gültig für Ganzzahl-Operanden.

**|**
Bitweises OR. Gültig für Ganzzahl-Operanden.

**&&**
Logisches AND. Sind beide Operanden wahr, ist das Ergebnis 1, sonst 0. Dieser Operator wertet faul aus; er wertet den zweiten Operanden nur aus, wenn das für das Ergebnis nötig ist.

**||**
Logisches OR. Sind beide Operanden falsch, ist das Ergebnis 0, sonst 1. Dieser Operator wertet faul aus; er wertet den zweiten Operanden nur aus, wenn das für das Ergebnis nötig ist.

***x*? *y*: *z***
Wenn-dann-sonst wie in C. Ist *x* wahr, ist das Ergebnis der Wert von *y*, sonst der von *z*. Dieser Operator wertet faul aus; er wertet nur *y* oder *z* aus.

Der Potenzierungsoperator fördert Typen wie Multiplikation und Division; das Ergebnis entspricht **pow**. Potenzierung gruppiert innerhalb einer Prioritätsstufe von rechts nach links, andere binäre Operatoren von links nach rechts. Beispielsweise ist der Wert von

```
expr {4*2 < 7}
```

0, der von

```
expr {2**3**2}
```

512.

Wie in C werten **&&**, **||** und**?:** "faul" aus: Operanden werden nicht ausgewertet, wenn das für das Ergebnis nicht nötig ist. In

```
expr {$v?[a]:[b]}
```

wird nur **[a]** oder **[b]** ausgewertet, je nach Wert von **$v**. Das gilt nicht für den normalen Tcl-Parser; daher empfiehlt es sich, die Argumente von **expr** in geschweifte Klammern zu setzen. Ohne Klammern, wie in **expr** $v ? [a]: [b], werden **[a]** und **[b]** ausgewertet, bevor **expr** aufgerufen wird.

Einzelheiten zu den Ergebnissen der Operatoren siehe die C-Dokumentation.

## Mathematische Funktionen

Eine mathematische Funktion wie **sin($x)** wird durch einen Aufruf eines gewöhnlichen Tcl-Befehls im Namespace **::tcl::mathfunc** ersetzt. Die Auswertung von

```
expr {sin($x+$y)}
```

entspricht in jeder Hinsicht der Auswertung von

```
expr {[::tcl::mathfunc::sin [expr {$x+$y}]]}
```

was wiederum gleichbedeutend ist mit der Auswertung von

```
::tcl::mathfunc::sin [expr {$x+$y}]
```

::tcl::mathfunc::sin wird wie in **NAMESPACE-AUFLÖSUNG** in namespace(n) beschrieben aufgelöst. Bei Standardwert von namespace path sind **[namespace current]::tcl::mathfunc::sin** oder ::tcl::mathfunc::sin typische Auflösungen.

Wie in C kann eine mathematische Funktion mehrere durch Kommas getrennte Argumente haben. So wird

```
expr {hypot($x,$y)}
```

zu

```
::tcl::mathfunc::hypot $x $y
```

Verfügbare Standard-Mathefunktionen siehe mathfunc(n).

### Typen, Überlauf und Genauigkeit

Interne Gleitkomma-Berechnungen nutzen den C-Typ *double*. Beim Umwandeln einer Zeichenkette in Gleitkomma wird Exponentenüberlauf erkannt und liefert den *double*-Wert **Inf** bzw. **-Inf**. Gleitkomma-Über- und Unterlauf werden soweit von der Hardware unterstützt erkannt, was in der Regel zuverlässig ist.

Die Umwandlung zwischen internen Darstellungen für Ganzzahl, Gleitkomma und Zeichenkette erfolgt bei Bedarf automatisch. Bei arithmetischen Berechnungen werden Ganzzahlen verwendet, bis eine Gleitkommazahl ins Spiel kommt; danach Gleitkomma. Beispielsweise liefert

```
expr {5 / 4}
```

1, während

```
expr {5 / 4.0}
expr {5 / ( [string length "abcd"] + 0.0 )}
```

beide 1.25 ergeben. Ein Gleitkomma-Ergebnis erkennt man am "**.**" oder "**e**".

Beispielsweise liefert

```
expr {20.0/5.0}
```

**4.0**, nicht **4**.

## Leistungshinweise

Enthält ein Ausdruck Syntax, die Tcl sonst substituieren würde, erlauben geschweifte Klammern oder sonstige Anführung als statischer Wert dem Tcl-Compiler, Bytecode für den Ausdruck zu erzeugen — bessere Geschwindigkeit, geringerer Speicherbedarf. Das vermeidet auch Probleme, wenn Tcl die Substitution vor dem Aufruf von **expr** ausführt.

Im folgenden Beispiel ist der Ausdruckswert 11, weil der Tcl-Parser zuerst **$b** und **expr** dann **$a** beim Auswerten von "$a + 2*4" substituiert. Klammern um den Ausdruck führten zu einem Syntaxfehler, da **$b** keinen numerischen Wert liefert.

```
set a 3
set b {$a + 2}
expr $b*4
```

Wird ein Ausdruck zur Laufzeit erzeugt (wie oben), muss der Bytecode-Compiler bei jeder Auswertung neuen Code erzeugen — der aufwändigste Fall. Dann sind die in mathfunc(n) oder mathop(n) beschriebenen Befehle oft die bessere Wahl als **expr**.

Die meisten Ausdrücke entstehen nicht zur Laufzeit, sondern sind Literale oder enthalten Substitutionen ohne weitere Substitutionen. Damit der Bytecode-Compiler zur Kompilierzeit mit einem Zeichenkettenliteral arbeiten kann, darf der Ausdruck keine Substitutionen enthalten oder muss in Klammern stehen, damit **expr** statt Tcl substituiert.

Muss ein nicht-konstanter Ausdruck in einen sonst konstanten Ausdruck, ist ein rekursives **expr** für den variablen Teil am effizientesten: Der äußere Teil kann kompiliert werden, der variable Teil wird separat ausgewertet. Im Beispiel ist das Ergebnis 20; der äußere Ausdruck profitiert von vollständig gecachtem Bytecode.

```
set a 3
set b {$a + 2}
expr {[expr $b] * 4}
```

Setzen Sie Ausdrücke nach Möglichkeit in geschweifte Klammern; wo das nicht geht, sollte das **expr**-Argument möglichst einfach woanders definiert sein. Oft sind Befehle aus **tcl::mathop** effizienter und sicherer als komplexe Ausdrucksgenerierung.

## Beispiele

Numerischer Vergleich mit Ergebnis 1:

```
expr {"0x03" > "2"}
```

Zeichenkettenvergleich mit Ergebnis 1:

```
expr {"0y" > "0x12"}
```

Erzwungener Zeichenkettenvergleich mit Ergebnis 0:

```
expr {"0x03" gt "2"}
```

Prozedur für eine "interessante" mathematische Funktion:

```
proc ::tcl::mathfunc::calc {x y} {
    expr { ($x**2 - $y**2) / exp($x**2 + $y**2) }
}
```

Polarkoordinaten in kartesische umrechnen:

```
# Umrechnung von ($radius,$angle)
set x [expr { $radius * cos($angle) }]
set y [expr { $radius * sin($angle) }]
```

Kartesische in Polarkoordinaten umrechnen:

```
# Umrechnung von ($x,$y)
set radius [expr { hypot($y, $x) }]
set angle  [expr { atan2($y, $x) }]
```

Meldung über die Beziehung zweier Zeichenkettenwerte:

```
puts "a and b are [expr {$a eq $b ? {equal} : {different}}]"
```

Variable setzen, ob eine Umgebungsvariable existiert und den Wert "wahr" hat:

```
set isTrue [expr {
    # Existiert die Umgebungsvariable, und...
    [info exists ::env(SOME_ENV_VAR)] &&
    # ...enthält sie einen gültigen Wahrheitswert?
    [string is true -strict $::env(SOME_ENV_VAR)]
}]
```

Zufällige Ganzzahl im Bereich 0..99 einschließlich:

```
set randNum [expr { int(100 * rand()) }]
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-2000 Sun Microsystems, Inc.*
*Copyright (c) 2005 Kevin B. Kenny* kennykb@acm.org*. All rights reserved*

# fblocked

*Stichwörter:* fblocked

## Name

fblocked - Prüft, ob die letzte Eingabeoperation alle verfügbaren Eingabedaten verbraucht hat

## Übersicht

**fblocked** *channel*

## Beschreibung

Der Befehl **fblocked** wurde durch den Befehl chan blocked abgelöst, der dieselbe Syntax und dieselben Optionen unterstützt.

*Copyright (c) 1996 Sun Microsystems, Inc.*

# fconfigure

*Stichwörter:* fconfigure

## Name

fconfigure - Legt Kanaloptionen fest und gibt sie zurück

## Übersicht

**fconfigure** *channel*
**fconfigure** *channel name*

**fconfigure** *channel name value* ?*name value* ...?

## Beschreibung

Der Befehl **fconfigure** wurde durch den Befehl chan configure abgelöst, der dieselbe Syntax und dieselben Optionen unterstützt.

*Copyright (c) 1995-1996 Sun Microsystems, Inc.*

# fcopy

*Stichwörter:* fcopy, blocking, channel, end of line, end of file, nonblocking, read, translation

## Name

fcopy - Daten von einem Kanal in einen anderen kopieren

## Übersicht

**fcopy** *inputChan outputChan* ?*-size size*? ?*-command callback*?

## Beschreibung

Der Befehl **fcopy** kopiert Daten von einem Ein-/Ausgabekanal *inchan* auf einen anderen Ein-/Ausgabekanal *outchan*. **fcopy** nutzt die Pufferung des Tcl-Ein-/Ausgabesystems, damit keine zusätzlichen Kopien entstehen und beim Kopieren großer Dateien auf Ziele wie Netzwerksockets nicht zu viele Daten im Hauptspeicher liegen.

### Datenmenge

Alle Daten bis *EOF* werden kopiert. Zusätzlich kann die Menge der kopierten Daten mit der Option **-size** angegeben werden. Die angegebene Größe gilt in Bytes, wenn der Eingabekanal im Binärmodus ist, sonst in Zeichen.

Veraltetes Verhalten: die Übertragung gilt als binär, wenn das Kodierungsprofil "tcl8" ist und die Eingabekodierung zur Ausgabekodierung passt. Kodierungsfehler werden dann nicht behandelt. Eine angegebene Größe gilt in diesem Fall in Bytes.

### Blockierender Betrieb

Ohne die Option **-command** blockiert **fcopy**, bis die Kopie fertig ist, und gibt die Anzahl der nach *outchan* geschriebenen Bytes oder Zeichen zurück (dieselben Regeln wie bei **-size**).

### Betrieb im Hintergrund

Das Argument **-command** lässt **fcopy** im Hintergrund arbeiten. Der Befehl kehrt dann sofort zurück, und *callback* wird später aufgerufen, wenn die Kopie fertig ist. *callback* erhält ein oder zwei zusätzliche Argumente, die angeben, wie viele Bytes nach *outchan* geschrieben wurden. Trat beim Kopieren im Hintergrund ein Fehler auf, ist das zweite Argument die zum Fehler gehörende Fehlermeldung. Bei einer Hintergrundkopie müssen *inchan* und *outchan* nicht in den nicht-blockierenden Modus gesetzt werden; **fcopy** erledigt das. Die Ereignisschleife muss aber mit vwait oder mit Tk betreten werden.

Während eines **fcopy** im Hintergrund sind andere Eingabeoperationen auf *inchan* und Ausgabeoperationen auf *outchan* nicht zulässig. Die Gegenrichtung ist zulässig, wie das bidirektionale Beispiel unten zeigt.

Wird *inchan* oder *outchan* geschlossen, während die Kopie läuft, stoppt die aktuelle Kopie, und der Rückruf wird *nicht* ausgeführt. Wird *inchan* geschlossen, werden alle für *outchan* schon eingereihten Daten noch geschrieben.

*inchan* kann während einer Hintergrundkopie lesbar werden. fileevent-Handler sollten während einer Hintergrundkopie abgeschaltet werden, damit sie die Kopie nicht stören. Ein-/Ausgabe auf der falschen Seite (durch einen fileevent-Handler oder sonst) ergibt den Fehler "channel busy".

### Zeilenende-Übersetzung

**fcopy** übersetzt Zeilenende-Folgen in *inchan* und *outchan* gemäß der Option **-translation** dieser Kanäle. Einzelheiten stehen in der Handbuchseite fconfigure. Durch die Übersetzung kann die Zahl der von *inchan* gelesenen Bytes von der Zahl der nach *outchan* geschriebenen Bytes abweichen. Gemeldet wird nur die nach *outchan* geschriebene Bytezahl, entweder als Rückgabewert eines synchronen **fcopy** oder als Argument des Rückrufs eines asynchronen **fcopy**.

### Kanalkodierung

**fcopy** beachtet die für die Kanäle eingestellten Kodierungen, Profile und Zeichenübersetzungen. Eingehende Zeichen werden aus der Kodierung des Eingabekanals umgewandelt und dann in der Kodierung des Ausgabekanals kodiert. Einzelheiten zu **-encoding** und **-profile** stehen in der Handbuchseite fconfigure. Stehen beide Kanäle auf der Kodierung "binary" und passen die Übersetzungen zusammen, findet keine Umwandlung statt.

**fcopy** kann Kodierungsfehler auslösen (Fehlercode **EILSEQ**), wenn Eingabe- oder Ausgabekanal das Profil "strict" hat.

Entsteht der Kodierungsfehler auf dem Eingabekanal, werden alle Daten vor dem fehlerhaften Byte auf den Ausgabekanal geschrieben. Der Dateizeiger der Eingabe steht unmittelbar vor den Werten, die den Kodierungsfehler verursachen. Der Fehler lässt sich prüfen oder beheben, indem die Kodierungsparameter geändert und ein Dateibefehl aufgerufen wird (read, **fcopy**).

Entsteht der Kodierungsfehler auf dem Ausgabekanal, sind die fehlerhaften Daten verloren. Eingabe- und Ausgabefehler unterscheiden sich nur in der Fehlermeldung (read oder write); beide lösen den Fehlercode *EILSEQ* aus.

## Beispiele

Das erste Beispiel überträgt den Inhalt eines Kanals exakt auf einen anderen. Beim Kopieren einer Datei auf eine andere ist file copy besser geeignet, da dabei auch Dateimetadaten (z. B. Zugriffsrechte) kopiert werden, wo möglich.

```
fconfigure $in -translation binary
fconfigure $out -translation binary
fcopy $in $out
```

Das zweite Beispiel zeigt, wie der Rückruf die Zahl der übertragenen Bytes erhält. Es verwendet vwait, damit die Anwendung in die Ereignisschleife geht. Dieses vereinfachte Beispiel käme auch ohne den Befehlsrückruf aus.

```
proc Cleanup {in out bytes {error {}}} {
    global total
    set total $bytes
    close $in
    close $out
    if {[string length $error] != 0} {
        # Fehler während der Kopie
    }
}
set in [open $file1]
set out [socket $server $port]
fcopy $in $out -command [list Cleanup $in $out]
vwait total
```

Das dritte Beispiel kopiert in Stücken und prüft im Befehlsrückruf auf Dateiende.

```
proc CopyMore {in out chunk bytes {error {}}} {
    global total done
    incr total $bytes
    if {([string length $error] != 0) || [eof $in]} {
        set done $total
        close $in
        close $out
    } else {
        fcopy $in $out -size $chunk \
                -command [list CopyMore $in $out $chunk]
    }
}
set in [open $file1]
set out [socket $server $port]
set chunk 1024
set total 0
fcopy $in $out -size $chunk \
        -command [list CopyMore $in $out $chunk]
vwait done
```

Das vierte Beispiel startet ein asynchrones, bidirektionales fcopy zwischen zwei Sockets. Das können auch Pipes aus zwei [open "|hal 9000" r+] sein (ihr Gespräch bliebe dem Skript verborgen, weil alle vier fileevent-Plätze belegt sind).

```
set flows 2
proc Done {dir args} {
     global flows done
     puts "$dir is over."
     incr flows -1
     if {$flows<=0} {set done 1}
}
fcopy $sok1 $sok2 -command [list Done UP]
fcopy $sok2 $sok1 -command [list Done DOWN]
vwait done
```

*Copyright (c) 1993 The Regents of the University of California.*

*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# file

*Stichwörter:* file, attributes, copy files, delete files, directory, move files, name, rename files, stat , user

## Name

file - Manipuliert Dateinamen und Dateiattribute

## Übersicht

**file** *option name* ?*arg arg* ...?

## Beschreibung

Dieser Befehl stellt mehrere Operationen am Namen oder den Attributen einer Datei bereit. Das Argument *name* ist in den meisten Fällen der Name einer Datei. Das Argument *option* gibt an, was mit dem Dateinamen geschehen soll. Jede eindeutige Abkürzung für *option* ist zulässig. Die gültigen Optionen sind:

**file atime *name* ?*time*?**

Gibt eine Dezimalzeichenkette zurück, die den Zeitpunkt angibt, zu dem auf die Datei *name* zuletzt zugegriffen wurde. Ist *time* angegeben, handelt es sich um eine Zugriffszeit, die für die Datei gesetzt werden soll. Die Zeit wird in der üblichen POSIX-Weise als Sekunden seit einem festen Startzeitpunkt (oft der 1. Januar 1970) gemessen. Existiert die Datei nicht oder kann ihre Zugriffszeit weder abgefragt noch gesetzt werden, wird ein Fehler erzeugt. Unter Windows unterstützen FAT-Dateisysteme keine Zugriffszeit. Auf zipfs-Dateisystemen wird die Zugriffszeit der Änderungszeit zugeordnet.

**file attributes *name***

siehe unten ...

**file attributes *name* ?*option*?**

siehe unten ...

**file attributes *name* ?*option value option value...*?**

Dieser Unterbefehl gibt plattformspezifische Werte zurück, die mit einer Datei verknüpft sind, oder setzt sie. Die erste Form gibt eine Liste der plattformspezifischen Optionen und ihrer Werte zurück. Die zweite Form gibt den Wert für die angegebene Option zurück. Die dritte Form setzt einen oder mehrere der Werte. Die Werte sind wie folgt:

Unter Unix liest oder setzt **-group** den Gruppennamen der Datei. Dem Befehl kann eine Gruppen-ID übergeben werden; zurückgegeben wird jedoch ein Gruppenname. **-owner** liest oder setzt den Benutzernamen des Dateibesitzers. Der Befehl gibt den Besitzernamen zurück; beim Setzen des Besitzers kann die numerische ID übergeben werden. **-permissions** liest oder setzt die Zugriffsrechte einer Datei, standardmäßig in Oktalnotation. Diese Option unterstützt außerdem eingeschränkt das Setzen von Rechten mit der vom Befehl **chmod** akzeptierten symbolischen Notation gemäß der Form [**ugo**]?[[**+-=**][**rwxst**],[...]]. Mehrere Rechtsspezifikationen können durch Kommas getrennt angegeben werden. Z. B. würde **u+s,go-rw** das setuid-Bit für den Besitzer der Datei setzen sowie Lese- und Schreibrechte für Gruppe und andere entfernen. Eine **ls**-artige Zeichenkette der Form **rwxrwxrwx** wird ebenfalls akzeptiert, muss aber immer 9 Zeichen lang sein. Z. B. ist **rwxr-xr-t** äquivalent zu

**01755**. Auf Unix-Versionen mit Dateiflags gibt **-readonly** den Wert des schreibgeschützten Attributs einer Datei zurück oder setzt bzw. löscht es, d. h. das Benutzer-Immutable-Flag ( **uchg**) des Befehls **chflags**.

Unter Windows liefert **-archive** den Wert oder setzt bzw. löscht das Archiv-Attribut der Datei. **-hidden** liefert den Wert oder setzt bzw. löscht das versteckte Attribut der Datei. **-longname** erweitert jedes Pfadelement auf seine Langversion. Dieses Attribut kann nicht gesetzt werden. **-readonly** liefert den Wert oder setzt bzw. löscht das schreibgeschützte Attribut der Datei. **-shortname** liefert eine Zeichenkette, in der jedes Pfadelement nach Möglichkeit durch seine kurze (8.3-)Version des Namens ersetzt wird. Pfadelemente, die keiner Kurzform zugeordnet werden können, behalten den langen Namen. Dieses Attribut kann nicht gesetzt werden. **-system** liefert den Wert oder setzt bzw. löscht das System-Attribut der Datei.

Unter macOS und Darwin liefert oder setzt **-creator** den Finder-Erstellertyp der Datei. **-hidden** liefert den Wert oder setzt bzw. löscht das versteckte Attribut der Datei. **-readonly** liefert den Wert oder setzt bzw. löscht das schreibgeschützte Attribut der Datei. **-rsrclength** gibt die Länge des Resource Fork der Datei an; dieses Attribut kann nur auf den Wert 0 gesetzt werden, wodurch der Resource Fork von der Datei entfernt wird.

Auf allen Plattformen geben Dateien in eingehängten zipfs-Archiven die folgenden Attribute zurück. Diese sind alle schreibgeschützt und können nicht direkt gesetzt werden.

**-archive**
Der Pfad des eingehängten ZIP-Archivs, das die Datei enthält.

**-compsize**
Die komprimierte Größe der Datei innerhalb des Archivs. Für Verzeichnisse ist dies **0**.

**-crc**
Die CRC der Datei, falls vorhanden, sonst **0**.

**-mount**
Der Pfad, unter dem das enthaltende Archiv eingehängt ist.

**-offset**
Der Offset der Datei innerhalb des Archivs.

**-uncompsize**
Die unkomprimierte Größe der Datei. Für Verzeichnisse ist dies **0**.

Weitere Attribute können in der zurückgegebenen Liste enthalten sein. Diese sollten ignoriert werden.

**file channels ?*pattern*?**
Ist *pattern* nicht angegeben, gibt der Befehl eine Liste der Namen aller in diesem Interpreter registrierten offenen Kanäle zurück. Ist *pattern* angegeben, werden nur Namen zurückgegeben, die zu *pattern* passen. Die Übereinstimmung erfolgt nach denselben Regeln wie bei string match.

**file copy ?-force? ?--? *source target***
siehe unten ...

**file copy ?-force? ?--? *source ?source …? targetDir***
Die erste Form erstellt eine Kopie der Datei oder des Verzeichnisses *source* unter dem Pfadnamen *target*. Ist *target* ein vorhandenes Verzeichnis, wird die zweite Form verwendet. Die zweite Form kopiert jede aufgeführte *source*-Datei in *targetDir*. Wird ein Verzeichnis als

*source* angegeben, werden dessen Inhalte rekursiv nach *targetDir* kopiert. Vorhandene Dateien werden nicht überschrieben, sofern die Option **-force** nicht angegeben ist (dann versucht Tcl außerdem, die Berechtigungen der Zieldatei oder des Zielverzeichnisses anzupassen, falls das zum Fortsetzen der Kopie nötig ist). Beim Kopieren innerhalb eines einzelnen Dateisystems kopiert *file copy* symbolische Links (d. h. die Links selbst, nicht die Ziele). Der Versuch, ein nicht leeres Verzeichnis zu überschreiben, ein Verzeichnis mit einer Datei zu überschreiben oder eine Datei mit einem Verzeichnis zu überschreiben, führt auch bei **-force** zu Fehlern. Die Argumente werden in der angegebenen Reihenfolge verarbeitet und beim ersten Fehler abgebrochen. Ein **--** markiert das Ende der Schalter; das Argument nach **--** wird als *source* behandelt, auch wenn es mit **-** beginnt.

**file delete ?-force? ?--? ?*pathname* ...?**

Entfernt die Datei oder das Verzeichnis, das von jedem Argument *pathname* angegeben wird. Nicht leere Verzeichnisse werden nur entfernt, wenn die Option **-force** angegeben ist. Bei symbolischen Links werden die Links selbst gelöscht, nicht die Ziele. Das Löschen einer nicht existierenden Datei gilt nicht als Fehler. Beim Löschen einer schreibgeschützten Datei wird die Datei gelöscht, auch wenn **-force** nicht angegeben ist. Ist **-force** für ein Verzeichnis angegeben, versucht Tcl sowohl, die Berechtigungen zu ändern, als auch das aktuelle Arbeitsverzeichnis "pwd" aus dem angegebenen Pfad herauszubewegen, falls das zum Löschen nötig ist. Die Argumente werden in der angegebenen Reihenfolge verarbeitet und beim ersten Fehler abgebrochen. Ein **--** markiert das Ende der Schalter; das Argument nach **--** wird als *pathname* behandelt, auch wenn es mit **-** beginnt.

**file dirname *name***

Gibt einen Namen zurück, der aus allen Pfadkomponenten in *name* ohne das letzte Element besteht. Ist *name* ein relativer Dateiname und enthält nur ein Pfadelement, wird "**.**" zurückgegeben. Verweist *name* auf ein Stammverzeichnis, wird das Stammverzeichnis zurückgegeben. Beispiel:

```
file dirname c:/
```

gibt **c:/** zurück.

**file executable *name***

Gibt **1** zurück, wenn die Datei *name* für den aktuellen Benutzer ausführbar ist, sonst **0**. Unter Windows, wo es kein Ausführbarkeitsattribut gibt, behandelt der Befehl alle Verzeichnisse und Dateien mit den Erweiterungen **exe**, **com**, **cmd** oder **bat** als ausführbar.

**file exists *name***

Gibt **1** zurück, wenn die Datei *name* existiert und der aktuelle Benutzer Suchrechte für die führenden Verzeichnisse hat, sonst **0**.

**file extension *name***

Gibt alle Zeichen in *name* nach und einschließlich des letzten Punkts im letzten Element von *name* zurück. Enthält das letzte Element von *name* keinen Punkt, wird die leere Zeichenkette zurückgegeben.

**file home ?*username*?**

Ist kein Argument angegeben, gibt der Befehl das Home-Verzeichnis des aktuellen Benutzers zurück. Das ist im Allgemeinen der Wert der Umgebungsvariable $HOME, wobei unter Windows in Pfaden Backslashes durch Schrägstriche ersetzt werden. Ist die Umgebungsvariable $HOME nicht gesetzt, wird ein Fehler ausgelöst.

Ist *username* angegeben, gibt der Befehl das im System für den angegebenen Benutzer konfigurierte Home-Verzeichnis zurück. Beachten Sie, dass dies von dem Wert der Umgebungsvariable **$HOME** abweichen kann, selbst wenn *username* dem aktuellen Benutzer entspricht. Entspricht *username* keinem Benutzerkonto im System, wird ein Fehler ausgelöst.

**file isdirectory *name***

Gibt **1** zurück, wenn *name* ein Verzeichnis ist, sonst **0**.

**file isfile *name***

Gibt **1** zurück, wenn *name* eine reguläre Datei ist, sonst **0**.

**file join *name* ?*name* ...?**

Nimmt einen oder mehrere Dateinamen und verbindet sie mit dem für die aktuelle Plattform korrekten Pfadtrennzeichen. Ist ein bestimmter *name* relativ, wird er an das vorherige Dateinamenargument angehängt. Andernfalls werden frühere Argumente verworfen, und die Verknüpfung setzt beim aktuellen Argument fort. Beispiel:

```
file join a b /foo bar
```

gibt **/foo/bar** zurück.

Beachten Sie, dass die Namen Trennzeichen enthalten können und das Ergebnis für die aktuelle Plattform stets kanonisch ist: **/** unter Unix und Windows.

**file link ?*-linktype*? *linkName* ?*target*?**

Wird nur ein Argument übergeben, wird dieses als *linkName* angenommen, und der Befehl gibt den Wert des Links *linkName* zurück (d. h. den Namen der Datei, auf die er verweist). Ist *linkName* kein Link oder kann sein Wert nicht gelesen werden (wie z. B. bei Hardlinks, die wie normale Dateien aussehen), wird ein Fehler zurückgegeben.

Werden zwei Argumente übergeben, werden diese als *linkName* und *target* angenommen. Existiert *linkName* bereits oder existiert *target* nicht, wird ein Fehler zurückgegeben. Andernfalls erstellt Tcl einen neuen Link *linkName*, der auf das vorhandene Dateisystemobjekt bei *target* verweist (dies ist auch der Rückgabewert); der Linktyp ist plattformspezifisch (unter Unix standardmäßig ein symbolischer Link). Das ist nützlich, wenn der Benutzer einen Link plattformübergreifend anlegen will, ohne den konkreten Linktyp zu kennen.

Soll nur ein Link eines bestimmten Typs erstellt werden (mit Fehler, falls das nicht möglich ist), ist das optionale Argument -*linktype* anzugeben. Zulässige Werte für -*linktype* sind " **-symbolic**" und "**-hard**".

Unter Unix können symbolische Links auf relative Pfade zeigen; diese Pfade müssen relativ zum tatsächlichen Speicherort von *linkName* sein (nicht zum aktuellen Arbeitsverzeichnis). Auf allen anderen Plattformen ohne Unterstützung relativer Links werden Zielpfade vor dem Anlegen des Links stets in absolute, normalisierte Form umgewandelt (relative Pfade beziehen sich also auf das aktuelle Arbeitsverzeichnis). Beim Anlegen von Links auf Dateisystemen, die keine Links oder nicht den angeforderten Typ unterstützen, wird eine Fehlermeldung zurückgegeben. Die meisten Unix-Plattformen unterstützen symbolische und Hardlinks (letztere nur für Dateien). Windows unterstützt symbolische Verzeichnislinks und Hardlinks für Dateien auf NTFS-Laufwerken.

**file lstat *name* ?*varName*?**
Entspricht der Option **stat** (siehe unten), verwendet jedoch den Kernel-Aufruf *lstat* statt *stat*. Verweist *name* auf einen symbolischen Link, beziehen sich die zurückgegebenen Informationen auf den Link, nicht auf die Zieldatei. Auf Systemen ohne symbolische Links verhält sich diese Option genau wie **stat**.

**file mkdir ?*dir* ...?**
Erstellt jedes angegebene Verzeichnis. Für jeden angegebenen Pfadnamen *dir* legt der Befehl alle nicht vorhandenen übergeordneten Verzeichnisse sowie *dir* selbst an. Ist ein vorhandenes Verzeichnis angegeben, erfolgt keine Aktion und kein Fehler. Der Versuch, eine vorhandene Datei mit einem Verzeichnis zu überschreiben, führt zu einem Fehler. Die Argumente werden in der angegebenen Reihenfolge verarbeitet und beim ersten Fehler abgebrochen.

**file mtime *name* ?*time*?**
Gibt eine Dezimalzeichenkette zurück, die den Zeitpunkt der letzten Änderung der Datei *name* angibt. Ist *time* angegeben, handelt es sich um eine Änderungszeit, die für die Datei gesetzt werden soll (entspricht Unix **touch**). Die Zeit wird in der üblichen POSIX-Weise als Sekunden seit einem festen Startzeitpunkt (oft der 1. Januar 1970) gemessen. Existiert die Datei nicht oder kann ihre Änderungszeit weder abgefragt noch gesetzt werden, wird ein Fehler erzeugt. Auf zipfs-Dateisystemen kann die Änderungszeit nicht explizit gesetzt werden.

**file nativename *name***
Gibt den plattformspezifischen Namen der Datei zurück. Das ist nützlich, wenn der Dateiname an einen plattformspezifischen Aufruf übergeben werden muss, z. B. an einen Unterprozess über exec unter Windows (siehe Beispiele unten).

**file normalize *name***
Gibt eine eindeutige normalisierte Pfaddarstellung für das Dateisystemobjekt (Datei, Verzeichnis, Link usw.) zurück, deren Zeichenkettenwert als eindeutiger Bezeichner verwendet werden kann. Ein normalisierter Pfad ist ein absoluter Pfad ohne "../" und "./". Außerdem liegt er im "Standard"-Format der nativen Plattform. Unter Unix bedeutet das, dass die Segmente bis zum Pfad frei von symbolischen Links/Aliasen sein müssen (das letzte Pfadelement darf ein symbolischer Link sein); unter Windows wird außerdem die lange Form mit deren groß-/kleinschreibungsabhängiger Darstellung gewünscht (eindeutiger, fallabhängiger Pfad). Die Ausnahme für das letzte Link-Element im Pfad ist nötig, weil Tcl oder der Benutzer auf den symbolischen Link selbst operieren können (z. B. sind **file delete**, **file rename** und **file copy** für symbolische Links definiert, nicht für deren Ziele).

**file owned *name***
Gibt **1** zurück, wenn die Datei *name* dem aktuellen Benutzer gehört, sonst **0**.

**file pathtype *name***
Gibt einen der Werte **absolute**, **relative**, **volumerelative** zurück. Verweist *name* auf eine bestimmte Datei auf einem bestimmten Volume, ist der Pfadtyp **absolute**. Verweist *name* relativ zum aktuellen Arbeitsverzeichnis, ist der Typ **relative**. Verweist *name* relativ zum aktuellen Arbeitsverzeichnis auf einem angegebenen Volume oder auf eine bestimmte Datei auf dem aktuellen Arbeits-Volume, ist der Typ **volumerelative**.

**file readable *name***
Gibt **1** zurück, wenn die Datei *name* für den aktuellen Benutzer lesbar ist, sonst **0**.

**file readlink *name***
Gibt den Wert des symbolischen Links *name* zurück (d. h. den Namen der Zieldatei). Ist *name*

kein symbolischer Link oder kann sein Wert nicht gelesen werden, wird ein Fehler zurückgegeben. Auf Systemen ohne symbolische Links ist diese Option nicht definiert.

**file rename ?-force? ?--?** ***source target***

siehe unten ...

**file rename ?-force? ?--?** ***source*** **?*source* ...?** ***targetDir***

Die erste Form benennt die Datei oder das Verzeichnis *source* in *target* um und verschiebt sie, wenn *target* einen Namen in einem anderen Verzeichnis angibt. Ist *target* ein vorhandenes Verzeichnis, wird die zweite Form verwendet. Die zweite Form verschiebt jede *source*-Datei oder jedes *source*-Verzeichnis in *targetDir*. Vorhandene Dateien werden nicht überschrieben, sofern **-force** nicht angegeben ist. Innerhalb eines einzelnen Dateisystems benennt Tcl symbolische Links um, nicht deren Ziele. Der Versuch, ein nicht leeres Verzeichnis zu überschreiben, ein Verzeichnis mit einer Datei zu überschreiben oder eine Datei mit einem Verzeichnis zu überschreiben, führt zu Fehlern. Die Argumente werden in der angegebenen Reihenfolge verarbeitet und beim ersten Fehler abgebrochen. Ein **--** markiert das Ende der Schalter; das Argument nach **--** wird als *source* behandelt, auch wenn es mit **-** beginnt.

**file rootname** ***name***

Gibt alle Zeichen in *name* bis, aber ohne das letzte "."-Zeichen im letzten Komponenten von *name* zurück. Enthält die letzte Komponente von *name* keinen Punkt, wird *name* zurückgegeben.

**file separator ?*name*?**

Ohne Argument gibt der Befehl das Zeichen zurück, das Pfadsegmente nativer Dateien auf dieser Plattform trennt. Ist ein Pfad angegeben, wird das zuständige Dateisystem nach seinem Trennzeichen befragt. Akzeptiert kein Dateisystem *name*, wird ein Fehler erzeugt.

**file size** ***name***

Gibt eine Dezimalzeichenkette mit der Größe der Datei *name* in Byte zurück. Existiert die Datei nicht oder kann ihre Größe nicht abgefragt werden, wird ein Fehler erzeugt.

**file split** ***name***

Gibt eine Liste zurück, deren Elemente die Pfadkomponenten in *name* sind. Das erste Element hat denselben Pfadtyp wie *name*. Alle weiteren Elemente sind relativ. Pfadtrennzeichen werden verworfen, sofern sie nicht nötig sind, um ein Element eindeutig relativ zu machen.

**file stat** ***name*** **?*varName*?**

Ruft den Kernel-Aufruf **stat** für *name* auf und gibt ein Dictionary mit den vom Kernel zurückgegebenen Informationen zurück. Ist *varName* angegeben, werden die Informationen in der Variable abgelegt. *VarName* wird als Array-Variable behandelt; in diesem Fall gibt der Befehl die leere Zeichenkette zurück. Folgende Elemente werden gesetzt: **atime**, **ctime**, **dev**, **gid**, **ino**, **mode**, **mtime**, **nlink**, **size**, **type**, **uid**. Jedes Element außer **type** ist eine Dezimalzeichenkette mit dem Wert des entsprechenden Felds aus der **stat**-Rückgabestruktur; Details zu den Bedeutungen stehen in der Manpage **stat**. Das Element **type** gibt den Dateityp in derselben Form zurück wie der Befehl **file type**.

**file system** ***name***

Gibt eine Liste mit einem oder zwei Elementen zurück: das erste ist der Name des für die Datei zu verwendenden Dateisystems, das zweite (falls vorhanden) eine beliebige Zeichenkette für die dateisystemspezifische Art oder den Typ des Speicherorts innerhalb dieses Dateisystems. Unterstützt ein Dateisystem nur einen Dateityp, fehlt das zweite Element möglicherweise. Native Dateien haben z. B. als erstes Element "native" und als zweites (falls vorhanden) einen plattformspezifischen Typnamen (z. B. "NTFS", "FAT" unter Windows). Ein generisches virtuelles Dateisystem könnte "vfs ftp" zurückgeben für eine Datei auf einer per Erweiterung "vfs" eingehängten FTP-Site. Gehört die Datei keinem Dateisystem an, wird ein Fehler erzeugt.

**file tail *name***

Gibt alle Zeichen der letzten Dateisystemkomponente von *name* zurück. Ein abschließender Verzeichnistrenner in *name* wird ignoriert. Enthält *name* keine Trennzeichen, wird *name* zurückgegeben. Also liefern **file tail a/b**, **file tail a/b/** und **file tail b** alle **b**.

**file tempdir ?*template*?**

Erstellt ein temporäres Verzeichnis (garantiert neu angelegt und für das aktuelle Skript beschreibbar) und gibt dessen Namen zurück. Ist template angegeben, legt es das vorhandene Verzeichnis fest (oder beides), in dem das temporäre Verzeichnis liegen soll, und/oder den Basisnamen; ein Verzeichnistrenner im Namen bedeutet, dass der Ort des Verzeichnisses angegeben ist, und der Basisname ist alles nach dem letzten Trenner (falls nicht leer). Standardmäßig wird das enthaltende Verzeichnis systemabhängig bestimmt, der Standard-Basisnamenpräfix ist "tcl".

Die folgende Ausgabe ist typisch und veranschaulichend; die tatsächliche Ausgabe variiert je nach Plattform:

```
% file tempdir
/var/tmp/tcl_u0kuy5
 % file tempdir /tmp/myapp
/tmp/myapp_8o7r9L
% file tempdir /tmp/
/tmp/tcl_1mOJHD
% file tempdir myapp
/var/tmp/myapp_0ihS0n
```

**file tempfile ?*nameVar*? ?*template*?**

Erstellt eine temporäre Datei und gibt einen Lese-Schreib-Kanal zurück, der auf dieser Datei geöffnet ist. Ist *nameVar* angegeben, wird der Name der temporären Datei in diese Variable geschrieben; fehlt sie, versucht Tcl, die temporäre Datei zu löschen, sobald sie nicht mehr benötigt wird. Ist *template* vorhanden, legt es Teile der Vorlage des Dateinamens fest (z. B. Verzeichnis, Basisname oder Erweiterung); einige Plattformen ignorieren Teile davon und verwenden einen eingebauten Standard.

Temporäre Dateien werden *nur* auf dem nativen Dateisystem erstellt. Sie eignen sich daher für Betriebssystem-APIs und externe Programme, die einen Dateinamen benötigen.

**file tildeexpand *name***

Gibt das Ergebnis der Tilden-Substitution für name zurück. Beginnt der Name mit einer Tilde, wird der Dateiname so interpretiert, als wäre das erste Element durch das Home-Verzeichnis des angegebenen Benutzers ersetzt. Folgt der Tilde unmittelbar ein Pfadtrenner, wird die Umgebungsvariable $HOME substituiert. Andernfalls gelten die Zeichen zwischen Tilde und dem nächsten Trenner als Benutzername, dessen Home-Verzeichnis für die Substitution verwendet wird. Ist die Umgebungsvariable $HOME oder der Benutzer nicht vorhanden, wird ein Fehler ausgelöst.

Beginnt der Dateiname nicht mit einer Tilde, wird er unverändert zurückgegeben.

**file type *name***

Gibt eine Zeichenkette mit dem Typ der Datei *name* zurück: **file**, **directory**, **characterSpecial**, **blockSpecial**, **fifo**, **link** oder **socket**.

**file volumes**

Gibt die absoluten Pfade der im System eingehängten Volumes als ordentliche Tcl-Liste zurück. Ohne zusätzliche virtuelle Dateisysteme als Stamm-Volumes liefert der Befehl unter UNIX "//zipfs:/" oder "/" (bei einem Bau mit --disable-zipfs), da alle Dateisysteme lokal eingehängt sind. Unter Windows eine Liste der verfügbaren lokalen Laufwerke (z. B. "//zipfs:/ C:/"). Hat ein virtuelles Dateisystem weitere Volumes eingehängt, erscheinen diese ebenfalls in der Liste.

**file writable *name***

Gibt **1** zurück, wenn die Datei *name* für den aktuellen Benutzer beschreibbar ist, sonst **0**.

## Portabilitätsfragen

**Unix**

Diese Befehle arbeiten stets mit den realen Benutzer- und Gruppen-IDs, nicht mit den effektiven.

**Windows**

Der Unterbefehl **file owned** verwendet die Benutzer-ID (SID) des Prozess-Tokens, nicht das Thread-Token, das möglicherweise einen anderen Benutzer impersoniert.

## Beispiele

Diese Prozedur zeigt, wie in einem Verzeichnis nach C-Dateien gesucht wird, zu denen im aktuellen Verzeichnis eine gleichnamige Objektdatei existiert:

```
proc findMatchingCFiles {dir} {
    set files {}
    switch $::tcl_platform(platform) {
        windows {
            set ext .obj
        }
        unix {
           set ext .o
        }
    }
    foreach file [glob -nocomplain -directory $dir *.c] {
        set objectFile [file tail [file rootname $file]]$ext
        if {[file exists $objectFile]} {
            lappend files $file
        }
    }
    return $files
}
```

Eine Datei umbenennen und einen symbolischen Link vom alten zum neuen Ort hinterlassen:

```
set oldName foobar.txt
set newName foo/bar.txt
# Sicherstellen, dass das Zielverzeichnis existiert...
if {![file isdirectory [file dirname $newName]]} {
    file mkdir [file dirname $newName]
}
file rename $oldName $newName
file link -symbolic $oldName $newName
```

Unter Windows kann eine Datei leicht "gestartet" werden (entspricht Doppelklick im Explorer), der an das Betriebssystem übergebene Name muss jedoch im nativen Format vorliegen:

```
exec {*}[auto_execok start] {} [file nativename
    C:/Users/fred/example.txt]
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# fileevent

*Stichwörter:* fileevent

## Name

fileevent - Führt ein Skript aus, wenn ein Kanal lesbar oder beschreibbar wird

## Übersicht

**fileevent** *channel* **readable** ?*script*?
**fileevent** *channel* **writable** ?*script*?

## Beschreibung

Der Befehl **fileevent** wurde durch den Befehl chan event abgelöst, der dieselbe Syntax und dieselben Optionen unterstützt.

*Copyright (c) 1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2008 Pat Thoyts*

# filename

*Stichwörter:* filename, current directory, absolute file name, relative file name, volume-relative file name, portability

## Name

filename - Von Tcl-Befehlen unterstützte Dateinamenskonventionen

## Einleitung

Alle Tcl-Befehle und C-Prozeduren, die Dateinamen als Argumente erwarten, verlangen Dateinamen in einer von drei Formen, abhängig von der aktuellen Plattform. Auf jeder Plattform unterstützt Tcl Dateinamen in der oder den Standardform(en) dieser Plattform. Zusätzlich unterstützt Tcl auf allen Plattformen eine Unix-ähnliche Syntax, um einfache Dateinamen bequem zu konstruieren. Portierbare Skripte sollten jedoch keine bestimmte Form für

Dateinamen voraussetzen. Stattdessen müssen portierbare Skripte die Befehle file split und file join zum Bearbeiten von Dateinamen verwenden (Einzelheiten im Manpage-Eintrag file).

## Pfadtypen

Dateinamen werden in drei allgemeine Typen eingeteilt, je nach Ausgangspunkt des Pfads zur Angabe der Datei: absolut, relativ und volumen-relativ. Absolute Namen sind vollständig qualifiziert und geben einen Pfad zur Datei relativ zu einem bestimmten Volume und dem Stammverzeichnis auf diesem Volume an. Relative Namen sind unqualifiziert und geben einen Pfad relativ zum aktuellen Arbeitsverzeichnis an. Volumen-relative Namen sind teilweise qualifiziert — entweder relativ zum Stammverzeichnis auf dem aktuellen Volume oder relativ zum aktuellen Verzeichnis des angegebenen Volumes. Der Befehl file pathtype kann verwendet werden, um den Typ eines gegebenen Pfads zu bestimmen.

## Pfad-Syntax

Die Regeln für native Namen hängen vom Wert des Tcl-Elements **platform** im Array tcl_platform ab:

### Unix

Auf Unix- und Apple-macOS-Plattformen verwendet Tcl Pfadnamen, deren Komponenten durch Schrägstriche getrennt sind. Pfadnamen können relativ oder absolut sein, und Dateinamen dürfen jedes Zeichen außer Schrägstrich enthalten. Die Dateinamen**.** und**..** sind speziell und bezeichnen das aktuelle bzw. das übergeordnete Verzeichnis. Mehrere benachbarte Schrägstriche gelten als ein einzelnes Trennzeichen, außer dem ersten Doppel-Schrägstrich **//** in absoluten Pfaden. Beliebig viele abschließende Schrägstriche am Pfadende werden ignoriert, sodass **foo**, **foo/** und **foo//** identisch sind; insbesondere bedeutet **foo/** nicht zwingend ein Verzeichnis.

Die folgenden Beispiele veranschaulichen verschiedene Pfadformen:

**/**
Absoluter Pfad zum Stammverzeichnis.

**/etc/passwd**
Absoluter Pfad zur Datei **passwd** im Verzeichnis **etc** im Stammverzeichnis.

**.**
Relativer Pfad zum aktuellen Verzeichnis.

**foo**
Relativer Pfad zur Datei **foo** im aktuellen Verzeichnis.

**foo/bar**
Relativer Pfad zur Datei **bar** im Verzeichnis **foo** im aktuellen Verzeichnis.

**../foo**
Relativer Pfad zur Datei **foo** im Verzeichnis über dem aktuellen Verzeichnis.

### Windows

Auf Microsoft-Windows-Plattformen unterstützt Tcl sowohl laufwerks-relative als auch UNC-Namen. Sowohl **/** als auch **** können als Verzeichnistrenner verwendet werden. Laufwerks-relative Namen bestehen aus einem optionalen Laufwerksbezeichner gefolgt von einem absoluten oder relativen Pfad. UNC-Pfade folgen der Form **\\servername\sharename\path\file**, müssen aber mindestens Server- und

Freigabe-Komponenten enthalten, also **\\servername\sharename**. In beiden Formen sind **.** und **..** speziell und bezeichnen das aktuelle bzw. das übergeordnete Verzeichnis. Die folgenden Beispiele veranschaulichen verschiedene Pfadformen:

**\\Host\share/file**

Absoluter UNC-Pfad zu einer Datei file im Stammverzeichnis der Freigabe **share** auf dem Host **Host**. Wiederholtes file dirname auf diesem Pfad liefert **//Host/share**, nie nur **//Host**.

**c:foo**

Volumen-relativer Pfad zur Datei **foo** im aktuellen Verzeichnis auf Laufwerk **c**.

**c:/foo**

Absoluter Pfad zur Datei **foo** im Stammverzeichnis von Laufwerk **c**.

**foo\bar**

Relativer Pfad zur Datei **bar** im Verzeichnis **foo** im aktuellen Verzeichnis auf dem aktuellen Volume.

**\foo**

Volumen-relativer Pfad zur Datei **foo** im Stammverzeichnis des aktuellen Volumes.

**\\foo**

Volumen-relativer Pfad zur Datei **foo** im Stammverzeichnis des aktuellen Volumes. Das ist kein gültiger UNC-Pfad; die zusätzlichen Backslashes gelten als überflüssig.

**Zipfs**

Auf allen Plattformen mit aktivierter zipfs-Unterstützung beginnen Pfade in eingehängten ZIP-Archiven mit der von zipfs root zurückgegebenen Zeichenkette. Zipfs-Pfade sind auf allen Plattformen groß-/kleinschreibungssensitiv.

## Tilde-Ersetzung

Anders als frühere Tcl-Versionen führt Tcl 9 keine implizite Tilde-Ersetzung in Dateipfaden durch — mit der unten genannten Ausnahme. Die Befehle file home und file tildeexpand können dasselbe explizit leisten.

Die Ausnahme betrifft die Initialisierung der Variable auto_path und der Tcl-Modul-Suchpfade wie in den Manpages tclvars und tm dokumentiert. Beginnt ein Pfad in einer Umgebungsvariable für diese Initialisierung mit einer Tilde, wird er so interpretiert, als wäre das erste Element durch das Home-Verzeichnis des angegebenen Benutzers ersetzt. Folgt der Tilde unmittelbar ein Trennzeichen, wird die Umgebungsvariable **$HOME** substituiert. Andernfalls werden die Zeichen zwischen Tilde und dem nächsten Trennzeichen als Benutzername genommen, um das Home-Verzeichnis für die Substitution zu ermitteln. Das gilt auf POSIX-, macOS- und Windows-Plattformen.

## Portabilitätsfragen

Nicht alle Dateisysteme unterscheiden Groß-/Kleinschreibung; Skripte sollten Code vermeiden, der von der Schreibweise in Dateinamen abhängt. Zudem können erlaubte Zeichensätze je Gerät variieren; Skripte sollten Dateinamen ohne Sonderzeichen wie **<>:?"/|** wählen. Am sichersten sind Namen nur aus alphanumerischen Zeichen. Vorsicht bei Dateinamen mit Leerzeichen (häufig unter Windows) und bei Dateinamen, bei denen Backslash das Verzeichnistrennzeichen ist (native Windows-Pfade).

Unter Windows gelten Längenbeschränkungen für Dateien und Pfade. Vollständige Pfade oder

Dateinamen länger als etwa 260 Zeichen führen bei den meisten Dateioperationen zu Fehlern.

Eine weitere Windows-Besonderheit: beliebig viele abschließende Punkte "." in Dateinamen werden völlig ignoriert — der Versuch, eine Datei oder ein Verzeichnis "foo." zu erstellen, erzeugt "foo". Das spiegelt sich in den Ergebnissen von file normalize wider. Außerdem ist ein Dateiname, der nur aus Punkten besteht ("........") oder aus Punkten mit nachfolgenden Zeichen (".....abc"), unzulässig.

*Copyright (c) 1995-1996 Sun Microsystems, Inc.*

# flush

*Stichwörter:* flush

## Name

flush - Leert die gepufferte Ausgabe eines Kanals

## Übersicht

**flush** *channel*

## Beschreibung

Der Befehl **flush** wurde durch den Befehl chan flush ersetzt, der die gleiche Syntax und die gleichen Optionen unterstützt.

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# for

*Stichwörter:* for, boolean, iteration, loop

## Name

for - 'For'-Schleife

## Übersicht

**for** *start test next body*

## Beschreibung

**for** ist ein Schleifen-Befehl, ähnlich in der Struktur zur C **for**-Anweisung. Die Argumente *start*, *next* und *body* müssen Tcl-Befehlsstrings sein, und *test* ist ein Ausdrucksstring. Der **for**-Befehl ruft zuerst den Tcl-Interpreter auf, um *start* auszuführen. Dann wertet er wiederholt *test* als Ausdruck aus; wenn das Ergebnis nicht null ist, ruft er den Tcl-Interpreter für *body* auf, dann ruft er den Tcl-Interpreter für *next* auf und wiederholt dann die Schleife. Der Befehl endet, wenn *test* zu 0 ausgewertet wird. Wenn ein continue-Befehl innerhalb von *body* aufgerufen wird, werden alle verbleibenden Befehle in der aktuellen Ausführung von *body* übersprungen; die Verarbeitung wird fortgesetzt, indem der Tcl-Interpreter für *next* aufgerufen wird, dann *test* ausgewertet wird und so weiter. Wenn ein break-Befehl innerhalb von *body* oder *next* aufgerufen wird, kehrt der **for**-Befehl sofort zurück. Die Funktionsweise von break und continue ist ähnlich den entsprechenden Anweisungen in C. **for** gibt einen leeren String zurück.

Beachten Sie, dass *test* fast immer in geschweifte Klammern eingeschlossen werden sollte. Wenn nicht, werden Variablenersetzungen vorgenommen, bevor der **for**-Befehl mit der

Ausführung beginnt, was bedeutet, dass Variablenänderungen durch den Schleifenkörper nicht im Ausdruck berücksichtigt werden. Dies führt wahrscheinlich zu einer Endlosschleife. Wenn *test* in geschweifte Klammern eingeschlossen ist, werden Variablenersetzungen verzögert, bis der Ausdruck ausgewertet wird (vor jeder Schleifeniteration), sodass Änderungen an den Variablen sichtbar werden. Siehe unten ein Beispiel:

### Beispiele

Gibt eine Zeile für jede Ganzzahl von 0 bis 9 aus:

```
for {set x 0} {$x<10} {incr x} {
    puts "x is $x"
}
```

Entweder läuft die Schleife endlos oder überhaupt nicht, weil der auszuwertende Ausdruck tatsächlich die Konstante ist, oder es wird sogar ein Fehler erzeugt! Das tatsächliche Verhalten hängt davon ab, ob die Variable *x* vor der Ausführung des **for**-Befehls existiert und ob ihr Wert kleiner als oder größer/gleich zehn ist, und das liegt daran, dass der Ausdruck ersetzt wird, bevor der **for**-Befehl ausgeführt wird.

```
for {set x 0} $x<10 {incr x} {
    puts "x is $x"
}
```

Gibt die Zweierpotenzen von 1 bis 1024 aus:

```
for {set x 1} {$x<=1024} {set x [expr {$x * 2}]} {
    puts "x is $x"
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

## foreach

*Stichwörter:* foreach, iteration, list, loop

### Name

foreach - Iteriert über alle Elemente einer oder mehrerer Listen

### Übersicht

**foreach** *varname list body*
**foreach** *varlist1 list1* ?*varlist2 list2* ...? *body*

### Beschreibung

**foreach** implementiert eine Schleife, in der die Schleifenvariable(n) Werte aus einer oder mehreren Listen annehmen. Im einfachsten Fall gibt es *varname* und *list*. Das Argument *body* ist ein Tcl-Skript. Für jedes Element von *list* (von vorne nach hinten) wird der Inhalt *varname* zugewiesen (wie mit lindex) und *body* ausgeführt.

Allgemein können mehrere Wertelisten (*list1*, *list2*, ...) jeweils mit Variablenlisten (*varlist1*, *varlist2*, ...) verknüpft sein. Pro Iteration erhalten die Variablen jeder *varlist* aufeinanderfolgende

Werte aus der zugehörigen Liste; Werte werden der Reihe nach verbraucht, jeder genau einmal. Die Iterationsanzahl reicht aus, alle Wertelisten zu erschöpfen. Reicht eine Liste für ihre Variablen in einer Iteration nicht, werden fehlende Elemente leer gesetzt.

break und continue in *body* wirken wie bei for. **foreach** gibt eine leere Zeichenkette zurück.

## Beispiele

Jeden Wert mit Quadrat und Kubik ausgeben:

```
set values {1 3 5 7 2 4 6 8}    ;# Ungerade zuerst
puts "Value\tSquare\tCube"  ;# Neat-looking header
foreach x $values { ;# Now loop and print...
    puts " $x\t [expr {$x**2}]\t [expr {$x**3}]"
}
```

*i* und *j* iterieren über Paare einer Liste:

```
set x {}
foreach {i j} {a b c d e f} {
    lappend x $j $i
}
# The value of x is "b a d c f e"
# There are 3 iterations of the loop.
```

Parallele Iteration über zwei Listen:

```
set x {}
foreach i {a b c} j {d e f g} {
    lappend x $i $j
}
# The value of x is "a d b e c f {} g"
# There are 4 iterations of the loop.
```

Kombination beider Formen:

```
set x {}
foreach i {a b c} {j k} {d e f g} {
    lappend x $i $j $k
}
# The value of x is "a d e b f g c {} {}"
# There are 3 iterations of the loop.
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# format

*Stichwörter:* format, conversion specifier, sprintf, string, substitution

## Name

format - Zeichenkette im Stil von sprintf formatieren

## Übersicht

**format** *formatString* ?*arg arg* ...?

## Einleitung

Der Befehl erzeugt eine formatierte Zeichenkette analog zu ANSI-C **sprintf**. *formatString* legt das Format fest (**%**-Konvertierungen wie bei **sprintf**); weitere Argumente liefern die einzusetzenden Werte. Rückgabe: die formatierte Zeichenkette.

## Details zum Formatieren

**format** durchläuft *formatString* von links nach rechts. Jedes Zeichen wird an die Ergebniszeichenkette angehängt, außer es ist **%**. Dann wird nicht kopiert; die folgenden Zeichen sind ein Konvertierungsspezifizierer. Er steuert die Umwandlung des nächsten *arg* und ersetzt den Spezifizierer im Ergebnis. Mehrere Spezifizierer verarbeiten weitere *arg*s in Reihenfolge; es müssen genug *arg*s für alle Spezifizierer vorhanden sein.

Ein Spezifizierer kann bis zu sechs Teile haben: XPG3-Positionsangabe, Flags, Mindestfeldbreite, Genauigkeit, Größenmodifikator, Konvertierungszeichen. Pflicht ist nur das Konvertierungszeichen; vorhandene Teile in der genannten Reihenfolge. Die folgenden Absätze behandeln diese Felder der Reihe nach.

### Optionale Positionsangabe

Folgt auf **%** eine Dezimalzahl und **$** (z. B. **%2$d**), stammt der Wert nicht vom nächsten sequenziellen Argument, sondern vom angegebenen (1 = erstes *arg*). Benötigt der Spezifizierer mehrere Argumente wegen ***** in der Angabe, werden ab diesem Argument weitere verwendet (XPG3). Sind Positionsangaben vorhanden, müssen alle Spezifizierer positional sein.

### Optionale Flags

Zweiter Teil des Spezifizierers, beliebige Reihenfolge:

**-**

Linksbündig im Feld (Zahlen normalerweise rechtsbündig mit führenden Leerzeichen).

**+**

Vorzeichen immer ausgeben, auch bei positiven Zahlen.

***Leerzeichen***

Führendes Leerzeichen, wenn das erste Zeichen kein Vorzeichen ist.

**0**

Links mit Nullen statt Leerzeichen auffüllen.

**#**

Alternative Ausgabeform. **o**: **0o** am Anfang (außer Null). **x**/**X**: **0x** (außer Null). **b**: **0b** (außer Null). **d**: **0d** am Anfang (außer Null). Gleitkomma (**e**, **E**, **f**, **g**, **G**): Dezimalpunkt immer. **g**/**G** mit **#**: nachgestellte Nullen nicht entfernen.

### Optionale Feldbreite

Dritter Teil: Dezimalzahl als Mindestfeldbreite (Spaltenausrichtung). Ist das konvertierte Argument kürzer, wird aufgefüllt. Standard: Leerzeichen links; **0** und **-** steuern Nullen links bzw. Leerzeichen rechts. Steht ***** statt einer Zahl, liefert das nächste *arg* die Breite (Ganzzahl).

### Optionale Genauigkeit

Vierter Teil: Punkt und Zahl. Bedeutung je nach Konvertierung: **e**, **E**, **f**: Stellen rechts vom Dezimalpunkt. **g**, **G**: Gesamtstellenzahl beider Seiten (nachgestellte Nullen nach dem Punkt ohne **#** dennoch weggelassen). Ganzzahl: Mindeststellenzahl (führende Nullen). **s**: maximale Zeichenanzahl; Rest wird abgeschnitten. ***** als Genauigkeit: nächstes *arg* (numerische Zeichenkette).

### Optionaler Größenmodifikator

Fünfter Teil: **ll**, **h**, **l**, **z**, **t** oder **L**. **ll**: Ganzzahl ohne Kürzung. **h**: auf 16-Bit-Bereich kürzen (selten nützlich). **l** (oder **j** oder **q**): Bereich wie **wide()** in expr (mindestens 64 Bit). **z**/**t**: Bereich gemäß **pointerSize** in tcl_platform. **L**: Ganzzahl oder Gleitkomma ohne Kürzung. Fehlt der Modifikator: Ganzzahl auf 32-Bit-Bereich gekürzt.

### Pflicht-Konvertierungstyp

Letztes Zeichen: Konvertierungsart:

**d**

Ganzzahl → vorzeichenbehafteter Dezimalstring.

**u**

Ganzzahl → vorzeichenloser Dezimalstring.

**i**

wie **d**.

**o**

Ganzzahl → vorzeichenlose Oktalzeichenkette.

**x oder X**

Ganzzahl → vorzeichenlose Hexzeichenkette ("0123456789abcdef" bzw. "0123456789ABCDEF").

**b**

Ganzzahl → vorzeichenlose Binärzeichenkette (0 und 1).

**c**

Ganzzahl → zugehöriges Unicode-Zeichen.

**s**

keine Konvertierung; Zeichenkette einfügen.

**f**

Zahl → Dezimalstring *xx.yyy*; Anzahl *y* durch Genauigkeit (Standard 6). Genauigkeit 0: kein Dezimalpunkt.

**e oder E**

wissenschaftliche Notation *x.yyy***e±***zz*; *y*-Anzahl wie oben (Standard 6). Genauigkeit 0: kein Dezimalpunkt. **E** statt **e**.

**g oder G**

Exponent < -4 oder ≥ Genauigkeit → wie **%e**/**%E**, sonst wie **%f**. Nachgestellte Nullen und Punkt am Ende weggelassen.

**a oder A**

Gleitkomma hexadezimal *0x1.yyy***p±***zz* (Standard 13 *y*). **A**: Hex in Großbuchstaben.

**%**

Literal **%**.

**p**

Kurzform für **0x%zx** (Hex mit **0x**-Präfix).

## Unterschiede zu ANSI sprintf

Wie **sprintf**, außer:

1. Tcl arbeitet mit UNICODE-Zeichen.
2. **%n** wird nicht unterstützt.
3. Bei **%c** muss das Argument eine Ganzzahl sein (wird zum Zeichen).
4. Größenmodifikatoren bei Gleitkomma werden ignoriert. **b** hat kein **sprintf**-Pendant.

## Beispiele

Numerischen Unicode-Wert in ein Zeichen:

```
set value 120
set char [format %c $value]
```

time-Ausgabe in Sekunden mit Hundertsteln:

```
set us [lindex [time $someTclCode] 0]
puts [format "%.2f seconds to execute" [expr {$us / 1e6}]]
```

X11-Farbspezifikation:

```
# Each color-component should be in range (0..255)
set color [format "#%02x%02x%02x" $r $g $b]
```

XPG3-Positionscodes für lokalisierte Meldungen (msgcat), um Felder umzuordnen, ohne die an **format** übergebenen Werte umzuordnen:

```
set fmt1 "Today, %d shares in %s were bought at $%.2f each"
puts [format $fmt1 123 "Global BigCorp" 19.37]

set fmt2 "Bought %2\$s equity ($%3$.2f x %1\$d) today"
puts [format $fmt2 123 "Global BigCorp" 19.37]
```

Tabelle der Dreierpotenzen:

```
# Set up the column widths
set w1 5
set w2 10

# Make a nice header (with separator) for the table first
set sep +-[string repeat - $w1]-+-[string repeat - $w2]-+
puts $sep
puts [format "| %-*s | %-*s |" $w1 "Index" $w2 "Power"]
puts $sep

# Print the contents of the table
set p 1
for {set i 0} {$i<=20} {incr i} {
    puts [format "| %*d | %*ld |" $w1 $i $w2 $p]
    set p [expr {wide($p) * 3}]
}

# Finish off by printing the separator again
puts $sep
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# fpclassify

*Stichwörter:* fpclassify, floating point

## Name

fpclassify - Klassifikation von Gleitkommazahlen bei Tcl-Werten

## Übersicht

**package require tcl 9.0**
**fpclassify** *value*

## Beschreibung

Der Befehl **fpclassify** nimmt eine Gleitkommazahl *value* und gibt eine der folgenden beschreibenden Zeichenketten zurück:

**zero**
*value* ist eine Gleitkomma-Null.

**subnormal**
*value* ist das Ergebnis eines graduellen Underflows.

**normal**
*value* ist eine gewöhnliche Gleitkommazahl (weder Null, subnormal, unendlich noch NaN).

**infinite**
*value* ist eine Gleitkomma-Unendlichkeit.

**nan**
*value* ist Not-a-Number.

Der Befehl **fpclassify** löst einen Fehler aus, wenn *value* kein Gleitkommawert ist und nicht in einen umgewandelt werden kann.

## Beispiel

Dies zeigt, wie geprüft wird, ob das Ergebnis einer Berechnung numerisch sicher ist. (Es schützt jedoch nicht vor numerischen Fehlern, sondern nur vor Darstellungsproblemen.)

```
set value [command-that-computes-a-value]
switch [fpclassify $value] {
    normal - zero {
        puts "Result is $value"
    }
    infinite {
        puts "Result is infinite"
    }
    subnormal {
        puts "Result is $value - WARNING! precision lost"
    }
    nan {
        puts "Computation completely failed"
    }
}
```

### Standards

Dieser Befehl setzt voraus, dass das C-Makro **fpclassify**() der Norm "ISO C99" entspricht (d. h. ISO/IEC 9899:1999).

### Copyright

```
Copyright © 2018 Kevin B. Kenny <kennykb@acm.org>. All rights reserved
```

*Copyright (c) 2018 Kevin B. Kenny* kennykb@acm.org*. All rights reserved*
*Copyright (c) 2019 Donal Fellows*

## gets

*Stichwörter:* gets

### Name

gets - Liest eine Zeile aus einem Kanal

### Übersicht

**gets** *channel* ?*varName*?

### Beschreibung

Der Befehl **gets** wurde durch den Befehl chan gets ersetzt, der die gleiche Syntax und die gleichen Optionen unterstützt.

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## glob

*Stichwörter:* glob, exist, file, pattern

### Name

glob - Dateinamen zurückgeben, die Mustern entsprechen

### Übersicht

**glob** ?*switches*? ?*pattern* ...?

### Beschreibung

Führt Dateinamen-"Globbing" ähnlich wie in csh oder bash aus. Gibt eine Liste der Dateien zurück, deren Namen einem der *pattern*-Argumente entsprechen. Die Reihenfolge ist nicht garantiert; für eine sortierte Liste lsort verwenden.

## Optionen

Beginnen die ersten Argumente von **glob** mit **-**, gelten sie als Schalter. Unterstützt werden:

**-directory *directory***

Suche nach passenden Mustern ab dem angegebenen *directory*. Ermöglicht das Durchsuchen von Verzeichnissen, deren Name Glob-Sonderzeichen enthält, ohne diese explizit zu quoten. Nicht zusammen mit **-path** (Suche nach vollständigen Pfaden mit Glob-Sonderzeichen im Namen).

**-join**

Die verbleibenden Musterargumente nach der Optionenverarbeitung werden als ein einziges Muster behandelt, das durch Verkettung mit Verzeichnistrennzeichen entsteht.

**-nocomplain**

Erlaubt die Rückgabe einer leeren Liste ohne Fehler. In Tcl 9.0 ist das Standardverhalten; der Schalter hat daher keine Wirkung mehr.

**-path *pathPrefix***

Suche nach Dateien mit dem angegebenen *pathPrefix*, deren Rest dem Muster entspricht. Ermöglicht Suche nach Dateien mit Namen ähnlich einer gegebenen Datei (nicht nur einem Verzeichnis), auch wenn der Name Glob-Sonderzeichen enthält. Nicht zusammen mit **-directory**. Beispiel: alle Dateien mit gleichem Stamm wie $path, aber anderer Endung: "**glob -path [file rootname $path].***" — funktioniert auch bei vielen Glob-Sonderzeichen in **$path**.

**-tails**

Nur der Teil jedes Treffers, der auf das zuletzt in einer **-directory**- oder **-path**-Angabe genannte Verzeichnis folgt. "**glob -tails -directory $dir ***" entspricht "**set pwd [pwd]; cd $dir; glob *; cd $pwd**". Bei **-path** enthält der Rückgabename das letzte Pfadsegment, z. B. "**glob -tails -path [file rootname /home/fred/foo.tex].***" → **foo.aux foo.bib foo.tex** usw.

**-types *typeList***

Nur Dateien oder Verzeichnisse listen, die *typeList* entsprechen. Die Listeneinträge haben zwei Formen. Erste Form wie die -type-Option von Unix find: *b* (Blockgerät), *c* (Zeichengerät), *d* (Verzeichnis), *f* (normale Datei), *l* (symbolischer Link), *p* (Named Pipe), *s* (Socket); mehrere Typen möglich. **Glob** liefert alle Dateien, die mindestens einem Typ entsprechen. Symbolische Links werden zurückgegeben, wenn **-types l** gesetzt ist oder das Linkziel den gewünschten Typ hat (Link auf Verzeichnis bei **-types d**).

Zweite Form: alle angegebenen Typen müssen zutreffen — *r*, *w*, *x* als Dateirechte sowie *readonly*, *hidden*. Auf dem Macintosh werden macOS-Typen und -Erzeuger unterstützt; vier Zeichen lang = macOS-Typ (z. B. **TEXT**). *{macintosh type XXXX}* bzw. *{macintosh creator XXXX}* matchen Typ bzw. Erzeuger. Unbekannte Typen oder mehrere macOS-Typen/Erzeuger in einer Spezifikation: Fehler.

Beide Formen sind mischbar; **-types {d f r w}** findet normale Dateien ODER Verzeichnisse mit Lese- UND Schreibrecht. Äquivalent:

```
glob -type d *
glob */
```

außer dass der erste Fall kein abschließendes "/" liefert und plattformunabhängiger ist.

**--**
Ende der Schalter; das folgende Argument gilt als *pattern*, auch wenn es mit **-** beginnt.

### Glob-Muster

Die *pattern*-Argumente können folgende Sonderzeichen enthalten (Obermenge von string match ):

**?**
Ein beliebiges Zeichen.

*****
Null oder mehr beliebige Zeichen.

**[*chars*]**
Ein Zeichen aus *chars*. Enthält *chars* *a*-*b*, jedes Zeichen von *a* bis *b* (einschließlich).

**\*x***
Das Zeichen *x*.

**{*a*,*b*,...}**
Eines der Teilmuster *a*, *b*, ...

Unter Unix muss wie bei csh ein "." am Anfang des Dateinamens oder direkt nach "/" explizit oder mit {} getroffen werden, außer **-types hidden** ist gesetzt ("." = versteckt). Auf anderen Plattformen keine Sonderbehandlung, außer "." und ".." müssen explizit getroffen werden (verhindert Rekursion nach oben bei "glob -join * * * *"). Alle "/" müssen explizit gematcht werden.

**glob** unterscheidet sich vom csh-Globbing: keine Sortierung der Ergebnisliste (lsort bei Bedarf); **glob** liefert nur Namen existierender Dateien — csh prüft die Existenz nur bei ?, *, [] im Muster.

## Portabilität unter Windows

Bei Windows-UNC-Namen dürfen Server- und Freigabename keine ?, *, [] enthalten.

Da Backslash für glob speziell ist, brauchen Windows-Pfadmuster Sorgfalt. "_C:\\foo\*_" wird als "_C:\foo\*_" gelesen, "_\f_" matcht ein "_f_", "*" ein "*", nicht als Wildcard. Lösung: Unix-Schrägstrich als Trenner; Windows-Pfade mit "file join **$path**" oder "file normalize **$path**" umwandeln.

## Beispiele

Alle Tcl-Dateien im aktuellen Verzeichnis:

```
glob *.tcl
```

Alle Tcl-Dateien im Home-Verzeichnis des Benutzers, unabhängig vom aktuellen Verzeichnis:

```
glob -directory [file home] *.tcl
```

Alle Unterverzeichnisse des aktuellen Verzeichnisses:

```
glob -type d *
```

Alle Dateien, deren Name "a", "b" oder die Sequenz "cde" enthält:

```
glob -type f *{a,b,cde}*
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# global

*Stichwörter:* global, namespace, procedure, variable

## Name

global - Zugriff auf globale Variablen

## Übersicht

**global** ?*varname* ...?

## Beschreibung

Dieser Befehl hat keine Wirkung, sofern er nicht im Kontext eines Prozedurrumpfs ausgeführt wird. Wird **global** im Kontext eines Prozedurrumpfs ausgeführt, werden lokale Variablen erzeugt, die mit den entsprechenden globalen Variablen verknüpft sind (diese verknüpften Variablen erscheinen, wie bei upvar erzeugte, nicht in der von info locals zurückgegebenen Liste).

Enthält *varname* Namespace-Qualifizierer, ist der Name der lokalen Variable der unqualifizierte Name der globalen Variable, wie vom Befehl namespace tail bestimmt.

*varname* wird stets als Variablenname behandelt, nicht als Array-Element. Sieht der Name wie ein Array-Element aus, etwa **a(b)**, wird ein Fehler zurückgegeben.

## Beispiele

Diese Prozedur setzt die Namespace-Variable *::a::x*:

```
proc reset {} {
    global a::x
    set x 0
}
```

Diese Prozedur sammelt die übergebenen Zeichenketten in einem globalen Puffer, durch Zeilenumbrüche getrennt. Nützlich, wenn eine Nachricht stückweise aufgebaut werden soll (wie mit puts), aber als Ganzes versendet werden soll (z. B. über eine mit socket geöffnete Verbindung oder als Teil einer HTTP-Antwort mit Content-Length).

```
proc accum {string} {
    global accumulator
    append accumulator $string \n
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# history

*Stichwörter:* history, event, record

## Name

history - Manipuliert die Verlaufsliste

## Übersicht

**history** ?*option*? ?*arg arg* ...?

## Beschreibung

Der Befehl **history** führt eine von mehreren Operationen aus, die sich auf kürzlich ausgeführte Befehle beziehen, die in einer Verlaufsliste gespeichert sind. Jeder dieser gespeicherten Befehle wird als "Ereignis" bezeichnet. Beim Angeben eines Ereignisses für **history** können folgende Formen verwendet werden:

1. Eine Zahl: Ist sie positiv, bezieht sie sich auf das Ereignis mit dieser Nummer (alle Ereignisse werden ab 1 nummeriert). Ist die Zahl negativ, wählt sie ein Ereignis relativ zum aktuellen Ereignis (**-1** bezieht sich auf das vorherige Ereignis, **-2** auf das davor, und so weiter). Ereignis **0** bezeichnet das aktuelle Ereignis.
2. Eine Zeichenkette: Wählt das jüngste Ereignis, das zur Zeichenkette passt. Ein Ereignis gilt als passend, wenn die Zeichenkette mit den ersten Zeichen des Ereignisses übereinstimmt oder wenn die Zeichenkette im Sinne des Befehls string match zum Ereignis passt.

Der Befehl **history** kann eine der folgenden Formen annehmen:

**history**

Entspricht **history info**, siehe unten.

**history add *command* ?exec?**

Fügt das Argument *command* als neues Ereignis zur Verlaufsliste hinzu. Ist **exec** angegeben (oder abgekürzt), wird der Befehl auch ausgeführt und sein Ergebnis zurückgegeben. Ist **exec** nicht angegeben, wird als Ergebnis eine leere Zeichenkette zurückgegeben.

**history change *newValue* ?*event*?**

Ersetzt den für ein Ereignis gespeicherten Wert durch *newValue*. *Event* gibt das zu ersetzende Ereignis an und standardmäßig das *aktuelle* Ereignis (nicht Ereignis **-1**). Der Befehl ist für Befehle gedacht, die neue Formen der Verlaufssubstitution implementieren und das aktuelle Ereignis (das die Substitution auslöst) durch den durch Substitution erzeugten Befehl ersetzen wollen. Der Rückgabewert ist eine leere Zeichenkette.

**history clear**

Löscht die Verlaufsliste. Das aktuelle Behalte-Limit bleibt erhalten. Die Ereignisnummern werden zurückgesetzt.

**history event ?*event*?**

Gibt den Wert des durch *event* angegebenen Ereignisses zurück. *Event* standardmäßig **-1**.

**history info ?*count*?**

Gibt eine formatierte Zeichenkette zurück (für Menschen lesbar) mit Ereignisnummer und Inhalt für jedes Ereignis in der Verlaufsliste außer dem aktuellen. Ist *count* angegeben, werden nur die *count* jüngsten Ereignisse zurückgegeben.

**history keep ?*count*?**

Kann verwendet werden, um die Größe der Verlaufsliste auf *count* Ereignisse zu ändern. Anfangs werden 20 Ereignisse in der Verlaufsliste behalten. Ist *count* nicht angegeben, wird das aktuelle Behalte-Limit zurückgegeben.

**history nextid**

Gibt die Nummer des nächsten in der Verlaufsliste zu speichernden Ereignisses zurück. Nützlich etwa zum Anzeigen der Ereignisnummer in Kommandozeilen-Prompts.

**history redo ?*event*?**

Führt den durch *event* angegebenen Befehl erneut aus und gibt sein Ergebnis zurück. *Event* standardmäßig **-1**. Dieser Befehl führt zu einer Verlaufsrevision; siehe unten.

### Verlaufs-Revision

Tcl vor Version 8.0 hatte einen komplexen Mechanismus zur Verlaufsrevision. Der aktuelle Mechanismus ist eingeschränkter; die alten Verlaufsoperationen **substitute** und **words** wurden entfernt. (Als Trost wurde die Operation **clear** hinzugefügt.)

Die Verlaufsoption **redo** führt zu einer deutlich einfacheren "Verlaufsrevision". Wird diese Option aufgerufen, wird das jüngste Ereignis so geändert, dass der Befehl **history** entfernt und durch das Ergebnis des Befehls **history** ersetzt wird. Soll ein Ereignis ohne Änderung des Verlaufs erneut ausgeführt werden, verwenden Sie die Operation **event**, um ein Ereignis abzurufen, und die Operation **add**, um es zum Verlauf hinzuzufügen und auszuführen.

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

## http

*Stichwörter:* http, internet, security policy, socket, www

### Name

http - Clientseitige Implementierung des HTTP/1.1-Protokolls

### Übersicht

**package require http** ?**2.10**?

**::http::config** ?*-option value* ...?
**::http::geturl** *url* ?*-option value* ...?
**::http::formatQuery** *key value* ?*key value* ...?
**::http::quoteString** *value*
**::http::reset** *token* ?*why*?
**::http::wait** *token*
**::http::status** *token*
**::http::size** *token*
**::http::error** *token*
**::http::postError** *token*
**::http::cleanup** *token*
**::http::requestLine** *token*
**::http::requestHeaders** *token* ?*headerName*?
**::http::requestHeaderValue** *token headerName*
**::http::responseLine** *token*
**::http::responseCode** *token*
**::http::reasonPhrase** *code*
**::http::responseHeaders** *token* ?*headerName*?
**::http::responseHeaderValue** *token headerName*
**::http::responseInfo** *token*
**::http::responseBody** *token*
**::http::register** *proto port command* ?*socketCmdVarName*? ?*useSockThread*? ?*endToEndProxy*?
**::http::registerError** *sock* ?*message*?
**::http::unregister** *proto*
**::http::code** *token*

**::http::data** *token*
**::http::meta** *token* ?*headerName*?
**::http::metaValue** *token headerName*
**::http::ncode** *token*

## Exportierte Befehle

Der Namespace **http** exportiert die Befehle **config**, **formatQuery**, **geturl**, **postError**, **quoteString**, **reasonPhrase**, **register**, **registerError**, **requestHeaders**, **requestHeaderValue**, **requestLine**, **responseBody**, **responseCode**, **responseHeaders**, **responseHeaderValue**, **responseInfo**, **responseLine**, **reset**, **unregister** und **wait**.

Er exportiert nicht die Befehle **cleanup**, **code**, **data**, **error**, **meta**, **metaValue**, **ncode**, **size** oder **status**.

## Beschreibung

Das Paket **http** stellt die Client-Seite des HTTP/1.1-Protokolls bereit, wie in RFC 9110 bis 9112 definiert, die RFC 7230 bis RFC 7235 ablösen, die wiederum RFC 2616 ablösen. Das Paket implementiert die GET-, POST- und HEAD-Operationen von HTTP/1.1. Es ermöglicht die Konfiguration eines Proxy-Hosts, um Firewalls zu passieren. Das Paket ist mit der **Safesock** -Sicherheitsrichtlinie kompatibel und kann daher von nicht vertrauenswürdigen Applets zum Abrufen von URLs von einer eingeschränkten Menge von Hosts verwendet werden. Dieses Paket kann erweitert werden, um zusätzliche HTTP-Transportprotokolle wie HTTPS zu unterstützen, indem ein benutzerdefinierter socket-Befehl über **::http::register** bereitgestellt wird.

Die Prozedur **::http::geturl** führt eine HTTP-Transaktion aus. Ihre *Optionen* legen fest, ob eine GET-, POST- oder HEAD-Transaktion durchgeführt wird. Der Rückgabewert von **::http::geturl** ist ein Token für die Transaktion. Das Token kann als Argument an andere Befehle übergeben werden, um die Transaktion zu verwalten und ihre Ergebnisse zu prüfen.

Ist die Option **-command** angegeben, wird die HTTP-Operation im Hintergrund ausgeführt. **::http::geturl** kehrt unmittelbar nach dem Erzeugen der HTTP-Anfrage zurück, und die **-command**-Rückruffunktion wird aufgerufen, wenn die Transaktion abgeschlossen ist. Damit dies funktioniert, muss die Tcl-Ereignisschleife aktiv sein. In Tk-Anwendungen ist dies stets der Fall. In reinen Tcl-Anwendungen kann der Aufrufer nach dem Aufruf von **::http::geturl** **::http::wait** verwenden, um die Ereignisschleife zu starten.

**Hinweis:** Die Ereigniswarteschlange wird auch ohne die Option **-command** verwendet. Als Nebeneffekt können beliebige Befehle verarbeitet werden, während **http::geturl** läuft.

Hat der HTTP-Server auf die Anfrage geantwortet, rufen Sie den Befehl **::http::responseInfo** auf, der ein dict mit Metadaten zurückgibt, die für die Erkennung einer erfolgreichen Transaktion und die Nutzung der Antwort wesentlich sind. Details zu den zurückgegebenen Informationen finden Sie im Abschnitt Metadaten. Die Antwort selbst wird vom Befehl **::http::responseBody** zurückgegeben, sofern sie nicht durch die Option -*channel* von **::http::geturl** in eine Datei umgeleitet wurde.

## Befehle

**::http::config ?*options*?**

Der Befehl **::http::config** dient zum Setzen und Abfragen des Namens des Proxy-Servers und -Ports sowie des in den HTTP-Anfragen verwendeten User-Agent-Namens. Sind keine Optionen angegeben, wird die aktuelle Konfiguration zurückgegeben. Ist ein einzelnes Argument angegeben, sollte es eine der unten beschriebenen Flags sein; in diesem Fall wird der aktuelle Wert dieser Einstellung zurückgegeben. Andernfalls sollten die Optionen eine Menge von Flags und Werten sein, die die Konfiguration definieren:

**-accept *mimetypes***

Der Accept-Header der Anfrage. Der Standardwert ist */*, was bedeutet, dass alle Dokumenttypen akzeptiert werden. Andernfalls können Sie eine durch Kommas getrennte Liste von MIME-Typ-Mustern angeben, die Sie empfangen möchten. Zum Beispiel "image/gif, image/jpeg, text/*".

**-cookiejar *command***

Der Cookie-Speicher, den das Paket zur Verwaltung von HTTP-Cookies verwenden soll. command ist eine Befehlspräfix-Liste; wird die leere Liste (der Standardwert) verwendet, senden Anfragen keine Cookies und speichern keine aus Antworten. Der durch command angegebene Befehl muss, falls angegeben, dem unten beschriebenen Cookie-Jar-Protokoll entsprechen.

**-pipeline *boolean***

Legt fest, ob HTTP/1.1-Transaktionen auf einem persistenten Socket gepipelined werden. Siehe Abschnitt **PERSISTENTE SOCKETS** für Details. Der Standardwert ist 1.

**-postfresh *boolean***

Legt fest, ob Anfragen, die die Methode **POST** verwenden, stets einen frischen Socket nutzen und damit die Option **-keepalive** des Befehls **http::geturl** überschreiben. Siehe Abschnitt **PERSISTENTE SOCKETS** für Details. Der Standardwert ist 0.

**-proxyauth *string***

Ist der String nicht leer, wird er dem Proxy-Server als Wert des Anfrage-Headers Proxy-Authorization übergeben. Diese Option kann für HTTP Basic Authentication verwendet werden. Erfordert der Proxy-Server eine Authentifizierung mit einer anderen Technik, z. B. Digest Authentication, ist die Option **-proxyauth** nicht nützlich. In diesem Fall muss der Aufrufer mit einer 407-Antwort des Proxys rechnen, den bereitzustellenden Authentifizierungswert berechnen und über die Option **-headers** als Wert des Headers Proxy-Authorization liefern.

**-proxyfilter *command***

Der Befehl ist eine Rückruffunktion, die während **::http::geturl** aufgerufen wird, um zu ermitteln, ob für einen bestimmten Host ein Proxy erforderlich ist. Beim Aufruf wird dem *command* ein Argument, ein Hostname, angehängt. Ist ein Proxy erforderlich, soll die Rückruffunktion eine zweielementige Liste mit Proxy-Server und Proxy-Port zurückgeben. Andernfalls soll der Filterbefehl eine leere Liste zurückgeben.

Der Standardwert von **-proxyfilter** ist **http::ProxyRequired**; dieser Befehl gibt die Werte der Einstellungen **-proxyhost** und **-proxyport** zurück, sofern sie nicht leer sind. Die Optionen **-proxyhost**, **-proxyport** und **-proxynot** werden nur von **http::ProxyRequired** verwendet und sonst nirgends in **::http::geturl**. Ein benutzerdefinierter **-proxyfilter** -Befehl kann diese Optionen nutzen oder Werte aus anderer Stelle im aufrufenden Skript beziehen. Im letzteren Fall bleiben alle für **-proxyhost**, **-proxyport** und **-proxynot** angegebenen Werte ungenutzt.

Der Befehl **::http::geturl** führt die **-proxyfilter**-Rückruffunktion innerhalb eines catch

-Befehls aus. Ein Fehler in der Rückruffunktion ruft daher nicht den bgerror-Handler auf. Siehe Abschnitt Fehler für Details.

**-proxyhost *hostname***

Der Hostname oder die IP-Adresse des Proxy-Servers, falls vorhanden. Ist dieser Wert die leere Zeichenkette, wird der URL-Host direkt kontaktiert. Siehe **-proxyfilter** zur Verwendung des Werts.

**-proxynot *list***

Eine Tcl-Liste von Domänennamen und IP-Adressen, die direkt und nicht über den Proxy-Server erreicht werden sollen. Der Ziel-Hostname wird mit jedem Listenelement per string match ohne Berücksichtigung der Groß-/Kleinschreibung verglichen. Oft ist es praktisch, das Platzhalterzeichen "*" am Anfang eines Domänennamens (z. B. *.example.com) oder am Ende einer IP-Adresse (z. B. 192.168.0.*) zu verwenden. Siehe **-proxyfilter** zur Verwendung des Werts.

**-proxyport *number***

Die Portnummer des Proxy-Servers. Siehe **-proxyfilter** zur Verwendung des Werts.

**-repost *boolean***

Legt fest, was zu tun ist, wenn eine POST-Anfrage über eine persistente Verbindung fehlschlägt, weil der Server die Verbindung halb geschlossen hat. Ist der boolesche Wert **true**, wird die Anfrage automatisch wiederholt; ist er **false**, nicht, und die Anwendung, die **http::geturl** verwendet, soll vor einer erneuten POST-Anfrage eine Bestätigung des Benutzers einholen. Der Wert **true** sollte nur unter bestimmten Bedingungen verwendet werden. Siehe Abschnitt **PERSISTENTE SOCKETS** für Details. Der Standardwert ist 0.

**-threadlevel *level***

Legt fest, ob und wie das Paket **Thread** verwendet wird. Mögliche Werte von *level* sind 0, 1 oder 2.

**0**

(Standard) Thread nicht verwenden

**1**

Thread verwenden, falls verfügbar; andernfalls nicht verwenden

**2**

Thread verwenden, falls verfügbar; andernfalls Fehler auslösen

Der Tcl-Befehl socket -async kann in ungünstigen Fällen blockieren (z. B. bei einer langsamen DNS-Abfrage). Die Verwendung des Thread-Pakets umgeht dieses Problem sowohl bei HTTP- als auch bei HTTPS-Transaktionen. Werte von *level* ungleich 0 stehen nur dem Haupt-Interpreter in jedem Thread zur Verfügung. Siehe Abschnitt Threads für weitere Informationen.

**-urlencoding *encoding***

Die *encoding*, die zum Erzeugen x-url-kodierter URLs mit **::http::formatQuery** und **::http::quoteString** verwendet wird. Der Standardwert ist **utf-8**, wie in RFC 2718 festgelegt.

**-useragent *string***

Der Wert des User-Agent-Headers in der HTTP-Anfrage. In einem unsicheren Interpreter hängt der Standardwert vom Betriebssystem sowie von den Versionsnummern von **http** und **Tcl** ab und lautet (zum Beispiel) "**Mozilla/5.0 (Windows; U; Windows NT 10.0)**

**http/2.10.3 Tcl/9.0.4**". Ein sicherer Interpreter kann sein Betriebssystem nicht ermitteln; der Standardwert in einem sicheren Interpreter ist daher ein Windows-10-Wert mit den aktuellen Versionsnummern von **http** und **Tcl**.

**-zip *boolean***

Ist der Wert boolesch **true**, senden Anfragen standardmäßig einen Header " **Accept-Encoding: gzip,deflate**". Ist der Wert boolesch **false**, senden Anfragen standardmäßig einen Header "**Accept-Encoding: identity**". In beiden Fällen kann der Standardwert für eine einzelne Anfrage überschrieben werden, indem ein benutzerdefinierter **Accept-Encoding**-Header in der Option **-headers** von **http::geturl** angegeben wird. Der Standardwert ist 1.

**::http::geturl *url ?options*?**

Der Befehl **::http::geturl** ist die Hauptprozedur im Paket. Die Option **-query** oder **-querychannel** bewirkt eine POST-Operation, die Option **-validate** eine HEAD-Operation; andernfalls wird eine GET-Operation ausgeführt. Der Befehl **::http::geturl** gibt einen *token* -Wert zurück, der als Argument an andere Befehle übergeben werden kann, um Informationen über die Transaktion zu erhalten. Siehe die Abschnitte Metadaten und Fehler für Details. Der Befehl **::http::geturl** blockiert, bis die Operation abgeschlossen ist, es sei denn, die Option **-command** gibt eine Rückruffunktion an, die aufgerufen wird, wenn die HTTP-Transaktion abgeschlossen ist. **::http::geturl** unterstützt mehrere Optionen:

**-binary *boolean***

Legt fest, ob die URL-Daten zwangsweise als Binärdaten interpretiert werden. Normalerweise erfolgt dies automatisch (alles, was nicht mit einem **text**-Content-Type beginnt oder dessen Content-Encoding **gzip** oder **deflate** ist, gilt als Binärdaten).

**-blocksize *size***

Die Blockgröße beim Lesen der URL. Es werden höchstens *size* Bytes auf einmal gelesen. Nach jedem Block wird die **-progress**-Rückruffunktion aufgerufen (falls diese Option angegeben ist).

**-channel *name***

Kopiert den URL-Inhalt in den Kanal *name*, statt ihn in einer Tcl-Variable für den Abruf durch **::http::responseBody** zu speichern.

**-command *callback***

Ist diese Option gesetzt, kehrt **::http::geturl** unmittelbar zurück. Nach Abschluss der HTTP-Transaktion wird *callback* expandiert, ein zusätzliches Argument angehängt und der resultierende Befehl ausgewertet. Das zusätzliche Argument ist der von **::http::geturl** zurückgegebene *token*. Dieser Token ist der Name eines Arrays, das im Abschnitt Status-Array beschrieben ist. Hier ist eine Vorlage für die Rückruffunktion:

```
proc httpCallback {token} {
    upvar 0 $token state
    # Access state as a Tcl array defined in this proc
    ...
    return
}
```

Der Befehl **::http::geturl** führt die **-command**-Rückruffunktion innerhalb eines catch

-Befehls aus. Ein Fehler in der Rückruffunktion ruft daher nicht den bgerror-Handler auf. Siehe Abschnitt Fehler für Details.

**-guesstype *boolean***

Versucht, **Content-Type** und Zeichensatz zu erraten, wenn ein falsch konfigurierter Server keine Angaben liefert. Der Standardwert ist *false* (nichts tun). Ist der boolesche Wert *true* und sendet der Server keinen **Content-Type**-Header oder sendet er den Wert "application/octet-stream", versucht **http::geturl**, passende Werte zu erraten. Dies ist nicht als allgemeines Werkzeug gedacht und derzeit auf die Erkennung von XML-Dokumenten beschränkt, die mit einer XML-Deklaration beginnen. In diesem Fall wird **Content-Type** auf "application/xml" geändert, die Binär-Flag state(binary) auf 0 gesetzt und der Zeichensatz auf den in der XML-Zeile angegebenen Wert des Tags "encoding" oder auf utf-8, falls keine Kodierung angegeben ist. Wird nicht verwendet, wenn **-channel** angegeben ist.

**-handler *callback***

Fehlt diese Option, verarbeitet **http::geturl** eingehende Daten selbst, entweder durch Anhängen an die Variable state(body) oder durch Schreiben in den -channel. Ist die Option **-handler** gesetzt, führt **http::geturl** diese Verarbeitung nicht durch; stattdessen wird *callback* aufgerufen. Sobald HTTP-Daten verfügbar sind, wird *callback* expandiert, zwei zusätzliche Argumente angehängt und der resultierende Befehl ausgewertet. Die beiden zusätzlichen Argumente sind: der Socket für die HTTP-Daten und der von **::http::geturl** zurückgegebene *token*. Der Token ist der Name eines globalen Arrays, das im Abschnitt Status-Array beschrieben ist. Die Prozedur soll die Anzahl der vom Socket gelesenen Bytes zurückgeben. Hier ist eine Vorlage für die Rückruffunktion:

```
proc httpHandlerCallback {socket token} {
    upvar 0 $token state
    # Access socket, and state as a Tcl array defined in this proc
    # For example...
    ...
    set data [read $socket 1000]
    set nbytes [string length $data]
    ...
    return $nbytes
}
```

Der Code von **http::geturl** für die Option **-handler** ist weder mit Kompression noch mit Chunked Transfer-Encoding kompatibel. Ist **-handler** angegeben, setzt **http::geturl** zur Umgehung dieser Probleme das HTTP-Protokoll auf 1.0 herab und überschreibt die Option **-zip** (d. h. es sendet den Header **Accept-Encoding: identity** statt **Accept-Encoding: gzip,deflate**).

Werden die Optionen **-handler** und **-channel** gemeinsam verwendet, ist der Handler dafür verantwortlich, die Daten vom HTTP-Socket in den angegebenen Kanal zu kopieren. Der Kanalname ist dem Handler als Element **-channel** des Token-Arrays verfügbar.

Der Befehl **::http::geturl** führt die **-handler**-Rückruffunktion innerhalb eines catch -Befehls aus. Ein Fehler in der Rückruffunktion ruft daher nicht den bgerror-Handler auf. Siehe Abschnitt Fehler für Details.

**-headers *keyvaluelist***

Diese Option fügt der HTTP-Anfrage Header hinzu, die nicht bereits durch **::http::config** festgelegt sind. Das Argument *keyvaluelist* muss eine Liste mit einer geraden Anzahl von Elementen sein, die zwischen Schlüsseln und Werten wechseln. Die Schlüssel werden zu Header-Feldnamen. Zeilenumbrüche werden aus den Werten entfernt, damit der Header nicht beschädigt wird. Ist *keyvaluelist* z. B. **Pragma no-cache**, wird folgender Header in die HTTP-Anfrage aufgenommen:

```
Pragma: no-cache
```

**-keepalive *boolean***

Ist der Wert boolesch **true**, wird versucht, die Verbindung für die Bearbeitung mehrerer Anfragen offen zu halten. Der Standardwert ist 0.

**-method *type***

Erzwingt die HTTP-Anfragemethode *type*. **::http::geturl** wählt GET, POST oder HEAD anhand anderer Optionen automatisch; diese Option überschreibt die Auswahl und ermöglicht z. B. PUT und DELETE für WebDAV-Unterstützung.

Es liegt in der Verantwortung des Aufrufers, dass Header und Anfragekörper (falls vorhanden) den Anforderungen der Anfragemethode entsprechen. Wird z. B. **-method** *POST* verwendet, um eine POST-Anfrage mit leerem Anfragekörper zu senden, muss der Aufrufer außerdem die Option

```
-headers {Content-Length 0}
```

**-myaddr *address***

Übergibt eine bestimmte lokale Adresse an den zugrunde liegenden socket-Aufruf, falls mehrere Schnittstellen verfügbar sind.

**-progress *callback***

Ist die Option **-progress** gesetzt, wird *callback* nach jedem Datenübertrag vom URL aufgerufen. *callback* wird expandiert, drei zusätzliche Argumente angehängt und der resultierende Befehl ausgewertet. Die drei zusätzlichen Argumente sind: der von **::http::geturl** zurückgegebene *token*, die erwartete Gesamtgröße des Inhalts aus dem Antwort-Header **Content-Length** und die bisher übertragene Byteanzahl. Der Token ist der Name eines globalen Arrays, das im Abschnitt Status-Array beschrieben ist. Die erwartete Gesamtgröße kann unbekannt sein; dann wird 0 an die Rückruffunktion übergeben. Hier ist eine Vorlage für die Fortschritts-Rückruffunktion:

```
proc httpProgress {token total current} {
    upvar 0 $token state
    # Access state as a Tcl array defined in this proc
    ...
    return
}
```

**-protocol *version***

Wählt die zu verwendende HTTP-Protokollversion. Dies sollte 1.0 oder 1.1 sein (Standard). Nur bei Servern nötig, die HTTP/1.1 nicht verstehen oder sich anderweitig beschweren.

**-query** ***query***

Dieses Flag (bei nicht leerem Wert) bewirkt, dass **::http::geturl** eine POST-Anfrage ausführt, die die Zeichenkette *query* unverändert als Anfrage-Nutzlast an den Server übergibt. Das Format (und die Kodierung) von *query* wird durch den Anfrage-Header **Content-Type** angekündigt, der durch die Option **-type** gesetzt wird. Jeder Wert von **-type** ist zulässig; es liegt in der Verantwortung des Aufrufers, *query* im richtigen Format zu liefern.

Ist **-type** nicht angegeben, gilt standardmäßig *application/x-www-form-urlencoded*, was erfordert, dass *query* eine x-url-kodierte Query-Zeichenkette ist (dieses **-type** und Query-Format werden bei einem aus einem HTML-Formular abgesendeten POST verwendet). Die Prozedur **::http::formatQuery** kann für die Formatierung verwendet werden.

**-queryblocksize** ***size***

Die Blockgröße beim Senden von Query-Daten an die URL. Es werden höchstens *size* Bytes auf einmal geschrieben. Nach jedem Block wird die **-queryprogress**-Rückruffunktion aufgerufen (falls diese Option angegeben ist).

**-querychannel** ***channelID***

Dieses Flag bewirkt, dass **::http::geturl** eine POST-Anfrage ausführt, die die in *channelID* enthaltenen Daten an den Server übergibt. Die Daten in *channelID* müssen eine x-url-kodierte Query sein, sofern die Option **-type** unten nicht verwendet wird. Ist kein **Content-Length**-Header über die Option **-headers** angegeben, versucht **::http::geturl**, die Größe der POST-Daten zu ermitteln, um diesen Header zu erzeugen. Kann die Größe nicht ermittelt werden, gibt es einen Fehler zurück.

**-queryprogress** ***callback***

Ist die Option **-queryprogress** gesetzt, wird *callback* nach jedem Datenübertrag an die URL in einer POST-Anfrage aufgerufen (d. h. bei einem Aufruf von **::http::geturl** mit Option **-query** oder **-querychannel**) und verhält sich genau wie die Option **-progress** (das Rückrufformat ist dasselbe).

**-strict** ***boolean***

Ist der Wert wahr, prüft der Befehl, ob die URL RFC 3986 entspricht, d. h. keine Zeichen enthält, die x-url-kodiert werden müssten (z. B. muss ein Leerzeichen als "%20" kodiert sein). Der Standardwert ist 1.

**-timeout** ***milliseconds***

Ist *milliseconds* ungleich null, richtet **::http::geturl** einen Timeout nach der angegebenen Anzahl Millisekunden ein. Ein Timeout führt zu einem Aufruf von **::http::reset** und zur **-command**-Rückruffunktion, falls angegeben. Der Rückgabewert von **::http::status** (und der Wert des Schlüssels *status* im von **::http::responseInfo** zurückgegebenen Dictionary) ist **timeout**, nachdem ein Timeout aufgetreten ist.

**-type** ***mime-type***

Verwendet *mime-type* als Wert von **Content-Type** statt des Standardwerts ( **application/x-www-form-urlencoded**) während einer POST-Operation.

**-validate** ***boolean***

Ist *boolean* ungleich null, führt **::http::geturl** eine HTTP-HEAD-Anfrage aus. Der Server liefert dieselbe Statuszeile und dieselben Antwort-Header wie bei einer HTTP-GET-Anfrage, lässt aber die Antwort-Entität (den URL-"Inhalt") weg. Die Antwort-Header sind nach der Transaktion mit dem Befehl **::http::responseHeaders** oder für ausgewählte Informationen mit **::http::responseInfo** verfügbar.

**::http::formatQuery *key value ?key value ...?***

Diese Prozedur führt x-url-Kodierung von Query-Daten durch. Sie erwartet eine gerade Anzahl von Argumenten als Schlüssel und Werte der Query. Sie kodiert Schlüssel und Werte und erzeugt eine Zeichenkette mit den passenden &- und =-Trennzeichen. Das Ergebnis eignet sich als Wert für **-query** bei **::http::geturl**.

**::http::quoteString *value***

Diese Prozedur führt x-url-Kodierung einer Zeichenkette durch. Sie erwartet ein einzelnes Argument und kodiert es.

**::http::reset *token* ?*why*?**

Dieser Befehl setzt die durch *token* identifizierte HTTP-Transaktion zurück, falls vorhanden. Dabei wird **state(status)** auf *why* gesetzt, standardmäßig **reset**, und die registrierte **-command**-Rückruffunktion aufgerufen.

**::http::wait *token***

Dieser Befehl blockiert und wartet auf den Abschluss der Transaktion. Dies funktioniert nur in vertrauenswürdigem Code, weil vwait verwendet wird. Außerdem ist es nicht nützlich, wenn **::http::geturl** *ohne* die Option **-command** aufgerufen wird, da in diesem Fall der Aufruf von **::http::geturl** erst zurückkehrt, wenn die HTTP-Transaktion abgeschlossen ist, und es nichts zu erwarten gibt.

**::http::status *token***

Dieser Befehl gibt eine Beschreibung des Status der HTTP-Transaktion zurück. Der Rückgabewert ist die leere Zeichenkette, bis die HTTP-Transaktion abgeschlossen ist; danach hat er einen der Werte ok, eof, error, timeout und reset. Die Bedeutung dieser Werte wird im Abschnitt Fehler (unten) beschrieben.

Der Name "status" steht in keinem Zusammenhang mit den Begriffen "status line" und "status code", die für eine HTTP-Antwort definiert sind.

**::http::size *token***

Dieser Befehl gibt die Anzahl der bisher vom URL im Aufruf von **::http::geturl** empfangenen Bytes zurück.

**::http::error *token***

Dieser Befehl gibt die Fehlerinformationen zurück, wenn die HTTP-Transaktion fehlgeschlagen ist, oder die leere Zeichenkette, wenn kein Fehler aufgetreten ist. Die Information ist eine Tcl-Liste aus Fehlermeldung, Stack-Trace und Fehlercode.

**::http::postError *token***

Eine POST-Anfrage ist ein Aufruf von **::http::geturl** mit der Option **-query** oder **-querychannel**. Der Befehl **::http::postError** gibt die Fehlerinformationen zurück, die entstehen, wenn eine HTTP-POST-Anfrage ihren Anfragekörper an den Server sendet; oder die leere Zeichenkette, wenn kein Fehler aufgetreten ist. Die Information ist eine Tcl-Liste aus Fehlermeldung, Stack-Trace und Fehlercode. Tritt dieser Fehlertyp auf, setzt der Befehl **::http::geturl** die Transaktion fort und versucht, eine Antwort vom Server zu empfangen.

**::http::cleanup *token***

Diese Prozedur bereinigt den mit der durch *token* identifizierten Verbindung verknüpften Zustand. Nach diesem Aufruf können Befehle wie **::http::responseBody** keine Informationen mehr über die Operation liefern. Es wird *dringend* empfohlen, diese Funktion aufzurufen, wenn eine HTTP-Anfrage abgeschlossen ist. Andernfalls wird Speicher nicht freigegeben; ruft Ihre Anwendung **::http::geturl** oft genug auf, kann das Speicherleck die Leistung beeinträchtigen ... oder schlimmer.

**::http::requestLine *token***

Dieser Befehl gibt die an den Server gesendete "request line" zurück. Die "request line" ist die erste Zeile einer HTTP-Client-Anfrage und hat drei durch Leerzeichen getrennte Elemente: die HTTP-Methode, die URL relativ zum Server und die HTTP-Version. Beispiele:

GET / HTTP/1.1 GET /introduction.html?subject=plumbing HTTP/1.1 POST /forms/order.html HTTP/1.1

**::http::requestHeaders *token ?headerName?***

Dieser Befehl gibt die Namen und Werte der HTTP-Anfrage-Header in der Reihenfolge zurück, in der sie an den Server gesendet wurden, als Tcl-Liste der Form ?name value ...? Header-Namen sind nicht case-sensitiv und werden in Kleinbuchstaben umgewandelt. Der Rückgabewert ist kein dict, weil einige Header-Namen mehrfach vorkommen können. Wird das zweite Argument nicht übergeben, werden alle Anfrage-Header zurückgegeben. Wird es übergeben, liefert es den Wert eines Header-Namens. Es werden nur Header mit dem angeforderten Namen (in Kleinbuchstaben) zurückgegeben. Werden keine solchen Header gefunden, wird eine leere Liste zurückgegeben.

**::http::requestHeaderValue *token headerName***

Dieser Befehl gibt den Wert des HTTP-Anfrage-Headers *headerName* zurück. Header-Namen sind nicht case-sensitiv und werden in Kleinbuchstaben umgewandelt. Existiert kein solcher Header, ist der Rückgabewert die leere Zeichenkette. Gibt es mehrere Header mit dem Namen *headerName*, wird das Ergebnis durch Verknüpfen der einzelnen Werte mit der Zeichenkette ", " (Komma und Leerzeichen) erzeugt, wobei die Reihenfolge erhalten bleibt.

**::http::responseLine *token***

Dieser Befehl gibt die erste Zeile der Serverantwort zurück: die HTTP-"status line". Die "status line" hat drei durch Leerzeichen getrennte Elemente: die HTTP-Version, einen dreistelligen numerischen "status code" und eine "reason phrase". Nur die reason phrase darf Leerzeichen enthalten. Beispiele:

HTTP/1.1 200 OK HTTP/1.0 404 Not Found Der "status code" ist eine dreistellige Zahl im Bereich 100 bis 599. Ein Wert von 200 ist die normale Rückgabe einer GET-Anfrage; die zugehörige "reason phrase" ist "OK". Codes, die mit 4 oder 5 beginnen, weisen auf Fehler hin. Codes, die mit 3 beginnen, sind Weiterleitungsfehler. In diesem Fall gibt der Antwort-Header **Location** eine neue URL an, die die angeforderten Informationen enthält.

Die "reason phrase" ist eine textuelle Beschreibung des "status code": sie kann von Server zu Server variieren und kann ohne Auswirkung auf das HTTP-Protokoll geändert werden. Die empfohlenen Werte (RFC 7231 und IANA-Zuweisungen) für jeden Code liefert der Befehl **::http::reasonPhrase**.

**::http::responseCode *token***

Dieser Befehl gibt den "status code" (200, 404 usw.) der Server-"status line" zurück. Kann kein dreistelliger Code gefunden werden, wird die vollständige Statuszeile zurückgegeben. Siehe Befehl **::http::responseLine** für weitere Informationen zur "status line".

**::http::reasonPhrase *code***

Dieser Befehl gibt die von der IANA empfohlene "reason phrase" für einen bestimmten von einem HTTP-Server zurückgegebenen "status code" zurück. Das Argument *code* ist ein gültiger Statuscode und damit eine Ganzzahl im Bereich 100 bis 599 einschließlich. Für Zahlen in diesem Bereich ohne zugewiesene Bedeutung gibt der Befehl den Wert "Unassigned" zurück. Mehrere Statuscodes werden nur als Antwort auf die von HTTP-Erweiterungen wie WebDAV definierten Methoden verwendet und nicht als Antwort auf HEAD, GET oder POST.

Die von einem HTTP-Server zurückgegebene "reason phrase" kann vom empfohlenen Wert abweichen, ohne das HTTP-Protokoll zu beeinträchtigen. Der von **::http::geturl** gelieferte Wert kann durch Aufruf von **::http::responseLine** (vollständige Statuszeile) oder **::http::responseInfo** (Dictionary mit der "reason phrase" im Schlüssel *reasonPhrase*) ermittelt werden.

Ein Register gültiger Statuscodes wird geführt unter https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml

**::http::responseHeaders *token* ?*headerName*?**

Die Antwort eines HTTP-Servers enthält Metadaten-Header, die den Antwortkörper und die Transaktion selbst beschreiben. Dieser Befehl gibt die Namen und Werte der HTTP-Antwort-Header in der Reihenfolge zurück, in der sie vom Server empfangen wurden, als Tcl-Liste der Form ?name value ...? Header-Namen sind nicht case-sensitiv und werden in Kleinbuchstaben umgewandelt. Der Rückgabewert ist kein dict, weil einige Header-Namen mehrfach vorkommen können, insbesondere **Set-Cookie**. Wird das zweite Argument nicht übergeben, werden alle Antwort-Header zurückgegeben. Wird es übergeben, liefert es den Wert eines Header-Namens. Es werden nur Header mit dem angeforderten Namen (in Kleinbuchstaben) zurückgegeben. Werden keine solchen Header gefunden, wird eine leere Liste zurückgegeben. Siehe Abschnitt Metadaten für weitere Informationen.

**::http::responseHeaderValue *token headerName***

Dieser Befehl gibt den Wert des HTTP-Antwort-Headers *headerName* zurück. Header-Namen sind nicht case-sensitiv und werden in Kleinbuchstaben umgewandelt. Existiert kein solcher Header, ist der Rückgabewert die leere Zeichenkette. Gibt es mehrere Header mit dem Namen *headerName*, wird das Ergebnis durch Verknüpfen der einzelnen Werte mit der Zeichenkette ", " (Komma und Leerzeichen) erzeugt, wobei die Reihenfolge erhalten bleibt. Mehrere Header mit demselben Namen können so verarbeitet werden, außer **Set-Cookie**, das der Syntax einer durch Kommas getrennten Liste nicht entspricht und nicht zu einem einzelnen Wert zusammengefasst werden kann. Jeder **Set-Cookie**-Header muss einzeln verarbeitet werden, z. B. durch Auswertung des Rückgabewerts von **::http::responseHeaders** *token* **Set-Cookie**.

**::http::responseInfo *token***

Dieser Befehl gibt ein dict aus ausgewählten Antwort-Metadaten zurück, die für die Erkennung einer erfolgreichen Transaktion und die Nutzung der Antwort wesentlich sind, sowie weitere informative Metadaten. Die Schlüssel des dict sind *stage*, *status*, *responseCode*, *reasonPhrase*, *contentType*, *binary*, *redirection*, *upgrade*, *error*, *postError*, *method*, *charset*, *compression*, *httpRequest*, *httpResponse*, *url*, *connectionRequest*, *connectionResponse*, *connectionActual*, *transferEncoding*, *totalPost*, *currentPost*, *totalSize* und *currentSize*. Die Bedeutung dieser Schlüssel wird im Abschnitt Metadaten unten beschrieben.

Es lohnt sich stets, nach einer HTTP-Transaktion den Wert von *binary* zu prüfen, um festzustellen, ob ein falsch konfigurierter Server **http** veranlasst hat, eine Textressource als Binärdaten oder umgekehrt zu interpretieren.

Nach einer POST-Transaktion prüfen Sie den Wert von *postError*, um sicherzustellen, dass der Anfragekörper fehlerfrei hochgeladen wurde.

**::http::responseBody *token***

Dieser Befehl gibt die vom HTTP-Server gesendete Entität zurück (es sei denn, **-channel** wurde verwendet; dann wurde die Entität an den Kanal geliefert und der Befehl gibt die leere Zeichenkette zurück).

Weitere Begriffe für "entity" mit unterschiedlicher Genauigkeit sind "Repräsentation der Ressource", "Ressource", "Antwortkörper nach Dekodierung", "Nutzlast", "Nachrichtenkörper nach Dekodierung", "Inhalt(e)" und "Datei".

**::http::register *proto port command* ?*socketCmdVarName*? ?*useSockThread*? ?*endToEndProxy*?**

Diese Prozedur ermöglicht benutzerdefinierte HTTP-Transporttypen wie HTTPS durch Registrierung eines Präfixes, des Standardports und des Befehls zum Erzeugen des Tcl-**channel**. Die optionalen Argumente konfigurieren, wie **http** den benutzerdefinierten Transport nutzt, und haben Standardwerte, die mit älteren Versionen von **http** kompatibel sind, in denen **::http::register** keine optionalen Argumente hat.

Das Argument *socketCmdVarName* ist der Name einer vom Transport bereitgestellten Variablen, deren Wert der Befehl ist, mit dem der Transport einen Socket öffnet. Sein Standardwert wird vom Transport gesetzt und lautet "::socket"; wird der Variablenname an **::http::register** übergeben, setzt **http** einen neuen Wert, um optionale Einrichtungen verfügbar zu machen. Diese werden durch die optionalen Argumente *useSockThread* und *endToEndProxy* aktiviert, die boolesche Werte mit Standardwert *false* annehmen.

Ist das Argument *useSockThread* angegeben und boolesch *true*, und erlaubt es der Wert von [ **http::config** *-threadlevel*] sowie die Verfügbarkeit des Pakets **Thread**, werden für den Transport erzeugte Sockets in einem anderen Thread geöffnet, damit eine langsame DNS-Abfrage das Skript nicht blockiert.

Ist das Argument *endToEndProxy* angegeben und boolesch *true*, öffnet **http::geturl** beim Zugriff auf einen Server über einen Proxy einen Kanal durch Senden einer CONNECT-Anfrage an den Proxy und stellt die eigentliche Anfrage über diesen Kanal. Das ermöglicht Ende-zu-Ende-Verschlüsselung für HTTPS-Anfragen über einen Proxy.

Zum Beispiel,

```
package require http
package require tls

::http::register https 443 ::tls::socket ::tls::socketCmd 1 1

set token [::http::geturl https://my.secure.site/]
```

**::http::registerError *sock* ?*message*?**

Diese Prozedur ermöglicht es einem registrierten Protokoll-Handler, eine Fehlermeldung für **http** bereitzustellen. Der Aufruf dieses Befehls löst keinen Fehler aus. Der Befehl ist nützlich, wenn ein registrierter Protokoll-Handler ein Problem erkennt (z. B. ein ungültiges TLS-Zertifikat), das einen Fehler bis zu **http** durchreichen lässt. Der Befehl ermöglicht **http**, eine präzise Fehlermeldung statt einer allgemeinen zu liefern. Der Befehl gibt den Wert des letzten Aufrufs mit Argument *message* zurück oder die leere Zeichenkette, wenn kein solcher Aufruf erfolgt ist.

**::http::unregister *proto***

Diese Prozedur hebt die Registrierung eines zuvor über **::http::register** registrierten Protokoll-Handlers auf und gibt eine sechselementige Liste der zuvor an **::http::register** übergebenen Werte zurück, falls ein solcher Handler existierte; andernfalls einen Fehler.

**::http::code *token***

Ein alternativer Name für den Befehl **::http::responseLine**

**::http::data *token***

Ein alternativer Name für den Befehl **::http::responseBody**.

**::http::meta *token* ?*headerName*?**

Ein alternativer Name für den Befehl **::http::responseHeaders**

**::http::ncode *token***

Ein alternativer Name für den Befehl **::http::responseCode**

## Fehler

Die Prozedur **::http::geturl** löst in folgenden Fällen Fehler aus: ungültige Befehlszeilenoptionen oder eine ungültige URL. Diese Fehler bedeuten, dass die Netzwerktransaktion gar nicht erst gestartet werden kann. Bei synchronen Aufrufen von **::http::geturl** (wenn **-command** nicht angegeben ist) wird ein Fehler ausgelöst, wenn die URL auf einem nicht existierenden Host liegt oder auf einem bestehenden Host an einem ungültigen Port. Es wird auch ein Fehler bei I/O-Fehlern beim Schreiben der HTTP-Anfragezeile und -Header oder beim Lesen der HTTP-Antwort-Header oder -Daten ausgelöst. Da **::http::geturl** in diesen Fällen keinen Token zurückgibt, führt es die erforderliche Bereinigung selbst durch; die Anwendung muss **::http::cleanup** nicht aufrufen.

Bei asynchronen Aufrufen von **::http::geturl** gelten alle obigen Fehlersituationen, außer dass beim Lesen der HTTP-Antwort-Header oder -Daten keine Ausnahme geworfen wird. Nach dem Schreiben der HTTP-Header kehrt **::http::geturl** zurück; der Rest der HTTP-Transaktion läuft im Hintergrund. Die Befehls-Rückruffunktion kann mit **::http::responseInfo** prüfen, ob beim Lesen ein Fehler aufgetreten ist.

Alternativ kann der Hauptprogrammfluss an einer Stelle **::http::wait** aufrufen und dann Status und Fehler prüfen, wie beim synchronen Aufruf.

Der Befehl **::http::geturl** führt die Rückruffunktionen **-command**, **-handler** und **-proxyfilter** innerhalb eines catch-Befehls aus. Ein Fehler in der Rückruffunktion ruft daher nicht den bgerror -Handler auf. Beim Debuggen einer dieser Rückruffunktionen kann es praktisch sein, Fehler innerhalb der Rückruffunktion selbst mit einem catch-Befehl zu melden, z. B. um eine Fehlermeldung nach stdout zu schreiben.

In jedem Fall müssen Sie dennoch **::http::cleanup** aufrufen, um das Status-Array zu löschen, wenn Sie fertig sind.

Weitere mögliche Ergebnisse der HTTP-Transaktion ergeben sich aus dem Status von **::http::status** (oder dem Wert des Schlüssels *status* im von **::http::responseInfo** zurückgegebenen Dictionary). Diese werden unten beschrieben.

**ok**

Schließt die HTTP-Transaktion vollständig ab, ist der Status **ok**. Sie sollten dennoch den Wert von **::http::responseLine** prüfen, um den HTTP-Status zu erhalten. Die Prozedur **::http::responseCode** liefert nur den numerischen Code (z. B. 200, 404 oder 500), während **::http::responseLine** einen Wert wie "HTTP 404 Datei nicht gefunden" zurückgibt.

eof

Schließt der Server den Socket ohne Antwort, wird kein Fehler ausgelöst, aber der Status der Transaktion ist eof.

**error**

Fehlermeldung, Stack-Trace und Fehlercode sind über **::http::error** zugänglich. Die Fehlermeldung wird auch durch den Wert des Schlüssels *error* im von **::http::responseInfo** zurückgegebenen Dictionary bereitgestellt.

**timeout**

Ein Timeout ist aufgetreten, bevor die Transaktion abgeschlossen werden konnte.

**reset**

Der Benutzer hat **::http::reset** aufgerufen.

**""**

(leere Zeichenkette) Die Transaktion ist noch nicht abgeschlossen.

Eine weitere Fehlermöglichkeit ist, dass **::http::geturl** den gesamten POST-Anfragekörper ( **-query**- oder **-querychannel**-Daten) nicht an den Server schreiben konnte. **::http::geturl** speichert die Fehlermeldung für den späteren Abruf durch **::http::postError** oder **::http::responseInfo** und versucht dann, die Transaktion abzuschließen. Kann die Antwort des Servers gelesen werden, ist der Status **ok**; es ist jedoch wichtig, nach jedem POST **::http::postError** oder **::http::responseInfo** aufzurufen, um zu prüfen, ob die Daten vollständig gesendet wurden. Hat der Server die Verbindung geschlossen, ist der Status eof.

## Metadaten

### Nützlichste Metadaten

Antwortet ein HTTP-Server auf eine Anfrage, liefert er nicht nur die angeforderte Entität, sondern auch Metadaten. Diese stammen aus der ersten Zeile (der "status line") der Antwort und aus einer Reihe von HTTP-Headern. Weitere Metadaten betreffen, wie **::http::geturl** die Antwort des Servers verarbeitet hat.

Die wichtigsten Metadaten sind mit dem Befehl **::http::responseInfo** zugänglich. Dieser Befehl gibt ein dict aus Metadaten zurück, die für die Erkennung einer erfolgreichen Transaktion und die

Nutzung der Antwort wesentlich sind, sowie weitere informative Metadaten. Die Schlüssel des dict sind:

**===== Wesentliche Werte =====**

**stage**
Dieser von **::http::geturl** gesetzte Wert beschreibt die erreichte Phase der Transaktion. Werte in der Reihenfolge des Transaktionslebenszyklus sind: "created", "connecting", "header", "body" und "complete". Die anderen dict-Schlüssel stehen erst zur Verfügung, wenn **stage** "body" oder "complete" ist. Der Schlüssel **currentSize** hat seinen endgültigen Wert erst, wenn **stage** "complete" ist.

**status**
Dieser von **::http::geturl** gesetzte Wert ist "ok" bei einer erfolgreichen Transaktion; "eof", "error", "timeout" oder "reset" bei einer fehlgeschlagenen Transaktion; oder "" solange die Transaktion noch läuft. Der Wert entspricht dem Rückgabewert von **::http::status**. Die Bedeutung dieser Werte wird im Abschnitt Fehler (oben) beschrieben.

**responseCode**
Der vom Server in der ersten Zeile (der "status line") der Antwort gesendete "HTTP status code". Kann der Wert nicht aus der Statuszeile extrahiert werden, wird die vollständige Statuszeile zurückgegeben.

**reasonPhrase**
Die vom Server als Beschreibung des HTTP-Statuscodes gesendete "reason phrase". Kann der Wert nicht aus der Statuszeile extrahiert werden, wird die vollständige Statuszeile zurückgegeben.

**contentType**
Der Wert des Antwort-Headers **Content-Type** oder, falls der Header nicht geliefert wurde, der Standardwert "application/octet-stream".

**binary**
Dieser boolesche, von **::http::geturl** gesetzte Wert beschreibt, wie der Befehl die vom Server zurückgegebene Entität (nach Dekodierung einer in **Content-Encoding** angegebenen Kompression) interpretiert hat. Auf diese dekodierte Entität greift der Rückgabewert von **::http::responseBody** zu.

Der Wert ist **true**, wenn **http** die dekodierte Entität als Binärdaten interpretiert hat. Der Rückgabewert von **::http::responseBody** ist dann eine Tcl-Binärzeichenkette. Das eignet sich für Bilddaten, Zip-Dateien usw. **::http::geturl** wählt diesen Wert, wenn der Benutzer die Option **-binary** übergeben hat, der Server einen binären Content-Type in einem **Content-Type**-Antwort-Header geliefert hat oder keinen **Content-Type**-Header gesendet hat.

Der Wert ist in anderen Fällen **false**; dann hat **http** die dekodierte Entität als Text interpretiert. Der Text wurde aus dem vom Server angegebenen Zeichensatz in Tcls internes Unicode-Format konvertiert; der Rückgabewert von **::http::responseBody** ist eine gewöhnliche Tcl-Zeichenkette.

Es lohnt sich stets, nach einer HTTP-Transaktion den Wert von "binary" zu prüfen, um festzustellen, ob ein falsch konfigurierter Server **http** veranlasst hat, eine Textressource als Binärdaten oder umgekehrt zu interpretieren.

**redirection**
Die URL, die Ziel der Weiterleitung ist. Der Wert stammt vom Antwort-Header **Location**. Dieser Header wird gesendet, wenn die Antwort den Statuscode 3XX (Weiterleitung) hat.

**upgrade**
Ist der Wert nicht leer, gibt er die Protokoll(e) an, auf die der Server nach Abschluss dieser Transaktion bei weiterhin derselben Verbindung wechseln wird. Will der Server das Protokoll wechseln, sendet er außerdem "101" als Statuscode (Schlüssel **responseCode**), das Wort "upgrade" als Element des Antwort-Headers **Connection** (Schlüssel **connectionResponse**) und keinen Antwortkörper. Siehe Abschnitt Protokoll-Upgrades für weitere Informationen.

**error**
Die Fehlermeldung, falls vorhanden. Weitere Informationen einschließlich Stack-Trace und Fehlercode liefert **::http::error**.

**postError**
Die Fehlermeldung (falls vorhanden), die entsteht, wenn eine HTTP-POST-Anfrage ihren Anfragekörper an den Server sendet. Weitere Informationen liefert **::http::postError**. Eine POST-Transaktion kann laut den Schlüsseln **stage**, **status** und **responseCode** abgeschlossen erscheinen; es ist jedoch wichtig, **postError** zu prüfen, falls beim Hochladen des Anfragekörpers ein Fehler aufgetreten ist.

**===== Informative Werte =====**

**method**
Die in der Anfrage verwendete HTTP-Methode.

**charset**
Der Wert des charset-Attributs des Antwort-Headers **Content-Type**. Der charset-Wert wird nur für Textressourcen verwendet. Hat der Server keinen charset angegeben, gilt der Wert der Variable **::http::defaultCharset**, die, sofern der Aufrufer sie nicht geändert hat, **iso8859-1** ist. Eingehende Textdaten werden automatisch aus dem durch **charset** definierten Zeichensatz in Tcls interne Unicode-Darstellung, d. h. in eine Tcl-Zeichenkette, konvertiert.

**compression**
Eine Kopie des Werts des Antwort-Headers **Content-Encoding**.

**httpRequest**
Die in der Anfrage angegebene HTTP-Version (in der Anfragezeile gesendet). Der Wert entspricht der Option **-protocol** von **::http::geturl** (Standard "1.1"), sofern der Befehl den Wert nicht auf "1.0" herabsetzte, weil **-handler** übergeben wurde.

**httpResponse**
Die vom Server verwendete HTTP-Version (aus der Antwort-"status line"). Der Server verwendet diese Version in der Antwort, stellt aber sicher, dass die Antwort mit der in der Client-Anfrage angegebenen HTTP-Version kompatibel ist. Kann der Wert nicht aus der Statuszeile extrahiert werden, wird die vollständige Statuszeile zurückgegeben.

**url**
Die angeforderte URL, typischerweise die als Argument an **::http::geturl** übergebene URL ohne ihr "Fragment" (der letzte Teil der URL ab "#").

**connectionRequest**
Der Wert, falls vorhanden, der dem Server in **Connection**-Anfrage-Header(n) gesendet wurde.

**connectionResponse**
Der Wert, falls vorhanden, der vom Server in **Connection**-Antwort-Header(n) empfangen wurde.

**connectionActual**
Dieser von **::http::geturl** gesetzte Wert gibt an, ob die Verbindung nach der Transaktion geschlossen wurde (Wert "close") oder offen blieb (Wert "keep-alive").

**transferEncoding**
Der Wert des Antwort-Headers Transfer-Encoding, falls vorhanden. Der Wert ist entweder "chunked" (HTTP/1.1 "chunked encoding") oder die leere Zeichenkette.

**totalPost**
Die Gesamtlänge des Anfragekörpers einer POST-Anfrage.

**currentPost**
Die Anzahl der Bytes des POST-Anfragekörpers, die bisher an den Server gesendet wurden. Der Wert entspricht dem Rückgabewert von **::http::size**.

**totalSize**
Eine Kopie des Werts des Antwort-Headers **Content-Length**. Die in einem **Content-Length** -Header angegebene Byteanzahl, falls gesendet. Wurde keiner gesendet, ist der Wert 0. Ein korrekt konfigurierter Server lässt diesen Header weg, wenn die Transfer-Kodierung "chunked" ist, oder (bei älteren Servern) wenn der Server die Verbindung am Ende der Ressource schließt.

**currentSize**
Die bisher vom Server abgerufene Byteanzahl.

## Weitere Metadaten

Das von **::http::responseInfo** zurückgegebene Dictionary ist die nützlichste Teilmenge der verfügbaren Metadaten. Weitere Metadaten umfassen:

1. Die vollständige "status line" der Antwort, verfügbar als Rückgabewert von **::http::responseLine**.
2. Die vollständigen Antwort-Header, verfügbar als Rückgabewert von **::http::responseHeaders**. Der Rückgabewert ist eine Liste der Namen und Werte der Antwort-Header in der Empfangsreihenfolge vom Server.

Der Rückgabewert ist kein dict, weil einige Header-Namen mehrfach vorkommen können, insbesondere **Set-Cookie**. Wird der Wert in ein dict oder ein Array gelesen (mit array set), bleibt nur der letzte Header mit jedem Namen erhalten.

Einige der Header-Namen (Metadaten-Schlüssel) sind unten aufgeführt; der HTTP-Standard definiert jedoch weitere, und Server können eigene hinzufügen. Wird unten ein Dictionary-Schlüssel erwähnt, bezieht sich das auf den von **::http::responseInfo** zurückgegebenen dict-Wert.

**Content-Type**
Der Content-Type des URL-Inhalts. Beispiele sind **text/html**, **image/gif, application/postscript** und **application/x-tcl**. Textwerte geben typischerweise einen Zeichensatz an, z. B. **text/html; charset=UTF-8**. Dictionary-Schlüssel *contentType*.

**Content-Length**
Die angegebene Größe des Inhalts in Bytes, verfügbar als Dictionary-Schlüssel *totalSize*. Die

tatsächlich von **::http::geturl** bisher gelesene Byteanzahl ist Dictionary-Schlüssel *currentSize*.

**Content-Encoding**

Der für den Inhalt verwendete Kompressionsalgorithmus. Beispiele sind **gzip**, **deflate**. Dictionary-Schlüssel *compression*.

**Location**

Dieser Header wird gesendet, wenn die Antwort den Statuscode 3XX (Weiterleitung) hat. Er liefert die URL, die Weiterleitungsziel ist. Dictionary-Schlüssel *redirection*.

**Set-Cookie**

Dieser Header bietet dem Client ein Cookie an. Die Cookie-Verwaltung erfolgt über die Option **-cookiejar** von **::http::config**; die **Set-Cookie**-Header müssen daher nicht von Benutzerskripten geparst werden. Siehe Abschnitt Cookie-Jar-Protokoll.

**Connection**

Der Wert kann als durch Kommas getrennte Liste oder durch mehrere Header geliefert werden. Die Liste hat oft nur ein Element, entweder "close" oder "keep-alive". Der Wert "upgrade" zeigt eine erfolgreiche Upgrade-Anfrage an und wird typischerweise mit Statuscode 101, einem **Upgrade**-Antwort-Header und ohne Antwortkörper kombiniert. Dictionary-Schlüssel *connectionResponse*.

**Upgrade**

Der Wert gibt die Protokoll(e) an, auf die der Server unmittelbar nach der leeren Zeile, die die 101-Antwort-Header beendet, wechselt. Dictionary-Schlüssel *upgrade*.

### Noch weitere Metadaten

**1.**

Details der HTTP-Anfrage. Die Anfrage wird durch die an **::http::geturl** und **::http::config** übergebenen Optionen bestimmt. Es ist manchmal hilfreich zu prüfen, was **::http::geturl** tatsächlich an den Server gesendet hat; diese Informationen sind über **::http::requestHeaders** und **::http::requestLine** verfügbar.

**2.**

Das Status-Array: die internen Variablen von **::http::geturl**. Es kann manchmal hilfreich sein, dieses Array zu untersuchen. Details im nächsten Abschnitt.

## Status-Array

Die Prozedur **::http::geturl** gibt einen *token* zurück, der als Argument an andere **::http::***-Befehle übergeben werden kann, die den Zustand der HTTP-Transaktion untersuchen und verwalten. Für die meisten Zwecke reichen diese Befehle. Der *token* kann auch verwendet werden, um auf den internen Zustand der Transaktion zuzugreifen, der in einem Tcl-Array gespeichert ist. Das ist besonders nützlich beim Schreiben von Rückruffunktionen für die Optionen **-command**, **-handler**, **-progress** oder **-queryprogress**. Verwenden Sie innerhalb der Prozedur folgenden Befehl, um ein leicht nutzbares Array *state* als lokale Variable in der Prozedur zu definieren:

```
upvar 0 $token state
```

Sind die mit der URL verknüpften Daten nicht mehr nötig, sollte das Array entfernt werden, um Speicher freizugeben. Dafür dient die Prozedur **::http::cleanup**.

Die folgenden Array-Elemente werden unterstützt und sind die Quelle der von Befehlen zurückgegebenen Werte, wie unten beschrieben. Wird unten ein Dictionary-Schlüssel erwähnt, bezieht sich das auf den von **::http::responseInfo** zurückgegebenen dict-Wert.

**binary**
Für Dictionary-Schlüssel *binary*.

**body**
Für Befehl **::http::responseBody**.

**charset**
Für Dictionary-Schlüssel *charset*.

**coding**
Für Dictionary-Schlüssel *compression*.

**connection**
Für Dictionary-Schlüssel *connectionActual*.

**currentsize**
Für Befehl **::http::size**; und für Dictionary-Schlüssel *currentSize*.

**error**
Für Befehl **::http::error**; ein Teil wird im Dictionary-Schlüssel *error* verwendet.

**http**
Für Befehl **::http::responseLine**.

**httpResponse**
Für Dictionary-Schlüssel *httpResponse*.

**meta**
Für Befehl **::http::responseHeaders**. Weitere Erläuterung oben im Abschnitt Weitere Metadaten.

**method**
Für Dictionary-Schlüssel *method*.

**posterror**
Für Dictionary-Schlüssel *postError*.

**postErrorFull**
Für Befehl **::http::postError**.

**-protocol**
Für Dictionary-Schlüssel *httpRequest*.

**querylength**
Für Dictionary-Schlüssel *totalPost*.

**queryoffset**
Für Dictionary-Schlüssel *currentPost*.

**reasonPhrase**
Für Dictionary-Schlüssel *reasonPhrase*.

**requestHeaders**
Für Befehl **::http::requestHeaders**.

**requestLine**
Für Befehl **::http::requestLine**.

**responseCode**

Für Dictionary-Schlüssel *responseCode*.

**state**

Für Dictionary-Schlüssel *stage*.

**status**

Für Befehl **::http::status**; und für Dictionary-Schlüssel *status*.

**totalsize**

Für Dictionary-Schlüssel *totalSize*.

**transfer**

Für Dictionary-Schlüssel *transferEncoding*.

**type**

Für Dictionary-Schlüssel *contentType*.

**upgrade**

Für Dictionary-Schlüssel *upgrade*.

**url**

Für Dictionary-Schlüssel *url*.

## Persistente Verbindungen

### Grundlagen

Siehe RFC 7230 Abschnitt 6, der RFC 2616 Abschnitt 8.1 ablöst.

Eine persistente Verbindung ermöglicht mehrere HTTP/1.1-Transaktionen über dieselbe TCP-Verbindung. Pipelining ermöglicht es einem Client, mehrere Anfragen über eine persistente Verbindung zu senden, ohne auf jede Antwort zu warten. Der Server sendet Antworten in derselben Reihenfolge, in der die Anfragen empfangen wurden.

Schlägt eine POST-Anfrage fehl, ist typischerweise eine Bestätigung des Benutzers nötig, bevor die Anfrage erneut gesendet wird. Der Benutzer möchte prüfen, ob der Server durch die fehlgeschlagene POST-Anfrage verändert wurde, bevor dieselbe Anfrage erneut gesendet wird.

Eine HTTP-Anfrage verwendet einen persistenten Socket, wenn der Aufruf von **http::geturl** die Option **-keepalive true** hat. Pipelining wird angewendet, wo erlaubt, wenn die **http::config** -Option **-pipeline** boolesch **true** ist (ihr Standardwert).

Das http-Paket hält höchstens eine persistente Verbindung zu jedem Server (d. h. zu jedem Wert von "domain:port"). Wird **http::geturl** aufgerufen, um eine Anfrage über eine persistente Verbindung zu stellen, während die Verbindung mit einer anderen Anfrage beschäftigt ist, wird die neue Anfrage in eine Warteschlange gestellt, bis die Verbindung frei ist.

Das http-Paket unterstützt keine HTTP/1.0-persistenten Verbindungen, die durch den Header **Keep-Alive** gesteuert werden.

### Sonderfälle

Dieser Unterabschnitt behandelt Fragen im Zusammenhang mit dem Schließen der persistenten Verbindung durch den Server, automatische Wiederholung fehlgeschlagener Anfragen, die besondere Behandlung von POST-Anfragen und die Optionen zum Umgang mit diesen Fällen.

Gemäß RFC 7230 pipelined **http::geturl** keine Anfragen mit der POST-Methode. Nutzt eine POST-Anfrage eine persistente Verbindung und ist nicht die erste Anfrage auf dieser Verbindung, wartet **http::geturl**, bis die Antwort auf die vorherige Anfrage empfangen wurde; oder (wenn die **http::config**-Option **-postfresh** boolesch **true** ist) verwendet es für jede POST-Anfrage eine neue Verbindung.

Verarbeitet der Server mehrere gepipelinete Anfragen und sendet er mit einer der Antworten (außer der letzten) einen Antwort-Header "**Connection: close**", bleiben nachfolgende Antworten unerfüllt. **http::geturl** sendet die unerfüllten Anfragen erneut über eine neue Verbindung.

Eine Schwierigkeit entsteht, wenn ein HTTP-Client eine Anfrage über eine persistente Verbindung sendet, die eine Weile untätig war. Der HTTP-Server kann eine scheinbar untätige Verbindung halb schließen, während der Client eine Anfrage sendet, aber bevor die Anfrage den Server erreicht: in diesem Fall (ein **asynchronous close event**) schlägt die Anfrage fehl. Die Schwierigkeit entsteht, weil der Client nicht sicher sein kann, ob die POST den Zustand des Servers verändert hat. Bei HEAD- oder GET-Anfragen öffnet **http::geturl** eine weitere Verbindung und sendet die fehlgeschlagene Anfrage erneut. War die Anfrage jedoch eine POST-Anfrage, verbietet RFC 7230 standardmäßig eine automatische Wiederholung und schlägt entweder eine Benutzerbestätigung oder eine Bestätigung durch User-Agent-Software mit semantischem Verständnis der Anwendung vor. Die **http::config**-Option **-repost** ermöglicht beide Möglichkeiten.

Asynchrone Schließ-Ereignisse können nur in einem kurzen Zeitintervall auftreten. Das Paket **http** überwacht jede persistente Verbindung auf Schließung durch den Server. Nach Erkennung wird die Verbindung auch auf der Client-Seite geschlossen; nachfolgende Anfragen verwenden eine frische Verbindung.

Wird **http::geturl** mit der Option **-keepalive true** aufgerufen, versucht es sowohl eine bestehende persistente Verbindung zu nutzen (falls verfügbar), als auch dem Server einen Anfrage-Header "**Connection: keep-alive**" zu senden, der die Verbindung für künftige Anfragen offen halten soll.

Die **http::config**-Optionen **-pipeline**, **-postfresh** und **-repost** betreffen persistente Verbindungen.

Ist die Option **-pipeline** boolesch **true**, werden GET- und HEAD-Anfragen über eine persistente Verbindung gepipelined. POST-Anfragen werden nicht gepipelined — ist die POST-Anfrage nicht die erste Transaktion auf der Verbindung, wird ihre Anfrage erst gesendet, wenn die vorherige Antwort abgeschlossen ist. GET- und HEAD-Anfragen nach einer POST-Anfrage werden erst gesendet, wenn die POST-Antwort geliefert wurde, und nicht, wenn die POST-Anfrage fehlschlägt.

Ist die Option **-postfresh** boolesch **true**, überschreibt sie die Option **-keepalive** von **http::geturl** und öffnet für jede POST-Anfrage stets eine frische Verbindung.

Ist die Option **-repost true**, ist die automatische Wiederholung einer POST-Anfrage erlaubt, die fehlschlägt, weil sie eine persistente Verbindung nutzt, die der Server halb geschlossen hat (ein **asynchronous close event**). Nachfolgende GET- und HEAD-Anfragen in einer fehlgeschlagenen Pipeline werden ebenfalls wiederholt. *Die Option -repost sollte nur verwendet werden, wenn die Anwendung versteht*, dass die Wiederholung angemessen ist — die Anwendung muss wissen, dass eine Wiederholung der POST-Anfrage keine nachteiligen Auswirkungen hat, falls die fehlgeschlagene POST den Serverzustand erfolgreich verändert hat.

## Cookie-Jar-Protokoll

Cookies sind kurze Schlüssel-Wert-Paare zur Implementierung von Sitzungen im ansonsten zustandslosen HTTP-Protokoll. (Details siehe RFC 6265; Tcl implementiert das Cookie2-Protokoll nicht, da es in der Praxis selten vorkommt.)

Cookie-Speicherverwaltungsbefehle — **cookie jars** — müssen diese Unterbefehle unterstützen, die das HTTP-Cookie-Speicherverwaltungsprotokoll bilden. *cookieJar* unten muss kein Befehlsname sein; es ist ein Befehlspräfix (eine Tcl-Liste von Wörtern, die an Ort und Stelle expandiert wird) und lässt viele Implementierungen zu.

Obwohl nicht formell Teil des Protokolls, wird erwartet, dass bestimmte Werte von *cookieJar* Sitzungen entsprechen; es liegt beim Aufrufer von **::http::config**, welche Sitzung gilt und wann diese Sitzungen gelöscht werden, wenn sie nicht mehr gewünscht sind (was der Fall sein sollte, wenn sie nicht mehr als aktueller Cookie-Jar konfiguriert sind).

***cookieJar* getCookies *protocol host requestPath***

Dieser Befehl fragt den Cookie-Jar, welche Cookies für eine bestimmte Anfrage geliefert werden sollen. Er soll *protocol* (typischerweise **http** oder **https**), *host* und *requestPath* (aus dem *url*-Argument von **::http::geturl** geparst) entgegennehmen und eine Liste von Cookie-Schlüsseln und -Werten zurückgeben, die die an den Remote-Host zu sendenden Cookies beschreiben. Die Liste muss eine gerade Anzahl von Elementen haben.

Pro Anfrage sollte höchstens ein Cookie mit einem bestimmten Schlüssel existieren (typischerweise das mit der spezifischsten *host*/domain- und *requestPath* /path-Übereinstimmung), aber es kann viele Cookies mit unterschiedlichen Namen in einer Anfrage geben.

***cookieJar* storeCookie *cookieDictionary***

Dieser Befehl bittet den Cookie-Jar, ein bestimmtes Cookie zu speichern, das von einer Anfrage zurückgegeben wurde; das Ergebnis dieses Befehls wird ignoriert. Das Cookie (vom http-Paket geparst) wird durch ein Dictionary *cookieDictionary* beschrieben, das folgende Schlüssel haben kann:

**domain**

Immer vorhanden. Sein Wert beschreibt den Domänen-Hostnamen *oder* das Präfix, für das das Cookie zurückgegeben werden soll. Die Prüfung der Domäne gegen den Ursprung (unten) soll sorgfältig erfolgen, da Sites, die Cookies ausstellen, dies nur für mit ihnen verbundene Domänen tun sollten. Cookies, die keine relevante Origin-Matching-Regel einhalten, sind zu ignorieren.

**expires**

Optional. Ist er vorhanden, ist das Cookie als persistentes Cookie gedacht; der Wert ist der Tcl-Zeitstempel (in Sekunden zur gleichen Basis wie clock seconds) des Ablaufzeitpunkts (der in der Vergangenheit liegen kann; dann soll das Cookie sofort gelöscht werden). Fehlt er, ist das Cookie ein Sitzungs-Cookie, das nicht über die Lebensdauer des Cookie-Jars hinaus persistiert werden soll.

**hostonly**

Immer vorhanden. Sein Wert ist ein Boolescher Wert, der angibt, ob es ein Single-Host-Cookie (**true**) oder ein Domänen-Cookie (**false**) ist.

**httponly**

Immer vorhanden. Sein Wert ist **true**, wenn die Site wünscht, dass das Cookie nur mit HTTP-(oder HTTPS-)Verkehr verwendet wird.

**key**

Immer vorhanden. Sein Wert ist der *key* des Cookies, Teil der Information, die beim erneuten Senden dieses Cookies in einer künftigen Anfrage zurückgegeben werden muss.

**origin**

Immer vorhanden. Sein Wert beschreibt, wo das http-Paket glaubt, das Cookie erhalten zu haben; nützlich zur Prüfung, ob die Domäne des Cookies gültig ist.

**path**

Immer vorhanden. Sein Wert beschreibt das Pfadpräfix von Anfragen an die Cookie-Domäne, für die das Cookie zurückgegeben werden soll.

**secure**

Immer vorhanden. Sein Wert ist **true**, wenn das Cookie nur bei Anfragen über sichere Kanäle (typischerweise HTTPS) verwendet werden soll.

**value**

Immer vorhanden. Sein Wert ist der Wert des Cookies, Teil der Information, die beim erneuten Senden in einer künftigen Anfrage zurückgegeben werden muss.

Andere Schlüssel können stets ignoriert werden; sie haben in diesem Protokoll keine Bedeutung.

## Protokoll-Upgrades

Die HTTP/1.1-Anfrage-Header **Connection** und **Upgrade** teilen dem Server mit, dass der Client das über die bestehende Verbindung genutzte Protokoll ändern möchte (RFC 7230). Dieser Mechanismus kann einen WebSocket (RFC 6455), eine höhere HTTP-Version (HTTP 2) oder TLS-Verschlüsselung anfordern. Nimmt der Server die Upgrade-Anfrage an, lautet sein Antwortcode 101.

Um beim Aufruf von **http::geturl** ein Protokoll-Upgrade anzufordern, muss die Option **-headers** passende Werte für **Connection** und **Upgrade** liefern, und die Option **-command** muss einen Befehl liefern, der das angeforderte Protokoll implementiert und auch die Serverantwort verarbeiten kann, wenn der Server das Upgrade ablehnt. Bei Upgrade-Anfragen ignoriert **http::geturl** den Wert der Option **-keepalive** und verwendet stets **0**, damit die Upgrade-Anfrage nicht über eine Verbindung erfolgt, die für mehrere HTTP-Anfragen gedacht ist.

Die Tcllib-Bibliothek **websocket** implementiert WebSockets und ruft die nötigen Befehle im Paket **http** auf.

Derzeit gibt es keine native Tcl-Client-Bibliothek für HTTP/2.

Der **Upgrade**-Mechanismus wird in Webbrowsern nicht verwendet, um TLS anzufordern, weil **http** und **https** über unterschiedliche Ports laufen. Er wird von Protokollen wie Internet Printing Protocol (IPP) verwendet, die auf **http(s)** aufbauen und dieselbe TCP-Portnummer für sichere und unsichere Verbindungen nutzen.

In Browsern wird opportunistische Verschlüsselung stattdessen durch den Client-Header **Upgrade-Insecure-Requests** umgesetzt. Ist ein sicherer Dienst verfügbar, lautet der Server-Antwortcode eine 307-Weiterleitung, und der Antwort-Header **Location** gibt die Ziel-URL an. Der Browser muss **http::geturl** erneut aufrufen, um diese URL abzurufen. Siehe https://w3c.github.io/webappsec-upgrade-insecure-requests/

## Threads

### Zweck

Der Befehl **::http::geturl** verwendet den Tcl-Befehl **::socket** mit der Option **-async**, um eine Verbindung zu einem Remote-Server herzustellen; die Rückkehr dieses Befehls kann in ungünstigen Fällen verzögert sein (z. B. bei einer langsamen DNS-Abfrage) und verhindert, dass die Ereignisschleife andere Ereignisse verarbeitet. Diese Verzögerung entfällt, wenn **::socket** in einem anderen Thread ausgewertet wird. Das Thread-Paket ist nicht Teil von Tcl, wird aber in "Batteries Included"-Distributionen mitgeliefert. Statt **::socket** verwendet das http-Paket **::http::socket**, das Verbindungen gemäß dem Wert von **-threadlevel** und der Verfügbarkeit des Pakets Thread herstellt.

### Mit TLS (HTTPS)

Dieselbe **-threadlevel**-Konfiguration gilt für HTTP- und HTTPS-Verbindungen. HTTPS wird durch **http::register** aktiviert, typischerweise durch Angabe des Befehls **::tls::socket** des tls-Pakets für TLS-Kryptografie. **::tls::socket** verbindet zum Remote-Server mit dem Befehl, der durch den Wert der Variable **::tls::socketCmd** angegeben ist; dieser Standardwert ist "::socket". Findet http::geturl, dass **::tls::socketCmd** diesen Wert hat, ersetzt es ihn durch "::http::socket". Hat **::tls::socketCmd** einen anderen Wert als "::socket", d. h. hat das Skript oder die Tcl-Installation den Wert "::socket" durch einen anderen Befehlsnamen ersetzt, ändert http den Wert nicht. Das Skript oder die Installation, die **::tls::socketCmd** geändert hat, ist dafür verantwortlich, **::http::socket** in den eigenen Ersatzbefehl zu integrieren.

### Mit einem Unter-Interpreter

Der Peer-Thread kann den Socket nur an den Haupt-Interpreter des Threads des Skripts übertragen. Daher funktioniert der thread-basierte **::http::socket** mit **-threadlevel**-Werten ungleich 0 nur, wenn das Skript im Haupt-Interpreter läuft. Ein Unter-Interpreter muss **-threadlevel 0** verwenden, sofern der Eltern-Interpreter keine alternativen Einrichtungen bereitgestellt hat. Der Haupt-Eltern-Interpreter kann einem Unter-Interpreter volle **-threadlevel** -Einrichtungen gewähren, z. B. durch Aliasierung von **::http::socket** im Kind auf einen Befehl, der **http::socket** im Eltern ausführt und den Socket dann an das Kind überträgt.

## Beispiele

Dieses Beispiel erstellt eine Prozedur zum Kopieren einer URL in eine Datei mit Fortschrittsanzeige und gibt die mit der URL verknüpften Antwort-Header aus.

```
proc httpcopy { url file {chunk 4096} } {
    set out [open $file w]
    set token [::http::geturl $url -channel $out \
            -progress httpCopyProgress -blocksize $chunk]
    close $out

    # Beendet die von httpCopyProgress begonnene Zeile
    puts stderr ""

    upvar 0 $token state
    set max 0
    foreach {name value} $state(meta) {
        if {[string length $name] > $max} {
            set max [string length $name]
        }
        if {[regexp -nocase ^location$ $name]} {
            # URL-Weiterleitungen behandeln
            puts stderr "Location:$value"
            return [httpcopy [string trim $value] $file $chunk]
        }
    }
    incr max
    foreach {name value} $state(meta) {
        puts [format "%-*s %s" $max $name: $value]
    }

    return $token
}
proc httpCopyProgress {args} {
    puts -nonewline stderr .
    flush stderr
}
```

*Copyright (c) 1995-1997 Sun Microsystems, Inc.*
*Copyright (c) 1998-2000 Ajuba Solutions.*
*Copyright (c) 2004 ActiveState Corporation.*

# idna

*Stichwörter:* idna, internet, www

## Name

tcl::idna - Unterstützung für die Normalisierung internationalisierter Domainnamen

## Übersicht

**package require tcl::idna 1.0**

**tcl::idna decode** *hostname*
**tcl::idna encode** *hostname*
**tcl::idna puny decode** *string* ?*case*?
**tcl::idna puny encode** *string* ?*case*?
**tcl::idna version**

## Beschreibung

Dieses Paket stellt eine Implementierung des in internationalisierten Domainnamen verwendeten Punycode-Verfahrens sowie einige Zugriffsbefehle bereit. (Siehe RFC 3492 für eine Beschreibung von Punycode.)

**tcl::idna decode *hostname***

Nimmt einen Hostnamen *hostname*, der möglicherweise punycode-kodierte Zeichenfolgen enthält, und gibt den Hostnamen zurück, wie er einem Benutzer angezeigt werden könnte. Beachten Sie, dass es oft UNICODE-Zeichen mit extrem ähnlichen Glyphen gibt; Hostnamen sollten Benutzern daher mit Vorsicht angezeigt werden.

**tcl::idna encode *hostname***

Nimmt einen Hostnamen *hostname*, wie er einem Benutzer angezeigt werden könnte, und gibt die Variante zurück, in der in einfachen Hostnamen nicht zulässige Zeichen mit Punycode kodiert sind.

**tcl::idna puny *subcommand ...***

Bietet direkten Zugriff auf den grundlegenden Punycode-Encoder und -Decoder. Unterstützt zwei *subcommand*s:

**tcl::idna puny decode *string* ?*case*?**

Dekodiert die punycode-kodierte Zeichenkette *string* und gibt das Ergebnis zurück. Ist *case* angegeben, ist es ein boolescher Wert: bei true wird die Schreibweise während der Dekodierung in Großbuchstaben, bei false in Kleinbuchstaben gefaltet; fehlt er, findet keine Schreibweisen-Transformation statt.

**tcl::idna puny encode *string* ?*case*?**

Kodiert die Zeichenkette *string* und gibt die punycode-kodierte Version zurück. Ist *case* angegeben, ist es ein boolescher Wert: bei true wird die Schreibweise während der Kodierung in Großbuchstaben, bei false in Kleinbuchstaben gefaltet; fehlt er, findet keine Schreibweisen-Transformation statt.

**tcl::idna version**

Gibt die Version des Pakets **tcl::idna** zurück.

## Beispiel

Beispiel für die Punycode-Kodierung einer Zeichenkette:

```
package require tcl::idna

puts [tcl::idna puny encode "abc→def"]
#    gibt aus: abcdef-kn2c
puts [tcl::idna puny decode "abcdef-kn2c"]
#    gibt aus: abc→def
```

*Copyright (c) 2014-2018 Donal K. Fellows.*

# if

*Stichwörter:* if, boolean, conditional, else, false, true

## Name

if - Führt Scripte bedingt aus

## Übersicht

**if** *expr1* ?**then**? *body1* **elseif** *expr2* ?**then**? *body2* **elseif** ... ?**else**? ?*bodyN*?

## Beschreibung

Der Befehl *if* wertet *expr1* als Ausdruck aus (auf dieselbe Weise wie expr sein Argument auswertet). Der Wert des Ausdrucks muss ein Boolean sein (ein numerischer Wert, wobei 0 falsch und alles andere wahr ist, oder ein String-Wert wie **true** oder **yes** für wahr und **false** oder

**no** für falsch); wenn er wahr ist, wird *body1* ausgeführt, indem es an den Tcl-Interpreter übergeben wird. Andernfalls wird *expr2* als Ausdruck ausgewertet, und wenn dieser wahr ist, wird **body2** ausgeführt, und so weiter. Wenn keiner der Ausdrücke zu wahr ausgewertet wird, wird *bodyN* ausgeführt. Die Argumente **then** und **else** sind optionale "Füllwörter", um den Befehl leichter lesbar zu machen. Es kann eine beliebige Anzahl von **elseif**-Klauseln geben, einschließlich null. *BodyN* kann auch weggelassen werden, solange **else** ebenfalls weggelassen wird. Der Rückgabewert des Befehls ist das Ergebnis des Body-Scripts, das ausgeführt wurde, oder ein leerer String, wenn keiner der Ausdrücke nicht null war und es kein *bodyN* gab.

## Beispiele

Eine einfache Bedingung:

```
if {$vbl == 1} { puts "vbl is one" }
```

Mit einer **else**-Klausel:

```
if {$vbl == 1} {
    puts "vbl is one"
} else {
    puts "vbl is not one"
}
```

Mit einer **elseif**-Klausel:

```
if {$vbl == 1} {
    puts "vbl is one"
} elseif {$vbl == 2} {
    puts "vbl is two"
} else {
    puts "vbl is not one or two"
}
```

Beachten Sie: Ausdrücke können mehrzeilig sein, aber in diesem Fall kann es sinnvoll sein, das optionale Schlüsselwort **then** für mehr Klarheit zu verwenden:

```
if {
    $vbl == 1
    || $vbl == 2
    || $vbl == 3
} then {
    puts "vbl is one, two or three"
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# incr

*Stichwörter:* incr, add, increment, variable, value

## Name

incr - Erhöht den Wert einer Variable

## Übersicht

**incr** *varName* ?*increment*?

## Beschreibung

Erhöht den in der Variable *varName* gespeicherten Wert. Der Variablenwert muss eine Ganzzahl sein. Ist *increment* angegeben, wird dessen Wert (ebenfalls eine Ganzzahl) zum Wert von *varName* addiert; andernfalls wird 1 addiert. Der neue Wert wird als Dezimalzeichenkette in *varName* gespeichert und auch als Ergebnis zurückgegeben.

Ab Tcl 8.5 darf *varName* unset sein; dann wird sie auf den Wert *increment* oder den Standard-Inkrementwert **1** gesetzt. Verweist varName auf ein nicht existierendes Element eines Arrays mit gesetztem Standardwert, wird die Summe aus Standardwert und increment (oder 1) im Array-Element gespeichert.

## Beispiele

Addiert eins zum Inhalt der Variable *x*:

```
incr x
```

Addiert 42 zum Inhalt der Variable *x*:

```
incr x 42
```

Addiert den Inhalt der Variable *y* zum Inhalt der Variable *x*:

```
incr x $y
```

Addiert nichts zur Variable *x* (oft nützlich, um zu prüfen, ob ein Prozedurargument wirklich ganzzahlig ist, und andernfalls einen Fehler zu erzeugen):

```
incr x 0
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# info

*Stichwörter:* info, command, information, interpreter, introspection, level, namespace, object, procedure, variable

## Name

info - Informationen über den Zustand des Tcl-Interpreters

## Übersicht

**info** *option* ?*arg arg* ...?

## Beschreibung

Verfügbare Befehle:

**info args *procname***

Gibt die Namen der Parameter der Prozedur *procname* zurück.

**info body *procname***

Gibt den Rumpf der Prozedur *procname* zurück.

**info class *subcommand class* ?*arg* ...?**

Gibt Informationen über die Klasse *class* zurück. Siehe Klassen-Introspection unten.

**info cmdcount**

Gibt die Gesamtzahl der in diesem Interpreter ausgewerteten Befehle zurück.

**info cmdtype *commandName***

Gibt den Typ des Befehls commandName zurück. Eingebaute Typen sind:

**alias**

commandName wurde mit interp alias erzeugt. In einem Safe-Interpreter ist ein Alias nur sichtbar, wenn sowohl der Alias als auch das Ziel sichtbar sind.

**coroutine**

commandName wurde mit coroutine erzeugt.

**ensemble**

commandName wurde mit namespace ensemble erzeugt.

**import**

commandName wurde mit namespace import erzeugt.

**native**

commandName wurde direkt über die Schnittstelle Tcl_CreateObjCommand erzeugt, ohne weitere Registrierung des Befehlstyps.

**object**

commandName ist der öffentliche Befehl, der eine Instanz von oo::object oder einer seiner Unterklassen repräsentiert.

**privateObject**

commandName ist der private Befehl (standardmäßig my), der eine Instanz von oo::object oder einer seiner Unterklassen repräsentiert.

**proc**

commandName wurde mit proc erzeugt.

**interp**

commandName wurde mit interp create erzeugt.

**zlibStream**

commandName wurde mit zlib stream erzeugt.

**info commands ?*pattern*?**

Gibt die Namen aller im aktuellen Namespace sichtbaren Befehle zurück. Ist *pattern* angegeben, nur die Namen, die gemäß string match passen. Nur die letzte Komponente von *pattern* ist ein Muster; andere Komponenten bezeichnen einen Namespace. Siehe **NAMENSAUFLÖSUNG** in der Dokumentation zu namespace(n).

**info complete *command***

Gibt **1** zurück, wenn *command* ein vollständiger Befehl ist, sonst **0**. Typischerweise in zeilenorientierten Eingabeumgebungen, damit Benutzer Befehle über mehrere Zeilen eingeben können.

**info constant *varName***

Gibt 1 zurück, wenn varName eine Konstantenvariable ist (siehe const), sonst 0.

**info consts ?*pattern*?**

Gibt die Liste der Konstantenvariablen (siehe const) im aktuellen Gültigkeitsbereich zurück, oder — ist pattern angegeben — die Liste der Konstantenvariablen, die pattern entsprechen, analog zu info vars.

**info coroutine**

Gibt den Namen der aktuellen **coroutine** zurück oder die leere Zeichenkette, wenn keine aktuelle Coroutine existiert oder die aktuelle Coroutine gelöscht wurde.

**info default *procname parameter varname***

Hat der Parameter *parameter* der Prozedur *procname* einen Standardwert, wird dieser in *varname* gespeichert und **1** zurückgegeben. Andernfalls **0**.

**info errorstack ?*interp*?**

Gibt eine Beschreibung des aktiven Befehls auf jeder Ebene für den letzten Fehler im aktuellen Interpreter zurück, oder im Interpreter *interp*, falls angegeben.

Die Beschreibung ist ein Dictionary aus Token und Parametern. Token sind derzeit **CALL**, **UP** oder **INNER**; weitere Werte können künftig eingeführt werden. **CALL** kennzeichnet einen Befehlsaufruf; der Parameter ist das entsprechende **info level 0**. **UP** kennzeichnet einen durch uplevel oder Ähnliches erzeugten Wechsel der Variablenframes und bezieht sich auf das vorherige **CALL**-Element; der Parameter ist der Ebenenversatz. **INNER** bezeichnet den "inneren Kontext" — den innersten atomaren Befehl oder Bytecode-Befehl, der den Fehler auslöste, samt Argumenten, sofern verfügbar. Während **CALL** und **UP** den Aufrufpfad abbilden, liefert **INNER** Details der fehlerhaften Operation im innersten Prozeduraufruf, bis auf Subausdruck-Ebene.

Diese Information steht auch im Eintrag **-errorstack** des Options-Dictionarys von catch mit drei Argumenten; **info errorstack** ist eine bequeme Möglichkeit, sie für nicht abgefangene Fehler auf oberster Ebene in einem interaktiven **interpreter** abzurufen.

**info exists *varName***

Gibt **1** zurück, wenn eine Variable *varName* sichtbar und definiert ist, sonst **0**.

**info frame ?*depth*?**

Gibt die Tiefe des Aufrufs von **info frame** selbst zurück. Andernfalls ein Dictionary, das den aktiven Befehl auf der *depth*-ten Ebene beschreibt; gezählt werden alle für **info level** sichtbaren Befehle plus Befehle ohne neue Ebene, z. B. eval, source oder *uplevel*. Die Frame-Tiefe ist stets größer als die aktuelle Ebene.

Ist *depth* größer als **0**, bezeichnet sie den Frame in dieser Tiefe. Andernfalls die Anzahl Frames oberhalb des aktuellen Frames.

Wie bei **info level** und Fehler-Traces sieht **info frame** bei verschachtelten Befehlen wie "foo [bar [x]]" nur "x", wenn es innerhalb von "x" aufgerufen wird.

Das Dictionary kann folgende Schlüssel enthalten:

**type**

Immer vorhanden. Mögliche Werte: source, **proc**, **eval** und **precompiled**.

source

Ein per source geladenes Skript.

**proc**

Der Rumpf einer Prozedur, der nicht auf eine Zeile in einem bestimmten Skript zurückverfolgt werden konnte.

eval

Der Rumpf eines an eval oder uplevel übergebenen Skripts.

**precompiled**

Ein vorkompiliertes Skript (ladbar mit dem Paket **tbcload**); weitere Informationen sind nicht verfügbar.

**line**

Die Zeilennummer des Befehls innerhalb seines Skripts. Nicht verfügbar für **precompiled**. Bei Typ source relativ zum Dateianfang, bei den letzten beiden Typen relativ zum Skriptanfang.

file

Bei Typ source der normalisierte Pfad der Datei, die den Befehl enthält.

**cmd**

Der Befehl vor durchgeführten Substitutionen.

**proc**

Bei Typ **proc** der Name der Prozedur, die den Befehl enthält.

**lambda**

Für einen Befehl in einem per apply als Rumpf einer unbenannten Routine ausgewerteten Skript die Definition dieser Routine.

**level**

Für einen Frame, der einer Ebene entspricht (noch festzulegen).

Wenn ein Befehl bis zu seiner wörtlichen Definition in einem Skript zurückverfolgt werden kann — z. B. in statisch definierten Prozeduren verschachtelte Prozeduren oder wörtliche **eval** -Skripte in Dateien oder statischen Prozeduren — ist sein Typ source und die Position die absolute Zeilennummer im Skript. Andernfalls ist der Typ **proc** und die Position die Zeilennummer im Prozedurrumpf.

Prozedurdefinitionen und eval in einer dynamisch evaluierten Umgebung zählen Zeilennummern relativ zum Skriptanfang, auch wenn sie relativ zum äußeren dynamischen Skript zählen könnten. Diese Art von Nummer ist meist aussagekräftiger.

Anders formuliert: dateibasierte Positionen werden so tief wie möglich verfolgt; wo das nicht möglich ist, werden Zeilen relativ zum kleinstmöglichen eval- oder Prozedurrumpf gezählt — dieser Gültigkeitsbereich ist meist leichter zu finden als ein dynamischer äußerer.

Die syntaktische Form **{*}** wird wie eval behandelt: Bei wörtlichem Listenargument verfolgt das System die Zeilennummer innerhalb der Listenelemente; andernfalls werden alle Zeilennummern relativ zum Anfang jedes Worts gezählt (kleinster Gültigkeitsbereich).

**info functions ?*pattern*?**

Ohne *pattern* eine Liste aller derzeit definierten mathematischen Funktionen. Mit *pattern* nur Namen, die *pattern* gemäß string match entsprechen.

**info globals ?*pattern*?**

Ohne *pattern* alle Namen derzeit definierter globaler Variablen (Variablen im globalen Namespace). Mit *pattern* nur passende Namen; Abgleich wie bei string match.

**info hostname**

Gibt den Namen des aktuellen Rechners zurück.

Dieser Name ist nicht garantiert der vollqualifizierte Domainname. Haben Rechner mehrere Namen (häufig bei TCP/IP (DNS) und NetBIOS), wird der für TCP/IP-Netzwerke geeignete Name zurückgegeben.

**info level ?*level*?**

Ohne *number* die Ebene, von der diese Routine aufgerufen wurde. Andernfalls den vollständigen aktiven Befehl auf der angegebenen Ebene. Ist *number* größer als **0**, die gewünschte Ebene; andernfalls *number* Ebenen oberhalb der aktuellen. Ein vollständiger Befehl sind die Wörter des Befehls nach allen Substitutionen, also eine Liste. Siehe uplevel zu Ebenen.

**info library**

Gibt den Wert von tcl_library zurück — das Verzeichnis, in dem die mit Tcl verteilten Skripte liegen.

**info loaded ?*interp*? ?*prefix*?**

Gibt den Namen jeder Datei zurück, die in *interp* mit load und Präfix *prefix* geladen wurde. Ohne *prefix* eine Liste, deren Elemente je Dateiname und verwendetes Präfix sind. Bei statisch geladenen Paketen ist der Dateiname die leere Zeichenkette. Für *interp* bezeichnet die leere Zeichenkette den aktuellen Interpreter.

**info locals ?*pattern*?**

Mit *pattern* die Namen lokaler Variablen, die *pattern* gemäß string match entsprechen. Andernfalls alle lokalen Variablen. Mit global, upvar oder variable definierte Variablen sind nicht lokal.

**info nameofexecutable**

Gibt den absoluten Pfadnamen des Programms für den aktuellen Interpreter zurück. Kann keine Datei identifiziert werden, die leere Zeichenkette.

**info object *subcommand object* ?*arg* ...?**

Gibt Informationen über das Objekt *object* zurück. *subcommand* siehe Objekt-Introspection unten.

**info patchlevel**

Gibt den Wert der globalen Variable tcl_patchLevel zurück — die exakte Version der Tcl-Bibliothek bei der Initialisierung.

**info procs ?*pattern*?**

Gibt die Namen aller sichtbaren Prozeduren zurück. Mit *pattern* nur passende Namen gemäß string match. Nur die letzte Komponente von *pattern* ist ein Muster; andere Komponenten wählen einen Namespace. Siehe **NAMENSAUFLÖSUNG** in namespace(n).

**info script ?*filename*?**

Gibt den Pfadnamen des innersten gerade ausgewerteten Skripts zurück oder die leere Zeichenkette, wenn keiner bestimmt werden kann. Mit *filename* wird der Rückgabewert künftiger **info script**-Aufrufe für die Dauer des innersten aktiven Skripts gesetzt — nützlich in virtuellen Dateisystemen.

**info sharedlibextension**

Gibt die Erweiterung für Shared-Library-Dateinamen auf dieser Plattform zurück, z. B. **.so** unter Solaris. Leere Zeichenkette, wenn Shared Libraries nicht unterstützt werden.

**info tclversion**

Gibt den Wert der globalen Variable tcl_version zurück (Haupt- und Nebenversion der Tcl-Bibliothek).

**info vars ?*pattern*?**

Ohne *pattern* alle sichtbaren Variablennamen. Mit *pattern* nur passende gemäß string match. Nur die letzte Komponente von *pattern* ist ein Muster; andere Komponenten bezeichnen einen Namespace. Siehe **NAMENSAUFLÖSUNG** in namespace(n). Bei qualifiziertem *pattern* sind die Ergebnisse vollqualifiziert.

Deklarierte, aber noch nicht bewertete Variablen sind in den Ergebnissen enthalten.

## Klassen-Introspection

Folgende *subcommand*-Werte unterstützt **info class**:

**info class call *class method***

Gibt eine Beschreibung der Methodenimplementierungen zurück, die eine stereotype Instanz der Implementierung von *method* in *class* liefern (stereotype Instanzen: vom Klasse erzeugte Objekte ohne objektspezifische Definitionen). Das Ergebnis ist eine Liste von Viererlisten; jedes Element besteht aus einem Wort für den Implementierungstyp (**method** für eine normale Methode, **filter** für einen angewendeten Filter, private für eine private Methode, unknown für unbekannte Methodenbehandlung), dem aufgerufenen Methodennamen (bei **method** stets *method*, bei unknown "unknown"), dem vollqualifizierten Klassennamen der Definition und einem Wort zum Implementierungstyp (siehe **info class methodtype**).

Es wird nicht geprüft, ob die Implementierungen next zur Weitergabe entlang der Aufrufkette nutzen; die von diesem Befehl gelieferten Aufrufketten enthalten private Methoden nicht.

**info class constructor *class***

Beschreibung der Konstruktordefinition von *class* als Zweierliste: erstes Element die Argumentliste (geeignet für **proc** oder Methodendefinition), zweites der Konstruktorrumpf. Ohne Konstruktor die leere Liste.

**info class definition *class method***

Beschreibung der Definition der Methode *method* von *class* als Zweierliste (Argumentliste und Methodenrumpf).

**info class definitionnamespace *class* ?*kind*?**

Gibt den Definitions-Namespace für Definitionen des Typs kind der Klasse class zurück; er betrifft nur Instanzen von class, nicht class selbst. kind kann -class (Namespace für oo::define) oder -instance (für oo::objdefine) sein; Standard ist -class (nur bei Unterklassen von oo::class wirklich nützlich).

Liefert *class* keinen Definitions-Namespace des Typs, die leere Zeichenkette. Dann ermitteln oo::define und oo::objdefine den Namespace über die Vererbungshierarchie.

**info class destructor *class***

Rumpf des Destruktors von *class*. Ohne Destruktor die leere Zeichenkette.

**info class filters *class***

Liste der auf der Klasse gesetzten Filtermethoden.

**info class forward *class method***

Argumentliste der Methodenweiterleitung *method* auf der Klasse *class*.

**info class instances *class* ?*pattern*?**

Liste der Instanzen von *class*. Mit optionalem *pattern* nur passende Instanzen gemäß string match.

**info class methods *class* ?*options...*?**

Liste aller öffentlichen (exportierten) Methoden von *class*. Folgende *option*s steuern die Auswahl:

**-all**

Mit **-all** und ohne -scope Methoden der Klasse sowie ihrer Superklassen und Mixins.

**-private**

Mit **-private** und ohne -scope auch nicht exportierte Methoden (und bei **-all** von Superklassen und Mixins). Hinweis: unglückliche Namenskollision mit echten privaten Methoden; der Optionsname bleibt aus Kompatibilitätsgründen.

**-scope *scope***

Alle Methoden von class mit Sichtbarkeit scope. Bei dieser Option werden -all und -private ignoriert. Gültige scope-Werte:

**public**

Nur Methoden mit public-Sichtbarkeit (von überall und jeder Instanz dieser Klasse aufrufbar).

**unexported**

Nur Methoden mit unexported-Sichtbarkeit (nur über my aufrufbar).

**private**

Nur Methoden mit private-Sichtbarkeit (nur innerhalb der Methoden dieser Klasse aufrufbar).

**info class methodtype *class method***

Beschreibung des Implementierungstyps der Methode *method* von *class*. Bei Ergebnis **method** weitere Infos mit **info class definition**, bei **forward** mit **info class forward**.

**info class mixins *class***

Liste aller in *class* gemischten Klassen.

**info class properties *class* ?*options...*?**

Sortierte Liste der Eigenschaften von class. options steuern die Auswahl:

**-all**

Mit dieser Option auch Eigenschaften von Superklassen und Mixins.

**-readable**

Diese Option (Standard) liefert lesbare Eigenschaften. Es werden nur lesbare oder schreibbare Eigenschaften zurückgegeben, nicht beides.

**-writable**

Liefert schreibbare Eigenschaften. Es werden nur lesbare oder schreibbare Eigenschaften zurückgegeben, nicht beides.

**info class subclasses *class* ?*pattern*?**

Liste direkter Unterklassen von *class*. Mit *pattern* nur passende Klassen gemäß string match.

**info class superclasses *class***

Liste direkter Superklassen von *class* in Vererbungsreihenfolge.

**info class variables *class* ?-private?**

Liste aller für *class* deklarierten Variablen (automatisch in Methoden, Konstruktor und Destruktor vorhanden). Mit -private die deklarierten privaten Variablen.

## Objekt-Introspection

Folgende *subcommand*-Werte unterstützt **info object**:

**info object call *object method***

Beschreibung der Methodenimplementierungen für *object*'s Implementierung von *method*. Viererlisten wie bei **info class call**; das dritte Wort gibt an, was die Methode definierte (vollqualifizierter Klassenname oder **object** bei Instanzimplementierung), das vierte den Implementierungstyp (siehe **info object methodtype**).

Es wird nicht geprüft, ob die Implementierungen next zur Weitergabe entlang der Aufrufkette nutzen; die von diesem Befehl gelieferten Aufrufketten enthalten private Methoden nicht.

**info object class *object* ?*className*?**

Ohne *className* die Klasse von *object*. Mit *className* ein boolescher Wert, ob *object* von dieser Klasse ist.

**info object creationid *object***

Eindeutige Erstellungskennung von object. Sie ist pro Objekt (innerhalb eines Tcl-Interpreters) eindeutig und kann weder bei der Erstellung gesetzt noch später geändert werden.

*Implementierungshinweis:* Die Erstellungskennung erzeugt eindeutige Bezeichner für das Objekt, insbesondere für private Variablen.

**info object definition *object method***

Beschreibung der Definition der Methode *method* von *object* als Zweierliste (Argumentliste und Methodenrumpf).

**info object filters *object***

Liste der auf dem Objekt gesetzten Filtermethoden.

**info object forward *object method***

Argumentliste der Methodenweiterleitung *method* auf *object*.

**info object isa *category object* ?*arg*?**

Prüft, ob ein Objekt zu einer Kategorie gehört; boolescher Wert. Unterstützte Kategorien:

**info object isa class *object***

Ob *object* eine Klasse ist (Instanz von oo::class oder Unterklasse).

**info object isa metaclass *object***

Ob *object* eine Klasse ist, die Klassen erzeugen kann (oo::class oder Unterklasse).

**info object isa mixin *object class***

Ob *class* direkt in *object* gemischt ist.

**info object isa object *object***

Ob *object* wirklich ein Objekt ist.

**info object isa typeof *object class***

Ob *class* der Typ von *object* ist (*object* Instanz von *class* oder Unterklasse, direkt oder indirekt).

**info object methods *object* ?*option...*?**

Liste aller öffentlichen Methoden von *object*. *option*s wie bei **info class methods**:

**-all**

Mit **-all** und ohne -scope auch Methoden von Klasse, Mixins und deren Superklassen.

**-private**

Mit **-private** und ohne -scope auch nicht exportierte Methoden (und bei **-all** von Klassen). Namenskollision mit echten privaten Methoden; Optionsname aus Kompatibilität.

**-scope *scope***

Methoden von object mit Sichtbarkeit scope; -all und -private werden ignoriert. Gültige Werte:

**public**

Nur public (von überall aufrufbar).

**unexported**

Nur unexported (nur über my).

**private**

Nur private (nur innerhalb der Instanzmethoden dieses Objekts).

**info object methodtype *object method***

Implementierungstyp der Methode *method* von *object*. Bei **method** → **info object definition** , bei **forward** → **info object forward**.

**info object mixins *object***

Liste aller in *object* gemischten Klassen.

**info object namespace *object***

Name des internen Namespace von *object*.

**info object properties *object* ?*options...*?**

Sortierte Liste der Eigenschaften von object. options wie bei info class properties:

**-all**

Auch Eigenschaften von Klasse, Superklassen und Mixins.

**-readable**

Standard: lesbare Eigenschaften. Nur lesbare oder schreibbare, nicht beides.

**-writable**

Schreibbare Eigenschaften. Nur lesbare oder schreibbare, nicht beides.

**info object variables *object* ?-private?**

Liste aller für *object* deklarierten Variablen (automatisch in den Methoden des Objekts). Mit -private die privaten Variablen.

**info object vars *object* ?*pattern*?**

Liste aller Variablen im privaten Namespace von *object*. Mit *pattern* ein Filter (string match -Glob). Anders als **info object variables**: jene kann noch nicht gesetzte Variablen enthalten; dieser Befehl Variablen, die von keiner Methode von *object* (oder Klasse, Superklassen, Mixins) automatisch eingebunden werden.

## Beispiele

Dieser Befehl gibt eine zum Speichern in einem Tcl-Skript geeignete Prozedur aus:

```
proc printProc {procName} {
    set result [list proc $procName]
    set formals {}
    foreach var [info args $procName] {
        if {[info default $procName $var def]} {
            lappend formals [list $var $def]
        } else {
            # Still need the list-quoting because variable
            # names may properly contain spaces.
            lappend formals [list $var]
        }
    }
    puts [lappend result $formals [info body $procName]]
}
```

### Beispiele mit Objekten

Jedes Objekt kennt seine Klasse; diese Information ist per Introspection leicht abrufbar:

```
oo::class create c
c create o
puts [info object class o]
                     → prints "::c"
puts [info object class c]
                     → prints "::oo::class"
```

Die Introspection kann ermitteln, welche Klasse eine Methode implementiert und wie sie definiert ist:

```
proc getDef {obj method} {
    foreach inf [info object call $obj $method] {
        lassign $inf calltype name locus methodtype

        # Annahme: keine Weiterleitungen oder Filter, daher keine
        # Prüfung von $calltype oder $methodtype...

        if {$locus eq "object"} {
            return [info object definition $obj $name]
        } else {
            return [info class definition $locus $name]
        }
    }
    error "no definition for $method"
}
```

Alternativ durch manuelles Durchlaufen der Vererbungskette (einfache Einzelvererbung, keine komplexen Mixins; dann ist **info object call** deutlich einfacher):

```
proc getDef {obj method} {
    if {$method in [info object methods $obj]} {
        # Annahme: keine Weiterleitungen
        return [info object definition $obj $method]
    }

    set cls [info object class $obj]

    while {$method ni [info class methods $cls]} {
        # Einfacher Fall angenommen
        set cls [lindex [info class superclass $cls] 0]
        if {$cls eq ""} {
            error "no definition for $method"
        }
    }

    # Annahme: keine Weiterleitungen
    return [info class definition $cls $method]
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*
*Copyright (c) 1993-1997 Bell Labs Innovations for Lucent Technologies*
*Copyright (c) 1998-2000 Ajuba Solutions*
*Copyright (c) 2007-2012 Donal K. Fellows*

# interp

*Stichwörter:* interp, alias, parent interpreter, safe interpreter, child interpreter

## Name

interp - Erstellt und manipuliert Tcl-Interpreter

## Übersicht

**interp** *subcommand* ?*arg arg* ...?

## Beschreibung

Dieser Befehl ermöglicht das Erstellen eines oder mehrerer neuer Tcl-Interpreter, die zusammen mit dem erstellenden Interpreter in derselben Anwendung koexistieren. Der erstellende Interpreter heißt *parent* (Eltern-Interpreter) und der neue Interpreter heißt *child* (Kind-Interpreter). Ein Eltern-Interpreter kann beliebig viele Kinder erstellen, und jedes Kind kann selbst weitere Kinder erstellen, für die es Eltern-Interpreter ist — so entsteht eine Hierarchie von Interpretern.

Jeder Interpreter ist unabhängig von den anderen: Er hat seinen eigenen Namensraum für Befehle, Prozeduren und globale Variablen. Ein Eltern-Interpreter kann Verbindungen zwischen seinen Kindern und sich selbst über einen Mechanismus namens *Alias* herstellen. Ein *Alias* ist ein Befehl in einem Kind-Interpreter, der bei Aufruf dazu führt, dass ein Befehl im Eltern-Interpreter oder in einem anderen Kind-Interpreter aufgerufen wird. Die einzigen weiteren Verbindungen zwischen Interpretern bestehen über Umgebungsvariablen (die Variable env), die normalerweise von allen Interpretern in der Anwendung gemeinsam genutzt werden, sowie über Callbacks bei überschrittenen Ressourcenlimits. Beachten Sie, dass der Namensraum für Dateien (wie die von open zurückgegebenen Namen) zwischen Interpretern nicht mehr gemeinsam genutzt wird. Explizite Befehle ermöglichen das Teilen von Dateien und das Übertragen von Referenzen auf geöffnete Dateien von einem Interpreter zu einem anderen.

Der Befehl **interp** unterstützt auch *sichere* Interpreter. Ein sicherer Interpreter ist ein Kind, dessen Funktionen stark eingeschränkt sind, sodass nicht vertrauenswürdige Skripte ausgeführt werden können, ohne dass andere Interpreter oder die Umgebung der Anwendung beschädigt werden. Beispielsweise sind alle Befehle zur Erstellung von IO-Kanälen und zur Erstellung von Unterprozessen für sichere Interpreter nicht zugänglich. Siehe Sichere Interpreter unten für weitere Informationen, welche Funktionen in einem sicheren Interpreter vorhanden sind. Die gefährliche Funktionalität wird nicht aus dem sicheren Interpreter entfernt; stattdessen wird sie *verborgen*, sodass nur vertrauenswürdige Interpreter Zugriff darauf erhalten. Eine ausführliche Erklärung verborgener Befehle finden Sie unter Verborgene Befehle unten. Der Alias-Mechanismus kann für geschützte Kommunikation (analog zu einem Kernel-Aufruf) zwischen einem Kind-Interpreter und seinem Eltern-Interpreter verwendet werden. Siehe Alias-Aufruf unten für Details zum Alias-Mechanismus.

Ein qualifizierter Interpretername ist eine wohlgeformte Tcl-Liste, die eine Teilmenge seiner Vorfahren in der Interpreter-Hierarchie enthält, abgeschlossen durch die Zeichenkette, die den Interpreter in seinem unmittelbaren Eltern-Interpreter benennt. Interpreter-Namen sind relativ zum Interpreter, in dem sie verwendet werden. Wenn beispielsweise "**a**" ein Kind des aktuellen Interpreters ist und ein Kind "**a1**" hat, das wiederum ein Kind "**a11**" hat, ist der qualifizierte Name von "**a11**" in "**a**" die Liste "**a1 a11**".

Der unten beschriebene Befehl **interp** akzeptiert qualifizierte Interpreter-Namen als Argumente; der Interpreter, in dem der Befehl ausgewertet wird, kann stets als **{}** (die leere Liste oder Zeichenkette) bezeichnet werden. Beachten Sie, dass es unmöglich ist, in einem Kind-Interpreter per Name auf einen Eltern- (Vorfahren-)Interpreter zu verweisen, außer über Aliase. Außerdem gibt es keinen globalen Namen, mit dem man auf den ersten in einer Anwendung erstellten Interpreter verweisen kann. Beide Einschränkungen dienen Sicherheitszwecken.

## Der interp-Befehl

Der Befehl **interp** dient zum Erstellen, Löschen und Manipulieren von Kind-Interpretern sowie zum Teilen oder Übertragen von Kanälen zwischen Interpretern. Je nach dem Argument *subcommand* kann er mehrere Formen annehmen:

**interp alias *srcPath srcToken***

Gibt eine Tcl-Liste zurück, deren Elemente *targetCmd* und *arg* des durch *srcToken* repräsentierten Alias sind (dies ist der Wert, der bei Erstellung des Alias zurückgegeben wurde; der Name des Quellbefehls im Kind kann von *srcToken* abweichen).

**interp alias *srcPath srcToken* {}**

Löscht den Alias für *srcToken* im Kind-Interpreter, der durch *srcPath* identifiziert wird. *srcToken* bezieht sich auf den bei Erstellung des Alias zurückgegebenen Wert; wurde der Quellbefehl umbenannt, wird der umbenannte Befehl gelöscht.

**interp alias *srcPath srcCmd targetPath targetCmd ?arg arg ...?***

Dieser Befehl erstellt einen Alias zwischen zwei Kind-Interpretern (siehe den Child-Befehl **alias** unten für Aliase zwischen einem Kind und seinem Eltern-Interpreter). In diesem Befehl kann jeder der Kind-Interpreter beliebig in der Hierarchie der Interpreter unter dem aufrufenden Interpreter stehen. *SrcPath* und *srcCmd* identifizieren die Quelle des Alias. *SrcPath* ist eine Tcl-Liste, deren Elemente einen bestimmten Interpreter auswählen. Beispielsweise identifiziert "**a b**" einen Interpreter "**b**", der Kind des Interpreters "**a**" ist, der wiederum Kind des aufrufenden Interpreters ist. Eine leere Liste bezeichnet den aufrufenden Interpreter. *srcCmd* gibt den Namen eines neuen Befehls an, der im Quell-Interpreter erstellt wird. *TargetPath* und *targetCmd* geben Ziel-Interpreter und -Befehl an; die *arg*-Argumente, falls vorhanden, legen zusätzliche Argumente für *targetCmd* fest, die den bei Aufruf von *srcCmd* angegebenen Argumenten vorangestellt werden. *TargetCmd* kann zum Zeitpunkt dieses Aufrufs undefiniert sein oder bereits existieren; er wird durch diesen Befehl nicht erstellt. Der Alias sorgt dafür, dass der angegebene Zielbefehl im Ziel-Interpreter aufgerufen wird, sobald der angegebene Quellbefehl im Quell-Interpreter aufgerufen wird. Siehe Alias-Aufruf unten für weitere Details. Der Befehl gibt ein Token zurück, das den erstellten Befehl *srcCmd* eindeutig identifiziert, auch wenn der Befehl später umbenannt wird. Das Token kann, muss aber nicht gleich *srcCmd* sein.

**interp aliases ?*path*?**

Dieser Befehl gibt eine Tcl-Liste der Tokens aller Quellbefehle für im Interpreter *path* definierte Aliase zurück. Die Tokens entsprechen den bei Erstellung der Aliase zurückgegebenen Werten (die von den aktuellen Befehlsnamen abweichen können).

**interp bgerror *path* ?*cmdPrefix*?**

Dieser Befehl liest oder setzt den aktuellen Handler für Hintergrund-Ausnahmen des durch *path* identifizierten Interpreters. Fehlt *cmdPrefix*, wird der aktuelle Handler zurückgegeben; ist er vorhanden, ist es eine Liste von Wörtern (mindestens Länge eins), die beschreibt, womit der Hintergrund-Ausnahme-Handler des Interpreters gesetzt werden soll. Siehe den Abschnitt Hintergrund-Ausnahmebehandlung für weitere Details.

**interp cancel ?-unwind? ?--? ?*path*? ?*result*?**

Bricht die Auswertung des Skripts im durch *path* identifizierten Interpreter ab. Ohne den Schalter **-unwind** wird der Auswertungsstack des Interpreters entladen, bis ein umschließender catch-Befehl gefunden wird oder keine weiteren Aufrufe des Interpreters auf dem Call-Stack verbleiben. Mit **-unwind** wird der Auswertungsstack ohne Rücksicht auf zwischenliegende catch-Befehle entladen, bis keine weiteren Aufrufe des Interpreters auf dem Call-Stack verbleiben. Der Schalter **-|-** markiert das Ende der Schalter; er kann nötig sein, wenn *path* ein unüblicher Wert wie **-safe** ist. Ist *result* angegeben, wird es als Fehlermeldung verwendet; andernfalls eine Standard-Fehlermeldung.

**interp children ?*path*?**

Gibt eine Tcl-Liste der Namen aller Kind-Interpreter zurück, die mit dem durch *path* identifizierten Interpreter verknüpft sind. Fehlt *path*, wird der aufrufende Interpreter verwendet.

**interp create ?-safe? ?--? ?*path*?**

Erstellt einen Kind-Interpreter *path* und einen neuen Befehl, den *Child-Befehl*. Der Name des Child-Befehls ist die letzte Komponente von *path*. Der neue Kind-Interpreter und der Child-Befehl werden im Interpreter erstellt, der durch den Pfad identifiziert wird, der durch Entfernen der letzten Komponente von *path* entsteht. Ist *path* beispielsweise **a b c**, werden ein neuer Kind-Interpreter und ein Child-Befehl **c** im Interpreter mit Pfad **a b** erstellt. Der

Child-Befehl kann den neuen Interpreter wie unten beschrieben steuern. Fehlt *path*, erzeugt Tcl einen eindeutigen Namen der Form **interp***x*, wobei *x* eine Ganzzahl ist, und verwendet ihn für Interpreter und Child-Befehl. Ist der Schalter **-safe** gesetzt (oder ist der Eltern-Interpreter ein sicherer Interpreter), wird der neue Kind-Interpreter als sicherer Interpreter mit eingeschränkter Funktionalität erstellt; andernfalls enthält das Kind den vollen Satz Tcl-Builtin-Befehle und -Variablen. **-|-** markiert das Ende der Schalter; nötig, wenn *path* ein unüblicher Wert wie **-safe** ist. Das Ergebnis ist der Name des neuen Interpreters. Der Name eines Kind-Interpreters muss unter allen Kindern seines Eltern-Interpreters eindeutig sein; existiert bereits ein Kind mit diesem Namen, tritt ein Fehler auf. Das anfängliche Rekursionslimit des Kind-Interpreters entspricht dem aktuellen Rekursionslimit seines Eltern-Interpreters.

**interp debug *path* ?*-frame bool*?**

Steuert, ob Frame-Level-Stack-Informationen im durch *path* identifizierten Kind-Interpreter erfasst werden. Ohne Argumente werden Option und aktuelle Einstellung zurückgegeben. Mit **-frame** wird die Debug-Einstellung auf den angegebenen booleschen Wert gesetzt (falls angegeben) und die aktuelle Einstellung zurückgegeben. Dies betrifft nur die Ausgabe von info frame: exakte Frame-Level-Informationen für Befehlsaufrufe auf Bytecode-Ebene werden nur mit dieser Einstellung erfasst.

Beispielsweise liefert bei Code wie

```
proc mycontrol {... script} {
  ...
  uplevel 1 $script
  ...
}

proc dosomething {...} {
  ...
  mycontrol {
    somecode
  }
}
```

die Standardeinstellung eine relative Zeilennummer für **somecode** und der relevante Frame ist vom Typ eval. Ist Frame-Debug aktiv, reicht die Verfolgung so weit, dass Datei und absolute Zeilennummer des Befehls bestimmt werden können und ein Frame vom Typ source zurückgegeben wird. Diese genauere Information kostet langsamere Ausführung aller Befehle.

Beachten Sie: Ist das Flag einmal gesetzt, kann es nicht wieder deaktiviert werden; solche Versuche werden still ignoriert. Das dient der Konsistenz des Zustands des zugrunde liegenden Interpreters.

**interp delete ?*path* ...?**

Löscht null oder mehr Interpreter gemäß den optionalen *path*-Argumenten; für jeden Interpreter werden auch seine Kinder gelöscht. Der Befehl löscht außerdem den Child-Befehl jedes gelöschten Interpreters. Existiert für ein *path*-Argument kein Interpreter mit diesem Namen, löst der Befehl einen Fehler aus.

**interp eval *path arg ?arg …?***

Dieser Befehl verkettet alle *arg*-Argumente wie concat und wertet die resultierende Zeichenkette als Tcl-Skript im durch *path* identifizierten Kind-Interpreter aus. Das Ergebnis (einschließlich aller return-Optionen wie **-errorinfo** und **-errorcode** bei Fehlern) wird an den aufrufenden Interpreter zurückgegeben.

Das Skript wird im aktuellen Kontext-Stack-Frame des *path*-Interpreters ausgeführt; so können Implementierungen (im Eltern-Interpreter) von Aliasen in einem Kind-Interpreter Skripte im Kind ausführen, die Informationen über aktuellen Zustand und Stack-Frame des Kindes ermitteln.

**interp exists *path***

Gibt **1** zurück, wenn ein Kind-Interpreter mit dem angegebenen *path* in diesem Eltern-Interpreter existiert, sonst **0**. Fehlt *path*, wird der aufrufende Interpreter verwendet.

**interp expose *path hiddenName ?exposedCmdName?***

Macht den verborgenen Befehl *hiddenName* sichtbar und stellt ihn gegebenenfalls unter dem neuen Namen *exposedCmdName* wieder her (dieser Name wird derzeit nur akzeptiert, wenn er ein gültiger globaler Namensraumname ohne :: ist), im durch *path* bezeichneten Interpreter. Existiert bereits ein sichtbarer Befehl mit dem Zielnamen, schlägt der Befehl fehl. Verborgene Befehle werden unter Verborgene Befehle unten ausführlicher erklärt.

**interp hide *path exposedCmdName ?hiddenCmdName?***

Macht den sichtbaren Befehl *exposedCmdName* verborgen, benennt ihn in *hiddenCmdName* um oder behält den Namen bei, wenn *hiddenCmdName* fehlt, im durch *path* bezeichneten Interpreter. Existiert bereits ein verborgener Befehl mit dem Zielnamen, schlägt der Befehl fehl. Derzeit dürfen weder *exposedCmdName* noch *hiddenCmdName* Namespace-Qualifizierer enthalten, sonst wird ein Fehler ausgelöst. Von **interp hide** zu verbergende Befehle werden im globalen Namespace gesucht, auch wenn der aktuelle Namespace nicht der globale ist. Das verhindert, dass Kinder einen Eltern-Interpreter dazu bringen, den falschen Befehl zu verbergen, indem sie den aktuellen Namespace vom globalen unterscheiden. Verborgene Befehle werden unter Verborgene Befehle unten ausführlicher erklärt.

**interp hidden *path***

Gibt eine Liste der Namen aller verborgenen Befehle im durch *path* identifizierten Interpreter zurück.

**interp invokehidden *path ?-option …? hiddenCmdName ?arg …?***

Ruft den verborgenen Befehl *hiddenCmdName* mit den übergebenen Argumenten im durch *path* bezeichneten Interpreter auf. Auf die Argumente werden keine Substitutionen oder Auswertungen angewendet. Drei *-option*en werden unterstützt, alle beginnend mit **-**: **-namespace** (mit einem folgenden Argument *nsName*), **-global** und **-|-**. Ist **-namespace** gesetzt, wird der verborgene Befehl im Namespace *nsName* im Ziel-Interpreter aufgerufen. Ist **-global** gesetzt, wird er auf globaler Ebene im Ziel-Interpreter aufgerufen; andernfalls im aktuellen Call-Frame mit Zugriff auf lokale Variablen in diesem und äußeren Frames. **-|-** erlaubt, dass *hiddenCmdName* mit "-" beginnt, und ist sonst unnötig. Sind **-namespace** und **-global** gesetzt, wird **-namespace** ignoriert.

Der verborgene Befehl wird (standardmäßig) im aktuellen Kontext-Stack-Frame des *path* -Interpreters ausgeführt.

Verborgene Befehle werden unter Verborgene Befehle unten ausführlicher erklärt.

**interp issafe ?*path*?**

Gibt **1** zurück, wenn der durch *path* identifizierte Interpreter sicher ist, sonst **0**.

**interp limit *path limitType* ?*-option*? ?*value* ...?**

Richtet die Konfiguration des Ressourcenlimits *limitType* für den durch *path* bezeichneten Interpreter ein, manipuliert und abfragt sie. Ist keine *-option* angegeben, wird die aktuelle Konfiguration zurückgegeben. Ist *-option* das einzige Argument, der Wert dieser Option. Andernfalls muss eine Liste von *-option*/*value*-Paaren angegeben werden. Siehe Ressourcenlimits unten für Limits und Optionen.

**interp marktrusted *path***

Markiert den durch *path* identifizierten Interpreter als vertrauenswürdig. Macht verborgene Befehle nicht sichtbar. Der Befehl kann nur aus einem vertrauenswürdigen Interpreter aufgerufen werden. Hat keine Wirkung, wenn der Interpreter bereits vertrauenswürdig ist.

**interp recursionlimit *path* ?*newlimit*?**

Gibt die maximal zulässige Verschachtelungstiefe für den durch *path* angegebenen Interpreter zurück. Ist *newlimit* gesetzt, wird das Rekursionslimit so gesetzt, dass mehr als *newlimit* Aufrufe von Tcl_Eval und verwandten Prozeduren in diesem Interpreter einen Fehler ergeben. *newlimit* wird ebenfalls zurückgegeben. *newlimit* muss eine positive Ganzzahl zwischen 1 und dem Maximum eines non-long-Integer auf der Plattform sein.

Der Befehl begrenzt nur die Größe des Tcl-Call-Stacks. Er verhindert allein keine Stack-Überläufe auf dem vom Programm genutzten C-Stack. Hat Ihr System ein Limit für die C-Stack-Größe, können Überläufe auftreten, bevor das durch den Befehl gesetzte Limit erreicht ist. Prüfen Sie in diesem Fall, ob sich die maximale C-Stack-Größe erhöhen lässt.

**interp share *srcPath channel destPath***

Bewirkt, dass der IO-Kanal *channel* zwischen dem durch *srcPath* und *destPath* identifizierten Interpreter geteilt wird. Beide haben dieselben Berechtigungen auf dem Kanal. Beide müssen schließen, um den zugrunde liegenden Kanal zu schließen; in einem Interpreter zugängliche Kanäle werden beim Zerstören des Interpreters automatisch geschlossen.

**interp target *path alias***

Gibt eine Tcl-Liste zurück, die den Ziel-Interpreter für einen Alias beschreibt. Der Alias wird wie bei **interp alias** oben durch Interpreter-Pfad und Quellbefehlsname angegeben. Der Ziel-Interpreter-Name wird als Interpreter-Pfad relativ zum aufrufenden Interpreter zurückgegeben. Ist der Ziel-Interpreter der aufrufende Interpreter, wird eine leere Liste zurückgegeben. Ist er weder der aufrufende noch einer seiner Nachkommen, wird ein Fehler erzeugt. Der Zielbefehl muss zum Zeitpunkt des Aufrufs nicht definiert sein.

**interp transfer *srcPath channel destPath***

Bewirkt, dass der IO-Kanal *channel* im durch *destPath* identifizierten Interpreter verfügbar und im durch *srcPath* identifizierten Interpreter nicht mehr verfügbar ist.

## Child-Befehl

Für jeden mit **interp** erstellten Kind-Interpreter wird im Eltern-Interpreter ein neuer Tcl-Befehl mit demselben Namen wie der Interpreter erstellt. Dieser Befehl steuert verschiedene Operationen am Interpreter. Allgemeine Form:

```
child command ?arg arg ...?
```

*Child* ist der Interpretername; *command* und die *arg*s bestimmen das Verhalten. Gültige Formen:

***child* aliases**

Gibt eine Tcl-Liste der Tokens aller Aliase in *child* zurück. Die Tokens entsprechen den bei Erstellung zurückgegebenen Werten (die von den aktuellen Befehlsnamen abweichen können).

***child* alias *srcToken***

Gibt eine Tcl-Liste zurück, deren Elemente *targetCmd* und *arg* des durch *srcToken* repräsentierten Alias sind (Wert bei Erstellung; der tatsächliche Quellbefehl im Kind kann von *srcToken* abweichen).

***child* alias *srcToken* {}**

Löscht den Alias für *srcToken* im Kind-Interpreter. *srcToken* bezieht sich auf den bei Erstellung zurückgegebenen Wert; wurde der Quellbefehl umbenannt, wird der umbenannte Befehl gelöscht.

***child* alias *srcCmd targetCmd* ?*arg..*?**

Erstellt einen Alias: wird *srcCmd* in *child* aufgerufen, wird *targetCmd* im Eltern-Interpreter aufgerufen. Die *arg*-Argumente werden *targetCmd* vorangestellt, vor den Argumenten des *srcCmd*-Aufrufs. Siehe Alias-Aufruf unten. Der Befehl gibt ein Token zurück, das den erstellten Befehl *srcCmd* eindeutig identifiziert, auch nach Umbenennung. Das Token kann, muss aber nicht gleich *srcCmd* sein.

***child* bgerror ?*cmdPrefix*?**

Liest oder setzt den Handler für Hintergrund-Ausnahmen des *child*-Interpreters. Fehlt *cmdPrefix*, wird der aktuelle Handler zurückgegeben; ist er vorhanden, eine Liste von Wörtern (mindestens Länge eins) für die neue Konfiguration. Siehe Hintergrund-Ausnahmebehandlung.

***child* eval *arg* ?*arg..*?**

Verkettet alle *arg*-Argumente wie concat und wertet die Zeichenkette als Tcl-Skript in *child* aus. Das Ergebnis (einschließlich return-Optionen wie **-errorinfo** und **-errorcode** bei Fehlern) geht an den aufrufenden Interpreter zurück.

Das Skript läuft im aktuellen Kontext-Stack-Frame von *child*; so können Alias-Implementierungen im Eltern-Interpreter Skripte im Kind ausführen, die Zustand und Stack-Frame des Kindes ermitteln.

***child* expose *hiddenName* ?*exposedCmdName*?**

Macht den verborgenen Befehl *hiddenName* in *child* sichtbar und stellt ihn gegebenenfalls unter *exposedCmdName* wieder her (nur gültiger globaler Namensraumname ohne ::). Existiert bereits ein sichtbarer Befehl mit dem Zielnamen, schlägt der Befehl fehl. Siehe Verborgene Befehle unten.

***child* hide *exposedCmdName* ?*hiddenCmdName*?**

Verbirgt *exposedCmdName* in *child*, benennt ihn in *hiddenCmdName* um oder behält den Namen. Existiert bereits ein verborgener Befehl mit dem Zielnamen, schlägt der Befehl fehl. Derzeit dürfen weder *exposedCmdName* noch *hiddenCmdName* Namespace-Qualifizierer enthalten. Zu verbergende Befehle werden im globalen Namespace gesucht, auch wenn der aktuelle Namespace nicht global ist — damit Kinder den Eltern-Interpreter nicht zum Verbergen des falschen Befehls verleiten. Siehe Verborgene Befehle unten.

***child*** **hidden**

Gibt eine Liste aller verborgenen Befehlsnamen in *child* zurück.

***child*** **invokehidden ?*-option* ...? *hiddenName* ?*arg*..?**

Ruft *hiddenName* in *child* mit den übergebenen Argumenten auf. Keine Substitutionen oder Auswertungen auf den Argumenten. Drei *-option*en: **-namespace** (mit *nsName*), **-global**, **-|-**. Mit **-namespace** Aufruf im angegebenen Namespace im Kind; mit **-global** auf globaler Ebene; sonst im aktuellen Call-Frame mit Zugriff auf lokale Variablen. **-|-** erlaubt Befehlsnamen mit führendem "-". Sind **-namespace** und **-global** gesetzt, wird **-namespace** ignoriert.

Der verborgene Befehl läuft (standardmäßig) im aktuellen Kontext-Stack-Frame von *child*.

Siehe Verborgene Befehle unten.

***child*** **issafe**

Gibt **1** zurück, wenn der Kind-Interpreter sicher ist, sonst **0**.

***child*** **limit *limitType* ?*-option*? ?*value* ...?**

Richtet die Konfiguration des Ressourcenlimits *limitType* für *child* ein, manipuliert und abfragt sie. Ohne *-option* die aktuelle Konfiguration; nur *-option* der Optionswert; sonst *-option*/*value* -Paare. Siehe Ressourcenlimits unten.

***child*** **marktrusted**

Markiert den Kind-Interpreter als vertrauenswürdig. Nur aus vertrauenswürdigem Interpreter aufrufbar. Macht keine verborgenen Befehle sichtbar. Keine Wirkung, wenn das Kind bereits vertrauenswürdig ist.

***child*** **recursionlimit ?*newlimit*?**

Gibt die maximal zulässige Verschachtelungstiefe für *child* zurück. Mit *newlimit* wird das Rekursionslimit so gesetzt, dass mehr als *newlimit* Aufrufe von Tcl_Eval**()** und verwandten Prozeduren in *child* einen Fehler ergeben. *newlimit* wird zurückgegeben; muss eine positive Ganzzahl zwischen 1 und dem Plattform-Maximum sein.

Begrenzt nur den Tcl-Call-Stack, nicht C-Stack-Überläufe. Bei C-Stack-Limits des Systems können Überläufe vor Erreichen des gesetzten Limits auftreten; prüfen Sie dann eine Erhöhung der maximalen C-Stack-Größe.

## Sichere Interpreter

Ein sicherer Interpreter hat eingeschränkte Funktionalität, sodass beliebige Skripte selbst von nicht vertrauenswürdigen Quellen ausgeführt werden können, ohne die umschließende Anwendung oder die übrige Umgebung zu beschädigen. Um einen Interpreter sicher zu machen, werden bestimmte Befehle und Variablen entfernt. Beispielsweise entfallen Befehle zum Anlegen von Dateien auf der Platte und der Befehl exec, da er über Unterprozesse Schaden anrichten könnte. Eingeschränkter Zugriff kann über Aliase zum Eltern-Interpreter bereitgestellt werden, die Argumente prüfen und nur eine sichere Teilmenge erlauben — z. B. Dateierstellung nur in einem Unterverzeichnis oder Unterprozessaufrufe nur für eine fest definierte Programmliste.

Ein sicherer Interpreter entsteht durch den Schalter **-safe** bei **interp create**. Jedes von einem sicheren Interpreter erzeugte Kind ist ebenfalls sicher.

Ein sicherer Interpreter wird mit genau folgendem Satz Builtin-Befehle erstellt:

after append apply array
binary break catch chan
clock close concat continue
dict eof error eval
expr fblocked fcopy fileevent
flush for foreach format
gets global if incr
info **interp** join lappend
lassign ledit lindex linsert
list llength lrange lrepeat
lreplace lsearch lseq lset
lsort namespace package pid
proc puts read regexp
regsub rename return scan
seek set split string
subst switch tell **time**
trace unset update uplevel
upvar variable vwait while
zlib

Folgende Befehle verbirgt **interp create** beim Erzeugen eines sicheren Interpreters:

cd encoding exec exit
fconfigure file glob load
open pwd socket source
unload zipfs

Diese Befehle können später als Tcl-Prozeduren oder Aliase neu erstellt oder mit **interp expose** wieder sichtbar gemacht werden.

Folgende Befehle aus Tcl's Bibliothek von Hilfsprozeduren fehlen in einem sicheren Interpreter:

**auto_exec_ok** auto_import auto_load
**auto_load_index** auto_qualify unknown

Insbesondere haben sichere Interpreter keinen Standard-unknown-Befehl; Tcl's Autoload-Funktionen stehen nicht zur Verfügung. Autoload-Zugriff auf normalerweise per Autoload geladene Tcl-Befehle:

auto_mkindex **auto_mkindex_old**
auto_reset history
parray pkg_mkIndex
::pkg::create ::safe::interpAddToAccessPath
::safe::interpCreate ::safe::interpConfigure
::safe::interpDelete ::safe::interpFindInAccessPath
::safe::interpInit ::safe::setLogCmd
tcl_endOfWord tcl_findLibrary
tcl_startOfNextWord tcl_startOfPreviousWord
tcl_wordBreakAfter tcl_wordBreakBefore

ist nur durch explizite Definition eines unknown-Befehls im sicheren Interpreter möglich — dazu muss source sichtbar gemacht werden. Am einfachsten durch Erstellung des sicheren

Interpreters mit Tcl's **Safe-Tcl**-Mechanismus; **Safe-Tcl** liefert sichere Varianten von source, load und weiteren Befehlen für Autoload und Paketladen.

Außerdem fehlt die Variable env im sicheren Interpreter; Umgebungsvariablen können nicht mit anderen Interpretern geteilt werden. env birgt ein Sicherheitsrisiko, da Nutzer sensible Daten in Umgebungsvariablen ablegen können — z. B. empfiehlt das PGP-Handbuch das Passwort zum Schutz des privaten Schlüssels in *PGPPASS*. Diese Variable für nicht vertrauenswürdigen Code in einem sicheren Interpreter verfügbar zu machen, wäre ein Sicherheitsrisiko.

Werden Erweiterungen in einen sicheren Interpreter geladen, können sie ihre Funktionalität ebenfalls einschränken, um unsichere Befehle zu entfernen. Zur Verwaltung von Erweiterungen im Hinblick auf Sicherheit siehe **Safe-Tcl** und den Tcl-Befehl load.

Ein sicherer Interpreter darf das Rekursionslimit keines Interpreters ändern, auch nicht seines eigenen.

## Alias-Aufruf

Der Alias-Mechanismus ist so ausgelegt, dass er sicher in nicht vertrauenswürdigen Skripten in einem sicheren Interpreter verwendet werden kann, selbst wenn das Alias-Ziel kein sicherer Interpreter ist. Entscheidend für die Sicherheit ist, dass vom Kind an den Eltern übergebene Informationen im Eltern-Interpreter nie ausgewertet oder substituiert werden; sonst könnte ein bösartiges Kind-Skript beliebige Funktionen im Eltern aufrufen und die Sicherheit gefährden.

Wird die Alias-Quelle im Kind-Interpreter aufgerufen, erfolgen beim Parsen die üblichen Tcl-Substitutionen im Quell-Interpreter wie bei jedem anderen Befehl. Die Befehlsprozedur der Quelle führt ihre Argumente mit *targetCmd* und *arg* des Alias zu einem neuen Argumentarray zusammen. Waren die Wörter von *srcCmd* "*srcCmd arg1 arg2 ... argN*", lautet die neue Wortfolge "*targetCmd arg arg ... arg arg1 arg2 ... argN*", wobei *targetCmd* und *arg* die bei Alias-Erstellung gesetzten Werte sind. *TargetCmd* wird verwendet, um die Befehlsprozedur im Ziel-Interpreter zu finden; diese wird mit den neuen Argumenten aufgerufen. Fehlt ein Befehl *targetCmd* im Ziel-Interpreter, tritt ein Fehler auf. Auf den Wörtern erfolgen keine weiteren Substitutionen: Die Zielbefehlsprozedur wird direkt aufgerufen, ohne den normalen Tcl-Auswertungsmechanismus. Jede Substitutionsrunde erfolgt genau einmal: *targetCmd* und *args* beim Parsen des Alias-Erstellungsbefehls, *arg1* bis *argN* beim Parsen des Alias-Quellbefehls im Quell-Interpreter.

Beim Schreiben von *targetCmd* für Aliase in sicheren Interpretern dürfen die Argumente dieses Befehls nie ausgewertet oder substituiert werden — sonst entstünde ein Ausweg, über den das Kind beliebigen Code im Eltern ausführen könnte und die Systemsicherheit gefährdet wäre.

## Verborgene Befehle

Sichere Interpreter schränken die für Tcl-Programme verfügbare Funktionalität stark ein. Direkter Zugriff für nicht vertrauenswürdige Skripte wäre unsicher und ermöglicht Angriffe auf die Umgebung. Trotzdem gibt es legitime Gründe, gefährliche Funktionen im Kontext des sicheren Interpreters zu nutzen — z. B. muss manchmal ein Programm per source in den Interpreter geladen werden. Bei Tk sind Fenster an die Fensterhierarchie eines Interpreters gebunden; potenziell gefährliche Funktionen wie Fensterverwaltung müssen im Interpreter-Kontext auf diesen Fenstern ausgeführt werden.

Der Befehl **interp** löst das mit *verborgenen Befehlen*. Statt gefährliche Befehle vollständig zu entfernen, werden sie verborgen und sind Tcl-Skripten im Interpreter nicht verfügbar.

Vertrauenswürdige Vorfahren des sicheren Interpreters können sie jedoch im Kontext des sicheren Interpreters mit **interp invoke** aufrufen. Verborgene und sichtbare Befehle liegen in getrennten Namensräumen; im selben Interpreter können verborgener und sichtbarer Befehl denselben Namen tragen.

Verborgene Befehle in einem Kind können im Rumpf von Prozeduren im Eltern während Alias-Aufrufen ausgeführt werden. Beispiel: Alias für source im Kind — beim Aufruf prüft eine Prozedur im Eltern, ob die Operation erlaubt ist (z. B. ob die Datei für das Kind zugänglich ist), und ruft dann den verborgenen source-Befehl im Kind auf, um den Inhalt zu laden. Im Kind existieren zwei Befehle source: der Alias und der verborgene Befehl.

Da der Eltern-Interpreter verborgene Befehle beim Alias-Handling aufrufen kann, dürfen über den Alias übergebene Argumente nicht ausgewertet werden — sonst könnten bösartige Kinder vertrauenswürdige Eltern zu gefährlichen Befehlen verleiten. Siehe Alias-Aufruf. Zur Vermeidung werden auf Argumente von **interp invokehidden** keine Substitutionen oder Auswertungen angewendet.

Sichere Interpreter dürfen verborgene Befehle weder in sich selbst noch in Nachkommen aufrufen — kein Zugriff auf verborgene Funktionalität in sich oder Nachfahren.

Die Menge verborgener Befehle kann von einem vertrauenswürdigen Interpreter mit **interp expose** und **interp hide** geändert werden. **interp expose** verschiebt einen verborgenen Befehl in die sichtbaren Befehle des durch *path* identifizierten Interpreters, ggf. mit Umbenennung; existiert bereits ein sichtbarer Befehl mit dem Zielnamen, schlägt die Operation fehl. **interp hide** verschiebt einen sichtbaren Befehl in die verborgenen. Sichere Interpreter dürfen Befehle weder bei sich noch bei Nachkommen zwischen verborgen und sichtbar verschieben.

Derzeit dürfen Namen verborgener Befehle keine Namespace-Qualifizierer enthalten; ein Befehl in einem Namespace muss zuerst in den globalen Namespace umbenannt werden, bevor er verborgen werden kann. Von **interp hide** gesuchte Befehle werden im globalen Namespace aufgelöst, auch wenn der aktuelle Namespace nicht global ist — damit Kinder den Eltern nicht zum Verbergen des falschen Befehls verleiten.

## Ressourcenlimits

Jeder Interpreter hat zwei Arten von Ressourcenlimits, die ein Eltern-Interpreter seinen Kindern auferlegen kann. Befehlslimits (Typ **command**) begrenzen die Gesamtzahl ausführbarer Tcl-Befehle (prüfbar mit info cmdcount); Zeitlimits (Typ **time**) legen fest, bis wann die Ausführung im Interpreter abgeschlossen sein muss. Zeitlimits sind *absolute* Zeiten (wie clock seconds), nicht relative (wie after), da sie nach Erstellung geändert werden können.

Wird ein Limit überschritten, werden zuerst Handler-Callbacks der Eltern-Interpreter aufgerufen. Erhöhen oder entfernen diese das Limit, läuft die Ausführung im (zuvor) limitierten Interpreter weiter. Bleibt das Limit aktiv, wird ein Fehler erzeugt und die normale Fehlerbehandlung im Interpreter (durch catch) deaktiviert; der Fehler breitet sich nach außen aus (mit Stack-Trace) bis zum Punkt, an dem der limitierte Interpreter aufgerufen wurde (z. B. **interp eval**), wo der aufrufende Code fangen und behandeln muss.

### Limit-Optionen

Jedes Limit hat Optionen — einige für alle Limittypen, andere typspezifisch.

**-command**

Diese Option (für alle Limittypen) legt (falls nicht leer) ein Tcl-Skript fest, das im globalen Namespace des Interpreters ausgeführt wird, der die Option liest/schreibt, wenn das jeweilige Limit im limitierten Interpreter überschritten wird. Der Callback kann das Limit ändern, damit der limitierte Interpreter weiterläuft. Erzeugt der Callback eine Ausnahme, wird sie über den Hintergrund-Ausnahme-Mechanismus gemeldet (siehe Hintergrund-Ausnahmebehandlung).

Callbacks eines Interpreters sind von denen eines anderen isoliert; die Aufrufreihenfolge ist undefiniert.

**-granularity**

Diese Option (für alle Limittypen) legt fest, wie häufig (an Punkten konsistenten Interpreter-Zustands, an denen Limits geprüft werden können) das Limit tatsächlich geprüft wird. Damit lässt sich die Prüfhäufigkeit und der Prüf-Overhead (bei Zeitlimits ggf. erheblich) abstimmen.

**-milliseconds**

Anzahl Millisekunden nach dem in **-seconds** definierten Zeitpunkt, zu dem das Zeitlimit auslöst. Nur zusammen mit **-seconds** anzugeben (ob zuvor gesetzt oder in diesem Aufruf).

**-seconds**

Sekunden seit der Epoche (siehe clock seconds), zu deren Beginn das Zeitlimit auslöst — am Sekundenanfang, sofern nicht mit **-milliseconds** sub-sekundengenau gesetzt. Kann die leere Zeichenkette sein (kein Zeitlimit).

**-value**

Anzahl Befehle, die der Interpreter ausführen darf, bevor das Befehlslimit auslöst. Kann die leere Zeichenkette sein (kein Befehlslimit).

Erzeugt ein Interpreter mit gesetztem Ressourcenlimit ein Kind, erhält dieses mindestens so restriktive Limits wie der erzeugende Eltern-Interpreter. Will der Eltern-Interpreter des limitierten Eltern die Bedingungen lockern, soll er **interp** im Kind verbergen und über Aliase sowie **interp invokehidden** den limitierten Eltern den gewünschten **interp**-Zugriff geben.

## Hintergrund-Ausnahmebehandlung

Tritt eine Ausnahme auf, die nicht direkt den Stack hinauf gemeldet werden kann (z. B. bei Event-Verarbeitung in update oder vwait), erfolgt die Meldung über den Mechanismus für Hintergrund-Ausnahmen. Jeder Interpreter hat einen registrierten Handler; der Standard ruft bgerror im globalen Namespace auf, andere Handler können Hintergrund-Ausnahmen anders verarbeiten.

Ein Hintergrund-Ausnahme-Handler ist eine nicht leere Wortliste, der beim Aufruf zwei weitere Wörter angehängt werden: das Interpreter-Ergebnis zum Zeitpunkt der Ausnahme (typischerweise eine Fehlermeldung) und das Dictionary der Return-Optionen — dieselben Werte, die catch in Nicht-Hintergrund-Situationen erfassen kann. Die resultierende Liste wird im globalen Namespace ohne weitere Substitutionen ausgeführt.

## Danksagungen

Der Mechanismus sicherer Interpreter basiert auf dem Safe-Tcl-Prototyp von Nathaniel Borenstein und Marshall Rose.

## Beispiele

Alias für einen Befehl im aktuellen Interpreter erstellen und verwenden:

```
interp alias {} getIndex {} lsearch {alpha beta gamma delta}
set idx [getIndex delta]
```

Beliebiges Skript in einem sicheren Interpreter ausführen, wobei jeder Aufruf von lappend protokolliert wird:

```
set i [interp create -safe]
interp hide $i lappend
interp alias $i lappend {} loggedLappend $i
proc loggedLappend {i args} {
    puts "logged invocation of lappend $args"
    interp invokehidden $i lappend {*}$args
}
interp eval $i $someUntrustedScript
```

Ressourcenlimit auf einem Interpreter setzen, damit eine Endlosschleife abbricht:

```
set i [interp create]
interp limit $i command -value 1000
interp eval $i {
    set x 0
    while {1} {
        puts "Counting up... [incr x]"
    }
}
```

*Copyright (c) 1995-1996 Sun Microsystems, Inc.*
*Copyright (c) 2004 Donal K. Fellows*
*Copyright (c) 2006-2008 Joe Mistachkin.*

# join

*Stichwörter:* join, element, list, separator

## Name

join - Erzeugt eine Zeichenkette durch Verbinden von Listenelementen

## Übersicht

**join** *list* ?*joinString*?

## Beschreibung

Das Argument *list* muss eine gültige Tcl-Liste sein. Der Befehl gibt die Zeichenkette zurück, die entsteht, wenn alle Elemente von *list* mit *joinString* zwischen jeweils benachbarten Elementen verbunden werden. *joinString* ist standardmäßig ein Leerzeichen.

## Beispiele

Erzeugt eine kommagetrennte Liste:

```
set data {1 2 3 4 5}
join $data ", "
     → 1, 2, 3, 4, 5
```

Verwendet **join**, um eine Liste um eine Ebene zu flachen:

```
set data {1 {2 3} 4 {5 {6 7} 8}}
join $data
     → 1 2 3 4 5 {6 7} 8
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# lappend

*Stichwörter:* lappend, append, element, list, variable

## Name

lappend - Hängt Listenelemente an eine Variable an

## Übersicht

**lappend** *varName* ?*value value value* ...?

## Beschreibung

Der Befehl behandelt die Variable *varName* als Liste und hängt jedes *value*-Argument als separates Element mit Leerzeichen dazwischen an. Existiert *varName* nicht, wird sie als Liste mit den *value*-Argumenten als Elementen angelegt. Verweist varName auf ein nicht existierendes Element eines Arrays mit gesetztem Standardwert, wird eine Liste aus dem Standardwert und allen angehängten value-Elementen im Array-Element gespeichert. **lappend** ähnelt append, hängt die *value*s aber als Listenelemente an, nicht als Rohtext. Der Befehl eignet sich relativ effizient zum Aufbau großer Listen. Beispielsweise ist "**lappend a $b**" deutlich effizienter als " **set a [concat $a [list $b]]**", wenn **$a** lang ist.

## Beispiele

**lappend** zum Aufbau einer Zahlenliste:

```
% set var 1
1
% lappend var 2
1 2
% lappend var 3 4 5
1 2 3 4 5
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2001 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*

# lassign

*Stichwörter:* lassign, assign, element, list, multiple, set, variable

## Name

lassign - Weist Listenelemente Variablen zu

## Übersicht

**lassign** *list* ?*varName* ...?

## Beschreibung

Der Befehl behandelt *list* als Liste und weist aufeinanderfolgende Elemente der *varName* -Argumente in Reihenfolge zu. Gibt es mehr Variablen als Elemente, erhalten die übrigen Variablen eine leere Zeichenkette. Gibt es mehr Elemente als Variablen, wird eine Liste der nicht zugewiesenen Elemente zurückgegeben.

## Beispiele

Mehrfachzuweisung; zu wenige oder zu viele Elemente:

```
lassign {a b c} x y z       ;# Leerer Rückgabewert
puts $x                     ;# Gibt a aus
puts $y                     ;# Gibt b aus
puts $z                     ;# Gibt c aus

lassign {d e} x y z         ;# Leerer Rückgabewert
puts $x                     ;# Gibt d aus
puts $y                     ;# Gibt e aus
puts $z                     ;# Gibt "" aus

lassign {f g h i} x y       ;# Gibt "h i" zurück
puts $x                     ;# Gibt f aus
puts $y                     ;# Gibt g aus
```

Analog zu "shift" in Shell-Sprachen:

```
set ::argv [lassign $::argv argumentToReadOff]
```

*Copyright (c) 1992-1999 Karl Lehenbauer & Mark Diekhans*
*Copyright (c) 2004 Donal K. Fellows*

# ledit

*Stichwörter:* ledit, element, list, replace

## Name

ledit - Ersetzt Elemente einer in einer Variable gespeicherten Liste

## Übersicht

**ledit** *listVar first last* ?*value value* ...?

## Beschreibung

Der Befehl liest den Listenwert in der Variable *listVar* und ersetzt die Elemente im durch die Indizes *first* bis *last* (einschließlich) angegebenen Bereich durch die Argumente *value*. Die resultierende Liste wird wieder in *listVar* gespeichert und als Ergebnis des Befehls zurückgegeben.

Die Argumente *first* und *last* sind Indexwerte für das erste und letzte Element des zu ersetzenden Bereichs. Sie werden wie Indexwerte für string index interpretiert, einschließlich einfacher Index-Arithmetik und Indizes relativ zum Listenende. Der Index **0** bezeichnet das erste Element der Liste, **end** das letzte. (Anders als bei lpop, lset und lindex werden Indizes in Unterlisten nicht unterstützt.)

Ist *first* oder *last* kleiner als null, bezieht es sich auf die Position vor dem ersten Element der Liste. So können Elemente vorangestellt werden.

Steht *first* oder *last* für eine Position größer als der Index des letzten Elements, wird es behandelt als Index um eins hinter dem letzten Element. So können Elemente angehängt werden.

Ist *last* kleiner als *first*, werden die *element*s vor dem von *first* bezeichneten Element eingefügt, ohne dass etwas gelöscht wird.

Die Argumente *value* legen null oder mehr neue Elemente fest, die anstelle der gelöschten in die Liste eingefügt werden. Jedes *value* wird ein separates Listenelement. Sind keine *value* -Argumente angegeben, werden die Elemente zwischen *first* und *last* nur gelöscht.

## Beispiele

Am Listenanfang einfügen:

```
set lst {c d e f g}
      → c d e f g
ledit lst -1 -1 a b
      → a b c d e f g
```

An die Liste anhängen:

```
ledit lst end+1 end+1 h i
      → a b c d e f g h i
```

Drittes und viertes Element löschen:

```
ledit lst 2 3
      → a b e f g h i
```

Zwei Elemente durch drei ersetzen:

```
ledit lst 2 3 x y z
      → a b x y z g h i
set lst
      → a b x y z g h i
```

*Copyright (c) 2022 Ashok P. Nadkarni* apnmbx-public@yahoo.com. *All rights reserved.*

# library

*Stichwörter:* library, auto-exec, auto-load, unknown, word, whitespace

## Name

auto_execok, auto_import, auto_load, auto_mkindex, auto_qualify, auto_reset, foreachLine, parray, readFile, tcl_findLibrary, tcl_endOfWord, tcl_nonwordchars, tcl_startOfNextWord, tcl_startOfPreviousWord, tcl_wordBreakAfter, tcl_wordBreakBefore, tcl_wordchars, writeFile - Standardbibliothek von Tcl-Prozeduren

## Übersicht

**auto_execok** *cmd*
**auto_import** *pattern*
**auto_load** *cmd*
**auto_mkindex** *dir pattern pattern ...*
**auto_qualify** *command namespace*
**auto_reset**
**tcl_findLibrary** *basename version patch initScript enVarName varName*
**parray** *arrayName* ?*pattern*?
**tcl_endOfWord** *str start*
**tcl_startOfNextWord** *str start*
**tcl_startOfPreviousWord** *str start*
**tcl_wordBreakAfter** *str start*
**tcl_wordBreakBefore** *str start*
**foreachLine** *filename varName body*
**readFile** *filename* ?**text|binary**?
**writeFile** *filename* ?**text|binary**? *contents*

## Einleitung

Tcl enthält eine Bibliothek von Tcl-Prozeduren für häufig benötigte Funktionen. Die in der Tcl-Bibliothek definierten Prozeduren sind allgemeine Prozeduren, die von vielen verschiedenen Anwendungen genutzt werden können. Der Speicherort der Tcl-Bibliothek wird vom Befehl info library zurückgegeben. Zusätzlich zur Tcl-Bibliothek verfügt jede Anwendung in der Regel über eine eigene Bibliothek unterstützender Prozeduren; der Speicherort dieser Bibliothek wird normalerweise durch den Wert der globalen Variable **$***app***_library** angegeben, wobei *app* der Name der Anwendung ist. Beispielsweise wird der Speicherort der Tk-Bibliothek in der Variable tk_library gehalten.

Um auf die Prozeduren in der Tcl-Bibliothek zuzugreifen, sollte eine Anwendung die Datei **init.tcl** in der Bibliothek sourcen, beispielsweise mit dem Tcl-Befehl

```
source [file join [info library] init.tcl]
```

Wird die Bibliotheksprozedur Tcl_Init aus der Tcl_AppInit-Prozedur einer Anwendung aufgerufen, geschieht dies automatisch. Der Code in **init.tcl** definiert die Prozedur unknown und sorgt dafür, dass die übrigen Prozeduren bei Bedarf über den unten beschriebenen Auto-Load-Mechanismus geladen werden.

## Befehlsprozeduren

Die folgenden Prozeduren werden in der Tcl-Bibliothek bereitgestellt:

**auto_execok** ***cmd***

Ermittelt, ob es eine ausführbare Datei oder ein Shell-Builtin mit dem Namen *cmd* gibt. Falls ja, gibt sie eine Liste von Argumenten zurück, die an exec übergeben werden, um die ausführbare Datei oder das Shell-Builtin namens *cmd* auszuführen. Andernfalls gibt sie eine leere Zeichenkette zurück. Dieser Befehl durchsucht bei der Suche nach einer ausführbaren Datei namens *cmd* die Verzeichnisse im aktuellen Suchpfad (angegeben durch die Umgebungsvariable PATH). Auf Windows-Plattformen wird die Suche mit denselben Verzeichnissen und Dateierweiterungen erweitert, die auch exec verwendet. **Auto_execok** merkt sich Informationen über frühere Suchen in einem Array namens **auto_execs**; dies vermeidet die Pfadsuche bei künftigen Aufrufen für dasselbe *cmd*. Der Befehl **auto_reset** kann verwendet werden, um **auto_execok** zum Vergessen der zwischengespeicherten Informationen zu zwingen.

Beispielsweise würde man auf Linux das Shell-Builtin *umask* so ausführen:

```
exec {*}[auto_execok umask]
```

Um das Shell-Builtin *DIR* unter Windows auszuführen, würde man Folgendes tun:

```
exec {*}[auto_execok dir]
```

Um festzustellen, ob sich ein *frobnicate*-Binary im PATH des Benutzers befindet, würde man Folgendes tun:

```
set mayFrob [expr {[llength [auto_execok frobnicate]] > 0}]
```

**auto_import *pattern***

**Auto_import** wird während namespace import aufgerufen, um zu prüfen, ob die durch *pattern* angegebenen importierten Befehle in einer per Auto-Load geladenen Bibliothek liegen. Falls ja, werden die Befehle geladen, damit sie dem Interpreter zum Erstellen der Import-Links zur Verfügung stehen. Liegen die Befehle nicht in einer per Auto-Load geladenen Bibliothek, tut **auto_import** nichts. Der Mustervergleich erfolgt gemäß den Abgleichregeln von namespace import.

Es ist normalerweise nicht erforderlich, diesen Befehl direkt aufzurufen.

**auto_load *cmd***

Dieser Befehl versucht, die Definition für einen Tcl-Befehl namens *cmd* zu laden. Dazu durchsucht er einen *Auto-Load-Pfad*, eine Liste aus einem oder mehreren Verzeichnissen. Der Auto-Load-Pfad wird durch die globale Variable auto_path angegeben, falls diese existiert. Gibt es keine Variable auto_path, wird die Umgebungsvariable **TCLLIBPATH** verwendet, falls sie existiert. Andernfalls besteht der Auto-Load-Pfad nur aus dem Tcl-Bibliotheksverzeichnis. In jedem Verzeichnis im Auto-Load-Pfad muss eine Datei **tclIndex** vorhanden sein, die einen oder mehrere in diesem Verzeichnis definierte Befehle beschreibt sowie ein Skript zum Auswerten, um jeden der Befehle zu laden. Die Datei **tclIndex** sollte mit dem Befehl **auto_mkindex** erzeugt werden. Wird *cmd* in einer Indexdatei gefunden, wird das entsprechende Skript ausgewertet, um den Befehl zu erstellen. Der Befehl **auto_load** gibt 1 zurück, wenn *cmd* erfolgreich erstellt wurde. Der Befehl gibt 0 zurück, wenn es keinen Indexeintrag für *cmd* gab oder wenn das Skript *cmd* nicht tatsächlich definiert hat (z. B. weil die Indexinformationen veraltet sind). Tritt beim Verarbeiten des Skripts ein Fehler auf, wird dieser Fehler zurückgegeben. **Auto_load** liest die Indexinformationen nur einmal und speichert sie im Array **auto_index**; künftige Aufrufe von **auto_load** prüfen *cmd* im Array, anstatt die Indexdateien erneut zu lesen. Die zwischengespeicherten Indexinformationen können mit dem Befehl **auto_reset** gelöscht werden. Dies zwingt den nächsten **auto_load** -Befehl, die Indexdatenbank von der Festplatte neu zu laden.

Es ist normalerweise nicht erforderlich, diesen Befehl direkt aufzurufen; der Standard-unknown-Handler erledigt dies.

**auto_mkindex *dir pattern pattern ...***

Erzeugt einen Index, der für **auto_load** geeignet ist. Der Befehl durchsucht *dir* nach allen Dateien, deren Namen einem der *pattern*-Argumente entsprechen (der Abgleich erfolgt mit dem Befehl glob), erzeugt einen Index aller in allen passenden Dateien definierten Tcl-Befehlsprozeduren und speichert die Indexinformationen in einer Datei namens **tclIndex** in *dir*. Wird kein Muster angegeben, wird ein Muster von ***.tcl** angenommen. Beispielsweise liest der Befehl

```
auto_mkindex foo *.tcl
```

alle **.tcl**-Dateien im Unterverzeichnis **foo** und erzeugt eine neue Indexdatei **foo/tclIndex**.

**Auto_mkindex** parst die Tcl-Skripte, indem es sie in einen Kind-Interpreter sourcet und die ausgeführten proc- und namespace-Befehle überwacht. Erweiterungen können das (undokumentierte) Paket auto_mkindex_parser verwenden, um andere Befehle zu registrieren, die zum Auto-Load-Index beitragen können. Wie dies funktioniert, muss man in auto.tcl nachlesen.

**Auto_mkindex_old** (mit derselben Syntax wie **auto_mkindex**) parst die Tcl-Skripte auf relativ einfache Weise: Enthält eine Zeile das Wort "proc" als erste Zeichen, wird angenommen, dass es sich um eine Prozedurdefinition handelt, und das nächste Wort der Zeile wird als Name der Prozedur genommen. Prozedurdefinitionen, die nicht in dieser Form erscheinen (z. B. mit Leerzeichen vor proc), werden nicht indiziert. Enthält Ihr Skript "gefährlichen" Code, etwa globalen Initialisierungscode oder Prozedurnamen mit Sonderzeichen wie **$**, *****, **[** oder **]**, ist die Verwendung von **auto_mkindex_old** sicherer.

**auto_reset**

Löscht alle von **auto_execok** und **auto_load** zwischengespeicherten Informationen. Diese Informationen werden beim nächsten Bedarf von der Festplatte neu gelesen. **Auto_reset** löscht außerdem alle im Auto-Load-Index aufgeführten Prozeduren, sodass beim nächsten Gebrauch frische Kopien geladen werden.

**auto_qualify *command namespace***

Berechnet eine Liste vollständig qualifizierter Namen für *command*. Diese Liste spiegelt den Pfad wider, den ein Standard-Tcl-Interpreter bei der Befehlssuche verfolgt: Zuerst sucht er den Befehl im aktuellen Namespace, dann im globalen Namespace. Entsprechend hat die zurückgegebene Liste zwei Elemente, wenn *command* relativ ist und *namespace* nicht **::** ist: *command* im Gültigkeitsbereich von *namespace*, als wäre es ein Befehl im Namespace *namespace*; und *command*, als wäre es ein Befehl im globalen Namespace. Andernfalls, wenn entweder *command* absolut ist (mit **::** beginnt) oder *namespace* **::** ist, enthält die Liste nur *command*, als wäre es ein Befehl im globalen Namespace.

**Auto_qualify** wird von den Auto-Load-Einrichtungen in Tcl verwendet, sowohl zum Erzeugen von Auto-Load-Indizes wie *pkgIndex.tcl* als auch zum tatsächlichen Auto-Load von Funktionen zur Laufzeit.

**tcl_findLibrary *basename version patch initScript enVarName varName***

Dies ist eine Standard-Suchprozedur für Erweiterungen während ihrer Initialisierung. Sie rufen diese Prozedur auf, um ihre Skriptbibliothek in mehreren Standardverzeichnissen zu suchen. Die letzte Komponente des Namens des Bibliotheksverzeichnisses ist normalerweise *basenameversion* (z. B. tk8.0), kann aber in Build-Hierarchien "library" sein. Die Datei *initScript* wird in den Interpreter gesourcet, sobald sie gefunden wurde. Das Verzeichnis, in dem diese Datei gefunden wurde, wird in der globalen Variable *varName* gespeichert. Ist diese Variable bereits definiert (z. B. durch C-Code während der Anwendungsinitialisierung), wird nicht gesucht. Andernfalls erfolgt die Suche in diesen Verzeichnissen: dem durch die Umgebungsvariable *enVarName* benannten Verzeichnis; relativ zum Tcl-Bibliotheksverzeichnis; relativ zur ausführbaren Datei im Standard-Installationsverzeichnis bin oder bin/*arch*; relativ zur ausführbaren Datei im aktuellen Build-Baum; relativ zur ausführbaren Datei in einem parallelen Build-Baum.

**parray *arrayName* ?*pattern*?**

Gibt auf der Standardausgabe die Namen und Werte aller Elemente im Array *arrayName* aus, oder nur die Namen, die auf *pattern* passen (gemäß den Abgleichregeln von string match), und deren Werte, falls *pattern* angegeben ist. *ArrayName* muss ein Array sein, das dem Aufrufer von **parray** zugänglich ist. Es kann lokal oder global sein. Das Ergebnis dieses Befehls ist die leere Zeichenkette.

Beispielsweise gibt man zum Ausgeben des Inhalts des Arrays tcl_platform Folgendes ein:

```
parray tcl_platform
```

## Hilfsprozeduren für Wortgrenzen

Diese Prozeduren werden hauptsächlich intern von Tk verwendet.

**tcl_endOfWord *str start***

Gibt den Index der ersten Wortende-Position zurück, die nach einem Startindex *start* in der Zeichenkette *str* auftritt. Eine Wortende-Position ist definiert als das erste Nicht-Wort-Zeichen, das auf das erste Wort-Zeichen nach dem Startpunkt folgt. Gibt -1 zurück, wenn nach dem Startpunkt keine weiteren Wortende-Positionen vorhanden sind. Siehe die Beschreibung von **tcl_wordchars** und **tcl_nonwordchars** unten für weitere Details, wie Tcl bestimmt, welche Zeichen Wort-Zeichen sind.

**tcl_startOfNextWord *str start***

Gibt den Index der ersten Wortanfang-Position zurück, die nach einem Startindex *start* in der Zeichenkette *str* auftritt. Eine Wortanfang-Position ist definiert als das erste Wort-Zeichen, das auf ein Nicht-Wort-Zeichen folgt. Gibt -1 zurück, wenn nach dem Startpunkt keine weiteren Wortanfang-Positionen vorhanden sind.

Beispielsweise gibt man zum Ausgeben der Indizes der Wortanfänge in einer Zeichenkette gemäß Plattformregeln Folgendes aus:

```
set theString "The quick brown fox"
for {set idx 0} {$idx >= 0} {
        set idx [tcl_startOfNextWord $theString $idx]} {
    puts "Word start index: $idx"
}
```

**tcl_startOfPreviousWord *str start***

Gibt den Index der ersten Wortanfang-Position zurück, die vor einem Startindex *start* in der Zeichenkette *str* auftritt. Gibt -1 zurück, wenn vor dem Startpunkt keine weiteren Wortanfang-Positionen vorhanden sind.

**tcl_wordBreakAfter *str start***

Gibt den Index der ersten Wortgrenze nach dem Startindex *start* in der Zeichenkette *str* zurück. Gibt -1 zurück, wenn in der angegebenen Zeichenkette nach dem Startpunkt keine weiteren Grenzen vorhanden sind. Der zurückgegebene Index bezieht sich auf das zweite Zeichen des Paares, das eine Grenze bildet.

**tcl_wordBreakBefore *str start***

Gibt den Index der ersten Wortgrenze vor dem Startindex *start* in der Zeichenkette *str* zurück. Gibt -1 zurück, wenn in der angegebenen Zeichenkette vor dem Startpunkt keine weiteren Grenzen vorhanden sind. Der zurückgegebene Index bezieht sich auf das zweite Zeichen des Paares, das eine Grenze bildet.

### Hilfsprozeduren für Dateizugriff

**foreachLine *varName filename body***

Liest die Textdatei namens filename zeilenweise ein (unter Verwendung der Systemvorgaben zum Lesen von Textdateien). Sie schreibt diese Zeile in die Variable namens varName und führt anschließend body für diese Zeile aus. Der Ergebniswert von body wird ignoriert, aber error, return, break und continue können darin verwendet werden, um einen Fehler zu erzeugen, aus dem aufrufenden Kontext zurückzukehren, die Schleife zu beenden bzw. zur nächsten Zeile zu gehen. Das Gesamtergebnis von foreachLine ist die leere Zeichenkette (vorausgesetzt, keine Fehler durch E/A oder durch Auswerten des Schleifenkörpers); die Datei wird vor dem Zurückkehren der Prozedur geschlossen.

**readFile *filename* ?text|binary?**

Liest die in filename benannte Datei ein und gibt deren Inhalt zurück. Das zweite Argument legt fest, wie die Datei eingelesen wird, entweder als text (unter Verwendung der Systemvorgaben zum Lesen von Textdateien) oder als binary (als uninterpretierte Bytes). Standard ist text. Beim Lesen als Text ist eine abschließende Newline enthalten. Die Datei wird vor dem Zurückkehren der Prozedur geschlossen.

**writeFile *filename* ?text|binary? *contents***

Schreibt contents in die in filename benannte Datei. Das optionale zweite Argument legt fest, wie in die Datei geschrieben wird, entweder als text (unter Verwendung der Systemvorgaben zum Schreiben von Textdateien) oder als binary (als uninterpretierte Bytes). Standard ist text. Ist eine abschließende Newline erforderlich, muss sie in contents bereitgestellt werden. Das Ergebnis dieses Befehls ist die leere Zeichenkette; die Datei wird vor dem Zurückkehren der Prozedur geschlossen.

## Variablen

Die folgenden globalen Variablen werden von den Prozeduren in der Tcl-Bibliothek definiert oder verwendet. Sie fallen in zwei große Kategorien: Umgang mit unbekannten Befehlen und Paketen sowie Bestimmung dessen, was Wörter sind.

## Variablen für Auto-Load und Paketverwaltung

**auto_execs**

Wird von **auto_execok** verwendet, um Informationen darüber zu speichern, ob bestimmte Befehle als ausführbare Dateien existieren.

Wird normalerweise nicht sinnvoll direkt durch Benutzercode angesprochen.

**auto_index**

Wird von **auto_load** verwendet, um die von der Festplatte gelesenen Indexinformationen zu speichern.

Wird normalerweise nicht sinnvoll direkt durch Benutzercode angesprochen.

**auto_noexec**

Ist sie auf einen beliebigen Wert gesetzt, versucht unknown nicht, Befehle per Auto-Exec auszuführen.

**auto_noload**

Ist sie auf einen beliebigen Wert gesetzt, versucht unknown nicht, Befehle per Auto-Load zu laden.

auto_path

Ist sie gesetzt, muss sie eine gültige Tcl-Liste mit Verzeichnissen enthalten, die während Auto-Load-Operationen durchsucht werden (einschließlich Paketindexdateien bei Verwendung des Standard-package unknown-Handlers). Diese Variable wird beim Start initialisiert und enthält der Reihe nach: die in der Umgebungsvariable **TCLLIBPATH** aufgeführten Verzeichnisse, das durch die globale Variable tcl_library benannte Verzeichnis, das übergeordnete Verzeichnis von tcl_library, die in der Variable tcl_pkgPath aufgeführten Verzeichnisse. Zusätzliche Speicherorte für Dateien und Paketindizes sollten normalerweise mit lappend zu dieser Variable hinzugefügt werden.

Beispielsweise würde man zum Hinzufügen des Verzeichnisses *lib* neben dem laufenden Skript Folgendes tun:

```
lappend auto_path [file dirname [info script]]/lib
```

Nutzt das Skript cd, sollten die Einträge auf auto_path mit file normalize normalisiert werden.

**env(TCL_LIBRARY)**

Ist sie gesetzt, legt sie den Ort des Verzeichnisses mit Bibliotheksskripten fest (der Wert dieser Variable wird der Variable tcl_library zugewiesen und daher vom Befehl info library zurückgegeben). Ist diese Variable nicht gesetzt, wird ein Standardwert verwendet.

Die Verwendung dieser Umgebungsvariable wird außerhalb von Tests nicht empfohlen. Tcl-Installationen sollten bereits wissen, wo ihre eigenen Skriptdateien zu finden sind, da der Wert beim Build oder bei der Installation fest eingebaut wird.

**env(TCLLIBPATH)**

Ist sie gesetzt, muss sie eine gültige Tcl-Liste mit Verzeichnissen enthalten, die bei

Auto-Load-Vorgängen durchsucht werden. Verzeichnisse müssen im Tcl-Format angegeben werden, mit "/" als Pfadtrenner, unabhängig von der Plattform. Diese Variable wird nur bei der Initialisierung der Variable auto_path verwendet.

Eine wichtige Folge dieser Variable ist, dass sie dem Benutzer eines Skripts eine Möglichkeit gibt, die Liste der Orte anzugeben, von denen aus dieses Skript mit package require Pakete laden darf. Sie ist normalerweise nicht sinnvoll innerhalb eines Tcl-Skripts selbst setzbar, *außer* um zu beeinflussen, von wo andere Interpreter laden (ob mit interp create erstellt oder als eigene Threads oder Subprozesse gestartet).

### Variablen zur Bestimmung von Wortgrenzen

Diese Variablen werden nur in den Befehlen **tcl_endOfWord**, **tcl_startOfNextWord**, **tcl_startOfPreviousWord**, **tcl_wordBreakAfter** und **tcl_wordBreakBefore** verwendet.

**tcl_nonwordchars**

Diese Variable enthält einen regulären Ausdruck, der von Routinen wie **tcl_endOfWord** verwendet wird, um zu ermitteln, ob ein Zeichen Teil eines Wortes ist oder nicht. Passt das Muster auf ein Zeichen, gilt das Zeichen als Nicht-Wort-Zeichen. Der Standardwert ist **\W**.

**tcl_wordchars**

Diese Variable enthält einen regulären Ausdruck, der von Routinen wie **tcl_endOfWord** verwendet wird, um zu ermitteln, ob ein Zeichen Teil eines Wortes ist oder nicht. Passt das Muster auf ein Zeichen, gilt das Zeichen als Wort-Zeichen. Der Standardwert ist **\w**.

*Copyright (c) 1991-1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# lindex

*Stichwörter:* lindex, element, index, list

## Name

lindex - Liest ein Element aus einer Liste

## Übersicht

**lindex** *list* ?*index* ...?

## Beschreibung

Der Befehl **lindex** nimmt den Parameter *list*, den er als Tcl-Liste behandelt, und null oder mehr *index*-Angaben in die Liste. Die Indizes können nacheinander auf der Befehlszeile stehen oder in einer Tcl-Liste gruppiert als ein einziges Argument übergeben werden.

Ohne Indizes hat der Befehl die Form:

```
lindex list
```

oder

```
lindex list {}
```

In diesem Fall ist der Rückgabewert von **lindex** einfach der Wert des Parameters *list*.

Bei einem einzelnen Index behandelt **lindex** *list* als Tcl-Liste und gibt das *index*-te Element zurück (0 bezeichnet das erste Element). Beim Extrahieren gelten dieselben Regeln für geschweifte Klammern, Anführungszeichen und Backslashes wie beim Tcl-Befehlsinterpreter; Variablen- und Befehlssubstitution finden jedoch nicht statt. Ist *index* negativ oder größer oder gleich der Elementanzahl in *list*, wird eine leere Zeichenkette zurückgegeben. Die Auslegung jedes einfachen *index*-Werts entspricht string index, einschließlich einfacher Indexarithmetik und Indizes relativ zum Listenende.

Weitere *index*-Argumente wählen nacheinander aus dem Ergebnis der vorherigen Indexoperation ein Element — damit lassen sich Elemente aus Unterlisten wählen. Der Befehl

```
lindex $a 1 2 3
```

oder

```
lindex $a {1 2 3}
```

ist gleichbedeutend mit

```
lindex [lindex [lindex $a 1] 2] 3
```

## Beispiele

Listen können von beiden Enden indiziert werden:

```
lindex {a b c} 0
      → a
lindex {a b c} 2
      → c
lindex {a b c} end
      → c
lindex {a b c} end-1
      → b
```

Listen oder Indexfolgen ermöglichen Zugriff in Listen von Listen:

```
lindex {a b c}
      → a b c
lindex {a b c} {}
      → a b c
lindex {{a b c} {d e f} {g h i}} 2 1
      → h
lindex {{a b c} {d e f} {g h i}} {2 1}
      → h
lindex {{{a b} {c d}} {{e f} {g h}}} 1 1 0
      → g
lindex {{{a b} {c d}} {{e f} {g h}}} {1 1 0}
      → g
```

Listenindizes können begrenzte Berechnungen ausführen, indem feste Beträge zu anderen Indizes addiert oder subtrahiert werden:

```
set idx 1
lindex {a b c d e f} $idx+2
      → d
set idx 3
lindex {a b c d e f} $idx+2
      → f
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2001 Kevin B. Kenny kennykb@acm.org. All rights reserved.*

# link

*Stichwörter:* link, command, method, object

## Name

link - Verknüpfung von Befehl mit Methode eines Objekts erstellen

## Übersicht

**package require tcl::oo**

**link** *methodName* ?...?
**link** *{commandName methodName}* ?...?

## Beschreibung

Der Befehl **link** steht innerhalb von Methoden zur Verfügung. Er nimmt eine Reihe von einem oder mehreren Methodennamen (*methodName ...*) und/oder Paaren aus Befehls- und Methodennamen (**{***commandName methodName***}**) entgegen und macht die genannten Methoden als Befehle verfügbar, ohne dass der Objektname oder der Befehl my ausdrücklich verwendet werden muss. Die Methode muss zum Zeitpunkt der Verknüpfung noch nicht existieren; wird der verknüpfte Befehl aufgerufen, während die Methode nicht existiert, greift das übliche **unknown**-Behandlungssystem für Methoden.

Der Befehlsname, unter dem die Methode verfügbar wird, entspricht standardmäßig dem Methodennamen, außer wenn er über ein Alias-/Methoden-Paar ausdrücklich angegeben wird. Formal muss jedes Argument eine Liste sein; hat die Liste zwei Elemente, ist das erste Element der Name des zu erzeugenden Befehls und das zweite Element der Name der Methode des aktuellen Objekts, mit der der Befehl verknüpft wird; andernfalls sind Befehlsname und Methodenname dieselbe Zeichenkette (das erste Element der Liste).

Ist der Befehlsname kein voll qualifizierter Befehlsname, wird er relativ zum aktuellen Namespace aufgelöst (d. h. dem Objekt-Namespace).

## Beispiele

Dies zeigt die Verknüpfung einer einzelnen Methode auf verschiedene Weise. Zuerst eine einfache Verknüpfung, dann eine umbenannte Verknüpfung, dann eine externe Verknüpfung. Beachten Sie, dass die Methode selbst nicht exportiert ist, aber dennoch direkt von außerhalb der Klasse aufgerufen werden kann.

```
oo::class create ABC {
    method Foo {} {
        puts "This is Foo in [self]"
    }

    constructor {} {
        link Foo
        # Die Methode Foo ist hier jetzt direkt als Foo aufrufbar
        link {bar Foo}
        # Die Methode Foo ist jetzt direkt als bar aufrufbar
        link {::ExternalCall Foo}
        # Die Methode Foo ist jetzt im globalen
        # Namespace als ExternalCall direkt aufrufbar
    }

    method grill {} {
        puts "Step 1:"
        Foo
        puts "Step 2:"
        bar
    }
}

ABC create abc
abc grill
        → Step 1:
        → This is Foo in ::abc
        → Step 2:
        → This is Foo in ::abc
# Direkter Zugriff über den verknüpften Befehl
puts "Step 3:"; ExternalCall
        → Step 3:
        → This is Foo in ::abc
```

Dieses Beispiel zeigt, dass in einem Aufruf von **link** mehrere verknüpfte Befehle erzeugt werden können und dass diese Argumente verarbeiten können.

```
oo::class create Ex {
    constructor {} {
        link a b c
        # Die Methoden a, b und c (unten definiert) sind nun
        # in Methoden direkt unter ihren eigenen Namen aufrufbar.
    }

    method a {} {
        puts "This is a"
    }
    method b {x} {
        puts "This is b($x)"
    }
    method c {y z} {
        puts "This is c($y,$z)"
    }

    method call {p q r} {
        a
        b $p
        c $q $r
    }
}

set o [Ex new]
$o call 3 5 7
        → This is a
        → This is b(3)
        → This is c(5,7)
```

*Copyright (c) 2011-2015 Andreas Kupries*
*Copyright (c) 2018 Donal K. Fellows*

# linsert

*Stichwörter:* linsert, element, insert, list

## Name

linsert - Fügt Elemente in eine Liste ein

## Übersicht

**linsert** *list index* ?*element element* ...?

## Beschreibung

Der Befehl erzeugt aus *list* eine neue Liste, indem alle *element*-Argumente unmittelbar vor dem *index*-ten Element von *list* eingefügt werden. Jedes *element* wird ein separates Element der neuen Liste. Ist *index* kleiner oder gleich null, kommen die neuen Elemente an den Anfang; ist *index* größer oder gleich der Listenlänge, gilt es wie **end**. Wie bei string index unterstützt *index* Indexarithmetik und end-relative Indizes.

Unter der Voraussetzung, dass Indizes gültige Positionen in der Liste bezeichnen und die *element*s in der angegebenen Reihenfolge eingefügt werden, gilt: bei start-relativem *index* steht das erste *element* an diesem Index in der Ergebnisliste; bei end-relativem *index* steht das letzte *element* an diesem Index.

## Beispiele

Werte in eine Liste einfügen — einmal vom Anfang, einmal vom Ende, dann verkettet:

```
set oldList {the fox jumps over the dog}
set midList [linsert $oldList 1 quick]
set newList [linsert $midList end-1 lazy]
# Die alten Listen existieren weiter...
set newerList [linsert [linsert $oldList end-1 quick] 1 lazy]
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2001 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*

# list

*Stichwörter:* list, element, quoting

## Name

list - Erstellt eine Liste

## Übersicht

**list** ?*arg arg* ...?

## Beschreibung

Dieser Befehl gibt eine Liste zurück, die aus allen *arg*s besteht, oder einen leeren String, wenn keine *arg*s angegeben sind. Geschweifte Klammern und Backslashes werden bei Bedarf hinzugefügt, sodass der Befehl lindex auf das Ergebnis angewendet werden kann, um die ursprünglichen Argumente wieder zu extrahieren, und sodass eval verwendet werden kann, um die resultierende Liste auszuführen, wobei *arg1* den Namen des Befehls und die anderen *args* dessen Argumente bilden. **list** erzeugt leicht andere Ergebnisse als concat: concat entfernt eine Ebene der Gruppierung, bevor die Liste gebildet wird, während **list** direkt mit den ursprünglichen Argumenten arbeitet.

## Beispiel

Der Befehl

```
list a b "c d e  " "  f {g h}"
```

gibt zurück

```
a b {c d e  } {  f {g h}}
```

während concat mit den gleichen Argumenten zurückgibt

```
a b c d e f {g h}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2001 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*

# llength

*Stichwörter:* llength, element, list, length

## Name

llength - Zählt die Elemente einer Liste

## Übersicht

**llength** *list*

## Beschreibung

Behandelt *list* als Liste und gibt eine Dezimalzeichenkette mit der Elementanzahl zurück.

## Beispiele

Das Ergebnis ist die Anzahl der Elemente:

```
% llength {a b c d e}
5
% llength {a b c}
3
% llength {}
0
```

Elemente sind nicht zwingend "Wörter" im Wörterbuch-Sinn, besonders bei Quoting:

```
% llength {a b {c d} e}
4
% llength {a b { } c d e}
6
```

Eine leere Liste ist nicht notwendigerweise eine leere Zeichenkette:

```
% set var { }; puts "[string length $var],[llength $var]"
1,0
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2001 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*

# lmap

*Stichwörter:* lmap, foreach, iteration, list, loop, map

## Name

lmap - Iteriert über alle Elemente einer oder mehrerer Listen und sammelt Ergebnisse

## Übersicht

**lmap** *varname list body*
**lmap** *varlist1 list1 ?varlist2 list2 ...? body*

## Beschreibung

Der Befehl **lmap** implementiert eine Schleife, in der die Schleifenvariable(n) Werte aus einer oder mehreren Listen annehmen und die Schleife eine Liste der bei jeder Iteration gesammelten Ergebnisse zurückgibt.

Im einfachsten Fall gibt es eine Schleifenvariable *varname* und eine Liste *list* mit Werten, die *varname* zugewiesen werden. Das Argument *body* ist ein Tcl-Skript. Für jedes Element von *list* (in Reihenfolge von erstem zu letztem) weist **lmap** den Inhalt des Elements *varname* zu, als ob lindex zum Extrahieren verwendet worden wäre, und ruft dann den Tcl-Interpreter auf, um *body* auszuführen. Schließt die Ausführung von *body* normal ab, wird das Ergebnis von *body* an eine Akkumulatorliste angehängt. **lmap** gibt die Akkumulatorliste zurück.

Im allgemeinen Fall kann es mehr als eine Werteliste geben (z. B. *list1* und *list2*), und jeder Werteliste kann eine Liste von Schleifenvariablen zugeordnet sein (z. B. *varlist1* und *varlist2*). In jeder Iteration werden den Variablen jeder *varlist* aufeinanderfolgende Werte aus der zugehörigen *list* zugewiesen. Werte in jeder *list* werden in Reihenfolge von erstem zu letztem verwendet, und jeder Wert genau einmal. Die Gesamtzahl der Schleifeniterationen ist groß genug, um alle Werte aus allen Wertelisten aufzubrauchen. Enthält eine Werteliste nicht genug Elemente für alle ihre Schleifenvariablen in einer Iteration, werden leere Werte für die fehlenden Elemente verwendet.

break und continue können innerhalb von *body* aufgerufen werden, mit derselben Wirkung wie bei for und foreach. In diesen Fällen schließt *body* nicht normal ab, und das Ergebnis wird nicht an die Akkumulatorliste angehängt.

### Beispiele

Listen zusammenführen:

```
set list1 {a b c d}
set list2 {1 2 3 4}
set zipped [lmap a $list1 b $list2 {list $a $b}]
# Der Wert von zipped ist "{a 1} {b 2} {c 3} {d 4}"
```

Liste filtern, um ungerade Werte zu entfernen:

```
set values {1 2 3 4 5 6 7 8}
proc isEven {n} {expr {($n % 2) == 0}}
set goodOnes [lmap x $values {expr {
    [isEven $x] ? $x : [continue]
}}]
# Der Wert von goodOnes ist "2 4 6 8"
```

Präfix einer Liste anhand ihres Inhalts nehmen:

```
set values {8 7 6 5 4 3 2 1}
proc isGood {counter} {expr {$n > 3}}
set prefix [lmap x $values {expr {
    [isGood $x] ? $x : [break]
}}]
# Der Wert von prefix ist "8 7 6 5 4"
```

*Copyright (c) 2012 Trevor Davel*

## load

*Stichwörter:* load, binary code, dynamic library, safe interpreter, shared library

## Name

load - Lädt Maschinencode und initialisiert neue Befehle

## Übersicht

**load** ?**-global**? ?**-lazy**? ?**--**? *fileName*
**load** ?**-global**? ?**-lazy**? ?**--**? *fileName prefix*
**load** ?**-global**? ?**-lazy**? ?**--**? *fileName prefix interp*

## Beschreibung

Dieser Befehl lädt Binärcode aus einer Datei in den Adressraum der Anwendung und ruft eine Initialisierungsprozedur in der Bibliothek auf, um sie in einen Interpreter einzubinden. *fileName* ist der Name der Datei mit dem Code; die genaue Form variiert von System zu System, auf den meisten Systemen handelt es sich jedoch um eine Shared Library, etwa eine **.so**-Datei unter Solaris oder eine DLL unter Windows. *prefix* wird verwendet, um den Namen einer Initialisierungsprozedur zu berechnen. *interp* ist der Pfadname des Interpreters, in den die Bibliothek geladen werden soll (Details siehe Manpage interp); fehlt *interp*, wird standardmäßig der Interpreter verwendet, in dem der Befehl **load** aufgerufen wurde.

Sobald die Datei in den Adressraum der Anwendung geladen wurde, wird im neuen Code eine von zwei Initialisierungsprozeduren aufgerufen. Typischerweise fügt die Initialisierungsprozedur neue Befehle zu einem Tcl-Interpreter hinzu. Der Name der Initialisierungsprozedur wird durch *prefix* und davon abhängig, ob der Zielinterpreter ein Safe-Interpreter ist, bestimmt. Für normale Interpreter hat die Initialisierungsprozedur die Form *prefix*_**Init**. Ist *prefix* beispielsweise **Foo**, lautet der Name der Initialisierungsprozedur **Foo_Init**.

Ist der Zielinterpreter ein Safe-Interpreter, heißt die Initialisierungsprozedur *prefix*_**SafeInit** statt *prefix*_**Init**. Die Funktion *prefix*_**SafeInit** sollte sorgfältig geschrieben werden, damit der Safe-Interpreter nur mit der Teilfunktionalität der Bibliothek initialisiert wird, die für nicht vertrauenswürdigen Code sicher ist. Weitere Informationen zu Safe-Tcl siehe die Manpage safe.

Die Initialisierungsprozedur muss folgendem Prototyp entsprechen:

```
typedef int Tcl_LibraryInitProc(
        Tcl_Interp *interp);
```

Das Argument *interp* bezeichnet den Interpreter, in den die Bibliothek geladen werden soll. Die Initialisierungsprozedur muss TCL_OK oder TCL_ERROR zurückgeben, um anzuzeigen, ob sie erfolgreich abgeschlossen wurde; bei einem Fehler soll sie das Ergebnis des Interpreters auf eine Fehlermeldung setzen. Das Ergebnis des Befehls **load** ist das von der Initialisierungsprozedur zurückgegebene Ergebnis.

Das eigentliche Laden einer Datei erfolgt pro *fileName* in einer Anwendung nur einmal. Wird ein bestimmter *fileName* in mehrere Interpreter geladen, lädt der erste **load** den Code und ruft die Initialisierungsprozedur auf; nachfolgende **load**-Aufrufe rufen die Initialisierungsprozedur auf, ohne den Code erneut zu laden. Bei Tcl-Versionen unter 8.5 ist das Entladen oder erneute Laden einer Bibliothek nicht möglich. Ab Version 8.5 erlaubt der Befehl unload das Entladen mit **load** geladener Bibliotheken, sofern diese Tcl-Entlademechanismus unterstützen.

Der Befehl **load** unterstützt auch Bibliotheken, die statisch mit der Anwendung verlinkt sind, wenn diese durch Aufruf von Tcl_StaticLibrary registriert wurden. Ist *fileName* ein leerer String, muss *prefix* angegeben werden.

Fehlt *prefix* oder ist er ein leerer String, versucht Tcl das Präfix zu erraten, indem das letzte Element von *fileName* genommen, die ersten drei Zeichen entfernt werden, falls sie **lib** lauten, anschließend die nächsten vier Zeichen, falls sie **tcl9** lauten, und alle folgenden Wortzeichen (aber keine Ziffern) in Titlecase als Präfix verwendet werden. Beispielsweise verwendet **load libxyz4.2.so** das Präfix **Xyz** und **load bin/last.so {}** das Präfix **Last**.

Ist *fileName* ein leerer String, muss *prefix* angegeben werden. Der Befehl **load** sucht zuerst nach einer statisch geladenen Bibliothek (registriert durch Tcl_StaticLibrary) mit diesem Namen; wird eine gefunden, wird sie verwendet. Andernfalls sucht **load** nach einer dynamisch geladenen Bibliothek mit diesem Namen und verwendet sie, falls vorhanden. Wurden mehrere verschiedene Dateien mit unterschiedlichen Versionen der Bibliothek mit **load** geladen, wählt Tcl die zuerst geladene Datei.

Wird **-global** vor dem Dateinamen angegeben, werden alle Symbole der Shared Library für die globale Nutzung durch andere Bibliotheken exportiert. Die Option **-lazy** verzögert das tatsächliche Laden von Symbolen bis zu deren erster Verwendung. Die Optionen dürfen abgekürzt werden. Die Option **--** markiert das Ende der Optionen und sollte verwendet werden, wenn der Dateiname mit **-** beginnt und ein Präfix für **load** angegeben wird.

Auf Plattformen ohne Unterstützung für **-global** oder **-lazy** existieren die Optionen weiterhin, haben aber keine Wirkung. Die Verwendung von **-global** oder **-lazy** kann später zu Abstürzen der Anwendung führen (bei Symbolkonflikten bzw. fehlenden Symbolen), die während **load** nicht erkannt werden. Verwenden Sie diese Optionen daher nur, wenn Sie wissen, was Sie tun — bei Problemen mit der geladenen Bibliothek erhalten Sie keine aussagekräftige Fehlermeldung.

## Portabilitätsfragen

### Windows

Schlägt ein Laden mit der Fehlermeldung "library not found" fehl, kann auch eine abhängige Bibliothek fehlen. Um abhängige Bibliotheken anzuzeigen, geben Sie in einer DOS-Konsole "dumpbin -imports " ein, um die Importe der Bibliothek zu sehen. Beim Laden einer DLL im aktuellen Verzeichnis ignoriert Windows "./" als Pfadangabe und verwendet stattdessen eine Suchheuristik. Um das zu vermeiden, laden Sie die DLL mit:

```
load [file join [pwd] mylib.DLL]
```

## Bugs

Wird dieselbe Datei mit verschiedenen *fileName*s mit **load** geladen, wird sie mehrfach in den Adressraum des Prozesses geladen. Das Verhalten variiert je nach System (manche Systeme erkennen redundante Ladungen, andere nicht).

## Beispiele

Im Folgenden eine minimale Erweiterung:

```
#include <tcl.h>
#include <stdio.h>
static int fooCmd(void *clientData,
        Tcl_Interp *interp, int objc, Tcl_Obj *const objv[]) {
    printf("called with %d arguments\n", objc);
    return TCL_OK;
}
int Foo_Init(Tcl_Interp *interp) {
    if (Tcl_InitStubs(interp, "8.6-", 0) == NULL) {
    return TCL_ERROR;
    }
    printf("creating foo command");
    Tcl_CreateObjCommand(interp, "foo", fooCmd, NULL, NULL);
    return TCL_OK;
}
```

Wird sie als Shared-/Dynamic Library mit passendem Namen gebaut (z. B. **foo.dll** unter Windows, **libfoo.so** unter Solaris und Linux), kann sie in Tcl wie folgt geladen werden:

```
# Erweiterung laden
switch $tcl_platform(platform) {
    windows {
        load [file join [pwd] foo.dll]
    }
    unix {
        load [file join [pwd] libfoo[info sharedlibextension]]
    }
}

# Befehl der Erweiterung ausführen
foo
```

*Copyright (c) 1995-1996 Sun Microsystems, Inc.*

# lpop

*Stichwörter:* lpop, element, index, list, remove, pop, stack, queue

## Name

lpop - Holt ein Listenelement und entfernt es

## Übersicht

**lpop** *varName* ?index ...?

## Beschreibung

**lpop** erwartet *varName* als Namen einer Variable mit einer Tcl-Liste sowie einen oder mehrere Indizes in die Liste. Ohne Indizes gilt **end**.

Mit einem Index wird das *index*-te Element geholt, aus der Liste entfernt und zurückgegeben.

Ist *index* negativ oder größer oder gleich der Elementanzahl in *varName*, tritt ein Fehler auf.

Die Auslegung jedes einfachen *index* entspricht string index (Arithmetik, end-relativ).

Weitere *index*-Argumente adressieren wie bei lindex und lset verschachtelte Unterlisten:

```
lpop a 1 2
```

holt und entfernt Element 2 der Unterliste 1.

## Beispiele

Anfangswert von *x* in jedem Beispiel:

```
set x [list [list a b c] [list d e f] [list g h i]]
      → {a b c} {d e f} {g h i}
```

Der angezeigte Wert wird der neue Wert von *x* (außer beim letzten: Fehler, *x* unverändert).

```
lpop x 0
      → {d e f} {g h i}
lpop x 2
      → {a b c} {d e f}
lpop x end
      → {a b c} {d e f}
lpop x end-1
      → {a b c} {g h i}
lpop x 2 1
      → {a b c} {d e f} {g i}
lpop x 2 3 j
      → list index out of range
```

Weitere Beispiele, Anfangswert von *x*:

```
set x [list [list [list a b] [list c d]] \
            [list [list e f] [list g h]]]
      → {{a b} {c d}} {{e f} {g h}}
```

Der angezeigte Wert wird der neue Wert von *x*.

```
lpop x 1 1 0
      → {{a b} {c d}} {{e f} h}
```

*Copyright (c) 2018 Peter Spjuth. All rights reserved.*

# lrange

*Stichwörter:* lrange, element, list, range, sublist

## Name

lrange - Gibt ein oder mehrere benachbarte Listenelemente zurück

## Übersicht

**lrange** *list first last*

## Beschreibung

*list* muss eine gültige Tcl-Liste sein. Der Befehl gibt eine neue Liste aus den Elementen *first* bis *last* einschließlich zurück. *first* und *last* werden wie Indizes bei string index ausgelegt, einschließlich einfacher Indexarithmetik und Indizes relativ zum Listenende. Ist *first* kleiner als null, gilt es als 0. Ist *last* größer oder gleich der Elementanzahl, gilt es als **end**. Ist *first* größer als *last*, wird eine leere Zeichenkette zurückgegeben.

"**lrange** *list first first*" liefert nicht immer dasselbe wie "lindex *list first*" (bei einfachen, nicht in

Klammern stehenden Feldern oft schon); es entspricht aber genau "list [lindex *list first*]".

## Beispiele

Die ersten zwei Elemente:

```
% lrange {a b c d e} 0 1
a b
```

Die letzten drei Elemente:

```
% lrange {a b c d e} end-2 end
c d e
```

Alles außer erstem und letztem Element:

```
% lrange {a b c d e} 1 end-1
b c d
```

Ein Element mit **lrange** statt lindex:

```
% set var {some {elements to} select}
some {elements to} select
% lindex $var 1
elements to
% lrange $var 1 1
{elements to}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2001 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*

# lremove

*Stichwörter:* lremove, element, list, remove

## Name

lremove - Entfernt Listenelemente nach Index

## Übersicht

**lremove** *list* ?*index* ...?

## Beschreibung

**lremove** gibt eine neue Liste zurück, in der gleichzeitig null oder mehr Elemente von *list* an den durch beliebig viele *index*-Argumente genannten Positionen entfernt werden. Indizes in beliebiger Reihenfolge und wiederholbar; jedes Element wird höchstens einmal entfernt. Die Auslegung entspricht string index (Arithmetik, end-relativ). 0 = erstes Element, **end** = letztes.

## Beispiele

Drittes Element entfernen:

```
% lremove {a b c d e} 2
a b d e
```

Zwei Elemente entfernen:

```
% lremove {a b c d e} end-1 1
a c e
```

Dasselbe Element zweifach angegeben:

```
% lremove {a b c d e} 2 end-2
a b d e
```

*Copyright (c) 2019 Donal K. Fellows.*

# lrepeat

*Stichwörter:* lrepeat, element, index, list

## Name

lrepeat - Erzeugt eine Liste durch Wiederholen von Elementen

## Übersicht

**lrepeat** *count* ?*element* ...?

## Beschreibung

Der Befehl **lrepeat** erzeugt eine Liste der Größe *count* × *Anzahl der Elemente*, indem die Elementfolge *element ... count*-mal wiederholt wird. *count* muss eine nicht-negative Ganzzahl sein; *element* kann ein beliebiger Tcl-Wert sein.

Beachten Sie, dass **lrepeat 1 element ...** identisch zu **list element ...** ist.

## Beispiele

```
lrepeat 3 a
      → a a a
lrepeat 3 [lrepeat 3 0]
      → {0 0 0} {0 0 0} {0 0 0}
lrepeat 3 a b c
      → a b c a b c a b c
lrepeat 3 [lrepeat 2 a] b c
      → {a a} b c {a a} b c {a a} b c
```

*Copyright (c) 2003 Simon Geard. All rights reserved.*

# lreplace

*Stichwörter:* lreplace, element, list, replace

## Name

lreplace - Ersetzt Elemente in einer Liste durch neue Elemente

## Übersicht

**lreplace** *list first last* ?*element element* ...?

## Beschreibung

**lreplace** gibt eine neue Liste zurück, die entsteht, wenn null oder mehr Elemente von *list* durch die *element*-Argumente ersetzt werden. *first* und *last* begrenzen den zu ersetzenden Bereich; ihre Auslegung entspricht string index (Arithmetik, end-relativ). 0 ist das erste Element, **end** das letzte.

Ist *first* oder *last* kleiner als null, gilt die Position vor dem ersten Element — so können Elemente vorangestellt werden. Liegt *first* oder *last* hinter dem letzten Index, gilt Index "eins hinter dem letzten" — so können Elemente angehängt werden.

Ist *last* kleiner als *first*, werden die *element*s vor dem von *first* bezeichneten Element eingefügt, ohne dass etwas gelöscht wird.

Die *element*-Argumente sind die neuen Elemente an Stelle der gelöschten; jedes wird ein separates Listenelement. Ohne *element* werden nur die Elemente von *first* bis *last* entfernt.

## Beispiele

Ein Element ersetzen:

```
% lreplace {a b c d e} 1 1 foo
a foo c d e
```

Zwei Elemente durch drei ersetzen:

```
% lreplace {a b c d e} 1 2 three more elements
a three more elements d e
```

Letztes Element aus einer Variablenliste entfernen:

```
% set var {a b c d e}
a b c d e
% set var [lreplace $var end end]
a b c d
```

Prozedur zum Entfernen eines Werts:

```
proc lremove {listVariable value} {
    upvar 1 $listVariable var
    set idx [lsearch -exact $var $value]
    set var [lreplace $var $idx $idx]
}
```

Anhängen; **end+2** wird zunächst wie 6 behandelt, liegt aber hinter dem letzten Index und verhält sich wie ein Index hinter dem Ende:

```
% set var {a b c d e}
a b c d e
% set var [lreplace $var 12345 end+2 f g h i]
a b c d e f g h i
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2001 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*

# lreverse

*Stichwörter:* lreverse, element, list, reverse

## Name

lreverse - Kehrt die Reihenfolge einer Liste um

## Übersicht

**lreverse** *list*

## Beschreibung

**lreverse** gibt eine Liste mit denselben Elementen wie *list* zurück, jedoch in umgekehrter Reihenfolge.

## Beispiele

```
lreverse {a a b c}
      → c b a a
lreverse {a b {c d} e f}
      → f e {c d} b a
```

*Copyright (c) 2006 Donal K. Fellows. All rights reserved.*

# lsearch

*Stichwörter:* lsearch, binary search, linear search, list, match, pattern, regular expression, search , string

## Name

lsearch - Prüft, ob eine Liste ein bestimmtes Element enthält

## Übersicht

**lsearch** ?*options*? *list pattern*

## Beschreibung

Der Befehl durchsucht die Elemente von *list*, ob eines zu *pattern* passt. Dann gibt er den Index des ersten Treffers zurück (außer bei **-all** oder **-inline**). Sonst **-1** oder bei **-all**/**-inline** eine leere Zeichenkette. Die *option*-Argumente legen fest, wie verglichen wird; es muss einer der unten genannten Werte sein:

### Optionen für den Abgleichstil

Fehlen alle Abgleichstil-Optionen, ist der Standard **-glob**. Mehrere Abgleichstile: der zuletzt angegebene gilt.

**-exact**
*pattern* ist eine Literalzeichenkette; exakter Vergleich mit jedem Listenelement.

**-glob**
*pattern* ist ein Glob-Muster; Abgleich wie bei string match.

**-regexp**
*pattern* ist ein regulärer Ausdruck; Regeln siehe re_syntax.

**-sorted**
Die Listenelemente sind sortiert. Dann nutzt **lsearch** einen effizienteren Suchalgorithmus. Ohne weitere Optionen gilt: sortiert aufsteigend, ASCII-Zeichenketten. Schließt **-glob** und **-regexp** aus; bei **-all** oder **-not** wie **-exact**.

## Allgemeine Modifikatoren

Gelten für alle Abgleichstile.

**-all**
Ergebnis ist die Liste aller passenden Indizes (oder aller passenden Werte mit **-inline**). Indizes in aufsteigender numerischer Reihenfolge; Werte in der Reihenfolge innerhalb von *list*.

**-inline**
Der passende Wert statt seines Index (leere Zeichenkette ohne Treffer). Mit **-all**: Liste aller passenden Werte.

**-not**
Negiert den Treffer; Index des ersten Nicht-Treffers.

**-start *index***
Suche ab Position *index*; Auslegung wie bei string index (Arithmetik, end-relativ).

## Optionen zur Inhaltsauslegung

Bedeuten nur mit **-exact** und **-sorted**. Mehrere: der letzte gilt. Standard: **-ascii**.

**-ascii**
Elemente als Tcl-Zeichenketten, also Folgen von Unicode-Codepunkten (der Name aus Kompatibilitätsgründen).

**-dictionary**
Wörterbuch-Vergleich (siehe lsort). Nur mit **-sorted** sinnvoll anders als **-ascii**, da dictionary-gleich nur bei exakter Gleichheit.

**-integer**
Vergleich als Ganzzahlen.

**-nocase**
Vergleich ohne Groß-/Kleinschreibung. Keine Wirkung mit **-dictionary**, **-integer** oder **-real**.

**-real**
Vergleich als Fließkommazahlen.

## Optionen für sortierte Listen

Nur mit **-sorted**; mehrere: der letzte gilt. Standard: **-increasing**.

**-decreasing**
Liste absteigend sortiert. Nur mit **-sorted**.

**-increasing**
Liste aufsteigend sortiert. Nur mit **-sorted**.

**-bisect**
Ungenauer Suchtreffer bei sortierter Liste. Aufsteigend: letzter Index, an dem das Element kleiner oder gleich *pattern* ist. Absteigend: größer oder gleich. Liegt *pattern* vor dem ersten Element oder ist die Liste leer: **-1**. Impliziert **-sorted**; nicht mit **-all** oder **-not**.

### Optionen für verschachtelte Listen

Für Listen von Listen; kombinierbar mit anderen Optionen.

**-stride *strideLength***

Die Liste gilt als Gruppen zu je *strideLength* Elementen; Gruppen werden am ersten Element oder (mit **-index**) am Element der Gruppe gesucht, das der erste **-index**-Wert angibt (**-index** ignoriert diesen Teil dann). Der Index zeigt stets auf das erste Element einer Gruppe.

Die Listenlänge muss ein ganzzahliges Vielfaches von *strideLength* sein (mindestens 1). Standard 1 = keine Gruppierung.

**-index *indexList***

Für verschachtelte Listen: Pfad von Indizes (wie bei lindex/lset) innerhalb jedes Elements zum Vergleichswert.

**-subindices**

Der Index (oder jeder bei **-all**) ist ein vollständiger Pfad im Gesamtlistensinn für lindex/lset. Nur sinnvoll mit **-index**; Kurzform.

## Beispiele

Einfache Suche:

```
lsearch {a b c d e} c
      → 2
lsearch -all {a b c a b c} c
      → 2 5
```

Filtern mit **lsearch**:

```
lsearch -inline {a20 b35 c47} b*
      → b35
lsearch -inline -not {a20 b35 c47} b*
      → a20
lsearch -all -inline -not {a20 b35 c47} b*
      → a20 c47
lsearch -all -not {a20 b35 c47} b*
      → 0 2
```

"Set-artiges" Entfernen:

```
lsearch -all -inline -not -exact {a b c a d e a f g a} a
      → b c d e f g
```

Suche ab einer Position:

```
lsearch -start 3 {a b c a b c} c
      → 5
```

Suche innerhalb von Elementen:

```
lsearch -index 1 -all -inline {{abc abc} {abc bcd} {abc cde}} *bc*
      → {abc abc} {abc bcd}
```

Dasselbe mit flacher Liste:

```
lsearch -stride 2 -index 1 -all -inline {abc abc abc bcd abc cde} *bc*
      → abc abc abc bcd
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2001 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*
*Copyright (c) 2003-2004 Donal K. Fellows.*

# lseq

*Stichwörter:* lseq, element, index, list

## Name

lseq - Numerische Folge erzeugen und als Liste zurückgeben

## Übersicht

**lseq** *start* ?*(..|to)*? *end* ??*by*? *step?*
**lseq** *start* **count** *count* ??*by*? *step?*
**lseq** *count* ?*by step*?

## Beschreibung

Der Befehl **lseq** erzeugt eine Folge numerischer Werte, die entweder breite Ganzzahlen oder Gleitkommazahlen sein können, anhand der Parameter *start*, *end* und *step*. Das Argument *operation* "**..**" oder "**to**" definiert den Bereich. Die Option "**count**" legt die Anzahl der Elemente in der Liste fest. In der Kurzform mit nur einem Zählwert entsteht ein Bereich von 0 bis *count*-1.

Der Befehl **lseq** kann sowohl aufsteigende als auch absteigende Folgen erzeugen. Sind *start* und *end* ohne *step* angegeben, ist die Folge aufsteigend, wenn *start* <= *end*, und absteigend, wenn *start* > *end*. Ist ein *step*-Wert angegeben, sollte sein Vorzeichen zur Richtung der Folge passen (absteigend → negativ, aufsteigend → positiv); andernfalls wird eine leere Liste zurückgegeben. Beispiele:

```
# aufsteigend:
% lseq 1 to 5
→ 1 2 3 4 5

# absteigend:
% lseq 5 to 1
→ 5 4 3 2 1

# Gleitkommazahlen:
% lseq 0 0.5 by 0.1
→ 0.0 0.1 0.2 0.3 0.4 0.5

# absteigend, Schritt mit falschem Vorzeichen:
% lseq 6 to 1 by 2
→ {}

# Schritt 0,
% lseq 3 to 9 by 0
→ 3
```

*Start* legt den Anfangswert fest und *end* die Grenze, nicht notwendigerweise den letzten Wert. **lseq** erzeugt immer eine Liste mit *count* Elementen, auch wenn *step* 0 ist. Ist *count* nicht angegeben, wird es berechnet als:

```
count = int( ( (end - start) / step ) + 1 )
```

## Beispiele

```
lseq 3
→ 0 1 2

lseq 3 0
→ 3 2 1 0

lseq 10 .. 1 by -2
→ 10 8 6 4 2

set l [lseq 0 -5]
→ 0 -1 -2 -3 -4 -5

lseq 1 count 5 by 0
→ 1 1 1 1 1

foreach i [lseq [llength $l]] {
    puts l($i)=[lindex $l $i]
}
→ l(0)=0
→ l(1)=-1
→ l(2)=-2
→ l(3)=-3
→ l(4)=-4
→ l(5)=-5

foreach i [lseq {[llength $l]-1} 0] {
    puts l($i)=[lindex $l $i]
}
→ l(5)=-5
→ l(4)=-4
→ l(3)=-3
→ l(2)=-2
→ l(1)=-1
→ l(0)=0

set i 17
→ 17

if {$i in [lseq 0 50]} { # äquivalent zu: (0 <= $i && $i <= 50)
    puts "Ok"
} else {
    puts "outside :("
} → Ok

set sqrs [lmap i [lseq 1 10] { expr {$i*$i} }]
→ 1 4 9 16 25 36 49 64 81 100
```

*Copyright (c) 2022 Eric Taylor. All rights reserved.*

# lset

*Stichwörter:* lset, element, index, list, replace, set

## Name

lset - Ändert ein Element in einer Liste

## Übersicht

**lset** *varName* ?index ...? newValue

## Beschreibung

**lset** nimmt *varName* als Namen einer Variable mit einer Tcl-Liste, null oder mehr Indizes in die Liste und einen neuen Wert *newValue* für ein Element. Die Indizes können entweder nacheinander auf der Befehlszeile oder als Tcl-Liste gruppiert in einem einzigen Argument angegeben werden.

Ohne Indizes:

```
lset varName newValue
```

oder

```
lset varName {} newValue
```

Dann ersetzt *newValue* den alten Inhalt von *varName*.

Bei einem Index behandelt **lset** den Inhalt von *varName* als Tcl-Liste und adressiert das *index*-te Element (0 = erstes Element). Beim Interpretieren gelten dieselben Regeln für Klammern, Anführungszeichen und Backslashes wie beim Interpreter; Variablen- und Befehlssubstitution finden nicht statt. Es entsteht eine neue Liste mit ersetztem Element, wird in *varName* geschrieben und ist der Rückgabewert.

Ist *index* negativ oder größer als die Elementanzahl in *$varName*, tritt ein Fehler auf.

Ist *index* gleich der Elementanzahl in *$varName*, wird das gegebene Element angehängt.

Die Auslegung jedes einfachen *index* entspricht string index (Arithmetik, end-relativ).

Weitere *index*-Argumente adressieren nacheinander Elemente in Unterlisten der vorherigen Indexoperation — Änderung oder Anhängen in Unterlisten. Der Befehl

```
lset a 1 2 newValue
```

oder

```
lset a {1 2} newValue
```

ersetzt Element 2 der Unterliste 1 durch *newValue*.

Jede Ganzzahl in einem *index* muss ≥ 0 und ≤ der Länge der zugehörigen Liste sein. **lset** kann die Listengröße nur durch Anhängen (Setzen des Elements hinter dem aktuellen Ende) vergrößern. Liegt ein Index außerhalb, wird ein Fehler gemeldet.

## Beispiele

Anfangswert von *x* in jedem Beispiel:

```
set x [list [list a b c] [list d e f] [list g h i]]
      → {a b c} {d e f} {g h i}
```

Der angegebene Rückgabewert wird neuer Wert von *x* (außer beim letzten Fall — Fehler, *x* unverändert):

```
lset x {j k l}
      → j k l
lset x {} {j k l}
      → j k l
lset x 0 j
      → j {d e f} {g h i}
lset x 2 j
      → {a b c} {d e f} j
lset x end j
      → {a b c} {d e f} j
lset x end-1 j
      → {a b c} j {g h i}
lset x 2 1 j
```

```
      → {a b c} {d e f} {g j i}
lset x {2 1} j
      → {a b c} {d e f} {g j i}
lset x {2 3} j
      → {a b c} {d e f} {g h i j}
lset x {2 4} j
      → list index out of range
```

Weitere Beispiele; Anfangswert von *x*:

```
set x [list [list [list a b] [list c d]] \
            [list [list e f] [list g h]]]
      → {{a b} {c d}} {{e f} {g h}}
```

Der angegebene Rückgabewert wird neuer Wert von *x*.

```
lset x 1 1 0 j
      → {{a b} {c d}} {{e f} {j h}}
lset x {1 1 0} j
      → {{a b} {c d}} {{e f} {j h}}
```

*Copyright (c) 2001 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*

# lsort

*Stichwörter:* lsort, element, list, order, sort

## Name

lsort - Sortiert die Elemente einer Liste

## Übersicht

**lsort** ?*options*? *list*

## Beschreibung

Der Befehl sortiert die Elemente von *list* und gibt eine neue, sortierte Liste zurück. Die Implementierung nutzt Merge-Sort — stabil, O(n log n).

Standard ist ASCII-Sortierung in aufsteigender Reihenfolge. Vor *list* können Optionen stehen (Abkürzungen erlaubt):

**-ascii**
Zeichenkettenvergleich in Unicode-Codepoint-Reihenfolge (Name aus Kompatibilitätsgründen). Standard.

**-dictionary**
Wörterbuch-Vergleich. Wie **-ascii**, aber (a) Groß-/Klein nur als Tie-Breaker und (b) eingebettete Zahlen vergleichen als Ganzzahlen. Beispiel: **bigBoy** zwischen **bigbang** und **bigboy**; **x10y** zwischen **x9y** und **x11y**. Überschreibt **-nocase**.

**-integer**
Elemente als Ganzzahlen vergleichen.

**-real**
Elemente als Fließkommazahlen vergleichen.

**-command *command***

*command* als Vergleichsbefehl. Zwei Elemente werden verglichen, indem ein Tcl-Skript aus *command* mit den beiden Elementen als zusätzliche Argumente ausgewertet wird. Rückgabe: Ganzzahl < 0, = 0 oder > 0, wenn das erste Element kleiner, gleich oder größer sein soll.

**-increasing**

Aufsteigend sortieren ("kleinste" zuerst). Standard.

**-decreasing**

Absteigend sortieren ("größte" zuerst).

**-indices**

Liste der Indizes in *list* in sortierter Reihenfolge statt der Werte.

**-index *indexList***

Jedes Element von *list* muss eine Tcl-Unterliste sein (außer mit **-stride**). Sortierung nach dem *indexList*-ten Element jeder Unterliste (wie lindex). Beispiel:

```
lsort -integer -index 1 \
      {{First 24} {Second 18} {Third 30}}
```

liefert **{Second 18} {First 24} {Third 30}**,

```
lsort -index end-1 \
        {{a 1 e i} {b 2 3 f g} {c 4 5 6 d h}}
```

liefert **{c 4 5 6 d h} {a 1 e i} {b 2 3 f g}**, und

```
lsort -index {0 1} {
    {{b i g} 12345}
    {{d e m o} 34512}
    {{c o d e} 54321}
}
```

liefert **{{d e m o} 34512} {{b i g} 12345} {{c o d e} 54321}** (weil **e** vor **i** vor **o**). Effizienter als **-command** für denselben Effekt.

**-stride *strideLength***

Die Liste gilt als Gruppen zu je *strideLength* Elementen; Gruppen werden am ersten Element oder (mit **-index**) am durch den ersten **-index** bezeichneten Element der Gruppe sortiert ( **-index** ignoriert diesen Teil). Elemente bleiben innerhalb der Gruppe an ihrer Position.

Listenlänge muss Vielfaches von *strideLength* sein (mindestens 2).

Beispiel:

```
lsort -stride 2 {carrot 10 apple 50 banana 25}
```

liefert "apple 50 banana 25 carrot 10", und

```
lsort -stride 2 -index 1 -integer {carrot 10 apple 50 banana 25}
```

liefert "carrot 10 banana 25 apple 50".

**-nocase**

Vergleich ohne Groß-/Kleinschreibung. Keine Wirkung mit **-dictionary**, **-integer** oder **-real**.

**-unique**

Nur die letzte Gruppe doppelter Elemente bleibt. Duplikate relativ zum verwendeten Vergleich; mit **-index 0** gelten **{1 a}** und **{1 b}** als Duplikat — nur **{1 b}** bleibt.

## Hinweise

Die **lsort**-Optionen steuern nur die Vergleichsart, nicht zwingend den tatsächlichen Typ der Werte. Das fällt nur bei Listen mit weniger als zwei Elementen auf.

**lsort** ist reentrant — sicher als Implementierung eines in **-command** verwendeten Befehls.

## Beispiele

ASCII-Sortierung:

```
% lsort {a10 B2 b1 a1 a2}
B2 a1 a10 a2 b1
```

Wörterbuch-Sortierung:

```
% lsort -dictionary {a10 B2 b1 a1 a2}
a1 a2 a10 b1 B2
```

Ganzzahllisten:

```
% lsort -integer {5 3 1 2 11 4}
1 2 3 4 5 11
% lsort -integer {1 2 0x5 7 0 4 -1}
-1 0 1 2 4 0x5 7
```

Fließkommalisten:

```
% lsort -real {5 3 1 2 11 4}
1 2 3 4 5 11
% lsort -real {.5 0.07e1 0.4 6e-1}
0.4 .5 6e-1 0.07e1
```

Sortieren mit Indizes:

```
% # Leerzeichen vor dem c beachten
% lsort {{a 5} { c 3} {b 4} {e 1} {d 2}}
{ c 3} {a 5} {b 4} {d 2} {e 1}
% lsort -index 0 {{a 5} { c 3} {b 4} {e 1} {d 2}}
{a 5} {b 4} { c 3} {d 2} {e 1}
% lsort -index 1 {{a 5} { c 3} {b 4} {e 1} {d 2}}
{e 1} {d 2} { c 3} {b 4} {a 5}
```

Dictionary sortieren:

```
% set d [dict create c d a b h i f g c e]
c e a b h i f g
% lsort -stride 2 $d
a b c e f g h i
```

Stride und mehrere Indizes:

```
% # Erster Indexwert ist relativ zur Gruppe
% lsort -stride 3 -index {0 1} \
     {{Bob Smith} 25 Audi {Jane Doe} 40 Ford}
{{Jane Doe} 40 Ford {Bob Smith} 25 Audi}
```

Duplikate entfernen durch Sortieren:

```
% lsort -unique {a b c a b c a b c}
a b c
```

Vergleichsfunktion:

```
% proc compare {a b} {
    set a0 [lindex $a 0]
    set b0 [lindex $b 0]
    if {$a0 < $b0} {
        return -1
    } elseif {$a0 > $b0} {
        return 1
    }
    return [string compare [lindex $a 1] [lindex $b 1]]
}
% lsort -command compare \
        {{3 apple} {0x2 carrot} {1 dingo} {2 banana}}
{1 dingo} {2 banana} {0x2 carrot} {3 apple}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 1999 Scriptics Corporation*
*Copyright (c) 2001 Kevin B. Kenny* kennykb@acm.org*. All rights reserved.*

# mathfunc

*Stichwörter:* mathfunc

## Name

mathfunc - Mathematische Funktionen für Tcl-Ausdrücke

## Übersicht

**package require Tcl 8.5-**

**::tcl::mathfunc::abs** *arg*
**::tcl::mathfunc::acos** *arg*
**::tcl::mathfunc::asin** *arg*
**::tcl::mathfunc::atan** *arg*
**::tcl::mathfunc::atan2** *y x*
**::tcl::mathfunc::bool** *arg*
**::tcl::mathfunc::ceil** *arg*
**::tcl::mathfunc::cos** *arg*

**::tcl::mathfunc::cosh** *arg*
**::tcl::mathfunc::double** *arg*
**::tcl::mathfunc::entier** *arg*
**::tcl::mathfunc::exp** *arg*
**::tcl::mathfunc::floor** *arg*
**::tcl::mathfunc::fmod** *x y*
**::tcl::mathfunc::hypot** *x y*
**::tcl::mathfunc::int** *arg*
**::tcl::mathfunc::isfinite** *arg*
**::tcl::mathfunc::isinf** *arg*
**::tcl::mathfunc::isnan** *arg*
**::tcl::mathfunc::isnormal** *arg*
**::tcl::mathfunc::isqrt** *arg*
**::tcl::mathfunc::issubnormal** *arg*
**::tcl::mathfunc::isunordered** *x y*
**::tcl::mathfunc::log** *arg*
**::tcl::mathfunc::log10** *arg*
**::tcl::mathfunc::max** *arg ?arg ...?*
**::tcl::mathfunc::min** *arg ?arg ...?*
**::tcl::mathfunc::pow** *x y*
**::tcl::mathfunc::rand**
**::tcl::mathfunc::round** *arg*
**::tcl::mathfunc::sin** *arg*
**::tcl::mathfunc::sinh** *arg*
**::tcl::mathfunc::sqrt** *arg*
**::tcl::mathfunc::srand** *arg*
**::tcl::mathfunc::tan** *arg*
**::tcl::mathfunc::tanh** *arg*
**::tcl::mathfunc::wide** *arg*

## Beschreibung

Der Befehl expr behandelt mathematische Funktionen der Form **sin($x)** oder **atan2($y,$x)**, indem er sie in Aufrufe der Form **[tcl::mathfunc::sin [expr {$x}]]** oder **[tcl::mathfunc::atan2 [expr {$y}] [expr {$x}]]** umwandelt. Im Namespace **::tcl::mathfunc** stehen standardmäßig zahlreiche mathematische Funktionen zur Verfügung; diese sind auch außerhalb von expr nutzbar, indem die genannten Befehle direkt aufgerufen werden.

Tcl unterstützt in Ausdrücken die folgenden mathematischen Funktionen, die — sofern nicht anders angegeben — ausschließlich mit Gleitkommazahlen arbeiten:

**abs acos asin atan**
**atan2 bool ceil cos**
**cosh double entier exp**
**floor fmod hypot int**
**isfinite isinf isnan isnormal**
**isqrt issubnormal isunordered log**
**log10 max min pow**

**rand round sin sinh**
**sqrt srand tan tanh**
**wide**

Zusätzlich zu diesen vordefinierten Funktionen können Anwendungen weitere Funktionen definieren, indem sie proc (oder eine andere Methode wie interp alias oder Tcl_CreateObjCommand) verwenden, um neue Befehle im Namespace **tcl::mathfunc** anzulegen.

## Detaillierte Definitionen

**abs *arg***

Gibt den Absolutwert von *arg* zurück. *Arg* kann Ganzzahl oder Gleitkommazahl sein; das Ergebnis wird in derselben Form zurückgegeben.

**acos *arg***

Gibt den Arkuskosinus von *arg* im Bereich [*0*,*pi*] Radiant zurück. *Arg* sollte im Bereich [*-1*,*1*] liegen.

**asin *arg***

Gibt den Arkussinus von *arg* im Bereich [*-pi/2*,*pi/2*] Radiant zurück. *Arg* sollte im Bereich [*-1*,*1*] liegen.

**atan *arg***

Gibt den Arkustangens von *arg* im Bereich [*-pi/2*,*pi/2*] Radiant zurück.

**atan2 *y x***

Gibt den Arkustangens von *y*/*x* im Bereich [*-pi*,*pi*] Radiant zurück. *x* und *y* dürfen nicht beide 0 sein. Ist *x* größer als *0*, ist dies äquivalent zu "**atan** [expr {*y*/*x*}]".

**bool *arg***

Akzeptiert jeden numerischen Wert oder jede Zeichenkette, die **string is boolean** akzeptiert, und gibt den entsprechenden booleschen Wert **0** oder **1** zurück. Von Null verschiedene Zahlen sind wahr. Andere Zahlen sind falsch. Nicht-numerische Zeichenketten liefern boolesche Werte gemäß **string is true** und **string is false**.

**ceil *arg***

Gibt den kleinsten integralen Gleitkommawert (d. h. mit Null Nachkommastellen) zurück, der nicht kleiner als *arg* ist. Das Argument kann ein beliebiger numerischer Wert sein.

**cos *arg***

Gibt den Kosinus von *arg* zurück, gemessen in Radiant.

**cosh *arg***

Gibt den hyperbolischen Kosinus von *arg* zurück. Würde das Ergebnis einen Überlauf verursachen, wird ein Fehler zurückgegeben.

**double *arg***

Das Argument kann ein beliebiger numerischer Wert sein. Ist *arg* ein Gleitkommawert, gibt der Befehl *arg* zurück, andernfalls wandelt er *arg* in Gleitkomma um und gibt den konvertierten Wert zurück. Kann **Inf** oder **-Inf** zurückgeben, wenn das Argument den Gleitkommabereich überschreitet.

**entier *arg***

Das Argument kann ein beliebiger numerischer Wert sein. Der Ganzzahl-Anteil von *arg* wird ermittelt und zurückgegeben. Der von dieser Funktion zurückgegebene Ganzzahl-Bereich ist unbegrenzt, im Gegensatz zu **int** und **wide**, die ihren Bereich auf bestimmte Speicherbreiten kürzen.

**exp** ***arg***

Gibt die Exponentialfunktion von *arg* zurück, definiert als *e****arg*. Würde das Ergebnis einen Überlauf verursachen, wird ein Fehler zurückgegeben.

**floor** ***arg***

Gibt den größten integralen Gleitkommawert (d. h. mit Null Nachkommastellen) zurück, der nicht größer als *arg* ist. Das Argument kann ein beliebiger numerischer Wert sein.

**fmod** ***x y***

Gibt den Gleitkomma-Rest der Division von *x* durch *y* zurück. Ist *y* gleich 0, wird ein Fehler zurückgegeben.

**hypot** ***x y***

Berechnet die Länge der Hypotenuse eines rechtwinkligen Dreiecks, ungefähr "**sqrt** [expr {*x***x*+*y***y*}]", außer dass sie numerisch stabiler ist, wenn die beiden Argumente deutlich unterschiedliche Größenordnungen haben.

**int** ***arg***

Das Argument kann ein beliebiger numerischer Wert sein. Der Ganzzahl-Anteil von *arg* wird ermittelt; anschließend werden die niederwertigen Bits dieses Ganzzahl-Werts bis zur Maschinenwortgröße als Ganzzahl-Wert zurückgegeben. Zur Referenz: Die Anzahl der Bytes im Maschinenwort ist im Element **wordSize** des Arrays tcl_platform gespeichert.

**isfinite** ***arg***

Gibt 1 zurück, wenn die Gleitkommazahl arg endlich ist, d. h. Null, subnormal oder normal. Gibt 0 zurück, wenn die Zahl unendlich oder NaN ist. Löst einen Fehler aus, wenn arg nicht in einen Gleitkommawert umgewandelt werden kann.

**isinf** ***arg***

Gibt 1 zurück, wenn die Gleitkommazahl arg unendlich ist. Gibt 0 zurück, wenn die Zahl endlich oder NaN ist. Löst einen Fehler aus, wenn arg nicht in einen Gleitkommawert umgewandelt werden kann.

**isnan** ***arg***

Gibt 1 zurück, wenn die Gleitkommazahl arg Not-a-Number (NaN) ist. Gibt 0 zurück, wenn die Zahl endlich oder unendlich ist. Löst einen Fehler aus, wenn arg nicht in einen Gleitkommawert umgewandelt werden kann.

**isnormal** ***arg***

Gibt 1 zurück, wenn die Gleitkommazahl arg normal ist. Gibt 0 zurück, wenn die Zahl Null, subnormal, unendlich oder NaN ist. Löst einen Fehler aus, wenn arg nicht in einen Gleitkommawert umgewandelt werden kann.

**isqrt** ***arg***

Berechnet den Ganzzahl-Anteil der Quadratwurzel von *arg*. *Arg* muss ein positiver Wert sein, Ganzzahl oder Gleitkommazahl. Im Gegensatz zu **sqrt**, das auf die Genauigkeit einer Gleitkommazahl beschränkt ist, liefert *isqrt* ein Ergebnis beliebiger Genauigkeit.

**issubnormal** ***arg***

Gibt 1 zurück, wenn die Gleitkommazahl arg subnormal ist, d. h. das Ergebnis graduellen Underflows. Gibt 0 zurück, wenn die Zahl Null, normal, unendlich oder NaN ist. Löst einen Fehler aus, wenn arg nicht in einen Gleitkommawert umgewandelt werden kann.

**isunordered** ***x y***

Gibt 1 zurück, wenn x und y nicht vergleichbar geordnet werden können, d. h. wenn mindestens einer NaN ist. Gibt 0 zurück, wenn beide Werte geordnet werden können, d. h. wenn beide aus der Menge Null, subnormal, normal und unendlich stammen. Löst einen Fehler aus, wenn x oder y nicht in einen Gleitkommawert umgewandelt werden kann.

**log *arg***

Gibt den natürlichen Logarithmus von *arg* zurück. *Arg* muss ein positiver Wert sein.

**log10 *arg***

Gibt den Logarithmus zur Basis 10 von *arg* zurück. *Arg* muss ein positiver Wert sein.

**max *arg* ...**

Akzeptiert ein oder mehrere numerische Argumente. Gibt das Argument mit dem größten Wert zurück.

**min *arg* ...**

Akzeptiert ein oder mehrere numerische Argumente. Gibt das Argument mit dem kleinsten Wert zurück.

**pow *x y***

Berechnet *x* hoch *y*. Ist *x* negativ, muss *y* ein Ganzzahl-Wert sein.

**rand**

Gibt einen pseudozufälligen Gleitkommawert im Bereich (*0*,*1*) zurück. Der Generator ist ein einfacher linearer Kongruenzgenerator ohne kryptographische Sicherheit. Jedes Ergebnis von **rand** bestimmt vollständig alle zukünftigen Ergebnisse nachfolgender Aufrufe von **rand**; **rand** sollte daher nicht zur Erzeugung geheimer Folgen, etwa Einmalpasswörter, verwendet werden. Der Startwert wird aus der internen Systemuhr initialisiert oder kann mit der Funktion **srand** gesetzt werden.

**round *arg***

Ist *arg* ein Ganzzahlwert, gibt der Befehl *arg* zurück, andernfalls wird *arg* durch Runden in eine Ganzzahl umgewandelt und der umgewandelte Wert zurückgegeben.

**sin *arg***

Gibt den Sinus von *arg* zurück, gemessen in Radiant.

**sinh *arg***

Gibt den hyperbolischen Sinus von *arg* zurück. Würde das Ergebnis einen Überlauf verursachen, wird ein Fehler zurückgegeben.

**sqrt *arg***

Das Argument kann ein beliebiger nicht-negativer numerischer Wert sein. Gibt einen Gleitkommawert zurück, der die Quadratwurzel von *arg* ist. Kann **Inf** zurückgeben, wenn das Argument den Quadrat des maximalen Gleitkommawerts überschreitet.

**srand *arg***

Das *arg*, das eine Ganzzahl sein muss, setzt den Startwert des Zufallszahlengenerators von **rand** zurück. Gibt die erste Zufallszahl (siehe **rand**) aus diesem Startwert zurück. Jeder Interpreter hat seinen eigenen Startwert.

**tan *arg***

Gibt den Tangens von *arg* zurück, gemessen in Radiant.

**tanh *arg***

Gibt den hyperbolischen Tangens von *arg* zurück.

**wide *arg***

Das Argument kann ein beliebiger numerischer Wert sein. Der ganzzahlige Teil von *arg* wird bestimmt, dann werden die niederwertigen 64 Bits dieses Ganzzahlwerts als Ganzzahl zurückgegeben.

## Copyright

```
Copyright © 1993 The Regents of the University of California.
Copyright © 1994-2000 Sun Microsystems Incorporated.
Copyright © 2005-2006 Kevin B. Kenny <kennykb@acm.org>.
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-2000 Sun Microsystems, Inc.*
*Copyright (c) 2005 Kevin B. Kenny* kennykb@acm.org*. All rights reserved*

# mathop

*Stichwörter:* mathop, command, expression, operator

## Name

mathop - Mathematische Operatoren als Tcl-Befehle

## Übersicht

**package require Tcl 8.5-**

**::tcl::mathop::!** *number*
**::tcl::mathop::~** *number*
**::tcl::mathop::+** ?*number* ...?
**::tcl::mathop::-** *number* ?*number* ...?
**::tcl::mathop::*** ?*number* ...?
**::tcl::mathop::/** *number* ?*number* ...?
**::tcl::mathop::%** *number number*
**::tcl::mathop::**** ?*number* ...?
**::tcl::mathop::&** ?*number* ...?
**::tcl::mathop::|** ?*number* ...?
**::tcl::mathop::^** ?*number* ...?
**::tcl::mathop::<<** *number number*
**::tcl::mathop::>>** *number number*
**::tcl::mathop::==** ?*arg* ...?
**::tcl::mathop::!=** *arg arg*
**::tcl::mathop::<** ?*arg* ...?
**::tcl::mathop::<=** ?*arg* ...?
**::tcl::mathop::>=** ?*arg* ...?
**::tcl::mathop::>** ?*arg* ...?
**::tcl::mathop::eq** ?*arg* ...?
**::tcl::mathop::ne** *arg arg*
**::tcl::mathop::lt** ?*arg* ...?
**::tcl::mathop::le** ?*arg* ...?
**::tcl::mathop::gt** ?*arg* ...?
**::tcl::mathop::ge** ?*arg* ...?
**::tcl::mathop::in** *arg list*

**::tcl::mathop::ni** *arg list*

## Beschreibung

Die Befehle im Namespace **::tcl::mathop** implementieren dieselbe Menge von Operationen, die der Befehl expr unterstützt. Alle werden aus dem Namespace exportiert, aber standardmäßig in keinen anderen Namespace importiert. Beachten Sie, dass Umbenennen, Neuimplementieren oder Löschen von Befehlen im Namespace *nicht* das Verhalten des Befehls expr ändert, ebenso wenig wie das Definieren neuer Befehle im Namespace **::tcl::mathop**.

Folgende Operator-Befehle werden unterstützt:

**~! + - ***
**/ % ** & |**
**^ >> << == eq**
**!= ne < <= >**
**>= in ni lt le**
**gt ge**

### Mathematische Operatoren

Das Verhalten der mathematischen Operator-Befehle ist wie folgt:

**! *boolean***

Gibt die boolesche Negation von *boolean* zurück, wobei *boolean* ein beliebiger numerischer Wert oder eine andere Form eines booleschen Werts sein kann (*d. h.* es gibt Wahrheit zurück, wenn das Argument Falschheit oder Null ist, und Falschheit, wenn das Argument Wahrheit oder ungleich Null ist).

**+ ?*number* ...?**

Gibt die Summe beliebig vieler Argumente zurück. Jedes *number*-Argument kann ein beliebiger numerischer Wert sein. Sind keine Argumente angegeben, ist das Ergebnis Null (das neutrale Element der Addition).

**- *number* ?*number* ...?**

Ist nur ein einzelnes *number*-Argument angegeben, gibt der Befehl die Negation dieses numerischen Werts zurück. Andernfalls gibt er die Zahl zurück, die entsteht, wenn alle folgenden numerischen Werte vom ersten subtrahiert werden. Alle *number*-Argumente müssen numerische Werte sein. Mindestens ein Argument muss angegeben sein.

*** ?*number* ...?**

Gibt das Produkt beliebig vieler Argumente zurück. Jedes *number* kann ein beliebiger numerischer Wert sein. Sind keine Argumente angegeben, ist das Ergebnis eins (das neutrale Element der Multiplikation).

**/ *number* ?*number* ...?**

Ist nur ein einzelnes *number*-Argument angegeben, gibt der Befehl den Kehrwert dieses numerischen Werts zurück (*d. h.* den Wert, der durch Division von 1.0 durch diesen Wert entsteht). Andernfalls gibt er die Zahl zurück, die entsteht, wenn das erste numerische Argument durch alle folgenden numerischen Argumente dividiert wird. Alle *number* -Argumente müssen numerische Werte sein. Mindestens ein Argument muss angegeben sein.

Beachten Sie, dass bei führenden Integer-Werten in der Argumentliste für diese anfänglichen Schritte Integer-Division verwendet wird (*d. h.* die Zwischenergebnisse sind so, als würden die

Funktionen *floor* und *int* in dieser Reihenfolge angewendet). Sind alle Werte der Operation Integer, ist das Ergebnis ein Integer.

**% *number number***

Gibt den integralen Modulus (*d. h.* Rest) des ersten Arguments bezüglich des zweiten zurück. Jedes *number* muss einen integralen Wert haben. Außerdem hat das Ergebnis dasselbe Vorzeichen wie das zweite *number*, das nicht Null sein darf.

Beachten Sie, dass Tcl diese Operation auch für negative Zahlen exakt definiert, sodass folgender Befehl einen wahren Wert zurückgibt (der Namespace wird zur Klarheit weggelassen):

```
== [* [/ x y] y] [- x [% x y]]
```

**** ?*number* ...?**

Gibt das Ergebnis zurück, bei dem jeder Wert potenziert wird mit dem Ergebnis nach rekursiver Verarbeitung der folgenden Argumente, sodass "**** 2 3 4**" dasselbe ist wie "**** 2 [** 3 4]**". Jedes *number* kann ein beliebiger numerischer Wert sein, wobei die zweite Zahl nicht bruchteilig sein darf, wenn die erste negativ ist. Der maximale Exponentenwert, den Tcl verarbeiten kann, wenn die erste Zahl ein Integer > 1 ist, beträgt 268435455. Sind keine Argumente angegeben, ist das Ergebnis eins; ist nur ein Argument angegeben, ist das Ergebnis dieses Argument. Das Ergebnis hat nur dann einen integralen Wert, wenn alle Argumente integrale Werte sind.

## Vergleichsoperatoren

Das Verhalten der Vergleichsoperator-Befehle (die überwiegend numerische Argumente bevorzugen) ist wie folgt:

**== ?*arg* ...?**

Gibt zurück, ob jedes Argument den Argumenten auf beiden Seiten im Sinne des expr -Operators == gleich ist (*d. h.* numerischer Vergleich wenn möglich, sonst exakter Zeichenkettenvergleich). Sind weniger als zwei Argumente angegeben, gibt diese Operation stets einen wahren Wert zurück.

**eq ?*arg* ...?**

Gibt zurück, ob jedes Argument den Argumenten auf beiden Seiten per exaktem Zeichenkettenvergleich gleich ist. Sind weniger als zwei Argumente angegeben, gibt diese Operation stets einen wahren Wert zurück.

**!= *arg arg***

Gibt zurück, ob die beiden Argumente einander im Sinne des expr-Operators!= ungleich sind (*d. h.* numerischer Vergleich wenn möglich, sonst exakter Zeichenkettenvergleich).

**ne *arg arg***

Gibt zurück, ob die beiden Argumente einander per exaktem Zeichenkettenvergleich ungleich sind.

**< ?*arg* ...?**

Gibt zurück, ob die beliebig vielen Argumente geordnet sind, wobei jedes Argument nach dem ersten strikt größer als das vorhergehende sein muss. Vergleiche erfolgen vorzugsweise auf numerischen Werten, andernfalls per UNICODE-Zeichenkettenvergleich. Sind weniger als zwei Argumente vorhanden, gibt diese Operation stets einen wahren Wert zurück. Sollen numerische Argumente als Zeichenketten verglichen werden, sollte stattdessen der Operator

**lt** oder der Befehl string compare verwendet werden.

**<= ?*arg* ...?**
Gibt zurück, ob die beliebig vielen Argumente geordnet sind, wobei jedes Argument nach dem ersten gleich oder größer als das vorhergehende sein muss. Vergleiche erfolgen vorzugsweise auf numerischen Werten, andernfalls per UNICODE-Zeichenkettenvergleich. Sind weniger als zwei Argumente vorhanden, gibt diese Operation stets einen wahren Wert zurück. Sollen numerische Argumente als Zeichenketten verglichen werden, sollte stattdessen der Operator **le** oder der Befehl string compare verwendet werden.

**> ?*arg* ...?**
Gibt zurück, ob die beliebig vielen Argumente geordnet sind, wobei jedes Argument nach dem ersten strikt kleiner als das vorhergehende sein muss. Vergleiche erfolgen vorzugsweise auf numerischen Werten, andernfalls per UNICODE-Zeichenkettenvergleich. Sind weniger als zwei Argumente vorhanden, gibt diese Operation stets einen wahren Wert zurück. Sollen numerische Argumente als Zeichenketten verglichen werden, sollte stattdessen der Operator **gt** oder der Befehl string compare verwendet werden.

**>= ?*arg* ...?**
Gibt zurück, ob die beliebig vielen Argumente geordnet sind, wobei jedes Argument nach dem ersten gleich oder kleiner als das vorhergehende sein muss. Vergleiche erfolgen vorzugsweise auf numerischen Werten, andernfalls per UNICODE-Zeichenkettenvergleich. Sind weniger als zwei Argumente vorhanden, gibt diese Operation stets einen wahren Wert zurück. Sollen numerische Argumente als Zeichenketten verglichen werden, sollte stattdessen der Operator **ge** oder der Befehl string compare verwendet werden.

**lt ?*arg* ...?**
Gibt zurück, ob die beliebig vielen Argumente geordnet sind, wobei jedes Argument nach dem ersten strikt größer als das vorhergehende sein muss. Vergleiche erfolgen per UNICODE-Zeichenkettenvergleich. Sind weniger als zwei Argumente vorhanden, gibt diese Operation stets einen wahren Wert zurück.

**le ?*arg* ...?**
Gibt zurück, ob die beliebig vielen Argumente geordnet sind, wobei jedes Argument nach dem ersten gleich oder strikt größer als das vorhergehende sein muss. Vergleiche erfolgen per UNICODE-Zeichenkettenvergleich. Sind weniger als zwei Argumente vorhanden, gibt diese Operation stets einen wahren Wert zurück.

**gt ?*arg* ...?**
Gibt zurück, ob die beliebig vielen Argumente geordnet sind, wobei jedes Argument nach dem ersten strikt kleiner als das vorhergehende sein muss. Vergleiche erfolgen per UNICODE-Zeichenkettenvergleich. Sind weniger als zwei Argumente vorhanden, gibt diese Operation stets einen wahren Wert zurück.

**ge ?*arg* ...?**
Gibt zurück, ob die beliebig vielen Argumente geordnet sind, wobei jedes Argument nach dem ersten gleich oder strikt kleiner als das vorhergehende sein muss. Vergleiche erfolgen per UNICODE-Zeichenkettenvergleich. Sind weniger als zwei Argumente vorhanden, gibt diese Operation stets einen wahren Wert zurück.

## Bitweise Operatoren

Das Verhalten der bitweisen Operator-Befehle (die alle nur mit integralen Argumenten arbeiten) ist wie folgt:

**~ *number***
Gibt die bitweise Negation von *number* zurück. *Number* kann ein Integer beliebiger Größe sein. Beachten Sie, dass das Ergebnis dieser Operation stets das entgegengesetzte Vorzeichen zum Eingabe-*number* hat.

**& ?*number* ...?**
Gibt das bitweise UND aller beliebig vielen Argumente zurück. Jedes *number* muss einen integralen Wert haben. Sind keine Argumente angegeben, ist das Ergebnis minus eins.

**| ?*number* ...?**
Gibt das bitweise ODER aller beliebig vielen Argumente zurück. Jedes *number* muss einen integralen Wert haben. Sind keine Argumente angegeben, ist das Ergebnis Null.

**^ ?*number* ...?**
Gibt das bitweise XOR aller beliebig vielen Argumente zurück. Jedes *number* muss einen integralen Wert haben. Sind keine Argumente angegeben, ist das Ergebnis Null.

**<< *number number***
Gibt das Ergebnis zurück, bei dem das erste Argument um die im zweiten Argument angegebene Anzahl Bits nach links verschoben wird. Jedes *number* muss einen integralen Wert haben.

**>> *number number***
Gibt das Ergebnis zurück, bei dem das erste Argument um die im zweiten Argument angegebene Anzahl Bits nach rechts verschoben wird. Jedes *number* muss einen integralen Wert haben.

### Listenoperatoren

Das Verhalten der listenorientierten Operator-Befehle ist wie folgt:

**in *arg list***
Gibt zurück, ob der Wert *arg* in der Liste *list* enthalten ist (gemäß exaktem Zeichenkettenvergleich der Elemente).

**ni *arg list***
Gibt zurück, ob der Wert *arg* nicht in der Liste *list* enthalten ist (gemäß exaktem Zeichenkettenvergleich der Elemente).

## Beispiele

Der einfachste Weg, die Operatoren zu nutzen, ist oft namespace path, um die Befehle verfügbar zu machen. Das hat den Vorteil, dass die im aktuellen Namespace definierten Befehle nicht verändert werden.

```
namespace path {::tcl::mathop ::tcl::mathfunc}

# Summe einiger Zahlen berechnen
set sum [+ 1 2 3]

# Mittelwert einer Liste berechnen
set list {1 2 3 4 5 6}
set mean [/ [+ {*}$list] [double [llength $list]]]
```

```
# Listenmitgliedschaft prüfen
set gotIt [in 3 $list]

# Prüfen, ob ein Wert in einem definierten Bereich liegt
set inRange [<= 1 $x 5]

# Prüfen, ob eine Liste numerisch sortiert ist
set sorted [<= {*}$list]

# Prüfen, ob eine Liste lexikalisch sortiert ist
set alphaList {a b c d e f}
set sorted [le {*}$alphaList]
```

*Copyright (c) 2006-2007 Donal K. Fellows.*

## memory

*Stichwörter:* memory, debug

### Name

memory - Steuert die Tcl-Speicher-Debugging-Funktionen

### Übersicht

**memory** *option* ?*arg arg* ...?

### Beschreibung

Der Befehl **memory** gibt Tcl-Entwicklern die Kontrolle über die Speicher-Debugging-Funktionen von Tcl. Der Befehl **memory** hat mehrere Unteroptionen, die unten beschrieben werden. Er ist nur verfügbar, wenn Tcl mit aktiviertem Speicher-Debugging kompiliert wurde (wenn TCL_MEM_DEBUG zur Kompilierzeit definiert ist) und nachdem Tcl_InitMemory aufgerufen wurde.

**memory active *file***

Schreibt eine Liste aller derzeit allozierten Speicherbereiche in die angegebene *file*.

**memory break_on_malloc *count***

Nach *count* durchgeführten Allozierungen gibt Tcl_Alloc eine entsprechende Meldung aus und versucht, in den C-Debugger einzutreten. Tcl sendet sich dann ein *SIGINT*-Signal. Wenn Tcl unter einem C-Debugger läuft, sollte er in den Debugger-Befehlsmodus wechseln.

**memory info**

Gibt einen Bericht zurück mit den Gesamtanzahlen von Allozierungen und Freigaben seit Tcl-Start, derzeit allozierten Paketen (Anzahl von Tcl_Alloc-Aufrufen ohne entsprechenden Tcl_Free-Aufruf), derzeit allozierten Bytes sowie der maximalen Anzahl allozierter Pakete und Bytes.

**memory init [on|off]**

Schaltet die Vorinitialisierung allen allozierten Speichers mit Dummy-Bytes ein oder aus. Nützlich zum Erkennen der Verwendung nicht initialisierter Werte.

**memory objs *file***

Veranlasst, dass sofort eine Liste aller allozierten Tcl_Obj-Werte zusammen mit dem Ort ihrer

Allozierung in die angegebene *file* geschrieben wird. Nützlich zur Prüfung auf Lecks von Werten.

**memory onexit *file***

Veranlasst, dass beim Finalisieren des Tcl-Speicher-Subsystems eine Liste aller allozierten Speicherbereiche in die angegebene *file* geschrieben wird. Nützlich zur Prüfung, dass der Speicher beim Prozessende ordnungsgemäß freigegeben wird.

**memory tag *string***

Jedem von Tcl_Alloc allozierten Speicherpaket kann ein Zeichenketten-Tag zugeordnet werden. In den von **memory active** und **memory onexit** erzeugten Listen wird das Tag jedes Pakets zusammen mit weiteren Informationen ausgegeben. Der Befehl **memory tag** setzt den Tag-Wert für nachfolgende Tcl_Alloc-Aufrufe auf *string*.

**memory trace [on|off]**

Schaltet Speicher-Tracing ein oder aus. Ist Speicher-Tracing aktiv, verursacht jeder Tcl_Alloc -Aufruf eine Trace-Zeile auf *stderr*, bestehend aus dem Wort Tcl_Alloc, gefolgt von der zurückgegebenen Adresse, der allozierten Speichermenge sowie C-Dateiname und Zeilennummer des allozierenden Codes. Beispiel:

```
Tcl_Alloc 40e478 98 tclProc.c 1406
```

Tcl_Free-Aufrufe werden auf dieselbe Weise protokolliert.

**memory trace_on_at_malloc *count***

Aktiviert Speicher-Tracing nach *count* Tcl_Alloc-Aufrufen. Wird z. B. **memory trace_on_at_malloc 100** eingegeben, werden ab dem 100. Tcl_Alloc-Aufruf Trace-Informationen für alle Allozierungen und Freigaben ausgegeben. Da vor einem Fehler viel Speicheraktivität auftreten kann, verringert ein gezielter Einsatz dieser Option die Verlangsamung durch Tracing (und die Menge der Trace-Daten), wenn die Anzahl der Allozierungen vor dem Auftreten des Problems bekannt ist. Bei einem Guard-Zone-Fehler wird die aktuelle Anzahl der Speicherallozierungen seit Tcl-Start ausgegeben.

**memory validate [on|off]**

Schaltet Speicher-Validierung ein oder aus. Ist sie aktiv, werden bei jedem Tcl_Alloc- oder Tcl_Free-Aufruf die Guard-Zonen jedes noch existierenden, von Tcl_Alloc allozierten Speicherbereichs geprüft. Das hat große Auswirkungen auf die Performance und sollte nur bei starkem Verdacht auf Überschreibungen verwendet werden. Vorteil: Eine Guard-Zone-Überschreibung kann schon beim ersten Tcl_Alloc- oder Tcl_Free-Aufruf nach der Überschreibung erkannt werden, nicht erst bei Freigabe des betroffenen Speichers — was lange danach erfolgen kann.

*Copyright (c) 1992-1999 Karl Lehenbauer & Mark Diekhans*
*Copyright (c) 2000 Scriptics Corporation.*

# msgcat

*Stichwörter:* msgcat, internationalization, i18n, localization, l10n, message, text, translation, class, object

## Name

msgcat - Tcl-Message-Katalog

## Übersicht

**package require tcl 9.0**
**package require msgcat 1.7**

**::msgcat::mc** *src-string* ?*arg arg* ...?
**::msgcat::mcmax** ?*src-string src-string* ...?
**::msgcat::mcexists** ?**-exactnamespace**? ?**-exactlocale**? *src-string*
**::msgcat::mcpackagenamespaceget**
**::msgcat::mclocale** ?*newLocale*?
**::msgcat::mcpreferences** ?*locale preference*? ...
**::msgcat::mcloadedlocales subcommand**
**::msgcat::mcload** *dirname*
**::msgcat::mcset** *locale src-string* ?*translate-string*?
**::msgcat::mcmset** *locale src-trans-list*
**::msgcat::mcflset** *src-string* ?*translate-string*?
**::msgcat::mcflmset** *src-trans-list*
**::msgcat::mcunknown** *locale src-string* ?*arg arg* ...?
**::msgcat::mcpackagelocale subcommand** ?*locale*?
**::msgcat::mcpackageconfig subcommand** *option* ?*value*?
**::msgcat::mcforgetpackage**
**::msgcat::mcutil subcommand** ?*locale*?

## Beschreibung

Das Paket **msgcat** stellt eine Reihe von Funktionen bereit, mit denen mehrsprachige Benutzeroberflächen verwaltet werden können. Textzeichenketten werden in einem Message-Katalog definiert, der unabhängig von der Anwendung ist und bearbeitet oder lokalisiert werden kann, ohne den Anwendungsquellcode zu ändern. Neue Sprachen oder Locales können durch Hinzufügen einer neuen Datei zum Message-Katalog bereitgestellt werden.

**msgcat** unterscheidet Pakete anhand ihres Namespace. Jedes Paket hat seinen eigenen Message-Katalog und seine Konfigurationseinstellungen in **msgcat**.

Ein *locale* ist eine Spezifikationszeichenkette, die eine Benutzersprache beschreibt, z. B. **de_ch** für Schweizerdeutsch. In **msgcat** gibt es ein globales Locale, das anhand des System-Locale des aktuellen Systems initialisiert wird. Jedes Paket kann entscheiden, das globale Locale oder ein paketspezifisches Locale zu verwenden.

Das globale Locale kann bei Bedarf geändert werden, z. B. durch eine vom Benutzer ausgelöste Sprachumschaltung oder in einer Mehrbenutzer-Anwendung wie einem Web-Server.

Objektorientierte Programmierung wird durch die Verwendung eines Paket-Namespace unterstützt.

## Befehle

### ::msgcat::mc *src-string ?arg arg ...?*

Gibt eine Übersetzung von *src-string* gemäß dem aktuellen Locale zurück. Wenn nach *src-string* weitere Argumente angegeben werden, wird der Befehl format verwendet, um die zusätzlichen Argumente in die Übersetzung von *src-string* einzusetzen.

**::msgcat::mc** durchsucht die im aktuellen Namespace definierten Meldungen nach einer

Übersetzung von *src-string*; wird keine gefunden, wird im übergeordneten Namespace des aktuellen Namespace gesucht und so weiter, bis der globale Namespace erreicht ist. Existiert keine Übersetzungszeichenkette, wird **::msgcat::mcunknown** aufgerufen und die von **::msgcat::mcunknown** zurückgegebene Zeichenkette wird zurückgegeben.

**::msgcat::mc** ist die Hauptfunktion zur Lokalisierung einer Anwendung. Statt eine englische Zeichenkette direkt zu verwenden, kann eine Anwendung die englische Zeichenkette durch **::msgcat::mc** leiten und das Ergebnis nutzen. Wird eine Anwendung auf diese Weise für eine einzige Sprache geschrieben, lässt sich später leicht Unterstützung für weitere Sprachen hinzufügen, indem einfach neue Message-Katalog-Einträge definiert werden.

**::msgcat::mcn *namespace src-string* ?*arg arg* ...?**

Wie ::msgcat::mc, jedoch mit dem Message-Namespace als erstem Argument.

**mcn** kann verwendet werden, wenn der Paket-Namespace nicht der Namespace des Aufrufers ist. Ein Beispiel steht in der Beschreibung des Befehls **::msgcat::mcpackagenamespaceget** weiter unten.

**::msgcat::mcmax ?*src-string src-string* ...?**

Bei mehreren Quellzeichenketten gibt **::msgcat::mcmax** die Länge der längsten übersetzten Zeichenkette zurück. Das ist nützlich beim Entwurf lokalisierter GUIs, die z. B. erfordern können, dass alle Schaltflächen eine feste Breite haben (nämlich die Breite der breitesten Schaltfläche).

**::msgcat::mcexists ?-exactnamespace? ?-exactlocale? ?-*namespace namespace*? *src-string***

Gibt **true** zurück, wenn eine Übersetzung für die angegebene *src-string* existiert.

Die Suche kann mit der Option **-exactnamespace** auf den aktuellen Namespace beschränkt werden, ohne übergeordnete Namespaces zu prüfen.

Sie kann auch mit der Option **-exactlocale** auf das erste bevorzugte Locale beschränkt werden (z. B. das erste Element von **::msgcat::mcpreferences**, wenn das globale Locale verwendet wird).

Ein expliziter Paket-Namespace kann mit der Option -namespace angegeben werden. Wird er nicht explizit angegeben, wird der Namespace des Aufrufers verwendet.

**::msgcat::mcpackagenamespaceget**

Gibt den Paket-Namespace des Aufrufers zurück. Dieser Befehl behandelt alle in Abschnitt Objektorientierte Programmierung beschriebenen Fälle.

Beispiel: Ein Tooltip-Paket speichert den Paket-Namespace des Aufrufers, um die Übersetzung bei jedem Anzeigen des Tooltips zu aktualisieren:

```
proc ::tooltip::tooltip {widget message} {
    ...
    set messagenamespace [uplevel 1
    {::msgcat::mcpackagenamespaceget}]
    ...
    bind $widget  [list ::tooltip::show $widget $messagenamespace
    $message]
}

proc ::tooltip::show {widget messagenamespace message} {
    ...
    set message [::msgcat::mcn $messagenamespace $message]
    ...
}
```

**::msgcat::mclocale ?*newLocale*?**

Wenn *newLocale* weggelassen wird, wird das aktuelle Locale zurückgegeben, andernfalls wird das aktuelle Locale auf *newLocale* gesetzt.

Wird das neue Locale auf *newLocale* gesetzt, werden die zugehörigen Präferenzen berechnet und gesetzt. Wenn das aktuelle Locale z. B. en_US_funky ist, gibt **::msgcat::mcpreferences {en_us_funky en_us en {}}** zurück.

Dasselbe Ergebnis lässt sich mit **::msgcat::mcpreferences** {*}[**::msgcat::mcutil getpreferences** *newLocale*] erzielen.

Das aktuelle Locale ist immer das erste Element der von **mcpreferences** zurückgegebenen Liste.

msgcat speichert und vergleicht das Locale ohne Berücksichtigung der Groß-/Kleinschreibung und gibt Locales in Kleinbuchstaben zurück. Das anfängliche Locale wird durch das in der Umgebung des Benutzers angegebene Locale bestimmt. Siehe Locale-Spezifikation weiter unten für das Locale-Zeichenkettenformat.

Wenn das Locale gesetzt wird, wird die Präferenzliste der Locales ausgewertet. Locales in dieser Liste werden jetzt geladen, falls sie noch nicht geladen sind.

**::msgcat::mcpreferences ?*locale preference*? ...**

Ohne Argumente gibt dieser Befehl eine geordnete Liste der vom Benutzer bevorzugten Locales zurück. Die Liste ist von der spezifischsten zur geringsten Präferenz geordnet.

Es kann eine Menge von Locale-Präferenzen angegeben werden, um die Liste der Locale-Präferenzen zu setzen. Das aktuelle Locale wird ebenfalls gesetzt; es ist das erste Element der Locale-Präferenzliste.

Locale-Präferenzen werden jetzt geladen, falls sie noch nicht geladen sind.

Beispiel: Der Benutzer bevorzugt französischen oder englischen Text. Das lässt sich so konfigurieren:

```
::msgcat::mcpreferences fr en {}
```

**::msgcat::mcloadedlocales subcommand**

Diese Gruppe von Befehlen verwaltet die Liste der geladenen Locales für Pakete, die kein

Paket-Locale setzen.

Der Unterbefehl **loaded** gibt die Liste der aktuell geladenen Locales zurück.

Der Unterbefehl **clear** entfernt alle Locales und ihre Daten, die nicht in der aktuellen Präferenzliste stehen.

**::msgcat::mcload *dirname***

Durchsucht das angegebene Verzeichnis nach Dateien, die den von **::msgcat::mcloadedlocales loaded** zurückgegebenen Sprachspezifikationen entsprechen (oder **msgcat::mcpackagelocale preferences**, wenn ein Paket-Locale gesetzt ist) (diese sind alle in Kleinbuchstaben), erweitert um die Dateierweiterung ".msg". Jede passende Datei wird der Reihe nach gelesen, unter Annahme einer UTF-8-Kodierung. Der Dateiinhalt wird dann als Tcl-Skript ausgewertet. Das bedeutet, dass Unicode-Zeichen in der Message-Datei entweder direkt in UTF-8-kodierter Form oder durch von der Tcl-Auswertung erkanntes Backslash-u-Quoting vorhanden sein können. Die Anzahl der Message-Dateien, die der Spezifikation entsprachen und geladen wurden, wird zurückgegeben.

Zusätzlich wird der angegebene Ordner in der Paketkonfigurationsoption *mcfolder* von **msgcat** gespeichert, um bei einer Locale-Änderung gegebenenfalls benötigte Message-Katalog-Dateien zu laden.

**::msgcat::mcset *locale src-string* ?*translate-string*?**

Legt die Übersetzung für *src-string* auf *translate-string* im angegebenen *locale* und im aktuellen Namespace fest. Wenn *translate-string* nicht angegeben ist, wird *src-string* für beides verwendet. Die Funktion gibt *translate-string* zurück.

**::msgcat::mcmset *locale src-trans-list***

Legt die Übersetzung für mehrere Quellzeichenketten in *src-trans-list* im angegebenen *locale* und im aktuellen Namespace fest. *src-trans-list* muss eine gerade Anzahl von Elementen haben und hat die Form {*src-string translate-string* ?*src-string translate-string ...*?} **::msgcat::mcmset** kann deutlich schneller sein als mehrere Aufrufe von **::msgcat::mcset**. Die Funktion gibt die Anzahl der gesetzten Übersetzungen zurück.

**::msgcat::mcflset *src-string* ?*translate-string*?**

Legt die Übersetzung für *src-string* auf *translate-string* im aktuellen Namespace für das Locale fest, das sich aus dem Namen des über **::msgcat::mcload** geladenen Message-Katalogs ergibt. Wenn *translate-string* nicht angegeben ist, wird *src-string* für beides verwendet. Die Funktion gibt *translate-string* zurück.

**::msgcat::mcflmset *src-trans-list***

Legt die Übersetzung für mehrere Quellzeichenketten in *src-trans-list* im aktuellen Namespace für das Locale fest, das sich aus dem Namen des über **::msgcat::mcload** geladenen Message-Katalogs ergibt. *src-trans-list* muss eine gerade Anzahl von Elementen haben und hat die Form {*src-string translate-string* ?*src-string translate-string ...*?} **::msgcat::mcflmset** kann deutlich schneller sein als mehrere Aufrufe von **::msgcat::mcflset**. Die Funktion gibt die Anzahl der gesetzten Übersetzungen zurück.

**::msgcat::mcunknown *locale src-string* ?*arg arg* ...?**

Diese Routine wird von **::msgcat::mc** aufgerufen, wenn für *src-string* im aktuellen Locale keine Übersetzung definiert ist. Die Standardaktion ist, *src-string* per format zurückzugeben, wenn Argumente vorhanden sind. Diese Prozedur kann von der Anwendung neu definiert werden, z. B. um Fehlermeldungen für jede unbekannte Zeichenkette zu protokollieren. Die Prozedur **::msgcat::mcunknown** wird im gleichen Stack-Kontext wie der Aufruf von **::msgcat::mc** ausgeführt. Der Rückgabewert von **::msgcat::mcunknown** wird als Rückgabewert für den Aufruf von **::msgcat::mc** verwendet.

Hinweis: Diese Routine wird nur aufgerufen, wenn das betreffende Paket keinen paketspezifischen Unknown-Befehl gesetzt hat.

**::msgcat::mcforgetpackage**

Das aufrufende Paket löscht seinen gesamten Zustand innerhalb des Pakets **msgcat** einschließlich aller Einstellungen und Übersetzungen.

**::msgcat::mcutil getpreferences *locale***

Gibt die Präferenzliste des angegebenen Locale zurück, wie im Abschnitt Locale-Spezifikation beschrieben. Ein Beispiel ist die Zusammensetzung einer Präferenzliste für die zweisprachige Region "Biel/Bienne" als Verkettung von Schweizerdeutsch und Schweizerfranzösisch:

```
% concat [lrange [msgcat::mcutil getpreferences fr_CH] 0 end-1]
    [msgcat::mcutil getpreferences de_CH]
fr_ch fr de_ch de {}
```

**::msgcat::mcutil getsystemlocale**

Gibt das System-Locale zurück, wie im Abschnitt Locale-Spezifikation beschrieben.

## Locale-Spezifikation

Das Locale wird **msgcat** durch eine an **::msgcat::mclocale** übergebene Locale-Zeichenkette mitgeteilt. Die Locale-Zeichenkette besteht aus einem Sprachcode, einem optionalen Ländercode und einem optionalen systemspezifischen Code, jeweils getrennt durch "_". Die Länder- und Sprachcodes sind in den Standards ISO-639 und ISO-3166 festgelegt. Zum Beispiel legt das Locale "en" Englisch fest und "en_US" U.S.-Englisch.

Wenn das Paket msgcat zum ersten Mal geladen wird, wird das Locale gemäß der Umgebung des Benutzers initialisiert. Die Variablen **env(LC_ALL)**, **env(LC_MESSAGES)** und **env(LANG)** werden der Reihe nach geprüft. Der erste von ihnen mit nicht leerem Wert wird zur Bestimmung des anfänglichen Locale verwendet. Der Wert wird gemäß dem XPG4-Muster

```
language[_country][.codeset][@modifier]
```

zerlegt. Das anfängliche Locale wird dann gesetzt, indem **::msgcat::mclocale** mit folgendem Argument aufgerufen wird:

```
language[_country][_modifier]
```

Unter Windows und Cygwin versucht msgcat, falls keine dieser Umgebungsvariablen gesetzt ist, Locale-Informationen aus der Registry zu extrahieren. Der RFC4747-Locale-Name "lang-script-country-options" wird in das Locale als "lang_country_script" umgewandelt (Beispiel: sr-Latn-CS -> sr_cs_latin). Schlagen alle Versuche fehl, ein anfängliches Locale aus der Umgebung des Benutzers zu ermitteln, verwendet msgcat standardmäßig ein anfängliches Locale von "C".

Wenn der Benutzer ein Locale angibt, wird bei der Zeichenkettenübersetzung eine "best match"-Suche durchgeführt. Gibt ein Benutzer z. B. en_GB_Funky an, werden die Locales "en_gb_funky", "en_gb", "en" und "" (die leere Zeichenkette) der Reihe nach durchsucht, bis eine passende Übersetzungszeichenkette gefunden wird. Ist keine Übersetzungszeichenkette verfügbar, wird der Unknown-Handler aufgerufen.

## Namespaces und Message-Kataloge

Im Message-Katalog gespeicherte Zeichenketten werden relativ zum Namespace gespeichert, aus dem sie hinzugefügt wurden. Das ermöglicht mehreren Paketen, dieselben Zeichenketten zu verwenden, ohne Kollisionen mit anderen Paketen zu befürchten. Außerdem können Quellzeichenketten kürzer und weniger fehleranfällig bei Tippfehlern sein.

Zum Beispiel gibt die Ausführung des Codes

```
::msgcat::mcset en hello "hello from ::"
namespace eval foo {
    ::msgcat::mcset en hello "hello from ::foo"
}
puts [::msgcat::mc hello]
namespace eval foo {puts [::msgcat::mc hello]}
```

aus

```
hello from ::
hello from ::foo
```

Bei der Suche nach einer Übersetzung einer Meldung durchsucht der Message-Katalog zuerst den aktuellen Namespace, dann den übergeordneten des aktuellen Namespace und so weiter, bis der globale Namespace erreicht ist. Das ermöglicht Kind-Namespaces, Meldungen vom übergeordneten Namespace zu übernehmen.

Zum Beispiel gibt die Ausführung (im Locale "en") des Codes

```
::msgcat::mcset en m1 ":: message1"
::msgcat::mcset en m2 ":: message2"
::msgcat::mcset en m3 ":: message3"
namespace eval ::foo {
    ::msgcat::mcset en m2 "::foo message2"
    ::msgcat::mcset en m3 "::foo message3"
}
namespace eval ::foo::bar {
    ::msgcat::mcset en m3 "::foo::bar message3"
}
namespace import ::msgcat::mc
puts "[mc m1]; [mc m2]; [mc m3]"
namespace eval ::foo {puts "[mc m1]; [mc m2]; [mc m3]"}
namespace eval ::foo::bar {puts "[mc m1]; [mc m2]; [mc m3]"}
```

aus

```
:: message1; :: message2; :: message3
:: message1; ::foo message2; ::foo message3
:: message1; ::foo message2; ::foo::bar message3
```

## Speicherort und Format von Message-Dateien

Message-Dateien können in jedem Verzeichnis liegen, vorbehaltlich der folgenden Bedingungen:

1. Alle Message-Dateien eines Pakets liegen im selben Verzeichnis.
2. Der Dateiname ist ein msgcat-Locale-Spezifizierer (alles Kleinbuchstaben) gefolgt von ".msg". Zum Beispiel:

```
es.msg    — spanish
en_gb.msg — United Kingdom English
```

*Ausnahme:* Die Message-Datei für das Root-Locale "" heißt "**ROOT.msg**". Diese Ausnahme verhindert ungewöhnliches Verhalten, z. B. dass die Message-Datei als "hidden" auf Unix-Dateisystemen markiert wird.

1. Die Datei enthält eine Reihe von Aufrufen von **mcflset** und **mcflmset**, die die nötigen Übersetzungszeichenketten für die Sprache setzen, typischerweise in einem namespace eval, damit alle Quellzeichenketten an den Namespace des Pakets gebunden sind. Eine kurze **es.msg** könnte z. B. enthalten:

```
namespace eval ::mypackage {
    ::msgcat::mcflset "Free Beer" "Cerveza Gratis"
}
```

## Empfohlene Message-Einrichtung für Pakete

Wenn ein Paket in ein Unterverzeichnis von tcl_pkgPath installiert und über package require geladen wird, empfiehlt sich folgendes Vorgehen.

1. Erstellen Sie bei der Paketinstallation ein Unterverzeichnis **msgs** unter Ihrem Paketverzeichnis.
2. Kopieren Sie Ihre *.msg-Dateien in dieses Verzeichnis.
3. Fügen Sie den folgenden Befehl in Ihr Paket-Initialisierungsskript ein:

```
# Sprachdateien laden, gespeichert im Unterverzeichnis msgs
::msgcat::mcload [file join [file dirname [info script]] msgs]
```

## Positionale Platzhalter für Format- und Scan-Befehle

Es ist möglich, dass eine als Argument für format verwendete Message-Zeichenkette positionsabhängige Parameter hat, die beim Übersetzen umgestellt werden müssen. Zum Beispiel kann es syntaktisch sinnvoll sein, die Satzstruktur umzuordnen.

```
format "We produced %d units in location %s" $num $city
format "In location %s we produced %d units" $city $num
```

Das lässt sich mit positionalen Parametern handhaben:

```
format "We produced %1\$d units in location %2\$s" $num $city
format "In location %2\$s we produced %1\$d units" $num $city
```

Analog können positionale Parameter mit scan verwendet werden, um Werte aus internationalisierten Zeichenketten zu extrahieren. Hinweis: Es ist nicht nötig, die Ausgabe von **::msgcat::mc** direkt an format zu übergeben; durch Übergabe der einzusetzenden Werte als Argumente erfolgt die Format-Ersetzung direkt.

```
msgcat::mc {Produced %1$d at %2$s} $num $city
# ... wobei dieser Schlüssel durch msgcat::mcset auf eine der
# menschenlesbaren Versionen abgebildet wird
```

## Paketprivate Locale

Ein Paket, das **msgcat** verwendet, kann ein eigenes paketprivates Locale und eine eigene Menge geladener Locales verwenden, unabhängig vom globalen Locale, das durch **::msgcat::mclocale** gesetzt wird.

So kann ein Paket sein Locale ändern, ohne Locales in anderen Paketen zu laden oder zu entfernen und ohne den globalen Locale-Änderungs-Rückruf aufzurufen (siehe unten).

Dieses Verhalten wird durch folgendes Ensemble gesteuert:

**::msgcat::mcpackagelocale set ?*locale*?**

Setzt oder ändert ein paketprivates Locale. Das paketprivate Locale wird auf das angegebene *locale* gesetzt, wenn *locale* angegeben ist. Wenn die Option *locale* nicht angegeben ist, wird das Paket in den Modus paketprivates Locale versetzt, aber kein Locale geändert (z. B. wenn das globale Locale zuvor für das Paket gültig war, wird es ins paketprivate Locale kopiert).

Dieser Befehl kann das Laden von Locales auslösen.

**::msgcat::mcpackagelocale get**

Gibt das paketprivate Locale oder das globale Locale zurück, wenn kein paketprivates Locale gesetzt ist.

**::msgcat::mcpackagelocale preferences ?*locale preference*? ...**

Ohne Parameter gibt dieser Befehl die paketprivaten Präferenzen oder die globalen Präferenzen zurück, wenn kein paketprivates Locale gesetzt ist. Der Paket-Locale-Zustand (gesetzt oder nicht) wird nicht geändert (im Gegensatz zu **::msgcat::mcpackagelocale set** ).

Wird eine Menge von Locale-Präferenzen angegeben, wird sie als paketlocale Präferenzliste gesetzt. Das Paket-Locale wird auf das erste Element der Präferenzliste gesetzt. Ein Paket-Locale wird aktiviert, falls es bisher nicht gesetzt war.

Locale-Präferenzen werden für das Paket jetzt geladen, falls noch nicht geladen.

**::msgcat::mcpackagelocale loaded**

Gibt die Liste der für dieses Paket geladenen Locales zurück.

**::msgcat::mcpackagelocale isset**

Gibt **true** zurück, wenn ein paketprivates Locale gesetzt ist.

**::msgcat::mcpackagelocale unset**

Hebt das paketprivate Locale auf und verwendet das globale Locale. Lädt und entfernt Locales, um die Liste der geladenen Locales des Pakets an die globale Liste geladener Locales anzupassen.

**::msgcat::mcpackagelocale present *locale***

Gibt **true** zurück, wenn das angegebene Locale für das Paket geladen ist.

**::msgcat::mcpackagelocale clear**

Entfernt alle geladenen Locales des Pakets, die nicht in den Paket-Präferenzen enthalten sind.

## Änderung der Paketoptionen

Jedes Paket, das msgcat verwendet, hat eine Menge von Optionen innerhalb von **msgcat**. Die Paketoptionen werden im nächsten Abschnitt Paketoptionen beschrieben. Jede Paketoption kann einzeln mit folgendem Ensemble gesetzt oder zurückgesetzt werden:

**::msgcat::mcpackageconfig get *option***

Gibt den aktuellen Wert der angegebenen *option* zurück. Dieser Aufruf meldet einen Fehler, wenn die Option für das Paket nicht gesetzt ist.

**::msgcat::mcpackageconfig isset *option***

Gibt 1 zurück, wenn die angegebene *option* für das Paket gesetzt ist, sonst 0.

**::msgcat::mcpackageconfig set *option value***

Setzt die angegebene *option* auf den angegebenen *value*. Das kann je nach *option* weitere Aktionen auslösen. Der Rückgabewert ist 0 oder die Anzahl geladener Pakete für die Option **mcfolder**.

**::msgcat::mcpackageconfig unset *option***

Setzt die angegebene *option* für das Paket zurück. Es erfolgt keine Aktion, wenn die *option* für das Paket nicht gesetzt ist. Die leere Zeichenkette wird zurückgegeben.

### Paketoptionen

Folgende Paketoptionen stehen für jedes Paket zur Verfügung:

**mcfolder**

Dies ist der Message-Ordner des Pakets. Diese Option wird durch mcload und den Unterbefehl set gesetzt. Beides ist identisch und gibt die Anzahl geladener Message-Katalog-Dateien zurück.

Das Setzen oder Ändern dieses Werts lädt alle Locales, die in den für das Paket gültigen Präferenzen enthalten sind. Das impliziert auch den Aufruf eines gesetzten loadcmd (siehe unten).

Das Zurücksetzen dieses Werts deaktiviert das Laden von Message-Dateien für das Paket.

**loadcmd**

Dieser Rückruf wird aufgerufen, bevor eine Menge von Message-Katalog-Dateien für das Paket geladen wird, das diese Eigenschaft gesetzt hat.

Dieser Rückruf kann Vorbereitungsarbeiten für das Laden von Message-Dateien leisten oder Message-Daten aus einer anderen Quelle wie einer Datenbank holen. In diesem Fall werden keine Message-Dateien verwendet (mcfolder ist nicht gesetzt).

Siehe Abschnitt Rückruf-Aufruf unten. Die an diesen Rückruf angehängte Parameterliste ist die Liste der zu ladenden Locales.

Wird dieser Rückruf geändert, wird er mit den für das Paket gültigen Präferenzen aufgerufen.

**changecmd**

Dieser Rückruf wird aufgerufen, wenn eine Standard-Locale-Änderung durchgeführt wurde. Er soll einem Paket erlauben, Abhängigkeiten vom Standard-Locale zu aktualisieren, z. B. die GUI in einer anderen Sprache anzuzeigen.

Siehe den Abschnitt Rückruf-Aufruf unten. Die angehängte Parameterliste ist **mcpreferences** . Die registrierten Rückrufe werden in beliebiger Reihenfolge aufgerufen.

**unknowncmd**

Verwendet eine paketlocale mcunknown-Prozedur statt der von msgcat bereitgestellten Standardversion (**msgcat::mcunknown**).

Die aufgerufene Prozedur muss die formatierte Meldung zurückgeben, die schließlich von **msgcat::mc** zurückgegeben wird.

Ein generischer Unknown-Handler wird verwendet, wenn auf die leere Zeichenkette gesetzt. Dieser gibt den Schlüssel zurück, wenn keine Argumente übergeben werden. Mit Argumenten wird der Befehl format zur Verarbeitung verwendet.

Siehe Abschnitt Rückruf-Aufruf unten. Die angehängten Argumente sind identisch zu **msgcat::mcunknown**.

## Rückruf-Aufruf

Ein Paket kann einen oder mehrere Rückrufe registrieren, wie oben beschrieben.

Rückrufe werden aufgerufen, wenn:

1. der Rückrufbefehl gesetzt ist,
2. der Befehl nicht die leere Zeichenkette ist,
3. der registrierende Namespace existiert.

Schlägt eine aufgerufene Routine mit einem Fehler fehl, wird nach Befehlsende die bgerror -Routine des Interpreters aufgerufen. Ausnahme ist der Rückruf **unknowncmd**, bei dem ein Fehler den aufrufenden **mc**-Befehl mit diesem Fehler fehlschlagen lässt.

## Objektorientierte Programmierung

**msgcat** unterstützt Pakete, die objektorientiert implementiert sind. Objekte und Klassen sollten innerhalb eines Paket-Namespace definiert werden.

Es gibt drei unterstützte Fälle, in denen paketnamespace-sensitive Befehle von msgcat (**mc**, **mcexists**, **mcpackagelocale**, **mcforgetpackage**, **mcpackagenamespaceget**, **mcpackageconfig**, **mcset** und **mcmset**) aufgerufen werden können:

**1) Im Klassendefinitionsskript**

Ein **msgcat**-Befehl wird innerhalb eines Klassendefinitionsskripts aufgerufen.

```
namespace eval ::N2 {
    mcload $dir/msgs
    oo::class create C1 {puts [mc Hi!]}
}
```

**2) Methode in einer Klasse**

Ein **msgcat**-Befehl wird aus einer Methode in einem Objekt aufgerufen, und die Methode ist in einer Klasse definiert.

```
namespace eval ::N3Class {
    mcload $dir/msgs
    oo::class create C1
    oo::define C1 method m1 {
        puts [mc Hi!]
    }
}
```

**3) Methode in einem klassenlosen Objekt**

Ein **msgcat**-Befehl wird aus einer Methode eines klassenlosen Objekts aufgerufen.

```
namespace eval ::N4 {
    mcload $dir/msgs
    oo::object create O1
    oo::objdefine O1 method m1 {} {
        puts [mc Hi!]
    }
}
```

## Beispiele

Pakete mit GUI können ihre Widgets aktualisieren, wenn sich das globale Locale ändert. Zur Registrierung eines Rückrufs:

```
namespace eval gui {
    msgcat::mcpackageconfig changecmd updateGUI

    proc updateGUI args {
        puts "New locale is '[lindex $args 0]'."
    }
}
% msgcat::mclocale fr
fr
% New locale is 'fr'.
```

Liegen Locales (oder zusätzliche Locales) in einer anderen Quelle wie einer Datenbank, kann ein Paket den Lade-Rückruf statt **mcload** verwenden:

```
namespace eval db {
    msgcat::mcpackageconfig loadcmd loadMessages

    proc loadMessages args {
        foreach locale $args {
            if {[LocaleInDB $locale]} {
                msgcat::mcmset $locale [GetLocaleList $locale]
            }
        }
    }
}
```

Die clock-Befehlsimplementierung nutzt **msgcat** mit einem Paket-Locale für den Befehlszeilenparameter **-locale**. Skizzen der Implementierung:

Zuerst wird ein Paket-Locale initialisiert und die generische Unknown-Funktion deaktiviert:

```
msgcat::mcpackagelocale set
msgcat::mcpackageconfig unknowncmd ""
```

Als Beispiel benötigt der Benutzer den Wochentag in einem bestimmten Locale:

```
clock format [clock seconds] -format %A -locale fr
```

clock setzt das Paket-Locale auf **fr** und sucht den Tagesnamen wie folgt:

```
msgcat::mcpackagelocale set $locale
return [lindex [msgcat::mc DAYS_OF_WEEK_FULL] $day]
### Gibt "mercredi" zurück
```

Innerhalb von clock sind einige Message-Katalog-Einträge rechenintensiv und werden daher dynamisch gecacht:

```
proc ::tcl::clock::LocalizeFormat { locale format } {
    set key FORMAT_$format
    if { [::msgcat::mcexists -exactlocale -exactnamespace $key] } {
        return [mc $key]
    }
    # ... aufwendige Berechnung des Formats gekürzt ...
    mcset $locale $key $format
    return $format
}
```

## Danksagungen

Der Message-Katalog-Code wurde von Mark Harrison entwickelt.

*Copyright (c) 1998 Mark Harrison.*

# my

*Stichwörter:* my, method, method visibility, object, private method, public method

## Name

my, myclass - Ruft eine beliebige Methode des aktuellen Objekts oder seiner Klasse auf

## Übersicht

**package require tcl::oo**

**my** *methodName* ?*arg* ...?
**myclass** *methodName* ?*arg* ...?

## Beschreibung

Der Befehl **my** ermöglicht es Methoden von Objekten, Methoden des Objekts (oder seiner Klasse) aufzurufen, und der Befehl myclass ermöglicht es Methoden von Objekten, Methoden der aktuellen Klasse des Objekts als Objekt aufzurufen. Insbesondere umfasst die Menge gültiger Werte für *methodName* alle Methoden, die ein Objekt und seine Superklassen unterstützen, einschließlich der nicht exportierten sowie privater Methoden des Objekts oder der Klasse, wenn sie innerhalb einer anderen von diesem Objekt oder dieser Klasse definierten Methode verwendet werden.

Das Objekt, auf dem die Methode über **my** aufgerufen wird, ist dasjenige, dem der Namespace gehört, in dem der Befehl **my** ursprünglich enthalten ist (**Hinweis:** die Verknüpfung bleibt bestehen, wenn der Befehl umbenannt wird); standardmäßig ist das das gerade aufgerufene Objekt. Entsprechend ist das Objekt, auf dem die Methode über myclass aufgerufen wird, das Objekt, das die aktuelle Klasse des Objekts ist, dem der Namespace gehört, in dem der Befehl myclass ursprünglich enthalten ist. Wie bei my bleibt die Verknüpfung bestehen, auch wenn der Befehl in einen anderen Namespace umbenannt wird, und verweist standardmäßig auf die erzeugende Klasse des aktuellen Objekts.

Jedes Objekt hat seine eigenen Befehle **my** und **myclass**, die in seinem Instanz-Namespace enthalten sind.

## Beispiele

Dieses Beispiel zeigt die grundlegende Verwendung von **my**, um die Methode **variables** der Klasse oo::object aufzurufen, die standardmäßig nicht öffentlich sichtbar ist:

```
oo::class create c {
    method count {} {
        my variable counter
        puts [incr counter]
    }
}

c create o
o count                   → prints "1"
o count                   → prints "2"
o count                   → prints "3"
```

Dieses Beispiel zeigt, wie **my** verwendet werden kann, um von außerhalb des Objekts (über einen trace) Callbacks zu privaten Methoden zu machen, wobei namespace code den richtigen Kontext herstellt. (Siehe den Befehl callback für die empfohlene Vorgehensweise.)

```
oo::class create HasCallback {
    method makeCallback {} {
        return [namespace code {
            my Callback
        }]
    }

    method Callback {args} {
        puts "callback: $args"
    }
}

set o [HasCallback new]
trace add variable xyz write [$o makeCallback]
set xyz "called"     → prints "callback: xyz {} write"
```

Dieses Beispiel zeigt den Zugriff auf eine private Methode einer Klasse von einer Instanz dieser Klasse aus. (Siehe die Deklaration classmethod in oo::define für eine höhere Schnittstelle dafür.)

```
oo::class create CountedSteps {
    self {
        variable count
        method Count {} {
            return [incr count]
        }
    }
    method advanceTwice {} {
        puts "in [self] step A: [myclass Count]"
        puts "in [self] step B: [myclass Count]"
    }
}

CountedSteps create x
CountedSteps create y
x advanceTwice      → prints "in ::x step A: 1"
                    → prints "in ::x step B: 2"
y advanceTwice      → prints "in ::y step A: 3"
                    → prints "in ::y step B: 4"
x advanceTwice      → prints "in ::x step A: 5"
                    → prints "in ::x step B: 6"
y advanceTwice      → prints "in ::y step A: 7"
                    → prints "in ::y step B: 8"
```

*Copyright (c) 2007 Donal K. Fellows*

# namespace

*Stichwörter:* namespace, command, ensemble, exported, internal, variable

## Name

namespace - Erstellt und manipuliert Kontexte für Befehle und Variablen

## Übersicht

**namespace** *subcommand* ?*arg* ...?

## Beschreibung

Der Befehl **namespace** ermöglicht das Erstellen, den Zugriff auf und das Löschen getrennter

Kontexte für Befehle und Variablen. Eine kurze Einführung zu Namespaces finden Sie im Abschnitt Was ist ein Namespace? unten. Die zulässigen Werte von *subcommand* sind unten aufgeführt. Beachten Sie, dass Sie die *subcommand*s abkürzen können.

**namespace children ?*namespace*? ?*pattern*?**

Gibt eine Liste aller Kind-Namespaces zurück, die zum Namespace *namespace* gehören. Ist *namespace* nicht angegeben, werden die Kinder des aktuellen Namespace zurückgegeben. Dieser Befehl liefert vollqualifizierte Namen, die mit einem doppelten Doppelpunkt (**::**) beginnen. Wird das optionale *pattern* angegeben, werden nur Namen zurückgegeben, die dem Glob-Muster entsprechen. Das tatsächlich verwendete Muster wird wie folgt bestimmt: Ein Muster, das mit einem doppelten Doppelpunkt (**::**) beginnt, wird direkt verwendet, andernfalls wird der Namespace *namespace* (oder der vollqualifizierte Name des aktuellen Namespace) dem Muster vorangestellt.

**namespace code *script***

Erfasst den aktuellen Namespace-Kontext für die spätere Ausführung des Skripts *script*. Er gibt ein neues Skript zurück, in dem *script* in einen Befehl **namespace inscope** eingebettet wurde. Das neue Skript hat zwei wichtige Eigenschaften. Erstens kann es in jedem Namespace ausgewertet werden und bewirkt, dass *script* im aktuellen Namespace (dem, in dem **namespace code** aufgerufen wurde) ausgewertet wird. Zweitens können dem resultierenden Skript weitere Argumente angehängt werden; diese werden *script* als zusätzliche Argumente übergeben. Angenommen, der Befehl **set script [namespace code {foo bar}]** wird im Namespace **::a::b** aufgerufen. Dann kann **eval $script [list x y]** in jedem Namespace ausgeführt werden (vorausgesetzt, der Wert von **script** wurde korrekt übergeben) und hat dieselbe Wirkung wie **::namespace eval ::a::b {foo bar x y}**. Dieser Befehl ist nötig, weil Erweiterungen wie Tk Callback-Skripte normalerweise im globalen Namespace ausführen. Ein Befehl mit Gültigkeitsbereich erfasst einen Befehl zusammen mit seinem Namespace-Kontext so, dass er später korrekt ausgeführt werden kann. Siehe den Abschnitt Gültigkeitsbereiche für Skripte für Beispiele zur Erstellung von Callback-Skripten.

**namespace current**

Gibt den vollqualifizierten Namen des aktuellen Namespace zurück. Der tatsächliche Name des globalen Namespace ist **{}** (d. h. eine leere Zeichenkette), dieser Befehl gibt jedoch aus Gründen der Programmiererfreundlichkeit **::** für den globalen Namespace zurück.

**namespace delete ?*namespace namespace* ...?**

Jeder Namespace *namespace* wird gelöscht; alle Variablen, Prozeduren und Kind-Namespaces in diesem Namespace werden ebenfalls gelöscht. Führt gerade eine Prozedur innerhalb des Namespace aus, bleibt der Namespace bis zur Rückkehr der Prozedur erhalten; er wird jedoch markiert, damit anderer Code ihn nicht mehr per Namen findet. Existiert ein Namespace nicht, liefert dieser Befehl einen Fehler. Werden keine Namespaces angegeben, tut der Befehl nichts.

**namespace ensemble *subcommand* ?*arg* ...?**

Erstellt und manipuliert einen Befehl, der aus einem Ensemble von Unterbefehlen gebildet ist. Siehe den Abschnitt Ensembles unten.

**namespace eval *namespace arg* ?*arg* ...?**

Aktiviert einen Namespace namens *namespace* und wertet Code in diesem Kontext aus. Existiert der Namespace noch nicht, wird er erstellt. Sind mehrere *arg*-Argumente angegeben, werden die Argumente wie beim Befehl eval mit Leerzeichen dazwischen verkettet und das

Ergebnis ausgewertet.

Hat *namespace* führende Namespace-Qualifizierer und existieren führende Namespaces nicht, werden sie automatisch erstellt.

**namespace exists *namespace***

Gibt **1** zurück, wenn *namespace* im aktuellen Kontext ein gültiger Namespace ist, sonst **0**.

**namespace export ?-clear? ?*pattern pattern* ...?**

Legt fest, welche Befehle aus einem Namespace exportiert werden. Exportierte Befehle können später mit **namespace import** in einen anderen Namespace importiert werden. Sowohl im Namespace definierte Befehle als auch zuvor importierte Befehle können exportiert werden. Die Befehle müssen zum Zeitpunkt von **namespace export** noch nicht definiert sein. Jedes *pattern* darf Glob-Sonderzeichen enthalten, aber keine Namespace-Qualifizierer — das Muster kann nur Befehle im aktuellen (exportierenden) Namespace angeben. Jedes *pattern* wird an die Exportmusterliste des Namespace angehängt. Mit der Option **-clear** wird die Exportmusterliste vor dem Anhängen der *pattern*-Argumente geleert. Ohne *pattern* und ohne **-clear** gibt der Befehl die aktuelle Exportliste des Namespace zurück.

**namespace forget ?*pattern pattern* ...?**

Entfernt zuvor importierte Befehle aus einem Namespace. Jedes *pattern* ist ein einfacher oder qualifizierter Name wie **x**, **foo::x** oder **a::b::p***. Qualifizierte Namen enthalten doppelte Doppelpunkte (**::**) und qualifizieren einen Namen mit einem oder mehreren Namespaces. Jedes "qualifizierte Muster" ist mit dem Namen eines exportierenden Namespace qualifiziert und darf am Ende des qualifizierten Namens Glob-Sonderzeichen im Befehlsnamen enthalten. Glob-Zeichen dürfen nicht in einem Namespacenamen vorkommen. Für jedes "einfache Muster" löscht der Befehl passende Befehle des aktuellen Namespace, die aus einem anderen Namespace importiert wurden. Bei "qualifizierten Mustern" findet der Befehl zuerst die passenden exportierten Befehle und prüft, ob der aktuelle Namespace sie zuvor importiert hat; falls ja, werden die entsprechenden importierten Befehle gelöscht. Damit macht er die Wirkung von **namespace import** rückgängig.

**namespace import ?-force? ?*pattern pattern* ...?**

Importiert Befehle in einen Namespace oder liefert die Menge importierter Befehle. Ohne Argumente gibt **namespace import** die Liste der Befehle im aktuellen Namespace zurück, die aus anderen Namespaces importiert wurden. Die Befehle in der Liste haben einfache Namen ohne Namespace-Qualifizierer — geeignet zur Verwendung mit **namespace forget** (siehe Beispiele unten).

Mit *pattern*-Argumenten ist jedes *pattern* ein qualifizierter Name wie **foo::x** oder **a::p*** — Name des exportierenden Namespace plus optional Glob-Sonderzeichen am Ende des Befehlsnamens. Glob-Zeichen dürfen nicht im Namespacenamen stehen. Ist der Namespacename nicht vollqualifiziert (beginnt nicht mit einem Namespace-Trennzeichen), wird er wie im Abschnitt Namensauflösung beschrieben aufgelöst; existiert kein passender Namespace, tritt ein Fehler auf.

Alle Befehle, die einem *pattern* entsprechen und aus ihrem Namespace exportiert sind, werden dem aktuellen Namespace hinzugefügt: ein neuer Befehl im aktuellen Namespace verweist auf den exportierten Befehl im Ursprungs-Namespace; beim Aufruf des importierten

Befehls wird der exportierte Befehl ausgeführt. Normalerweise liefert der Befehl einen Fehler, wenn ein importierter Befehl mit einem bestehenden kollidiert. Mit **-force** ersetzen importierte Befehle bestehende stillschweigend. **namespace import** hat Snapshot-Semantik: importiert werden nur die zum Ausführungszeitpunkt im exportierenden Namespace definierten, angeforderten Befehle. Später definierte und exportierte Befehle werden nicht importiert.

**namespace inscope *namespace script* ?*arg* ...?**

Führt ein Skript im Kontext des angegebenen *namespace* aus. Programmierer rufen diesen Befehl nicht direkt auf; Aufrufe entstehen implizit, wenn Anwendungen mit **namespace code** Callback-Skripte erzeugen und z. B. bei Tk-Widgets registrieren. **namespace inscope** ähnelt **namespace eval**, aber *namespace* muss bereits existieren, und **namespace inscope** hängt zusätzliche *arg*s als korrekte Listenelemente an.

```
namespace inscope ::foo $script $x $y $z
```

entspricht

```
namespace eval ::foo [concat $script [list $x $y $z]]
```

zusätzliche Argumente durchlaufen also keine zweite Substitutionsrunde wie bei **namespace eval**.

**namespace origin *command***

Gibt den vollqualifizierten Namen des ursprünglichen Befehls zurück, auf den der importierte Befehl *command* verweist. Beim Import in einen Namespace wird dort ein neuer Befehl erzeugt, der auf den Befehl im exportierenden Namespace zeigt. Wurde ein Befehl in eine Kette von Namespaces *a, b, ..., n* importiert (jeder importiert nur aus dem vorherigen), liefert dieser Befehl den vollqualifizierten Namen des ursprünglichen Befehls im ersten Namespace *a* . Verweist *command* nicht auf einen importierten Befehl, wird der eigene vollqualifizierte Name des Befehls zurückgegeben.

**namespace parent ?*namespace*?**

Gibt den vollqualifizierten Namen des übergeordneten Namespace von *namespace* zurück. Ohne *namespace* der des übergeordneten Namespace des aktuellen Namespace.

**namespace path ?*namespaceList*?**

Gibt den Befehlsauflösungspfad des aktuellen Namespace zurück. Wird *namespaceList* als Liste benannter Namespaces angegeben, wird der Pfad auf diese Namespaces gesetzt und die leere Liste zurückgegeben. Standardmäßig ist der Pfad immer leer. Siehe Namensauflösung für die Auflösungsregeln.

**namespace qualifiers *string***

Gibt führende Namespace-Qualifizierer von *string* zurück. Qualifizierer sind durch **::** getrennte Namespaces. Für **::foo::bar::x** liefert der Befehl **::foo::bar**, für **::** eine leere Zeichenkette. Ergänzung zu **namespace tail**. Es wird nicht geprüft, ob die Namespaces existieren.

**namespace tail *string***

Gibt den einfachen Namen am Ende einer qualifizierten Zeichenkette zurück. Für **::foo::bar::x** liefert der Befehl **x**, für **::** eine leere Zeichenkette. Ergänzung zu **namespace qualifiers**. Es wird nicht geprüft, ob die Namespaces existieren.

**namespace upvar *namespace* ?*otherVar myVar* ...?**

Verknüpft null oder mehr lokale Variablen der aktuellen Prozedur mit Variablen in *namespace*. Der Namespacename wird wie in Namensauflösung beschrieben aufgelöst. **namespace upvar $ns a b** verhält sich wie **upvar 0 ${ns}::a b**, außer bei den Auflösungsregeln für qualifizierte Namespace- oder Variablennamen. **namespace upvar** gibt eine leere Zeichenkette zurück.

**namespace unknown ?*script*?**

Setzt oder liefert den Unknown-Befehlshandler des aktuellen Namespace. Der Handler wird aufgerufen, wenn ein aus dem Namespace aufgerufener Befehl weder im aktuellen Namespace, noch auf dessen Pfad noch im globalen Namespace gefunden wird. Ist *script* angegeben, soll es eine wohlgeformte Liste mit Befehlsname und optionalen Argumenten sein. Beim Aufruf wird die vollständige Aufrufzeile angehängt und im Namespace-Kontext ausgewertet. Standard für alle Namespaces ist **::unknown**. Ohne Argument liefert der Befehl den Handler des aktuellen Namespace.

**namespace which ?-command? ?-variable? *name***

Sucht *name* als Befehl oder Variable und gibt den vollqualifizierten Namen zurück. Existiert *name* nicht im aktuellen, aber im globalen Namespace, liefert der Befehl einen vollqualifizierten Namen im globalen Namespace. Existiert Befehl oder Variable nicht, wird eine leere Zeichenkette zurückgegeben. Wurde die Variable erzeugt, aber nicht definiert (z. B. mit variable oder per trace), liefert der Befehl den vollqualifizierten Variablennamen. Ohne Flag wird *name* als Befehlsname behandelt. Siehe Namensauflösung.

## Was ist ein Namespace?

Ein Namespace ist eine Sammlung von Befehlen und Variablen. Er kapselt Befehle und Variablen, damit sie nicht mit denen anderer Namespaces interferieren. Tcl hatte stets eine solche Sammlung — den *globalen Namespace*. Der globale Namespace enthält alle globalen Variablen und Befehle. Mit **namespace eval** legen Sie neue Namespaces an. Beispiel:

```
namespace eval Counter {
    namespace export bump
    variable num 0

    proc bump {} {
        variable num
        incr num
    }
}
```

erzeugt einen neuen Namespace mit der Variable **num** und der Prozedur **bump**. Befehle und Variablen in diesem Namespace sind von anderen im selben Programm getrennt. Gibt es im globalen Namespace einen Befehl **bump**, unterscheidet er sich vom **bump** im Namespace **Counter**.

Namespace-Variablen ähneln globalen Variablen in Tcl. Sie existieren außerhalb der Prozeduren im Namespace, sind in Prozeduren aber über variable erreichbar (siehe oben).

Namespaces sind dynamisch. Befehle und Variablen können jederzeit hinzugefügt oder entfernt werden; der Inhalt kann mit mehreren **namespace eval**-Aufrufen aufgebaut werden. Die folgende Folge hat dieselbe Wirkung wie die Namespace-Definition oben:

```
namespace eval Counter {
    variable num 0
    proc bump {} {
        variable num
        return [incr num]
    }
}
namespace eval Counter {
    proc test {args} {
        return $args
    }
}
namespace eval Counter {
     rename test ""
}
```

Die Prozedur **test** wird dem Namespace **Counter** hinzugefügt und später per rename entfernt.

Namespaces können weitere Namespaces enthalten und verschachteln sich hierarchisch. Ein verschachtelter Namespace liegt im übergeordneten Namespace und interferiert nicht mit anderen.

## Qualifizierte Namen

Jeder Namespace hat einen Textnamen wie history oder **::safe::interp**. Da Namespaces verschachtelt sein können, werden qualifizierte Namen verwendet, um Befehle, Variablen und Kind-Namespaces zu bezeichnen. Qualifizierte Namen ähneln hierarchischen Pfaden bei Unix-Dateien oder Tk-Widgets, nur ist **::** das Trennzeichen statt **/** oder**.**. Der oberste bzw. globale Namespace heißt **{}** (leere Zeichenkette); **::** ist ein Synonym. Beispiel: **::safe::interp::create** bezeichnet den Befehl **create** im Namespace interp, Kind von **::safe**, Kind des globalen Namespace **::**.

Um Befehle und Variablen aus einem anderen Namespace zu nutzen, ist zusätzliche Syntax nötig. Namen müssen durch den enthaltenden Namespace qualifiziert sein. Aus dem globalen Namespace könnte man die **Counter**-Prozeduren so aufrufen:

```
Counter::bump 5
Counter::Reset
```

Den aktuellen Zählerstand liest man so:

```
puts "count = $Counter::num"
```

Enthält ein Namespace einen anderen, können mehrere Qualifizierer nötig sein. Enthält **Foo** den Namespace **Counter**, lässt sich **bump** aus dem globalen Namespace so aufrufen:

```
Foo::Counter::bump 3
```

Qualifizierte Namen gelten auch beim Erzeugen und Umbenennen von Befehlen:

```
proc Foo::Test {args} {return $args}
```

Verschieben in einen anderen Namespace:

```
rename Foo::Test Bar::Test
```

Weitere Punkte zu qualifizierten Namen: Namespaces haben nichtleere Namen, außer dem globalen. **::** ist in einfachen Befehls-, Variablen- und Namespacenamen außer als Trennzeichen unzulässig. Zusätzliche Doppelpunkte in einem Trennteil werden ignoriert; zwei oder mehr Doppelpunkte gelten als ein Trennzeichen. Ein abschließendes **::** in einem qualifizierten Variablen- oder Befehlsnamen bezeichnet die Variable bzw. den Befehl mit dem Namen {}. Ein abschließendes **::** in einem qualifizierten Namespacenamen wird ignoriert.

## Namensauflösung

Im Allgemeinen unterstützen Tcl-Befehle, die Variablen- und Befehlsnamen erwarten, qualifizierte Namen — z. B. set, proc, rename, interp alias. Beginnt ein vollqualifizierter Name mit **::**, ist der Bezug eindeutig. Beginnt der Name nicht mit **::** (ist *relativ*), gelten folgende Regeln:

- **Variablennamen** werden stets im aktuellen Namespace aufgelöst. Ohne spezielle Resolver bezeichnet foo::bar::baz eine Variable "baz" in einem Namespace "bar", Kind von "foo", Kind des aktuellen Namespace des Interpreters.
- **Befehlsnamen** werden zuerst im aktuellen Namespace gesucht, dann in jedem Namespace auf dem Befehlsauflösungspfad des aktuellen Namespace (standardmäßig leer), dann im globalen Namespace (oder der passende **namespace unknown**-Handler).
- **Namespacenamen** werden nur im aktuellen Namespace gesucht.

Im folgenden Beispiel

```
set traceLevel 0
namespace eval Debug {
    printTrace $traceLevel
}
```

sucht Tcl **traceLevel** im Namespace **Debug**. **printTrace** wird gleichermaßen aufgelöst. Wird ein Name nicht gefunden, ist er undefiniert. Zur Verdeutlichung:

```
set traceLevel 0
namespace eval Foo {
    variable traceLevel 3

    namespace eval Debug {
        printTrace $traceLevel
    }
}
```

sucht Tcl **traceLevel** im Namespace **Foo::Debug**. **Foo::traceLevel** und **Foo::Debug::traceLevel** werden bei der Auflösung ignoriert.

Mit **namespace which** klären Sie Auflösungsfragen. Beispiel:

```
namespace eval Foo::Debug {namespace which -variable traceLevel}
```

liefert die leere Zeichenkette. Ebenso

```
namespace eval Foo {namespace which -variable traceLevel}
```

liefert ebenfalls die leere Zeichenkette.

Namespacenamen und Variablen werden anders als Befehlsnamen aufgelöst — stets im aktuellen Namespace. Ein **namespace eval**, der einen neuen Namespace erzeugt, legt daher standardmäßig ein Kind des aktuellen Namespace an, sofern der neue Name nicht mit **::** beginnt.

Tcl hat keine Zugriffskontrolle: Mit einem qualifizierten Namen, der nach obigen Regeln aufgelöst wird, ist das Element erreichbar.

Eine Namespace-Variable in einer Prozedur desselben Namespace erreicht man mit variable — analog global, mit lokaler Verknüpfung. Bei Bedarf wird die Variable im aktuellen Namespace angelegt und initialisiert. global verknüpft nur mit dem globalen Namespace. variable ist entbehrlich, wenn Sie stets den qualifizierten Namen verwenden.

## Import von Befehlen

Namespaces repräsentieren oft Bibliotheken. Häufig genutzte Befehle mit qualifiziertem Namen zu tippen ist lästig. Enthält ein Paket wie BLT alle Befehle im Namespace **Blt**, nutzt man sie so:

```
Blt::graph .g -background red
Blt::table . .g 0,0
```

Nutzen Sie **graph** und **table** oft, können Sie sie ohne Präfix **Blt::** importieren:

```
namespace import Blt::*
```

Damit landen alle exportierten Befehle aus **Blt** im aktuellen Namespace-Kontext:

```
graph .g -background red
table . .g 0,0
```

**namespace import** importiert nur Befehle, die der Namespace mit **namespace export** exportiert hat.

*Alle* Befehle eines Namespace zu importieren ist meist schlecht, weil unklar ist, was ankommt. Besser nur benötigte Befehle:

```
namespace import Blt::graph Blt::table
```

importiert nur **graph** und **table**.

Existiert der Befehl bereits, gibt es einen Fehler — Schutz vor doppeltem Import aus verschiedenen Paketen. Beim Debuggen kann **namespace import** erneut ausgeführt werden, um neue Befehle zu holen; mit **-force** werden bestehende Befehle still überschrieben:

```
namespace import -force Blt::graph Blt::table
```

Importierte Befehle entfernt **namespace forget**:

```
namespace forget Blt::*
```

Sucht im aktuellen Namespace importierte Befehle aus **Blt** und entfernt sie; sonst keine Wirkung. Danach sind **Blt**-Befehle wieder nur mit **Blt::** erreichbar.

Löschen Sie einen Befehl im exportierenden Namespace:

```
rename Blt::graph ""
```

wird er automatisch aus allen Namespaces entfernt, die ihn importiert haben.

## Export von Befehlen

Befehle exportieren Sie so:

```
namespace eval Counter {
    namespace export bump reset
    variable Num 0
    variable Max 100

    proc bump {{by 1}} {
        variable Num
        incr Num $by
        Check
        return $Num
    }
    proc reset {} {
        variable Num
        set Num 0
    }
    proc Check {} {
        variable Num
        variable Max
        if {$Num > $Max} {
            error "too high!"
        }
    }
}
```

**bump** und **reset** sind exportiert und werden beim Import aus **Counter** einbezogen:

```
namespace import Counter::*
```

**Check** ist nicht exportiert und wird beim Import ignoriert.

**namespace import** importiert nur als exportiert deklarierte Befehle. **namespace export** legt fest, was andere Namespaces importieren dürfen. Ist ein in **namespace import** genannter Befehl nicht exportiert, wird er nicht importiert.

## Gültigkeitsbereiche für Skripte

**namespace code** verpackt ein Skript zur Auswertung in einem anderen Namespace als dem der Erstellung — häufig für Event-Handler, Tk-Bindings und Traces im globalen Kontext. Beispiel: trace-Callback im aktuellen Namespace:

```
namespace eval a {
    variable b
    proc theTraceCallback { n1 n2 op } {
        upvar 1 $n1 var
        puts "the value of $n1 has changed to $var"
        return
    }
    trace add variable b write [namespace code theTraceCallback]
}
set a::b c
```

Ausgabe bei Ausführung:

```
the value of a::b has changed to c
```

## Ensembles

**namespace ensemble** erstellt und manipuliert Ensemble-Befehle — Befehle aus gruppierten Unterbefehlen. Die Befehle stammen typischerweise aus dem aktuellen Namespace bei der Erstellung (konfigurierbar). Pro Namespace können beliebig viele Ensembles existieren (standardmäßig keines); alle Ensembles eines Namespace werden mit diesem gelöscht. Die Verbindung zum Namespace bleibt beim Umbenennen des Ensembles erhalten.

Drei Unterbefehle von **namespace ensemble** sind definiert:

**namespace ensemble create ?*option value* ...?**

Erstellt einen neuen Ensemble-Befehl, verknüpft mit dem aktuellen Namespace, und gibt dessen vollqualifizierten Namen zurück. Die Argumente konfigurieren den Befehl wie bei **namespace ensemble configure**. Ohne **-command** hat das Ensemble denselben Namen wie der verknüpfte Namespace. Siehe Ensemble-Optionen für alle Optionen.

**namespace ensemble configure *command* ?*option*? ?*value* ...?**

Liest eine Option des Ensemble-Befehls *command* oder setzt Optionen. Siehe Ensemble-Optionen.

**namespace ensemble exists *command***

Gibt einen booleschen Wert zurück, ob *command* existiert und ein Ensemble-Befehl ist. Fehler nur bei falscher Argumentanzahl.

Beim Aufruf nimmt ein Ensemble-Befehl das erste Argument und ermittelt (nach den Regeln unten) eine Wortliste, die Ensemble-Befehl und Unterbefehl ersetzt. Die Wortliste wird ohne weitere Substitution ausgewertet (als ob so eingegeben — über Tcl_EvalObjv), und das Ergebnis zurückgegeben. Das Ziel einer Ensemble-Umschreibung darf ein anderes (oder dasselbe) Ensemble sein. Der Ensemble-Befehl ist über uplevel oder info level nicht sichtbar.

### Ensemble-Optionen

Die Optionen von **namespace ensemble create** und **namespace ensemble configure** steuern das Verhalten:

**-map**

Ist die Option nicht leer, liefert sie ein Dictionary, das Unterbefehlsnamen auf eine Präfix-Wortliste abbildet, die Ensemble-Befehl und Unterbefehl ersetzt (ähnlich interp alias; keine erneute Auswertung nach Substitution). Ist das erste Zielwort beim Setzen nicht vollqualifiziert, gilt es relativ zum *aktuellen* Namespace und wird beim Lesen vollqualifiziert. Ist die Option leer, mappt der lokale Unterbefehlsname auf den vollqualifizierten Namen. Ist **-map** nicht leer und **-subcommands** leer, sind die Unterbefehlsnamen genau die Schlüssel des Dictionary.

**-parameters**

Liste benannter Argumente (Namen für Fehlermeldungen), die der Aufrufer zwischen Ensemble-Namen und Unterbefehl übergibt. Standard: leere Liste.

**-prefixes**

Standardmäßig aktiv: erkennt eindeutige Präfixe der Unterbefehle. Ausgeschaltet: exakte Übereinstimmung der Unterbefehlsnamen.

**-subcommands**

Ist die Option nicht leer, listet sie exakt die Unterbefehle des Ensembles. Die Abbildung kommt aus **-map** oder dem gleichnamigen Befehl im verknüpften Namespace. Ist die Option leer, sind die Unterbefehle die Schlüssel von **-map** oder die zum Aufrufzeitpunkt exportierten Befehle des verknüpften Namespace.

**-unknown**

Ist die Option nicht leer, liefert sie einen partiellen Befehl (an den alle Argumente des Ensemble-Aufrufs inkl. vollqualifiziertem Ensemble-Namen angehängt werden), wenn ein Unterbefehl nicht erkannt wird und sonst ein Fehler entstünde. Ist die Option leer (Standard), entsteht ein Fehler im Stil von Tcl_GetIndexFromObj, wenn das Ensemble den Unterbefehl nicht zuordnen kann. Siehe Verhalten des Unknown-Handlers.

Zusätzliche Option nur bei **namespace ensemble create**:

**-command**

Schreibgeschützte Option: Name des Ensembles beliebig in einem bestehenden Namespace. Standard: vollqualifizierter Name des Namespace, in dem **namespace ensemble create** aufgerufen wurde.

Zusätzliche Option nur bei **namespace ensemble configure**:

**-namespace**

Leseoption: vollqualifizierter Name des Namespace, in dem das Ensemble erzeugt wurde.

## Verhalten des Unknown-Handlers

Ist ein Unknown-Handler für ein Ensemble gesetzt, wird er aufgerufen, wenn der Ensemble-Befehl sonst wegen unklarer Unterbefehlswahl einen Fehler zurückgeben würde. Die genauen Bedingungen steuern **-subcommands**, **-map** und **-prefixes** (siehe oben).

Zur Ausführung nimmt der Ensemble-Mechanismus die **-unknown**-Option und hängt jedes Argument des fehlgeschlagenen Ensemble-Aufrufs an (inkl. des Ensemble-Befehls als vollqualifiziertem Namen) und wertet den resultierenden Befehl im Gültigkeitsbereich des Versuchs aus. Endet der Handler normal, parst die Ensemble-Engine den Unterbefehl erneut und dispatcht noch einmal — ideal, wenn der Handler die Ensemble-Konfiguration aktualisiert hat. Jede andere Beendigung gilt als Fehler.

Das Ergebnis des Unknown-Handlers muss eine Liste sein (sonst Fehler). Ist die Liste leer, sucht der Ensemble-Befehl den ursprünglichen Unterbefehl erneut; wird er dann nicht gefunden, entsteht ein Fehler wie ohne **-unknown**-Handler (der Unknown-Handler wird pro Ensemble-Aufruf höchstens einmal aufgerufen). So kann der Handler Ensemble oder Namespace aktualisieren und erneut parsen lassen.

Ist die Liste nicht leer, ersetzen ihre Wörter Ensemble-Befehl und Unterbefehl wie bei einem Treffer in **-map**. Der Unknown-Handler muss alle Namespace-Qualifizierer liefern, wenn der implementierende Unterbefehl nicht im Namespace des Ensemble-Aufrufers liegt. Bei verketteten Ensembles (z. B. ein Unterbefehl-Implementierer ist selbst ein Ensemble, analog zu **text**-Tags und -Marken) erfolgt die Umschreibung im Kontext des Aufrufers des äußersten Ensembles — Ensembles legen selbst keinen Namespace-Kontext auf den Tcl-Aufrufstack.

Ohne **-unknown**-Handler (Standard) erzeugt der Ensemble-Befehl eine Fehlermeldung aus der Liste definierter Befehle (Format wie Tcl_GetIndexFromObj). Das ist der Fehler, wenn der Unterbefehl beim erneuten Parsen noch unbekannt ist. Es ist auch ein Fehler, wenn ein **-unknown**-Handler seinen eigenen Namespace löscht.

## Beispiele

Namespace mit Variable und exportiertem Befehl anlegen:

```
namespace eval foo {
    variable bar 0
    proc grill {} {
        variable bar
        puts "called [incr bar] times"
    }
    namespace export grill
}
```

Den Befehl aus dem vorherigen Beispiel auf verschiedene Weise aufrufen:

```
# Direct call
::foo::grill

# Use the command resolution path to find the name
namespace eval boo {
    namespace path ::foo
    grill
}

# Import into current namespace, then call local alias
namespace import foo::grill
grill

# Create two ensembles, one with the default name and one with a
# specified name.  Then call through the ensembles.
namespace eval foo {
    namespace ensemble create
    namespace ensemble create -command ::foobar
}
foo grill
foobar grill
```

Herkunft des importierten Befehls ermitteln:

```
puts "grill came from [namespace origin grill]"
```

Alle importierten Befehle aus dem aktuellen Namespace entfernen:

```
namespace forget {*}[namespace import]
```

Erstellt ein Ensemble für einfache Arbeit mit Zahlen und nutzt die Option **-parameters**, damit der Operator zwischen dem ersten und dem zweiten Argument stehen kann.

```
namespace eval do {
    namespace export *
    namespace ensemble create -parameters x
    proc plus  {x y} {expr { $x + $y }}
    proc minus {x y} {expr { $x - $y }}
}

# Verwendung des Ensembles:
puts [do 1 plus [do 9 minus 7]]
```

*Copyright (c) 1993-1997 Bell Labs Innovations for Lucent Technologies*
*Copyright (c) 1997 Sun Microsystems, Inc.*
*Copyright (c) 2000 Scriptics Corporation.*
*Copyright (c) 2004-2005 Donal K. Fellows.*

# next

*Stichwörter:* next, call, method, method chain

## Name

next, nextto - Implementierungen von Oberklassenmethoden aufrufen

## Übersicht

**package require tcl::oo**

**next** ?*arg* ...?
**nextto** *class* ?*arg* ...?

## Beschreibung

Der Befehl **next** wird verwendet, um Implementierungen einer Methode durch eine Klasse, Oberklasse oder Mixin aufzurufen, die von der aktuellen Methode überschrieben wurden. Er kann nur innerhalb einer Methode verwendet werden. Er wird auch innerhalb von Filtern verwendet, um die Stelle anzugeben, an der ein Filter die eigentliche Implementierung aufruft (der Filter kann entscheiden, die Kette nicht fortzusetzen, und kann die Ergebnisse der Methodenkette nach Belieben verarbeiten). Das Ergebnis des Befehls **next** ist das Ergebnis der nächsten Methode in der Methodenkette; gibt es keine weiteren Methoden in der Methodenkette, ist das Ergebnis von **next** ein Fehler. Die Argumente *arg* von **next** sind die Argumente, die an die nächste Methode in der Kette übergeben werden.

Der Befehl **nextto** entspricht dem Befehl **next**, nimmt jedoch ein zusätzliches Argument *class* entgegen, das eine Klasse identifiziert, deren Implementierung der aktuellen Methodenkette (siehe info object **call**) verwendet werden soll; die ausgewählte Methodenimplementierung ist die von der angegebenen Klasse bereitgestellte, und sie muss sich auf einen existierenden Nicht-Filter-Aufruf beziehen, der weiter entlang der Kette liegt als die aktuelle Implementierung.

## Die Methodenkette

Wenn eine Methode eines Objekts aufgerufen wird, geschieht dies in mehreren Schritten:

1. Die Struktur des Objekts, seiner Klasse, Oberklassen, Filter und Mixins wird untersucht, um eine *Methodenkette* aufzubauen, die eine Liste von aufzurufenden Methodenimplementierungen enthält.
2. Die erste Methodenimplementierung in der Kette wird aufgerufen.
3. Ruft diese Methodenimplementierung den Befehl **next** auf, wird die nächste Methodenimplementierung aufgerufen (mit den Argumenten, die an **next** übergeben wurden).
4. Das Ergebnis des gesamten Methodenaufrufs ist das Ergebnis der äußersten Methodenimplementierung; innere Methodenimplementierungen geben ihre Ergebnisse über **next** zurück.
5. Die Methodenkette wird für die spätere Verwendung zwischengespeichert.

### Suchreihenfolge für Methoden

Beim Aufbau der Methodenkette werden Methodenimplementierungen in folgender Reihenfolge gesucht:

1. In die Objekte eingemischten Klassen, in Traversierungsreihenfolge der Klassen. Die Liste der Mixins wird in natürlicher Reihenfolge geprüft.
2. In die Klassen des Objekts eingemischten Klassen, wobei Quellen des Einschmischens in Traversierungsreihenfolge der Klassen gesucht werden. Innerhalb jeder Klasse wird die Liste der Mixins in natürlicher Reihenfolge verarbeitet.
3. Im Objekt selbst.
4. In der Klasse des Objekts.
5. In den Oberklassen der Klasse, wobei jede Oberklasse in Tiefensuche in der natürlichen Reihenfolge der Oberklassenliste durchlaufen wird.

Jede bestimmte Methodenimplementierung steht in der resultierenden Implementierungsliste stets so *spät* wie möglich; das bedeutet, wenn eine Klasse A sowohl in eine Klasse B eingemischt ist als auch Oberklasse von B ist, behandeln Instanzen von B A stets als Oberklasse aus Sicht der Vererbung. Dies gilt auch, wenn die Mehrfachvererbung indirekt verarbeitet wird.

### Filter

Hat ein Objekt eine Liste von Filternamen oder ist es Instanz einer Klasse (oder hat eine Klasse eingemischt), die eine Liste von Filternamen hat, werden vor jedem Aufruf einer Methode die Filter verarbeitet. Filterimplementierungen werden in Traversierungsreihenfolge der Klassen gefunden, ebenso die Listen von Filternamen (jede wird in natürlicher Listenreihenfolge durchlaufen). Der explizite Aufruf einer als Filter verwendeten Methode führt dazu, dass diese Methode zweimal aufgerufen wird — einmal als Filter und einmal als normale Methode.

Jeder Filter sollte selbst entscheiden, ob die Ausführung zur eigentlichen Implementierung der Methode fortgesetzt werden darf (was durch Aufruf des Befehls **next** geschieht, da Filter an den Anfang der Methodenaufrufkette eingefügt werden), und ist dafür verantwortlich, das Ergebnis von **next** zurückzugeben.

Filter werden bei der Verarbeitung eines Aufrufs der Methode **unknown** aufgerufen, weil keine Methodenimplementierung gefunden werden konnte, aber *nicht* beim Aufruf von Konstruktoren oder Destruktoren. (Beachten Sie jedoch, dass die Methode **destroy** eine konventionelle

Methode ist und Filter bei ihrem Aufruf normal verarbeitet werden.)

## Beispiele

Dieses Beispiel zeigt, wie der Befehl **next** verwendet wird, um die Implementierung einer Methode der (Ober-)Klasse aufzurufen. Das Skript:

```
oo::class create theSuperclass {
    method example {args} {
        puts "in the superclass, args = $args"
    }
}

oo::class create theSubclass {
    superclass theSuperclass
    method example {args} {
        puts "before chaining from subclass, args = $args"
        next a {*}$args b
        next pureSynthesis
        puts "after chaining from subclass"
    }
}

theSubclass create obj
oo::objdefine obj method example args {
    puts "per-object method, args = $args"
    next x {*}$args y
    next
}
obj example 1 2 3
```

gibt Folgendes aus:

```
per-object method, args = 1 2 3
before chaining from subclass, args = x 1 2 3 y
in the superclass, args = a x 1 2 3 y b
in the superclass, args = pureSynthesis
after chaining from subclass
before chaining from subclass, args =
in the superclass, args = a b
in the superclass, args = pureSynthesis
after chaining from subclass
```

Dieses Beispiel zeigt, wie eine einfache Cache-Klasse erstellt wird, die Memoization auf alle Methodenaufrufe der Objekte anwendet, in die sie eingemischt ist, und wie sich dies auf die Berechnungszeiten auswirken kann:

```
oo::class create cache {
    filter Memoize
    method Memoize args {
        # Kernmethodenimplementierungen nicht filtern
        if {[lindex [self target] 0] eq "::oo::object"} {
            return [next {*}$args]
        }

        # Prüfen, ob der Wert bereits im Cache liegt
        my variable ValueCache
        set key [self target],$args
        if {[info exist ValueCache($key)]} {
            return $ValueCache($key)
        }

        # Wert berechnen, in Cache einfügen und zurückgeben
        return [set ValueCache($key) [next {*}$args]]
    }

    method flushCache {} {
        my variable ValueCache
        unset ValueCache
        # Caching überspringen
        return -level 2 ""
    }
}

oo::object create demo
oo::objdefine demo {
    mixin cache

    method compute {a b c} {
        after 3000 ;# Tiefes Nachdenken simulieren
        return [expr {$a + $b * $c}]
    }

    method compute2 {a b c} {
        after 3000 ;# Tiefes Nachdenken simulieren
        return [expr {$a * $b + $c}]
    }
}

puts [demo compute  1 2 3]      → prints "7" after delay
puts [demo compute2 4 5 6]      → prints "26" after delay
puts [demo compute  1 2 3]      → prints "7" instantly
puts [demo compute2 4 5 6]      → prints "26" instantly
puts [demo compute  4 5 6]      → prints "34" after delay
puts [demo compute  4 5 6]      → prints "34" instantly
puts [demo compute  1 2 3]      → prints "7" instantly
demo flushCache
puts [demo compute  1 2 3]      → prints "7" after delay
```

*Copyright (c) 2007 Donal K. Fellows*

# object

*Stichwörter:* object, base class, class, root class

## Name

oo::object - Wurzelklasse der Klassenhierarchie

## Übersicht

**package require tcl::oo**

**oo::object** *method* ?*arg* ...?

## Klassenhierarchie

```
oo::object
```

## Beschreibung

Die Klasse **oo::object** ist die Wurzelklasse der Objekthierarchie; jedes Objekt ist eine Instanz dieser Klasse. Da Klassen selbst Objekte sind, sind sie ebenfalls Instanzen dieser Klasse. Objekte werden stets über ihren Namen angesprochen und können mit rename umbenannt werden, ohne ihre Identität zu verlieren.

Instanzen von Objekten können mit den Methoden **create** oder **new** des Objekts **oo::object** selbst oder durch Aufruf dieser Methoden auf Unterklassenobjekten erzeugt werden; siehe oo::class für Details. Die Konfiguration einzelner Objekte (z. B. instanzspezifische Methoden, gemischte Klassen usw.) kann mit oo::objdefine gesteuert werden.

Jedes Objekt hat einen eindeutigen Namespace, den Instanz-Namespace. Dieser Namespace enthält alle Instanzvariablen des Objekts und ist der aktuelle Namespace, wenn eine Methode des Objekts (einschließlich einer Methode der Klasse des Objekts) aufgerufen wird. Wird das Objekt zerstört, wird sein Instanz-Namespace gelöscht. Der Instanz-Namespace enthält den my -Befehl des Objekts, mit dem nicht exportierte Methoden aufgerufen oder eine Referenz auf das Objekt für Aufrufe erzeugt werden kann, die Umbenennungen überdauert.

### Konstruktor

Die Klasse **oo::object** definiert keinen expliziten Konstruktor.

### Destruktor

Die Klasse **oo::object** definiert keinen expliziten Destruktor.

### Exportierte Methoden

Die Klasse **oo::object** unterstützt folgende exportierte Methoden:

***obj* destroy**

Diese Methode zerstört das Objekt *obj*, auf dem sie aufgerufen wird, und ruft dabei Destruktoren der Klasse des Objekts auf. Sie entspricht rename, um den Objektbefehl zu löschen. Das Ergebnis dieser Methode ist stets die leere Zeichenkette.

### Nicht exportierte Methoden

Die Klasse **oo::object** unterstützt folgende nicht exportierte Methoden:

***obj* eval ?*arg* ...?**

Diese Methode verkettet die Argumente *arg* wie mit concat und wertet das resultierende Skript im Namespace aus, der eindeutig mit *obj* verknüpft ist; der Rückgabewert der Auswertung wird zurückgegeben.

Beachten Sie, dass objektinterne Befehle wie my und self in diesem Kontext aufgerufen werden können.

***obj* unknown ?*methodName*? ?*arg* ...?**

Diese Methode wird aufgerufen, wenn der Versuch, die Methode *methodName* auf dem Objekt *obj* aufzurufen, fehlschlägt. Die vom Benutzer übergebenen Argumente stehen als *arg* zur Verfügung. Fehlt methodName, wurde das Objekt ohne Methodennamen (oder andere Argumente) aufgerufen. Die Standardimplementierung (in **oo::object**) erzeugt einen passenden Fehler mit Angabe der unterstützten Methoden, je nachdem ob das Objekt über seinen öffentlichen Namen oder über my aufgerufen wurde.

***obj* variable ?*varName* ...?**

Diese Methode verknüpft jede Variable *varName* aus dem eindeutigen Namespace des Objekts *obj* in den Kontext des Aufrufers. Wird sie innerhalb einer Prozedur aufgerufen, wird die Namespace-Variable mit der lokalen Variable der Prozedur verknüpft. Jedes *varName* darf keine Namespace-Trenner enthalten. Das Ergebnis ist die leere Zeichenkette.

***obj* varname *varName***

Diese Methode gibt den global qualifizierten Namen der Variable *varName* im eindeutigen Namespace des Objekts *obj* zurück.

***obj sourceObjectName***

Diese Methode wird vom Befehl oo::object verwendet, um den Zustand eines Objekts auf ein anderes zu kopieren. Sie kopiert die Prozeduren und Variablen des Namespace von sourceObjectName in das aktuelle Objekt. Andere Befehlstypen oder Spuren auf Variablen werden nicht kopiert; das kann durch Überschreiben dieser Methode in einer Unterklasse ergänzt werden.

## Beispiele

Dieses Beispiel zeigt die grundlegende Verwendung eines Objekts.

```
set obj [oo::object new]
$obj foo             → error "unknown method foo"
oo::objdefine $obj method foo {} {
    my variable count
    puts "bar[incr count]"
}
$obj foo             → prints "bar1"
$obj foo             → prints "bar2"
```

```
$obj variable count  → error "unknown method variable"
$obj destroy
$obj foo             → error "unknown command obj"
```

*Copyright (c) 2007-2008 Donal K. Fellows*

# open

*Stichwörter:* open, access mode, append, create, file, non-blocking, permissions, pipeline, process, serial

## Name

open - Öffnet einen dateibasierten Kanal oder einen Befehlspipeline-Kanal

## Übersicht

**open** *fileName*
**open** *fileName access*
**open** *fileName access permissions*

## Beschreibung

Dieser Befehl öffnet eine Datei, einen seriellen Port oder eine Befehlspipeline und gibt einen Kanalbezeichner zurück, der in späteren Aufrufen von Befehlen wie read, puts und close verwendet werden kann. Ist das erste Zeichen von *fileName* nicht **|**, öffnet der Befehl eine Datei: *fileName* gibt den Namen der zu öffnenden Datei an, und er muss den in der Manpage filename beschriebenen Konventionen entsprechen.

Das Argument *access*, falls angegeben, gibt an, wie auf die Datei (oder Befehlspipeline) zugegriffen werden soll. In der ersten Form kann *access* einen der folgenden Werte haben:

**r**
Die Datei nur zum Lesen öffnen; die Datei muss bereits existieren. Dies ist der Standardwert, wenn *access* nicht angegeben wird.

**r+**
Die Datei zum Lesen und Schreiben öffnen; die Datei muss bereits existieren.

**w**
Die Datei nur zum Schreiben öffnen. Existiert sie, wird sie gekürzt. Existiert sie nicht, wird eine neue Datei erstellt.

**w+**
Die Datei zum Lesen und Schreiben öffnen. Existiert sie, wird sie gekürzt. Existiert sie nicht, wird eine neue Datei erstellt.

**a**
Die Datei nur zum Schreiben öffnen. Existiert die Datei nicht, wird eine neue leere Datei erstellt. Der Dateizeiger wird vor jedem Schreibvorgang ans Dateiende gesetzt.

**a+**
Die Datei zum Lesen und Schreiben öffnen. Existiert die Datei nicht, wird eine neue leere Datei erstellt. Die anfängliche Zugriffsposition wird auf das Dateiende gesetzt.

Alle oben genannten zulässigen *access*-Werte können das Zeichen **b** als zweites oder drittes Zeichen im Wert erhalten, um anzugeben, dass der geöffnete Kanal so konfiguriert werden soll, als wäre die Option fconfigure **-translation binary** gesetzt, sodass der Kanal zum Lesen oder Schreiben binärer Daten geeignet ist.

In der zweiten Form besteht *access* aus einer Liste beliebiger der folgenden Flags, von denen die meisten die übliche POSIX-Bedeutung haben. Eines der Flags muss **RDONLY**, **WRONLY** oder **RDWR** sein.

**RDONLY**
Die Datei nur zum Lesen öffnen.

**WRONLY**
Die Datei nur zum Schreiben öffnen.

**RDWR**
Die Datei zum Lesen und Schreiben öffnen.

**APPEND**
Den Dateizeiger vor jedem Schreibvorgang ans Dateiende setzen.

**BINARY**
Den geöffneten Kanal mit der Option **-translation binary** konfigurieren.

**CREAT**
Die Datei erstellen, falls sie noch nicht existiert (ohne dieses Flag ist es ein Fehler, wenn die Datei nicht existiert).

**EXCL**
Ist **CREAT** ebenfalls angegeben, wird ein Fehler zurückgegeben, wenn die Datei bereits existiert.

**NOCTTY**
Ist die Datei ein Terminalgerät, verhindert dieses Flag, dass die Datei zum steuernden Terminal des Prozesses wird.

**NONBLOCK**
Verhindert, dass der Prozess beim Öffnen der Datei und möglicherweise bei nachfolgenden E/A-Operationen blockiert. Das genaue Verhalten dieses Flags ist system- und geräteabhängig; seine Verwendung wird nicht empfohlen (besser ist es, den Befehl fconfigure zu verwenden, um eine Datei in den nicht blockierenden Modus zu versetzen). Einzelheiten finden sich in der Systemdokumentation zum Flag **O_NONBLOCK** des Systemaufrufs **open**.

**TRUNC**
Existiert die Datei, wird sie auf die Länge null gekürzt.

Wird beim Öffnen eine neue Datei erstellt, wird *permissions* (eine Ganzzahl) zusammen mit der Dateimodus-Erstellungsmaske des Prozesses verwendet, um die Berechtigungen für die neue Datei festzulegen. *Permissions* hat standardmäßig den Wert 0666.

Wenn die geöffnete Datei eine normale Festplattendatei ist, können die Befehle chan configure und fconfigure verwendet werden, um diese zusätzliche Konfigurationsoption abzufragen:

**-stat**
Beim Lesen dieser Option wird ein Wörterbuch mit Werten zurückgegeben, wie es der Befehl file stat liefert, wobei sich die Stat-Informationen auf die tatsächlich geöffnete Datei beziehen. Schlüssel im Wörterbuch können unter anderem atime, ctime, dev, gid, ino, mode, mtime, nlink, size, type und uid sein; die Felder mtime, size und type sind auf allen Plattformen garantiert vorhanden und aussagekräftig; weitere Schlüssel können ebenfalls vorhanden sein.

*Implementierungshinweis:* Diese Option wird auf POSIX-Plattformen auf einen Aufruf von **fstat()** abgebildet und unter Windows auf einen Aufruf von **GetFileInformationByHandle()**; die gemeldeten Informationen entsprechen dem, was diese Systemaufrufe liefern.

## Befehlspipelines

Ist das erste Zeichen von *fileName* "**|**", werden die übrigen Zeichen von *fileName* als Liste von Argumenten behandelt, die eine Befehlspipeline beschreiben, im gleichen Stil wie die Argumente für exec. In diesem Fall kann der von **open** zurückgegebene Kanalbezeichner verwendet werden, um in die Eingabepipeline des Befehls zu schreiben oder aus seiner Ausgabepipeline zu lesen, abhängig vom Wert von *access*. Wird nur Schreibzugriff verwendet (z. B. *access* ist "**w**"), wird die Standardausgabe der Pipeline auf die aktuelle Standardausgabe geleitet, sofern der Befehl nichts anderes vorgibt. Wird nur Lesezugriff verwendet (z. B. *access* ist "**r**"), stammt die Standardeingabe der Pipeline von der aktuellen Standardeingabe, sofern der Befehl nichts anderes vorgibt. Die ID des gestarteten Prozesses ist über den Befehl pid zugänglich, wobei der von **open** zurückgegebene Kanalbezeichner als Argument verwendet wird.

Gibt der Befehl (oder einer der Befehle) in der Befehlspipeline einen Fehler zurück (gemäß der Definition in exec), wird beim Aufruf von close auf dem Kanal ein Tcl-Fehler erzeugt, es sei denn, die Pipeline befindet sich im nicht blockierenden Modus; dann wird kein Exit-Status zurückgegeben (ein stilles close mit -blocking 0).

Es ist oft sinnvoll, den Befehl fileevent mit Pipelines zu verwenden, damit andere Verarbeitung parallel zum Ausführen des Befehls im Hintergrund ablaufen kann.

## Serielle Kommunikation

Bezieht sich *fileName* auf einen seriellen Port, wird der angegebene serielle Port plattformabhängig geöffnet und initialisiert. Zulässige Werte für *fileName* zum Öffnen eines seriellen Ports sind im Abschnitt PORTABILITÄTSFRAGEN beschrieben.

Die Befehle chan configure und fconfigure können verwendet werden, um zusätzliche, serielle Ports betreffende Konfigurationsoptionen abzufragen und zu setzen (wo unterstützt):

**-mode *baud,parity,data,stop***

Diese Option ist ein Satz aus vier durch Kommas getrennten Werten: Baudrate, Parität, Anzahl der Datenbits und Anzahl der Stoppbits für diesen seriellen Port. Die *baud*-Rate ist eine einfache Ganzzahl, die die Verbindungsgeschwindigkeit angibt. *Parity* ist einer der Buchstaben **n**, **o**, **e**, **m**, **s**; sie stehen für die Paritätsoptionen "none", "odd", "even", "mark" bzw. "space". *Data* ist die Anzahl der Datenbits und sollte eine Ganzzahl von 5 bis 8 sein, *stop* ist die Anzahl der Stoppbits und sollte die Ganzzahl 1 oder 2 sein.

**-handshake *type***

(Windows und Unix). Diese Option dient zur Einrichtung der automatischen Handshake-Steuerung. Beachten Sie, dass nicht alle Handshake-Typen von Ihrem Betriebssystem unterstützt werden. Der Parameter *type* ist unabhängig von der Groß-/Kleinschreibung.

Ist *type* **none**, ist jeder Handshake ausgeschaltet. **rtscts** aktiviert den Hardware-Handshake. Die Hardware-Handshake-Signale sind unten beschrieben. Für den Software-Handshake **xonxoff** können die Handshake-Zeichen mit **-xchar** neu definiert werden. Ein zusätzlicher Hardware-Handshake **dtrdsr** ist nur unter Windows verfügbar. Es gibt keine

Standard-Handshake-Konfiguration; der Anfangswert hängt von den Betriebssystemeinstellungen ab. Die Option **-handshake** kann nicht abgefragt werden.

**-queue**

(Windows und Unix). Die Option **-queue** kann nur abgefragt werden. Sie gibt eine Liste aus zwei Ganzzahlen zurück, die die aktuelle Anzahl der Bytes in der Eingabe- bzw. Ausgabewarteschlange darstellen.

**-timeout *msec***

(Windows und Unix). Diese Option legt das Timeout für blockierende Leseoperationen fest. Sie gibt das maximale Intervall zwischen dem Empfang zweier Bytes in Millisekunden an. Unter Unix-Systemen beträgt die Granularität 100 Millisekunden. Die Option **-timeout** beeinflusst Schreiboperationen oder nicht blockierende Lesezugriffe nicht. Diese Option kann nicht abgefragt werden.

**-ttycontrol *{signal boolean signal boolean ...}***

(Windows und Unix). Diese Option dient zur dauerhaften Einrichtung der Handshake-Ausgangsleitungen (siehe unten) oder zum Senden eines BREAK über die serielle Leitung. Die *signal*-Namen sind unabhängig von der Groß-/Kleinschreibung. **{RTS 1 DTR 0}** setzt die RTS-Ausgabe auf high und die DTR-Ausgabe auf low. Die BREAK-Bedingung (siehe unten) wird mit **{BREAK 1}** bzw. **{BREAK 0}** aktiviert und deaktiviert. Es ist keine gute Idee, das Signal **RTS** (oder **DTR**) bei aktivem Hardware-Handshake **rtscts** (oder **dtrdsr**) zu ändern. Das Ergebnis ist unvorhersehbar. Die Option **-ttycontrol** kann nicht abgefragt werden.

**-ttystatus**

(Windows und Unix). Die Option **-ttystatus** kann nur abgefragt werden. Sie liefert den aktuellen Modemstatus und die Handshake-Eingangssignale (siehe unten). Das Ergebnis ist eine Liste von Signal-Wert-Paaren in fester Reihenfolge, z. B. **{CTS 1 DSR 0 RING 1 DCD 0}**. Die *signal*-Namen werden in Großbuchstaben zurückgegeben.

**-xchar *{xonChar xoffChar}***

(Windows und Unix). Diese Option dient zum Abfragen oder Ändern der Software-Handshake-Zeichen. Normalerweise sollten die Betriebssystem-Standardwerte DC1 (0x11) und DC3 (0x13) sein, die den ASCII-Standardzeichen XON und XOFF entsprechen.

**-closemode *closeMode***

(Windows und Unix). Diese Option dient zum Abfragen oder Ändern des Schließmodus des seriellen Kanals; sie legt fest, wie mit ausstehender Ausgabe in Betriebssystempuffern beim Schließen des Kanals verfahren wird. Folgende Werte für closeMode werden unterstützt:

**default**

bedeutet, dass eine systemseitige Standardoperation verwendet werden soll; alle seriellen Kanäle verwenden standardmäßig diesen Wert.

**discard**

bedeutet, dass der Inhalt der Betriebssystempuffer verworfen werden soll. Beachten Sie, dass dies beim Schreiben auf ein POSIX-Terminal nicht empfohlen ist, da es unerwartet mit der Behandlung von stderr interagieren kann.

**drain**

bedeutet, dass Tcl beim Schließen des Kanals wartet, bis die gesamte Ausgabe verarbeitet wurde. Das kann close spürbar verlangsamen.

**-inputmode *inputMode***
(nur Unix; Windows hat die gleichwertige Option bei Konsolenkanälen). Diese Option dient zum Abfragen oder Ändern des Eingabemodus des seriellen Kanals unter der Annahme, dass mit einem Terminal kommuniziert wird; sie steuert, wie interaktive Benutzereingabe behandelt wird. Folgende Werte für inputMode werden unterstützt:

**normal**
bedeutet, dass normale zeilenorientierte Eingabe mit den üblichen Terminal-Bearbeitungsfunktionen verwendet werden soll.

**password**
bedeutet, dass nicht echoende Eingabe mit den üblichen Terminal-Bearbeitungsfunktionen verwendet werden soll, aber ohne Ausgabe getippter Zeichen auf das Terminal (außer Zeilenumbrüchen). Einige Terminals können dies besonders kennzeichnen.

**raw**
bedeutet, dass die gesamte Tastatureingabe unverarbeitet an Tcl weitergegeben werden soll, ohne dass das Terminal etwas verarbeitet. Die Tasten werden nicht echoiert; die Interpretation obliegt dem Tcl-Skript.

**reset (nur setzen)**
bedeutet, dass das Terminal auf den Zustand zurückgesetzt werden soll, in dem es sich beim Öffnen befand.

Beachten Sie, dass das Setzen dieser Option (technisch gesehen alles, was den Terminalzustand über diese Option vom Anfangswert abweicht) bewirkt, dass der Kanal beim Schließen automatisch das Terminal zurücksetzt.

**-winsize**
(nur Unix; Windows hat die gleichwertige Option bei Konsolenkanälen). Diese Option ist nur abfragbar. Sie liefert eine zweielementige Liste mit der aktuellen Breite und Höhe des Terminals.

**-pollinterval *msec***
(nur Windows). Diese Option legt die maximale Zeit zwischen Abfragen auf Datei-Events fest. Das betrifft das Intervall zwischen Ereignisprüfungen im gesamten Tcl-Interpreter (der kleinste Wert gewinnt immer). Verwenden Sie diese Option nur, wenn Sie den seriellen Port häufiger oder seltener als alle 10 ms (Standard) abfragen möchten.

**-sysbuffer *inSize***
siehe unten ...

**-sysbuffer *{inSize outSize}***
(nur Windows). Diese Option dient zur Änderung der Größe der Windows-Systempuffer für einen seriellen Kanal. Besonders bei höheren Übertragungsraten kann der Standard-Eingabepuffer von 4096 Bytes auf trägen Systemen überlaufen. Die erste Form legt nur die Größe des Eingabepuffers fest, in der zweiten Form werden Eingabe- und Ausgabepuffer definiert.

**-lasterror**
(nur Windows). Diese Option ist nur abfragbar. Tritt bei der seriellen Kommunikation ein Fehler auf, liefern read oder puts einen allgemeinen Tcl-Datei-E/A-Fehler. fconfigure **-lasterror** kann aufgerufen werden, um eine Liste mit Fehlerdetails zu erhalten. Die verschiedenen Fehlercodes werden unten erläutert.

## Signale der seriellen Schnittstelle

RS-232 ist die am häufigsten verwendete standardisierte elektrische Schnittstelle für serielle Kommunikation. Eine negative Spannung (−3 V bis −12 V) definiert ein Mark-Bit (on=1), eine positive Spannung (+3 V bis +12 V) ein Space-Bit (off=0) (RS-232C). Für eingehende und ausgehende Daten, Statusleitungen und Handshake gelten die folgenden Signale. Hier verwenden wir die Begriffe *Workstation* für Ihren Computer und *Modem* für das externe Gerät, weil einige Signalnamen (DCD, RI) von Modems stammen. Natürlich kann Ihr externes Gerät diese Signalleitungen auch für andere Zwecke nutzen.

**TXD (output)**

**Transmitted Data (gesendete Daten):** Ausgehende serielle Daten.

**RXD (input)**

**Received Data (empfangene Daten):** Eingehende serielle Daten.

**RTS (output)**

**Request To Send:** Diese Hardware-Handshake-Leitung teilt dem Modem mit, dass Ihre Workstation bereit ist, Daten zu empfangen. Ihre Workstation kann dieses Signal automatisch zurücksetzen, um anzuzeigen, dass der Eingabepuffer voll ist.

**CTS (input)**

**Clear To Send:** Das Gegenstück zu RTS. Zeigt an, dass das Modem bereit ist, Daten zu empfangen.

**DTR (output)**

**Data Terminal Ready:** Dieses Signal teilt dem Modem mit, dass die Workstation bereit ist, eine Verbindung aufzubauen. DTR wird oft automatisch aktiviert, sobald ein serieller Port geöffnet wird.

**DSR (input)**

**Data Set Ready:** Das Gegenstück zu DTR. Teilt der Workstation mit, dass das Modem bereit ist, eine Verbindung aufzubauen.

**DCD (input)**

**Data Carrier Detect:** Diese Leitung wird aktiv, wenn ein Modem ein "Carrier"-Signal erkennt.

**RI (input)**

**Ring Indicator:** Wird aktiv, wenn das Modem einen eingehenden Anruf erkennt.

**BREAK**

Eine BREAK-Bedingung ist keine Hardware-Signalleitung, sondern logische Null auf den Leitungen TXD oder RXD über einen längeren Zeitraum, üblicherweise 250 bis 500 Millisekunden. Normalerweise bleibt ein Empfangs- oder Sende-Datensignal bis zur Übertragung des nächsten Zeichens auf Mark-Spannung (on=1). Ein BREAK wird manchmal verwendet, um die Kommunikationsleitung zurückzusetzen oder den Betriebsmodus der Kommunikationshardware zu ändern.

## Fehlercodes (nur Windows)

Bei seriellen Leseoperationen oder bei der Ereignisabfrage im Hintergrund können viele verschiedene Fehler auftreten. Das externe Gerät kann ausgeschaltet sein, die Datenleitungen können verrauscht sein, Systempuffer können überlaufen oder Ihre Moduseinstellungen können falsch sein. Deshalb sollte zuverlässige Software serielle Leseoperationen immer catchen. Im Fehlerfall gibt Tcl einen allgemeinen Datei-E/A-Fehler zurück. Dann kann fconfigure **-lasterror** helfen, das Problem einzugrenzen. Folgende Fehlercodes können zurückgegeben werden.

**RXOVER**

Überlauf des Windows-Eingabepuffers. Die Daten kommen schneller an, als Ihr Skript sie liest, oder Ihr System ist überlastet. Verwenden Sie fconfigure **-sysbuffer**, um einen temporären Engpass zu vermeiden und/oder beschleunigen Sie Ihr Skript.

**TXFULL**

Überlauf des Windows-Ausgabepuffers. Gegenstück zu RXOVER. Dieser Fehler sollte praktisch nicht auftreten, weil Tcl den Status des Ausgabepuffers überwacht.

**OVERRUN**

UART-Pufferüberlauf (Hardware) mit Datenverlust. Die Daten kommen schneller an, als der Systemtreiber sie empfängt. Prüfen Sie die erweiterten seriellen Port-Einstellungen, um den FIFO-Puffer (16550) zu aktivieren und/oder einen niedrigeren(1) Interrupt-Schwellenwert zu setzen.

**RXPARITY**

Ihr UART hat einen Paritätsfehler erkannt. Falsche Paritätseinstellungen mit fconfigure **-mode** oder eine verrauschte Datenleitung (RXD) können diesen Fehler verursachen.

**FRAME**

Ihr UART hat einen Stoppbit-Fehler erkannt. Falsche Moduseinstellungen mit fconfigure **-mode** oder eine verrauschte Datenleitung (RXD) können diesen Fehler verursachen.

**BREAK**

Ihr UART hat eine BREAK-Bedingung erkannt (siehe oben).

## Portabilitätsfragen

### Windows

Zulässige Werte für *fileName* zum Öffnen eines seriellen Ports haben die Form **com***X*, wobei *X* eine Zahl ist, in der Regel von 1 bis 9. Eine ältere, ebenfalls akzeptierte Form ist **com***X***:**. Diese Notation funktioniert nur für serielle Ports von 1 bis 9. Der Versuch, einen nicht existierenden seriellen Port oder einen mit einer Nummer größer als 9 zu öffnen, schlägt fehl. Eine alternative Form zum Öffnen serieller Ports ist der Dateiname **//./comX**, wobei *X* eine beliebige Nummer ist, die einem seriellen Port entspricht.

Wird Tcl interaktiv ausgeführt, kann es zu seltsamen Wechselwirkungen zwischen der echten Konsole, falls vorhanden, und einer Befehlspipeline kommen, die Standardeingabe oder -ausgabe nutzt. Wird eine Befehlspipeline zum Lesen geöffnet, gehen einige an der Konsole eingegebene Zeilen in die Befehlspipeline, andere an den Tcl-Interpreter. Wird eine Befehlspipeline zum Schreiben geöffnet, sind Tastatureingaben an der Konsole erst sichtbar, wenn die Pipeline geschlossen wird. Diese Probleme treten nur auf, weil Tcl und die Kindanwendung gleichzeitig um die Konsole konkurrieren. Wird die Befehlspipeline aus einem Skript gestartet, sodass Tcl die Konsole nicht nutzt, oder verwendet die Befehlspipeline weder Standardeingabe noch -ausgabe, sondern ist von einer Datei umgeleitet oder dorthin umgeleitet, treten die genannten Probleme nicht auf.

Dateien, die im Modus "**a**" oder mit gesetztem Flag **APPEND** geöffnet werden, werden implementiert, indem unmittelbar vor jedem Schreibvorgang gesucht wird; das ist keine atomare Operation und bietet nicht die auf POSIX-Plattformen vorhandene Garantie des strikten Anhängens.

### Unix

Zulässige Werte für *fileName* zum Öffnen eines seriellen Ports haben in der Regel die Form **/dev/tty***X*, wobei *X* **a** oder **b** ist; es kann jedoch jeder Pseudodateiname verwendet werden, der einem seriellen Port zugeordnet ist. Erweiterte Konfigurationsoptionen werden für serielle Ports nur unterstützt, wenn Tcl mit der POSIX-Schnittstelle für serielle Ports gebaut wurde.

Wird Tcl interaktiv ausgeführt, kann es zu seltsamen Wechselwirkungen zwischen der Konsole, falls vorhanden, und einer Befehlspipeline kommen, die die Standardeingabe nutzt. Wird eine Befehlspipeline zum Lesen geöffnet, gehen einige an der Konsole eingegebene Zeilen in die Befehlspipeline, andere an den Tcl-Interpreter. Dieses Problem tritt nur auf, weil Tcl und die Kindanwendung gleichzeitig um die Konsole konkurrieren. Wird die Befehlspipeline aus einem Skript gestartet, sodass Tcl die Konsole nicht nutzt, oder verwendet die Befehlspipeline keine Standardeingabe, sondern ist von einer Datei umgeleitet, tritt das genannte Problem nicht auf.

Siehe den Abschnitt **PORTABILITÄTSFRAGEN** des Befehls exec für weitere Informationen zum Ausführen von Anwendungen auf den verschiedenen Plattformen, die nicht spezifisch für Befehlspipelines sind.

## Konsolenkanäle

Nur unter Windows unterstützen Konsolenkanäle (üblicherweise stdin oder stdout) die folgenden Optionen:

**-inputmode *inputMode***

Diese Option dient zum Abfragen oder Ändern des Eingabemodus des Konsolenkanals; sie steuert, wie interaktive Benutzereingabe behandelt wird. Folgende Werte für inputMode werden unterstützt:

**normal**

bedeutet, dass normale zeilenorientierte Eingabe mit den üblichen Konsolen-Bearbeitungsfunktionen verwendet werden soll.

**password**

bedeutet, dass nicht echoende Eingabe mit den üblichen Konsolen-Bearbeitungsfunktionen verwendet werden soll, aber ohne Ausgabe getippter Zeichen auf das Terminal (außer Zeilenumbrüchen).

**raw**

bedeutet, dass die gesamte Tastatureingabe unverarbeitet an Tcl weitergegeben werden soll, ohne dass die Konsole etwas verarbeitet. Die Tasten werden nicht echoiert; die Interpretation obliegt dem Tcl-Skript.

**reset (nur setzen)**

bedeutet, dass die Konsole auf den Zustand zurückgesetzt werden soll, in dem sie sich beim Öffnen des Konsolenkanals befand.

Beachten Sie, dass das Setzen dieser Option (technisch gesehen alles, was den Konsolenzustand über diese Option vom Standard abweicht) bewirkt, dass der Kanal beim Schließen automatisch die Konsole zurücksetzt.

**-winsize**

Diese Option ist nur abfragbar. Sie liefert eine zweielementige Liste mit der aktuellen Breite und Höhe der Konsole, mit der dieser Kanal kommuniziert.

Beachten Sie, dass gleichwertige Optionen unter Unix existieren, aber beim Kanaltyp "serieller Port" angeboten werden.

### Beispiele

Eine Datei zum Schreiben öffnen, Erstellung erzwingen und einen Fehler auslösen, falls sie bereits existiert.

```
set myNewFile [open filename.txt {WRONLY CREAT EXCL}]
```

Eine Datei zum Schreiben als Protokolldatei öffnen.

```
set myLogFile [open filename.log "a"]
fconfigure $myLogFile -buffering line
```

Eine Befehlspipeline öffnen und Fehler abfangen:

```
set fl [open "| ls this_file_does_not_exist"]
set data [read $fl]
if {[catch {close $fl} err]} {
    puts "ls command failed: $err"
}
```

Eine Befehlspipeline öffnen und binäre Daten daraus lesen. Beachten Sie die ungewöhnliche Form mit "|[list", die nicht triviale Randfälle mit Argumenten behandelt, die möglicherweise Leerzeichen enthalten.

```
set fl [open |[list create_image_data $input] "rb"]
set binData [read $fl]
close $fl
```

Ein Passwort sicher vom Benutzer lesen (vorausgesetzt, das Skript wird interaktiv ausgeführt):

```
chan configure stdin -inputmode password
try {
    chan puts -nonewline "Password: "
    chan flush stdout
    set thePassword [chan gets stdin]
} finally {
    chan configure stdin -inputmode reset
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## package

*Stichwörter:* package, version

### Name

package - Paketladen und Versionskontrolle

### Übersicht

**package files** *package*
**package forget** ?*package package* ...?
**package ifneeded** *package version* ?*script*?
**package names**
**package present** *package* ?*requirement...*?
**package present -exact** *package version*
**package provide** *package* ?*version*?
**package require** *package* ?*requirement...*?
**package require -exact** *package version*
**package unknown** ?*command*?
**package vcompare** *version1 version2*
**package versions** *package*
**package vsatisfies** *version requirement...*
**package prefer** ?**latest**|**stable**?

## Beschreibung

Der Befehl verwaltet eine einfache Datenbank der für den aktuellen Interpreter verfügbaren Pakete und wie sie geladen werden. Mehrere Versionen pro Paket werden unterstützt; die passende Version wird anhand der Anforderungen der Anwendung geladen. Versionskonflikte werden erkannt und gemeldet. In normalen Tcl-Skripten werden meist nur **package require** und **package provide** verwendet; die übrigen Unterbefehle dienen vor allem Systemskripten zur Pflege der Paketdatenbank.

Das Verhalten wird durch das erste Argument bestimmt. Zulässige Formen:

**package files *package***

Listet alle Dateien auf, die zu *package* gehören. Auto-geladene Dateien sind nicht enthalten, nur direkt beim Paketinitialisieren per **source** geladene. Die Reihenfolge entspricht der **source**-Reihenfolge.

**package forget ?*package package* ...?**

Entfernt alle Informationen zu jedem angegebenen Paket aus diesem Interpreter, einschließlich **package ifneeded** und **package provide**.

**package ifneeded *package version* ?*script*?**

Typischerweise nur in System-Konfigurationsskripten. Kennzeichnet, dass *version* von *package* bei Bedarf verfügbar ist und per *script* geladen werden kann. *script* wird für spätere **package require** gespeichert; üblicherweise richtet *script* Auto-Loading ein (oder load/source) und ruft **package provide** auf. Für ein Paket können mehrere Versionen eingetragen sein. Existiert bereits ein Eintrag für *package* und *version*, ersetzt *script* den alten. Ohne *script*: Rückgabe des aktuellen Skripts für diese Version, oder leerer String wenn kein **package ifneeded** gesetzt wurde.

**package names**

Liste aller Paketnamen, für die eine Version bereitgestellt wurde (**package provide**) oder ein **package ifneeded**-Skript existiert. Reihenfolge beliebig.

**package present ?-exact? *package* ?*requirement...*?**

Wie **package require**, lädt das Paket aber nicht, wenn es noch nicht geladen ist.

**package provide *package* ?*version*?**

Kennzeichnet, dass *version* von *package* im Interpreter vorhanden ist. Wird typischerweise im

**ifneeded**-Skript und beim endgültigen Laden des Pakets aufgerufen. Fehler, wenn eine andere Version bereits per **package provide** gemeldet wurde. Ohne *version*: aktuell bereitgestellte Versionsnummer, oder leerer String wenn nie **package provide** für *package* aufgerufen wurde.

**package require *package* ?*requirement...*?**

Wird von Tcl-Code aufgerufen, der eine bestimmte Paketversion braucht. Bei Erfolg: geladene Versionsnummer; sonst Fehler.

Eine passende Version erfüllt mindestens eine *requirement* (Abschnitt Anforderung). Bei mehreren passenden Versionen wird die höchste gewählt, zusätzlich beeinflusst durch **package prefer**.

Modus **stable**: höchste stabile passende Version; falls keine, höchste instabile. Modus **latest**: höchste passende Version unabhängig von Stabilität.

Ist *package* bereits bereitgestellt (**package provide**), muss die Version die *requirement*s erfüllen — sofortige Rückkehr. Sonst Suche in der **package ifneeded**-Datenbank; bei Treffer Auswertung des Skripts der höchsten passenden Version im globalen Namespace (inkl. **package provide**). Fehlt ein passender Eintrag und ist **package unknown** gesetzt, wird dieser ausgewertet; danach erneute Prüfung. Scheitern aller Schritte: Fehler.

**package require -exact *package version***

Nur die angegebene *version* ist akzeptabel. Äquivalent zu **package require** *package version-version*.

**package unknown ?*command*?**

"Letzter Ausweg" bei **package require**, wenn in **package ifneeded** nichts Passendes steht. Mit *command*: Präfix eines Befehls; Tcl hängt Paketname und Anforderungen an (z. B. **foo bar** + **package require test 2.4** → **foo bar test 2.4**). Ohne Anforderungen nur der Name. Ohne *command*: aktuelles **package unknown**-Skript oder leerer String. *command* als leerer String entfernt das Skript.

**package vcompare *version1 version2***

Vergleicht zwei Versionsnummern. Rückgabe -1 (früher), 0 (gleich), 1 (später).

**package versions *package***

Liste aller Versionsnummern von *package* mit **package ifneeded**-Einträgen.

**package vsatisfies *version requirement...***

1 wenn *version* mindestens eine Anforderung erfüllt, sonst 0. Anforderungen: Abschnitt Anforderung.

**package prefer ?latest|stable?**

Ohne Argument: **latest** oder **stable** — aktueller Auswahlmodus für **package require**.

**latest**: Modus auf "latest" setzen.

**stable**: Wenn bereits **stable**, bleibt es; von **latest** zurück auf **stable** ist wirkungslos — Modus bleibt **latest**.

Anderes Argument: Fehler "invalid argument".

Neuer Interpreter: initial **stable**, außer Umgebungsvariable **TCL_PKG_PREFER_LATEST** ist gesetzt (beliebiger Wert) oder das Tcl-Paket selbst ist instabil — dann initial und dauerhaft **latest**.

## Versionsnummern

Versionsnummern bestehen aus einer oder mehreren durch Punkte getrennten Dezimalzahlen (z. B. 2, 1.162, 3.1.13.1). Die erste Zahl ist die Hauptversionsnummer; größere Zahlen = spätere Versionen, links mehr Signifikanz. Fehlende Felder = Nullen: 1.3 = 1.3.0 = 1.3.0.0, früher als 1.3.1. Die Buchstaben "a" (alpha) und "b" (beta) dürfen je einmal einen Punkt ersetzen und fügen semantisch einen negativen Anteil ein ("a" = -2, "b" = -1); nur einmal, gegenseitig ausschließend. 1.3a1 → 1.3.-2.1, 1.3b1 → 1.3.-1.1. Negative Zahlen sind in Spezifizierern nicht direkt erlaubt. Ohne "a"/"b": **stable**; mit Buchstaben: **unstable**. Spätere Versionen gelten als aufwärtskompatibel zur früheren, solange die Hauptversionsnummer gleich ist. Änderung der Hauptversion: inkompatible Änderungen.

## Paket-Indizes

Empfohlen: **package require** und **package provide** in Skripten, pkg_mkIndex für Indexdateien — Pakete laden dann automatisch bei **package require**. Siehe pkg_mkIndex.

## Anforderung

Eine *requirement*-Zeichenkette prüft, ob eine kompatible Paketversion vorhanden ist. Die meisten Befehle akzeptieren eine Liste von Anforderungen; die höchste passende Version wird gewählt.

Formen:

***min***
"min-bounded" (untere Grenze).

***min-***
"min-unbound" (nur untere Grenze, offen nach oben).

***min-max***
"bounded" (Intervall).

wobei *min* und *max* gültige Versionsnummern sind. Legacy-Syntax ist Spezialfall der erweiterten Syntax. Regeln:

1. Die *version* muss mindestens eine *requirement* erfüllen.
2. "bounded": erfüllt wenn

   **[a]**
   *min* gleich *max* genau dann, wenn *version* gleich *min* ist.

   **[b]**
   sonst genau dann, wenn *version* ≥ *min* und *version* < *max* (intern beide mit "a0" aufgefüllt). Vergleich zu *min* inklusiv, zu *max* exklusiv.

1. "min-bounded": "bounded" mit implizitem *max* = nächste höhere Hauptversion von *min*.
2. "min-unbound": *version* ≥ *min* (*min* intern mit "a0" aufgefüllt), keine Obergrenze.

## Beispiele

Tk und http am Skriptanfang anfordern:

```
package require Tk
package require http
```

Snack optional laden:

```
if {[catch {package require Snack}]} {
    # Error thrown - package not found.
    # Set up a dummy interface to work around the absence
} else {
    # We have the package, configure the app to use it
}
```

*Copyright (c) 1996 Sun Microsystems, Inc.*

# packagens

*Stichwörter:* packagens, auto-load, index, package, version

## Name

pkg::create - Passenden Befehl package ifneeded zu einer Paketangabe erzeugen

## Übersicht

**::pkg::create -name** *packageName* **-version** *packageVersion* ?*-load filespec*? ... ?*-source filespec*? ...

## Beschreibung

**::pkg::create** ist eine Hilfsprozedur der Tcl-Standardbibliothek. Sie erzeugt zu einer Paketangabe den passenden Befehl package ifneeded. Damit lässt sich eine Datei **pkgIndex.tcl** für den package-Mechanismus bauen.

## Optionen

Unterstützt werden diese Parameter:

**-name *packageName***

Legt den Namen des Pakets fest. Erforderlich.

**-version *packageVersion***

Legt die Version des Pakets fest. Erforderlich.

**-load *filespec***

Legt eine Bibliothek fest, die mit load geladen werden muss. *filespec* ist eine Liste mit zwei Elementen. Das erste ist der Name der zu ladenden Datei. Das zweite, optionale Element ist eine Liste der Befehle, die das Laden dieser Datei bereitstellt. Ist die Prozedurliste leer oder fehlt sie, richtet **::pkg::create** die Bibliothek zum direkten Laden ein (siehe pkg_mkIndex). Beliebig viele Parameter **-load** sind zulässig.

**-source *filespec***

Wie **-load**, gibt aber eine Tcl-Bibliothek an, die mit source geladen werden muss. Beliebig viele Parameter **-source** sind zulässig.

Mindestens ein **-load** oder **-source** muss angegeben sein.

*Copyright (c) 1998-2000 Scriptics Corporation.*

# pid

*Stichwörter:* pid, file, pipeline, process identifier

## Name

pid - Ermittelt Prozesskennungen

## Übersicht

**pid** ?*fileId*?

## Beschreibung

Ist das Argument *fileId* angegeben, sollte es normalerweise auf eine mit open erzeugte Prozess-Pipeline verweisen. In diesem Fall gibt **pid** eine Liste zurück, deren Elemente die Prozesskennungen aller Prozesse in der Pipeline in Reihenfolge sind. Die Liste ist leer, wenn *fileId* eine geöffnete Datei bezeichnet, die keine Prozess-Pipeline ist. Ohne *fileId* gibt **pid** die Prozesskennung des aktuellen Prozesses zurück. Alle Prozesskennungen werden als Dezimalzeichenketten zurückgegeben.

## Beispiel

Prozessinformationen zu den Prozessen einer Pipeline mit dem SysV-Programm **ps** ausgeben, bevor die Pipeline-Ausgabe gelesen wird:

```
set pipeline [open "| zcat somefile.gz | grep foobar | sort -u"]
# Prozessinformationen ausgeben
exec ps -fp [pid $pipeline] >@stdout
# Trennzeile und dann die Pipeline-Ausgabe
puts [string repeat - 70]
puts [read $pipeline]
close $pipeline
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# pkgMkIndex

*Stichwörter:* pkgMkIndex, auto-load, index, package, version

## Name

pkg_mkIndex - Erstellt einen Index für automatisches Laden von Paketen

## Übersicht

**pkg_mkIndex** ?*options...*? *dir* ?*pattern pattern* ...?

## Beschreibung

**Pkg_mkIndex** ist eine Hilfsprozedur der Standard-Tcl-Bibliothek. Sie erstellt Indexdateien, mit denen Pakete automatisch geladen werden, wenn package require ausgeführt wird. So verwenden Sie **pkg_mkIndex**:

1. Erstellen Sie die Pakete. Jedes Paket kann aus einer oder mehreren Tcl-Skriptdateien oder

Binärdateien bestehen. Binärdateien müssen mit dem Befehl load und einem einzigen Argument ladbar sein; ist die Datei beispielsweise **test.so**, muss load test.so möglich sein. Jede Skriptdatei muss einen Befehl package provide enthalten, der Paket und Versionsnummer deklariert; jede Binärdatei muss einen Aufruf von Tcl_PkgProvide enthalten.

2. Erstellen Sie den Index durch Aufruf von **pkg_mkIndex**. Das Argument *dir* gibt ein Verzeichnis an, und jedes Argument *pattern* ist ein glob-artiges Muster, das Skript- oder Binärdateien in *dir* auswählt. Standardmuster sind ***.tcl** und ***.[info sharedlibextension]**.

   **Pkg_mkIndex** erzeugt in *dir* die Datei **pkgIndex.tcl** mit Paketinformationen zu allen durch die *pattern*-Argumente ausgewählten Dateien. Dazu lädt es jede Datei in einen Kind-Interpreter und prüft, welche Pakete und neuen Befehle erscheinen (daher sind package provide bzw. Tcl_PkgProvide in den Dateien erforderlich). Ist ein Paket auf Skripte und Binärdateien verteilt oder bestehen Abhängigkeiten zwischen Dateien, müssen Sie ggf. die Option **-load** nutzen oder die Verarbeitungsreihenfolge anpassen. Siehe Komplexe Fälle unten.
3. Installieren Sie das Paket als Unterverzeichnis eines der Verzeichnisse in der Variable tcl_pkgPath. Enthält **$tcl_pkgPath** mehr als ein Verzeichnis, sollten maschinenabhängige Pakete (z. B. mit binären Shared Libraries) normalerweise im ersten Verzeichnis und maschinenunabhängige Pakete (nur Tcl-Skripte) im zweiten installiert werden. Das Unterverzeichnis sollte die Skript- und/oder Binärdateien des Pakets sowie **pkgIndex.tcl** enthalten. Liegt das Paket als Unterverzeichnis eines Verzeichnisses in **$tcl_pkgPath**, wird es bei package require automatisch gefunden.

   Installieren Sie das Paket anderswo, muss das Paketverzeichnis in der globalen Variable auto_path oder in einem unmittelbaren Unterverzeichnis eines Eintrags in auto_path liegen. **Auto_path** listet Verzeichnisse, die Auto-Loader und Paket-Loader durchsuchen; standardmäßig enthält sie **$tcl_pkgPath**. Der Paket-Loader prüft auch alle Unterverzeichnisse der Einträge in auto_path. Sie können ein Verzeichnis in der Anwendung explizit zu auto_path hinzufügen oder die Umgebungsvariable **TCLLIBPATH** setzen: ist sie gesetzt, initialisiert Tcl auto_path beim Start daraus.
4. Nach diesen Schritten genügt package require. Sind die Versionen 2.1, 2.3 und 3.1 von Paket **Test** per **pkg_mkIndex** indexiert, macht **package require Test** Version 3.1 verfügbar und **package require -exact Test 2.1** Version 2.1. In den Indexdateien unter auto_path können viele Versionen eines Pakets stehen, aber pro Interpreter wird nur eine geladen — je nach erstem package require. Verschiedene Versionen können in verschiedenen Interpretern geladen sein.

## Optionen

Die optionalen Schalter sind:

**-direct**

Der erzeugte Index lädt das Paket bei package require direkt. Das ist die Voreinstellung.

**-lazy**

Der erzeugte Index verzögert das Laden des Pakets bis zur ersten Verwendung eines vom Paket bereitgestellten Befehls, statt es sofort bei package require zu laden. Das ist nicht mit *auto_reset* vereinbar; die Verwendung wird daher nicht empfohlen.

**-load *pkgPat***

Beim Indexieren werden alle Pakete, die im aktuellen Interpreter existieren und *pkgPat* entsprechen, in den Kind-Interpreter geladen, der den Index erzeugt. Der Mustervergleich folgt string match, ohne Berücksichtigung der Groß-/Kleinschreibung. Siehe Komplexe Fälle unten.

**-verbose**

Erzeugt Ausgabe während der Indexierung über die Prozedur **tclLog**, die standardmäßig nach stderr schreibt.

**--**

Ende der Schalter, falls *dir* mit einem Bindestrich beginnt.

## Pakete und der Auto-Loader

Die Paketverwaltung überschneidet sich etwas mit dem Auto-Loader, da beide Dateien bei Bedarf laden. Die Paketverwaltung ist jedoch eine höhere Ebene und nutzt den Auto-Loader für den letzten Ladeschritt. Pakete sollten eher mit **pkg_mkIndex** als mit auto_mkindex indexiert werden, weil die Paketmechanismen Versionskontrolle bieten: mehrere Versionen können in Indexdateien verfügbar sein, und Anwendungen wählen per package require die Version. auto_mkindex kennt keine Versionen und kann nur eine Version pro Paket verwalten. Ein Paket sollte nicht gleichzeitig mit **pkg_mkIndex** und auto_mkindex indexiert werden. Wurde ein Paket mit **pkg_mkIndex** indexiert, sind seine Befehle erst nach package require aufrufbar; mit auto_mkindex indexierte Pakete sind sofort nutzbar, weil es keine Versionskontrolle gibt.

## Funktionsweise

**Pkg_mkIndex** nutzt package unknown, package ifneeded und den Auto-Loader. Beim ersten package require wird das package unknown-Skript ausgeführt, das Tcl beim Start so setzt, dass alle **pkgIndex.tcl** in auto_path ausgewertet werden. Diese Dateien enthalten package ifneeded für jede Version jedes verfügbaren Pakets; diese rufen package provide auf und richten Auto-Loader-Informationen zum Laden der Paketdateien ein. Wurde **pkgIndex.tcl** mit **-lazy** erzeugt, wird eine Datei einer Version eines Pakets erst beim ersten Aufruf eines ihrer Befehle geladen. Nach package require sind die Paketbefehle im Interpreter ggf. noch nicht sichtbar, können aber aufgerufen werden und werden dann automatisch geladen.

## Direktes Laden

Manche Pakete — z. B. solche mit Namespaces und exportierten Befehlen oder mit spezieller Initialisierung — sollen ihre Dateien sofort bei package require laden, statt das Laden bis zur ersten Befehlsnutzung zu verzögern. Das ist die Voreinstellung beim Erzeugen des Paketindex und kann mit **-lazy** überschrieben werden.

## Komplexe Fälle

Die meisten komplexen Abhängigkeiten zwischen Skripten und Binärdateien sowie aufgeteilte Pakete funktionieren in der Regel. Sie müssen ggf. die Verarbeitungsreihenfolge von **pkg_mkIndex** anpassen. Details folgen.

Enthält jede Skript- oder Datei genau ein Paket und liegt jedes Paket nur in einer Datei, reicht es, alle zu indexierenden Dateien in beliebiger Reihenfolge per Glob-Muster anzugeben.

Generell dürfen Skripte von anderen Paketen abhängen. Enthalten Skripte package require, werden diese im Interpreter zum Verarbeiten der Skripte ausgehängt und verursachen keine Probleme. Rufen Skripte im globalen Code andere Pakete auf, fängt ein Stub-unknown-Befehl das ab. Verweisen Skripte im globalen Code auf Variablen anderer Pakete, entstehen Fehler — das ist zudem schlechter Stil.

Haben Binärdateien Abhängigkeiten zu anderen Paketen, wird es schwieriger, weil C-APIs wie Tcl_PkgRequire beim Laden einer Binärdatei nicht ausgehängt werden können. Benötigt das BLT-Paket Tk und ruft Tcl_PkgRequire in **Blt_Init** auf, muss **pkg_mkIndex** in einem Interpreter mit geladenem Tk laufen. Das erreichen Sie mit **-load** *pkgPat*: **pkg_mkIndex** lädt dann alle von info loaded gelisteten Pakete, die *pkgPat* entsprechen, in den Verarbeitungs-Interpreter — in den meisten Fällen genügt das für Tcl_PkgRequire in Binärdateien.

Indexieren Sie zwei Binärdateien und hängt eine von der anderen ab, geben Sie die abhängige zuletzt an. So wird die unabhängige zuerst geladen und indexiert, und ihr Paket steht beim Verarbeiten der zweiten Datei bereit. Ggf. müssen Sie das erste Paket mit **-load** in den temporären Interpreter laden; es schadet nicht, Muster für noch nicht geladene Pakete anzugeben.

Ist ein Paket auf Skripte und eine Binärdatei verteilt, vermeiden Sie **-load**. Wird ein Paket vor der Indexberechnung geladen, verdeckt es andere Dateien desselben Pakets. Müssen Sie **-load** verwenden, geben Sie die Skripte zuerst an — sonst kann das aus der Binärdatei geladene Paket das der Skripte verdecken.

*Copyright (c) 1996 Sun Microsystems, Inc.*

# platform

*Stichwörter:* platform, operating system, cpu architecture, architecture

## Name

platform - Unterstützungscode und Hilfsmittel zur Systemidentifikation

## Übersicht

**package require platform** ?**1.0.10**?

**platform::generic**
**platform::identify**
**platform::patterns** *identifier*

## Beschreibung

Das Paket **platform** stellt mehrere Hilfsbefehle bereit, die zur Identifikation der Architektur einer Maschine nützlich sind, auf der Tcl läuft.

Tcl bietet zwar das Array tcl_platform zur Identifikation der aktuellen Architektur (insbesondere die Elemente platform und machine), das reicht nicht immer aus. Auf Unix-Systemen spiegelt tcl_platform die von **uname** gelieferten Werte wider, die über Plattformen und Architekturen nicht standardisiert sind. Auf mindestens einer Plattform (AIX) enthält **tcl_platform(machine)** außerdem die CPU-Seriennummer.

Einzelne Anwendungen müssen daher Werte in tcl_platform (zusammen mit Ausgaben systemspezifischer Hilfsprogramme) selbst aufbereiten — unbequem für Entwickler und fehleranfällig bei Inkonsistenzen bei Architektur-Identifikation und Namenskonventionen.

Das Paket **platform** verhindert solche Fragmentierung: Es etabliert eine standardisierte Namenskonvention für Architekturen mit Tcl und erleichtert Entwicklern die Identifikation der Architektur, unter der ein Tcl-Programm läuft.

## Befehle

**platform::identify**

Dieser Befehl gibt eine Kennung zurück, die die Plattform beschreibt, auf der der Tcl-Kern läuft. Die Kennung hat im Allgemeinen das Format *OS-CPU*. Der *OS*-Teil kann Details wie Kernel-Version, libc-Version usw. enthalten und selbst Bindestriche; der *CPU*-Teil enthält keine Bindestriche, sodass der vorletzte Bindestrich in der Kennung der letzte ist.

**platform::generic**

Dieser Befehl gibt eine vereinfachte Kennung zurück, die die Plattform beschreibt, auf der der Tcl-Kern läuft. Im Gegensatz zu **platform::identify** werden Details wie Kernel- oder libc-Version weggelassen. Das Format ist im Allgemeinen *OS-CPU*.

**platform::patterns *identifier***

Dieser Befehl nimmt eine von **platform::identify** zurückgegebene Kennung und liefert eine Liste von Kennungen kompatibler Architekturen.

## Beispiele

Ermöglicht z. B., eine Anwendung mit mehreren Builds einer Shared Library auszuliefern, damit dasselbe Paket auf vielen Versionen eines Betriebssystems funktioniert:

```
package require platform
# Annahme: App-Skript ist .../theapp/bin/theapp.tcl
set binDir [file dirname [file normalize [info script]]]
set libDir [file join $binDir .. lib]
set platLibDir [file join $libDir [platform::identify]]
load [file join $platLibDir support[info sharedlibextension]]
```

*Copyright (c) 2006 ActiveState Software Inc*

# platform_shell

*Stichwörter:* platform_shell, operating system, cpu architecture, platform, architecture

## Name

platform::shell - Unterstützungscode und Hilfsmittel zur Systemidentifikation

## Übersicht

**package require platform::shell** ?**1.1.4**?

**platform::shell::generic** *shell*
**platform::shell::identify** *shell*
**platform::shell::platform** *shell*

## Beschreibung

Das Paket **platform::shell** stellt mehrere Hilfsbefehle bereit, die zur Identifikation der Architektur einer bestimmten Tcl-Shell nützlich sind.

Das Paket erlaubt die Identifikation der Architektur einer Tcl-Shell, die sich von der Shell unterscheidet, die das Paket ausführt. Voraussetzung ist nur, dass die andere Shell (über ihren Pfad identifiziert) auf der aktuellen Maschine ausführbar ist.

Auf den meisten Plattformen ist die Architektur der abgefragten Shell mit der der laufenden Shell identisch; das gilt nicht allgemein. Gegenbeispiel sind Plattformen mit 32- und

64-Bit-Varianten, auf denen ein 64-Bit-System 32-Bit-Code ausführen kann: Laufende und abgefragte Shell können unterschiedliche 32/64-Bit-Einstellungen und damit unterschiedliche Kennungen haben.

Für Anwendungen wie ein Code-Repository ist es wichtig, die Architektur der Shell zu kennen, die installierte Pakete tatsächlich ausführt — nicht nur die Architektur der Shell, die die Repository-Software betreibt.

## Befehle

**platform::shell::identify *shell***
Führt dieselbe Identifikation wie platform::identify durch, jedoch für die angegebene Tcl-Shell statt für die laufende Shell.

**platform::shell::generic *shell***
Führt dieselbe Identifikation wie platform::generic durch, jedoch für die angegebene Tcl-Shell statt für die laufende Shell.

**platform::shell::platform *shell***
Gibt den Inhalt von **tcl_platform(platform)** für die angegebene Tcl-Shell zurück.

*Copyright (c) 2006-2008 ActiveState Software Inc*

# prefix

*Stichwörter:* prefix, table lookup

## Name

tcl::prefix - Hilfsmittel für Präfixabgleich

## Übersicht

**::tcl::prefix all** *table string*
**::tcl::prefix longest** *table string*
**::tcl::prefix match** ?*option* ...? *table string*

## Beschreibung

Dieses Dokument beschreibt Befehle zum Nachschlagen eines Präfixes in einer Liste von Zeichenketten. Folgende Befehle werden unterstützt:

**::tcl::prefix all *table string***
Gibt eine Liste aller Elemente in *table* zurück, die mit dem Präfix *string* beginnen.

**::tcl::prefix longest *table string***
Gibt das längste gemeinsame Präfix aller Elemente in *table* zurück, die mit dem Präfix *string* beginnen.

**::tcl::prefix match ?*option* ...? *table string***
Wenn *string* einem Element in *table* entspricht oder das Präfix genau eines Elements ist, wird das gefundene Element zurückgegeben. Andernfalls hängt das Ergebnis von der Option **-error** ab. (Es wird empfohlen, *table* vor der Verwendung mit diesem Unterbefehl zu sortieren, damit die Liste der Übereinstimmungen in der Fehlermeldung ebenfalls sortiert ist; dies ist für die Funktion des Unterbefehls selbst jedoch nicht zwingend erforderlich.) Folgende Optionen werden unterstützt:

**-exact**

Nur exakte Übereinstimmungen akzeptieren.

**-message *string***

*string* in der Fehlermeldung bei Nichtübereinstimmung verwenden. Standard ist "option".

**-error *options***

Die *options* werden verwendet, wenn keine Übereinstimmung gefunden wird. Ist *options* leer, wird kein Fehler erzeugt und eine leere Zeichenkette zurückgegeben. Andernfalls werden die *options* als return-Optionen bei der Erzeugung der Fehlermeldung verwendet. Der Standard entspricht dem Setzen von "-level 0". Beispiel: Wird "**-error** {-errorcode MyError -level 1}" verwendet, würde ein Fehler wie folgt erzeugt:

```
return -errorcode MyError -level 1 -code error \
       "ambiguous option ..."
```

## Beispiele

Grundlegende Verwendung:

```
namespace import ::tcl::prefix
prefix match {apa bepa cepa} apa
     → apa
prefix match {apa bepa cepa} a
     → apa
prefix match -exact {apa bepa cepa} a
     → bad option "a": must be apa, bepa, or cepa
prefix match -message "switch" {apa ada bepa cepa} a
     → ambiguous switch "a": must be apa, ada, bepa, or cepa
prefix longest {fblocked fconfigure fcopy file fileevent flush} fc
     → fco
prefix all {fblocked fconfigure fcopy file fileevent flush} fc
     → fconfigure fcopy
```

Optionenabgleich vereinfachen:

```
array set opts {-apa 1 -bepa "" -cepa 0}
foreach {arg val} $args {
    set opts([prefix match {-apa -bepa -cepa} $arg]) $val
}
```

Erzeugen eines switch, der Präfixe unterstützt:

```
switch [prefix match {apa bepa cepa} $arg] {
    apa  { }
    bepa { }
    cepa { }
}
```

*Copyright (c) 2008 Peter Spjuth* pspjuth@users.sourceforge.net

# proc

*Stichwörter:* proc, argument, procedure

## Name

proc - Erzeugt eine Tcl-Prozedur

## Übersicht

**proc** *name args body*

## Beschreibung

Der Befehl **proc** erzeugt eine neue Tcl-Prozedur *name* und ersetzt einen vorhandenen Befehl oder eine Prozedur gleichen Namens. Bei jedem Aufruf führt der Interpreter *body* aus. Normalerweise ist *name* unqualifiziert; die Prozedur entsteht im aktuellen Namespace. Enthält *name* Namespace-Qualifizierer, entsteht sie im angegebenen Namespace. *args* listet die formalen Argumente (möglicherweise leer). Jedes Element ist eine Liste mit einem oder zwei Feldern: nur Name, oder Name und Standardwert. Argumente mit Standardwert, denen nicht-standardisierte Argumente folgen, werden faktisch erforderlich — es müssen genug tatsächliche Argumente geliefert werden, um alle formalen Argumente bis einschließlich des letzten erforderlichen abzudecken. Portable Skripte sollten alle optionalen Argumente nach allen erforderlichen platzieren.

Beim Aufruf von *name* entsteht pro formales Argument eine lokale Variable mit dem Wert des Aufrufarguments oder dem Standardwert. Zuweisung strikt in Reihenfolge; optionale Argumente können weggelassen werden, aber es dürfen weder zu wenige noch zu viele tatsächliche Argumente übergeben werden.

Sonderfall variable Argumentanzahl: heißt das letzte formale Argument **args**, darf der Aufruf mehr tatsächliche als formale Argumente haben; ab dem **args** zugewiesenen Argument werden alle restlichen tatsächlichen Argumente zu einer Liste kombiniert (wie mit list) und **args** zugewiesen.

Während *body* läuft, beziehen sich Variablennamen auf lokale Variablen (automatisch erzeugt und beim Verlassen der Prozedur gelöscht); pro Argument eine lokale Variable. Andere Variablen nur über global, variable, upvar oder namespace upvar. Der aktuelle Namespace während *body* ist der des Prozedurnamens (Erstellungs-Namespace, sofern rename nicht geändert hat).

**proc** gibt eine leere Zeichenkette zurück. Der Rückgabewert der Prozedur ist der Wert eines return-Befehls; ohne explizites return der Wert des letzten Befehls in *body*. Tritt in *body* ein Fehler auf, gibt die Prozedur denselben Fehler zurück.

## Beispiele

Summe und Produkt zweier Argumente ausgeben; explizites Ergebnis **OK**:

```
proc printSumProduct {x y} {
    set sum [expr {$x + $y}]
    set prod [expr {$x * $y}]
    puts "sum is $sum, product is $prod"
    return "OK"
}
```

Beliebig viele Argumente ausgeben:

```
proc printArguments args {
    foreach arg $args {
        puts $arg
    }
}
```

Variable mit dem Faktor multiplizieren (Standard 2), ähnlich incr:

```
proc mult {varName {multiplier 2}} {
    upvar 1 $varName var
    set var [expr {$var * $multiplier}]
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# process

*Stichwörter:* process, background, child, detach, wait

## Name

tcl::process - Verwaltung von Unterprozessen

## Übersicht

**::tcl::process** *option* ?*arg arg* ...?

## Beschreibung

Dieser Befehl bietet eine Möglichkeit, Unterprozesse zu verwalten, die von den Befehlen open und exec erstellt wurden, identifiziert anhand der Prozess-IDs (PIDs) dieser Unterprozesse. Die zulässigen *options* (können abgekürzt werden) sind:

**::tcl::process autopurge ?*flag*?**

Automatische Bereinigungsfunktion. Wird *flag* als boolescher Wert angegeben, aktiviert oder deaktiviert er autopurge. In allen Fällen gibt er den aktuellen Status als booleschen Wert zurück. Ist autopurge aktiv, wird Tcl_ReapDetachedProcs bei jedem Aufruf von exec oder beim Schließen eines von open erzeugten Pipe-Kanals aufgerufen. Ist autopurge inaktiv, muss **::tcl::process** purge explizit aufgerufen werden. Standardmäßig ist autopurge aktiv.

**::tcl::process list**

Gibt die Liste der Unterprozess-PIDs zurück. Dazu gehören alle derzeit laufenden Unterprozesse sowie alle beendeten Unterprozesse, deren zugehörige Prozesstabelleneinträge noch nicht bereinigt wurden.

**::tcl::process purge ?*pids*?**

Bereinigt alle Daten zu beendeten Unterprozessen. Wird *pids* als Liste von PIDs angegeben, bereinigt der Befehl nur Daten der passenden Unterprozesse, sofern sie existieren. Ist ein gelisteter Prozess noch aktiv, tut der Befehl für diesen Prozess nichts. Jede PID, die keinem Unterprozess entspricht, wird ignoriert.

**::tcl::process status ?*switches*? ?*pids*?**

Gibt ein Dictionary zurück, das Unterprozess-PIDs ihren jeweiligen Status zuordnet. Wird *pids* als Liste von PIDs angegeben, liefert der Befehl nur den Status der passenden Unterprozesse, sofern sie existieren. Jede PID, die keinem Unterprozess entspricht, wird ignoriert. Für aktive Prozesse ist der Status ein leerer Wert. Für beendete Prozesse ist der Status eine Liste im folgenden Format: "**{***code* ?*msg errorCode*?**}**", wobei:

***code***
ein standardmäßiger Tcl-Rückgabecode ist, d. h. **0** für TCL_OK und **1** für TCL_ERROR,

***msg***
die menschenlesbare Fehlermeldung ist,

***errorCode***
verwendet dasselbe Format wie die globale Variable errorCode

**msg** und errorCode sind nur bei abnormal beendeten Prozessen vorhanden (d. h. wenn *code* ungleich null ist). Intern ruft dieser Befehl Tcl_WaitPid mit gesetztem Flag **WNOHANG** für nicht-blockierendes Verhalten auf, sofern der Schalter **-wait** nicht gesetzt ist (siehe unten).

Zusätzlich akzeptiert **::tcl::process status** folgende Schalter:

**-wait**
Standardmäßig kehrt der Befehl sofort zurück (der zugrunde liegende Tcl_WaitPid wird mit gesetztem Flag **WNOHANG** aufgerufen), sofern dieser Schalter nicht gesetzt ist. Wird *pids* als Liste von PIDs angegeben, wartet der Befehl, bis der Status der passenden Unterprozesse verfügbar ist. Wurde *pids* nicht angegeben, wartet dieser Befehl auf alle bekannten Unterprozesse.

**--**
Markiert das Ende der Schalter. Das folgende Argument wird als erstes *arg* behandelt, auch wenn es mit **-** beginnt.

## Beispiele

Die folgenden Beispiele zeigen die Verwendung von **::tcl::process**. Einige Ergebnisse von **::tcl::process status** sind zur Lesbarkeit über mehrere Zeilen verteilt.

```
::tcl::process autopurge
     → true
::tcl::process autopurge false
     → false

set pid1 [exec command1 a b c | command2 d e f &]
     → 123 456
set chan [open "|command1 a b c | command2 d e f"]
     → file123
set pid2 [pid $chan]
     → 789 1011

::tcl::process list
     → 123 456 789 1011

::tcl::process status
     → 123 0
```

```
       456 {1 "child killed: write on pipe with no readers" {
         CHILDKILLED 456 SIGPIPE "write on pipe with no readers"}}
       789 {1 "child suspended: background tty read" {
         CHILDSUSP 789 SIGTTIN "background tty read"}}
       1011 {}

::tcl::process status 123
     → 123 0

::tcl::process status 1011
     → 1011 {}

::tcl::process status -wait
     → 123 0
       456 {1 "child killed: write on pipe with no readers" {
         CHILDKILLED 456 SIGPIPE "write on pipe with no readers"}}
       789 {1 "child suspended: background tty read" {
         CHILDSUSP 789 SIGTTIN "background tty read"}}
       1011 {1 "child process exited abnormally" {
         CHILDSTATUS 1011 -1}}

::tcl::process status 1011
     → 1011 {1 "child process exited abnormally" {
         CHILDSTATUS 1011 -1}}

::tcl::process purge
exec command1 1 2 3 &
     → 1213
::tcl::process list
     → 1213
```

*Copyright (c) 2017 Frederic Bonnet.*

# puts

*Stichwörter:* puts

## Name

puts - In einen Kanal schreiben

## Übersicht

**puts** ?**-nonewline**? ?*channel*? *string*

## Beschreibung

Der Befehl **puts** wurde durch chan puts abgelöst, der dieselbe Syntax und dieselben Optionen unterstützt.

*Copyright (c) 1993 The Regents of the University of California.*

*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# pwd

*Stichwörter:* pwd, working directory

## Name

pwd - Gibt den absoluten Pfad des aktuellen Arbeitsverzeichnisses zurück

## Übersicht

**pwd**

## Beschreibung

Gibt den absoluten Pfadnamen des aktuellen Arbeitsverzeichnisses zurück.

## Beispiele

Manchmal ist es nützlich, für einen externen Befehl mit exec in ein bekanntes Verzeichnis zu wechseln, während die Anwendung normalerweise im Verzeichnis weiterläuft, in dem sie gestartet wurde (sofern der Benutzer nichts anderes angibt), um Verwirrung zu vermeiden. Dazu speichert man das aktuelle Verzeichnis während des externen Befehls und wechselt danach zurück:

```
set tarFile [file normalize somefile.tar]
set savedDir [pwd]
cd /tmp
exec tar -xf $tarFile
cd $savedDir
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# re_syntax

*Stichwörter:* re_syntax, match, regular expression, string

## Name

re_syntax - Syntax von Tcl-Regulären Ausdrücken

## Beschreibung

Ein *regulärer Ausdruck* beschreibt Zeichenketten. Er ist ein Muster, das bestimmte Zeichenketten trifft und andere nicht.

## Verschiedene Varianten von REs

Reguläre Ausdrücke ("RE"s), wie von POSIX definiert, gibt es in zwei Varianten: *erweiterte* REs ("ERE"s) und *einfache* REs ("BRE"s). EREs entsprechen grob denen des traditionellen *egrep*, BREs grob denen des traditionellen *ed*. Diese Implementierung fügt eine dritte Variante hinzu: *fortgeschrittene* REs ("ARE"s), im Wesentlichen EREs mit einigen wesentlichen Erweiterungen.

Diese Manualseite beschreibt vor allem AREs. BREs existieren meist nur zur Abwärtskompatibilität in einigen alten Programmen; sie werden am Ende besprochen. POSIX-EREs sind fast eine exakte Teilmenge von AREs. Merkmale von AREs, die in EREs fehlen, werden gekennzeichnet.

## Syntax regulärer Ausdrücke

Tcl-Reguläre Ausdrücke werden mit dem Paket von Henry Spencer implementiert, basierend auf der Spezifikation 1003.2 und einigen (nicht allen) Perl5-Erweiterungen (danke, Henry!). Vieles der folgenden Beschreibung regulärer Ausdrücke ist wörtlich aus seinem Manual-Eintrag übernommen.

Ein ARE besteht aus einem oder mehreren *Zweigen*, getrennt durch "**|**", und trifft alles, was irgendeinen der Zweige trifft.

Ein Zweig besteht aus null oder mehr *Einschränkungen* oder *quantifizierten Atomen*, aneinandergereiht. Er trifft eine Übereinstimmung für das erste, gefolgt von einer für das zweite usw.; ein leerer Zweig trifft die leere Zeichenkette.

### Quantifizierer

Ein quantifiziertes Atom ist ein *Atom*, optional gefolgt von einem einzelnen *Quantifizierer*. Ohne Quantifizierer trifft es genau eine Übereinstimmung für das Atom. Die Quantifizierer und was ein so quantifiziertes Atom trifft, sind:

*****

eine Folge von 0 oder mehr Übereinstimmungen des Atoms

**+**

eine Folge von 1 oder mehr Übereinstimmungen des Atoms

**?**

eine Folge von 0 oder 1 Übereinstimmungen des Atoms

**{*m*}**

eine Folge von genau *m* Übereinstimmungen des Atoms

**{*m*,}**

eine Folge von *m* oder mehr Übereinstimmungen des Atoms

**{*m*,*n*}**

eine Folge von *m* bis *n* (einschließlich) Übereinstimmungen des Atoms; *m* darf *n* nicht überschreiten

***? +? ?? {*m*}? {*m*,}? {*m*,*n*}?**

*nicht-gierige* Quantifizierer, die dieselben Möglichkeiten treffen, aber die kleinste statt der größten Anzahl von Übereinstimmungen bevorzugen (siehe Abgleich)

Die Formen mit **{** und **}** heißen *Grenzen*. Die Zahlen *m* und *n* sind vorzeichenlose Dezimalzahlen mit zulässigen Werten von 0 bis 255 einschließlich.

### Atome

Ein Atom ist eines von:

**(*re*)**

trifft eine Übereinstimmung für *re* (*re* ist ein beliebiger regulärer Ausdruck), wobei die Übereinstimmung für eine mögliche Rückmeldung vermerkt wird

**(?:*re*)**

wie zuvor, aber ohne Rückmeldung (eine "nicht erfassende" Klammergruppe)

**()**

trifft eine leere Zeichenkette, vermerkt für eine mögliche Rückmeldung

**(?:)**

trifft eine leere Zeichenkette, ohne Rückmeldung

**[*chars*]**

ein *Klammerausdruck*, der genau eines der *chars* trifft (siehe Klammerausdrücke für Details)

**.**

trifft ein beliebiges einzelnes Zeichen

**\*k***

trifft das nicht-alphanumerische Zeichen *k* als gewöhnliches Zeichen, z. B. trifft **\\** ein Backslash-Zeichen

**\*c***

wobei *c* alphanumerisch ist (ggf. von weiteren Zeichen gefolgt), ein *Escape* (nur AREs), siehe Escapes unten

**{**

wenn von einem Zeichen gefolgt, das keine Ziffer ist, trifft es das linke geschweifte Klammerzeichen "**{**"; wenn von einer Ziffer gefolgt, ist es der Beginn einer *Grenze* (siehe oben)

***x***

wobei *x* ein einzelnes Zeichen ohne weitere Bedeutung ist, trifft dieses Zeichen.

## Einschränkungen

Eine *Einschränkung* trifft eine leere Zeichenkette, wenn bestimmte Bedingungen erfüllt sind. Auf eine Einschränkung darf kein Quantifizierer folgen. Die einfachen Einschränkungen sind wie folgt; weitere Einschränkungen werden später unter Escapes beschrieben.

**^**

trifft am Anfang der Zeichenkette oder einer Zeile (je nachdem, ob der Abgleich zeilenumbruch-sensitiv ist oder nicht, wie unter Abgleich unten beschrieben).

**$**

trifft am Ende der Zeichenkette oder einer Zeile (je nachdem, ob der Abgleich zeilenumbruch-sensitiv ist oder nicht, wie unter Abgleich unten beschrieben).

Der Unterschied zwischen String- und Zeilen-Abgleichmodus ist bedeutungslos, wenn die Zeichenkette kein Zeilenumbruchzeichen enthält. Die Einschränkungs-Escapes **\A** und **\Z** haben einen ähnlichen Zweck, sind aber stets Einschränkungen für die gesamte Zeichenkette.

Die Standard-Zeilenumbruch-Sensitivität hängt vom Befehl ab, der den regulären Ausdruck verwendet, und kann wie unter Metasyntax unten überschrieben werden.

**(?=*re*)**

*positives Lookahead* (nur AREs), trifft an jeder Stelle, an der eine Teilzeichenkette beginnt, die *re* trifft

**(?!*re*)**

*negatives Lookahead* (nur AREs), trifft an jeder Stelle, an der keine Teilzeichenkette beginnt, die *re* trifft

Die Lookahead-Einschränkungen dürfen keine Rückverweise enthalten (siehe später), und alle

Klammern darin gelten als nicht erfassend.

Ein RE darf nicht mit "\" enden.

## Klammerausdrücke

Ein *Klammerausdruck* ist eine Liste von Zeichen in "**[]**". Er trifft normalerweise ein beliebiges einzelnes Zeichen aus der Liste (siehe unten). Beginnt die Liste mit "**^**", trifft er ein beliebiges einzelnes Zeichen (siehe unten) *nicht* aus dem Rest der Liste.

Stehen zwei Zeichen in der Liste durch "**-**" getrennt, ist das eine Abkürzung für den vollen *Bereich* der Zeichen zwischen diesen beiden (einschließlich) in der Sortierfolge, z. B. trifft "**[0-9]**" in Unicode jede übliche Dezimalziffer. Zwei Bereiche dürfen keinen gemeinsamen Endpunkt haben, z. B. ist "**a-c-e**" unzulässig. Bereiche in Tcl verwenden stets die Unicode-Sortierfolge; andere Programme können andere Sortierfolgen verwenden — das kann zu Inkompatibilitäten zwischen Programmen führen.

Um ein wörtliches **]** oder **-** in die Liste aufzunehmen, umschließt man es am einfachsten mit **[.** und**.]**, um ein Sortierelement daraus zu machen (siehe unten). Alternativ macht man es zum ersten Zeichen (nach einem möglichen "**^**") oder (nur AREs) setzt "\" davor. Für "**-**" alternativ: letztes Zeichen oder zweiter Endpunkt eines Bereichs. Soll "**-**" wörtlich erster Endpunkt eines Bereichs sein, macht man es ein Sortierelement oder (nur AREs) setzt "\" davor. Mit Ausnahme dieser Fälle, einiger Kombinationen mit **[** (siehe nächste Absätze) und Escapes verlieren alle anderen Sonderzeichen in einem Klammerausdruck ihre Sonderbedeutung.

### Zeichenklassen

In einem Klammerausdruck steht der Name einer *Zeichenklasse* in **[:** und**:]** für die Liste aller Zeichen (nicht aller Sortierelemente!), die zu dieser Klasse gehören. Standard-Zeichenklassen sind:

**alpha**
ein Buchstabe.

**upper**
ein Großbuchstabe.

**lower**
ein Kleinbuchstabe.

**digit**
eine Dezimalziffer.

**xdigit**
eine hexadezimale Ziffer.

**alnum**
alphanumerisch (Buchstabe oder Ziffer).

**print**
"druckbar" (wie graph, zusätzlich Leerzeichen).

**blank**
Leerzeichen oder Tabulator.

**space**
ein Zeichen, das in angezeigtem Text Weißraum erzeugt.

**punct**

ein Interpunktionszeichen.

**graph**
ein Zeichen mit sichtbarer Darstellung (umfasst **alnum** und **punct**).

**cntrl**
ein Steuerzeichen.

Ein Gebietsschema kann weitere bereitstellen. Eine Zeichenklasse darf nicht Endpunkt eines Bereichs sein.

(*Hinweis:* die aktuelle Tcl-Implementierung hat nur ein Gebietsschema, das Unicode-Gebietsschema, das genau die obigen Klassen unterstützt.)

### Klammerausdrücke als Einschränkungen

Es gibt zwei Sonderfälle von Klammerausdrücken: die Klammerausdrücke "**[[:<:]]**" und "**[[:>:]]**" sind Einschränkungen und treffen leere Zeichenketten am Anfang bzw. Ende eines Wortes. Ein Wort ist eine Folge von Wortzeichen, der weder vorher noch nachher Wortzeichen folgen. Ein Wortzeichen ist ein *alnum*-Zeichen oder ein Unterstrich ("**_**"). Diese speziellen Klammerausdrücke sind veraltet; ARE-Nutzer sollten stattdessen Einschränkungs-Escapes verwenden (siehe unten).

### Sortierelemente

In einem Klammerausdruck steht ein Sortierelement (ein Zeichen, eine mehrzeichenige Folge, die wie ein einzelnes Zeichen sortiert, oder ein Sortierfolgen-Name für eines von beiden) in **[.** und**.]** für die Zeichenfolge dieses Sortierelements. Die Folge ist ein einzelnes Element der Liste des Klammerausdrucks. Ein Klammerausdruck in einem Gebietsschema mit mehrzeichenigen Sortierelementen kann daher mehr als ein Zeichen treffen. So kann (heimtückisch) ein Klammerausdruck, der mit **^** beginnt, mehrzeichenige Sortierelemente treffen, auch wenn keines in der Liste des Klammerausdrucks vorkommt!

(*Hinweis:* Tcl hat keine mehrzeichenigen Sortierelemente. Diese Information dient nur zur Veranschaulichung.)

Nehmen wir an, die Sortierfolge enthält ein mehrzeicheniges Sortierelement **ch**. Dann trifft der RE "**[[.ch.]]*c**" (null oder mehr "**ch**", gefolgt von "**c**") die ersten fünf Zeichen von "**chchcc**". Außerdem trifft der RE "**[^c]b**" ganz "**chb**" (weil "**[^c]**" das mehrzeichenige "**ch**" trifft).

### Äquivalenzklassen

In einem Klammerausdruck ist ein Sortierelement in **[=** und **=]** eine Äquivalenzklasse und steht für die Zeichenfolgen aller Sortierelemente, die diesem äquivalent sind, einschließlich es selbst. (Gibt es keine anderen äquivalenten Sortierelemente, wird behandelt, als ob die umschließenden Begrenzer "**[.**" und "**.]**" wären.) Sind z. B. **o** und **ô** Mitglieder einer Äquivalenzklasse, sind " **[[=o=]]**", "**[[=ô=]]**" und "**[oô]**" synonym. Eine Äquivalenzklasse darf nicht Endpunkt eines Bereichs sein.

(*Hinweis:* Tcl implementiert nur das Unicode-Gebietsschema. Es definiert keine Äquivalenzklassen. Die Beispiele oben dienen nur zur Veranschaulichung.)

## Escapes

Escapes (nur AREs), die mit \ gefolgt von einem alphanumerischen Zeichen beginnen, gibt es in mehreren Arten: Zeicheneingabe, Klassenkurzformen, Einschränkungs-Escapes und

Rückverweise. **** gefolgt von einem alphanumerischen Zeichen, das kein gültiges Escape bildet, ist in AREs unzulässig. In EREs gibt es keine Escapes: außerhalb eines Klammerausdrucks steht **** gefolgt von einem alphanumerischen Zeichen nur für dieses Zeichen als gewöhnliches Zeichen; innerhalb eines Klammerausdrucks ist **** ein gewöhnliches Zeichen. (Letzteres ist die einzige echte Inkompatibilität zwischen EREs und AREs.)

## Zeicheneingabe-Escapes

Zeicheneingabe-Escapes (nur AREs) erleichtern die Angabe nicht druckbarer und sonst unhandlicher Zeichen in REs:

**\a**
: Alarm (Glocke), wie in C

**\b**
: Rücktaste, wie in C

**\B**
: Synonym für ****, um doppeltes Backslash-Escaping in Anwendungen mit mehreren Ebenen von Backslash-Verarbeitung zu verringern

**\c*X***
: (wobei *X* ein beliebiges Zeichen ist) das Zeichen, dessen niederwertige 5 Bits mit denen von *X* übereinstimmen und dessen übrige Bits alle null sind

**\e**
: das Zeichen, dessen Sortierfolgen-Name "**ESC**" ist, oder andernfalls das Zeichen mit Oktalwert 033

**\f**
: Seitenvorschub, wie in C

**\n**
: Zeilenumbruch, wie in C

**\r**
: Wagenrücklauf, wie in C

**\t**
: horizontaler Tabulator, wie in C

**\u*wxyz***
: (wobei *wxyz* ein bis vier hexadezimale Ziffern sind) das Unicode-Zeichen **U+***wxyz* in der lokalen Byte-Reihenfolge

**\U*stuvwxyz***
: (wobei *stuvwxyz* ein bis acht hexadezimale Ziffern sind) reserviert für eine Unicode-Erweiterung bis 21 Bit. Die Ziffern werden gelesen, bis das erste nicht-hexadezimale Zeichen erreicht wird, maximal acht hexadezimale Ziffern erreicht sind oder ein Überlauf beim Maximalwert **U+***10ffff* auftreten würde.

**\v**
: vertikaler Tabulator, wie in C

**\x*hh***
: (wobei *hh* ein oder zwei hexadezimale Ziffern sind) das Zeichen mit hexadezimalem Wert **0x** *hh*.

**\0**

das Zeichen mit Wert **0**

***\xyz***

(wobei *xyz* genau drei Oktalziffern sind und kein *Rückverweis* (siehe unten)) das Zeichen mit Oktalwert **0***xyz*. Die erste Ziffer muss im Bereich 0–3 liegen, sonst gilt die zweistellige Form.

***\xy***

(wobei *xy* genau zwei Oktalziffern sind und kein *Rückverweis* (siehe unten)) das Zeichen mit Oktalwert **0***xy*

Hexadezimale Ziffern sind "**0**–**9**", "**a**–**f**" und "**A**–**F**". Oktalziffern sind "**0**–**7**".

Die Zeicheneingabe-Escapes gelten stets als gewöhnliche Zeichen. Z. B. ist **\135** in Unicode **]**, aber **\135** beendet keinen Klammerausdruck. Beachten Sie jedoch, dass manche Anwendungen (z. B. C-Compiler und der Tcl-Interpreter, wenn der reguläre Ausdruck nicht in geschweifte Klammern gesetzt ist) solche Folgen selbst verarbeiten, bevor das Regex-Paket sie sieht — das kann Verdoppelung (Vervierfachung usw.) von "\" erfordern.

## Klassenkurzformen

Klassenkurzformen (nur AREs) sind Abkürzungen für häufig verwendete Zeichenklassen:

**\d**

**[[:digit:]]**

**\s**

**[[:space:]]**

**\w**

**[[:alnum:]_\u203F\u2040\u2054\uFE33\uFE34\uFE4D\uFE4E\uFE4F\uFF3F]** (einschließlich Interpunktions-Verbindungszeichen)

**\D**

**[^[:digit:]]**

**\S**

**[^[:space:]]**

**\W**

**[^[:alnum:]_\u203F\u2040\u2054\uFE33\uFE34\uFE4D\uFE4E\uFE4F\uFF3F]** (einschließlich Interpunktions-Verbindungszeichen)

In Klammerausdrücken verlieren "**\d**", "**\s**" und "**\w**" ihre äußeren Klammern, und "**\D**", "**\S**" und "**\W**" sind unzulässig. (Z. B. ist "**[a-c\d]**" äquivalent zu "**[a-c[:digit:]]**". "**[a-c\D]**", äquivalent zu "**[a-c^[:digit:]]**", ist unzulässig.)

## Einschränkungs-Escapes

Ein Einschränkungs-Escape (nur AREs) ist eine Einschränkung, die die leere Zeichenkette trifft, wenn bestimmte Bedingungen erfüllt sind, geschrieben als Escape:

**\A**

trifft nur am Anfang der Zeichenkette (siehe Abgleich unten, wie sich das von "**^**" unterscheidet)

**\m**

trifft nur am Anfang eines Wortes

**\M**

trifft nur am Ende eines Wortes

**\y**
trifft nur am Anfang oder Ende eines Wortes

**\Y**
trifft nur an einer Stelle, die weder Wortanfang noch -ende ist

**\Z**
trifft nur am Ende der Zeichenkette (siehe Abgleich unten, wie sich das von "**$**" unterscheidet)

***m***
(wobei *m* eine von Null verschiedene Ziffer ist) ein *Rückverweis*, siehe unten

***mnn***
(wobei *m* eine von Null verschiedene Ziffer ist, *nn* weitere Ziffern, und der Dezimalwert *mnn* nicht größer ist als die bisher gesehene Anzahl schließender erfassender Klammern) ein *Rückverweis*, siehe unten

Ein Wort ist wie bei "**[[:<:]]**" und "**[[:>:]]**" oben definiert. Einschränkungs-Escapes sind in Klammerausdrücken unzulässig.

### Rückverweise

Ein Rückverweis (nur AREs) trifft dieselbe Zeichenkette wie der geklammerte Unterausdruck mit der angegebenen Nummer, sodass z. B. "**([bc])\1**" "**bb**" oder "**cc**", aber nicht "**bc**" trifft. Der Unterausdruck muss dem Rückverweis im RE vollständig vorausgehen. Unterausdrücke werden in der Reihenfolge ihrer führenden Klammern nummeriert. Nicht erfassende Klammern definieren keine Unterausdrücke.

Es gibt eine historische Mehrdeutigkeit zwischen oktalen Zeicheneingabe-Escapes und Rückverweisen, die heuristisch aufgelöst wird (siehe oben). Eine führende Null bedeutet stets ein Oktal-Escape. Eine einzelne von Null verschiedene Ziffer ohne weitere Ziffer danach ist stets ein Rückverweis. Eine mehrstellige Folge ohne führende Null ist ein Rückverweis, wenn sie auf einen passenden Unterausdruck folgt (d. h. die Nummer liegt im gültigen Bereich), andernfalls Oktal.

## Metasyntax

Zusätzlich zur Hauptsyntax oben gibt es Sonderformen und weitere syntaktische Einrichtungen.

Normalerweise wird die RE-Variante anwendungsabhängig festgelegt. Das kann durch einen *Direktor* überschrieben werden. Beginnt ein RE beliebiger Variante mit "*****:**", ist der Rest ein ARE. Beginnt ein RE beliebiger Variante mit "*****=**", gilt der Rest als wörtliche Zeichenkette; alle Zeichen sind gewöhnliche Zeichen.

Ein ARE kann mit *eingebetteten Optionen* beginnen: eine Folge **(?***xyz***)** (wobei *xyz* ein oder mehr Buchstaben sind) legt Optionen für den Rest des RE fest. Sie ergänzen und können Anwendungsoptionen überschreiben. Verfügbare Optionsbuchstaben:

**b**
Rest des RE ist ein BRE

**c**
groß-/kleinschreibungs-sensitiver Abgleich (üblicher Standard)

**e**
Rest des RE ist ein ERE

**i**

groß-/kleinschreibungs-insensitiver Abgleich (siehe Abgleich unten)

**m**

historisches Synonym für **n**

**n**

zeilenumbruch-sensitiver Abgleich (siehe Abgleich unten)

**p**

teilweise zeilenumbruch-sensitiver Abgleich (siehe Abgleich unten)

**q**

Rest des RE ist eine wörtliche ("quoted") Zeichenkette, alle Zeichen gewöhnlich

**s**

nicht zeilenumbruch-sensitiver Abgleich (üblicher Standard)

**t**

straffe Syntax (üblicher Standard; siehe unten)

**w**

invers teilweise zeilenumbruch-sensitiv ("weird") (siehe Abgleich unten)

**x**

erweiterte Syntax (siehe unten)

Eingebettete Optionen gelten ab der**)**, die die Folge beendet. Sie sind nur am Anfang eines ARE verfügbar und dürfen später im ARE nicht verwendet werden.

Neben der üblichen (*straffen*) RE-Syntax, in der alle Zeichen bedeutsam sind, gibt es eine *erweiterte* Syntax, in allen RE-Varianten mit dem Schalter **-expanded** oder in AREs mit der eingebetteten Option x. In der erweiterten Syntax werden Leerraumzeichen ignoriert und alle Zeichen zwischen **#** und dem folgenden Zeilenumbruch (oder dem Ende des RE) ignoriert — so lassen sich komplexe REs gliedern und kommentieren. Drei Ausnahmen:

- ein Leerraumzeichen oder "**#**" mit vorangehendem "\" bleibt erhalten
- Leerraum oder "**#**" innerhalb eines Klammerausdrucks bleibt erhalten
- Leerraum und Kommentare sind in mehrzeichenigen Symbolen wie dem ARE "**(?:**" oder dem BRE "**\(**" unzulässig

Leerraumzeichen der erweiterten Syntax sind Leerzeichen, Tabulator, Zeilenumbruch und jedes Zeichen der Zeichenklasse *space*.

Schließlich ist in einem ARE außerhalb von Klammerausdrücken die Folge "**(?#***ttt***)**" (wobei *ttt* beliebiger Text ohne "**)**" ist) ein Kommentar, der völlig ignoriert wird. Auch das ist nicht zwischen den Zeichen mehrzeicheniger Symbole wie "**(?:**" erlaubt. Solche Kommentare sind eher historisches Artefakt als nützliche Einrichtung; ihre Verwendung ist veraltet — stattdessen die erweiterte Syntax verwenden.

*Keine* dieser Metasyntax-Erweiterungen ist verfügbar, wenn die Anwendung (oder ein anfänglicher "*****=**"-Direktor) festlegt, dass die Eingabe als wörtliche Zeichenkette und nicht als RE behandelt wird.

## Abgleich

Kann ein RE mehr als eine Teilzeichenkette einer gegebenen Zeichenkette treffen, trifft der RE diejenige, die in der Zeichenkette am frühesten beginnt. Können an dieser Stelle mehrere Teilzeichenketten getroffen werden, bestimmt die *Präferenz* die Wahl: entweder die längste oder die kürzeste Teilzeichenkette.

Die meisten Atome und alle Einschränkungen haben keine Präferenz. Ein geklammerter RE hat dieselbe Präferenz (ggf. keine) wie der RE. Ein quantifiziertes Atom mit Quantifizierer **{***m***}** oder **{***m***}?** hat dieselbe Präferenz (ggf. keine) wie das Atom selbst. Ein quantifiziertes Atom mit anderen normalen Quantifizierern (einschließlich **{***m***,***n***}** mit *m* gleich *n*) bevorzugt die längste Übereinstimmung. Ein quantifiziertes Atom mit anderen nicht-gierigen Quantifizierern (einschließlich **{***m***,***n***}?** mit *m* gleich *n*) bevorzugt die kürzeste. Ein Zweig hat dieselbe Präferenz wie das erste quantifizierte Atom darin mit Präferenz. Ein RE aus zwei oder mehr durch **|** verbundenen Zweigen bevorzugt die längste Übereinstimmung.

Unter den Einschränkungen der Regeln für den Abgleich des gesamten RE treffen Unterausdrücke ebenfalls die längstmöglichen oder kürzestmöglichen Teilzeichenketten gemäß ihrer Präferenz; Unterausdrücke, die im RE früher beginnen, haben Vorrang vor späteren. Äußere Unterausdrücke haben damit Vorrang vor ihren Bestandteilen.

Die Quantifizierer **{1,1}** und **{1,1}?** können verwendet werden, um auf einen Unterausdruck oder ein ganzes RE jeweils längste bzw. kürzeste Präferenz zu erzwingen.

**HINWEIS:** Man kann einen RE meist insgesamt nicht-gierig machen, indem man **{1,1}?** nach einem der ersten Nicht-Einschränkungs-Atome oder geklammerten Unterausdrücke setzt. *Es lohnt sich zu experimentieren* mit der Platzierung dieser Nicht-Gierigkeits-Überschreibung an geeigneten Eingabetexten, wenn man einen RE dieser Komplexität schreibt.

Z. B. ist dieser reguläre Ausdruck nicht-gierig und trifft die kürzestmögliche Teilzeichenkette, wobei "**abc**" so früh wie möglich getroffen wird (der Quantifizierer ändert das nicht):

```
ab{1,1}?c.*x.*cba
```

Das Atom "**a**" hat keine Gierigkeits-Präferenz; für "**b**" setzen wir eine explizit; die übrigen Quantifizierer werden durch den vorangehenden nicht-gierigen Quantifizierer auf nicht-gierig überschrieben.

Trefferlängen werden in Zeichen, nicht in Sortierelementen gemessen. Eine leere Zeichenkette gilt als länger als gar kein Treffer. Z. B. trifft "**bb***" die drei mittleren Zeichen von "**abbbc**", "**(week|wee)(night|knights)**" alle zehn Zeichen von "**weeknights**", bei "**(.*).***" gegen "**abc**" trifft der geklammerte Unterausdruck alle drei Zeichen, und bei "**(a*)***" gegen "**bc**" treffen sowohl der gesamte RE als auch der geklammerte Unterausdruck eine leere Zeichenkette.

Ist groß-/kleinschreibungs-unabhängiger Abgleich angegeben, wirkt es etwa so, als ob alle Groß-/Kleinschreibungs-Unterschiede aus dem Alphabet verschwunden wären. Erscheint ein Buchstabe in mehreren Schreibungen außerhalb eines Klammerausdrucks als gewöhnliches Zeichen, wird er effektiv in einen Klammerausdruck mit beiden Schreibungen verwandelt, sodass **x** zu "**[xX]**" wird. Erscheint er innerhalb eines Klammerausdrucks, werden alle Schreibvarianten hinzugefügt, sodass "**[x]**" zu "**[xX]**" und "**[^x]**" zu "**[^xX]**" wird.

Ist zeilenumbruch-sensitiver Abgleich angegeben, treffen**.** und Klammerausdrücke mit **^** nie das Zeilenumbruchzeichen (Treffer überschreiten Zeilenumbrüche nicht, es sei denn, der RE ordnet es ausdrücklich an), und **^** sowie **$** treffen jeweils die leere Zeichenkette nach bzw. vor einem Zeilenumbruch zusätzlich zu Anfang und Ende der Zeichenkette. ARE **\A** und **\Z** treffen weiterhin *nur* Anfang bzw. Ende der Zeichenkette.

Bei teilweise zeilenumbruch-sensitivem Abgleich betrifft das**.** und Klammerausdrücke wie beim zeilenumbruch-sensitiven Abgleich, aber nicht **^** und **$**.

Bei invers teilweise zeilenumbruch-sensitivem Abgleich betrifft das **^** und **$** wie beim zeilenumbruch-sensitiven Abgleich, aber nicht**.** und Klammerausdrücke. Das ist wenig nützlich, wird aber der Symmetrie wegen bereitgestellt.

## Grenzen und Kompatibilität

Für die Länge von REs gibt es keine besondere Grenze. Hoch portable Programme sollten keine REs länger als 256 Byte verwenden, da eine POSIX-konforme Implementierung solche REs ablehnen darf.

Das einzige Merkmal von AREs, das tatsächlich mit POSIX-EREs inkompatibel ist: **** verliert in Klammerausdrücken nicht seine Sonderbedeutung. Alle anderen ARE-Merkmale nutzen Syntax, die in POSIX-EREs unzulässig ist oder undefinierte bzw. nicht spezifizierte Wirkung hat; die "******* "-Syntax der Direktoren liegt ebenfalls außerhalb der POSIX-Syntax für BREs und EREs.

Viele ARE-Erweiterungen stammen aus Perl; einige wurden bereinigt, einige Perl-Erweiterungen fehlen. Bemerkenswerte Inkompatibilitäten: "**\b**", "**\B**", fehlende Sonderbehandlung eines abschließenden Zeilenumbruchs, Ergänzung komplementierter Klammerausdrücke bei zeilenumbruch-sensitivem Abgleich, Einschränkungen für Klammern und Rückverweise in Lookahead-Einschränkungen sowie längster/kürzester-Treffer-Semantik (statt erster Treffer).

Die Abgleichregeln für REs mit normalen und nicht-gierigen Quantifizierern haben sich seit frühen Beta-Versionen dieses Pakets geändert. (Die neuen Regeln sind einfacher und klarer, raten aber weniger die wirklichen Absichten des Nutzers.)

Henry Spencers ursprüngliches *regexp*-Paket von 1986, noch weit verbreitet (z. B. in Tcl vor 8.1), implementierte eine frühe Version der heutigen EREs. Es gibt vier Inkompatibilitäten zwischen den nahe-EREs von *regexp* ("RREs") und AREs, in etwa steigender Bedeutung:

- In AREs ist **** gefolgt von einem alphanumerischen Zeichen entweder ein Escape oder ein Fehler; in RREs war es nur eine andere Schreibweise des alphanumerischen Zeichens. Das sollte kein Problem sein, da es keinen Grund gab, solche Folgen in RREs zu schreiben.
- **{** gefolgt von einer Ziffer in einem ARE ist der Beginn einer Grenze; in RREs war **{** stets ein gewöhnliches Zeichen. Solche Folgen sollten selten sein und führen oft zu einem Fehler, weil folgende Zeichen nicht wie eine gültige Grenze aussehen.
- In AREs bleibt **** innerhalb von "**[]**" ein Sonderzeichen; ein wörtliches **** in **[]** muss "**\**" geschrieben werden. **\** liefert in RREs ebenfalls ein wörtliches **** in **[]**, aber nur wirklich Paranoide verdoppelten routinemäßig den Backslash.
- AREs melden den längsten/kürzesten Treffer für den RE, nicht den ersten in einer festgelegten Suchreihenfolge. Das kann RREs betreffen, die darauf ausgelegt waren, dass der erste Treffer gemeldet wird. (Die sorgfältige Ausgestaltung von RREs zur Optimierung der Suchreihenfolge ist obsolet — AREs prüfen alle möglichen Treffer parallel, ihre Leistung hängt wenig von der Komplexität ab — Fälle, in denen die Suchreihenfolge genutzt wurde, um absichtlich einen Treffer zu finden, der *nicht* der längste/kürzeste ist, müssen umgeschrieben werden.)

## Einfache reguläre Ausdrücke

BREs unterscheiden sich in mehreren Punkten von EREs. "**|**", "**+**" und**?** sind gewöhnliche Zeichen; es gibt kein Äquivalent für ihre Funktion. Die Begrenzer für Grenzen sind **\{** und "**\}**", **{** und **}** allein sind gewöhnliche Zeichen. Die Klammern für verschachtelte Unterausdrücke sind **\(** und "**\)**", **(** und**)** allein sind gewöhnliche Zeichen. **^** ist ein gewöhnliches Zeichen außer am Anfang des RE oder am Anfang eines geklammerten Unterausdrucks, **$** außer am Ende des RE oder am Ende eines geklammerten Unterausdrucks, und ***** ist ein gewöhnliches Zeichen, wenn es am Anfang des RE oder am Anfang eines geklammerten Unterausdrucks steht (nach einem möglichen führenden "**^**"). Schließlich gibt es einstellige Rückverweise, und **\<** sowie **\>** sind Synonyme für "**[[:<:]]**" bzw. "**[[:>:]]**"; weitere Escapes gibt es nicht.

*Copyright (c) 1998 Sun Microsystems, Inc.*
*Copyright (c) 1999 Scriptics Corporation*

# read

*Stichwörter:* read

## Name

read - Aus einem Kanal lesen

## Übersicht

**read** ?**-nonewline**? *channel*
**read** *channel numChars*

## Beschreibung

Der Befehl **read** wurde durch chan read abgelöst, der dieselbe Syntax und dieselben Optionen unterstützt.

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# refchan

*Stichwörter:* refchan, API, channel, ensemble, prefix, reflection

## Name

refchan - API des Befehlshandlers für in Tcl-Code implementierte Kanäle

## Übersicht

**chan create** *mode cmdPrefix*

*cmdPrefix* **blocking** *channel mode*
*cmdPrefix* **cget** *channel option*
*cmdPrefix* **cgetall** *channel*
*cmdPrefix* **configure** *channel option value*
*cmdPrefix* **finalize** *channel*
*cmdPrefix* **initialize** *channel mode*
*cmdPrefix* **read** *channel count*
*cmdPrefix* **seek** *channel offset base*
*cmdPrefix* **watch** *channel eventspec*
*cmdPrefix* **write** *channel data*

## Beschreibung

Der Tcl-Ebenen-Handler für einen reflektierten Kanal muss ein Befehl mit Unterbefehlen sein (als *Ensemble* bezeichnet, wie ein Befehl, der durch namespace ensemble **create** erzeugt wird; die Implementierung von Handlern für reflektierte Kanäle ist jedoch in keiner Weise an namespace ensemble gebunden; siehe Beispiele unten, wie man eine oo::class aufbaut, die die API unterstützt). *cmdPrefix* ist das, was beim Aufruf von chan create angegeben wurde, und kann aus mehreren Argumenten bestehen; es wird an Stelle des Präfixes zu mehreren Wörtern expandiert.

Von allen möglichen Unterbefehlen muss der Handler **initialize**, **finalize** und **watch** unterstützen. Die Unterstützung der übrigen Unterbefehle ist optional.

## Pflicht-Unterbefehle

***cmdPrefix* initialize *channel mode***

Ein Aufruf dieses Unterbefehls ist der erste Aufruf, den *cmdPrefix* für den angegebenen neuen *channel* erhält. Dieser Unterbefehl ist dafür verantwortlich, alle internen Datenstrukturen einzurichten, die zum Verfolgen des Kanals und seines Zustands erforderlich sind.

Der Rückgabewert der Methode muss eine Liste mit den Namen aller von *cmdPrefix* unterstützten Unterbefehle sein. Damit teilt er dem Tcl-Kern mit, welche Version der API für reflektierte Kanäle von diesem Befehlshandler verwendet wird.

Jeder von der Methode geworfene Fehler bricht die Erzeugung des Kanals ab, und es wird kein Kanal erzeugt. Der geworfene Fehler erscheint als vom Befehl chan create geworfener Fehler. Jede andere Ausnahme als error (z. B. break usw.) wird als Fehler behandelt (und in einen Fehler umgewandelt).

**Hinweis:** Wurde die Erzeugung des Kanals hier wegen Fehlern abgebrochen, wird der Unterbefehl **finalize** nicht aufgerufen.

Das Argument *mode* teilt dem Handler mit, ob der Kanal zum Lesen, Schreiben oder beides geöffnet wurde. Es ist eine Liste, die beliebig die Zeichenketten read oder **write** enthalten kann. Die Liste kann leer sein, enthält aber üblicherweise mindestens ein Element.

Der Unterbefehl muss einen Fehler werfen, wenn der gewählte Modus von *cmdPrefix* nicht unterstützt wird.

***cmdPrefix* finalize *channel***

Ein Aufruf dieses Unterbefehls ist der letzte Aufruf, den *cmdPrefix* für den angegebenen *channel* erhält. Er wird unmittelbar vor der Zerstörung der vom Tcl-Kern gehaltenen Datenstrukturen des Kanals erzeugt. Der Befehlshandler darf auf den *channel* in keiner Weise mehr zugreifen. Wenn dieser Unterbefehl aufgerufen wird, müssen alle diesem Kanal zugeordneten internen Ressourcen freigegeben werden.

Der Rückgabewert dieses Unterbefehls wird ignoriert.

Wirft der Unterbefehl einen Fehler, erscheint es, als hätte der Aufruf, der seine Ausführung auslöste (üblicherweise chan close), diesen Fehler geworfen. Jede Ausnahme außer error (z. B. break usw.) wird als Fehler behandelt (und in einen Fehler umgewandelt).

Dieser Unterbefehl wird nicht aufgerufen, wenn die Erzeugung des Kanals während **initialize** abgebrochen wurde (siehe oben).

***cmdPrefix* watch *channel eventspec***

Dieser Unterbefehl benachrichtigt *cmdPrefix*, dass der angegebene *channel* an den in *eventspec* aufgeführten Ereignissen interessiert ist. Dieses Argument ist eine Liste, die beliebig read und **write** enthalten kann. Die Liste kann leer sein; das signalisiert, dass der Kanal über keine Ereignisse benachrichtigt werden möchte. In diesem Fall soll der Handler die Ereigniserzeugung vollständig deaktivieren.

**Warnung:** Jeder Rückgabewert des Unterbefehls wird ignoriert. Das schließt alle vom Unterbefehl geworfenen Fehler, break, continue und benutzerdefinierte Rückgabecodes ein.

Dieser Unterbefehl interagiert mit chan postevent. Der Versuch, ein Ereignis zu melden, das im letzten Aufruf von **watch** nicht aufgeführt war, führt dazu, dass chan postevent einen Fehler wirft.

## Optionale Unterbefehle

### *cmdPrefix* read *channel count*

Dieser *optionale* Unterbefehl wird aufgerufen, wenn der Benutzer Daten vom Kanal *channel* anfordert. *count* gibt an, wie viele *Bytes* angefordert wurden. Wird der Unterbefehl nicht unterstützt, ist ein Lesen vom durch den Befehl verwalteten Kanal nicht möglich.

Der Rückgabewert dieses Unterbefehls wird als die angeforderten Daten-*Bytes* interpretiert. Enthält die zurückgegebenen Daten mehr Bytes als angefordert, wird ein Fehler signalisiert und später vom Befehl geworfen, der das Lesen ausgeführt hat (üblicherweise gets oder read ). Weniger Bytes als angefordert zurückzugeben ist jedoch zulässig.

Beachten Sie, dass das Zurückgeben von nichts (0 Bytes) den höheren Schichten signalisiert, dass auf dem Kanal **EOF** erreicht wurde. Um zu signalisieren, dass der Kanal derzeit keine Daten hat, **EOF** aber noch nicht erreicht ist, muss der Fehler "EAGAIN" geworfen werden, also entweder

```
return -code error EAGAIN
```

oder

```
error EAGAIN
```

Zur Erweiterbarkeit bewirkt jeder Fehler, dessen Wert eine negative Ganzzahl ist, dass die höheren Schichten die C-Variable "**errno**" auf den Absolutwert dieser Zahl setzen und so einen Systemfehler signalisieren. Beachten Sie jedoch, dass die genaue Zuordnung zwischen diesen Fehlernummern und ihrer Bedeutung vom Betriebssystem abhängt.

Beispielsweise sind unter Linux sowohl

```
return -code error -11
```

als auch

```
error -11
```

den obigen Beispielen äquivalent, wenn die lesbarere Zeichenkette "EAGAIN" verwendet wird; unter BSD ist das nicht der Fall, dort ist die äquivalente Zahl -35.

Die symbolische Zeichenkette ist jedoch systemübergreifend dieselbe und wird intern in die richtige Zahl übersetzt. Kein anderer Fehlerwert hat eine solche Zuordnung zu einer symbolischen Zeichenkette.

Wirft der Unterbefehl einen anderen Fehler, erscheint es, als hätte der Aufruf, der seine Ausführung auslöste (üblicherweise gets oder read), diesen Fehler geworfen. Jede Ausnahme außer error (z. B. break usw.) wird als Fehler behandelt und in einen Fehler umgewandelt.

***cmdPrefix* write *channel data***

Dieser *optionale* Unterbefehl wird aufgerufen, wenn der Benutzer Daten in den Kanal *channel* schreibt. Das Argument *data* enthält *Bytes*, keine Zeichen. Jede für den Kanal konfigurierte Transformation (EOL, Kodierung) wurde zu diesem Zeitpunkt bereits angewendet. Wird dieser Unterbefehl nicht unterstützt, ist ein Schreiben in den durch den Befehl verwalteten Kanal nicht möglich.

Der Rückgabewert des Unterbefehls wird als die vom Kanal geschriebene Byte-Anzahl interpretiert. Jeder nicht numerische Wert führt zu einem Fehler, der später vom Befehl geworfen wird, der das Schreiben ausgeführt hat. Ein negativer Wert bedeutet, dass das Schreiben fehlgeschlagen ist. Ein Wert größer als die dem Handler übergebenen Bytes oder Null ist verboten und führt dazu, dass der Tcl-Kern einen Fehler wirft.

Um zu signalisieren, dass der Kanal derzeit keine Daten zum Schreiben annehmen kann, muss der Fehler "EAGAIN" geworfen werden, also entweder

```
return -code error EAGAIN
```

oder

```
error EAGAIN
```

Zur Erweiterbarkeit bewirkt jeder Fehler, dessen Wert eine negative Ganzzahl ist, dass die höheren Schichten die C-Variable "**errno**" auf den Absolutwert dieser Zahl setzen und so einen Systemfehler signalisieren. Beachten Sie jedoch, dass die genaue Zuordnung zwischen diesen Fehlernummern und ihrer Bedeutung vom Betriebssystem abhängt.

Beispielsweise sind unter Linux sowohl

```
return -code error -11
```

als auch

```
error -11
```

den obigen Beispielen äquivalent, wenn die lesbarere Zeichenkette "EAGAIN" verwendet wird; unter BSD ist das nicht der Fall, dort ist die äquivalente Zahl -35.

Die symbolische Zeichenkette ist jedoch systemübergreifend dieselbe und wird intern in die richtige Zahl übersetzt. Kein anderer Fehlerwert hat eine solche Zuordnung zu einer symbolischen Zeichenkette.

Wirft der Unterbefehl einen anderen Fehler, erscheint es, als hätte der Aufruf, der seine Ausführung auslöste (üblicherweise puts), diesen Fehler geworfen. Jede Ausnahme außer error (z. B. break usw.) wird als Fehler behandelt und in einen Fehler umgewandelt.

***cmdPrefix* seek *channel offset base***

Dieser *optionale* Unterbefehl ist für die Bearbeitung von chan seek- und chan tell-Anfragen auf dem Kanal *channel* zuständig. Wird er nicht unterstützt, ist Suchen auf dem Kanal nicht möglich.

Das Argument *base* entspricht dem gleichnamigen Argument des eingebauten chan seek, nämlich:

**start**

Die Positionierung ist relativ zum Anfang des Kanals.

**current**

Die Positionierung ist relativ zur aktuellen Suchposition.

**end**

Die Positionierung ist relativ zum Ende des Kanals.

*offset* ist eine Ganzzahl, die angibt, um wie viele **Bytes** vor- oder zurückgesucht werden soll. Eine positive Zahl soll vorwärts suchen, eine negative rückwärts. Ein Kanal kann nur eingeschränktes Suchen anbieten. Sockets können beispielsweise nur vorwärts suchen, nicht rückwärts.

Der Rückgabewert des Unterbefehls wird als die (neue) Position des Kanals vom Anfang an gezählt genommen. Das muss eine Ganzzahl größer oder gleich null sein. Wirft der Unterbefehl einen Fehler, erscheint es, als hätte der Aufruf, der seine Ausführung auslöste (üblicherweise chan seek oder chan tell), diesen Fehler geworfen. Jede Ausnahme außer error (z. B. break usw.) wird als Fehler behandelt und in einen Fehler umgewandelt.

Die Kombination *offset*/*base* 0/**current** signalisiert eine chan tell-Anfrage, d. h. relativ zur aktuellen Position nichts zu suchen, sodass die neue Position mit der aktuellen identisch ist, die dann zurückgegeben wird.

***cmdPrefix* configure *channel option value***

Dieser *optionale* Unterbefehl dient zum Setzen der typspezifischen Optionen des Kanals *channel*. Das Argument *option* gibt die zu schreibende Option an, das Argument *value* den Wert, auf den die Option gesetzt werden soll.

Dieser Unterbefehl versucht niemals, mehr als eine Option gleichzeitig zu aktualisieren; dieses Verhalten ist im Tcl-Kanal-Kern implementiert.

Der Rückgabewert des Unterbefehls wird ignoriert.

Wirft der Unterbefehl einen Fehler, erscheint es, als hätte der Befehl, der die (Neu-)Konfiguration oder Abfrage ausgeführt hat (üblicherweise fconfigure oder chan configure), diesen Fehler geworfen. Jede Ausnahme außer error (z. B. break usw.) wird als Fehler behandelt und in einen Fehler umgewandelt.

***cmdPrefix* cget *channel option***

Dieser *optionale* Unterbefehl wird beim Lesen einer einzelnen typspezifischen Option des Kanals *channel* verwendet. Wird dieser Unterbefehl unterstützt, muss auch der Unterbefehl **cgetall** unterstützt werden.

Der Unterbefehl soll den Wert der angegebenen *option* zurückgeben.

Wirft der Unterbefehl einen Fehler, erscheint es, als hätte der Befehl, der die (Neu-)Konfiguration oder Abfrage ausgeführt hat (üblicherweise fconfigure oder chan configure), diesen Fehler geworfen. Jede Ausnahme außer *error* (z. B. break usw.) wird als Fehler behandelt und in einen Fehler umgewandelt.

***cmdPrefix* cgetall *channel***

Dieser *optionale* Unterbefehl dient zum Lesen aller typspezifischen Optionen des Kanals *channel*. Wird dieser Unterbefehl unterstützt, muss auch der Unterbefehl **cget** unterstützt werden.

Der Unterbefehl soll eine Liste aller Optionen und ihrer Werte zurückgeben. Diese Liste muss eine gerade Anzahl von Elementen haben.

Wirft der Unterbefehl einen Fehler, erscheint es, als hätte der Befehl, der die (Neu-)Konfiguration oder Abfrage ausgeführt hat (üblicherweise fconfigure oder chan configure), diesen Fehler geworfen. Jede Ausnahme außer error (z. B. break usw.) wird als Fehler behandelt und in einen Fehler umgewandelt.

***cmdPrefix* blocking *channel mode***

Dieser *optionale* Unterbefehl behandelt Änderungen am Blockiermodus des Kanals *channel*. *mode* ist ein boolesches Flag. Ein wahrer Wert bedeutet, dass der Kanal blockierend gesetzt werden muss, ein falscher Wert, dass der Kanal nicht blockierend sein soll.

Der Rückgabewert des Unterbefehls wird ignoriert.

Wirft der Unterbefehl einen Fehler, erscheint es, als hätte der Aufruf, der seine Ausführung auslöste (üblicherweise fconfigure oder chan configure), diesen Fehler geworfen. Jede Ausnahme außer error (z. B. break usw.) wird als Fehler behandelt und in einen Fehler umgewandelt.

***cmdPrefix* truncate *channel length***

Dieser *optionale* Unterbefehl behandelt die Änderung der Länge des zugrunde liegenden Datenstroms für den Kanal *channel*. Seine Länge wird auf *length* gesetzt.

Wirft der Unterbefehl einen Fehler, erscheint es, als hätte der Aufruf, der seine Ausführung auslöste (üblicherweise chan truncate), diesen Fehler geworfen. Jede Ausnahme außer error (z. B. break usw.) wird als Fehler behandelt und in einen Fehler umgewandelt.

## Hinweise

Einige der in Kanälen unterstützten Funktionen, die in der C-Schnittstelle von Tcl definiert sind, stehen Kanälen, die auf die Tcl-Ebene reflektiert werden, nicht zur Verfügung.

Die Funktion **Tcl_DriverGetHandleProc** wird nicht unterstützt; d. h. reflektierte Kanäle haben keine betriebssystemspezifischen Handles.

Die Funktion **Tcl_DriverHandlerProc** wird nicht unterstützt. Diese Treiberfunktion ist nur für gestapelte Kanäle, d. h. Transformationen, relevant. Reflektierte Kanäle sind stets Basiskanäle, keine Transformationen.

Die Funktion **Tcl_DriverFlushProc** wird nicht unterstützt. Das liegt daran, dass die aktuelle generische E/A-Schicht von Tcl diese Funktion nirgends verwendet. Daher ergibt Unterstützung auf Tcl-Ebene ebenfalls keinen Sinn. Das kann in Zukunft geändert werden (durch Erweiterung der hier definierten API und Änderung ihrer Versionsnummer), falls die Funktion künftig verwendet wird.

## Beispiele

Dies zeigt, wie man einen Kanal erzeugt, der aus einer Zeichenkette liest.

```
oo::class create stringchan {
    variable data pos
    constructor {string {encoding {}}} {
        if {$encoding eq ""} {set encoding [encoding system]}
        set data [encoding convertto $encoding $string]
        set pos 0
    }

    method initialize {ch mode} {
        return "initialize finalize watch read seek"
    }
    method finalize {ch} {
        my destroy
    }
    method watch {ch events} {
        # Muss vorhanden sein, wird aber ignoriert, weil wir
        # keine Ereignisse melden
    }

    # Muss bei einem lesbaren Kanal vorhanden sein
    method read {ch count} {
        set d [string range $data $pos [expr {$pos+$count-1}]]
        incr pos [string length $d]
        return $d
    }

    # Diese Methode ist optional, für das Beispiel unten aber nützlich
    method seek {ch offset base} {
        switch $base {
            start {
                set pos $offset
            }
            current {
                incr pos $offset
            }
            end {
                set pos [string length $data]
                incr pos $offset
            }
        }
        if {$pos < 0} {
            set pos 0
        } elseif {$pos > [string length $data]} {
            set pos [string length $data]
        }
        return $pos
    }
}

# Jetzt erzeugen wir eine Instanz...
set string "The quick brown fox jumps over the lazy dog.\n"
set ch [chan create read [stringchan new $string]]

puts [gets $ch];   # Gibt die ganze Zeichenkette aus

seek $ch -5 end;
puts [read $ch];   # Gibt nur das letzte Wort aus
```

*Copyright (c) 2006 Andreas Kupries* andreas_kupries@users.sourceforge.net

# regexp

*Stichwörter:* regexp, match, parsing, pattern, regular expression, splitting, string

## Name

regexp - Regulären Ausdruck gegen eine Zeichenkette prüfen

## Übersicht

**regexp** ?*switches*? *exp string* ?*matchVar*? ?*subMatchVar subMatchVar* ...?

## Beschreibung

Prüft, ob der reguläre Ausdruck *exp* einen Teil oder die gesamte *string* trifft, und gibt 1 zurück, wenn ja, sonst 0 — außer **-inline** ist angegeben (siehe unten). (Reguläre Ausdrücke werden auf der Referenzseite re_syntax beschrieben.)

Werden nach *string* weitere Argumente angegeben, gelten sie als Namen von Variablen, in denen Informationen darüber zurückgegeben wird, welche Teil(e) von *string* zu *exp* passten. *matchVar* erhält den Bereich von *string*, der ganz zu *exp* passte. Die erste *subMatchVar* enthält die Zeichen in *string*, die zur linken geklammerten Unterausdrucks in *exp* passten, die nächste *subMatchVar* die zum nächsten geklammerten Unterausdruck rechts davon usw.

Beginnen die ersten Argumente von **regexp** mit **-**, werden sie als Schalter behandelt (Abkürzungen erlaubt). Unterstützt werden:

**-about**

Statt zu matchen wird eine Liste mit Informationen über den regulären Ausdruck zurückgegeben. Erstes Element: Anzahl der Unterausdrücke. Zweites: Liste von Eigenschaftsnamen zu Attributen des Ausdrucks. Vor allem für Debugging.

**-expanded**

Erweiterte RE-Syntax: Leerraum und Kommentare werden ignoriert. Entspricht der eingebetteten Option **(?x)** (siehe re_syntax).

**-indices**

Ändert, was in *matchVar* und *subMatchVar* gespeichert wird: statt der passenden Zeichen je zwei Dezimalstrings mit den Indizes des ersten und letzten Zeichens des Trefferbereichs in *string*.

**-line**

Zeilenumbruch-sensitives Matching. Standard: Zeilenumbruch ist ein normales Zeichen. Mit diesem Flag: "^"-Klammerausdrücke und "." treffen keinen Zeilenumbruch; "^" zusätzlich leere Zeichenkette nach jedem Zeilenumbruch; "$" zusätzlich davor. Entspricht -linestop und -lineanchor zusammen oder (?n) (siehe [re_syntax).

**-linestop**

"^" und "." stoppen an Zeilenumbrüchen. Wie (?p) (siehe [re_syntax).

**-lineanchor**

"^" und "$" (Anker) treffen Zeilenanfang bzw. -ende. Wie **(?w)** (siehe re_syntax).

**-nocase**

Großbuchstaben in *string* werden beim Matching wie Kleinbuchstaben behandelt.

**-all**

So oft wie möglich matchen; Rückgabe der Gesamtzahl der Treffer. Mit Match-Variablen: nur Informationen zum letzten Treffer.

**-inline**

Gibt als Liste die Daten zurück, die sonst in Match-Variablen landen würden. Mit **-inline** keine Match-Variablen. Mit **-all** wird die Liste pro Iteration angehängt (flache Liste). Pro Treffer: Gesamttreffer plus je ein Element pro Unterausdruck. Beispiele:

```
regexp -inline -- {\w(\w)} " inlined "
      → in n
regexp -all -inline -- {\w(\w)} " inlined "
      → in n li i ne e
```

**-start *index***

Zeichenindex-Offset in *string*, ab dem gematcht wird (wie string index). Mit diesem Schalter trifft "^" nicht den Zeilenanfang; \A trifft weiter den Stringanfang bei *index*. Mit **-indices** Indizes relativ zum absoluten Anfang von *string*. *index* wird auf die Stringgrenzen beschränkt.

**--**

Ende der Schalter; das folgende Argument gilt als *exp*, auch wenn es mit **-** beginnt.

Gibt es mehr *subMatchVar* als geklammerte Unterausdrücke in *exp*, oder trifft ein Unterausdruck nicht (weil er in einem nicht gematchten Teil lag), wird die *subMatchVar* auf "**-1 -1**" gesetzt bei **-indices**, sonst auf leeren String.

## Beispiele

Erstes Vorkommen eines Worts, das mit **foo** beginnt, aber kein **foobar** ist; Buchstaben danach bis Wortende in eine Variable:

```
regexp {\mfoo(?!bar\M)(\w*)} $string -> restOfWord
```

Die gesamte Trefferzeichenkette liegt in "**->**" — ein Name, der gut aussieht, obwohl der Inhalt uninteressant ist.

Index des Worts **badger** (beliebige Schreibweise) in **location**:

```
regexp -indices {(?i)\mbadger\M} $string location
```

Als *basic* regular expression (statt Standard *advanced*):

```
regexp -indices {(?ib)\<badger\>} $string location
```

Anzahl der Oktalziffern in einer Zeichenkette:

```
regexp -all {[0-7]} $string
```

Alle Wörter (Nicht-Leerzeichen-Sequenzen) als Liste — mächtigere Variante von split:

```
regexp -all -inline {\S+} $string
```

*Copyright (c) 1998 Sun Microsystems, Inc.*

# registry

*Stichwörter:* registry

## Name

registry - Windows-Registrierung manipulieren

## Übersicht

**package require registry 1.3**

**registry** ?*-mode*? *option keyName* ?*arg arg* ...?

## Beschreibung

Das Paket **registry** stellt einen allgemeinen Satz von Operationen zum Manipulieren der Windows-Registrierung bereit. Das Paket implementiert den Tcl-Befehl **registry**. Dieser Befehl wird nur auf der Windows-Plattform unterstützt. Warnung: Dieser Befehl sollte mit Vorsicht verwendet werden, da eine beschädigte Registrierung das System unbrauchbar machen kann.

*KeyName* ist der Name eines Registrierungsschlüssels. Registrierungsschlüssel müssen eine der folgenden Formen haben:

\*hostname**rootname**keypath*

*rootname**keypath*

*rootname*

*Hostname* gibt den Namen eines beliebigen gültigen Windows-Hosts an, der seine Registrierung exportiert. Die Komponente *rootname* muss eine von **HKEY_LOCAL_MACHINE**, **HKEY_USERS**, **HKEY_CLASSES_ROOT**, **HKEY_CURRENT_USER**, **HKEY_CURRENT_CONFIG**, **HKEY_PERFORMANCE_DATA** oder **HKEY_DYN_DATA** sein. Der *keypath* kann aus einem oder mehreren Registrierungsschlüsselnamen bestehen, die durch Backslash-Zeichen (\) getrennt sind.

Das optionale Argument *-mode* gibt an, mit welcher Registrierung gearbeitet werden soll; bei **-32bit** wird die 32-Bit-Registrierung verwendet, bei **-64bit** die 64-Bit-Registrierung. Wenn dieses

Argument weggelassen wird, ist die Standard-Registrierung des Systems Gegenstand der angeforderten Operation.

*Option* gibt an, was mit dem Registrierungsschlüsselnamen geschehen soll. Jede eindeutige Abkürzung für *option* ist zulässig. Die gültigen Optionen sind:

**registry broadcast *keyName* ?*-timeout milliseconds*?**

Sendet eine Broadcast-Nachricht an das System und laufende Programme, um sie über bestimmte Aktualisierungen zu informieren. Dies ist notwendig, um Änderungen an wichtigen Registrierungsschlüsseln wie Environment zu propagieren. Der Timeout gibt die Wartezeit in Millisekunden an, bis Anwendungen auf die Broadcast-Nachricht reagieren. Standardwert ist 3000. Das folgende Beispiel zeigt, wie ein Pfad zur globalen Umgebungsvariable hinzugefügt und Anwendungen über die Änderung informiert werden kann, ohne Ab- und Anmelden zu erfordern (setzt Administratorrechte voraus):

```
set regPath [join {
    HKEY_LOCAL_MACHINE
    SYSTEM
    CurrentControlSet
    Control
    {Session Manager}
    Environment
} "\\"]
set curPath [registry get $regPath "Path"]
registry set $regPath "Path" "$curPath;$addPath"
registry broadcast "Environment"
```

**registry delete *keyName* ?*valueName*?**

Wenn das optionale Argument *valueName* angegeben ist, wird der angegebene Wert unter *keyName* aus der Registrierung gelöscht. Wenn das optionale *valueName* weggelassen wird, werden der angegebene Schlüssel sowie alle Unterschlüssel oder Werte darunter in der Registrierungshierarchie gelöscht. Wenn der Schlüssel nicht gelöscht werden konnte, wird ein Fehler erzeugt. Wenn der Schlüssel nicht existierte, hat der Befehl keine Wirkung.

**registry get *keyName valueName***

Gibt die mit dem Wert *valueName* unter dem Schlüssel *keyName* verknüpften Daten zurück. Wenn entweder der Schlüssel oder der Wert nicht existiert, wird ein Fehler erzeugt. Weitere Details zum Format der zurückgegebenen Daten siehe Unterstützte Typen weiter unten.

**registry keys *keyName* ?*pattern*?**

Wenn *pattern* nicht angegeben ist, gibt eine Liste der Namen aller Unterschlüssel von *keyName* zurück. Wenn *pattern* angegeben ist, werden nur die Namen zurückgegeben, die auf *pattern* passen. Die Übereinstimmung wird nach denselben Regeln wie bei string match bestimmt. Wenn der angegebene *keyName* nicht existiert, wird ein Fehler erzeugt.

**registry set *keyName* ?*valueName data* ?*type*?**

Wenn *valueName* nicht angegeben ist, wird der Schlüssel *keyName* erstellt, falls er noch nicht existiert. Wenn *valueName* angegeben ist, werden der Schlüssel *keyName* und der Wert *valueName* bei Bedarf erstellt. Der Inhalt von *valueName* wird auf *data* mit dem durch *type* angegebenen Typ gesetzt. Wenn *type* nicht angegeben ist, wird der Typ **sz** angenommen. Weitere Details zu den Argumenten *data* und *type* siehe Unterstützte Typen weiter unten.

**registry type *keyName valueName***

Gibt den Typ des Werts *valueName* im Schlüssel *keyName* zurück. Weitere Informationen zu den möglichen Typen siehe Unterstützte Typen weiter unten.

**registry values *keyName* ?*pattern*?**

Wenn *pattern* nicht angegeben ist, gibt eine Liste der Namen aller Werte von *keyName* zurück. Wenn *pattern* angegeben ist, werden nur die Namen zurückgegeben, die auf *pattern* passen. Die Übereinstimmung wird nach denselben Regeln wie bei string match bestimmt.

## Unterstützte Typen

Jeder Wert unter einem Schlüssel in der Registrierung enthält Daten eines bestimmten Typs in einer typspezifischen Darstellung. Der Befehl **registry** konvertiert zwischen dieser internen Darstellung und einer, die von Tcl-Skripten manipuliert werden kann. In den meisten Fällen werden die Daten einfach als Tcl-String zurückgegeben. Der Typ gibt die beabsichtigte Verwendung der Daten an, ändert aber nicht die Darstellung selbst. Für einige Typen gibt der Befehl **registry** die Daten in einer anderen Form zurück, um die Manipulation zu erleichtern. Die folgenden Typen werden vom Befehl **registry** erkannt:

binary

Der Registrierungswert enthält beliebige Binärdaten. Die Daten werden in Tcl exakt dargestellt, einschließlich eingebetteter Nullen.

**none**

Der Registrierungswert enthält beliebige Binärdaten ohne definierten Typ. Die Daten werden in Tcl exakt dargestellt, einschließlich eingebetteter Nullen.

**sz**

Der Registrierungswert enthält eine nullterminierte Zeichenkette. Die Daten werden in Tcl als String dargestellt.

**expand_sz**

Der Registrierungswert enthält eine nullterminierte Zeichenkette mit nicht expandierten Verweisen auf Umgebungsvariablen im üblichen Windows-Stil (zum Beispiel "%PATH%"). Die Daten werden in Tcl als String dargestellt.

**dword**

Der Registrierungswert enthält eine 32-Bit-Zahl in Little-Endian-Darstellung. Die Daten werden in Tcl als Dezimalstring dargestellt.

**dword_big_endian**

Der Registrierungswert enthält eine 32-Bit-Zahl in Big-Endian-Darstellung. Die Daten werden in Tcl als Dezimalstring dargestellt.

link

Der Registrierungswert enthält einen symbolischen Link. Die Daten werden in Tcl exakt dargestellt, einschließlich eingebetteter Nullen.

**multi_sz**

Der Registrierungswert enthält ein Array nullterminierter Zeichenketten. Die Daten werden in Tcl als Liste von Strings dargestellt.

**resource_list**

Der Registrierungswert enthält eine Gerätetreiber-Ressourcenliste. Die Daten werden in Tcl exakt dargestellt, einschließlich eingebetteter Nullen.

Zusätzlich zu den oben symbolisch benannten Typen werden unbekannte Typen durch eine 32-Bit-Ganzzahl identifiziert, die dem von den System-Schnittstellen zurückgegebenen Typcode entspricht. In diesem Fall werden die Daten in Tcl exakt dargestellt, einschließlich eingebetteter Nullen.

### Portabilität

Der Befehl **registry** ist nur unter Windows verfügbar.

### Beispiele

Gibt aus, wie durch Doppelklick auf eine Tcl-Skriptdatei ein Tcl-Interpreter aufgerufen wird:

```
package require registry
set ext .tcl

# Typnamen lesen
set type [registry get HKEY_CLASSES_ROOT\\$ext {}]
# Ermitteln, wo der Befehl zu finden ist
set path HKEY_CLASSES_ROOT\\$type\\Shell\\Open\\command
# Befehl lesen!
set command [registry get $path {}]

puts "$ext opens with $command"
```

*Copyright (c) 1997 Sun Microsystems, Inc.*
*Copyright (c) 2002 ActiveState Corporation.*

## regsub

*Stichwörter:* regsub, match, pattern, quoting, regular expression, substitution

### Name

regsub - Ersetzungen auf Basis regulärer Ausdrücke

### Übersicht

**regsub** ?*switches*? *exp string subSpec* ?*varName*?

### Beschreibung

Matcht den regulären Ausdruck *exp* gegen *string* und kopiert *string* entweder in die Variable *varName* oder gibt *string* zurück, wenn *varName* fehlt. (Reguläre Ausdrücke: Referenzseite re_syntax.) Bei einem Treffer wird beim Kopieren von *string* nach *varName* (bzw. als Rückgabewert ohne *varName*) der zu *exp* passende Teil von *string* durch *subSpec* ersetzt. Enthält *subSpec* "&" oder "\0", wird es in der Ersetzung durch den zu *exp* passenden Teil von *string* ersetzt. Enthält *subSpec* "*n*", wobei *n* eine Ziffer von 1 bis 9 ist, wird es durch den Teil von *string* ersetzt, der zum *n*. geklammerten Unterausdruck von *exp* passte. Zusätzliche Backslashes in *subSpec* verhindern die Sonderbedeutung von "&", "\0", "*n*" und Backslashes. Backslashes in *subSpec* kollidieren oft mit der Tcl-Parser-Verarbeitung; *subSpec* daher am besten in geschweifte Klammern setzen, wenn Backslashes vorkommen.

Beginnen die ersten Argumente von **regsub** mit **-**, gelten sie als Schalter. Unterstützt werden:

**-all**

Alle in *string* zu *exp* passenden Bereiche werden gefunden und ersetzt. Ohne diesen Schalter nur der erste Treffer. Mit **-all** werden "&"- und "\*n*"-Sequenzen pro Ersetzung mit den Daten des jeweiligen Treffers aufgelöst.

**-command**

subSpec ist kein Ersetzungs-Template mehr; "&" und "\n" haben keine Sonderbedeutung. subSpec muss ein Befehlspräfix sein (nicht-leere Liste). Der zu exp passende Teil von string und dann je der Treffer jedes erfassenden Unter-RE innerhalb von exp werden als weitere Listenelemente angehängt (ähnlich regexp -inline). Die fertige Liste wird als Tcl-Befehl ausgewertet; dessen Ergebnis ist die Ersetzungszeichenkette. Fehler oder Ausnahmen aus der Befehlsauswertung werden zu Fehlern/Ausnahmen von regsub.

Ohne **-all** wird der Callback höchstens einmal aufgerufen (genau wenn der Ausdruck matcht). Mit **-all** für jede Trefferposition der Reihe nach. Die exakten Indexpositionen der Treffer stehen dem Skript nicht zur Verfügung.

Siehe Beispiele unten.

**-expanded**

Erweiterte RE-Syntax: Leerraum und Kommentare werden ignoriert. Entspricht **(?x)** (siehe re_syntax).

**-line**

Zeilenumbruch-sensitives Matching. Standard: Zeilenumbruch ist ein normales Zeichen. Mit diesem Flag: "[^"-Klammerausdrücke und "." treffen keinen Zeilenumbruch; "^" zusätzlich leere Zeichenkette nach jedem Zeilenumbruch; "$" zusätzlich davor. Entspricht **-linestop** und **-lineanchor** zusammen oder **(?n)** (siehe re_syntax).

**-linestop**

"[^" und "." stoppen an Zeilenumbrüchen. Wie **(?p)** (siehe re_syntax).

**-lineanchor**

"^" und "$" (Anker) treffen Zeilenanfang bzw. -ende. Wie **(?w)** (siehe re_syntax).

**-nocase**

Großbuchstaben in *string* werden vor dem Match gegen *exp* in Kleinbuchstaben umgewandelt; Ersetzungen aus *subSpec* beziehen sich aber auf die unveränderte Form von *string*.

**-start *index***

Zeichenindex-Offset in *string*, ab dem gematcht wird (wie string index). Mit diesem Schalter trifft "^" nicht den Zeilenanfang; \A trifft weiter den Stringanfang bei *index*. *index* wird auf die Stringgrenzen beschränkt.

**--**

Ende der Schalter; das folgende Argument gilt als *exp*, auch wenn es mit **-** beginnt.

Ist *varName* angegeben, gibt der Befehl die Anzahl der gefundenen und ersetzten Trefferbereiche zurück, sonst die Zeichenkette nach der Ersetzung. Details zur Interpretation regulärer Ausdrücke: Manpage regexp.

## Beispiele

Jedes eigenständige Wort **foo** in der Variable *string* durch **bar** ersetzen:

```
regsub -all {\mfoo\M} $string bar string
```

oder (Syntax *basic regular expression*):

```
regsub -all {(?b)\<foo\>} $string bar string
```

Das erste Vorkommen des Worts **interesting** (beliebige Schreibweise) in Anführungszeichen setzen:

```
regsub -nocase {\yinteresting\y} $string {"&"} string
```

Alle Nicht-ASCII- und Tcl-relevanten Zeichen per **regsub** und subst in \u-Escapes umwandeln:

```
# This RE is just a character class for almost everything "bad"
set RE {[][{};#\\\$ \r\t\u0080-\uffff]}

# We will substitute with a fragment of Tcl script in brackets
set substitution {[format \\\\u%04x [scan "\\&" %c]]}

# Now we apply the substitution to get a subst-string that
# will perform the computational parts of the conversion. Note
# that newline is handled specially through string map since
# backslash-newline is a special sequence.
set quoted [subst [string map {\n {\\u000a}} \
        [regsub -all $RE $string $substitution]]]
```

Dasselbe lässt sich mit regsub -command erledigen, oft schneller. (Eine vollständig vorberechnete string map wäre noch schneller; der Aufwand für die Map bei so komplexer Transformation kann aber groß sein.)

```
# This RE is just a character class for everything "bad"
set RE {[][{};#\\\$\s\u0080-\uffff]}

# This encodes what the RE described above matches
proc encodeChar {ch} {
    # newline is handled specially since backslash-newline is a
    # special sequence.
    if {$ch eq "\n"} {
        return "\\u000a"
    }
    # No point in writing this as a one-liner
    scan $ch %c charNumber
    format "\\u%04x" $charNumber
}

set quoted [regsub -all -command $RE $string encodeChar]
```

URL-kodierte Zeichenkette dekodieren mit **regsub -command**, Lambda und apply:

```
# Match one of the sequences in a URL-encoded string that needs
# fixing, converting + to space and %XX to the right character
# (e.g., %7e becomes ~)
set RE {(\+)|%([0-9A-Fa-f]{2})}

# Note that -command uses a command prefix, not a command name
set decoded [regsub -all -command $RE $string {apply {{- p h} {
    # + is a special case; handle directly
    if {$p eq "+"} {
        return " "
    }
    # convert hex to a char
    scan $h %x charNumber
    format %c $charNumber
}}}]
```

Die Option **-command** eignet sich auch, um den Wirkungsbereich von Befehlen wie string totitle einzuschränken:

```
set message "the quIck broWn fOX JUmped oVer the laZy dogS..."
puts [regsub -all -command {\w+} $message {string totitle}]
# → The Quick Brown Fox Jumped Over The Lazy Dogs..
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2000 Scriptics Corporation.*

## rename

*Stichwörter:* rename, command, delete, namespace

### Name

rename - Benennt einen Befehl um oder löscht ihn

### Übersicht

**rename** *oldName newName*

### Beschreibung

Benennt den Befehl *oldName* in *newName* um. Ist *newName* eine leere Zeichenkette, wird *oldName* gelöscht. *oldName* und *newName* dürfen Namespace-Qualifizierer enthalten. Wird ein Befehl in einen anderen Namespace umbenannt, laufen künftige Aufrufe in diesem Namespace. **rename** gibt eine leere Zeichenkette zurück.

### Beispiele

**rename** kann Standardbefehle mit eigener Überwachung umschließen — z. B. zählen, wie oft source aufgerufen wird:

```
rename ::source ::theRealSource
set sourceCount 0
proc ::source args {
    global sourceCount
    puts "called source for the [incr sourceCount]'th time"
    uplevel 1 ::theRealSource $args
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# return

*Stichwörter:* return, break, catch, continue, error, exception, procedure, result

## Name

return - Kehrt aus einer Prozedur zurück oder setzt den Rückgabecode eines Skripts

## Übersicht

**return** ?*result*?
**return** ?*-code code*? ?*result*?
**return** ?*option value* ...? ?*result*?

## Beschreibung

In der einfachsten Form beendet **return** ohne Optionen im Prozedurrumpf sofort die Ausführung und gibt die Kontrolle an den Aufrufer zurück. Mit *result* wird dieser Wert zum Ergebnis der Prozedur; ohne *result* eine leere Zeichenkette.

In von source ausgewerteten Skriptdateien beendet **return** die Skriptauswertung; *result* (oder eine leere Zeichenkette) wird Ergebnis von source.

## Ausnahmerückgabecodes

Mit **-code** kann die Prozedur außergewöhnlich zurückkehren. *code* kann sein:

**ok (oder 0)**
Normaler Return; Rückgabecode 0 (TCL_OK).

error **(oder 1)**
Fehler-Return; Rückgabecode 1 (TCL_ERROR). Der Prozedurbefehl verhält sich im Aufrufkontext wie error *result*. Weitere Optionen siehe unten.

**return (oder 2)**
Rückgabecode 2 (TCL_RETURN); Verhalten wie **return** ohne Argumente.

break **(oder 3)**
Rückgabecode 3 (TCL_BREAK); Verhalten wie break.

continue **(oder 4)**
Rückgabecode 4 (TCL_CONTINUE); Verhalten wie continue.

***value***
Ganzzahl als Rückgabecode (Anwendungen/Pakete: 5–1073741823; außerhalb reserviert für Tcl).

Ungültige Argumente vom Aufrufer melden mit **return -code error** und passender Meldung in *result*. **return -code** dient sonst vor allem neuen Kontrollstrukturen.

**return -code** in von source ausgewerteten Dateien setzt den Rückgabecode von source auf *code*.

## Rückgabeoptionen

Neben Ergebnis und Rückgabecode liefert jede Befehlsauswertung ein Dictionary von Rückgabeoptionen. Alle *option value*-Paare von **return** werden Einträge (mit Ausnahmen unten). catch kann Rückgabecode, Ergebnis und Dictionary erfassen.

**-code** wird von Tcl speziell behandelt. Weitere spezielle Optionen:

**-errorcode *list***

Nur bei **-code** TCL_ERROR. Zusatzinformation als Tcl-Liste für Programme. Ohne Angabe bei **-code error**: Standard **NONE**; auch in errorCode.

**-errorinfo *info***

Nur bei TCL_ERROR. Dann ist *info* der anfängliche Stack-Trace mit Zusatzinformationen zum Fehlerkontext für Menschen; er wird auch in errorInfo gespeichert. Ohne **-errorinfo** bei **-code error** setzt Tcl einen eigenen anfänglichen Stack-Trace: nur der Prozeduraufruf; beim Unwinden kommen höhere Ebenen hinzu, nicht der Kontext innerhalb der Prozedur. Typisch stammt *info* aus **-errorinfo** eines von catch erfassten Rückgabeoptionen-Dictionarys (oder aus errorInfo).

**-errorstack *list***

Nur bei TCL_ERROR. Dann ist *list* der anfängliche Fehler-Stack mit den tatsächlich übergebenen Argumentwerten je Prozedurebene; erreichbar auch über info errorstack. Ohne **-errorstack** bei **-code error** liefert Tcl einen eigenen anfänglichen Fehler-Stack (nur Prozeduraufruf; Unwinden ergänzt höhere Ebenen). Typisch stammt *list* aus **-errorstack** eines von catch erfassten Dictionarys (oder von info errorstack).

**-level *level***

**-level** und **-code** setzen gemeinsam den Rückgabecode eines der gerade ausgewerteten Befehle. *level* ist eine nicht-negative Ganzzahl: Anzahl Ebenen auf dem Aufrufstack, auf denen der Rückgabecode *code* gelten soll. Ohne **-level** ist der Standard 1 — **return** setzt den Code, den die aktuelle Prozedur an ihren Aufrufer zurückgibt. Details siehe unten.

**-options *options***

Gültiges Dictionary; Einträge werden zusätzliche *option value*-Paare für **return**.

## Rückgabecode-Behandlung

Rückgabecodes steuern den Programmfluss in Tcl. Ein Skript ist eine Folge von Befehlen. Liefert jede Auswertung TCL_OK, geht es zum nächsten Befehl. Jeder außergewöhnliche Code (ungleich TCL_OK) unterbricht den weiteren Ablauf: die Skriptauswertung endet, und der Code wird Ergebnis der gesamten Auswertung. So führen Fehler zum Unterbrechen und Unwinden des Aufrufstapels; so bewirken break, continue und **return**, dass nicht alle Befehle nacheinander laufen.

Einige eingebaute Befehle werten Skripte aus und nutzen außergewöhnliche Codes für Sonderfunktionen. Schleifenbefehle wie while, for und foreach werten den Schleifenrumpf aus; liefert der Rumpf TCL_BREAK oder TCL_CONTINUE, reagieren sie so, dass break und continue in Schleifen ihre dokumentierte Wirkung haben.

Prozeduraufrufe werten den Prozedurrumpf aus. Liefert der Rumpf TCL_RETURN, wird der Eintrag **-level** im Rückgabeoptionen-Dictionary dekrementiert. Ist **-level** danach 0, wird **-code** zum Rückgabecode der Prozedur. Ist **-level** danach größer als 0, bleibt der Prozedur-Rückgabecode TCL_RETURN. Erfolgte der Aufruf während der Auswertung einer anderen Prozedur, wiederholt sich der Vorgang aufwärts: **-level** wird je Ebene dekrementiert, bis *code* der Rückgabecode des Befehls *level* Ebenen höher ist. source behandelt TCL_RETURN analog — daher ähnelt **return** in source dem **return** in einer Prozedur.

Der Rückgabecode des **return**-Befehls selbst löst dieselbe Sonderbehandlung aus. Mit **return -level 0** ist der Rückgabecode des **return**-Befehls *code* aus **-code** (Standard TCL_OK). Jeder andere **-level**-Wert (einschließlich Standard 1) macht den Rückgabecode des **return**-Befehls zu TCL_RETURN und löst Return aus der umgebenden Prozedur aus.

## Beispiele

Vorzeitiger Return aus einer Prozedur:

```
proc printOneLine {} {
    puts "line 1"    ;# wird ausgegeben
    return
    puts "line 2"    ;# wird nicht ausgegeben
}
```

Rückgabewert setzen:

```
proc returnX {} {return X}
puts [returnX]    ;# gibt X aus
```

Ungültige Argumente mit **return -code error**:

```
proc factorial {n} {
    if {![string is integer $n] || ($n < 0)} {
        return -code error \
                "expected non-negative integer,\
                but got \"$n\""
    }
    if {$n < 2} {
        return 1
    }
    set factor [factorial [expr {$n - 1}]]
    set product [expr {$n * $factor}]
    return $product
}
```

Ersatz für break:

```
proc myBreak {} {
    return -code break
}
```

Mit **return -level 0** und interp alias als break-Ersatz:

```
interp alias {} Break {} return -level 0 -code break
```

Fehler mit catch und **return -options** erneut auslösen:

```
proc doSomething {} {
    set resource [allocate]
    catch {
        # Long script of operations
        # that might raise an error
    } result options
    deallocate $resource
    return -options $options $result
}
```

Erweitert: Prozedur-Ersatz für **return** selbst:

```
proc myReturn {args} {
    set result ""
    if {[llength $args] % 2} {
        set result [lindex $args end]
        set args [lrange $args 0 end-1]
    }
    set options [dict merge {-level 1} $args]
    dict incr options -level
    return -options $options $result
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# safe

*Stichwörter:* safe, alias, auto-loading, auto_mkindex, load, parent interpreter, safe interpreter, child interpreter, source

## Name

safe - Erstellt und manipuliert sichere Interpreter

## Übersicht

**::safe::interpCreate** ?*child*? ?*options...*?
**::safe::interpInit** *child* ?*options...*?
**::safe::interpConfigure** *child* ?*options...*?
**::safe::interpDelete** *child*
**::safe::interpAddToAccessPath** *child directory*
**::safe::interpFindInAccessPath** *child directory*
**::safe::setSyncMode** ?*newValue*?
**::safe::setLogCmd** ?*cmd arg...*?
?-accessPath *pathList?*
?-autoPath *pathList?*

?-statics *boolean?*
?-noStatics?
?-nested *boolean?*
?-nestedLoadOk?
?-deleteHook *script?*

## Beschreibung

Safe Tcl ist ein Mechanismus zum sicheren Ausführen nicht vertrauenswürdiger Tcl-Skripte und zur vermittelten Bereitstellung potenziell gefährlicher Funktionalität für solche Skripte.

Safe Tcl stellt sicher, dass nicht vertrauenswürdige Tcl-Skripte der hostenden Anwendung keinen Schaden zufügen können. Es verhindert Angriffe auf Integrität und Vertraulichkeit. Nicht vertrauenswürdige Tcl-Skripte können den Zustand der hostenden Anwendung oder des Computers nicht beschädigen. Nicht vertrauenswürdige Skripte können außerdem keine auf dem hostenden Computer oder in der hostenden Anwendung gespeicherten Informationen an Dritte weitergeben.

Safe Tcl ermöglicht einem Eltern-Interpreter, sichere, eingeschränkte Interpreter zu erzeugen, die eine vordefinierte Menge von Aliasen für die Befehle source, load, file, encoding und exit enthalten und die Auto-Load- und Paketmechanismen nutzen können.

Dem sicheren Interpreter werden keine Kenntnisse über die Struktur des Dateisystems preisgegeben, weil er nur Zugriff auf einen virtualisierten Pfad mit Token hat. Wenn der sichere Interpreter eine Datei sourcen möchte, verwendet er das Token im virtuellen Pfad als Teil des zu sourcenden Dateinamens; der Eltern-Interpreter übersetzt das Token transparent in einen echten Verzeichnisnamen und führt die angeforderte Operation aus (siehe den Abschnitt Sicherheit unten). Verschiedene Sicherheitsstufen können über die optionalen Schalter der unten beschriebenen Befehle gewählt werden.

Alle von Safe Tcl im Eltern-Interpreter bereitgestellten Befehle befinden sich im Namespace **safe**
.

## Befehle

Im Eltern-Interpreter stehen folgende Befehle zur Verfügung:

**::safe::interpCreate ?*child*? ?*options...*?**

Erstellt einen sicheren Interpreter, installiert die im Abschnitt Aliasse beschriebenen Aliasen und initialisiert den Auto-Load- und Paketmechanismus gemäß den übergebenen *options*. Siehe den Abschnitt Optionen unten für eine Beschreibung der optionalen Argumente. Wird das Argument *child* weggelassen, wird ein Name erzeugt. **::safe::interpCreate** gibt stets den Interpreter-Namen zurück.

Der Interpreter-Name *child* darf Namespace-Trennzeichen enthalten, jedoch weder führende noch nachgestellte Namespace-Trennzeichen und keine überzähligen Doppelpunkte in Namespace-Trennzeichen. Der Interpreter-Name ist relativ zum globalen Namespace :: qualifiziert, nicht relativ zum Namespace, in dem der Befehl **::safe::interpCreate** ausgewertet wird.

**::safe::interpInit *child* ?*options...*?**

Dieser Befehl ähnelt **interpCreate**, erzeugt den sicheren Interpreter jedoch nicht. *child* muss auf andere Weise erzeugt worden sein, z. B. mit interp create **-safe**. Der Interpreter-Name *child* darf Namespace-Trennzeichen enthalten, unter denselben Einschränkungen wie bei **interpCreate**.

**::safe::interpConfigure *child* ?*options...*?**

Werden keine *options* angegeben, gibt der Befehl die Einstellungen aller Optionen für den benannten sicheren Interpreter als Liste von Optionen und deren aktuellen Werten für dieses *child* zurück. Wird genau ein weiteres Argument übergeben, liefert er eine Liste mit 2 Elementen *name* und *value*, wobei *name* der vollständige Name dieser Option ist und *value* der aktuelle Wert für diese Option und das *child*. Werden mehr als zwei weitere Argumente übergeben, wird der sichere Interpreter neu konfiguriert und ausschließlich die angegebenen Optionen geändert. Siehe den Abschnitt Optionen unten für die Optionsbeschreibung. Beispiel:

```
# Neuen Interp mit derselben Konfiguration wie "$i0" erzeugen:
set i1 [safe::interpCreate {*}[safe::interpConfigure $i0]]

# Aktuellen deleteHook abfragen
set dh [safe::interpConfigure $i0  -del]

# Nur das Attribut statics loading ok eines Interpreters und seinen
# deleteHook ändern (Rest unverändert lassen):
safe::interpConfigure $i0  -delete {foo bar} -statics 0
```

**::safe::interpDelete *child***

Löscht den sicheren Interpreter und räumt die zugehörigen Datenstrukturen des Eltern-Interpreters auf. Wurde für diesen Interpreter ein *deleteHook*-Skript angegeben, wird es vor dem Löschen des Interpreters mit dem Namen des Interpreters als zusätzlichem Argument ausgewertet.

**::safe::interpFindInAccessPath *child directory***

Dieser Befehl sucht und gibt das Token für das reale Verzeichnis *directory* im aktuellen virtuellen Zugriffspfad des sicheren Interpreters zurück. Er erzeugt einen Fehler, wenn das Verzeichnis nicht gefunden wird. Beispiel:

```
$child eval [list set tk_library \
      [::safe::interpFindInAccessPath $name $tk_library]]
```

**::safe::interpAddToAccessPath *child directory***

Dieser Befehl fügt *directory* dem im Eltern-Interpreter für den sicheren Interpreter verwalteten virtuellen Pfad hinzu und gibt das Token zurück, mit dem der sichere Interpreter auf Dateien in diesem Verzeichnis zugreifen kann. Befindet sich das Verzeichnis bereits im virtuellen Pfad, wird nur das Token zurückgegeben, ohne das Verzeichnis erneut hinzuzufügen. Beispiel:

```
$child eval [list set tk_library \
      [::safe::interpAddToAccessPath $name $tk_library]]
```

**::safe::setSyncMode ?*newValue*?**

Dieser Befehl liest oder setzt den "Sync Mode" der Safe Base. Wird ein Argument übergeben, liefert der Befehl einen Fehler, wenn das Argument kein boolescher Wert ist oder wenn

Safe-Base-Interpreter existieren. Typischerweise wird der Wert bei der Initialisierung gesetzt — boolesch **true** für "Sync Mode" ein (Standard), **false** für "Sync Mode" aus. Bei eingeschaltetem "Sync Mode" hält die Safe Base den ::auto_path jedes Kind-Interpreters mit seinem Zugriffspfad synchron. Siehe den Abschnitt Sync-Modus unten.

**::safe::setLogCmd ?*cmd arg...*?**

Dieser Befehl installiert ein Skript, das bei interessanten Ereignissen im Lebenszyklus eines sicheren Interpreters aufgerufen wird. Ohne Argumente gibt er das aktuell installierte Skript zurück. Mit einem Argument, der leeren Zeichenkette, wird das aktuell installierte Skript entfernt und die Protokollierung abgeschaltet. Das Skript wird mit einem zusätzlichen Argument aufgerufen, einer Zeichenkette, die das Ereignis beschreibt. Der Hauptzweck ist die Unterstützung beim Debuggen sicherer Interpreter. Mit dieser Einrichtung können vollständige Fehlermeldungen erhalten werden, während der sichere Interpreter nur generische Fehlermeldungen erhält. So kann ein sicherer Interpreter keine Meldungen über Fehler und andere Ereignisse sehen, die sensible Informationen wie echte Verzeichnisnamen enthalten könnten.

Beispiel:

```
::safe::setLogCmd puts stderr
```

Unten steht die Ausgabe einer Beispielsitzung, in der ein sicherer Interpreter versuchte, eine Datei zu sourcen, die nicht in seinem virtuellen Zugriffspfad gefunden wurde. Beachten Sie, dass der sichere Interpreter nur eine Fehlermeldung erhielt, dass die Datei nicht gefunden wurde:

```
NOTICE for child interp10 : Created
NOTICE for child interp10 : Setting accessPath=(/foo/bar)
    staticsok=1 nestedok=0 deletehook=()
NOTICE for child interp10 : auto_path in interp10 has been set to
    {$p(:0:)}
ERROR for child interp10 : /foo/bar/init.tcl: no such file or
    directory
```

## Optionen

Die folgenden Optionen gelten gemeinsam für **::safe::interpCreate**, **::safe::interpInit** und **::safe::interpConfigure**. Jeder Optionsname kann auf seinen minimalen eindeutigen Namen abgekürzt werden. Optionsnamen sind nicht case-sensitiv.

**-accessPath *directoryList***

Diese Option legt die Liste der Verzeichnisse fest, aus denen der sichere Interpreter Dateien sourcen und loaden darf. Wird die Option nicht angegeben oder als leere Liste übergeben, verwendet der sichere Interpreter dieselben Verzeichnisse wie sein Eltern-Interpreter für Auto-Loading. Siehe den Abschnitt Sicherheit unten für Details zu virtuellen Pfaden, Token und Zugriffskontrolle.

**-autoPath *directoryList***

Diese Option legt die Liste der Verzeichnisse im ::auto_path des sicheren Interpreters fest. Die Option ist undefiniert, wenn die Safe Base den "Sync Mode" eingeschaltet hat — dann wird der ::auto_path des sicheren Interpreters von der Safe Base verwaltet und ist eine tokenisierte

Form seines Zugriffspfads. Siehe den Abschnitt Sync-Modus unten.

**-statics *boolean***

Diese Option legt fest, ob der sichere Interpreter statisch verlinkte Pakete laden darf (wie **load {} Tk**). Der Standardwert ist **true**: sichere Interpreter dürfen statisch verlinkte Pakete laden.

**-noStatics**

Diese Option ist eine Kurzform für **-statics false** und legt fest, dass der sichere Interpreter keine statisch verlinkten Pakete laden darf.

**-nested *boolean***

Diese Option legt fest, ob der sichere Interpreter Pakete in seine eigenen Unter-Interpreter laden darf. Der Standardwert ist **false**: sichere Interpreter dürfen keine Pakete in ihre eigenen Unter-Interpreter laden.

**-nestedLoadOk**

Diese Option ist eine Kurzform für **-nested true** und legt fest, dass der sichere Interpreter Pakete in seine eigenen Unter-Interpreter laden darf.

**-deleteHook *script***

Wird diese Option mit einem nicht-leeren *script* angegeben, wird es im Eltern-Interpreter kurz vor dem tatsächlichen Löschen des sicheren Interpreters mit dem Namen des sicheren Interpreters als zusätzlichem Argument ausgewertet. Ein leerer Wert entfernt jedes aktuell installierte Lösch-Hook-Skript für diesen sicheren Interpreter. Der Standardwert (**{}**) bedeutet: kein Lösch-Rückruf.

## Aliasse

In einem sicheren Interpreter stehen folgende Aliasen zur Verfügung:

source ***fileName***

Die angeforderte Datei, eine Tcl-Quelldatei, wird in den sicheren Interpreter gesourct, wenn sie gefunden wird. Der Alias source kann nur Dateien aus Verzeichnissen im virtuellen Pfad des sicheren Interpreters sourcen. Der Alias source verlangt, dass der sichere Interpreter eines der Token-Namen in seinem virtuellen Pfad verwendet, um das Verzeichnis anzugeben, in dem die zu sourcende Datei liegt. Siehe den Abschnitt Sicherheit für weitere Einschränkungen gültiger Dateinamen.

load ***fileName***

Die angeforderte Datei, eine Shared-Object-Datei, wird dynamisch in den sicheren Interpreter geladen, wenn sie gefunden wird. Der Dateiname muss ein im virtuellen Pfad des sicheren Interpreters genanntes Token enthalten, damit das Laden erfolgreich ist. Zusätzlich muss die Shared-Object-Datei einen sicheren Einstiegspunkt enthalten; siehe die Manpage des Befehls load für Details.

file **?*subcommand args...*?**

Der Alias file bietet Zugriff auf eine sichere Teilmenge der Unterbefehle des Befehls file; er erlaubt nur die Unterbefehle **dirname**, **join**, **extension**, **root**, **tail**, **pathtype** und **split**. Für Details zu diesen Unterbefehlen siehe die Manpage des Befehls file.

encoding **?*subcommand args...*?**

Der Alias encoding bietet Zugriff auf eine sichere Teilmenge der Unterbefehle des Befehls encoding; er verbietet das Setzen der Systemkodierung, erlaubt aber alle anderen Unterbefehle einschließlich **system** zum Abfragen der aktuellen Kodierung.

exit

Der aufrufende Interpreter wird gelöscht und seine Auswertung gestoppt, der Tcl-Prozess, in dem dieser Interpreter existiert, wird jedoch nicht beendet.

## Sicherheit

Safe Tcl versucht nicht, Belästigungs- und Denial-of-Service-Angriffe vollständig zu verhindern. Diese Angriffsformen verhindern, dass Anwendung oder Benutzer den Computer vorübergehend für nützliche Arbeit nutzen können, z. B. durch Verbrauch der gesamten CPU-Zeit oder des gesamten Bildschirmplatzes. Diese Angriffe sind zwar ärgerlich, gelten im Allgemeinen jedoch als weniger wichtig als Integritäts- und Vertraulichkeitsangriffe, die Safe Tcl verhindern soll.

Die in einem sicheren Interpreter verfügbaren Befehle sind zusätzlich zur sicheren Menge gemäß der Manpage interp vermittelte Aliasen für source, load, exit sowie sichere Teilmengen von file und encoding. Der sichere Interpreter kann Code auch per Auto-Load laden und das Laden von Paketen anfordern.

Da einige dieser Befehle auf das lokale Dateisystem zugreifen, besteht die Möglichkeit von Informationslecks über dessen Verzeichnisstruktur. Um dies zu verhindern, verwenden Befehle, die in einem sicheren Interpreter Dateinamen als Argumente erwarten, Token statt echter Verzeichnisnamen. Diese Token werden in einen echten Verzeichnisnamen übersetzt, während z. B. eine Anforderung zum Sourcen einer Datei vom Eltern-Interpreter vermittelt wird. Dieses virtuelle Pfadsystem wird im Eltern-Interpreter für jeden von **::safe::interpCreate** erzeugten oder von **::safe::interpInit** initialisierten sicheren Interpreter verwaltet; der Pfad bildet im sicheren Interpreter zugängliche Token auf reale Pfadnamen im lokalen Dateisystem ab und verhindert so, dass sichere Interpreter Kenntnis über die Dateisystemstruktur des Hosts erlangen, auf dem der Interpreter läuft. Die einzigen gültigen Dateinamen-Argumente für die dem Kind bereitgestellten Aliasen source und load sind Pfade der Form **[file join** *token filename* **]** (d. h. im nativen Pfadformat: *token***/***filename* unter Unix und *token***\***filename* unter Windows), wobei *token* eines der Verzeichnisse der *accessPath*-Liste repräsentiert und *filename* eine Datei in diesem Verzeichnis ist (Zugriff auf Unterverzeichnisse ist nicht erlaubt).

Wird ein Token in einem sicheren Interpreter in einer Anforderung zum Sourcen oder Laden einer Datei verwendet, wird das Token geprüft und in einen realen Pfadnamen übersetzt; die zu sourcende oder ladende Datei wird im Dateisystem gesucht. Der sichere Interpreter erlangt niemals Kenntnis des tatsächlichen Pfadnamens, unter dem die Datei im Dateisystem gespeichert ist.

Um weiteres potenzielles Informationslecken aus sensiblen Dateien zu verhindern, die versehentlich in der Menge der vom sicheren Interpreter sourcbaren Dateien enthalten sind, schränkt der Alias source den Zugriff auf Dateien ein, die folgende Bedingungen erfüllen: Der Dateiname darf höchstens vierzehn Zeichen lang sein, darf höchstens einen Punkt**.** enthalten, muss mit der Erweiterung **.tcl** enden oder **tclIndex** heißen.

Jedes Element der anfänglichen Zugriffspfad-Liste erhält ein Token, das im Kind-auto_path gesetzt wird; das erste Element dieser Liste wird als tcl_library des Kindes gesetzt.

Wird das Zugriffspfad-Argument **::safe::interpCreate** oder **::safe::interpInit** nicht angegeben oder ist es die leere Liste, greift das Standardverhalten: Das Kind erhält Zugriff auf dieselben Pakete wie der Eltern-Interpreter (genauer: nur in Tcl geschriebene Pakete, die per Definition nicht gefährlich sind, weil sie im Kind-Interpreter laufen, und C-Erweiterungen mit einem _SafeInit-Einstiegspunkt). Dazu wird der auto_path des Eltern-Interpreters zur Konstruktion des

Kind-Zugriffspfads verwendet. Damit das Kind die Tcl-Bibliotheksdateien (einschließlich des Auto-Load-Mechanismus selbst) erfolgreich laden kann, wird tcl_library bei Bedarf hinzugefügt oder an die erste Position verschoben, sodass die Kind-tcl_library dieselbe wie die des Eltern-Interpreters ist (der reale Pfad bleibt dem Kind verborgen). Damit Auto-Loading für Kind und Eltern in diesem Standardfall gleich funktioniert, werden die Unterverzeichnisse erster Ebene jedes Verzeichnisses im Eltern-auto_path (falls noch nicht enthalten) ebenfalls dem Kind-Zugriffspfad hinzugefügt. Sie können jederzeit einen restriktiveren Pfad angeben, in dem bestimmte Unterverzeichnisse nie durchsucht werden, indem Sie Ihre Verzeichnisliste explizit mit dem Schalter **-accessPath** statt dieses Standardmechanismus angeben.

Wird *accessPath* nach der ersten Erzeugung oder Initialisierung geändert (z. B. über **interpConfigure -accessPath** *list*), wird im sicheren Interpreter automatisch auto_reset ausgewertet, um **auto_index** mit der neuen Token-Liste zu synchronisieren.

## Typische Verwendung

In vielen Fällen können die Eigenschaften eines Safe-Base-Interpreters bei der Erzeugung festgelegt und für die Lebensdauer des Interpreters unverändert bleiben.

Wenn Sie Safe-Base-Interpreter mit ausgeschaltetem "Sync Mode" verwenden möchten, werten Sie aus:

```
safe::setSyncMode 0
```

Verwenden Sie **::safe::interpCreate** oder **::safe::interpInit**, um einen Interpreter mit den gewünschten Eigenschaften zu erzeugen. Am einfachsten ist es, **-accessPath** oder **-autoPath** nicht anzugeben; dann verwendet der sichere Interpreter dieselben Pfade wie der Eltern-Interpreter. Wird jedoch **-accessPath** angegeben, muss auch **-autoPath** angegeben werden, sonst wird es auf {} gesetzt.

Der Wert von **-autoPath** muss den Zugriff auf tclIndex- und pkgIndex.tcl-Dateien nach denselben Regeln wie ein unsicherer Interpreter ermöglichen (siehe pkg_mkIndex(n) und library(n)).

Bei eingeschaltetem "Sync Mode" ist die Option **-autoPath** undefiniert; die Safe Base setzt den ::auto_path des Kindes auf eine tokenisierte Form des Zugriffspfads. Zusätzlich zu den Verzeichnissen bei ausgeschaltetem "Sync Mode" enthält der ::auto_path zahlreiche Unterverzeichnisse und Modulpfade des Zugriffspfads.

## Sync-Modus

Vor Tcl Version 9.0 hielt die Safe Base den ::auto_path jedes sicheren Interpreters mit einer tokenisierten Form seines Zugriffspfads synchron. Einschränkungen von Tcl 8.4 und früher machten dieses Feature nötig. Diese Definition von ::auto_path entsprach nicht der Spezifikation in library(n) und pkg_mkIndex(n), funktionierte aber dennoch zuverlässig für das Auffinden und Laden von Paketen. Die Einführung von Tcl-Modulen in Tcl 8.5 fügte dem Zugriffspfad viele Verzeichnisse hinzu; es ist unbequem, diese zusätzlichen Verzeichnisse unnötig an ::auto_path anzuhängen.

Zur Kompatibilität mit bestehendem Code ist diese Synchronisation von ::auto_path und Zugriffspfad ("Sync Mode" ein) weiterhin Standard. Die Safe Base bietet jedoch die Option, den ::auto_path des sicheren Interpreters auf die deutlich kürzere Liste der für seine Funktion nötigen Verzeichnisse zu beschränken ("Sync Mode" aus). Verwenden Sie **::safe::setSyncMode**, um den Modus zu wählen, bevor Sie Safe-Base-Interpreter erzeugen.

In beiden Modi ist der bequemste Weg zur Initialisierung eines sicheren Interpreters, **::safe::interpCreate** oder **::safe::interpInit** ohne die Optionen **-accessPath** oder **-autoPath** (oder mit **-accessPath** als leerer Liste) aufzurufen; dann erhält der sichere Interpreter denselben Zugriff wie der Eltern-Interpreter auf Pakete, Module und Auto-Loader-Dateien. Bei ausgeschaltetem "Sync Mode" setzt die Safe Base den Wert von **-autoPath** auf den ::auto_path des Eltern-Interpreters und den ::auto_path des Kindes auf eine tokenisierte Form des ::auto_path des Eltern-Interpreters.

Bei ausgeschaltetem "Sync Mode": Wird in einem Aufruf von **::safe::interpCreate**, **::safe::interpInit** oder **::safe::interpConfigure** ein Wert für **-autoPath** angegeben, auch die leere Liste, wird er tokenisiert und als ::auto_path des sicheren Interpreters verwendet. Verzeichnisse, die nicht auch zum Zugriffspfad gehören, können nicht tokenisiert werden und werden still ignoriert. Der Wert von **-autoPath** bleibt jedoch wie angegeben und wird zur erneuten Tokenisierung des ::auto_path des Kindes verwendet, wenn **::safe::interpConfigure** den Wert von **-accessPath** ändert.

Bei ausgeschaltetem "Sync Mode": Wird der Zugriffspfad durch **::safe::interpConfigure** mit **-accessPath** {} auf die Werte des Eltern-Interpreters zurückgesetzt, wird auch ::auto_path zurückgesetzt, sofern nicht **-autoPath** einen anderen Wert vorgibt.

Bei ausgeschaltetem "Sync Mode": Wird ein nicht-leerer Wert für **-accessPath** übergeben, wird der ::auto_path des sicheren Interpreters auf {} gesetzt (durch **::safe::interpCreate**, **::safe::interpInit**) oder unverändert gelassen (durch **::safe::interpConfigure**). Gibt derselbe Befehl einen neuen Wert für **-autoPath** an, wird er nach der Verarbeitung von **-accessPath** angewendet.

Beispiele bei ausgeschaltetem "Sync Mode": Jeder dieser Befehle setzt ::auto_path auf eine tokenisierte Form seines Werts im Eltern-Interpreter:

```
safe::interpCreate foo
safe::interpCreate foo -accessPath {}
safe::interpInit bar
safe::interpInit bar -accessPath {}
safe::interpConfigure foo -accessPath {}
```

Beispiel bei ausgeschaltetem "Sync Mode": Bei Initialisierung mit nicht-leerem Zugriffspfad wird ::auto_path auf {} gesetzt, sofern kein eigener Wert angegeben wird:

```
safe::interpCreate foo -accessPath {
    /usr/local/TclHome/lib/tcl9.0
    /usr/local/TclHome/lib/tcl9.0/http1.0
    /usr/local/TclHome/lib/tcl9.0/opt0.4
    /usr/local/TclHome/lib/tcl9.0/msgs
    /usr/local/TclHome/lib/tcl9.0/encoding
    /usr/local/TclHome/lib
}

# Dem Kind muss ein passender ::auto_path-Wert gegeben werden:
```

```
safe::interpConfigure foo -autoPath {
    /usr/local/TclHome/lib/tcl9.0
    /usr/local/TclHome/lib
}

# Beide Befehle können kombiniert werden:

safe::interpCreate foo -accessPath {
    /usr/local/TclHome/lib/tcl9.0
    /usr/local/TclHome/lib/tcl9.0/http1.0
    /usr/local/TclHome/lib/tcl9.0/opt0.4
    /usr/local/TclHome/lib/tcl9.0/msgs
    /usr/local/TclHome/lib/tcl9.0/encoding
    /usr/local/TclHome/lib
} -autoPath {
    /usr/local/TclHome/lib/tcl9.0
    /usr/local/TclHome/lib
}
```

Beispiel bei ausgeschaltetem "Sync Mode": Der Befehl **safe::interpAddToAccessPath** ändert den ::auto_path des sicheren Interpreters nicht; nötige Änderungen müssen im Skript vorgenommen werden:

```
safe::interpAddToAccessPath foo /usr/local/TclHome/lib/extras/Img1.4.11

lassign [safe::interpConfigure foo -autoPath] DUM childAutoPath
lappend childAutoPath /usr/local/TclHome/lib/extras/Img1.4.11
safe::interpConfigure foo -autoPath $childAutoPath
```

*Copyright (c) 1995-1996 Sun Microsystems, Inc.*

# scan

*Stichwörter:* scan, conversion specifier, parse

## Name

scan - Zeichenkette mit Konvertierungsspezifizierern im sscanf-Stil parsen

## Übersicht

**scan** *string format* ?*varName varName* ...?

## Einleitung

Der Befehl parst Teilstrings aus einer Eingabezeichenkette analog zu ANSI-C **sscanf** und gibt die Anzahl erfolgreicher Konvertierungen zurück, oder -1, wenn das Ende von *string* erreicht wird, bevor eine Konvertierung gelang. *string* ist die Eingabe, *format* legt das Muster fest (**%** wie bei **sscanf**). Jedes *varName* benennt eine Variable; passt ein Teilstring zu einem Spezifizierer, wird er der Variable zugewiesen. Ohne *varName* arbeitet **scan** inline und gibt die Werte als Liste zurück; am Ende ohne Konvertierung: leerer String.

## Details zum Scannen

**scan** durchläuft *string* und *format* gemeinsam. Ist das nächste Zeichen in *format* Leerzeichen oder Tab, entspricht es beliebig viel Leerraum in *string* (auch null). Sonst, wenn es kein **%** ist, muss es mit dem nächsten Zeichen von *string* übereinstimmen. **%** beginnt einen Konvertierungsspezifizierer mit bis zu vier Feldern: XPG3-Position (oder * zum Verwerfen), maximale Teilstringbreite, Größenmodifikator, Konvertierungszeichen (Pflicht). Reihenfolge wie oben.

Bei einem Spezifizierer überspringt **scan** zunächst Leerraum in *string* (außer bei **[** oder **c**), konvertiert dann die nächsten Zeichen und speichert das Ergebnis in der nächsten Variable.

### Optionale Positionsangabe

**%** gefolgt von Zahl und **$** (z. B. **%2$d**): Variable nicht sequenziell, sondern vom angegebenen *varName* (1 = erstes). Sind Positionsangaben vorhanden, müssen alle positional sein. Jedes *varName* muss genau einem Spezifizierer entsprechen, sonst Fehler; inline darf jede Position höchstens einmal vorkommen, leere Positionen werden mit leeren Strings gefüllt.

### Optionaler Größenmodifikator

Nur bei Tcl-Ganzzahlen relevant; begrenzt den speicherbaren Bereich (bzw. Listenposition inline). Gültig: **h**, **l**, **z**, **t**, **q**, **j**, **ll**, **L**. **h** oder fehlend: 32-Bit-Bereich. **L** oder **ll**: unbegrenzt. **l** (**q**, **j**): Bereich wie **wide()** in expr. **z**/**t**: wie **h** oder **l** je nach **pointerSize** in tcl_platform.

### Pflicht-Konvertierungszeichen

Folgende Konvertierungszeichen werden unterstützt:

**d**

Dezimalganzzahl; Wert in Variable, gekürzt gemäß Größenmodifikator.

**o**

Oktalganzzahl; wie oben.

**x oder X**

Hexadezimalganzzahl; wie oben.

**b**

Binärganzzahl; wie oben.

**u**

Dezimalganzzahl; nach Kürzung entsprechender vorzeichenloser Wert als Dezimalstring in der Variable.

**i**

Ganzzahl; Basis per C-Konvention (führende 0 Oktal, **0x** Hex); Kürzung wie oben.

**c**

Ein Zeichen; Unicode-Wert als Ganzzahl in der Variable. Kein Überspringen von Leerraum; Teilstring kann Leerzeichen sein.

**s**

Alle Zeichen bis zum nächsten Leerraum; Kopie in die Variable.

**e oder f oder g oder E oder G**

Gleitkommazahl mit optionalem Vorzeichen, Dezimalziffern optional mit Punkt, optionaler Exponent **e**/**E** mit Vorzeichen und Ziffern; intern als Gleitkomma.

**[*chars*]**

Ein oder mehr Zeichen aus *chars*; Treffer in Variable. Steht **]** zuerst in *chars*, gehört es zu *chars*. Bereich *a*-*b* inklusive. Führendes oder abschließendes **-** in *chars* ist Literal, kein Bereich.

**[^*chars*]**

Zeichen nicht in *chars*; Treffer in Variable. **^** gefolgt von **]**: **]** gehört zur Menge. Bereiche und **-** wie oben.

**n**

Kein Verbrauch aus *string*; bisher gescannte Zeichenanzahl in die Variable.

Gelesen wird jeweils die längste sinnvolle Folge (**%d**: Dezimalziffern, **%o**: Oktal, usw.). Abbruch bei Leerzeichen oder maximaler Breite. ***** im Spezifizierer: keine Zuweisung, nächstes Scan-Argument wird nicht verbraucht.

## Unterschiede zu ANSI sscanf

Wie **sscanf**, außer:

1. **%p** wird nicht unterstützt.
2. Bei **%c** wird ein Zeichen in einen Dezimalstring umgewandelt und zugewiesen; keine Teilstringbreite.
3. **h** wird ignoriert; **l** und **L** bei reellen Werten ignoriert (intern **double**). **ll** hat kein **sscanf** -Äquivalent.
4. Ende von *string* vor Konvertierungen ohne Variablen: leerer String.

## Beispiele

Unicode-Zeichen in numerischen Wert:

```
set char "x"
set value [scan $char %c]
```

Farbe *#RRGGBB* mit Hex und Feldgrößen:

```
set string "#08D03F"
scan $string "#%2x%2x%2x" r g b
```

Zeit *HH:MM*:

```
set string "08:08"
if {[scan $string "%d:%d" hours minutes] != 2} {
    error "not a valid time string"
}
# We have to understand numeric ranges ourselves...
if {$minutes < 0 || $minutes > 59} {
    error "invalid number of minutes"
}
```

Nicht-Leerwort-Sequenzen (**%n** für korrektes Überspringen von Leerraum):

```
set string " a string {with braced words} + leading space "
set words {}
while {[scan $string %s%n word length] == 2} {
    lappend words $word
    set string [string range $string $length end]
}
```

Koordinatenstring mit explizitem Abschlusszeichen:

```
set string "(5.2,-4e-2)"
# Note that the spaces before the literal parts of
# the scan pattern are significant, and that ")" is
# the Unicode character \u0029
if {
    [scan $string " (%f ,%f %c" x y last] != 3
    || $last != 0x0029
} then {
    error "invalid coordinate string"
}
puts "X=$x, Y=$y"
```

Kürzung durch Größenmodifikatoren:

```
% scan 20000000000000000000 %d
2147483647
% scan 20000000000000000000 %ld
9223372036854775807
% scan 20000000000000000000 %lld
20000000000000000000
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2000 Scriptics Corporation.*

# seek

*Stichwörter:* seek

## Name

seek - Ändert die Zugriffsposition eines offenen Kanals

## Übersicht

**seek** *channel offset* ?*origin*?

## Beschreibung

Der Befehl **seek** wurde durch den Befehl chan seek ersetzt, der die gleiche Syntax und die gleichen Optionen unterstützt.

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# self

*Stichwörter:* self, call, introspection, object

## Name

self - Introspektion des aktuellen Methodenaufrufs

## Übersicht

**package require tcl::oo**

**self** ?*subcommand*?

## Beschreibung

Der Befehl **self**, der nur innerhalb des Kontexts eines Methodenaufrufs verwendet werden sollte (d. h. im Rumpf einer Methode, eines Konstruktors oder Destruktors), ermöglicht es der Methode, Informationen darüber zu ermitteln, wie sie aufgerufen wurde. Er nimmt ein Argument *subcommand* entgegen, das angibt, welche Art von Information gewünscht ist; fehlt es, entspricht das Ergebnis dem von **self object**. Die unterstützten Unterbefehle sind:

**self call**

Gibt eine Liste mit zwei Elementen zurück, die die zur Implementierung der aktuellen Aufrufkette verwendeten Methodenimplementierungen beschreibt. Das erste Element entspricht dem, was info object **call** für die aktuelle Methode melden würde (außer dass hier auch sinnvolle Werte aus Konstruktoren und Destruktoren geliefert werden, deren Namen als bzw. gemeldet werden, sowie für private Methoden, die als private statt als method beschrieben werden), und das zweite Element ist ein Index in die Liste des ersten Elements, der angibt, welche Implementierung gerade ausgeführt wird (die erste auszuführende Implementierung steht stets bei Index 0).

**self caller**

Wurde die Methode von innerhalb einer anderen Objektmethode aufgerufen, gibt dieser Unterbefehl eine Liste mit drei Elementen zurück, die das enthaltende Objekt und die Methode beschreiben. Das erste Element beschreibt das deklarierende Objekt oder die Klasse der Methode, das zweite ist der Name des Objekts, auf dem die enthaltende Methode aufgerufen wurde, und das dritte ist der Name der Methode (wobei die Zeichenketten und Konstruktoren bzw. Destruktoren kennzeichnen).

**self class**

Gibt den Namen der Klasse zurück, in der die aktuelle Methode definiert wurde. Beachten Sie, dass sich dieser Wert ändert, wenn die Kette der Methodenimplementierungen mit next durchlaufen wird, und dass dies fehlschlägt, wenn die Methode auf einem Objekt definiert wurde.

Wenn Sie die Klasse des aktuellen Objekts benötigen, verwenden Sie stattdessen:

```
info object class [self object]
```

**self filter**

Wird er innerhalb eines Filters aufgerufen, gibt dieser Unterbefehl eine Liste mit drei Elementen zurück, die den Filter beschreibt. Das erste Element gibt den Namen des Objekts oder der Klasse an, die den Filter deklariert hat (dies kann von dem Objekt oder der Klasse abweichen, die die Implementierung des Filters bereitstellt), das zweite ist entweder **object** oder **class**, je nachdem ob die deklarierende Entität ein Objekt oder eine Klasse war, und das dritte ist der Name des Filters.

**self method**

Gibt den Namen der aktuellen Methode zurück (wobei die Zeichenketten und Konstruktoren bzw. Destruktoren kennzeichnen).

**self namespace**

Gibt den Namen des eindeutigen Namespace des Objekts zurück, auf dem die Methode aufgerufen wurde.

**self next**

Wird er von einer Methode aufgerufen, die nicht am Ende einer Aufrufkette steht (d. h. wo der Befehl next eine tatsächliche Methodenimplementierung aufrufen wird), gibt dieser Unterbefehl eine Liste mit zwei Elementen zurück, die das nächste Element in der Methodenaufrufkette beschreibt; das erste Element ist der Name der Klasse oder des Objekts, die den nächsten Teil der Aufrufkette deklariert, und das zweite ist der Name der Methode (wobei die Zeichenketten und Konstruktoren bzw. Destruktoren kennzeichnen). Wird er von einer Methode am Ende einer Aufrufkette aufgerufen, gibt dieser Unterbefehl die leere Zeichenkette zurück.

**self object**

Gibt den Namen des Objekts zurück, auf dem die Methode aufgerufen wurde.

**self target**

Wird er innerhalb einer Filterimplementierung aufgerufen, gibt dieser Unterbefehl eine Liste mit zwei Elementen zurück, die die gefilterte Methode beschreibt. Das erste Element ist der Name des Deklarators der Methode, das zweite der tatsächliche Name der Methode.

## Beispiele

Dieses Beispiel zeigt die grundlegende Verwendung von **self**, um Informationen über das aktuelle Objekt zu liefern:

```
oo::class create c {
    method foo {} {
        puts "this is the [self] object"
    }
}
c create a
c create b
a foo                → prints "this is the ::a object"
b foo                → prints "this is the ::b object"
```

Dies veranschaulicht, wie eine Methodenaufrufkette aussieht und wie sich der Index beim Durchlaufen ändert:

```
oo::class create c {
    method x {} {
        puts "Cls: [self call]"
    }
}
c create a
```

```
oo::objdefine a {
    method x {} {
        puts "Obj: [self call]"
        next
        puts "Obj: [self call]"
    }
}
a x      → Obj: {{method x object method} {method x ::c method}} 0
         → Cls: {{method x object method} {method x ::c method}} 1
         → Obj: {{method x object method} {method x ::c method}} 0
```

*Copyright (c) 2007 Donal K. Fellows*

# set

*Stichwörter:* set, read, write, variable

## Name

set - Liest und schreibt Variablen

## Übersicht

**set** *varName* ?*value*?

## Beschreibung

Gibt den Wert der Variable *varName* zurück. Wenn *value* angegeben ist, wird der Wert von *varName* auf *value* gesetzt, wobei eine neue Variable erstellt wird, falls noch keine existiert, und ihr Wert zurückgegeben wird. Wenn *varName* eine öffnende Klammer enthält und mit einer schließenden Klammer endet, bezieht es sich auf ein Array-Element: Die Zeichen vor der ersten öffnenden Klammer sind der Name des Arrays, und die Zeichen zwischen den Klammern sind der Index innerhalb des Arrays. Andernfalls bezieht sich *varName* auf eine skalare Variable.

Wenn *varName* Namespace-Qualifizierer enthält (im Array-Namen, falls es sich auf ein Array-Element bezieht), oder wenn *varName* unqualifiziert ist (keine Namen von umgebenden Namespaces enthält), aber keine Prozedur aktiv ist, bezieht sich *varName* auf eine Namespace-Variable, die gemäß den Regeln unter **NAME RESOLUTION** in der namespace -Manpage aufgelöst wird.

Wenn eine Prozedur aktiv ist und *varName* unqualifiziert ist, bezieht sich *varName* auf einen Parameter oder eine lokale Variable der Prozedur, es sei denn, *varName* wurde durch einen der Befehle global, variable oder upvar anders deklariert.

## Beispiele

Speichert eine Zufallszahl in der Variable *r*:

```
set r [expr {rand()}]
```

Speichert eine kurze Nachricht in einem Array-Element:

```
set anAry(msg) "Hello, World!"
```

Speichert eine kurze Nachricht in einem Array-Element, das durch eine Variable spezifiziert ist:

```
set elemName "msg"
set anAry($elemName) "Hello, World!"
```

Kopiert einen Wert in die Variable *out* aus einer Variable, deren Name in *vbl* gespeichert ist (beachten Sie, dass es in der Praxis oft einfacher ist, Arrays zu verwenden, statt doppelte Dereferenzierung):

```
set in0 "small random"
set in1 "large random"
set vbl in[expr {rand() >= 0.5}]
set out [set $vbl]
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# singleton

*Stichwörter:* singleton, class, metaclass, object, single instance

## Name

oo::singleton - Klasse, die höchstens eine Instanz von sich selbst zulässt

## Übersicht

**package require tcl::oo**

**oo::singleton** *method* ?*arg* ...?

## Klassenhierarchie

```
oo::object
   → oo::class
       → oo::singleton
```

## Beschreibung

Singleton-Klassen sind Klassen, die höchstens eine Instanz von sich selbst zulassen. Sie exportieren die Methoden **create** und **createWithNamespace** vollständig nicht und überschreiben **new** so, dass nur dann eine neue Instanz erzeugt wird, wenn noch keine existiert. Von einer Singleton-Klasse zu erben wird nicht empfohlen; Singleton-Eigenschaft wird *nicht* vererbt. Es wird nicht empfohlen, dass der Konstruktor einer Singleton-Klasse Argumente annimmt.

Instanzen haben ihre Methode **destroy** durch eine Methode ersetzt, die stets einen Fehler zurückgibt, um die Zerstörung des Objekts zu erschweren; Zerstörung bleibt bei striktem Bedarf möglich (z. B. durch Zerstörung der Klasse oder rename zum Löschen). Sie definieren außerdem eine (nicht exportierte) Methode, die ebenfalls stets Fehler zurückgibt, damit Versuche, die Singleton-Instanz mit oo::copy zu nutzen, scheitern.

## Konstruktor

Die Klasse **oo::singleton** definiert keinen expliziten Konstruktor; sie verhält sich damit wie der Konstruktor der Klasse oo::class.

### Destruktor

Die Klasse **oo::singleton** definiert keinen expliziten Destruktor; das Zerstören einer Instanz entspricht dem Zerstören einer gewöhnlichen Klasse (und zerstört das Singleton-Objekt).

### Exportierte Methoden

***cls* new ?*arg* ...?**

Gibt die aktuelle Instanz der Singleton-Klasse zurück, falls eine existiert, und erzeugt nur dann eine neue Instanz, wenn noch keine existiert. Die zusätzlichen Argumente *arg ...* werden nur verwendet, wenn tatsächlich eine neue Instanz erzeugt wird; die Erzeugung erfolgt über **new** der Klasse oo::class.

Dies überschreibt das Verhalten einer Superklassen-Methode mit derselben Aufrufsignatur.

### Nicht exportierte Methoden

Die Klasse **oo::singleton** erklärt ausdrücklich, dass **create** und **createWithNamespace** nicht exportiert sind; Aufrufer dürfen nicht annehmen, dass sie Name oder Namespace-Name der Singleton-Instanz steuern können.

## Beispiele

Dieses Beispiel zeigt, dass es nur eine Instanz gibt, obwohl **new** dreimal aufgerufen wird.

```
oo::singleton create Highlander {
    method say {} {
        puts "there can be only one"
    }
}

set h1 [Highlander new]
set h2 [Highlander new]
if {$h1 eq $h2} {
    puts "equal objects"    → prints "equal objects"
}
set h3 [Highlander new]
if {$h1 eq $h3} {
    puts "equal objects"    → prints "equal objects"
}
```

Beachten Sie, dass der Name der Singleton-Instanz nicht garantiert ist.

*Copyright (c) 2018 Donal K. Fellows*

# socket

*Stichwörter:* socket, asynchronous I/O, bind, channel, connection, domain name, host, network address, tcp

## Name

socket - Öffnet eine TCP-Netzwerkverbindung

## Übersicht

**socket** ?*options*? *host port*
**socket -server** *command* ?*options*? *port*

## Beschreibung

Dieser Befehl öffnet einen Netzwerk-Socket und gibt einen Kanalbezeichner zurück, der in späteren Aufrufen von Befehlen wie read, puts und flush verwendet werden kann. Derzeit wird nur das TCP-Netzwerkprotokoll über IPv4 und IPv6 unterstützt; zukünftige Versionen können zusätzliche Protokolle unterstützen. Der Befehl **socket** kann verwendet werden, um je nachdem, ob der Schalter **-server** angegeben ist, entweder die Client- oder die Server-Seite einer Verbindung zu öffnen.

Beachten Sie, dass die Standardkodierung für *alle* Sockets die Systemkodierung ist, wie von encoding system zurückgegeben. Meistens müssen Sie chan configure verwenden, um diese auf etwas anderes zu ändern, wie *utf-8* (ideal für die Kommunikation mit anderen Tcl-Prozessen) oder *iso8859-1* (nützlich für viele Netzwerkprotokolle, besonders die älteren).

## Client-Sockets

Ist die Option **-server** nicht angegeben, wird die Client-Seite einer Verbindung geöffnet und der Befehl gibt einen Kanalbezeichner zurück, der sowohl zum Lesen als auch zum Schreiben verwendet werden kann. *Port* und *host* legen einen Port fest, mit dem eine Verbindung hergestellt werden soll; auf diesem Port muss ein Server Verbindungen annehmen. *Port* ist eine ganzzahlige Portnummer (oder ein Dienstname, sofern vom Host-Betriebssystem unterstützt und verstanden) und *host* ist entweder ein Domain-Name wie **www.tcl.tk** oder eine numerische IPv4- oder IPv6-Adresse wie **127.0.0.1** oder **2001:DB8::1**. Verwenden Sie *localhost*, um auf den Host zu verweisen, auf dem der Befehl ausgeführt wird.

Die folgenden Optionen können ebenfalls vor *host* stehen, um zusätzliche Informationen über die Verbindung anzugeben:

**-myaddr *addr***

*Addr* gibt den Domain-Namen oder die numerische IP-Adresse der clientseitigen Netzwerkschnittstelle an, die für die Verbindung verwendet werden soll. Diese Option kann nützlich sein, wenn der Client-Rechner mehrere Netzwerkschnittstellen hat. Wird die Option weggelassen, wählt die Systemsoftware die clientseitige Schnittstelle.

**-myport *port***

*Port* legt eine ganzzahlige Portnummer (oder einen Dienstnamen, sofern vom Host-Betriebssystem unterstützt und verstanden) für die Client-Seite der Verbindung fest. Wird diese Option weggelassen, wird die Client-Portnummer zufällig von der Systemsoftware gewählt.

**-async**

Diese Option bewirkt, dass der Client-Socket asynchron verbunden wird. Das heißt, der Socket wird sofort erstellt, ist aber möglicherweise noch nicht mit dem Server verbunden, wenn der Aufruf von **socket** zurückkehrt.

Wird auf dem Socket ein gets oder flush ausgeführt, bevor der Verbindungsversuch erfolgreich ist oder fehlschlägt, wartet die Operation im blockierenden Modus des Sockets, bis die Verbindung abgeschlossen ist oder fehlschlägt. Ist der Socket im nicht blockierenden Modus und wird ein gets oder flush ausgeführt, bevor der Verbindungsversuch erfolgreich ist oder fehlschlägt, kehrt die Operation sofort zurück und fblocked auf dem Socket gibt 1 zurück. Synchrone Client-Sockets können (nach erfolgreicher Verbindung) in den asynchronen Betrieb versetzt werden mit:

```
chan configure chan -blocking 0
```

Siehe den Befehl chan configure für weitere Einzelheiten.

Die Tcl-Ereignisschleife sollte laufen, während eine asynchrone Verbindung aufgebaut wird, da im Hintergrund mehrere Verbindungsversuche nötig sein können. Das Ausführen der Ereignisschleife ermöglicht außerdem, ein beschreibbares Kanalereignis auf dem Socket einzurichten, um benachrichtigt zu werden, wenn die asynchrone Verbindung erfolgreich war oder fehlgeschlagen ist. Siehe die Befehle vwait und chan für weitere Einzelheiten zur Ereignisschleife und zu Kanalereignissen.

Die chan configure-Option **-connecting** kann verwendet werden, um zu prüfen, ob der Verbindungsaufbau noch läuft. Um eine erfolgreiche Verbindung zu verifizieren, kann die Option **-error** geprüft werden, wenn **-connecting** 0 zurückgegeben hat.

Der Betrieb ohne Ereigniswarteschlange erfordert derzeit Aufrufe von chan configure, um die interne Zustandsmaschine voranzutreiben.

## Server-Sockets

Ist die Option **-server** angegeben, ist der neue Socket ein Server, der auf dem angegebenen *port* lauscht (entweder eine Ganzzahl oder ein Dienstname, sofern vom Host-Betriebssystem unterstützt und verstanden; ist *port* null, weist das Betriebssystem dem Server-Socket einen freien Port zu, der mit chan configure und der Option **-sockname** ermittelt werden kann). Unterstützt der Host sowohl IPv4 als auch IPv6, lauscht der Socket auf beiden Adressfamilien. Tcl nimmt Verbindungen zum angegebenen Port automatisch an. Für jede Verbindung erstellt Tcl einen neuen Kanal, über den mit dem Client kommuniziert werden kann. Tcl ruft dann *command* (korrekt als Befehlspräfixliste, siehe Beispiele unten) mit drei zusätzlichen Argumenten auf: den Namen des neuen Kanals, die Adresse des Client-Hosts in Netzwerkadressnotation und die Portnummer des Clients.

Die folgende zusätzliche Option kann ebenfalls vor *port* angegeben werden:

**-myaddr *addr***

*Addr* gibt den Domain-Namen oder die numerische IP-Adresse der serverseitigen Netzwerkschnittstelle für die Verbindung an. Diese Option kann nützlich sein, wenn der Server-Rechner mehrere Netzwerkschnittstellen hat. Wird die Option weggelassen, ist der Server-Socket an die Platzhalteradresse gebunden, sodass Verbindungen von jeder Schnittstelle angenommen werden können. Ist *addr* ein Domain-Name, der in mehrere auf dem lokalen Rechner verfügbare IP-Adressen aufgelöst wird, lauscht der Socket auf allen.

**-reuseaddr *boolean***

Teilt dem Kernel mit, ob die lokale Adresse wiederverwendet werden soll, wenn kein Socket aktiv darauf lauscht. Dies ist unter Windows die Standardeinstellung.

**-reuseport *boolean***

Teilt dem Kernel mit, ob mehrere Sockets an dieselbe Adresse und denselben Port gebunden werden dürfen.

Server-Kanäle können nicht zum Lesen oder Schreiben verwendet werden; sie dienen ausschließlich dazu, neue Client-Verbindungen anzunehmen. Die für jede eingehende Client-Verbindung erstellten Kanäle werden zum Lesen und Schreiben geöffnet. Das Schließen des Server-Kanals beendet den Server, sodass keine neuen Verbindungen mehr angenommen werden; bestehende Verbindungen bleiben jedoch unberührt.

Server-Sockets sind auf den Tcl-Ereignismechanismus angewiesen, um zu erfahren, wann neue Verbindungen geöffnet werden. Betritt die Anwendung die Ereignisschleife nicht, etwa durch Aufruf von vwait oder der C-Prozedur Tcl_DoOneEvent, werden keine Verbindungen angenommen.

Wird *port* als null angegeben, weist das Betriebssystem einen unbenutzten Port für den Server-Socket zu. Die tatsächlich zugewiesene Portnummer kann vom erstellten Server-Socket mit chan configure und der Option **-sockname** wie unten beschrieben abgefragt werden.

## Konfigurationsoptionen

Der Befehl chan configure kann verwendet werden, um mehrere schreibgeschützte Konfigurationsoptionen für Socket-Kanäle abzufragen oder in einigen Fällen alternative Eigenschaften auf Socket-Kanälen zu setzen:

**-error**

Diese Option liefert den aktuellen Fehlerstatus des angegebenen Sockets. Das ist nützlich, wenn festgestellt werden soll, ob ein asynchroner Verbindungsaufbau erfolgreich war. Bei einem Fehler wird die Fehlermeldung zurückgegeben. Gab es keinen Fehler, wird eine leere Zeichenkette zurückgegeben.

Beachten Sie, dass der Fehlerstatus durch die Leseoperation zurückgesetzt wird; das entspricht dem zugrunde liegenden Aufruf getsockopt(SO_ERROR).

**-sockname**

Bei Client-Sockets (einschließlich der Kanäle, die erstellt werden, wenn sich ein Client mit einem Server-Socket verbindet) gibt diese Option eine Liste aus drei Elementen zurück: Adresse, Hostname und Portnummer des Sockets. Kann der Hostname nicht ermittelt werden, ist das zweite Element identisch mit der Adresse, dem ersten Element der Liste.

Bei Server-Sockets gibt diese Option eine Liste mit einem Vielfachen von drei Elementen zurück; jede Gruppe hat dieselbe Bedeutung wie oben beschrieben. Die Liste enthält mehr als eine Gruppe, wenn der Server-Socket ohne **-myaddr** erstellt wurde oder das Argument von **-myaddr** ein Domain-Name ist, der in mehrere auf dem aufrufenden Host lokale IP-Adressen aufgelöst wird.

**-peername**

Diese Option wird von Server-Sockets nicht unterstützt. Bei Client- und angenommenen Sockets gibt sie eine Liste aus drei Elementen zurück: Adresse, Hostname und Port, mit dem der Peer-Socket verbunden ist oder an den er gebunden ist. Kann der Hostname nicht ermittelt werden, ist das zweite Element der Liste identisch mit der Adresse, ihrem ersten Element.

**-connecting**

Diese Option wird von Server-Sockets nicht unterstützt. Bei Client-Sockets gibt sie 1 zurück, wenn ein asynchroner Verbindungsaufbau noch läuft, sonst 0.

**-keepalive**
Diese Option setzt oder fragt die TCP-Keepalive-Option des Sockets ab als 1, wenn Keepalive eingeschaltet ist, sonst 0.

**-nodelay**
Diese Option setzt oder fragt die TCP-Nodelay-Option des Sockets ab als 1, wenn Nodelay eingeschaltet ist, sonst 0.

## Beispiele

Hier ist ein sehr einfacher Zeitserver:

```
proc Server {startTime channel clientaddr clientport} {
    puts "Connection from $clientaddr registered"
    set now [clock seconds]
    puts $channel [clock format $now]
    puts $channel "[expr {$now - $startTime}] since start"
    close $channel
}

socket -server [list Server [clock seconds]] 9900
vwait forever
```

Und hier ist der zugehörige Client, der mit dem Server spricht und Informationen ausliest:

```
set server localhost
set sockChan [socket $server 9900]
gets $sockChan line1
gets $sockChan line2
close $sockChan
puts "The time on $server is $line1"
puts "That is [lindex $line2 0]s since the server started"
```

*Copyright (c) 1996 Sun Microsystems, Inc.*
*Copyright (c) 1998-1999 Scriptics Corporation.*

# source

*Stichwörter:* source, file, script

## Name

source - Wertet eine Datei oder Ressource als Tcl-Skript aus

## Übersicht

**source** *fileName*
**source -encoding** *encodingName fileName*

## Beschreibung

Der Befehl liest die angegebene Datei oder Ressource und übergibt den Inhalt dem Tcl-Interpreter als Textskript. Der Rückgabewert von **source** ist der des zuletzt im Skript ausgeführten Befehls. Tritt bei der Auswertung ein Fehler auf, gibt **source** diesen zurück. Wird im Skript return aufgerufen, wird der Rest der Datei übersprungen und **source** kehrt normal mit dem Ergebnis von return zurück.

Das Dateiende-Zeichen ist auf allen Plattformen "\32" (^Z). **source** liest bis zu diesem Zeichen. read und gets kennen diese Einschränkung nicht (Skripte mit Code- und Datensegmenten). Für einen Literal-Vergleich mit ^Z im Code: "\x1A" — der Interpreter wandelt das sicher in ^Z um.

Eine führende BOM (Byte Order Mark) in der Datei wird bei Unicode-Kodierungen (utf-8, utf-16, ucs-2) ignoriert.

Die Option **-encoding** legt die Kodierung der Daten in *fileName* fest. Ohne **-encoding** wird utf-8 angenommen.

### Beispiele

Skripte in **foo.tcl** und **bar.tcl** nacheinander ausführen:

```
source foo.tcl
source bar.tcl
```

Alternativ:

```
foreach scriptFile {foo.tcl bar.tcl} {
    source $scriptFile
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2000 Scriptics Corporation.*

## split

*Stichwörter:* split, list, string

### Name

split - Teilt eine Zeichenkette in eine gültige Tcl-Liste

### Übersicht

**split** *string* ?*splitChars*?

### Beschreibung

Gibt eine Liste zurück, die entsteht, wenn *string* an jedem Zeichen aus *splitChars* geteilt wird. Jedes Listenelement besteht aus den Zeichen von *string* zwischen solchen Trennzeichen. Leere Elemente entstehen bei benachbarten Trennzeichen oder wenn *string* mit einem Trennzeichen beginnt oder endet. Ist *splitChars* leer, wird jedes Zeichen von *string* ein eigenes Element. Standard: übliche Leerraumzeichen.

### Beispiele

USENET-Gruppenname in Hierarchieebenen:

```
split "comp.lang.tcl" .
      → comp lang tcl
```

**split** trennt an jedem Zeichen in *splitChars* — bei unpassender Wahl Informationsverlust:

```
split "alpha beta gamma" "temp"
      → al {ha b} {} {a ga} {} a
```

Wörter aus einer nicht wohlgeformten Liste:

```
split "Example with {unbalanced brace character"
      → Example with \{unbalanced brace character
```

Zeichenkette in Einzelzeichen:

```
split "Hello world" {}
      → H e l l o { } w o r l d
```

### Zeilenorientierte Dateien parsen

Unix-/etc/passwd (Zeilen mit Doppelpunkt-Feldern):

```
## Datei lesen
set fid [open /etc/passwd]
set content [read $fid]
close $fid

## Records an Zeilenumbrüchen
set records [split $content "\n"]

## Über Records
foreach rec $records {

    ## Felder an Doppelpunkten
    set fields [split $rec ":"]

    ## Variablen zuweisen und ausgeben...
    lassign $fields \
            userName password uid grp longName homeDir shell
    puts "$longName uses [file tail $shell] for a login shell"
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## string

*Stichwörter:* string, case conversion, compare, index, integer value, match, pattern, word, equal, ctype, character, reverse

### Name

string - Zeichenketten bearbeiten

### Übersicht

**string** *option arg* ?*arg* ...?

## Beschreibung

Führt je nach *option* eine von mehreren Zeichenkettenoperationen aus.

**Dem üblichen Sprachgebrauch folgend verwenden die Befehlsbeschreibungen unten den Begriff** *Zeichen* **an Stelle des genaueren Begriffs** *Codepunkt*.

Die zulässigen *option*en (Abkürzungen erlaubt) sind:

**string cat ?*string1*? ?*string2...*?**

Verkettet die *string*s wie direktes Nebeneinanderstellen und gibt die zusammengesetzte Zeichenkette zurück. Ohne Argumente: leerer String.

Gelegentlich handlicher als reine Juxtaposition bei gemischten Anführungszeichen oder wenn das Ergebnis einer Verkettung ohne return **-level 0** zurückgegeben werden soll; effizienter als Liste bauen und join mit leerem Trennzeichen.

**string compare ?-nocase? ?*-length length*? *string1 string2***

Zeichenweiser Vergleich von *string1* und *string2*. Rückgabe -1, 0 oder 1 (lexikographisch kleiner, gleich, größer). Mit **-length** nur die ersten *length* Zeichen; negative Länge wird ignoriert. **-nocase**: ohne Berücksichtigung der Groß-/Kleinschreibung.

**string equal ?-nocase? ?*-length length*? *string1 string2***

Zeichenweiser Vergleich; 1 bei Identität, sonst 0. **-length** und **-nocase** wie bei **compare**.

**string first *needleString haystackString* ?*startIndex*?**

Sucht in *haystackString* die erste exakte Übereinstimmung mit *needleString*; Rückgabe Index des ersten Zeichens oder -1. Mit *startIndex* (Formen siehe String-Indizes) beginnt die Suche ab diesem Index. Beispiel:

```
string first a 0a23456789abcdef 5
```

liefert **10**,

```
string first a 0123456789abcdef 11
```

liefert **-1**.

**string index *string charIndex***

Gibt das *charIndex*-te Zeichen von *string* zurück (0 = erstes Zeichen). *charIndex* wie in String-Indizes.

Liegt *charIndex* außerhalb [0, Länge), leerer String.

**string insert *string index insertString***

Kopie von string mit insertString an Position index (siehe String-Indizes).

start-relativ: erstes eingefügtes Zeichen an *index*; end-relativ: letztes an *index*.

*index* am Anfang (z. B. **0**): voranstellen; am Ende (**end**): anhängen.

**string is *class* ?-strict? ?*-failindex varname*? *string***

1, wenn *string* zur Zeichenklasse gehört, sonst 0. **-strict**: leerer String → 0; sonst leerer String → 1 für jede Klasse. **-failindex**: bei 0 Index der ersten ungültigen Stelle in *varname* (bei 1 nicht gesetzt). Klassen (abkürzbar):

**alnum**
Jedes Unicode-Alphabet- oder Ziffernzeichen.

**alpha**
Jedes Unicode-Alphabetzeichen.

**ascii**
Zeichenwert < \u0080 (7-Bit-ASCII).

**boolean**
Formen wie für Tcl_GetBoolean.

**control**
Jedes Unicode-Steuerzeichen.

dict
Gültige Dict-Struktur, optional umgebender Leerraum. Bei Fehler: 0, varname = Index des fehlerhaften "Elements" oder -1.

**digit**
Jedes Unicode-Ziffernzeichen (nicht nur [0-9]).

**double**
Formen wie für Tcl_GetDoubleFromObj.

**entier**
Synonym für **integer**.

**false**
Tcl_GetBoolean-Formen mit Wert false.

**graph**
Druckbares Unicode-Zeichen außer Leerzeichen.

**integer**
Gültige Tcl-Ganzzahlstrings beliebiger Größe, optional Leerraum; wie Tcl_GetBignumFromObj.

list
Gültige Listenstruktur, optional Leerraum. Bei Fehler: 0, *varname* wie bei **dict**.

**lower**
Unicode-Kleinbuchstabe.

**print**
Druckbares Unicode-Zeichen inkl. Leerzeichen.

**punct**
Unicode-Interpunktion.

**space**
Unicode-Whitespace, mongolisches Vokaltrennzeichen (U+180e), zero width space (U+200b), word joiner (U+2060), zero width no-break space (U+feff, BOM).

**true**
Tcl_GetBoolean mit Wert true.

**upper**
Unicode-Großbuchstabe.

**wideinteger**
Gültige Wide-Integer-Form; bei Überlauf 0, *varname* = -1.

**wordchar**

Unicode-Wortzeichen (alphanumerisch und Verbinder-Interpunktion, z. B. Unterstrich).

**xdigit**

Hexadezimalziffer ([0-9A-Fa-f]).

Bei **boolean**, **true**, **false**: bei 0 wird *varname* stets auf 0 gesetzt.

**string last *needleString haystackString* ?*lastIndex*?**

Letzte exakte Übereinstimmung mit *needleString* in *haystackString*; Index des ersten Zeichens dieser Übereinstimmung oder -1. Mit *lastIndex* nur Zeichen an oder vor diesem Index. Beispiel:

```
string last a 0a23456789abcdef 15
```

→ **10**,

```
string last a 0a23456789abcdef 9
```

→ **1**.

**string length *string***

Dezimalstring mit Zeichenanzahl (nicht zwingend Byteanzahl). Bei Byte-Array-Werten (z. B. binärer Kanal): tatsächliche Bytelänge.

**string map ?-nocase? *mapping string***

Ersetzt Teilstrings gemäß Schlüssel-Wert-Paaren in *mapping* (Liste *key value key value ...* wie array get). **-nocase**: ohne Groß-/Kleinschreibung. Schlüssel und Wert können mehrzeilig sein; Reihenfolge in *mapping* entscheidet; *string* wird nur einmal durchlaufen. Beispiel:

```
string map {abc 1 ab 2 a 3 1 0} 1abcaababcabababc
```

→ **01321221**.

Ist ein früherer *key* Präfix eines späteren, maskiert er diesen vollständig:

```
string map {1 0 ab 2 a 3 abc 1} 1abcaababcabababc
```

→ **02c322c222c**.

**string match ?-nocase? *pattern string***

Prüft, ob *pattern* auf *string* passt; 1 oder 0. **-nocase**: case-insensitive. Sondersequenzen in *pattern*:

*****

Beliebige Zeichenfolge in *string*, auch leer.

**?**

Genau ein Zeichen in *string*.

**[*chars*]**

Ein Zeichen aus *chars*. Bereich *x*-*y* inklusive. Mit **-nocase** werden Bereichsgrenzen zuerst kleingeschrieben; {[A-z]} trifft bei case-sensitive "_" (zwischen "Z" und "a"), mit **-nocase** eher wie {[A-Za-z]}.

**\*x***

Genau das Zeichen *x*; entzieht ***?[]** die Sonderbedeutung.

**string range *string first last***

Aufeinanderfolgende Zeichen von Index *first* bis *last* (String-Indizes). **0** = erstes, **end** = letztes. *first* < 0 → 0; *last* ≥ Länge → **end**. *first* > *last* → leerer String.

**string repeat *string count***

*string count*-mal verkettet; *count* 0 → leerer String.

**string replace *string first last* ?*newstring*?**

Entfernt Zeichen *first* bis *last*; optional *newstring* an dieser Stelle. Indizes wie **index**. *first* > *last*, *first* ≥ Länge oder *last* < 0: ursprünglicher String unverändert.

**string reverse *string***

Gleiche Länge, Zeichen in umgekehrter Reihenfolge.

**string tolower *string* ?*first*? ?*last*?**

*string* mit Kleinbuchstaben statt Groß-/Titelfall; optional Bereich *first*..*last* (String-Indizes).

**string totitle *string* ?*first*? ?*last*?**

Erstes Zeichen in Unicode-Titelfall (oder Großbuchstabe), Rest klein; optional Bereich.

**string toupper *string* ?*first*? ?*last*?**

Klein-/Titelfall → Großbuchstaben; optional Bereich.

**string trim *string* ?*chars*?**

*string* ohne führende/nachfolgende Zeichen aus *chars*; Standard: Leerzeichen (**string is space** = 1) und "\0".

**string trimleft *string* ?*chars*?**

Nur führende Zeichen aus *chars* entfernen; Standard wie **trim**.

**string trimright *string* ?*chars*?**

Nur nachfolgende Zeichen entfernen; Standard wie **trim**.

## Veraltete Unterbefehle

Unterstützt, voraussichtlich entfernt (selten genutzt oder irreführend):

**string wordend *string charIndex***

Index des Zeichens unmittelbar nach dem letzten des Worts um *charIndex* (String-Indizes). Wort: zusammenhängende alphanumerische Unicode-Zeichen oder Unterstrich (Verbinder-Interpunktion), sonst ein einzelnes anderes Zeichen.

**string wordstart *string charIndex***

Index des ersten Zeichens des Worts um *charIndex*; Wortdefinition wie oben.

## String-Indizes

Indizes in Zeichenketten (z. B. **string index**, **string range**):

***integer***

Jeder Index, der **string is integer -strict** besteht (z. B. **2** → "c" in "abcd").

**end**

Letztes Zeichen ("d" in "abcd").

**end-*N***

Letztes Zeichen minus *N* (**end-1** → "c").

**end+*N***

Letztes Zeichen plus *N* (**end+-1** → "c").

***M+N***

Zeichen an Index *M*+*N* (**1+1** → "c").

***M-N***

Zeichen an Index *M-N* (**2-1** → "b").

*M* hat keinen nachgestellten Leerraum, *N* keinen führenden.

### Beispiele

Prüfen, ob *string* ein echtes, nicht leeres Präfix von **foobar** ist:

```
set length [string length $string]
if {$length == 0} {
    set isPrefix 0
} else {
    set isPrefix [string equal -length $length $string "foobar"]
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## subst

*Stichwörter:* subst, backslash substitution, command substitution, quoting, substitution, variable substitution

### Name

subst - Führt Backslash-, Befehls- und Variablensubstitutionen durch

### Übersicht

**subst** ?**-nobackslashes**? ?**-nocommands**? ?**-novariables**? *string*

### Beschreibung

Dieser Befehl führt Variablen-, Befehls- und Backslash-Substitutionen an seinem Argument *string* durch und gibt das vollständig substituierte Ergebnis zurück. Die Substitutionen erfolgen genau wie bei Tcl-Befehlen. Dadurch wird das Argument *string* tatsächlich zweimal substituiert: einmal durch den Tcl-Parser wie üblich für Tcl-Befehle und erneut durch den Befehl *subst*.

Ist eine der Optionen **-nobackslashes**, **-nocommands** oder **-novariables** angegeben, werden die entsprechenden Substitutionen nicht durchgeführt. Ist zum Beispiel **-nocommands** gesetzt, findet keine Befehlssubstitution statt: öffnende und schließende eckige Klammern gelten als normale Zeichen ohne besondere Bedeutung.

Beachten Sie, dass die Substitution einer Art Substitutionen anderer Art einschließen kann. Ist zum Beispiel **-novariables** gesetzt, wird die Befehlssubstitution dennoch uneingeschränkt durchgeführt. Jede für die Befehlssubstitution nötige Variablensubstitution findet also weiter statt. Ebenso findet jede für eine Variablensubstitution nötige Befehlssubstitution statt, auch wenn **-nocommands** gesetzt ist. Siehe Beispiele unten.

Tritt während der Substitution ein Fehler auf, gibt **subst** diesen Fehler zurück. Tritt während einer Befehls- oder Variablensubstitution eine break-Ausnahme auf, ist das Ergebnis der gesamten Substitution die Zeichenkette (bereits substituiert) bis zum Beginn der Substitution, die die Ausnahme auslöste. Tritt während der Auswertung einer Befehls- oder Variablensubstitution eine continue-Ausnahme auf, wird für diese gesamte Befehls- oder Variablensubstitution eine leere Zeichenkette substituiert (sofern sie wohlgeformtes Tcl ist). Tritt eine return-Ausnahme auf oder wird während einer Befehls- oder Variablensubstitution ein anderer Rückgabecode zurückgegeben, wird der zurückgegebene Wert für diese Substitution eingesetzt. Siehe Beispiele unten. So "fängt" **subst** alle außergewöhnlichen Rückgabecodes. Der Befehl **subst** selbst gibt entweder einen Fehler zurück oder schließt erfolgreich ab.

## Beispiele

Bei seinen Substitutionen behandelt *subst* doppelte Anführungszeichen und geschweifte Klammern nicht besonders (außer innerhalb von Befehlssubstitutionen). Das Skript

```
set a 44
subst {xyz {$a}}
```

liefert "**xyz {44}**", nicht "**xyz {$a}**", und das Skript

```
set a "p\} q \{r"
subst {xyz {$a}}
```

liefert "**xyz {p} q {r}**", nicht "**xyz {p\} q \{r}**".

Bei Befehlssubstitution wird jede zur Auswertung des Skripts nötige Variablensubstitution einbezogen.

```
set a 44
subst -novariables {$a [format $a]}
```

liefert "**$a 44**", nicht "**$a $a**". Bei Variablensubstitution wird entsprechend jede zum Lesen des Variablenwerts nötige Befehlssubstitution einbezogen.

```
proc b {} {return c}
array set a {c c [b] tricky}
subst -nocommands {[b] $a([b])}
```

liefert "**[b] c**", nicht "**[b] tricky**".

Die Ausnahmen continue und break erlauben Befehlssubstitutionen, die Substitution des restlichen Befehlsteils bzw. des restlichen *string* zu verhindern und Skriptautoren mehr Möglichkeiten bei der Textverarbeitung mit *subst*. Das Skript

```
subst {abc,[break],def}
```

liefert "**abc,**", nicht "**abc,,def**", und das Skript

```
subst {abc,[continue;expr {1+2}],def}
```

liefert "**abc,,def**", nicht "**abc,3,def**".

Andere außergewöhnliche Rückgabecodes substituieren den zurückgegebenen Wert:

```
subst {abc,[return foo;expr {1+2}],def}
```

liefert "**abc,foo,def**", nicht "**abc,3,def**", und

```
subst {abc,[return -code 10 foo;expr {1+2}],def}
```

liefert ebenfalls "**abc,foo,def**", nicht "**abc,3,def**".

*Copyright (c) 1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2001 Donal K. Fellows*

# switch

*Stichwörter:* switch, match, regular expression

## Name

switch - Wertet eines von mehreren Skripten aus, abhängig von einem Wert

## Übersicht

**switch** ?*options*? *value pattern body* ?*pattern body* ...?
**switch** ?*options*? *value {pattern body ?pattern body ...?}*

## Beschreibung

**switch** vergleicht sein Argument *value* nacheinander mit jedem *pattern*. Sobald ein *pattern* zu *value* passt, wird das folgende *body* rekursiv ausgewertet; dessen Ergebnis liefert **switch**. Ist das letzte *pattern* **default**, passt es auf alles. Passt kein *pattern* und gibt es kein **default**, gibt **switch** eine leere Zeichenkette zurück.

Beginnen die ersten Argumente mit **-**, gelten sie als Optionen — außer es gibt genau zwei Argumente (dann ist das erste *value*, das zweite die *pattern*/*body*-Liste). Unterstützte Optionen:

**-exact**
Exakter Vergleich von *value* mit dem Muster (Standard).

**-glob**
Glob-Vergleich (wie string match).

**-regexp**
Reguläre Ausdrücke (siehe re_syntax).

**-nocase**
Vergleich ohne Berücksichtigung der Groß-/Kleinschreibung.

**-matchvar *varName***
Nur mit **-regexp**: Variable erhält die Trefferliste der Engine (gesamtes Match, dann Capturing Groups). Bei **default**: leere Liste. Kombinierbar mit **-indexvar**.

**-indexvar *varName***
Nur mit **-regexp**: Indizes der Treffer (zwei Elemente Start/Ende pro Teilstring, analog zur Option **-indices** von regexp). Bei **default**: leere Liste. Kombinierbar mit **-matchvar**.

**--**
Ende der Optionen; das folgende Argument ist *value*, auch wenn es mit **-** beginnt. Nicht nötig, wenn Muster und Körper in einem Listenargument stehen.

Zwei Syntaxen für *pattern* und *body*: getrennte Argumente (Substitutionen in Mustern/Körpern möglich) oder eine Liste aller Paare (mehrzeilige **switch**-Befehle; Muster in geschweiften Klammern ohne Substitution — Verhalten kann abweichen).

Ist *body* als **-** angegeben, gilt der *body* des nächsten Musters auch hier (Kette von **-** möglich). So teilen sich mehrere Muster einen *body*.

Kommentare in **switch** nur **innerhalb** des Ausführungskörpers eines Musters, nicht zwischen den Mustern.

## Beispiele

Abgleich mit Variablen (Ergebnis *2*):

```
set foo "abc"
switch abc a - b {expr {1}} $foo {expr {2}} default {expr {3}}
```

Glob und Durchfall (**-** als *body*); Ergebnis *1*:

```
switch -glob aaab {
    a*b     -
    b       {expr {1}}
    a*      {expr {2}}
    default {expr {3}}
}
```

**default** (muss zuletzt stehen); Ergebnis *3*:

```
switch xyz {
    a -
    b {
        # Kommentar hier erlaubt
        expr {1}
    }
    c {
        expr {2}
    }
    default {
        expr {3}
    }
}
```

**-matchvar** bei regulären Ausdrücken:

```
switch -regexp -matchvar foo -- $bar {
    a(b*)c {
        puts "Found [string length [lindex $foo 1]] 'b's"
    }
    d(e*)f(g*)h {
        puts "Found [string length [lindex $foo 1]] 'e's and\
                [string length [lindex $foo 2]] 'g's"
    }
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# tailcall

*Stichwörter:* tailcall, call, recursion, tail recursion

## Name

tailcall - Ersetzt die aktuelle Prozedur durch einen anderen Befehl

## Übersicht

**tailcall** *command* ?*arg* ...?

## Beschreibung

Der Befehl **tailcall** ersetzt die gerade ausgeführte Prozedur, Lambda-Anwendung oder Methode durch einen anderen Befehl. *command* wird mit den Argumenten *arg ...* (falls angegeben) aufgerufen und im aktuellen Namespace-Kontext aufgelöst, nicht in dem des Aufrufers. Abgesehen von dieser Auflösungsdifferenz ist er äquivalent zu:

```
return [uplevel 1 [list command ?arg ...?]]
```

Dieser Befehl darf nicht aus einem uplevel in eine Prozedur oder aus einem catch innerhalb einer Prozedur oder eines Lambdas aufgerufen werden.

## Beispiele

Berechnet die Fakultät einer Zahl.

```
proc factorial {n {accum 1}} {
    if {$n < 2} {
        return $accum
    }
    tailcall factorial [expr {$n - 1}] [expr {$accum * $n}]
}
```

Gibt die Elemente einer Liste aus, wobei abwechselnde Zeilen unterschiedliche Einrückungen haben.

```
proc printList {theList} {
    if {[llength $theList]} {
        puts "> [lindex $theList 0]"
        tailcall printList2 [lrange $theList 1 end]
    }
}
proc printList2 {theList} {
    if {[llength $theList]} {
        puts "< [lindex $theList 0]"
        tailcall printList [lrange $theList 1 end]
    }
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# tclsh

*Stichwörter:* tclsh, application, argument, interpreter, prompt, script file, shell, zipfs

## Name

tclsh - Einfache Shell mit Tcl-Interpreter

## Übersicht

**tclsh** ?*-encoding name*? ?*fileName arg arg* ...?

## Beschreibung

**Tclsh** ist eine shell-artige Anwendung, die Tcl-Befehle von der Standardeingabe oder aus einer Datei liest und auswertet. Ohne Argumente läuft sie interaktiv, liest Tcl-Befehle von der Standardeingabe und schreibt Ergebnisse und Fehlermeldungen auf die Standardausgabe. Sie läuft, bis der Befehl exit aufgerufen wird oder die Standardeingabe das Dateiende erreicht. Liegt im Home-Verzeichnis des Benutzers eine Datei **.tclshrc** (unter Windows **tclshrc.tcl**), wertet interaktives **tclsh** diese Datei als Tcl-Skript aus, unmittelbar bevor der erste Befehl von der Standardeingabe gelesen wird.

## Skriptdateien

Wird **tclsh** mit Argumenten aufgerufen, nennen die ersten Argumente den Namen einer Skriptdatei und optional die Kodierung der darin gespeicherten Textdaten. Weitere Argumente stehen dem Skript als Variablen zur Verfügung (siehe unten). Statt von der Standardeingabe liest **tclsh** die Tcl-Befehle aus der genannten Datei und beendet sich an ihrem Ende. Das Ende der Datei ist das physische Ende des Mediums oder das Zeichen ("\x1A", Strg-Z). Steht dieses Zeichen in der Datei, liest **tclsh** den Text bis unmittelbar vor dieses Zeichen. Eine Anwendung, die das Zeichen in der Datei braucht, kann es gefahrlos als "\x1A" kodieren oder mit Befehlen wie format oder binary erzeugen. Eine automatische Auswertung von **.tclshrc** findet nicht statt, wenn auf der Befehlszeile von **tclsh** ein Skriptdateiname steht; die Skriptdatei kann die Datei bei Bedarf mit source laden.

Steht in der ersten Zeile eines Tcl-Skripts

```
#!/usr/local/bin/tclsh
```

lässt sich die Datei direkt aus der Shell aufrufen, sobald sie als ausführbar gekennzeichnet ist. Das setzt voraus, dass **tclsh** am üblichen Ort /usr/local/bin installiert ist; sonst muss die Zeile angepasst werden. Viele UNIX-Systeme lassen die Zeile **#!** nur etwa 30 Zeichen lang werden. Die ausführbare Datei **tclsh** muss daher über einen kurzen Dateinamen erreichbar sein.

Besser beginnen Skriptdateien mit diesen drei Zeilen:

```
#!/bin/sh
# the next line restarts using tclsh \
exec tclsh "$0" ${1+"$@"}
```

Das hat gegenüber dem vorigen Absatz drei Vorteile. Erstens muss der Ort der **tclsh**-Binärdatei nicht fest im Skript stehen: sie darf irgendwo im Suchpfad der Shell liegen. Zweitens entfällt die Grenze von etwa 30 Zeichen für den Dateinamen. Drittens funktioniert das auch, wenn **tclsh** selbst ein Shell-Skript ist (auf manchen Systemen wählt das **tclsh**-Skript eine von mehreren Binärdateien, um mehrere Architekturen oder Betriebssysteme zu bedienen). Die drei Zeilen lassen sowohl **sh** als auch **tclsh** das Skript verarbeiten, exec führt aber nur **sh** aus. **sh** verarbeitet das Skript zuerst, behandelt die zweite Zeile als Kommentar und führt die dritte aus. exec beendet die Verarbeitung durch die Shell und startet **tclsh**, das das ganze Skript erneut verarbeitet. Beim Start von **tclsh** gelten alle drei Zeilen als Kommentar, weil der Backslash am Ende der zweiten Zeile die dritte Zeile zum Kommentar der zweiten macht.

Häufig wird tclsh mit der Versionsnummer im Namen installiert. Das erlaubt mehrere Tcl-Versionen auf demselben System, macht es aber schwerer, Skripte zu schreiben, die über verschiedene Tcl-Versionen einheitlich starten.

Alternativ findet /usr/bin/env tclsh im Pfad:

```
#!/usr/bin/env tclsh
```

Das ist kurz und einfach, aber unflexibel, wenn die Binärdatei unterschiedliche Namen trägt.

## Variablen

**Tclsh** setzt zusätzlich zu den Variablen der Tcl-Bibliothek (etwa env, das Umgebungsvariablen wie **PATH** nach Tcl abbildet) diese globalen Tcl-Variablen:

argc
    Anzahl der *arg*-Argumente (0, wenn keine), ohne den Namen der Skriptdatei.

argv
    Tcl-Liste der *arg*-Argumente in derselben Reihenfolge, oder eine leere Zeichenkette, wenn keine *arg*-Argumente da sind.

argv0
    *fileName*, wenn er angegeben wurde. Sonst der Name, mit dem **tclsh** aufgerufen wurde.

tcl_interactive
    1, wenn **tclsh** interaktiv läuft (kein *fileName* und die Standardeingabe ist ein terminalartiges Gerät), sonst 0.

## Eingabeaufforderung

Interaktives **tclsh** fordert jeden Befehl mit "**%** " an. Die globalen Variablen **tcl_prompt1** und **tcl_prompt2** ändern die Aufforderung. Existiert **tcl_prompt1**, muss sie ein Tcl-Skript sein, das die Aufforderung ausgibt; statt selbst eine Aufforderung zu schreiben, wertet **tclsh** das Skript in **tcl_prompt1** aus. **tcl_prompt2** gilt entsprechend, wenn ein Zeilenumbruch kommt, der aktuelle Befehl aber noch nicht vollständig ist. Ist **tcl_prompt2** nicht gesetzt, gibt es für unvollständige Befehle keine Aufforderung.

## Standardkanäle

Weitere Erläuterungen zum Standardkanal stehen bei Tcl_StandardChannels.

## Virtuelles Dateisystem Zipfs

Wird eine Zip-Datei an das Ende von **tclsh** angehängt, wird ihr Inhalt beim Start unter einem virtuellen Dateisystem (VFS) eingehängt. Die Wurzel dieses VFS liefert der Befehl zipfs root. Das Archiv liegt im Verzeichnis **app** innerhalb des VFS. Liegt im obersten Verzeichnis des Archivs eine Datei **main.tcl**, wird sie geladen statt der üblichen Befehlszeilenverarbeitung von tclsh. Liegt dort ein Verzeichnis tcl_library, wird es als env(TCL_LIBRARY) geladen. Liegt im obersten Verzeichnis des Archivs eine Datei **main.tcl**, wird sie geladen statt der üblichen Befehlszeilenverarbeitung der Shell.

An das Ende eines Programmbilds (tclsh oder wish) kann nur eine Zip-Datei angehängt werden. Sind mehrere angehängt, gilt nur die letzte. Dieses Dateisystem ist nur lesbar. Dateien darin lassen sich nicht hinzufügen oder ändern. Einzelheiten stehen in zipfs(n).

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# tcltest

*Stichwörter:* tcltest, test, test harness, test suite

## Name

tcltest - Unterstützungscode und Hilfsbefehle für Test-Harness

## Übersicht

**package require tcltest** ?**2.5**?

**tcltest::test** *name description* ?*-option value* ...?
**tcltest::test** *name description* ?*constraints*? *body result*

**tcltest::loadTestedCommands**
**tcltest::makeDirectory** *name* ?*directory*?
**tcltest::removeDirectory** *name* ?*directory*?
**tcltest::makeFile** *contents name* ?*directory*?
**tcltest::removeFile** *name* ?*directory*?
**tcltest::viewFile** *name* ?*directory*?
**tcltest::cleanupTests** ?*runningMultipleTests*?
**tcltest::runAllTests**

**tcltest::configure**
**tcltest::configure** *-option*
**tcltest::configure** *-option value* ?*-option value* ...?
**tcltest::customMatch** *mode command*
**tcltest::testConstraint** *constraint* ?*value*?
**tcltest::outputChannel** ?*channel*?
**tcltest::errorChannel** ?*channel*?
**tcltest::interpreter** ?*interp*?

**tcltest::debug** ?*level*?
**tcltest::errorFile** ?*filename*?
**tcltest::limitConstraints** ?*boolean*?
**tcltest::loadFile** ?*filename*?
**tcltest::loadScript** ?*script*?
**tcltest::match** ?*patternList*?
**tcltest::matchDirectories** ?*patternList*?
**tcltest::matchFiles** ?*patternList*?
**tcltest::outputFile** ?*filename*?
**tcltest::preserveCore** ?*level*?
**tcltest::singleProcess** ?*boolean*?
**tcltest::skip** ?*patternList*?
**tcltest::skipDirectories** ?*patternList*?

**tcltest::skipFiles** ?*patternList*?
**tcltest::temporaryDirectory** ?*directory*?
**tcltest::testsDirectory** ?*directory*?
**tcltest::verbose** ?*level*?

**tcltest::test** *name description optionList*
**tcltest::normalizeMsg** *msg*
**tcltest::normalizePath** *pathVar*
**tcltest::workingDirectory** ?*dir*?

## Beschreibung

Das Paket **tcltest** stellt mehrere Hilfsbefehle bereit, die beim Aufbau von Test-Suites für Code nützlich sind, der durch Auswertung von Tcl-Befehlen getestet werden soll. Insbesondere werden die eingebauten Befehle der Tcl-Bibliothek selbst mit einer Test-Suite getestet, die das Paket tcltest verwendet.

Alle vom Paket **tcltest** bereitgestellten Befehle sind im Namespace **::tcltest** definiert und werden von dort exportiert, wie in der Übersicht oben angegeben. In den folgenden Abschnitten werden alle Befehle der Kürze halber mit ihren einfachen Namen beschrieben.

Der zentrale Befehl von **tcltest** ist **test**, der einen Test definiert und ausführt. Tests mit **test** umfassen die Auswertung eines Tcl-Skripts und den Vergleich des Ergebnisses mit einem erwarteten Ergebnis, wie durch zahlreiche Optionen konfiguriert und gesteuert. Weitere Befehle von **tcltest** steuern die Konfiguration von **test** und das Zusammenfassen vieler **test**-Befehle zu Test-Suites.

Siehe Test-Suites mit tcltest erstellen unten für ein ausführliches Beispiel, wie die Befehle von **tcltest** zum Erzeugen von Test-Suites für Tcl-basierten Code verwendet werden.

## Befehle

**test *name description* ?*-option value* ...?**

Definiert und führt gegebenenfalls einen Test mit dem Namen *name* und der Beschreibung *description* aus. Name und Beschreibung eines Tests werden in von **test** während des Tests gemeldeten Meldungen verwendet, wie durch die Optionen von **tcltest** konfiguriert. Die übrigen *option value*-Argumente von **test** definieren den Test, einschließlich der auszuführenden Skripte, der Bedingungen für deren Ausführung, des erwarteten Ergebnisses und der Art, wie erwartete und tatsächliche Ergebnisse verglichen werden sollen. Siehe Tests unten für eine vollständige Beschreibung der gültigen Optionen und wie sie einen Test definieren. Der Befehl **test** gibt einen leeren String zurück.

**test *name description* ?*constraints*? *body result***

Diese Form von **test** unterstützt Test-Suites, die für Version 1 des Pakets **tcltest** geschrieben wurden, und bietet außerdem eine einfachere Schnittstelle für häufige Verwendung. Sie entspricht "**test** *name description* **-constraints** *constraints* **-body** *body* **-result** *result*". Alle anderen Optionen von **test** behalten ihre Standardwerte. Wenn *constraints* weggelassen wird, lässt sich diese Form von **test** von der ersten unterscheiden, weil alle *option*-Namen mit "-" beginnen.

**loadTestedCommands**

Wertet im Kontext des Aufrufers das von **configure -load** oder **configure -loadfile** angegebene Skript aus. Gibt das Ergebnis dieser Skriptauswertung zurück, einschließlich jedes vom Skript ausgelösten Fehlers. Verwenden Sie diesen Befehl und die zugehörigen Konfigurationsoptionen, um die zu testenden Befehle dem Interpreter bereitzustellen, der die Test-Suite ausführt.

**makeFile *contents name* ?*directory*?**

Erstellt eine Datei namens *name* relativ zum Verzeichnis *directory* und schreibt *contents* mit der Kodierung encoding system in diese Datei. Endet *contents* nicht mit einem Zeilenumbruch, wird einer angehängt, sodass die Datei *name* mit einem Zeilenumbruch endet. Da die Systemkodierung verwendet wird, eignet sich dieser Befehl nur zum Erzeugen von Textdateien. Die Datei wird bei der nächsten Auswertung von **cleanupTests** entfernt, sofern sie nicht zuvor mit **removeFile** gelöscht wurde. Der Standardwert von *directory* ist das Verzeichnis **configure -tmpdir**. Gibt den vollständigen Pfad der erstellten Datei zurück. Verwenden Sie diesen Befehl, um beliebige vom Test benötigte Textdateien mit passendem Inhalt zu erzeugen.

**removeFile *name* ?*directory*?**

Erzwingt das Entfernen der durch *name* referenzierten Datei. Der Dateiname sollte relativ zu *directory* sein. Der Standardwert von *directory* ist das Verzeichnis **configure -tmpdir**. Gibt einen leeren String zurück. Verwenden Sie diesen Befehl, um mit **makeFile** erstellte Dateien zu löschen.

**makeDirectory *name* ?*directory*?**

Erstellt ein Verzeichnis namens *name* relativ zum Verzeichnis *directory*. Das Verzeichnis wird bei der nächsten Auswertung von **cleanupTests** entfernt, sofern es nicht zuvor mit **removeDirectory** gelöscht wurde. Der Standardwert von *directory* ist das Verzeichnis **configure -tmpdir**. Gibt den vollständigen Pfad des erstellten Verzeichnisses zurück. Verwenden Sie diesen Befehl, um alle von einem Test benötigten Verzeichnisse zu erzeugen.

**removeDirectory *name* ?*directory*?**

Erzwingt das Entfernen des durch *name* referenzierten Verzeichnisses. Das Verzeichnis sollte relativ zu *directory* sein. Der Standardwert von *directory* ist das Verzeichnis **configure -tmpdir**. Gibt einen leeren String zurück. Verwenden Sie diesen Befehl, um mit **makeDirectory** erstellte Verzeichnisse zu löschen.

**viewFile *file* ?*directory*?**

Gibt den Inhalt von *file* zurück, ohne einen abschließenden Zeilenumbruch, wie read -nonewline ihn liefern würde. Der Dateiname sollte relativ zu *directory* sein. Der Standardwert von *directory* ist das Verzeichnis **configure -tmpdir**. Verwenden Sie diesen Befehl, um den Inhalt einer vom Test erzeugten Datei bequem als Testergebnis für den Abgleich mit einem erwarteten Ergebnis zu verwenden. Der Dateiinhalt wird mit der Systemkodierung gelesen; die Nützlichkeit beschränkt sich daher auf Textdateien.

**cleanupTests**

Dient dazu, nach mehreren ausgeführten Tests aufzuräumen und zusammenzufassen. Wird typischerweise einmal pro Testdatei am Ende der Datei aufgerufen, nachdem alle Tests abgeschlossen sind. Damit er zuverlässig wirkt, sollte **cleanupTests** auch dann ausgewertet werden, wenn zuvor bei der Auswertung der Testdatei ein Fehler auftrat.

Gibt Statistiken über die ausgeführten Tests aus und entfernt Dateien, die seit dem letzten **cleanupTests** mit **makeDirectory** und **makeFile** erstellt wurden. Namen von Dateien und Verzeichnissen im Verzeichnis **configure -tmpdir**, die seit dem letzten **cleanupTests** entstanden, aber nicht mit **makeFile** oder **makeDirectory** erzeugt wurden, werden auf **outputChannel** ausgegeben. Der Befehl stellt außerdem die ursprüngliche Shell-Umgebung wieder her, wie im globalen Array env beschrieben. Gibt einen leeren String zurück.

**runAllTests**

Hauptbefehl zum Ausführen einer gesamten Test-Suite über mehrere Dateien und/oder Verzeichnisse, gesteuert durch die konfigurierbaren Optionen von **tcltest**. Siehe Alle Tests ausführen unten für eine vollständige Beschreibung der vielen Varianten mit **runAllTests**.

## Konfigurationsbefehle

**configure**

Gibt die Liste der von **tcltest** unterstützten konfigurierbaren Optionen zurück. Siehe Konfigurierbare Optionen unten für die vollständige Liste der Optionen, ihrer gültigen Werte und ihrer Wirkung auf **tcltest**-Operationen.

**configure *option***

Gibt den aktuellen Wert der unterstützten konfigurierbaren Option *option* zurück. Löst einen Fehler aus, wenn *option* keine unterstützte konfigurierbare Option ist.

**configure *option value* ?*-option value* ...?**

Setzt den Wert jeder konfigurierbaren Option *option* der Reihe nach auf den entsprechenden Wert *value*. Löst einen Fehler aus, wenn eine *option* keine unterstützte konfigurierbare Option ist, wenn *value* kein gültiger Wert für die entsprechende *option* ist oder wenn kein *value* angegeben wurde. Bei einem Fehler wird die Operation von **configure** abgebrochen; nachfolgende *option value*-Argumente werden nicht verarbeitet.

Existiert die Umgebungsvariable **::env(TCLTEST_OPTIONS)**, wenn das Paket **tcltest** geladen wird (durch package require **tcltest**), wird deren Wert als Argumentliste für **configure** verwendet. So können Standardwerte der Konfigurationsoptionen über die Umgebung gesetzt werden.

**customMatch *mode script***

Registriert *mode* als neuen gültigen Wert der **-match**-Option von **test**. Wird die Option **-match** *mode* an **test** übergeben, wird das Skript *script* ausgewertet, um das tatsächliche Ergebnis der Auswertung des Testkörpers mit dem erwarteten Ergebnis zu vergleichen. Für den Abgleich wird das *script* um zwei zusätzliche Wörter ergänzt — das erwartete und das tatsächliche Ergebnis — und das vollständige Skript im globalen Namespace ausgewertet. Das vollständige Skript soll einen booleschen Wert zurückgeben, der angibt, ob die Ergebnisse übereinstimmen. Die eingebauten Abgleichsmodi von **test** sind **exact**, glob und regexp.

**testConstraint *constraint* ?*boolean*?**

Setzt oder gibt den booleschen Wert zurück, der der benannten *constraint* zugeordnet ist. Siehe Test-Einschränkungen unten für weitere Informationen.

**interpreter ?*executableName*?**

Setzt oder gibt den Namen der ausführbaren Datei zurück, die von **runAllTests** per exec gestartet wird, um jede Testdatei auszuführen, wenn **configure -singleproc** false ist. Der Standardwert für **interpreter** ist der Name des aktuell laufenden Programms, wie von info nameofexecutable zurückgegeben.

**outputChannel ?*channel*?**

Setzt oder gibt den Ausgabekanal zurück. Standard ist **stdout**. Jeder Test, der testbezogene Ausgabe erzeugt, sollte diese an **outputChannel** senden, statt sie standardmäßig nach **stdout** zu leiten.

**errorChannel ?*channel*?**
Setzt oder gibt den Fehlerkanal zurück. Standard ist **stderr**. Jeder Test, der Fehlermeldungen ausgibt, sollte diese an **errorChannel** senden, statt direkt nach **stderr** zu schreiben.

## Kurzbefehle für die Konfiguration

**debug ?*level*?**
Entspricht "**configure -debug** ?*level*?".

**errorFile ?*filename*?**
Entspricht "**configure -errfile** ?*filename*?".

**limitConstraints ?*boolean*?**
Entspricht "**configure -limitconstraints** ?*boolean*?".

**loadFile ?*filename*?**
Entspricht "**configure -loadfile** ?*filename*?".

**loadScript ?*script*?**
Entspricht "**configure -load** ?*script*?".

**match ?*patternList*?**
Entspricht "**configure -match** ?*patternList*?".

**matchDirectories ?*patternList*?**
Entspricht "**configure -relateddir** ?*patternList*?".

**matchFiles ?*patternList*?**
Entspricht "**configure -file** ?*patternList*?".

**outputFile ?*filename*?**
Entspricht "**configure -outfile** ?*filename*?".

**preserveCore ?*level*?**
Entspricht "**configure -preservecore** ?*level*?".

**singleProcess ?*boolean*?**
Entspricht "**configure -singleproc** ?*boolean*?".

**skip ?*patternList*?**
Entspricht "**configure -skip** ?*patternList*?".

**skipDirectories ?*patternList*?**
Entspricht "**configure -asidefromdir** ?*patternList*?".

**skipFiles ?*patternList*?**
Entspricht "**configure -notfile** ?*patternList*?".

**temporaryDirectory ?*directory*?**
Entspricht "**configure -tmpdir** ?*directory*?".

**testsDirectory ?*directory*?**
Entspricht "**configure -testdir** ?*directory*?".

**verbose ?*level*?**
Entspricht "**configure -verbose** ?*level*?".

## Weitere Befehle

Die übrigen von **tcltest** bereitgestellten Befehle haben bessere Alternativen in **tcltest** oder **Tcl** selbst. Sie bleiben zur Unterstützung bestehender Test-Suites erhalten, sollten in neuem Code aber vermieden werden.

**test *name description optionList***

Diese Form von **test** ermöglichte, viele über mehrere Zeilen verteilte Optionen als ein einziges in geschweiften Klammern gesetztes Argument an **test** zu übergeben, statt Zeilenumbrüche zwischen Argumenten mit Backslash zu quoten. Das Argument *optionList* soll eine Liste mit gerader Elementanzahl sein, die *option*- und *value*-Argumente für **test** darstellt. Diese Werte werden jedoch nicht direkt übergeben, wie bei den alternativen Formen von switch. Stattdessen versucht diese Form leider, Tcls Ersetzungsregeln zu umgehen, indem auf einigen Listenelementen Ersetzungen vorgenommen werden, um eine "mach-was-ich-meine"-Interpretation eines in Klammern gesetzten "Blocks" zu implementieren. Das Ergebnis ist nahezu undokumentierbar; diese Form wird daher nicht empfohlen. Die Beispiele in Test-Suites mit tcltest erstellen unten zeigen, dass diese Form wirklich nicht nötig ist, um Backslash-quotierte Zeilenumbrüche zu vermeiden. Wenn Sie diese Form dennoch verwenden wollen, untersuchen Sie den Quellcode von **tcltest** für Ersetzungsdetails, oder setzen Sie das dritte bis letzte Argument von **test** in Klammern und hoffen Sie auf das Beste.

**workingDirectory ?*directoryName*?**

Setzt oder gibt das aktuelle Arbeitsverzeichnis während der Ausführung der Test-Suite zurück. Der Standardwert für workingDirectory ist das Verzeichnis, in dem die Test-Suite gestartet wurde. Die Tcl-Befehle cd und pwd sind ausreichende Ersatzlösungen.

**normalizeMsg *msg***

Gibt das Ergebnis zurück, *msg* von "überflüssigen" Zeilenumbrüchen zu befreien — wobei "überflüssig" recht unpräzise ist. Tcl bietet zahlreiche String-Befehle zur beliebigen Modifikation von Strings, und **customMatch** ermöglicht flexiblen Abgleich tatsächlicher und erwarteter Ergebnisse.

**normalizePath *pathVar***

Löst symbolische Links in einem Pfad auf und erzeugt so einen Pfad ohne interne Umleitungen. Es wird angenommen, dass *pathVar* absolut ist. *pathVar* wird an Ort und Stelle geändert. Der Tcl-Befehl file normalize ist eine ausreichende Ersatzlösung.

## Tests

Der Befehl **test** ist das Herzstück des Pakets **tcltest**. Seine wesentliche Funktion ist die Auswertung eines Tcl-Skripts und der Vergleich des Ergebnisses mit einem erwarteten Ergebnis. Die Optionen von **test** definieren das Testskript, die Umgebung für die Auswertung, das erwartete Ergebnis und die Art des Vergleichs zwischen tatsächlichem und erwartetem Ergebnis. Einige Konfigurationsoptionen von **tcltest** beeinflussen ebenfalls das Verhalten von **test**.

Die gültigen Optionen von **test** sind zusammengefasst:

```
test name description
        ?-constraints keywordList|expression?
        ?-setup setupScript?
        ?-body testScript?
        ?-cleanup cleanupScript?
        ?-result expectedAnswer?
        ?-output expectedOutput?
        ?-errorOutput expectedError?
        ?-returnCodes codeList?
        ?-errorCode expectedErrorCode?
        ?-match mode?
```

Der *name* kann ein beliebiger String sein. Üblich ist die Wahl eines *name* nach dem Muster:

```
target-majorNum.minorNum
```

Bei White-Box-(Regressions-)Tests sollte das Ziel der Name der getesteten C-Funktion oder Tcl-Prozedur sein. Bei Black-Box-Tests sollte das Ziel der Name des getesteten Merkmals sein. Einige Konventionen verlangen für Black-Box-Tests das Suffix **_bb**. Verwandte Tests sollten dieselbe Hauptnummer teilen. Mit der Entwicklung einer Test-Suite ist es am besten, wenn derselbe Testname weiterhin demselben Test entspricht, sodass Aussagen wie "Test foo-1.3 bestand in allen Versionen bis 3.4, schlug aber ab Version 3.5 fehl" sinnvoll bleiben.

Bei der Auswertung von **test** wird der *name* mit den von **configure -match** und **configure -skip** zurückgegebenen Listen von String-Abgleichmustern verglichen. Der Test wird nur ausgeführt, wenn *name* mindestens einem Muster von **configure -match** entspricht und keinem Muster von **configure -skip**.

Die *description* sollte eine kurze textuelle Beschreibung des Tests sein. Die *description* erscheint in der vom Test erzeugten Ausgabe, typischerweise in Fehlermeldungen bei Testfehlern. Gute *description*-Werte erklären Nutzern einer Test-Suite kurz den Zweck des Tests. Bei Regressions-Tests sollte der Name der getesteten Tcl- oder C-Funktion in der Beschreibung stehen. Existiert der Testfall, um einen Fehler zu reproduzieren, gehört die Fehler-ID in die Beschreibung.

Gültige Attribute und zugehörige Werte sind:

**-constraints *keywordList*|*expression***

Das optionale Attribut **-constraints** kann eine Liste aus einem oder mehreren Schlüsselwörtern oder ein Ausdruck sein. Ist der **-constraints**-Wert eine Schlüsselwortliste, sollte jedes Schlüsselwort der Name einer durch **testConstraint** definierten Einschränkung sein. Ist eine der gelisteten Einschränkungen false oder existiert nicht, wird der Test übersprungen. Ist der **-constraints**-Wert ein Ausdruck, wird dieser ausgewertet. Wertet der Ausdruck zu true aus, wird der Test ausgeführt.

Beachten Sie, dass die Ausdrucksform von **-constraints** mit **configure -constraints** und **configure -limitconstraints** interferieren kann und nicht empfohlen wird.

Tests, die nicht immer laufen sollen, sollten passende Einschränkungen erhalten. Bedingte Ausführung eines Tests soll über die Option **-constraints** erfolgen, nicht durch bedingte Auswertung von **test**. So meldet die Test-Suite stets dieselbe Anzahl Tests; die Zahl übersprungener Tests kann je nach Testumgebung variieren. Standardwert ist eine leere Liste. Siehe Test-Einschränkungen unten für eingebaute Einschränkungen und das Hinzufügen eigener Einschränkungen.

**-setup *script***

Das optionale Attribut **-setup** gibt ein *script* an, das vor dem durch **-body** angegebenen Skript ausgeführt wird. Löst die Auswertung von *script* einen Fehler aus, schlägt der Test fehl. Standardwert ist ein leeres Skript.

**-body *script***

Das Attribut **-body** gibt das *script* an, das den Test ausführt und ein auf Korrektheit prüfbares Ergebnis zurückgeben muss. Löst die Auswertung von *script* einen Fehler aus, schlägt der Test fehl (es sei denn, **-returnCodes** legt fest, dass ein Fehler erwartet wird). Standardwert ist ein leeres Skript.

**-cleanup *script***

Das optionale Attribut **-cleanup** gibt ein *script* an, das nach dem durch **-body** angegebenen Skript ausgeführt wird. Löst die Auswertung von *script* einen Fehler aus, schlägt der Test fehl. Standardwert ist ein leeres Skript.

**-match *mode***

Das Attribut **-match** legt fest, wie die von **-result**, **-output** und **-errorOutput** gelieferten erwarteten Antworten verglichen werden. Gültige Werte für *mode* sind regexp, glob, **exact** und jeder durch einen früheren Aufruf von **customMatch** registrierte Wert. Standardwert ist **exact**.

**-result *expectedValue***

Das Attribut **-result** liefert den *expectedValue*, mit dem der Rückgabewert des Skripts verglichen wird. Standardwert ist ein leerer String.

**-output *expectedValue***

Das Attribut **-output** liefert den *expectedValue*, mit dem jede während der Auswertung der Skripte an **stdout** oder **outputChannel** gesendete Ausgabe verglichen wird. Beachten Sie, dass nur Ausgabe des globalen Befehls puts zum Vergleich verwendet wird. Ist **-output** nicht angegeben, wird an **stdout** und **outputChannel** gesendete Ausgabe nicht zum Vergleich verarbeitet.

**-errorOutput *expectedValue***

Das Attribut **-errorOutput** liefert den *expectedValue*, mit dem jede während der Auswertung der Skripte an **stderr** oder **errorChannel** gesendete Ausgabe verglichen wird. Beachten Sie, dass nur Ausgabe des globalen Befehls puts zum Vergleich verwendet wird. Ist **-errorOutput** nicht angegeben, wird an **stderr** und **errorChannel** gesendete Ausgabe nicht zum Vergleich verarbeitet.

**-returnCodes *expectedCodeList***

Das optionale Attribut **-returnCodes** liefert *expectedCodeList*, eine Liste von Rückgabecodes, die bei der Auswertung des **-body**-Skripts akzeptiert werden dürfen. Liefert die Auswertung des **-body**-Skripts einen Code außerhalb von *expectedCodeList*, schlägt der Test fehl. Alle return bekannten Rückgabecodes in numerischer und symbolischer Form, einschließlich erweiterter Rückgabecodes, sind zulässige Elemente in *expectedCodeList*. Standardwert ist " **ok return**".

**-errorCode *expectedErrorCode***

Das optionale Attribut **-errorCode** liefert *expectedErrorCode*, ein Glob-Muster, das zum bei der Auswertung des **-body**-Skripts gemeldeten Fehlercode passen soll. Liefert die Auswertung des **-body**-Skripts keinen zu *expectedErrorCode* passenden Code, schlägt der Test fehl. Standardwert ist "*". Enthält **-returnCodes** nicht **error**, wird es auf **error** gesetzt.

Damit ein Test besteht, müssen **-setup**, **-body** und **-cleanup** erfolgreich ausgewertet werden.

Rückgabecode und Ergebnis des **-body**-Skripts müssen den erwarteten Werten entsprechen; sind angegeben, müssen Ausgabe- und Fehlerdaten des Tests den erwarteten **-output**- und **-errorOutput**-Werten entsprechen. Ist eine dieser Bedingungen nicht erfüllt, schlägt der Test fehl. Beachten Sie, dass alle Skripte im Kontext des Aufrufers von **test** ausgewertet werden.

Solange **test** mit gültiger Syntax und legalen Werten für alle Attribute aufgerufen wird, löst es keinen Fehler aus. Testfehler werden stattdessen als Ausgabe auf **outputChannel** gemeldet. Im Standardbetrieb erzeugt ein erfolgreicher Test keine Ausgabe. Die von **test** erzeugten Meldungen werden durch **configure -verbose** gesteuert, wie in Konfigurierbare Optionen unten beschrieben. Ausgabe der Testskripte selbst sollte mit puts nach **outputChannel** oder **errorChannel** erfolgen, damit Nutzer der Test-Suite Ausgabe leicht mit **configure -outfile** und **configure -errfile** erfassen können und die Attribute **-output** und **-errorOutput** korrekt funktionieren.

## Test-Einschränkungen

Einschränkungen bestimmen, ob ein Test übersprungen werden soll. Jede Einschränkung hat einen Namen (beliebiger String) und einen booleschen Wert. Jeder **test** hat einen **-constraints** -Wert als Liste von Einschränkungsnamen. Es gibt zwei Modi der Einschränkungssteuerung. Am häufigsten wird der Standardmodus verwendet, angezeigt durch **configure -limitconstraints** false. Der Test läuft nur, wenn alle Einschränkungen in der Liste true sind. Die **-constraints** -Option von **test** ist daher eine bequeme, symbolische Möglichkeit, Bedingungen für einen sinnvollen oder möglichen Test zu definieren. Ein **test** mit **-constraints unix** läuft nur, wenn die Einschränkung **unix** true ist, was anzeigt, dass die Test-Suite auf einer Unix-Plattform läuft.

Jeder **test** sollte die **-constraints** enthalten, die ihn auf passende Umgebungen beschränken. Mehrere Einschränkungen sind im Paket **tcltest** vordefiniert (unten aufgelistet). Benutzerdefinierte Einschränkungen werden mit **testConstraint** registriert. Sie können in einer Testdatei oder im Skript von **configure -load** bzw. **configure -loadfile** stehen.

Folgende Einschränkungen sind im Paket **tcltest** selbst vordefiniert:

***singleTestInterp***
Dieser Test kann nur ausgeführt werden, wenn alle Testdateien in einen einzigen Interpreter geladen werden.

***unix***
Dieser Test kann nur auf einer Unix-Plattform ausgeführt werden.

***win***
Dieser Test kann nur auf einer Windows-Plattform ausgeführt werden.

***nt***
Dieser Test kann nur auf einer Windows-NT-Plattform ausgeführt werden.

***mac***
Dieser Test kann nur auf einer Mac-Plattform ausgeführt werden.

***unixOrWin***
Dieser Test kann nur auf Unix oder Windows ausgeführt werden.

***macOrWin***
Dieser Test kann nur auf Mac oder Windows ausgeführt werden.

***macOrUnix***
Dieser Test kann nur auf Mac oder Unix ausgeführt werden.

***tempNotWin***
Dieser Test kann unter Windows nicht ausgeführt werden. Dieses Flag dient zum vorübergehenden Deaktivieren.

***tempNotMac***
Dieser Test kann auf dem Mac nicht ausgeführt werden. Dieses Flag dient zum vorübergehenden Deaktivieren.

***unixCrash***
Dieser Test stürzt unter Unix ab. Dieses Flag dient zum vorübergehenden Deaktivieren.

***winCrash***
Dieser Test stürzt unter Windows ab. Dieses Flag dient zum vorübergehenden Deaktivieren.

***macCrash***
Dieser Test stürzt auf dem Mac ab. Dieses Flag dient zum vorübergehenden Deaktivieren.

***emptyTest***
Dieser Test ist leer und lohnt sich nicht, bleibt aber als Platzhalter für einen späteren Test. Diese Bedingung ist false, damit Tests übersprungen werden, sofern der Nutzer nichts anderes angibt.

***knownBug***
Dieser Test ist als fehlschlagend bekannt; der Fehler ist noch nicht behoben. Diese Bedingung ist false, damit Tests übersprungen werden, sofern der Nutzer nichts anderes angibt.

***nonPortable***
Dieser Test kann nur in einer bekannten Entwicklungsumgebung ausgeführt werden. Manche Tests sind von Natur aus nicht portabel (Wortlänge, Dateisystem, Window-Manager usw.). Diese Bedingung ist false, damit Tests übersprungen werden, sofern der Nutzer nichts anderes angibt.

***userInteraction***
Dieser Test erfordert Benutzerinteraktion. Diese Bedingung ist false, damit Tests übersprungen werden, sofern der Nutzer nichts anderes angibt.

***interactive***
Dieser Test kann nur ausgeführt werden, wenn der Interpreter im interaktiven Modus ist (wenn die globale Variable **::tcl_interactive** auf 1 gesetzt ist).

***nonBlockFiles***
Dieser Test kann nur ausgeführt werden, wenn die Plattform Dateien im nicht-blockierenden Modus setzen kann.

***asyncPipeClose***
Dieser Test kann nur ausgeführt werden, wenn die Plattform asynchrones Leeren und Schließen von Pipes unterstützt.

***unixExecs***
Dieser Test kann nur ausgeführt werden, wenn auf dieser Maschine Unix-Befehle **cat**, **echo**, **sh**, **wc**, **rm**, **sleep**, **fgrep**, **ps**, **chmod** und **mkdir** verfügbar sind.

***hasIsoLocale***
Dieser Test kann nur ausgeführt werden, wenn zu einer ISO-Locale gewechselt werden kann.

***root***
Dieser Test kann nur ausgeführt werden, wenn der Unix-Benutzer root ist.

***notRoot***

Dieser Test kann nur ausgeführt werden, wenn der Unix-Benutzer nicht root ist.

***eformat***

Dieser Test kann nur ausgeführt werden, wenn die Anwendung eine funktionierende sprintf-Version für das "e"-Format von Gleitkommazahlen hat.

***stdio***

Dieser Test kann nur ausgeführt werden, wenn **interpreter** als Pipe open werden kann.

Der alternative Modus wird mit **configure -limitconstraints** true aktiviert. Dann werden alle Bedingungen außer denen in der von **configure -constraints** zurückgegebenen Liste auf false gesetzt. Beim Setzen von **configure -constraints** werden diese Bedingungen auf true gesetzt. Mit beiden Optionen laufen nur Tests, deren Bedingungen ausschließlich aus der **configure -constraints**-Liste stammen; alle anderen werden übersprungen. Beispielkonfiguration:

```
configure -constraints knownBug \
          -limitconstraints true \
          -verbose pass
```

um genau die Tests mit bekannten Fehlern auszuführen und zu prüfen, ob einer besteht — was auf eine Behebung hindeutet.

### Alle Tests ausführen

Der Befehl **runAllTests** führt eine gesamte Test-Suite über viele Dateien und Verzeichnisse aus. Die Konfigurationsoptionen von **tcltest** steuern die genauen Abläufe. **runAllTests** gibt zuerst eine Zusammenfassung der Konfiguration auf **outputChannel** aus.

Zu auswertende Testdateien werden in **configure -testdir** gesucht. Es wird eine sortierte Liste der Dateien erzeugt, die **configure -file** treffen und **configure -notfile** nicht. Jede Datei wird nacheinander ausgewertet. Bei **configure -singleproc** true wird jede Datei im Kontext des Aufrufers sourced. Bei false startet eine Kopie von **interpreter** per exec je Datei. Der Mehrprozess-Modus hilft, wenn Tests Prozesse beenden können: Der Hauptprozess setzt die Suite fort. Die **tcltest**-Konfiguration des Hauptprozesses wird als Kommandozeilenargumente an Kindprozesse übergeben, außer **configure -outfile**. **runAllTests** im Hauptprozess sammelt Kindausgaben zu einem Gesamtbericht. Einzelne Fehlermeldungen oder **configure -verbose** -Meldungen gehen direkt an **outputChannel**.

Nach allen ausgewählten Testdateien folgt eine Zusammenfassung auf **outputChannel**: Gesamtzahl der **test**-Auswertungen, aufgeteilt in übersprungene, bestandene und fehlgeschlagene; Anzahl der Dateien; Namen fehlerhafter Dateien; Bedingungen mit Übersprungszahlen; Hinweise, wenn Testdateien temporäre Dateien in **configure -tmpdir** hinterlassen haben.

Nach Abschluss wertet **runAllTests** Unterverzeichnisse von **configure -testdir** rekursiv aus. Unterverzeichnisse, die **configure -relateddir** treffen und **configure -asidefromdir** nicht, werden untersucht. Enthält ein Verzeichnis **all.tcl**, wird es im Kontext des Aufrufers sourced. Mit oder ohne **all.tcl** werden Unterverzeichnisse weiter geprüft. So kann ein **runAllTests**-Aufruf Testdateien in vielen Verzeichnissen eines Baums auswerten.

## Konfigurierbare Optionen

Der Befehl **configure** setzt und fragt die konfigurierbaren Optionen von **tcltest** ab. Gültige Optionen:

**-singleproc *boolean***

Steuert, ob **runAllTests** je Testdatei einen Kindprozess erzeugt. Kein Spawnen bei *boolean* true. Standardwert ist false.

**-debug *level***

Setzt die Debug-Stufe *level* (Ganzzahl) für die Menge an Debug-Informationen auf **stdout**. Debug-Meldungen gehen stets nach **stdout**, unabhängig von **configure -outfile**. Standardwert ist 0. Stufen:

**0**

Keine Debug-Informationen anzeigen.

**1**

Informationen, ob ein Test wegen **configure -match** (userSpecifiedNonMatch) oder **configure -skip** (userSpecifiedSkip) übersprungen wird; Warnungen zu fehlendem Aufräumen oder Ungleichgewicht in Testdateien; Warnungen bei Wiederverwendung von Testnamen.

**2**

Flag-Array des Kommandozeilenprozessors, Inhalt des globalen env-Arrays und alle benutzerdefinierten Variablen im aktuellen Namespace bei Verwendung.

**3**

Informationen, was einzelne Procs im Test-Harness tun.

**-verbose *level***

Setzt die gewünschte Ausgabeausführlichkeit *level* als Liste aus null oder mehr Elementen **body**, **pass**, **skip**, **start**, **error**, **line**, **msec** und **usec**. Standardwert ist "**body error**". Stufen:

**body (b)**

Körper fehlgeschlagener Tests anzeigen

**pass (p)**

Ausgabe, wenn ein Test besteht

**skip (s)**

Ausgabe, wenn ein Test übersprungen wird

**start (t)**

Ausgabe beim Start jedes Tests

**error (e)**

errorInfo und errorCode ausgeben, falls vorhanden, wenn der Rückgabecode nicht dem erwarteten entspricht

**line (l)**

Zeileninformationen fehlgeschlagener Tests ausgeben

**msec (m)**

Ausführungszeit jedes Tests in Millisekunden ausgeben

**usec (u)**

Ausführungszeit jedes Tests in Mikrosekunden ausgeben

Die Stufen **msec** und **usec** sind nur indikative Maße. Sie lösen nicht das Problem der Wiederholbarkeit bei Performance-Tests oder Benchmarks. Für gründliche Verfolgung von Leistungseinbußen Testkörper mit time umschließen.

Die Einbuchstaben-Abkürzungen gelten ebenfalls, sodass "**configure -verbose pt**" dasselbe ist wie "**configure -verbose {pass start}**".

**-preservecore** ***level***

Setzt die Core-Erhaltungsstufe *level*, die bestimmt, wie streng Core-Dateien geprüft werden. Standardwert ist 0. Stufen:

**0**

Keine Prüfung — keine Core-Dateien am Ende jedes Testbefehls; Prüfung in **runAllTests** nach allen Testdateien.

**1**

Zusätzlich Core-Dateien am Ende jedes **test**-Befehls prüfen.

**2**

Core-Dateien in allen oben genannten Fällen prüfen und eine Kopie jeder Core-Datei in **configure -tmpdir** speichern.

**-limitconstraints** ***boolean***

Setzt den Modus, in dem **test** Bedingungen wie in Tests oben beschrieben berücksichtigt. Standardwert ist false.

**-constraints** ***list***

Setzt alle Bedingungen in *list* auf true. Zusammen mit **configure -limitconstraints true** für den alternativen Modus in Tests oben. Standardwert ist eine leere Liste.

**-tmpdir** ***directory***

Setzt das temporäre Verzeichnis für **makeFile**, **makeDirectory**, **viewFile**, **removeFile** und **removeDirectory** als Standardort für temporäre Dateien und Verzeichnisse aus Testdateien. Standardwert ist **workingDirectory**.

**-testdir** ***directory***

Setzt das von **runAllTests** durchsuchte Verzeichnis für Testdateien und Unterverzeichnisse. Standardwert ist **workingDirectory**.

**-file** ***patternList***

Setzt die Musterliste, mit der **runAllTests** Testdateien zur Auswertung bestimmt. Standardwert ist "***.test**".

**-notfile** ***patternList***

Setzt die Musterliste für von **runAllTests** zu überspringende Testdateien. Standardwert ist " **l.*.test**", damit SCCS-Sperrdateien übersprungen werden.

**-relateddir** ***patternList***

Setzt die Musterliste für Unterverzeichnisse, in denen **runAllTests** nach **all.tcl** sucht. Standardwert ist "*****".

**-asidefromdir** ***patternList***

Setzt die Musterliste für zu überspringende Unterverzeichnisse bei der **all.tcl**-Suche. Standardwert ist eine leere Liste.

**-match** ***patternList***

Setzt die Musterliste, mit der **test** bestimmt, ob ein Test ausgeführt wird. Standardwert ist "*****".

**-skip** ***patternList***

Setzt die Musterliste, mit der **test** bestimmt, ob ein Test übersprungen wird. Standardwert ist eine leere Liste.

**-load** ***script***

Setzt ein von **loadTestedCommands** auszuwertendes Skript. Standardwert ist ein leeres Skript.

**-loadfile *filename***

Setzt den Dateinamen für ein von **loadTestedCommands** auszuwertendes Skript. Alternative zu **-load**; nicht gemeinsam verwendbar.

**-outfile *filename***

Setzt die Datei für alle von tcltest erzeugten Ausgaben. Die Datei *filename* wird zum Schreiben openet; der Kanal wird als **outputChannel** gesetzt.

**-errfile *filename***

Setzt die Datei für alle Fehlerausgaben von tcltest. Die Datei *filename* wird zum Schreiben openet; der Kanal wird als **errorChannel** gesetzt.

## Test-Suites mit tcltest erstellen

Das grundlegende Element einer Test-Suite ist der einzelne Befehl **test**. Wir beginnen mit mehreren Beispielen.

1. Test eines Skripts mit normalem Rückgabewert.

```
test example-1.0 {normal return} {
    format %s value
} value
```

2. Test eines Skripts mit Kontext-Setup und -Aufräumen. Beachten Sie Klammerung und Einrückung ohne Zeilenfortsetzung.

```
test example-1.1 {test file existence} -setup {
    set file [makeFile {} test]
} -body {
    file exists $file
} -cleanup {
    removeFile test
} -result 1
```

3. Test eines Skripts, das einen Fehler auslöst.

```
test example-1.2 {error return} -body {
    error message
} -returnCodes error -result message
```

4. Test mit einer Bedingung.

```
test example-1.3 {user owns created files} -constraints {
    unix
} -setup {
    set file [makeFile {} test]
} -body {
    file attributes $file -owner
} -cleanup {
    removeFile test
} -result $::tcl_platform(user)
```

Auf der nächsten Organisationsebene werden mehrere **test**-Befehle in einer Testdatei zusammengefasst. Testdateien sollten die Endung "**.test**" haben — das Standardmuster von **runAllTests**. Als Faustregel gilt: eine Testdatei pro Quellcodedatei. Test- und Quelldatei gemeinsam bearbeiten, Tests mit Codeänderungen synchron halten.

Der Großteil einer Testdatei sollte **test**-Befehle sein. Tests per Bedingungen überspringen, nicht durch bedingte Auswertung von **test**.

1. Empfohlenes System für bedingte Tests mit Bedingungen als Schutz:

```
testConstraint X [expr $myRequirement]
test goodConditionalTest {} X {
    # body
} result
```

2. Nicht empfohlenes System mit if als Schutz:

```
if $myRequirement {
    test badConditionalTest {} {
        #body
    } result
}
```

Verwenden Sie **-setup** und **-cleanup**, um den Kontext des Testkörpers herzustellen und freizugeben. Tests nicht von vorherigen Tests in der Datei abhängig machen — diese können übersprungen sein. Benötigen mehrere Tests denselben Kontext, können Setup- und Cleanup-Skripte in Variablen für **-setup** und **-cleanup** jedes Tests gespeichert werden. Besser als Setup außerhalb von **test**: Setup nur bei Bedarf; Setup-Fehler werden gemeldet und brechen die Datei nicht ab.

Eine Testdatei sollte sich mit anderen kombinieren lassen, ohne sie zu stören — auch wenn **configure -singleproc 1** alle Dateien in einem Interpreter auswertet. Definieren Sie Befehle und Variablen in einem Namespace, der nach der Testdatei gelöscht wird. Geeignet ist ein Kind-Namespace **test** des Modul-Namespaces.

Eine Testdatei sollte auch direkt als Skript auswertbar sein, ohne **runAllTests**. Jede Testdatei sollte Kommandozeilenargumente verarbeiten, damit der Tester die volle **tcltest**-Konfiguration hat.

Nach allen **test**-Befehlen in einer Testdatei sollte **cleanupTests** aufgerufen werden.

1. Skizze einer Beispiel-Testdatei:

```
package require tcltest 2.5
::tcltest::configure {*}$::argv
package require example
namespace eval ::example::test {
    namespace import ::tcltest::*
    testConstraint X [expr {...}]
    variable SETUP {#common setup code}
    variable CLEANUP {#common cleanup code}
    test example-1 {} -setup $SETUP -body {
        # First test
    } -cleanup $CLEANUP -result {...}
    test example-2 {} -constraints X -setup $SETUP -body {
        # Second test; constrained
    } -cleanup $CLEANUP -result {...}
    test example-3 {} {
        # Third test; no context required
    } {...}
    cleanupTests
}
namespace delete ::example::test
```

Die nächste Ebene ist eine vollständige Test-Suite aus mehreren Testdateien. Ein Skript steuert die Suite und ruft nach Setup **runAllTests** auf. Üblicher Name: **all.tcl** — Standardname von **runAllTests** beim Zusammenfassen mehrerer Suites.

1. Skizze eines Hauptskripts einer Test-Suite:

```
package require tcltest 2.5
package require example
::tcltest::configure -testdir \
        [file dirname [file normalize [info script]]]
::tcltest::configure {*}$::argv
::tcltest::runAllTests
```

## Kompatibilität

Einige Befehle und Variablen im Namespace **::tcltest** früherer **tcltest**-Versionen sind hier nicht dokumentiert. Sie gehören nicht mehr zur unterstützten öffentlichen Schnittstelle und sollten in neuen Test-Suites nicht verwendet werden. Zur Unterstützung bestehender Suites funktionieren viele veraltete Befehle und Variablen noch. So wird **configure** oft kurz nach erfolgreichen package require **tcltest 2.1** automatisch mit Argumenten aus **::argv** aufgerufen — für Suites, die auf automatische Konfiguration aus Kommandozeilenargumenten angewiesen sind. Neue Testdateien sollten das nicht voraussetzen, sondern explizit enthalten:

```
::tcltest::configure {*}$::argv
```

um eine Konfiguration aus Kommandozeilenargumenten herzustellen.

## Bekannte Probleme

Es gibt zwei bekannte Probleme bei verschachtelten **test**-Auswertungen. Das erste betrifft die Stack-Ebene der Testskripte. Verschachtelte Tests können auf derselben Stack-Ebene wie der äußerste Test laufen. Beispiel:

```
test level-1.1 {level 1} {
    -body {
        test level-2.1 {level 2} {
        }
    }
}
```

jedes in level-2.1 ausgeführte Skript kann auf derselben Stack-Ebene wie das Skript für level-1.1 laufen.

Zudem meldet **cleanupTests** nach zwei **test**-Läufen Ergebnisse nur für Tests auf derselben Ebene wie level-1.1. Ergebnisse aller Tests vor level-1.1 sind verfügbar, wenn level-2.1 läuft. Beim Zugriff auf Ergebnisse von level-2.1 kann stehen, dass "m" Tests liefen, "n" übersprungen, "o" bestanden und "p" fehlschlugen — wobei m, n, o und p Tests auf derselben Ebene wie level-1.1 meinen.

Ausgabe- und Fehlervergleich in **test** hängen von puts in der Anwendung ab. Ausgabe wird abgefangen, indem puts global neu definiert wird. Von C-Prozeduren geworfene oder direkt gedruckte Fehler erfasst **test** nicht. **-output** und **-errorOutput** sind daher nur für reine Tcl-Anwendungen mit puts-Ausgabe sinnvoll.

*Copyright (c) 1990-1994 The Regents of the University of California*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*
*Copyright (c) 1998-1999 Scriptics Corporation*
*Copyright (c) 2000 Ajuba Solutions*

# tclvars

*Stichwörter:* tclvars, arithmetic, bytecode, compiler, error, environment, POSIX, precision, subprocess, user, variables

## Name

argc, argv, argv0, auto_path, env, errorCode, errorInfo, tcl_interactive, tcl_library, tcl_patchLevel, tcl_pkgPath, tcl_platform, tcl_rcFileName, tcl_traceCompile, tcl_traceExec, tcl_version - Von Tcl verwendete Variablen

## Beschreibung

Die folgenden globalen Variablen werden von der Tcl-Bibliothek automatisch erzeugt und verwaltet. Sofern unten nicht anders vermerkt, sollten diese Variablen von anwendungsspezifischem Code und von Benutzern in der Regel als schreibgeschützt behandelt werden.

**auto_path**

Ist sie gesetzt, muss sie eine gültige Tcl-Liste mit Verzeichnissen enthalten, die bei Auto-Load-Vorgängen durchsucht werden (einschließlich Paketindexdateien bei Verwendung des Standard-Handlers package unknown). Diese Variable wird beim Start initialisiert und enthält in dieser Reihenfolge: die in der Umgebungsvariable **TCLLIBPATH** aufgeführten Verzeichnisse, das durch die globale Variable **tcl_library** benannte Verzeichnis, das übergeordnete Verzeichnis von **tcl_library**, **[file dirname [file dirname [info nameofexecutable]]]/lib**, die in der Variable **tcl_pkgPath** aufgeführten Verzeichnisse. Weitere Orte für Dateien und Paketindizes sollten normalerweise mit lappend zu dieser Variable hinzugefügt werden. Bei der Initialisierung von **auto_path** aus der Umgebungsvariable TCLLIBPATH wird für jeden Pfad eine Tilden-Substitution durchgeführt (siehe filename). Jede Tilden-Substitution, die fehlschlägt, weil der Benutzer unbekannt ist, wird aus **auto_path** weggelassen.

Weitere Variablen zur Paketverwaltung existieren. Weitere Details stehen im Abschnitt

**VARIABLES** der Manpage library.

**env**

Diese Variable wird von Tcl als Array verwaltet, dessen Elemente die Umgebungsvariablen des Prozesses sind. Das Lesen eines Elements liefert den Wert der entsprechenden Umgebungsvariable. Das Setzen eines Array-Elements ändert die entsprechende Umgebungsvariable oder legt eine neue an, falls sie noch nicht existiert. Das Entfernen eines Elements von **env** löscht die entsprechende Umgebungsvariable. Änderungen am Array **env** wirken sich auf die Umgebung aus, die an Kindprozesse durch Befehle wie exec übergeben wird. Wird das gesamte Array **env** entfernt, überwacht Tcl keine Zugriffe auf **env** mehr und aktualisiert keine Umgebungsvariablen.

Unter Windows werden die Umgebungsvariablen PATH und COMSPEC unabhängig von der Groß-/Kleinschreibung automatisch in Großbuchstaben umgewandelt. So kann die PATH-Variable vom Betriebssystem als "path", "Path", "PaTh" usw. exportiert werden, was ansonsten einfachen Tcl-Code zu vielen Sonderfällen zwingen würde. Alle anderen von Tcl geerbten Umgebungsvariablen bleiben unverändert. Das Setzen eines **env**-Array-Elements auf einen leeren Wert entspricht dem Entfernen, da dies dem Verhalten des zugrunde liegenden Windows-Betriebssystems entspricht. Es ist zu beachten, dass sich auf Windows nicht auf eine vorhandene und leere Umgebungsvariable verlassen lässt; für plattformübergreifenden Einsatz wird dies nicht empfohlen.

Die folgenden Elemente von **env** sind für Tcl besonders:

**env(HOME)**

Diese Umgebungsvariable gibt, falls gesetzt, den Ort des Verzeichnisses an, das als Heimatverzeichnis des aktuellen Benutzers gilt. Der Wert dieser Variable wird vom Befehl file home zurückgegeben. Die meisten Plattformen setzen dies standardmäßig korrekt; sie muss normalerweise nicht durch Benutzercode gesetzt werden. Unter Windows wird sie, falls noch nicht gesetzt, auf den Wert der Umgebungsvariable **USERPROFILE** gesetzt.

**env(TCL_LIBRARY)**

Ist sie gesetzt, legt sie den Ort des Verzeichnisses mit Bibliotheksskripten fest (der Wert dieser Variable wird der Variable **tcl_library** zugewiesen und daher vom Befehl info library zurückgegeben). Ist diese Variable nicht gesetzt, wird ein Standardwert verwendet.

Beachten Sie, dass diese Umgebungsvariable normalerweise *nicht* gesetzt werden sollte.

**env(TCLLIBPATH)**

Ist sie gesetzt, muss sie eine gültige Tcl-Liste mit Verzeichnissen enthalten, die bei Auto-Load-Vorgängen durchsucht werden. Verzeichnisse müssen im Tcl-Format angegeben werden, mit "/" als Pfadtrenner, unabhängig von der Plattform. Diese Variable wird nur bei der Initialisierung der Variable **auto_path** verwendet.

**env(TCL_TZ), env(TZ)**

Diese legen die Standardzeitzone für das Parsen und Formatieren von Zeiten und Datumsangaben im Befehl clock fest. Auf vielen Plattformen wird die TZ-Umgebungsvariable vom Betriebssystem eingerichtet.

**env(LC_ALL), env(LC_MESSAGES), env(LANG)**

Diese Umgebungsvariablen werden vom Paket msgcat verwendet, um festzulegen, mit welcher Locale Nachrichten formatiert werden.

**env(TCL_INTERP_DEBUG_FRAME)**

Existiert sie, hat sie dieselbe Wirkung wie die Ausführung von interp debug **{} -frame 1** als allererstem Befehl jedes neuen Tcl-Interpreters.

**errorCode**

Diese Variable enthält den Wert der Rückgabeoption **-errorcode**, die durch den zuletzt in diesem Interpreter aufgetretenen Fehler gesetzt wurde. Dieser Listenwert stellt zusätzliche Informationen über den Fehler in einer für Programme leicht verarbeitbaren Form dar. Das erste Element der Liste identifiziert eine allgemeine Fehlerklasse und bestimmt das Format des restlichen Teils der Liste. Die folgenden Formate für Rückgabeoptionen **-errorcode** werden vom Tcl-Kern verwendet; einzelne Anwendungen können weitere Formate definieren.

**ARITH *code msg***

Dieses Format wird bei einem arithmetischen Fehler verwendet (z. B. ein Versuch, Null durch Null im Befehl expr zu teilen). *code* identifiziert den genauen Fehler und *msg* liefert eine für Menschen lesbare Fehlerbeschreibung. *code* ist entweder DIVZERO (bei einem Versuch, durch Null zu teilen), DOMAIN (wenn ein Argument außerhalb des Definitionsbereichs einer Funktion liegt, z. B. acos(-3)), IOVERFLOW (bei Integer-Überlauf), OVERFLOW (bei Gleitkomma-Überlauf) oder UNKNOWN (wenn die Fehlerursache nicht bestimmt werden kann).

Die Erkennung dieser Fehler hängt teilweise von der zugrunde liegenden Hardware und den Systembibliotheken ab.

**CHILDKILLED *pid sigName msg***

Dieses Format wird verwendet, wenn ein Kindprozess wegen eines Signals beendet wurde. Das Element *pid* ist die Prozesskennung (dezimal). Das Element *sigName* ist der symbolische Name des Signals, das den Prozess beendet hat; es ist einer der Namen aus der Include-Datei signal.h, z. B. **SIGPIPE**. Das Element *msg* ist eine kurze, für Menschen lesbare Meldung zum Signal, z. B. "Schreiben in Pipe ohne Leser" für **SIGPIPE**.

**CHILDSTATUS *pid code***

Dieses Format wird verwendet, wenn ein Kindprozess mit einem von Null verschiedenen Exit-Status beendet wurde. Das Element *pid* ist die Prozesskennung (dezimal) und das Element *code* ist der vom Prozess zurückgegebene Exit-Code (ebenfalls dezimal).

**CHILDSUSP *pid sigName msg***

Dieses Format wird verwendet, wenn ein Kindprozess wegen eines Signals angehalten wurde. Das Element *pid* ist die Prozesskennung in Dezimalform. Das Element *sigName* ist der symbolische Name des Signals, das den Prozess angehalten hat; es ist einer der Namen aus der Include-Datei signal.h, z. B. **SIGTTIN**. Das Element *msg* ist eine kurze, für Menschen lesbare Meldung zum Signal, z. B. "Hintergrund-TTY-Lesen" für **SIGTTIN**.

**NONE**

Dieses Format wird für Fehler verwendet, bei denen außer der mit dem Fehler zurückgegebenen Meldung keine weiteren Informationen verfügbar sind. In diesen Fällen besteht die Rückgabeoption **-errorcode** aus einer Liste mit einem einzigen Element, dessen Inhalt **NONE** ist.

**POSIX *errName msg***

Ist das erste Element **POSIX**, trat der Fehler bei einem POSIX-Kernelaufruf auf. Das Element *errName* enthält den symbolischen Namen des aufgetretenen Fehlers, z. B. **ENOENT**; es ist einer der in der Include-Datei errno.h definierten Werte. Das Element *msg* ist eine für Menschen lesbare Meldung zu *errName*, z. B. "Datei oder Verzeichnis nicht vorhanden" im Fall **ENOENT**.

**TCL ...**

Weist auf ein Problem im Zusammenhang mit Tcl selbst hin, z. B. das Fehlschlagen der Suche nach einem Kanal oder einer Variable.

Um die Rückgabeoption **-errorcode** zu setzen, sollten Anwendungen Bibliotheksprozeduren wie Tcl_SetObjErrorCode, Tcl_SetReturnOptions und Tcl_PosixError verwenden oder die Option **-errorcode** des Befehls return aufrufen. Wurde keine dieser Methoden zum Setzen des Fehlercodes verwendet, setzt der Tcl-Interpreter die Variable nach dem nächsten Fehler auf **NONE** zurück.

**errorInfo**

Diese Variable enthält den Wert der Rückgabeoption **-errorinfo**, die durch den zuletzt in diesem Interpreter aufgetretenen Fehler gesetzt wurde. Dieser Zeichenkettenwert enthält eine oder mehrere Zeilen, die die Tcl-Befehle und Prozeduren identifizieren, die bei dem zuletzt aufgetretenen Fehler ausgeführt wurden. Der Inhalt hat die Form eines Stack-Trace mit den verschachtelten Tcl-Befehlen, die zum Zeitpunkt des Fehlers aufgerufen worden waren.

**tcl_library**

Diese Variable enthält den Namen eines Verzeichnisses mit der Systembibliothek von Tcl-Skripten, z. B. solchen für Auto-Loading. Der Wert dieser Variable wird vom Befehl info library zurückgegeben. Details zu den Einrichtungen der Tcl-Skriptbibliothek stehen im Manpage-Eintrag library. Normalerweise verfügt jede Anwendung oder jedes Paket zusätzlich zur Tcl-Skriptbibliothek über eine eigene anwendungsspezifische Skriptbibliothek; jede Anwendung sollte eine globale Variable mit einem Namen wie **$***app***_library** setzen (wobei *app* der Anwendungsname ist), die den Netzwerkdateinamen des Bibliotheksverzeichnisses der Anwendung enthält. Der Anfangswert von **tcl_library** wird beim Erzeugen eines Interpreters gesetzt, indem mehrere Verzeichnisse durchsucht werden, bis eines gefunden wird, das ein geeignetes Tcl-Startskript enthält. Existiert die Umgebungsvariable **TCL_LIBRARY**, wird das von ihr benannte Verzeichnis zuerst geprüft. Ist **TCL_LIBRARY** nicht gesetzt oder verweist nicht auf ein geeignetes Verzeichnis, prüft Tcl weitere Verzeichnisse anhand eines fest eingebauten Standardorts, des Speicherorts der Binärdatei der Anwendung und des aktuellen Arbeitsverzeichnisses.

**tcl_patchLevel**

Beim Erzeugen eines Interpreters initialisiert Tcl diese Variable mit einer Zeichenkette, die die aktuelle Patch-Stufe von Tcl angibt, z. B. **8.4.16** für Tcl 8.4 mit den ersten sechzehn offiziellen Patches oder **8.5b3** für die dritte Beta-Version von Tcl 8.5. Der Wert dieser Variable wird vom Befehl info patchlevel zurückgegeben.

**tcl_pkgPath**

Diese Variable enthält eine Liste von Verzeichnissen, die angibt, wo Pakete normalerweise installiert sind. Sie wird unter Windows nicht verwendet. Sie enthält typischerweise einen oder zwei Einträge; bei zwei Einträgen ist der erste normalerweise ein Verzeichnis für plattformabhängige Pakete (z. B. Shared-Library-Binärdateien) und der zweite ein Verzeichnis für plattformunabhängige Pakete (z. B. Skriptdateien). Ein Paket wird typischerweise als Unterverzeichnis eines Eintrags in **tcl_pkgPath** installiert. Die Verzeichnisse in **tcl_pkgPath** sind standardmäßig in **auto_path** enthalten, sodass sie und ihre unmittelbaren Unterverzeichnisse bei package require automatisch nach Paketen durchsucht werden.

Hinweis: **tcl_pkgPath** ist nicht zur Änderung durch die Anwendung vorgesehen. Ihr Wert wird beim Start zu **auto_path** hinzugefügt; Änderungen an **tcl_pkgPath** spiegeln sich nicht in **auto_path** wider. Sollen Tcl weitere Verzeichnisse nach Paketen durchsuchen, fügen Sie deren Namen **auto_path** hinzu, nicht **tcl_pkgPath**.

**tcl_platform**

Dies ist ein assoziatives Array, dessen Elemente Informationen über die Plattform enthalten, auf der die Anwendung läuft, z. B. Name des Betriebssystems, aktuelle Versionsnummer und Befehlssatz der Maschine. Die unten aufgeführten Elemente sind immer definiert, können aber leere Zeichenketten als Wert haben, wenn Tcl keine relevanten Informationen ermitteln konnte. Erweiterungen und Anwendungen können dem Array weitere Werte hinzufügen. Die vordefinierten Elemente sind:

**byteOrder**

Die native Byte-Reihenfolge dieser Maschine: entweder **littleEndian** oder **bigEndian**.

**debug**

Existiert diese Variable, wurde der Interpreter mit einer debug-fähigen C-Laufzeit kompiliert und verlinkt. Diese Variable existiert nur unter Windows, damit Erweiterungsautoren je nach verwendeter C-Laufzeitbibliothek festlegen können, welches Paket geladen wird. Dies ist kein Hinweis darauf, dass dieser Kern Symbole enthält.

**engine**

Der Name der Tcl-Sprachimplementierung. Beim ersten Erzeugen des Interpreters ist dies immer die Zeichenkette **Tcl**.

**machine**

Der von dieser Maschine ausgeführte Befehlssatz, z. B. **intel**, **PPC**, **68k** oder **sun4m**. Auf UNIX-Maschinen ist dies der von **uname -m** zurückgegebene Wert.

**os**

Der Name des auf dieser Maschine laufenden Betriebssystems, z. B. **Windows NT** oder **SunOS**. Auf UNIX-Maschinen ist dies der von **uname -s** zurückgegebene Wert.

**osVersion**

Die Versionsnummer des auf dieser Maschine laufenden Betriebssystems. Auf UNIX-Maschinen ist dies der von **uname -r** zurückgegebene Wert.

**pathSeparator**

Das Zeichen, mit dem PATH-ähnliche Umgebungsvariablen per **split** in die entsprechende Liste von Verzeichnisnamen aufgeteilt werden sollten.

**platform**

Entweder **windows** oder **unix**. Identifiziert die allgemeine Betriebsumgebung der Maschine.

**pointerSize**

Gibt die Größe des nativen Maschinenzeigers in Bytes an (genau genommen entspricht sie dem Ergebnis der Auswertung von *sizeof(void*)* in C.)

**user**

Identifiziert den aktuellen Benutzer anhand der auf der Plattform verfügbaren Anmeldeinformationen. Dieser Wert stammt unter Unix von den Systemaufrufen getuid() und getpwuid(), unter Windows vom Systemaufruf GetUserName().

**wordSize**

Gibt die Größe des nativen Maschinenworts in Bytes an (genau genommen entspricht sie dem Ergebnis der Auswertung von *sizeof(long)* in C.)

**tcl_rcFileName**

Diese Variable wird bei der Initialisierung verwendet, um den Namen einer benutzerspezifischen Startdatei anzugeben. Wird sie durch anwendungsspezifische Initialisierung gesetzt, prüft der Tcl-Startcode auf das Vorhandensein dieser Datei und führt source darauf aus, falls sie existiert. Für wish ist die Variable z. B. unter Unix auf **~/.wishrc** und unter Windows auf **~/wishrc.tcl** gesetzt.

**tcl_traceCompile**

Der Wert dieser Variable kann gesetzt werden, um zu steuern, wie viele Trace-Informationen während der Bytecode-Kompilierung ausgegeben werden. Standardmäßig ist **tcl_traceCompile** null und es wird nichts ausgegeben. Der Wert 1 erzeugt eine einzeilige Zusammenfassung in **stdout**, sobald eine Prozedur oder ein Top-Level-Befehl kompiliert wird. Der Wert 2 erzeugt eine detaillierte Auflistung in **stdout** der bei jeder Kompilierung emittierten Bytecode-Anweisungen. Diese Variable ist hilfreich beim Aufspüren vermuteter Probleme mit dem Tcl-Compiler.

Diese Variable und die Funktionalität existieren nur, wenn **TCL_COMPILE_DEBUG** bei der Kompilierung von Tcl definiert wurde.

**tcl_traceExec**

Der Wert dieser Variable kann gesetzt werden, um zu steuern, wie viele Trace-Informationen während der Bytecode-Ausführung ausgegeben werden. Standardmäßig ist **tcl_traceExec** null und es wird nichts ausgegeben. Der Wert 1 erzeugt bei jedem Aufruf einer Tcl-Prozedur eine einzeilige Spur in **stdout**. Der Wert 2 erzeugt bei jedem aufgerufenen Tcl-Befehl eine Zeile mit Befehlsname und Argumenten. Der Wert 3 erzeugt eine detaillierte Spur mit dem Ergebnis der Ausführung jeder Bytecode-Anweisung. Bei **tcl_traceExec** 2 oder 3 werden Befehle wie set und incr, die vollständig durch eine Folge von Bytecode-Anweisungen ersetzt wurden, nicht angezeigt. Das Setzen dieser Variable ist hilfreich beim Aufspüren vermuteter Probleme mit Bytecode-Compiler und -Interpreter.

Diese Variable und die Funktionalität existieren nur, wenn **TCL_COMPILE_DEBUG** bei der Kompilierung von Tcl definiert wurde.

**tcl_version**

Beim Erzeugen eines Interpreters initialisiert Tcl diese Variable mit der Versionsnummer dieser Tcl-Version in der Form *x.y*. Änderungen an *x* stehen für größere Änderungen mit wahrscheinlichen Inkompatibilitäten; Änderungen an *y* stehen für kleinere Erweiterungen und Fehlerbehebungen mit Abwärtskompatibilität. Der Wert dieser Variable wird vom Befehl info tclversion zurückgegeben.

## Weitere globale Variablen

Die folgenden Variablen sind nur in den ausführbaren Programmen tclsh und wish garantiert vorhanden; die Tcl-Bibliothek definiert sie selbst nicht, viele Tcl-Umgebungen jedoch schon.

**argc**

Die Anzahl der Argumente für tclsh oder wish.

**argv**

Tcl-Liste der Argumente für tclsh oder wish.

**argv0**
Das Skript, mit dem tclsh oder wish die Ausführung begonnen hat (falls angegeben), andernfalls der Name, unter dem tclsh oder wish aufgerufen wurde.

**tcl_interactive**
Enthält 1, wenn tclsh oder wish interaktiv läuft (kein Skript angegeben und die Standardeingabe ein terminalähnliches Gerät ist), sonst 0.

## Beispiele

Um ein Verzeichnis zur Menge der von package require durchsuchten Orte hinzuzufügen, z. B. wegen anwendungsspezifischer Pakete, muss die Variable **auto_path** aktualisiert werden:

```
lappend ::auto_path [file join [pwd] "theLibDir"]
```

Ein einfacher, wenn auch nicht sehr robuster Weg, Kommandozeilenargumente der Form "-foo 1 -bar 2" zu verarbeiten, ist, sie in ein Array zu laden, nachdem zuvor die Standardwerte gesetzt wurden:

```
array set arguments {-foo 0 -bar 0 -grill 0}
array set arguments $::argv
puts "foo is $arguments(-foo)"
puts "bar is $arguments(-bar)"
puts "grill is $arguments(-grill)"
```

Die globale Variable **argv0** kann (zusammen mit dem Befehl info script) verwendet werden, um festzustellen, ob das aktuelle Skript als Hauptskript ausgeführt oder als Bibliothek geladen wird. Das ist nützlich, weil ein einziges Skript sowohl als Bibliothek als auch als Demonstration dieser Bibliothek dienen kann:

```
if {$::argv0 eq [info script]} {
    # Ausführung als: tclsh example.tcl
} else {
    package provide Example 1.0
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# tell

*Stichwörter:* tell

## Name

tell - Gibt die aktuelle Zugriffsposition eines offenen Kanals zurück

## Übersicht

**tell** *channel*

## Beschreibung

Der Befehl **tell** wurde durch den Befehl chan tell ersetzt, der die gleiche Syntax und die gleichen Optionen unterstützt.

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# throw

*Stichwörter:* throw, error, exception

## Name

throw - Erzeugt einen maschinenlesbaren Fehler

## Übersicht

**throw** *type message*

## Beschreibung

Dieser Befehl bricht die aktuelle Auswertung mit einem Fehler ab. Der Fehler wird durch die Argumente *type* und *message* beschrieben: *type* muss eine Liste von Wörtern enthalten, die den Fehler in maschinenlesbarer Form beschreiben (und den error-code-Teil des Ergebnis-Dictionarys bilden), *message* soll Text für die Anzeige an Menschen enthalten.

Der Stack wird entwunden, bis der Fehler von einem passenden catch- oder try-Befehl abgefangen wird. Erreicht er die Event-Schleife ungefangen, wird er über den Mechanismus bgerror gemeldet. Erreicht er in tclsh die oberste Skriptebene, wird er auf der Konsole ausgegeben und beendet im nicht-interaktiven Fall das Programm (in anderen Programmen hängt das Verhalten von der Einbettung von Tcl ab).

Üblicherweise gehen die Wörter in *type* vom Allgemeinsten zum Spezifischsten.

## Beispiele

Erzeugt einen Fehler, der dem von expr beim Dividieren durch Null entspricht:

```
throw {ARITH DIVZERO {divide by zero}} {divide by zero}
```

*Copyright (c) 2008 Donal K. Fellows*

# time

*Stichwörter:* time, script

## Name

time - Misst die Ausführungszeit eines Skripts

## Übersicht

**time** *script* ?*count*?

## Beschreibung

Der Befehl wertet *script* *count*-mal aus (oder einmal, wenn *count* fehlt) und gibt eine Zeichenkette der Form

```
503.2 microseconds per iteration
```

zurück — die durchschnittliche Dauer pro Durchlauf in Mikrosekunden. Gemessen wird verstrichene Zeit, nicht CPU-Zeit.

## Beispiele

Schätzt, wie lange eine einfache for-Schleife bis 1000 braucht:

```
time {
    for {set i 0} {$i<1000} {incr i} {
        # leerer Rumpf
    }
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# timerate

*Stichwörter:* timerate, performance measurement, script, time

## Name

timerate - Kalibrierte Messung der Ausführungszeit eines Skripts

## Übersicht

**timerate** *script* ?*time*? ?*max-count*?
**timerate** ?**-direct**? ?*-overhead estimate*? *script* ?*time*? ?*max-count*?
**timerate** ?**-calibrate**? ?**-direct**? *script* ?*time*? ?*max-count*?

## Beschreibung

Der Befehl **timerate** misst kalibriert die Leistung eines Tcl-Befehls oder Skripts *script*. *script* soll so geschrieben sein, dass es während der Messung mehrfach ausgeführt werden kann. Gemessen wird die verstrichene Zeit mit der feinsten verfügbaren Auflösung des Zeitgebers, nicht die CPU-Zeit; interagiert *script* mit dem Betriebssystem, ist dieser Aufwand enthalten. Der Befehl zeigt, wie gut ein Skript oder Tcl-Befehl läuft, und hilft, Engpässe zu finden und die Anwendungsleistung zu justieren.

Die erste und die zweite Form werten *script* aus, bis das Intervall *time* in Millisekunden verstrichen ist, oder 1000 Millisekunden (1 Sekunde), wenn *time* fehlt.

Der Parameter *max-count* kann zusätzlich die Höchstzahl der Auswertungen begrenzen. Ist *max-count* angegeben, stoppt die Auswertung, wenn diese Zahl der Durchläufe erreicht ist oder die Zeit überschritten wird.

Danach kommt eine kanonische Tcl-Liste dieser Form zurück:

```
0.095977 µs/# 52095836 # 10419167 #/sec 5000.000 net-ms
```

Sie bedeutet:

- die durchschnittliche Zeit je Durchlauf in Mikrosekunden ([**lindex** $result 0])
- wie oft das Skript ausgeführt wurde ([**lindex** $result 2])
- die geschätzte Rate je Sekunde ([**lindex** $result 4])
- die geschätzte echte Ausführungszeit ohne Messaufwand ([**lindex** $result 6])

Folgende Optionen kann **timerate** erhalten:

**-calibrate**
Um sehr schnelle Skripte so genau wie möglich zu messen, kann eine Kalibrierung nötig sein.

**-calibrate** kalibriert **timerate** selbst und berechnet den geschätzten Aufwand des angegebenen Skripts als Standardaufwand für spätere Aufrufe von **timerate**. Fehlt *time*, läuft die Kalibrierung bis zu 10 Sekunden.

Die Kalibrierung ist in der derzeitigen Implementierung nicht thread-sicher.

**-overhead *estimate***

**-overhead** gibt eine Schätzung (in Mikrosekunden, auch als Gleitkommazahl) des Messaufwands je Durchlauf des getesteten Skripts an. Der übergebene Wert ersetzt für diesen Aufruf von **timerate** den durch eine frühere Kalibrierung geschätzten Aufwand. Solche Ersetzungen lassen sich selbst mit **timerate** messen, wie ein späteres Beispiel zeigt.

**-direct**

**-direct** führt das angegebene Skript direkt aus, ohne Kompilierung, ähnlich dem Befehl time. Damit lässt sich der Aufwand von Tcl_EvalObjEx, des Aufrufs kanonischer Listen und der unkompilierten Fassungen bytecodierter Befehle messen.

Anders als time, das das getestete Skript eine feste Zahl von Durchläufen ausführt, lässt **timerate** es eine feste Zeit laufen. Zusätzlich wird während der gesamten Messung die kompilierte Fassung des Skripts verwendet, als wäre das Skript Teil einer kompilierten Prozedur, wenn **-direct** nicht angegeben ist. Die feste Zeitspanne und die mögliche Kompilierung ergeben genauere Ergebnisse und verhindern sehr lange Läufe langsamer Skripte; das eignet sich für Skripte mit stark schwankender Ausführungszeit.

## Beispiele

Schätzen, wie schnell eine einfache Tcl-Schleife for (einschließlich der Operationen an der Variable *i*) bis zehn zählt:

```
# kalibrieren
timerate -calibrate {}

# messen
timerate { for {set i 0} {$i<10} {incr i} {} } 5000
```

Schätzen, wie schnell eine einfache Tcl-Schleife for ist, ohne den Aufwand der Verwaltung der Schleifenvariable:

```
# Aufwand der Variablenoperationen kalibrieren
set i 0; timerate -calibrate {expr {$i<10}; incr i} 1000

# messen
timerate {
    for {set i 0} {$i<10} {incr i} {}
} 5000
```

Die Geschwindigkeit schätzen, mit der clock format nur die Stunde des Tages berechnet, ohne den Aufwand des Skriptteils, der die Zeit dafür vorbereitet:

```
# kalibrieren
timerate -calibrate {}

# Aufwand schätzen
set tm 0
set ovh [lindex [timerate -overhead 0 {
    incr tm [expr {24*60*60}]
}] 0]

# mit geschätztem Aufwand messen
set tm 0
timerate -overhead $ovh {
    clock format $tm -format %H
    incr tm [expr {24*60*60}]; # dieser Aufwand wird ignoriert
} 5000
```

Im letzten Beispiel wird der Aufwand selbst mit **timerate** und **-overhead 0** gemessen. Das ist nötig, weil ausdrückliche Aufwandswerte als absolute Werte gelten und nicht als Zuschlag auf den kalibrierten Standardaufwand. Der kalibrierte Aufwand muss also aus der Messung des Aufwandswerts selbst herausbleiben. Dazu wird beim Messen des Aufwands **-overhead 0** übergeben.

*Copyright (c) 2005 Sergey Brester aka sebres.*

# tm

*Stichwörter:* tm, modules, package

## Name

tm - Einrichtungen zum Auffinden und Laden von Tcl-Modulen

## Übersicht

**::tcl::tm::path add** ?*path* ...?
**::tcl::tm::path remove** ?*path* ...?
**::tcl::tm::path list**
**::tcl::tm::roots** *paths*

## Beschreibung

Dieses Dokument beschreibt die Einrichtungen zum Auffinden und Laden von Tcl-Modulen (siehe Moduldefinition für die Definition eines Tcl-Moduls). Folgende Befehle werden unterstützt:

**::tcl::tm::path add ?*path* ...?**

Die Pfade werden in der Reihenfolge ihres Auftretens am Kopf der Liste der Modulpfade eingefügt. Das bedeutet, dass das letzte Argument zum neuen Kopf der Liste wird.

Der Befehl erzwingt die Einschränkung, dass kein Pfad ein Vorfahrenverzeichnis eines anderen Pfads in der Liste sein darf. Verletzt einer der neuen Pfade diese Einschränkung, wird ein Fehler ausgelöst, bevor Pfade hinzugefügt wurden. Mit anderen Worten: Verletzt nur ein Pfadargument die Einschränkung, wird keiner hinzugefügt.

Ist ein Pfad bereits unverändert vorhanden, wird kein Fehler ausgelöst und keine Aktion ausgeführt.

Pfade werden später in der Reihenfolge ihres Auftretens in der Liste durchsucht. Da sie am Anfang der Liste eingefügt werden, werden sie in umgekehrter Reihenfolge der Hinzufügung durchsucht. Mit anderen Worten: Zuletzt hinzugefügte Pfade werden zuerst betrachtet.

**::tcl::tm::path remove ?*path* ...?**

Entfernt die Pfade aus der Liste der Modulpfade. Der Befehl ignoriert still alle Pfade, die nicht in der Liste stehen.

**::tcl::tm::path list**

Gibt eine Liste aller registrierten Modulpfade in der Reihenfolge zurück, in der nach Modulen gesucht wird.

**::tcl::tm::roots *paths***

Ähnlich wie **path add** und darauf aufbauend. Dieser Befehl nimmt ein einzelnes Argument mit einer Liste von Pfaden, erweitert jeden um "**tcl***X***/site-tcl**" und "**tcl***X*/*X***.***y*" für die Hauptversion *X* des Tcl-Interpreters und die Nebenversion *y* kleiner oder gleich der Nebenversion des Interpreters und fügt die resultierende Menge von Pfaden der Suchliste hinzu.

Dieser Befehl wird intern vom System verwendet, um die systemspezifischen Standardpfade einzurichten.

Der Befehl wurde exponiert, damit ein Build-System zusätzliche Wurzelpfade über die in diesem Dokument beschriebenen hinaus definieren kann.

## Moduldefinition

Ein Tcl-Modul ist ein Tcl-Paket in einer einzigen Datei, ohne weitere von ihm benötigte Dateien. Diese Datei muss source-bar sein. Mit anderen Worten wird ein Tcl-Modul immer so importiert:

```
source module_file
```

Der Befehl load wird nicht direkt verwendet. Diese Einschränkung ist keine tatsächliche Begrenzung, wie manche glauben mögen. Seit 8.4 liest der Tcl-Befehl source nur bis zum ersten ^Z-Zeichen. So können ein beliebiges Tcl-Skript und beliebige Binärdaten in einer Datei kombiniert werden, wobei das Skript die angehängten Daten nach Belieben verarbeitet, um das Paket vollständig zu importieren und zu aktivieren.

Der Name einer Moduldatei muss dem regulären Ausdruck entsprechen:

```
([_[:alpha:]][:_[:alnum:]]*)-([[:digit:]].*)\.tm
```

Die erste erfassende Klammer liefert den Paketnamen, die zweite seine Version. Zusätzlich zum Muster muss die extrahierte Versionsnummer keinen Fehler erzeugen, wenn sie im Befehl verwendet wird:

```
package vcompare $version 0
```

## Modulsuche

Der Verzeichnisbaum zum Speichern von Tcl-Modulen ist vom übrigen Dateisystem getrennt und unabhängig von auto_path.

Tcl-Module werden in allen Verzeichnissen gesucht, die in der Rückgabe von **::tcl::tm::path list** aufgeführt sind. Dies wird der *Modulpfad* genannt. Weder auto_path noch die Variable tcl_pkgPath werden verwendet. Alle Verzeichnisse auf dem Modulpfad müssen eine Einschränkung einhalten:

Für je zwei Verzeichnisse ist keines ein Vorfahrenverzeichnis des anderen.

Das ist nötig, um Mehrdeutigkeiten bei Paketnamen zu vermeiden. Wären beispielsweise die beiden Verzeichnisse "*foo/*" und "*foo/cool*" auf dem Pfad, könnte ein Paket **cool::ice** über die Namen **cool::ice** oder **ice** gefunden werden; letzteres könnte ein unqualifiziertes Paket **ice** verdecken.

Vor Beginn der Suche wird der Name des angeforderten Pakets mit folgendem Algorithmus in einen Teilpfad übersetzt:

Alle Vorkommen von "**::**" im Paketnamen werden durch das passende Verzeichnistrennzeichen der Plattform ersetzt. Unter Unix ist das beispielsweise "**/**".

Beispiel:

Das angeforderte Paket ist **encoding::base64**. Der erzeugte Teilpfad ist "*encoding/base64*".

Danach wird das Paket in allen Modulpfaden gesucht, indem sie nacheinander, vom ersten zum letzten, mit dem Teilpfad zu einem vollständigen Suchmuster kombiniert werden. Der Suchalgorithmus verwirft alle Dateien, deren Dateiname nicht dem regulären Ausdruck im Abschnitt Moduldefinition entspricht. Für die übrigen Dateien werden *provide scripts* erzeugt und der Paket-ifneeded-Datenbank hinzugefügt.

Der Algorithmus fällt auf den vorherigen unknown-Handler zurück, wenn keine gefundene Moduldatei die Anfrage erfüllt. Wurde die Anfrage erfüllt, wird der Fallback ignoriert.

Beachten Sie, dass Pakete in Modulform *keine* Kontrolle über die in die Paketdatenbank eingetragenen *index*- und *provide script*s haben. Für eine Moduldatei **MF** ist das *index script* immer:

```
package ifneeded PNAME PVERSION [list source MF]
```

und das darin eingebettete *provide script* ist:

```
source MF
```

Sowohl Paketname **PNAME** als auch Paketversion **PVERSION** werden aus dem Dateinamen **MF** wie folgt extrahiert:

```
MF = /module_path/PNAME'-PVERSION.tm
```

Dabei ist **PNAME'** der Teilpfad des Moduls wie im Abschnitt Modulsuche definiert und wird in **PNAME** übersetzt, indem alle Verzeichnistrennzeichen durch "**::**" ersetzt werden; **module_path** ist der Pfad (aus der Suchliste), unter dem die Moduldatei gefunden wurde.

Beachten Sie außerdem, dass hier eine Verbindung zwischen Paketnamen und Pfaden entsteht. Tcl vergleicht Paketnamen case-sensitiv, es gibt jedoch Dateisysteme, die das nicht tun, wie NTFS. Glücklicherweise speichern diese Dateisysteme die Groß-/Kleinschreibung des Namens, obwohl sie die Information beim Vergleichen nicht nutzen.

Daraus erlauben wir gemischte Schreibweise für Paketnamen in Tcl-Modulen, verlangen aber, dass es keine Kollisionen gibt, wenn Namen case-insensitiv verglichen werden. Ist ein Paket **Foo** als Tcl-Modul bereitgestellt, sind Pakete wie **foo**, **fOo** usw. nicht mehr erlaubt.

## Standardpfade

Die Standardliste der Pfade auf dem Modulpfad wird von einer tclsh wie folgt berechnet, wobei *X* die Hauptversion des Tcl-Interpreters und *y* kleiner oder gleich der Nebenversion des Tcl-Interpreters ist.

Alle Standardpfade werden dem Modulpfad hinzugefügt, auch nicht existierende. Nicht existierende Pfade werden bei tatsächlichen Suchen herausgefiltert. So kann ein Benutzer einen der durchsuchten Pfade bei Bedarf anlegen, und alle laufenden Anwendungen übernehmen automatisch dort abgelegte Module.

Die Pfade werden in der unten aufgeführten Reihenfolge hinzugefügt; bei Listen aus Umgebungsvariablen in der Reihenfolge, in der sie in der Variable vorkommen.

### Systemspezifische Pfade

**file normalize [info library]/../tcl*X*/*X*.*y***

Mit anderen Worten sucht der Interpreter in einem Verzeichnis, das durch seine Hauptversion festgelegt ist und dessen Nebenversionen kleiner oder gleich der Nebenversion des Interpreters sind.

Für Tcl 8.4 werden beispielsweise folgende Pfade durchsucht:

```
[info library]/../tcl8/8.4
[info library]/../tcl8/8.3
[info library]/../tcl8/8.2
[info library]/../tcl8/8.1
[info library]/../tcl8/8.0
```

Diese Definition setzt voraus, dass ein für Tcl *X*.*y* definiertes Paket auch von allen Interpretern mit derselben Hauptnummer *X* und einer Nebennummer größer als *y* genutzt werden kann.

**file normalize EXEC/tcl*X*/*X*.*y***

Dabei ist **EXEC file normalize [info nameofexecutable]/../lib** oder **file normalize [::tcl::pkgconfig get libdir,runtime]**

Diese Pfadmenge wird wie die vorherige behandelt, außer dass sie in **EXEC_PREFIX** verankert ist. Bei einem Build mit **PREFIX** = **EXEC_PREFIX** sind beide Mengen identisch.

### Site-spezifische Pfade

**file normalize [info library]/../tcl*X*/site-tcl**

Beachten Sie, dass dies immer ein einzelner Eintrag ist, weil *X* stets ein konkreter Wert ist (die aktuelle Hauptversion von Tcl).

### Benutzerspezifische Pfade

**$::env(TCL*X*_*y*_TM_PATH)**

Eine Liste von Pfaden, getrennt durch **:** (Unix) oder **;** (Windows). Das ist benutzer- und site-spezifisch, da diese Umgebungsvariable nicht nur im Benutzerprofil, sondern auch durch Systemkonfigurationsskripte gesetzt werden kann.

**$::env(TCL*X*.*y*_TM_PATH)**

Gleiche Bedeutung und gleicher Inhalt wie die vorherige Variable. Die Verwendung von Punkt '.' zur Trennung von Haupt- und Nebenversion macht den Namen jedoch weniger bis nicht portabel; die Verwendung wird abgeraten. Die Unterstützung wurde nur aus Kompatibilität mit der ursprünglichen Spezifikation beibehalten, d. h. TIP 189.

Diese Pfade werden von allen Tcl-Shells im **$::env(PATH)** des Benutzers gesehen und damit geteilt.

Beachten Sie, dass *X* und *y* den oben genannten allgemeinen Regeln folgen. Tcl 9.1 betrachtet beispielsweise diese 4 Umgebungsvariablen:

```
$::env(TCL9.1_TM_PATH)  $::env(TCL9_1_TM_PATH)
$::env(TCL9.0_TM_PATH)  $::env(TCL9_0_TM_PATH)
```

Pfade aus Umgebungsvariablen unterliegen Tilde-Ersetzung (siehe filename). Jeder Pfad, bei dem die Tilde-Ersetzung fehlschlägt, weil der Benutzer unbekannt ist, wird von den Suchpfaden weggelassen.

*Copyright (c) 2004-2010 Andreas Kupries* andreas_kupries@users.sourceforge.net

# trace

*Stichwörter:* trace, read, command, rename, variable, write, unset

## Name

trace - Überwacht Variablenzugriffe, Befehlsnutzung und Befehlsausführungen

## Übersicht

**trace** *option* ?*arg arg* ...?

## Beschreibung

Dieser Befehl führt Tcl-Befehle aus, sobald bestimmte Operationen ausgelöst werden. Die zulässigen *options* (abkürzbar) sind:

**trace add *type name ops* ?*args*?**

Dabei ist *type* **command**, **execution** oder **variable**.

**trace add command *name ops commandPrefix***

*commandPrefix* wird (mit zusätzlichen Argumenten) ausgeführt, sobald der Befehl *name* auf eine der in der Liste *ops* genannten Weisen geändert wird. *Name* wird mit den üblichen Namespace-Auflösungsregeln für Befehle aufgelöst. Existiert der Befehl nicht, wird ein Fehler ausgelöst.

*Ops* gibt an, welche Operationen von Interesse sind, als Liste aus einem oder mehreren der folgenden Einträge:

**rename**

*commandPrefix* wird aufgerufen, sobald der überwachte Befehl umbenannt wird. Umbenennung in die leere Zeichenkette gilt als Löschen und wird nicht mit "**rename**" überwacht.

**delete**

*commandPrefix* wird aufgerufen, wenn der überwachte Befehl gelöscht wird. Befehle können explizit mit **rename** auf eine leere Zeichenkette gelöscht werden. Befehle werden auch gelöscht, wenn der Interpreter gelöscht wird; Traces werden dann nicht aufgerufen, weil kein Interpreter zur Ausführung existiert.

Beim Auslösen des Traces werden je nach überwachter Operation Argumente an *commandPrefix* angehängt, sodass der tatsächliche Befehl lautet:

```
commandPrefix oldName newName op
```

*OldName* und *newName* sind der aktuelle (alte) Name des überwachten Befehls bzw. der Zielname der Umbenennung (leere Zeichenkette bei "delete"). *Op* ist **rename** oder **delete** wie oben. Der Trace kann das Löschen eines Befehls nicht verhindern; Tcl entfernt den Befehl nach Abschluss des Traces. Rekursives Umbenennen oder Löschen löst keine weiteren Traces desselben Typs aus; ein Delete-Trace, der den Befehl selbst löscht, oder ein Rename-Trace, der selbst umbenennt, führt nicht zu weiteren Trace-Auswertungen. *oldName* und *newName* sind vollständig mit allen Namespaces qualifiziert.

**trace add execution *name ops commandPrefix***

*commandPrefix* wird (mit zusätzlichen Argumenten) ausgeführt, sobald der Befehl *name* ausgeführt wird; die Traces erfolgen an den in *ops* genannten Punkten. *Name* wird mit den üblichen Namespace-Auflösungsregeln aufgelöst. Existiert der Befehl nicht, wird ein Fehler ausgelöst.

*Ops* ist eine Liste aus einem oder mehreren der folgenden Einträge:

**enter**

*commandPrefix* wird aufgerufen, sobald der Befehl *name* ausgeführt wird, unmittelbar vor der eigentlichen Ausführung.

**leave**

*commandPrefix* wird aufgerufen, sobald der Befehl *name* ausgeführt wird, unmittelbar nach der eigentlichen Ausführung.

**enterstep**

*commandPrefix* wird für jeden Tcl-Befehl aufgerufen, der von Beginn der Ausführung der Prozedur *name* bis zu deren Ende ausgeführt wird. *commandPrefix* wird unmittelbar vor der Ausführung des gemeldeten Tcl-Befehls aufgerufen. Hat man z. B. "proc foo {} { puts "hello" }", wird ein *enterstep*-Trace unmittelbar vor der Ausführung von "*puts "hello"*" aufgerufen. Ein *enterstep*-Trace auf einen *name*, der keine Prozedur bezeichnet, führt nicht zu einem Fehler und wird ignoriert.

**leavestep**

*commandPrefix* wird für jeden Tcl-Befehl aufgerufen, der von Beginn der Ausführung der Prozedur *name* bis zu deren Ende ausgeführt wird. *commandPrefix* wird unmittelbar nach der Ausführung des gemeldeten Tcl-Befehls aufgerufen. Ein *leavestep*-Trace auf einen *name*, der keine Prozedur bezeichnet, führt nicht zu einem Fehler und wird ignoriert.

Beim Auslösen werden je nach Operation Argumente an *commandPrefix* angehängt:

Bei **enter** und **enterstep**:

```
commandPrefix command-string op
```

*Command-string* ist der vollständige aktuelle Befehl (der überwachte Befehl bei **enter**, ein beliebiger Befehl bei **enterstep**), einschließlich aller Argumente in voll expandierter Form. *Op* ist **enter** oder **enterstep** wie oben. Der Trace kann die Ausführung stoppen, indem der betreffende Befehl gelöscht wird; bei späterer Ausführung tritt ein "invalid command"-Fehler auf.

Bei **leave** und **leavestep**:

```
commandPrefix command-string code result op
```

*Command-string* wie oben. *Code* ist der Ergebniscode der Ausführung, *result* die Ergebniszeichenkette. *Op* ist **leave** oder **leavestep** wie oben.

Viele **enterstep**- oder **leavestep**-Traces können zu unintuitiven Ergebnissen führen, weil aus einem Trace aufgerufene Befehle weitere Befehlsaufrufe für andere Traces auslösen können.

*commandPrefix* läuft im gleichen Kontext wie der Code, der die überwachte Operation auslöste: wird er aus einer Prozedur aufgerufen, hat er Zugriff auf dieselben lokalen Variablen. Der Kontext kann vom Erstellungskontext des Traces abweichen. Ruft *commandPrefix* eine Prozedur auf (üblich), muss diese upvar oder uplevel nutzen, um auf lokale Variablen des auslösenden Codes zuzugreifen.

Während *commandPrefix* bei einem Execution-Trace läuft, sind Traces auf *name* vorübergehend deaktiviert. So kann *commandPrefix name* im Rumpf ausführen, ohne weitere Traces auszulösen. Tritt beim Ausführen von *commandPrefix* ein Fehler auf, liefert der Befehl *name* insgesamt denselben Fehler.

Sind mehrere Traces auf *name* gesetzt, werden bei *enter* und *enterstep* die Trace-Befehle in umgekehrter Erstellungsreihenfolge aufgerufen; bei *leave* und *leavestep* in der ursprünglichen Erstellungsreihenfolge.

Das Verhalten von Execution-Traces ist für einen in einen anderen Namespace importierten Befehlsnamen *name* derzeit undefiniert.

**trace add variable *name ops commandPrefix***

*commandPrefix* wird ausgeführt, sobald auf die Variable *name* auf eine der in *ops* genannten Weisen zugegriffen wird. *Name* kann eine normale Variable, ein Arrayelement oder ein ganzes Array sein (d. h. *name* nur der Arrayname ohne Index in Klammern). Verweist *name* auf ein ganzes Array, wird *commandPrefix* bei jeder Manipulation eines Elements aufgerufen. Existiert die Variable nicht, wird sie erzeugt, aber ohne Wert; sie ist für namespace which sichtbar, nicht für info exists.

*Ops* ist eine Liste aus einem oder mehreren der folgenden Einträge:

array

*commandPrefix* wird aufgerufen, wenn auf die Variable über den Befehl array zugegriffen oder sie darüber geändert wird, sofern *name* zum Zeitpunkt des array -Aufrufs kein Skalar ist. Ist *name* ein Skalar, löst Zugriff über array den Trace nicht aus.

**read**

*commandPrefix* wird aufgerufen, wenn die Variable gelesen wird.

**write**

*commandPrefix* wird aufgerufen, wenn die Variable geschrieben wird.

**unset**

*commandPrefix* wird aufgerufen, wenn die Variable entfernt wird. Variablen können explizit mit **unset** oder implizit beim Prozedurrückkehr entfernt werden. Variablen werden auch entfernt, wenn Interpreter gelöscht werden; Traces werden dann nicht aufgerufen.

Beim Auslösen werden drei Argumente an *commandPrefix* angehängt:

```
commandPrefix name1 name2 op
```

*Name1* gibt den Namen der Variablen an, auf die zugegriffen wird. Das ist nicht notwendigerweise derselbe Name wie im Befehl **trace add variable**: mit upvar kann eine Prozedur eine Variable unter anderem Namen referenzieren. War der Trace ursprünglich auf ein Array oder ein Arrayelement gesetzt, liefert *name2* den betroffenen Array-Index. Diese Information liegt auch vor, wenn *name1* auf einen Skalar verweist, was vorkommen kann, wenn upvar eine Referenz auf ein einzelnes Arrayelement erzeugt hat. Wird ein ganzes Array gelöscht und der Trace auf das gesamte Array (nicht ein einzelnes Element) registriert, ist *name1* der Arrayname und *name2* eine leere Zeichenkette. *Op* ist **read**, **write** oder **unset** wie oben.

*commandPrefix* läuft im gleichen Kontext wie der auslösende Code: wurde die Variable in einer Tcl-Prozedur verwendet, hat *commandPrefix* Zugriff auf dieselben lokalen Variablen. Der Kontext kann vom Erstellungskontext abweichen. Ruft *commandPrefix* eine Prozedur auf, muss diese upvar oder uplevel nutzen, um auf die überwachte Variable zuzugreifen. *name1* kann vom beim Setzen des Traces verwendeten Namen abweichen, z. B. bei Zugriff über mit upvar definierte Variablen.

Bei Lese- und Schreib-Traces kann *commandPrefix* die Variable ändern und so das Ergebnis der überwachten Operation beeinflussen. Ändert *commandPrefix* den Wert während eines Lese- oder Schreib-Traces, wird der neue Wert als Ergebnis der überwachten Operation zurückgegeben. Der Rückgabewert von *commandPrefix* wird ignoriert, außer liefert er einen Fehler — dann liefert auch die überwachte Operation denselben Fehlertext (z. B. für schreibgeschützte Variablen). Bei Schreib-Traces wird *commandPrefix* nach der Wertänderung aufgerufen; er kann einen neuen Wert setzen und den ursprünglichen überschreiben. Für schreibgeschützte Variablen muss *commandPrefix* den alten Wert wiederherstellen.

Während *commandPrefix* bei einem Lese- oder Schreib-Trace läuft, sind Traces auf die Variable vorübergehend deaktiviert: Lese- und Schreibzugriffe in *commandPrefix* erfolgen direkt ohne erneuten Aufruf von *commandPrefix* (oder anderer Traces). Entfernt *commandPrefix* die Variable, werden Unset-Traces aufgerufen.

Bei einem Unset-Trace ist die Variable bereits gelöscht: sie erscheint undefiniert ohne Traces. Erfolgt das Unset wegen Prozedurrückkehr, läuft der Trace im Variablenkontext der Zielprozedur; der Stack-Frame der zurückkehrenden Prozedur existiert nicht mehr. Unset-Traces werden nicht deaktiviert; legt ein Unset-Trace einen neuen Trace an und greift auf die Variable zu, wird der Trace aufgerufen. Fehler in Unset-Traces werden ignoriert.

Gibt es mehrere Traces auf einer Variable, werden sie in Erstellungsreihenfolge aufgerufen, zuletzt erstellter zuerst. Liefert ein Trace einen Fehler, werden keine weiteren Traces für die Variable aufgerufen. Hat ein Arrayelement einen Trace und das Array als Ganzes ebenfalls, wird zuerst der Trace auf dem gesamten Array aufgerufen.

Der Trace bleibt bis zum Entfernen mit **trace remove variable** (unten), bis zum Unset der Variable oder bis zum Löschen des Interpreters aktiv. Entfernen eines Arrayelements entfernt Traces auf diesem Element, nicht auf dem gesamten Array.

Der Befehl gibt eine leere Zeichenkette zurück.

**trace remove *type name opList commandPrefix***

*type* ist **command**, **execution** oder **variable**.

**trace remove command *name opList commandPrefix***

Gibt es einen Trace auf den Befehl *name* mit den Operationen und dem Befehl aus *opList* und *commandPrefix*, wird er entfernt; *commandPrefix* wird nicht mehr aufgerufen. Gibt eine leere Zeichenkette zurück. Existiert *name* nicht, wird ein Fehler ausgelöst.

**trace remove execution *name opList commandPrefix***

Wie bei **trace remove command**; *commandPrefix* wird nicht mehr aufgerufen. Leere Zeichenkette. Existiert *name* nicht, Fehler.

**trace remove variable *name opList commandPrefix***

Gibt es einen Trace auf die Variable *name* mit *opList* und *commandPrefix*, wird er entfernt. Leere Zeichenkette.

**trace info *type name***

*type* ist **command**, **execution** oder **variable**.

**trace info command *name***

Gibt eine Liste zurück mit einem Element pro Trace auf dem Befehl *name*. Jedes Element ist eine Liste mit *opList* und *commandPrefix*. Hat *name* keine Traces, leere Zeichenkette. Existiert *name* nicht, Fehler.

**trace info execution *name***

Wie **trace info command**.

**trace info variable *name***

Gibt eine Liste zurück mit einem Element pro Trace auf der Variable *name*. Jedes Element enthält *opList* und *commandPrefix*. Existiert *name* nicht oder hat keine Traces, leere Zeichenkette.

## Beispiele

Eine Meldung ausgeben, sobald eine der globalen Variablen **foo** oder **bar** aktualisiert wird, auch bei anderem lokalem Namen (z. B. mit upvar):

```
proc tracer {varname args} {
    upvar #0 $varname var
    puts "$varname was updated to be \"$var\""
}
trace add variable foo write "tracer foo"
trace add variable bar write "tracer bar"
```

Sicherstellen, dass die globale Variable **foobar** stets das Produkt von **foo** und **bar** enthält:

```
proc doMult args {
    global foo bar foobar
    set foobar [expr {$foo * $bar}]
}
trace add variable foo write doMult
trace add variable bar write doMult
```

Ausgabe, welche Befehle während der Verarbeitung einer Tcl-Prozedur ausgeführt werden:

```
proc x {} { y }
proc y {} { z }
proc z {} { puts hello }
proc report args {puts [info level 0]}
trace add execution x enterstep report
x
  → report y enterstep
    report z enterstep
    report {puts hello} enterstep
    hello
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2000 Ajuba Solutions.*

# transchan

*Stichwörter:* transchan, API, channel, ensemble, prefix, transformation

## Name

transchan - API des Befehlshandlers für in Tcl-Code implementierte Kanaltransformationen

## Übersicht

**chan push** *channel cmdPrefix*

*cmdPrefix* **clear** *handle*
*cmdPrefix* **drain** *handle*
*cmdPrefix* **finalize** *handle*
*cmdPrefix* **flush** *handle*
*cmdPrefix* **initialize** *handle mode*
*cmdPrefix* **limit?** *handle*
*cmdPrefix* **read** *handle buffer*

*cmdPrefix* **write** *handle buffer*

## Beschreibung

Der Tcl-Ebenen-Handler für eine Kanaltransformation muss ein Befehl mit Unterbefehlen sein (als *Ensemble* bezeichnet, ohne dass er mit **namespace ensemble create** erzeugt sein muss; dieser Mechanismus ist in keiner Weise an namespace ensemble gebunden). *cmdPrefix* ist das, was beim Aufruf von chan push angegeben wurde, und kann aus mehreren Argumenten bestehen; es wird an Stelle des Präfixes zu mehreren Wörtern expandiert.

Von allen möglichen Unterbefehlen muss der Handler **initialize** und **finalize** unterstützen. Transformationen für beschreibbare Kanäle müssen außerdem **write** unterstützen, Transformationen für lesbare Kanäle müssen **read** unterstützen.

Beachten Sie, dass *cmdPrefix* in den Beschreibungen unten aus mehr als einem Wort bestehen kann und *handle* der von chan push beim Erzeugen der Transformation zurückgegebene Wert ist.

### Allgemeine Unterbefehle

Die folgenden Unterbefehle sind für alle Kanaltypen relevant.

***cmdPrefix* clear *handle***

Dieser optionale Unterbefehl wird aufgerufen, um der Transformation mitzuteilen, dass alle in internen Puffern gespeicherten Daten (eingehend oder ausgehend) geleert werden müssen. Er wird aufgerufen, wenn auf dem transformierten Kanal ein chan seek ausgeführt wird.

***cmdPrefix* finalize *handle***

Dieser Pflicht-Unterbefehl wird zuletzt für den angegebenen *handle* aufgerufen und danach nie wieder; er dient dazu, alle mit der Transformation verbundenen Tcl-Ebenen-Datenstrukturen freizugeben. *Warnung!* Alle von diesem Unterbefehl geworfenen Fehler werden ignoriert. Es ist nicht garantiert, dass er aufgerufen wird, wenn der Interpreter gelöscht wird.

***cmdPrefix* initialize *handle mode***

Dieser Pflicht-Unterbefehl wird zuerst aufgerufen und danach nie wieder (für den angegebenen *handle*). Er ist dafür verantwortlich, alle Teile der Transformation auf Tcl-Ebene zu initialisieren. *mode* ist eine Liste, die beliebig **read** und **write** enthalten kann.

**write**

bedeutet, dass der Kanal beschreibbar ist.

**read**

bedeutet, dass der Kanal lesbar ist.

Der Rückgabewert des Unterbefehls soll eine Liste mit den Namen aller von diesem Handler unterstützten Unterbefehle sein. Jeder vom Unterbefehl geworfene Fehler verhindert die Erzeugung der Transformation. Der geworfene Fehler erscheint als vom Befehl chan push geworfener Fehler.

### Lesebezogene Unterbefehle

Diese Unterbefehle werden für Transformationen auf lesbaren Kanälen verwendet; streng genommen ist **read** optional, muss aber unterstützt werden, wenn einer der anderen unterstützt wird, sonst wird der Kanal nicht lesbar gemacht.

***cmdPrefix* drain *handle***

Dieser optionale Unterbefehl wird aufgerufen, wenn Daten im Eingabe- (d. h. Lese-)Puffer der Transformation nach oben, d. h. zum Benutzer oder Skript, erzwungen werden müssen. Das von der Methode zurückgegebene Ergebnis wird als die *binären* Daten interpretiert, die nach oben an die Ebene über dieser Transformation (den Leser oder eine Transformation höherer Ebene) weitergereicht werden.

Mit anderen Worten: Wenn diese Methode aufgerufen wird, kann die Transformation die eigentliche Transformationsoperation nicht mehr aufschieben und muss alle in ihren internen Lesepuffern wartenden Daten transformieren und das Ergebnis dieser Aktion zurückgeben.

***cmdPrefix* limit? *handle***

Dieser optionale Unterbefehl wird aufgerufen, damit die Tcl-E/A-Engine bestimmen kann, wie weit sie vorauslesen soll. Ist er vorhanden, soll er eine Ganzzahl größer als null zurückgeben, die angibt, wie viele Bytes vorausgelesen werden sollen, oder eine Ganzzahl kleiner als null, um anzuzeigen, dass die E/A-Engine beliebig weit vorauslesen darf.

***cmdPrefix* read *handle buffer***

Dieser Unterbefehl, der vorhanden sein muss, wenn die Transformation mit lesbaren Kanälen arbeiten soll, wird aufgerufen, wenn der Basiskanal oder eine Transformation unterhalb dieser Transformation Daten nach oben weitergibt. *buffer* enthält die von unten übergebenen binären Daten. Es ist Aufgabe dieses Unterbefehls, die Daten tatsächlich zu transformieren. Der Rückgabewert des Unterbefehls wird als die binären Daten interpretiert, die weiter nach oben an die Transformation über dieser Transformation weitergereicht werden. Das kann auch der Benutzer oder das Skript sein, das ursprünglich vom Kanal gelesen hat.

Beachten Sie, dass das Ergebnis leer sein darf oder sogar weniger als die empfangenen Daten; die Transformation muss nicht alles, was ihr jetzt übergeben wird, sofort transformieren. Sie darf eingehende Daten in internen Puffern speichern und die eigentliche Transformation aufschieben, bis sie mehr Daten hat.

## Schreibbezogene Unterbefehle

Diese Unterbefehle werden für Transformationen auf beschreibbaren Kanälen verwendet; streng genommen ist **write** optional, muss aber unterstützt werden, wenn einer der anderen unterstützt wird, sonst wird der Kanal nicht beschreibbar gemacht.

***cmdPrefix* flush *handle***

Dieser optionale Unterbefehl wird aufgerufen, wenn Daten im Schreibpuffer der Transformation nach unten, d. h. zum Basiskanal, erzwungen werden müssen. Der Rückgabewert des Unterbefehls wird als die binären Daten interpretiert, die an die Transformation unterhalb der aktuellen Transformation geschrieben werden. Das kann auch der Basiskanal sein.

Mit anderen Worten: Wenn dieser Unterbefehl aufgerufen wird, kann die Transformation die eigentliche Transformationsoperation nicht mehr aufschieben und muss alle in ihren internen Schreibpuffern wartenden Daten transformieren und das Ergebnis dieser Aktion zurückgeben.

***cmdPrefix* write *handle buffer***

Dieser Unterbefehl, der vorhanden sein muss, wenn die Transformation mit beschreibbaren Kanälen arbeiten soll, wird aufgerufen, wenn der Benutzer oder eine Transformation über dieser Transformation Daten nach unten schreibt. *buffer* enthält die an uns geschriebenen binären Daten. Es ist Aufgabe dieses Unterbefehls, die Daten tatsächlich zu transformieren.

Der Rückgabewert des Unterbefehls wird als die binären Daten interpretiert, die an die Transformation unterhalb dieser Transformation geschrieben werden. Das kann auch der Basiskanal sein. Beachten Sie, dass das Ergebnis leer sein darf oder weniger als die empfangenen Daten; die Transformation muss nicht alles, was ihr jetzt geschrieben wird, sofort transformieren. Sie darf diese Daten in internen Puffern speichern und die eigentliche Transformation aufschieben, bis sie mehr Daten hat.

*Copyright (c) 2008 Donal K. Fellows*

# try

*Stichwörter:* try, cleanup, error, exception, final, resource management

## Name

try - Fehler und Ausnahmen abfangen und verarbeiten

## Übersicht

**try** *body* ?*handler...*? ?*finally script*?

## Beschreibung

Dieser Befehl führt das Skript *body* aus und startet je nach Ergebnis dieses Skripts (normales Ende, Fehler oder ein anderes Ausnahmeergebnis) ein Handler-Skript für diesen Fall. Danach, wenn eine Klausel **finally** vorhanden ist, läuft das darin enthaltene *script*, und das Ergebnis des Handlers (oder von *body*, wenn kein Handler gepasst hat) darf sich weiter ausbreiten. Die Klausel **finally** wird auch bei einem Fehler verarbeitet, unabhängig davon, welcher *handler* verwendet wird, falls überhaupt einer.

Jede *handler*-Klausel besteht aus mehreren Wörtern und muss eine der folgenden Formen haben:

**on *code variableList script***

Diese Klausel passt, wenn die Auswertung von *body* mit dem Ausnahmecode *code* endete. *code* darf eine Ganzzahl sein oder eines der Wörter **ok**, **error**, **return**, **break** oder **continue**. Diese Wörter entsprechen den Ganzzahlen 0 bis 4.

**trap *pattern variableList script***

Diese Klausel passt, wenn die Auswertung von *body* einen Fehler ergab und das Präfix von **-errorcode** aus dem Status-Dictionary des Interpreters gleich *pattern* ist. Die Anzahl der Präfixwörter aus **-errorcode** ist die Listenlänge von *pattern*; Leerzeichen zwischen den Wörtern werden in **-errorcode** und *pattern* vor dem Vergleich vereinheitlicht.

Das Wort *variableList* jedes *handler* gilt immer als Liste von Variablennamen. Ist das erste Wort der Liste vorhanden und nicht leer, nennt es eine Variable, in die das Ergebnis der Auswertung von *body* (aus dem umgebenden **try**) kommt; das enthält die lesbare Form von Fehlern. Ist das zweite Wort vorhanden und nicht leer, nennt es eine Variable für das Options-Dictionary des Interpreters zum Zeitpunkt, an dem *body* endete.

Das Wort *script* jedes *handler* gilt immer gleich: als Tcl-Skript, das ausgewertet wird, wenn die Klausel passt. Ist *script* das Literal "-" und der *handler* nicht der letzte, wird stattdessen das *script* des folgenden *handler* aufgerufen (wie beim Befehl switch).

Die *handler*-Klauseln werden der Reihe nach geprüft, und die erste passende wird gewählt.

Höchstens eine *handler*-Klausel wird gewählt. Deshalb verdeckt ein **on error** jedes nachfolgende **trap** im **try**. **on error** ist gleichwertig zu **trap {}**.

Tritt bei der Auswertung des *handler* oder der Klausel **finally** eine Ausnahme auf (jedes Ergebnis außer **ok**), wird das Status-Dictionary der ursprünglichen Ausnahme dem Status-Dictionary der neuen Ausnahme unter dem Schlüssel **-during** hinzugefügt.

### Beispiele

Eine Datei in jedem Fall schließen:

```
set f [open /some/file/name a]
try {
    puts $f "some message"
    # ...
} finally {
    close $f
}
```

Verschiedene Gründe behandeln, warum eine Datei nicht zum Lesen geöffnet werden kann:

```
try {
    set f [open /some/file/name r]
} trap {POSIX EISDIR} {} {
    puts "failed to open /some/file/name: it's a directory"
} trap {POSIX ENOENT} {} {
    puts "failed to open /some/file/name: it doesn't exist"
}
```

Prozedur, die eine Datei in UTF-8 liest und den Inhalt zurückgibt. Die Datei wird bei Erfolg und bei Fehler durch die Klausel **finally** geschlossen. **return** innerhalb des Blocks **try** ist zulässig. Ab Tcl 9 kann **read** auch UTF-8-Umwandlungsfehler auslösen:

```
proc readfile {filename} {
    set f [open $filename r]
    try {
        fconfigure $f -encoding utf-8 -profile strict
        return [read $f]
    } finally {
        close $f
    }
}
```

*Copyright (c) 2008 Donal K. Fellows*

## unknown

*Stichwörter:* unknown, error, non-existent command

### Name

unknown - Behandelt Versuche, nicht existierende Befehle zu verwenden

## Übersicht

**unknown** *cmdName* ?*arg arg* ...?

## Beschreibung

Dieser Befehl wird vom Tcl-Interpreter aufgerufen, wenn ein Skript versucht, einen Befehl aufzurufen, der nicht existiert. Die Standardimplementierung von **unknown** ist eine Bibliotheksprozedur, die beim Initialisieren eines Interpreters definiert wird. Sie können die Standard-**unknown** überschreiben, um ihr Verhalten zu ändern, oder für einzelne Namespaces einen neuen Handler mit namespace unknown registrieren. Beachten Sie, dass es in einem sicheren Interpreter keine Standardimplementierung von **unknown** gibt.

Trifft der Tcl-Interpreter auf einen Befehlsnamen, für den weder im aktuellen Namespace noch im globalen Namespace ein Befehl definiert ist, prüft Tcl, ob ein unknown-Handler für den aktuellen Namespace existiert. Standardmäßig ist dieser Handler ein Befehl namens **::unknown**. Gibt es keinen solchen Befehl, liefert der Interpreter einen Fehler. Existiert **unknown** (oder wurde für den aktuellen Namespace ein neuer Handler registriert), wird er mit Argumenten aufgerufen, die aus dem vollständig substituierten Namen und den Argumenten des ursprünglich nicht existierenden Befehls bestehen. **unknown** führt typischerweise Aktionen aus wie das Durchsuchen von Bibliotheksverzeichnissen nach einer Befehlsprozedur mit dem Namen *cmdName*, das Erweitern abgekürzter Befehlsnamen auf volle Länge oder das automatische Ausführen unbekannter Befehle als Unterprozesse. In manchen Fällen (z. B. beim Erweitern von Abkürzungen) ändert **unknown** den ursprünglichen Befehl leicht und führt ihn dann (erneut) aus. Das Ergebnis von **unknown** wird als Ergebnis des ursprünglich nicht existierenden Befehls verwendet.

Die Standardimplementierung von **unknown** verhält sich wie folgt. Zuerst ruft sie die Bibliotheksprozedur auto_load auf, um den Befehl zu laden. Gelingt das, führt sie den ursprünglichen Befehl mit seinen ursprünglichen Argumenten aus. Schlägt das Auto-Load fehl und läuft Tcl interaktiv, ruft **unknown** auto_execok auf, um zu prüfen, ob es eine ausführbare Datei mit dem Namen *cmd* gibt. Ist das der Fall, ruft sie den Tcl-Befehl exec mit *cmd* und allen *args* als Argumente auf. Kann *cmd* nicht auto-ausgeführt werden, prüft **unknown**, ob der Befehl auf oberster Ebene und außerhalb eines Skripts aufgerufen wurde. Ist das so, führt **unknown** zwei weitere Schritte aus. Zuerst prüft sie, ob *cmd* eine der folgenden drei Formen hat:**!!**,**!***event* oder **^***old***^***new*?**^**?. Ist das der Fall, führt **unknown** History-Substitution wie **csh** für diese Konstrukte durch. Schließlich prüft **unknown**, ob *cmd* eine eindeutige Abkürzung eines existierenden Tcl-Befehls ist. Ist das so, erweitert sie den Befehlsnamen und führt den Befehl mit den ursprünglichen Argumenten aus. Konnte keiner dieser Schritte den Befehl ausführen, erzeugt **unknown** einen Fehlerrückgabewert. Ist die globale Variable **auto_noload** definiert, wird der Auto-Load-Schritt übersprungen. Ist **auto_noexec** definiert, wird der Auto-Exec-Schritt übersprungen. Unter normalen Umständen ist der Rückgabewert von **unknown** der Rückgabewert des schließlich ausgeführten Befehls.

## Beispiele

**unknown** soll sein Standardverhalten haben, aber zuerst protokollieren, dass ein Befehl nicht gefunden wurde:

```
# Original speichern, damit wir daran anknüpfen können
rename unknown _original_unknown

# Eigene Implementierung bereitstellen
proc unknown args {
    puts stderr "WARNING: unknown command: $args"
    uplevel 1 [list _original_unknown {*}$args]
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# unload

*Stichwörter:* unload, binary code, unloading, safe interpreter, shared library

## Name

unload - Entlädt Maschinencode

## Übersicht

**unload** ?*switches*? *fileName*
**unload** ?*switches*? *fileName prefix*
**unload** ?*switches*? *fileName prefix interp*

## Beschreibung

Dieser Befehl versucht, Shared Libraries, die zuvor mit load geladen wurden, aus dem Adressraum der Anwendung zu entladen. *fileName* ist der Name der Datei mit der zu entladenden Bibliothek; er muss derselbe Dateiname sein, der load beim Laden übergeben wurde. Das Argument *prefix* ist das Präfix (wie von load ermittelt oder übergeben) und wird verwendet, um den Namen der Entladeprozedur zu berechnen; fehlt es, wird es aus *fileName* auf dieselbe Weise wie bei load berechnet. Das Argument *interp* ist der Pfadname des Interpreters, aus dem das Paket entladen werden soll (Details siehe Manpage interp); fehlt *interp*, wird standardmäßig der Interpreter verwendet, in dem der Befehl **unload** aufgerufen wurde.

Beginnen die ersten Argumente von **unload** mit **-**, werden sie als Schalter behandelt. Folgende Schalter werden derzeit unterstützt:

**-nocomplain**
Unterdrückt alle Fehlermeldungen. Mit diesem Schalter meldet **unload** niemals einen Fehler.

**-keeplibrary**
Verhindert, dass **unload** den Systemaufruf ausführt, der die Bibliothek aus dem Prozess entlädt.

**--**
Markiert das Ende der Schalter. Das folgende Argument wird als *fileName* behandelt, auch wenn es mit **-** beginnt.

## Entladevorgang

Wird eine Datei mit einer Shared Library über den Befehl load geladen, führt Tcl zwei Referenzzähler für die Bibliotheksdatei. Der erste zählt, wie oft die Bibliothek in normale (vertrauenswürdige) Interpreter geladen wurde, der zweite, wie oft sie in Safe-Interpreter geladen wurde. Da eine Datei mit einer Shared Library von Tcl nur einmal geladen werden kann (beim ersten load-Aufruf für die Datei), verfolgen diese Zähler, wie viele Interpreter die Bibliothek nutzen. Jeder weitere load-Aufruf nach dem ersten erhöht nur den jeweiligen Referenzzähler.

**unload** arbeitet in die entgegengesetzte Richtung. Zunächst prüft **unload**, ob die Bibliothek entladbar ist: eine entladbare Bibliothek exportiert eine spezielle Entladeprozedur. Der Name der Entladeprozedur wird durch *prefix* und davon abhängig, ob der Zielinterpreter ein Safe-Interpreter ist, bestimmt. Für normale Interpreter hat die Initialisierungsprozedur die Form *pfx*_**Unload**, wobei *pfx* dasselbe wie *prefix* ist, außer dass der erste Buchstabe groß und alle übrigen klein geschrieben werden. Ist *prefix* beispielsweise **foo** oder **FOo**, lautet der Name der Initialisierungsprozedur **Foo_Unload**. Ist der Zielinterpreter ein Safe-Interpreter, lautet der Name der Initialisierungsprozedur *pkg*_**SafeUnload** statt *pkg*_**Unload**.

Stellt **unload** fest, dass eine Bibliothek nicht entladbar ist (oder die Entladefunktionalität bei der Kompilierung deaktiviert wurde), wird ein Fehler zurückgegeben. Ist die Bibliothek entladbar, ruft **unload** die Entladeprozedur auf. Gibt die Entladeprozedur TCL_OK zurück, setzt **unload** fort und verringert den jeweiligen Referenzzähler (je nach Zielinterpreter-Typ). Haben beide Referenzzähler 0 erreicht, wird die Bibliothek vom Prozess getrennt.

## Prototyp der Entlade-Hook-Funktion

Die Entladeprozedur muss folgendem Prototyp entsprechen:

```
typedef int Tcl_LibraryUnloadProc(
        Tcl_Interp *interp,
        int flags);
```

Das Argument *interp* bezeichnet den Interpreter, aus dem die Bibliothek entladen werden soll. Die Entladeprozedur muss TCL_OK oder TCL_ERROR zurückgeben, um anzuzeigen, ob sie erfolgreich abgeschlossen wurde; bei einem Fehler soll sie das Ergebnis des Interpreters auf eine Fehlermeldung setzen. Das Ergebnis des Befehls **unload** ist dann das von der Entladeprozedur zurückgegebene Ergebnis.

Das Argument *flags* kann entweder **TCL_UNLOAD_DETACH_FROM_INTERPRETER** oder **TCL_UNLOAD_DETACH_FROM_PROCESS** sein. Bleibt die Bibliothek nach Rückkehr der Entladeprozedur am Prozess hängen (weil sie von anderen Interpretern genutzt wird), ist **TCL_UNLOAD_DETACH_FROM_INTERPRETER** gesetzt. Wird die Bibliothek jedoch nur vom Zielinterpreter genutzt und beim Rückkehren der Entladeprozedur vom Prozess getrennt, ist *flags* auf **TCL_UNLOAD_DETACH_FROM_PROCESS** gesetzt.

## Hinweise

Der Befehl **unload** kann Bibliotheken nicht entladen, die statisch mit der Anwendung verlinkt sind. Ist *fileName* eine leere Zeichenkette, muss das Argument *prefix* angegeben werden.

Fehlt *prefix* oder ist es eine leere Zeichenkette, versucht Tcl das Präfix zu erraten; das kann je nach Plattform unterschiedlich erfolgen. Die Standardannahme auf den meisten UNIX-Plattformen nimmt das letzte Element von *fileName*, entfernt die ersten drei Zeichen, falls sie **lib** lauten, anschließend die nächsten drei Zeichen, falls sie **tcl9** lauten, und verwendet alle folgenden Wortzeichen (aber keine Ziffern), in Titlecase konvertiert, als Präfix. Beispielsweise verwendet **unload libxyz4.2.so** das Präfix **Xyz** und **unload bin/last.so {}** das Präfix **Last**.

## Portabilitätsfragen

### Unix

Nicht alle Unix-Betriebssysteme unterstützen das Entladen von Bibliotheken. Unter einem solchen System gibt **unload** einen Fehler zurück (sofern **-nocomplain** nicht angegeben wurde).

## Bugs

Wird dieselbe Datei mit verschiedenen *fileName*s mit load geladen, wird sie mehrfach in den Adressraum des Prozesses geladen. Das Verhalten variiert je nach System (manche Systeme erkennen redundante Ladungen, andere nicht). Wurde eine Bibliothek vom Betriebssystem still entladen (und Tcl hält sie daher noch für geladen), kann **unload** auf einer solchen Bibliothek gefährlich sein (die Bibliothek wird vollständig vom Prozess getrennt, während einige Interpreter sie weiter nutzen).

## Beispiele

Wurde ein entladbares Modul in der Datei **foobar.dll** unter Windows so mit load geladen:

```
load c:/some/dir/foobar.dll
```

wird es so entladen:

```
unload c:/some/dir/foobar.dll
```

Damit kann ein C-Code-Modul vorübergehend in ein lang laufendes Tcl-Programm eingebunden und später wieder entfernt werden (weil es nicht mehr benötigt wird oder weil eine neue Version installiert wird), ohne den gesamten Tcl-Prozess beenden zu müssen.

*Copyright (c) 2003 George Petasis* petasis@iit.demokritos.gr*.*

# unset

*Stichwörter:* unset, remove, variable

## Name

unset - Löscht Variablen

## Übersicht

**unset** ?**-nocomplain**? ?**--**? ?*name name name* ...?

## Beschreibung

Dieser Befehl entfernt eine oder mehrere Variablen. Jedes *name* ist ein Variablenname, angegeben auf eine der für den Befehl set zulässigen Arten. Verweist *name* auf ein Array-Element, wird nur dieses Element entfernt, ohne den Rest des Arrays zu beeinflussen. Besteht *name* aus einem Array-Namen ohne indexierende Klammern, wird das gesamte Array gelöscht. Der Befehl **unset** gibt als Ergebnis eine leere Zeichenkette zurück. Wird **-nocomplain** als erstes Argument angegeben, werden mögliche Fehler unterdrückt. Die Option darf nicht abgekürzt werden, um sie von möglichen Variablennamen zu unterscheiden. Die Option **--** markiert das Ende der Optionen und sollte verwendet werden, wenn eine Variable denselben Namen wie eine der Optionen trägt. Tritt beim Löschen ein Fehler auf, werden alle Variablen nach der fehlerauslösenden nicht gelöscht. Ein Fehler kann auftreten, wenn die benannte

Variable nicht existiert, der Name auf ein Array-Element verweist, die Variable aber skalar ist, oder der Name auf eine Variable in einem nicht existierenden Namespace verweist.

### Beispiele

Erzeugt ein Array mit einer Abbildung von Zahlen auf ihre Quadrate und entfernt die Array-Elemente für Nicht-Primzahlen:

```
array set squares {
    1 1    6 36
    2 4    7 49
    3 9    8 64
    4 16   9 81
    5 25  10 100
}

puts "The squares are:"
parray squares

unset squares(1) squares(4) squares(6)
unset squares(8) squares(9) squares(10)

puts "The prime squares are:"
parray squares
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2000 Ajuba Solutions.*

## update

*Stichwörter:* update, asynchronous I/O, event, flush, handler, idle

### Name

update - Verarbeitet anstehende Events und Idle-Callbacks

### Übersicht

**update** ?**idletasks**?

### Beschreibung

Dieser Befehl bringt die Anwendung "auf den neuesten Stand", indem er wiederholt die Event-Schleife betritt, bis alle anstehenden Events (einschließlich Idle-Callbacks) verarbeitet wurden.

Wird das Schlüsselwort **idletasks** als Argument angegeben, werden keine neuen Events oder Fehler verarbeitet; es werden nur Idle-Callbacks aufgerufen. Dadurch werden normalerweise verzögerte Operationen wie Anzeigeaktualisierungen und Fensterlayout-Berechnungen sofort ausgeführt.

Der Befehl **update idletasks** ist in Skripten nützlich, in denen der Zustand der Anwendung geändert wurde und diese Änderungen sofort auf der Anzeige erscheinen sollen, statt erst nach

Abschluss des Skripts. Die meisten Anzeigeaktualisierungen erfolgen als Idle-Callbacks; **update idletasks** lässt sie laufen. Es gibt jedoch Aktualisierungen, die nur als Reaktion auf Events erfolgen, etwa bei Fenstergrößenänderungen; diese erfolgen nicht bei **update idletasks**.

Der Befehl **update** ohne Optionen ist in Skripten nützlich, die lange Berechnungen ausführen, bei denen die Anwendung dennoch auf Events wie Benutzerinteraktionen reagieren soll; ruft man gelegentlich **update** auf, wird Benutzereingabe beim nächsten Aufruf von **update** verarbeitet.

### Beispiel

Berechnungen etwa eine Sekunde lang ausführen und dann beenden:

```
set x 1000
set done 0
after 1000 set done 1
while {!$done} {
    # Ein sehr albernes Beispiel!
    set x [expr {log($x) ** 2.8}]

    # Prüfen, ob das Zeitlimit erreicht ist. Das gibt auch
    # Gelegenheit, Netzwerk-Sockets zu bedienen und, wenn das
    # Tk-Paket geladen ist, eine Benutzeroberfläche zu aktualisieren.
    update
}
```

*Copyright (c) 1990-1992 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## uplevel

*Stichwörter:* uplevel, context, level, namespace, stack frame, variable

### Name

uplevel - Führt ein Skript in einem anderen Stack-Frame aus

### Übersicht

**uplevel** ?*level*? *arg* ?*arg* ...?

### Beschreibung

Alle *arg*-Argumente werden verkettet, als wären sie an concat übergeben worden; das Ergebnis wird im durch *level* angegebenen Variablenkontext ausgewertet. **Uplevel** gibt das Ergebnis dieser Auswertung zurück.

Ist *level* eine Ganzzahl, gibt sie einen Abstand (aufwärts auf dem Prozeduraufrufstack) an, um den vor der Ausführung des Befehls verschoben wird. Besteht *level* aus **#** gefolgt von einer Ganzzahl, gibt *level* eine absolute Ebene an. Fehlt *level*, ist der Standardwert **1**. *Level* kann nicht weggelassen werden, wenn das erste *command*-Argument eine Ganzzahl ist oder mit **#** beginnt.

Angenommen, die Prozedur **a** wurde von der obersten Ebene aufgerufen, **a** rief **b** auf und **b** rief **c** auf. **c** ruft **uplevel** auf. Ist *level* **1** oder **#2** oder weggelassen, wird der Befehl im Variablenkontext von **b** ausgeführt. Ist *level* **2** oder **#1**, im Kontext von **a**. Ist *level* **3** oder **#0**, auf der obersten Ebene (nur globale Variablen sind sichtbar).

Der Befehl **uplevel** lässt die aufrufende Prozedur während der Ausführung des Befehls vom Prozeduraufrufstack verschwinden. Im obigen Beispiel ruft **c** auf:

```
uplevel 1 {set x 43; d}
```

wobei **d** eine weitere Tcl-Prozedur ist. Der Befehl set ändert die Variable **x** im Kontext von **b**, und **d** läuft auf Ebene 3, als wäre sie von **b** aufgerufen. Führt **d** wiederum aus:

```
uplevel {set x 42}
```

so ändert set dieselbe Variable **x** im Kontext von **b**: die Prozedur **c** scheint beim Ausführen von **d** nicht auf dem Aufrufstack zu liegen. Mit info level kann die Ebene der aktuellen Prozedur ermittelt werden.

**Uplevel** ermöglicht neue Steuerkonstrukte als Tcl-Prozeduren (z. B. ließe sich while als Tcl-Prozedur mit **uplevel** implementieren).

Die Befehle namespace eval und apply bieten weitere Wege (neben Prozeduraufrufen), den Tcl-Namenskontext zu wechseln. Sie legen einen Aufrufframe auf den Stack, der den Namespace-Kontext repräsentiert. Jeder Befehl namespace eval zählt damit als weitere Aufrufebene für **uplevel** und upvar. **info level 1** liefert zum Beispiel eine Liste, die einen Befehl beschreibt, der entweder der äußerste Prozeduraufruf oder der äußerste namespace eval-Befehl ist. **uplevel #0** wertet ein Skript auf der obersten Ebene im äußersten Namespace (dem globalen Namespace) aus.

## Beispiel

Wie oben beschrieben, eignet sich **uplevel** zum Erzeugen neuer Steuerkonstrukte. Dieses Beispiel zeigt (ohne Fehlerbehandlung), wie ein **do**-Befehl als Gegenstück zu while erstellt wird, der die Bedingung stets nach dem Schleifenkörper prüft:

```
proc do {body while condition} {
    if {$while ne "while"} {
        error "required word missing"
    }
    set conditionCmd [list expr $condition]
    while {1} {
        uplevel 1 $body
        if {![uplevel 1 $conditionCmd]} {
            break
        }
    }
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# upvar

*Stichwörter:* upvar, context, frame, global, level, namespace, procedure, variable

## Name

upvar - Verknüpfung zu einer Variable in einem anderen Stack-Frame erstellen

## Übersicht

**upvar** ?*level*? *otherVar myVar* ?*otherVar myVar* ...?

## Beschreibung

Dieser Befehl richtet ein, dass eine oder mehrere lokale Variablen in der aktuellen Prozedur auf Variablen in einem umschließenden Prozeduraufruf oder auf globale Variablen verweisen. *Level* kann jede für den Befehl uplevel zulässige Form haben und kann weggelassen werden (Standard **1**). Für jedes Argument *otherVar* macht **upvar** die Variable mit diesem Namen im durch *level* angegebenen Prozedur-Frame (oder auf globaler Ebene, wenn *level* **#0** ist) in der aktuellen Prozedur unter dem Namen aus dem zugehörigen Argument *myVar* zugänglich. Die von *otherVar* benannte Variable muss zum Zeitpunkt des Aufrufs noch nicht existieren; sie wird beim ersten Zugriff auf *myVar* erzeugt, wie eine gewöhnliche Variable. Zum Zeitpunkt des Aufrufs von **upvar** darf noch keine Variable mit dem Namen *myVar* existieren. *MyVar* wird stets als Name einer Variable behandelt, nicht als Array-Element. Wenn der Name wie ein Array-Element aussieht, z. B. **a(b)**, wird ein Fehler zurückgegeben. *OtherVar* kann eine Skalarvariable, ein Array oder ein Array-Element bezeichnen. **Upvar** gibt eine leere Zeichenkette zurück.

Der Befehl **upvar** vereinfacht die Implementierung von Call-by-Name-Prozeduraufrufen und erleichtert den Aufbau neuer Steuerkonstrukte als Tcl-Prozeduren. Als Beispiel dient folgende Prozedur:

```
proc add2 name {
    upvar $name x
    set x [expr {$x + 2}]
}
```

Wird *add2* mit einem Argument aufgerufen, das den Namen einer Variable angibt, addiert es zwei zum Wert dieser Variable. Obwohl *add2* auch mit uplevel statt **upvar** implementiert werden könnte, erleichtert **upvar** *add2* den Zugriff auf die Variable im Prozedur-Frame des Aufrufers.

namespace eval ist ein weiterer Weg (neben Prozeduraufrufen), wie sich der Tcl-Namenskontext ändern kann. Er fügt dem Stack einen Aufruf-Frame hinzu, der den Namespace-Kontext repräsentiert. Das bedeutet, dass jeder Befehl namespace eval als weitere Aufrufebene für uplevel und **upvar** zählt. Beispielsweise liefert info level **1** eine Liste, die einen Befehl beschreibt, der entweder der äußerste Prozeduraufruf oder der äußerste Befehl namespace eval ist. Außerdem wertet uplevel #0 ein Skript auf oberster Ebene im äußersten Namespace (dem globalen Namespace) aus.

Wird eine upvar-Variable gelöscht (z. B. **x** in **add2** oben), betrifft unset die verknüpfte Variable, nicht die upvar-Variable. Eine upvar-Variable lässt sich nur durch Verlassen der Prozedur löschen, in der sie definiert ist. Es ist jedoch möglich, eine upvar-Variable umzubinden, indem ein weiterer **upvar**-Befehl ausgeführt wird.

## Traces und upvar

Upvar interagiert mit Traces auf einfache, aber möglicherweise unerwartete Weise. Ist auf *otherVar* ein Variablen-Trace definiert, wird dieser Trace durch Aktionen ausgelöst, die *myVar* betreffen. Die Trace-Prozedur erhält jedoch den Namen von *myVar*, nicht den Namen von *otherVar*. Die Ausgabe des folgenden Codes lautet daher "*localVar*" und nicht "*originalVar*":

```
proc traceproc { name index op } {
    puts $name
}
proc setByUpvar { name value } {
    upvar $name localVar
    set localVar $value
}
set originalVar 1
trace add variable originalVar write traceproc
setByUpvar originalVar 2
```

Bezieht sich *otherVar* auf ein Element eines Arrays, wird der Elementname als zweites Argument an die Trace-Prozedur übergeben. Das kann wichtig sein, wenn Traces auf ein ganzes Array gesetzt sind.

### Beispiel

Ein Befehl **decr**, der wie incr funktioniert, aber den Wert von der Variable subtrahiert statt addiert:

```
proc decr {varName {decrement 1}} {
    upvar 1 $varName var
    incr var [expr {-$decrement}]
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

## variable

*Stichwörter:* variable, global, namespace, procedure

### Name

variable - Erstellt und initialisiert eine Namespace-Variable

### Übersicht

**variable** *name*
**variable** ?*name value...*?

### Beschreibung

Dieser Befehl wird normalerweise innerhalb eines namespace eval-Befehls verwendet, um eine oder mehrere Variablen innerhalb eines Namespace zu erstellen. Jede Variable *name* wird mit *value* initialisiert. Der *value* für die letzte Variable ist optional.

Existiert eine Variable *name* noch nicht, wird sie erstellt. In diesem Fall wird, wenn *value* angegeben ist, der neu erstellten Variable dieser Wert zugewiesen. Ist kein *value* angegeben, bleibt die neue Variable undefiniert. Existiert die Variable bereits, wird sie auf *value* gesetzt, wenn *value* angegeben ist, oder unverändert gelassen, wenn kein *value* übergeben wird. Normalerweise ist *name* unqualifiziert (enthält keine Namen umgebender Namespaces), und die

Variable wird im aktuellen Namespace erstellt. Enthält *name* Namespace-Qualifizierer, wird die Variable im angegebenen Namespace erstellt. Ist die Variable nicht definiert, ist sie für den Befehl namespace which sichtbar, aber nicht für info exists.

Wird der Befehl **variable** innerhalb einer Tcl-Prozedur ausgeführt, erzeugt er lokale Variablen, die mit den entsprechenden Namespace-Variablen verknüpft sind (und daher von info vars aufgelistet werden). So ähnelt der Befehl **variable** dem Befehl global, wobei global Variablennamen bezüglich des globalen Namespace auflöst statt bezüglich des aktuellen Namespace der Prozedur. Werden *value*-Argumente angegeben, werden sie verwendet, um die Werte der zugehörigen Namespace-Variablen zu ändern. Existiert eine Namespace-Variable noch nicht, wird sie erstellt und optional initialisiert.

Ein *name*-Argument kann nicht auf ein Element innerhalb eines Arrays verweisen. Stattdessen sollte *name* auf das gesamte Array verweisen, und der Initialisierungs-*value* sollte weggelassen werden. Nach der Deklaration der Variable können Elemente innerhalb des Arrays mit den üblichen Befehlen set oder array gesetzt werden.

## Beispiele

Variable in einem Namespace erstellen:

```
namespace eval foo {
    variable bar 12345
}
```

Array in einem Namespace erstellen:

```
namespace eval someNS {
    variable someAry
    array set someAry {
        someName  someValue
        otherName otherValue
    }
}
```

Variablen in Namespaces aus einer Prozedur heraus zugreifen:

```
namespace eval foo {
    proc spong {} {
        # Variable in diesem Namespace
        variable bar
        puts "bar is $bar"

        # Variable in einem anderen Namespace
        variable ::someNS::someAry
        parray someAry
    }
}
```

*Copyright (c) 1993-1997 Bell Labs Innovations for Lucent Technologies*
*Copyright (c) 1997 Sun Microsystems, Inc.*

# vwait

*Stichwörter:* vwait, asynchronous I/O, event, variable, wait

## Name

vwait - Verarbeitet Events bis eine Variable geschrieben wird

## Übersicht

**vwait** *varName*
**vwait** ?*options*? ?*varName* ...?

## Beschreibung

Dieser Befehl betritt die Tcl-Event-Schleife zur Verarbeitung von Events und blockiert die Anwendung, wenn keine Events bereitstehen. Er verarbeitet Events weiter, bis ein Event-Handler den Wert der globalen Variable *varName* setzt. Sobald *varName* gesetzt wurde, kehrt der Befehl **vwait** zurück, sobald der Event-Handler, der *varName* geändert hat, abgeschlossen ist. Das Argument *varName* wird stets als Variablenname bezogen auf den globalen Namespace interpretiert, kann aber auf Variablen beliebiger Namespaces verweisen, wenn der vollqualifizierte Name angegeben ist.

In der zweiten, komplexeren Befehlsform ermöglichen *options* feinere Steuerung der Warteoperation und den Umgang mit mehreren Event-Quellen. *Options* können bestehen aus:

**--**
Markiert das Ende der Optionen. Alle folgenden Argumente werden als Variablennamen behandelt.

**-all**
Alle Bedingungen der Warteoperation müssen erfüllt sein, damit die Warteoperation endet. Andernfalls (Standard) beendet das erste Ereignis das Warten.

**-extended**
Es wird ein erweitertes Ergebnis in Listenform zurückgegeben; siehe unten.

**-nofileevents**
Datei-Events werden in der Warteoperation nicht bearbeitet.

**-noidleevents**
Idle-Handler werden während der Warteoperation nicht aufgerufen.

**-notimerevents**
Timer-Handler werden während der Warteoperation nicht bedient.

**-nowindowevents**
Events des Fenstersystems werden während der Warteoperation nicht bearbeitet.

**-readable *channel***
*Channel* muss einen Tcl-Kanal benennen, der zum Lesen geöffnet ist. Ist *channel* lesbar oder wird lesbar, endet die Warteoperation.

**-timeout *milliseconds***
Die Warteoperation ist auf *milliseconds* begrenzt.

**-variable *varName***
*VarName* muss der Name einer globalen Variable sein. Schreiben oder Entfernen dieser Variable beendet die Warteoperation.

**-writable *channel***
: *Channel* muss einen Tcl-Kanal benennen, der zum Schreiben geöffnet ist. Ist *channel* beschreibbar oder wird beschreibbar, endet die Warteoperation.

Der Rückgabewert von **vwait** ist bei der einfachen Form eine leere Zeichenkette. Ist die Option **-timeout** angegeben, ist das Ergebnis die verbleibende Anzahl Millisekunden, wenn die Wartebedingung erfüllt wurde, oder -1 bei Zeitüberschreitung.

Ist die Option **-extended** angegeben, besteht das Ergebnis aus einer Tcl-Liste mit gerader Elementanzahl. Ungerade Elemente haben die Werte **readable**, **timeleft**, **variable** und **writable**. Gerade Elemente sind die zugehörigen Variablen- und Kanalnamen bzw. die verbleibende Anzahl Millisekunden. Die Liste ist nach dem Auftreten der Event(s) geordnet, mit Ausnahme von **timeleft**, das stets zuletzt steht.

In manchen Fällen kehrt **vwait** nicht sofort zurück, nachdem *varName* usw. gesetzt wurden. Das passiert, wenn der Event-Handler, der *varName* setzt, nicht sofort abgeschlossen wird. Setzt etwa ein Event-Handler *varName* und ruft anschließend selbst **vwait** auf, um auf eine andere Variable zu warten, kann er lange nicht zurückkehren. In dieser Zeit ist der oberste **vwait** blockiert und wartet auf den Abschluss des Event-Handlers, er kann also ebenfalls nicht zurückkehren. (Siehe Verschachtelte vwaits anhand eines Beispiels unten.)

Zur Klarstellung: *mehrere* **vwait***-Aufrufe verschachteln sich und laufen nicht parallel*. Der äußerste Aufruf von **vwait** kehrt erst zurück, wenn alle inneren zurückgekehrt sind. Es wird empfohlen, **vwait**-Aufrufe nie zu verschachteln (indem man sie nicht in Event-Callbacks platziert); wenn das nicht möglich ist, sollte man Sperrvariablen einbauen, um alle nicht *strikt* notwendigen reentranten **vwait**-Aufrufe zu verhindern. Beachten Sie, dass synchrone Betriebsarten einiger Tcl-Pakete (z. B. **http**) **vwait** intern verwenden; bei Nutzung der Event-Schleife sind nach Möglichkeit die asynchronen, callback-basierten Betriebsarten dieser Pakete zu bevorzugen.

## Beispiele

Die Event-Schleife dauerhaft laufen lassen, bis ein Event exit aufruft. (Sie können jede Variable verwenden, die sonst nicht vorkommt; der Name *forever* erinnert auf den ersten Blick an die Absicht.)

```
vwait forever
```

Fünf Sekunden auf eine Verbindung zu einem Server-Socket warten, andernfalls den Socket schließen und das Skript fortsetzen:

```
# Zustand initialisieren
after 5000 set state timeout
set server [socket -server accept 12345]
proc accept {args} {
    global state connectionInfo
    set state accepted
    set connectionInfo $args
}

# Auf ein Ereignis warten
```

```
vwait state

# Mögliche Events aufräumen
close $server
after cancel set state timeout

# Je nach vwait-Ende handeln ...
switch $state {
    timeout {
        puts "no connection on port 12345"
    }
    accepted {
       puts "connection: $connectionInfo"
       puts [lindex $connectionInfo 0] "Hello there!"
    }
}
```

Ein Befehl, der eine Zeitverzögerung abwartet, indem auf das Setzen einer Namespace-Variable gewartet wird. Enthält eine Sperre gegen verschachtelte Waits.

```
namespace eval example {
    variable v done
    proc wait {delay} {
        variable v
        if {$v ne "waiting"} {
            set v waiting
            after $delay [namespace code {set v done}]
            vwait [namespace which -variable v]
        }
        return $v
    }
}
```

Läuft der Code in einer coroutine, ist eine Alternative zu **vwait**, mit yield an eine äußere Event-Schleife zu übergeben und fortgesetzt zu werden, wenn die Variable gesetzt wird oder danach in einem Idle-Moment.

```
coroutine task apply {{} {
    # [after 1000] simulieren
    after 1000 [info coroutine]
    yield

    # Setzen einer globalen Variable planen, wie üblich
    after 2000 {set var 1}

    # [vwait var] simulieren
    proc updatedVar {task args} {
        after idle $task
        trace remove variable ::var write "updatedVar $task"
    }
    trace add variable ::var write "updatedVar [info coroutine]"
    yield
}}
```

### Verschachtelte vwaits anhand eines Beispiels

Dieses Beispiel zeigt, was bei verschachtelten **vwait**-Befehlen passieren kann. Das Skript wird nie enden, weil das Warten auf die Variable *a* nie abgeschlossen wird; dieser **vwait** wartet noch darauf, dass ein mit after geplantes Skript endet, das zufällig einen inneren **vwait** (für *b*) ausführt, obwohl das Ereignis, auf das der äußere **vwait** wartete (das Setzen von *a*), bereits eingetreten ist.

```
after 500 {
    puts "waiting for b"
    vwait b
    puts "b was set"
}
after 1000 {
    puts "setting a"
    set a 10
}
puts "waiting for a"
vwait a
puts "a was set"
puts "setting b"
set b 42
```

Führt man den obigen Code aus, erscheint diese Ausgabe:

```
waiting for a
waiting for b
setting a
```

Das Skript gibt "a was set" erst aus, nachdem "b was set" gedruckt wurde, wegen der Verschachtelung der **vwait**-Befehle; *b* wird jedoch erst gesetzt, nachdem der äußere **vwait** zurückkehrt — das Skript ist verklemmt. Vermeiden lässt sich das nur, indem man das Gesamtprogramm im Continuation-Passing-Stil strukturiert oder coroutine nutzt, um die Fortsetzungen implizit zu machen. Die erste Option sieht so aus:

```
after 500 {
    puts "waiting for b"
    trace add variable b write {apply {args {
        global a b
        trace remove variable ::b write \
                [lrange [info level 0] 0 1]
        puts "b was set"
        set ::done ok
    }}}
}
after 1000 {
    puts "setting a"
    set a 10
}
puts "waiting for a"
trace add variable a write {apply {args {
    global a b
    trace remove variable a write [lrange [info level 0] 0 1]
    puts "a was set"
    puts "setting b"
    set b 42
}}}
vwait done
```

Die zweite Option mit coroutine und einigen Hilfsprozeduren:

```
# Koroutine-basierter Befehl: auf Variable warten
proc waitvar globalVar {
    trace add variable ::$globalVar write \
            [list apply {{v c args} {
        trace remove variable $v write \
                [lrange [info level 0] 0 3]
        after 0 $c
    }} ::$globalVar [info coroutine]]
    yield
}
# Koroutine-basierter Befehl: Zeit abwarten
proc waittime ms {
    after $ms [info coroutine]
    yield
}

coroutine task-1 eval {
    puts "waiting for a"
    waitvar a
    puts "a was set"
    puts "setting b"
    set b 42
}
coroutine task-2 eval {
    waittime 500
    puts "waiting for b"
    waitvar b
    puts "b was set"
    set done ok
}
coroutine task-3 eval {
    waittime 1000
    puts "setting a"
    set a 10
}
vwait done
```

*Copyright (c) 1995-1996 Sun Microsystems, Inc.*

# while

*Stichwörter:* while, boolean, loop, test

## Name

while - Führt ein Script wiederholt aus, solange eine Bedingung erfüllt ist

## Übersicht

**while** *test body*

## Beschreibung

Der Befehl **while** wertet *test* als Ausdruck aus (auf dieselbe Weise wie expr sein Argument auswertet). Der Wert des Ausdrucks muss ein gültiger Boolean-Wert sein; wenn er wahr ist, wird *body* ausgeführt, indem es an den Tcl-Interpreter übergeben wird. Sobald *body* ausgeführt wurde, wird *test* erneut ausgewertet, und der Prozess wiederholt sich, bis *test* schließlich zu einem falschen Boolean-Wert ausgewertet wird. **continue**-Befehle können innerhalb von *body* ausgeführt werden, um die aktuelle Iteration der Schleife zu beenden, und break-Befehle können innerhalb von *body* ausgeführt werden, um den **while**-Befehl sofort zu beenden. Der **while** -Befehl gibt immer einen leeren String zurück.

Beachten Sie, dass *test* fast immer in geschweifte Klammern eingeschlossen werden sollte. Wenn nicht, werden Variablenersetzungen vorgenommen, bevor der **while**-Befehl mit der Ausführung beginnt, was bedeutet, dass Variablenänderungen durch den Schleifenkörper nicht im Ausdruck berücksichtigt werden. Dies führt wahrscheinlich zu einer Endlosschleife. Wenn *test* in geschweifte Klammern eingeschlossen ist, werden Variablenersetzungen verzögert, bis der Ausdruck ausgewertet wird (vor jeder Schleifeniteration), sodass Änderungen an den Variablen sichtbar werden. Probieren Sie als Beispiel das folgende Script mit und ohne die geschweiften Klammern um **$x<10**:

```
set x 0
while {$x<10} {
    puts "x is $x"
    incr x
}
```

## Beispiel

Liest Zeilen aus einem Kanal bis zum Ende des Streams und gibt sie mit vorangestellter Zeilennummer aus:

```
set lineCount 0
while {[gets $chan line] >= 0} {
    puts "[incr lineCount]: $line"
}
```

*Copyright (c) 1993 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# zipfs

*Stichwörter:* zipfs, compress, filesystem, zip

## Name

zipfs - ZIP-Archive in Tcl einbinden und damit arbeiten

## Übersicht

**zipfs canonical** ?*mountpoint*? *filename*
**zipfs exists** *filename*
**zipfs find** *directoryName*
**zipfs info** *filename*
**zipfs list** ?*(-glob*|*-regexp)*? ?*pattern*?
**zipfs lmkimg** *outfile inlist* ?*password*? ?*infile*?
**zipfs lmkzip** *outfile inlist* ?*password*?
**zipfs mkimg** *outfile indir* ?*strip*? ?*password*? ?*infile*?
**zipfs mkkey** *password*
**zipfs mkzip** *outfile indir* ?*strip*? ?*password*?
**zipfs mount** ?*zipfile*? ?*mountpoint*? ?*password*?
**zipfs mountdata** *data mountpoint*
**zipfs root**
**zipfs unmount** *mountpoint*

## Beschreibung

Der Befehl **zipfs** ermöglicht Tcl, den Inhalt einer ZIP-Archivdatei als virtuelles Dateisystem einzubinden. Die ZIP-Archiv-Unterstützung von Tcl ist auf grundlegende Funktionen und Optionen beschränkt. Unterstützte Speichermethoden umfassen nur STORE und DEFLATE mit optionaler einfacher Verschlüsselung, ausreichend um eine oberflächliche Einsicht in den Inhalt zu verhindern, aber nicht um den Zugriff selbst durch einen nur mäßig entschlossenen Angreifer zu verhindern. Starke Verschlüsselung, mehrteilige Archive, Plattform-Metadaten, zip64-Formate und andere Kompressionsmethoden wie bzip2 werden nicht unterstützt.

Dateien in eingebundenen Archiven können beschrieben werden, aber neue Dateien oder Verzeichnisse können nicht erstellt werden. Außerdem sind Änderungen an Dateien auf das eingebundene Archiv im Speicher beschränkt und werden nicht auf die Platte geschrieben.

Pfade in eingebundenen Archiven sind auf allen Plattformen groß-/kleinschreibungssensitiv.

**zipfs canonical ?*mountpoint*? *filename***

Nimmt den Namen einer Datei, *filename*, und liefert als Ergebnis, wohin sie in einem zipfs-Mountpunkt abgebildet wäre. Wenn angegeben, gibt *mountpoint* an, innerhalb welches Mounts die Abbildung erfolgt; wenn weggelassen, wird die Hauptwurzel des zipfs-Systems verwendet.

**zipfs exists *filename***

Gibt 1 zurück, wenn der angegebene Dateiname im eingebundenen zipfs existiert, und 0, wenn nicht.

**zipfs find *directoryName***

Gibt die Liste der Pfade unter dem Verzeichnis *directoryName* zurück, das nicht innerhalb eines zipfs-eingebundenen Archivs liegen muss. Die Pfade werden mit *directoryName* vorangestellt. Dieser Befehl wird auch von den Befehlen **zipfs mkzip** und **zipfs mkimg** verwendet.

**zipfs info *file***

Gibt Informationen über die angegebene *file* im eingebundenen zipfs zurück. Die Informationen bestehen aus:

1. dem Namen der ZIP-Archivdatei, die die Datei enthält,
2. der Größe der Datei nach der Dekompression,
3. der komprimierten Größe der Datei und
4. dem Offset der komprimierten Daten in der ZIP-Archivdatei.

Als Sonderfall liefert eine Abfrage des Mountpunkts den Beginn der ZIP-Daten als Offset in (4), was verwendet werden kann, um die ZIP-Informationen aus einer ausführbaren Datei abzuschneiden. Eine Abfrage eines Vorfahren eines Mountpunkts löst einen Fehler aus.

**zipfs list ?*(-glob*|*-regexp)*? ?*pattern*?**

Wenn *pattern* nicht angegeben ist, gibt der Befehl eine Liste von Dateien über alle zipfs-eingebundenen Archive zurück. Wenn *pattern* angegeben ist, werden nur die Pfade zurückgegeben, die dem Muster entsprechen. Standardmäßig oder mit der Option **-glob** wird das Muster als Glob-Muster behandelt und der Abgleich erfolgt wie für den Befehl string match beschrieben. Alternativ kann die Option **-regexp** verwendet werden, um den Abgleich von **pattern** als regulären Ausdruck festzulegen. Die Dateinamen werden in beliebiger Reihenfolge zurückgegeben. Beachten Sie, dass Pfadtrennzeichen beim Abgleich als normale Zeichen behandelt werden. Daher sollten Schrägstriche als Pfadtrennzeichen im Muster verwendet werden. Die zurückgegebenen Pfade umfassen nur diejenigen, die tatsächlich im Archiv sind, und nicht Zwischenverzeichnisse in Mount-Pfaden.

**zipfs mount**

siehe unten ...

**zipfs mount *mountpoint***

siehe unten ...

**zipfs mount *zipfile mountpoint* ?*password*?**

Der Befehl **zipfs mount** bindet ZIP-Archive als Tcl-virtuelle Dateisysteme ein und gibt Informationen über aktuelle Mounts zurück.

Ohne Argumente gibt der Befehl ein Dictionary zurück, das Mountpunkte den Pfaden der zugehörigen ZIP-Archive zuordnet.

In der Form mit einem Argument gibt der Befehl den Dateipfad des ZIP-Archivs zurück, das am angegebenen Mountpunkt eingebunden ist.

In der dritten Form bindet der Befehl das ZIP-Archiv *zipfile* als Tcl-virtuelles Dateisystem am *mountpoint* ein. Nach Ausführung dieses Befehls erscheinen die in *zipfile* enthaltenen Dateien für Tcl als reguläre Dateien am Mountpunkt. Wenn *mountpoint* als leerer String angegeben wird, wird standardmäßig **[zipfs root]** verwendet. Der Befehl gibt den normalisierten Mountpunkt-Pfad zurück.

Wenn *mountpoint* nicht unter der zipfs-Dateisystemwurzel liegt, wird er relativ dazu normalisiert. Beispielsweise würde ein Mountpunkt, der als **mt** oder **/mt** übergeben wird, zu **//zipfs:/mt** normalisiert (wenn **zipfs root** "//zipfs:/" zurückgibt). Es wird ein Fehler ausgelöst, wenn der Mountpunkt ein Laufwerk oder ein UNC-Volume enthält.

**Hinweis:** Da das aktuelle Arbeitsverzeichnis ein vom Betriebssystem verwaltetes Konzept ist, funktioniert cd in ein eingebundenes Archiv nur im aktuellen Prozess und dann nicht vollständig konsistent (z. B. wenn eine Shared Library direkten Zugriff auf das Betriebssystem statt über Tcls Dateisystem-API verwendet, sieht sie das aktuelle Verzeichnis nicht innerhalb des Mounts und kann nicht auf die Dateien im Mount zugreifen).

**zipfs mountdata *data mountpoint***

Bindet den ZIP-Archivinhalt *data* als Tcl-virtuelles Dateisystem am *mountpoint* ein.

**zipfs root**

Gibt einen konstanten String zurück, der den Mountpunkt für zipfs-Volumes auf der aktuellen Plattform angibt. Der Benutzer sollte sich nicht darauf verlassen, dass der Mountpunkt auf allen Plattformen derselbe konstante String ist.

**zipfs unmount *mountpoint***

Hängt ein zuvor am *mountpoint* eingebundenes ZIP-Archiv ab. Der Befehl schlägt mit einer Fehlerausnahme fehl, wenn noch Dateien innerhalb des eingebundenen Archivs geöffnet sind.

## Befehle zur ZIP-Erstellung

Dieses Paket stellt außerdem mehrere Befehle bereit, die bei der Erstellung von ZIP-Archiven als Tcl-Anwendungen helfen.

**zipfs mkzip *outfile indir* ?*strip*? ?*password*?**

Erstellt eine ZIP-Archivdatei namens *outfile* aus dem Inhalt des Eingabeverzeichnisses *indir* (nur reguläre Dateien) mit optionalem ZIP-Passwort *password*. Beim Verarbeiten der Dateien unter *indir* wird das optionale Dateinamenpräfix in *strip* vom Anfang des jeweiligen Dateinamens entfernt, falls es nicht leer ist. Beim Strippen ist es üblich, entweder den gesamten Quellverzeichnisnamen oder den Namen des übergeordneten Verzeichnisses zu entfernen.

**Vorsicht:** Die Wahl des Parameters *indir* (abzüglich des optional gestrippten Präfixes) bestimmt den späteren Wurzelnamen des Archivinhalts.

**zipfs mkimg *outfile indir* ?*strip*? ?*password*? ?*infile*?**

Erstellt ein Image (potenziell eine neue ausführbare Datei) ähnlich wie **zipfs mkzip**; siehe diesen Befehl für eine Beschreibung der meisten Parameter dieses Befehls, da sie sich hier identisch verhalten. Wenn *outfile* existiert, wird es stillschweigend überschrieben.

Wenn der Parameter *infile* angegeben ist, wird diese Datei vor das ZIP-Archiv gesetzt, andernfalls wird die von info nameofexecutable zurückgegebene Datei verwendet (d. h. die ausführbare Datei des laufenden Prozesses, typischerweise wish oder tclsh). Wenn der

Parameter *password* nicht der leere String ist, wird eine verschleierte Version dieses Passworts (siehe **zipfs mkkey**) zwischen Image- und ZIP-Teil der Ausgabedatei platziert und der Inhalt des ZIP-Teils mit diesem Passwort geschützt. Wenn das Ausgangs-Image bereits ein ZIP-Archiv angehängt hat, wird es aus der Kopie in *outfile* entfernt, bevor das neue ZIP-Archiv hinzugefügt wird.

Wenn es eine Datei **main.tcl** im Wurzelverzeichnis des resultierenden Archivs gibt und die Image-Datei, an die das Archiv angehängt ist, eine tclsh- (oder wish-)Instanz ist (standardmäßig wahr, hängt aber von der Konfiguration ab), dann ist das resultierende Image eine ausführbare Datei, die nach dem Einbinden des ZIP-Archivs das Skript in **main.tcl** per source ausführt und nach Ausführung dieses Skripts per exit beendet.

**Hinweis:** tclsh und wish können mit dynamischer oder statischer Bindung der Kernimplementierungsbibliotheken gebaut werden. Bei dynamischer Bindung sind die Tcl_Library-Inhalte der Basisanwendung an die Shared Libraries **libtcl** bzw. **libtk** angehängt. Bei statischer Bindung sind die Tcl_Library-Inhalte usw. an die Anwendung tclsh oder wish angehängt. Bei Verwendung von **mkimg** mit einem statisch gebauten tclsh liegt es in der Verantwortung des Benutzers, das angehängte Archiv zu erhalten, indem er es zuerst an einen temporären Ort extrahiert und dann die gewünschten zusätzlichen Dateien hinzufügt, bevor das neue Archiv erstellt und an die neue Anwendung angehängt wird.

**zipfs mkkey *password***

Gibt bei dem Klartext-Argument *password* eine verschleierte Stringversion im gleichen Format zurück, das im Befehl **zipfs mkimg** verwendet wird.

**zipfs lmkimg *outfile inlist* ?*password*? ?*infile*?**

Dieser Befehl entspricht **zipfs mkimg**, aber anstelle eines Eingabeverzeichnisses muss *inlist* eine Tcl-Liste sein, deren ungerade Elemente die Namen der in das Archiv im Image zu kopierenden Dateien sind und deren gerade Elemente ihre jeweiligen Namen innerhalb dieses Archivs.

**zipfs lmkzip *outfile inlist* ?*password*?**

Dieser Befehl entspricht **zipfs mkzip**, aber anstelle eines Eingabeverzeichnisses muss *inlist* eine Tcl-Liste sein, deren ungerade Elemente die Namen der in das Archiv zu kopierenden Dateien sind und deren gerade Elemente ihre jeweiligen Namen innerhalb dieses Archivs.

## Hinweis

Die aktuelle Syntax bestimmter Unterbefehle mit mehreren optionalen Parametern könnte sich in Zukunft ändern, um stattdessen ein Muster*?-option value?* zu unterstützen. Daher sollte die aktuelle Syntax nicht als stabil betrachtet werden.

## Beispiele

Einbinden eines ZIP-Archivs als Anwendungsverzeichnis, Ausführen von Code daraus und anschließendes Abhängen:

```
set zip myApp.zip
set base [file join [zipfs root] myApp]

zipfs mount $zip $base
```

```
# $base enthält jetzt den Inhalt von myApp.zip

source [file join $base app.tcl]
# Inhalt nutzen, Bibliotheken daraus laden, usw.

zipfs unmount $base
```

Erstellen eines ZIP-Archivs, wenn ein Verzeichnis mit dem einzupackenden Inhalt existiert. Beachten Sie, dass das Quellverzeichnis zweimal angegeben wird, um den äußeren Verzeichnisnamen von jedem Dateinamen im Archiv zu entfernen.

```
set sourceDirectory [file normalize myApp]
set targetZip myApp.zip

zipfs mkzip $targetZip $sourceDirectory $sourceDirectory
```

Verschlüsselung kann auf ZIP-Archive angewendet werden, indem beim Erstellen und beim Einbinden ein Passwort angegeben wird.

```
set zip myApp.zip
set sourceDir [file normalize myApp]
set password "hunter2"
set base [file join [zipfs root] myApp]

# Mit Passwort erstellen
zipfs mkzip $targetZip $sourceDir $sourceDir $password

# Mit Passwort einbinden
zipfs mount $zip $base $password
```

Das folgende Beispiel erstellt eine ausführbare Anwendung, indem ein ZIP-Archiv an die tclsh-Datei angehängt wird, von der es aufgerufen wurde, und die resultierende ausführbare Datei in der Datei "myApp.bin" gespeichert wird. Beim Erstellen eines ausführbaren Images mit Passwort wird das Passwort in verschleierter Form in die ausführbare Datei eingebettet, sodass die Anwendung Dateien im eingebetteten ZIP-Archiv lesen kann, eine oberflächliche Einsicht es jedoch nicht lesen kann.

```
set appDir [file normalize myApp]
set img "myApp.bin"
set password "hunter2"

# Einfachen Inhalt für eine Basisanwendung anlegen
file mkdir $appDir
set f [open $appDir/main.tcl w]
puts $f {
    puts "Hi. This is [info script]"
}
close $f
```

```
# Ausführbare Anwendung erstellen
zipfs mkimg $img $appDir $appDir $password

# jetzt überflüssigen temporären appDir-Ordner entfernen
file delete -force $appDir

# Ausführbare Datei starten, Ausgabe nach stdout
exec $img >@stdout
# gibt folgende Zeile aus, wenn [zipfs root] "//zipfs:/" zurückgibt:
# Hi. This is //zipfs:/app/main.tcl
```

*Copyright (c) 2015 Jan Nijtmans* jan.nijtmans@gmail.com
*Copyright (c) 2015 Christian Werner* chw@ch-werner.de
*Copyright (c) 2015 Sean Woods* yoda@etoyoc.com

# zlib

*Stichwörter:* zlib, compress, decompress, deflate, gzip, inflate

## Name

zlib - Komprimierungs- und Dekomprimierungsoperationen

## Übersicht

**zlib** *subcommand arg ...*

## Beschreibung

Der Befehl **zlib** stellt Zugriff auf die Komprimierungs- und Prüfsummenfunktionen der Zlib-Bibliothek von Jean-loup Gailly und Mark Adler bereit. Er verfügt über folgende Unterbefehle.

### Komprimierungs-Unterbefehle

**zlib compress *string* ?*level*?**

Gibt die komprimierten Binärdaten im zlib-Format der Binärzeichenkette in *string* zurück. Ist *level* angegeben, legt es die zu verwendende Komprimierungsstufe fest (von 0, unkomprimiert, bis 9, maximal komprimiert).

**zlib decompress *string* ?*bufferSize*?**

Gibt die unkomprimierte Version der rohen komprimierten Binärdaten in *string* zurück. Ist *bufferSize* angegeben, ist es ein Hinweis auf die Puffergröße, die zum Empfang der Daten verwendet werden soll.

**zlib deflate *string* ?*level*?**

Gibt die rohen komprimierten Binärdaten der Binärzeichenkette in *string* zurück. Ist *level* angegeben, legt es die zu verwendende Komprimierungsstufe fest (von 0, unkomprimiert, bis 9, maximal komprimiert).

**zlib gunzip *string* ?-*headerVar varName*?**

Gibt den unkomprimierten Inhalt der Binärzeichenkette *string* zurück, die im gzip-Format vorliegen muss. Ist **-headerVar** angegeben, speichert er ein Dictionary, das den Inhalt des gzip-Headers beschreibt, in der Variable *varName*. Die Schlüssel des Dictionarys, die vorhanden sein können, sind:

**comment**
Das Kommentarfeld aus dem Header, falls vorhanden.

**crc**
Ein boolescher Wert, der angibt, ob eine CRC des Headers berechnet wird.

**filename**
Das Dateinamenfeld aus dem Header, falls vorhanden.

**os**
Das Feld für den Betriebssystemtypcode aus dem Header (wenn nicht der QW-Unbekannt-Wert). Siehe RFC 1952 für die Bedeutung dieser Codes.

**size**
Die Größe der unkomprimierten Daten.

**time**
Das Zeitfeld aus dem Header, falls ungleich null; erwartet wird die Zeit, zu der die Datei mit dem Namen aus dem Feld **filename** geändert wurde. Geeignet für die Verwendung mit clock format.

**type**
Der Typ der unkomprimierten Daten (**binary** oder **text**), falls bekannt.

**zlib gzip *string* ?-*level level*? ?-*header dict*?**

Gibt den komprimierten Inhalt der Binärzeichenkette *string* im gzip-Format zurück. Ist **-level** angegeben, legt *level* die zu verwendende Komprimierungsstufe fest (von 0, unkomprimiert, bis 9, maximal komprimiert). Ist **-header** angegeben, ist *dict* ein Dictionary mit Werten für den gzip-Header. Folgende Schlüssel können definiert werden:

**comment**
Fügt den angegebenen Kommentar dem Header der gzip-formatierten Daten hinzu.

**crc**
Ein boolescher Wert, der angibt, ob eine CRC des Headers berechnet werden soll. Beachten Sie, dass bei Datenaustausch mit dem Programm **gzip** keine Header-CRC berechnet werden sollte.

**filename**
Der Name der Datei, aus der die zu komprimierenden Daten stammen.

**os**
Der Betriebssystemtypcode, der einer der in RFC 1952 beschriebenen Werte sein sollte.

**time**
Die Zeit, zu der die Datei mit dem Schlüssel **filename** zuletzt geändert wurde. Entspricht dem Rückgabewert von clock seconds oder file mtime.

**type**
Der Typ der komprimierten Daten, entweder **binary** oder **text**.

**zlib inflate *string* ?*bufferSize*?**

Gibt die unkomprimierte Version der rohen komprimierten Binärdaten in *string* zurück. Ist *bufferSize* angegeben, ist es ein Hinweis auf die Puffergröße, die zum Empfang der Daten verwendet werden soll.

## Kanal-Unterbefehl

**zlib push *mode channel* ?*options* ...?**

Legt eine komprimierende oder dekomprimierende Transformation auf den Kanal *channel* auf. Die Transformation kann mit chan pop wieder entfernt werden. Das Argument *mode* bestimmt, welche Art von Transformation aufgelegt wird; Folgendes wird unterstützt:

**compress**

Die Transformation komprimiert und erzeugt zlib-formatierte Daten auf *channel*, der beschreibbar sein muss.

**decompress**

Die Transformation dekomprimiert und liest zlib-formatierte Daten von *channel*, der lesbar sein muss.

**deflate**

Die Transformation komprimiert und erzeugt rohe komprimierte Daten auf *channel*, der beschreibbar sein muss.

**gunzip**

Die Transformation dekomprimiert und liest gzip-formatierte Daten von *channel*, der lesbar sein muss.

**gzip**

Die Transformation komprimiert und erzeugt gzip-formatierte Daten auf *channel*, der beschreibbar sein muss.

**inflate**

Die Transformation dekomprimiert und liest rohe komprimierte Daten von *channel*, der lesbar sein muss.

Folgende Optionen können beim Erstellen einer Transformation über die "*options ...*" des Befehls **zlib push** gesetzt werden:

**-dictionary *binData***

Legt das zu verwendende Komprimierungswörterbuch auf *binData* fest, wenn komprimierende oder dekomprimierende Transformationen angewendet werden. Nicht gültig für Transformationen, die mit gzip-formatierten Daten arbeiten. Das Wörterbuch sollte aus Zeichenketten (Bytefolgen) bestehen, die später in den zu komprimierenden Daten wahrscheinlich vorkommen, wobei die am häufigsten verwendeten Zeichenketten vorzugsweise ans Ende des Wörterbuchs gesetzt werden sollten. Tcl bietet keinen Mechanismus zur Auswahl eines geeigneten Wörterbuchs für eine bestimmte Datenfolge.

**-header *dictionary***

Übergibt eine Beschreibung des zu erstellenden gzip-Headers im gleichen Format, das **zlib gzip** versteht.

**-level *compressionLevel***

Legt fest, wie stark die Daten komprimiert werden. Muss eine Ganzzahl von 0 (unkomprimiert) bis 9 (maximal komprimiert) sein.

**-limit *readaheadLimit***

Die maximale Anzahl von Bytes, die beim Dekomprimieren vorausgelesen werden.

Diese Option ist **obsolet** geworden. Sie wurde ursprünglich eingeführt, um zu verhindern, dass Tcl in Mehrfachstream-Kanälen über das Ende eines komprimierten Streams hinaus liest, damit die anschließenden Daten für weiteres Lesen unberührt bleiben — auf Kosten der Geschwindigkeit.

Tcl gibt jetzt automatisch alle Bytes, die über das Ende eines komprimierten Streams hinaus gelesen wurden, an den Kanal zurück, sodass sie für nachfolgende Leser als ungelesen erscheinen.

Sowohl komprimierende als auch dekomprimierende Kanaltransformationen fügen zusätzliche Konfigurationsoptionen hinzu, die über chan configure abgerufen werden können. Die Optionen sind:

**-checksum *checksum***

Diese schreibgeschützte Option liefert die aktuelle Prüfsumme für die unkomprimierten Daten, die die Komprimierungs-Engine bisher gesehen hat. Gültig für komprimierende und dekomprimierende Transformationen, aber nicht für die rohen inflate- und deflate-Formate. Der Komprimierungsalgorithmus hängt davon ab, welches Format erzeugt oder verarbeitet wird.

**-dictionary *binData***

Diese lese-schreibbare Option liest oder setzt das anfängliche Komprimierungswörterbuch auf *binData*, wenn komprimierende oder dekomprimierende Transformationen angewendet werden. Nicht gültig für Transformationen, die mit gzip-formatierten Daten arbeiten, und sollte bei komprimierenden Transformationen normalerweise nur beim Auflegen der Transformation gesetzt werden. Beachten Sie, dass damit nicht das aktuell aktive Komprimierungswörterbuch mitten im Stream abgefragt werden kann, da diese Information von der zugrunde liegenden Bibliothek nicht bereitgestellt wird.

**-flush *type***

Diese nur-schreibbare Operation leert den aktuellen Zustand des Komprimierers in den zugrunde liegenden Kanal. Nur gültig für komprimierende Transformationen. *type* muss entweder **sync** oder **full** sein — für ein normales bzw. aufwendiges Leeren. Leeren verschlechtert das Komprimierungsverhältnis, erleichtert aber einem Dekomprimierer die Wiederherstellung eines größeren Teils der Datei bei Datenbeschädigung.

**-header *dictionary***

Diese schreibgeschützte Option, nur gültig für dekomprimierende Transformationen, die gzip-formatierte Daten verarbeiten, gibt das Dictionary zurück, das den aus dem Datenstrom gelesenen Header beschreibt.

**-limit *readaheadLimit***

Diese lese-schreibbare Option steuert bei dekomprimierenden Kanälen die maximale Anzahl vorausgelesener Bytes aus der zugrunde liegenden Datenquelle. Siehe oben für weitere Informationen.

## Streaming-Unterbefehl

### zlib stream *mode* ?*options*?

Erstellt einen Streaming-Komprimierungs- oder Dekomprimierungsbefehl basierend auf *mode* und gibt den Namen des Befehls zurück. Eine Beschreibung der Funktionsweise finden Sie unter Streaming-Instanzbefehl weiter unten. Folgende Modi und *options* werden unterstützt:

**zlib stream compress ?*-dictionary bindata*? ?*-level level*?**

Der Stream komprimiert und erzeugt Ausgabe im zlib-Format, mit der Komprimierungsstufe *level* (falls angegeben; eine Ganzzahl von 0 bis 9) und dem Komprimierungswörterbuch *bindata* (falls angegeben).

**zlib stream decompress ?*-dictionary bindata*?**

Der Stream dekomprimiert Eingabe im zlib-Format und erzeugt unkomprimierte Ausgabe. Ist *bindata* angegeben, ist es ein bei Bedarf zu verwendendes Komprimierungswörterbuch.

**zlib stream deflate ?*-dictionary bindata*? ?*-level level*?**

Der Stream komprimiert und erzeugt rohe Ausgabe, mit der Komprimierungsstufe *level* (falls angegeben; eine Ganzzahl von 0 bis 9) und dem Komprimierungswörterbuch *bindata* (falls angegeben). Beachten Sie, dass die rohen komprimierten Daten keine Metadaten darüber enthalten, welches Komprimierungswörterbuch — falls überhaupt — verwendet wurde; das ist ein Merkmal der zlib-formatierten Daten.

**zlib stream gunzip**

Der Stream dekomprimiert Eingabe im gzip-Format und erzeugt unkomprimierte Ausgabe.

**zlib stream gzip ?*-header header*? ?*-level level*?**

Der Stream komprimiert und erzeugt Ausgabe im gzip-Format, mit der Komprimierungsstufe *level* (falls angegeben; eine Ganzzahl von 0 bis 9) und dem Header-Deskriptor-Dictionary *header* (falls angegeben; Schlüssel siehe **zlib gzip**).

**zlib stream inflate ?*-dictionary bindata*?**

Der Stream dekomprimiert rohe komprimierte Eingabe und erzeugt unkomprimierte Ausgabe. Ist *bindata* angegeben, ist es ein zu verwendendes Komprimierungswörterbuch. Beachten Sie, dass keine Prüfungen vorhanden sind, ob das Komprimierungswörterbuch korrekt ist.

### Prüfsummen-Unterbefehle

**zlib adler32 *string* ?*initValue*?**

Berechnet eine Prüfsumme der Binärzeichenkette *string* mit dem Adler-32-Algorithmus. Ist *initValue* angegeben, initialisiert er damit die Prüfsummen-Engine.

**zlib crc32 *string* ?*initValue*?**

Berechnet eine Prüfsumme der Binärzeichenkette *string* mit dem CRC-32-Algorithmus. Ist *initValue* angegeben, initialisiert er damit die Prüfsummen-Engine.

## Streaming-Instanzbefehl

Streaming-Komprimierungs-Instanzbefehle werden vom Befehl **zlib stream** erzeugt. Sie werden verwendet, indem ihr Unterbefehl **put** ein- oder mehrmals aufgerufen wird, um Daten zu laden, und ihr Unterbefehl **get** ein- oder mehrmals, um die transformierten Daten zu extrahieren.

Die vollständige Menge der von einem Streaming-Instanzbefehl *stream* unterstützten Unterbefehle lautet:

***stream* add ?*option...*? *data***

Eine Abkürzung für "*stream* **put** ?*option...*? *data*" gefolgt von "*stream* **get**".

***stream* checksum**

Gibt die Prüfsumme der unkomprimierten Daten zurück, die dieser Stream bisher gesehen hat.

***stream* close**

Löscht diesen Stream und gibt alle zugehörigen Ressourcen frei.

***stream* eof**

Gibt einen booleschen Wert zurück, der angibt, ob das Ende des Streams (wie durch die komprimierten Daten selbst bestimmt) erreicht wurde. Nicht alle Formate unterstützen die Erkennung des Streamendes.

***stream* finalize**

Abkürzung für "*stream* **put -finalize {}**".

***stream* flush**

Abkürzung für "*stream* **put -flush {}**".

***stream* fullflush**

Abkürzung für "*stream* **put -fullflush {}**".

***stream* get ?*count*?**

Gibt bis zu *count* Bytes aus den internen Puffern von *stream* mit angewendeter Transformation zurück. Fehlt *count*, wird der gesamte Pufferinhalt zurückgegeben.

***stream* header**

Gibt das Dictionary mit der gzip-Header-Beschreibung zurück, das aus dem Stream extrahiert wurde. Nur unterstützt für Streams, die mit dem *mode*-Parameter **gunzip** erstellt wurden.

***stream* put ?*option...*? *data***

Hängt den Inhalt der Binärzeichenkette *data* an die internen Puffer von *stream* an und wendet dabei die Transformation an. Folgende *option*en werden unterstützt (oder ein eindeutiges Präfix davon), die die Art der Transformation beeinflussen:

**-dictionary *binData***

Legt das zu verwendende Komprimierungswörterbuch auf *binData* fest, wenn komprimierende oder dekomprimierende Transformationen angewendet werden.

**-finalize**

Markiert den Stream als abgeschlossen und stellt sicher, dass alle Bytes vollständig komprimiert oder dekomprimiert wurden. Bei gzip-Streams wird damit auch sichergestellt, dass der Footer in den Stream geschrieben wird. Der Stream muss danach zurückgesetzt werden, bevor weitere Daten geschrieben werden können; gelesen werden kann er weiterhin mit dem Unterbefehl **get**.

Diese Option schließt sich gegenseitig mit den Optionen **-flush** und **-fullflush** aus.

**-flush**

Stellt sicher, dass ein Dekomprimierer, der die vom aktuellen (komprimierenden) Stream erzeugten Bytes verarbeitet, alle bisher komprimierten Bytes ausgeben kann — mit gewissen Leistungseinbußen.

Diese Option schließt sich gegenseitig mit den Optionen **-finalize** und **-fullflush** aus.

**-fullflush**

Stellt sicher, dass nicht nur ein Dekomprimierer alle bisher erzeugten Bytes verarbeiten kann (wie bei **-flush** oben), sondern auch von diesem Punkt aus neu starten kann, wenn er feststellt, dass der Stream teilweise beschädigt ist. Dies verursacht erhebliche Leistungseinbußen.

Diese Option schließt sich gegenseitig mit den Optionen **-finalize** und **-flush** aus.

***stream* reset**

Setzt jeden Stream, einschließlich finalisierter oder EOF-erreichte, in einen Zustand zurück, in dem er weitere Daten verarbeiten kann. Verwirft alle intern gepufferten Daten.

### Beispiele

Um eine Tcl-Zeichenkette zu komprimieren, sollte sie zuerst in eine bestimmte Zeichensatzkodierung umgewandelt werden, da der Befehl **zlib** immer auf Binärzeichenketten arbeitet.

```
set binData [encoding convertto utf-8 $string]
set compData [zlib compress $binData]
```

Beim Rückwandeln ist es ebenfalls wichtig, die Zeichensatzkodierung rückgängig zu machen:

```
set binData [zlib decompress $compData]
set string [encoding convertfrom utf-8 $binData]
```

Die Komprimierung von oben kann auch mit Streams durchgeführt werden, was besonders hilfreich ist, wenn die Daten schrittweise aufgebaut werden sollen:

```
set strm [zlib stream compress]
$strm put [encoding convertto utf-8 $string]
# ...
$strm finalize
set compData [$strm get]
$strm close
```

*Copyright (c) 2008-2012 Donal K. Fellows*

# Tk

## bell

*Stichwörter:* bell, beep, ring

### Name

bell - Läutet die Klingel eines Displays

### Übersicht

**bell** ?-*displayof window*? ?**-nice**?

### Beschreibung

Dieser Befehl läutet die Klingel auf dem Display von *window* und gibt eine leere Zeichenkette zurück. Fehlt die Option **-displayof**, wird das Display des Hauptfensters der Anwendung verwendet. Der Befehl nutzt die aktuellen Klingel-Einstellungen des Displays; Programme wie **xset** können sie ändern.

Ohne **-nice** setzt dieser Befehl auch den Bildschirmschoner des Bildschirms zurück. Manche Bildschirmschoner ignorieren das, andere setzen sich so zurück, dass der Bildschirm wieder sichtbar wird.

*Copyright (c) 1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

*Copyright (c) 2000 Ajuba Solutions.*

# bind

*Stichwörter:* bind, binding, event

## Name

bind - Verknüpft X-Ereignisse mit Tcl-Scripten

## Übersicht

**bind** *tag* ?*sequence*? ?**+**? ?*script*?

## Einleitung

Der Befehl **bind** verknüpft Tcl-Scripte mit X-Ereignissen. Wenn alle drei Argumente angegeben sind, sorgt **bind** dafür, dass *script* (ein Tcl-Script, das "Binding-Script" genannt wird) ausgewertet wird, wann immer das/die durch *sequence* gegebene(n) Ereignis(se) in dem/den durch *tag* identifizierten Fenster(n) auftreten. Wenn *script* mit einem "+" vorangestellt ist, wird es an ein vorhandenes Binding für *sequence* angehängt; andernfalls ersetzt *script* ein vorhandenes Binding. Wenn *script* ein leerer String ist, wird das aktuelle Binding für *sequence* zerstört, sodass *sequence* ungebunden bleibt. In allen Fällen, in denen ein *script*-Argument bereitgestellt wird, gibt **bind** einen leeren String zurück.

Wenn *sequence* ohne *script* angegeben ist, wird das aktuell an *sequence* gebundene Script zurückgegeben, oder ein leerer String wird zurückgegeben, wenn es kein Binding für *sequence* gibt. Wenn weder *sequence* noch *script* angegeben ist, ist der Rückgabewert eine Liste, deren Elemente alle Sequenzen sind, für die Bindings für *tag* existieren.

Das *tag*-Argument bestimmt, auf welche(s) Fenster das Binding angewendet wird. Wenn *tag* mit einem Punkt beginnt, wie in **.a.b.c**, muss es der Pfadname für ein Fenster sein; andernfalls kann es ein beliebiger String sein. Jedes Fenster hat eine zugeordnete Liste von Tags, und ein Binding gilt für ein bestimmtes Fenster, wenn sein Tag zu denen gehört, die für das Fenster angegeben sind. Obwohl der Befehl bindtags verwendet werden kann, um einem Fenster eine beliebige Menge von Binding-Tags zuzuweisen, bieten die Standard-Binding-Tags folgendes Verhalten:

- Wenn ein Tag der Name eines internen Fensters ist, gilt das Binding für dieses Fenster.
- Wenn der Tag der Name einer Widget-Klasse ist, wie z.B. **Button**, gilt das Binding für alle Widgets in dieser Klasse.
- Wenn der Tag der Name eines Toplevel-Fensters ist, gilt das Binding für das Toplevel-Fenster und alle seine internen Fenster.
- Wenn *tag* den Wert **all** hat, gilt das Binding für alle Fenster in der Anwendung.

Tk legt selbst einige Bindings für den Tag **all** an, zum Beispiel für die Tastaturnavigation von Menüs. Unter Windows geben die Bindings für,,, und die Alt-Tasten die Taste an Windows weiter; Windows bedient damit die native Menüleiste und Systemtasten wie Alt-F4. Wer diese Bindings ersetzt, schaltet das ab; erweitern Sie sie, indem Sie dem Skript **+** voranstellen.

## Ereignismuster (Event Patterns)

Das *sequence*-Argument gibt eine Sequenz von einem oder mehreren Ereignismustern an, mit optionalem Leerraum zwischen den Mustern. Jedes Ereignismuster kann eine von drei Formen annehmen. Im einfachsten Fall ist es ein einzelnes druckbares ASCII-Zeichen, wie z.B. **a** oder **[**. Das Zeichen darf kein Leerzeichen oder das Zeichen **<** sein. Diese Form des Musters passt auf ein **Key**-Ereignis für das bestimmte Zeichen. Die zweite Form des Musters ist länger, aber allgemeiner. Sie hat folgende Syntax:

```
<modifier-modifier-type-detail>
```

Das gesamte Ereignismuster ist von spitzen Klammern umgeben. Innerhalb der spitzen Klammern befinden sich null oder mehr Modifizierer, ein Ereignistyp und eine zusätzliche Information (*detail*), die einen bestimmten Button oder Keysym identifiziert. Jedes der Felder kann weggelassen werden, solange mindestens eines von *type* und *detail* vorhanden ist. Die Felder müssen durch Leerzeichen oder Bindestriche getrennt sein.

Die dritte Form des Musters wird verwendet, um ein benutzerdefiniertes, benanntes virtuelles Ereignis anzugeben. Sie hat folgende Syntax:

```
<<name>>
```

Das gesamte virtuelle Ereignismuster ist von doppelten spitzen Klammern umgeben. Innerhalb der spitzen Klammern befindet sich der benutzerdefinierte Name des virtuellen Ereignisses. Modifizierer wie **Shift** oder **Control** können nicht mit einem virtuellen Ereignis kombiniert werden, um es zu modifizieren. Bindings auf ein virtuelles Ereignis können erstellt werden, bevor das virtuelle Ereignis definiert ist, und wenn sich die Definition eines virtuellen Ereignisses dynamisch ändert, reagieren alle an dieses virtuelle Ereignis gebundenen Fenster sofort auf die neue Definition.

Einige Widgets (z.B. menu und text) erzeugen virtuelle Ereignisse, wenn ihr interner Zustand auf bestimmte Weise aktualisiert wird. Bitte sehen Sie die Manualseite für jedes Widget für Details.

### Modifizierer

Modifizierer bestehen aus einem der folgenden Werte:

**Control Mod1**, **M1**, **Command**
**Alt Mod2**, **M2**, **Option**
**Shift Mod3**, **M3**, **Num**
**Lock Mod4**, **M4**, **Fn**
**Extended Mod5**, **M5**
**Button1**, **B1 Meta**, **M**
**Button2**, **B2 Double**
**Button3**, **B3 Triple**
**Button4**, **B4 Quadruple**
**Button5**, **B5**

Wenn mehr als ein Wert aufgeführt ist, getrennt durch Kommas, sind die Werte gleichwertig. Die meisten Modifizierer haben die offensichtliche X-Bedeutung. Zum Beispiel erfordert **Button1**, dass Taste 1 gedrückt ist, wenn das Ereignis auftritt. Damit ein Binding auf ein gegebenes Ereignis passt, müssen die Modifizierer im Ereignis alle diejenigen enthalten, die im Ereignismuster angegeben sind. Ein Ereignis kann auch zusätzliche Modifizierer enthalten, die im Binding nicht angegeben sind. Wenn zum Beispiel Taste 1 gedrückt wird, während die Umschalt- und Steuerungstasten gedrückt sind, passt das Muster auf das Ereignis, jedoch nicht. Wenn keine Modifizierer angegeben sind, kann im Ereignis jede Kombination von Modifizierern vorhanden sein.

**Meta** und **M** beziehen sich auf denjenigen der Modifizierer **M1** bis **M5**, der mit der/den

Meta-Taste(n) auf der Tastatur verknüpft ist (keysyms **Meta_R** und **Meta_L**). Wenn es keine Meta-Tasten gibt oder wenn sie mit keinen Modifizierern verknüpft sind, passen **Meta** und **M** auf kein Ereignis. Entsprechend bezieht sich der Modifizierer **Alt** auf den Modifizierer, der mit der/den Alt-Taste(n) auf der Tastatur verknüpft ist (keysyms **Alt_L** und **Alt_R**).

Die Modifizierer **Double**, **Triple** und **Quadruple** sind eine Bequemlichkeit zur Angabe von Doppelklicks und anderen wiederholten Ereignissen. Sie bewirken, dass ein bestimmtes Ereignismuster 2, 3 oder 4 Mal wiederholt wird, und stellen außerdem eine Zeit- und Abstandsanforderung an die Sequenz: Damit eine Ereignissequenz auf ein **Double**-, **Triple**- oder **Quadruple**-Muster passt, müssen alle Ereignisse zeitlich nahe beieinander auftreten und dazwischen darf keine nennenswerte Mausbewegung stattfinden. Zum Beispiel ist äquivalent zu mit der zusätzlichen Zeit- und Abstandsanforderung.

Die Modifizierer **Command**, **Option**, **Num** und **Fn** sind Äquivalente von **Mod1** bis **Mod4**; sie entsprechen Macintosh-spezifischen Modifizierungstasten.

Der Modifizierer **Extended** ist derzeit Windows-spezifisch. Er tritt bei Ereignissen auf, die mit den Tasten der "erweiterten Tastatur" verknüpft sind. Auf einer US-Tastatur umfassen die erweiterten Tasten die **Alt**- und **Control**-Tasten rechts auf der Tastatur, die Cursortasten im Block links vom Ziffernblock, die **NumLock**-Taste, die **Break**-Taste, die **PrintScreen**-Taste sowie die **/**- und **Enter**-Tasten im Ziffernblock.

## Ereignistypen

Das Feld *type* kann einer der standardmäßigen X-Ereignistypen sein, mit einigen zusätzlichen Abkürzungen. Das Feld *type* akzeptiert auch einige nicht standardmäßige X-Ereignistypen, die hinzugefügt wurden, um die Macintosh- und Windows-Plattformen besser zu unterstützen. Nachfolgend eine Liste aller gültigen Typen; wenn zwei Namen zusammen erscheinen, sind sie Synonyme.

**Activate Destroy Map**
**Button**, **ButtonPress Enter MapRequest**
**ButtonRelease Expose Motion**
**Circulate FocusIn MouseWheel**
**TouchpadScroll CirculateRequest FocusOut**
**Property Colormap Gravity**
**Reparent Configure Key**, **KeyPress**
**ResizeRequest ConfigureRequest KeyRelease**
**Unmap Create Leave Visibility**
**Deactivate**

Die meisten der obigen Ereignisse haben dieselben Felder und Verhaltensweisen wie Ereignisse im X-Windowing-System. Ausführlichere Beschreibungen dieser Ereignisse finden sich in jedem X-Window-Programmierbuch. Einige der Ereignisse sind Erweiterungen des X-Ereignissystems zur Unterstützung von Funktionen, die für die Macintosh- und Windows-Plattformen spezifisch sind. Hier geben wir etwas mehr Detail zu diesen Ereignissen. Dazu gehören:

**Activate, Deactivate**

Diese beiden Ereignisse werden an jedes Unterfenster eines Toplevel gesendet, wenn sich dessen Zustand ändert. Zusätzlich zum Fokus-Fenster kennen die Macintosh- und Windows-Plattformen den Begriff eines aktiven Fensters (das oft, aber nicht zwingend den Fokus hat). Auf dem Macintosh haben Widgets im aktiven Fenster ein anderes

Erscheinungsbild als Widgets in inaktiven Fenstern. Das Ereignis **Activate** wird an alle Unterfenster eines Toplevel gesendet, wenn es vom inaktiven zum aktiven Zustand wechselt. Entsprechend wird das Ereignis **Deactivate** gesendet, wenn sich der Fensterzustand von aktiv zu inaktiv ändert. Für Bindings auf diese Ereignisse gibt es keine nützlichen Prozent-Substitutionen.

**MouseWheel**

Viele moderne Mäuse haben ein Mausrad, mit dem Dokumente ohne Scrollleisten gescrollt werden können. Durch Drehen des Rads erzeugt das System **MouseWheel**-Ereignisse, die die Anwendung zum Scrollen nutzen kann. Das Ereignis wird an das Fenster unter dem Mauszeiger weitergeleitet. Beim Empfang des Ereignisses kann die Substitution **%D** verwendet werden, um das *delta*-Feld des Ereignisses zu erhalten — einen ganzzahligen Wert, der beschreibt, wie weit das Mausrad bewegt wurde. Der kleinste Wert, den das System meldet, ist vom Betriebssystem abhängig. Das Vorzeichen des Werts bestimmt, in welche Richtung das Widget scrollen soll. Positive Werte bedeuten Scrollen nach oben, negative nach unten.

Horizontales Scrollen verwendet **Shift-MouseWheel**-Ereignisse; eine positive **%D**-*delta* -Substitution bedeutet Scrollen nach links, eine negative nach rechts. Horizontale Scroll-Ereignisse entstehen bei Kipp-Rädern mancher Mäuse. Horizontales Scrollen kann auch emuliert werden, indem Shift gedrückt gehalten und vertikal gescrollt wird.

**TouchpadScroll**

Auf einigen Plattformen (derzeit Windows und macOS) gibt es Unterstützung für hochauflösende Scroll-Geräte wie Touchpads. Dies wird über **TouchpadScroll**-Ereignisse bereitgestellt. Diese Ereignisse speichern zwei 16-Bit-Delta-Werte in der Ganzzahl, die durch die Substitution **%D** bereitgestellt wird. Das *X*-Delta steht in den höherwertigen 16 Bits, das *Y* -Delta in den niederwertigen 16 Bits. Diese Werte können mit der Hilfsprozedur tk::PreciseScrollDeltas ausgepackt werden. Zum Beispiel:

```
lassign [tk::PreciseScrollDeltas %D] deltaX deltaY
```

Die Substitution **$#** ist ein Zähler für **TouchpadScroll**-Ereignisse, den Widgets nutzen können, die nur Einheiten-Scrolling unterstützen, um einen Teil der Ereignisse zu ignorieren.

**KeyPress, KeyRelease**

Die Ereignisse **KeyPress** und **KeyRelease** werden erzeugt, wann immer eine Taste gedrückt oder losgelassen wird. **KeyPress** und **KeyRelease**

**Key, KeyRelease**

Die Ereignisse **Key** und **KeyRelease** werden erzeugt, wann immer eine Taste gedrückt oder losgelassen wird. **Key**- und **KeyRelease**-Ereignisse werden an das Fenster gesendet, das derzeit den Tastaturfokus hat.

**Button, ButtonRelease, Motion**

Die Ereignisse **Button** und **ButtonRelease** werden erzeugt, wenn der Benutzer eine Maustaste drückt oder loslässt. **Motion**-Ereignisse werden erzeugt, wann immer der Zeiger bewegt wird. **Button**-, **ButtonRelease**- und **Motion**-Ereignisse werden normalerweise an das Fenster gesendet, das den Zeiger enthält.

Wenn eine Maustaste gedrückt wird, erhält das Fenster mit dem Zeiger automatisch einen temporären Pointer-Grab. Nachfolgende **Button**-, **ButtonRelease**- und **Motion**-Ereignisse werden an dieses Fenster gesendet, unabhängig davon, welches Fenster den Zeiger enthält, bis alle Tasten losgelassen wurden.

**Configure**

Ein **Configure**-Ereignis wird an ein Fenster gesendet, wann immer sich dessen Größe, Position oder Rahmenbreite ändert, und manchmal, wenn sich seine Position in der Stapelreihenfolge geändert hat.

**Map, Unmap**

Die Ereignisse **Map** und **Unmap** werden erzeugt, wann immer sich der Mapping-Zustand eines Fensters ändert.

Fenster werden im unmapped-Zustand erzeugt. Toplevel-Fenster werden gemappt, wenn sie in den Zustand **normal** wechseln, und unmapped im Zustand **withdrawn** und **iconic**. Andere Fenster werden gemappt, wenn sie unter Kontrolle eines Geometry-Managers (z.B. pack oder grid) gestellt werden.

Ein Fenster ist nur dann *sichtbar* (*viewable*), wenn es und alle seine Vorfahren gemappt sind. Geometry-Manager mappen ihre Kinder typischerweise erst, wenn sie selbst gemappt wurden, und unmapen alle Kinder, wenn sie unmapped werden; daher zeigen **Map**- und **Unmap**-Ereignisse in Tk an, ob ein Fenster sichtbar ist oder nicht.

**Visibility**

Ein Fenster gilt als *verdeckt* (*obscured*), wenn ein anderes Fenster darüber in der Stapelreihenfolge es vollständig oder teilweise überlappt. **Visibility**-Ereignisse werden erzeugt, wann immer sich der Verdeckungszustand eines Fensters ändert; das Feld *state* (**%s**) gibt den neuen Zustand an.

**Expose**

Ein **Expose**-Ereignis wird erzeugt, wann immer ein Fenster ganz oder teilweise neu gezeichnet werden muss (z.B. wenn ein Fenster erstmals gemappt wird oder wieder sichtbar wird). Client-Anwendungen müssen **Expose**-Ereignisse normalerweise nicht behandeln, da Tk sie intern verarbeitet.

**Destroy**

Ein **Destroy**-Ereignis wird an ein Fenster geliefert, wenn es zerstört wird.

Wenn das **Destroy**-Ereignis an ein Widget geliefert wird, befindet es sich in einem "halb-toten" Zustand: Das Widget existiert noch, aber Operationen, die es betreffen, können ungültige Ergebnisse liefern oder einen Fehler zurückgeben.

**FocusIn, FocusOut**

Die Ereignisse **FocusIn** und **FocusOut** werden erzeugt, wann immer sich der Tastaturfokus ändert. Ein **FocusOut**-Ereignis wird an das alte Fokus-Fenster gesendet, ein **FocusIn**-Ereignis an das neue.

Wenn die alten und neuen Fokus-Fenster keinen gemeinsamen Parent haben, werden "virtuelle Crossing"-Fokus-Ereignisse an die Zwischenfenster in der Hierarchie gesendet. Ein **FocusIn**-Ereignis zeigt also an, dass das Zielfenster oder einer seiner Nachfahren den Fokus erhalten hat; ein **FocusOut**-Ereignis zeigt an, dass der Fokus auf ein Fenster außerhalb der

Hierarchie des Zielfensters gewechselt ist.

Der Tastaturfokus kann explizit durch einen Aufruf von focus oder implizit durch den Window-Manager geändert werden.

**Enter, Leave**

Ein **Enter**-Ereignis wird an ein Fenster gesendet, wenn der Zeiger dieses Fenster betritt; ein **Leave**-Ereignis, wenn er es verlässt.

Wenn ein Pointer-Grab aktiv ist, werden **Enter**- und **Leave**-Ereignisse nur an das Fenster geliefert, das den Grab besitzt.

Wenn der Zeiger zwischen zwei Fenstern wechselt, werden **Enter**- und **Leave**-"virtuelle Crossing"-Ereignisse auf dieselbe Weise an Zwischenfenster in der Hierarchie gesendet wie bei **FocusIn** und **FocusOut**.

**Property**

Ein **Property**-Ereignis wird an ein Fenster gesendet, wann immer eine diesem Fenster gehörende X-Property geändert oder gelöscht wird. **Property**-Ereignisse werden Tk-Anwendungen normalerweise nicht zugestellt, da sie vom Tk-Kern verarbeitet werden.

**Colormap**

Ein **Colormap**-Ereignis wird erzeugt, wann immer die mit einem Fenster verknüpfte Colormap geändert, installiert oder deinstalliert wurde.

Widgets können eine private Colormap durch Angabe der Option **-colormap** zugewiesen bekommen; der Window-Manager ist für die Installation und Deinstallation von Colormaps verantwortlich.

Tk liefert für diesen Ereignistyp keine nützlichen Details.

**MapRequest, CirculateRequest, ResizeRequest, ConfigureRequest, Create**

Diese Ereignisse werden Tk-Anwendungen normalerweise nicht zugestellt. Sie sind der Vollständigkeit halber enthalten, damit X11-Window-Manager in Tk geschrieben werden können. (Diese Ereignisse werden nur zugestellt, wenn ein Client **SubstructureRedirectMask** für ein Fenster gewählt hat; der Tk-Kern verwendet diese Maske nicht.)

**Gravity, Reparent, Circulate**

Die Ereignisse **Gravity** und **Reparent** werden Tk-Anwendungen normalerweise nicht zugestellt. Sie sind der Vollständigkeit halber enthalten.

Ein **Circulate**-Ereignis zeigt an, dass das Fenster infolge einer **XCirculateSubwindows** -Protokollanfrage an den Anfang oder das Ende der Stapelreihenfolge gerückt ist. Die Stapelreihenfolge kann sich auch aus anderen Gründen ändern, die kein **Circulate**-Ereignis erzeugen, und Tk verwendet **XCirculateSubwindows()** intern nicht. Dieser Ereignistyp ist nur der Vollständigkeit halber enthalten; es gibt keine zuverlässige Möglichkeit, Änderungen der Position eines Fensters in der Stapelreihenfolge zu verfolgen.

## Ereignisdetails

Der letzte Teil einer langen Ereignisspezifikation ist *detail*. Bei einem **Button**- oder

**ButtonRelease**-Ereignis ist es die Nummer einer Taste (1–9). Wenn eine Tastennummer angegeben ist, passt nur ein Ereignis auf genau diese Taste; wenn keine Tastennummer angegeben ist, passt ein Ereignis auf jede Taste. Hinweis: Eine bestimmte Tastennummer anzugeben unterscheidet sich von der Angabe eines Button-Modifizierers; im ersten Fall bezieht es sich auf eine Taste, die gedrückt oder losgelassen wird, im zweiten auf eine andere Taste, die bereits gedrückt ist, wenn das passende Ereignis auftritt. Wenn eine Tastennummer angegeben ist, kann *type* weggelassen werden; es wird standardmäßig **Button** angenommen. Zum Beispiel ist der Spezifizierer **<1>** äquivalent zu.

Wenn der Ereignistyp **Key** oder **KeyRelease** ist, kann *detail* in Form eines X-Keysym angegeben werden. Keysym sind textuelle Spezifikationen für bestimmte Tasten auf der Tastatur; sie umfassen alle alphanumerischen ASCII-Zeichen (z.B. ist "a" der Keysym für das ASCII-Zeichen "a"), Beschreibungen für nicht-alphanumerische Zeichen ("comma" ist der Keysym für das Kommazeichen) sowie Beschreibungen für alle nicht-ASCII-Tasten (z.B. ist "Shift_L" der Keysym für die linke Umschalttaste und "F1" der Keysym für die F1-Funktionstaste, falls vorhanden). Die vollständige Keysym-Liste wird hier nicht aufgeführt; sie ist in anderer X-Dokumentation verfügbar und kann von System zu System variieren. Bei Bedarf kann die unten beschriebene Notation **%K** verwendet werden, um den Keysym-Namen für eine bestimmte Taste auszugeben. Wenn ein Keysym als *detail* angegeben ist, kann das Feld *type* weggelassen werden; es wird standardmäßig **Key** angenommen. Zum Beispiel ist äquivalent zu.

Unter Windows ist der Keysym des Ereignisses der lateinische Buchstabe oder die Ziffer, die Windows der Taste zuordnet (meist neben dem nicht-lateinischen Zeichen aufgedruckt), wenn **Control** oder **Alt** gedrückt ist und das Zeichen, das die aktuelle Tastaturbelegung einer Buchstaben- oder Zifferntaste zuweist, kein lateinischer Buchstabe und keine Ziffer ist (zum Beispiel bei kyrillischer oder griechischer Belegung). Bindings wie oder funktionieren damit unabhängig von der Tastaturbelegung, so wie Tastaturkürzel in anderen Windows-Anwendungen. Die Substitution **%A** bleibt davon unberührt und liefert weiter das Zeichen der aktuellen Belegung.

Unter X11 ist der Keysym des Ereignisses der, den die aktuelle Tastaturbelegung (XKB-Gruppe) der Taste zuweist. Ist **Control** oder **Alt** gedrückt und dieser Keysym kein Latin-1-Zeichen (zum Beispiel bei kyrillischer oder griechischer Belegung), wird stattdessen der Keysym derselben Taste in einer lateinischen Belegung verwendet, sofern die Tastatur eine hat. Bindings wie oder funktionieren damit unabhängig von der Tastaturbelegung, so wie Tastaturkürzel in anderen X11-Toolkits. Die Substitution **%A** bleibt davon unberührt.

## Binding-Skripte und Substitutionen

Das *script*-Argument von **bind** ist ein Tcl-Script, das "Binding-Script" genannt wird und ausgeführt wird, wann immer die gegebene Ereignissequenz auftritt. *Command* wird im selben Interpreter ausgeführt, in dem der Befehl **bind** ausgeführt wurde, und auf globaler Ebene (nur globale Variablen sind zugänglich). Enthält *script* **%**-Zeichen, wird das Script nicht direkt ausgeführt. Stattdessen wird ein neues Script erzeugt, indem jedes **%** und das darauf folgende Zeichen durch Informationen aus dem aktuellen Ereignis ersetzt werden. Die Ersetzung hängt vom Zeichen nach dem **%** ab, wie in der folgenden Liste definiert. Sofern nicht anders angegeben, ist die Ersetzungszeichenkette der Dezimalwert des angegebenen Feldes aus dem aktuellen Ereignis. Einige Substitutionen gelten nur für bestimmte Ereignistypen; werden sie für andere Ereignistypen verwendet, ist der substituierte Wert undefiniert.

**%%**

Wird durch ein einzelnes Prozentzeichen ersetzt.

**%#**

Die Nummer der letzten vom Server verarbeiteten Client-Anfrage (das Feld *serial* aus dem Ereignis). Gültig für alle Ereignistypen.

**%a**

Das Feld *above* aus dem Ereignis, formatiert als hexadezimale Zahl. Nur gültig für **Configure** -Ereignisse. Gibt das Geschwisterfenster unmittelbar unter dem empfangenden Fenster in der Stapelreihenfolge an, oder **0**, wenn das empfangende Fenster unten ist.

**%b**

Die Nummer der gedrückten oder losgelassenen Taste. Nur gültig für **Button**- und **ButtonRelease**-Ereignisse.

**%c**

Das Feld *count* aus dem Ereignis. Nur gültig für **Expose**-Ereignisse. Zeigt an, dass *count* ausstehende **Expose**-Ereignisse vorliegen, die dem Fenster noch nicht zugestellt wurden.

**%d**

Das Feld *detail* oder *user_data* aus dem Ereignis. **%d** wird durch eine Zeichenkette ersetzt, die das Detail identifiziert. Für **Enter**-, **Leave**-, **FocusIn**- und **FocusOut**-Ereignisse ist die Zeichenkette eine der folgenden:

**NotifyAncestor NotifyNonlinearVirtual**
**NotifyDetailNone NotifyPointer**
**NotifyInferior NotifyPointerRoot**
**NotifyNonlinear NotifyVirtual**

Für **ConfigureRequest**-Ereignisse ist die Zeichenkette eine von:

**Above Opposite**
**Below None**
**BottomIf TopIf**

Bei virtuellen Ereignissen ist die Zeichenkette der Wert, der beim Erzeugen des Ereignisses (typischerweise mit event generate) im Feld *user_data* gespeichert wurde, oder der leere String, wenn das Feld NULL ist. Virtuelle Ereignisse, die Tastensequenz-Drücke entsprechen (siehe event add), setzen *user_data* auf NULL. Für andere Ereignisse ist die substituierte Zeichenkette undefiniert.

**%f**

Das Feld *focus* aus dem Ereignis (**0** oder **1**). Nur gültig für **Enter**- und **Leave**-Ereignisse. **1**, wenn das empfangende Fenster das Fokus-Fenster oder ein Nachfahre des Fokus-Fensters ist, sonst **0**.

**%h**

Das Feld *height* aus dem Ereignis. Gültig für **Configure**-, **ConfigureRequest**-, **Create**-, **ResizeRequest**- und **Expose**-Ereignisse. Gibt die neue oder angeforderte Höhe des Fensters an.

**%i**

Das Feld *window* aus dem Ereignis, dargestellt als hexadezimale Ganzzahl. Gültig für alle Ereignistypen.

**%k**

Das Feld *keycode* aus dem Ereignis. Nur gültig für **Key**- und **KeyRelease**-Ereignisse.

**%m**

Das Feld *mode* aus dem Ereignis. Die substituierte Zeichenkette ist eine von **NotifyNormal**, **NotifyGrab**, **NotifyUngrab** oder **NotifyWhileGrabbed**. Nur gültig für **Enter**-, **FocusIn**-, **FocusOut**- und **Leave**-Ereignisse.

**%o**

Das Feld *override_redirect* aus dem Ereignis. Nur gültig für **Map**-, **Reparent**- und **Configure** -Ereignisse.

**%p**

Das Feld *place* aus dem Ereignis, substituiert als eine der Zeichenketten **PlaceOnTop** oder **PlaceOnBottom**. Nur gültig für **Circulate**- und **CirculateRequest**-Ereignisse.

**%s**

Das Feld *state* aus dem Ereignis. Für **Button**-, **ButtonRelease**-, **Enter**-, **Key**-, **KeyRelease**-, **Leave**- und **Motion**-Ereignisse wird eine Dezimalzeichenkette substituiert. Für **Visibility** eine von **VisibilityUnobscured**, **VisibilityPartiallyObscured** und **VisibilityFullyObscured**. Für **Property**-Ereignisse entweder **NewValue** (Property wurde erstellt oder geändert) oder **Delete** (Property wurde entfernt).

**%t**

Das Feld *time* aus dem Ereignis. Dies ist der X-Server-Zeitstempel (typischerweise die Zeit seit dem letzten Server-Reset) in Millisekunden, als das Ereignis auftrat. Gültig für die meisten Ereignisse.

**%w**

Das Feld *width* aus dem Ereignis. Gibt die neue oder angeforderte Breite des Fensters an. Nur gültig für **Configure**-, **ConfigureRequest**-, **Create**-, **ResizeRequest**- und **Expose** -Ereignisse.

**%x, %y**

Die Felder *x* und *y* aus dem Ereignis. Für **Button**-, **ButtonRelease**-, **Motion**-, **Key**-, **KeyRelease**- und **MouseWheel**-Ereignisse geben **%x** und **%y** die Position des Mauszeigers relativ zum empfangenden Fenster an. Bei Tastenereignissen auf dem Macintosh sind dies die Mauskoordinaten im Moment, in dem ein X11-KeyEvent an Tk gesendet wird, was etwas später als der physische Druck oder das Loslassen sein kann. Für **Enter**- und **Leave** -Ereignisse die Position, an der der Mauszeiger das Fenster gekreuzt hat, relativ zum empfangenden Fenster. Für **Configure**- und **Create**-Anfragen die *x*- und *y*-Koordinaten des Fensters relativ zum Parent-Fenster.

**%A**

Substitutiert das dem Ereignis entsprechende UNICODE-Zeichen oder den leeren String, wenn das Ereignis keinem UNICODE-Zeichen entspricht (z.B. wenn die Umschalttaste gedrückt wurde). Unter X11 übernimmt **XmbLookupString** (oder **XLookupString**, wenn Input-Method-Unterstützung aus ist) die Übersetzung vom Ereignis zum UNICODE-Zeichen. Unter X11 nur gültig für **Key**-Ereignisse. Unter Windows und macOS/aqua nur gültig für **Key**- und **KeyRelease**-Ereignisse.

**%B**

Das Feld *border_width* aus dem Ereignis. Nur gültig für **Configure**-, **ConfigureRequest**- und **Create**-Ereignisse.

**%D**
: Meldet den *delta*-Wert eines **MouseWheel**-Ereignisses. Der *delta*-Wert gibt die Dreheinheiten des Mausrads an. Das Vorzeichen gibt die Scroll-Richtung an.

**%E**
: Das Feld *send_event* aus dem Ereignis. Gültig für alle Ereignistypen. **0** bedeutet ein "normales" Ereignis, **1** ein "synthetisches" Ereignis, erzeugt durch **SendEvent**.

**%K**
: Der dem Ereignis entsprechende Keysym, substituiert als Textzeichenkette. Nur gültig für **Key** - und **KeyRelease**-Ereignisse.

**%M**
: Die Anzahl der bisher für das Ereignis passenden script-basierten Binding-Muster. Gültig für alle Ereignistypen.

**%N**
: Der dem Ereignis entsprechende Keysym, substituiert als Dezimalzahl. Nur gültig für **Key**- und **KeyRelease**-Ereignisse.

**%P**
: Der Name der aktualisierten oder gelöschten Property (kann mit winfo atom in ein XAtom umgewandelt werden). Nur gültig für **Property**-Ereignisse.

**%R**
: Die *root*-Fensterkennung aus dem Ereignis. Nur gültig für Ereignisse mit einem *root*-Feld.

**%S**
: Die *subwindow*-Fensterkennung aus dem Ereignis, formatiert als hexadezimale Zahl. Nur gültig für Ereignisse mit einem *subwindow*-Feld.

**%T**
: Das Feld *type* aus dem Ereignis. Gültig für alle Ereignistypen.

**%W**
: Der Pfadname des Fensters, an das das Ereignis gemeldet wurde (das Feld *window* aus dem Ereignis). Gültig für alle Ereignistypen.

**%X, %Y**
: Die Felder *x_root* und *y_root* aus dem Ereignis. Bei einem Virtual-Root-Window-Manager sind die substituierten Werte die entsprechenden x- und y-Koordinaten im virtuellen Root. Nur gültig für **Button**-, **ButtonRelease**-, **Enter**-, **Key**-, **KeyRelease**-, **Leave**- und **Motion** -Ereignisse. Dieselbe Bedeutung wie **%x** und **%y**, jedoch relativ zum (virtuellen) Root-Fenster.

Die Ersetzungszeichenkette für eine %-Ersetzung wird als gültiges Tcl-Listenelement formatiert. Leerzeichen oder Sonderzeichen wie **$** und **{** können mit Backslashes versehen werden. So wird sichergestellt, dass die Zeichenkette beim Auswerten des Binding-Scripts durch den Tcl-Parser geht. Die meisten Ersetzungen sind Zahlen oder wohldefinierte Zeichenketten wie **Above**; für diese ist keine spezielle Formatierung nötig. Am häufigsten tritt Neuformatierung bei der Substitution **%A** auf. Wenn *script* zum Beispiel

```
insert %A
```

ist und die eingegebene Taste eine öffnende eckige Klammer ist, wird tatsächlich ausgeführt:

```
insert \[
```

Dadurch erhält **insert** die ursprüngliche Ersetzungszeichenkette (öffnende eckige Klammer) als erstes Argument. Ohne den zusätzlichen Backslash könnte Tcl das Script nicht korrekt parsen.

## Mehrere Treffer

Es ist möglich, dass mehrere Bindings auf ein gegebenes X-Ereignis passen. Sind die Bindings verschiedenen *tag*-Werten zugeordnet, wird jedes Binding der Reihe nach ausgeführt. Standardmäßig wird zuerst ein Binding für das Widget ausgeführt, gefolgt von einem Klassen-Binding, einem Binding für sein Toplevel und einem **all**-Binding. Mit dem Befehl bindtags kann diese Reihenfolge für ein bestimmtes Fenster geändert oder zusätzliche Binding-Tags zugeordnet werden.

Die Befehle continue und break können innerhalb eines Binding-Scripts verwendet werden, um die Verarbeitung passender Scripts zu steuern. Wird continue in einem Binding-Script aufgerufen, wird dieses Binding-Script einschließlich aller mit "+" angehängten Scripts beendet, Tk verarbeitet jedoch weiter Binding-Scripts, die anderen *tag*-Werten zugeordnet sind. Wird break in einem Binding-Script aufgerufen, endet dieses Script und es werden keine weiteren Scripts für das Ereignis aufgerufen.

In einem vom Binding-Script aufgerufenen Script kann return **-code ok** die Verarbeitung fortsetzen (einschließlich mit "+" angehängter Scripts), oder return **-code break** die Verarbeitung aller anderen Binding-Scripts stoppen.

Passen mehrere Bindings auf ein bestimmtes Ereignis und haben dasselbe *tag*, wird das spezifischste Binding gewählt und dessen Script ausgewertet. Zur Bestimmung, welche von mehreren passenden Sequenzen spezifischer ist, werden nacheinander folgende Tests angewendet:

**(a)**

Ein Ereignismuster, das eine bestimmte Maustaste oder Taste angibt, ist spezifischer als eines ohne solche Angabe;

**(b)**

Eine Sequenz mit den höchststufigsten Mustern (im Sinne der höchsten Wiederholungsanzahl) ist spezifischer als eine Sequenz mit weniger höchststufigen Mustern;

**(c)**

Sind die Modifizierer eines Musters eine Teilmenge der Modifizierer eines anderen Musters, ist das Muster mit mehr Modifizierern spezifischer;

**(d)**

Ein virtuelles Ereignis, dessen physisches Muster auf die Sequenz passt, ist weniger spezifisch als dasselbe physische Muster ohne Zuordnung zu einem virtuellen Ereignis;

**(e)**

Passt eine Sequenz auf zwei oder mehr virtuelle Ereignisse, wird eines der virtuellen Ereignisse gewählt, die Reihenfolge ist jedoch undefiniert.

Enthalten die passenden Sequenzen mehr als ein Ereignis, werden die Tests (c)–(e) in der Reihenfolge vom jüngsten zum ältesten Ereignis in den Sequenzen angewendet. Bestimmen diese Tests keinen Sieger, gewinnt die zuletzt registrierte Sequenz.

Gibt es zwei (oder mehr) virtuelle Ereignisse, die durch dieselbe Sequenz ausgelöst werden, und sind beide an dasselbe Fenster-Tag gebunden, wird nur eines der virtuellen Ereignisse ausgelöst, und zwar zufällig gewählt:

```
event add <<Paste>> <Control-y>
event add <<Paste>> <Button-2>
event add <<Scroll>> <Button-2>
bind Entry <<Paste>> {puts Paste}
bind Entry <<Scroll>> {puts Scroll}
```

Tippt der Benutzer Control-y, wird das Binding **<>** aufgerufen; drückt er Taste 2, wird eines der Bindings **<>** oder **<>** aufgerufen — welches genau, ist undefiniert.

Passt ein X-Ereignis auf keines der vorhandenen Bindings, wird es ignoriert. Ein ungebundenes Ereignis gilt nicht als Fehler.

## Mehrere-Ereignis-Sequenzen und ignorierte Ereignisse

Enthält eine in einem **bind**-Befehl angegebene *sequence* mehr als ein Ereignismuster, wird ihr Script ausgeführt, wann immer die jüngsten Ereignisse (bis einschließlich des aktuellen) auf die gegebene Sequenz passen. Das bedeutet z.B., dass bei wiederholtem Klick auf Taste 1 die Sequenz auf jeden Tastendruck außer den ersten passt. Treten in der Mitte einer Ereignissequenz überflüssige Ereignisse auf, die eine Übereinstimmung verhindern würden, werden diese ignoriert, es sei denn, es sind **Key**- oder **Button**-Ereignisse. Zum Beispiel passt auf eine Folge von Drücken von Taste 1, auch wenn zwischen den **Button**-Ereignissen **ButtonRelease** -Ereignisse (und ggf. **Motion**-Ereignisse) liegen. Außerdem kann ein **Key**-Ereignis von beliebig vielen anderen **Key**-Ereignissen für Modifizierungstasten vorangehen, ohne dass diese eine Übereinstimmung verhindern. Die Ereignissequenz **aB** passt z.B. auf Drücken von **a**, Loslassen von **a**, Drücken von **Shift** und Drücken von **b**: Das Drücken von **Shift** wird ignoriert, weil es eine Modifizierungstaste ist. Treten schließlich mehrere **Motion**-Ereignisse hintereinander auf, wird nur das letzte zum Abgleich von Binding-Sequenzen verwendet.

## Fehler

Tritt beim Ausführen des Scripts für ein Binding ein Fehler auf, wird der Fehler über den Mechanismus bgerror gemeldet. Der Befehl bgerror wird auf globaler Ebene ausgeführt (außerhalb des Kontexts einer Tcl-Prozedur).

## Beispiele

Beim Doppelklick mit der Maus soll eine Zeichenkette mit der Mausbewegung ausgegeben werden:

```
bind . <Double-1> {
    puts "hi from (%x,%y)"
}
```

Eine kleine GUI, die den Keysym-Namen der zuletzt gedrückten Taste anzeigt:

```
set keysym "Press any key"
pack [label .l -textvariable keysym -padx 2m -pady 1m]
bind . <Key> {
    set keysym "You pressed %K"
}
```

*Copyright (c) 1990 The Regents of the University of California.*

*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 1998 Scriptics Corporation.*

# bindtags

*Stichwörter:* bindtags, binding, event, tag

## Name

bindtags - Legt anwendbare Bindings und ihre Auswertungsreihenfolge fest

## Übersicht

**bindtags** *window* ?*tagList*?

## Beschreibung

Wird ein Binding mit dem Befehl bind erzeugt, wird es entweder einem bestimmten Fenster wie **.a.b.c**, einem Klassennamen wie **Button**, dem Schlüsselwort **all** oder einer anderen Zeichenkette zugeordnet. All diese Formen heißen *Binding-Tags*. Jedes Fenster enthält eine Liste von Binding-Tags, die bestimmen, wie Ereignisse für das Fenster verarbeitet werden. Tritt ein Ereignis in einem Fenster ein, wird es nacheinander auf jedes Tag des Fensters angewendet: Für jedes Tag wird das spezifischste Binding ausgeführt, das zum Tag und Ereignis passt. Details zum Abgleich siehe Befehl bind.

Standardmäßig hat jedes Fenster vier Binding-Tags in dieser Reihenfolge: Fenstername, Klassenname des Fensters, Name des nächsten **toplevel**-Vorfahren und **all**. **toplevel**-Fenster haben standardmäßig nur drei Tags, da der Toplevel-Name mit dem Fensternamen übereinstimmt. Mit **bindtags** können die Binding-Tags eines Fensters gelesen und geändert werden.

Wird **bindtags** nur mit einem Argument aufgerufen, gibt der Befehl die aktuelle Binding-Tag-Liste von *window* als Liste zurück. Ist das Argument *tagList* angegeben, muss es eine gültige Liste sein; die Tags von *window* werden auf die Listenelemente gesetzt. Die Elemente von *tagList* dürfen beliebige Zeichenketten sein; beginnt ein Tag mit einem Punkt, wird er als Fenstername behandelt — existiert zum Zeitpunkt der Ereignisverarbeitung kein Fenster mit diesem Namen, wird das Tag für dieses Ereignis ignoriert. Die Reihenfolge der Elemente in *tagList* legt fest, in welcher Reihenfolge Binding-Skripte bei Ereignissen ausgeführt werden. Beispiel:

```
bindtags .b {all . Button .b}
```

kehrt die Auswertungsreihenfolge für einen Button **.b** um: zuerst **all**-Bindings, dann Bindings des Toplevels von **.b** ("."), dann Klassen-Bindings, zuletzt Bindings für **.b**. Ist *tagList* eine leere Liste, werden die Binding-Tags von *window* auf den oben beschriebenen Standard zurückgesetzt.

Mit **bindtags** lassen sich beliebige zusätzliche Binding-Tags einführen oder Standard-Tags entfernen. Beispiel:

```
bindtags .b {.b TrickyButton . all}
```

ersetzt den **Button**-Tag von **.b** durch **TrickyButton**. Die Standard-Widget-Bindings für Buttons (Tag **Button**) gelten dann nicht mehr für **.b**; Bindings für **TrickyButton** (z. B. neues Button-Verhalten) schon.

### Beispiele

Bei verschachtelten frame-Widgets sollen Ereignisse eines button-Widgets auch an alle Widgets bis zum aktuellen toplevel weitergegeben werden (im Gegensatz zum Tk-Standard, der Zwischenfenster auslässt) — praktisch für Beschleuniger, die nur für einen Teil des Fensters aktiv sind. Hilfsprozedur:

```
proc setupBindtagsForTreeDelivery {widget} {
    set tags [list $widget [winfo class $widget]]
    set w $widget
    set t [winfo toplevel $w]
    while {$w ne $t} {
        set w [winfo parent $w]
        lappend tags $w
    }
    lappend tags all
    bindtags $widget $tags
}
```

*Copyright (c) 1990 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## bitmap

*Stichwörter:* bitmap, image

### Name

bitmap - Bilder mit zwei Farben

### Übersicht

**image create bitmap** ?*name*? ?*options*?

*imageName* **cget** *option*
*imageName* **configure** ?*option*? ?*value option value* ...?

### Beschreibung

Ein Bitmap ist ein Bild, dessen Pixel entweder eine von zwei Farben anzeigen oder transparent sein können. Ein Bitmap-Bild wird durch vier Dinge definiert: eine Hintergrundfarbe, eine Vordergrundfarbe und zwei Bitmaps, *source* und *mask* genannt. Jedes Bitmap gibt 0/1-Werte für ein rechteckiges Pixelraster an; beide Bitmaps müssen dieselben Abmessungen haben. Ist die Maske an einer Pixelposition null, zeigt das Bild nichts und wirkt transparent. An allen anderen Pixeln zeigt das Bild die Vordergrundfarbe, wenn die Quelldaten eins sind, und die Hintergrundfarbe, wenn die Quelldaten null sind.

### Erstellen von Bitmaps

Wie alle Bilder werden Bitmaps mit dem Befehl image create erzeugt. Bitmaps unterstützen folgende *Optionen*:

**-background *color***

Legt eine Hintergrundfarbe für das Bild auf eine der von Tk akzeptierten Standardarten fest. Ist diese Option auf eine leere Zeichenkette gesetzt, sind die Hintergrundpixel transparent.

Der Effekt entsteht, indem das Quell-Bitmap als Masken-Bitmap verwendet wird; Optionen **-maskdata** und **-maskfile** werden ignoriert.

**-data *string***

Legt den Inhalt des Quell-Bitmaps als Zeichenkette fest. Die Zeichenkette muss dem X11-Bitmap-Format entsprechen (z. B. wie vom Programm **bitmap** erzeugt). Sind **-data** und **-file** gesetzt, hat **-data** Vorrang.

**-file *name***

*name* ist der Name einer Datei, deren Inhalt das Quell-Bitmap definiert. Die Datei muss dem X11-Bitmap-Format entsprechen (z. B. wie vom Programm **bitmap** erzeugt).

**-foreground *color***

Legt eine Vordergrundfarbe für das Bild auf eine der von Tk akzeptierten Standardarten fest.

**-maskdata *string***

Legt den Inhalt der Maske als Zeichenkette fest. Die Zeichenkette muss dem X11-Bitmap-Format entsprechen (z. B. wie vom Programm **bitmap** erzeugt). Sind **-maskdata** und **-maskfile** gesetzt, hat **-maskdata** Vorrang.

**-maskfile *name***

*name* ist der Name einer Datei, deren Inhalt die Maske definiert. Die Datei muss dem X11-Bitmap-Format entsprechen (z. B. wie vom Programm **bitmap** erzeugt).

## Bild-Befehl

Wird ein Bitmap-Bild erzeugt, legt Tk auch einen neuen Befehl an, dessen Name mit dem des Bildes übereinstimmt. Mit diesem Befehl können verschiedene Operationen auf dem Bild ausgeführt werden. Er hat die allgemeine Form:

```
imageName option ?arg ...?
```

*Option* und die *arg*-Argumente bestimmen das genaue Verhalten des Befehls. Für Bitmap-Bilder sind folgende Befehle möglich:

***imageName* cget *option***

Gibt den aktuellen Wert der Konfigurationsoption *option* zurück. *Option* kann einen der von image create **bitmap** akzeptierten Werte haben.

***imageName* configure ?*option*? ?*value option value* ...?**

Fragt die Konfigurationsoptionen des Bildes ab oder ändert sie. Ist keine *option* angegeben, gibt der Befehl eine Liste aller verfügbaren Optionen für *imageName* zurück (Format siehe Tk_ConfigureInfo). Ist *option* ohne *value* angegeben, liefert der Befehl eine Liste zur benannten Option (identisch mit der entsprechenden Teilliste ohne *option*). Sind eine oder mehrere *option*-*value*-Paare angegeben, setzt der Befehl die Option(en) auf die angegebenen Werte und gibt eine leere Zeichenkette zurück. *Option* kann einen der von image create **bitmap** akzeptierten Werte haben.

*Copyright (c) 1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# busy

*Stichwörter:* busy, keyboard events, pointer events, window

## Name

busy - Tk-Widgets vorübergehend als beschäftigt markieren und Benutzerinteraktionen blockieren

## Übersicht

**tk busy** *window* ?*options*?
**tk busy busywindow** *window*
**tk busy hold** *window* ?*options*?
**tk busy configure** *window* ?*option value* ...?
**tk busy forget** *window* ?*window* ...?
**tk busy current** ?*pattern*?
**tk busy status** *window*

## Beschreibung

Der Befehl tk busy bietet ein einfaches Mittel, Mauszeiger-Ereignisse von Tk-Widgets zu blockieren und dabei den Cursor des Widgets durch einen konfigurierbaren Busy-Cursor zu ersetzen. Dieser Befehl verhindert nicht, dass Tastaturereignisse an die als beschäftigt markierten Widgets gesendet werden.

## Einleitung

In Anwendungen gibt es oft Situationen, in denen Sie die Aktionen des Benutzers vorübergehend einschränken möchten. Eine Anwendung könnte beispielsweise einen "Run"-Button haben, dessen Drücken eine Verarbeitung auslöst. Während die Anwendung jedoch mit der Verarbeitung beschäftigt ist, möchten Sie wahrscheinlich nicht, dass der Benutzer den "Run"-Button erneut anklicken kann. Sie möchten den Benutzer möglicherweise auch von anderen Aufgaben abhalten, etwa dem Anklicken eines "Print"-Buttons.

Der Befehl tk busy ermöglicht es, Tk-Widgets als beschäftigt zu markieren. Das bedeutet, dass Benutzerinteraktionen wie Button-Klicks, Mausbewegungen, Tastatureingaben usw. vom Widget ignoriert werden. Sie können einen speziellen Cursor (z. B. eine Uhr) setzen, der den normalen Cursor des Widgets überschreibt und Rückmeldung gibt, dass die Anwendung (das Widget) vorübergehend beschäftigt ist.

Wenn ein Widget als beschäftigt markiert ist, ignorieren das Widget und alle seine Nachkommen Zeigerereignisse. Es ist einfach, ein ganzes Panel von Widgets als beschäftigt zu markieren: Machen Sie einfach das Toplevel-Widget (z. B. ".") beschäftigt. Das ist einfacher und weitaus effizienter, als die Widget-Hierarchie rekursiv zu durchlaufen, jedes Widget zu deaktivieren und seinen Cursor neu zu konfigurieren.

Oft kann der Befehl tk busy anstelle des Tk-Befehls grab verwendet werden. Im Gegensatz zu grab, das alle Benutzerinteraktionen auf ein Widget beschränkt, können mit tk busy mehrere Widgets gleichzeitig aktiv sein (z. B. ein "Cancel"-Dialog und ein "Help"-Button).

## Beispiel

Sie können mehrere Widgets als beschäftigt markieren, indem Sie einfach ihr Vorfahren-Widget mit der Operation **hold** als beschäftigt markieren.

```
frame .top
button .top.button; canvas .top.canvas
pack .top.button .top.canvas
pack .top
# . . .
tk busy hold .top
update
```

Alle Widgets innerhalb von **.top** (einschließlich **.top**) sind jetzt beschäftigt. update stellt sicher, dass der Befehl tk busy wirksam wird, bevor andere Benutzerereignisse auftreten können.

Wenn die Anwendung nicht mehr mit der Verarbeitung beschäftigt ist, können Sie Benutzerinteraktionen wieder zulassen und mit der Operation **forget** alle von ihr belegten Ressourcen freigeben.

```
tk busy forget .top
```

Das Busy-Fenster hat einen konfigurierbaren Cursor. Sie können den Busy-Cursor mit der Operation **configure** ändern.

```
tk busy configure .top -cursor "watch"
```

Das Zerstören des Widgets räumt auch alle vom Befehl tk busy belegten Ressourcen auf.

## Operationen

Für den Befehl tk busy stehen folgende Operationen zur Verfügung:

**tk busy *window* ?*option value*?...**

Kurzform für den Befehl **tk busy hold**.

**tk busy busywindow *window***

Gibt den Pfadnamen des Busy-Fensters zurück (d. h. des transparenten Fensters, das das als beschäftigt erscheinende Widget abschirmt), das der Befehl **tk busy hold** für *window* erzeugt hat, oder die leere Zeichenkette, wenn *window* nicht beschäftigt ist.

**tk busy cget *window option***

Fragt die Konfigurationsoptionen des Befehls tk busy für *window* ab. *Window* muss der Pfadname eines Widgets sein, das zuvor durch die Operation **hold** als beschäftigt markiert wurde. Der Befehl gibt den aktuellen Wert der angegebenen *option* zurück. *Option* kann einen der von der Operation **hold** akzeptierten Werte haben.

**tk busy configure *window* ?*option value* ...?**

Fragt die Konfigurationsoptionen des Befehls tk busy für *window* ab oder ändert sie. *Window* muss der Pfadname eines Widgets sein, das zuvor durch die Operation **hold** als beschäftigt markiert wurde. Sind keine Optionen angegeben, wird eine Liste aller verfügbaren Optionen für *window* zurückgegeben (siehe Tk_ConfigureInfo zum Format dieser Liste). Ist *option* ohne *value* angegeben, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Teilliste des Rückgabewerts, wenn keine *option* angegeben ist). Sind ein oder mehrere *option*-*value*-Paare angegeben, setzt der Befehl die angegebenen Widget-Option(en) auf die angegebenen Werte; in diesem Fall gibt der Befehl die leere Zeichenkette zurück. *Option* kann einen der von der Operation **hold** akzeptierten Werte haben.

Beachten Sie, dass die Optionsdatenbank über *window* referenziert wird. Soll beispielsweise das Widget **.frame** als beschäftigt markiert werden, kann der Busy-Cursor dafür mit einem der folgenden option-Befehle festgelegt werden:

```
option add *frame.busyCursor gumby
option add *Frame.BusyCursor gumby
```

**tk busy current ?*pattern*?**

Gibt die Pfadnamen aller Widgets zurück, die derzeit als beschäftigt markiert sind. Ist ein *pattern* angegeben, werden nur die Pfadnamen beschäftigter Widgets zurückgegeben, die *pattern* entsprechen.

**tk busy forget *window* ?*window?...*?**

Gibt die vom Befehl tk busy für *window* belegten Ressourcen frei, einschließlich des transparenten Fensters. *Window* empfängt wieder Benutzerereignisse. Ressourcen werden auch freigegeben, wenn *window* zerstört wird. *Window* muss der Name eines in der Operation **hold** angegebenen Widgets sein, andernfalls wird ein Fehler gemeldet.

**tk busy hold *window* ?*option value* ...?**

Lässt das angegebene *window* (und seine Nachkommen in der Tk-Fensterhierarchie) als beschäftigt erscheinen. *Window* muss ein gültiger Pfadname eines Tk-Widgets sein. Vor dem angegebenen Fenster wird ein transparentes Fenster platziert. Dieses transparente Fenster wird beim nächsten Abarbeiten von Idle-Aufgaben abgebildet, und das angegebene Fenster sowie seine Nachkommen werden für Benutzerinteraktionen blockiert. Normalerweise sollte unmittelbar danach update aufgerufen werden, damit die Operation **hold** wirksam ist, bevor die Anwendung mit ihrer Verarbeitung beginnt. Der Befehl gibt den Pfadnamen des erzeugten Busy-Fensters zurück (d. h. des transparenten Fensters, das das als beschäftigt erscheinende Widget abschirmt). Folgende Konfigurationsoptionen sind gültig:

**-cursor *cursorName***

Legt den Cursor fest, der angezeigt wird, wenn das Widget als beschäftigt markiert ist. *CursorName* kann in jeder von Tk_GetCursor akzeptierten Form angegeben werden. Der Standardcursor ist unter Windows **wait** und auf anderen Plattformen **watch**.

**tk busy status *window***

Gibt den Status eines Widgets *window* zurück. Kann *window* derzeit keine Benutzerinteraktionen empfangen, wird **1** zurückgegeben, andernfalls **0**.

## Ereignisbehandlung

### Bindings

Die Ereignisblockierung wird implementiert, indem ein transparentes Fenster erzeugt und abgebildet wird, das das Widget vollständig abdeckt. Wenn das Busy-Fenster abgebildet ist, schirmt es das Widget und seine Hierarchie unsichtbar von allen möglicherweise gesendeten Ereignissen ab. Wie Tk-Widgets haben Busy-Fenster Widget-Namen in der Tk-Fensterhierarchie. Das bedeutet, dass Sie den Befehl bind verwenden können, um Ereignisse im Busy-Fenster zu behandeln:

```
tk busy hold .frame.canvas
bind [tk busy busywindow .frame.canvas] <Enter> { ... }
```

oder

```
set busyWin [tk busy hold .frame.canvas]
bind $busyWin <Enter> { ... }
```

### Enter/Leave-Ereignisse

Das Abbilden und Entfernen von Busy-Fenstern erzeugt Enter/Leave-Ereignisse für alle von ihnen abgedeckten Widgets. Beachten Sie dies, wenn Sie Enter/Leave-Ereignisse in Widgets verfolgen.

### Tastaturereignisse

Wenn ein Widget als beschäftigt markiert ist, verhindert das Busy-Fenster, dass der Benutzer durch Anklicken den Tastaturfokus auf das Widget setzen kann. Hatte das Widget jedoch bereits den Fokus, kann es weiterhin Tastaturereignisse empfangen. Das Widget kann den Fokus auch weiterhin per Tastatur-Traversal erhalten. Um das zu verhindern, müssen Sie den Fokus auf ein anderes Fenster verschieben und sicherstellen, dass der Fokus nicht zu den als beschäftigt markierten Widgets zurückkehren kann (z. B. durch Beschränkung des Fokus auf einen Abbrechen-Button).

```
pack [frame .frame]
pack [text .frame.text]
tk busy hold .frame
pack [button .cancel -text "Cancel" -command exit]
focus .cancel
bind .cancel <Tab> {break}
bind .cancel <Shift-Tab> {break}
update
```

Das obige Beispiel verschiebt den Fokus von .frame unmittelbar nach dem Aufruf von **hold**, sodass keine Tastaturereignisse an **.frame** oder seine Nachkommen gesendet werden. Es stellt außerdem sicher, dass der Button **.cancel** per Tastatur nicht verlassen werden kann.

## Portierbarkeit

Beachten Sie, dass der Befehl tk busy unter macOS derzeit keine Wirkung hat, wenn Tk mit Aqua-Unterstützung gebaut wurde.

*Copyright (c) 1993-1998 Lucent Technologies, Inc.*
*Copyright (c) 2008 Jos Decoster*

# button

*Stichwörter:* button, widget

## Name

button - Erstellt und manipuliert 'Button'-Action-Widgets

## Übersicht

**button** *pathName* ?*options*?

## Standardoptionen

**-activebackground**, **-font**, **-relief**, **-activeforeground**, **-foreground**, **-repeatdelay**,

**-anchor**, **-highlightbackground**, **-repeatinterval**, **-background**, **-highlightcolor**, **-takefocus**, **-bitmap**, **-highlightthickness**, **-text**, **-borderwidth**, **-image**, **-textvariable**, **-compound**, **-justify**, **-underline**, **-cursor**, **-padx**, **-wraplength**, **-disabledforeground**, **-pady**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-command (command/Command)**

Legt einen Tcl-Befehl fest, der mit dem Button verknüpft wird. Dieser Befehl wird typischerweise aufgerufen, wenn Maustaste 1 über dem Button-Fenster losgelassen wird.

**-default (default/Default)**

Legt einen von drei Zuständen für den Standard-Ring fest: **normal**, **active** oder **disabled**. Im aktiven Zustand wird der Button mit der plattformspezifischen Darstellung für einen Standard-Button gezeichnet. Im normalen Zustand wird der Button mit der plattformspezifischen Darstellung für einen Nicht-Standard-Button gezeichnet, wobei genug Platz für die Standard-Button-Darstellung gelassen wird. Die normalen und aktiven Zustände ergeben Buttons der gleichen Größe. Im deaktivierten Zustand wird der Button mit der Nicht-Standard-Button-Darstellung ohne Platz für die Standard-Darstellung gezeichnet. Der deaktivierte Zustand kann zu einem kleineren Button als der aktive Zustand führen.

**-height (height/Height)**

Legt eine gewünschte Höhe für den Button fest. Wenn ein Bild oder Bitmap im Button angezeigt wird, ist der Wert in Bildschirmeinheiten (d.h. alle von Tk_GetPixels akzeptierten Formen); für Text ist es in Textzeilen. Wenn diese Option nicht angegeben ist, wird die gewünschte Höhe des Buttons aus der Größe des angezeigten Bildes, Bitmaps oder Textes berechnet.

**-overrelief (overRelief/OverRelief)**

Legt ein alternatives Relief für den Button fest, das verwendet wird, wenn der Mauszeiger über dem Widget ist. Diese Option kann verwendet werden, um Toolbar-Buttons zu erstellen, indem **-relief flat -overrelief raised** konfiguriert wird. Wenn der Wert dieser Option der leere String ist, wird kein alternatives Relief verwendet, wenn der Mauszeiger über dem Button ist. Der leere String ist der Standardwert.

**-state (state/State)**

Legt einen von drei Zuständen für den Button fest: **normal**, **active** oder **disabled**. Im normalen Zustand wird der Button mit den Optionen **-foreground** und **-background** angezeigt. Der aktive Zustand wird typischerweise verwendet, wenn der Zeiger über dem Button ist. Im aktiven Zustand wird der Button mit den Optionen **-activeforeground** und **-activebackground** angezeigt. Der deaktivierte Zustand bedeutet, dass der Button unempfindlich sein sollte: Die Standard-Bindings weigern sich, das Widget zu aktivieren und ignorieren Mausklicks. In diesem Zustand bestimmen die Optionen **-disabledforeground** und **-background**, wie der Button angezeigt wird.

**-width (width/Width)**

Legt eine gewünschte Breite für den Button fest. Wenn ein Bild oder Bitmap im Button angezeigt wird, ist der Wert in Bildschirmeinheiten (d.h. alle von Tk_GetPixels akzeptierten Formen). Für einen Text-Button (kein Bild oder mit **-compound none**) gibt die Breite an, wie viel Platz in Zeichen für das Text-Label reserviert werden soll. Wenn die Breite negativ ist, wird eine Mindestbreite angegeben. Wenn diese Option nicht angegeben ist, wird die gewünschte Breite des Buttons aus der Größe des angezeigten Bildes, Bitmaps oder Textes berechnet.

## Beschreibung

Der Befehl **button** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Button-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des Buttons wie Farben, Schriftart, Text und anfängliches Relief zu konfigurieren. Der Befehl **button** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Ein Button ist ein Widget, das einen Text-String, ein Bitmap oder ein Bild anzeigt. Wenn Text angezeigt wird, muss er in einer einzigen Schriftart sein, kann aber mehrere Zeilen auf dem Bildschirm einnehmen (wenn er Zeilenumbrüche enthält oder wenn ein Umbruch wegen der Option **-wraplength** erfolgt), und eines der Zeichen kann optional mit der Option **-underline** unterstrichen werden. Er kann sich auf drei verschiedene Arten darstellen, je nach Option **-state** ; er kann erhaben, eingesenkt oder flach dargestellt werden; und er kann blinken. Wenn ein Benutzer den Button aufruft (durch Drücken von Maustaste 1 mit dem Cursor über dem Button), wird der in der Option **-command** angegebene Tcl-Befehl aufgerufen.

## Widget-Befehl

Der Befehl **button** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Button-Widgets möglich:

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **button** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **button** akzeptierten Werte haben.

***pathName* flash**

Lässt den Button blinken. Dies wird erreicht, indem der Button mehrmals neu angezeigt wird und dabei zwischen den konfigurierten Farben activebackground und background wechselt. Am Ende des Blinkens bleibt der Button im gleichen normal/aktiven Zustand wie beim Aufruf des Befehls. Dieser Befehl wird ignoriert, wenn der Zustand des Buttons **disabled** ist.

***pathName* invoke**

Ruft den mit dem Button verknüpften Tcl-Befehl auf, falls einer vorhanden ist. Der Rückgabewert ist der Rückgabewert des Tcl-Befehls oder ein leerer String, wenn kein Befehl mit dem Button verknüpft ist. Dieser Befehl wird ignoriert, wenn der Zustand des Buttons **disabled** ist.

## Standard-Bindings

Tk erstellt automatisch Klassen-Bindings für Buttons, die ihnen Standardverhalten geben:

1. Ein Button wird aktiviert, wenn die Maus über ihn fährt, und deaktiviert, wenn die Maus ihn verlässt. Unter Windows ist dieses Binding nur aktiv, wenn Maustaste 1 über dem Button gedrückt wurde.
2. Das Relief eines Buttons wird auf eingesenkt geändert, wenn Maustaste 1 über dem Button gedrückt wird, und das Relief wird auf seinen ursprünglichen Wert wiederhergestellt, wenn Maustaste 1 später losgelassen wird.
3. Wenn Maustaste 1 über einem Button gedrückt und später über dem Button losgelassen wird, wird der Button aufgerufen. Wenn sich die Maus jedoch nicht über dem Button befindet, wenn Maustaste 1 losgelassen wird, erfolgt kein Aufruf.
4. Wenn ein Button den Eingabefokus hat, bewirkt die Leertaste, dass der Button aufgerufen wird.

Wenn der Zustand des Buttons **disabled** ist, tritt keine der obigen Aktionen auf: Der Button reagiert überhaupt nicht.

Das Verhalten von Buttons kann geändert werden, indem neue Bindings für einzelne Widgets definiert werden oder indem die Klassen-Bindings neu definiert werden.

## Plattform-Hinweise

Unter Aqua/macOS werden einige Konfigurationsoptionen zum Zweck des Zeichnens des Widgets ignoriert, da sie andernfalls mit Plattform-Richtlinien in Konflikt stehen würden. Die Unterbefehle **configure** und **cget** können die Werte weiterhin manipulieren, verursachen jedoch keine Variation im Aussehen des Widgets. Die betroffenen Optionen umfassen insbesondere **-background** und **-relief**.

## Beispiele

Dies ist die klassische Tk "Hello, World!"-Demonstration:

```
button .b -text "Hello, World!" -command exit
pack .b
```

Dieses Beispiel demonstriert, wie Button-Beschleuniger behandelt werden:

```
button .b1 -text Hello -underline 0
button .b2 -text World -underline 0
bind . <Key-h> {.b1 flash; .b1 invoke}
bind . <Key-w> {.b2 flash; .b2 invoke}
pack .b1 .b2
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# canvas

*Stichwörter:* canvas, widget

## Name

canvas - Erstellt und manipuliert 'Canvas'-Hypergraphik-Zeichen-Oberflächen-Widgets

## Übersicht

**canvas** *pathName* ?*options*?

## Standardoptionen

**-background**, **-borderwidth**, **-cursor**, **-highlightbackground**, **-highlightcolor**, **-highlightthickness**, **-insertbackground**, **-insertborderwidth**, **-insertofftime**, **-insertontime**, **-insertwidth**, **-relief**, **-selectbackground**, **-selectborderwidth**, **-selectforeground**, **-takefocus**, **-xscrollcommand**, **-yscrollcommand**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-closeenough (closeEnough/CloseEnough)**

Legt einen Gleitkommawert fest, der angibt, wie nah der Mauszeiger an einem Item sein muss, bevor es als "innerhalb" des Items gilt. Standardwert 1.0.

**-confine (confine/Confine)**

Legt einen booleschen Wert fest, der angibt, ob die Ansicht des Canvas außerhalb der durch das Argument **scrollRegion** definierten Region gesetzt werden darf. Standardwert true, d.h. die Ansicht wird auf die Scroll-Region beschränkt.

**-height (height/Height)**

Legt eine gewünschte Fensterhöhe fest, die das Canvas-Widget von seinem Geometry-Manager anfordern soll. Der Wert kann in einer der im Abschnitt Koordinaten unten beschriebenen Formen angegeben werden.

**-scrollregion (scrollRegion/ScrollRegion)**

Legt eine Liste mit vier Koordinaten fest, die die linke, obere, rechte und untere Koordinate einer rechteckigen Region beschreiben. Diese Region dient dem Scrollen und gilt als Grenze der Informationen im Canvas. Jede Koordinate kann in einer der im Abschnitt Koordinaten angegebenen Formen angegeben werden.

**-state (state/State)**

Ändert den Standardzustand des Canvas; *state* kann **normal**, **disabled** oder **hidden** sein. Einzelne Canvas-Objekte haben jeweils eine eigene Zustandsoption, die den Standardzustand überschreiben kann. Viele Optionen können getrennte Angaben für verschiedene Situationen haben, sodass das Erscheinungsbild des Items je nach Zustand unterschiedlich sein kann. Optionen, die mit **active** beginnen, steuern das Erscheinungsbild, wenn der Mauszeiger darüber ist; die mit **disabled** beginnende Option steuert das Erscheinungsbild im deaktivierten Zustand. Canvas-Items im Zustand **disabled** reagieren nicht auf Canvas-Bindings.

**-width (width/width)**

Legt eine gewünschte Fensterbreite fest, die das Canvas-Widget von seinem Geometry-Manager anfordern soll. Der Wert kann in einer der im Abschnitt Koordinaten beschriebenen Formen angegeben werden.

**-xscrollincrement (xScrollIncrement/ScrollIncrement)**

Legt ein Inkrement für horizontales Scrollen fest, in einer der üblichen für Bildschirmabstände zulässigen Formen. Ist der Wert größer als null, wird die horizontale Ansicht so beschränkt, dass die Canvas-x-Koordinate am linken Fensterrand stets ein gerades Vielfaches von **xScrollIncrement** ist; außerdem sind die Scroll-Einheiten (z.B. die Ansichtsänderung bei Auswahl der linken und rechten Pfeile einer Scrollbar) ebenfalls **xScrollIncrement**. Ist der Wert negativ oder null, ist horizontales Scrollen unbeschränkt.

**-yscrollincrement (yScrollIncrement/ScrollIncrement)**

Legt ein Inkrement für vertikales Scrollen fest, in einer der üblichen für Bildschirmabstände zulässigen Formen. Ist der Wert größer als null, wird die vertikale Ansicht so beschränkt, dass die Canvas-y-Koordinate am oberen Fensterrand stets ein gerades Vielfaches von **yScrollIncrement** ist; außerdem sind die Scroll-Einheiten (z.B. die Ansichtsänderung bei Auswahl der oberen und unteren Pfeile einer Scrollbar) ebenfalls **yScrollIncrement**. Ist der Wert negativ oder null, ist vertikales Scrollen unbeschränkt.

## Einleitung

Der Befehl **canvas** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Canvas-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des Canvas wie Farben und 3-D-Relief zu konfigurieren. Der Befehl **canvas** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Canvas-Widgets implementieren strukturierte Grafiken. Ein Canvas zeigt beliebig viele *Items* an, die Dinge wie Rechtecke, Kreise, Linien und Text sein können. Items können manipuliert werden (z.B. verschoben oder umgefärbt) und Befehle können mit Items verknüpft werden, ähnlich wie der Befehl bind es erlaubt, Befehle an Widgets zu binden. Zum Beispiel kann ein bestimmter Befehl mit dem Ereignis verknüpft werden, sodass der Befehl aufgerufen wird, wenn Maustaste 1 gedrückt wird, während sich der Mauszeiger über einem Item befindet. Das bedeutet, dass Items in einem Canvas Verhalten haben können, die durch die an sie gebundenen Tcl-Scripte definiert sind.

## Anzeigeliste

Die Items in einem Canvas sind für Anzeigezwecke geordnet, wobei das erste Item in der Anzeigeliste zuerst angezeigt wird, gefolgt vom nächsten Item in der Liste usw. Items, die später in der Anzeigeliste stehen, verdecken diejenigen, die früher in der Anzeigeliste stehen, und werden manchmal als "oben" auf früheren Items bezeichnet. Wenn ein neues Item erstellt wird, wird es am Ende der Anzeigeliste platziert, oben auf allem anderen. Widget-Befehle können verwendet werden, um die Reihenfolge der Anzeigeliste neu anzuordnen.

Fenster-Items sind eine Ausnahme von den obigen Regeln. Die zugrunde liegenden Fenstersysteme erfordern, dass sie immer oben auf anderen Items gezeichnet werden. Außerdem wird die Stapelreihenfolge von Fenster-Items nicht durch Canvas-Widget-Befehle beeinflusst; stattdessen müssen die Tk-Befehle raise und lower verwendet werden.

## Item-IDs und Tags

Items in einem Canvas-Widget können auf zwei Arten benannt werden: per ID oder per Tag.

Jedes Item hat eine eindeutige Identifikationsnummer, die diesem Item bei seiner Erstellung zugewiesen wird. Die ID eines Items ändert sich nie und ID-Nummern werden innerhalb der Lebensdauer eines Canvas-Widgets nie wiederverwendet.

Jedes Item kann auch eine beliebige Anzahl von *Tags* haben. Ein Tag ist einfach eine Zeichenkette und kann jede Form annehmen, außer der einer Ganzzahl. Zum Beispiel ist "x123" OK, aber "123" nicht. Derselbe Tag kann vielen verschiedenen Items zugeordnet werden. Dies wird häufig verwendet, um Items auf verschiedene interessante Arten zu gruppieren; zum Beispiel könnten alle ausgewählten Items den Tag "selected" erhalten.

Der Tag **all** ist implizit mit jedem Item im Canvas verknüpft; er kann verwendet werden, um Operationen auf allen Items im Canvas aufzurufen.

Der Tag **current** wird automatisch von Tk verwaltet; er gilt für das *aktuelle Item*, das das oberste Item ist, dessen gezeichneter Bereich die Position des Mauszeigers abdeckt (verschiedene Item-Typen interpretieren dies auf unterschiedliche Weise; siehe die Dokumentation der einzelnen Item-Typen für Details). Wenn sich die Maus nicht im Canvas-Widget befindet oder nicht über einem Item ist, hat kein Item den Tag **current**.

Wenn Items in Canvas-Widget-Befehlen angegeben werden: Wenn der Spezifizierer eine Ganzzahl ist, wird angenommen, dass er sich auf das einzelne Item mit dieser ID bezieht. Wenn der Spezifizierer keine Ganzzahl ist, wird angenommen, dass er sich auf alle Items im Canvas bezieht, die einen zum Spezifizierer passenden Tag haben. Das Symbol *tagOrId* wird unten verwendet, um anzuzeigen, dass ein Argument entweder eine ID angibt, die ein einzelnes Item auswählt, oder einen Tag, der null oder mehr Items auswählt.

*tagOrId* kann logische Ausdrücke von Tags enthalten, indem die Operatoren "**&&**", "**||**", "**^**", "**!**" und geklammerte Unterausdrücke verwendet werden. Zum Beispiel:

```
      .c find withtag {(a&&!b)||(!a&&b)}
```

oder äquivalent:

```
      .c find withtag {a^b}
```

findet nur die Items mit entweder "a" oder "b" Tags, aber nicht beide.

Einige Widget-Befehle operieren nur auf jeweils einem Item; wenn *tagOrId* so angegeben wird, dass mehrere Items benannt werden, besteht das normale Verhalten darin, dass der Befehl das erste (niedrigste) dieser Items in der Anzeigeliste verwendet, das für den Befehl geeignet ist. Ausnahmen werden in den Widget-Befehlsbeschreibungen unten vermerkt.

## Koordinaten

Alle mit Canvas zusammenhängenden Koordinaten werden als Gleitkommazahlen gespeichert. Koordinaten und Abstände werden in Bildschirmeinheiten angegeben — Gleitkommazahlen, optional gefolgt von einem Buchstaben. Ohne Buchstabe gilt der Abstand in Pixeln. **m** bedeutet Millimeter auf dem Bildschirm, **c** Zentimeter, **i** Zoll und **p** Druckerpunkte (1/72 Zoll). Größere y-Koordinaten bezeichnen Punkte weiter unten auf dem Bildschirm; größere x-Koordinaten Punkte weiter rechts. Koordinaten können entweder als gerade Anzahl von Parametern oder als ein einzelner Listenparameter mit gerader Anzahl von x- und y-Koordinatenwerten angegeben werden.

### Transformationen

Normalerweise liegt der Ursprung des Canvas-Koordinatensystems in der oberen linken Ecke des Fensters, das den Canvas enthält. Der Ursprung des Canvas-Koordinatensystems relativ zum Fensterursprung kann mit den Widget-Befehlen **xview** und **yview** angepasst werden; typischerweise für Scrollen. Canvas unterstützen keine Skalierung oder Rotation des Canvas-Koordinatensystems relativ zum Fenster-Koordinatensystem.

Einzelne Items können verschoben, skaliert oder gedreht werden — mit den unten beschriebenen Widget-Befehlen.

Beachten Sie, dass der Standardursprung des sichtbaren Canvas-Bereichs mit dem Ursprung des gesamten Fensters zusammenfällt; das erleichtert Bindings mit der Mausposition — **canvasx** und **canvasy** sind nur nötig, wenn Sie den Ursprung des sichtbaren Bereichs verschieben. Gleichzeitig müssen Fokusring (Option **-highlightthickness**) und Fensterrand (Option **-borderwidth**) berücksichtigt werden, bevor der sichtbare Canvas-Bereich erreicht ist.

## Indizes

Text-Items unterstützen den Begriff eines *Index* zur Identifikation bestimmter Positionen innerhalb des Items. Entsprechend unterstützen Linien- und Polygon-Items *Index* zum Identifizieren, Einfügen und Löschen von Teilmengen ihrer Koordinaten. Indizes werden für Befehle wie Einfügen oder Löschen eines Zeichen- oder Koordinatenbereichs und zum Setzen der Einfügemarken-Position verwendet. Ein Index kann auf verschiedene Weise angegeben werden; verschiedene Item-Typen können unterschiedliche Formen unterstützen. Text-Items unterstützen folgende Index-Formen; bei neuen textähnlichen Item-Typen ist es ratsam, möglichst viele dieser Formen zu unterstützen. Es ist möglich, auf das Zeichen unmittelbar nach dem letzten im Text-Item zu verweisen — nötig z.B. zum Einfügen neuer Texte am Ende. Linien und Polygone unterstützen weder Einfügemarke noch Auswahl; ihre Indizes sollen stets gerade sein, da Koordinaten stets paarweise auftreten.

***number***

Eine Dezimalzahl für die Position des gewünschten Zeichens im Text-Item. 0 bezeichnet das erste Zeichen, 1 das nächste usw. Bei Linien und Polygonen werden ungerade Indizes automatisch um eins verringert. Eine negative Zahl wird wie null behandelt; eine Zahl größer als die Textlänge wie die Textlänge. Bei Polygonen werden negative Zahlen oder Zahlen größer als die Koordinatenlistenlänge durch Addieren oder Subtrahieren der Länge angepasst, bis das Ergebnis zwischen null und der Länge (einschließlich) liegt.

**end**

Bezeichnet das Zeichen oder die Koordinate unmittelbar nach dem letzten im Item (entspricht der Anzahl der Zeichen bzw. Koordinaten im Item).

**insert**

Bezeichnet das Zeichen unmittelbar vor der Einfügemarke in diesem Item. Für Linien und Polygone ungültig.

**sel.first**

Bezeichnet das erste ausgewählte Zeichen im Item. Ungültig, wenn die Auswahl nicht in diesem Item liegt.

**sel.last**

Bezeichnet das letzte ausgewählte Zeichen im Item. Ungültig, wenn die Auswahl nicht in diesem Item liegt.

**@*x,y***

Bezeichnet das Zeichen oder die Koordinate am Punkt *x*, *y* im Canvas-Koordinatensystem. Liegen *x* und *y* außerhalb der vom Text-Item abgedeckten Koordinaten, beziehen sie sich auf das erste bzw. letzte Zeichen in der dem Punkt nächsten Zeile.

## Strichmuster

Viele Items unterstützen Strichmuster für Umrisse.

Die erste mögliche Syntax ist eine Liste von Ganzzahlen. Jedes Element gibt die Pixelanzahl eines Liniensegments an. Nur die ungeraden Segmente werden in der "outline"-Farbe gezeichnet; die anderen sind transparent.

Die zweite mögliche Syntax ist eine Zeichenliste mit nur 5 Zeichen "**.,-_** ". Leerzeichen vergrößern den Abstand zwischen anderen Linienelementen und dürfen nicht an erster Stelle stehen. Beispiele:

```
-dash .     → -dash {2 4}
-dash -     → -dash {6 4}
-dash -.    → -dash {6 4 2 4}
-dash -..   → -dash {6 4 2 4 2 4}
-dash {. }  → -dash {2 8}
-dash ,     → -dash {4 4}
```

Der Hauptunterschied dieser Syntax zur ersten ist die Formtreue: Alle Werte der Strichliste werden vor der Anzeige mit der Linienbreite multipliziert. So wird "." stets als Punkt und "-" stets als Strich dargestellt, unabhängig von der Linienbreite.

Auf Systemen mit nur begrenzt unterstützten Strichmustern wird das nächstliegende verfügbare Muster verwendet. Unter Windows sind z.B. nur die ersten 4 der obigen Beispiele verfügbar; die letzten 2 werden wie das erste dargestellt.

## Widget-Befehl

Der Befehl **canvas** erstellt einen neuen Tcl-Befehl mit dem Namen *pathName*. Dieser Befehl kann verschiedene Operationen auf dem Widget aufrufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Folgende Widget-Befehle sind für Canvas-Widgets möglich:

***pathName* addtag *tag searchSpec* ?*arg* ...?**

Für jedes Item, das die durch *searchSpec* und die *arg*s angegebenen Bedingungen erfüllt, wird *tag* zur Tag-Liste des Items hinzugefügt, falls noch nicht vorhanden. Es ist möglich, dass kein Item die Bedingungen erfüllt; dann hat der Befehl keine Wirkung. Rückgabe: leerer String. *SearchSpec* und *arg*s können folgende Formen haben:

**above *tagOrId***

Wählt das Item unmittelbar nach (oberhalb) dem durch *tagOrId* bezeichneten in der Anzeigeliste. Bezeichnet *tagOrId* mehrere Items, wird das letzte (oberste) davon verwendet.

**all**

Wählt alle Items im Canvas.

**below *tagOrId***

Wählt das Item unmittelbar vor (unterhalb) dem durch *tagOrId* bezeichneten in der Anzeigeliste. Bezeichnet *tagOrId* mehrere Items, wird das erste (unterste) davon verwendet.

**closest *x y* ?*halo*? ?*start*?**

Wählt das dem Punkt *x*, *y* nächste Item. Liegen mehrere Items in gleicher Mindestentfernung (z.B. Überlappung), wird das oberste (letzte in der Anzeigeliste) gewählt. *halo* muss, falls angegeben, nicht negativ sein; jedes Item näher als *halo* zum Punkt gilt als überlappend. *start* ermöglicht zyklisches Durchsteppen aller nächsten Items. Ist *start* angegeben, benennt es ein Item per Tag oder ID (bei Tag: erstes Item in der Anzeigeliste mit diesem Tag). Statt des obersten nächsten Items wird das oberste nächste unterhalb von *start* in der Anzeigeliste gewählt; existiert keines, verhält sich die Auswahl wie ohne *start*.

**enclosed *x1 y1 x2 y2***

Wählt alle Items, die vollständig im Rechteck *x1*, *y1*, *x2*, *y2* liegen. *x1* darf *x2* nicht überschreiten, *y1* nicht *y2*.

**overlapping *x1 y1 x2 y2***

Wählt alle Items, die das Rechteck *x1*, *y1*, *x2*, *y2* überlappen oder einschließen. *x1* darf *x2* nicht überschreiten, *y1* nicht *y2*.

**withtag *tagOrId***

Wählt alle durch *tagOrId* bezeichneten Items.

***pathName* bbox *tagOrId* ?*tagOrId tagOrId* ...?**

Gibt eine Liste mit vier Elementen zurück — eine näherungsweise Bounding-Box für alle durch die *tagOrId*-Argumente benannten Items. Form "*x1 y1 x2 y2*", sodass die gezeichneten Bereiche aller benannten Elemente innerhalb der Region links *x1*, rechts *x2*, oben *y1*, unten *y2* liegen. Der Rückgabewert kann die tatsächliche Box um wenige Pixel überschätzen. Stimmen keine Items überein oder haben leere Bounding-Boxes (nichts Darstellbares), wird ein leerer String zurückgegeben.

***pathName* bind *tagOrId* ?*sequence*? ?*command*?**

Verknüpft *command* mit allen durch *tagOrId* bezeichneten Items, sodass der Befehl aufgerufen wird, wenn die Ereignisfolge *sequence* für eines der Items eintritt. Ähnlich dem Befehl bind, operiert aber auf Canvas-Items statt ganzen Widgets. Siehe bind für Syntax von *sequence* und Substitutionen in *command*. Sind alle Argumente angegeben, wird ein neues Binding erstellt und ersetzt ein bestehendes für dieselbe *sequence* und *tagOrId* (beginnt *command* mit "+", wird ein bestehendes Binding ergänzt). Rückgabe: leerer String. Fehlt *command*, wird der mit *tagOrId* und *sequence* verknüpfte Befehl zurückgegeben (Fehler, wenn kein Binding existiert). Fehlen *command* und *sequence*, eine Liste aller Sequenzen mit Bindings für *tagOrId*.

Bindings sind nur für Maus- und Tastaturereignisse (z.B. **Enter**, **Leave**, **Button**, **Motion**, **Key** ) oder virtuelle Ereignisse möglich. Die Ereignisbehandlung nutzt das aktuelle Item aus Item-IDs und Tags oben. **Enter** und **Leave** für ein Item, wenn es aktuelles Item wird oder aufhört — anders als Fenster-**Enter**/**Leave**. Mausereignisse gehen an das aktuelle Item; Tastaturereignisse an das Fokus-Item (siehe **focus** unten). Virtuelle Bindings lösen nur aus, wenn das virtuelle Ereignis von einem Maus- oder Tastaturereignis abgeleitet ist. Mehrere Bindings können zu einem Ereignis passen (z.B. Binding an Item-ID und an einen Tag). Dann werden alle passenden aufgerufen: zuerst **all**, dann je ein Binding pro Tag des Items (in Reihenfolge), dann Binding an die Item-ID. Bei mehreren passenden Bindings für einen Tag nur das spezifischste. continue beendet das Skript; break beendet es und überspringt weitere Skripte für das Ereignis — wie bei bind. Bindings am Canvas-Fenster per bind werden zusätzlich zu Item-Bindings per **bind**-Widget-Befehl aufgerufen; Item-Bindings vor Fenster-Bindings.

***pathName* canvasx *screenx* ?*gridspacing*?**

Zu einer Fenster-x-Koordinate *screenx* im Canvas die dort angezeigte Canvas-x-Koordinate. Mit *gridspacing* wird auf das nächste Vielfache von *gridspacing* gerundet.

***pathName* canvasy *screeny* ?*gridspacing*?**

Zu einer Fenster-y-Koordinate *screeny* im Canvas die dort angezeigte Canvas-y-Koordinate. Mit *gridspacing* wird auf das nächste Vielfache von *gridspacing* gerundet.

***pathName* cget *option***

Gibt den aktuellen Wert der Konfigurationsoption *option* zurück. *Option* kann jeden vom Befehl **canvas** akzeptierten Wert haben.

***pathName* configure ?*option*? ?*value*? ?*option value* ...?**

Abfrage oder Änderung der Widget-Konfigurationsoptionen. Ohne *option*: Liste aller verfügbaren Optionen für *pathName* (Format siehe Tk_ConfigureInfo). *option* ohne *value*: Liste für diese eine Option. Ein oder mehrere *option*-*value*-Paare ändern die Option(en); Rückgabe: leerer String. *Option* kann jeden vom Befehl **canvas** akzeptierten Wert haben.

***pathName* coords *tagOrId* ?*x0 y0* ...?**

...siehe nächstes...

***pathName* coords *tagOrId* ?*coordList*?**

Abfrage oder Änderung der Koordinaten, die ein Item definieren. Ohne Koordinaten: Liste der Koordinaten des durch *tagOrId* benannten Items. Mit Koordinaten: Ersetzen der aktuellen Koordinaten. Bei mehreren Items durch *tagOrId* wird das erste in der Anzeigeliste verwendet.

Bei Rechtecken, Ovals und Bögen hat die zurückgegebene Koordinatenliste feste Reihenfolge: links, oben, rechts, unten — nicht unbedingt die ursprünglich angegebene. Koordinaten werden stets in Bildschirmeinheiten ohne Einheit (Pixel) zurückgegeben; waren sie z.B. in Zentimetern angegeben, sind die Rückgabewerte dennoch in Pixeln.

***pathName* create *type x y* ?*x y* ...? ?*option value* ...?**

...siehe nächstes...

***pathName* create *type coordList* ?*option value* ...?**

Erstellt ein neues Item vom Typ *type* in *pathName*. Das genaue Format der Argumente nach *type* hängt vom Typ ab, meist Koordinaten eines oder mehrerer Punkte, gefolgt von null oder mehr Item-Optionen. Siehe die Unterabschnitte zu den Item-Typen unten. Rückgabe: ID des neuen Items.

***pathName* dchars *tagOrId first* ?*last*?**

Für jedes durch *tagOrId* bezeichnete Item werden die Zeichen bzw. Koordinaten im Bereich *first* bis *last* (einschließlich) gelöscht. Items ohne Index-Unterstützung ignorieren die Operation. Text-Items interpretieren *first* und *last* als Zeichenindizes; Linien- und Polygon-Items als Koordinatenindizes (x,y-Paar). Indizes siehe Indizes oben. Fehlt *last*, entspricht es *first*. Rückgabe: leerer String.

***pathName* delete ?*tagOrId tagOrId* ...?**

Löscht jedes durch *tagOrId* bezeichnete Item; Rückgabe: leerer String.

***pathName* dtag *tagOrId* ?*tagToDelete*?**

Entfernt für jedes durch *tagOrId* bezeichnete Item den Tag *tagToDelete* aus der zugehörigen Liste. Hat ein Item den Tag nicht, bleibt es unverändert. Fehlt *tagToDelete*, entspricht es *tagOrId*. Rückgabe: leerer String.

***pathName* find *searchCommand* ?*arg* ...?**

Gibt eine Liste aller Items zurück, die die durch *searchCommand* und *arg*s angegebenen Bedingungen erfüllen. *SearchCommand* und *args* haben die von **addtag** akzeptierten Formen. Items in Stapelreihenfolge, unterstes zuerst.

***pathName* focus ?*tagOrId*?**

Setzt den Tastaturfokus des Canvas auf das durch *tagOrId* bezeichnete Item. Bei mehreren Items: erstes in der Anzeigeliste mit Einfügemarken-Unterstützung. Bezeichnet *tagOrId* keine Items oder unterstützt keines die Einfügemarke, bleibt der Fokus unverändert. Ist *tagOrId* ein leerer String, hat kein Item den Fokus. Ohne *tagOrId*: ID des Items mit Fokus oder leerer String.

Mit Fokus auf einem Item zeigt es die Einfügemarke; alle Tastaturereignisse gehen dorthin. Fokus-Item im Canvas und Fokus-Fenster auf dem Bildschirm (per focus) sind unabhängig: Ein Item hat den Eingabefokus nur, wenn (a) sein Canvas das Fokus-Fenster ist und (b) es das Fokus-Item im Canvas ist. Meist sollte auf den **focus**-Widget-Befehl der focus-Befehl folgen, um das Fokus-Fenster auf den Canvas zu setzen.

***pathName* gettags *tagOrId***

Gibt eine Liste der mit dem durch *tagOrId* bezeichneten Item verknüpften Tags zurück. Bei mehreren Items: Tags des ersten in der Anzeigeliste. Bezeichnet *tagOrId* keine Items oder hat das Item keine Tags: leerer String.

***pathName* icursor *tagOrId index***

Setzt die Einfügemarken-Position für die durch *tagOrId* bezeichneten Items unmittelbar vor das Zeichen an Position *index*. Items ohne Einfügemarke bleiben unverändert. Zulässige Formen für *index* siehe Indizes oben.

Die Einfügemarke wird nur angezeigt, wenn das Item den Tastaturfokus hat (siehe **focus** oben); die Position kann auch ohne Fokus gesetzt werden.

Rückgabe: leerer String.

***pathName* image *imagename* ?*subsample*? ?*zoom*?**

Zeichnet den Canvas in das Tk-Foto-Image *imagename*. Ist **-scrollregion** definiert, sind das die Grenzen der gezeichneten Canvas-Region und die endgültige Bildgröße. Sonst Widget-Breite und -Höhe mit Ursprung 0,0. Optional ganzzahliger *subsample*-Faktor zur Verkleinerung.

Zusätzlich zu *subsample* kann ein ganzzahliger *zoom*-Faktor die Vergrößerung steuern. Der Bildhintergrund wird mit der Canvas-Hintergrundfarbe gefüllt. Der Canvas muss nicht gemappt sein; mindestens ein Vorfahre muss gemappt sein.

Rückgabe: leerer String.

***pathName* imove *tagOrId index x y***

Verschiebt die *index*. Koordinate jedes durch *tagOrId* bezeichneten Items nach (*x*,*y*). Jedes Item interpretiert *index* gemäß Indizes oben. Von den Standard-Items können nur Linien und Polygone ihre Koordinaten so verschieben.

***pathName* index *tagOrId index***

Gibt eine Dezimalzeichenkette mit dem numerischen Index innerhalb von *tagOrId* zurück, der *index* entspricht. *index* beschreibt die gewünschte Position gemäß Indizes oben. Text-Items: Zeichenindex; Linien und Polygone: Koordinatenindex (x,y-Paar). Der Rückgabewert liegt garantiert zwischen 0 und der Anzahl der Zeichen bzw. Koordinaten im Item (einschließlich). Bei mehreren Items wird das erste mit Index-Unterstützung in Anzeigelistenreihenfolge verwendet.

***pathName* insert *tagOrId beforeThis string***

Für jedes durch *tagOrId* bezeichnete Item mit Text- oder Koordinaten-Einfügeunterstützung wird *string* unmittelbar vor das Zeichen bzw. die Koordinate mit Index *beforeThis* eingefügt. Text-Items: Zeichenindex; Linien und Polygone: Koordinatenindex. Bei Linien und Polygonen muss *string* eine gültige Koordinatenfolge sein.

Zulässige Formen für *beforeThis* siehe Indizes oben.

Rückgabe: leerer String.

***pathName* itemcget *tagOrId option***

Gibt den aktuellen Wert der Konfigurationsoption *option* für das durch *tagOrId* bezeichnete Item zurück. Ähnlich **cget**, aber für ein einzelnes Item. *option* kann jeden beim **create** -Widget-Befehl akzeptierten Wert haben. Bei Tag mit mehreren Items: erstes (unterstes).

***pathName* itemconfigure *tagOrId* ?*option*? ?*value*? ?*option value* ...?**

Ähnlich **configure**, ändert aber Item-spezifische Optionen der durch *tagOrId* bezeichneten Items statt der Canvas-Gesamtoptionen. Ohne *option*: Liste aller Optionen für das erste Item (Format Tk_ConfigureInfo). *option* ohne *value*: Liste für diese Option. *option*-*value*-Paare setzen die Werte in allen genannten Items; Rückgabe: leerer String. *option* und *value* wie beim **create**-Widget-Befehl; Details siehe Item-Typ-Abschnitte unten.

***pathName* lower *tagOrId* ?*belowThis*?**

Verschiebt alle durch *tagOrId* bezeichneten Items in der Anzeigeliste unmittelbar vor das durch *belowThis* bezeichnete Item. Bei mehreren Items werden alle verschoben; die relative Reihenfolge der verschobenen Items bleibt. *belowThis* ist Tag oder ID; bei mehreren Items wird das erste (unterste) als Ziel verwendet. Keine Wirkung auf Fenster-Items — deren Stapelreihenfolge bestimmen die Befehle raise und lower, nicht die Canvas-Widget-Befehle **raise** und **lower**. Rückgabe: leerer String.

***pathName* move *tagOrId xAmount yAmount***

Verschiebt jedes durch *tagOrId* bezeichnete Item im Canvas-Koordinatenraum um Addition von *xAmount* zu jeder x-Koordinate und *yAmount* zu jeder y-Koordinate aller zugehörigen Punkte. Rückgabe: leerer String.

***pathName* moveto *tagOrId xPos yPos***

Verschiebt die durch *tagOrId* bezeichneten Items so, dass das erste Koordinatenpaar (obere linke Ecke der Bounding-Box) des ersten Items (unterstes in der Anzeigeliste) mit Tag *tagOrId* an Position (*xPos*,*yPos*) liegt. *xPos* und *yPos* können leerer String sein (Koordinate unverändert). Alle passenden Items behalten ihre relative Position zueinander. Rückgabe: leerer String.

***pathName* postscript ?*option value option value* ...?**

Erzeugt eine Postscript-Darstellung für einen Teil oder den gesamten Canvas. Mit Option **-file** wird Postscript in eine Datei geschrieben und ein leerer String zurückgegeben; sonst ist Postscript das Ergebnis des Befehls. Ist der Interpreter des Canvas als safe markiert, schlägt die Operation fehl (safe Interpreter dürfen keine Dateien schreiben). Mit **-channel** bezeichnet das Argument einen bereits zum Schreiben geöffneten Kanal; Postscript wird dorthin geschrieben, der Kanal bleibt offen. Postscript wird als Encapsulated Postscript nach Document Structuring Conventions Version 3.0 erzeugt. Hinweis: Standardmäßig wird Postscript nur für im Canvas-Fenster sichtbare Information erzeugt. Ein neu erstellter Canvas kann noch 1×1 Pixel groß sein — dann erscheint nichts im Postscript. Abhilfe: update aufrufen, bis das Fenster seine endgültige Größe hat, oder **-width** und **-height** zur Angabe des Druckbereichs. Die *option*-*value*-Paare steuern die Postscript-Erzeugung. Unterstützte Optionen:

**-channel *channelName***

Name des Kanals, in den Postscript geschrieben wird. Ohne diese und ohne **-file** wird Postscript als Befehlsergebnis zurückgegeben.

**-colormap *varName***

*varName* muss der Name einer Array-Variable mit Farbzuordnung für Postscript sein. Jedes Element besteht aus Postscript-Code zum Setzen einer Farbe (z.B. "**1.0 1.0 0.0 setrgbcolor**"). Beim Ausgeben von Farbinformation prüft Tk, ob ein Element von *varName* denselben Namen wie die Farbe hat; dann wird dessen Wert als Postscript-Befehl verwendet.

Ohne diese Option oder ohne Eintrag in *varName* für eine Farbe verwendet Tk die Rot-, Grün- und Blau-Intensitäten der X-Farbe.

**-colormode *mode***

Steuert die Farbausgabe. *mode* muss **color** (Vollfarbe), **gray** (Graustufen) oder **mono** (Schwarz/Weiß) sein.

**-file *fileName***

Dateiname für Postscript. Ohne diese und ohne **-channel** wird Postscript als Befehlsergebnis zurückgegeben.

**-fontmap *varName***

*varName* muss der Name einer Array-Variable mit Schriftzuordnung für Postscript sein. Jedes Element ist eine Tcl-Liste mit zwei Elementen: Name und Punktgröße einer Postscript-Schrift. Beim Ausgeben prüft Tk, ob *varName* ein Element mit demselben Namen wie die Schrift enthält; dann werden dessen Angaben verwendet. Sonst versucht Tk zu erraten. Das gelingt meist nur bei bekannten Schriften wie Times, Helvetica und Courier und nur, wenn der X-Schriftname bis zur Punktgröße keine Bindestriche auslässt.

Z.B. funktioniert **-*-Courier-Bold-R-Normal--*-120-***, nicht aber ***Courier-Bold-R-Normal*120*** — Tk braucht die Bindestriche zum Parsen).

**-height *size***

Höhe des zu druckenden Canvas-Bereichs. Standard: Höhe des Canvas-Fensters.

**-pageanchor *anchor***

Welcher Punkt des gedruckten Canvas-Bereichs über dem Positionierungspunkt auf der Seite liegt (gegeben durch **-pagex** und **-pagey**).

Z.B. bedeutet **-pageanchor n**, dass die obere Mitte des gedruckten Bereichs (wie im Canvas-Fenster) über dem Positionierungspunkt liegt. Standard: **center**.

**-pageheight *size***

Postscript wird in x und y skaliert, sodass der Druckbereich auf der Postscript-Seite *size* hoch ist. *size*: Gleitkommazahl gefolgt von **c** (Zentimeter), **i** (Zoll), **m** (Millimeter) oder **p** bzw. nichts (Druckerpunkte, 1/72 Zoll). Standard: Bildschirmhöhe des Druckbereichs. Sind **-pageheight** und **-pagewidth** gesetzt, wird der Skalierungsfaktor von **-pagewidth** verwendet (ungleichmäßige Skalierung ist nicht implementiert).

**-pagewidth *size***

Wie **-pageheight**, aber für die Breite. *size* hat dieselbe Form. Standard: Bildschirmbreite. Bei beiden Optionen gilt der Skalierungsfaktor von **-pagewidth**.

**-pagex *position***

*position* gibt die x-Koordinate des Positionierungspunkts auf der Postscript-Seite in einer für **-pageheight** zulässigen Form. Zusammen mit **-pagey** und **-pageanchor** bestimmt dies die Platzierung. Standard: Seitenmitte.

**-pagey *position***

*position* gibt die y-Koordinate des Positionierungspunkts auf der Postscript-Seite. Zusammen mit **-pagex** und **-pageanchor**. Standard: Seitenmitte.

**-rotate *boolean***

*boolean* gibt an, ob der Druckbereich um 90 Grad gedreht wird. Ohne Drehung verläuft die x-Achse entlang der kurzen Seite ("Hochformat"); mit Drehung entlang der langen Seite ("Querformat"). Standard: nicht gedreht.

**-width *size***

Breite des zu druckenden Canvas-Bereichs. Standard: Breite des Canvas-Fensters.

**-x *position***

x-Koordinate der linken Kante des zu druckenden Bereichs in Canvas-Koordinaten, nicht Fensterkoordinaten. Standard: linke Fensterkante.

**-y *position***

y-Koordinate der oberen Kante des zu druckenden Bereichs in Canvas-Koordinaten. Standard: obere Fensterkante.

***pathName* raise *tagOrId ?aboveThis?***

Verschiebt alle durch *tagOrId* bezeichneten Items in der Anzeigeliste unmittelbar nach dem durch *aboveThis* bezeichneten Item. Bei mehreren Items werden alle verschoben; die relative Reihenfolge bleibt. *aboveThis* ist Tag oder ID; bei mehreren Items wird das letzte (oberste) als Ziel verwendet. Rückgabe: leerer String.

Dieser Befehl hat keine Wirkung auf Fenster-Items. Deren Stapelreihenfolge bestimmen die Befehle raise und lower, nicht die Canvas-Widget-Befehle **raise** und **lower**.

***pathName* rchars *tagOrId first last string***

Ersetzt für jedes durch *tagOrId* bezeichnete Item den Text bzw. die Koordinaten zwischen *first* und *last* durch *string*. Jedes Item interpretiert *first* und *last* gemäß Indizes oben. Text-Items ändern ihren Text; Linien und Polygone ihre Koordinatenliste (*string* dann Koordinatenliste). Andere Items ignorieren die Operation.

***pathName* rotate *tagOrId xOrigin yOrigin angle***

Dreht die Koordinaten aller durch tagOrId bezeichneten Items im Canvas-Koordinatenraum. xOrigin und yOrigin sind der Drehmittelpunkt; angle der Drehwinkel gegen den Uhrzeigersinn in Grad (negative Werte: im Uhrzeigersinn). Rückgabe: leerer String.

Implementierungshinweis: Nicht alle Item-Typen verhalten sich gleich. **bitmap**, **image**, **text** und **window** drehen nur ihre Ankerpunkte, nicht die Items selbst; **arc**, **oval** und **rectangle** drehen um einen berechneten Mittelpunkt statt die Bounding-Box-Koordinaten direkt zu verschieben.

Einige Items (**arc** und **text**) haben Winkel in ihren Optionen; dieser Befehl ändert diese Optionen *nicht*.

***pathName* scale *tagOrId xOrigin yOrigin xScale yScale***

Skaliert die Koordinaten aller durch *tagOrId* bezeichneten Items im Canvas-Koordinatenraum. *xOrigin* und *yOrigin* sind der Skalierungsursprung; *xScale* und *yScale* die Skalierungsfaktoren für x- bzw. y-Koordinaten (1.0 = keine Änderung). Für jeden Punkt wird die x-Koordinate so angepasst, dass der Abstand von *xOrigin* mit *xScale* multipliziert wird; analog für y. Rückgabe: leerer String.

Einige Items haben nur ein Koordinatenpaar (Text, Bilder, Fenster) — Skalierung verschiebt sie dann nur.

***pathName* scan *option args***

Implementiert Scanning auf Canvas. Zwei Formen je nach *option*:

***pathName* scan mark *x y***

Speichert *x*, *y* und die aktuelle Canvas-Ansicht; für spätere **scan dragto**-Befehle. Typischerweise bei Maustastendruck; *x* und *y* sind Mauskoordinaten. Rückgabe: leerer String.

***pathName* scan dragto *x y* ?gain?**

Berechnet die Differenz zwischen *x*, *y* (meist Mauskoordinaten) und den Argumenten des letzten **scan mark** für das Widget. Passt die Ansicht um *gain* mal die Koordinatendifferenz an (*gain* standardmäßig 10). Typischerweise bei Mausbewegung, um schnelles Ziehen des Canvas durch das Fenster zu simulieren. Rückgabe: leerer String.

***pathName* select *option* ?*tagOrId arg*?**

Manipuliert die Auswahl auf verschiedene Weise je nach *option*. Formen siehe unten. *tagOrId* muss ein Item mit Index- und Auswahl-Unterstützung bezeichnen; bei mehreren Items das erste mit dieser Unterstützung. *index* beschreibt eine Position in *tagOrId* gemäß Indizes oben.

***pathName* select adjust *tagOrId index***

Findet das Auswahlende in *tagOrId*, das dem Zeichen *index* am nächsten ist, und setzt dieses Ende auf *index* (einschließlich, aber nicht darüber hinaus). Das andere Ende wird Anker für künftige **select to**-Befehle. Liegt die Auswahl nicht in *tagOrId*, verhält sich der Befehl wie **select to**. Rückgabe: leerer String.

***pathName* select clear**

Löscht die Auswahl, wenn sie in diesem Widget liegt. Sonst keine Wirkung. Rückgabe: leerer String.

***pathName* select from *tagOrId index***

Setzt den Auswahlanker des Widgets unmittelbar vor das durch *index* bezeichnete Zeichen im durch *tagOrId* bezeichneten Item. Ändert die Auswahl nicht; setzt nur das feste Ende für künftige **select to**-Befehle. Rückgabe: leerer String.

***pathName* select item**

Gibt die ID des ausgewählten Items zurück, wenn die Auswahl in einem Item dieses Canvas liegt. Sonst leerer String.

***pathName* select to *tagOrId index***

Setzt die Auswahl auf die Zeichen von *tagOrId* zwischen Auswahlanker und *index*. Die neue Auswahl schließt das Zeichen *index* ein; das Ankerzeichen nur, wenn *index* größer oder gleich dem Anker ist. Der Anker ergibt sich aus dem letzten **select adjust** oder **select from** für dieses Widget. Liegt der Anker nicht in *tagOrId*, wird er auf dasselbe Zeichen wie *index* gesetzt. Rückgabe: leerer String.

***pathName* type *tagOrId***

Gibt den Typ des durch *tagOrId* bezeichneten Items zurück, z.B. **rectangle** oder **text**. Bei mehreren Items: Typ des ersten in der Anzeigeliste. Bezeichnet *tagOrId* keine Items: leerer String.

***pathName* xview ?*args*?**

Abfrage und Änderung der horizontalen Position der im Canvas-Fenster angezeigten Information. Formen:

***pathName* xview**

Liste mit zwei Elementen — reelle Brüche zwischen 0 und 1, die den sichtbaren horizontalen Bereich beschreiben. Z.B. .2 und .6: 20 % der Canvas-Fläche (laut **-scrollregion**) links außerhalb, mittlere 40 % sichtbar, 40 % rechts außerhalb. Dieselben Werte wie an Scrollbars über **-xscrollcommand**.

***pathName* xview moveto *fraction***

Passt die Ansicht so an, dass *fraction* der Gesamtbreite des Canvas links außerhalb liegt. *fraction* zwischen 0 und 1.

***pathName* xview scroll *number what***

Verschiebt die Ansicht links oder rechts gemäß *number* und *what*. *number* muss Ganzzahl oder Gleitkomma sein; bei Gleitkomma wird auf Ganzzahl gerundet (weg von 0). *what* muss **pages** oder **units** (oder Abkürzung) sein. Bei **pages** in Einheiten von neun Zehnteln der Fensterbreite. Negatives *number*: weiter links sichtbar; positives: weiter rechts. Bei **units** in Einheiten von **xScrollIncrement**, falls > 0, sonst ein Zehntel der Fensterbreite.

***pathName* yview ?*args*?**

Abfrage und Änderung der vertikalen Position der im Canvas-Fenster angezeigten Information. Formen:

***pathName* yview**

Liste mit zwei Elementen — Brüche zwischen 0 und 1 für den sichtbaren vertikalen Bereich. Z.B. .6 und 1.0: unterste 40 % der Canvas-Fläche (laut **-scrollregion**) sichtbar. Dieselben Werte wie an Scrollbars über **-yscrollcommand**.

***pathName* yview moveto *fraction***

Passt die Ansicht so an, dass *fraction* der Canvas-Fläche oberhalb außerhalb liegt. *fraction* zwischen 0 und 1.

***pathName* yview scroll *number what***

Verschiebt die Ansicht auf oder ab gemäß *number* und *what*. *number* wie bei **xview scroll**. *what* muss **pages** oder **units** sein. Bei **pages** in Einheiten von neun Zehnteln der Fensterhöhe. Negatives *number*: höher gelegene Information sichtbar; positives: tiefer gelegene. Bei **units** in Einheiten von **yScrollIncrement**, falls > 0, sonst ein Zehntel der Fensterhöhe.

## Übersicht der Item-Typen

Die folgenden Abschnitte beschreiben die von Canvas-Widgets unterstützten Item-Typen. Jeder Item-Typ ist durch zwei Dinge gekennzeichnet: die Form des **create**-Befehls zum Erzeugen von Instanzen des Typs und eine Menge Konfigurationsoptionen für Items dieses Typs, verwendbar in den Widget-Befehlen **create** und **itemconfigure**. Die meisten Items unterstützen weder Indexierung noch Auswahl noch die zugehörigen Befehle wie **index** und **insert**. Wo Items diese Funktionen unterstützen, ist es in den Beschreibungen unten ausdrücklich vermerkt. Derzeit bieten Text-, Linien- und Polygon-Items diese Unterstützung. Bei Linien und Polygonen dient die Indexierung der Manipulation der Item-Koordinaten.

### Gemeinsame Item-Optionen

Viele Items teilen einen gemeinsamen Optionssatz. Diese Optionen werden hier erläutert und von jedem Widget-Typ der Kürze halber nur referenziert.

**-anchor *anchorPos***

*anchorPos* legt fest, wie das Item relativ zum Positionierungspunkt platziert wird; zulässig sind alle von Tk_GetAnchor akzeptierten Formen. Ist *anchorPos* **center**, ist das Item auf dem Punkt zentriert; ist es **n**, wird es so gezeichnet, dass seine obere Mitte am Positionierungspunkt liegt. Standard: **center**.

**-dash *pattern***

...siehe nächstes...

**-activedash *pattern***

...siehe nächstes...

**-disableddash *pattern***

Diese Optionen legen Strichmuster für den normalen, aktiven und deaktivierten Zustand eines Items fest. *pattern* kann jede von Tk_GetDash akzeptierte Form haben. Fehlen die Strichoptionen, ist der Standard ein durchgezogener Umriss. Siehe Strichmuster.

**-dashoffset *offset***

Start-*offset* in Pixeln in das von **-dash** bereitgestellte Muster. **-dashoffset** wird ignoriert, wenn kein **-dash**-Muster gesetzt ist. *offset* kann jede im Abschnitt Koordinaten beschriebene Form haben.

**-fill *color***

...siehe nächstes...

**-activefill *color***

...siehe nächstes...

**-disabledfill *color***

Diese Optionen legen die Füllfarbe des Item-Bereichs in normalen, aktiven und deaktivierten

Zuständen fest. Es gilt die Even-Odd-Füllregel. *color* kann jede von Tk_GetColor akzeptierte Form haben. Bei Linien-Items: Farbe der gezeichneten Linie. Bei Text-Items: Vordergrundfarbe des Textes. Ist *color* ein leerer String (Standard für alle Canvas-Items außer Linie und Text), wird das Item nicht gefüllt.

**-outline *color***

...siehe nächstes...

**-activeoutline *color***

...siehe nächstes...

**-disabledoutline *color***

Diese Optionen legen die Umrissfarbe in normalen, aktiven und deaktivierten Zuständen fest. *color* kann jede von Tk_GetColor akzeptierte Form haben. Ist *color* ein leerer String, wird kein Umriss gezeichnet.

**-offset *offset***

Legt den Versatz von Stippelmuster fest. Der Wert kann die Form **x,y** oder *side* haben, wobei *side* **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw** oder **center** sein kann. Im ersten Fall ist der Ursprung der des Toplevel des aktuellen Fensters. Für den Canvas selbst und Canvas-Objekte ist der Ursprung der Canvas-Ursprung; **#** vor dem Koordinatenpaar bedeutet Toplevel-Ursprung. Bei Canvas-Objekten steuert **-offset** auch Stippelung. Bei Linien- und Polygon-Items kann ein Index angegeben werden, der den Stippelursprung mit einem Koordinatenpunkt verbindet. Stippelversätze werden *nur unter X11 unterstützt*; auf anderen Plattformen still ignoriert.

**-outlinestipple *bitmap***

...siehe nächstes...

**-activeoutlinestipple *bitmap***

...siehe nächstes...

**-disabledoutlinestipple *bitmap***

Diese Optionen legen Stippelmuster für den Umriss in normalen, aktiven und deaktivierten Zuständen fest. *bitmap* gibt das Muster in einer von Tk_GetBitmap akzeptierten Form an. Ohne **-outline** keine Wirkung. Ist *bitmap* leerer String (Standard), wird der Umriss durchgezogen gezeichnet. *Stippel werden auf Plattformen ohne X11 als Zeichen-API nur eingeschränkt unterstützt.*

**-outlineoffset *offset***

Versatz des Stippelmusters für Umrisse, analog zur Steuerung von Füll-Stippeln durch **-stipple**. (Syntax von *offset* siehe **-offset**.)

**-stipple *bitmap***

...siehe nächstes...

**-activestipple *bitmap***

...siehe nächstes...

**-disabledstipple *bitmap***

Diese Optionen legen Stippelmuster für die Füllung in normalen, aktiven und deaktivierten Zuständen fest. *bitmap* in einer von Tk_GetBitmap akzeptierten Form. Ohne **-fill** keine Wirkung. Ist *bitmap* leerer String (Standard), erfolgt Füllung durchgezogen. Bei Text-Items betrifft es den eigentlichen Text. *Stippel werden auf Plattformen ohne X11 als Zeichen-API nur eingeschränkt unterstützt.*

**-state *state***

Ermöglicht einem Item, die globale *state*-Option des Canvas zu überschreiben. Dieselben

Werte: *normal*, *disabled* oder *hidden*.

**-tags *tagList***

Legt Tags für das Item fest. *tagList* ist eine Liste von Tag-Namen, die bestehende Tags ersetzen. *tagList* kann leer sein.

**-width *outlineWidth***

...siehe nächstes...

**-activewidth *outlineWidth***

...siehe nächstes...

**-disabledwidth *outlineWidth***

Diese Optionen legen die Umrissbreite um den Item-Bereich in normalen, aktiven und deaktivierten Zuständen fest. *outlineWidth* in einer im Abschnitt Koordinaten beschriebenen Form. Ist **-outline** leerer String, keine Wirkung. Standard 1.0. Bei Bögen werden breite Umrisse zentriert auf die Kanten des Bogenbereichs gezeichnet.

## Standard-Item-Typen

### Bogen-Items

Items vom Typ **arc** erscheinen als bogenförmige Bereiche. Ein Bogen ist ein Abschnitt einer Ovalform, begrenzt durch zwei Winkel (über **-start** und **-extent** oder **-height**) und auf eine von mehreren Arten dargestellt (**-style**). Bögen werden mit Widget-Befehlen folgender Form erstellt:

```
pathName create arc x1 y1 x2 y2 ?option value ...?
pathName create arc coordList ?option value ...?
```

Die Argumente *x1*, *y1*, *x2* und *y2* oder *coordList* geben die Koordinaten zweier diagonal gegenüberliegender Ecken eines Rechtecks an, das die den Bogen definierende Ovalform umschließt (außer bei **-height** — siehe unten).

Nach den Koordinaten können beliebig viele *option-value*-Paare folgen, die Konfigurationsoptionen setzen. Dieselben Paare können in **itemconfigure**-Widget-Befehlen verwendet werden. Ein Bogen-Item wird aktuelles Item, wenn der Mauszeiger über einen gezeichneten Teil liegt oder (bei vollständiger Transparenz) über einen Teil, der gezeichnet würde, wenn **-fill** und **-outline** nicht leer wären.

Folgende Standardoptionen unterstützen Bögen:

**-dash -activedash**
**-disableddash -dashoffset**
**-fill -activefill**
**-disabledfill -offset**
**-outline -activeoutline**
**-disabledoutline -outlineoffset**
**-outlinestipple -activeoutlinestipple**
**-disabledoutlinestipple -stipple**
**-activestipple -disabledstipple**
**-state -tags**
**-width -activewidth**
**-disabledwidth**

Zusätzliche Optionen für Bögen:

**-extent *degrees***

Größe des Winkelbereichs des Bogens. Der Bereich erstreckt sich *degrees* Grad gegen den Uhrzeigersinn ab dem Startwinkel von **-start**. *degrees* kann negativ sein. Ist der Wert größer als 360 oder kleiner als -360, wird *degrees* modulo 360 als Ausdehnung verwendet.

**-start *degrees***

Beginn des Winkelbereichs. *degrees* in Grad gegen den Uhrzeigersinn ab der 3-Uhr-Position; positiv oder negativ.

**-height *distance***

Abkürzung zum Erzeugen eines Kreisbogensegments durch den Abstand des Bogenmittelpunkts von seiner Sehne. Dann werden die Koordinaten als Start- und Endkoordinaten der Sehne interpretiert; **-start** und **-extent** werden ignoriert. Bedeutung von *distance*:

*distance* > 0 erzeugt einen Bogen im Uhrzeigersinn

*distance* < 0 gegen den Uhrzeigersinn

*distance* = 0 wie ohne diese Option

Für einen bestimmten Radius *r* gilt:

*distance* = *r* ± sqrt(*r***2 - (chordLength / 2)**2)

Minuszeichen für den kleineren, Pluszeichen für den größeren Bogen.

**itemcget -height** liefert stets 0, damit Introspektionscode einfach bleibt.

**-style *type***

Zeichenweise des Bogens. Bei **pieslice** (Standard) wird der Bereich durch einen Abschnitt des Ovalumfangs plus zwei Linien vom Ovalzentrum zu den Endpunkten des Abschnitts definiert. Bei **chord** ein Umfangsabschnitt plus eine Verbindungslinie zwischen den Endpunkten. Bei **arc** nur der Umfangsabschnitt; dann wird **-fill** ignoriert.

## Bitmap-Items

Items vom Typ **bitmap** erscheinen als Bilder mit zwei Farben, Vorder- und Hintergrund. Bitmaps werden mit Widget-Befehlen folgender Form erstellt:

```
pathName create bitmap x y ?option value ...?
pathName create bitmap coordList ?option value ...?
```

Die Argumente *x* und *y* oder *coordList* (zwei Elemente) geben die Koordinaten eines Punkts zur Positionierung des Bitmaps an, gesteuert durch **-anchor**. Nach den Koordinaten können *option-value*-Paare folgen; dieselben Paare in **itemconfigure**. Ein Bitmap-Item wird aktuelles Item, wenn der Mauszeiger über einen Teil seiner Bounding-Box liegt.

Folgende Standardoptionen unterstützen Bitmaps:

**-anchor -state**
**-tags**

Zusätzliche Optionen für Bitmaps:

**-background *color***
  ...siehe nächstes...

**-activebackground *color***
  ...siehe nächstes...

**-disabledbackground *color***
  Farbe für die "0"-Pixel des Bitmaps in normalen, aktiven und deaktivierten Zuständen. *color* kann jede von Tk_GetColor akzeptierte Form haben. Ohne Angabe oder bei leerem String wird an Stellen mit 0-Pixeln nichts gezeichnet — transparenter Effekt.

**-bitmap *bitmap***
  ...siehe nächstes...

**-activebitmap *bitmap***
  ...siehe nächstes...

**-disabledbitmap *bitmap***
  Diese Optionen legen die in normalen, aktiven und deaktivierten Zuständen anzuzeigenden Bitmaps fest. *bitmap* kann jede von Tk_GetBitmap akzeptierte Form haben.

**-foreground *color***
  ...siehe nächstes...

**-activeforeground *color***
  ...siehe nächstes...

**-disabledforeground *color***
  Diese Optionen legen die Farbe für die "1"-Pixel des Bitmaps in normalen, aktiven und deaktivierten Zuständen fest. *color* kann jede von Tk_GetColor akzeptierte Form haben.

### Bild-Items

Items vom Typ image zeigen Bilder auf einem Canvas an. Bilder werden mit Widget-Befehlen folgender Form erstellt:

```
pathName create image x y ?option value ...?
pathName create image coordList ?option value ...?
```

Die Argumente *x* und *y* oder *coordList* geben den Positionierungspunkt an (**-anchor**). Nach den Koordinaten *option-value*-Paare; dieselben in **itemconfigure**. Ein Bild-Item wird aktuelles Item über der Bounding-Box.

Folgende Standardoptionen unterstützen Bilder:

**-anchor -state**
**-tags**

Zusätzliche Optionen für Bilder:

**-image *name***
  ...siehe nächstes...

**-activeimage *name***
  ...siehe nächstes...

**-disabledimage *name***
  Namen der Bilder in normalen, aktiven und deaktivierten Zuständen. Das Bild muss zuvor mit image create erstellt worden sein.

## Linien-Items

Items vom Typ **line** erscheinen als eine oder mehrere verbundene Liniensegmente oder Kurven. Linien-Items unterstützen Koordinaten-Indexierung mit **dchars**, **index** und **insert**. Linien werden mit Widget-Befehlen folgender Form erstellt:

```
pathName create line x1 y1... xn yn ?option value ...?
pathName create line coordList ?option value ...?
```

Die Argumente *x1* bis *yn* oder *coordList* geben Koordinaten für zwei oder mehr Punkte verbundener Segmente. Danach *option-value*-Paare; dieselben in **itemconfigure**. Eine Linie ist aktuelles Item, wenn der Mauszeiger über einem Segment liegt — gezeichnet oder nicht, geglättet oder nicht.

Folgende Standardoptionen unterstützen Linien:

**-dash -activedash**
**-disableddash -dashoffset**
**-fill -activefill**
**-disabledfill -stipple**
**-activestipple -disabledstipple**
**-state -tags**
**-width -activewidth**
**-disabledwidth**

Zusätzliche Optionen für Linien:

**-arrow *where***

Gibt an, ob Pfeilspitzen an einem oder beiden Linienenden gezeichnet werden. *where* muss **none**, **first**, **last** oder **both** sein. Standard **none**. Beim Zeichnen einer Pfeilspitze passt Tk den Linienendpunkt so an, dass die Linie am Pfeilhals endet, nicht an der Spitze. Das kann **Leave** auslösen, wenn die Maus am Linienende schwebt; beim Entfernen umgekehrt **Enter**.

**-arrowshape *shape***

Form der Pfeilspitzen. *shape* ist eine Liste mit drei Elementen — Abstände in Formen aus Koordinaten. Erstes Element: Abstand entlang der Linie vom Pfeilhals zur Spitze. Zweites: von den Hinterpunkten zur Spitze. Drittes: vom Linienrand zu den Hinterpunkten. Ohne Angabe wählt Tk eine "vernünftige" Form.

**-capstyle *style***

Endenzeichnung an Linienenden. *style* kann jede von Tk_GetCapStyle akzeptierte Form haben (**butt**, **projecting**, **round**). Standard **butt**. Bei Pfeilspitzen wird der Cap-Stil ignoriert.

**-joinstyle *style***

Verbindungszeichnung an Linienecken. *style* kann jede von Tk_GetJoinStyle akzeptierte Form haben (**bevel**, **miter**, **round**). Standard **round**. Bei nur zwei Punkten irrelevant.

**-smooth *smoothMethod***

*smoothMethod* muss eine von Tcl_GetBoolean akzeptierte Form oder eine Glättungsmethode sein. Im Kern nur **true** und **raw** (**bezier** Alias für **true**), weitere zur Laufzeit. Bei false oder leerem String keine Glättung; bei true **true**-Glättung. Bei **true** wird die Linie als Kurve mit quadratischen Splines gezeichnet: je ein Spline pro benachbartes Segmentpaar. Gerade Segmente durch Duplizieren der Endpunkte. Bei **raw** ist jedes erste (und jedes dritte) Koordinatenpaar ein Knotenpunkt einer kubischen Bézier-Kurve, die anderen Kontrollpunkte.

Gerade Segmente durch gleiche Kontroll- und Knotenpunkte. Ist der letzte Punkt nur Kontrollpunkt, wird er ein- oder zweimal wiederholt.

**-splinesteps *number***

Gewünschter Glättegrad für Kurven: jeder Spline wird mit *number* Liniensegmenten approximiert. Ignoriert, wenn **-smooth** nicht true oder **raw** ist.

### Oval-Items

Items vom Typ **oval** erscheinen als kreisförmige oder ovale Bereiche. Jedes Oval kann Umriss, Füllung oder beides haben. Ovale werden mit Widget-Befehlen folgender Form erstellt:

```
pathName create oval x1 y1 x2 y2 ?option value ...?
pathName create oval coordList ?option value ...?
```

Die Argumente *x1*, *y1*, *x2*, *y2* oder *coordList* geben diagonal gegenüberliegende Ecken des umschließenden Rechtecks an. Das Oval schließt die obere und linke Kante ein, nicht die untere und rechte. Ist das Rechteck quadratisch, ist das Oval kreisförmig; sonst oval. Danach *option-value*-Paare; dieselben in **itemconfigure**. Aktuelles Item bei Maus über gezeichnetem Teil oder (transparent) über Teil, der bei nicht leeren **-fill** und **-outline** gezeichnet würde.

Folgende Standardoptionen unterstützen Ovale:

**-dash -activedash**
**-disableddash -dashoffset**
**-fill -activefill**
**-disabledfill -offset**
**-outline -activeoutline**
**-disabledoutline -outlineoffset**
**-outlinestipple -activeoutlinestipple**
**-disabledoutlinestipple -stipple**
**-activestipple -disabledstipple**
**-state -tags**
**-width -activewidth**
**-disabledwidth**

Es gibt keine oval-spezifischen Optionen.

### Polygon-Items

Items vom Typ **polygon** erscheinen als polygonale oder gekrümmte gefüllte Bereiche. Polygon-Items unterstützen Koordinaten-Indexierung mit **dchars**, **index** und **insert**. Polygone werden mit Widget-Befehlen folgender Form erstellt:

```
pathName create polygon x1 y1 ... xn yn ?option value ...?
pathName create polygon coordList ?option value ...?
```

Die Argumente *x1* bis *yn* oder *coordList* geben Koordinaten für drei oder mehr Eckpunkte. Der erste Punkt soll nicht als letzter wiederholt werden; Tk schließt den Umfang automatisch. Danach *option-value*-Paare; dieselben in **itemconfigure**. Aktuelles Item bei Maus über beliebigem Teilstück — gezeichnet oder nicht, Umriss geglättet oder nicht.

Folgende Standardoptionen unterstützen Polygone:

**-dash -activedash**

**-disableddash -dashoffset**
**-fill -activefill**
**-disabledfill -offset**
**-outline -activeoutline**
**-disabledoutline -outlineoffset**
**-outlinestipple -activeoutlinestipple**
**-disabledoutlinestipple -stipple**
**-activestipple -disabledstipple**
**-state -tags**
**-width -activewidth**
**-disabledwidth**

Zusätzliche Optionen für Polygone:

**-joinstyle *style***

Verbindungszeichnung an Umriss-Ecken. *style* kann jede von Tk_GetJoinStyle akzeptierte Form haben. Standard **round**.

**-smooth *boolean***

*boolean* muss eine von Tcl_GetBoolean akzeptierte Form oder Glättungsmethode sein. Im Kern nur **true** und **raw** (**bezier** Alias für **true**). Bei false oder leerem String keine Glättung. Bei **true** quadratische Splines wie bei Linien. Bei **raw** kubische Bézier-Knoten und Kontrollpunkte wie bei Linien; fehlt am Ende das zweite Kontrollpunkt-Paar, wird der Punkt ein- oder zweimal wiederholt.

**-splinesteps *number***

Gewünschter Glättegrad für Kurven: jeder Spline wird mit *number* Liniensegmenten approximiert. Ignoriert, wenn **-smooth** nicht true oder **raw** ist.

Polygon-Items unterscheiden sich von Rechtecken, Ovalen und Bögen: Innenpunkte gelten als "innerhalb" (z.B. für **find closest** und **find overlapping**), auch ohne Füllung. Bei den meisten anderen Typen nur bei Füllung oder ohne Füllung und Umriss. Für ein ungefülltes Polygon, dessen Innen nicht als "innerhalb" gilt, ein Linien-Item verwenden.

### Rechteck-Items

Items vom Typ **rectangle** erscheinen als rechteckige Bereiche mit optionalem Umriss, Füllung oder beidem. Erstellung mit Widget-Befehlen folgender Form:

```
pathName create rectangle x1 y1 x2 y2 ?option value ...?
pathName create rectangle coordList ?option value ...?
```

Die Argumente *x1*, *y1*, *x2*, *y2* oder *coordList* (vier Elemente) geben diagonal gegenüberliegende Ecken an (obere und linke Kante eingeschlossen, untere und rechte nicht). Danach *option-value* -Paare; dieselben in **itemconfigure**. Aktuelles Item wie bei Ovalen.

Folgende Standardoptionen unterstützen Rechtecke:

**-dash -activedash**
**-disableddash -dashoffset**
**-fill -activefill**
**-disabledfill -offset**
**-outline -activeoutline**
**-disabledoutline -outlineoffset**

**-outlinestipple -activeoutlinestipple**
**-disabledoutlinestipple -stipple**
**-activestipple -disabledstipple**
**-state -tags**
**-width -activewidth**
**-disabledwidth**

Es gibt keine rechteck-spezifischen Optionen.

### Text-Items

Ein Text-Item zeigt eine Zeichenkette in einer oder mehreren Zeilen. Text-Items unterstützen Indexierung, Bearbeitung und Auswahl über **dchars**, **focus**, **icursor**, **index**, **insert** und **select**. Erstellung mit Widget-Befehlen folgender Form:

```
pathName create text x y ?option value ...?
pathName create text coordList ?option value ...?
```

Die Argumente *x* und *y* oder *coordList* (zwei Elemente) positionieren den Text (siehe Optionen unten). Danach *option*-*value*-Paare; dieselben in **itemconfigure**. Aktuelles Item über der Bounding-Box.

Folgende Standardoptionen unterstützen Text-Items:

**-anchor -fill**
**-activefill -disabledfill**
**-stipple -activestipple**
**-disabledstipple -state**
**-tags**

Zusätzliche Optionen für Text-Items:

**-angle *rotationDegrees***
*rotationDegrees* gibt die Drehung des Textes gegen den Uhrzeigersinn um den Positionierungspunkt in Grad (0.0 bis 360.0). Bei **90** wird der Text von unten nach oben gezeichnet. Standard **0.0**.

**-font *fontName***
Schrift für das Text-Item. *fontName* jeder von Tk_GetFont akzeptierte String. Standard: systemspezifische Schrift.

**-justify *how***
Ausrichtung des Textes im Begrenzungsbereich. *how* muss **left**, **right** oder **center** sein. Nur relevant bei mehrzeiligem Text. Standard **left**.

**-text *string***
*string* gibt die anzuzeigenden Zeichen an. Zeilenumbrüche durch Newline. Änderbar mit **insert** und **delete**. Standard: leerer String.

**-underline *number***
Index des zu unterstreichenden Zeichens. 0 = erstes Zeichen, 1 = nächstes usw. -1 = keine Unterstreichung (für gesamten Text passende Schrift verwenden).

**-width *lineLength***
Maximale Zeilenlänge in Formen aus Koordinaten. Bei 0 (Standard) nur Umbruch bei Newline. Sonst wird vor einem Leerzeichen umgebrochen, wenn die Zeile länger als *lineLength* wäre;

das Leerzeichen gilt wie Newline.

### Fenster-Items

Items vom Typ **window** zeigen ein bestimmtes Fenster an einer Position auf dem Canvas. Erstellung mit Widget-Befehlen folgender Form:

```
pathName create window x y ?option value ...?
pathName create window coordList ?option value ...?
```

Die Argumente *x* und *y* oder *coordList* (zwei Elemente) positionieren das Fenster (**-anchor**). Danach *option-value*-Paare; dieselben in **itemconfigure**. Theoretisch aktuelles Item über der Bounding-Box; praktisch oft nicht, weil der Mauszeiger dann nicht mehr über dem Canvas liegt.

Folgende Standardoptionen unterstützen Fenster-Items:

**-anchor -state**
**-tags**

Zusätzliche Optionen für Fenster-Items:

**-height *pixels***
Höhe des Item-Fensters. *pixels* in Formen aus Koordinaten. Ohne Angabe oder bei 0 die intern angeforderte Höhe.

**-width *pixels***
Breite des Item-Fensters. *pixels* in Formen aus Koordinaten. Ohne Angabe oder bei 0 die intern angeforderte Breite.

**-window *pathName***
Mit diesem Item zu verknüpfendes Fenster. *pathName* muss Kind des Canvas oder eines Vorfahren sein, nicht Toplevel.

Aufgrund von Fensterverwaltungsbeschränkungen können andere grafische Items (Linien, Bilder) nicht über Fenster-Items gezeichnet werden. Ein Fenster-Item verdeckt stets überlappende Grafik, unabhängig von der Anzeigeliste. Fenster-Items werden nicht am Canvas-Rand beschnitten, sondern am Eltern-Widget des in **-window** angegebenen Fensters; ist das der Canvas, kann das Fenster-Item über den Canvas-Rand hinausragen.

## Anwendungsdefinierte Item-Typen

Einzelne Anwendungen können mit C-Code neue Item-Typen für Canvas-Widgets definieren. Siehe Tk_CreateItemType.

## Bindings

In der aktuellen Implementierung erhalten neue Canvas kein Standardverhalten: Sie müssen explizite Tcl-Befehle ausführen, um dem Canvas sein Verhalten zu geben.

## Danksagungen

Das Canvas-Widget von Tk übernimmt Ideen aus Joel Bartletts Programm *ezd*. *Ezd* bietet strukturierte Grafik in einer Scheme-Umgebung und ging Canvas um ein bis zwei Jahre voraus. Seine einfachen Mechanismen zum Platzieren und Animieren grafischer Objekte inspirierten die Canvas-Funktionen.

*Copyright (c) 1992-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

*Copyright (c) 1997-1999 Scriptics Corporation.*

# checkbutton

*Stichwörter:* checkbutton, widget

## Name

checkbutton - Erstellt und manipuliert 'Checkbutton'-Widgets für boolesche Auswahl

## Übersicht

**checkbutton** *pathName* ?*options*?

## Standardoptionen

**-activebackground**, **-disabledforeground**, **-padx**, **-activeforeground**, **-font**, **-pady**, **-anchor**, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-takefocus**, **-bitmap**, **-highlightcolor**, **-text**, **-borderwidth**, **-highlightthickness**, **-textvariable**, **-compound**, **-image**, **-underline**, **-cursor**, **-justify**, **-wraplength**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-command (command/Command)**

Legt einen Tcl-Befehl fest, der mit dem Button verknüpft wird. Dieser Befehl wird typischerweise aufgerufen, wenn Maustaste 1 über dem Button-Fenster losgelassen wird. Die globale Variable des Buttons (Option **-variable**) wird vor dem Aufruf des Befehls aktualisiert.

**-height (height/Height)**

Legt eine gewünschte Höhe für den Button fest. Wenn ein Bild oder Bitmap im Button angezeigt wird, ist der Wert in Bildschirmeinheiten (d.h. alle von Tk_GetPixels akzeptierten Formen); für Text ist es in Textzeilen. Wenn diese Option nicht angegeben ist, wird die gewünschte Höhe des Buttons aus der Größe des angezeigten Bildes, Bitmaps oder Textes berechnet.

**-indicatoron (indicatorOn/IndicatorOn)**

Legt fest, ob der Indikator gezeichnet werden soll. Muss ein gültiger boolescher Wert sein. Wenn false, wird die Option **-relief** ignoriert und das Relief des Widgets ist stets eingesenkt, wenn das Widget ausgewählt ist, und sonst erhaben.

**-offrelief (offRelief/OffRelief)**

Legt das Relief für den Checkbutton fest, wenn der Indikator nicht gezeichnet wird und der Checkbutton ausgeschaltet ist. Der Standardwert ist "raised". Durch Setzen dieser Option auf "flat" und **-indicatoron** auf false sowie **-overrelief** auf "raised" entsteht ein flacher Button, der bei Maus-Over erhaben wird und beim Aktivieren eingedrückt ist. Dies ist typisches Verhalten der Fett-, Kursiv- und Unterstrichen-Checkbuttons in der Symbolleiste einer Textverarbeitung.

**-offvalue (offValue/Value)**

Gibt den Wert an, der in der zugehörigen Variable des Buttons gespeichert wird, wenn dieser Button abgewählt wird. Standardwert ist "0".

**-onvalue (onValue/Value)**

Gibt den Wert an, der in der zugehörigen Variable des Buttons gespeichert wird, wenn dieser Button ausgewählt wird. Standardwert ist "1".

**-overrelief (overRelief/OverRelief)**

Legt ein alternatives Relief für den Checkbutton fest, das verwendet wird, wenn der Mauszeiger über dem Widget ist. Diese Option kann verwendet werden, um Toolbar-Buttons zu erstellen, indem **-relief flat -overrelief raised** konfiguriert wird. Wenn der Wert dieser Option der leere String ist, wird kein alternatives Relief verwendet, wenn der Mauszeiger über dem Checkbutton ist. Der leere String ist der Standardwert.

**-selectcolor (selectColor/Background)**

Legt eine Hintergrundfarbe fest, die verwendet wird, wenn der Button ausgewählt ist. Wenn **indicatorOn** true ist, wird die Farbe als Hintergrund für den Indikator verwendet, unabhängig vom Auswahlzustand. Wenn **indicatorOn** false ist, wird diese Farbe als Hintergrund für das gesamte Widget anstelle von **background** oder **activeBackground** verwendet, sobald das Widget ausgewählt ist. Wenn als leerer String angegeben, wird keine besondere Farbe für die Anzeige im ausgewählten Zustand verwendet.

**-selectimage (selectImage/SelectImage)**

Legt ein Bild fest, das (anstelle der Option **-image**) angezeigt wird, wenn der Checkbutton ausgewählt ist. Diese Option wird ignoriert, sofern die Option **-image** nicht angegeben wurde.

**-state (state/State)**

Legt einen von drei Zuständen für den Checkbutton fest: **normal**, **active** oder **disabled**. Im normalen Zustand wird der Checkbutton mit den Optionen **-foreground** und **-background** angezeigt. Der aktive Zustand wird typischerweise verwendet, wenn der Zeiger über dem Checkbutton ist. Im aktiven Zustand wird der Checkbutton mit den Optionen **-activeforeground** und **-activebackground** angezeigt. Der deaktivierte Zustand bedeutet, dass der Checkbutton unempfindlich sein sollte: Die Standard-Bindings weigern sich, das Widget zu aktivieren und ignorieren Mausklicks. In diesem Zustand bestimmen die Optionen **-disabledforeground** und **-background**, wie der Checkbutton angezeigt wird.

**-tristateimage (tristateImage/TristateImage)**

Legt ein Bild fest, das (anstelle der Option **-image**) angezeigt wird, wenn der Checkbutton im Drei-Zustand-Modus ist. Diese Option wird ignoriert, sofern die Option **-image** nicht angegeben wurde.

**-tristatevalue (tristateValue/Value)**

Gibt den Wert an, der bewirkt, dass der Checkbutton die Mehrfachauswahl anzeigt, auch bekannt als Drei-Zustand-Modus. Standardwert ist "".

**-variable (**variable**/Variable)**

Gibt den Namen einer globalen Variable an, die gesetzt wird, um anzuzeigen, ob dieser Button ausgewählt ist oder nicht. Standardwert ist der Name des Buttons innerhalb seines Eltern-Elements (d.h. das letzte Element des Pfadnamens des Button-Fensters).

**-width (width/Width)**

Legt eine gewünschte Breite für den Button fest. Wenn ein Bild oder Bitmap im Button angezeigt wird, ist der Wert in Bildschirmeinheiten (d.h. alle von Tk_GetPixels akzeptierten Formen); für Text ist es in Zeichen. Wenn diese Option nicht angegeben ist, wird die gewünschte Breite des Buttons aus der Größe des angezeigten Bildes, Bitmaps oder Textes berechnet.

## Beschreibung

Der Befehl **checkbutton** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Checkbutton-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des Checkbuttons wie Farben, Schriftart, Text und anfängliches Relief zu konfigurieren. Der Befehl **checkbutton** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Ein Checkbutton ist ein Widget, das einen Text-String, ein Bitmap oder ein Bild sowie ein Quadrat namens *Indikator* anzeigt. Wenn Text angezeigt wird, muss er in einer einzigen Schriftart sein, kann aber mehrere Zeilen auf dem Bildschirm einnehmen (wenn er Zeilenumbrüche enthält oder wenn ein Umbruch wegen der Option **-wraplength** erfolgt), und eines der Zeichen kann optional mit der Option **-underline** unterstrichen werden. Ein Checkbutton hat das gesamte Verhalten eines einfachen Buttons, einschließlich Folgendem: Er kann sich je nach Option **-state** auf drei verschiedene Arten darstellen; er kann erhaben, eingesenkt oder flach dargestellt werden; er kann blinken; und er ruft einen Tcl-Befehl auf, wenn Maustaste 1 über dem Checkbutton geklickt wird.

Darüber hinaus können Checkbuttons *ausgewählt* werden. Wenn ein Checkbutton ausgewählt ist, wird der Indikator normalerweise mit ausgewähltem Erscheinungsbild gezeichnet, und eine mit dem Checkbutton verknüpfte Tcl-Variable wird auf einen bestimmten Wert gesetzt (normalerweise 1). Der Indikator wird mit einem Häkchen darin gezeichnet. Wenn der Checkbutton nicht ausgewählt ist, wird der Indikator mit nicht ausgewähltem Erscheinungsbild gezeichnet, und die zugehörige Variable wird auf einen anderen Wert gesetzt (typischerweise 0). Der Indikator wird ohne Häkchen darin gezeichnet. Im Sonderfall, in dem die Variable (falls angegeben) einen Wert hat, der mit dem **-tristatevalue** übereinstimmt, wird der Indikator mit Drei-Zustand-Erscheinungsbild gezeichnet und befindet sich im Drei-Zustand-Modus, der gemischte oder mehrere Werte anzeigt. (Dies wird verwendet, wenn das Kontrollkästchen den Zustand mehrerer Elemente repräsentiert.) Der Indikator wird plattformabhängig gezeichnet. Unter Unix und Windows wird der Hintergrund im Inneren des Kästchens "ausgegraut". Unter Mac wird der Indikator mit einem Strich darin gezeichnet. Standardmäßig ist der Name der mit einem Checkbutton verknüpften Variable derselbe wie der *name*, der zum Erstellen des Checkbuttons verwendet wurde. Der Variablenname sowie die darin gespeicherten "on"-, "off"- und "tristate"-Werte können mit Optionen in der Befehlszeile oder in der Optionsdatenbank geändert werden. Konfigurationsoptionen können auch verwendet werden, um die Art der Indikatoranzeige zu ändern (oder ob er überhaupt angezeigt wird). Standardmäßig ist ein Checkbutton so konfiguriert, dass er sich bei abwechselnden Button-Klicks selbst aus- und abwählt. Zusätzlich überwacht jeder Checkbutton seine zugehörige Variable und wählt sich automatisch ab bzw. an, wenn sich der Wert der Variable zu den "on"-, "off"- und "tristate"-Werten des Buttons ändert.

## Widget-Befehl

Der Befehl **checkbutton** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Checkbutton-Widgets möglich:

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **checkbutton** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option-value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **checkbutton** akzeptierten Werte haben.

***pathName* deselect**

Wählt den Checkbutton ab und setzt die zugehörige Variable auf ihren "off"-Wert.

***pathName* flash**

Lässt den Checkbutton blinken. Dies wird erreicht, indem der Checkbutton mehrmals neu angezeigt wird und dabei zwischen aktiven und normalen Farben gewechselt wird. Am Ende des Blinkens bleibt der Checkbutton im gleichen normal/aktiven Zustand wie beim Aufruf des Befehls. Dieser Befehl wird ignoriert, wenn der Zustand des Checkbuttons **disabled** ist.

***pathName* invoke**

Bewirkt genau das, was passiert wäre, wenn der Benutzer den Checkbutton mit der Maus aufgerufen hätte: Schaltet den Auswahlzustand des Buttons um und ruft den mit dem Checkbutton verknüpften Tcl-Befehl auf, falls einer vorhanden ist. Der Rückgabewert ist der Rückgabewert des Tcl-Befehls oder ein leerer String, wenn kein Befehl mit dem Checkbutton verknüpft ist. Dieser Befehl wird ignoriert, wenn der Zustand des Checkbuttons **disabled** ist.

***pathName* select**

Wählt den Checkbutton aus und setzt die zugehörige Variable auf ihren "on"-Wert.

***pathName* toggle**

Schaltet den Auswahlzustand des Buttons um, zeigt ihn neu an und ändert die zugehörige Variable entsprechend dem neuen Zustand.

## Standard-Bindings

Tk erstellt automatisch Klassen-Bindings für Checkbuttons, die ihnen folgendes Standardverhalten geben:

1. Auf Unix-Systemen wird ein Checkbutton aktiviert, wenn die Maus über ihn fährt, und deaktiviert, wenn die Maus den Checkbutton verlässt. Auf Mac- und Windows-Systemen wird der Button aktiviert, wenn Maustaste 1 über einem Checkbutton gedrückt wird und der Mauszeiger sich innerhalb des Buttons befindet, und deaktiviert, wenn der Mauszeiger den Button verlässt.
2. Wenn Maustaste 1 über einem Checkbutton gedrückt wird, wird er aufgerufen (sein Auswahlzustand schaltet um und der mit dem Button verknüpfte Befehl wird aufgerufen, falls einer vorhanden ist).
3. Wenn ein Checkbutton den Eingabefokus hat, bewirkt die Leertaste, dass der Checkbutton aufgerufen wird. Unter Windows gibt es zusätzliche Tasten-Bindings; Plus (**+**) und Gleich (**=**)

wählen den Button aus, und Minus (**-**) wählt ihn ab.

Wenn der Zustand des Checkbuttons **disabled** ist, tritt keine der obigen Aktionen auf: Der Checkbutton reagiert überhaupt nicht.

Das Verhalten von Checkbuttons kann geändert werden, indem neue Bindings für einzelne Widgets definiert werden oder indem die Klassen-Bindings neu definiert werden.

### Beispiele

Dieses Beispiel zeigt eine Gruppe unabhängiger Checkbuttons.

```
labelframe .lbl -text "Steps:"
checkbutton .c1 -text Lights  -variable lights
checkbutton .c2 -text Cameras -variable cameras
checkbutton .c3 -text Action! -variable action
pack .c1 .c2 .c3 -in .lbl
pack .lbl
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## chooseColor

*Stichwörter:* chooseColor, color, color selection, dialog

### Name

tk_chooseColor - Zeigt einen Dialog zur Farbauswahl an

### Übersicht

**tk_chooseColor** ?*option value* ...?

### Beschreibung

Die Prozedur **tk_chooseColor** zeigt einen Dialog an, in dem der Benutzer eine Farbe auswählen kann. Folgende *option*-*value*-Paare sind als Befehlszeilenargumente möglich:

**-initialcolor *color***

Legt fest, welche Farbe beim Öffnen des Farbdialogs angezeigt wird. *color* muss in einer für die Funktion Tk_GetColor akzeptablen Form vorliegen.

**-parent *window***

Macht *window* zum logischen Elternfenster des Farbdialogs. Der Farbdialog wird über seinem Elternfenster angezeigt.

**-title *titleString***

Legt eine Zeichenkette fest, die als Titel des Dialogfelds angezeigt wird. Wird diese Option nicht angegeben, wird ein Standardtitel angezeigt.

Wenn der Benutzer eine Farbe auswählt, gibt **tk_chooseColor** den Namen der Farbe in einer für Tk_GetColor akzeptablen Form zurück. Bricht der Benutzer den Vorgang ab, gibt der Befehl eine leere Zeichenkette zurück.

### Beispiel

```
button .b -bg [tk_chooseColor -initialcolor gray -title "Choose color"]
```

*Copyright (c) 1996 Sun Microsystems, Inc.*

# chooseDirectory

*Stichwörter:* chooseDirectory, directory, selection, dialog, platform-specific

## Name

tk_chooseDirectory - Zeigt einen Dialog zur Verzeichnisauswahl an

## Übersicht

**tk_chooseDirectory** ?*option value* ...?

## Beschreibung

Die Prozedur **tk_chooseDirectory** zeigt einen Dialog an, in dem der Benutzer ein Verzeichnis auswählen kann. Folgende *option-value*-Paare sind als Befehlszeilenargumente möglich:

**-command *string***

Legt das Präfix eines Tcl-Befehls fest, der ausgeführt wird, wenn der Benutzer den Dialog nach einer Auswahl schließt. Dieser Rückruf wird nicht aufgerufen, wenn der Benutzer den Dialog abbricht. Der tatsächliche Befehl besteht aus *string*, gefolgt von einem Leerzeichen und dem vom Benutzer im Dialog gewählten Wert. Nur unter macOS verfügbar.

**-initialdir *dirname***

Legt fest, dass beim Öffnen des Dialogs die Verzeichnisse in *directory* angezeigt werden. Wird dieser Parameter nicht angegeben, ist das Anfangsverzeichnis auf Nicht-Windows-Systemen und auf Windows vor Vista das aktuelle Arbeitsverzeichnis. Unter Vista und neuer ist es das zuletzt vom Benutzer für die Anwendung gewählte Verzeichnis. Gibt der Parameter einen relativen Pfad an, wandelt der Rückgabewert diesen in einen absoluten Pfad um.

**-message *string***

Legt eine Meldung fest, die im Client-Bereich des Dialogs angezeigt wird. Nur unter macOS verfügbar.

**-mustexist *boolean***

Legt fest, ob der Benutzer nicht vorhandene Verzeichnisse angeben darf. Ist dieser Parameter wahr, kann der Benutzer nur bereits existierende Verzeichnisse wählen. Der Standardwert ist *false*.

**-parent *window***

Macht *window* zum logischen Elternfenster des Dialogs. Der Dialog wird über seinem Elternfenster angezeigt. Unter macOS wird der Dateidialog zu einem an das Elternfenster angehängten Sheet.

**-title *titleString***

Legt eine Zeichenkette fest, die als Titel des Dialogfelds angezeigt wird. Wird diese Option nicht angegeben, wird ein Standardtitel angezeigt.

## Beispiel

```
set dir [tk_chooseDirectory \
        -initialdir ~ -title "Choose a directory"]
if {$dir eq ""} {
   label .l -text "No directory selected"
} else {
   label .l -text "Selected $dir"
}
```

*Copyright (c) 1998-2000 Scriptics Corporation.*

# clipboard

*Stichwörter:* clipboard, clear, format, append, selection, type

## Name

clipboard - Tk-Zwischenablage steuern

## Übersicht

**clipboard** *option* ?*arg* ...?

## Beschreibung

Dieser Befehl bietet eine Tcl-Schnittstelle zur Tk-Zwischenablage, die Daten für spätere Abfrage über die Selection-Mechanismen speichert (Option **-selection CLIPBOARD**). Zum Kopieren: zuerst **clipboard clear**, dann eine oder mehrere **clipboard append**-Aufrufe. Für atomare Updates alle Appends vor der Rückkehr in die Ereignisschleife abschließen.

Das erste Argument von **clipboard** legt Format der weiteren Argumente und Verhalten fest. Unterstützt werden:

**clipboard append ?*-displayof window*? ?*-format format*? ?*-type type*? ?--? *data***

Hängt *data* an die Zwischenablage auf dem Display von *window* an, in der Form *type* mit Darstellung *format*, und beansprucht die Zwischenablage auf diesem Display.

*Type* ist die Rückgabeform der Selection (ICCCM-"target"), z. B. **STRING** oder **FILE_NAME**; Details im Inter-Client Communication Conventions Manual. Standard **STRING**.

*format* ist die Übertragungsdarstellung (ICCCM Tabelle 2, Spalte 2), Standard **STRING**. Bei **STRING**: 8-Bit-ASCII. Bei **ATOM**: *data* in Felder durch Leerzeichen geteilt, jedes Feld zum Atom-Wert, 32-Bit-Atom-Wert statt Name. Sonst: Felder zu 32-Bit-Integer, Array an den Anforderer. Strings an **clipboard append** werden vor der Konvertierung verkettet — passende Trennung beachten. Gleicher *type* erfordert gleichen *format*.

*format* nur für Nicht-Tk-Anforderer nötig; bei Tk-Abfrage der **CLIPBOARD**-Selection wird am Ende wieder ein String geliefert — *format* irrelevant.

**--** markiert Optionenende: nächstes Argument ist immer *data* (z. B. wenn *data* mit **-** beginnt).

**clipboard clear ?*-displayof window*?**

Beansprucht die Zwischenablage auf dem Display von *window* und entfernt frühere Inhalte. *window* standardmäßig ".". Leerer String als Rückgabe.

**clipboard get ?*-displayof window*? ?*-type type*?**

Daten aus der Zwischenablage auf dem Display von *window*. *window* standardmäßig ".". *type* z. B. **STRING** oder **FILE_NAME**, Standard **STRING**. Entspricht "selection get **-selection CLIPBOARD**".

Auf modernen X11-Systemen für Text oft **UTF8_STRING** statt **STRING** sinnvoller.

## Beispiele

Aktuellen Zwischenablage-Inhalt lesen:

```
if {[catch {clipboard get} contents]} {
    # There were no clipboard contents at all
}
```

Zwischenablage mit festem String setzen:

```
clipboard clear
clipboard append "some fixed string"
```

Eigene Daten per **-type** (nicht unbedingt portabel, aber nützlich). Tcl-Skripte so zu übergeben ist wirksam; in Produktion mit Safe-Interpretern kombinieren.

```
# This is a very simple canvas serializer;
# it produces a script that recreates the item(s) when executed
proc getItemConfig {canvas tag} {
   set script {}
   foreach item [$canvas find withtag $tag] {
      append script {$canvas create } [$canvas type $item]
      append script { } [$canvas coords $item] { }
      foreach config [$canvas itemconf $item] {
         lassign $config name - - - value
         append script [list $name $value] { }
      }
      append script \n
   }
   return [string trim $script]
}

# Set up a binding on a canvas to cut and paste an item
set c [canvas .c]
pack $c
$c create text 150 30 -text "cut and paste me"
bind $c <<Cut>> {
   clipboard clear
   clipboard append -type TkCanvasItem \
         [getItemConfig %W current]
   # Delete because this is cut, not copy.
   %W delete current
}
bind $c <<Paste>> {
   catch {
      set canvas %W
      eval [clipboard get -type TkCanvasItem]
   }
}
```

*Copyright (c) 1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# colors

*Stichwörter:* colors, color, option

## Name

colors - Von Tk erkannte symbolische Farbnamen

## Beschreibung

Tk erkennt viele symbolische Farbnamen (z. B. **red**), wenn Farben angegeben werden. Die von Tk erkannten symbolischen Namen und ihre RGB-Werte mit 8 Bit pro Kanal:

| Name | **Red** | **Green** | **Blue** |
|---|---|---|---|
| alice blue | 240 | 248 | 255 |
| AliceBlue | 240 | 248 | 255 |
| antique white | 250 | 235 | 215 |
| AntiqueWhite | 250 | 235 | 215 |
| AntiqueWhite1 | 255 | 239 | 219 |
| AntiqueWhite2 | 238 | 223 | 204 |
| AntiqueWhite3 | 205 | 192 | 176 |
| AntiqueWhite4 | 139 | 131 | 120 |
| aqua | 0 | 255 | 255 |
| aquamarine | 127 | 255 | 212 |
| aquamarine1 | 127 | 255 | 212 |
| aquamarine2 | 118 | 238 | 198 |
| aquamarine3 | 102 | 205 | 170 |
| aquamarine4 | 69 | 139 | 116 |
| azure | 240 | 255 | 255 |
| azure1 | 240 | 255 | 255 |
| azure2 | 224 | 238 | 238 |
| azure3 | 193 | 205 | 205 |
| azure4 | 131 | 139 | 139 |
| beige | 245 | 245 | 220 |
| bisque | 255 | 228 | 196 |
| bisque1 | 255 | 228 | 196 |
| bisque2 | 238 | 213 | 183 |
| bisque3 | 205 | 183 | 158 |
| bisque4 | 139 | 125 | 107 |
| black | 0 | 0 | 0 |
| blanched almond | 255 | 235 | 205 |
| BlanchedAlmond | 255 | 235 | 205 |
| blue | 0 | 0 | 255 |

| | | | |
|---|---|---|---|
| blue violet | 138 | 43 | 226 |
| blue1 | 0 | 0 | 255 |
| blue2 | 0 | 0 | 238 |
| blue3 | 0 | 0 | 205 |
| blue4 | 0 | 0 | 139 |
| BlueViolet | 138 | 43 | 226 |
| brown | 165 | 42 | 42 |
| brown1 | 255 | 64 | 64 |
| brown2 | 238 | 59 | 59 |
| brown3 | 205 | 51 | 51 |
| brown4 | 139 | 35 | 35 |
| burlywood | 222 | 184 | 135 |
| burlywood1 | 255 | 211 | 155 |
| burlywood2 | 238 | 197 | 145 |
| burlywood3 | 205 | 170 | 125 |
| burlywood4 | 139 | 115 | 85 |
| cadet blue | 95 | 158 | 160 |
| CadetBlue | 95 | 158 | 160 |
| CadetBlue1 | 152 | 245 | 255 |
| CadetBlue2 | 142 | 229 | 238 |
| CadetBlue3 | 122 | 197 | 205 |
| CadetBlue4 | 83 | 134 | 139 |
| chartreuse | 127 | 255 | 0 |
| chartreuse1 | 127 | 255 | 0 |
| chartreuse2 | 118 | 238 | 0 |
| chartreuse3 | 102 | 205 | 0 |
| chartreuse4 | 69 | 139 | 0 |
| chocolate | 210 | 105 | 30 |
| chocolate1 | 255 | 127 | 36 |
| chocolate2 | 238 | 118 | 33 |
| chocolate3 | 205 | 102 | 29 |
| chocolate4 | 139 | 69 | 19 |
| coral | 255 | 127 | 80 |
| coral1 | 255 | 114 | 86 |
| coral2 | 238 | 106 | 80 |
| coral3 | 205 | 91 | 69 |
| coral4 | 139 | 62 | 47 |
| cornflower blue | 100 | 149 | 237 |
| CornflowerBlue | 100 | 149 | 237 |
| cornsilk | 255 | 248 | 220 |
| cornsilk1 | 255 | 248 | 220 |
| cornsilk2 | 238 | 232 | 205 |
| cornsilk3 | 205 | 200 | 177 |
| cornsilk4 | 139 | 136 | 120 |
| crimson | 220 | 20 | 60 |
| cyan | 0 | 255 | 255 |

| | | | |
|---|---|---|---|
| cyan1 | 0 | 255 | 255 |
| cyan2 | 0 | 238 | 238 |
| cyan3 | 0 | 205 | 205 |
| cyan4 | 0 | 139 | 139 |
| dark blue | 0 | 0 | 139 |
| dark cyan | 0 | 139 | 139 |
| dark goldenrod | 184 | 134 | 11 |
| dark gray | 169 | 169 | 169 |
| dark green | 0 | 100 | 0 |
| dark grey | 169 | 169 | 169 |
| dark khaki | 189 | 183 | 107 |
| dark magenta | 139 | 0 | 139 |
| dark olive green | 85 | 107 | 47 |
| dark orange | 255 | 140 | 0 |
| dark orchid | 153 | 50 | 204 |
| dark red | 139 | 0 | 0 |
| dark salmon | 233 | 150 | 122 |
| dark sea green | 143 | 188 | 143 |
| dark slate blue | 72 | 61 | 139 |
| dark slate gray | 47 | 79 | 79 |
| dark slate grey | 47 | 79 | 79 |
| dark turquoise | 0 | 206 | 209 |
| dark violet | 148 | 0 | 211 |
| DarkBlue | 0 | 0 | 139 |
| DarkCyan | 0 | 139 | 139 |
| DarkGoldenrod | 184 | 134 | 11 |
| DarkGoldenrod1 | 255 | 185 | 15 |
| DarkGoldenrod2 | 238 | 173 | 14 |
| DarkGoldenrod3 | 205 | 149 | 12 |
| DarkGoldenrod4 | 139 | 101 | 8 |
| DarkGray | 169 | 169 | 169 |
| DarkGreen | 0 | 100 | 0 |
| DarkGrey | 169 | 169 | 169 |
| DarkKhaki | 189 | 183 | 107 |
| DarkMagenta | 139 | 0 | 139 |
| DarkOliveGreen | 85 | 107 | 47 |
| DarkOliveGreen1 | 202 | 255 | 112 |
| DarkOliveGreen2 | 188 | 238 | 104 |
| DarkOliveGreen3 | 162 | 205 | 90 |
| DarkOliveGreen4 | 110 | 139 | 61 |
| DarkOrange | 255 | 140 | 0 |
| DarkOrange1 | 255 | 127 | 0 |
| DarkOrange2 | 238 | 118 | 0 |
| DarkOrange3 | 205 | 102 | 0 |
| DarkOrange4 | 139 | 69 | 0 |
| DarkOrchid | 153 | 50 | 204 |

| | | | |
|---|---|---|---|
| DarkOrchid1 | 191 | 62 | 255 |
| DarkOrchid2 | 178 | 58 | 238 |
| DarkOrchid3 | 154 | 50 | 205 |
| DarkOrchid4 | 104 | 34 | 139 |
| DarkRed | 139 | 0 | 0 |
| DarkSalmon | 233 | 150 | 122 |
| DarkSeaGreen | 143 | 188 | 143 |
| DarkSeaGreen1 | 193 | 255 | 193 |
| DarkSeaGreen2 | 180 | 238 | 180 |
| DarkSeaGreen3 | 155 | 205 | 155 |
| DarkSeaGreen4 | 105 | 139 | 105 |
| DarkSlateBlue | 72 | 61 | 139 |
| DarkSlateGray | 47 | 79 | 79 |
| DarkSlateGray1 | 151 | 255 | 255 |
| DarkSlateGray2 | 141 | 238 | 238 |
| DarkSlateGray3 | 121 | 205 | 205 |
| DarkSlateGray4 | 82 | 139 | 139 |
| DarkSlateGrey | 47 | 79 | 79 |
| DarkTurquoise | 0 | 206 | 209 |
| DarkViolet | 148 | 0 | 211 |
| deep pink | 255 | 20 | 147 |
| deep sky blue | 0 | 191 | 255 |
| DeepPink | 255 | 20 | 147 |
| DeepPink1 | 255 | 20 | 147 |
| DeepPink2 | 238 | 18 | 137 |
| DeepPink3 | 205 | 16 | 118 |
| DeepPink4 | 139 | 10 | 80 |
| DeepSkyBlue | 0 | 191 | 255 |
| DeepSkyBlue1 | 0 | 191 | 255 |
| DeepSkyBlue2 | 0 | 178 | 238 |
| DeepSkyBlue3 | 0 | 154 | 205 |
| DeepSkyBlue4 | 0 | 104 | 139 |
| dim gray | 105 | 105 | 105 |
| dim grey | 105 | 105 | 105 |
| DimGray | 105 | 105 | 105 |
| DimGrey | 105 | 105 | 105 |
| dodger blue | 30 | 144 | 255 |
| DodgerBlue | 30 | 144 | 255 |
| DodgerBlue1 | 30 | 144 | 255 |
| DodgerBlue2 | 28 | 134 | 238 |
| DodgerBlue3 | 24 | 116 | 205 |
| DodgerBlue4 | 16 | 78 | 139 |
| firebrick | 178 | 34 | 34 |
| firebrick1 | 255 | 48 | 48 |
| firebrick2 | 238 | 44 | 44 |
| firebrick3 | 205 | 38 | 38 |

| | | | |
|---|---|---|---|
| firebrick4 | 139 | 26 | 26 |
| floral white | 255 | 250 | 240 |
| FloralWhite | 255 | 250 | 240 |
| forest green | 34 | 139 | 34 |
| ForestGreen | 34 | 139 | 34 |
| fuchsia | 255 | 0 | 255 |
| gainsboro | 220 | 220 | 220 |
| ghost white | 248 | 248 | 255 |
| GhostWhite | 248 | 248 | 255 |
| gold | 255 | 215 | 0 |
| gold1 | 255 | 215 | 0 |
| gold2 | 238 | 201 | 0 |
| gold3 | 205 | 173 | 0 |
| gold4 | 139 | 117 | 0 |
| goldenrod | 218 | 165 | 32 |
| goldenrod1 | 255 | 193 | 37 |
| goldenrod2 | 238 | 180 | 34 |
| goldenrod3 | 205 | 155 | 29 |
| goldenrod4 | 139 | 105 | 20 |
| gray | 128 | 128 | 128 |
| gray0 | 0 | 0 | 0 |
| gray1 | 3 | 3 | 3 |
| gray2 | 5 | 5 | 5 |
| gray3 | 8 | 8 | 8 |
| gray4 | 10 | 10 | 10 |
| gray5 | 13 | 13 | 13 |
| gray6 | 15 | 15 | 15 |
| gray7 | 18 | 18 | 18 |
| gray8 | 20 | 20 | 20 |
| gray9 | 23 | 23 | 23 |
| gray10 | 26 | 26 | 26 |
| gray11 | 28 | 28 | 28 |
| gray12 | 31 | 31 | 31 |
| gray13 | 33 | 33 | 33 |
| gray14 | 36 | 36 | 36 |
| gray15 | 38 | 38 | 38 |
| gray16 | 41 | 41 | 41 |
| gray17 | 43 | 43 | 43 |
| gray18 | 46 | 46 | 46 |
| gray19 | 48 | 48 | 48 |
| gray20 | 51 | 51 | 51 |
| gray21 | 54 | 54 | 54 |
| gray22 | 56 | 56 | 56 |
| gray23 | 59 | 59 | 59 |
| gray24 | 61 | 61 | 61 |
| gray25 | 64 | 64 | 64 |

| | | | |
|---|---|---|---|
| gray26 | 66 | 66 | 66 |
| gray27 | 69 | 69 | 69 |
| gray28 | 71 | 71 | 71 |
| gray29 | 74 | 74 | 74 |
| gray30 | 77 | 77 | 77 |
| gray31 | 79 | 79 | 79 |
| gray32 | 82 | 82 | 82 |
| gray33 | 84 | 84 | 84 |
| gray34 | 87 | 87 | 87 |
| gray35 | 89 | 89 | 89 |
| gray36 | 92 | 92 | 92 |
| gray37 | 94 | 94 | 94 |
| gray38 | 97 | 97 | 97 |
| gray39 | 99 | 99 | 99 |
| gray40 | 102 | 102 | 102 |
| gray41 | 105 | 105 | 105 |
| gray42 | 107 | 107 | 107 |
| gray43 | 110 | 110 | 110 |
| gray44 | 112 | 112 | 112 |
| gray45 | 115 | 115 | 115 |
| gray46 | 117 | 117 | 117 |
| gray47 | 120 | 120 | 120 |
| gray48 | 122 | 122 | 122 |
| gray49 | 125 | 125 | 125 |
| gray50 | 127 | 127 | 127 |
| gray51 | 130 | 130 | 130 |
| gray52 | 133 | 133 | 133 |
| gray53 | 135 | 135 | 135 |
| gray54 | 138 | 138 | 138 |
| gray55 | 140 | 140 | 140 |
| gray56 | 143 | 143 | 143 |
| gray57 | 145 | 145 | 145 |
| gray58 | 148 | 148 | 148 |
| gray59 | 150 | 150 | 150 |
| gray60 | 153 | 153 | 153 |
| gray61 | 156 | 156 | 156 |
| gray62 | 158 | 158 | 158 |
| gray63 | 161 | 161 | 161 |
| gray64 | 163 | 163 | 163 |
| gray65 | 166 | 166 | 166 |
| gray66 | 168 | 168 | 168 |
| gray67 | 171 | 171 | 171 |
| gray68 | 173 | 173 | 173 |
| gray69 | 176 | 176 | 176 |
| gray70 | 179 | 179 | 179 |
| gray71 | 181 | 181 | 181 |

| | | | |
|---|---|---|---|
| gray72 | 184 | 184 | 184 |
| gray73 | 186 | 186 | 186 |
| gray74 | 189 | 189 | 189 |
| gray75 | 191 | 191 | 191 |
| gray76 | 194 | 194 | 194 |
| gray77 | 196 | 196 | 196 |
| gray78 | 199 | 199 | 199 |
| gray79 | 201 | 201 | 201 |
| gray80 | 204 | 204 | 204 |
| gray81 | 207 | 207 | 207 |
| gray82 | 209 | 209 | 209 |
| gray83 | 212 | 212 | 212 |
| gray84 | 214 | 214 | 214 |
| gray85 | 217 | 217 | 217 |
| gray86 | 219 | 219 | 219 |
| gray87 | 222 | 222 | 222 |
| gray88 | 224 | 224 | 224 |
| gray89 | 227 | 227 | 227 |
| gray90 | 229 | 229 | 229 |
| gray91 | 232 | 232 | 232 |
| gray92 | 235 | 235 | 235 |
| gray93 | 237 | 237 | 237 |
| gray94 | 240 | 240 | 240 |
| gray95 | 242 | 242 | 242 |
| gray96 | 245 | 245 | 245 |
| gray97 | 247 | 247 | 247 |
| gray98 | 250 | 250 | 250 |
| gray99 | 252 | 252 | 252 |
| gray100 | 255 | 255 | 255 |
| green | 0 | 128 | 0 |
| green yellow | 173 | 255 | 47 |
| green1 | 0 | 255 | 0 |
| green2 | 0 | 238 | 0 |
| green3 | 0 | 205 | 0 |
| green4 | 0 | 139 | 0 |
| GreenYellow | 173 | 255 | 47 |
| grey | 128 | 128 | 128 |
| grey0 | 0 | 0 | 0 |
| grey1 | 3 | 3 | 3 |
| grey2 | 5 | 5 | 5 |
| grey3 | 8 | 8 | 8 |
| grey4 | 10 | 10 | 10 |
| grey5 | 13 | 13 | 13 |
| grey6 | 15 | 15 | 15 |
| grey7 | 18 | 18 | 18 |
| grey8 | 20 | 20 | 20 |

| | | | |
|---|---|---|---|
| grey9 | 23 | 23 | 23 |
| grey10 | 26 | 26 | 26 |
| grey11 | 28 | 28 | 28 |
| grey12 | 31 | 31 | 31 |
| grey13 | 33 | 33 | 33 |
| grey14 | 36 | 36 | 36 |
| grey15 | 38 | 38 | 38 |
| grey16 | 41 | 41 | 41 |
| grey17 | 43 | 43 | 43 |
| grey18 | 46 | 46 | 46 |
| grey19 | 48 | 48 | 48 |
| grey20 | 51 | 51 | 51 |
| grey21 | 54 | 54 | 54 |
| grey22 | 56 | 56 | 56 |
| grey23 | 59 | 59 | 59 |
| grey24 | 61 | 61 | 61 |
| grey25 | 64 | 64 | 64 |
| grey26 | 66 | 66 | 66 |
| grey27 | 69 | 69 | 69 |
| grey28 | 71 | 71 | 71 |
| grey29 | 74 | 74 | 74 |
| grey30 | 77 | 77 | 77 |
| grey31 | 79 | 79 | 79 |
| grey32 | 82 | 82 | 82 |
| grey33 | 84 | 84 | 84 |
| grey34 | 87 | 87 | 87 |
| grey35 | 89 | 89 | 89 |
| grey36 | 92 | 92 | 92 |
| grey37 | 94 | 94 | 94 |
| grey38 | 97 | 97 | 97 |
| grey39 | 99 | 99 | 99 |
| grey40 | 102 | 102 | 102 |
| grey41 | 105 | 105 | 105 |
| grey42 | 107 | 107 | 107 |
| grey43 | 110 | 110 | 110 |
| grey44 | 112 | 112 | 112 |
| grey45 | 115 | 115 | 115 |
| grey46 | 117 | 117 | 117 |
| grey47 | 120 | 120 | 120 |
| grey48 | 122 | 122 | 122 |
| grey49 | 125 | 125 | 125 |
| grey50 | 127 | 127 | 127 |
| grey51 | 130 | 130 | 130 |
| grey52 | 133 | 133 | 133 |
| grey53 | 135 | 135 | 135 |
| grey54 | 138 | 138 | 138 |

| | | | |
|---|---|---|---|
| grey55 | 140 | 140 | 140 |
| grey56 | 143 | 143 | 143 |
| grey57 | 145 | 145 | 145 |
| grey58 | 148 | 148 | 148 |
| grey59 | 150 | 150 | 150 |
| grey60 | 153 | 153 | 153 |
| grey61 | 156 | 156 | 156 |
| grey62 | 158 | 158 | 158 |
| grey63 | 161 | 161 | 161 |
| grey64 | 163 | 163 | 163 |
| grey65 | 166 | 166 | 166 |
| grey66 | 168 | 168 | 168 |
| grey67 | 171 | 171 | 171 |
| grey68 | 173 | 173 | 173 |
| grey69 | 176 | 176 | 176 |
| grey70 | 179 | 179 | 179 |
| grey71 | 181 | 181 | 181 |
| grey72 | 184 | 184 | 184 |
| grey73 | 186 | 186 | 186 |
| grey74 | 189 | 189 | 189 |
| grey75 | 191 | 191 | 191 |
| grey76 | 194 | 194 | 194 |
| grey77 | 196 | 196 | 196 |
| grey78 | 199 | 199 | 199 |
| grey79 | 201 | 201 | 201 |
| grey80 | 204 | 204 | 204 |
| grey81 | 207 | 207 | 207 |
| grey82 | 209 | 209 | 209 |
| grey83 | 212 | 212 | 212 |
| grey84 | 214 | 214 | 214 |
| grey85 | 217 | 217 | 217 |
| grey86 | 219 | 219 | 219 |
| grey87 | 222 | 222 | 222 |
| grey88 | 224 | 224 | 224 |
| grey89 | 227 | 227 | 227 |
| grey90 | 229 | 229 | 229 |
| grey91 | 232 | 232 | 232 |
| grey92 | 235 | 235 | 235 |
| grey93 | 237 | 237 | 237 |
| grey94 | 240 | 240 | 240 |
| grey95 | 242 | 242 | 242 |
| grey96 | 245 | 245 | 245 |
| grey97 | 247 | 247 | 247 |
| grey98 | 250 | 250 | 250 |
| grey99 | 252 | 252 | 252 |
| grey100 | 255 | 255 | 255 |

| | | | |
|---|---|---|---|
| honeydew | 240 | 255 | 240 |
| honeydew1 | 240 | 255 | 240 |
| honeydew2 | 224 | 238 | 224 |
| honeydew3 | 193 | 205 | 193 |
| honeydew4 | 131 | 139 | 131 |
| hot pink | 255 | 105 | 180 |
| HotPink | 255 | 105 | 180 |
| HotPink1 | 255 | 110 | 180 |
| HotPink2 | 238 | 106 | 167 |
| HotPink3 | 205 | 96 | 144 |
| HotPink4 | 139 | 58 | 98 |
| indian red | 205 | 92 | 92 |
| IndianRed | 205 | 92 | 92 |
| IndianRed1 | 255 | 106 | 106 |
| IndianRed2 | 238 | 99 | 99 |
| IndianRed3 | 205 | 85 | 85 |
| IndianRed4 | 139 | 58 | 58 |
| indigo | 75 | 0 | 130 |
| ivory | 255 | 255 | 240 |
| ivory1 | 255 | 255 | 240 |
| ivory2 | 238 | 238 | 224 |
| ivory3 | 205 | 205 | 193 |
| ivory4 | 139 | 139 | 131 |
| khaki | 240 | 230 | 140 |
| khaki1 | 255 | 246 | 143 |
| khaki2 | 238 | 230 | 133 |
| khaki3 | 205 | 198 | 115 |
| khaki4 | 139 | 134 | 78 |
| lavender | 230 | 230 | 250 |
| lavender blush | 255 | 240 | 245 |
| LavenderBlush | 255 | 240 | 245 |
| LavenderBlush1 | 255 | 240 | 245 |
| LavenderBlush2 | 238 | 224 | 229 |
| LavenderBlush3 | 205 | 193 | 197 |
| LavenderBlush4 | 139 | 131 | 134 |
| lawn green | 124 | 252 | 0 |
| LawnGreen | 124 | 252 | 0 |
| lemon chiffon | 255 | 250 | 205 |
| LemonChiffon | 255 | 250 | 205 |
| LemonChiffon1 | 255 | 250 | 205 |
| LemonChiffon2 | 238 | 233 | 191 |
| LemonChiffon3 | 205 | 201 | 165 |
| LemonChiffon4 | 139 | 137 | 112 |
| light blue | 173 | 216 | 230 |
| light coral | 240 | 128 | 128 |
| light cyan | 224 | 255 | 255 |

| | | | |
|---|---|---|---|
| light goldenrod | 238 | 221 | 130 |
| light goldenrod yellow | 250 | 250 | 210 |
| light gray | 211 | 211 | 211 |
| light green | 144 | 238 | 144 |
| light grey | 211 | 211 | 211 |
| light pink | 255 | 182 | 193 |
| light salmon | 255 | 160 | 122 |
| light sea green | 32 | 178 | 170 |
| light sky blue | 135 | 206 | 250 |
| light slate blue | 132 | 112 | 255 |
| light slate gray | 119 | 136 | 153 |
| light slate grey | 119 | 136 | 153 |
| light steel blue | 176 | 196 | 222 |
| light yellow | 255 | 255 | 224 |
| LightBlue | 173 | 216 | 230 |
| LightBlue1 | 191 | 239 | 255 |
| LightBlue2 | 178 | 223 | 238 |
| LightBlue3 | 154 | 192 | 205 |
| LightBlue4 | 104 | 131 | 139 |
| LightCoral | 240 | 128 | 128 |
| LightCyan | 224 | 255 | 255 |
| LightCyan1 | 224 | 255 | 255 |
| LightCyan2 | 209 | 238 | 238 |
| LightCyan3 | 180 | 205 | 205 |
| LightCyan4 | 122 | 139 | 139 |
| LightGoldenrod | 238 | 221 | 130 |
| LightGoldenrod1 | 255 | 236 | 139 |
| LightGoldenrod2 | 238 | 220 | 130 |
| LightGoldenrod3 | 205 | 190 | 112 |
| LightGoldenrod4 | 139 | 129 | 76 |
| LightGoldenrodYellow | 250 | 250 | 210 |
| LightGray | 211 | 211 | 211 |
| LightGreen | 144 | 238 | 144 |
| LightGrey | 211 | 211 | 211 |
| LightPink | 255 | 182 | 193 |
| LightPink1 | 255 | 174 | 185 |
| LightPink2 | 238 | 162 | 173 |
| LightPink3 | 205 | 140 | 149 |
| LightPink4 | 139 | 95 | 101 |
| LightSalmon | 255 | 160 | 122 |
| LightSalmon1 | 255 | 160 | 122 |
| LightSalmon2 | 238 | 149 | 114 |
| LightSalmon3 | 205 | 129 | 98 |
| LightSalmon4 | 139 | 87 | 66 |
| LightSeaGreen | 32 | 178 | 170 |
| LightSkyBlue | 135 | 206 | 250 |

| | | | |
|---|---|---|---|
| LightSkyBlue1 | 176 | 226 | 255 |
| LightSkyBlue2 | 164 | 211 | 238 |
| LightSkyBlue3 | 141 | 182 | 205 |
| LightSkyBlue4 | 96 | 123 | 139 |
| LightSlateBlue | 132 | 112 | 255 |
| LightSlateGray | 119 | 136 | 153 |
| LightSlateGrey | 119 | 136 | 153 |
| LightSteelBlue | 176 | 196 | 222 |
| LightSteelBlue1 | 202 | 225 | 255 |
| LightSteelBlue2 | 188 | 210 | 238 |
| LightSteelBlue3 | 162 | 181 | 205 |
| LightSteelBlue4 | 110 | 123 | 139 |
| LightYellow | 255 | 255 | 224 |
| LightYellow1 | 255 | 255 | 224 |
| LightYellow2 | 238 | 238 | 209 |
| LightYellow3 | 205 | 205 | 180 |
| LightYellow4 | 139 | 139 | 122 |
| lime | 0 | 255 | 0 |
| lime green | 50 | 205 | 50 |
| LimeGreen | 50 | 205 | 50 |
| linen | 250 | 240 | 230 |
| magenta | 255 | 0 | 255 |
| magenta1 | 255 | 0 | 255 |
| magenta2 | 238 | 0 | 238 |
| magenta3 | 205 | 0 | 205 |
| magenta4 | 139 | 0 | 139 |
| maroon | 128 | 0 | 0 |
| maroon1 | 255 | 52 | 179 |
| maroon2 | 238 | 48 | 167 |
| maroon3 | 205 | 41 | 144 |
| maroon4 | 139 | 28 | 98 |
| medium aquamarine | 102 | 205 | 170 |
| medium blue | 0 | 0 | 205 |
| medium orchid | 186 | 85 | 211 |
| medium purple | 147 | 112 | 219 |
| medium sea green | 60 | 179 | 113 |
| medium slate blue | 123 | 104 | 238 |
| medium spring green | 0 | 250 | 154 |
| medium turquoise | 72 | 209 | 204 |
| medium violet red | 199 | 21 | 133 |
| MediumAquamarine | 102 | 205 | 170 |
| MediumBlue | 0 | 0 | 205 |
| MediumOrchid | 186 | 85 | 211 |
| MediumOrchid1 | 224 | 102 | 255 |
| MediumOrchid2 | 209 | 95 | 238 |
| MediumOrchid3 | 180 | 82 | 205 |

| | | | |
|---|---|---|---|
| MediumOrchid4 | 122 | 55 | 139 |
| MediumPurple | 147 | 112 | 219 |
| MediumPurple1 | 171 | 130 | 255 |
| MediumPurple2 | 159 | 121 | 238 |
| MediumPurple3 | 137 | 104 | 205 |
| MediumPurple4 | 93 | 71 | 139 |
| MediumSeaGreen | 60 | 179 | 113 |
| MediumSlateBlue | 123 | 104 | 238 |
| MediumSpringGreen | 0 | 250 | 154 |
| MediumTurquoise | 72 | 209 | 204 |
| MediumVioletRed | 199 | 21 | 133 |
| midnight blue | 25 | 25 | 112 |
| MidnightBlue | 25 | 25 | 112 |
| mint cream | 245 | 255 | 250 |
| MintCream | 245 | 255 | 250 |
| misty rose | 255 | 228 | 225 |
| MistyRose | 255 | 228 | 225 |
| MistyRose1 | 255 | 228 | 225 |
| MistyRose2 | 238 | 213 | 210 |
| MistyRose3 | 205 | 183 | 181 |
| MistyRose4 | 139 | 125 | 123 |
| moccasin | 255 | 228 | 181 |
| navajo white | 255 | 222 | 173 |
| NavajoWhite | 255 | 222 | 173 |
| NavajoWhite1 | 255 | 222 | 173 |
| NavajoWhite2 | 238 | 207 | 161 |
| NavajoWhite3 | 205 | 179 | 139 |
| NavajoWhite4 | 139 | 121 | 94 |
| navy | 0 | 0 | 128 |
| navy blue | 0 | 0 | 128 |
| NavyBlue | 0 | 0 | 128 |
| old lace | 253 | 245 | 230 |
| OldLace | 253 | 245 | 230 |
| olive | 128 | 128 | 0 |
| olive drab | 107 | 142 | 35 |
| OliveDrab | 107 | 142 | 35 |
| OliveDrab1 | 192 | 255 | 62 |
| OliveDrab2 | 179 | 238 | 58 |
| OliveDrab3 | 154 | 205 | 50 |
| OliveDrab4 | 105 | 139 | 34 |
| orange | 255 | 165 | 0 |
| orange red | 255 | 69 | 0 |
| orange1 | 255 | 165 | 0 |
| orange2 | 238 | 154 | 0 |
| orange3 | 205 | 133 | 0 |
| orange4 | 139 | 90 | 0 |

| | | | |
|---|---|---|---|
| OrangeRed | 255 | 69 | 0 |
| OrangeRed1 | 255 | 69 | 0 |
| OrangeRed2 | 238 | 64 | 0 |
| OrangeRed3 | 205 | 55 | 0 |
| OrangeRed4 | 139 | 37 | 0 |
| orchid | 218 | 112 | 214 |
| orchid1 | 255 | 131 | 250 |
| orchid2 | 238 | 122 | 233 |
| orchid3 | 205 | 105 | 201 |
| orchid4 | 139 | 71 | 137 |
| pale goldenrod | 238 | 232 | 170 |
| pale green | 152 | 251 | 152 |
| pale turquoise | 175 | 238 | 238 |
| pale violet red | 219 | 112 | 147 |
| PaleGoldenrod | 238 | 232 | 170 |
| PaleGreen | 152 | 251 | 152 |
| PaleGreen1 | 154 | 255 | 154 |
| PaleGreen2 | 144 | 238 | 144 |
| PaleGreen3 | 124 | 205 | 124 |
| PaleGreen4 | 84 | 139 | 84 |
| PaleTurquoise | 175 | 238 | 238 |
| PaleTurquoise1 | 187 | 255 | 255 |
| PaleTurquoise2 | 174 | 238 | 238 |
| PaleTurquoise3 | 150 | 205 | 205 |
| PaleTurquoise4 | 102 | 139 | 139 |
| PaleVioletRed | 219 | 112 | 147 |
| PaleVioletRed1 | 255 | 130 | 171 |
| PaleVioletRed2 | 238 | 121 | 159 |
| PaleVioletRed3 | 205 | 104 | 127 |
| PaleVioletRed4 | 139 | 71 | 93 |
| papaya whip | 255 | 239 | 213 |
| PapayaWhip | 255 | 239 | 213 |
| peach puff | 255 | 218 | 185 |
| PeachPuff | 255 | 218 | 185 |
| PeachPuff1 | 255 | 218 | 185 |
| PeachPuff2 | 238 | 203 | 173 |
| PeachPuff3 | 205 | 175 | 149 |
| PeachPuff4 | 139 | 119 | 101 |
| peru | 205 | 133 | 63 |
| pink | 255 | 192 | 203 |
| pink1 | 255 | 181 | 197 |
| pink2 | 238 | 169 | 184 |
| pink3 | 205 | 145 | 158 |
| pink4 | 139 | 99 | 108 |
| plum | 221 | 160 | 221 |
| plum1 | 255 | 187 | 255 |

| | | | |
|---|---|---|---|
| plum2 | 238 | 174 | 238 |
| plum3 | 205 | 150 | 205 |
| plum4 | 139 | 102 | 139 |
| powder blue | 176 | 224 | 230 |
| PowderBlue | 176 | 224 | 230 |
| purple | 128 | 0 | 128 |
| purple1 | 155 | 48 | 255 |
| purple2 | 145 | 44 | 238 |
| purple3 | 125 | 38 | 205 |
| purple4 | 85 | 26 | 139 |
| red | 255 | 0 | 0 |
| red1 | 255 | 0 | 0 |
| red2 | 238 | 0 | 0 |
| red3 | 205 | 0 | 0 |
| red4 | 139 | 0 | 0 |
| rosy brown | 188 | 143 | 143 |
| RosyBrown | 188 | 143 | 143 |
| RosyBrown1 | 255 | 193 | 193 |
| RosyBrown2 | 238 | 180 | 180 |
| RosyBrown3 | 205 | 155 | 155 |
| RosyBrown4 | 139 | 105 | 105 |
| royal blue | 65 | 105 | 225 |
| RoyalBlue | 65 | 105 | 225 |
| RoyalBlue1 | 72 | 118 | 255 |
| RoyalBlue2 | 67 | 110 | 238 |
| RoyalBlue3 | 58 | 95 | 205 |
| RoyalBlue4 | 39 | 64 | 139 |
| saddle brown | 139 | 69 | 19 |
| SaddleBrown | 139 | 69 | 19 |
| salmon | 250 | 128 | 114 |
| salmon1 | 255 | 140 | 105 |
| salmon2 | 238 | 130 | 98 |
| salmon3 | 205 | 112 | 84 |
| salmon4 | 139 | 76 | 57 |
| sandy brown | 244 | 164 | 96 |
| SandyBrown | 244 | 164 | 96 |
| sea green | 46 | 139 | 87 |
| SeaGreen | 46 | 139 | 87 |
| SeaGreen1 | 84 | 255 | 159 |
| SeaGreen2 | 78 | 238 | 148 |
| SeaGreen3 | 67 | 205 | 128 |
| SeaGreen4 | 46 | 139 | 87 |
| seashell | 255 | 245 | 238 |
| seashell1 | 255 | 245 | 238 |
| seashell2 | 238 | 229 | 222 |
| seashell3 | 205 | 197 | 191 |

| | | | |
|---|---|---|---|
| seashell4 | 139 | 134 | 130 |
| sienna | 160 | 82 | 45 |
| sienna1 | 255 | 130 | 71 |
| sienna2 | 238 | 121 | 66 |
| sienna3 | 205 | 104 | 57 |
| sienna4 | 139 | 71 | 38 |
| silver | 192 | 192 | 192 |
| sky blue | 135 | 206 | 235 |
| SkyBlue | 135 | 206 | 235 |
| SkyBlue1 | 135 | 206 | 255 |
| SkyBlue2 | 126 | 192 | 238 |
| SkyBlue3 | 108 | 166 | 205 |
| SkyBlue4 | 74 | 112 | 139 |
| slate blue | 106 | 90 | 205 |
| slate gray | 112 | 128 | 144 |
| slate grey | 112 | 128 | 144 |
| SlateBlue | 106 | 90 | 205 |
| SlateBlue1 | 131 | 111 | 255 |
| SlateBlue2 | 122 | 103 | 238 |
| SlateBlue3 | 105 | 89 | 205 |
| SlateBlue4 | 71 | 60 | 139 |
| SlateGray | 112 | 128 | 144 |
| SlateGray1 | 198 | 226 | 255 |
| SlateGray2 | 185 | 211 | 238 |
| SlateGray3 | 159 | 182 | 205 |
| SlateGray4 | 108 | 123 | 139 |
| SlateGrey | 112 | 128 | 144 |
| snow | 255 | 250 | 250 |
| snow1 | 255 | 250 | 250 |
| snow2 | 238 | 233 | 233 |
| snow3 | 205 | 201 | 201 |
| snow4 | 139 | 137 | 137 |
| spring green | 0 | 255 | 127 |
| SpringGreen | 0 | 255 | 127 |
| SpringGreen1 | 0 | 255 | 127 |
| SpringGreen2 | 0 | 238 | 118 |
| SpringGreen3 | 0 | 205 | 102 |
| SpringGreen4 | 0 | 139 | 69 |
| steel blue | 70 | 130 | 180 |
| SteelBlue | 70 | 130 | 180 |
| SteelBlue1 | 99 | 184 | 255 |
| SteelBlue2 | 92 | 172 | 238 |
| SteelBlue3 | 79 | 148 | 205 |
| SteelBlue4 | 54 | 100 | 139 |
| tan | 210 | 180 | 140 |
| tan1 | 255 | 165 | 79 |

| | | | |
|---|---|---|---|
| tan2 | 238 | 154 | 73 |
| tan3 | 205 | 133 | 63 |
| tan4 | 139 | 90 | 43 |
| teal | 0 | 128 | 128 |
| thistle | 216 | 191 | 216 |
| thistle1 | 255 | 225 | 255 |
| thistle2 | 238 | 210 | 238 |
| thistle3 | 205 | 181 | 205 |
| thistle4 | 139 | 123 | 139 |
| tomato | 255 | 99 | 71 |
| tomato1 | 255 | 99 | 71 |
| tomato2 | 238 | 92 | 66 |
| tomato3 | 205 | 79 | 57 |
| tomato4 | 139 | 54 | 38 |
| turquoise | 64 | 224 | 208 |
| turquoise1 | 0 | 245 | 255 |
| turquoise2 | 0 | 229 | 238 |
| turquoise3 | 0 | 197 | 205 |
| turquoise4 | 0 | 134 | 139 |
| violet | 238 | 130 | 238 |
| violet red | 208 | 32 | 144 |
| VioletRed | 208 | 32 | 144 |
| VioletRed1 | 255 | 62 | 150 |
| VioletRed2 | 238 | 58 | 140 |
| VioletRed3 | 205 | 50 | 120 |
| VioletRed4 | 139 | 34 | 82 |
| wheat | 245 | 222 | 179 |
| wheat1 | 255 | 231 | 186 |
| wheat2 | 238 | 216 | 174 |
| wheat3 | 205 | 186 | 150 |
| wheat4 | 139 | 126 | 102 |
| white | 255 | 255 | 255 |
| white smoke | 245 | 245 | 245 |
| WhiteSmoke | 245 | 245 | 245 |
| yellow | 255 | 255 | 0 |
| yellow green | 154 | 205 | 50 |
| yellow1 | 255 | 255 | 0 |
| yellow2 | 238 | 238 | 0 |
| yellow3 | 205 | 205 | 0 |
| yellow4 | 139 | 139 | 0 |
| YellowGreen | 154 | 205 | 50 |

## Portabilität

### macOS

Unter macOS sind die folgenden zusätzlichen Systemfarben verfügbar. Diese erste Gruppe enthält alle HIBrush-Farben, die in der HIToolbox-Bibliothek verfügbar sind. Beachten Sie, dass unter macOS 10.14 (Mojave) und später diese Farben wahrscheinlich nicht mit der Farbe übereinstimmen, die tatsächlich für den durch den Farbnamen angedeuteten Zweck verwendet wird..

systemActiveAreaFill
systemAlertBackgroundActive
systemAlertBackgroundInactive
systemAlternatePrimaryHighlightColor
systemAppleGuideCoachmark
systemBevelActiveDark
systemBevelActiveLight
systemBevelInactiveDark
systemBevelInactiveLight
systemBlack
systemButtonActiveDarkHighlight
systemButtonActiveDarkShadow
systemButtonActiveLightHighlight
systemButtonActiveLightShadow
systemButtonFace
systemButtonFaceActive
systemButtonFaceInactive
systemButtonFacePressed
systemButtonFrame
systemButtonFrameActive
systemButtonFrameInactive
systemButtonInactiveDarkHighlight
systemButtonInactiveDarkShadow
systemButtonInactiveLightHighlight
systemButtonInactiveLightShadow
systemButtonPressedDarkHighlight
systemButtonPressedDarkShadow
systemButtonPressedLightHighlight
systemButtonPressedLightShadow
systemChasingArrows
systemDialogBackgroundActive
systemDialogBackgroundInactive
systemDocumentWindowBackground
systemDragHilite
systemDrawerBackground
systemFinderWindowBackground
systemFocusHighlight
systemHighlight
systemHighlightAlternate
systemHighlightSecondary
systemIconLabelBackground
systemIconLabelBackgroundSelected
systemListViewBackground

systemListViewColumnDivider
systemListViewEvenRowBackground
systemListViewOddRowBackground
systemListViewSeparator
systemListViewSortColumnBackground
systemMenu
systemMenuActive
systemMenuBackground
systemMenuBackgroundSelected
systemModelessDialogBackgroundActive
systemModelessDialogBackgroundInactive
systemMovableModalBackground
systemNotificationWindowBackground
systemPopupArrowActive
systemPopupArrowInactive
systemPopupArrowPressed
systemPrimaryHighlightColor
systemScrollBarDelimiterActive
systemScrollBarDelimiterInactive
systemSecondaryHighlightColor
systemSelectedTabTextColor
systemSheetBackground
systemSheetBackgroundOpaque
systemSheetBackgroundTransparent
systemStaticAreaFill
systemToolbarBackground
systemTransparent
systemUtilityWindowBackgroundActive
systemUtilityWindowBackgroundInactive
systemWhite
systemWindowBody

Tk unterstützt alle NSColors in der macOS System ColorList. Die Namenskonvention für diese Farben ist, dass der Tk-Name durch Großschreibung des macOS-Namens und Hinzufügen des Präfixes "system" erzeugt wird. Unter macOS 10.14 (Mojave) und später erscheinen viele dieser "semantischen" Farben unterschiedlich, je nachdem, ob das NSWindow, in dem sie verwendet werden, das Aqua- oder DarkAqua-Erscheinungsbild hat. Die System ColorList unterscheidet sich zwischen macOS-Versionen, und einige Farben, wie systemLinkColor und systemControlAccentColor, werden auf älteren Systemen simuliert, die sie nicht bereitstellten. Alle unten aufgeführten Farben sind auf allen unterstützten macOS-Versionen verfügbar, aber neuere Systeme unterstützen zusätzliche Farben..

systemControlAccentColor
systemControlTextColor
systemDisabledControlTextColor
systemLabelColor
systemLinkColor

systemPlaceholderTextColor
systemSelectedTextBackgroundColor
systemSelectedTextColor
systemSeparatorColor
systemTextBackgroundColor
systemTextColor

Die nummerierten systemWindowBackgroundColors unten werden in den Widgets ttk::notebook und ttk::labelframe verwendet, um einen kontrastierenden Hintergrund bereitzustellen. Jede nummerierte Farbe kontrastiert mit ihrem Vorgänger..

systemWindowBackgroundColor
systemWindowBackgroundColor1
systemWindowBackgroundColor2
systemWindowBackgroundColor3
systemWindowBackgroundColor4
systemWindowBackgroundColor5
systemWindowBackgroundColor6
systemWindowBackgroundColor7

**Windows**

Unter Windows sind die folgenden zusätzlichen Systemfarben verfügbar (beachten Sie, dass die tatsächlichen Farbwerte vom derzeit aktiven Betriebssystem-Design abhängen):.

system3dDarkShadow
system3dLight
systemActiveBorder
systemActiveCaption
systemAppWorkspace
systemBackground
systemButtonFace
systemButtonHighlight
systemButtonShadow
systemButtonText
systemCaptionText
systemDisabledText
systemGrayText
systemHighlight
systemHighlightText
systemInactiveBorder
systemInactiveCaption
systemInactiveCaptionText
systemInfoBackground
systemInfoText
systemMenu
systemMenuText
systemPlaceholderText
systemScrollbar
systemWindow
systemWindowFrame
systemWindowText

*Copyright (c) 1998-2000 Scriptics Corporation.*
*Copyright (c) 2003 ActiveState Corporation.*
*Copyright (c) 2006-2007 Daniel A. Steffen* das@users.sourceforge.net
*Copyright (c) 2008 Donal K. Fellows*

# console

*Stichwörter:* console, interpreter, window, interactive, output channels

## Name

console - Steuert die Konsole auf Systemen ohne echte Konsole

## Übersicht

**console** *subcommand* ?*arg* ...?

## Beschreibung

Das Konsolenfenster ersetzt eine echte Konsole, um Ein- und Ausgabe auf den Standard-I/O-Kanälen auf Plattformen ohne echte Konsole zu ermöglichen. Es ist als separater Interpreter mit geladenem Tk-Toolkit implementiert; die Steuerung erfolgt über den Befehl **console**. Das Verhalten des Konsolenfensters wird vor allem durch den Inhalt der Datei *console.tcl* in der Tk-Bibliothek definiert. Außer bei TkAqua ist dieser Befehl nicht verfügbar, wenn Tk mit "**package require tk**" in einen tclsh-Interpreter geladen wird, da dort ein herkömmliches Terminal erwartet wird. Unter TkAqua ist der Befehl deaktiviert, wenn ein Startskript vorliegt und stdin **/dev/null** ist (wie z. B. wenn eine gebündelte Tk-Anwendung vom macOS Launcher gestartet wird). Um den Befehl in diesem Fall zu aktivieren, definieren Sie die Umgebungsvariable **TK_CONSOLE**. Dies kann durch Anpassen der Info.plist erfolgen: Schlüssel LSEnvironment zum Haupt-dict hinzufügen und als Wert ein dict mit dem Schlüssel **TK_CONSOLE** setzen.

**console eval *script***

Wertet das Argument *script* als Tcl-Skript im Konsolen-Interpreter aus. Auf den normalen Interpreter wird im Konsolen-Interpreter über den Befehl **consoleinterp** zugegriffen.

**console hide**

Blendet das Konsolenfenster aus. Entspricht genau dem Zurückziehen des Fensters **.** im Konsolen-Interpreter.

**console show**

Zeigt das Konsolenfenster an. Entspricht genau dem Deiconify des Fensters **.** im Konsolen-Interpreter.

**console title ?*string*?**

Fragt den Titel des Konsolenfensters ab oder ändert ihn. Ist *string* nicht angegeben, wird der Titel abgefragt; andernfalls auf *string* gesetzt. Entspricht genau dem Befehl wm title im Konsolen-Interpreter.

## Zugriff auf den Haupt-Interpreter

Der Befehl **consoleinterp** im Konsolen-Interpreter erlaubt die Auswertung von Skripten im Haupt-Interpreter. Er unterstützt zwei Unterbefehle: **eval** und **record**.

**consoleinterp eval *script***

Wertet *script* als Tcl-Skript auf globaler Ebene im Haupt-Interpreter aus.

**consoleinterp record *script***

Zeichnet *script* auf und wertet es auf globaler Ebene im Haupt-Interpreter aus, als ob *script* an der Konsole eingegeben worden wäre.

## Zusätzliche Trap-Aufrufe

Im Konsolen-Interpreter gibt es mehrere zusätzliche Befehle, die als Reaktion auf Aktivität im Haupt-Interpreter aufgerufen werden. *Sie sind hier nur der Vollständigkeit halber dokumentiert; sie gehören zur internen Implementierung der Konsole und können ohne Vorwarnung geändert werden.*

Ausgabe der Konsole aus dem Haupt-Interpreter über die Kanäle stdout und stderr wird durch

Aufruf des Befehls **tk::ConsoleOutput** im Konsolen-Interpreter mit zwei Argumenten behandelt. Das erste Argument ist der Name des beschriebenen Kanals, das zweite der String, der in den Kanal geschrieben wird (nach Kodierungs- und Zeilenend-Verarbeitung).

Wenn das Fenster**.** des Haupt-Interpreters zerstört wird, wird der Befehl **tk::ConsoleExit** im Konsolen-Interpreter aufgerufen (sofern der Konsolen-Interpreter nicht bereits selbst gelöscht wurde).

## Standard-Bindings

Das Standardskript erzeugt ein Konsolenfenster (implementiert mit einem Text-Widget) mit folgendem Verhalten:

1. Tab oder Escape versucht, den Text unmittelbar vor dem Cursor zu vervollständigen. Die Konsole probiert der Reihe nach einen Dateinamen im aktuellen Verzeichnis, dann einen Befehlsnamen, dann einen Variablennamen und hört beim ersten Typ auf, der einen Treffer liefert.

   Nur dieser Typ wird danach verwendet. Gibt es darin mehr als einen Treffer, wird die längste gemeinsame Übereinstimmung eingesetzt; die vollständige Liste erscheint erst, wenn die Vervollständigung erneut versucht wird und nichts mehr zu verlängern ist.

   Eigene Vervollständigungs-Bindings für Dateien, Befehle und Variablen stehen unten (Punkte 12–14).
2. Return-Taste übergibt die aktuelle Zeile (wenn sie nach den Regeln von info complete vollständig ist) zur Auswertung an den Haupt-Interpreter.
3. Delete-Taste löscht den ausgewählten Text (falls vorhanden) oder das Zeichen rechts vom Cursor (wenn nicht am Zeilenende).
4. Backspace-Taste löscht den ausgewählten Text (falls vorhanden) oder das Zeichen links vom Cursor (wenn nicht am Zeilenanfang).
5. Control+A oder Home-Taste setzt den Cursor an den Zeilenanfang (aber nach der Eingabeaufforderung, falls eine auf der Zeile steht).
6. Control+E oder End-Taste setzt den Cursor ans Zeilenende.
7. Control+P oder Pfeil-hoch wählt den vorherigen Eintrag in der Befehlshistorie.
8. Control+N oder Pfeil-runter wählt den nächsten Eintrag in der Befehlshistorie.
9. Control+B oder Pfeil-links bewegt den Cursor ein Zeichen zurück, solange der Cursor nicht bei der Eingabeaufforderung steht.
10. Control+F oder Pfeil-rechts bewegt den Cursor ein Zeichen vorwärts.
11. F9 baut das Konsolenfenster neu auf, indem alle Kinder zerstört und das Tcl-Skript neu geladen wird, das das Konsolenverhalten definiert.
12. Control+Umschalt+F vervollständigt nur gegen Dateinamen im aktuellen Verzeichnis, unabhängig davon, was Tab oder Escape sonst zuerst versuchen würde.
13. Control+Umschalt+P vervollständigt nur gegen Befehlsnamen (intern "proc"-Vervollständigung, trifft aber jeden Befehl, nicht nur benutzerdefinierte Prozeduren).
14. Control+Umschalt+V vervollständigt nur gegen Variablennamen. Sieht der zu vervollständigende Text wie ein Array-Element aus (array(name)), wird gegen die Elementnamen dieses Arrays vervollständigt statt allgemein gegen Variablennamen.
15. Drehen am Mausrad bei gedrückter Cmd-Taste (macOS) oder Ctrl (andere Plattformen) vergrößert (nach oben) oder verkleinert (nach unten) die Konsolenschrift.

Das übrige Verhalten entspricht einem herkömmlichen Text-Widget, außer dass das Ereignis <>

identisch zu <> behandelt wird.

### Beispiel

Nicht alle Plattformen haben den Befehl **console**; Debug-Code enthält daher oft folgendes Fragment, damit Ausgabe von puts während der Entwicklung sichtbar ist:

```
catch {console show}
```

*Copyright (c) 2001 Donal K. Fellows*

## cursors

*Stichwörter:* cursors, cursor, option

### Name

cursors - in Tk verfügbare Mauszeiger

### Beschreibung

Die Widget-Option **-cursor** ermöglicht es, den Mauszeiger für ein bestimmtes Widget zu ändern. Die von Tk auf allen Plattformen erkannten Zeigernamen sind:

```
X_cursor
arrow
based_arrow_down
based_arrow_up
boat
bogosity
bottom_left_corner
bottom_right_corner
bottom_side
bottom_tee
box_spiral
center_ptr
circle
clock
coffee_mug
cross
cross_reverse
crosshair
diamond_cross
dot
dotbox
double_arrow
draft_large
draft_small
draped_box
exchange
fleur
gobbler
```

```
gumby
hand1
hand2
heart
icon
iron_cross
left_ptr
left_side
left_tee
leftbutton
ll_angle
lr_angle
man
middlebutton
mouse
none
pencil
pirate
plus
question_arrow
right_ptr
right_side
right_tee
rightbutton
rtl_logo
sailboat
sb_down_arrow
sb_h_double_arrow
sb_left_arrow
sb_right_arrow
sb_up_arrow
sb_v_double_arrow
shuttle
sizing
spider
spraycan
star
target
tcross
top_left_arrow
top_left_corner
top_right_corner
top_side
top_tee
trek
ul_angle
```

```
umbrella
ur_angle
watch
xterm
```

Der Zeiger **none** kann angegeben werden, um den Zeiger zu unterdrücken.

## Portierbarkeitsprobleme

### Windows

Unter Windows werden folgende Zeiger nativen Zeigern zugeordnet:

```
arrow
center_ptr
crosshair
fleur
ibeam
icon
none
sb_h_double_arrow
sb_v_double_arrow
watch
xterm
```

Zusätzlich stehen folgende Zeiger zur Verfügung:

```
no
starting
size
size_ne_sw
size_ns
size_nw_se
size_we
uparrow
wait
```

### macOS

Unter macOS werden folgende Zeiger nativen Zeigern zugeordnet:

```
arrow
top_left_arrow
left_ptr
cross
crosshair
tcross
ibeam
none
xterm
```

Zusätzlich stehen folgende native Zeiger zur Verfügung:

```
copyarrow
aliasarrow
contextualmenuarrow
movearrow
text
cross-hair
hand
openhand
closedhand
fist
pointinghand
resize
resizeleft
resizeright
resizeleftright
resizeup
resizedown
resizeupdown
resizebottomleft
resizetopleft
resizebottomright
resizetopright
notallowed
poof
wait
countinguphand
countingdownhand
countingupanddownhand
spinning
help
bucket
cancel
eyedrop
eyedrop-full
zoom-in
zoom-out
```

*Copyright (c) 1998-2000 Scriptics Corporation.*
*Copyright (c) 2006-2007 Daniel A. Steffen* das@users.sourceforge.net

# destroy

*Stichwörter:* destroy, application, window

## Name

destroy - Zerstört ein oder mehrere Fenster

## Übersicht

**destroy** ?*window window* ...?

## Beschreibung

Dieser Befehl löscht die durch die *window*-Argumente angegebenen Fenster sowie alle ihre Nachkommen. Wird das Fenster "." gelöscht, werden alle Fenster zerstört und die Anwendung beendet (normalerweise) sich. Die Fenster werden in der angegebenen Reihenfolge zerstört; tritt beim Zerstören eines Fensters ein Fehler auf, bricht der Befehl ab, ohne die verbleibenden Fenster zu zerstören. Existiert *window* nicht, wird kein Fehler zurückgegeben.

## Beispiele

Zerstört alle Checkbuttons, die direkte Kinder des angegebenen Widgets sind:

```
proc killCheckbuttonChildren {parent} {
   foreach w [winfo children $parent] {
      if {[winfo class $w] eq "Checkbutton"} {
         destroy $w
      }
   }
}
```

*Copyright (c) 1990 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# dialog

*Stichwörter:* dialog, bitmap, modal

## Name

tk_dialog - Erstellt einen modalen Dialog und wartet auf die Antwort

## Übersicht

**tk_dialog** *window title text bitmap default string string ...*

## Beschreibung

Diese Prozedur gehört zur Tk-Skriptbibliothek. Sie ist weitgehend durch tk_messageBox *veraltet*. Ihre Argumente beschreiben ein Dialogfeld:

***window***
Name des Toplevel-Fensters für den Dialog. Ein bereits vorhandenes Fenster mit diesem Namen wird zerstört.

***title***
Text in der Titelleiste des Fenstermanagers für den Dialog.

***text***
Meldung im oberen Bereich des Dialogfelds.

***bitmap***
Wenn nicht leer, legt dies ein Bitmap fest (in einer für Tk_GetBitmap geeigneten Form), das im oberen Bereich des Dialogs links neben dem Text angezeigt wird. Bei einer leeren Zeichenkette wird kein Bitmap angezeigt.

***default***

Ist dies eine ganze Zahl größer oder gleich null, gibt sie den Index des Standard-Buttons für den Dialog an (0 für den linken Button usw.). Ist der Wert negativ oder eine leere Zeichenkette, gibt es keinen Standard-Button.

***string***

Für jedes dieser Argumente gibt es einen Button. Jede *string*-Angabe legt den auf dem Button anzuzeigenden Text fest, von links nach rechts.

Nach dem Erzeugen des Dialogfelds wartet **tk_dialog**, bis der Benutzer einen der Buttons wählt — per Mausklick oder durch Drücken von Return für den Standard-Button (falls vorhanden). Anschließend gibt der Befehl den Index des gewählten Buttons zurück: 0 für den linken, 1 für den nächsten usw. Wird das Dialogfenster zerstört, bevor der Benutzer einen Button wählt, wird -1 zurückgegeben.

Während des Wartens setzt **tk_dialog** einen lokalen Grab. Der Benutzer kann die Anwendung dann nur noch über das Dialogfeld bedienen.

## Beispiel

```
set reply [tk_dialog .foo "The Title" "Do you want to say yes?" \
        questhead 0 Yes No "I'm not sure"]
```

*Copyright (c) 1992 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# entry

*Stichwörter:* entry, widget

## Name

entry - Erstellt und manipuliert 'Entry'-Einzeilige-Text-Eingabe-Widgets

## Übersicht

**entry** *pathName* ?*options*?

## Standardoptionen

**-background**, **-highlightthickness**, **-selectbackground**, **-borderwidth**, **-insertbackground**, **-selectborderwidth**, **-cursor**, **-insertborderwidth**, **-selectforeground**, **-exportselection**, **-insertofftime**, **-takefocus**, **-font**, **-insertontime**, **-textvariable**, **-foreground**, **-insertwidth**, **-xscrollcommand**, **-highlightbackground**, **-justify**, **-highlightcolor**, **-relief**, **-placeholder**, **-placeholderforeground**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-disabledbackground (disabledBackground/DisabledBackground)**

Legt die Hintergrundfarbe fest, die verwendet wird, wenn das Entry deaktiviert ist. Wenn diese Option der leere String ist, wird die normale Hintergrundfarbe verwendet.

**-disabledforeground (disabledForeground/DisabledForeground)**

Legt die Vordergrundfarbe fest, die verwendet wird, wenn das Entry deaktiviert ist. Wenn diese Option der leere String ist, wird die normale Vordergrundfarbe verwendet.

**-invalidcommand, -invcmd (invalidCommand/InvalidCommand)**
Legt ein Script fest, das ausgeführt wird, wenn **-validatecommand** 0 zurückgibt. Das Setzen auf {} deaktiviert diese Funktion (Standard). Die beste Verwendung dieser Option ist, sie auf *bell* zu setzen. Siehe Validierung unten für weitere Informationen.

**-readonlybackground (readonlyBackground/ReadonlyBackground)**
Legt die Hintergrundfarbe fest, die verwendet wird, wenn das Entry schreibgeschützt ist. Wenn diese Option der leere String ist, wird die normale Hintergrundfarbe verwendet.

**-show (show/Show)**
Wenn diese Option angegeben ist, wird der wahre Inhalt des Entrys nicht im Fenster angezeigt. Stattdessen wird jedes Zeichen im Wert des Entrys als das erste Zeichen im Wert dieser Option angezeigt, wie z.B. "*". Dies ist nützlich, wenn das Entry beispielsweise zur Eingabe eines Passworts verwendet werden soll. Wenn Zeichen im Entry ausgewählt und anderswo kopiert werden, werden die angezeigten Informationen kopiert, nicht der wahre Inhalt des Entrys.

**-state (state/State)**
Legt einen von drei Zuständen für das Entry fest: **normal**, **disabled** oder **readonly**. Wenn das Entry schreibgeschützt ist, kann der Wert nicht mit Widget-Befehlen geändert werden und es wird kein Einfügecursor angezeigt, auch wenn der Eingabefokus im Widget ist; der Inhalt des Widgets kann weiterhin ausgewählt werden. Wenn das Entry deaktiviert ist, kann der Wert nicht geändert werden, es wird kein Einfügecursor angezeigt, der Inhalt ist nicht auswählbar, und das Entry kann in einer anderen Farbe angezeigt werden, abhängig von den Werten der Optionen **-disabledforeground** und **-disabledbackground**.

**-validate (validate/Validate)**
Legt den Modus fest, in dem die Validierung arbeiten soll: **none**, **focus**, **focusin**, **focusout**, **key** oder **all**. Der Standard ist **none**. Wenn Sie Validierung wünschen, müssen Sie explizit angeben, welchen Modus Sie verwenden möchten. Siehe Validierung unten für weitere Informationen.

**-validatecommand, -vcmd (validateCommand/ValidateCommand)**
Legt ein Script fest, das ausgeführt wird, wenn Sie die Eingabe in das Entry-Widget validieren möchten. Das Setzen auf {} deaktiviert diese Funktion (Standard). Dieser Befehl muss einen gültigen Tcl-Boolean-Wert zurückgeben. Wenn er 0 zurückgibt (oder das gültige Tcl-Boolean-Äquivalent), bedeutet dies, dass Sie die neue Bearbeitung ablehnen und sie nicht erfolgt und das **-invalidcommand** ausgewertet wird, falls gesetzt. Wenn er 1 zurückgibt, erfolgt die neue Bearbeitung. Siehe Validierung unten für weitere Informationen.

**-width (width/Width)**
Legt einen ganzzahligen Wert fest, der die gewünschte Breite des Entry-Fensters angibt, in Zeichen durchschnittlicher Größe der Schriftart des Widgets. Wenn der Wert negativ oder null ist, wählt das Widget eine Größe, die gerade groß genug ist, um seinen aktuellen Text zu halten.

## Beschreibung

Der Befehl **entry** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Entry-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des Entrys wie Farben, Schriftart und Relief zu konfigurieren. Der Befehl **entry** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Ein Entry ist ein Widget, das einen einzeiligen Text-String anzeigt und es erlaubt, diesen String mit unten beschriebenen Widget-Befehlen zu bearbeiten, die typischerweise an Tastendrücke und Mausaktionen gebunden sind. Wenn es erstellt wird, ist der String des Entrys leer. Ein Teil des Entrys kann wie unten beschrieben ausgewählt werden. Wenn ein Entry seine Auswahl exportiert (siehe die Option **-exportselection**), beachtet es die Standard-X11-Protokolle für die Behandlung der Auswahl; Entry-Auswahlen sind als Typ **STRING** verfügbar. Entrys beachten auch die Standard-Tk-Regeln für den Umgang mit dem Eingabefokus. Wenn ein Entry den Eingabefokus hat, zeigt es einen *Einfügecursor* an, um anzuzeigen, wo neue Zeichen eingefügt werden.

Entrys können Strings anzeigen, die zu lang sind, um vollständig in das Fenster des Widgets zu passen. In diesem Fall wird nur ein Teil des Strings angezeigt; unten beschriebene Befehle können verwendet werden, um die Ansicht im Fenster zu ändern. Entrys verwenden den Standard-Mechanismus **-xscrollcommand** für die Interaktion mit Scrollbars (siehe die Beschreibung der Option **-xscrollcommand** für Details). Sie unterstützen auch das Scannen, wie unten beschrieben.

## Validierung

Die Validierung funktioniert, indem die Option **-validatecommand** auf ein Script ( *validateCommand*) gesetzt wird, das gemäß der Option **-validate** wie folgt ausgewertet wird:

**none**
Standard. Es findet keine Validierung statt.

focus
*validateCommand* wird aufgerufen, wenn das Entry den Fokus erhält oder verliert.

**focusin**
*validateCommand* wird aufgerufen, wenn das Entry den Fokus erhält.

**focusout**
*validateCommand* wird aufgerufen, wenn das Entry den Fokus verliert.

**key**
*validateCommand* wird aufgerufen, wenn das Entry bearbeitet wird.

**all**
*validateCommand* wird bei allen obigen Bedingungen aufgerufen.

Es ist möglich, Prozent-Substitutionen auf den Werten der Optionen **-validatecommand** und **-invalidcommand** durchzuführen, genau wie in einem bind-Script. Folgende Substitutionen werden erkannt:

**%d**
Art der Aktion: 1 für **insert**, 0 für **delete** oder -1 für Fokus-, erzwungene oder **textvariable** -Validierung.

**%i**
Index des einzufügenden/zu löschenden Zeichenstrings, falls vorhanden, sonst -1.

**%P**
Der Wert des Entrys, wenn die Bearbeitung erlaubt ist. Wenn Sie das Entry-Widget mit einer neuen **textvariable** konfigurieren, ist dies der Wert dieser **textvariable**.

**%s**
Der aktuelle Wert des Entrys vor der Bearbeitung.

**%S**
Der einzufügende/zu löschende Textstring, falls vorhanden, sonst {}.

**%v**
Der Typ der aktuell gesetzten Validierung.

**%V**
Der Typ der Validierung, die die Rückruffunktion ausgelöst hat (key, focusin, focusout, forced).

**%W**
Der Name des Entry-Widgets.

Im Allgemeinen können die Optionen **-textvariable** und **-validatecommand** problematisch sein, wenn sie gemischt werden. Alle Probleme wurden behoben, sodass die Verwendung von **-validatecommand** das traditionelle Verhalten des Entry-Widgets nicht stört. Die Verwendung von **-textvariable** nur zu Lesezwecken verursacht nie Probleme. Die Gefahr entsteht, wenn Sie versuchen, **-textvariable** auf etwas zu setzen, was **-validatecommand** nicht akzeptieren würde; dadurch wird **-validate** auf *none* gesetzt (**-invalidcommand** wird nicht ausgelöst). Dasselbe geschieht, wenn beim Auswerten von **-validatecommand** ein Fehler auftritt.

Vor allem tritt ein Fehler auf, wenn **-validatecommand** oder **-invalidcommand** beim Auswerten einen Fehler in seinem Script hat oder **-validatecommand** keinen gültigen Tcl-Boolean-Wert zurückgibt. Die Option **-validate** setzt sich außerdem auf **none**, wenn Sie das Entry-Widget innerhalb von **-validatecommand** oder **-invalidcommand** bearbeiten. Solche Bearbeitungen überschreiben die gerade validierte. Wenn Sie das Entry-Widget während der Validierung bearbeiten möchten (z. B. auf {} setzen) und die Option **-validate** gesetzt bleiben soll, sollten Sie den Befehl

```
after idle {%W config -validate %v}
```

in **-validatecommand** oder **-invalidcommand** (je nachdem, von welchem aus Sie das Entry bearbeitet haben) aufnehmen. Es wird auch empfohlen, während der Validierung keine zugehörige **-textvariable** zu setzen, da das Entry-Widget sonst aus dem Takt mit **-textvariable** geraten kann.

## Widget-Befehl

Der Befehl **entry** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName subcommand ?arg ...?
```

*Subcommand* und die *arg*s bestimmen das genaue Verhalten des Befehls.

### Indizes

Viele Widget-Befehle für Entrys nehmen einen oder mehrere Indizes als Argumente. Ein Index bezeichnet ein bestimmtes Zeichen im String des Entrys, auf eine der folgenden Arten:

***number***
Bezeichnet das Zeichen als numerischen Index, wobei 0 dem ersten Zeichen im String entspricht.

**anchor**
Bezeichnet den Ankerpunkt für die Auswahl, der mit den Widget-Befehlen **select from** und **select adjust** gesetzt wird.

**end**
Bezeichnet das Zeichen unmittelbar nach dem letzten im String des Entrys. Entspricht einem numerischen Index gleich der Länge des Strings des Entrys.

**insert**
Bezeichnet das Zeichen neben und unmittelbar nach dem Einfügecursor.

**sel.first**
Bezeichnet das erste Zeichen in der Auswahl. Die Verwendung dieser Form ist ein Fehler, wenn die Auswahl nicht im Entry-Fenster liegt.

**sel.last**
Bezeichnet das Zeichen unmittelbar nach dem letzten in der Auswahl. Die Verwendung dieser Form ist ein Fehler, wenn die Auswahl nicht im Entry-Fenster liegt.

**@*number***
In dieser Form wird *number* als x-Koordinate im Fenster des Entrys behandelt; das Zeichen, das diese x-Koordinate überspannt, wird verwendet. Beispielsweise bezeichnet "**@0**" das Zeichen ganz links im Fenster.

Abkürzungen sind für alle obigen Formen möglich, z. B. "**e**" oder "**sel.f**". Im Allgemeinen werden außerhalb des gültigen Bereichs liegende Indizes automatisch auf den nächsten zulässigen Wert gerundet. Indizes unterstützen dieselbe einfache Interpretation wie beim Befehl **string index**, mit einfacher ganzzahliger Indexarithmetik und Indexierung relativ zu **end**.

## Unterbefehle

Folgende Befehle sind für Entry-Widgets möglich:

***pathName* bbox *index***
Gibt eine Liste von vier Zahlen zurück, die die Begrenzungsbox des durch *index* angegebenen Zeichens beschreiben. Die ersten beiden Elemente der Liste geben die x- und y-Koordinaten der oberen linken Ecke des vom Zeichen bedeckten Bildschirmbereichs an (in Pixeln relativ zum Widget), die letzten beiden geben Breite und Höhe des Zeichens in Pixeln an. Die Begrenzungsbox kann sich auf einen Bereich außerhalb des sichtbaren Fensterbereichs beziehen.

***pathName* cget *option***
Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **entry** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**
Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **entry** akzeptierten Werte haben.

***pathName* delete *first ?last?***

Löscht ein oder mehrere Elemente des Entrys. *First* ist der Index des ersten zu löschenden Zeichens, *last* der Index des Zeichens unmittelbar nach dem letzten zu löschenden. Wenn *last* nicht angegeben ist, entspricht es *first*+1, d. h. es wird ein einzelnes Zeichen gelöscht. Dieser Befehl gibt einen leeren String zurück.

***pathName* get**

Gibt den String des Entrys zurück.

***pathName* icursor *index***

Sorgt dafür, dass der Einfügecursor unmittelbar vor dem durch *index* gegebenen Zeichen angezeigt wird. Gibt einen leeren String zurück.

***pathName* index *index***

Gibt den numerischen Index zurück, der *index* entspricht.

***pathName* insert *index string***

Fügt die Zeichen von *string* unmittelbar vor dem durch *index* angegebenen Zeichen ein. Gibt einen leeren String zurück.

***pathName* scan *option args***

Dieser Befehl implementiert das Scannen auf Entrys. Er hat zwei Formen, abhängig von *option*:

***pathName* scan mark *x***

Speichert *x* und die aktuelle Ansicht im Entry-Fenster; wird zusammen mit späteren **scan dragto**-Befehlen verwendet. Typischerweise ist dieser Befehl mit einem Maustastendruck im Widget verknüpft. Er gibt einen leeren String zurück.

***pathName* scan dragto *x***

Dieser Befehl berechnet die Differenz zwischen seinem *x*-Argument und dem *x*-Argument des letzten **scan mark**-Befehls für das Widget. Anschließend wird die Ansicht um das Zehnfache der Differenz der x-Koordinaten nach links oder rechts verschoben. Typischerweise ist dieser Befehl mit Mausbewegungs-Events im Widget verknüpft, um den Effekt zu erzeugen, das Entry mit hoher Geschwindigkeit durch das Fenster zu ziehen. Der Rückgabewert ist ein leerer String.

***pathName* selection *option arg***

Dieser Befehl passt die Auswahl innerhalb eines Entrys an. Er hat mehrere Formen, abhängig von *option*:

***pathName* selection adjust *index***

Findet das Ende der Auswahl, das dem durch *index* gegebenen Zeichen am nächsten liegt, und passt dieses Ende der Auswahl auf *index* an (d. h. einschließlich, aber nicht über *index* hinaus). Das andere Ende der Auswahl wird zum Ankerpunkt für zukünftige **select to** -Befehle. Befindet sich die Auswahl derzeit nicht im Entry, wird eine neue Auswahl erstellt, die die Zeichen zwischen *index* und dem zuletzt gesetzten Auswahl-Ankerpunkt einschließt. Gibt einen leeren String zurück.

***pathName* selection clear**

Löscht die Auswahl, wenn sie derzeit in diesem Widget liegt. Liegt die Auswahl nicht in diesem Widget, hat der Befehl keine Wirkung. Gibt einen leeren String zurück.

***pathName* selection from *index***

Setzt den Auswahl-Ankerpunkt unmittelbar vor das durch *index* gegebene Zeichen. Ändert die Auswahl nicht. Gibt einen leeren String zurück.

***pathName* selection present**

Gibt 1 zurück, wenn im Entry Zeichen ausgewählt sind, 0 wenn nichts ausgewählt ist.

***pathName* selection range *start end***

Setzt die Auswahl so, dass sie die Zeichen ab dem durch *start* indizierten bis vor dem durch *end* indizierten einschließt. Verweist *end* auf dasselbe Zeichen wie *start* oder ein früheres, wird die Auswahl des Entrys gelöscht.

***pathName* selection to *index***

Liegt *index* vor dem Ankerpunkt, wird die Auswahl auf die Zeichen von *index* bis vor den Ankerpunkt gesetzt. Ist *index* gleich dem Ankerpunkt, geschieht nichts. Liegt *index* nach dem Ankerpunkt, wird die Auswahl auf die Zeichen vom Ankerpunkt bis vor *index* gesetzt. Der Ankerpunkt wird durch den zuletzt in diesem Widget ausgeführten Befehl **select from** oder **select adjust** bestimmt. Liegt die Auswahl nicht in diesem Widget, wird eine neue Auswahl mit dem zuletzt für das Widget angegebenen Ankerpunkt erstellt. Gibt einen leeren String zurück.

***pathName* validate**

Dieser Befehl erzwingt eine Auswertung von **-validatecommand** unabhängig von den durch die Option **-validate** festgelegten Bedingungen. Dazu wird **-validate** vorübergehend auf **all** gesetzt. Er gibt 0 oder 1 zurück.

***pathName* xview *args***

Dieser Befehl fragt die horizontale Position des Texts im Fenster des Widgets ab und ändert sie. Er kann eine der folgenden Formen haben:

***pathName* xview**

Gibt eine Liste mit zwei Elementen zurück. Jedes Element ist ein reeller Bruchteil zwischen 0 und 1; zusammen beschreiben sie den horizontal sichtbaren Bereich im Fenster. Beispielsweise bedeuten die Elemente .2 und .6, dass 20 % des Entry-Texts links außerhalb des Bildschirms liegen, die mittleren 40 % im Fenster sichtbar sind und 40 % rechts außerhalb. Dies sind dieselben Werte, die Scrollbars über die Option **-xscrollcommand** übergeben werden.

***pathName* xview *index***

Passt die Ansicht im Fenster so an, dass das durch *index* gegebene Zeichen am linken Rand des Fensters angezeigt wird.

***pathName* xview moveto *fraction***

Passt die Ansicht im Fenster so an, dass das Zeichen *fraction* des Weges durch den Text am linken Rand des Fensters erscheint. *Fraction* muss ein Bruchteil zwischen 0 und 1 sein.

***pathName* xview scroll *number what***

Dieser Befehl verschiebt die Ansicht im Fenster nach links oder rechts gemäß *number* und *what*. *Number* muss eine Ganzzahl oder eine Fließkommazahl sein; ist es eine Fließkommazahl, wird sie in eine Ganzzahl umgewandelt, von 0 weg gerundet. *What* muss **pages** oder **units** oder eine Abkürzung davon sein. Ist *what* **pages**, passt sich die Ansicht um *number* Bildschirmseiten an. Ist *number* negativ, werden Zeichen weiter links sichtbar; ist es positiv, werden Zeichen weiter rechts sichtbar. Ist *what* **units**, verschiebt sich die Ansicht um *number* Zeichen durchschnittlicher Breite auf der Anzeige nach links oder rechts.

## Standard-Bindings

Tk erstellt automatisch Klassen-Bindings für Entrys, die ihnen folgendes Standardverhalten geben. In den Beschreibungen unten bezeichnet "Wort" eine zusammenhängende Gruppe aus Buchstaben, Ziffern oder "_"-Zeichen oder ein einzelnes anderes Zeichen.

1. Ein Klick mit Maustaste 1 positioniert den Einfügecursor unmittelbar vor dem Zeichen unter dem Mauszeiger, setzt den Eingabefokus auf dieses Widget und löscht jede Auswahl im Widget. Ziehen mit Maustaste 1 erzeugt eine Auswahl zwischen Einfügecursor und dem Zeichen unter der Maus.
2. Doppelklick mit Maustaste 1 wählt das Wort unter der Maus aus und positioniert den Einfügecursor am Ende des Worts. Ziehen nach einem Doppelklick erzeugt eine Auswahl aus ganzen Wörtern.
3. Dreifachklick mit Maustaste 1 wählt den gesamten Text im Entry aus und positioniert den Einfügecursor am Zeilenende.
4. Die Enden der Auswahl können durch Ziehen mit Maustaste 1 bei gedrückter Umschalttaste angepasst werden; dabei wird das Ende der Auswahl angepasst, das dem Mauszeiger beim Drücken von Taste 1 am nächsten war. Wurde vor dem Ziehen doppelgeklickt, wird die Auswahl in Einheiten ganzer Wörter angepasst.
5. Klick mit Maustaste 1 bei gedrückter Steuerung positioniert den Einfügecursor im Entry, ohne die Auswahl zu beeinflussen.
6. Werden normale druckbare Zeichen in einem Entry eingegeben, werden sie an der Position des Einfügecursors eingefügt.
7. Die Ansicht im Entry kann durch Ziehen mit der mittleren Maustaste (Taste 2, oder Taste 3 unter TkAqua) angepasst werden. Wird die mittlere Maustaste ohne Mausbewegung geklickt, wird die Auswahl an der Mausposition in das Entry kopiert.
8. Wird die Maus beim Gedrückthalten von Taste 1 links oder rechts aus dem Entry gezogen, scrollt das Entry automatisch, um mehr Text sichtbar zu machen (falls auf der Seite, auf der die Maus das Fenster verließ, noch Text außerhalb des Bildschirms liegt).
9. Die Tasten Links und Rechts bewegen den Einfügecursor um ein Zeichen nach links oder rechts; sie löschen außerdem jede Auswahl im Entry und setzen den Auswahl-Anker. Wird Links oder Rechts mit gedrückter Umschalttaste eingegeben, bewegt sich der Einfügecursor und die Auswahl wird um das neue Zeichen erweitert. Steuerung-Links und Steuerung-Rechts bewegen den Einfügecursor wortweise; Steuerung-Umschalt-Links und Steuerung-Umschalt-Rechts bewegen ihn wortweise und erweitern die Auswahl. Steuerung-b und Steuerung-f verhalten sich wie Links bzw. Rechts. Meta-b und Meta-f verhalten sich wie Steuerung-Links bzw. Steuerung-Rechts.
10. Die Pos1-Taste oder Steuerung-a bewegt den Einfügecursor an den Anfang des Entrys und löscht jede Auswahl. Umschalt-Pos1 bewegt den Einfügecursor an den Anfang und erweitert die Auswahl bis dorthin.
11. Die Ende-Taste oder Steuerung-e bewegt den Einfügecursor ans Ende des Entrys und löscht jede Auswahl. Umschalt-Ende bewegt den Cursor ans Ende und erweitert die Auswahl bis dorthin.
12. Die Select-Taste und Steuerung-Leertaste setzen den Auswahl-Anker auf die Position des Einfügecursors. Sie beeinflussen die aktuelle Auswahl nicht. Umschalt-Select und Steuerung-Umschalt-Leertaste passen die Auswahl an die aktuelle Position des Einfügecursors an und wählen vom Anker bis zum Einfügecursor, falls zuvor keine Auswahl bestand.

13. Steuerung-/ wählt den gesamten Text im Entry aus.
14. Steuerung-\ löscht jede Auswahl im Entry.
15. Die F16-Taste (auf vielen Sun-Arbeitsplätzen mit Copy beschriftet) oder Meta-w kopiert die Auswahl im Widget in die Zwischenablage, falls eine Auswahl besteht.
16. Die F20-Taste (auf vielen Sun-Arbeitsplätzen mit Cut beschriftet) oder Steuerung-w kopiert die Auswahl in die Zwischenablage und löscht sie. Gibt es keine Auswahl im Widget, haben diese Tasten keine Wirkung.
17. Die F18-Taste (auf vielen Sun-Arbeitsplätzen mit Paste beschriftet) oder Steuerung-y fügt den Inhalt der Zwischenablage an der Position des Einfügecursors ein.
18. Die Entf-Taste löscht die Auswahl, falls eine im Entry besteht. Gibt es keine Auswahl, löscht sie das Zeichen rechts vom Einfügecursor.
19. Die Rücktaste und Steuerung-h löschen die Auswahl, falls eine im Entry besteht. Gibt es keine Auswahl, löschen sie das Zeichen links vom Einfügecursor.
20. Steuerung-d löscht das Zeichen rechts vom Einfügecursor.
21. Meta-d löscht das Wort rechts vom Einfügecursor.
22. Steuerung-k löscht alle Zeichen rechts vom Einfügecursor.
23. Steuerung-t kehrt die Reihenfolge der beiden Zeichen rechts vom Einfügecursor um.

Ist das Entry mit der Option **-state** deaktiviert, kann die Ansicht des Entrys weiterhin angepasst und Text im Entry weiterhin ausgewählt werden, aber es wird kein Einfügecursor angezeigt und es finden keine Textänderungen statt, außer wenn das Entry über die Option **-textvariable** mit einer Variable verknüpft ist; in diesem Fall spiegelt das Entry jede Änderung der Variable wider, unabhängig vom Wert seiner Option **-state**.

Das Verhalten von Entrys kann geändert werden, indem neue Bindings für einzelne Widgets definiert oder die Klassen-Bindings neu definiert werden.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 1998-2000 Scriptics Corporation.*

# event

*Stichwörter:* event, binding, define, handle, virtual event

## Name

event - Verschiedene Ereignisfunktionen: virtuelle Ereignisse definieren und Ereignisse erzeugen

## Übersicht

**event** *option* ?*arg* ...?

## Beschreibung

Der Befehl **event** bietet mehrere Einrichtungen zum Umgang mit Fenstersystem-Ereignissen, etwa zum Definieren virtueller Ereignisse und zum Synthetisieren von Ereignissen. Der Befehl hat mehrere Formen, die durch das erste Argument bestimmt werden. Derzeit werden folgende Formen unterstützt:

**event add <> *sequence* ?*sequence* ...?**

Ordnet dem virtuellen Ereignis *virtual* die physischen Ereignissequenz(en) zu, die durch die Argumente *sequence* angegeben werden, sodass das virtuelle Ereignis ausgelöst wird, sobald eine der *sequence*s auftritt. *Virtual* kann ein beliebiger Zeichenkettenwert sein und *sequence* kann jeden der für das Argument *sequence* des Befehls bind zulässigen Werte haben. Ist *virtual* bereits definiert, werden die neuen physischen Ereignissequenzen zu den bestehenden Sequenzen für das Ereignis hinzugefügt.

**event delete <> ?*sequence sequence* ...?**

Löscht jede der *sequence*s aus denen, die mit dem virtuellen Ereignis *virtual* verknüpft sind. *Virtual* kann ein beliebiger Zeichenkettenwert sein und *sequence* kann jeden der für das Argument *sequence* des Befehls bind zulässigen Werte haben. *sequence*s, die derzeit nicht mit *virtual* verknüpft sind, werden ignoriert. Wird kein Argument *sequence* angegeben, werden alle physischen Ereignissequenzen für *virtual* entfernt, sodass das virtuelle Ereignis nicht mehr ausgelöst wird.

**event generate *window event* ?*option value option value* ...?**

Erzeugt ein Fensterereignis und sorgt dafür, dass es genauso verarbeitet wird, als käme es vom Fenstersystem. *Window* gibt den Pfadnamen des Fensters an, für das das Ereignis erzeugt wird; es kann auch eine Kennung sein (wie von winfo id zurückgegeben), solange es ein Fenster der aktuellen Anwendung ist. *Event* liefert eine grundlegende Beschreibung des Ereignisses, etwa oder **<>**. Ist *Window* leer, ist der gesamte Bildschirm gemeint und die Koordinaten beziehen sich auf den Bildschirm. *Event* kann jede der für das Argument *sequence* des Befehls bind zulässigen Formen haben, muss aber aus genau einem Ereignismuster bestehen, nicht aus einer Sequenz. Paare *Option-Wert* können zusätzliche Attribute des Ereignisses angeben, etwa die x- und y-Position der Maus; siehe Ereignisfelder unten. Ist die Option **-when** nicht angegeben, wird das Ereignis sofort verarbeitet: alle Handler für das Ereignis werden abgeschlossen, bevor der Befehl **event generate** zurückkehrt. Ist die Option **-when** angegeben, legt sie fest, wann das Ereignis verarbeitet wird. Bestimmte Ereignisse, etwa Tastaturereignisse, erfordern, dass das Fenster den Fokus hat, um das Ereignis ordnungsgemäß zu empfangen.

**event info ?<>?**

Gibt Informationen über virtuelle Ereignisse zurück. Wird das Argument **<<***virtual***>>** weggelassen, ist der Rückgabewert eine Liste aller derzeit definierten virtuellen Ereignisse. Ist **<<***virtual***>>** angegeben, ist der Rückgabewert eine Liste, deren Elemente die für das angegebene virtuelle Ereignis derzeit definierten physischen Ereignissequenzen sind; ist das virtuelle Ereignis nicht definiert, wird eine leere Zeichenkette zurückgegeben.

Beachten Sie, dass virtuelle Ereignisse, die nicht an physische Ereignissequenzen gebunden sind, von **event info** *nicht* zurückgegeben werden.

## Ereignisfelder

Die folgenden Optionen werden für den Befehl **event generate** unterstützt. Sie entsprechen den "%"-Ersetzungen, die in Bindungsskripten für den Befehl bind erlaubt sind.

**-above *window***

*Window* legt das Feld *above* für das Ereignis fest, entweder als Fensterpfadnamen oder als ganzzahlige Fensterkennung. Gültig für **Configure**-Ereignisse. Entspricht der **%a**-Ersetzung in Bindungsskripten.

**-borderwidth *size***

*Size* muss ein Bildschirmabstand sein; es legt das Feld *border_width* für das Ereignis fest.

Gültig für **Configure**-Ereignisse. Entspricht der **%B**-Ersetzung in Bindungsskripten.

**-button *number***

*Number* muss eine ganze Zahl sein; es legt das Feld *detail* für ein **Button**- oder **ButtonRelease**-Ereignis fest und überschreibt jede im Basisargument *event* angegebene Tastennummer. Entspricht der **%b**-Ersetzung in Bindungsskripten.

**-count *number***

*Number* muss eine ganze Zahl sein; es legt das Feld *count* für das Ereignis fest. Gültig für **Expose**-Ereignisse. Entspricht der **%c**-Ersetzung in Bindungsskripten.

**-data *string***

*String* kann ein beliebiger Wert sein; er legt das Feld *user_data* für das Ereignis fest. Nur gültig für virtuelle Ereignisse. Entspricht der **%d**-Ersetzung für virtuelle Ereignisse in Bindungsskripten.

**-delta *number***

*Number* muss eine ganze Zahl sein; es legt das Feld *delta* für das **MouseWheel**-Ereignis fest. *Delta* bezeichnet Richtung und Ausmaß der Mausrad-Drehung. Der Wert ist kein Bildschirmabstand, sondern Einheiten der Mausrad-Bewegung. Typischerweise sind diese Werte Vielfache von 120. Beispielsweise sollte 120 das Text-Widget um 4 Zeilen nach oben scrollen und -240 um 8 Zeilen nach unten. Andere Widgets können für Mausrad-Bewegung anderes Verhalten definieren. Dieses Feld entspricht der **%D**-Ersetzung in Bindungsskripten.

**-detail *detail***

*Detail* legt das Feld *detail* für das Ereignis fest und muss eines der folgenden sein:

**NotifyAncestor NotifyNonlinearVirtual**
**NotifyDetailNone NotifyPointer**
**NotifyInferior NotifyPointerRoot**
**NotifyNonlinear NotifyVirtual**

Gültig für **Enter**-, **Leave**-, **FocusIn**- und **FocusOut**-Ereignisse. Entspricht der **%d**-Ersetzung in Bindungsskripten.

**-focus *boolean***

*Boolean* muss ein boolescher Wert sein; er legt das Feld *focus* für das Ereignis fest. Gültig für **Enter**- und **Leave**-Ereignisse. Entspricht der **%f**-Ersetzung in Bindungsskripten.

**-height *size***

*Size* muss ein Bildschirmabstand sein; er legt das Feld *height* für das Ereignis fest. Gültig für **Configure**-Ereignisse. Entspricht der **%h**-Ersetzung in Bindungsskripten.

**-keycode *number***

*Number* muss eine ganze Zahl sein; er legt das Feld *keycode* für das Ereignis fest. Gültig für **Key**- und **KeyRelease**-Ereignisse. Entspricht der **%k**-Ersetzung in Bindungsskripten.

**-keysym *name***

*Name* muss der Name eines gültigen Keysym sein, etwa **g**, **space** oder **Return**; der zugehörige Keycode-Wert wird als Feld *keycode* für das Ereignis verwendet und überschreibt jedes im Basisargument *event* angegebene Detail. Gültig für **Key**- und **KeyRelease** -Ereignisse. Entspricht der **%K**-Ersetzung in Bindungsskripten.

**-mode *notify***

*Notify* legt das Feld *mode* für das Ereignis fest und muss eines von **NotifyNormal**, **NotifyGrab**, **NotifyUngrab** oder **NotifyWhileGrabbed** sein. Gültig für **Enter**-, **Leave**-,

**FocusIn**- und **FocusOut**-Ereignisse. Entspricht der **%m**-Ersetzung in Bindungsskripten.

**-override *boolean***

*Boolean* muss ein boolescher Wert sein; er legt das Feld *override_redirect* für das Ereignis fest. Gültig für **Map**-, **Reparent**- und **Configure**-Ereignisse. Entspricht der **%o**-Ersetzung in Bindungsskripten.

**-place *where***

*Where* legt das Feld *place* für das Ereignis fest; es muss **PlaceOnTop** oder **PlaceOnBottom** sein. Gültig für **Circulate**-Ereignisse. Entspricht der **%p**-Ersetzung in Bindungsskripten.

**-root *window***

*Window* muss entweder ein Fensterpfadname oder eine ganzzahlige Fensterkennung sein; er legt das Feld *root* für das Ereignis fest. Gültig für **Key**-, **KeyRelease**-, **Button**-, **ButtonRelease**-, **Enter**-, **Leave**- und **Motion**-Ereignisse. Entspricht der **%R**-Ersetzung in Bindungsskripten.

**-rootx *coord***

*Coord* muss ein Bildschirmabstand sein; er legt das Feld *x_root* für das Ereignis fest. Gültig für **Key**-, **KeyRelease**-, **Button**-, **ButtonRelease**-, **Enter**-, **Leave**- und **Motion**-Ereignisse. Entspricht der **%X**-Ersetzung in Bindungsskripten.

**-rooty *coord***

*Coord* muss ein Bildschirmabstand sein; er legt das Feld *y_root* für das Ereignis fest. Gültig für **Key**-, **KeyRelease**-, **Button**-, **ButtonRelease**-, **Enter**-, **Leave**- und **Motion**-Ereignisse. Entspricht der **%Y**-Ersetzung in Bindungsskripten.

**-sendevent *boolean***

*Boolean* muss ein boolescher Wert sein; er legt das Feld *send_event* für das Ereignis fest. Gültig für alle Ereignisse. Entspricht der **%E**-Ersetzung in Bindungsskripten.

**-serial *number***

*Number* muss eine ganze Zahl sein; er legt das Feld *serial* für das Ereignis fest. Gültig für alle Ereignisse. Entspricht der **%#**-Ersetzung in Bindungsskripten.

**-state *state***

*State* legt das Feld *state* für das Ereignis fest. Für **Key**-, **KeyRelease**-, **Button**-, **ButtonRelease**-, **Enter**-, **Leave**- und **Motion**-Ereignisse muss es ein ganzzahliger Wert sein. Für **Visibility**-Ereignisse muss es eines von **VisibilityUnobscured**, **VisibilityPartiallyObscured** oder **VisibilityFullyObscured** sein. Diese Option überschreibt Modifikatoren wie **Meta** oder **Control**, die im Basis-*event* angegeben sind. Entspricht der **%s** -Ersetzung in Bindungsskripten.

**-subwindow *window***

*Window* legt das Feld *subwindow* für das Ereignis fest, entweder als Pfadnamen eines Tk-Widgets oder als ganzzahlige Fensterkennung. Gültig für **Key**-, **KeyRelease**-, **Button**-, **ButtonRelease**-, **Enter**-, **Leave**- und **Motion**-Ereignisse. Ähnlich der **%S**-Ersetzung in Bindungsskripten.

**-time *integer***

*Integer* muss ein ganzzahliger Wert sein; er legt das Feld *time* für das Ereignis fest. Zusätzlich ist der Sonderwert **current** erlaubt; dieser wird durch die aktuelle Ereigniszeit ersetzt. Gültig für **Key**-, **KeyRelease**-, **Button**-, **ButtonRelease**-, **Enter**-, **Leave**-, **Motion**- und **Property** -Ereignisse. Entspricht der **%t**-Ersetzung in Bindungsskripten.

**-warp *boolean***

*boolean* muss ein boolescher Wert sein; er legt fest, ob der Bildschirm-Mauszeiger ebenfalls verschoben ("gewarpt") werden soll. Gültig für **Key**-, **KeyRelease**-, **Button**-, **ButtonRelease**- und **Motion**-Ereignisse. Tk unterstützt Zeigerverschiebung relativ zum Root-Fenster des Bildschirms und relativ zu einem Tk-Fenster. Im zweiten Fall wird der Zeiger nur verschoben, wenn das Fenster abgebildet ist.

**-width *size***

*Size* muss ein Bildschirmabstand sein; er legt das Feld *width* für das Ereignis fest. Gültig für **Configure**-Ereignisse. Entspricht der **%w**-Ersetzung in Bindungsskripten.

**-when *when***

*When* bestimmt, wann das Ereignis verarbeitet wird; es muss einen der folgenden Werte haben:

**now**

Das Ereignis wird sofort verarbeitet, bevor der Befehl zurückkehrt. Das gilt auch, wenn die Option **-when** weggelassen wird.

**tail**

Das Ereignis wird hinten in die Tcl-Ereigniswarteschlange eingereiht, hinter alle für diese Anwendung bereits eingereihten Ereignisse.

**head**

Das Ereignis wird an den Anfang der Tcl-Ereigniswarteschlange gesetzt, sodass es vor allen anderen bereits eingereihten Ereignissen behandelt wird.

**mark**

Das Ereignis wird an den Anfang der Tcl-Ereigniswarteschlange gesetzt, aber hinter alle anderen Ereignisse, die bereits mit **-when mark** eingereiht wurden. Diese Option ist nützlich, wenn eine Reihe von Ereignissen erzeugt werden soll, die in Reihenfolge, aber am Anfang der Warteschlange verarbeitet werden sollen.

**-x *coord***

*Coord* muss ein Bildschirmabstand sein; er legt das Feld *x* für das Ereignis fest. Gültig für **Key**-, **KeyRelease**-, **Button**-, **ButtonRelease**-, **Motion**-, **Enter**-, **Leave**-, **Expose**-, **Configure**-, **Gravity**- und **Reparent**-Ereignisse. Entspricht der **%x**-Ersetzung in Bindungsskripten. Ist *Window* leer, bezieht sich die Koordinate auf den Bildschirm, und diese Option entspricht der **%X**-Ersetzung in Bindungsskripten.

**-y *coord***

*Coord* muss ein Bildschirmabstand sein; er legt das Feld *y* für das Ereignis fest. Gültig für **Key**-, **KeyRelease**-, **Button**-, **ButtonRelease**-, **Motion**-, **Enter**-, **Leave**-, **Expose**-, **Configure**-, **Gravity**- und **Reparent**-Ereignisse. Entspricht der **%y**-Ersetzung in Bindungsskripten. Ist *Window* leer, bezieht sich die Koordinate auf den Bildschirm, und diese Option entspricht der **%Y**-Ersetzung in Bindungsskripten.

Optionen, die beim Erzeugen eines Ereignisses nicht angegeben werden, werden mit dem Wert 0 gefüllt, außer *serial*, das mit der nächsten X-Ereignis-Seriennummer gefüllt wird.

## Vordefinierte virtuelle Ereignisse

Tk definiert die folgenden virtuellen Ereignisse zu Benachrichtigungszwecken:

**<>**

Wird an ein Widget gesendet, um es zu benachrichtigen, dass der Buchstabe, den es mit der

Option **-underline** unterstrichen hat (als Beschleuniger-Anzeige), zusammen mit der Alt-Taste gedrückt wurde. Übliche Reaktion: Fokus auf das Widget (oder ein verwandtes Widget) setzen oder das Widget auslösen.

**<>**

Kann an einige Widgets (z. B. button, listbox, menu) als Alternative zu gesendet werden.

**<>**

Wird an eine Listbox gesendet, wenn die Menge der ausgewählten Einträge in der Listbox aktualisiert wird.

**<>**

Wird an ein Menü gesendet, wenn sich der aktuell ausgewählte Eintrag im Menü ändert. Gedacht für kontextsensitive Hilfesysteme.

**<>**

Wird an ein Text-Widget gesendet, wenn sich der Inhalt des Widgets ändert.

**<>**

Wird an ein Text-Widget gesendet, wenn sich die Auswahl im Widget ändert.

**<>**

Wird an alle Widgets gesendet, wenn sich das ttk-Theme ändert. Die ttk-Widgets hören auf dieses Ereignis und zeichnen sich neu, wenn es ausgelöst wird. Die Legacy-Widgets ignorieren dieses Ereignis.

**<>**

Dieses Ereignis wird an alle Widgets gesendet, wenn eine Schriftart geändert wird, etwa durch [font configure]. Das Feld *user_data* (%d) hat den Wert "FontChanged". Bei anderen systemweiten Änderungen wird dieses Ereignis an alle Widgets gesendet, und das Feld *user_data* gibt die Ursache der Änderung an. HINWEIS: alle tk- und ttk-Widgets behandeln dieses Ereignis bereits intern.

**<>**

Wird an ein Widget gesendet, wenn der Fokus das Widget betritt, weil der Benutzer per Tab zum Widget wechselt.

**<>**

Wird an ein Widget gesendet, wenn der Fokus das Widget verlässt, weil der Benutzer per Tab zum Widget wechselt.

**<>**

Wird an ein Text-Widget gesendet, wenn sein Undo- oder Redo-Stack leer oder nicht mehr leer wird.

**<>**

Wird an ein Text-Widget gesendet, wenn seine internen Daten veraltet werden, und erneut, wenn diese internen Daten wieder mit der Widget-Ansicht synchron sind. Das Detailfeld (%d-Ersetzung) ist entweder true (wenn das Widget synchron ist) oder false (wenn nicht).

Tk definiert die folgenden virtuellen Ereignisse, um Bindungen über mehrere Plattformen hinweg zu vereinheitlichen. Benutzer erwarten folgendes Verhalten:

**<>**

Den aktuell ausgewählten Widget-Inhalt löschen.

**<>**

Den aktuell ausgewählten Widget-Inhalt in die Zwischenablage kopieren.

**<>**

Den aktuell ausgewählten Widget-Inhalt in die Zwischenablage verschieben.

**<>**

Zum Zeilenende im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Zum Zeilenanfang im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Zum nächsten Element (d. h. sichtbaren Zeichen) im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Zur nächsten Zeile im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Zum nächsten Absatz im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Zur nächsten Gruppe von Elementen (d. h. sichtbares Wort) im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Den aktuell ausgewählten Widget-Inhalt durch den Inhalt der Zwischenablage ersetzen.

**<>**

Den Inhalt der Selection an der Mausposition einfügen. (Dieses Ereignis hat sinnvolle **%x**- und **%y**-Ersetzungen).

**<>**

Zum vorherigen Element (d. h. sichtbaren Zeichen) im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Zur vorherigen Zeile im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Zum vorherigen Absatz im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Zum vorherigen Fenster wechseln (Traverse).

**<>**

Zur vorherigen Gruppe von Elementen (d. h. sichtbares Wort) im aktuellen Widget wechseln und dabei jede Auswahl aufheben.

**<>**

Eine rückgängig gemachte Aktion wiederherstellen.

**<>**

Den Bereich der ausgewählten Inhalte auf das gesamte Widget setzen.

**<>**

Zum Zeilenende im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Zum Zeilenanfang im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Zum nächsten Element (d. h. sichtbaren Zeichen) im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Zur nächsten Zeile im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Zum nächsten Absatz im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Zur nächsten Gruppe von Elementen (d. h. sichtbares Wort) im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Den Bereich der ausgewählten Inhalte auf leer zurücksetzen.

**<>**

Zum vorherigen Element (d. h. sichtbaren Zeichen) im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Zur vorherigen Zeile im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Zum vorherigen Absatz im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Zur vorherigen Gruppe von Elementen (d. h. sichtbares Wort) im aktuellen Widget wechseln und dabei den Bereich der ausgewählten Inhalte erweitern.

**<>**

Die Auswahl umschalten.

**<>**

Die letzte Aktion rückgängig machen.

## Beispiele

### Tasten auf virtuelle Ereignisse abbilden

Damit eine Bindung an ein virtuelles Ereignis ausgelöst wird, müssen zwei Dinge geschehen. Zuerst muss das virtuelle Ereignis mit dem Befehl **event add** definiert werden. Zweitens muss mit dem Befehl bind eine Bindung für das virtuelle Ereignis erstellt werden. Betrachten Sie die folgenden Definitionen virtueller Ereignisse:

```
event add <<Paste>> <Control-y>
event add <<Paste>> <Button-2>
event add <<Save>> <Control-X><Control-S>
event add <<Save>> <Shift-F12>
if {[tk windowingsystem] eq "aqua"} {
    event add <<Save>> <Command-s>
}
```

Im Befehl bind kann ein virtuelles Ereignis wie jeder andere eingebaute Ereignistyp gebunden werden:

```
bind Entry <<Paste>> {%W insert [selection get]}
```

Die doppelten spitzen Klammern geben an, dass ein virtuelles Ereignis gebunden wird. Tippt der Benutzer Control-y oder drückt Taste 2, oder wird ein virtuelles Ereignis **<>** mit **event generate** synthetisiert, wird die **<>**-Bindung aufgerufen.

Hat eine virtuelle Bindung exakt dieselbe Sequenz wie eine separate physische Bindung, hat die physische Bindung Vorrang. Betrachten Sie folgendes Beispiel:

```
event add <<Paste>> <Control-y> <Meta-Control-y>
bind Entry <Control-y> {puts Control-y}
bind Entry <<Paste>> {puts Paste}
```

Tippt der Benutzer Control-y, wird die Bindung aufgerufen, weil ein physisches Ereignis als spezifischer gilt als ein virtuelles, sofern sonst gleich. Tippt der Benutzer Meta-Control-y, wird die **<>**-Bindung aufgerufen, weil der **Meta**-Modifikator im physischen Muster der virtuellen Bindung spezifischer ist als die Sequenz **<Control-y**> für das physische Ereignis.

Bindungen an ein virtuelles Ereignis können erstellt werden, bevor das virtuelle Ereignis existiert. Das virtuelle Ereignis muss sogar nie definiert werden, etwa auf Plattformen, auf denen das jeweilige virtuelle Ereignis sinnlos oder nicht erzeugbar wäre.

Ändert sich die Definition eines virtuellen Ereignisses zur Laufzeit, reagieren alle Fenster sofort auf die neue Definition. Ausgehend vom vorherigen Beispiel, wenn folgender Code ausgeführt wird:

```
bind Entry <Control-y> {}
event add <<Paste>> <F6>
```

ändert sich das Verhalten in zweierlei Hinsicht. Zuerst tritt die verdeckte **<>**-Bindung hervor. Control-y ruft nicht mehr die Bindung auf, sondern das virtuelle Ereignis **<>**. Zweitens ruft das Drücken der F6-Taste nun ebenfalls die **<>**-Bindung auf.

### Mauszeiger bewegen

Manchmal ist es nützlich, den Mauszeiger wirklich zu bewegen. Beispielsweise bei Software, die dem Benutzer direkt zeigt, wie das Programm bedient wird. Dazu muss die Maus mit **event generate** "gewarpt" werden, etwa so:

```
for {set xy 0} {$xy < 200} {incr xy} {
    event generate . <Motion> -x $xy -y $xy -warp 1
    update
    after 50
}
```

Beachten Sie, dass es üblicherweise als schlechter Stil gilt, den Mauszeiger für den Benutzer zu bewegen, weil ihm die Kontrolle entzogen wird. Die Technik sollte daher mit Vorsicht eingesetzt werden. Außerdem ist nicht garantiert, dass sie auf allen Plattformen funktioniert.

*Copyright (c) 1996 Sun Microsystems, Inc.*
*Copyright (c) 1998-2000 Ajuba Solutions.*

# focus

*Stichwörter:* focus, events, keyboard, top-level, window manager

## Name

focus - Verwaltung des Eingabefokus

## Übersicht

**focus**
**focus** *window*
**focus** *option* ?*arg* ...?

## Beschreibung

Der Befehl **focus** verwaltet den Tk-Eingabefokus. Zu jedem Zeitpunkt ist auf jedem Display genau ein Fenster als *Fokusfenster* festgelegt; alle Tastendruck- und Tastenloslass-Ereignisse für dieses Display werden an dieses Fenster gesendet. Normalerweise leitet der Fenstermanager den Fokus zwischen den Toplevel-Fenstern eines Displays um. Einige Fenstermanager setzen den Eingabefokus beispielsweise automatisch auf ein Toplevel-Fenster, sobald die Maus hineinbewegt wird; andere geben den Fokus erst um, wenn der Benutzer auf ein Fenster klickt. Üblicherweise setzt der Fenstermanager den Fokus nur auf Toplevel-Fenster; die Anwendung leitet den Fokus unter den Kindern des Toplevels selbst um.

Tk merkt sich für jedes Toplevel ein Fokusfenster (den zuletzt fokussierten Nachkommen dieses Toplevels). Gibt der Fenstermanager einem Toplevel den Fokus, leitet Tk ihn automatisch auf das gemerkte Fenster um. Innerhalb eines Toplevels verwendet Tk standardmäßig ein *explizites* Fokusmodell: Das Bewegen der Maus innerhalb eines Toplevels ändert den Fokus normalerweise nicht; er wechselt nur, wenn ein Widget den Fokus ausdrücklich anfordert (z. B. durch einen Mausklick auf einen Button) oder wenn der Benutzer eine Taste wie Tab drückt, die den Fokus verschiebt.

Die Tcl-Prozedur tk_focusFollowsMouse kann aufgerufen werden, um ein *implizites* Fokusmodell einzurichten: Tk wird so konfiguriert, dass ein Fenster den Fokus erhält, sobald die Maus hineinbewegt wird. Die Tcl-Prozeduren tk_focusNext und tk_focusPrev implementieren eine Fokusreihenfolge unter den Fenstern eines Toplevels; sie werden unter anderem in den Standard-Bindings für Tab und Shift-Tab verwendet.

Der Befehl **focus** kann eine der folgenden Formen annehmen:

**focus**

Gibt den Pfadnamen des Fokusfensters auf dem Display zurück, das das Hauptfenster der Anwendung enthält, oder eine leere Zeichenkette, wenn kein Fenster dieser Anwendung auf diesem Display den Fokus hat. Hinweis: Es ist besser, das Display ausdrücklich mit **-displayof** anzugeben (siehe unten), damit der Code in Anwendungen mit mehreren Displays funktioniert.

**focus *window***

Hat die Anwendung derzeit den Eingabefokus auf dem Display von *window*, setzt dieser Befehl den Eingabefokus für das Display von *window* auf *window* und gibt eine leere Zeichenkette zurück. Hat die Anwendung den Eingabefokus auf dem Display von *window* derzeit nicht, wird *window* als Fokusfenster für sein Toplevel gemerkt; beim nächsten Erhalt des Fokus durch das Toplevel leitet Tk ihn auf *window* um. Ist *window* eine leere Zeichenkette, bewirkt der Befehl nichts.

**focus -displayof *window***

Gibt den Namen des Fokusfensters auf dem Display zurück, das *window* enthält. Liegt das Fokusfenster für das Display von *window* nicht in dieser Anwendung, ist der Rückgabewert eine leere Zeichenkette.

**focus -force *window***

Setzt den Fokus des Displays von *window* auf *window*, auch wenn die Anwendung derzeit nicht den Eingabefokus für dieses Display hat. Dieser Befehl sollte sparsam oder gar nicht verwendet werden. In normaler Nutzung sollte eine Anwendung den Fokus nicht selbst beanspruchen, sondern darauf warten, dass der Fenstermanager ihn vergibt. Ist *window* eine leere Zeichenkette, bewirkt der Befehl nichts.

**focus -lastfor *window***

Gibt den Namen des zuletzt den Eingabefokus gehabten Fensters unter allen Fenstern im selben Toplevel wie *window* zurück. Hat noch kein Fenster in diesem Toplevel den Eingabefokus gehabt oder wurde das zuletzt fokussierte Fenster gelöscht, wird der Name des Toplevels zurückgegeben. Der Rückgabewert ist das Fenster, das den Eingabefokus erhält, sobald der Fenstermanager dem Toplevel das nächste Mal den Fokus gibt.

## Besonderheiten

Erhält ein internes Fenster den Eingabefokus, setzt Tk den X-Fokus nicht tatsächlich auf dieses Fenster; aus Sicht von X bleibt der Fokus auf dem Toplevel-Fenster, das das fokussierte Fenster enthält. Tk erzeugt jedoch FocusIn- und FocusOut-Ereignisse, als ob der X-Fokus auf dem internen Fenster läge. Dieser Ansatz umgeht eine Reihe von Problemen, die entstünden, wenn der X-Fokus wirklich verschoben würde; dass der X-Fokus auf dem Toplevel liegt, ist unsichtbar, sofern der X-Server nicht direkt per C-Code abgefragt wird.

## Beispiele

Um ein Fenster nur dann in den Fokus-Traversierungsring einzubeziehen, wenn eine Variable gesetzt ist, fügen Sie die folgenden Bindings den Widgets *vor* und *nach* diesem Fenster im Ring hinzu:

```
button .before -text "Before"
button .middle -text "Middle"
button .after  -text "After"
checkbutton .flag -variable traverseToMiddle -takefocus 0
pack .flag -side left
pack .before .middle .after
bind .before <Tab> {
   if {!$traverseToMiddle} {
      focus .after
      break
   }
}
bind .after <Shift-Tab> {
   if {!$traverseToMiddle} {
      focus .before
      break
   }
}
focus .before
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# focusNext

*Stichwörter:* focusNext, focus, keyboard traversal, top-level

## Name

tk_focusNext, tk_focusPrev, tk_focusFollowsMouse - Hilfsprozeduren zur Verwaltung des Eingabefokus

## Übersicht

**tk_focusNext** *window*
**tk_focusPrev** *window*
**tk_focusFollowsMouse**

## Beschreibung

**tk_focusNext** ist eine Hilfsprozedur für die Tastatur-Navigation. Sie liefert das "nächste" Fenster nach *window* in der Fokusreihenfolge. Die Fokusreihenfolge ergibt sich aus der Stapelreihenfolge der Fenster und der Fensterhierarchie. Unter Geschwistern entspricht die Fokusreihenfolge der Stapelreihenfolge; das unterste Fenster ist zuerst. Hat ein Fenster Kinder, wird zuerst das Fenster selbst besucht, dann rekursiv die Kinder, danach das nächste Geschwister. Andere **toplevel** -Fenster als *window* werden übersprungen — **tk_focusNext** liefert nie ein Fenster in einem anderen Toplevel als *window*.

Nach dem nächsten Fenster prüft **tk_focusNext** die Option **-takefocus**, ob es übersprungen werden soll. Ist das der Fall, geht **tk_focusNext** zum nächsten Fenster in der Fokusreihenfolge, bis ein Fenster gefunden wird, das den Fokus annimmt, oder wieder *window* erreicht ist.

**tk_focusPrev** entspricht **tk_focusNext**, liefert aber das Fenster unmittelbar vor *window* in der Fokusreihenfolge.

**tk_focusFollowsMouse** stellt die Anwendung auf ein implizites Fokusmodell um: Das Fenster unter dem Mauszeiger erhält den Fokus. Nach dem Aufruf gibt Tk dem Fenster automatisch den Eingabefokus, sobald die Maus hineinbewegt wird. Mit dem Befehl focus kann der Fokus auf ein anderes Fenster als das unter der Maus gesetzt werden; bewegt sich die Maus in ein neues Fenster, springt der Fokus dorthin.

Derzeit gibt es keine eingebaute Unterstützung, zur expliziten Fokussteuerung zurückzukehren; dazu muss ein Skript die von **tk_focusFollowsMouse** angelegten Bindings entfernen.

*Copyright (c) 1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# font

*Stichwörter:* font

## Name

font - Schriftarten erzeugen und inspizieren

## Übersicht

**font** *option* ?*arg* ...?

## Beschreibung

Der Befehl **font** bietet mehrere Einrichtungen zum Umgang mit Schriftarten, etwa zum Definieren benannter Schriftarten und zum Inspizieren der tatsächlichen Attribute einer Schriftart. Der Befehl hat mehrere Formen, bestimmt durch das erste Argument. Derzeit werden folgende Formen unterstützt:

**font actual *font* ?-displayof *window*? ?*option*? ?--? ?*char*?**

Gibt Informationen über die tatsächlichen Attribute zurück, die erhalten werden, wenn *font* auf dem Display von *window* verwendet wird; die erhaltenen Attribute können sich von den angeforderten unterscheiden, z. B. wegen plattformabhängiger Einschränkungen wie Verfügbarkeit von Schriftfamilien und Punktgrößen. *font* ist eine Schriftbeschreibung; siehe Schriftbeschreibungen unten. Wenn das Argument *window* weggelassen wird, gilt standardmäßig das Hauptfenster. Wenn *option* angegeben ist, wird der Wert dieses Attributs zurückgegeben; andernfalls ist der Rückgabewert eine Liste aller Attribute und ihrer Werte. Siehe Schriftoptionen unten für die möglichen Attribute. Wenn das Argument *char* angegeben ist, muss es ein einzelnes Zeichen sein. Die zurückgegebenen Schriftattribute sind die der konkreten Schrift, mit der dieses Zeichen dargestellt wird; sie unterscheiden sich von der Basisschrift, wenn die Basisschrift das Zeichen nicht enthält. Kann *char* ein Bindestrich sein, sollte es mit **-|-** vorangestellt werden, um es von einer falsch geschriebenen *option* zu unterscheiden.

**font configure *fontname* ?*option*? ?*value option value* ...?**

Fragt die gewünschten Attribute der benannten Schrift *fontname* ab oder ändert sie. Wenn keine *option* angegeben ist, gibt der Befehl eine Liste aller Optionen und ihrer Werte für *fontname* zurück. Wenn eine einzelne *option* ohne *value* angegeben ist, wird der aktuelle Wert dieses Attributs zurückgegeben. Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die benannte Schrift entsprechend; in diesem Fall zeichnen sich alle Widgets, die diese Schrift verwenden, mit den neuen Attributen neu. Siehe Schriftoptionen unten für die möglichen Attribute.

Beachten Sie, dass unter Aqua/macOS die Systemschriften (siehe Plattformspezifische Schriften unten) möglicherweise nicht wirklich geändert werden können, weil sie vom System-Theme implementiert sind. Um den Effekt einer Änderung zu erzielen, verwenden Sie **font actual**, um die Konfiguration zu lesen, und **font create**, um eine kopierbare, änderbare Schrift zu erzeugen.

**font create ?*fontname*? ?*option value* ...?**

Erzeugt eine neue benannte Schrift und gibt ihren Namen zurück. *fontname* legt den Namen fest; wenn er weggelassen wird, erzeugt Tk einen neuen Namen der Form **font***x*, wobei *x* eine Ganzzahl ist. Es kann beliebig viele *option*-*value*-Paare geben, die die gewünschten Attribute der neuen benannten Schrift festlegen. Siehe Schriftoptionen unten für die möglichen Attribute.

**font delete *fontname* ?*fontname* ...?**

Löscht die angegebenen benannten Schriften. Wenn Widgets die benannte Schrift noch verwenden, wird sie erst gelöscht, wenn alle Instanzen freigegeben sind. Diese Widgets zeigen weiterhin die zuletzt bekannten Werte der benannten Schrift. Wird eine gelöschte benannte Schrift später mit **font create** neu erzeugt, verwenden die Widgets die neue benannte Schrift und zeichnen sich mit deren neuen Attributen neu.

**font families ?*-displayof window*?**

Der Rückgabewert ist eine Liste der namensunabhängigen Namen aller Schriftfamilien auf dem Display von *window*. Wenn *window* weggelassen wird, gilt standardmäßig das Hauptfenster.

**font measure *font* ?*-displayof window*? *text***

Misst, wie viel Platz die Zeichenkette *text* in der angegebenen Schrift *font* auf *window* einnehmen würde. *font* ist eine Schriftbeschreibung; siehe Schriftbeschreibungen unten. Wenn *window* weggelassen wird, gilt standardmäßig das Hauptfenster. Der Rückgabewert ist die Gesamtbreite von *text* in Pixeln, ohne zusätzliche Pixel für stark überzeichnete Zeichen wie ein geschwungenes "f". Enthält die Zeichenkette Zeilenumbrüche oder Tabulatoren, werden diese beim Messen nicht expandiert oder besonders behandelt.

**font metrics *font* ?*-displayof window*? ?*option*?**

Gibt Informationen über die Metriken (schriftspezifische Daten) von *font* auf dem Display von *window* zurück. *font* ist eine Schriftbeschreibung; siehe Schriftbeschreibungen unten. Wenn *window* weggelassen wird, gilt standardmäßig das Hauptfenster. Wenn *option* angegeben ist, wird der Wert dieser Metrik zurückgegeben; andernfalls ist der Rückgabewert eine Liste aller Metriken und ihrer Werte. Siehe Schriftmetriken unten für die möglichen Metriken.

**font names**

Der Rückgabewert ist eine Liste aller derzeit definierten benannten Schriften.

## Schriftbeschreibungen

Die folgenden Formate werden als Schriftbeschreibung überall akzeptiert, wo oben *font* als Argument angegeben ist; dieselben Formen sind auch bei der Option **-font** von Widgets zulässig.

**[1] *fontname***

Der Name einer benannten Schrift, erzeugt mit **font create**. Verwendet ein Widget eine benannte Schrift, ist garantiert, dass dies nie einen Fehler verursacht, solange die benannte Schrift existiert — unabhängig davon, wie ungültig oder sinnlos die Attribute der benannten Schrift sind. Kann die benannte Schrift nicht exakt mit den angegebenen Attributen dargestellt werden, wird automatisch eine ähnliche Schrift substituiert.

**[2] *systemfont***

Der plattformspezifische Name einer Schrift, vom Grafikserver interpretiert. Unter X gehört dazu auch ein XLFD (siehe [4]), bei dem ein einzelnes Zeichen "*" mehr als ein Feld in der Mitte des Namens ausgelassen hat. Siehe Plattformspezifische Schriften für die Systemschriften.

**[3] *family* ?*size*? ?*style*? ?*style ...*?**

Eine wohlgeformte Liste, deren erstes Element die gewünschte Schrift-*family* ist und deren optionales zweites Element die gewünschte *size* ist. Die Auslegung von *size* folgt denselben Regeln wie **-size** unter Schriftoptionen unten. Weitere optionale Argumente nach *size* sind Schrift-*style*s. Mögliche Werte für *style* sind:

**normal bold roman italic**
**underline overstrike**

**[4] X-Schriftnamen (XLFD)**

Ein Unix-orientierter Schriftname der Form *-foundry-family-weight-slant-setwidth-addstyle-pixel-point-resx-resy-spacing-width-charset-encoding* . Das Zeichen "*" kann verwendet werden, um einzelne Felder zu überspringen. Pro übersprungenem Feld muss genau ein "*" stehen, außer ein "*" am Ende des XLFD überspringt alle restlichen Felder; der kürzeste gültige XLFD ist einfach "*" (alle Felder Standard). Übersprungene Felder erhalten Standardwerte. Zur Kompatibilität wählt ein XLFD immer eine Schrift der angegebenen Pixelgröße (nicht Punktgröße); obwohl das nicht streng korrekt ist, gingen bestehende Anwendungen davon aus, dass ein "Punkt" ein Pixel war, und würden bei korrekter Größe falsch (meist größer) dargestellt.

**[5]** ***option value ?option value ...?***

Eine wohlgeformte Liste von *option-value*-Paaren, die die gewünschten Schriftattribute im gleichen Format wie bei einer benannten Schrift festlegen; siehe Schriftoptionen unten.

Wird die Schriftbeschreibung *font* verwendet, versucht das System, die Beschreibung nacheinander nach den fünf Regeln zu parsen. Fälle [1] und [2] müssen einem existierenden benannten oder System-Schriftnamen entsprechen. Fälle [3], [4] und [5] werden auf allen Plattformen akzeptiert; die nächstliegende verfügbare Schrift wird verwendet. Manchmal ist keine nahe Schrift möglich (z. B. ungültige Familie); dann wird eine systemabhängige Standardschrift gewählt. Entspricht die Beschreibung keinem Muster, entsteht ein Fehler.

## Schriftmetriken

Die folgenden Optionen werden von **font metrics** abgefragt; sie sind schriftspezifische Daten beim Erzeugen der Schrift. Sie gelten für die ganze Schrift, nicht für einzelne Zeichen. In den Definitionen ist die "Grundlinie" die horizontale Linie, an der die Unterkante der meisten Buchstaben ausgerichtet ist; manche Buchstaben wie "g" ragen darunter hinaus.

**-ascent**

Anzahl Pixel, um die der höchste Buchstabe über der Grundlinie ragt, plus eventueller Leerraum des Schriftdesigners.

**-descent**

Größter Betrag in Pixeln, um den ein Buchstabe unter die Grundlinie ragt, plus eventueller Leerraum des Schriftdesigners.

**-linespace**

Gibt zurück, wie weit zwei Textzeilen in derselben Schrift vertikal in Pixeln platziert werden sollen, damit sich keine Zeichen überlappen. Im Allgemeinen Summe aus Aufstieg über der Grundlinie und Abstieg darunter.

**-fixed**

Gibt ein boolesches Flag zurück: "**1**", wenn dies eine Festbreitenschrift ist (jedes normale Zeichen gleich breit), oder "**0**" bei proportionaler Schrift. Steuerzeichen, Tabulatoren und andere nicht druckbare Zeichen zählen nicht mit.

## Schriftoptionen

Die folgenden Optionen werden auf allen Plattformen unterstützt und beim Erzeugen einer benannten Schrift oder bei Schriftbeschreibung nach Stil [5] verwendet:

**-family *name***

Name der Schriftfamilie (Groß-/Kleinschreibung irrelevant). Tk garantiert **Courier** (Monospace-Schreibmaschine), **Times** (Serif-Zeitung) und **Helvetica** (Sans-Serif). Bei Verwendung einer dieser Familien wird automatisch die passendste native Familie substituiert. *name* kann auch eine native, plattformspezifische Familie sein; dann funktioniert sie auf einer Plattform, auf anderen evtl. nicht. Ist die Familie nicht angegeben oder unbekannt, wird eine plattformspezifische Standardschrift gewählt.

**-size *size***

Gewünschte Schriftgröße. Ist *size* positiv, wird sie in Punkten interpretiert. Ist *size* negativ, ist der Absolutwert die Größe in Pixeln. Kann die Schrift in der Größe nicht dargestellt werden, wird eine nahe Größe gewählt. Ist *size* nicht angegeben oder 0, wird eine plattformabhängige Standardgröße gewählt.

Größen sollten normalerweise in Punkten angegeben werden, damit die Anwendung auf dem Bildschirm gleich groß bleibt, auch bei geänderten Auflösungen oder beim Portieren. Pixel sind sinnvoll, wenn Text zu einem Bitmap mit fester Größe passen muss. Die Zuordnung Punkte/Pixel wird beim Start anhand des Monitors festgelegt, kann aber mit tk scaling überschrieben werden.

**-weight *weight***

Nominale Strichstärke. **normal** ist normal, **bold** fett. Die nächstliegende verfügbare Stärke wird gewählt. Standard **normal**.

**-slant *slant***

Neigung der Zeichen gegen die Vertikale. Gültig: **roman** und **italic**. **roman** ist aufrecht, **italic** geneigt. Die nächstliegende Neigung wird gewählt. Standard **roman**.

**-underline *boolean***

Boolesches Flag: Zeichen unterstrichen. Standard **false**.

**-overstrike *boolean***

Boolesches Flag: horizontale Linie durch die Mitte der Zeichen. Standard **false**.

## Standardschriften

Die folgenden benannten Schriften werden auf allen Systemen unterstützt und entsprechen passenden System-Standards.

**TkDefaultFont**

Standardschrift für alle GUI-Elemente ohne eigene Angabe.

**TkTextFont**

Für Benutzertext in entry, listbox usw.

**TkFixedFont**

Standard-Festbreitenschrift.

**TkMenuFont**

Für Menüeinträge.

**TkHeadingFont**

Für Spaltenüberschriften in Listen und Tabellen.

**TkCaptionFont**

Für Fenster- und Dialog-Titelleisten.

**TkSmallCaptionFont**
Für Beschriftungen in eingebetteten Fenstern oder Werkzeugdialogen.

**TkIconFont**
Für Symbolbeschriftungen.

**TkTooltipFont**
Für Tooltip-Fenster (kurzlebige Infofenster).

Diese Schriften sollten *nicht* geändert werden, da Tk sie bei Systemänderungen selbst anpassen kann. Stattdessen eine Kopie erzeugen und diese ändern.

## Plattformspezifische Schriften

Folgende Systemschriften werden unterstützt:

**X Windows**
Alle gültigen X-Schriftnamen, einschließlich der von xlsfonts(1) gelisteten.

**MS Windows**
Folgende Schriften, zugeordnet zu den Stil-Standards des Benutzers:

**system ansi device**
**systemfixed ansifixed oemfixed**

**macOS**
Folgende Schriften, zugeordnet zu den Stil-Standards des Benutzers:

**system application menu**

Zusätzlich liefern folgende benannte Schriften Zugriff auf Aqua-Theme-Schriften:

**systemSystemFont systemEmphasizedSystemFont**
**systemSmallSystemFont systemSmallEmphasizedSystemFont**
**systemApplicationFont systemLabelFont**
**systemViewsFont systemMenuTitleFont**
**systemMenuItemFont systemMenuItemMarkFont**
**systemMenuItemCmdKeyFont systemWindowTitleFont**
**systemPushButtonFont systemUtilityWindowTitleFont**
**systemAlertHeaderFont systemToolbarFont**
**systemMiniSystemFont systemDetailSystemFont**
**systemDetailEmphasizedSystemFont**

## Beispiel

Ein Text-Widget mit Schriftmustern für jede installierte Familie füllen:

```
pack [text .t -wrap none] -fill both -expand 1
set count 0
set tabwidth 0
foreach family [lsort -dictionary [font families]] {
    .t tag configure f[incr count] -font [list $family 10]
    .t insert end ${family}:\t {} \
            "This is a simple sampler\n" f$count
    set w [font measure [.t cget -font] ${family}:]
    if {$w + 5 > $tabwidth} {
        set tabwidth [expr {$w + 5}]
        .t configure -tabs $tabwidth
    }
}
```

*Copyright (c) 1996 Sun Microsystems, Inc.*
*Copyright (c) 2006-2007 Daniel A. Steffen* das@users.sourceforge.net

# fontchooser

*Stichwörter:* fontchooser, dialog, font, font selection, font chooser, font panel

## Name

fontchooser - Schriftauswahl-Dialog steuern

## Übersicht

**tk fontchooser configure** ?-*option value* ...?
**tk fontchooser show**
**tk fontchooser hide**

## Beschreibung

Der Befehl tk fontchooser steuert den Tk-Schriftauswahl-Dialog. Wo verfügbar, wird der native plattformspezifische Schriftauswahl-Dialog verwendet, andernfalls ein in Tcl implementierter Dialog.

Im Gegensatz zu den meisten anderen Tk-Dialogbefehlen liefert tk fontchooser kein unmittelbares Ergebnis zurück, da auf manchen Plattformen (macOS) der Standardschriftdialog modellos ist, auf anderen (Windows) modal. Um diesen Unterschied auszugleichen, wird die gesamte Benutzerinteraktion mit dem Dialog dem Aufrufer über Rückruffunktionen oder virtuelle Ereignisse mitgeteilt.

Der Befehl tk fontchooser kann eine der folgenden Formen haben:

**tk fontchooser configure ?-*option value* ...?**

Legt eine oder mehrere der unten stehenden Konfigurationsoptionen fest oder fragt sie ab (analog zur Widget-Konfiguration in Tk).

**tk fontchooser show**

Zeigt den Schriftauswahl-Dialog an. Je nach Plattform kann der Befehl sofort zurückkehren oder erst, wenn der Dialog geschlossen wurde.

**tk fontchooser hide**

Blendet den Schriftauswahl-Dialog aus, falls er sichtbar ist, und bewirkt, dass ein ausstehender tk fontchooser **show**-Befehl zurückkehrt.

## Konfigurationsoptionen

**-parent**

Legt fest bzw. gibt das logische Elternfenster des Schriftauswahl-Dialogs zurück (ähnlich der Option **-parent** anderer Dialoge). Der Schriftauswahl-Dialog wird ausgeblendet, wenn er sichtbar ist und das Elternfenster zerstört wird.

**-title**

Legt fest bzw. gibt den Dialogtitel zurück. Hat auf Plattformen keine Wirkung, deren Schriftauswahl-Dialog keine Titel unterstützt.

**-font**

Legt fest bzw. gibt die im Dialog aktuell ausgewählte Schriftart zurück, wenn er sichtbar ist, oder die beim Anzeigen des Dialogs zunächst ausgewählte Schriftart (falls von der Plattform unterstützt). Kann auf den leeren String gesetzt werden, um anzuzeigen, dass keine Schriftart ausgewählt sein soll. Schriftarten können in jeder im Abschnitt "SCHRIFTART-BESCHREIBUNG" der Manpage font beschriebenen Form angegeben werden.

**-command**

Legt fest bzw. gibt das Befehlspräfix zurück, das aufgerufen wird, wenn der Benutzer eine Schriftart gewählt hat. Das Präfix wird auf globaler Ebene ausgewertet, nachdem die Spezifikation der gewählten Schriftart angehängt wurde. Auf Plattformen, deren Schriftauswahl-Dialog weitere Schriftattribute (z. B. Farbe) anbietet, können vor der Auswertung zusätzliche Schlüssel/Wert-Paare angehängt werden. Kann auf den leeren String gesetzt werden, um anzuzeigen, dass keine Rückruffunktion aufgerufen werden soll. Schriftarten werden als Liste der Form aus Abschnitt [3] "SCHRIFTART-BESCHREIBUNG" in der Manpage font angegeben (d. h. eine Liste der Form *{family size style ?style ...?}*).

**-visible**

Schreibgeschützte Option; gibt einen booleschen Wert zurück, ob der Schriftauswahl-Dialog derzeit sichtbar ist. Ein Versuch, diese Option zu setzen, führt zu einem Fehler.

## Virtuelle Ereignisse

**<>**

Wird an das Dialog-Elternfenster gesendet, wenn sich die Sichtbarkeit des Schriftauswahl-Dialogs ändert — sowohl durch Benutzeraktion (z. B. Schließen über OK/Abbrechen oder Schließen-Kästchen) als auch durch tk fontchooser **show**/**hide**. Bindungs-Skripte können die aktuelle Sichtbarkeit über die Konfigurationsoption **-visible** ermitteln.

**<>**

Wird an das Dialog-Elternfenster gesendet, wenn der Schriftauswahl-Dialog sichtbar ist und sich die gewählte Schriftart ändert — sowohl durch Benutzeraktion als auch durch Setzen der Option **-font**. Bindungs-Skripte können die aktuell gewählte Schriftart über die Konfigurationsoption **-font** ermitteln.

## Hinweise

Aufrufer sollten von tk fontchooser **show** kein Ergebnis erwarten und dürfen nicht annehmen, dass der Dialog beim Zurückkehren des Befehls bereits geschlossen wurde. Die gesamte Benutzerinteraktion mit dem Dialog erfolgt über die Rückruffunktion **-command** und die virtuellen Ereignisse **<>**. Welche Benutzeraktionen genau die Rückruffunktion bzw. die virtuellen Ereignisse auslösen, ist implementierungsabhängig. Ist im Dialog eine Schaltfläche "Übernehmen" oder "OK" vorhanden, löst diese die Rückruffunktion **-command** und das virtuelle Ereignis **<>** aus. In manchen Implementierungen können auch andere Benutzeraktionen dies bewirken; unter macOS spiegelt der Standardschriftauswahl-Dialog alle Benutzerentscheidungen unmittelbar an den Aufrufer wider.

Wenn mehrere Widgets vom Schriftauswahl-Dialog beeinflusst werden sollen, muss der

Fokuswechsel korrekt behandelt werden: Die im Dialog gewählte Schriftart soll stets der aktuellen Schriftart des Widgets mit Fokus entsprechen, und die Rückruffunktion **-command** soll nur auf das Widget mit Fokus wirken. Empfohlen wird, die Konfigurationsoptionen **-font** und **-command** des Schriftdialogs in -Handlern pro Widget zu setzen (und bei Bedarf in entsprechenden -Handlern zurückzusetzen — d. h. auf den leeren String). Das ist besonders wichtig für Bibliotheksautoren, die den Schriftauswahl-Dialog nutzen, um Konflikte mit Anwendungscode zu vermeiden, der denselben Dialog ebenfalls verwenden möchte.

Da der Schriftauswahl-Dialog anwendungsweit global ist, wird bei mehreren Interpretern, die tk fontchooser aufrufen, nur die **-command**-Rückruffunktion des Interpreters aufgerufen, der zuletzt tk fontchooser **configure** oder tk fontchooser **show** aufgerufen hat; nur das von diesem Interpreter gesetzte **-parent** erhält die virtuellen Ereignisse **<>**.

Die Schriftdialog-Implementierung speichert und liefert unter Umständen nur font **actual**-Daten als Wert der Konfigurationsoption **-font**. Das kann problematisch sein, wenn **-font** auf eine benannte Schriftart gesetzt wird und diese später geändert wird: Die Option **-font** des Schriftdialogs muss erneut gesetzt werden, damit die gewählte Schriftart dem neuen Wert der benannten Schriftart entspricht.

## Beispiel

```
proc fontchooserDemo {} {
    wm title . "Font Chooser Demo"
    tk fontchooser configure -parent .
    button .b -command fontchooserToggle -takefocus 0
    fontchooserVisibility .b
    bind . <<TkFontchooserVisibility>> \
            [list fontchooserVisibility .b]
    foreach w {.t1 .t2} {
        text $w -width 20 -height 4 -borderwidth 1 -relief solid
        bind $w <FocusIn> [list fontchooserFocus $w]
        $w insert end "Text Widget $w"
    }
    .t1 configure -font {Courier 14}
    .t2 configure -font {Times 16}
    pack .b .t1 .t2; focus .t1
}
proc fontchooserToggle {} {
    tk fontchooser [expr {
            [tk fontchooser configure -visible] ?
            "hide" : "show"}]
}
proc fontchooserVisibility {w} {
    $w configure -text [expr {
            [tk fontchooser configure -visible] ?
            "Hide Font Dialog" : "Show Font Dialog"}]
}
proc fontchooserFocus {w} {
    tk fontchooser configure -font [$w cget -font] \
            -command [list fontchooserFontSelection $w]
}
proc fontchooserFontSelection {w font args} {
    $w configure -font [font actual $font]
}
fontchooserDemo
```

*Copyright (c) 2008 Daniel A. Steffen* das@users.sourceforge.net

# frame

*Stichwörter:* frame, widget

## Name

frame - Erstellt und manipuliert 'Frame'-Container-Widgets

## Übersicht

**frame** *pathName* ?*options*?

## Standardoptionen

**-borderwidth**, **-highlightcolor**, **-pady**, **-cursor**, **-highlightthickness**, **-relief**, **-highlightbackground**, **-padx**, **-takefocus**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-background (background/Background)**

Diese Option ist identisch mit der Standard-Option **-background**, außer dass ihr Wert auch als leerer String angegeben werden kann. In diesem Fall zeigt das Widget keinen Hintergrund oder Rahmen an, und es werden keine Farben aus seiner Farbkarte für Hintergrund und Rahmen verbraucht. Ein leerer Hintergrund deaktiviert das Zeichnen des Hintergrundbildes.

**-backgroundimage (backgroundImage/BackgroundImage)**

Dies gibt ein Bild an, das im Hintergrund des Frames innerhalb des Rahmens angezeigt werden soll (d.h. das Bild wird durch den Hervorhebungsring und den Rahmen des Frames abgeschnitten, falls vorhanden); Unter-Widgets des Frames werden darüber gezeichnet. Das Bild muss mit dem Befehl image create erstellt worden sein. Wenn als leerer String angegeben, wird kein Bild angezeigt.

**-class (class/Class)**

Legt eine Klasse für das Fenster fest. Diese Klasse wird verwendet, wenn die Optionsdatenbank für die anderen Optionen des Fensters abgefragt wird, und sie wird später auch für andere Zwecke wie Bindings verwendet. Die Option **-class** kann nicht mit dem Widget-Befehl **configure** geändert werden.

**-colormap (colormap/Colormap)**

Legt eine Farbkarte fest, die für das Fenster verwendet werden soll. Der Wert kann entweder **new** sein, in welchem Fall eine neue Farbkarte für das Fenster und seine Kinder erstellt wird, oder der Name eines anderen Fensters (das auf dem gleichen Bildschirm und mit der gleichen visuellen Darstellung wie *pathName* sein muss), in welchem Fall das neue Fenster die

Farbkarte vom angegebenen Fenster verwendet. Wenn die Option **-colormap** nicht angegeben ist, verwendet das neue Fenster die gleiche Farbkarte wie sein Eltern-Element. Diese Option kann nicht mit dem Widget-Befehl **configure** geändert werden.

**-container (container/Container)**

Der Wert muss ein Boolean sein. Wenn wahr, bedeutet dies, dass dieses Fenster als Container verwendet wird, in dem eine andere Anwendung eingebettet wird (zum Beispiel kann ein Tk-Toplevel mit der Option **-use** eingebettet werden). Das Fenster unterstützt die entsprechenden Window-Manager-Protokolle für Dinge wie Geometrieanforderungen. Das Fenster sollte in dieser Anwendung keine eigenen Kinder haben. Diese Option kann nicht mit dem Widget-Befehl **configure** geändert werden. Beachten Sie, dass **-borderwidth**, **-padx** und **-pady** ignoriert werden, wenn als Container konfiguriert, da ein Container keinen Rahmen hat.

**-height (height/Height)**

Legt die gewünschte Höhe für das Fenster in einer der von Tk_GetPixels akzeptierten Formen fest. Wenn diese Option negativ oder null ist, fordert das Fenster überhaupt keine Größe an. Beachten Sie, dass dies die Gesamthöhe des Frames festlegt, **-borderwidth** oder ähnliches wird nicht hinzugefügt. Normalerweise sollte **-height** nicht verwendet werden, wenn ein propagierender Geometrie-Manager wie grid oder pack innerhalb des Frames verwendet wird, da der Geometrie-Manager die Höhe des Frames überschreibt.

**-tile (tile/Tile)**

Dies legt fest, wie das Hintergrundbild (siehe -backgroundimage) auf dem Frame gezeichnet wird. Wenn wahr (gemäß Tcl_GetBoolean), wird das Bild gekachelt, um den gesamten Frame zu füllen, wobei der Ursprung der ersten Kopie des Bildes die obere linke Ecke des Innenraums des Frames ist. Wenn falsch (Standard), wird das Bild innerhalb des Frames zentriert.

**-visual (visual/Visual)**

Legt visuelle Informationen für das neue Fenster in einer der von Tk_GetVisual akzeptierten Formen fest. Wenn diese Option nicht angegeben ist, verwendet das neue Fenster die gleiche visuelle Darstellung wie sein Eltern-Element. Die Option **-visual** kann nicht mit dem Widget-Befehl **configure** geändert werden.

**-width (width/Width)**

Legt die gewünschte Breite für das Fenster in einer der von Tk_GetPixels akzeptierten Formen fest. Wenn diese Option negativ oder null ist, fordert das Fenster überhaupt keine Größe an. Beachten Sie, dass dies die Gesamtbreite des Frames festlegt, **-borderwidth** oder ähnliches wird nicht hinzugefügt. Normalerweise sollte **-width** nicht verwendet werden, wenn ein propagierender Geometrie-Manager wie grid oder pack innerhalb des Frames verwendet wird, da der Geometrie-Manager die Breite des Frames überschreibt.

## Beschreibung

Der Befehl **frame** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Frame-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des Frames wie Hintergrundfarbe und Relief zu konfigurieren. Der Befehl **frame** gibt den Pfadnamen des neuen Fensters zurück.

Ein Frame ist ein einfaches Widget. Sein Hauptzweck ist es, als Abstandshalter oder Container für komplexe Fensterlayouts zu dienen. Die einzigen Merkmale eines Frames sind sein Hintergrund und ein optionaler 3-D-Rahmen, um den Frame erhaben oder eingesenkt erscheinen zu lassen.

### Widget-Befehl

Der Befehl **frame** erstellt einen neuen Tcl-Befehl, dessen Name identisch mit dem Pfadnamen des Frame-Fensters ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*PathName* ist der Name des Befehls, der identisch mit dem Pfadnamen des Frame-Widgets ist. *Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Frame-Widgets möglich:

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **frame** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **frame** akzeptierten Werte haben.

### Bindings

Wenn ein neuer Frame erstellt wird, hat er keine Standard-Event-Bindings: Frames sind nicht für Interaktion gedacht.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## getOpenFile

*Stichwörter:* getOpenFile, file selection dialog

### Name

tk_getOpenFile, tk_getSaveFile - Blendet einen Dialog zum Öffnen oder Speichern einer Datei ein

### Übersicht

**tk_getOpenFile** ?*option value* ...?
**tk_getSaveFile** ?*option value* ...?

### Beschreibung

Die Prozeduren **tk_getOpenFile** und **tk_getSaveFile** blenden einen Dialog ein, in dem der Benutzer eine Datei zum Öffnen oder Speichern auswählt. Der Befehl **tk_getOpenFile** ist üblicherweise dem Menübefehl **Open** im Menü **File** zugeordnet. Er dient dazu, dass der Benutzer *nur* eine bestehende Datei auswählt. Gibt der Benutzer eine nicht existierende Datei ein, zeigt der Dialog eine Fehlermeldung und verlangt eine andere Auswahl. Erlaubt eine Anwendung das Anlegen neuer Dateien, sollte dies über einen separaten Menübefehl **New** erfolgen.

Der Befehl **tk_getSaveFile** ist üblicherweise dem Menübefehl **Save as** im Menü **File** zugeordnet. Gibt der Benutzer eine bereits existierende Datei ein, fragt der Dialog, ob die bestehende Datei überschrieben werden soll.

Die folgenden *Option-Wert*-Paare sind als Befehlszeilenargumente für diese beiden Befehle möglich:

**-command *string***

Legt das Präfix eines Tcl-Befehls fest, der ausgeführt wird, wenn der Benutzer den Dialog nach einer Auswahl schließt. Dieser Rückruf wird nicht aufgerufen, wenn der Benutzer den Dialog abbricht. Der tatsächliche Befehl besteht aus *string*, gefolgt von einem Leerzeichen und dem vom Benutzer im Dialog gewählten Wert. Nur unter macOS verfügbar.

**-confirmoverwrite *boolean***

Legt fest, wie der Speichern-Dialog reagiert, wenn die gewählte Datei bereits existiert und das Speichern sie überschreiben würde. Ein wahrer Wert fordert einen Bestätigungsdialog. Ein falscher Wert überschreibt ohne Bestätigung. Standardwert ist true.

**-defaultextension *extension***

Legt einen String fest, der an den Dateinamen angehängt wird, wenn der Benutzer einen Dateinamen ohne Erweiterung eingibt. Standard ist der leere String, d. h. es wird in keinem Fall eine Erweiterung angehängt. Diese Option wird unter macOS ignoriert, wo Dateinamen keine Erweiterung benötigen; die UNIX-Implementierung leitet vernünftige Werte aus **-filetypes** ab, wenn nichts angegeben ist.

**-filetypes *filePatternList***

Existiert auf der Plattform eine **File types**-Listbox im Dateidialog, gibt diese Option die *filetype*s in dieser Listbox an. Wählt der Benutzer einen Dateityp, werden nur Dateien dieses Typs aufgelistet. Ist die Option nicht angegeben, leer oder wird die **File types**-Listbox von der Plattform nicht unterstützt, werden alle Dateien unabhängig vom Typ aufgelistet. Siehe Abschnitt Dateimuster festlegen unten zur Struktur von *filePatternList*.

**-initialdir *directory***

Legt fest, dass beim Öffnen des Dialogs die Dateien in *directory* angezeigt werden. Ist der Parameter nicht angegeben, ist das Startverzeichnis auf Nicht-Windows-Systemen das aktuelle Arbeitsverzeichnis; unter Windows vor Vista ebenfalls. Unter Vista und später ist es das zuletzt vom Benutzer für die Anwendung gewählte Verzeichnis. Gibt der Parameter einen relativen Pfad an, wandelt der Rückgabewert ihn in einen absoluten Pfad um.

**-initialfile *filename***

Legt einen Dateinamen fest, der beim Öffnen des Dialogs angezeigt wird.

**-message *string***

Legt eine Meldung fest, die im Client-Bereich des Dialogs angezeigt wird. Nur unter macOS verfügbar.

**-multiple *boolean***

Erlaubt dem Benutzer, mehrere Dateien im Öffnen-Dialog auszuwählen.

**-parent *window***

Macht *window* zum logischen Elternfenster des Dateidialogs. Der Dialog wird über dem Elternfenster angezeigt. Unter macOS wird der Dialog zu einem Sheet am Elternfenster.

**-title *titleString***

Legt einen String als Titel des Dialogs fest. Ist die Option nicht angegeben, wird ein Standardtitel angezeigt.

**-typevariable *variableName***

Die globale Variable *variableName* wählt beim Öffnen des Dialogs den Filter aus *filterList* vor und wird beim Schließen auf den zuletzt gewählten Filter aktualisiert. Die Variable wird einmal zu Beginn gelesen. Existiert sie nicht, passt ihr Wert keinem Filter-Typnamen oder ist leer (**{}**), wählt der Dialog standardmäßig den ersten Filter. Wird der Dialog abgebrochen, bleibt die Variable unverändert.

Wählt der Benutzer eine Datei, geben **tk_getOpenFile** und **tk_getSaveFile** den vollständigen Pfadnamen zurück. Bricht der Benutzer ab, geben beide Befehle den leeren String zurück.

## Dateimuster festlegen

Der von **-filetypes** angegebene Wert *filePatternList* ist eine Liste von Dateimustern. Jedes Dateimuster ist eine Liste der Form

```
typeName {extension ?extension ...?} ?{macType ?macType ...?}?
```

*typeName* ist der Name des durch dieses Muster beschriebenen Dateityps und der Text in der **File types**-Listbox. *extension* ist eine Dateierweiterung für dieses Muster. *macType* ist ein vierstelliges Macintosh-Dateityp-Kürzel. Die Liste der *macType*s ist optional und kann für Anwendungen ohne Macintosh-Bedarf weggelassen werden.

Mehrere Dateimuster können denselben *typeName* haben; sie bezeichnen dann denselben Dateityp und teilen sich einen Listbox-Eintrag. Wählt der Benutzer einen Eintrag, werden alle Dateien aufgelistet, die mindestens einem der zugehörigen Muster entsprechen. Üblicherweise entspricht jedes Muster einem eigenen Dateityp. Mehrere Muster für einen Typ sind nur auf der Macintosh-Plattform nötig.

Auf Macintosh passt eine Datei zu einem Muster, wenn ihr Name mindestens einer *extension* entspricht UND sie zu mindestens einem *macType* des Musters gehört. Das Muster **C Source Files** im Beispielcode passt zu Dateien mit Erweiterung **.c** UND *macType* **TEXT**. Für die ODER-Regel verwenden Sie zwei Muster, eines nur mit *extensions*, eines nur mit *macType*. Der Dateityp **GIF Files** im Beispielcode passt zu Dateien, die *entweder* die Erweiterung **.gif** haben ODER zum *macType* **GIFF** gehören.

Auf Unix und Windows passt eine Datei, wenn ihr Name mindestens einer *extension* des Musters entspricht. Die *macType*s werden ignoriert.

## Erweiterungen festlegen

Auf Unix und Macintosh werden Erweiterungen per Glob-Muster abgeglichen. Unter Windows übernimmt das Betriebssystem den Abgleich. Mögliche Erweiterungstypen:

1. die Sondererweiterung "*" passt zu jeder Datei;
2. die Sondererweiterung "" passt zu Dateien ohne Erweiterung (d. h. der Dateiname enthält keinen Punkt);
3. jede Zeichenkette ohne Platzhalter (* und ?).

Wegen unterschiedlicher Abgleichregeln auf den Plattformen sind Platzhalter in Erweiterungen zur Portierbarkeit nicht erlaubt, außer bei der Sondererweiterung "*". Erweiterungen ohne Punkt (z. B. "~") sind erlaubt, funktionieren aber nicht auf allen Plattformen.

### Beispiel

```
set types {
    {{Text Files}       {.txt}        }
    {{TCL Scripts}      {.tcl}        }
    {{C Source Files}   {.c}      TEXT}
    {{GIF Files}        {.gif}        }
    {{GIF Files}        {}        GIFF}
    {{All Files}        *             }
}
set filename [tk_getOpenFile -filetypes $types]

if {$filename ne ""} {
    # Open the file ...
}
```

*Copyright (c) 1996 Sun Microsystems, Inc.*

## grab

*Stichwörter:* grab, keyboard events, pointer events, window

### Name

grab - Maus- und Tastaturereignisse auf einen Fenster-Unterbaum beschränken

### Übersicht

**grab** ?**-global**? *window*
**grab** *option* ?*arg* ...?

### Beschreibung

Dieser Befehl implementiert einfache Maus- und Tastatur-Grabs für Tk. Tk-Grabs unterscheiden sich von den in der Xlib-Dokumentation beschriebenen Grabs. Ist für ein bestimmtes Fenster ein Grab gesetzt, beschränkt Tk alle Mauszeigerereignisse auf das Grab-Fenster und dessen Nachkommen in der Tk-Fensterhierarchie. Befindet sich der Mauszeiger innerhalb des Unterbaums des Grab-Fensters, verhält er sich genau so, als ob kein Grab aktiv wäre, und alle Ereignisse werden wie üblich gemeldet. Befindet sich der Mauszeiger außerhalb des Baums von *window*, werden Tastendrücke und -loslassungen sowie Mausbewegungen an *window* gemeldet; Fenstereintritts- und -austrittsereignisse werden ignoriert. Der Grab-Unterbaum besitzt den Mauszeiger: Fenster außerhalb des Grab-Unterbaums sind auf dem Bildschirm sichtbar, sind jedoch unempfindlich, bis der Grab freigegeben wird. Der Baum der Fenster unter dem Grab-Fenster kann Toplevel-Fenster enthalten; dann erhalten alle diese Toplevel-Fenster und ihre Nachkommen während des Grabs weiterhin Mausereignisse.

Zwei Grab-Formen sind möglich: lokal und global. Ein lokaler Grab betrifft nur die anfordernde Anwendung: Ereignisse werden an andere Anwendungen so gemeldet, als ob kein Grab stattgefunden hätte. Grabs sind standardmäßig lokal. Ein globaler Grab sperrt alle Anwendungen auf dem Bildschirm, sodass nur der angegebene Unterbaum der anfordernden Anwendung auf Mauszeigerereignisse reagiert (Tastendrücke und -loslassungen, Mausbewegungen, Fenstereintritte und -austritte). Während globaler Grabs erhält auch der Window-Manager keine Mauszeigerereignisse.

Bei lokalen Grabs werden Tastaturereignisse (Tastendrücke und -loslassungen) wie gewohnt zugestellt: Der Window-Manager steuert, welche Anwendung Tastaturereignisse erhält; gehen sie an ein Fenster der anfordernden Anwendung, werden sie an das Fokusfenster umgeleitet. Bei einem globalen Grab übernimmt Tk die Tastatur; alle Tastaturereignisse gehen stets an die anfordernde Anwendung. Der Befehl focus legt weiterhin fest, welches Fenster in der Anwendung die Tastaturereignisse erhält. Der Tastatur-Grab wird freigegeben, wenn der Grab endet.

Unter macOS betrifft ein globaler Grab alle von einem Tk-Prozess erzeugten Fenster. Kein anderes Fenster in diesem Prozess kann fokussiert werden, daher erhält kein anderes Fenster Tastatur- oder Mausereignisse. Ein lokaler Grab unter macOS betrifft alle von einem Tcl-Interpreter erzeugten Fenster. Während eines lokalen Grabs kann jedes Fenster des Tk-Prozesses fokussiert werden, aber das Grab-Fenster ist das einzige Fenster seines Interpreters, das Tastatur- oder Mausereignisse erhält. Fenster desselben Prozesses, die von anderen Interpretern erzeugt wurden, erhalten Tastatur- und Mausereignisse weiterhin normal.

Grabs gelten für bestimmte Displays. Hat eine Anwendung Fenster auf mehreren Displays, kann sie pro Display einen separaten Grab setzen. Der Grab auf einem Display betrifft nur die Fenster auf diesem Display. Verschiedene Anwendungen auf einem Display können gleichzeitig lokale Grabs haben, aber nur eine Anwendung kann gleichzeitig einen globalen Grab auf einem Display haben.

Der Befehl **grab** kann eine der folgenden Formen annehmen:

**grab ?-global? *window***

Entspricht **grab set**, siehe unten.

**grab current ?*window*?**

Ist *window* angegeben, gibt der Befehl den Namen des aktuellen Grab-Fensters dieser Anwendung für das Display von *window* zurück, oder eine leere Zeichenkette, wenn keines existiert. Wird *window* weggelassen, liefert der Befehl eine Liste aller von dieser Anwendung auf allen Displays gehaltenen Grab-Fenster, oder eine leere Zeichenkette, wenn die Anwendung keine Grabs hat.

**grab release *window***

Gibt den Grab auf *window* frei, falls einer gesetzt ist; andernfalls keine Wirkung. Gibt eine leere Zeichenkette zurück.

**grab set ?-global? *window***

Setzt einen Grab auf *window*. Mit **-global** ist der Grab global, andernfalls lokal. War für diese Anwendung auf dem Display von *window* bereits ein Grab aktiv, wird er automatisch freigegeben. Existiert bereits ein Grab auf *window* in derselben global/lokal-Form wie angefordert, tut der Befehl nichts. Gibt eine leere Zeichenkette zurück.

**grab status *window***

Gibt **none** zurück, wenn auf *window* kein Grab gesetzt ist, **local** bei lokalem Grab und **global** bei globalem Grab.

## Warnung

Mit globalen Grabs lässt sich ein Display sehr leicht vollständig unbenutzbar machen (z. B. Grab auf ein Widget ohne Ereignisbehandlung, ohne Mechanismus zur Freigabe). Seien Sie bei der Verwendung *äußerst* vorsichtig!

## Fehler

Die beschriebene einfache Grab-Wirkung erforderte eine außerordentlich komplexe und unschöne Implementierung. Bei der gegenwärtigen Implementierung ist es für Anwendungen nicht sicher, die Xlib-Grab-Funktionen außer über die Tk-Grab-Prozeduren zu nutzen. Versuchen Anwendungen, X-Grab-Mechanismen direkt zu manipulieren, wird es vermutlich Probleme geben.

Verwaltet ein einzelner Prozess mehrere Tk-Anwendungen, kann jeweils nur eine davon gleichzeitig einen lokalen Grab für ein Display haben. Liegen die Anwendungen in verschiedenen Prozessen, gilt diese Einschränkung nicht.

## Beispiel

Grab setzen, sodass aus einer Gruppe nur eine Schaltfläche per Maus klickbar ist. Die anderen reagieren erst nach Klick auf die mittlere Schaltfläche.

```
pack [button .b1 -text "Click me! #1" -command {destroy .b1}]
pack [button .b2 -text "Click me! #2" -command {destroy .b2}]
pack [button .b3 -text "Click me! #3" -command {destroy .b3}]
grab .b2
```

*Copyright (c) 1992 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# grid

*Stichwörter:* grid, geometry manager, location, cell, propagation, size, pack

## Name

grid - Geometrie-Manager, der Widgets in einem Raster anordnet

## Übersicht

**grid** *option arg* ?*arg* ...?

## Beschreibung

Der Befehl **grid** dient der Kommunikation mit dem Grid-Geometrie-Manager, der Widgets in Zeilen und Spalten innerhalb eines anderen Fensters anordnet, des Geometrie-Containers (oder Container-Fensters). Der Befehl **grid** kann je nach dem Argument *option* mehrere Formen annehmen:

**grid *window* ?*window* ...? ?*options*?**

Wenn das erste Argument von **grid** als erstes Fensterargument für **grid configure** geeignet ist — entweder ein Fenstername (ein Wert, der mit**.** beginnt) oder eines der Zeichen **x** oder **^** (siehe Abschnitt Relative Platzierung unten) —, wird der Befehl auf dieselbe Weise verarbeitet wie **grid configure**.

**grid anchor *window* ?*anchor*?**

Der Ankerwert steuert, wie das Raster im Container-Fenster platziert wird, wenn keine Zeile oder Spalte ein Gewicht hat. Siehe Der Grid-Algorithmus unten. Der Standard-*anchor* ist *nw*.

**grid bbox *window* ?*column row*? ?*column2 row2*?**

Ohne Argumente wird die Begrenzungsbox (in Pixeln) des Rasters zurückgegeben. Der Rückgabewert besteht aus 4 Ganzzahlen. Die ersten beiden sind der Pixel-Offset vom Container-Fenster (zuerst x, dann y) der oberen linken Ecke des Rasters, die zweiten beiden sind Breite und Höhe des Rasters, ebenfalls in Pixeln. Wird auf der Befehlszeile genau eine *column* und *row* angegeben, wird die Begrenzungsbox für diese Zelle zurückgegeben; die Zelle oben links ist mit Null nummeriert. Werden sowohl *column* als auch *row* angegeben, wird die Begrenzungsbox zurückgegeben, die die angegebenen Zeilen und Spalten umfasst.

**grid columnconfigure *window index* ?*-option value...*?**

Abfrage oder Festlegung der Spalteneigenschaften der Spalte *index* des Geometrie-Containers *window*. Gültige Optionen sind **-minsize**, **-weight**, **-uniform** und **-pad** . Wenn eine oder mehrere Optionen angegeben werden, kann *index* als Liste von Spaltenindizes angegeben werden, auf die die Konfigurationsoptionen wirken. Indizes können Ganzzahlen, Fensternamen oder das Schlüsselwort *all* sein. Bei *all* gelten die Optionen für alle Spalten, die derzeit von Inhaltsfenstern belegt sind. Bei einem Fensternamen muss dieses Fenster ein Inhalt dieses Containers sein; die Optionen gelten für alle Spalten, die derzeit von diesem Inhalt belegt sind. Die Option **-minsize** legt die minimale Größe in Bildschirmeinheiten fest, die für diese Spalte zulässig ist. Die Option **-weight** (Ganzzahl) legt das relative Gewicht für die Verteilung zusätzlichen Platzes auf die Spalten fest. Ein Gewicht von Null (0) bedeutet, dass die Spalte nicht von ihrer angeforderten Größe abweicht. Eine Spalte mit Gewicht zwei wächst bei zusätzlichem Platz doppelt so schnell wie eine Spalte mit Gewicht eins. Die Option **-uniform**: Ist ein nicht-leerer Wert angegeben, wird die Spalte in eine *uniforme Gruppe* mit anderen Spalten gleichen **-uniform**-Werts eingeordnet. Der Platz für Spalten einer uniformen Gruppe wird so verteilt, dass ihre Größen stets im strengen Verhältnis zu ihren **-weight**-Werten stehen. Siehe Der Grid-Algorithmus unten. Die Option **-pad** gibt die Anzahl Bildschirmeinheiten an, die zum größten Fenster hinzugefügt werden, das vollständig in dieser Spalte liegt, wenn der Grid-Geometrie-Manager eine Größe vom Container-Fenster anfordert. Wird nur eine Option ohne Wert angegeben, wird der aktuelle Wert dieser Option zurückgegeben. Wird nur Container-Fenster und Index angegeben, werden alle aktuellen Einstellungen in einer Liste von "*-option value*"-Paaren zurückgegeben.

**grid configure *window* ?*window* ...? ?*options*?**

Die Argumente bestehen aus den Namen eines oder mehrerer Inhaltsfenster gefolgt von Argumentpaaren, die festlegen, wie der Inhalt verwaltet wird. Statt eines Fensternamens können die Zeichen **-**, **x** und **^** angegeben werden, um die Standardposition eines *window* zu ändern (siehe Abschnitt Relative Platzierung unten). Folgende Optionen werden unterstützt:

**-column *n***

Das Fenster wird so eingefügt, dass es die *n*-te Spalte im Raster belegt. Spaltennummern beginnen bei 0. Fehlt diese Option, wird das Fenster direkt rechts vom zuvor in diesem **grid** -Aufruf angegebenen Fenster platziert, oder in Spalte "0", wenn es das erste Fenster ist. Für jedes **x**, das unmittelbar dem *window* vorausgeht, wird die Spaltenposition um eins erhöht. **x** steht damit für eine leere Spalte in dieser Zeile des Rasters.

**-columnspan *n***

Das Fenster belegt *n* Spalten im Raster. Standard ist eine Spalte, außer der Fenstername wird von **-** gefolgt; dann wird der Spaltenumfang für jedes unmittelbar folgende **-** um eins erhöht.

**-in *container***

Die Fenster werden im Container-Fenster *container* eingefügt. Standard ist das Elternfenster des ersten Fensters.

**-ipadx *amount***

*amount* legt fest, wie viel horizontaler innerer Abstand auf jeder Seite des Inhalts bleibt. Dieser Abstand wird innerhalb des Inhaltsrahmens hinzugefügt. *amount* muss eine gültige Bildschirmdistanz sein, z. B. **2** oder **.5c**. Standard ist 0.

**-ipady *amount***

*amount* legt fest, wie viel vertikaler innerer Abstand oben und unten am Inhalt bleibt. Dieser Abstand wird innerhalb des Inhaltsrahmens hinzugefügt. *amount* hat standardmäßig 0.

**-padx *amount***

*amount* legt den horizontalen äußeren Abstand auf jeder Seite des Inhalts in Bildschirmeinheiten fest. *Amount* kann eine Liste aus zwei Werten sein, um Abstand links und rechts getrennt anzugeben. Standard ist 0. Dieser Abstand liegt außerhalb des Inhaltsrahmens.

**-pady *amount***

*amount* legt den vertikalen äußeren Abstand oben und unten am Inhalt in Bildschirmeinheiten fest. *Amount* kann eine Liste aus zwei Werten sein, um Abstand oben und unten getrennt anzugeben. Standard ist 0. Dieser Abstand liegt außerhalb des Inhaltsrahmens.

**-row *n***

Der Inhalt wird so eingefügt, dass er die *n*-te Zeile im Raster belegt. Zeilennummern beginnen bei 0. Fehlt diese Option, wird der Inhalt in derselben Zeile wie der zuvor in diesem **grid**-Aufruf angegebene Inhalt platziert, oder in der nächsten Zeile unter der höchsten belegten Zeile, wenn dies der erste Inhalt ist.

**-rowspan *n***

Der Inhalt belegt *n* Zeilen im Raster. Standard ist eine Zeile. Enthält der nächste **grid** -Befehl **^**-Zeichen statt *content*, die mit den Spalten dieses *content* ausgerichtet sind, wird der **rowspan** dieses *content* um eins verlängert.

**-sticky *style***

Ist die Zelle eines Inhalts größer als seine angeforderten Abmessungen, kann diese Option den Inhalt in der Zelle positionieren (oder strecken). *Style* ist eine Zeichenkette mit null oder mehr der Zeichen **n**, **s**, **e** oder **w**. Die Zeichenkette darf optional Leerzeichen oder Kommas enthalten; sie werden ignoriert. Jeder Buchstabe bezeichnet eine Seite (Norden, Süden, Osten oder Westen), an die der Inhalt "klebt". Sind sowohl **n** als auch **s** (oder **e** und **w**) angegeben, wird der Inhalt gestreckt, um die gesamte Höhe (oder Breite) seiner Zelle zu füllen. Die Option **-sticky** umfasst die Kombination aus **-anchor** und **-fill** von pack. Standard ist "", wodurch der Inhalt in seiner Zelle zentriert in der angeforderten Größe erscheint.

Wird Inhalt bereits vom Geometrie-Manager verwaltet, behalten nicht angegebene Optionen ihre bisherigen Werte statt Standardwerte zu erhalten.

**grid content *window* ?*-option value*?**

Ohne Optionen wird eine Liste aller Inhalte in *window* zurückgegeben, zuletzt verwaltete zuerst. *Option* kann **-row** oder **-column** sein; dann werden nur die Inhalte in der durch *value* angegebenen Zeile (bzw. Spalte) zurückgegeben.

**grid forget *window* ?*window* ...?**

Entfernt jedes *window* aus dem Raster seines Containers und entmappt die Fenster. Der Inhalt wird nicht mehr vom Grid-Geometrie-Manager verwaltet. Die Konfigurationsoptionen für dieses Fenster werden vergessen, sodass bei erneuter Verwaltung durch den Grid-Geometrie-Manager die anfänglichen Standardwerte gelten.

Wird das letzte verwaltete Inhaltsfenster des Containers nicht mehr verwaltet, wird außerdem das virtuelle Ereignis <> an den Container gesendet; der Container kann sich daraufhin selbst neu dimensionieren (oder anderweitig reagieren).

**grid info *window***

Gibt eine Liste zurück, deren Elemente den aktuellen Konfigurationszustand des Inhalts *window* in derselben Option-Wert-Form darstellen, die an **grid configure** übergeben werden könnte. Die ersten beiden Elemente sind "**-in** *container*", wobei *container* das Container-Fenster des Fensters ist.

**grid location *window x y***

Bei *x*- und *y*-Werten in Bildschirmeinheiten relativ zum Container-Fenster werden Spalten- und Zeilennummer an dieser Position zurückgegeben. Liegt die Position oberhalb oder links vom Raster, wird **-1** zurückgegeben.

**grid propagate *window* ?*boolean*?**

Hat *boolean* einen wahren booleschen Wert wie **1** oder **on**, ist Propagation für *window* aktiviert; *window* muss ein Fenstername sein (siehe Geometrie-Propagation unten). Hat *boolean* einen falschen booleschen Wert, ist Propagation für *window* deaktiviert. In beiden Fällen wird ein leerer String zurückgegeben. Fehlt *boolean*, gibt der Befehl **0** oder **1** zurück, je nachdem, ob Propagation für *window* derzeit aktiv ist.

Propagation ist standardmäßig aktiviert.

**grid rowconfigure *window index* ?*-option value...*?**

Abfrage oder Festlegung der Zeileneigenschaften der Zeile *index* des Geometrie-Containers *window*. Gültige Optionen sind **-minsize**, **-weight**, **-uniform** und **-pad**. Wenn eine oder mehrere Optionen angegeben werden, kann *index* als Liste von Zeilenindizes angegeben werden, auf die die Konfigurationsoptionen wirken. Indizes können Ganzzahlen, Fensternamen oder das Schlüsselwort *all* sein. Bei *all* gelten die Optionen für alle Zeilen, die derzeit von Inhaltsfenstern belegt sind. Bei einem Fensternamen muss dieses Fenster ein Inhaltsfenster dieses Containers sein; die Optionen gelten für alle Zeilen, die derzeit vom Container-Fenster belegt sind. Die Option **-minsize** legt die minimale Größe in Bildschirmeinheiten fest, die für diese Zeile zulässig ist. Die Option **-weight** (Ganzzahl) legt das relative Gewicht für die Verteilung zusätzlichen Platzes auf die Zeilen fest. Ein Gewicht von Null (0) bedeutet, dass die Zeile nicht von ihrer angeforderten Größe abweicht. Eine Zeile mit Gewicht zwei wächst bei zusätzlichem Platz doppelt so schnell wie eine Zeile mit Gewicht eins. Die Option **-uniform**: Ist ein nicht-leerer Wert angegeben, wird die Zeile in eine *uniforme Gruppe* mit anderen Zeilen gleichen **-uniform**-Werts eingeordnet. Der Platz für Zeilen einer uniformen Gruppe wird so verteilt, dass ihre Größen stets im strengen Verhältnis zu ihren **-weight**-Werten stehen. Siehe Der Grid-Algorithmus unten. Die Option **-pad** gibt die Anzahl Bildschirmeinheiten an, die zum größten Fenster hinzugefügt werden, das vollständig in dieser Zeile liegt, wenn der Grid-Geometrie-Manager eine Größe vom Container-Fenster anfordert. Wird nur eine Option ohne Wert angegeben, wird der aktuelle Wert dieser Option zurückgegeben. Wird nur Container-Fenster und Index angegeben, werden alle aktuellen Einstellungen in einer Liste von "-option value"-Paaren zurückgegeben.

**grid remove *window* ?*window* ...?**

Entfernt jedes *window* aus dem Raster seines Containers und entmappt die Fenster. Der Inhalt wird nicht mehr vom Grid-Geometrie-Manager verwaltet. Die Konfigurationsoptionen für dieses Fenster bleiben jedoch gespeichert, sodass bei erneuter Verwaltung die bisherigen Werte erhalten bleiben.

Wird das letzte verwaltete Inhaltsfenster des Containers nicht mehr verwaltet, wird außerdem das virtuelle Ereignis <> an den Container gesendet; der Container kann sich daraufhin selbst neu dimensionieren (oder anderweitig reagieren).

**grid size *container***

Gibt die Rastergröße (zuerst Spalten, dann Zeilen) für *container* zurück. Die Größe ergibt sich entweder aus dem *content*, der die größte Zeile oder Spalte belegt, oder aus der größten Spalte oder Zeile mit nicht-null **-minsize**, **-weight** oder **-pad**.

**grid slaves *window* ?*-option value*?**

Synonym für **grid content** *window* ?*-option value*?.

## Relative Platzierung

Der Befehl **grid** bietet begrenzte Möglichkeiten, Layouts zu erzeugen, ohne für jeden Inhalt Zeilen- und Spalteninformationen anzugeben. Inhalte können so umgeordnet, hinzugefügt oder entfernt werden, ohne Zeilen und Spalten explizit festzulegen. Fehlen Spalten- oder Zeilenangaben für einen *content*, werden beim Verwalten Standardwerte für **-column**, **-row**, **-columnspan** und **-rowspan** gewählt. Die Werte hängen vom aktuellen Rasterlayout, der Position des *content* relativ zu anderen *content*s im selben **grid**-Befehl und vom Vorkommen der Zeichen **-**, **x** und **^** an Stellen ab, an denen normalerweise *content*-Namen erwartet werden.

**-**

Erhöht den **-columnspan** des Inhalts links davon. Mehrere **-** hintereinander erhöhen die Spaltenanzahl schrittweise. Ein **-** darf nicht auf **^** oder **x** folgen und nicht das erste *content* -Argument von **grid configure** sein.

**x**

Lässt eine leere Spalte zwischen dem Inhalt links und dem Inhalt rechts.

**^**

Verlängert den **-rowspan** des Inhalts über den **^** im Raster. Die Anzahl der **^** in einer Zeile muss der Spaltenanzahl entsprechen, die der Inhalt darüber überspannt.

## Der Grid-Algorithmus

Der Grid-Geometrie-Manager legt seinen Inhalt in drei Schritten aus. Im ersten Schritt wird die Mindestgröße berechnet, die alle Inhalte benötigen; ist Propagation eingeschaltet, wird das Container-Fenster gebeten, diese Größe anzunehmen. Im zweiten Schritt wird die angeforderte Größe mit der tatsächlichen Größe des Containers verglichen. Weichen sie ab, wird dem Layout bei Bedarf Platz hinzugefügt oder entnommen. Im letzten Schritt wird jeder Inhalt in seine Zeile(n) und Spalte(n) gemäß der *sticky*-Einstellung gesetzt.

Zur Berechnung der Mindestgröße eines Layouts betrachtet der Grid-Geometrie-Manager zuerst allen Inhalt mit **-columnspan** und **-rowspan** gleich eins und ermittelt die nominale Größe jeder Zeile oder Spalte als entweder *minsize* für diese Zeile oder Spalte oder Summe aus *pad* und Größe des größten Inhalts — je nachdem, was größer ist. Danach passen sich Zeilen oder Spalten in jeder uniformen Gruppe aneinander an. Anschließend werden Inhalte mit Zeilen- oder Spaltenumfang größer als eins geprüft. Muss eine Gruppe von Zeilen oder Spalten vergrößert werden, um diese Inhalte unterzubringen, wird jeder Zeile oder Spalte der Gruppe zusätzlicher Platz gemäß ihrem *weight* zugewiesen. Hat eine Gruppe überall Gewicht null, wird der zusätzliche Platz gleichmäßig verteilt.

Gehören mehrere Zeilen oder Spalten einer uniformen Gruppe an, wird der zugewiesene Platz stets im Verhältnis ihrer Gewichte verteilt. (Gewicht null gilt als 1.) Eine Zeile oder Spalte mit **-weight 1 -uniform a** hat also dieselbe Größe wie jede andere mit **-weight 1 -uniform a**. Mit **-weight 2 -uniform b** ist sie genau doppelt so groß wie mit **-weight 1 -uniform b**.

Technisch hat jede Zeile oder Spalte der Gruppe eine Größe *k·weight* für eine Konstante *k*. *k* wird so gewählt, dass keine Zeile oder Spalte unter ihre Mindestgröße fällt. Haben alle Zeilen oder Spalten einer Gruppe dasselbe Gewicht, hat jede dieselbe Größe wie die größte Zeile oder Spalte der Gruppe.

Ist der Container größer als das angeforderte Layout, wird zusätzlicher Platz gemäß Zeilen- und Spaltengewichten verteilt. Sind alle Gewichte null, wird das Layout gemäß dem *anchor*-Wert im Container platziert. Ist der Container kleiner als das angeforderte Layout, wird Platz von Spalten und Zeilen gemäß ihren Gewichten entnommen. Sobald eine Spalte oder Zeile ihre *minsize* erreicht, gilt ihr Gewicht als null. Muss mehr Platz entfernt werden als zulässig (wenn alle Zeilen oder Spalten bei Mindestgröße sind), wird das Layout platziert und gemäß *anchor* beschnitten.

## Geometrie-Propagation

Der Grid-Geometrie-Manager berechnet normalerweise, wie groß ein Container sein muss, um den Bedarf seines Inhalts genau zu decken, und setzt angeforderte Breite und Höhe des Containers auf diese Maße. So propagieren Geometrieinformationen in der Fensterhierarchie bis zum Toplevel, sodass der gesamte Teilbaum sich den Blattfenstern anpasst. Mit **grid propagate** kann Propagation für einen oder mehrere Container jedoch abgeschaltet werden. Ist Propagation deaktiviert, setzt **grid** die angeforderte Breite und Höhe des Container-Fensters nicht. Das ist nützlich, wenn ein Container eine von Ihnen festgelegte feste Größe haben soll.

## Einschränkungen für Container-Fenster

Der Container jedes Inhalts muss entweder das Elternelement des Inhalts sein (Standard) oder ein Nachkomme des Elternelements. Diese Einschränkung stellt sicher, dass der Inhalt über jeden sichtbaren Teil seines Containers gelegt werden kann, ohne vom Elternfenster abgeschnitten zu werden. Außerdem müssen alle Inhalte eines **grid**-Aufrufs denselben Container haben.

## Stacking-Reihenfolge

Ist der Container eines Inhalts nicht dessen Elternelement, muss der Inhalt in der Stacking-Reihenfolge über dem Container liegen. Andernfalls verdeckt der Container den Inhalt, und es wirkt, als wäre der Inhalt nicht korrekt verwaltet. Am einfachsten stellt man das sicher, indem man zuerst das Container-Fenster erzeugt: das zuletzt erzeugte Fenster liegt oben in der Stacking-Reihenfolge.

## Danksagungen

Der Befehl **grid** basiert auf Ideen des Geometrie-Managers *GridBag* von Doug Stein und des Geometrie-Managers **blt_table** von George Howlett.

## Beispiele

Ein Toplevel-Fenster mit Text-Widget und zwei Scrollleisten:

```
# Widgets erzeugen
toplevel .t
text .t.txt -wrap none -xscroll {.t.h set} -yscroll {.t.v set}
scrollbar .t.v -orient vertical   -command {.t.txt yview}
scrollbar .t.h -orient horizontal -command {.t.txt xview}

# Anordnen
grid .t.txt .t.v -sticky nsew
grid .t.h        -sticky nsew

# Text-Widget soll den zusätzlichen Platz einnehmen
grid rowconfigure    .t .t.txt -weight 1
grid columnconfigure .t .t.txt -weight 1
```

Drei Widgets gleicher Breite trotz unterschiedlicher "natürlicher" Breite:

```
button .b -text "Foo"
entry .e -textvariable foo ; set foo "Hello World!"
label .l -text "This is a fairly long piece of text"

grid .b .e .l -sticky ew
grid columnconfigure . "all" -uniform allTheSame
```

*Copyright (c) 1996 Sun Microsystems, Inc.*

# image

*Stichwörter:* image, height, types of images, width

## Name

image - Erstellt und manipuliert Bilder

## Übersicht

**image** *option* ?*arg* ...?

## Beschreibung

Der Befehl **image** erstellt, löscht und fragt Bilder ab. Er kann je nach dem Argument *option* verschiedene Formen annehmen. Die zulässigen Formen sind:

**image create *type* ?*name*? ?*option value* ...?**

Erstellt ein neues Bild und einen Befehl mit demselben Namen und gibt dessen Namen zurück. *type* legt den Bildtyp fest, der einer der derzeit definierten Typen sein muss (z. B. bitmap). *name* gibt den Namen des Bildes an; wird er weggelassen, wählt Tk einen Namen der Form **image***x*, wobei *x* eine Ganzzahl ist. Es kann beliebig viele *option-value*-Paare geben, die Konfigurationsoptionen für das neue Bild liefern. Die zulässigen Optionen sind für jeden Bildtyp separat definiert; Details zu den Optionen eingebauter Bildtypen siehe unten. Existiert bereits ein Bild mit dem angegebenen Namen, wird es durch das neue Bild ersetzt und alle Instanzen dieses Bildes werden mit dem neuen Inhalt neu gezeichnet. Wichtig: Der Befehl **image** überschreibt stillschweigend jede Prozedur, die unter dem angegebenen Namen definiert sein könnte — den Namen also mit Bedacht wählen. Es wird empfohlen, einen separaten Namespace für Bildnamen zu verwenden (z. B. **::img::logo**, **::img::large**).

**image delete ?*name name* ...?**

Löscht jedes der benannten Bilder und gibt einen leeren String zurück. Werden Instanzen der Bilder in Widgets angezeigt, werden die Bilder erst wirklich gelöscht, wenn alle Instanzen freigegeben sind. Die Zuordnung zwischen Instanzen und dem Bildmanager wird jedoch aufgehoben. Bestehende Instanzen behalten ihre Größen, zeigen aber leere Bereiche. Wird ein gelöschtes Bild mit einem weiteren Aufruf von **image create** neu erzeugt, verwenden die bestehenden Instanzen das neue Bild.

**image height *name***

Gibt einen Dezimalstring mit der Höhe des Bildes *name* in Pixeln zurück.

**image inuse *name***

Gibt einen booleschen Wert zurück, der angibt, ob das Bild *name* von Widgets verwendet wird.

**image names**

Gibt eine Liste mit den Namen aller vorhandenen Bilder zurück.

**image type *name***

Gibt den Typ des Bildes *name* zurück (den Wert des Arguments *type* bei **image create**, als das Bild erzeugt wurde).

**image types**

Gibt eine Liste zurück, deren Elemente alle gültigen Bildtypen sind (d. h. alle Werte, die für das Argument *type* bei **image create** angegeben werden können).

**image width *name***

Gibt einen Dezimalstring mit der Breite des Bildes *name* in Pixeln zurück.

Weitere Operationen (z. B. Schreiben des Bildes in eine Datei) können als Unterbefehle des Bildinstanz-Befehls verfügbar sein. Details stehen in der Manpage des jeweiligen Bildtyps.

## Eingebaute Bildtypen

Die folgenden Bildtypen werden von Tk definiert und stehen in jeder Tk-Anwendung zur Verfügung. Einzelne Anwendungen oder Erweiterungen können weitere Typen definieren.

bitmap

Jedes Pixel im Bild zeigt eine Vordergrundfarbe, eine Hintergrundfarbe oder nichts. Weitere Informationen in der Manpage bitmap.

photo

Zeigt verschiedene Vollfarbbilder an und nutzt Dithering, um Farben auf Displays mit begrenzten Farbfähigkeiten anzunähern. Weitere Informationen in der Manpage photo.

nsimage

Dieser Typ ist nur auf der Aqua-Plattform verfügbar. Es handelt sich um ein Vollfarbbild, das aus einem benannten Systembild erzeugt werden kann. Er bietet Optionen zur einfachen Verwendung dieser Bilder in Buttons. Ein nsimage kann auf einem *Template-Bild* basieren — einem antialiasierten Zwei-Farben-Format, das Schwarz und Weiß automatisch tauscht, wenn macOS zwischen Hell- und Dunkelmodus wechselt. Ein nsimage kann auch eine alternative *pressed*-Variante erzeugen, die im Dunkelmodus heller und im Hellmodus dunkler ist. Diese dienen Bild-Buttons. Weitere Informationen in der Manpage nsimage.

*Copyright (c) 1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# keysyms

*Stichwörter:* keysyms, bind, binding, event, keysym

## Name

keysyms - von Tk erkannte Keysyms

## Beschreibung

Tk erkennt viele Keysyms beim Festlegen von Tasten-Bindings (z. B. "bind **. <***keysym***>**"). Die folgende Liste führt die Keysyms auf, die von Tk erkannt werden. Beachten Sie, dass nicht alle Keysyms auf allen Plattformen gültig sind und einige Keysyms auch auf Plattformen verfügbar sind, die für diese Taste einen anderen nativen Namen haben. Auf Unix-Systemen hängt z. B. die Verfügbarkeit eines bestimmten Keysyms von der Konfiguration der Tastatur-Modifier-Map ab. Die Liste zeigt Keysyms zusammen mit ihren Dezimal- und Hexadezimalwerten.

| | | |
|---|---|---|
| space | 32 | 0x20 |
| ! (exclam) | 33 | 0x21 |
| quotedbl | 34 | 0x22 |
| numbersign | 35 | 0x23 |
| dollar | 36 | 0x24 |
| % (percent) | 37 | 0x25 |
| & (ampersand) | 38 | 0x26 |
| ' (apostrophe) | 39 | 0x27 |
| ( (parenleft) | 40 | 0x28 |
| ) (parenright) | 41 | 0x29 |
| * (asterisk) | 42 | 0x2A |
| + (plus) | 43 | 0x2B |
| , (comma) | 44 | 0x2C |
| minus | 45 | 0x2D |
| . (period) | 46 | 0x2E |
| / (slash) | 47 | 0x2F |
| 0 | 48 | 0x30 |
| 1 | 49 | 0x31 |
| 2 | 50 | 0x32 |
| 3 | 51 | 0x33 |
| 4 | 52 | 0x34 |
| 5 | 53 | 0x35 |
| 6 | 54 | 0x36 |

```
7                                   55     0x37
8                                   56     0x38
9                                   57     0x39
: (colon)                           58     0x3A
semicolon                           59     0x3B
less                                60     0x3C
= (equal)                           61     0x3D
greater                             62     0x3E
? (question)                        63     0x3F
@ (at)                              64     0x40
A                                   65     0x41
B                                   66     0x42
C                                   67     0x43
D                                   68     0x44
E                                   69     0x45
F                                   70     0x46
G                                   71     0x47
H                                   72     0x48
I                                   73     0x49
J                                   74     0x4A
K                                   75     0x4B
L                                   76     0x4C
M                                   77     0x4D
N                                   78     0x4E
O                                   79     0x4F
P                                   80     0x50
Q                                   81     0x51
R                                   82     0x52
S                                   83     0x53
T                                   84     0x54
U                                   85     0x55
V                                   86     0x56
W                                   87     0x57
X                                   88     0x58
Y                                   89     0x59
Z                                   90     0x5A
bracketleft                         91     0x5B
backslash                           92     0x5C
bracketright                        93     0x5D
^ (asciicircum)                     94     0x5E
_ (underscore)                      95     0x5F
` (grave)                           96     0x60
a                                   97     0x61
b                                   98     0x62
c                                   99     0x63
d                                  100     0x64
```

```
e                                  101     0x65
f                                  102     0x66
g                                  103     0x67
h                                  104     0x68
i                                  105     0x69
j                                  106     0x6A
k                                  107     0x6B
l                                  108     0x6C
m                                  109     0x6D
n                                  110     0x6E
o                                  111     0x6F
p                                  112     0x70
q                                  113     0x71
r                                  114     0x72
s                                  115     0x73
t                                  116     0x74
u                                  117     0x75
v                                  118     0x76
w                                  119     0x77
x                                  120     0x78
y                                  121     0x79
z                                  122     0x7A
braceleft                          123     0x7B
| (bar)                            124     0x7C
braceright                         125     0x7D
~ (asciitilde)                     126     0x7E
nobreakspace                       160     0xA0
¡ (exclamdown)                     161     0xA1
¢ (cent)                           162     0xA2
£ (sterling)                       163     0xA3
¤ (currency)                       164     0xA4
¥ (yen)                            165     0xA5
¦ (brokenbar)                      166     0xA6
§ (section)                        167     0xA7
¨ (diaeresis)                      168     0xA8
© (copyright)                      169     0xA9
ª (ordfeminine)                    170     0xAA
« (guillemetleft)                  171     0xAB
¬ (notsign)                        172     0xAC
hyphen                             173     0xAD
® (registered)                     174     0xAE
¯ (macron)                         175     0xAF
° (degree)                         176     0xB0
± (plusminus)                      177     0xB1
² (twosuperior)                    178     0xB2
³ (threesuperior)                  179     0xB3
```

```
´ (acute)                           180     0xB4
µ (mu)                              181     0xB5
¶ (paragraph)                       182     0xB6
· (periodcentered)                  183     0xB7
¸ (cedilla)                         184     0xB8
¹ (onesuperior)                     185     0xB9
º (ordmasculine)                    186     0xBA
» (guillemetright)                  187     0xBB
¼ (onequarter)                      188     0xBC
½ (onehalf)                         189     0xBD
¾ (threequarters)                   190     0xBE
¿ (questiondown)                    191     0xBF
À (Agrave)                          192     0xC0
Á (Aacute)                          193     0xC1
Â (Acircumflex)                     194     0xC2
Ã (Atilde)                          195     0xC3
Ä (Adiaeresis)                      196     0xC4
Å (Aring)                           197     0xC5
Æ (AE)                              198     0xC6
Ç (Ccedilla)                        199     0xC7
È (Egrave)                          200     0xC8
É (Eacute)                          201     0xC9
Ê (Ecircumflex)                     202     0xCA
Ë (Ediaeresis)                      203     0xCB
Ì (Igrave)                          204     0xCC
Í (Iacute)                          205     0xCD
Î (Icircumflex)                     206     0xCE
Ï (Idiaeresis)                      207     0xCF
Ð (ETH)                             208     0xD0
Ñ (Ntilde)                          209     0xD1
Ò (Ograve)                          210     0xD2
Ó (Oacute)                          211     0xD3
Ô (Ocircumflex)                     212     0xD4
Õ (Otilde)                          213     0xD5
Ö (Odiaeresis)                      214     0xD6
× (multiply)                        215     0xD7
Ø (Oslash)                          216     0xD8
Ù (Ugrave)                          217     0xD9
Ú (Uacute)                          218     0xDA
Û (Ucircumflex)                     219     0xDB
Ü (Udiaeresis)                      220     0xDC
Ý (Yacute)                          221     0xDD
Þ (THORN)                           222     0xDE
ß (ssharp)                          223     0xDF
à (agrave)                          224     0xE0
á (aacute)                          225     0xE1
```

| | | |
|---|---|---|
| â (acircumflex) | 226 | 0xE2 |
| ã (atilde) | 227 | 0xE3 |
| ä (adiaeresis) | 228 | 0xE4 |
| å (aring) | 229 | 0xE5 |
| æ (ae) | 230 | 0xE6 |
| ç (ccedilla) | 231 | 0xE7 |
| è (egrave) | 232 | 0xE8 |
| é (eacute) | 233 | 0xE9 |
| ê (ecircumflex) | 234 | 0xEA |
| ë (ediaeresis) | 235 | 0xEB |
| ì (igrave) | 236 | 0xEC |
| í (iacute) | 237 | 0xED |
| î (icircumflex) | 238 | 0xEE |
| ï (idiaeresis) | 239 | 0xEF |
| ð (eth) | 240 | 0xF0 |
| ñ (ntilde) | 241 | 0xF1 |
| ò (ograve) | 242 | 0xF2 |
| ó (oacute) | 243 | 0xF3 |
| ô (ocircumflex) | 244 | 0xF4 |
| õ (otilde) | 245 | 0xF5 |
| ö (odiaeresis) | 246 | 0xF6 |
| ÷ (division) | 247 | 0xF7 |
| ø (oslash) | 248 | 0xF8 |
| ù (ugrave) | 249 | 0xF9 |
| ú (uacute) | 250 | 0xFA |
| û (ucircumflex) | 251 | 0xFB |
| ü (udiaeresis) | 252 | 0xFC |
| ý (yacute) | 253 | 0xFD |
| þ (thorn) | 254 | 0xFE |
| ÿ (ydiaeresis) | 255 | 0xFF |

| | | |
|---|---|---|
| Aogonek | 417 | 0x1A1 |
| breve | 418 | 0x1A2 |
| Lstroke | 419 | 0x1A3 |
| Lcaron | 421 | 0x1A5 |
| Sacute | 422 | 0x1A6 |
| Scaron | 425 | 0x1A9 |
| Scedilla | 426 | 0x1AA |
| Tcaron | 427 | 0x1AB |
| Zacute | 428 | 0x1AC |
| Zcaron | 430 | 0x1AE |
| Zabovedot | 431 | 0x1AF |
| aogonek | 433 | 0x1B1 |
| ogonek | 434 | 0x1B2 |
| lstroke | 435 | 0x1B3 |
| lcaron | 437 | 0x1B5 |

```
sacute                             438     0x1B6
caron                              439     0x1B7
scaron                             441     0x1B9
scedilla                           442     0x1BA
tcaron                             443     0x1BB
zacute                             444     0x1BC
doubleacute                        445     0x1BD
zcaron                             446     0x1BE
zabovedot                          447     0x1BF
Racute                             448     0x1C0
Abreve                             451     0x1C3
Lacute                             453     0x1C5
Cacute                             454     0x1C6
Ccaron                             456     0x1C8
Eogonek                            458     0x1CA
Ecaron                             460     0x1CC
Dcaron                             463     0x1CF
Dstroke                            464     0x1D0
Nacute                             465     0x1D1
Ncaron                             466     0x1D2
Odoubleacute                       469     0x1D5
Rcaron                             472     0x1D8
Uring                              473     0x1D9
Udoubleacute                       475     0x1DB
Tcedilla                           478     0x1DE
racute                             480     0x1E0
abreve                             483     0x1E3
lacute                             485     0x1E5
cacute                             486     0x1E6
ccaron                             488     0x1E8
eogonek                            490     0x1EA
ecaron                             492     0x1EC
dcaron                             495     0x1EF
dstroke                            496     0x1F0
nacute                             497     0x1F1
ncaron                             498     0x1F2
odoubleacute                       501     0x1F5
rcaron                             504     0x1F8
uring                              505     0x1F9
udoubleacute                       507     0x1FB
tcedilla                           510     0x1FE
abovedot                           511     0x1FF
Hstroke                            673     0x2A1
Hcircumflex                        678     0x2A6
Iabovedot                          681     0x2A9
Gbreve                             683     0x2AB
```

| | | |
|---|---|---|
| Jcircumflex | 684 | 0x2AC |
| hstroke | 689 | 0x2B1 |
| hcircumflex | 694 | 0x2B6 |
| idotless | 697 | 0x2B9 |
| gbreve | 699 | 0x2BB |
| jcircumflex | 700 | 0x2BC |
| Cabovedot | 709 | 0x2C5 |
| Ccircumflex | 710 | 0x2C6 |
| Gabovedot | 725 | 0x2D5 |
| Gcircumflex | 728 | 0x2D8 |
| Ubreve | 733 | 0x2DD |
| Scircumflex | 734 | 0x2DE |
| cabovedot | 741 | 0x2E5 |
| ccircumflex | 742 | 0x2E6 |
| gabovedot | 757 | 0x2F5 |
| gcircumflex | 760 | 0x2F8 |
| ubreve | 765 | 0x2FD |
| scircumflex | 766 | 0x2FE |
| kra | 930 | 0x3A2 |
| Rcedilla | 931 | 0x3A3 |
| Itilde | 933 | 0x3A5 |
| Lcedilla | 934 | 0x3A6 |
| Emacron | 938 | 0x3AA |
| Gcedilla | 939 | 0x3AB |
| Tslash | 940 | 0x3AC |
| rcedilla | 947 | 0x3B3 |
| itilde | 949 | 0x3B5 |
| lcedilla | 950 | 0x3B6 |
| emacron | 954 | 0x3BA |
| gcedilla | 955 | 0x3BB |
| tslash | 956 | 0x3BC |
| ENG | 957 | 0x3BD |
| eng | 959 | 0x3BF |
| Amacron | 960 | 0x3C0 |
| Iogonek | 967 | 0x3C7 |
| Eabovedot | 972 | 0x3CC |
| Imacron | 975 | 0x3CF |
| Ncedilla | 977 | 0x3D1 |
| Omacron | 978 | 0x3D2 |
| Kcedilla | 979 | 0x3D3 |
| Uogonek | 985 | 0x3D9 |
| Utilde | 989 | 0x3DD |
| Umacron | 990 | 0x3DE |
| amacron | 992 | 0x3E0 |
| iogonek | 999 | 0x3E7 |
| eabovedot | 1004 | 0x3EC |

| | | |
|---|---|---|
| imacron | 1007 | 0x3EF |
| ncedilla | 1009 | 0x3F1 |
| omacron | 1010 | 0x3F2 |
| kcedilla | 1011 | 0x3F3 |
| uogonek | 1017 | 0x3F9 |
| utilde | 1021 | 0x3FD |
| umacron | 1022 | 0x3FE |
| overline | 1150 | 0x47E |
| kana_fullstop | 1185 | 0x4A1 |
| kana_openingbracket | 1186 | 0x4A2 |
| kana_closingbracket | 1187 | 0x4A3 |
| kana_comma | 1188 | 0x4A4 |
| kana_conjunctive | 1189 | 0x4A5 |
| kana_WO | 1190 | 0x4A6 |
| kana_a | 1191 | 0x4A7 |
| kana_i | 1192 | 0x4A8 |
| kana_u | 1193 | 0x4A9 |
| kana_e | 1194 | 0x4AA |
| kana_o | 1195 | 0x4AB |
| kana_ya | 1196 | 0x4AC |
| kana_yu | 1197 | 0x4AD |
| kana_yo | 1198 | 0x4AE |
| kana_tsu | 1199 | 0x4AF |
| prolongedsound | 1200 | 0x4B0 |
| kana_A | 1201 | 0x4B1 |
| kana_I | 1202 | 0x4B2 |
| kana_U | 1203 | 0x4B3 |
| kana_E | 1204 | 0x4B4 |
| kana_O | 1205 | 0x4B5 |
| kana_KA | 1206 | 0x4B6 |
| kana_KI | 1207 | 0x4B7 |
| kana_KU | 1208 | 0x4B8 |
| kana_KE | 1209 | 0x4B9 |
| kana_KO | 1210 | 0x4BA |
| kana_SA | 1211 | 0x4BB |
| kana_SHI | 1212 | 0x4BC |
| kana_SU | 1213 | 0x4BD |
| kana_SE | 1214 | 0x4BE |
| kana_SO | 1215 | 0x4BF |
| kana_TA | 1216 | 0x4C0 |
| kana_CHI | 1217 | 0x4C1 |
| kana_TSU | 1218 | 0x4C2 |
| kana_TE | 1219 | 0x4C3 |
| kana_TO | 1220 | 0x4C4 |
| kana_NA | 1221 | 0x4C5 |
| kana_NI | 1222 | 0x4C6 |

| | | |
|---|---|---|
| kana_NU | 1223 | 0x4C7 |
| kana_NE | 1224 | 0x4C8 |
| kana_NO | 1225 | 0x4C9 |
| kana_HA | 1226 | 0x4CA |
| kana_HI | 1227 | 0x4CB |
| kana_FU | 1228 | 0x4CC |
| kana_HE | 1229 | 0x4CD |
| kana_HO | 1230 | 0x4CE |
| kana_MA | 1231 | 0x4CF |
| kana_MI | 1232 | 0x4D0 |
| kana_MU | 1233 | 0x4D1 |
| kana_ME | 1234 | 0x4D2 |
| kana_MO | 1235 | 0x4D3 |
| kana_YA | 1236 | 0x4D4 |
| kana_YU | 1237 | 0x4D5 |
| kana_YO | 1238 | 0x4D6 |
| kana_RA | 1239 | 0x4D7 |
| kana_RI | 1240 | 0x4D8 |
| kana_RU | 1241 | 0x4D9 |
| kana_RE | 1242 | 0x4DA |
| kana_RO | 1243 | 0x4DB |
| kana_WA | 1244 | 0x4DC |
| kana_N | 1245 | 0x4DD |
| voicedsound | 1246 | 0x4DE |
| semivoicedsound | 1247 | 0x4DF |

| | | |
|---|---|---|
| Arabic_comma | 1452 | 0x5AC |
| Arabic_semicolon | 1467 | 0x5BB |
| Arabic_question_mark | 1471 | 0x5BF |
| Arabic_hamza | 1473 | 0x5C1 |
| Arabic_maddaonalef | 1474 | 0x5C2 |
| Arabic_hamzaonalef | 1475 | 0x5C3 |
| Arabic_hamzaonwaw | 1476 | 0x5C4 |
| Arabic_hamzaunderalef | 1477 | 0x5C5 |
| Arabic_hamzaonyeh | 1478 | 0x5C6 |
| Arabic_alef | 1479 | 0x5C7 |
| Arabic_beh | 1480 | 0x5C8 |
| Arabic_tehmarbuta | 1481 | 0x5C9 |
| Arabic_teh | 1482 | 0x5CA |
| Arabic_theh | 1483 | 0x5CB |
| Arabic_jeem | 1484 | 0x5CC |
| Arabic_hah | 1485 | 0x5CD |
| Arabic_khah | 1486 | 0x5CE |
| Arabic_dal | 1487 | 0x5CF |
| Arabic_thal | 1488 | 0x5D0 |
| Arabic_ra | 1489 | 0x5D1 |

| | | |
|---|---|---|
| Arabic_zain | 1490 | 0x5D2 |
| Arabic_seen | 1491 | 0x5D3 |
| Arabic_sheen | 1492 | 0x5D4 |
| Arabic_sad | 1493 | 0x5D5 |
| Arabic_dad | 1494 | 0x5D6 |
| Arabic_tah | 1495 | 0x5D7 |
| Arabic_zah | 1496 | 0x5D8 |
| Arabic_ain | 1497 | 0x5D9 |
| Arabic_ghain | 1498 | 0x5DA |
| Arabic_tatweel | 1504 | 0x5E0 |
| Arabic_feh | 1505 | 0x5E1 |
| Arabic_qaf | 1506 | 0x5E2 |
| Arabic_kaf | 1507 | 0x5E3 |
| Arabic_lam | 1508 | 0x5E4 |
| Arabic_meem | 1509 | 0x5E5 |
| Arabic_noon | 1510 | 0x5E6 |
| Arabic_ha | 1511 | 0x5E7 |
| Arabic_waw | 1512 | 0x5E8 |
| Arabic_alefmaksura | 1513 | 0x5E9 |
| Arabic_yeh | 1514 | 0x5EA |
| Arabic_fathatan | 1515 | 0x5EB |
| Arabic_dammatan | 1516 | 0x5EC |
| Arabic_kasratan | 1517 | 0x5ED |
| Arabic_fatha | 1518 | 0x5EE |
| Arabic_damma | 1519 | 0x5EF |
| Arabic_kasra | 1520 | 0x5F0 |
| Arabic_shadda | 1521 | 0x5F1 |
| Arabic_sukun | 1522 | 0x5F2 |
| Serbian_dje | 1697 | 0x6A1 |
| Macedonia_gje | 1698 | 0x6A2 |
| Cyrillic_io | 1699 | 0x6A3 |
| Ukrainian_ie | 1700 | 0x6A4 |
| Macedonia_dse | 1701 | 0x6A5 |
| Ukrainian_i | 1702 | 0x6A6 |
| Ukrainian_yi | 1703 | 0x6A7 |
| Cyrillic_je | 1704 | 0x6A8 |
| Cyrillic_lje | 1705 | 0x6A9 |
| Cyrillic_nje | 1706 | 0x6AA |
| Serbian_tshe | 1707 | 0x6AB |
| Macedonia_kje | 1708 | 0x6AC |
| Ukrainian_ghe_with_upturn | 1709 | 0x6AD |
| Byelorussian_shortu | 1710 | 0x6AE |
| Cyrillic_dzhe | 1711 | 0x6AF |
| numerosign | 1712 | 0x6B0 |
| Serbian_DJE | 1713 | 0x6B1 |
| Macedonia_GJE | 1714 | 0x6B2 |

| | | |
|---|---|---|
| Cyrillic_IO | 1715 | 0x6B3 |
| Ukrainian_IE | 1716 | 0x6B4 |
| Macedonia_DSE | 1717 | 0x6B5 |
| Ukrainian_I | 1718 | 0x6B6 |
| Ukrainian_YI | 1719 | 0x6B7 |
| Cyrillic_JE | 1720 | 0x6B8 |
| Cyrillic_LJE | 1721 | 0x6B9 |
| Cyrillic_NJE | 1722 | 0x6BA |
| Serbian_TSHE | 1723 | 0x6BB |
| Macedonia_KJE | 1724 | 0x6BC |
| Ukrainian_GHE_WITH_UPTURN | 1725 | 0x6BD |
| Byelorussian_SHORTU | 1726 | 0x6BE |
| Cyrillic_DZHE | 1727 | 0x6BF |
| Cyrillic_yu | 1728 | 0x6C0 |
| Cyrillic_a | 1729 | 0x6C1 |
| Cyrillic_be | 1730 | 0x6C2 |
| Cyrillic_tse | 1731 | 0x6C3 |
| Cyrillic_de | 1732 | 0x6C4 |
| Cyrillic_ie | 1733 | 0x6C5 |
| Cyrillic_ef | 1734 | 0x6C6 |
| Cyrillic_ghe | 1735 | 0x6C7 |
| Cyrillic_ha | 1736 | 0x6C8 |
| Cyrillic_i | 1737 | 0x6C9 |
| Cyrillic_shorti | 1738 | 0x6CA |
| Cyrillic_ka | 1739 | 0x6CB |
| Cyrillic_el | 1740 | 0x6CC |
| Cyrillic_em | 1741 | 0x6CD |
| Cyrillic_en | 1742 | 0x6CE |
| Cyrillic_o | 1743 | 0x6CF |
| Cyrillic_pe | 1744 | 0x6D0 |
| Cyrillic_ya | 1745 | 0x6D1 |
| Cyrillic_er | 1746 | 0x6D2 |
| Cyrillic_es | 1747 | 0x6D3 |
| Cyrillic_te | 1748 | 0x6D4 |
| Cyrillic_u | 1749 | 0x6D5 |
| Cyrillic_zhe | 1750 | 0x6D6 |
| Cyrillic_ve | 1751 | 0x6D7 |
| Cyrillic_softsign | 1752 | 0x6D8 |
| Cyrillic_yeru | 1753 | 0x6D9 |
| Cyrillic_ze | 1754 | 0x6DA |
| Cyrillic_sha | 1755 | 0x6DB |
| Cyrillic_e | 1756 | 0x6DC |
| Cyrillic_shcha | 1757 | 0x6DD |
| Cyrillic_che | 1758 | 0x6DE |
| Cyrillic_hardsign | 1759 | 0x6DF |
| Cyrillic_YU | 1760 | 0x6E0 |

```
Cyrillic_A                        1761     0x6E1
Cyrillic_BE                       1762     0x6E2
Cyrillic_TSE                      1763     0x6E3
Cyrillic_DE                       1764     0x6E4
Cyrillic_IE                       1765     0x6E5
Cyrillic_EF                       1766     0x6E6
Cyrillic_GHE                      1767     0x6E7
Cyrillic_HA                       1768     0x6E8
Cyrillic_I                        1769     0x6E9
Cyrillic_SHORTI                   1770     0x6EA
Cyrillic_KA                       1771     0x6EB
Cyrillic_EL                       1772     0x6EC
Cyrillic_EM                       1773     0x6ED
Cyrillic_EN                       1774     0x6EE
Cyrillic_O                        1775     0x6EF
Cyrillic_PE                       1776     0x6F0
Cyrillic_YA                       1777     0x6F1
Cyrillic_ER                       1778     0x6F2
Cyrillic_ES                       1779     0x6F3
Cyrillic_TE                       1780     0x6F4
Cyrillic_U                        1781     0x6F5
Cyrillic_ZHE                      1782     0x6F6
Cyrillic_VE                       1783     0x6F7
Cyrillic_SOFTSIGN                 1784     0x6F8
Cyrillic_YERU                     1785     0x6F9
Cyrillic_ZE                       1786     0x6FA
Cyrillic_SHA                      1787     0x6FB
Cyrillic_E                        1788     0x6FC
Cyrillic_SHCHA                    1789     0x6FD
Cyrillic_CHE                      1790     0x6FE
Cyrillic_HARDSIGN                 1791     0x6FF
Greek_ALPHAaccent                 1953     0x7A1
Greek_EPSILONaccent               1954     0x7A2
Greek_ETAaccent                   1955     0x7A3
Greek_IOTAaccent                  1956     0x7A4
Greek_IOTAdieresis                1957     0x7A5
Greek_IOTAaccentdiaeresis         1958     0x7A6
Greek_OMICRONaccent               1959     0x7A7
Greek_UPSILONaccent               1960     0x7A8
Greek_UPSILONdieresis             1961     0x7A9
Greek_UPSILONaccentdieresis       1962     0x7AA
Greek_OMEGAaccent                 1963     0x7AB
Greek_accentdieresis              1966     0x7AE
Greek_horizbar                    1967     0x7AF
Greek_alphaaccent                 1969     0x7B1
Greek_epsilonaccent               1970     0x7B2
```

```
Greek_etaaccent                   1971     0x7B3
Greek_iotaaccent                  1972     0x7B4
Greek_iotadieresis                1973     0x7B5
Greek_iotaaccentdieresis          1974     0x7B6
Greek_omicronaccent               1975     0x7B7
Greek_upsilonaccent               1976     0x7B8
Greek_upsilondieresis             1977     0x7B9
Greek_upsilonaccentdieresis       1978     0x7BA
Greek_omegaaccent                 1979     0x7BB
Greek_ALPHA                       1985     0x7C1
Greek_BETA                        1986     0x7C2
Greek_GAMMA                       1987     0x7C3
Greek_DELTA                       1988     0x7C4
Greek_EPSILON                     1989     0x7C5
Greek_ZETA                        1990     0x7C6
Greek_ETA                         1991     0x7C7
Greek_THETA                       1992     0x7C8
Greek_IOTA                        1993     0x7C9
Greek_KAPPA                       1994     0x7CA
Greek_LAMBDA                      1995     0x7CB
Greek_MU                          1996     0x7CC
Greek_NU                          1997     0x7CD
Greek_XI                          1998     0x7CE
Greek_OMICRON                     1999     0x7CF
Greek_PI                          2000     0x7D0
Greek_RHO                         2001     0x7D1
Greek_SIGMA                       2002     0x7D2
Greek_TAU                         2004     0x7D4
Greek_UPSILON                     2005     0x7D5
Greek_PHI                         2006     0x7D6
Greek_CHI                         2007     0x7D7
Greek_PSI                         2008     0x7D8
Greek_OMEGA                       2009     0x7D9
Greek_alpha                       2017     0x7E1
Greek_beta                        2018     0x7E2
Greek_gamma                       2019     0x7E3
Greek_delta                       2020     0x7E4
Greek_epsilon                     2021     0x7E5
Greek_zeta                        2022     0x7E6
Greek_eta                         2023     0x7E7
Greek_theta                       2024     0x7E8
Greek_iota                        2025     0x7E9
Greek_kappa                       2026     0x7EA
Greek_lambda                      2027     0x7EB
Greek_mu                          2028     0x7EC
Greek_nu                          2029     0x7ED
```

| | | |
|---|---|---|
| Greek_xi | 2030 | 0x7EE |
| Greek_omicron | 2031 | 0x7EF |
| Greek_pi | 2032 | 0x7F0 |
| Greek_rho | 2033 | 0x7F1 |
| Greek_sigma | 2034 | 0x7F2 |
| Greek_finalsmallsigma | 2035 | 0x7F3 |
| Greek_tau | 2036 | 0x7F4 |
| Greek_upsilon | 2037 | 0x7F5 |
| Greek_phi | 2038 | 0x7F6 |
| Greek_chi | 2039 | 0x7F7 |
| Greek_psi | 2040 | 0x7F8 |
| Greek_omega | 2041 | 0x7F9 |

| | | |
|---|---|---|
| leftradical | 2209 | 0x8A1 |
| topleftradical | 2210 | 0x8A2 |
| horizconnector | 2211 | 0x8A3 |
| topintegral | 2212 | 0x8A4 |
| botintegral | 2213 | 0x8A5 |
| vertconnector | 2214 | 0x8A6 |
| topleftsqbracket | 2215 | 0x8A7 |
| botleftsqbracket | 2216 | 0x8A8 |
| toprightsqbracket | 2217 | 0x8A9 |
| botrightsqbracket | 2218 | 0x8AA |
| topleftparens | 2219 | 0x8AB |
| botleftparens | 2220 | 0x8AC |
| toprightparens | 2221 | 0x8AD |
| botrightparens | 2222 | 0x8AE |
| leftmiddlecurlybrace | 2223 | 0x8AF |
| rightmiddlecurlybrace | 2224 | 0x8B0 |
| topleftsummation | 2225 | 0x8B1 |
| botleftsummation | 2226 | 0x8B2 |
| topvertsummationconnector | 2227 | 0x8B3 |
| botvertsummationconnector | 2228 | 0x8B4 |
| toprightsummation | 2229 | 0x8B5 |
| botrightsummation | 2230 | 0x8B6 |
| rightmiddlesummation | 2231 | 0x8B7 |
| lessthanequal | 2236 | 0x8BC |
| notequal | 2237 | 0x8BD |
| greaterthanequal | 2238 | 0x8BE |
| integral | 2239 | 0x8BF |
| therefore | 2240 | 0x8C0 |
| variation | 2241 | 0x8C1 |
| infinity | 2242 | 0x8C2 |
| nabla | 2245 | 0x8C5 |
| approximate | 2248 | 0x8C8 |
| similarequal | 2249 | 0x8C9 |

| | | |
|---|---|---|
| ifonlyif | 2253 | 0x8CD |
| implies | 2254 | 0x8CE |
| identical | 2255 | 0x8CF |
| radical | 2262 | 0x8D6 |
| includedin | 2266 | 0x8DA |
| includes | 2267 | 0x8DB |
| intersection | 2268 | 0x8DC |
| union | 2269 | 0x8DD |
| logicaland | 2270 | 0x8DE |
| logicalor | 2271 | 0x8DF |
| partialderivative | 2287 | 0x8EF |
| function | 2294 | 0x8F6 |
| leftarrow | 2299 | 0x8FB |
| uparrow | 2300 | 0x8FC |
| rightarrow | 2301 | 0x8FD |
| downarrow | 2302 | 0x8FE |
| blank | 2527 | 0x9DF |
| soliddiamond | 2528 | 0x9E0 |
| checkerboard | 2529 | 0x9E1 |
| ht | 2530 | 0x9E2 |
| ff | 2531 | 0x9E3 |
| cr | 2532 | 0x9E4 |
| lf | 2533 | 0x9E5 |
| nl | 2536 | 0x9E8 |
| vt | 2537 | 0x9E9 |
| lowrightcorner | 2538 | 0x9EA |
| uprightcorner | 2539 | 0x9EB |
| upleftcorner | 2540 | 0x9EC |
| lowleftcorner | 2541 | 0x9ED |
| crossinglines | 2542 | 0x9EE |
| horizlinescan1 | 2543 | 0x9EF |
| horizlinescan3 | 2544 | 0x9F0 |
| horizlinescan5 | 2545 | 0x9F1 |
| horizlinescan7 | 2546 | 0x9F2 |
| horizlinescan9 | 2547 | 0x9F3 |
| leftt | 2548 | 0x9F4 |
| rightt | 2549 | 0x9F5 |
| bott | 2550 | 0x9F6 |
| topt | 2551 | 0x9F7 |
| vertbar | 2552 | 0x9F8 |
| emspace | 2721 | 0xAA1 |
| enspace | 2722 | 0xAA2 |
| em3space | 2723 | 0xAA3 |
| em4space | 2724 | 0xAA4 |
| digitspace | 2725 | 0xAA5 |
| punctspace | 2726 | 0xAA6 |

| | | |
|---|---|---|
| thinspace | 2727 | 0xAA7 |
| hairspace | 2728 | 0xAA8 |
| emdash | 2729 | 0xAA9 |
| endash | 2730 | 0xAAA |
| signifblank | 2732 | 0xAAC |
| ellipsis | 2734 | 0xAAE |
| doubbaselinedot | 2735 | 0xAAF |
| onethird | 2736 | 0xAB0 |
| twothirds | 2737 | 0xAB1 |
| onefifth | 2738 | 0xAB2 |
| twofifths | 2739 | 0xAB3 |
| threefifths | 2740 | 0xAB4 |
| fourfifths | 2741 | 0xAB5 |
| onesixth | 2742 | 0xAB6 |
| fivesixths | 2743 | 0xAB7 |
| careof | 2744 | 0xAB8 |
| figdash | 2747 | 0xABB |
| leftanglebracket | 2748 | 0xABC |
| decimalpoint | 2749 | 0xABD |
| rightanglebracket | 2750 | 0xABE |
| marker | 2751 | 0xABF |
| oneeighth | 2755 | 0xAC3 |
| threeeighths | 2756 | 0xAC4 |
| fiveeighths | 2757 | 0xAC5 |
| seveneighths | 2758 | 0xAC6 |
| trademark | 2761 | 0xAC9 |
| signaturemark | 2762 | 0xACA |
| trademarkincircle | 2763 | 0xACB |
| leftopentriangle | 2764 | 0xACC |
| rightopentriangle | 2765 | 0xACD |
| emopencircle | 2766 | 0xACE |
| emopenrectangle | 2767 | 0xACF |
| leftsinglequotemark | 2768 | 0xAD0 |
| rightsinglequotemark | 2769 | 0xAD1 |
| leftdoublequotemark | 2770 | 0xAD2 |
| rightdoublequotemark | 2771 | 0xAD3 |
| prescription | 2772 | 0xAD4 |
| permille | 2773 | 0xAD5 |
| minutes | 2774 | 0xAD6 |
| seconds | 2775 | 0xAD7 |
| latincross | 2777 | 0xAD9 |
| hexagram | 2778 | 0xADA |
| filledrectbullet | 2779 | 0xADB |
| filledlefttribullet | 2780 | 0xADC |
| filledrighttribullet | 2781 | 0xADD |
| emfilledcircle | 2782 | 0xADE |

| | | |
|---|---|---|
| emfilledrect | 2783 | 0xADF |
| enopencircbullet | 2784 | 0xAE0 |
| enopensquarebullet | 2785 | 0xAE1 |
| openrectbullet | 2786 | 0xAE2 |
| opentribulletup | 2787 | 0xAE3 |
| opentribulletdown | 2788 | 0xAE4 |
| openstar | 2789 | 0xAE5 |
| enfilledcircbullet | 2790 | 0xAE6 |
| enfilledsqbullet | 2791 | 0xAE7 |
| filledtribulletup | 2792 | 0xAE8 |
| filledtribulletdown | 2793 | 0xAE9 |
| leftpointer | 2794 | 0xAEA |
| rightpointer | 2795 | 0xAEB |
| club | 2796 | 0xAEC |
| diamond | 2797 | 0xAED |
| heart | 2798 | 0xAEE |
| maltesecross | 2800 | 0xAF0 |
| dagger | 2801 | 0xAF1 |
| doubledagger | 2802 | 0xAF2 |
| checkmark | 2803 | 0xAF3 |
| ballotcross | 2804 | 0xAF4 |
| musicalsharp | 2805 | 0xAF5 |
| musicalflat | 2806 | 0xAF6 |
| malesymbol | 2807 | 0xAF7 |
| femalesymbol | 2808 | 0xAF8 |
| telephone | 2809 | 0xAF9 |
| telephonerecorder | 2810 | 0xAFA |
| phonographcopyright | 2811 | 0xAFB |
| caret | 2812 | 0xAFC |
| singlelowquotemark | 2813 | 0xAFD |
| doublelowquotemark | 2814 | 0xAFE |
| cursor | 2815 | 0xAFF |
| leftcaret | 2979 | 0xBA3 |
| rightcaret | 2982 | 0xBA6 |
| downcaret | 2984 | 0xBA8 |
| upcaret | 2985 | 0xBA9 |
| overbar | 3008 | 0xBC0 |
| downtack | 3010 | 0xBC2 |
| upshoe | 3011 | 0xBC3 |
| downstile | 3012 | 0xBC4 |
| underbar | 3014 | 0xBC6 |
| jot | 3018 | 0xBCA |
| quad | 3020 | 0xBCC |
| uptack | 3022 | 0xBCE |
| circle | 3023 | 0xBCF |
| upstile | 3027 | 0xBD3 |

```
downshoe                          3030     0xBD6
rightshoe                         3032     0xBD8
leftshoe                          3034     0xBDA
lefttack                          3036     0xBDC
righttack                         3068     0xBFC
hebrew_doublelowline              3295     0xCDF
hebrew_aleph                      3296     0xCE0
hebrew_bet                        3297     0xCE1
hebrew_gimel                      3298     0xCE2
hebrew_dalet                      3299     0xCE3
hebrew_he                         3300     0xCE4
hebrew_waw                        3301     0xCE5
hebrew_zain                       3302     0xCE6
hebrew_chet                       3303     0xCE7
hebrew_tet                        3304     0xCE8
hebrew_yod                        3305     0xCE9
hebrew_finalkaph                  3306     0xCEA
hebrew_kaph                       3307     0xCEB
hebrew_lamed                      3308     0xCEC
hebrew_finalmem                   3309     0xCED
hebrew_mem                        3310     0xCEE
hebrew_finalnun                   3311     0xCEF
hebrew_nun                        3312     0xCF0
hebrew_samech                     3313     0xCF1
hebrew_ayin                       3314     0xCF2
hebrew_finalpe                    3315     0xCF3
hebrew_pe                         3316     0xCF4
hebrew_finalzade                  3317     0xCF5
hebrew_zade                       3318     0xCF6
hebrew_qoph                       3319     0xCF7
hebrew_resh                       3320     0xCF8
hebrew_shin                       3321     0xCF9
hebrew_taw                        3322     0xCFA
```

```
Thai_kokai                        3489     0xDA1
Thai_khokhai                      3490     0xDA2
Thai_khokhuat                     3491     0xDA3
Thai_khokhwai                     3492     0xDA4
Thai_khokhon                      3493     0xDA5
Thai_khorakhang                   3494     0xDA6
Thai_ngongu                       3495     0xDA7
Thai_chochan                      3496     0xDA8
Thai_choching                     3497     0xDA9
Thai_chochang                     3498     0xDAA
Thai_soso                         3499     0xDAB
Thai_chochoe                      3500     0xDAC
```

```
Thai_yoying                       3501     0xDAD
Thai_dochada                      3502     0xDAE
Thai_topatak                      3503     0xDAF
Thai_thothan                      3504     0xDB0
Thai_thonangmontho                3505     0xDB1
Thai_thophuthao                   3506     0xDB2
Thai_nonen                        3507     0xDB3
Thai_dodek                        3508     0xDB4
Thai_totao                        3509     0xDB5
Thai_thothung                     3510     0xDB6
Thai_thothahan                    3511     0xDB7
Thai_thothong                     3512     0xDB8
Thai_nonu                         3513     0xDB9
Thai_bobaimai                     3514     0xDBA
Thai_popla                        3515     0xDBB
Thai_phophung                     3516     0xDBC
Thai_fofa                         3517     0xDBD
Thai_phophan                      3518     0xDBE
Thai_fofan                        3519     0xDBF
Thai_phosamphao                   3520     0xDC0
Thai_moma                         3521     0xDC1
Thai_yoyak                        3522     0xDC2
Thai_rorua                        3523     0xDC3
Thai_ru                           3524     0xDC4
Thai_loling                       3525     0xDC5
Thai_lu                           3526     0xDC6
Thai_wowaen                       3527     0xDC7
Thai_sosala                       3528     0xDC8
Thai_sorusi                       3529     0xDC9
Thai_sosua                        3530     0xDCA
Thai_hohip                        3531     0xDCB
Thai_lochula                      3532     0xDCC
Thai_oang                         3533     0xDCD
Thai_honokhuk                     3534     0xDCE
Thai_paiyannoi                    3535     0xDCF
Thai_saraa                        3536     0xDD0
Thai_maihanakat                   3537     0xDD1
Thai_saraaa                       3538     0xDD2
Thai_saraam                       3539     0xDD3
Thai_sarai                        3540     0xDD4
Thai_saraii                       3541     0xDD5
Thai_saraue                       3542     0xDD6
Thai_sarauee                      3543     0xDD7
Thai_sarau                        3544     0xDD8
Thai_sarauu                       3545     0xDD9
Thai_phinthu                      3546     0xDDA
```

```
Thai_maihanakat_maitho            3550     0xDDE
Thai_baht                         3551     0xDDF
Thai_sarae                        3552     0xDE0
Thai_saraae                       3553     0xDE1
Thai_sarao                        3554     0xDE2
Thai_saraaimaimuan                3555     0xDE3
Thai_saraaimaimalai               3556     0xDE4
Thai_lakkhangyao                  3557     0xDE5
Thai_maiyamok                     3558     0xDE6
Thai_maitaikhu                    3559     0xDE7
Thai_maiek                        3560     0xDE8
Thai_maitho                       3561     0xDE9
Thai_maitri                       3562     0xDEA
Thai_maichattawa                  3563     0xDEB
Thai_thanthakhat                  3564     0xDEC
Thai_nikhahit                     3565     0xDED
Thai_leksun                       3568     0xDF0
Thai_leknung                      3569     0xDF1
Thai_leksong                      3570     0xDF2
Thai_leksam                       3571     0xDF3
Thai_leksi                        3572     0xDF4
Thai_lekha                        3573     0xDF5
Thai_lekhok                       3574     0xDF6
Thai_lekchet                      3575     0xDF7
Thai_lekpaet                      3576     0xDF8
Thai_lekkao                       3577     0xDF9
Hangul_Kiyeog                     3745     0xEA1
Hangul_SsangKiyeog                3746     0xEA2
Hangul_KiyeogSios                 3747     0xEA3
Hangul_Nieun                      3748     0xEA4
Hangul_NieunJieuj                 3749     0xEA5
Hangul_NieunHieuh                 3750     0xEA6
Hangul_Dikeud                     3751     0xEA7
Hangul_SsangDikeud                3752     0xEA8
Hangul_Rieul                      3753     0xEA9
Hangul_RieulKiyeog                3754     0xEAA
Hangul_RieulMieum                 3755     0xEAB
Hangul_RieulPieub                 3756     0xEAC
Hangul_RieulSios                  3757     0xEAD
Hangul_RieulTieut                 3758     0xEAE
Hangul_RieulPhieuf                3759     0xEAF
Hangul_RieulHieuh                 3760     0xEB0
Hangul_Mieum                      3761     0xEB1
Hangul_Pieub                      3762     0xEB2
Hangul_SsangPieub                 3763     0xEB3
Hangul_PieubSios                  3764     0xEB4
```

| | | |
|---|---|---|
| Hangul_Sios | 3765 | 0xEB5 |
| Hangul_SsangSios | 3766 | 0xEB6 |
| Hangul_Ieung | 3767 | 0xEB7 |
| Hangul_Jieuj | 3768 | 0xEB8 |
| Hangul_SsangJieuj | 3769 | 0xEB9 |
| Hangul_Cieuc | 3770 | 0xEBA |
| Hangul_Khieuq | 3771 | 0xEBB |
| Hangul_Tieut | 3772 | 0xEBC |
| Hangul_Phieuf | 3773 | 0xEBD |
| Hangul_Hieuh | 3774 | 0xEBE |
| Hangul_A | 3775 | 0xEBF |
| Hangul_AE | 3776 | 0xEC0 |
| Hangul_YA | 3777 | 0xEC1 |
| Hangul_YAE | 3778 | 0xEC2 |
| Hangul_EO | 3779 | 0xEC3 |
| Hangul_E | 3780 | 0xEC4 |
| Hangul_YEO | 3781 | 0xEC5 |
| Hangul_YE | 3782 | 0xEC6 |
| Hangul_O | 3783 | 0xEC7 |
| Hangul_WA | 3784 | 0xEC8 |
| Hangul_WAE | 3785 | 0xEC9 |
| Hangul_OE | 3786 | 0xECA |
| Hangul_YO | 3787 | 0xECB |
| Hangul_U | 3788 | 0xECC |
| Hangul_WEO | 3789 | 0xECD |
| Hangul_WE | 3790 | 0xECE |
| Hangul_WI | 3791 | 0xECF |
| Hangul_YU | 3792 | 0xED0 |
| Hangul_EU | 3793 | 0xED1 |
| Hangul_YI | 3794 | 0xED2 |
| Hangul_I | 3795 | 0xED3 |
| Hangul_J_Kiyeog | 3796 | 0xED4 |
| Hangul_J_SsangKiyeog | 3797 | 0xED5 |
| Hangul_J_KiyeogSios | 3798 | 0xED6 |
| Hangul_J_Nieun | 3799 | 0xED7 |
| Hangul_J_NieunJieuj | 3800 | 0xED8 |
| Hangul_J_NieunHieuh | 3801 | 0xED9 |
| Hangul_J_Dikeud | 3802 | 0xEDA |
| Hangul_J_Rieul | 3803 | 0xEDB |
| Hangul_J_RieulKiyeog | 3804 | 0xEDC |
| Hangul_J_RieulMieum | 3805 | 0xEDD |
| Hangul_J_RieulPieub | 3806 | 0xEDE |
| Hangul_J_RieulSios | 3807 | 0xEDF |
| Hangul_J_RieulTieut | 3808 | 0xEE0 |
| Hangul_J_RieulPhieuf | 3809 | 0xEE1 |
| Hangul_J_RieulHieuh | 3810 | 0xEE2 |

| | | |
|---|---|---|
| Hangul_J_Mieum | 3811 | 0xEE3 |
| Hangul_J_Pieub | 3812 | 0xEE4 |
| Hangul_J_PieubSios | 3813 | 0xEE5 |
| Hangul_J_Sios | 3814 | 0xEE6 |
| Hangul_J_SsangSios | 3815 | 0xEE7 |
| Hangul_J_Ieung | 3816 | 0xEE8 |
| Hangul_J_Jieuj | 3817 | 0xEE9 |
| Hangul_J_Cieuc | 3818 | 0xEEA |
| Hangul_J_Khieuq | 3819 | 0xEEB |
| Hangul_J_Tieut | 3820 | 0xEEC |
| Hangul_J_Phieuf | 3821 | 0xEED |
| Hangul_J_Hieuh | 3822 | 0xEEE |
| Hangul_RieulYeorinHieuh | 3823 | 0xEEF |
| Hangul_SunkyeongeumMieum | 3824 | 0xEF0 |
| Hangul_SunkyeongeumPieub | 3825 | 0xEF1 |
| Hangul_PanSios | 3826 | 0xEF2 |
| Hangul_KkogjiDalrinIeung | 3827 | 0xEF3 |
| Hangul_SunkyeongeumPhieuf | 3828 | 0xEF4 |
| Hangul_YeorinHieuh | 3829 | 0xEF5 |
| Hangul_AraeA | 3830 | 0xEF6 |
| Hangul_AraeAE | 3831 | 0xEF7 |
| Hangul_J_PanSios | 3832 | 0xEF8 |
| Hangul_J_KkogjiDalrinIeung | 3833 | 0xEF9 |
| Hangul_J_YeorinHieuh | 3834 | 0xEFA |
| Korean_Won | 3839 | 0xEFF |
| OE | 5052 | 0x13BC |
| oe | 5053 | 0x13BD |
| Ydiaeresis | 5054 | 0x13BE |
| € (EuroSign) | 8364 | 0x20AC |

| | | |
|---|---|---|
| ISO_Lock | 65025 | 0xFE01 |
| ISO_Level2_Latch | 65026 | 0xFE02 |
| ISO_Level3_Shift | 65027 | 0xFE03 |
| ISO_Level3_Latch | 65028 | 0xFE04 |
| ISO_Level3_Lock | 65029 | 0xFE05 |
| ISO_Group_Latch | 65030 | 0xFE06 |
| ISO_Group_Lock | 65031 | 0xFE07 |
| ISO_Next_Group | 65032 | 0xFE08 |
| ISO_Next_Group_Lock | 65033 | 0xFE09 |
| ISO_Prev_Group | 65034 | 0xFE0A |
| ISO_Prev_Group_Lock | 65035 | 0xFE0B |
| ISO_First_Group | 65036 | 0xFE0C |
| ISO_First_Group_Lock | 65037 | 0xFE0D |
| ISO_Last_Group | 65038 | 0xFE0E |
| ISO_Last_Group_Lock | 65039 | 0xFE0F |
| ISO_Level5_Shift | 65041 | 0xFE11 |

| | | |
|---|---|---|
| ISO_Level5_Latch | 65042 | 0xFE12 |
| ISO_Level5_Lock | 65043 | 0xFE13 |
| ISO_Left_Tab | 65056 | 0xFE20 |
| ISO_Move_Line_Up | 65057 | 0xFE21 |
| ISO_Move_Line_Down | 65058 | 0xFE22 |
| ISO_Partial_Line_Up | 65059 | 0xFE23 |
| ISO_Partial_Line_Down | 65060 | 0xFE24 |
| ISO_Partial_Space_Left | 65061 | 0xFE25 |
| ISO_Partial_Space_Right | 65062 | 0xFE26 |
| ISO_Set_Margin_Left | 65063 | 0xFE27 |
| ISO_Set_Margin_Right | 65064 | 0xFE28 |
| ISO_Release_Margin_Left | 65065 | 0xFE29 |
| ISO_Release_Margin_Right | 65066 | 0xFE2A |
| ISO_Release_Both_Margins | 65067 | 0xFE2B |
| ISO_Fast_Cursor_Left | 65068 | 0xFE2C |
| ISO_Fast_Cursor_Right | 65069 | 0xFE2D |
| ISO_Fast_Cursor_Up | 65070 | 0xFE2E |
| ISO_Fast_Cursor_Down | 65071 | 0xFE2F |
| ISO_Continuous_Underline | 65072 | 0xFE30 |
| ISO_Discontinuous_Underline | 65073 | 0xFE31 |
| ISO_Emphasize | 65074 | 0xFE32 |
| ISO_Center_Object | 65075 | 0xFE33 |
| ISO_Enter | 65076 | 0xFE34 |
| dead_grave | 65104 | 0xFE50 |
| dead_acute | 65105 | 0xFE51 |
| dead_circumflex | 65106 | 0xFE52 |
| dead_tilde | 65107 | 0xFE53 |
| dead_macron | 65108 | 0xFE54 |
| dead_breve | 65109 | 0xFE55 |
| dead_abovedot | 65110 | 0xFE56 |
| dead_diaeresis | 65111 | 0xFE57 |
| dead_abovering | 65112 | 0xFE58 |
| dead_doubleacute | 65113 | 0xFE59 |
| dead_caron | 65114 | 0xFE5A |
| dead_cedilla | 65115 | 0xFE5B |
| dead_ogonek | 65116 | 0xFE5C |
| dead_iota | 65117 | 0xFE5D |
| dead_voiced_sound | 65118 | 0xFE5E |
| dead_semivoiced_sound | 65119 | 0xFE5F |
| dead_belowdot | 65120 | 0xFE60 |
| dead_hook | 65121 | 0xFE61 |
| dead_horn | 65122 | 0xFE62 |
| dead_stroke | 65123 | 0xFE63 |
| dead_abovecomma | 65124 | 0xFE64 |
| dead_abovereversedcomma | 65125 | 0xFE65 |
| dead_doublegrave | 65126 | 0xFE66 |

```
dead_belowring                   65127     0xFE67
dead_belowmacron                 65128     0xFE68
dead_belowcircumflex             65129     0xFE69
dead_belowtilde                  65130     0xFE6A
dead_belowbreve                  65131     0xFE6B
dead_belowdiaeresis              65132     0xFE6C
dead_invertedbreve               65133     0xFE6D
dead_belowcomma                  65134     0xFE6E
dead_currency                    65135     0xFE6F
AccessX_Enable                   65136     0xFE70
AccessX_Feedback_Enable          65137     0xFE71
RepeatKeys_Enable                65138     0xFE72
SlowKeys_Enable                  65139     0xFE73
BounceKeys_Enable                65140     0xFE74
StickyKeys_Enable                65141     0xFE75
MouseKeys_Enable                 65142     0xFE76
MouseKeys_Accel_Enable           65143     0xFE77
Overlay1_Enable                  65144     0xFE78
Overlay2_Enable                  65145     0xFE79
AudibleBell_Enable               65146     0xFE7A
dead_a                           65152     0xFE80
dead_A                           65153     0xFE81
dead_e                           65154     0xFE82
dead_E                           65155     0xFE83
dead_i                           65156     0xFE84
dead_I                           65157     0xFE85
dead_o                           65158     0xFE86
dead_O                           65159     0xFE87
dead_u                           65160     0xFE88
dead_U                           65161     0xFE89
dead_schwa                       65162     0xFE8A
dead_SCHWA                       65163     0xFE8B
dead_greek                       65164     0xFE8C
dead_hamza                       65165     0xFE8D
dead_lowline                     65168     0xFE90
dead_aboveverticalline           65169     0xFE91
dead_belowverticalline           65170     0xFE92
dead_longsolidusoverlay          65171     0xFE93
ch                               65184     0xFEA0
Ch                               65185     0xFEA1
CH                               65186     0xFEA2
c_h                              65187     0xFEA3
C_h                              65188     0xFEA4
C_H                              65189     0xFEA5
First_Virtual_Screen             65232     0xFED0
Prev_Virtual_Screen              65233     0xFED1
```

```
Next_Virtual_Screen              65234     0xFED2
Last_Virtual_Screen              65236     0xFED4
Terminate_Server                 65237     0xFED5
Pointer_Left                     65248     0xFEE0
Pointer_Right                    65249     0xFEE1
Pointer_Up                       65250     0xFEE2
Pointer_Down                     65251     0xFEE3
Pointer_UpLeft                   65252     0xFEE4
Pointer_UpRight                  65253     0xFEE5
Pointer_DownLeft                 65254     0xFEE6
Pointer_DownRight                65255     0xFEE7
Pointer_Button_Dflt              65256     0xFEE8
Pointer_Button1                  65257     0xFEE9
Pointer_Button2                  65258     0xFEEA
Pointer_Button3                  65259     0xFEEB
Pointer_Button4                  65260     0xFEEC
Pointer_Button5                  65261     0xFEED
Pointer_DblClick_Dflt            65262     0xFEEE
Pointer_DblClick1                65263     0xFEEF
Pointer_DblClick2                65264     0xFEF0
Pointer_DblClick3                65265     0xFEF1
Pointer_DblClick4                65266     0xFEF2
Pointer_DblClick5                65267     0xFEF3
Pointer_Drag_Dflt                65268     0xFEF4
Pointer_Drag1                    65269     0xFEF5
Pointer_Drag2                    65270     0xFEF6
Pointer_Drag3                    65271     0xFEF7
Pointer_Drag4                    65272     0xFEF8
Pointer_EnableKeys               65273     0xFEF9
Pointer_Accelerate               65274     0xFEFA
Pointer_DfltBtnNext              65275     0xFEFB
Pointer_DfltBtnPrev              65276     0xFEFC
Pointer_Drag5                    65277     0xFEFD
```

```
BackSpace                        65288     0xFF08
Tab                              65289     0xFF09
Linefeed                         65290     0xFF0A
Clear                            65291     0xFF0B
Return                           65293     0xFF0D
Pause                            65299     0xFF13
Scroll_Lock                      65300     0xFF14
Sys_Req                          65301     0xFF15
Escape                           65307     0xFF1B
Multi_key                        65312     0xFF20
Kanji                            65313     0xFF21
Muhenkan                         65314     0xFF22
```

| | | |
|---|---|---|
| Henkan | 65315 | 0xFF23 |
| Romaji | 65316 | 0xFF24 |
| Hiragana | 65317 | 0xFF25 |
| Katakana | 65318 | 0xFF26 |
| Hiragana_Katakana | 65319 | 0xFF27 |
| Zenkaku | 65320 | 0xFF28 |
| Hankaku | 65321 | 0xFF29 |
| Zenkaku_Hankaku | 65322 | 0xFF2A |
| Touroku | 65323 | 0xFF2B |
| Massyo | 65324 | 0xFF2C |
| Kana_Lock | 65325 | 0xFF2D |
| Kana_Shift | 65326 | 0xFF2E |
| Eisu_Shift | 65327 | 0xFF2F |
| Eisu_toggle | 65328 | 0xFF30 |
| Hangul | 65329 | 0xFF31 |
| Hangul_Start | 65330 | 0xFF32 |
| Hangul_End | 65331 | 0xFF33 |
| Hangul_Hanja | 65332 | 0xFF34 |
| Hangul_Jamo | 65333 | 0xFF35 |
| Hangul_Romaja | 65334 | 0xFF36 |
| Codeinput | 65335 | 0xFF37 |
| Hangul_Jeonja | 65336 | 0xFF38 |
| Hangul_Banja | 65337 | 0xFF39 |
| Hangul_PreHanja | 65338 | 0xFF3A |
| Hangul_PostHanja | 65339 | 0xFF3B |
| SingleCandidate | 65340 | 0xFF3C |
| MultipleCandidate | 65341 | 0xFF3D |
| PreviousCandidate | 65342 | 0xFF3E |
| Hangul_Special | 65343 | 0xFF3F |
| Home | 65360 | 0xFF50 |
| Left | 65361 | 0xFF51 |
| Up | 65362 | 0xFF52 |
| Right | 65363 | 0xFF53 |
| Down | 65364 | 0xFF54 |
| Prior | 65365 | 0xFF55 |
| Next | 65366 | 0xFF56 |
| End | 65367 | 0xFF57 |
| Begin | 65368 | 0xFF58 |
| Select | 65376 | 0xFF60 |
| Print | 65377 | 0xFF61 |
| Execute | 65378 | 0xFF62 |
| Insert | 65379 | 0xFF63 |
| Undo | 65381 | 0xFF65 |
| Redo | 65382 | 0xFF66 |
| Menu (App) | 65383 | 0xFF67 |
| Find | 65384 | 0xFF68 |

```
Cancel                           65385     0xFF69
Help                             65386     0xFF6A
Break                            65387     0xFF6B
Mode_switch                      65406     0xFF7E
Num_Lock                         65407     0xFF7F
KP_Space                         65408     0xFF80
KP_Tab                           65417     0xFF89
KP_Enter                         65421     0xFF8D
KP_F1                            65425     0xFF91
KP_F2                            65426     0xFF92
KP_F3                            65427     0xFF93
KP_F4                            65428     0xFF94
KP_Home                          65429     0xFF95
KP_Left                          65430     0xFF96
KP_Up                            65431     0xFF97
KP_Right                         65432     0xFF98
KP_Down                          65433     0xFF99
KP_Prior                         65434     0xFF9A
KP_Next                          65435     0xFF9B
KP_End                           65436     0xFF9C
KP_Begin                         65437     0xFF9D
KP_Insert                        65438     0xFF9E
KP_Delete                        65439     0xFF9F
KP_Multiply                      65450     0xFFAA
KP_Add                           65451     0xFFAB
KP_Separator                     65452     0xFFAC
KP_Subtract                      65453     0xFFAD
KP_Decimal                       65454     0xFFAE
KP_Divide                        65455     0xFFAF
KP_0                             65456     0xFFB0
KP_1                             65457     0xFFB1
KP_2                             65458     0xFFB2
KP_3                             65459     0xFFB3
KP_4                             65460     0xFFB4
KP_5                             65461     0xFFB5
KP_6                             65462     0xFFB6
KP_7                             65463     0xFFB7
KP_8                             65464     0xFFB8
KP_9                             65465     0xFFB9
KP_Equal                         65469     0xFFBD
F1                               65470     0xFFBE
F2                               65471     0xFFBF
F3                               65472     0xFFC0
F4                               65473     0xFFC1
F5                               65474     0xFFC2
F6                               65475     0xFFC3
```

| | | |
|---|---|---|
| F7 | 65476 | 0xFFC4 |
| F8 | 65477 | 0xFFC5 |
| F9 | 65478 | 0xFFC6 |
| F10 | 65479 | 0xFFC7 |
| F11 | 65480 | 0xFFC8 |
| F12 | 65481 | 0xFFC9 |
| F13 | 65482 | 0xFFCA |
| F14 | 65483 | 0xFFCB |
| F15 | 65484 | 0xFFCC |
| F16 | 65485 | 0xFFCD |
| F17 | 65486 | 0xFFCE |
| F18 | 65487 | 0xFFCF |
| F19 | 65488 | 0xFFD0 |
| F20 | 65489 | 0xFFD1 |
| F21 | 65490 | 0xFFD2 |
| F22 | 65491 | 0xFFD3 |
| F23 | 65492 | 0xFFD4 |
| F24 | 65493 | 0xFFD5 |
| F25 | 65494 | 0xFFD6 |
| F26 | 65495 | 0xFFD7 |
| F27 | 65496 | 0xFFD8 |
| F28 | 65497 | 0xFFD9 |
| F29 | 65498 | 0xFFDA |
| F30 | 65499 | 0xFFDB |
| F31 | 65500 | 0xFFDC |
| F32 | 65501 | 0xFFDD |
| F33 | 65502 | 0xFFDE |
| F34 | 65503 | 0xFFDF |
| F35 | 65504 | 0xFFE0 |
| Shift_L | 65505 | 0xFFE1 |
| Shift_R | 65506 | 0xFFE2 |
| Control_L | 65507 | 0xFFE3 |
| Control_R | 65508 | 0xFFE4 |
| Caps_Lock | 65509 | 0xFFE5 |
| Shift_Lock | 65510 | 0xFFE6 |
| Meta_L | 65511 | 0xFFE7 |
| Meta_R | 65512 | 0xFFE8 |
| Alt_L | 65513 | 0xFFE9 |
| Alt_R | 65514 | 0xFFEA |
| Super_L (Win_L) | 65515 | 0xFFEB |
| Super_R (Win_R) | 65516 | 0xFFEC |
| Hyper_L | 65517 | 0xFFED |
| Hyper_R | 65518 | 0xFFEE |
| braille_dot_1 | 65521 | 0xFFF1 |
| braille_dot_2 | 65522 | 0xFFF2 |
| braille_dot_3 | 65523 | 0xFFF3 |

```
braille_dot_4                    65524     0xFFF4
braille_dot_5                    65525     0xFFF5
braille_dot_6                    65526     0xFFF6
braille_dot_7                    65527     0xFFF7
braille_dot_8                    65528     0xFFF8
braille_dot_9                    65529     0xFFF9
braille_dot_10                   65530     0xFFFA
Delete                           65535     0xFFFF
```

```
SunFA_Grave                  268828416     0x1005FF00
SunFA_Circum                 268828417     0x1005FF01
SunFA_Tilde                  268828418     0x1005FF02
SunFA_Acute                  268828419     0x1005FF03
SunFA_Diaeresis              268828420     0x1005FF04
SunFA_Cedilla                268828421     0x1005FF05
SunF36                       268828432     0x1005FF10
SunF37                       268828433     0x1005FF11
SunSys_Req                   268828512     0x1005FF60
SunProps                     268828528     0x1005FF70
SunFront                     268828529     0x1005FF71
SunCopy                      268828530     0x1005FF72
SunOpen                      268828531     0x1005FF73
SunPaste                     268828532     0x1005FF74
SunCut                       268828533     0x1005FF75
SunPowerSwitch               268828534     0x1005FF76
SunAudioLowerVolume          268828535     0x1005FF77
SunAudioMute                 268828536     0x1005FF78
SunAudioRaiseVolume          268828537     0x1005FF79
SunVideoDegauss              268828538     0x1005FF7A
SunVideoLowerBrightness      268828539     0x1005FF7B
SunVideoRaiseBrightness      268828540     0x1005FF7C
SunPowerSwitchShift          268828541     0x1005FF7D
XF86BrightnessAuto           268964084     0x100810F4
XF86DisplayOff               268964085     0x100810F5
XF86Info                     268964198     0x10081166
XF86AspectRatio              268964215     0x10081177
XF86DVD                      268964229     0x10081185
XF86Audio                    268964232     0x10081188
XF86ChannelUp                268964242     0x10081192
XF86ChannelDown              268964243     0x10081193
XF86Break                    268964251     0x1008119B
XF86VideoPhone               268964256     0x100811A0
XF86ZoomReset                268964260     0x100811A4
XF86Editor                   268964262     0x100811A6
XF86GraphicsEditor           268964264     0x100811A8
XF86Presentation             268964265     0x100811A9
```

```
XF86Database                 268964266     0x100811AA
XF86Voicemail                268964268     0x100811AC
XF86Addressbook              268964269     0x100811AD
XF86DisplayToggle            268964271     0x100811AF
XF86SpellCheck               268964272     0x100811B0
XF86ContextMenu              268964278     0x100811B6
XF86MediaRepeat              268964279     0x100811B7
XF8610ChannelsUp             268964280     0x100811B8
XF8610ChannelsDown           268964281     0x100811B9
XF86Images                   268964282     0x100811BA
XF86NotificationCenter       268964284     0x100811BC
XF86PickupPhone              268964285     0x100811BD
XF86HangupPhone              268964286     0x100811BE
XF86Fn                       268964304     0x100811D0
XF86Fn_Esc                   268964305     0x100811D1
XF86FnRightShift             268964325     0x100811E5
XF86Numeric0                 268964352     0x10081200
XF86Numeric1                 268964353     0x10081201
XF86Numeric2                 268964354     0x10081202
XF86Numeric3                 268964355     0x10081203
XF86Numeric4                 268964356     0x10081204
XF86Numeric5                 268964357     0x10081205
XF86Numeric6                 268964358     0x10081206
XF86Numeric7                 268964359     0x10081207
XF86Numeric8                 268964360     0x10081208
XF86Numeric9                 268964361     0x10081209
XF86NumericStar              268964362     0x1008120A
XF86NumericPound             268964363     0x1008120B
XF86NumericA                 268964364     0x1008120C
XF86NumericB                 268964365     0x1008120D
XF86NumericC                 268964366     0x1008120E
XF86NumericD                 268964367     0x1008120F
XF86CameraFocus              268964368     0x10081210
XF86WPSButton                268964369     0x10081211
XF86CameraZoomIn             268964373     0x10081215
XF86CameraZoomOut            268964374     0x10081216
XF86CameraUp                 268964375     0x10081217
XF86CameraDown               268964376     0x10081218
XF86CameraLeft               268964377     0x10081219
XF86CameraRight              268964378     0x1008121A
XF86AttendantOn              268964379     0x1008121B
XF86AttendantOff             268964380     0x1008121C
XF86AttendantToggle          268964381     0x1008121D
XF86LightsToggle             268964382     0x1008121E
XF86ALSToggle                268964400     0x10081230
XF86Buttonconfig             268964416     0x10081240
```

```
XF86Taskmanager              268964417     0x10081241
XF86Journal                  268964418     0x10081242
XF86ControlPanel             268964419     0x10081243
XF86AppSelect                268964420     0x10081244
XF86Screensaver              268964421     0x10081245
XF86VoiceCommand             268964422     0x10081246
XF86Assistant                268964423     0x10081247
XF86EmojiPicker              268964425     0x10081249
XF86Dictate                  268964426     0x1008124A
XF86BrightnessMin            268964432     0x10081250
XF86BrightnessMax            268964433     0x10081251
XF86KbdInputAssistPrev       268964448     0x10081260
XF86KbdInputAssistNext       268964449     0x10081261
XF86KbdInputAssistPrevgroup  268964450     0x10081262
XF86KbdInputAssistNextgroup  268964451     0x10081263
XF86KbdInputAssistAccept     268964452     0x10081264
XF86KbdInputAssistCancel     268964453     0x10081265
XF86RightUp                  268964454     0x10081266
XF86RightDown                268964455     0x10081267
XF86LeftUp                   268964456     0x10081268
XF86LeftDown                 268964457     0x10081269
XF86RootMenu                 268964458     0x1008126A
XF86MediaTopMenu             268964459     0x1008126B
XF86Numeric11                268964460     0x1008126C
XF86Numeric12                268964461     0x1008126D
XF86AudioDesc                268964462     0x1008126E
XF863DMode                   268964463     0x1008126F
XF86NextFavorite             268964464     0x10081270
XF86StopRecord               268964465     0x10081271
XF86PauseRecord              268964466     0x10081272
XF86VOD                      268964467     0x10081273
XF86Unmute                   268964468     0x10081274
XF86FastReverse              268964469     0x10081275
XF86SlowReverse              268964470     0x10081276
XF86Data                     268964471     0x10081277
XF86OnScreenKeyboard         268964472     0x10081278
XF86PrivacyScreenToggle      268964473     0x10081279
XF86SelectiveScreenshot      268964474     0x1008127A
XF86Macro1                   268964496     0x10081290
XF86Macro2                   268964497     0x10081291
XF86Macro3                   268964498     0x10081292
XF86Macro4                   268964499     0x10081293
XF86Macro5                   268964500     0x10081294
XF86Macro6                   268964501     0x10081295
XF86Macro7                   268964502     0x10081296
XF86Macro8                   268964503     0x10081297
```

| | | |
|---|---|---|
| XF86Macro9 | 268964504 | 0x10081298 |
| XF86Macro10 | 268964505 | 0x10081299 |
| XF86Macro11 | 268964506 | 0x1008129A |
| XF86Macro12 | 268964507 | 0x1008129B |
| XF86Macro13 | 268964508 | 0x1008129C |
| XF86Macro14 | 268964509 | 0x1008129D |
| XF86Macro15 | 268964510 | 0x1008129E |
| XF86Macro16 | 268964511 | 0x1008129F |
| XF86Macro17 | 268964512 | 0x100812A0 |
| XF86Macro18 | 268964513 | 0x100812A1 |
| XF86Macro19 | 268964514 | 0x100812A2 |
| XF86Macro20 | 268964515 | 0x100812A3 |
| XF86Macro21 | 268964516 | 0x100812A4 |
| XF86Macro22 | 268964517 | 0x100812A5 |
| XF86Macro23 | 268964518 | 0x100812A6 |
| XF86Macro24 | 268964519 | 0x100812A7 |
| XF86Macro25 | 268964520 | 0x100812A8 |
| XF86Macro26 | 268964521 | 0x100812A9 |
| XF86Macro27 | 268964522 | 0x100812AA |
| XF86Macro28 | 268964523 | 0x100812AB |
| XF86Macro29 | 268964524 | 0x100812AC |
| XF86Macro30 | 268964525 | 0x100812AD |
| XF86MacroRecordStart | 268964528 | 0x100812B0 |
| XF86MacroRecordStop | 268964529 | 0x100812B1 |
| XF86MacroPresetCycle | 268964530 | 0x100812B2 |
| XF86MacroPreset1 | 268964531 | 0x100812B3 |
| XF86MacroPreset2 | 268964532 | 0x100812B4 |
| XF86MacroPreset3 | 268964533 | 0x100812B5 |
| XF86KbdLcdMenu1 | 268964536 | 0x100812B8 |
| XF86KbdLcdMenu2 | 268964537 | 0x100812B9 |
| XF86KbdLcdMenu3 | 268964538 | 0x100812BA |
| XF86KbdLcdMenu4 | 268964539 | 0x100812BB |
| XF86KbdLcdMenu5 | 268964540 | 0x100812BC |
| XF86Switch_VT_1 | 269024769 | 0x1008FE01 |
| XF86Switch_VT_2 | 269024770 | 0x1008FE02 |
| XF86Switch_VT_3 | 269024771 | 0x1008FE03 |
| XF86Switch_VT_4 | 269024772 | 0x1008FE04 |
| XF86Switch_VT_5 | 269024773 | 0x1008FE05 |
| XF86Switch_VT_6 | 269024774 | 0x1008FE06 |
| XF86Switch_VT_7 | 269024775 | 0x1008FE07 |
| XF86Switch_VT_8 | 269024776 | 0x1008FE08 |
| XF86Switch_VT_9 | 269024777 | 0x1008FE09 |
| XF86Switch_VT_10 | 269024778 | 0x1008FE0A |
| XF86Switch_VT_11 | 269024779 | 0x1008FE0B |
| XF86Switch_VT_12 | 269024780 | 0x1008FE0C |

```
XF86Ungrab                   269024800     0x1008FE20
XF86ClearGrab                269024801     0x1008FE21
XF86Next_VMode               269024802     0x1008FE22
XF86Prev_VMode               269024803     0x1008FE23
XF86LogWindowTree            269024804     0x1008FE24
XF86LogGrabInfo              269024805     0x1008FE25
XF86ModeLock                 269025025     0x1008FF01
XF86MonBrightnessUp          269025026     0x1008FF02
XF86MonBrightnessDown        269025027     0x1008FF03
XF86KbdLightOnOff            269025028     0x1008FF04
XF86KbdBrightnessUp          269025029     0x1008FF05
XF86KbdBrightnessDown        269025030     0x1008FF06
XF86MonBrightnessCycle       269025031     0x1008FF07
XF86Standby                  269025040     0x1008FF10
XF86AudioLowerVolume         269025041     0x1008FF11
XF86AudioMute                269025042     0x1008FF12
XF86AudioRaiseVolume         269025043     0x1008FF13
XF86AudioPlay                269025044     0x1008FF14
XF86AudioStop                269025045     0x1008FF15
XF86AudioPrev                269025046     0x1008FF16
XF86AudioNext                269025047     0x1008FF17
XF86HomePage                 269025048     0x1008FF18
XF86Mail                     269025049     0x1008FF19
XF86Start                    269025050     0x1008FF1A
XF86Search                   269025051     0x1008FF1B
XF86AudioRecord              269025052     0x1008FF1C
XF86Calculator               269025053     0x1008FF1D
XF86Memo                     269025054     0x1008FF1E
XF86ToDoList                 269025055     0x1008FF1F
XF86Calendar                 269025056     0x1008FF20
XF86PowerDown                269025057     0x1008FF21
XF86ContrastAdjust           269025058     0x1008FF22
XF86RockerUp                 269025059     0x1008FF23
XF86RockerDown               269025060     0x1008FF24
XF86RockerEnter              269025061     0x1008FF25
XF86Back                     269025062     0x1008FF26
XF86Forward                  269025063     0x1008FF27
XF86Stop                     269025064     0x1008FF28
XF86Refresh                  269025065     0x1008FF29
XF86PowerOff                 269025066     0x1008FF2A
XF86WakeUp                   269025067     0x1008FF2B
XF86Eject                    269025068     0x1008FF2C
XF86ScreenSaver              269025069     0x1008FF2D
XF86WWW                      269025070     0x1008FF2E
XF86Sleep                    269025071     0x1008FF2F
XF86Favorites                269025072     0x1008FF30
```

```
XF86AudioPause               269025073     0x1008FF31
XF86AudioMedia               269025074     0x1008FF32
XF86MyComputer               269025075     0x1008FF33
XF86VendorHome               269025076     0x1008FF34
XF86LightBulb                269025077     0x1008FF35
XF86Shop                     269025078     0x1008FF36
XF86History                  269025079     0x1008FF37
XF86OpenURL                  269025080     0x1008FF38
XF86AddFavorite              269025081     0x1008FF39
XF86HotLinks                 269025082     0x1008FF3A
XF86BrightnessAdjust         269025083     0x1008FF3B
XF86Finance                  269025084     0x1008FF3C
XF86Community                269025085     0x1008FF3D
XF86AudioRewind              269025086     0x1008FF3E
XF86BackForward              269025087     0x1008FF3F
XF86Launch0                  269025088     0x1008FF40
XF86Launch1                  269025089     0x1008FF41
XF86Launch2                  269025090     0x1008FF42
XF86Launch3                  269025091     0x1008FF43
XF86Launch4                  269025092     0x1008FF44
XF86Launch5                  269025093     0x1008FF45
XF86Launch6                  269025094     0x1008FF46
XF86Launch7                  269025095     0x1008FF47
XF86Launch8                  269025096     0x1008FF48
XF86Launch9                  269025097     0x1008FF49
XF86LaunchA                  269025098     0x1008FF4A
XF86LaunchB                  269025099     0x1008FF4B
XF86LaunchC                  269025100     0x1008FF4C
XF86LaunchD                  269025101     0x1008FF4D
XF86LaunchE                  269025102     0x1008FF4E
XF86LaunchF                  269025103     0x1008FF4F
XF86ApplicationLeft          269025104     0x1008FF50
XF86ApplicationRight         269025105     0x1008FF51
XF86Book                     269025106     0x1008FF52
XF86CD                       269025107     0x1008FF53
XF86Calculater               269025108     0x1008FF54
XF86Clear                    269025109     0x1008FF55
XF86Close                    269025110     0x1008FF56
XF86Copy                     269025111     0x1008FF57
XF86Cut                      269025112     0x1008FF58
XF86Display                  269025113     0x1008FF59
XF86DOS                      269025114     0x1008FF5A
XF86Documents                269025115     0x1008FF5B
XF86Excel                    269025116     0x1008FF5C
XF86Explorer                 269025117     0x1008FF5D
XF86Game                     269025118     0x1008FF5E
```

```
XF86Go                       269025119     0x1008FF5F
XF86iTouch                   269025120     0x1008FF60
XF86LogOff                   269025121     0x1008FF61
XF86Market                   269025122     0x1008FF62
XF86Meeting                  269025123     0x1008FF63
XF86MenuKB                   269025125     0x1008FF65
XF86MenuPB                   269025126     0x1008FF66
XF86MySites                  269025127     0x1008FF67
XF86New                      269025128     0x1008FF68
XF86News                     269025129     0x1008FF69
XF86OfficeHome               269025130     0x1008FF6A
XF86Open                     269025131     0x1008FF6B
XF86Option                   269025132     0x1008FF6C
XF86Paste                    269025133     0x1008FF6D
XF86Phone                    269025134     0x1008FF6E
XF86Q                        269025136     0x1008FF70
XF86Reply                    269025138     0x1008FF72
XF86Reload                   269025139     0x1008FF73
XF86RotateWindows            269025140     0x1008FF74
XF86RotationPB               269025141     0x1008FF75
XF86RotationKB               269025142     0x1008FF76
XF86Save                     269025143     0x1008FF77
XF86ScrollUp                 269025144     0x1008FF78
XF86ScrollDown               269025145     0x1008FF79
XF86ScrollClick              269025146     0x1008FF7A
XF86Send                     269025147     0x1008FF7B
XF86Spell                    269025148     0x1008FF7C
XF86SplitScreen              269025149     0x1008FF7D
XF86Support                  269025150     0x1008FF7E
XF86TaskPane                 269025151     0x1008FF7F
XF86Terminal                 269025152     0x1008FF80
XF86Tools                    269025153     0x1008FF81
XF86Travel                   269025154     0x1008FF82
XF86UserPB                   269025156     0x1008FF84
XF86User1KB                  269025157     0x1008FF85
XF86User2KB                  269025158     0x1008FF86
XF86Video                    269025159     0x1008FF87
XF86WheelButton              269025160     0x1008FF88
XF86Word                     269025161     0x1008FF89
XF86Xfer                     269025162     0x1008FF8A
XF86ZoomIn                   269025163     0x1008FF8B
XF86ZoomOut                  269025164     0x1008FF8C
XF86Away                     269025165     0x1008FF8D
XF86Messenger                269025166     0x1008FF8E
XF86WebCam                   269025167     0x1008FF8F
XF86MailForward              269025168     0x1008FF90
```

```
XF86Pictures                 269025169     0x1008FF91
XF86Music                    269025170     0x1008FF92
XF86Battery                  269025171     0x1008FF93
XF86Bluetooth                269025172     0x1008FF94
XF86WLAN                     269025173     0x1008FF95
XF86UWB                      269025174     0x1008FF96
XF86AudioForward             269025175     0x1008FF97
XF86AudioRepeat              269025176     0x1008FF98
XF86AudioRandomPlay          269025177     0x1008FF99
XF86Subtitle                 269025178     0x1008FF9A
XF86AudioCycleTrack          269025179     0x1008FF9B
XF86CycleAngle               269025180     0x1008FF9C
XF86FrameBack                269025181     0x1008FF9D
XF86FrameForward             269025182     0x1008FF9E
XF86Time                     269025183     0x1008FF9F
XF86Select                   269025184     0x1008FFA0
XF86View                     269025185     0x1008FFA1
XF86TopMenu                  269025186     0x1008FFA2
XF86Red                      269025187     0x1008FFA3
XF86Green                    269025188     0x1008FFA4
XF86Yellow                   269025189     0x1008FFA5
XF86Blue                     269025190     0x1008FFA6
XF86Suspend                  269025191     0x1008FFA7
XF86Hibernate                269025192     0x1008FFA8
XF86TouchpadToggle           269025193     0x1008FFA9
XF86TouchpadOn               269025200     0x1008FFB0
XF86TouchpadOff              269025201     0x1008FFB1
XF86AudioMicMute             269025202     0x1008FFB2
XF86Keyboard                 269025203     0x1008FFB3
XF86WWAN                     269025204     0x1008FFB4
XF86RFKill                   269025205     0x1008FFB5
XF86AudioPreset              269025206     0x1008FFB6
XF86RotationLockToggle       269025207     0x1008FFB7
XF86FullScreen               269025208     0x1008FFB8
```

*Copyright (c) 1998-2000 Scriptics Corporation.*

# label

*Stichwörter:* label, widget

## Name

label - Erstellt und manipuliert 'Label'-Widgets für nicht-interaktiven Text oder Bilder

## Übersicht

**label** *pathName* ?*options*?

## Standardoptionen

**-activebackground**, **-disabledforeground**, **-padx**, **-activeforeground**, **-font**, **-pady**, **-anchor**, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-takefocus**, **-bitmap**, **-highlightcolor**, **-text**, **-borderwidth**, **-highlightthickness**, **-textvariable**, **-compound**, **-image**, **-underline**, **-cursor**, **-justify**, **-wraplength**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-height (height/Height)**

Legt eine gewünschte Höhe für das Label fest. Wenn ein Bild oder Bitmap im Label angezeigt wird, ist der Wert in Bildschirmeinheiten (d.h. alle von Tk_GetPixels akzeptierten Formen); für Text ist es in Textzeilen. Wenn diese Option nicht angegeben ist, wird die gewünschte Höhe des Labels aus der Größe des angezeigten Bildes, Bitmaps oder Textes berechnet.

**-state (state/State)**

Legt einen von drei Zuständen für das Label fest: **normal**, **active** oder **disabled**. Im normalen Zustand wird das Label mit den Optionen **-foreground** und **-background** angezeigt. Im aktiven Zustand wird das Label mit den Optionen **-activeforeground** und **-activebackground** angezeigt. Im deaktivierten Zustand bestimmen die Optionen **-disabledforeground** und **-background**, wie das Label angezeigt wird.

**-width (width/Width)**

Legt eine gewünschte Breite für das Label fest. Wenn ein Bild oder Bitmap im Label angezeigt wird, ist der Wert in Bildschirmeinheiten (d.h. alle von Tk_GetPixels akzeptierten Formen); für Text ist es in Zeichen. Wenn diese Option nicht angegeben ist, wird die gewünschte Breite des Labels aus der Größe des angezeigten Bildes, Bitmaps oder Textes berechnet.

## Beschreibung

Der Befehl **label** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Label-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des Labels wie Farben, Schriftart, Text und anfängliches Relief zu konfigurieren. Der Befehl **label** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Ein Label ist ein Widget, das einen Text-String, ein Bitmap oder ein Bild anzeigt. Wenn Text angezeigt wird, muss er in einer einzigen Schriftart sein, kann aber mehrere Zeilen auf dem Bildschirm einnehmen (wenn er Zeilenumbrüche enthält oder wenn ein Umbruch wegen der Option **-wraplength** erfolgt), und eines der Zeichen kann optional mit der Option **-underline** unterstrichen werden. Das Label kann auf einige einfache Arten manipuliert werden, wie z.B. das Ändern seines Reliefs oder Textes, unter Verwendung der unten beschriebenen Befehle.

## Widget-Befehl

Der Befehl **label** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *args* bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Label-Widgets möglich:

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **label** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **label** akzeptierten Werte haben.

## Bindings

Wenn ein neues Label erstellt wird, hat es keine Standard-Event-Bindings: Labels sind nicht für Interaktion gedacht.

## Beispiel

```
# Erstelle die Widgets
label .t -text "This widget is at the top"    -bg red
label .b -text "This widget is at the bottom" -bg green
label .l -text "Left\nHand\nSide"
label .r -text "Right\nHand\nSide"
text .mid
.mid insert end "This layout is like Java's BorderLayout"
# Ordne sie an
pack .t   -side top    -fill x
pack .b   -side bottom -fill x
pack .l   -side left   -fill y
pack .r   -side right  -fill y
pack .mid -expand 1    -fill both
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# labelframe

*Stichwörter:* labelframe, widget

## Name

labelframe - Erstellt und manipuliert 'labelframe'-beschriftete Container-Widgets

## Übersicht

**labelframe** *pathName* ?*options*?

## Standardoptionen

**-borderwidth**, **-highlightbackground**, **-pady**, **-cursor**, **-highlightcolor**, **-relief**, **-font**, **-highlightthickness**, **-takefocus**, **-foreground**, **-padx**, **-text**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-background (background/Background)**

Diese Option ist identisch mit der Standard-Option **-background**, außer dass ihr Wert auch als leerer String angegeben werden kann. In diesem Fall zeigt das Widget keinen Hintergrund oder Rahmen an, und es werden keine Farben aus seiner Farbkarte für Hintergrund und Rahmen verbraucht.

**-class (class/Class)**

Legt eine Klasse für das Fenster fest. Diese Klasse wird verwendet, wenn die Optionsdatenbank für die anderen Optionen des Fensters abgefragt wird, und sie wird später auch für andere Zwecke wie Bindings verwendet. Die Option **-class** kann nicht mit dem Widget-Befehl **configure** geändert werden.

**-colormap (colormap/Colormap)**

Legt eine Farbkarte fest, die für das Fenster verwendet werden soll. Der Wert kann entweder **new** sein, in welchem Fall eine neue Farbkarte für das Fenster und seine Kinder erstellt wird, oder der Name eines anderen Fensters (das auf dem gleichen Bildschirm und mit der gleichen visuellen Darstellung wie *pathName* sein muss), in welchem Fall das neue Fenster die Farbkarte vom angegebenen Fenster verwendet. Wenn die Option **-colormap** nicht angegeben ist, verwendet das neue Fenster die gleiche Farbkarte wie sein Eltern-Element. Diese Option kann nicht mit dem Widget-Befehl **configure** geändert werden.

**-height (height/Height)**

Legt die gewünschte Höhe für das Fenster in einer der von Tk_GetPixels akzeptierten Formen fest. Wenn diese Option negativ oder null ist, fordert das Fenster überhaupt keine Größe an.

**-labelanchor (labelAnchor/LabelAnchor)**

Legt fest, wo die Beschriftung platziert wird. Eine Beschriftung wird nur angezeigt, wenn die Option **-text** nicht der leere String ist. Gültige Werte für diese Option sind (im Uhrzeigersinn aufgeführt) **nw**, **n**, **ne**, **en**, **e**, **es**, **se**, **s**, **sw**, **ws**, **w** und **wn**. Der Standardwert ist **nw**.

**-labelwidget (labelWidget/LabelWidget)**

Legt ein Widget fest, das als Beschriftung verwendet wird. Dies überschreibt jede **-text** -Option. Das Widget muss existieren, bevor es als **-labelwidget** verwendet wird, und wenn es kein Nachfahre dieses Fensters ist, wird es in der Stapelreihenfolge darüber angehoben.

**-visual (visual/Visual)**

Legt visuelle Informationen für das neue Fenster in einer der von Tk_GetVisual akzeptierten Formen fest. Wenn diese Option nicht angegeben ist, verwendet das neue Fenster die gleiche visuelle Darstellung wie sein Eltern-Element. Die Option **-visual** kann nicht mit dem Widget-Befehl **configure** geändert werden.

**-width (width/Width)**

Legt die gewünschte Breite für das Fenster in einer der von Tk_GetPixels akzeptierten Formen fest. Wenn diese Option negativ oder null ist, fordert das Fenster überhaupt keine Größe an.

## Beschreibung

Der Befehl **labelframe** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem labelframe-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des labelframe wie Hintergrundfarbe und Relief zu konfigurieren. Der Befehl **labelframe** gibt den Pfadnamen des neuen Fensters zurück.

Ein labelframe ist ein einfaches Widget. Sein Hauptzweck ist es, als Abstandshalter oder Container für komplexe Fensterlayouts zu dienen. Es hat die Merkmale eines frame plus die Fähigkeit, eine Beschriftung anzuzeigen.

## Widget-Befehl

Der Befehl **labelframe** erstellt einen neuen Tcl-Befehl, dessen Name identisch mit dem Pfadnamen des labelframe-Fensters ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*PathName* ist der Name des Befehls, der identisch mit dem Pfadnamen des labelframe-Widgets ist. *Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Frame-Widgets möglich:

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **labelframe** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option-value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **labelframe** akzeptierten Werte haben.

## Bindings

Wenn ein neues labelframe erstellt wird, hat es keine Standard-Event-Bindings: labelframes sind nicht für Interaktion gedacht.

## Beispiel

Dies zeigt, wie ein Teil einer GUI für einen Hamburger-Anbieter aufgebaut wird. Die **labelframe** -Widgets werden verwendet, um die verfügbaren Auswahlmöglichkeiten nach den Arten der Dinge zu gruppieren, über die gewählt wird.

```
grid [labelframe .burger -text "Burger"] \
     [labelframe .bun    -text "Bun"] -sticky news
grid [labelframe .cheese -text "Cheese Option"] \
     [labelframe .pickle -text "Pickle Option"] -sticky news
foreach {type name val} {
    burger Beef    beef
    burger Lamb    lamb
    burger Vegetarian beans

    bun    Plain   white
    bun    Sesame  seeds
    bun    Wholemeal brown

    cheese None    none
    cheese Cheddar cheddar
    cheese Edam    edam
    cheese Brie    brie
    cheese Gruy\u00e8re gruyere
    cheese "Monterey Jack" jack

    pickle None    none
    pickle Gherkins gherkins
    pickle Onions  onion
    pickle Chili   chili
} {
    set w [radiobutton .$type.$val -text $name -anchor w \
            -variable $type -value $val]
    pack $w -side top -fill x
}
set burger beef
set bun    white
set cheese none
set pickle none
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# listbox

*Stichwörter:* listbox, widget

## Name

listbox - Erstellt und manipuliert 'Listbox'-Elementlisten-Widgets

## Übersicht

**listbox** *pathName* ?*options*?

## Standardoptionen

**-background**, **-borderwidth**, **-cursor**, **-disabledforeground**, **-exportselection**, **-font**, **-foreground**, **-highlightbackground**, **-highlightcolor**, **-highlightthickness**, **-justify**, **-relief**, **-selectbackground**, **-selectborderwidth**, **-selectforeground**, **-setgrid**, **-takefocus**, **-xscrollcommand**, **-yscrollcommand**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-activestyle (activeStyle/ActiveStyle)**

Legt den Stil fest, in dem das aktive Element gezeichnet wird. Dies muss einer von **dotbox** (zeigt einen Fokusring um das aktive Element), **none** (keine besondere Kennzeichnung des aktiven Elements) oder **underline** (unterstreicht das aktive Element) sein. Der Standardwert ist **underline** unter Windows und **dotbox** anderswo.

**-height (height/Height)**

Legt die gewünschte Höhe für das Fenster in Zeilen fest. Wenn der Wert null oder kleiner ist, wird die gewünschte Höhe des Fensters gerade groß genug gemacht, um alle Elemente in der Listbox aufzunehmen.

**-listvariable (listVariable/Variable)**

Legt den Namen einer globalen Variable fest. Der Wert der Variable ist eine Liste, die innerhalb des Widgets angezeigt werden soll; wenn sich der Variablenwert ändert, aktualisiert sich das Widget automatisch, um den neuen Wert widerzuspiegeln. Versuche, einer Variable mit einem ungültigen Listenwert **-listvariable** zuzuweisen, führen zu einem Fehler. Versuche, eine als **-listvariable** verwendete Variable zu entfernen, schlagen fehl, erzeugen aber keinen Fehler.

**-selectmode (selectMode/SelectMode)**

Legt einen von mehreren Stilen für die Manipulation der Auswahl fest. Der Wert der Option kann beliebig sein, aber die Standard-Bindings erwarten **single**, **browse**, **multiple** oder **extended**; der Standardwert ist **browse**.

**-state (state/State)**

Legt einen von zwei Zuständen für die Listbox fest: **normal** oder **disabled**. Wenn die Listbox deaktiviert ist, können keine Elemente eingefügt oder gelöscht werden, Elemente werden in der Farbe **-disabledforeground** gezeichnet, und die Auswahl kann nicht geändert werden und wird nicht angezeigt (Auswahlinformationen bleiben jedoch erhalten).

**-width (width/Width)**

Legt die gewünschte Breite für das Fenster in Zeichen fest. Wenn die Schriftart keine einheitliche Breite hat, wird die Breite des Zeichens "0" bei der Umrechnung von Zeicheneinheiten in Bildschirmeinheiten verwendet. Wenn der Wert null oder kleiner ist, wird die gewünschte Breite des Fensters gerade groß genug gemacht, um alle Elemente in der Listbox aufzunehmen.

## Beschreibung

Der Befehl **listbox** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Listbox-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte der Listbox wie Farben, Schriftart, Text und Relief zu konfigurieren. Der Befehl **listbox** gibt sein *pathName* -Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Eine Listbox ist ein Widget, das eine Liste von Zeichenketten anzeigt, eine pro Zeile. Bei der Erstellung hat eine neue Listbox keine Elemente. Elemente können mit den unten beschriebenen Widget-Befehlen hinzugefügt oder gelöscht werden. Zusätzlich können ein oder mehrere

Elemente wie unten beschrieben ausgewählt werden. Wenn eine Listbox ihre Auswahl exportiert (siehe Option **-exportselection**), beachtet sie die Standard-X11-Protokolle für die Behandlung der Auswahl. Listbox-Auswahlen sind als Typ **STRING** verfügbar; der Wert der Auswahl ist der Text der ausgewählten Elemente, wobei die Elemente durch Zeilenumbrüche getrennt sind.

Es ist nicht erforderlich, dass alle Elemente gleichzeitig im Listbox-Fenster angezeigt werden; die unten beschriebenen Befehle können verwendet werden, um die Ansicht im Fenster zu ändern. Listboxes erlauben das Scrollen in beide Richtungen mit den Standardoptionen **-xscrollcommand** und **-yscrollcommand**. Sie unterstützen auch Scanning, wie unten beschrieben.

## Indizes

Viele der Widget-Befehle für Listboxes nehmen ein oder mehrere Indizes als Argumente. Ein Index gibt ein bestimmtes Element der Listbox an, auf eine der folgenden Arten:

***number***

Legt das Element als numerischen Index fest, wobei 0 dem ersten Element in der Listbox entspricht.

**active**

Bezeichnet das Element, das den Positionscursor hat. Dieses Element wird wie durch **-activestyle** festgelegt angezeigt, wenn die Listbox den Tastaturfokus hat, und wird mit dem Widget-Befehl **activate** festgelegt.

**anchor**

Bezeichnet den Ankerpunkt für die Auswahl, der mit dem Widget-Befehl **selection anchor** gesetzt wird.

**end**

Bezeichnet das Ende der Listbox. Für die meisten Befehle bezieht sich dies auf das letzte Element in der Listbox, aber für einige Befehle wie **index** und **insert** bezieht es sich auf das Element direkt nach dem letzten.

**@*x*,*y***

Bezeichnet das Element, das den Punkt im Listbox-Fenster abdeckt, der durch *x* und *y* (in Pixelkoordinaten) angegeben ist. Wenn kein Element diesen Punkt abdeckt, wird das dem Punkt am nächsten liegende Element verwendet.

Indizes unterstützen dieselbe einfache Interpretation wie beim Befehl string index, mit einfacher ganzzahliger Indexarithmetik und Indexierung relativ zu **end**. In den unten stehenden Widget-Befehlsbeschreibungen enthalten Argumente mit den Namen *index*, *first* und *last* immer Textindizes in einer der obigen Formen.

## Widget-Befehl

Der Befehl **listbox** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Listbox-Widgets möglich:

***pathName* activate *index***

Setzt das aktive Element auf das durch *index* angegebene. Wenn *index* außerhalb des Bereichs der Elemente in der Listbox liegt, wird das nächstgelegene Element aktiviert. Das aktive Element wird wie durch **-activestyle** festgelegt gezeichnet, wenn das Widget den Eingabefokus hat, und sein Index kann mit dem Index **active** abgerufen werden.

***pathName* bbox *index***

Gibt eine Liste von vier Zahlen zurück, die die Begrenzungsbox des Textes im durch *index* gegebenen Element beschreiben. Die ersten beiden Elemente der Liste geben die x- und y-Koordinaten der oberen linken Ecke des vom Text abgedeckten Bildschirmbereichs an (in Pixeln relativ zum Widget), und die letzten beiden Elemente geben die Breite und Höhe des Bereichs in Pixeln an. Wenn kein Teil des durch *index* gegebenen Elements auf dem Bildschirm sichtbar ist oder wenn *index* auf ein nicht existierendes Element verweist, ist das Ergebnis eine leere Zeichenkette; wenn das Element teilweise sichtbar ist, gibt das Ergebnis den vollständigen Bereich des Elements an, einschließlich aller nicht sichtbaren Teile.

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **listbox** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **listbox** akzeptierten Werte haben.

***pathName* curselection**

Gibt eine Liste zurück, die die numerischen Indizes aller Elemente in der Listbox enthält, die derzeit ausgewählt sind. Wenn in der Listbox keine Elemente ausgewählt sind, wird eine leere Zeichenkette zurückgegeben.

***pathName* delete *first* ?*last*?**

Löscht ein oder mehrere Elemente der Listbox. *First* und *last* sind Indizes, die das erste und letzte Element im zu löschenden Bereich angeben. Wenn *last* nicht angegeben ist, entspricht es *first*, d. h. ein einzelnes Element wird gelöscht.

***pathName* get *first* ?*last*?**

Wenn *last* weggelassen wird, gibt der Befehl den Inhalt des durch *first* angegebenen Listbox-Elements zurück oder eine leere Zeichenkette, wenn *first* auf ein nicht existierendes Element verweist. Wenn *last* angegeben ist, gibt der Befehl eine Liste zurück, deren Elemente alle Listbox-Elemente zwischen *first* und *last* einschließlich sind. Sowohl *first* als auch *last* können eine der Standardformen für Indizes haben.

***pathName* index *index***

Gibt den ganzzahligen Indexwert zurück, der *index* entspricht. Wenn *index* **end** ist, ist der Rückgabewert eine Anzahl der Elemente in der Listbox (nicht der Index des letzten Elements).

***pathName* insert *index* ?*element element* ...?**

Fügt null oder mehr neue Elemente in die Liste direkt vor dem durch *index* gegebenen

Element ein. Wenn *index* als **end** angegeben ist, werden die neuen Elemente am Ende der Liste hinzugefügt. Gibt einen leeren String zurück.

***pathName* itemcget *index option***

Gibt den aktuellen Wert der durch *option* gegebenen Item-Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **itemconfigure** akzeptierten Werte haben.

***pathName* itemconfigure *index* ?*option*? ?*value*? ?*option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen eines Items in der Listbox. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für das Item beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option-value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. Die folgenden Optionen werden derzeit für Items unterstützt:

**-background *color***

*Color* legt die Hintergrundfarbe fest, die beim Anzeigen des Items verwendet wird. Sie kann eine der von Tk_GetColor akzeptierten Formen haben.

**-foreground *color***

*Color* legt die Vordergrundfarbe fest, die beim Anzeigen des Items verwendet wird. Sie kann eine der von Tk_GetColor akzeptierten Formen haben.

**-selectbackground *color***

*color* legt die Hintergrundfarbe fest, die beim Anzeigen des Items verwendet wird, während es ausgewählt ist. Sie kann eine der von Tk_GetColor akzeptierten Formen haben.

**-selectforeground *color***

*color* legt die Vordergrundfarbe fest, die beim Anzeigen des Items verwendet wird, während es ausgewählt ist. Sie kann eine der von Tk_GetColor akzeptierten Formen haben.

***pathName* nearest *y***

Gibt bei einer y-Koordinate innerhalb des Listbox-Fensters den Index des (sichtbaren) Listbox-Elements zurück, das dieser y-Koordinate am nächsten liegt.

***pathName* scan *option args***

Dieser Befehl wird verwendet, um Scanning auf Listboxes zu implementieren. Er hat zwei Formen, abhängig von *option*:

***pathName* scan mark *x y***

Speichert *x* und *y* sowie die aktuelle Ansicht im Listbox-Fenster; wird in Verbindung mit späteren **scan dragto**-Befehlen verwendet. Typischerweise wird dieser Befehl mit einem Maustastendruck im Widget verknüpft. Gibt einen leeren String zurück.

***pathName* scan dragto *x y*.**

Dieser Befehl berechnet die Differenz zwischen seinen *x*- und *y*-Argumenten und den *x*- und *y*-Argumenten des letzten **scan mark**-Befehls für das Widget. Anschließend passt er die Ansicht um das Zehnfache der Koordinatendifferenz an. Dieser Befehl wird typischerweise mit Mausbewegungs-Events im Widget verknüpft, um den Effekt zu erzeugen, die Liste mit hoher Geschwindigkeit durch das Fenster zu ziehen. Der Rückgabewert ist ein leerer String.

***pathName* see *index***

Passt die Ansicht in der Listbox so an, dass das durch *index* gegebene Element sichtbar ist. Wenn das Element bereits sichtbar ist, hat der Befehl keine Wirkung; wenn das Element nahe an einer Kante des Fensters liegt, scrollt die Listbox, um das Element an der Kante sichtbar zu machen; andernfalls scrollt die Listbox, um das Element zu zentrieren.

***pathName* selection *option arg***

Dieser Befehl wird verwendet, um die Auswahl innerhalb einer Listbox anzupassen. Er hat mehrere Formen, abhängig von *option*:

***pathName* selection anchor *index***

Setzt den Auswahlanker auf das durch *index* gegebene Element. Wenn *index* auf ein nicht existierendes Element verweist, wird das nächstgelegene Element verwendet. Der Auswahlanker ist das Ende der Auswahl, das beim Ziehen einer Auswahl mit der Maus fest bleibt. Der Index **anchor** kann verwendet werden, um auf das Ankerelement zu verweisen.

***pathName* selection clear *first* ?*last*?**

Wenn eines der Elemente zwischen *first* und *last* (einschließlich) ausgewählt ist, wird die Auswahl aufgehoben. Der Auswahlzustand wird für Elemente außerhalb dieses Bereichs nicht geändert.

***pathName* selection includes *index***

Gibt 1 zurück, wenn das durch *index* angegebene Element derzeit ausgewählt ist, 0 wenn nicht.

***pathName* selection set *first* ?*last*?**

Wählt alle Elemente im Bereich zwischen *first* und *last* einschließlich aus, ohne den Auswahlzustand von Elementen außerhalb dieses Bereichs zu beeinflussen.

***pathName* size**

Gibt eine Dezimalzeichenkette zurück, die die Gesamtzahl der Elemente in der Listbox angibt.

***pathName* xview ?*args*?**

Dieser Befehl wird verwendet, um die horizontale Position der Informationen im Fenster des Widgets abzufragen und zu ändern. Er kann eine der folgenden Formen annehmen:

***pathName* xview**

Gibt eine Liste mit zwei Elementen zurück. Jedes Element ist ein reeller Bruch zwischen 0 und 1; zusammen beschreiben sie den horizontal sichtbaren Bereich im Fenster. Wenn beispielsweise das erste Element .2 und das zweite Element .6 ist, sind 20 % des Listbox-Textes links außerhalb des Bildschirms, die mittleren 40 % sind im Fenster sichtbar und 40 % des Textes sind rechts außerhalb des Bildschirms. Dies sind dieselben Werte, die über die Option **-xscrollcommand** an Scrollbars übergeben werden.

***pathName* xview *index***

Passt die Ansicht im Fenster so an, dass die durch *index* gegebene Zeichenposition am linken Rand des Fensters angezeigt wird. Zeichenpositionen werden durch die Breite des Zeichens **0** definiert.

***pathName* xview moveto *fraction***

Passt die Ansicht im Fenster so an, dass *fraction* der Gesamtbreite des Listbox-Textes links außerhalb des Bildschirms ist. *fraction* muss ein Bruch zwischen 0 und 1 sein.

***pathName* xview scroll *number what***

Dieser Befehl verschiebt die Ansicht im Fenster nach links oder rechts gemäß *number* und *what*. *Number* muss eine Ganzzahl oder eine Fließkommazahl sein, aber wenn es eine Fließkommazahl ist, wird sie in eine Ganzzahl umgewandelt, von 0 weg gerundet. *What* muss entweder **pages** oder **units** oder eine Abkürzung davon sein. Wenn *what* **pages** ist, passt sich die Ansicht um *number* Bildschirmfüllungen an. Wenn *number* negativ ist, werden Zeichen weiter links sichtbar; wenn es positiv ist, werden Zeichen weiter rechts sichtbar. Wenn *what* **units** ist, passt sich die Ansicht nach links oder rechts um *number* Zeicheneinheiten (die Breite des Zeichens **0**) auf der Anzeige an.

***pathName* yview ?*args*?**

Dieser Befehl wird verwendet, um die vertikale Position des Textes im Fenster des Widgets abzufragen und zu ändern. Er kann eine der folgenden Formen annehmen:

***pathName* yview**

Gibt eine Liste mit zwei Elementen zurück, die beide reelle Brüche zwischen 0 und 1 sind. Das erste Element gibt die Position des Listbox-Elements am oberen Rand des Fensters relativ zur Listbox als Ganzes an (0,5 bedeutet beispielsweise, dass es auf halber Strecke durch die Listbox ist). Das zweite Element gibt die Position des Listbox-Elements direkt nach dem letzten im Fenster relativ zur Listbox als Ganzes an. Dies sind dieselben Werte, die über die Option **-yscrollcommand** an Scrollbars übergeben werden.

***pathName* yview *index***

Passt die Ansicht im Fenster so an, dass das durch *index* gegebene Element am oberen Rand des Fensters angezeigt wird.

***pathName* yview moveto *fraction***

Passt die Ansicht im Fenster so an, dass das durch *fraction* gegebene Element am oberen Rand des Fensters erscheint. *Fraction* ist ein Bruch zwischen 0 und 1; 0 bezeichnet das erste Element in der Listbox, 0,33 bezeichnet das Element ein Drittel des Weges durch die Listbox und so weiter.

***pathName* yview scroll *number what***

Dieser Befehl passt die Ansicht im Fenster nach oben oder unten gemäß *number* und *what* an. *Number* muss eine Ganzzahl oder eine Fließkommazahl sein, aber wenn es eine Fließkommazahl ist, wird sie in eine Ganzzahl umgewandelt, von 0 weg gerundet. *What* muss entweder **pages** oder **units** sein. Wenn *what* **pages** ist, passt sich die Ansicht um *number* Bildschirmfüllungen an. Wenn *number* negativ ist, werden frühere Elemente sichtbar; wenn es positiv ist, werden spätere Elemente sichtbar. Wenn *what* **units** ist, passt sich die Ansicht nach oben oder unten um *number* Zeilen an.

## Standard-Bindings

Tk erstellt automatisch Klassen-Bindings für Listboxes, die ihnen Motif-ähnliches Verhalten geben. Ein Großteil des Verhaltens einer Listbox wird durch ihre Option **-selectmode** bestimmt, die eine von vier Arten der Behandlung der Auswahl auswählt.

Wenn der Auswahlmodus **single** oder **browse** ist, kann höchstens ein Element in der Listbox gleichzeitig ausgewählt sein. In beiden Modi wählt ein Klick mit Taste 1 auf ein Element es aus und hebt die Auswahl aller anderen ausgewählten Elemente auf. Im Modus **browse** ist es auch möglich, die Auswahl mit Taste 1 zu ziehen. Bei Taste 1 erhält die Listbox auch den Fokus, wenn sie den Zustand **normal** hat.

Wenn der Auswahlmodus **multiple** oder **extended** ist, können beliebig viele Elemente gleichzeitig ausgewählt werden, einschließlich nicht zusammenhängender Bereiche. Im Modus **multiple** schaltet ein Klick mit Taste 1 auf ein Element dessen Auswahlzustand um, ohne andere Elemente zu beeinflussen. Im Modus **extended** wählt das Drücken von Taste 1 auf ein Element es aus, hebt die Auswahl aller anderen auf und setzt den Anker auf das Element unter der Maus; das Ziehen der Maus mit gedrückter Taste 1 erweitert die Auswahl, um alle Elemente zwischen dem Anker und dem Element unter der Maus einschließlich einzubeziehen.

Die meisten Benutzer werden wahrscheinlich den Modus **browse** für Einzelauswahlen und den Modus **extended** für Mehrfachauswahlen verwenden wollen; die anderen Modi scheinen nur in besonderen Situationen nützlich zu sein.

Jedes Mal, wenn die Menge der ausgewählten Elemente in der Listbox vom Benutzer über Tastatur oder Maus aktualisiert wird, wird das virtuelle Event **<<ListboxSelect>>** erzeugt. Dieses virtuelle Event wird nicht erzeugt, wenn die Auswahl mit dem Befehl *pathName* selection angepasst wird. Am einfachsten ist es, an dieses Event zu binden, um über Benutzeränderungen der Listbox-Auswahl informiert zu werden.

Zusätzlich zu dem obigen Verhalten wird durch die Standard-Bindings folgendes weiteres Verhalten definiert:

1. Im Modus **extended** kann der ausgewählte Bereich angepasst werden, indem Taste 1 mit gedrückter Umschalttaste gedrückt wird: dies ändert die Auswahl so, dass sie aus den Elementen zwischen dem Anker und dem Element unter der Maus einschließlich besteht. Das nicht verankerte Ende dieser neuen Auswahl kann ebenfalls mit gedrückter Taste gezogen werden.
2. Im Modus **extended** startet das Drücken von Taste 1 mit gedrückter Steuerungstaste eine Umschaltoperation: der Anker wird auf das Element unter der Maus gesetzt und dessen Auswahlzustand wird umgekehrt. Der Auswahlzustand anderer Elemente wird nicht geändert. Wenn die Maus mit gedrückter Taste 1 gezogen wird, wird der Auswahlzustand aller Elemente zwischen dem Anker und dem Element unter der Maus so gesetzt, dass er dem des Ankerelements entspricht; der Auswahlzustand aller anderen Elemente bleibt, was er vor Beginn der Umschaltoperation war.
3. Wenn die Maus das Listbox-Fenster mit gedrückter Taste 1 verlässt, scrollt das Fenster von der Maus weg und macht Informationen sichtbar, die zuvor auf der Seite der Maus außerhalb des Bildschirms waren. Das Scrollen setzt sich fort, bis die Maus das Fenster wieder betritt, die Taste losgelassen wird oder das Ende der Listbox erreicht ist.
4. Maustaste 2 kann für Scanning verwendet werden. Wenn sie gedrückt und über die Listbox gezogen wird, zieht der Inhalt der Listbox mit hoher Geschwindigkeit in die Richtung, in die sich die Maus bewegt.
5. Wenn die Auf- oder Ab-Taste gedrückt wird, bewegt sich der Positionscursor (aktives Element) um ein Element nach oben oder unten. Wenn der Auswahlmodus **browse** oder **extended** ist, wird das neue aktive Element auch ausgewählt und alle anderen Elemente abgewählt. Im Modus **extended** wird das neue aktive Element zum Auswahlanker.
6. Im Modus **extended** bewegen Umschalt-Auf und Umschalt-Ab den Positionscursor (aktives Element) um ein Element nach oben oder unten und erweitern die Auswahl auf dieses Element in ähnlicher Weise wie das Ziehen mit Maustaste 1.
7. Die Links- und Rechts-Tasten scrollen die Listbox-Ansicht nach links und rechts um die Breite des Zeichens **0**. Steuerung-Links und Steuerung-Rechts scrollen die Listbox-Ansicht nach links und rechts um die Breite des Fensters. Steuerung-Bild-Auf und Steuerung-Bild-Ab scrollen

ebenfalls nach links und rechts um die Breite des Fensters.

8. Die Bild-Auf- und Bild-Ab-Tasten scrollen die Listbox-Ansicht nach oben und unten um eine Seite (die Höhe des Fensters).
9. Die Pos1- und Ende-Tasten scrollen die Listbox horizontal zu den linken bzw. rechten Rändern.
10. Steuerung-Pos1 setzt den Positionscursor auf das erste Element in der Listbox, wählt dieses Element aus und hebt die Auswahl aller anderen in der Listbox auf.
11. Steuerung-Ende setzt den Positionscursor auf das letzte Element in der Listbox, wählt dieses Element aus und hebt die Auswahl aller anderen in der Listbox auf.
12. Im Modus **extended** erweitert Steuerung-Umschalt-Pos1 die Auswahl auf das erste Element in der Listbox und Steuerung-Umschalt-Ende erweitert die Auswahl auf das letzte Element.
13. Im Modus **multiple** bewegt Steuerung-Umschalt-Pos1 den Positionscursor auf das erste Element in der Listbox und Steuerung-Umschalt-Ende bewegt den Positionscursor auf das letzte Element.
14. Die Leertaste und die Auswahl-Taste treffen eine Auswahl am Positionscursor (aktives Element), genau so, als wäre Maustaste 1 über diesem Element gedrückt worden.
15. Im Modus **extended** erweitern Steuerung-Umschalt-Leertaste und Umschalt-Auswahl die Auswahl auf das aktive Element, genau so, als wäre Taste 1 mit gedrückter Umschalttaste gedrückt worden.
16. Im Modus **extended** bricht die Escape-Taste die letzte Auswahl ab und stellt den vorherigen Auswahlzustand aller Elemente im ausgewählten Bereich wieder her.
17. Steuerung-/ wählt alles im Widget aus, außer in den Modi **single** und **browse**, in denen das aktive Element ausgewählt und alles andere abgewählt wird.
18. Steuerung-\ hebt die Auswahl alles im Widget auf, außer im Modus **browse**, wo es keine Wirkung hat.
19. Die F16-Taste (auf vielen Sun-Arbeitsplätzen mit Copy beschriftet) oder Meta-w kopiert die Auswahl im Widget in die Zwischenablage, sofern eine Auswahl vorhanden ist.

Das Verhalten von Listboxes kann geändert werden, indem neue Bindings für einzelne Widgets definiert werden oder indem die Klassen-Bindings neu definiert werden.

*Copyright (c) 1990 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# loadTk

*Stichwörter:* loadTk, alias, auto-loading, auto_mkindex, load, parent interpreter, safe interpreter, child interpreter, source

## Name

safe::loadTk - Tk in einen sicheren Interpreter laden

## Übersicht

**safe::loadTk** *child* ?*-use windowId*? ?*-display displayName*?

## Beschreibung

Safe Tk baut auf Safe Tcl auf. Safe Tcl erlaubt eingeschränkten und vermittelten Zugriff auf automatisches Laden und auf Pakete für sichere Interpreter. Safe Tk ergänzt die Möglichkeit, den Interpreter für sichere Tk-Operationen einzurichten und Tk in sichere Interpreter zu laden.

Der Befehl **safe::loadTk** initialisiert die nötigen Datenstrukturen im genannten sicheren

Interpreter und lädt dann Tk hinein. Der Interpreter muss mit **safe::interpCreate** erzeugt oder mit **safe::interpInit** initialisiert worden sein. Der Befehl gibt den Namen des sicheren Interpreters zurück. Ist **-use** angegeben, enthält das Fenster mit der genannten, systemabhängigen Kennung *windowId* das Fenster "." des sicheren Interpreters; jede gültige Kennung ist zulässig, auch eine, die auf ein Fenster einer anderen Anwendung zeigt. Gehört das vorgesehene Fenster zur eigenen Anwendung und ist ein Tk-Fenster, kann statt der Fensterkennung (etwa aus winfo id *.x.y*) der Fenstername stehen (etwa **.x.y**). Ohne **-use** wird für das Fenster "." des sicheren Interpreters ein neues Toplevel-Fenster erzeugt. Unter X11 legt **-display** ein anderes Display als das voreingestellte fest, wenn das eingebettete Fenster es nutzen soll. Einzelheiten zur Umsetzung stehen im Abschnitt Sicherheitshinweise.

### Sicherheitshinweise

Die Handbuchseite safe zu Tcl beschreibt die grundlegenden Sicherheitsüberlegungen für Safe Tcl.

**safe::loadTk** hängt den Wert von tk_library aus dem Eltern-Interpreter an den virtuellen Zugriffspfad des sicheren Interpreters, damit automatisches Laden im sicheren Interpreter funktioniert.

Die Tk-Initialisierung vertraut beim Start nicht mehr dem Zustand des Kind-Interpreters. **safe::loadTk** registriert den Namen des Kindes. Ruft die Tk-Initialisierung (Tk_SafeInit) dann **safe::InitTk** des Eltern-Interpreters auf, liefert das das gewünschte Gegenstück zu argv (**-use** *windowId*, das passende **-display** und so weiter).

Ohne **-use** wird das neue Toplevel besonders dekoriert, damit der Benutzer stets sieht, dass die Oberfläche von möglicherweise unsicherem Code stammt, und den zugehörigen Interpreter leicht löschen kann.

Unter X11 erzeugen widersprüchliche Angaben von **-use** und **-display** wahrscheinlich einen fatalen X-Fehler.

*Copyright (c) 1995-1996 Sun Microsystems, Inc.*

## lower

*Stichwörter:* lower, obscure, stacking order

### Name

lower - Ändert die Position eines Fensters in der Stapelreihenfolge

### Übersicht

**lower** *window* ?*belowThis*?

### Beschreibung

Ist das Argument *belowThis* weggelassen, senkt **lower** *window* so ab, dass es unter allen Geschwister-Fenstern in der Stapelreihenfolge liegt (es wird von überlappenden Geschwistern verdeckt und verdeckt keine Geschwister). Ist *belowThis* angegeben, muss es der Pfadname eines Fensters sein, das Geschwister von *window* ist oder Nachkomme eines Geschwisters von *window*. Dann fügt **lower** *window* in die Stapelreihenfolge direkt unter *belowThis* ein (oder unter den Vorfahren von *belowThis*, der Geschwister von *window* ist); das kann *window* anheben oder absenken.

Alle toplevel-Fenster können zueinander umgestapelt werden, unabhängig von ihren Pfadnamen; der Window-Manager ist jedoch nicht verpflichtet, Umstapelungswünsche strikt zu erfüllen.

*Copyright (c) 1990 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# menu

*Stichwörter:* menu, widget

## Name

menu, tk_menuSetFocus - Erstellt und manipuliert 'Menu'-Widgets und Menüleisten

## Übersicht

**menu** *pathName* ?*options*?
**tk_menuSetFocus** *pathName*

## Standardoptionen

**-activebackground**, **-borderwidth**, **-foreground**, **-activeborderwidth**, **-cursor**, **-relief**, **-activeforeground**, **-disabledforeground**, **-takefocus**, **-background**, **-font**, **-activerelief**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-postcommand (postCommand/Command)**

Wenn diese Option angegeben ist, stellt sie einen Tcl-Befehl bereit, der jedes Mal ausgeführt wird, wenn das Menü gepostet wird. Der Befehl wird vom Widget-Befehl **post** aufgerufen, bevor das Menü gepostet wird. Beachten Sie, dass in Tk 8.0 auf Macintosh und Windows alle Post-Befehle in einem Menüsystem ausgeführt werden, bevor eines dieser Menüs gepostet wird. Dies liegt an den Einschränkungen der Menü-Manager der einzelnen Plattformen.

**-selectcolor (selectColor/Background)**

Für Menüeinträge, die Checkbuttons oder Radiobuttons sind, legt diese Option die Farbe fest, die im Indikator angezeigt wird, wenn der Checkbutton oder Radiobutton ausgewählt ist.

**-tearoff (tearOff/TearOff)**

Diese Option muss einen gültigen booleschen Wert haben (Standard ist false) und legt fest, ob das Menü oben einen Tear-off-Eintrag enthalten soll. Wenn ja, existiert er als Eintrag 0 des Menüs und die anderen Einträge werden ab 1 nummeriert. Die Standard-Menü-Bindings richten ein, dass das Menü abgerissen wird, wenn der Tear-off-Eintrag aufgerufen wird. Diese Option wird unter Aqua/macOS ignoriert, wo Menüs nicht abgerissen werden können.

**-tearoffcommand (tearOffCommand/TearOffCommand)**

Wenn diese Option einen nicht-leeren Wert hat, legt sie einen Tcl-Befehl fest, der aufgerufen wird, wann immer das Menü abgerissen wird. Der tatsächliche Befehl besteht aus dem Wert dieser Option, gefolgt von einem Leerzeichen, gefolgt vom Namen des Menüfensters, gefolgt von einem Leerzeichen, gefolgt vom Namen des abgerissenen Menüfensters. Wenn der Optionswert beispielsweise "**a b**" ist und das Menü **.x.y** abgerissen wird, um ein neues Menü **.x.tearoff1** zu erzeugen, wird der Befehl "**a b .x.y .x.tearoff1**" aufgerufen. Diese Option wird unter Aqua/macOS ignoriert, wo Menüs nicht abgerissen werden können.

**-title (title/Title)**

Der String wird als Titel für das Fenster verwendet, das erzeugt wird, wenn dieses Menü abgerissen wird. Wenn der Titel NULL ist, hat das Fenster den Titel des Menubuttons oder den Text des Cascade-Eintrags, von dem aus dieses Menü aufgerufen wurde.

**-type (type/Type)**

Diese Option kann einer von **menubar**, **tearoff** oder **normal** sein und wird beim Erzeugen des Menüs gesetzt. Obwohl sich der von der Konfigurationsdatenbank zurückgegebene String ändert, wenn diese Option geändert wird, beeinflusst dies das Verhalten des Menü-Widgets nicht. Dies wird vom Klon-Mechanismus verwendet und wird normalerweise nicht außerhalb der Tk-Bibliothek gesetzt.

## Einleitung

Der Befehl **menu** erstellt ein neues Toplevel-Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Menu-Widget. Dieses Menu-Widget kann entweder als Pop-up-Fenster verwendet oder auf ein toplevel (mit dessen Option **-menu**) angewendet werden, um es zur Menüleiste für dieses Toplevel zu machen. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des Menüs wie Farben und Schriftart zu konfigurieren. Der Befehl **menu** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Ein Menü ist ein Widget, das eine Sammlung einzeiliger Einträge anzeigt, die in einer oder mehreren Spalten angeordnet sind. Es gibt mehrere verschiedene Eintragstypen, jeweils mit unterschiedlichen Eigenschaften. Einträge verschiedener Typen können in einem einzigen Menü kombiniert werden. Menüeinträge sind nicht dasselbe wie Entry-Widgets. Tatsächlich sind Menüeinträge nicht einmal separate Widgets; das gesamte Menü ist ein Widget.

Menüeinträge werden mit bis zu drei getrennten Feldern angezeigt. Das Hauptfeld ist eine Beschriftung in Form eines Textstrings, eines Bitmaps oder eines Bildes, gesteuert durch die Optionen **-label**, **-bitmap** und **-image** für den Eintrag. Wenn für einen Eintrag die Option **-accelerator** angegeben ist, wird rechts neben der Beschriftung ein zweites Textfeld angezeigt. Der Accelerator beschreibt typischerweise eine Tastenkombination, die in der Anwendung verwendet werden kann, um dasselbe Ergebnis wie beim Aufrufen des Menüeintrags zu erzielen. Dies ist eine Anzeigeoption; sie legt das entsprechende Binding nicht tatsächlich fest (was mit dem Befehl bind erreicht werden kann). Das dritte Feld ist ein *Indikator*. Der Indikator ist nur bei Checkbutton- oder Radiobutton-Einträgen vorhanden. Er zeigt an, ob der Eintrag ausgewählt ist oder nicht, und wird links vom String des Eintrags angezeigt.

Bei normaler Verwendung wird ein Eintrag aktiv (stellt sich anders dar), sobald der Mauszeiger über dem Eintrag ist. Wenn eine Maustaste über dem Eintrag losgelassen wird, wird der Eintrag *aufgerufen*. Die Wirkung des Aufrufs ist für jeden Eintragstyp unterschiedlich; diese Wirkungen werden unten in den Abschnitten zu den einzelnen Einträgen beschrieben.

Einträge können *deaktiviert* werden, wodurch ihre Beschriftungen und Accelerators mit gedimmteren Farben angezeigt werden. Die Standard-Menü-Bindings erlauben nicht, dass ein deaktivierter Eintrag aktiviert oder aufgerufen wird. Deaktivierte Einträge können wieder aktiviert werden; dann ist es wieder möglich, sie zu aktivieren und aufzurufen.

Wann immer sich der aktive Eintrag eines Menüs ändert, wird ein virtuelles Event <<MenuSelect>> an das Menü gesendet. Der aktive Eintrag kann dann vom Menü abgefragt werden, und es kann eine Aktion ausgeführt werden, z. B. das Setzen kontextsensitiver Hilfetexte für den Eintrag.

## Eintragstypen

### Befehlseinträge

Die häufigste Art von Menüeintrag ist ein Befehlseintrag, der sich ähnlich wie ein Button-Widget verhält. Wenn ein Befehlseintrag aufgerufen wird, wird ein Tcl-Befehl ausgeführt. Der Tcl-Befehl wird mit der Option **-command** angegeben.

### Trenner-Einträge

Ein Trenner ist ein Eintrag, der als horizontale Trennlinie angezeigt wird. Ein Trenner kann nicht aktiviert oder aufgerufen werden und hat kein Verhalten außer seinem Erscheinungsbild.

### Checkbutton-Einträge

Ein Checkbutton-Menüeintrag verhält sich ähnlich wie ein Checkbutton-Widget. Wenn er aufgerufen wird, schaltet er zwischen ausgewähltem und abgewähltem Zustand hin und her. Wenn der Eintrag ausgewählt ist, wird ein bestimmter Wert in einer bestimmten globalen Variable gespeichert (wie durch die Optionen **-onvalue** und **-variable** für den Eintrag festgelegt); wenn der Eintrag abgewählt wird, wird ein anderer Wert (festgelegt durch die Option **-offvalue**) in der globalen Variable gespeichert. Links von der Beschriftung wird bei einem Checkbutton-Eintrag ein Indikator-Kästchen angezeigt. Wenn der Eintrag ausgewählt ist, wird die Mitte des Indikators in der durch die Option **-selectcolor** für den Eintrag angegebenen Farbe angezeigt; andernfalls wird die Mitte des Indikators in der Hintergrundfarbe des Menüs angezeigt. Wenn für einen Checkbutton-Eintrag eine Option **-command** angegeben ist, wird ihr Wert bei jedem Aufruf des Eintrags als Tcl-Befehl ausgewertet; dies geschieht nach dem Umschalten des ausgewählten Zustands des Eintrags.

### Radiobutton-Einträge

Ein Radiobutton-Menüeintrag verhält sich ähnlich wie ein Radiobutton-Widget. Radiobutton-Einträge sind in Gruppen organisiert, von denen jeweils nur ein Eintrag gleichzeitig ausgewählt sein kann. Wann immer ein bestimmter Eintrag ausgewählt wird, speichert er einen bestimmten Wert in einer bestimmten globalen Variable (wie durch die Optionen **-value** und **-variable** für den Eintrag festgelegt). Diese Aktion bewirkt, dass sich jeder zuvor ausgewählte Eintrag in derselben Gruppe selbst abwählt. Sobald ein Eintrag ausgewählt wurde, führt jede Änderung an der zugehörigen Variable des Eintrags dazu, dass der Eintrag sich selbst abwählt. Die Gruppierung von Radiobutton-Einträgen wird durch ihre zugehörigen Variablen bestimmt: Haben zwei Einträge dieselbe zugehörige Variable, gehören sie zur selben Gruppe. Links von der Beschriftung wird bei jedem Radiobutton-Eintrag ein Indikator in Rautenform angezeigt. Wenn der Eintrag ausgewählt ist, wird die Mitte des Indikators in der durch die Option **-selectcolor** für den Eintrag angegebenen Farbe angezeigt; andernfalls wird die Mitte des Indikators in der Hintergrundfarbe des Menüs angezeigt. Wenn für einen Radiobutton-Eintrag eine Option **-command** angegeben ist, wird ihr Wert bei jedem Aufruf des Eintrags als Tcl-Befehl ausgewertet; dies geschieht nach der Auswahl des Eintrags.

### Cascade-Einträge

Ein Cascade-Eintrag ist ein Eintrag mit einem zugehörigen Menü (festgelegt durch die Option **-menu**). Cascade-Einträge ermöglichen den Aufbau kaskadierender Menüs. Der Widget-Befehl **postcascade** kann verwendet werden, um das zugehörige Menü direkt neben dem Cascade-Eintrag zu posten und wieder zu entfernen. Das zugehörige Menü muss ein Kind des Menüs sein, das den Cascade-Eintrag enthält (dies ist nötig, damit die Menü-Traversierung korrekt funktioniert).

Ein Cascade-Eintrag postet sein zugehöriges Menü, indem er einen Tcl-Befehl der Form

```
menu post x y
```

aufruft, wobei *menu* der Pfadname des zugehörigen Menüs ist und *x* und *y* die Root-Fenster-Koordinaten der oberen rechten Ecke des Cascade-Eintrags sind. Unter Unix wird das untergeordnete Menü durch Ausführen eines Tcl-Befehls der Form

```
menu unpost
```

wieder entfernt, wobei *menu* der Name des zugehörigen Menüs ist. Auf anderen Plattformen kümmert sich der native Plattformcode um das Entfernen des Menüs.

Wenn für einen Cascade-Eintrag eine Option **-command** angegeben ist, wird sie bei jedem Aufruf des Eintrags als Tcl-Befehl ausgewertet. Dies wird unter Windows nicht unterstützt.

### Tear-off-Einträge

Ein Tear-off-Eintrag erscheint oben im Menü, wenn er mit der Option **-tearoff** aktiviert ist. Er ist nicht wie andere Menüeinträge: Er kann nicht mit dem Widget-Befehl **add** erzeugt und nicht mit dem Widget-Befehl **delete** gelöscht werden. Wenn ein Tear-off-Eintrag erzeugt wird, erscheint er als gestrichelte Linie oben im Menü. Unter den Standard-Bindings bewirkt der Aufruf des Tear-off-Eintrags, dass eine abgerissene Kopie des Menüs und all seiner Untermenüs erstellt wird.

## Menüleisten

Jedes Menü kann als Menüleiste für ein Toplevel-Fenster gesetzt werden (siehe Befehl toplevel für die Syntax). Auf dem Macintosh erscheinen die Cascade-Einträge dieses Menüs in der Menüleiste am oberen Rand des Hauptbildschirms, sobald das Toplevel im Vordergrund ist. Unter Windows und Unix werden die Einträge dieses Menüs in einer Menüleiste am oberen Rand des Fensters angezeigt. Diese Menüs verhalten sich gemäß den Schnittstellenrichtlinien ihrer Plattformen. Da macOS verlangt, dass für die aktive App immer eine Menüleiste angezeigt wird, wird ein Standard-Menü angezeigt, wenn für ein Toplevel kein Menü per **configure -menu** zugewiesen wurde und dieses Toplevel den Fokus hat. Für jedes als Menüleiste gesetzte Menü wird ein Klon-Menü erstellt. Siehe Abschnitt Klone für weitere Informationen.

Wie erwähnt, können Menüleisten auf verschiedenen Plattformen unterschiedlich wirken. Ein Beispiel betrifft die Behandlung von Checkbuttons und Radiobuttons innerhalb des Menüs. Obwohl es erlaubt ist, diese Menüelemente in Menüleisten zu verwenden, werden sie auf einigen Plattformen aufgrund von Systembeschränkungen möglicherweise nicht mit Indikatoren gezeichnet.

### Spezielle Menüs in Menüleisten

Bestimmte Menüs in einer Menüleiste werden speziell behandelt. Auf dem Macintosh wird Zugriff auf die speziellen Menüs Application, Window und Help bereitgestellt. Unter Windows wird Zugriff auf das Windows-Systemmenü in jedem Fenster bereitgestellt. Unter X Windows kann ein spezielles rechtsbündiges Hilfe-Menü bereitgestellt werden, wenn Motif-Menü-Kompatibilität aktiviert ist. In allen Fällen müssen diese Menüs mit dem Befehlsnamen des Menüleisten-Menüs erstellt werden, an den der spezielle Name angehängt ist. Für eine Menüleiste namens .menubar wären auf dem Macintosh die speziellen Menüs .menubar.apple, .menubar.window und .menubar.help; unter Windows .menubar.system; unter X Windows .menubar.help.

Wenn Tk auf dem Macintosh ein .menubar.apple-Menü als erstes Menü in einer Menüleiste sieht, bilden die Inhalte dieses Menüs die ersten Einträge des Application-Menüs, sobald das Fenster mit der Menüleiste im Vordergrund ist. Nach allen von Tk definierten Einträgen folgt im Menü ein Trenner und danach alle Standard-Einträge des Application-Menüs. Ein solches .apple-Menü muss vorhanden sein, wenn das Menü erstmals als Menüleiste eines Toplevel konfiguriert wird; andernfalls wird zu diesem Zeitpunkt ein Standard-Application-Menü (vor Tk verborgen) in die Menüleiste eingefügt, und ein später hinzugefügtes .apple-Menü wird nicht mehr zum Application-Menü.

Wenn Tk auf dem Macintosh ein .menubar.window-Menü sieht, werden die Inhalte des Menüs in das Standard-Window-Menü der Benutzer-Menüleiste eingefügt, sobald die Menüleiste des Fensters im Vordergrund ist. Die ersten Einträge im Menü stammen von macOS; die Namen der aktuellen Toplevels werden automatisch nach allen von Tk definierten Einträgen und einem Trenner angehängt. Das Window-Menü auf dem Mac erlaubt auch das Umschalten des Fensters in einen Vollbildzustand und die Verwaltung einer Tab-Fenster-Oberfläche (mehrere Fenster in einem Fenster gruppiert), sofern die Betriebssystemversion dies unterstützt.

Wenn Tk auf dem Macintosh ein .menubar.help-Menü sieht, werden die Inhalte des Menüs an das Standard-Hilfe-Menü der Benutzer-Menüleiste angehängt, sobald die Menüleiste des Fensters im Vordergrund ist. Die ersten Einträge im Menü stammen von macOS.

Wenn Tk unter Windows ein System-Menü sieht, werden seine Einträge an das Systemmenü angehängt, an das die Menüleiste gebunden ist. Dieses Menü ist mit dem Anwendungssymbol verknüpft und kann mit der Maus oder durch Alt+Leertaste aufgerufen werden. Aufgrund von Einschränkungen in der Windows-API erscheinen Schriftänderungen, Farben, Bilder, Bitmaps oder Tear-off-Bilder nicht im Systemmenü.

Wenn Tk unter X Windows ein Hilfe-Menü sieht und Motif-Menü-Kompatibilität aktiviert ist, wird das Menü ans Ende der Menüleiste verschoben und rechtsbündig ausgerichtet. Motif-Menü-Kompatibilität wird aktiviert, indem die Tk-Option ***Menu.useMotifHelp** auf true gesetzt wird.

## Klone

Wenn ein Menü als Menüleiste für ein Toplevel-Fenster gesetzt wird oder wenn ein Menü abgerissen wird, wird ein Klon des Menüs erstellt. Dieser Klon ist ein Menu-Widget für sich, ist aber ein Kind des Originals. Änderungen an der Konfiguration des Originals werden im Klon widergespiegelt. Zusätzlich werden alle angesprochenen Cascades ebenfalls geklont, damit die Menü-Traversierung korrekt funktioniert. Klone werden zerstört, wenn entweder der Tear-off oder die Menüleiste verschwindet oder wenn das ursprüngliche Menü zerstört wird.

## Widget-Befehl

Der Befehl **menu** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls.

Viele Widget-Befehle für ein Menü nehmen als ein Argument einen Indikator dafür an, auf welchen Eintrag des Menüs gearbeitet werden soll. Diese Indikatoren heißen *Index*e und können in einer der folgenden Formen angegeben werden:

**active**
: Bezeichnet den Eintrag, der derzeit aktiv ist. Wenn kein Eintrag aktiv ist, ist diese Form äquivalent zu **{}**. Diese Form darf nicht abgekürzt werden.

**end**
: Bezeichnet den untersten Eintrag im Menü. Wenn das Menü keine Einträge hat, ist diese Form äquivalent zu **{}**. Diese Form darf nicht abgekürzt werden.

**last**
: Entspricht **end**.

**{}**
: Bezeichnet "überhaupt keinen Eintrag"; wird am häufigsten mit der Option **activate** verwendet, um alle Einträge im Menü zu deaktivieren. In den meisten Fällen bewirkt die Angabe von **{}**, dass im Widget-Befehl nichts passiert.

**@*x*,*y***
: Bezeichnet den Eintrag, der den im Menüfenster durch *x* und *y* angegebenen Punkt (in Pixelkoordinaten) abdeckt. Deckt kein Eintrag diesen Punkt ab, ist diese Form äquivalent zu **{}**. Wird nur eine einzelne Zahl angegeben, wird sie als y-Koordinate behandelt.

***number***
: Bezeichnet den Eintrag numerisch, wobei 0 dem obersten Eintrag des Menüs entspricht, 1 dem darunter und so weiter.

***id***
: Wenn der Index keine der obigen Formen erfüllt, wird das Menü nach einem Eintrag mit der angegebenen *id* durchsucht.

***pattern***
: Wenn alle obigen Methoden zum Finden eines Eintrags fehlschlagen, wird diese Form verwendet. *Pattern* wird mit der Beschriftung jedes Eintrags im Menü von oben nach unten abgeglichen, bis ein passender Eintrag gefunden wird. Es gelten die Regeln von string match.

Wenn der Index mehr als eine der obigen Formen treffen könnte, hat die früher in der obigen Liste stehende Form Vorrang. Indizes unterstützen dieselbe einfache Interpretation wie beim Befehl string index, mit einfacher ganzzahliger Indexarithmetik und Indexierung relativ zu **end**.

Folgende Widget-Befehle sind für Menu-Widgets möglich:

***pathName* activate *index***
: Ändert den Zustand des durch *index* angegebenen Eintrags auf **active** und zeigt ihn mit seinen aktiven Farben neu an. Jeder zuvor aktive Eintrag wird deaktiviert. Wenn *index* als **{}** angegeben ist oder der angegebene Eintrag deaktiviert ist, endet das Menü ohne aktiven Eintrag. Gibt einen leeren String zurück.

***pathName* add *type* ?*id*? ?*option value option value ...*?**
: Fügt einen neuen Eintrag am Ende des Menüs hinzu. Der Typ des neuen Eintrags wird durch *type* angegeben und muss einer von **cascade**, **checkbutton**, **command**, **radiobutton** oder **separator** sein oder eine eindeutige Abkürzung davon. Wenn das Argument *id* angegeben ist, wird es als Eintragskennung verwendet; *id* darf im Menü noch nicht existieren. Andernfalls wird eine neue eindeutige Kennung erzeugt. Sind weitere Argumente vorhanden, legen sie die unten im Abschnitt Menüeintrags-Optionen aufgeführten Optionen fest. Der Widget-Befehl **add** gibt die *id* des neuen Eintrags zurück.

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **menu** akzeptierten Werte haben.

***pathName* clone *newPathname* ?*cloneType*?**

Erstellt einen Klon des aktuellen Menüs mit dem Namen *newPathName*. Dieser Klon ist ein Menü für sich, aber Änderungen am Klon werden auf das ursprüngliche Menü und umgekehrt übertragen. *cloneType* kann **normal**, **menubar** oder **tearoff** sein. Sollte normalerweise nicht außerhalb der Tk-Bibliothek aufgerufen werden. Siehe Abschnitt Klone für weitere Informationen.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **menu** akzeptierten Werte haben.

***pathName* delete *index1* ?*index2*?**

Löscht alle Menüeinträge zwischen *index1* und *index2* einschließlich. Wenn *index2* weggelassen wird, entspricht es *index1*. Versuche, einen Tear-off-Menüeintrag zu löschen, werden ignoriert (stattdessen sollte die Option **-tearoff** geändert werden, um den Tear-off-Eintrag zu entfernen).

***pathName* entrycget *index option***

Gibt den aktuellen Wert einer Konfigurationsoption für den durch *index* gegebenen Eintrag zurück. *Option* kann einen der unten im Abschnitt Menüeintrags-Optionen beschriebenen Namen haben.

***pathName* entryconfigure *index* ?*options...*?**

Dieser Befehl ähnelt dem Befehl **configure**, gilt jedoch für die Optionen eines einzelnen Eintrags, während **configure** für die Optionen des Menüs als Ganzes gilt. *Options* kann jeden der im Abschnitt Menüeintrags-Optionen beschriebenen Werte haben. Wenn *options* angegeben sind, werden die Optionen wie im Befehl angegeben geändert und der Befehl gibt einen leeren String zurück. Wenn keine *options* angegeben sind, gibt eine Liste zurück, die die aktuellen Optionen für Eintrag *index* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste).

***pathName* id *index***

Gibt die *id* des durch *index* gegebenen Menüeintrags zurück. Dies ist die Kennung, die dem Eintrag beim Erzeugen mit dem Widget-Befehl **add** oder **insert** zugewiesen wurde. Gibt einen leeren String für den Tear-off-Eintrag zurück oder wenn *index* äquivalent zu **{}** ist.

***pathName* index *index***

Gibt den numerischen Index zurück, der *index* entspricht, oder **{}**, wenn *index* als **{}** angegeben wurde.

***pathName* insert *index type* ?*id*? ?*option value option value* ...?**

Entspricht dem Widget-Befehl **add**, fügt den neuen Eintrag jedoch unmittelbar vor dem durch *index* gegebenen Eintrag ein, statt ans Ende des Menüs anzuhängen. Die Argumente *type*, *id*, *option* und *value* haben dieselbe Bedeutung wie beim Widget-Befehl **add**. Es ist nicht möglich, neue Menüeinträge vor dem Tear-off-Eintrag einzufügen, falls das Menü einen hat. Der Widget-Befehl **insert** gibt die *id* des neuen Eintrags zurück.

***pathName* invoke *index***

Ruft die Aktion des Menüeintrags auf. Siehe die Abschnitte zu den einzelnen Einträgen oben für Details. Wenn der Menüeintrag deaktiviert ist, passiert nichts. Hat der Eintrag einen zugehörigen Befehl, wird das Ergebnis dieses Befehls als Ergebnis des Widget-Befehls **invoke** zurückgegeben. Andernfalls ist das Ergebnis ein leerer String. Hinweis: Das Aufrufen eines Menüeintrags entfernt das Menü nicht automatisch; die Standard-Bindings kümmern sich normalerweise darum, bevor **invoke** ausgeführt wird.

***pathName* post *x y* ?*index*?**

Sorgt dafür, dass das Menü an den Root-Fenster-Koordinaten *x* und *y* auf dem Bildschirm angezeigt wird. Wenn ein Index angegeben ist, wird das Menü so positioniert, dass der Eintrag mit diesem Index an dem Punkt angezeigt wird. Diese Koordinaten werden bei Bedarf angepasst, damit das gesamte Menü auf dem Bildschirm sichtbar ist. Dieser Befehl gibt normalerweise einen leeren String zurück. Wurde die Option **-postcommand** angegeben, wird ihr Wert als Tcl-Script vor dem Posten des Menüs ausgeführt und das Ergebnis dieses Scripts als Ergebnis des Widget-Befehls **post** zurückgegeben. Tritt beim Ausführen des Befehls ein Fehler auf, wird der Fehler zurückgegeben, ohne das Menü zu posten.

***pathName* postcascade *index***

Postet das Untermenü, das dem durch *index* gegebenen Cascade-Eintrag zugeordnet ist, und entfernt jedes zuvor gepostete Untermenü. Entspricht *index* keinem Cascade-Eintrag oder ist *pathName* nicht gepostet, hat der Befehl keine Wirkung außer das Entfernen eines aktuell geposteten Untermenüs.

***pathName* type *index***

Gibt den Typ des durch *index* gegebenen Menüeintrags zurück. Dies ist das *type*-Argument, das beim Erzeugen des Eintrags an **add** oder **insert** übergeben wurde, z. B. **command** oder **separator**, oder **tearoff** für einen Tear-off-Eintrag.

***pathName* unpost**

Entfernt die Zuordnung des Fensters, sodass es nicht mehr angezeigt wird. Ist ein kaskadiertes Untermenü auf niedrigerer Ebene gepostet, wird dieses entfernt. Gibt einen leeren String zurück. Dieser Unterbefehl funktioniert unter Windows und auf dem Macintosh nicht, da diese Plattformen ihr eigenes Entfernen von Menüs haben.

***pathName* xposition *index***

Gibt einen Dezimalstring zurück mit der x-Koordinate im Menüfenster des linken Pixel des durch *index* angegebenen Eintrags.

***pathName* yposition *index***

Gibt einen Dezimalstring zurück mit der y-Koordinate im Menüfenster des obersten Pixel des durch *index* angegebenen Eintrags.

## Menüeintrags-Optionen

Die folgenden Optionen sind für Menüeinträge erlaubt. Die meisten Optionen werden nicht von allen Eintragstypen unterstützt.

**-activebackground *value***

Legt eine Hintergrundfarbe fest, die für die Anzeige dieses Eintrags verwendet wird, wenn er aktiv ist. Diese Option wird unter Aqua/macOS ignoriert. Ist sie als leerer String angegeben (Standard), wird die Option **-activebackground** für das gesamte Menü verwendet. Wurde die Variable tk_strictMotif gesetzt, um strikte Motif-Konformität zu verlangen, wird diese Option ignoriert und stattdessen die Option **-background** verwendet. Diese Option ist für Trenner- oder Tear-off-Einträge nicht verfügbar.

**-activeforeground *value***

Legt eine Vordergrundfarbe fest, die für die Anzeige dieses Eintrags verwendet wird, wenn er aktiv ist. Diese Option wird unter Aqua/macOS ignoriert. Ist diese Option als leerer String angegeben (Standard), wird die Option **-activeforeground** für das gesamte Menü verwendet.

**-accelerator *value***

Legt einen String fest, der rechts vom Menüeintrag angezeigt wird. Beschreibt normalerweise eine Accelerator-Tastenkombination, mit der dieselbe Funktion wie der Menüeintrag aufgerufen werden kann. Dies ist eine Anzeigeoption; sie legt das entsprechende Binding nicht tatsächlich fest (was mit dem Befehl bind erreicht werden kann). Diese Option ist für Trenner- oder Tear-off-Einträge nicht verfügbar.

**-background *value***

Legt eine Hintergrundfarbe fest, die für die Anzeige dieses Eintrags im normalen Zustand (weder aktiv noch deaktiviert) verwendet wird. Diese Option wird unter Aqua/macOS ignoriert. Ist sie als leerer String angegeben (Standard), wird die Option **-background** für das gesamte Menü verwendet. Diese Option ist für Trenner- oder Tear-off-Einträge nicht verfügbar.

**-bitmap *value***

Legt ein Bitmap fest, das im Menü statt einer Textbeschriftung angezeigt wird, in einer von Tk_GetBitmap akzeptierten Form. Diese Option überschreibt die Option **-label** (gesteuert durch **-compound**), kann aber auf einen leeren String zurückgesetzt werden, um eine Textbeschriftung anzuzeigen. Wurde eine Option **-image** angegeben, überschreibt sie **-bitmap**. Diese Option ist für Trenner- oder Tear-off-Einträge nicht verfügbar.

**-columnbreak *value***

Ist diese Option null, erscheint der Eintrag unter dem vorherigen Eintrag. Ist sie eins, erscheint der Eintrag oben in einer neuen Spalte im Menü. Diese Option wird unter Aqua/macOS ignoriert, wo Menüs immer eine Spalte haben.

**-command *value***

Legt einen Tcl-Befehl fest, der ausgeführt wird, wenn der Menüeintrag aufgerufen wird. Nicht verfügbar für Trenner- oder Tear-off-Einträge.

**-compound *value***

Legt fest, ob der Menüeintrag sowohl ein Bild als auch Text anzeigen soll und falls ja, wo das Bild relativ zum Text platziert wird. Gültige Werte sind **bottom**, **center**, **left**, **none**, **right** und **top**. Der Standardwert ist **none**, d. h. es wird entweder ein Bild oder Text angezeigt, je nach den Werten der Optionen **-image** und **-bitmap**.

**-font *value***

Legt die Schriftart fest, mit der die Beschriftung oder der Accelerator-String in diesem Eintrag gezeichnet wird. Ist diese Option als leerer String angegeben (Standard), wird die Option **-font** für das gesamte Menü verwendet. Diese Option ist für Trenner- oder Tear-off-Einträge nicht verfügbar.

**-foreground *value***
Legt eine Vordergrundfarbe fest, die für die Anzeige dieses Eintrags im normalen Zustand verwendet wird. Diese Option wird unter Aqua/macOS ignoriert. Ist sie als leerer String angegeben (Standard), wird die Option **-foreground** für das gesamte Menü verwendet. Diese Option ist für Trenner- oder Tear-off-Einträge nicht verfügbar.

**-hidemargin *value***
Legt fest, ob die Standardränder für diesen Menüeintrag gezeichnet werden sollen. Nützlich beim Erzeugen von Paletten mit Bildern, z. B. Farb- oder Musterpaletten. 1 bedeutet, dass der Rand für den Eintrag verborgen ist; 0 bedeutet, dass der Rand verwendet wird.

**-image *value***
Legt ein Bild fest, das im Menü statt eines Textstrings oder Bitmaps angezeigt wird. Das Bild muss durch einen früheren Aufruf von image create erzeugt worden sein. Diese Option überschreibt die Optionen **-label** und **-bitmap** (gesteuert durch **-compound**), kann aber auf einen leeren String zurückgesetzt werden, um eine Text- oder Bitmap-Beschriftung anzuzeigen. Diese Option ist für Trenner- oder Tear-off-Einträge nicht verfügbar.

**-indicatoron *value***
Nur für Checkbutton- und Radiobutton-Einträge verfügbar. *Value* ist ein boolescher Wert, der festlegt, ob der Indikator angezeigt werden soll.

**-label *value***
Legt einen String fest, der als identifizierende Beschriftung im Menüeintrag angezeigt wird. Nicht verfügbar für Trenner- oder Tear-off-Einträge.

**-menu *value***
Nur für Cascade-Einträge verfügbar. Legt den Pfadnamen des mit diesem Eintrag verknüpften Untermenüs fest. Das Untermenü muss ein Kind des Menüs sein.

**-offvalue *value***
Nur für Checkbutton-Einträge verfügbar. Legt den Wert fest, der in der zugehörigen Variable des Eintrags gespeichert wird, wenn der Eintrag abgewählt wird.

**-onvalue *value***
Nur für Checkbutton-Einträge verfügbar. Legt den Wert fest, der in der zugehörigen Variable des Eintrags gespeichert wird, wenn der Eintrag ausgewählt wird.

**-selectcolor *value***
Nur für Checkbutton- und Radiobutton-Einträge verfügbar. Legt die Farbe fest, die im Indikator angezeigt wird, wenn der Eintrag ausgewählt ist. Ist der Wert ein leerer String (Standard), bestimmt die Option **-selectcolor** des Menüs die Indikatorfarbe.

**-selectimage *value***
Nur für Checkbutton- und Radiobutton-Einträge verfügbar. Legt ein Bild fest, das im Eintrag angezeigt wird (anstelle der Option **-image**), wenn er ausgewählt ist. *Value* ist der Name eines Bildes, das durch einen früheren Aufruf von image create erzeugt worden sein muss. Diese Option wird ignoriert, sofern die Option **-image** nicht angegeben wurde.

**-state *value***
Legt einen von drei Zuständen für den Eintrag fest: **normal**, **active** oder **disabled**. Im normalen Zustand wird der Eintrag mit der Option **-foreground** des Menüs und der Option **-background** des Eintrags oder des Menüs angezeigt. Der aktive Zustand wird typischerweise verwendet, wenn der Zeiger über dem Eintrag ist. Im aktiven Zustand werden **-activeforeground** des Menüs und **-activebackground** des Eintrags verwendet. Der

deaktivierte Zustand bedeutet, dass der Eintrag unempfindlich sein soll: Die Standard-Bindings weigern sich, ihn zu aktivieren oder aufzurufen. In diesem Zustand werden **-disabledforeground** des Menüs und **-background** des Eintrags verwendet. Diese Option ist für Trenner-Einträge nicht verfügbar.

**-underline *value***

Legt den ganzzahligen Index eines zu unterstreichenden Zeichens im Eintrag fest. Diese Option wird auch von den Standard-Bindings abgefragt und für die Tastatur-Traversierung verwendet. 0 entspricht dem ersten Zeichen des im Eintrag angezeigten Textes, 1 dem nächsten und so weiter. Wird im Eintrag ein Bitmap oder Bild angezeigt, wird diese Option ignoriert. Diese Option ist für Trenner- oder Tear-off-Einträge nicht verfügbar.

**-value *value***

Nur für Radiobutton-Einträge verfügbar. Legt den Wert fest, der in der zugehörigen Variable des Eintrags gespeichert wird, wenn der Eintrag ausgewählt ist. Wird ein leerer String angegeben, wird die Option **-label** des Eintrags als in der Variable zu speichernder Wert verwendet.

**-variable *value***

Nur für Checkbutton- und Radiobutton-Einträge verfügbar. Legt den Namen einer globalen Variable fest, die gesetzt wird, wenn der Eintrag ausgewählt wird. Bei Checkbutton-Einträgen wird die Variable auch gesetzt, wenn der Eintrag abgewählt wird. Bei Radiobutton-Einträgen bewirkt eine Änderung der Variable, dass sich der aktuell ausgewählte Eintrag abwählt.

Bei Checkbutton-Einträgen wird der Standardwert dieser Option aus der Option **-label** übernommen; bei Radiobutton-Einträgen wird ein einzelner fester Wert verwendet. Es wird empfohlen, die Option **-variable** beim Erzeugen eines Checkbuttons oder Radiobuttons immer zu setzen.

## Menü-Konfigurationen

Die Standard-Bindings unterstützen vier verschiedene Arten der Menüverwendung:

### Pulldown-Menüs in der Menüleiste

Dies ist der häufigste Fall. Sie erzeugen ein Menu-Widget, das die Menüleiste wird. Sie fügen diesem Menü Cascade-Einträge hinzu und geben die Pulldown-Menüs an, die Sie in der Menüleiste verwenden möchten. Anschließend erzeugen Sie alle Pulldowns. Danach geben Sie das Menü mit der Option **-menu** des Widget-Befehls des Toplevel an. Siehe die Manpage toplevel für Details.

### Pulldown-Menüs in Menubuttons

Dies ist der kompatible Weg für Menüleisten. Sie erzeugen ein Menubutton-Widget für jedes Top-Level-Menü und ordnen typischerweise eine Reihe von Menubuttons in einem Menüleisten-Fenster an. Sie erzeugen auch die Top-Level-Menüs und kaskadierten Untermenüs und verknüpfen sie mit **-menu**-Optionen in Menubuttons und Cascade-Menüeinträgen. Das Top-Level-Menü muss ein Kind des Menubuttons sein, und jedes Untermenü muss ein Kind des Menüs sein, das darauf verweist. Danach erlauben die Standard-Bindings den Benutzern, den Menübaum über den Menubutton zu traversieren und aufzurufen; siehe die Manpage menubutton für Details.

### Pop-up-Menüs

Pop-up-Menüs werden typischerweise als Reaktion auf einen Mausklick oder Tastendruck gepostet. Sie erzeugen die Pop-up-Menüs und kaskadierten Untermenüs und rufen zur passenden Zeit tk_popup auf, um das Top-Level-Menü zu posten.

**Optionsmenüs**

Ein Optionsmenü besteht aus einem Menubutton mit einem zugehörigen Menü, mit dem einer von mehreren Werten ausgewählt werden kann. Der aktuelle Wert wird im Menubutton angezeigt und in einer globalen Variable gespeichert. Verwenden Sie die Prozedur tk_optionMenu, um Options-Menubuttons und ihre Menüs zu erzeugen.

**Abgerissene Menüs**

Ein abgerissenes Menü erzeugen Sie, indem Sie den Tear-off-Eintrag oben in einem bestehenden Menü aufrufen. Die Standard-Bindings erzeugen ein neues Menü als Kopie des Originals und lassen es dauerhaft als Toplevel-Fenster gepostet. Das abgerissene Menü verhält sich wie das Originalmenü.

## Standard-Bindings

Tk erstellt automatisch Klassen-Bindings für Menüs, die folgendes Standardverhalten geben:

1. Wenn die Maus ein Menü betritt, wird der Eintrag unter dem Mauszeiger aktiviert; bewegt sich die Maus im Menü, ändert sich der aktive Eintrag entsprechend.
2. Wenn die Maus ein Menü verlässt, werden alle Einträge im Menü deaktiviert, außer im Spezialfall, wenn die Maus von einem Menü zu einem kaskadierten Untermenü wechselt.
3. Wird eine Taste über einem Menü losgelassen, wird der aktive Eintrag (falls vorhanden) aufgerufen. Das Menü wird auch entfernt, sofern es kein abgerissenes Menü ist.
4. Die Tasten Leertaste und Return rufen den aktiven Eintrag auf und entfernen das Menü.
5. Haben Einträge im Menü mit der Option **-underline** unterstrichene Buchstaben, ruft das Drücken eines dieser Buchstaben (oder der Groß-/Kleinschreibungsvariante) den Eintrag auf und entfernt das Menü.
6. Die Escape-Taste bricht eine laufende Menüauswahl ab, ohne einen Eintrag aufzurufen. Sie entfernt das Menü auch, sofern es kein abgerissenes Menü ist.
7. Die Tasten Auf und Ab aktivieren den nächsthöheren bzw. nächsttieferen Eintrag im Menü. Am Ende des Menüs springt der aktive Eintrag zum anderen Ende.
8. Die linke Pfeiltaste wechselt zum nächsten Menü links. Ist das aktuelle Menü ein kaskadiertes Untermenü, wird das Untermenü entfernt und der aktuelle Menüeintrag wird zum Cascade-Eintrag im Eltern-Menü. Ist das aktuelle Menü ein von einem Menubutton gepostetes Top-Level-Menü, wird der aktuelle Menubutton entfernt und der nächste Menubutton links gepostet. Andernfalls hat die Taste keine Wirkung. Die Links-Rechts-Reihenfolge der Menubuttons folgt ihrer Stapelreihenfolge: Tk nimmt an, dass der unterste Menubutton (standardmäßig der zuerst erzeugte) links ist.
9. Die rechte Pfeiltaste wechselt zum nächsten Menü rechts. Ist der aktuelle Eintrag ein Cascade-Eintrag, wird das Untermenü gepostet und der aktuelle Menüeintrag wird zum ersten Eintrag im Untermenü. War das aktuelle Menü von einem Menubutton gepostet, wird der aktuelle Menubutton entfernt und der nächste Menubutton rechts gepostet.

Deaktivierte Menüeinträge reagieren nicht: Sie werden nicht aktiviert und ignorieren Mausklicks.

Mehrere Bindings verwenden den Befehl **tk_menuSetFocus**. Er speichert den aktuellen Fokus und setzt den Fokus auf sein *pathName*-Argument, ein Menu-Widget.

Das Verhalten von Menüs kann geändert werden, indem neue Bindings für einzelne Widgets definiert oder die Klassen-Bindings neu definiert werden.

## Bugs

Derzeit ist es nicht möglich, die Optionsdatenbank zu verwenden, um Werte für die Optionen einzelner Einträge festzulegen.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# menubutton

*Stichwörter:* menubutton, widget

## Name

menubutton - Erstellt und manipuliert Menubutton-Widgets für Popup-Menüs

## Übersicht

**menubutton** *pathName* ?*options*?

## Standardoptionen

**-activebackground**, **-disabledforeground**, **-padx**, **-activeforeground**, **-font**, **-pady**, **-anchor**, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-takefocus**, **-bitmap**, **-highlightcolor**, **-text**, **-borderwidth**, **-highlightthickness**, **-textvariable**, **-cursor**, **-image**, **-underline**, **-compound**, **-justify**, **-wraplength**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-direction (direction/Direction)**

Gibt an, wo das Menü geöffnet wird. **above** öffnet oberhalb des Menubuttons, **below** darunter, **left** links, **right** rechts, flush direkt über dem Menubutton. Bei **above** oder **below** wird die Richtung umgekehrt, wenn das Menü sonst außerhalb des Bildschirms läge.

**-height (height/Height)**

Gewünschte Höhe des Menubuttons. Bei Bild oder Bitmap in Bildschirmeinheiten (Formen von Tk_GetPixels); bei Text in Textzeilen. Ohne Angabe: Höhe aus angezeigtem Bild/Bitmap/Text.

**-indicatoron (indicatorOn/IndicatorOn)**

Muss ein gültiger boolescher Wert sein. Ist er wahr, wird rechts ein kleines Indikator-Rechteck gezeichnet und die Standard-Menü-Bindings behandeln den Menubutton als Options-Menubutton. Ist er falsch, wird kein Indikator angezeigt.

**-menu (menu/MenuName)**

Pfadname des zugehörigen Menüs. Das Menü muss ein Kind des Menubuttons sein.

**-state (state/State)**

Einer von drei Zuständen: **normal**, **active** oder **disabled**. Im Zustand **normal** Anzeige mit **foreground** und **background**. **active** typischerweise bei Maus über dem Widget, mit **-activeforeground** und **-activebackground**. **disabled**: Widget unempfindlich; Standard-Bindings aktivieren es nicht und ignorieren Mausklicks; **-disabledforeground** und **-background** bestimmen die Darstellung.

**-width (width/Width)**

Gewünschte Breite. Bei Bild/Bitmap in Bildschirmeinheiten (Formen von Tk_GetPixels); bei Text in Zeichen. Ohne Angabe: Breite aus angezeigtem Inhalt.

## Beschreibung

Der Befehl **menubutton** erstellt ein neues Fenster (*pathName*) und macht es zu einem Menubutton-Widget. Weitere Optionen können in der Befehlszeile oder Optionsdatenbank gesetzt werden (Farben, Schrift, Text, anfängliches Relief). **menubutton** liefert *pathName*. Beim Aufruf darf kein Fenster *pathName* existieren; das Eltern-Element muss existieren.

Ein Menubutton zeigt Text, Bitmap oder Bild und ist mit einem Menü-Widget verknüpft. Text in einer Schrift, mehrzeilig möglich (Zeilenumbrüche oder **-wraplength**); ein Zeichen kann mit **-underline** unterstrichen werden. Üblicherweise öffnet Maustaste 1 das zugehörige Menü unter dem Menubutton. Wird die Maus vor dem Loslassen über das Menü bewegt, wird der Eintrag unter dem Cursor aufgerufen; danach wird das Menü geschlossen.

Menubuttons bilden tk_optionMenu — auf macOS die bevorzugte Auswahl aus einer Liste.

Früher wurden Menubuttons zu Menüleisten gruppiert (Scanning): Bleibt die Taste gedrückt und die Maus wechselt zu einem anderen Menubutton derselben Leiste, schließt sich das erste Menü und das neue öffnet sich. *Diese Nutzung ist veraltet*; stattdessen menu direkt als Menüleiste setzen (Option **-menu** von toplevel).

Wechselwirkungen zwischen Menubuttons und Menüs: siehe menu (Pulldown-, Options-Menüs usw.).

## Widget-Befehl

**menubutton** erzeugt einen Tcl-Befehl *pathName* mit folgender Form:

```
pathName option ?arg ...?
```

*Option* und *arg* bestimmen das Verhalten. Für Menubutton-Widgets möglich:

***pathName* cget *option***

Aktueller Wert der Konfigurationsoption *option* (alle von **menubutton** akzeptierten Werte).

***pathName* configure ?*option*? ?*value option value* ...?**

Optionen abfragen oder ändern. Ohne *option*: Liste aller Optionen (siehe Tk_ConfigureInfo). Nur *option*: Liste für diese Option. Mit *option*-*value*-Paaren: Optionen setzen, leerer String als Rückgabe. *option*: alle von **menubutton** akzeptierten Werte.

## Standard-Bindings

Tk legt Klassen-Bindings für Menubuttons an:

1. Aktivierung bei Maus über dem Widget, Deaktivierung beim Verlassen.
2. Maustaste 1 über einem Menubutton öffnet ihn (Relief erhaben) und postet das Menü darunter. Wird bei gedrückter Taste ins Menü gezogen und dort losgelassen, schließt sich der Menubutton und der Eintrag wird aufgerufen.
3. Taste 1 drücken und auf demselben Menubutton loslassen: Menubutton bleibt geöffnet; Maus über Menü, Klick auf Eintrag zum Aufruf; danach schließt sich der Menubutton.
4. Taste 1 auf einem Menubutton, Ziehen zu anderem Menubutton: erster schließt sich, zweiter öffnet sich.
5. Taste 1 auf Menubutton, Loslassen außerhalb von Menubutton/Menü: schließen ohne Aufruf.
6. Geöffneter Menubutton: zugehöriges Menü übernimmt Eingabefokus für Tastatur-Navigation (siehe menu).
7. Mit **-underline**: Alt+*x* (*x* = unterstrichenes Zeichen oder Groß-/Kleinschreibung) in jedem

Fenster unter dem Toplevel des Menubuttons öffnet ihn.
8. F10 in jedem Fenster öffnet den ersten nicht deaktivierten Menubutton unter dessen Toplevel.
9. Hat der Menubutton den Fokus, öffnen Leertaste und Return ihn.

Ist der Zustand **disabled**, trifft nichts davon zu — der Menubutton reagiert nicht.

Das Verhalten lässt sich durch neue Bindings pro Widget oder durch Neudefinition der Klassen-Bindings ändern.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# message

*Stichwörter:* message, widget

## Name

message - Erstellt und manipuliert 'Message'-Widgets für nicht-interaktiven Text

## Übersicht

**message** *pathName* ?*options*?

## Standardoptionen

**-anchor**, **-background**, **-borderwidth**, **-cursor**, **-font**, **-foreground**, **-highlightbackground**, **-highlightcolor**, **-highlightthickness**, **-padx**, **-pady**, **-relief**, **-takefocus**, **-text**, **-textvariable**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-aspect (aspect/Aspect)**

Legt einen nicht-negativen Ganzzahlwert fest, der das gewünschte Seitenverhältnis für den Text angibt. Das Seitenverhältnis wird als 100*Breite/Höhe angegeben. 100 bedeutet, dass der Text so breit wie hoch sein soll, 200 bedeutet, dass der Text doppelt so breit wie hoch sein soll, 50 bedeutet, dass der Text doppelt so hoch wie breit sein soll usw. Wird verwendet, um die Zeilenlänge für Text zu wählen, wenn die Option **-width** nicht angegeben ist. Standardwert ist 150.

**-justify (justify/Justify)**

Legt fest, wie Textzeilen ausgerichtet werden. Muss einer von **left**, **center** oder **right** sein. Standardwert ist **left**. Diese Option wirkt zusammen mit den Optionen **-anchor**, **-aspect**, **-padx**, **-pady** und **-width**, um verschiedene Anordnungen des Textes im Fenster zu ermöglichen. Die Optionen **-aspect** und **-width** bestimmen, wie viel Bildschirmplatz zum Anzeigen des Textes benötigt wird. Die Optionen **-anchor**, **-padx** und **-pady** bestimmen, wo dieser rechteckige Bereich im Fenster des Widgets angezeigt wird, und die Option **-justify** bestimmt, wie jede Zeile innerhalb dieses rechteckigen Bereichs angezeigt wird. Angenommen beispielsweise, **-anchor** ist **e** und **-justify** ist **left**, und das Message-Fenster ist viel größer als für den Text benötigt. Der Text wird so angezeigt, dass die linken Kanten aller Zeilen auf einer Linie liegen und die rechte Kante der längsten Zeile **-padx** vom rechten Rand des Fensters entfernt ist; der gesamte Textblock wird in der vertikalen Spanne des Fensters zentriert.

**-width (width/Width)**

Legt die Länge der Zeilen im Fenster fest. Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben. Wenn diese Option einen Wert größer als null hat, wird die Option **-aspect** ignoriert und die Option **-width** bestimmt die Zeilenlänge. Wenn dieser Optionswert negativ oder null ist, bestimmt die Option **-aspect** die Zeilenlänge.

## Beschreibung

Der Befehl **message** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Message-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte der Message wie Farben, Schriftart, Text und anfängliches Relief zu konfigurieren. Der Befehl **message** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Eine Message ist ein Widget, das einen Text-String anzeigt. Ein Message-Widget hat drei besondere Merkmale, die es von einem label-Widget unterscheiden. Erstens teilt es seinen String in Zeilen auf, um ein bestimmtes Seitenverhältnis für das Fenster zu erzeugen. Die Zeilenumbrüche werden nach Möglichkeit an Wortgrenzen gewählt (wenn nicht einmal ein einzelnes Wort in eine Zeile passt, wird das Wort über Zeilen hinweg geteilt). Zeilenumbruchzeichen im String erzwingen Zeilenumbrüche; sie können beispielsweise verwendet werden, um Leerzeilen in der Anzeige zu hinterlassen.

Das zweite Merkmal eines Message-Widgets ist die Ausrichtung. Der Text kann linksbündig angezeigt werden (jede Zeile beginnt am linken Rand des Fensters), zeilenweise zentriert oder rechtsbündig (jede Zeile endet am rechten Rand des Fensters).

Das dritte Merkmal eines Message-Widgets ist, dass Steuerzeichen und nicht druckbare Zeichen besonders behandelt werden. Tabulatorzeichen werden durch genügend Leerzeichen ersetzt, um an der nächsten 8-Zeichen-Grenze ausgerichtet zu sein. Zeilenumbrüche verursachen Zeilenumbrüche. Andere Steuerzeichen (ASCII-Code kleiner als 0x20) und in der Schriftart nicht definierte Zeichen werden als vierzeichenige Sequenz **\x***hh* angezeigt, wobei *hh* die zweistellige hexadezimale Zahl ist, die dem Zeichen entspricht. Im ungewöhnlichen Fall, dass die Schriftart nicht alle Zeichen in "0123456789abcdef\x" enthält, werden Steuerzeichen und undefinierte Zeichen überhaupt nicht angezeigt.

## Widget-Befehl

Der Befehl **message** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Message-Widgets möglich:

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **message** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option-value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **message** akzeptierten Werte haben.

## Standard-Bindings

Wenn eine neue Message erstellt wird, hat sie keine Standard-Event-Bindings: Messages sind nur für Ausgabe-Zwecke gedacht.

## Fehler

Tabulatoren funktionieren nicht sehr gut mit zentriertem oder rechtsbündigem Text. Das häufigste Ergebnis ist, dass die Zeile falsch ausgerichtet wird.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# messageBox

*Stichwörter:* messageBox, message box

## Name

tk_messageBox - Blendet ein Meldungsfenster ein und wartet auf die Benutzerantwort

## Übersicht

**tk_messageBox** ?*option value* ...?

## Beschreibung

Diese Prozedur erstellt und zeigt ein Meldungsfenster mit einer anwendungsspezifischen Nachricht, einem Symbol und einer Menge von Buttons. Jeder Button im Meldungsfenster wird durch einen eindeutigen symbolischen Namen identifiziert (siehe die Option **-type**). Nach dem Einblenden wartet **tk_messageBox**, bis der Benutzer einen der Buttons wählt. Anschließend gibt sie den symbolischen Namen des gewählten Buttons zurück.

Die folgenden Option-Wert-Paare werden unterstützt:

**-command *string***

Legt das Präfix eines Tcl-Befehls fest, der aufgerufen wird, wenn der Benutzer den Dialog schließt. Der tatsächliche Befehl besteht aus *string*, gefolgt von einem Leerzeichen und dem Namen des Buttons, mit dem der Benutzer den Dialog geschlossen hat. Nur unter macOS verfügbar.

**-default *name***

*name* gibt den symbolischen Namen des Standard-Buttons für dieses Meldungsfenster an ("ok", "cancel" usw.). Eine Liste der symbolischen Namen siehe **-type**. Ist diese Option nicht angegeben, wird der erste Button im Dialog zum Standard.

**-detail *string***

Legt eine Zusatznachricht zur Hauptnachricht der Option **-message** fest. Die Detailnachricht wird unter der Hauptnachricht angezeigt und — wo das Betriebssystem es unterstützt — in einer weniger betonten Schriftart als die Hauptnachricht.

**-icon *iconImage***

Legt ein anzuzeigendes Symbol fest. *iconImage* muss einer der folgenden Werte sein: **error**, **info**, **question** oder **warning**. Ist diese Option nicht angegeben, wird das Info-Symbol angezeigt.

**-message *string***

Legt die in dieser Meldungsbox anzuzeigende Nachricht fest. Standard ist ein leerer String.

**-parent *window***

Macht *window* zum logischen Elternfenster der Meldungsbox. Die Meldungsbox wird über ihrem Elternfenster angezeigt.

**-title *titleString***

Legt einen String als Titel der Meldungsbox fest. Standard ist ein leerer String.

**-type *predefinedType***

Ordnet eine vordefinierte Menge von Buttons an. Folgende Werte sind für *predefinedType* möglich:

**abortretryignore**

Zeigt drei Buttons mit den symbolischen Namen **abort**, **retry** und **ignore**.

**ok**

Zeigt einen Button mit dem symbolischen Namen **ok**. Das ist der Standard.

**okcancel**

Zeigt zwei Buttons mit den symbolischen Namen **ok** und **cancel**.

**retrycancel**

Zeigt zwei Buttons mit den symbolischen Namen **retry** und **cancel**.

**yesno**

Zeigt zwei Buttons mit den symbolischen Namen **yes** und **no**.

**yesnocancel**

Zeigt drei Buttons mit den symbolischen Namen **yes**, **no** und **cancel**.

## Beispiel

```
set answer [tk_messageBox -message "Really quit?" \
        -icon question -type yesno \
        -detail "Select \"Yes\" to make the application exit"]
switch -- $answer {
    yes exit
    no {tk_messageBox -message "I know you like this application!" \
            -type ok}
}
```

*Copyright (c) 1996 Sun Microsystems, Inc.*

# nsimage

*Stichwörter:* nsimage, height, image, types of images, width

## Name

nsimage - Tk-Bildtyp für macOS auf Basis der NSImage-Klasse

## Beschreibung

**nsimage** ist als Tk-Bildtyp implementiert. Der Befehl image erstellt, löscht und fragt alle Bilder ab, einschließlich Bilder vom Typ **nsimage**. Die verfügbaren Optionen sind spezifisch für den Typ **nsimage** und werden unten beschrieben.

Zum Erstellen eines **nsimage**:

**image create nsimage ?*name*? ?*option value ...*?**
  siehe unten ...

Erstellt ein neues **nsimage** und einen Befehl gleichen Namens; gibt dessen Namen zurück.

## Optionen

Gültige *options* sind:

**-source *string***
  siehe unten ...

Der Wert der Option **-source** ist eine Zeichenkette, die ein NSImage beschreibt. Es gibt mehrere Interpretationsmöglichkeiten; die Interpretation wird durch den Wert der Option **-as** bestimmt. Diese Option ist erforderlich.

**-as *type***
  siehe unten ...

Es gibt vier mögliche Werte für die Option **-as**, die festlegen, wie die Quellzeichenkette interpretiert werden soll. Die zulässigen Werte und ihre Bedeutungen sind:

**name**
  Die Quelle wird als Name eines vom System bereitgestellten benannten NSImage interpretiert. Dies ist der Standard, wenn die Option **-as** nicht angegeben ist.

file
  Die Quelle wird als Pfad zu einer Bilddatei in einem von der NSImage-Klasse unterstützten Format interpretiert.

**path**
  Die Quelle wird als Pfad zu einer beliebigen Datei interpretiert. Der Dateityp wird ermittelt; das resultierende Bild ist das Systemsymbol für Dateien dieses Typs.

**filetype**
  Die Quelle wird als Zeichenkette interpretiert, die einen bestimmten Dateityp identifiziert. Das kann eine Dateinamenserweiterung, ein Apple Uniform Type Identifier oder ein 4-Zeichen-OSType-Wert wie im HFS-Dateisystem sein.

**-width *pixels***
  siehe unten ...

Der Wert der Option *width* ist eine Ganzzahl und gibt die Breite des **nsimage** in Pixeln an. Wenn die Breite nicht angegeben ist, wird sie aus der Höhe berechnet, um das Seitenverhältnis beizubehalten. Wenn weder Breite noch Höhe angegeben sind, werden Breite und Höhe des zugrunde liegenden NSImage verwendet.

**-height *pixels***

siehe unten ...

Der Wert der Option *height* ist eine Ganzzahl und gibt die Höhe des **nsimage** in Pixeln an. Wenn die Höhe nicht angegeben ist, wird sie aus der Breite berechnet, um das Seitenverhältnis beizubehalten. Wenn weder Breite noch Höhe angegeben sind, werden Breite und Höhe des zugrunde liegenden NSImage verwendet.

**-radius *pixels***
siehe unten ...

Der Wert der Option *radius* ist eine Ganzzahl. Ist er ungleich null, wird das Bild auf ein abgerundetes Rechteck mit derselben Breite und Höhe wie das Bild zugeschnitten; die Ecken werden durch Kreisbögen des angegebenen Radius abgeschnitten.

**-ring *pixels***
siehe unten ...

Der Wert der Option *ring* ist eine Ganzzahl. Ist er ungleich null, gibt er die Dicke eines Fokusrahmens an, der um das Bild mit der in den Systemeinstellungen festgelegten Akzentfarbe der Steuerelemente gezeichnet wird. Das Bild wird verkleinert, sodass Breite und Höhe jeweils um das Doppelte der Ringdicke reduziert werden. Das kann eine geringe Verzerrung verursachen. Das Seitenverhältnis eines nicht quadratischen Bildes ändert sich leicht.

**-alpha *float***
siehe unten ...

Der Wert der Option *alpha* sollte eine Gleitkommazahl zwischen 0,0 und 1,0 sein. Dieser Alpha-Wert wird auf jeden Pixel des **nsimage** angewendet und erzeugt ein teilweise transparentes Bild. Der Standardwert ist 1,0 (undurchsichtig).

**-pressed *boolean***
siehe unten ...

Die Option *pressed* nimmt einen booleschen Wert an. Ist der Wert true oder 1, wird das Bild algorithmisch im Hellmodus dunkler bzw. im Dunkelmodus heller dargestellt. Der Standard ist false. Für einen Bild-Button sollte das Hauptbild false verwenden; das gedrückte Bild sollte dasselbe Bild sein, aber mit der Option *pressed* auf true.

**-template *boolean***
siehe unten ...

Die Option *template* nimmt einen booleschen Wert an. Ist der Wert true oder 1, wird das Bild als Vorlagenbild markiert. Das System wandelt das Bild dann algorithmisch im Dunkelmodus in ein hellfarbiges Bild um. Damit der Algorithmus korrekt arbeitet, darf das Bild nur aus schwarzen Pixeln mit Alpha-Werten bestehen.

*Copyright (c) 2022 Marc Culler*

# option

*Stichwörter:* option, database, priority, retrieve

## Name

option - Fensteroptionen zur/von der Optionsdatenbank hinzufügen/abfragen

## Übersicht

**option add** *pattern value* ?*priority*?
**option clear**
**option get** *window name class*
**option readfile** *fileName* ?*priority*?

## Beschreibung

Der Befehl **option** ermöglicht es, Einträge zur Tk-Optionsdatenbank hinzuzufügen oder Optionen aus der Datenbank abzufragen. Die Form **add** des Befehls fügt der Datenbank eine neue Option hinzu. *Pattern* enthält die anzugebende Option und besteht aus Namen und/oder Klassen, getrennt durch Sternchen oder Punkte, im üblichen X-Format (siehe Musterformat). *Value* enthält eine Textzeichenkette, die mit *pattern* verknüpft wird; dies ist der Wert, der bei Aufrufen von Tk_GetOption oder durch Aufrufe des Befehls **option get** zurückgegeben wird. Wenn *priority* angegeben ist, gibt sie die Prioritätsstufe für diese Option an (siehe unten für zulässige Werte); der Standardwert ist **interactive**. Dieser Befehl gibt immer einen leeren String zurück.

Der Befehl **option clear** leert die Optionsdatenbank. Standardoptionen (aus der Eigenschaft **RESOURCE_MANAGER** oder der Datei **.Xdefaults**) werden automatisch neu geladen, sobald das nächste Mal eine Option zur Datenbank hinzugefügt oder daraus entfernt wird. Dieser Befehl gibt immer einen leeren String zurück.

Der Befehl **option get** gibt den Wert der für *window* unter *name* und *class* angegebenen Option zurück. Wenn mehrere Einträge in der Optionsdatenbank zu *window*, *name* und *class* passen, gibt der Befehl denjenigen zurück, der mit der höchsten *priority*-Stufe erstellt wurde. Wenn es mehrere passende Einträge auf derselben Prioritätsstufe gibt, gibt er denjenigen zurück, der zuletzt in die Optionsdatenbank eingetragen wurde. Wenn es keine passenden Einträge gibt, wird der leere String zurückgegeben.

Die Form **readfile** des Befehls liest *fileName*, das das Standardformat für eine X-Ressourcendatenbank wie **.Xdefaults** haben sollte, und fügt alle in dieser Datei angegebenen Optionen zur Optionsdatenbank hinzu. Wenn *priority* angegeben ist, gibt sie die Prioritätsstufe an, mit der die Optionen eingetragen werden; *priority* hat standardmäßig den Wert **interactive**.

Die Datei wird über einen Kanal gelesen, der in der Kodierung "utf-8" ist; ungültige Bytefolgen werden automatisch in gültige umgewandelt. Das bedeutet, dass Kodierungen wie ISO 8859-1 oder cp1252 mit hoher Wahrscheinlichkeit ebenfalls funktionieren, dies kann jedoch nicht garantiert werden. Dies kann nicht geändert werden; das Setzen von [encoding system] hat keine Wirkung.

Die *priority*-Argumente des Befehls **option** werden normalerweise symbolisch mit einem der folgenden Werte angegeben:

**widgetDefault**
Stufe 20. Wird für Standardwerte verwendet, die fest in Widgets codiert sind.

**startupFile**
Stufe 40. Wird für Optionen verwendet, die in anwendungsspezifischen Startdateien angegeben sind.

**userDefault**
Stufe 60. Wird für Optionen verwendet, die in benutzerspezifischen Standarddateien angegeben sind, wie **.Xdefaults**, in den X-Server geladene Ressourcendatenbanken oder

benutzerspezifische Startdateien.

**interactive**

Stufe 80. Wird für Optionen verwendet, die interaktiv angegeben werden, nachdem die Anwendung gestartet wurde. Wenn *priority* nicht angegeben ist, ist dies der Standardwert.

Jedes der obigen Schlüsselwörter darf abgekürzt werden. Zusätzlich können Prioritäten numerisch mit Ganzzahlen zwischen 0 und 100 (einschließlich) angegeben werden. Die numerische Form ist wahrscheinlich eine schlechte Idee, außer für neue Prioritätsstufen, die nicht zu den oben genannten gehören.

## Musterformat

Muster bestehen aus einer Folge von Wörtern, getrennt durch entweder Punkte "." oder Sternchen "*". Das gesamte Muster kann optional auch von einem Sternchen vorangestellt werden.

Jedes Wort im Muster beginnt üblicherweise entweder mit einem Großbuchstaben (in diesem Fall bezeichnet es die Klasse eines Widgets oder einer Option) oder mit einem anderen Zeichen, wenn es den Namen eines Widgets oder einer Option bezeichnet. Das letzte Wort im Muster gibt immer die Option an; die vorhergehenden schränken ein, in welchen Widgets nach dieser Option gesucht wird.

Wenn zwei Wörter durch einen Punkt getrennt sind, muss das letztere Widget ein direktes Kind des ersteren sein (oder die Option muss nur für die angegebenen Widgets gelten). Wenn zwei Wörter durch ein Sternchen getrennt sind, kann zwischen den beiden Widgets eine beliebige Tiefe liegen (und die Option gilt für alle Widgets, die Kinder des ersteren Widgets sind).

Wenn dem gesamten Muster ein Sternchen vorangestellt ist, gilt das gesamte Muster überall dort in der gesamten Widget-Hierarchie, wo es passt. Andernfalls wird das erste Wort des Musters mit dem Namen und der Klasse des "**.**"-toplevel abgeglichen, die üblicherweise durch Optionen zu wish gesetzt werden.

## Beispiele

Weisen Sie jedem Button in der Anwendung roten Text zu, sofern nicht ausdrücklich überschrieben, indem Sie **foreground** für die Klasse **Button** setzen (beachten Sie, dass die Option auf einigen Plattformen ignoriert wird):

```
option add *Button.foreground red startupFile
```

Ermöglichen Sie Benutzern, das Verhalten in einem Entry-Widget beim Drücken der Return-Taste zu steuern, indem Sie ein Skript in der Optionsdatenbank angeben, und fügen Sie eine Standardoption hinzu, die die Glocke läutet:

```
entry .e
bind .e <Return> [option get .e returnCommand Command]
option add *.e.returnCommand bell widgetDefault
```

*Copyright (c) 1990 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# optionMenu

*Stichwörter:* optionMenu, option menu

## Name

tk_optionMenu - Erstellt ein Options-Menubutton und zugehöriges Menü

## Übersicht

**tk_optionMenu** *pathName varName value* ?*value value* ...?

## Beschreibung

Diese Prozedur erstellt einen Options-Menubutton mit dem Namen *pathName* sowie ein zugehöriges Menü. Zusammen ermöglichen sie dem Benutzer, einen der durch die *value* -Argumente angegebenen Werte auszuwählen. Der aktuelle Wert wird in der globalen Variable gespeichert, deren Name durch *varName* angegeben ist, und außerdem als Beschriftung im Options-Menubutton angezeigt. Der Benutzer kann auf den Menubutton klicken, um ein Menü mit allen *value*-Einträgen anzuzeigen und so einen neuen Wert auszuwählen. Sobald ein neuer Wert gewählt wurde, wird er in der Variable gespeichert und im Options-Menubutton angezeigt. Der aktuelle Wert kann auch durch Setzen der Variable geändert werden.

Der Rückgabewert von **tk_optionMenu** ist der Name des mit *pathName* verknüpften Menüs, sodass der Aufrufer dessen Konfigurationsoptionen ändern oder es auf andere Weise manipulieren kann.

## Beispiel

```
tk_optionMenu .foo myVar Foo Bar Boo Spong Wibble
pack .foo
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# options

*Stichwörter:* options, class, name, standard option, switch

## Name

options - Von Widgets unterstützte Standardoptionen

## Beschreibung

Dieser Handbuch-Eintrag beschreibt die gemeinsamen Konfigurationsoptionen, die von Widgets im Tk-Toolkit unterstützt werden. Nicht jedes Widget unterstützt notwendigerweise jede Option (siehe die Handbuch-Einträge für einzelne Widgets für eine Liste der von diesem Widget unterstützten Standardoptionen), aber wenn ein Widget eine Option mit einem der unten aufgeführten Namen unterstützt, hat die Option genau die unten beschriebene Wirkung.

In den folgenden Beschreibungen bezeichnet "Befehlszeilen-Name" den Schalter, der in Klassenbefehlen und **configure**-Widget-Befehlen verwendet wird, um diesen Wert festzulegen. Wenn beispielsweise der Befehlszeilen-Schalter einer Option **-foreground** ist und ein Widget **.a.b.c** existiert, dann kann der Befehl

```
.a.b.c  configure  -foreground black
```

verwendet werden, um den Wert **black** für die Option im Widget **.a.b.c** festzulegen. Befehlszeilen-Schalter dürfen abgekürzt werden, solange die Abkürzung eindeutig ist.

"Datenbank-Name" bezieht sich auf den Namen der Option in der Optionsdatenbank (z. B. in .Xdefaults-Dateien). "Datenbank-Klasse" bezieht sich auf den Klassenwert der Option in der Optionsdatenbank.

**-activebackground (activeBackground/Foreground)**

Legt die Hintergrundfarbe fest, die beim Zeichnen aktiver Elemente verwendet werden soll. Ein Element (ein Widget oder ein Teil eines Widgets) ist aktiv, wenn der Mauszeiger über dem Element positioniert ist und das Drücken einer Maustaste eine Aktion auslöst. Wenn strikte Motif-Kompatibilität durch Setzen der Variable tk_strictMotif angefordert wurde, wird diese Option normalerweise ignoriert; stattdessen wird die normale Hintergrundfarbe verwendet. Bei einigen Elementen unter Windows- und Macintosh-Systemen wird die aktive Farbe nur verwendet, während Maustaste 1 über dem Element gedrückt gehalten wird.

**-activeborderwidth (activeBorderWidth/BorderWidth)**

Legt einen nicht-negativen Wert fest, der die Breite des 3D-Rahmens angibt, der um aktive Elemente gezeichnet wird. Siehe oben für die Definition aktiver Elemente. Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben. Diese Option ist typischerweise nur in Widgets verfügbar, die mehr als ein Element gleichzeitig anzeigen (z. B. Menüs, aber nicht Buttons).

**-activeforeground (activeForeground/Background)**

Legt die Vordergrundfarbe fest, die beim Zeichnen aktiver Elemente verwendet werden soll. Siehe oben für die Definition aktiver Elemente.

**-activerelief (activeRelief/Relief)**

Legt den gewünschten 3D-Effekt für das aktive Element des Widgets fest. Siehe die Option **-relief** für Details.

**-anchor (anchor/Anchor)**

Legt fest, wie die Informationen in einem Widget (z. B. Text oder ein Bitmap) im Widget angezeigt werden sollen. Muss einer der Werte **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw** oder **center** sein. Beispielsweise bedeutet **nw**, dass die Informationen so angezeigt werden, dass ihre obere linke Ecke an der oberen linken Ecke des Widgets liegt.

**-background or -bg (background/Background)**

Legt die normale Hintergrundfarbe fest, die beim Anzeigen des Widgets verwendet werden soll.

**-bitmap (bitmap/Bitmap)**

Legt ein Bitmap fest, das im Widget angezeigt werden soll, in einer der von Tk_GetBitmap akzeptierten Formen. Die genaue Art, wie das Bitmap angezeigt wird, kann von anderen Optionen wie **-anchor** oder **-justify** beeinflusst werden. Typischerweise überschreibt diese Option, wenn sie angegeben ist, andere Optionen, die einen Textwert zur Anzeige im Widget festlegen, aber dies wird durch die Option **-compound** gesteuert; die Option **-bitmap** kann auf eine leere Zeichenkette zurückgesetzt werden, um eine Textanzeige wieder zu aktivieren. In Widgets, die sowohl **-bitmap** als auch **-image** unterstützen, überschreibt **-image** normalerweise **-bitmap**.

**-borderwidth or -bd (borderWidth/BorderWidth)**

Legt einen nicht-negativen Wert fest, der die Breite des 3D-Rahmens angibt, der um die Außenseite des Widgets gezeichnet wird (wenn ein solcher Rahmen gezeichnet wird; die Option **-relief** bestimmt dies typischerweise). Der Wert kann auch beim Zeichnen von 3D-Effekten im Inneren des Widgets verwendet werden. Der Wert kann eine der von

Tk_GetPixels akzeptierten Formen haben.

**-cursor (cursor/Cursor)**

Legt den Mauszeiger fest, der für das Widget verwendet werden soll. Der Wert kann eine der von Tk_GetCursor akzeptierten Formen haben. Wenn eine leere Zeichenkette angegeben ist, bedeutet dies, dass das Widget die Cursor-Spezifikation an sein übergeordnetes Widget delegieren soll.

**-compound (compound/Compound)**

Legt fest, ob das Widget Text und Bitmaps/Bilder gleichzeitig anzeigen soll und, falls ja, wo das Bitmap/Bild relativ zum Text platziert werden soll. Muss einer der Werte **none**, **bottom**, **top**, **left**, **right** oder **center** sein. Beispielsweise gibt der (Standard-)Wert **none** an, dass das Bitmap oder Bild (falls definiert) anstelle des Textes angezeigt werden soll, der Wert **left** gibt an, dass das Bitmap oder Bild links vom Text angezeigt werden soll, und der Wert **center** gibt an, dass das Bitmap oder Bild über dem Text angezeigt werden soll.

**-disabledforeground (disabledForeground/DisabledForeground)**

Legt die Vordergrundfarbe fest, die beim Zeichnen eines deaktivierten Elements verwendet werden soll. Wenn die Option als leere Zeichenkette angegeben ist (was typischerweise bei monochromen Displays der Fall ist), werden deaktivierte Elemente mit der normalen Vordergrundfarbe gezeichnet, aber sie werden abgedunkelt, indem sie mit einem gestrichelten Füllmuster gezeichnet werden.

**-exportselection (exportSelection/ExportSelection)**

Legt fest, ob eine Auswahl im Widget auch die X-Auswahl sein soll. Der Wert kann eine der von Tcl_GetBoolean akzeptierten Formen haben, wie **true**, **false**, **0**, **1**, **yes** oder **no**. Wenn die Auswahl exportiert wird, hebt die Auswahl im Widget die aktuelle X-Auswahl auf, die Auswahl außerhalb des Widgets hebt jede Widget-Auswahl auf, und das Widget reagiert auf Anfragen zur Auswahlübermittlung, wenn es eine Auswahl hat. Standardmäßig exportieren Widgets normalerweise Auswahlen.

**-font (font/Font)**

Legt die Schriftart fest, die beim Zeichnen von Text im Widget verwendet werden soll. Der Wert kann eine der im Handbuch-Eintrag font unter **FONT DESCRIPTION** beschriebenen Formen haben.

**-foreground or -fg (foreground/Foreground)**

Legt die normale Vordergrundfarbe fest, die beim Anzeigen des Widgets verwendet werden soll.

**-highlightbackground (highlightBackground/HighlightBackground)**

Legt die Farbe fest, die in der Traversierungs-Hervorhebungsregion angezeigt wird, wenn das Widget nicht den Eingabefokus hat.

**-highlightcolor (highlightColor/HighlightColor)**

Legt die Farbe fest, die für das Traversierungs-Hervorhebungsrechteck verwendet wird, das um das Widget gezeichnet wird, wenn es den Eingabefokus hat.

**-highlightthickness (highlightThickness/HighlightThickness)**

Legt einen nicht-negativen Wert fest, der die Breite des Hervorhebungsrechtecks angibt, das um die Außenseite des Widgets gezeichnet wird, wenn es den Eingabefokus hat. Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben. Wenn der Wert null ist, wird keine Fokus-Hervorhebung um das Widget gezeichnet.

**-image (image/Image)**

Legt ein Bild fest, das im Widget angezeigt werden soll, das mit dem Befehl **image create** erstellt worden sein muss. Typischerweise überschreibt die Option **-image**, wenn sie angegeben ist, andere Optionen, die ein Bitmap oder einen Textwert zur Anzeige im Widget festlegen, obwohl dies durch die Option **-compound** gesteuert wird; die Option **-image** kann auf eine leere Zeichenkette zurückgesetzt werden, um eine Bitmap- oder Textanzeige wieder zu aktivieren.

**-insertbackground (insertBackground/Foreground)**

Legt die Farbe fest, die als Hintergrund im Bereich verwendet wird, der vom Einfügecursor abgedeckt wird. Diese Farbe überschreibt normalerweise entweder den normalen Hintergrund des Widgets (oder den Auswahlhintergrund, wenn der Einfügecursor zufällig in der Auswahl liegt).

**-insertborderwidth (insertBorderWidth/BorderWidth)**

Legt einen nicht-negativen Wert fest, der die Breite des 3D-Rahmens angibt, der um den Einfügecursor gezeichnet wird. Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben.

**-insertofftime (insertOffTime/OffTime)**

Legt einen nicht-negativen Ganzzahlwert fest, der angibt, wie viele Millisekunden der Einfügecursor in jedem Blinkzyklus "aus" bleiben soll. Wenn diese Option null ist, blinkt der Cursor nicht: er ist die ganze Zeit an.

**-insertontime (insertOnTime/OnTime)**

Legt einen nicht-negativen Ganzzahlwert fest, der angibt, wie viele Millisekunden der Einfügecursor in jedem Blinkzyklus "an" bleiben soll.

**-insertwidth (insertWidth/InsertWidth)**

Legt einen nicht-negativen Wert fest, der die Gesamtbreite des Einfügecursors angibt. Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben. Wenn ein Rahmen für den Einfügecursor angegeben wurde (mit der Option **-insertborderwidth**), wird der Rahmen innerhalb der von der Option **-insertwidth** angegebenen Breite gezeichnet.

**-jump (jump/Jump)**

Bei Widgets mit einem Schieberegler, der zum Anpassen eines Werts gezogen werden kann, wie Scrollleisten, bestimmt diese Option, wann Benachrichtigungen über Änderungen des Werts erfolgen. Der Wert der Option muss ein boolescher Wert in der von Tcl_GetBoolean akzeptierten Form sein. Wenn der Wert false ist, werden Aktualisierungen kontinuierlich vorgenommen, während der Schieberegler gezogen wird. Wenn der Wert true ist, werden Aktualisierungen verzögert, bis die Maustaste losgelassen wird, um das Ziehen zu beenden; zu diesem Zeitpunkt erfolgt eine einzelne Benachrichtigung (der Wert "springt" anstatt sich stetig zu ändern).

**-justify (justify/Justify)**

Wenn mehrere Textzeilen in einem Widget angezeigt werden, bestimmt diese Option, wie die Zeilen zueinander ausgerichtet werden. Muss einer von **left**, **center** oder **right** sein. **Left** bedeutet, dass die linken Kanten der Zeilen alle auf einer Linie liegen, **center** bedeutet, dass die Mittelpunkte der Zeilen ausgerichtet sind, und **right** bedeutet, dass die rechten Kanten der Zeilen auf einer Linie liegen.

**-orient (orient/Orient)**

Bei Widgets, die sich horizontal oder vertikal ausrichten können, wie Scrollleisten, legt diese Option fest, welche Ausrichtung verwendet werden soll. Muss entweder **horizontal** oder

**vertical** oder eine Abkürzung davon sein.

**-padx (padX/Pad)**

Legt einen nicht-negativen Wert fest, der angibt, wie viel zusätzlicher Platz für das Widget in X-Richtung angefordert werden soll. Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben. Bei der Berechnung, wie groß ein Fenster benötigt wird, addiert das Widget diesen Betrag zur Breite, die es normalerweise benötigen würde (bestimmt durch die Breite der im Widget angezeigten Elemente); wenn der Geometrie-Manager diese Anforderung erfüllen kann, erhält das Widget zusätzlichen internen Platz links und/oder rechts von dem, was es innen anzeigt. Die meisten Widgets verwenden diese Option nur zum Polstern von Text: wenn sie ein Bitmap oder Bild anzeigen, ignorieren sie Polsterungsoptionen normalerweise.

**-pady (padY/Pad)**

Legt einen nicht-negativen Wert fest, der angibt, wie viel zusätzlicher Platz für das Widget in Y-Richtung angefordert werden soll. Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben. Bei der Berechnung, wie groß ein Fenster benötigt wird, addiert das Widget diesen Betrag zur Höhe, die es normalerweise benötigen würde (bestimmt durch die Höhe der im Widget angezeigten Elemente); wenn der Geometrie-Manager diese Anforderung erfüllen kann, erhält das Widget zusätzlichen internen Platz oberhalb und/oder unterhalb von dem, was es innen anzeigt. Die meisten Widgets verwenden diese Option nur zum Polstern von Text: wenn sie ein Bitmap oder Bild anzeigen, ignorieren sie Polsterungsoptionen normalerweise.

**-placeholder (placeHolder/PlaceHolder)**

Legt eine Hilfetext-Zeichenkette fest, die angezeigt wird, wenn kein Text anderweitig angezeigt wird, das heißt wenn das Widget leer ist. Der Platzhaltertext wird mit den Werten der Optionen **-font** und **-justify** angezeigt.

**-placeholderforeground (placeholderForeground/PlaceholderForeground)**

Legt die Vordergrundfarbe fest, die verwendet wird, wenn der Platzhaltertext angezeigt wird. Die Standardfarbe ist plattformspezifisch.

**-relief (relief/Relief)**

Legt den gewünschten 3D-Effekt für das Widget fest. Zulässige Werte sind **raised**, **sunken**, **flat**, **ridge**, **solid** und **groove**. Der Wert gibt an, wie das Innere des Widgets relativ zu seinem Äußeren erscheinen soll; beispielsweise bedeutet **raised**, dass das Innere des Widgets relativ zum Äußeren des Widgets aus dem Bildschirm hervorstehen soll.

**-repeatdelay (repeatDelay/RepeatDelay)**

Legt die Anzahl der Millisekunden fest, die eine Taste oder ein Schlüssel gedrückt gehalten werden muss, bevor die automatische Wiederholung beginnt. Wird beispielsweise bei den Auf- und Ab-Pfeilen in Scrollleisten verwendet.

**-repeatinterval (repeatInterval/RepeatInterval)**

Wird zusammen mit **-repeatdelay** verwendet: Sobald die automatische Wiederholung beginnt, bestimmt diese Option die Anzahl der Millisekunden zwischen automatischen Wiederholungen.

**-selectbackground (selectBackground/Foreground)**

Legt die Hintergrundfarbe fest, die beim Anzeigen ausgewählter Elemente verwendet werden soll.

**-selectborderwidth (selectBorderWidth/BorderWidth)**

Legt einen nicht-negativen Wert fest, der die Breite des 3D-Rahmens angibt, der um ausgewählte Elemente gezeichnet wird. Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben.

**-selectforeground (selectForeground/Background)**

Legt die Vordergrundfarbe fest, die beim Anzeigen ausgewählter Elemente verwendet werden soll.

**-setgrid (setGrid/SetGrid)**

Legt einen booleschen Wert fest, der bestimmt, ob dieses Widget das Größenänderungsraster für sein Toplevel-Fenster steuert. Diese Option wird typischerweise in Text-Widgets verwendet, bei denen die Informationen im Widget eine natürliche Größe haben (die Größe eines Zeichens) und es sinnvoll ist, dass die Fensterabmessungen ganzzahlige Vielfache dieser Einheiten sind. Diese natürlichen Fenstergrößen bilden ein Raster. Wenn die Option **-setgrid** auf true gesetzt ist, kommuniziert das Widget mit dem Fenstermanager, sodass beim interaktiven Größenändern des Toplevel-Fensters, das das Widget enthält, die Abmessungen des Fensters dem Benutzer in Rastereinheiten angezeigt werden und die Fenstergröße auf ganzzahlige Vielfache von Rastereinheiten beschränkt wird. Siehe den Abschnitt **GRIDDED GEOMETRY MANAGEMENT** im Handbuch-Eintrag wm für weitere Details.

**-takefocus (takeFocus/TakeFocus)**

Bestimmt, ob das Fenster während der Tastatur-Traversierung (z. B. Tab und Shift-Tab) den Fokus annimmt. Bevor der Fokus auf ein Fenster gesetzt wird, konsultieren die Traversierungs-Skripte den Wert der Option **-takefocus**. Ein Wert von **0** bedeutet, dass das Fenster während der Tastatur-Traversierung vollständig übersprungen werden soll. **1** bedeutet, dass das Fenster den Eingabefokus erhalten soll, solange es sichtbar ist (es und alle seine Vorfahren sind gemappt). Ein leerer Wert für die Option bedeutet, dass die Traversierungs-Skripte die Entscheidung treffen, ob auf das Fenster fokussiert werden soll oder nicht: Der aktuelle Algorithmus überspringt das Fenster, wenn es deaktiviert ist, wenn es keine Tastenbelegungen hat oder wenn es nicht sichtbar ist. Wenn der Wert eine andere Form hat, nehmen die Traversierungs-Skripte den Wert, hängen den Namen des Fensters an (mit einem Trennleerzeichen) und werten die resultierende Zeichenkette als Tcl-Skript aus. Das Skript muss **0**, **1** oder eine leere Zeichenkette zurückgeben: Ein **0**- oder **1**-Wert gibt an, ob das Fenster den Eingabefokus erhalten wird, und eine leere Zeichenkette führt zur oben beschriebenen Standardentscheidung. Hinweis: Diese Interpretation der Option wird vollständig durch die Tcl-Skripte definiert, die die Traversierung implementieren: Die Widget-Implementierungen ignorieren die Option vollständig, sodass Sie ihre Bedeutung ändern können, wenn Sie die Tastatur-Traversierungs-Skripte neu definieren.

**-text (**text**/Text)**

Legt eine Zeichenkette fest, die im Widget angezeigt werden soll. Die Art, wie die Zeichenkette angezeigt wird, hängt vom jeweiligen Widget ab und kann durch andere Optionen wie **-anchor** oder **-justify** bestimmt werden.

**-textvariable (textVariable/Variable)**

Legt den Namen einer globalen Variable fest. Der Wert der Variable ist eine Textzeichenkette, die im Widget angezeigt werden soll; wenn sich der Variablenwert ändert, aktualisiert sich das Widget automatisch, um den neuen Wert widerzuspiegeln. Die Art, wie die Zeichenkette im Widget angezeigt wird, hängt vom jeweiligen Widget ab und kann durch andere Optionen wie **-anchor** oder **-justify** bestimmt werden.

**-troughcolor (troughColor/Background)**

Legt die Farbe fest, die für die rechteckigen Trog-Bereiche in Widgets wie Scrollleisten und Skalen verwendet wird. Diese Option wird für Scrollleisten unter Windows ignoriert (das native Widget erkennt diese Option nicht).

**-underline (underline/Underline)**

Legt den ganzzahligen Index eines Zeichens fest, das im Widget unterstrichen werden soll. Diese Option wird von den Standard-Bindings verwendet, um die Tastatur-Traversierung für Menü-Buttons und Menüeinträge zu implementieren. 0 entspricht dem ersten Zeichen des im Widget angezeigten Textes, 1 dem nächsten Zeichen und so weiter. **end** entspricht dem letzten Zeichen, **end**-1 dem vorletzten Zeichen und so weiter.

**-wraplength (wrapLength/WrapLength)**

Bei Widgets, die Zeilenumbrüche durchführen können, legt diese Option die maximale Zeilenlänge fest. Zeilen, die diese Länge überschreiten würden, werden auf die nächste Zeile umgebrochen, sodass keine Zeile länger als die angegebene Länge ist. Der Wert kann in einer der Standardformen für Bildschirmabstände angegeben werden. Wenn dieser Wert negativ oder null ist, erfolgt kein Umbruch: Zeilen werden nur an Zeilenumbruchzeichen im Text getrennt.

**-xscrollcommand (xScrollCommand/ScrollCommand)**

Legt das Präfix für einen Befehl fest, der zur Kommunikation mit horizontalen Scrollleisten verwendet wird. Wenn sich die Ansicht im Fenster des Widgets ändert (oder immer wenn etwas anderes auftritt, das die Anzeige in einer Scrollleiste ändern könnte, wie eine Änderung der Gesamtgröße des Widget-Inhalts), erzeugt das Widget einen Tcl-Befehl durch Verkettung des Scroll-Befehls und zweier Zahlen. Jede der Zahlen ist ein Bruchteil zwischen 0 und 1, der eine Position im Dokument angibt. 0 gibt den Anfang des Dokuments an, 1 das Ende, .333 eine Position ein Drittel durch das Dokument und so weiter. Der erste Bruchteil gibt die ersten Informationen im Dokument an, die im Fenster sichtbar sind, und der zweite Bruchteil gibt die Informationen unmittelbar nach dem letzten sichtbaren Teil an. Der Befehl wird dann an den Tcl-Interpreter zur Ausführung übergeben. Typischerweise besteht die Option **-xscrollcommand** aus dem Pfadnamen eines Scrollleisten-Widgets gefolgt von "set", z. B. ".x.scrollbar set": Dies bewirkt, dass die Scrollleiste aktualisiert wird, wann immer sich die Ansicht im Fenster ändert. Wenn diese Option nicht angegeben ist, wird kein Befehl ausgeführt.

**-yscrollcommand (yScrollCommand/ScrollCommand)**

Legt das Präfix für einen Befehl fest, der zur Kommunikation mit vertikalen Scrollleisten verwendet wird. Diese Option wird auf die gleiche Weise behandelt wie die Option **-xscrollcommand**, außer dass sie für vertikale Scrollleisten verwendet wird und von Widgets bereitgestellt wird, die vertikales Scrollen unterstützen. Siehe die Beschreibung von **-xscrollcommand** für Details zur Verwendung dieser Option.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# pack

*Stichwörter:* pack, geometry manager, location, packer, parcel, propagation, size

## Name

pack - Geometrie-Manager, der entlang der Ränder des Hohlraums packt

## Übersicht

**pack** *option arg* ?*arg* ...?

## Beschreibung

Der Befehl **pack** dient der Kommunikation mit dem Packer, einem Geometrie-Manager, der die Kinder eines Eltern-Widgets anordnet, indem er sie der Reihe nach entlang der Ränder des Eltern-Widgets packt. Der Befehl **pack** kann je nach dem *option*-Argument mehrere Formen annehmen:

**pack *window* ?*window* ...? ?*options*?**

Wenn das erste Argument von **pack** ein Fenstername ist (ein Wert, der mit "." beginnt), wird der Befehl auf dieselbe Weise verarbeitet wie **pack configure**.

**pack configure *window* ?*window* ...? ?*options*?**

Die Argumente bestehen aus den Namen eines oder mehrerer Inhaltsfenster, gefolgt von Argumentpaaren, die festlegen, wie der Inhalt verwaltet wird. Details zur Verwendung der Optionen durch den Packer finden Sie unter Der Packer-Algorithmus weiter unten. Folgende Optionen werden unterstützt:

**-after *other***

*Other* muss der Name eines anderen Fensters sein. Dessen Container wird als Container für den Inhalt verwendet, und der Inhalt wird in der Packreihenfolge unmittelbar nach *other* eingefügt.

**-anchor *anchor***

*Anchor* muss eine gültige Ankerposition sein, etwa **n** oder **sw**; sie legt fest, wo jedes Inhaltsfenster in seinem Parcel positioniert wird. Standard ist **center**.

**-before *other***

*Other* muss der Name eines anderen Fensters sein. Dessen Container wird als Container für den Inhalt verwendet, und der Inhalt wird in der Packreihenfolge unmittelbar vor *other* eingefügt.

**-expand *boolean***

Legt fest, ob der Inhalt erweitert werden soll, um zusätzlichen Platz im Container zu nutzen. *Boolean* kann jeden gültigen booleschen Wert haben, etwa **1** oder **no**. Standard ist 0.

**-fill *style***

Ist das Parcel eines Inhalts größer als dessen angeforderte Abmessungen, kann diese Option verwendet werden, um den Inhalt zu strecken. *Style* muss einen der folgenden Werte haben:

**none**

Dem Inhalt werden seine angeforderten Abmessungen plus jeder mit **-ipadx** oder **-ipady** angeforderten internen Polsterung gegeben. Dies ist der Standardwert.

**x**

Den Inhalt horizontal strecken, um die gesamte Breite seines Parcels zu füllen (äußere Polsterung gemäß **-padx** bleibt ausgenommen).

**y**

Den Inhalt vertikal strecken, um die gesamte Höhe seines Parcels zu füllen (äußere

Polsterung gemäß **-pady** bleibt ausgenommen).

**both**

Den Inhalt sowohl horizontal als auch vertikal strecken.

**-in *container***

Das Fenster am Ende der Packreihenfolge für den durch *container* angegebenen Container einfügen.

**-ipadx *amount***

*Amount* legt fest, wie viel horizontale interne Polsterung auf jeder Seite des Inhalts verbleiben soll. *Amount* muss eine gültige Bildschirmabstandseinheit sein, etwa **2** oder **.5c**. Standard ist 0.

**-ipady *amount***

*Amount* legt fest, wie viel vertikale interne Polsterung auf jeder Seite des Inhalts verbleiben soll. *Amount* hat standardmäßig den Wert 0.

**-padx *amount***

*Amount* legt fest, wie viel horizontale äußere Polsterung auf jeder Seite des Inhalts verbleiben soll. *Amount* kann eine Liste aus zwei Werten sein, um Polsterung links und rechts getrennt anzugeben. Standard ist 0.

**-pady *amount***

*Amount* legt fest, wie viel vertikale äußere Polsterung auf jeder Seite des Inhalts verbleiben soll. *Amount* kann eine Liste aus zwei Werten sein, um Polsterung oben und unten getrennt anzugeben. Standard ist 0.

**-side *side***

Legt fest, an welcher Seite des Containers der Inhalt gepackt wird. Muss **left**, **right**, **top** oder **bottom** sein. Standard ist **top**.

Wenn keine Option **-in**, **-after** oder **-before** angegeben ist, wird jedes Inhaltsfenster am Ende der Packliste seines Eltern-Elements eingefügt, sofern es nicht bereits vom Packer verwaltet wird (dann bleibt es an seiner Position). Wird eine dieser Optionen angegeben, werden alle Inhalte am angegebenen Punkt eingefügt. Sind einige Inhalte bereits vom Geometrie-Manager verwaltet, behalten nicht angegebene Optionen ihre bisherigen Werte statt Standardwerte zu erhalten.

**pack content *window***

Gibt eine Liste aller Inhaltsfenster in der Packreihenfolge für *window* zurück. Die Reihenfolge der Inhaltsfenster in der Liste entspricht ihrer Reihenfolge in der Packreihenfolge. Hat *window* keinen Inhalt, wird ein leerer String zurückgegeben.

**pack forget *window* ?*window* ...?**

Entfernt jedes der *window* aus der Packreihenfolge seines Containers und entmappt die Fenster. Der Inhalt wird nicht mehr vom Packer verwaltet.

Wird das letzte verwaltete Inhaltsfenster des Containers nicht mehr verwaltet, wird außerdem das virtuelle Ereignis <> an den Container gesendet; der Container kann sich daraufhin selbst neu dimensionieren (oder anderweitig reagieren).

**pack info *window***

Gibt eine Liste zurück, deren Elemente den aktuellen Konfigurationszustand des durch *window* angegebenen Fensters in derselben Option-Wert-Form enthalten, die auch an **pack configure** übergeben werden könnte. Die ersten beiden Elemente der Liste sind "**-in** *container*", wobei *container* der Container des Fensters ist.

**pack propagate *container* ?*boolean*?**

Hat *boolean* einen wahren booleschen Wert wie **1** oder **on**, ist die Propagation für *container* aktiviert; *container* muss ein Fenstername sein (siehe Geometrie-Propagation weiter unten). Hat *boolean* einen falschen booleschen Wert, ist die Propagation für *container* deaktiviert. In beiden Fällen wird ein leerer String zurückgegeben. Wird *boolean* weggelassen, gibt der Befehl **0** oder **1** zurück, je nachdem, ob Propagation für *container* derzeit aktiviert ist. Propagation ist standardmäßig aktiviert.

**pack slaves *window***

Synonym für **pack content** *window*.

## Der Packer-Algorithmus

Für jeden Container führt der Packer eine geordnete Liste von Inhaltsfenstern, die *Packliste* genannt wird. Die Konfigurationsoptionen **-in**, **-after** und **-before** legen den Container für jeden Inhalt und dessen Position in der Packliste fest. Wird für einen Inhalt keine dieser Optionen angegeben, wird der Inhalt am Ende der Packliste seines Eltern-Elements hinzugefügt.

Der Packer ordnet die Inhaltsfenster eines Containers an, indem er die Packliste der Reihe nach durchläuft. Wenn er jeden Inhalt verarbeitet, ist innerhalb des Containers noch ein rechteckiger Bereich unbelegt. Dieser Bereich heißt *Hohlraum*; für den ersten Inhalt ist er die gesamte Fläche des Containers.

Für jeden Inhalt führt der Packer folgende Schritte aus:

1. Der Packer weist dem Inhalt ein rechteckiges *Parcel* entlang der durch die Option **-side** des Inhalts angegebenen Seite des Hohlraums zu. Ist die Seite oben oder unten, ist die Breite des Parcels die Breite des Hohlraums und seine Höhe die angeforderte Höhe des Inhalts plus die Optionen **-ipady** und **-pady**. Bei der Seite links oder rechts ist die Höhe des Parcels die Höhe des Hohlraums und die Breite die angeforderte Breite des Inhalts plus **-ipadx** und **-padx**. Das Parcel kann wegen der Option **-expand** weiter vergrößert werden (siehe Erweiterung weiter unten).
2. Der Packer wählt die Abmessungen des Inhalts. Die Breite ist normalerweise die angeforderte Breite des Inhalts plus das Doppelte seiner Option **-ipadx**, die Höhe normalerweise die angeforderte Höhe plus das Doppelte von **-ipady**. Ist die Option **-fill** jedoch **x** oder **both**, wird die Breite des Inhalts so erweitert, dass sie die Breite des Parcels abzüglich des Doppelten von **-padx** füllt. Ist **-fill y** oder **both**, wird die Höhe des Inhalts so erweitert, dass sie die Höhe des Parcels abzüglich des Doppelten von **-pady** füllt.
3. Der Packer positioniert den Inhalt über seinem Parcel. Ist der Inhalt kleiner als das Parcel, legt die Option **-anchor** fest, wo im Parcel der Inhalt platziert wird. Sind **-padx** oder **-pady** ungleich null, bleibt stets die angegebene äußere Polsterung zwischen Inhalt und Parcel-Rändern.

Sobald ein Inhalt gepackt ist, wird die Fläche seines Parcels vom Hohlraum abgezogen, sodass ein kleinerer rechteckiger Hohlraum für den nächsten Inhalt bleibt. Nutzt ein Inhalt sein Parcel nicht vollständig, wird der ungenutzte Platz nicht von nachfolgenden Inhalten verwendet. Wird der Hohlraum zu klein für die Anforderungen eines Inhalts, erhält der Inhalt den verbleibenden Platz im Hohlraum. Schrumpft der Hohlraum auf die Größe null, werden alle verbleibenden

Inhalte der Packliste vom Bildschirm entmappt, bis das Container-Fenster wieder groß genug ist.

### Erweiterung

Ist ein Container-Fenster so groß, dass nach dem Packen aller Inhalte noch zusätzlicher Platz übrig bleibt, wird dieser Platz gleichmäßig auf alle Inhalte verteilt, für die die Option **-expand** gesetzt ist. Zusätzlicher horizontaler Platz wird auf erweiterbare Inhalte mit **-side left** oder **right** verteilt, zusätzlicher vertikaler Platz auf erweiterbare Inhalte mit **-side top** oder **bottom**.

### Geometrie-Propagation

Der Packer berechnet normalerweise, wie groß ein Container sein muss, um genau den Bedarf seiner Inhalte zu decken, und setzt die angeforderte Breite und Höhe des Containers auf diese Werte. Dadurch propagieren Geometrieinformationen in der Fensterhierarchie bis zum Top-Level-Fenster, sodass sich der gesamte Unterbaum an die Blattfenster anpasst. Mit **pack propagate** kann die Propagation für einen oder mehrere Container jedoch abgeschaltet werden. Ist Propagation deaktiviert, setzt der Packer die angeforderte Breite und Höhe des Containers nicht. Das ist nützlich, wenn ein Container-Fenster etwa eine von Ihnen festgelegte feste Größe haben soll.

## Einschränkungen für Container-Fenster

Der Container jedes Inhalts muss entweder das Eltern-Element des Inhalts sein (Standard) oder ein Nachfahre des Eltern-Elements des Inhalts. Diese Einschränkung stellt sicher, dass der Inhalt über jeden sichtbaren Teil seines Containers gelegt werden kann, ohne vom Eltern-Element abgeschnitten zu werden.

## Packreihenfolge

Ist der Container eines Inhalts nicht dessen Eltern-Element, muss der Inhalt in der Stapelreihenfolge über dem Container liegen. Andernfalls verdeckt der Container den Inhalt, und es wirkt, als wäre der Inhalt nicht korrekt gepackt. Am einfachsten erreichen Sie das, indem Sie zuerst das Container-Fenster erzeugen: das zuletzt erzeugte Fenster liegt oben in der Stapelreihenfolge. Alternativ können raise und lower die Stapelreihenfolge von Container oder Inhalt ändern.

## Beispiele

```
# Widgets erzeugen
label .t -text "This widget is at the top"    -bg red
label .b -text "This widget is at the bottom" -bg green
label .l -text "Left\nHand\nSide"
label .r -text "Right\nHand\nSide"
text .mid
.mid insert end "This layout is like Java's BorderLayout"
# Anordnen
pack .t   -side top    -fill x
pack .b   -side bottom -fill x
pack .l   -side left   -fill y
pack .r   -side right  -fill y
pack .mid -expand 1    -fill both
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# palette

*Stichwörter:* palette, bisque, color

## Name

tk_setPalette, tk_bisque - Ändert die Tk-Farbpalette

## Übersicht

**tk_setPalette** *background*
**tk_setPalette** *name value* ?*name value* ...?
**tk_bisque**

## Beschreibung

Die Prozedur **tk_setPalette** ändert das Farbschema für Tk. Dazu werden die Farben bestehender Widgets geändert und die Optionsdatenbank so angepasst, dass zukünftig erstellte Widgets das neue Farbschema verwenden. Wird **tk_setPalette** mit einem einzigen Argument aufgerufen, ist dieses Argument der Name einer Farbe, die als normale Hintergrundfarbe verwendet werden soll; **tk_setPalette** berechnet daraus eine vollständige Farbpalette. Alternativ können die Argumente von **tk_setPalette** aus beliebig vielen *name*-*value*-Paaren bestehen, wobei das erste Argument des Paars der Name einer Option in der Tk-Optionsdatenbank und das zweite Argument der neue Wert für diese Option ist. Folgende Datenbanknamen werden derzeit unterstützt:

**activeBackground foreground selectColor**
**activeForeground highlightBackground selectBackground**
**background highlightColor selectForeground**
**disabledForeground insertBackground troughColor**

**tk_setPalette** versucht, sinnvolle Standardwerte für alle Optionen zu berechnen, die Sie nicht angeben. Sie können auch andere Optionen als die oben genannten angeben; Tk ändert dann diese Optionen an Widgets ebenfalls. Diese Funktion kann nützlich sein, wenn Sie eigene Widgets mit zusätzlichen Farboptionen verwenden.

Nachdem **tk_setPalette** für jede Farboption den neuen Wert ermittelt hat, durchsucht es die Widget-Hierarchie, um die Optionen aller bestehenden Widgets zu ändern. Für jedes Widget prüft es, ob eine der oben genannten Optionen für das Widget definiert ist. Ist das der Fall und hat die Option den aktuellen Standardwert, wird der Wert geändert; hat die Option einen anderen Wert als den Standard, ändert **tk_setPalette** ihn nicht. Der Standardwert einer Option ist der vom Widget bereitgestellte (**[lindex [$w configure $option] 3]**), sofern **tk_setPalette** nicht zuvor ausgeführt wurde; in diesem Fall ist es der Wert aus dem vorherigen Aufruf von **tk_setPalette**.

Nach der Änderung aller Widgets in der Anwendung fügt **tk_setPalette** der Optionsdatenbank Einträge hinzu, um die Standardwerte für künftig erstellte Widgets zu ändern. Die neuen Optionen werden mit der Priorität **widgetDefault** eingetragen, sodass sie durch Optionen aus der .Xdefaults-Datei oder durch Optionen auf der Befehlszeile beim Erstellen eines Widgets überschrieben werden können.

Die Prozedur **tk_bisque** dient der Abwärtskompatibilität: Sie stellt die Anwendungsfarben auf das hellbraune ("bisque") Farbschema von Tk 3.6 und früheren Versionen zurück.

*Copyright (c) 1995-1996 Sun Microsystems, Inc.*

# panedwindow

*Stichwörter:* panedwindow, widget, geometry management

## Name

panedwindow - Erstellt und manipuliert 'Panedwindow'-Widgets als geteilte Container

## Übersicht

**panedwindow** *pathName* ?*options*?

## Standardoptionen

**-background**, **-borderwidth**, **-cursor**, **-orient**, **-relief**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-handlepad (handlePad/HandlePad)**

Wenn Sash-Griffe gezeichnet werden, legt dies den Abstand vom oberen oder linken Ende des Sash (je nach Ausrichtung des Widgets) fest, an dem der Griff gezeichnet wird. Kann jeder von Tk_GetPixels akzeptierten Wert sein.

**-handlesize (handleSize/HandleSize)**

Legt die Seitenlänge eines Sash-Griffs fest. Griffe werden stets als Quadrate gezeichnet. Kann jeder von Tk_GetPixels akzeptierten Wert sein.

**-height (height/Height)**

Legt eine gewünschte Höhe für das gesamte Panedwindow-Widget fest. Kann jeder von Tk_GetPixels akzeptierten Wert sein. Wenn ein leerer String, wird das Widget hoch genug gemacht, damit alle enthaltenen Widgets ihre natürliche Höhe haben können.

**-opaqueresize (opaqueResize/OpaqueResize)**

Legt fest, ob Bereiche während des Verschiebens eines Sash in Echtzeit skaliert werden sollen (true), oder ob die Größenänderung bis zum Platzieren des Sash zurückgestellt werden soll (false). Im letzteren Fall wird während der Größenänderung eine "Geister"-Version des Sash angezeigt, um zu zeigen, wohin die Bereiche skaliert werden, wenn die Maustaste losgelassen wird. Diese "Geister"-Version des Sash ist der Proxy. Seine Darstellung kann mit den Optionen **-proxybackground**, **-proxyborderwidth** und **-proxyrelief** konfiguriert werden.

**-proxybackground (proxyBackground/ProxyBackground)**

Hintergrundfarbe zum Zeichnen des Proxys. Wenn ein leerer String, wird der Wert der Option **-background** verwendet.

**-proxyborderwidth (proxyBorderWidth/ProxyBorderWidth)**

Legt die Rahmenbreite des Proxys fest. Kann jeder von Tk_GetPixels akzeptierten Wert sein.

**-proxyrelief (proxyRelief/ProxyRelief)**

Relief zum Zeichnen des Proxys. Kann einer der Standard-Tk-Relief-Werte sein. Wenn ein leerer String, wird der Wert der Option **-sashrelief** verwendet.

**-sashcursor (sashCursor/SashCursor)**

Mauszeiger, der über einem Sash verwendet wird. Wenn null, wird **sb_h_double_arrow** für horizontale Panedwindows und **sb_v_double_arrow** für vertikale Panedwindows verwendet.

**-sashpad (sashPad/SashPad)**

Legt den Innenabstand fest, der auf jeder Seite eines Sash verbleiben soll. Kann jeder von Tk_GetPixels akzeptierten Wert sein.

**-sashrelief (sashRelief/SashRelief)**

Relief zum Zeichnen eines Sash. Kann einer der Standard-Tk-Relief-Werte sein.

**-sashwidth (sashWidth/SashWidth)**

Legt die Breite jedes Sash fest. Kann jeder von Tk_GetPixels akzeptierten Wert sein.

**-showhandle (showHandle/ShowHandle)**

Legt fest, ob Sash-Griffe angezeigt werden sollen. Kann jeder gültige Tcl-Boolesche Wert sein.

**-width (width/Width)**

Legt eine gewünschte Breite für das gesamte Panedwindow-Widget fest. Kann jeder von Tk_GetPixels akzeptierten Wert sein. Wenn ein leerer String, wird das Widget breit genug gemacht, damit alle enthaltenen Widgets ihre natürliche Breite haben können.

## Beschreibung

Der Befehl **panedwindow** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Panedwindow-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des Panedwindows wie Standard-Hintergrundfarbe und Relief zu konfigurieren. Der Befehl **panedwindow** gibt den Pfadnamen des neuen Fensters zurück.

Ein Panedwindow-Widget enthält beliebig viele Bereiche (*panes*), horizontal oder vertikal angeordnet, je nach Wert der Option **-orient**. Jeder Bereich enthält ein Widget, und jedes Paar von Bereichen wird durch einen verschiebbaren (per Mausbewegung) Sash getrennt. Das Verschieben eines Sash bewirkt, dass die Widgets auf beiden Seiten des Sash skaliert werden.

Ist ein Bereich kein Kind des Panedwindows, muss er in der Stapelreihenfolge über dem Panedwindow liegen, sonst verdeckt das Panedwindow ihn. Am einfachsten erzeugen Sie das Panedwindow zuerst: das zuletzt erzeugte Fenster liegt oben in der Stapelreihenfolge. Oder Sie verwenden die Befehle raise und lower.

## Widget-Befehl

Der Befehl **panedwindow** erstellt einen neuen Tcl-Befehl, dessen Name derselbe ist wie der Pfadname des Fensters des Panedwindows. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*PathName* ist der Name des Befehls, der mit dem Pfadnamen des Panedwindow-Widgets identisch ist. *Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Panedwindow-Widgets möglich:

***pathName* add *window* ?*window* ...? ?*option value* ...?**

Fügt ein oder mehrere Fenster zum Panedwindow hinzu, jeweils in einem separaten Bereich. Die Argumente bestehen aus den Namen eines oder mehrerer Fenster gefolgt von Argumentpaaren, die angeben, wie die Fenster verwaltet werden sollen. *Option* kann jeden der vom Unterbefehl **configure** akzeptierten Werte haben.

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **panedwindow** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **panedwindow** akzeptierten Werte haben.

***pathName* forget *window* ?*window* ...?**

Entfernt den Bereich, der *window* enthält, aus dem Panedwindow. Alle Geometrieverwaltungsoptionen für *window* werden vergessen.

***pathName* identify *x y***

Identifiziert die Panedwindow-Komponente unter dem durch *x* und *y* gegebenen Punkt in Fensterkoordinaten. Wenn der Punkt über einem Sash oder einem Sash-Griff liegt, ist das Ergebnis eine zweielementige Liste mit dem Index des Sash oder Griffs und einem Wort, das angibt, ob es sich um einen Sash oder einen Griff handelt, z. B. {0 sash} oder {2 handle}. Wenn der Punkt über einem anderen Teil des Panedwindows liegt, ist das Ergebnis eine leere Liste.

***pathName* panecget *window option***

Abfrage einer Verwaltungsoption für *window*. *Option* kann jeder vom Unterbefehl **paneconfigure** erlaubte Wert sein.

***pathName* paneconfigure *window* ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Verwaltungsoptionen für *window*. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. Folgende Optionen werden unterstützt:

**-after *window***

Fügt das Fenster nach dem angegebenen Fenster ein. *window* sollte der Name eines Fensters sein, das bereits von *pathName* verwaltet wird.

**-before *window***

Fügt das Fenster vor dem angegebenen Fenster ein. *window* sollte der Name eines Fensters sein, das bereits von *pathName* verwaltet wird.

**-height *size***

Legt eine Höhe für das Fenster fest. Die Höhe ist die äußere Abmessung des Fensters einschließlich seines Rahmens, falls vorhanden. Wenn *size* ein leerer String ist oder **-height** nicht angegeben ist, wird zunächst die intern vom Fenster angeforderte Höhe verwendet; die Höhe kann später durch Verschieben von Sash im Panedwindow angepasst werden. *Size* kann jeder von Tk_GetPixels akzeptierte Wert sein.

**-hide *boolean***

Steuert die Sichtbarkeit eines Bereichs. Wenn der *boolean* true ist (gemäß Tcl_GetBoolean), ist der Bereich nicht sichtbar, wird aber weiterhin in der Liste der Bereiche geführt.

**-minsize *n***

Legt fest, dass die Größe des Fensters nicht kleiner als *n* gemacht werden kann. Diese Einschränkung betrifft nur die Größe des Widgets in der Paned-Dimension — der x-Dimension bei horizontalen Panedwindows, der y-Dimension bei vertikalen Panedwindows. Kann jeder von Tk_GetPixels akzeptierte Wert sein.

**-padx *n***

Legt einen nicht-negativen Wert fest, der angibt, wie viel zusätzlicher Platz auf jeder Seite des Fensters in X-Richtung verbleiben soll. Der Wert kann in einer der von Tk_GetPixels akzeptierten Formen angegeben werden.

**-pady *n***

Legt einen nicht-negativen Wert fest, der angibt, wie viel zusätzlicher Platz auf jeder Seite des Fensters in Y-Richtung verbleiben soll. Der Wert kann in einer der von Tk_GetPixels akzeptierten Formen angegeben werden.

**-sticky *style***

Wenn der Bereich eines Fensters größer ist als die angeforderten Abmessungen des Fensters, kann diese Option verwendet werden, um das Fenster innerhalb seines Bereichs zu positionieren (oder zu strecken). *Style* ist ein String, der null oder mehr der Zeichen **n**, **s** , **e** oder **w** enthält. Der String kann optional Leerzeichen oder Kommas enthalten, diese werden jedoch ignoriert. Jeder Buchstabe bezieht sich auf eine Seite (Norden, Süden, Osten oder Westen), an der das Fenster "klebt". Wenn sowohl **n** als auch **s** (oder **e** und **w**) angegeben sind, wird das Fenster gestreckt, um die gesamte Höhe (oder Breite) seines Hohlraums auszufüllen.

**-stretch *when***

Steuert, wie zusätzlicher Platz den einzelnen Bereichen zugewiesen wird. *When* ist einer von **always**, **first**, **last**, **middle** und **never**. Das Panedwindow berechnet die erforderliche Größe aller Bereiche. Verbleibender (oder fehlender) Platz wird den zum Strecken markierten Bereichen zugewiesen. Der Platz wird basierend auf dem aktuellen Verhältnis jedes Bereichs am Ganzen verteilt. Die *when*-Werte haben folgende Bedeutung:

**always**

Dieser Bereich wird stets gestreckt.

**first**

Nur wenn dieser Bereich der erste Bereich (ganz links oder ganz oben) ist, wird er gestreckt.

**last**

Nur wenn dieser Bereich der letzte Bereich (ganz rechts oder ganz unten) ist, wird er gestreckt. Dies ist der Standardwert.

**middle**

Nur wenn dieser Bereich weder der erste noch der letzte Bereich ist, wird er gestreckt.

**never**

Dieser Bereich wird niemals gestreckt.

**-width *size***

Legt eine Breite für das Fenster fest. Die Breite ist die äußere Abmessung des Fensters einschließlich seines Rahmens, falls vorhanden. Wenn *size* ein leerer String ist oder **-width** nicht angegeben ist, wird zunächst die intern vom Fenster angeforderte Breite verwendet; die Breite kann später durch Verschieben von Sash im Panedwindow angepasst werden. *Size* kann jeder von Tk_GetPixels akzeptierte Wert sein.

***pathName* panes**

Gibt eine geordnete Liste der von *pathName* verwalteten Widgets zurück.

***pathName* proxy ?*args*?**

Dieser Befehl wird verwendet, um die Position des Sash-Proxys abzufragen und zu ändern, der für Rubberband-artige Größenänderung von Bereichen verwendet wird. Er kann eine der folgenden Formen annehmen:

***pathName* proxy coord**

Gibt eine Liste mit den x- und y-Koordinaten der zuletzt gespeicherten Proxy-Position zurück.

***pathName* proxy forget**

Entfernt den Proxy von der Anzeige.

***pathName* proxy place *x y***

Platziert den Proxy an den gegebenen *x*- und *y*-Koordinaten.

***pathName* sash ?*args*?**

Dieser Befehl wird verwendet, um die Position von Sash im Panedwindow abzufragen und zu ändern. Er kann eine der folgenden Formen annehmen:

***pathName* sash coord *index***

Gibt das aktuelle x-y-Koordinatenpaar für den durch *index* gegebenen Sash zurück. *Index* muss eine Ganzzahl zwischen 0 und 1 weniger als die Anzahl der Bereiche im Panedwindow sein. Die angegebenen Koordinaten sind die der oberen linken Ecke der Region, die den Sash enthält.

***pathName* sash dragto *index x y***

Dieser Befehl berechnet die Differenz zwischen den gegebenen Koordinaten und den Koordinaten, die dem letzten Befehl **sash mark** für den gegebenen Sash übergeben wurden. Er verschiebt diesen Sash dann um die berechnete Differenz. Der Rückgabewert ist der leere String.

***pathName* sash mark *index x y***

Speichert *x* und *y* für den durch *index* gegebenen Sash; wird zusammen mit späteren Befehlen **sash dragto** zum Verschieben des Sash verwendet.

***pathName* sash place *index x y***

Platziert den durch *index* gegebenen Sash an den gegebenen Koordinaten.

## Bereiche Größe ändern

Ein Bereich wird skaliert, indem der Sash (oder der Sash-Griff, falls vorhanden) gegriffen und mit der Maus gezogen wird. Dies geschieht über Mausbewegungs-Bindings am Widget. Wenn ein Sash verschoben wird, werden die Größen der Bereiche auf beiden Seiten des Sash und damit die Widgets in diesen Bereichen angepasst.

Wenn ein Bereich von außen skaliert wird (z. B. wenn er mit pack zum Expandieren und Ausfüllen gepackt ist und das enthaltende Toplevel-Fenster skaliert wird), wird Platz dem letzten (ganz rechts oder ganz unten liegenden) Bereich im Fenster hinzugefügt.

Anders als untergeordnete Fenster, die z. B. von pack oder grid verwaltet werden, ändern die von einem Panedwindow verwalteten Bereiche ihre Breite oder Höhe nicht, um Änderungen der angeforderten Breiten oder Höhen der Bereiche aufzunehmen, sobald diese eingeblendet ( *mapped*) wurden. Daher kann es ratsam sein, insbesondere bei interaktiver Layout-Erstellung, einen Bereich erst dann zum Panedwindow-Widget hinzuzufügen, nachdem die Geometrieanforderungen dieses Bereichs abgeschlossen sind (d.h. alle Komponenten des Bereichs eingefügt, alle die Geometrie beeinflussenden Optionen auf ihre richtigen Werte gesetzt usw.).

*Copyright (c) 1992 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# photo

*Stichwörter:* photo, image, color

## Name

photo - Vollfarbbilder

## Übersicht

**image create photo** ?*name*? ?*options*?

*imageName* **blank**
*imageName* **cget** *option*
*imageName* **configure** ?*option*? ?*value option value* ...?
*imageName* **copy** *sourceImage* ?*option value(s)* ...?
*imageName* **data** ?*option value(s)* ...?
*imageName* **get** *x y* ?*option*?
*imageName* **put** *data* ?*option value(s)* ...?
*imageName* **read** *filename* ?*option value(s)* ...?
*imageName* **redither**
*imageName* **transparency** *subcommand* ?*arg* ...?
*imageName* **write** *filename* ?*option value(s)* ...?

## Beschreibung

Ein Photo ist ein Bild, dessen Pixel jede Farbe mit variabler Transparenz (Alpha-Kanal) anzeigen können. Ein Foto-Image wird intern in Vollfarbe (32 Bit pro Pixel) gespeichert und bei Bedarf mit Dithering angezeigt. Bilddaten können aus einer Datei oder Zeichenkette stammen oder über eine prozedurale C-Schnittstelle geliefert werden. Derzeit werden nur PNG, GIF, PPM/PGM, und (nur lesend) SVG unterstützt; eine Schnittstelle erlaubt, weitere Bilddateiformate leicht hinzuzufügen. Ein Foto-Image ist (halb)transparent, wenn die Bilddaten Transparenzinformationen enthielten. In Bereichen ohne Bilddaten ist es vollständig transparent. Transparenz kann auch mit dem Unterbefehl **transparency set** geändert werden.

## Erstellung von Photos

Wie alle Images werden Photos mit image create erzeugt. Photos unterstützen folgende

*Optionen*:

**-data *string***
Legt den Bildinhalt als Zeichenkette fest. Die Zeichenkette soll Daten in der Standardform Liste-von-Listen, Binärdaten oder — für manche Formate — Base64-kodierte Daten enthalten (derzeit garantiert für PNG- und GIF-Bilder). Das Format der Zeichenkette muss eines der Formate sein, für die ein Bilddateiformat-Handler Stringdaten akzeptiert. Sind sowohl **-data** als auch **-file** angegeben, hat **-file** Vorrang.

**-format *{format-name ?option value ...?}***
Legt den Namen des Dateiformats für die mit **-data** oder **-file** angegebenen Daten fest und optionale Argumente für den Format-Handler. Der Wert dieser Option muss eine Tcl-Liste sein. Bei einem Wort können die geschweiften Klammern entfallen; statt Klammern können Anführungszeichen zur Quotierung verwendet werden.

**-file *name***
*name* gibt den Namen einer Datei an, aus der Daten für das Foto-Image gelesen werden. Das Dateiformat muss eines der Formate sein, die ein Bilddateiformat-Handler lesen kann.

**-gamma *value***
Legt fest, dass die zur Anzeige dieses Bildes in einem Fenster zugewiesenen Farben für ein nichtlineares Display mit dem angegebenen Gamma-Exponenten korrigiert werden. (Die von den meisten CRT-Displays erzeugte Intensität ist näherungsweise eine Potenzfunktion des Eingabewerts; Gamma ist der Exponent und liegt typischerweise bei etwa 2.) Der angegebene Wert muss größer als null sein. Standard ist eins (keine Korrektur). Im Allgemeinen machen Werte größer als eins das Bild heller, Werte kleiner als eins dunkler.

**-height *number***
Legt die Bildhöhe in Pixeln fest. Die Option ist vor allem nützlich, wenn der Benutzer den Bildinhalt stückweise aufbauen möchte. Null (Standard) erlaubt vertikales Anpassen an die gespeicherten Daten.

**-metadata *metadata***
Setzt das Metadaten-Dictionary des Images. Weitere Schlüssel können im Metadaten-Dictionary des Images gesetzt werden, wenn Bilddaten wegen der Optionen -file oder -data verarbeitet werden und der Treiber Metadaten-Schlüssel liefert. Siehe Abschnitt Metadaten-Dictionary unten.

**-palette *palette-spec***
Legt die Auflösung des zuzuweisenden Farbwürfels und damit die Anzahl der aus den Farbtabellen der Anzeigefenster genutzten Farben fest. *palette-spec* kann eine einzelne Dezimalzahl (Anzahl Graustufen) oder drei durch Schrägstriche (/) getrennte Dezimalzahlen (Anzahl Rot-, Grün- und Blautöne) sein. Bei der ersten Form wird das Bild monochrom (Graustufen) angezeigt.

**-width *number***
Legt die Bildbreite in Pixeln fest. Die Option ist vor allem nützlich, wenn der Benutzer den Bildinhalt stückweise aufbauen möchte. Null (Standard) erlaubt horizontales Anpassen an die gespeicherten Daten.

## Bild-Befehl

Beim Erzeugen eines Foto-Images legt Tk auch einen neuen Befehl an, dessen Name mit dem des Images übereinstimmt. Dieser Befehl kann verschiedene Operationen am Bild ausführen.

Allgemeine Form:

```
imageName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls.

Optionen, die Daten in das Bild schreiben, vergrößern das Bild bei Bedarf, um die geschriebenen Daten aufzunehmen — es sei denn, der Benutzer hat für die Konfigurationsoptionen **-width** und/oder **-height** von null abweichende Werte gesetzt; dann ändern sich Breite und/oder Höhe des Images nicht.

Folgende Befehle sind für Foto-Images möglich:

***imageName* blank**

Leert das Bild; das gesamte Image hat keinen Dateninhalt, wird transparent angezeigt, und der Hintergrund des Anzeigefensters scheint durch. Das Metadaten-Dictionary des Images bleibt unverändert.

***imageName* cget *option***

Gibt den aktuellen Wert der Konfigurationsoption *option* zurück. *Option* kann jeden von image create **photo** akzeptierten Wert haben.

***imageName* configure ?*option*? ?*value option value* ...?**

Fragt Konfigurationsoptionen ab oder ändert sie. Ohne *option* wird eine Liste aller verfügbaren Optionen für *imageName* zurückgegeben (Format siehe Tk_ConfigureInfo). Mit *option* ohne *value* eine Liste nur für diese Option (identisch mit dem entsprechenden Teillisten-Eintrag der vollständigen Liste). Bei *option-value*-Paaren werden die Optionen gesetzt; Rückgabe ist ein leerer String. *Option* kann jeden von image create **photo** akzeptierten Wert haben. Nur die Option -metadata setzen, ohne weitere Option, ruft den Bildformat-Treiber nicht auf, um das Bitmap neu zu erzeugen.

***imageName* copy *sourceImage* ?*option value(s)* ...?**

Kopiert eine Region vom Image *sourceImage* (muss ein Foto-Image sein) nach *imageName*, optional mit Pixel-Zoom und/oder -Unterabtastung. Ohne Optionen wird ganz *sourceImage* ab (0,0) in *imageName* kopiert. Mögliche Optionen:

**-from *x1 y1 x2 y2***

Rechteckige Teilregion der Quelle. (*x1*,*y1*) und (*x2*,*y2*) sind diagonal gegenüberliegende Ecken. Fehlen *x2* und *y2*, ist die Standardeinstellung die untere rechte Ecke der Quelle. Kopierte Pixel umfassen linke und obere Kante des Rechtecks, nicht untere und rechte. Ohne **-from** wird die ganze Quelle verwendet.

**-to *x1 y1 x2 y2***

Rechteckige Teilregion des Ziel-Images. (*x1*,*y1*) und (*x2*,*y2*) sind diagonal gegenüberliegende Ecken. Fehlen *x2* und *y2*, Standard ist (*x1*,*y1*) plus Größe der Quellregion (nach Unterabtastung/Zoom). Sind *x2* und *y2* angegeben, wird die Quellregion bei Bedarf gekachelt, um die Zielregion zu füllen.

**-shrink**

Verkleinert das Ziel-Image bei Bedarf, sodass die kopierte Region in der unteren rechten Ecke liegt. Die Option ändert Breite/Höhe nicht, wenn der Benutzer **-width** bzw. **-height** ungleich null gesetzt hat.

**-zoom *x y***

Vergrößert die Quellregion um Faktor *x* in X- und *y* in Y-Richtung. Fehlt *y*, entspricht es *x*. Jedes Quellpixel wird zu einem *x*×*y*-Block gleicher Farbe im Ziel. *x* und *y* müssen größer als

0 sein.

**-subsample *x y***

Verkleinert die Quelle, indem nur jedes *x*. Pixel in X und jedes *y*. Pixel in Y verwendet wird. Negative Werte spiegeln um die Y- bzw. X-Achse. Fehlt *y*, entspricht es *x*.

**-compositingrule *rule***

Legt fest, wie transparente Quellpixel mit dem Ziel kombiniert werden. Bei *overlay* bleibt alter Zielinhalt sichtbar, als läge die Quelle auf transparentem Film über dem Ziel. Bei *set* wird alter Zielinhalt verworfen und die Quelle unverändert übernommen. Standard ist *overlay*.

***imageName* data ?*option value(s)* ...?**

Gibt Bilddaten als Zeichenkette zurück. Das Format hängt vom Format-Handler ab. Standard ist ein menschenlesbares Format als Liste von Listen von Pixeldaten; andere Formate können mit -format gewählt werden. Siehe Bildformate unten. Mögliche Optionen:

**-background *color***

Ist die Farbe angegeben, enthalten die Daten keine Transparenzinformationen. In allen transparenten Pixeln wird die Farbe durch die angegebene Farbe ersetzt.

**-format *{format-name ?option value ...?}***

Legt den Namen des Bilddateiformat-Handlers fest und optional Argumente dafür. Konkret sucht dieser Unterbefehl den ersten Handler, dessen Name mit einem Anfangs-Substring von *format-name* übereinstimmt und der einen String mit diesen Bilddaten schreiben kann. Ohne diese Option wird das Standardformat verwendet: eine Liste (ein Element pro Zeile) von Listen (ein Element pro Pixel/Spalte) mit Farben im Format "#rrggbb" (siehe Bildformate unten). Der Wert muss eine Tcl-Liste sein; bei einem Wort können geschweifte Klammern entfallen; Anführungszeichen sind möglich.

**-from *x1 y1 x2 y2***

Rechteckige Region von *imageName*, die zurückgegeben wird. Nur *x1* und *y1*: von *(x1,y1)* bis zur unteren rechten Ecke. Alle vier Koordinaten: diagonal gegenüberliegende Ecken, einschließlich x1,y1, ausschließlich x2,y2. Standard: ganzes Bild.

**-grayscale**

Die Daten enthalten keine Farbinformation; alle Pixel werden in Graustufen umgewandelt.

**-metadata *metadata***

Der Bildformat-Handler kann Metadaten in den zurückgegebenen Datenstring aufnehmen. Die angegebenen metadata werden an den Treiber übergeben. Ohne -metadata werden die aktuellen Metadaten des Images verwendet.

***imageName* get *x y* ?-withalpha?**

Gibt die Farbe des Pixels bei (*x*,*y*) als Liste von drei Ganzzahlen zwischen 0 und 255 für Rot, Grün und Blau zurück. Mit **-withalpha** enthält die Liste ein viertes Element: den Alpha-Wert des Pixels als Ganzzahl zwischen 0 und 255.

***imageName* put *data* ?*option value(s)* ...?**

Setzt Pixel in *imageName* auf die in *data* angegebenen Daten. Der Befehl sucht einen Handler, der data interpretieren kann, und liest das kodierte Bild in imageName (Ziel-Image). Siehe Bildformate unten. Mögliche Optionen:

**-format *{format-name ?option value ..?}***

Legt das Format der Bilddaten in *data* fest und optional Argumente für den Format-Handler.

Es werden nur Handler verwendet, deren Name mit *format-name* beginnt. Der Wert muss eine Tcl-Liste sein; bei einem Wort können geschweifte Klammern entfallen; Anführungszeichen sind möglich.

**-metadata *metadata***

Die angegebenen metadata werden an den Bildformat-Treiber beim Interpretieren der Daten übergeben. Die aktuellen Metadaten des Images werden nicht an den Treiber übergeben und durch den Befehl nicht geändert.

**-to *x1 y1* ?*x2 y2*?**

Koordinaten der oberen linken Ecke (*x1*,*y1*) der Zielregion. Standard (0,0). Mit *x2*,*y2* und zu kleinen *data* wird gekachelt. Ist die mit dieser Option angegebene Region kleiner als *data*, werden überschüssige Daten still verworfen. Gibt *data* einen einzelnen Farbwert an, wird die Region bis (*x2*,*y2*) mit dieser Farbe gefüllt.

***imageName* read *filename* ?*option value(s)* ...?**

Liest Bilddaten aus der Datei *filename* ins Image. Zuerst wird ein passender Handler gesucht, dann wird das Bild in *imageName* gelesen. Mögliche Optionen:

**-format *{format-name ?option value ..?}***

Legt das Format der Bilddaten in *filename* fest und optional weitere Argumente für den Format-Handler. Es werden nur Handler verwendet, deren Name mit *format-name* beginnt. Der Wert muss eine Tcl-Liste sein; bei einem Wort können geschweifte Klammern entfallen; Anführungszeichen sind möglich.

**-from *x1 y1 x2 y2***

Rechteckige Teilregion der Dateidaten für das Ziel. Nur *x1* und *y1*: von (*x1,y1*) bis zur unteren rechten Ecke in der Datei. Alle vier Koordinaten: diagonal gegenüberliegende Ecken der Region. Standard: ganzes Bild in der Datei.

**-metadata *metadata***

Die angegebenen metadata werden an den Bildformat-Treiber beim Interpretieren der Daten übergeben. Die aktuellen Metadaten des Images werden nicht an den Treiber übergeben und durch den Befehl nicht geändert.

**-shrink**

Verkleinert *imageName* bei Bedarf, sodass die gelesene Region in der unteren rechten Ecke liegt. Breite/Höhe bleiben unverändert bei gesetztem **-width**/**-height** ungleich null.

**-to *x y***

Koordinaten der oberen linken Ecke der Zielregion. Standard (0,0).

***imageName* redither**

Das beim Anzeigen von Foto-Images verwendete Dithering propagiert Quantisierungsfehler zu Nachbarpixeln. Werden die Bilddaten für *imageName* stückweise geliefert, kann das gedittherte Bild abweichen. Normalerweise fällt das nicht auf; bei Bedarf berechnet dieser Befehl das geditherte Bild in jedem Fenster neu, in dem das Image angezeigt wird.

***imageName* transparency *subcommand* ?*arg* ...?**

Ermöglicht Abfrage und Änderung der Transparenzinformationen im Foto-Image. Verfügbare Unterbefehle:

***imageName* transparency get *x y* ?-alpha?**

Gibt wahr zurück, wenn der Pixel bei (x,y) vollständig transparent ist, sonst falsch. Mit -alpha wird stattdessen der Alpha-Wert des Pixels als Ganzzahl zwischen 0 und 255 zurückgegeben.

***imageName*** **transparency set** ***x y newVal*** **?-alpha?**

Ändert die Transparenz des Pixels bei (x,y) auf newVal. Ohne weitere Option wird newVal als boolescher Wert interpretiert: voll transparent bei wahr, sonst voll opak. Mit -alpha wird newVal als ganzzahliger Alpha-Wert für den Pixel interpretiert (0 bis 255).

***imageName*** **write** ***filename*** **?*option value(s)* ...?**

Schreibt Bilddaten von *imageName* in die Datei *filename*. Folgende Optionen können angegeben werden:

**-background *color***

Ist die Farbe angegeben, enthalten die Daten keine Transparenzinformationen. In allen transparenten Pixeln wird die Farbe durch die angegebene Farbe ersetzt.

**-format *{format-name ?option value ...?}***

Legt den Namen des Bilddateiformat-Handlers fest, der die Daten in die Datei schreibt, und optional Argumente für den Handler. Konkret sucht dieser Unterbefehl den ersten Handler, dessen Name mit einem Anfangs-Substring von *format-name* übereinstimmt und der Bilddateien schreiben kann. Ohne diese Option wird das Format aus der Dateiendung erraten; wenn das nicht möglich ist, wird der erste schreibfähige Handler verwendet. Der Wert muss eine Tcl-Liste sein; bei einem Wort können geschweifte Klammern entfallen, Anführungszeichen sind möglich.

**-from *x1 y1 x2 y2***

Rechteckige Region von *imageName*, die in die Bilddatei geschrieben wird. Nur *x1* und *y1*: von *(x1,y1)* bis zur unteren rechten Ecke von *imageName*. Alle vier Koordinaten: diagonal gegenüberliegende Ecken. Standard: ganzes Bild.

**-grayscale**

Die Daten enthalten keine Farbinformation; alle Pixel werden in Graustufen umgewandelt.

**-metadata *metadata***

Der Bildformat-Handler kann Metadaten in die geschriebene Datei aufnehmen. Die angegebenen metadata werden an den Treiber übergeben. Ohne -metadata werden die aktuellen Metadaten des Images verwendet.

## Bildformate

Der Photo-Image-Code ist so strukturiert, dass Handler für weitere Bilddateiformate leicht ergänzt werden können. Der Photo-Image-Code verwaltet eine Liste dieser Handler. Handler werden der Liste durch Registrierung mit einem Aufruf von Tk_CreatePhotoImageFormat hinzugefügt. Die Standard-Tk-Distribution enthält Handler für PPM/PGM, PNG, GIF und (nur lesend) SVG, sowie den Handler default zum Kodieren/Dekodieren von Bilddaten in menschenlesbarer Form. Diese Handler werden bei der Initialisierung automatisch registriert.

Beim Lesen einer Bilddatei oder Verarbeiten von Stringdaten, die mit der Konfigurationsoption **-data** angegeben wurden, ruft der Photo-Image-Code Handler nacheinander auf, bis einer die Daten in der Datei oder Zeichenkette lesen kann. Meist findet das den richtigen Handler; andernfalls kann der Benutzer mit **-format** einen Formatnamen vorgeben. Dann werden Handler versucht, deren Name mit dem für **-format** angegebenen String beginnt (Groß-/Kleinschreibung irrelevant). Bei **-format gif** können z. B. Handler namens GIF87 oder GIF89 aufgerufen werden, JPEG nicht (sofern registriert).

Beim Schreiben von Bilddaten in eine Datei ist die Verarbeitung von **-format** etwas anders: Der Stringwert für **-format** muss mit dem vollständigen Namen des gewünschten Handlers beginnen und kann weitere Informationen enthalten, die der Handler nutzen kann, z. B. um eine Variante der vom Handler unterstützten Formate zu wählen. Nicht alle Bildhandler unterstützen das Schreiben von Transparenzdaten in eine Datei, auch wenn das Zielformat es theoretisch erlaubt.

### Standard-Bildhandler

Der Handler **default** kann keine Daten aus Dateien lesen oder in Dateien schreiben. Sein einziger Zweck ist die Kodierung und Dekodierung von Bilddaten in Stringform in klarer, menschenlesbarer Form. Der Unterbefehl *imageName* **data** nutzt diesen Handler, wenn kein anderes Format angegeben ist. Beim Lesen von Bilddaten aus einer Zeichenkette mit *imageName* **put** oder der Option **-data** wird der Standard-Handler wie die anderen Handler behandelt.

Bilddaten im **default**-Stringformat sind eine (von oben nach unten) Liste von Scanlines, wobei jede Scanline eine (von links nach rechts) Liste von Pixeldaten ist. Jede Scanline hat dieselbe Länge. Farbe und optional Alpha-Wert jedes Pixels werden in einer der im Abschnitt Farbformate unten beschriebenen Formen angegeben.

### Format-Unteroptionen

Bildformate können Unteroptionen unterstützen, die über zusätzliche Wörter im Wert von -format angegeben werden. Diese Unteroptionen können beeinflussen, wie Bilddaten aus Datei oder String gelesen oder geschrieben werden. Art und Werte dieser Optionen liegen beim Format-Handler. Die eingebauten Handler unterstützen diese Unteroptionen:

**default -colorformat *formatType***

Die Option ist beim Schreiben von Bilddaten in einen String mit imageName data erlaubt. Legt das Format für die Farbzeichenkette jedes Pixels fest. formatType kann sein: rgb für Pixeldaten als #RRGGBB, rgba als #RRGGBBAA oder list als Liste mit vier Elementen. Details siehe Farbformate unten. Standard ist rgb.

**gif -index *indexValue***

Die Option wirkt beim Lesen von Bilddaten aus einer Datei. Beim Parsen eines mehrteiligen GIF-Images greift Tk normalerweise nur auf das erste Bild zu. Mit der Unteroption **-index** kann stattdessen das *indexValue*. Bild verwendet werden. *indexValue* muss eine Ganzzahl von 0 bis zur Anzahl der Bildteile in den GIF-Daten sein.

**png -alpha *alphaValue***

Die Option wirkt beim Lesen von Bilddaten aus einer Datei. Legt einen zusätzlichen Alpha-Filter für das Gesamtbild fest, sodass der Hintergrund der Anzeige durchscheinen kann; das hat meist auch eine Entsättigung zur Folge. *alphaValue* muss zwischen 0,0 und 1,0 liegen.

**svg -dpi *dpiValue* -scale *scaleValue* -scaletowidth *width* -scaletoheight *height***

*dpiValue* wird bei der Umrechnung zwischen angegebenen Koordinaten und Bildschirmauflösung verwendet. Der Wert muss größer als 0 sein; Standard ist 96.

*scaleValue* skaliert das resultierende Bild. Der Wert muss größer als 0 sein; Standard ist 1. *width* und *height* sind Breite bzw. Höhe, auf die das Bild angepasst wird. Nur einer der Parameter **-scale**, **-scaletowidth** und **-scaletoheight** darf angegeben werden; das Seitenverhältnis des Originalbilds bleibt erhalten. Das **svg**-Format unterstützt viele SVG-Funktionen, aber nicht den vollen SVG-Standard — z. B. fehlt die Funktion "text", die beim Lesen der SVG-Daten still ignoriert wird. Unterstützte SVG-Funktionen sind:

**elements:**
g, path, rect, circle, ellipse, line, polyline, polygon, linearGradient, radialGradient, stop, defs, svg, style

**attributes:**
width, height, viewBox, preserveAspectRatio mit none, xMin, xMid, xMax, yMin, yMid, yMax, slice

**gradient attributes:**
gradientUnits mit objectBoundingBox, gradientTransform, cx, cy, r fx, fy x1, y1, x2, y2 spreadMethod mit pad, reflect or repeat, xlink:href

**poly attributes:**
points

**line attributes:**
x1, y1, x2, y2

**ellipse attributes:**
cx, cy, rx, ry

**circle attributes:**
cx, cy, r

**rectangle attributes:**
x, y, width, height, rx, ry

**path attributes:**
d mit m, M, l, L, h, H, v, V, c, C, s, S, q, Q, t, T, a, A, z, Z

**style attributes:**
display mit none, visibility, hidden, visible, fill mit nonzero and evenodd, opacity, fill-opacity, stroke, stroke-width, stroke-dasharray, stroke-dashoffset, stroke-opacity, stroke-linecap mit butt, round and square, stroke-linejoin mit miter, round and bevel, stroke-miterlimit fill-rule, font-size, transform mit matrix, translate, scale, rotate, skewX and skewY, stop-color, stop-opacity, offset, id, class

Derzeit nur Lesen von SVG und Konvertierung in pixelbasierte Photos; Einbetten von Photos in SVG-Vektorgrafik wird (noch) nicht unterstützt.

## Farbformate

Der Standard-Bildhandler kann Farb- und Alpha-Werte eines Pixels in einem der unten aufgeführten Formate darstellen oder parsen. Enthält ein Farbformat keine Transparenzinformation, wird volle Deckkraft angenommen. Verfügbare Farbformate:

- Leerer String — wird als volle Transparenz interpretiert; der Farbwert ist undefiniert.
- Jeder von Tk_GetColor akzeptierte Wert, optional gefolgt von einem Alpha-Suffix. Das Alpha-Suffix kann eines von Folgendem sein:

***@A***
Der Alpha-Wert *A* muss ein Bruchteil im Bereich 0,0 (vollständig transparent) bis 1,0 (vollständig opak) sein.

***#X***
Der Alpha-Wert *X* ist eine Hexadezimalziffer, die einen ganzzahligen Alpha-Wert im Bereich 0 (vollständig transparent) bis 255 (vollständig opak) angibt. Der Wert wird von 4 Bit auf 8

Bit durch Multiplikation mit 0x11 erweitert.

***#XX***

Der Alpha-Wert *XX* wird als zwei Hexadezimalziffern übergeben, die einen ganzzahligen Alpha-Wert im Bereich 0 (vollständig transparent) bis 255 (vollständig opak) angeben.

- Eine Tcl-Liste mit drei oder vier Ganzzahlen im Bereich 0 bis 255 für die Kanäle Rot, Grün, Blau und (optional) Alpha.
- Format **#***RGBA*: ein **#** gefolgt von vier Hexadezimalziffern, wobei jede Ziffer den Wert für Rot, Grün, Blau und Alpha angibt. Jede Ziffer wird intern durch Multiplikation mit 0x11 auf 8 Bit erweitert.
- Format **#***RRGGBBAA*: **#** gefolgt von acht Hexadezimalziffern, wobei jedes Paar aufeinanderfolgender Ziffern den Wert für Rot, Grün, Blau und Alpha darstellt.

## Farbzuweisung

Wird ein Foto-Image in einem Fenster angezeigt, weist der Photo-Image-Code Farben zu und dithert das Bild bei Bedarf, um eine brauchbare Annäherung mit den verfügbaren Farben darzustellen. Die Farben werden als Farbwürfel zugewiesen: die Anzahl der zugewiesenen Farben ist das Produkt der Anzahl der Rot-, Grün- und Blautöne.

Normalerweise richtet sich die Anzahl der zugewiesenen Farben nach der Tiefe des Fensters. In einem 8-Bit-PseudoColor-Fenster versucht der Photo-Image-Code z. B., sieben Rottöne, sieben Grüntöne und vier Blautöne zuzuweisen (insgesamt 198 Farben). In einem 1-Bit-StaticGray-Fenster (Monochrom) werden zwei Farben, Schwarz und Weiß, zugewiesen. In einem 24-Bit-DirectColor- oder TrueColor-Fenster werden 256 Stufen je für Rot, Grün und Blau zugewiesen. Dank der Art, wie Pixelwerte in DirectColor- und TrueColor-Fenstern kombiniert werden können, reichen dafür 256 zugewiesene Farben. Können nicht alle Farben zugewiesen werden, reduziert der Photo-Image-Code die Anzahl der Stufen pro Grundfarbe und versucht es erneut.

Der Benutzer kann mit der Konfigurationsoption **-palette** steuern, wie viele Farben ein Foto-Image nutzt. Mit dieser Option wird die maximale Anzahl der Stufen pro Grundfarbe festgelegt, die zugewiesen werden sollen. Sie kann auch erzwingen, dass das Bild in Graustufen angezeigt wird, auch auf einem Farbdisplay, indem eine einzelne Zahl statt drei durch Schrägstriche getrennte Zahlen angegeben wird.

## Metadaten-Dictionary

Jedes Image hat eine Metadaten-Dictionary-Eigenschaft. Dieses Dictionary ist für die Bitmap-Darstellung des Images nicht relevant, kann aber zusätzliche Informationen wie Auflösung oder Kommentare enthalten. Bildformat-Treiber können beim Parsen von Bilddaten Metadaten ausgeben oder Metadaten zum Einbinden in Bilddateien oder -formate nutzen.

### Metadaten-Schlüssel (mehrere Formate)

Jeder Bildformat-Treiber unterstützt eine eigene Menge von Metadaten-Dictionary-Schlüsseln. Vordefinierte Schlüssel sind:

**DPI**

Horizontale Bildauflösung in DPI als double-Wert. Unterstützt vom Format **png**.

**aspect**

Seitenverhältnis horizontal geteilt durch vertikal als double-Wert. Unterstützt von den

Formaten **gif** und **png**.

**comment**
Textkommentar zum Bild. Unterstützt von den Formaten **gif** und **png**.

Es ist gültig, beliebige Schlüssel im Metadaten-Dict zu setzen. Ein Format-Treiber ignoriert Schlüssel, die er nicht verarbeitet.

### Metadaten-Schlüssel für animierte GIF-Informationen

Die folgenden Metadaten-Schlüssel werden beim Lesen einer **gif**-Formatdatei gemeldet. Sie werden typischerweise zusammen mit der Option **-index** einer animierten **gif**-Datei verwendet, um die Teilbildsequenz korrekt anzuzeigen. Die Optionen beziehen sich auf das jeweils mit **-index** gewählte Teilbild.

**delay time *time***
Aktualisierungsverzögerung in 10-ms-Einheiten. Dieser Schlüssel ist nur vorhanden, wenn die Verzögerung nicht 0 ist.

**disposal method *method***
Entsorgungsmethode des vorhergehenden Bildes, falls für das aktuelle Bild angegeben. Mögliche Werte: **do not dispose**, **restore to background color**, **restore to previous**.

**user interaction *bool***
Der Schlüssel ist mit dem Wert 1 vorhanden, wenn Benutzerinteraktion vorgesehen ist. Andernfalls fehlt der Schlüssel.

**update region *X0*, *Y0*, *width*, *height***
Aktualisierungsregion des aktuellen Teilbilds, wenn das Teilbild nicht dieselbe Größe wie das vollständige Bild hat. Pixel außerhalb dieses Bereichs sind vollständig transparent.

## Danksagungen

Der Photo-Image-Typ wurde von Paul Mackerras entworfen und implementiert, auf Basis seines früheren Photo-Widgets und einiger Vorschläge von John Ousterhout.

## Beispiel

Bild aus einer Datei laden und auf Fenstergröße kacheln — nützlich für einen gekachelten Hintergrund:

```
# Diese Zeilen sollten einmal aufgerufen werden
image create photo untiled -file "theFile.ppm"
image create photo tiled

# Diese Zeilen sollten aufgerufen werden, wenn sich .someWidget in der
    Größe ändert;
# eine <Configure>-Bindung ist hier hilfreich
set width  [winfo width .someWidget]
set height [winfo height .someWidget]
tiled copy untiled -to 0 0 $width $height -shrink
```

Der PNG-Bildlader erlaubt die Anwendung eines zusätzlichen Alpha-Faktors beim Laden — nützlich für Bilder deaktivierter Buttons:

```
image create photo icon -file "icon.png"
image create photo iconDisabled -file "icon.png" \
        -format "png -alpha 0.5"
button .b -image icon -disabledimage iconDisabled
```

Grüne Box mit einfachem Schatten

```
image create photo foo

# Einfache Verlaufsfüllung mit variierendem Alpha für den Schatten
for {set i 14} {$i > 0} {incr i -1} {
   set i2 [expr {$i + 30}]
   foo put [format black#%x [expr {15-$i}]] -to $i $i $i2 $i2
}

# Solides grünes Rechteck darüberlegen
foo put #F080 -to 0 0 30 30
```

*Copyright (c) 1994 The Australian National University*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# place

*Stichwörter:* place, geometry manager, height, location, container, rubber sheet, content, width

## Name

place - Geometriemanager für feste oder Gummiertafel-Platzierung

## Übersicht

**place** *option arg* ?*arg* ...?

## Beschreibung

Der Placer ist ein Geometriemanager für Tk. Er ermöglicht einfache feste Platzierung von Fenstern, bei der Sie die exakte Größe und Position eines Fensters, des *Inhalts*, innerhalb eines anderen Fensters, des *Containers*, angeben. Der Placer bietet außerdem Gummiertafel-Platzierung, bei der Sie Größe und Position des Inhalts in Bezug auf die Abmessungen des Containers angeben, sodass der Inhalt bei Größenänderungen des Containers seine Größe und Position anpasst. Schließlich erlaubt der Placer, diese Platzierungsstile zu mischen, sodass der Inhalt beispielsweise feste Breite und Höhe hat, aber im Container zentriert ist.

**place *window option value* ?*option value ...*?**

Veranlasst den Placer, die Geometrie eines Inhalts zu verwalten, dessen Pfadname *window* ist. Die übrigen Argumente bestehen aus einem oder mehreren *option-value*-Paaren, die festlegen, wie die Geometrie von *window* verwaltet wird. *Option* kann jeden von **place configure** akzeptierten Wert haben.

**place configure *window* ?*option*? ?*value option value ...*?**

Abfragen oder Ändern der Geometrie-Optionen des durch *window* angegebenen Inhalts. Ist keine *option* angegeben, gibt der Befehl eine Liste der verfügbaren Optionen zurück (Format siehe Tk_ConfigureInfo). Ist *option* ohne *value* angegeben, gibt der Befehl eine Liste zur benannten Option zurück (identisch mit dem entsprechenden Teillistenwert bei Abfrage aller Optionen). Sind ein oder mehrere *option-value*-Paare angegeben, werden die Optionen auf die angegebenen Werte gesetzt; der Befehl gibt dann einen leeren String zurück.

Folgende *option-value*-Paare werden unterstützt:

**-anchor *where***

*Where* legt fest, welcher Punkt von *window* an der durch **-x**, **-y**, **-relx** und **-rely** gewählten (x,y)-Position platziert wird. Der Ankerpunkt bezieht sich auf den äußeren Bereich von *window* einschließlich Rahmen, falls vorhanden. Ist *where* **se**, erscheint die untere rechte Ecke des Rahmens von *window* an der angegebenen (x,y)-Position im Container. Standard-Ankerposition ist **nw**.

**-bordermode *mode***

*Mode* bestimmt, inwieweit Rahmen innerhalb des Containers bei der Platzierung des Inhalts berücksichtigt werden. Standard und häufigster Wert ist **inside**. Der Placer betrachtet dann den Container-Bereich als innersten Bereich innerhalb jedes Rahmens: **-x 0** entspricht einer x-Koordinate knapp innerhalb des Rahmens und **-relwidth 1.0** bedeutet, dass *window* den Bereich innerhalb des Container-Rahmens füllt.

Ist *mode* **outside**, gilt der Container-Bereich inklusive Rahmen; typisch beim Platzieren von *window* außerhalb des Containers, z. B. mit **-x 0 -y 0 -anchor ne**. Ist *mode* **ignore**, werden Rahmen ignoriert: Der Container-Bereich ist die offizielle X-Fläche mit internem, aber ohne externem Rahmen. **ignore** ist vermutlich wenig nützlich.

**-height *size***

*Size* legt die Höhe von *window* in Bildschirmeinheiten fest (Formen wie von Tk_GetPixels akzeptiert). Die Höhe ist die äußere Abmessung von *window* einschließlich Rahmen. Ist *size* leer oder sind weder **-height** noch **-relheight** gesetzt, wird die intern angeforderte Fensterhöhe verwendet.

**-in *container***

*Container* gibt den Pfadnamen des Fensters an, relativ zu dem *window* platziert wird. *Container* muss Eltern von *window* oder ein Nachfahre des Eltern-Widgets sein. Außerdem müssen *container* und *window* Nachfahren desselben Toplevel-Fensters sein. Dies garantiert, dass *window* sichtbar ist, sobald *container* sichtbar ist. Ohne diese Option ist das Bezugsfenster standardmäßig das Eltern-Widget von *window*.

**-relheight *size***

*Size* legt die Höhe von *window* als Gleitkommazahl relativ zur Container-Höhe fest: 0.5 bedeutet halbe Container-Höhe, 1.0 gleiche Höhe usw. Sind **-height** und **-relheight** gesetzt, werden die Werte addiert. Beispiel: **-relheight 1.0 -height -2** macht den Inhalt 2 Pixel kürzer als den Container.

**-relwidth *size***

*Size* legt die Breite von *window* relativ zur Container-Breite fest. 0.5 bedeutet halbe Breite, 1.0 gleiche Breite usw. Sind **-width** und **-relwidth** gesetzt, werden die Werte addiert. Beispiel: **-relwidth 1.0 -width 5** macht den Inhalt 5 Pixel breiter als den Container.

**-relx *location***

*Location* legt die x-Koordinate des Ankerpunkts von *window* im Container relativ fest: 0.0 linke Kante, 1.0 rechte Kante. *Location* muss nicht zwischen 0.0 und 1.0 liegen. Sind **-x** und

**-relx** gesetzt, werden die Werte addiert. Beispiel: **-relx 0.5 -x -2** positioniert die linke Kante des Inhalts 2 Pixel links von der Container-Mitte.

**-rely *location***

*Location* legt die y-Koordinate des Ankerpunkts relativ fest: 0.0 obere Kante, 1.0 untere Kante. *Location* muss nicht zwischen 0.0 und 1.0 liegen. Sind **-y** und **-rely** gesetzt, werden die Werte addiert. Beispiel: **-rely 0.5 -x 3** positioniert die obere Kante des Inhalts 3 Pixel unter der Container-Mitte.

**-width *size***

*Size* legt die Breite von *window* in Bildschirmeinheiten fest (Formen wie von Tk_GetPixels akzeptiert). Die Breite ist die äußere Breite einschließlich Rahmen. Ist *size* leer oder sind weder **-width** noch **-relwidth** gesetzt, wird die intern angeforderte Breite verwendet.

**-x *location***

*Location* legt die x-Koordinate des Ankerpunkts im Container in Bildschirmeinheiten fest (Formen wie von Tk_GetPixels akzeptiert); sie muss nicht innerhalb des Containers liegen.

**-y *location***

*Location* legt die y-Koordinate des Ankerpunkts im Container in Bildschirmeinheiten fest (Formen wie von Tk_GetPixels akzeptiert); sie muss nicht innerhalb des Containers liegen.

Wird derselbe Wert mit zwei verschiedenen Optionen angegeben, etwa **-x** und **-relx**, gilt die zuletzt gesetzte Option; die ältere wird ignoriert.

**place content *window***

Gibt eine Liste aller Inhaltsfenster zurück, für die *window* der Container ist. Gibt es keinen Inhalt für *window*, wird ein leerer String zurückgegeben.

**place forget *window***

Der Placer verwaltet die Geometrie von *window* nicht mehr. Als Nebeneffekt wird *window* unmapped, sodass es nicht auf dem Bildschirm erscheint. Wird *window* nicht vom Placer verwaltet, hat der Befehl keine Wirkung. Rückgabe ist ein leerer String.

**place info *window***

Gibt eine Liste mit der aktuellen Konfiguration von *window* zurück. Die Liste besteht aus *option-value*-Paaren im gleichen Format wie für **place configure**.

**place slaves *window***

Synonym für **place content** *window*.

Wurde die Konfiguration eines Fensters mit **place info** abgerufen, kann sie später wiederhergestellt werden, indem zuerst **place forget** alle vorhandenen Informationen löscht und anschließend **place configure** mit den gespeicherten Werten aufgerufen wird.

## Feinheiten

Der Container muss nicht das Eltern-Widget des Inhalts sein. Das ist in mindestens zwei Situationen nützlich. Erstens können Sie bei komplexen Layouts eine Hierarchie von Unterfenstern anlegen, deren einziger Zweck das Layout des Eltern-Widgets ist. Die "echten Kinder" des Eltern-Widgets (die für die Benutzeroberfläche relevanten Fenster) können Kinder des Eltern-Widgets sein und dennoch innerhalb der Fenster der Geometrie-Hierarchie platziert werden. Die Pfadnamen der echten Kinder spiegeln die Geometrie-Hierarchie nicht wider, und Benutzer können Optionen für die echten Kinder setzen, ohne die Geometrie-Struktur zu kennen.

Ein zweiter Grund für einen von den Eltern verschiedenen Container ist die Verknüpfung zweier

Geschwister. Der Placer kann ein Fenster z. B. immer zentriert direkt unter einem Geschwister halten mit

```
-in sibling -relx 0.5 -rely 1.0 -anchor n -bordermode outside
```

Wird das Geschwister später verschoben, wird der Inhalt ebenfalls neu positioniert.

Anders als viele andere Geometriemanager (z. B. der Packer) manipuliert der Placer nicht die Geometrie der Container-Fenster oder der Eltern von Inhaltsfenstern (er setzt deren angeforderte Größen nicht). Um die Größen zu steuern, verwenden Sie Fenster wie **frame** und **canvas**, die Konfigurationsoptionen dafür bieten.

### Beispiel

Das Label soll den mittleren Bereich des Toplevel einnehmen, unabhängig von der Größenänderung:

```
label .l -text "In the\nMiddle!" -bg black -fg white
place .l -relwidth .3 -relx .35 -relheight .3 -rely .35
```

*Copyright (c) 1992 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## popup

*Stichwörter:* popup, menu

### Name

tk_popup - Blendet ein Popup-Menü ein

### Übersicht

**tk_popup** *menu x y* ?*entry*?

### Beschreibung

Diese Prozedur blendet ein Menü an einer gegebenen Bildschirmposition ein und konfiguriert Tk so, dass das Menü und seine kaskadierten Kindmenüs mit Maus oder Tastatur durchlaufen werden können. *Menu* ist der Name eines Menü-Widgets; *x* und *y* sind die Root-Koordinaten, an denen das Menü angezeigt wird. Wird *entry* weggelassen oder ist es eine leere Zeichenkette, wird die obere linke Ecke des Menüs an den angegebenen Punkt positioniert. Andernfalls gibt *entry* den Index eines Eintrags in *menu* an, und das Menü wird so positioniert, dass dieser Eintrag über dem angegebenen Punkt liegt.

### Beispiel

So bindet man ein einfaches Popup-Menü an ein Widget.

```
# Menü erzeugen
set m [menu .popupMenu]
$m add command -label "Example 1" -command bell
$m add command -label "Example 2" -command bell

# Ziel-Widget erzeugen
pack [label .l -text "Click me!"]
```

```
# Menü beim Klick auf das Label einblenden
bind .l <Button-1> {tk_popup .popupMenu %X %Y}
```

*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# print

*Stichwörter:* print, output, graphics, text, canvas

## Name

print - Canvas- und Text-Widgets mit nativen Dialogen und APIs drucken

## Übersicht

**tk print** *window*

## Beschreibung

Der Befehl tk print öffnet einen Dialog, mit dem Benutzer die Ausgabe von canvas- und text -Widgets drucken können. Der Druck erfolgt, wo verfügbar, über plattformnative APIs und Dialoge.

Das canvas-Widget unterstützt seit langem PostScript-Export; sowohl PostScript- als auch Textdateien können auf Unix-ähnlichen Systemen direkt mit den Unix-Befehlen "lp" und "lpr" an einen Drucker gesendet werden. Der Befehl tk print ersetzt diese Funktionalität nicht, sondern baut darauf auf. tk print ist eine vollständigere Implementierung, die auf macOS und Windows native Dialoge nutzt und auf X11 einen Tk-basierten Dialog mit vergleichbarer Funktionalität bereitstellt.

## Plattformhinweise

### macOS

Die Mac-Implementierung verwendet native Druckdialoge und stützt sich auf das darunterliegende Common Unix Printing System (CUPS), um Textausgabe aus dem Text-Widget und die PDF-Konvertierung der Canvas-Daten an den Drucker oder in eine PDF-Datei zu rendern.

### Windows

Die Windows-Implementierung basiert auf der GDI-API (Graphics Device Interface). Da GDI und das Tk-canvas-Widget Grafik leicht unterschiedlich darstellen, kann die gedruckte Canvas-Ausgabe unter Windows von der Bildschirmdarstellung abweichen.

### X11

Die X11-Implementierung nutzt eine Tk-GUI zur Konfiguration von Druckaufträgen, die per "lpr" oder "lp" oder über die API des Common Unix Printing System (CUPS) an einen Drucker gesendet werden, wenn libcups installiert ist. Diese Befehle haben viele Parameter zur Druckauftragskonfiguration; Drucker unterstützen sie jedoch sehr unterschiedlich. Daher werden nur Druckerauswahl und Anzahl der Kopien als Argumente des print-Befehls konfiguriert; viele Aspekte der Druckdarstellung (z. B. Graustufen oder Farbe für das Canvas) werden stattdessen bei der PostScript-Erzeugung eingestellt.

*Copyright (c) 2021 Kevin Walzer.*

# radiobutton

*Stichwörter:* radiobutton, widget

## Name

radiobutton - Erstellt und manipuliert 'Radiobutton'-Widgets für Einzelauswahl

## Übersicht

**radiobutton** *pathName* ?*options*?

## Standardoptionen

**-activebackground**, **-disabledforeground**, **-padx**, **-activeforeground**, **-font**, **-pady**, **-anchor**, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-takefocus**, **-bitmap**, **-highlightcolor**, **-text**, **-borderwidth**, **-highlightthickness**, **-textvariable**, **-compound**, **-image**, **-underline**, **-cursor**, **-justify**, **-wraplength**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-command (command/Command)**

Legt einen Tcl-Befehl fest, der mit dem Button verknüpft wird. Dieser Befehl wird typischerweise aufgerufen, wenn Maustaste 1 über dem Button-Fenster losgelassen wird. Die globale Variable des Buttons (Option **-variable**) wird vor dem Aufruf des Befehls aktualisiert.

**-height (height/Height)**

Legt eine gewünschte Höhe für den Button fest. Wenn ein Bild oder Bitmap im Button angezeigt wird, ist der Wert in Bildschirmeinheiten (d.h. alle von Tk_GetPixels akzeptierten Formen); für Text ist es in Textzeilen. Wenn diese Option nicht angegeben ist, wird die gewünschte Höhe des Buttons aus der Größe des angezeigten Bildes, Bitmaps oder Textes berechnet.

**-indicatoron (indicatorOn/IndicatorOn)**

Legt fest, ob der Indikator gezeichnet werden soll. Muss ein gültiger boolescher Wert sein. Wenn false, wird die Option **-relief** ignoriert und das Relief des Widgets ist stets eingesenkt, wenn das Widget ausgewählt ist, und sonst erhaben.

**-offrelief (offRelief/OffRelief)**

Legt das Relief für den Radiobutton fest, wenn der Indikator nicht gezeichnet wird und der Radiobutton abgewählt ist. Der Standardwert ist "raised". Durch Setzen dieser Option auf "flat" und **-indicatoron** auf false sowie **-overrelief** auf "raised" entsteht ein flacher Button, der bei Maus-Over erhaben wird und beim Aktivieren eingedrückt ist. Dies ist typisches Verhalten der Links-ausrichten-, Rechts-ausrichten- und Zentrieren-Radiobuttons in der Symbolleiste einer Textverarbeitung.

**-overrelief (overRelief/OverRelief)**

Legt ein alternatives Relief für den Radiobutton fest, das verwendet wird, wenn der Mauszeiger über dem Widget ist. Diese Option kann verwendet werden, um Toolbar-Buttons zu erstellen, indem **-relief flat -overrelief raised** konfiguriert wird. Wenn der Wert dieser Option der leere String ist, wird kein alternatives Relief verwendet, wenn der Mauszeiger über dem Radiobutton ist. Der leere String ist der Standardwert.

**-selectcolor (selectColor/Background)**

Legt eine Hintergrundfarbe fest, die verwendet wird, wenn der Button ausgewählt ist. Wenn **indicatorOn** true ist, wird die Farbe als Hintergrund für den Indikator verwendet, unabhängig vom Auswahlzustand. Wenn **-indicatoron** false ist, wird diese Farbe als Hintergrund für das gesamte Widget anstelle von **-background** oder **-activeBackground** verwendet, sobald das Widget ausgewählt ist. Wenn als leerer String angegeben, wird keine besondere Farbe für die Anzeige im ausgewählten Zustand verwendet.

**-selectimage (selectImage/SelectImage)**

Legt ein Bild fest, das (anstelle der Option **-image**) angezeigt wird, wenn der Radiobutton ausgewählt ist. Diese Option wird ignoriert, sofern die Option **-image** nicht angegeben wurde.

**-state (state/State)**

Legt einen von drei Zuständen für den Radiobutton fest: **normal**, **active** oder **disabled**. Im normalen Zustand wird der Radiobutton mit den Optionen **-foreground** und **-background** angezeigt. Der aktive Zustand wird typischerweise verwendet, wenn der Zeiger über dem Radiobutton ist. Im aktiven Zustand wird der Radiobutton mit den Optionen **-activeforeground** und **-activebackground** angezeigt. Der deaktivierte Zustand bedeutet, dass der Radiobutton unempfindlich sein sollte: Die Standard-Bindings weigern sich, das Widget zu aktivieren und ignorieren Mausklicks. In diesem Zustand bestimmen die Optionen **-disabledforeground** und **-background**, wie der Radiobutton angezeigt wird.

**-tristateimage (tristateImage/TristateImage)**

Legt ein Bild fest, das (anstelle der Option **-image**) angezeigt wird, wenn der Radiobutton ausgewählt ist. Diese Option wird ignoriert, sofern die Option **-image** nicht angegeben wurde.

**-tristatevalue (tristateValue/Value)**

Legt den Wert fest, der bewirkt, dass der Radiobutton die Mehrfachauswahl anzeigt, auch bekannt als Drei-Zustand-Modus. Standardwert ist "".

**-value (value/Value)**

Legt den Wert fest, der in der zugehörigen Variable des Buttons gespeichert wird, wenn dieser Button ausgewählt wird.

**-variable (**variable**/Variable)**

Gibt den Namen einer globalen Variable an, die gesetzt wird, wenn dieser Button ausgewählt wird. Änderungen an dieser Variable bewirken auch, dass der Button sich selbst aus- bzw. abwählt. Standardwert ist **selectedButton**.

**-width (width/Width)**

Legt eine gewünschte Breite für den Button fest. Wenn ein Bild oder Bitmap im Button angezeigt wird, ist der Wert in Bildschirmeinheiten (d.h. alle von Tk_GetPixels akzeptierten Formen); für Text ist es in Zeichen. Wenn diese Option nicht angegeben ist, wird die gewünschte Breite des Buttons aus der Größe des angezeigten Bildes, Bitmaps oder Textes berechnet.

## Beschreibung

Der Befehl **radiobutton** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Radiobutton-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte des Radiobuttons wie Farben, Schriftart, Text und anfängliches Relief zu konfigurieren. Der Befehl **radiobutton** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Ein Radiobutton ist ein Widget, das einen Text-String, ein Bitmap oder ein Bild sowie eine Raute oder einen Kreis namens *Indikator* anzeigt. Wenn Text angezeigt wird, muss er in einer einzigen Schriftart sein, kann aber mehrere Zeilen auf dem Bildschirm einnehmen (wenn er Zeilenumbrüche enthält oder wenn ein Umbruch wegen der Option **-wraplength** erfolgt), und eines der Zeichen kann optional mit der Option **-underline** unterstrichen werden. Ein Radiobutton hat das gesamte Verhalten eines einfachen Buttons: Er kann sich je nach Option **-state** auf drei verschiedene Arten darstellen; er kann erhaben, eingesenkt oder flach dargestellt werden; er kann blinken; und er ruft einen Tcl-Befehl auf, wenn Maustaste 1 über dem Radiobutton geklickt wird.

Darüber hinaus können Radiobuttons *ausgewählt* werden. Wenn ein Radiobutton ausgewählt ist, wird der Indikator normalerweise mit ausgewähltem Erscheinungsbild gezeichnet, und eine mit dem Radiobutton verknüpfte Tcl-Variable wird auf einen bestimmten Wert gesetzt (normalerweise 1). Unter Unix wird der Indikator mit eingesenktem Relief und einer besonderen Farbe gezeichnet. Unter Windows wird der Indikator mit einem runden Markierungspunkt im Inneren gezeichnet. Wenn der Radiobutton nicht ausgewählt ist, wird der Indikator mit nicht ausgewähltem Erscheinungsbild gezeichnet, und die zugehörige Variable wird auf einen anderen Wert gesetzt (typischerweise 0). Der Indikator wird ohne runden Markierungspunkt im Inneren gezeichnet. Typischerweise teilen sich mehrere Radiobuttons eine einzelne Variable, und der Wert der Variable gibt an, welcher Radiobutton ausgewählt werden soll. Wenn ein Radiobutton ausgewählt wird, setzt er den Wert der Variable, um dies anzuzeigen; jeder Radiobutton überwacht auch den Wert der Variable und wählt sich automatisch ab bzw. an, wenn sich der Wert der Variable ändert. Wenn der Wert der Variable mit **-tristatevalue** übereinstimmt, wird der Radiobutton im Drei-Zustand-Modus gezeichnet. Dieser Modus wird verwendet, um gemischte oder mehrere Werte anzuzeigen. (Dies wird verwendet, wenn der Radiobutton den Zustand mehrerer Elemente repräsentiert.) Standardmäßig wird die Variable **selectedButton** verwendet; ihr Inhalt gibt den Namen des ausgewählten Buttons an oder den leeren String, wenn kein mit dieser Variable verknüpfter Button ausgewählt ist. Der Name der Variable für einen Radiobutton sowie die darin zu speichernde Variable können mit Optionen in der Befehlszeile oder in der Optionsdatenbank geändert werden. Konfigurationsoptionen können auch verwendet werden, um die Art der Indikatoranzeige zu ändern (oder ob er überhaupt angezeigt wird). Standardmäßig ist ein Radiobutton so konfiguriert, dass er sich bei Button-Klicks selbst auswählt.

## Widget-Befehl

Der Befehl **radiobutton** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Radiobutton-Widgets möglich:

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **radiobutton** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **radiobutton** akzeptierten Werte haben.

***pathName* deselect**

Wählt den Radiobutton ab und setzt die zugehörige Variable auf einen leeren String. Wenn dieser Radiobutton derzeit nicht ausgewählt war, hat der Befehl keine Wirkung.

***pathName* flash**

Lässt den Radiobutton blinken. Dies wird erreicht, indem der Radiobutton mehrmals neu angezeigt wird und dabei zwischen aktiven und normalen Farben gewechselt wird. Am Ende des Blinkens bleibt der Radiobutton im gleichen normal/aktiven Zustand wie beim Aufruf des Befehls. Dieser Befehl wird ignoriert, wenn der Zustand des Radiobuttons **disabled** ist.

***pathName* invoke**

Bewirkt genau das, was passiert wäre, wenn der Benutzer den Radiobutton mit der Maus aufgerufen hätte: wählt den Button aus und ruft den mit dem Radiobutton verknüpften Tcl-Befehl auf, falls einer vorhanden ist. Der Rückgabewert ist der Rückgabewert des Tcl-Befehls oder ein leerer String, wenn kein Befehl mit dem Radiobutton verknüpft ist. Dieser Befehl wird ignoriert, wenn der Zustand des Radiobuttons **disabled** ist.

***pathName* select**

Wählt den Radiobutton aus und setzt die zugehörige Variable auf den Wert, der diesem Widget entspricht.

## Standard-Bindings

Tk erstellt automatisch Klassen-Bindings für Radiobuttons, die ihnen folgendes Standardverhalten geben:

1. Auf Unix-Systemen wird ein Radiobutton aktiviert, wenn die Maus über ihn fährt, und deaktiviert, wenn die Maus den Radiobutton verlässt. Auf Mac- und Windows-Systemen wird der Button aktiviert, wenn Maustaste 1 über einem Radiobutton gedrückt wird und der Mauszeiger sich innerhalb des Buttons befindet, und deaktiviert, wenn der Mauszeiger den Button verlässt.
2. Wenn Maustaste 1 über einem Radiobutton gedrückt wird, wird er aufgerufen (er wird ausgewählt und der mit dem Button verknüpfte Befehl wird aufgerufen, falls einer vorhanden ist).
3. Wenn ein Radiobutton den Eingabefokus hat, bewirkt die Leertaste, dass der Radiobutton aufgerufen wird.

Wenn der Zustand des Radiobuttons **disabled** ist, tritt keine der obigen Aktionen auf: Der Radiobutton reagiert überhaupt nicht.

Das Verhalten von Radiobuttons kann geändert werden, indem neue Bindings für einzelne Widgets definiert werden oder indem die Klassen-Bindings neu definiert werden.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# raise

*Stichwörter:* raise, obscure, stacking order

## Name

raise - Ändert die Position eines Fensters in der Stapelreihenfolge

## Übersicht

**raise** *window* ?*aboveThis*?

## Beschreibung

Ist das Argument *aboveThis* weggelassen, hebt **raise** *window* so an, dass es über allen Geschwister-Fenstern in der Stapelreihenfolge liegt (es wird von keinem Geschwister verdeckt und verdeckt überlappende Geschwister). Ist *aboveThis* angegeben, muss es der Pfadname eines Fensters sein, das Geschwister von *window* ist oder Nachkomme eines Geschwisters von *window*. Dann fügt **raise** *window* in die Stapelreihenfolge direkt über *aboveThis* ein (oder über den Vorfahren von *aboveThis*, der Geschwister von *window* ist); das kann *window* anheben oder absenken.

Alle toplevel-Fenster können zueinander umgestapelt werden, unabhängig von ihren Pfadnamen; der Window-Manager ist jedoch nicht verpflichtet, Umstapelungswünsche strikt zu erfüllen.

Unter macOS führt das Anheben eines iconisierten toplevel-Fensters zur De-Iconifizierung.

## Beispiele

Ein Button soll in einem Geschwister-Frame erscheinen, der nach dem Button erzeugt wurde — oft nötig, wenn Widgets zuerst erzeugt und dann layoutet werden:

```
button .b -text "Hi there!"
pack [frame .f -background blue]
pack [label .f.l1 -text "This is above"]
pack .b -in .f
pack [label .f.l2 -text "This is below"]
raise .b
```

*Copyright (c) 1990 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# scale

*Stichwörter:* scale, slider, trough, widget

## Name

scale - Erstellt und manipuliert 'Scale'-Widgets mit wertgesteuertem Schieberegler

## Übersicht

**scale** *pathName* ?*options*?

## Standardoptionen

**-activebackground**, **-foreground**, **-relief**, **-background**, **-highlightbackground**, **-repeatdelay**, **-borderwidth**, **-highlightcolor**, **-repeatinterval**, **-cursor**, **-highlightthickness**, **-takefocus**, **-font**, **-orient**, **-troughcolor**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-bigincrement (bigIncrement/BigIncrement)**

Einige Interaktionen mit der Scale bewirken, dass sich ihr Wert in "großen" Schritten ändert; diese Option legt die Größe der großen Schritte fest. Wenn als 0 angegeben, entsprechen die großen Schritte standardmäßig 1/10 des Wertebereichs der Scale.

**-command (command/Command)**

Legt das Präfix eines Tcl-Befehls fest, der aufgerufen wird, wenn sich der Wert der Scale über einen Widget-Befehl ändert. Der tatsächliche Befehl besteht aus dieser Option gefolgt von einem Leerzeichen und einer reellen Zahl, die den neuen Wert der Scale angibt.

**-digits (digits/Digits)**

Eine Ganzzahl, die angibt, wie viele signifikante Stellen beim Umwandeln des Werts der Scale in einen String beibehalten werden sollen. Wenn die Zahl negativ oder null ist, wählt die Scale den kleinsten Wert, der garantiert, dass jede mögliche Schiebereglerposition als unterschiedlicher String ausgegeben wird.

**-from (from/From)**

Ein reeller Wert, der dem linken oder oberen Ende der Scale entspricht.

**-label (label/Label)**

Ein String, der als Beschriftung für die Scale angezeigt wird. Bei vertikalen Scales wird die Beschriftung direkt rechts vom oberen Ende der Scale angezeigt. Bei horizontalen Scales wird die Beschriftung direkt über dem linken Ende der Scale angezeigt. Wenn die Option als leerer String angegeben ist, wird keine Beschriftung angezeigt.

**-length (length/Length)**

Legt die gewünschte Längenabmessung der Scale in Bildschirmeinheiten fest (d.h. alle von Tk_GetPixels akzeptierten Formen). Bei vertikalen Scales ist dies die Höhe der Scale; bei horizontalen Scales ist es die Breite der Scale.

**-resolution (resolution/Resolution)**

Ein reeller Wert, der die Auflösung für die Scale angibt. Wenn dieser Wert größer als null ist, wird der Wert der Scale stets auf ein gerades Vielfaches dieses Werts gerundet, ebenso die Endpunkte der Scale. Wenn der Wert negativ ist, erfolgt keine Rundung. Standardwert ist 1 (d.h. der Wert ist ganzzahlig).

**-showvalue (showValue/ShowValue)**

Legt einen booleschen Wert fest, der angibt, ob der aktuelle Wert der Scale angezeigt werden soll.

**-sliderlength (sliderLength/SliderLength)**

Legt die Größe des Schiebereglers fest, gemessen in Bildschirmeinheiten entlang der Längenabmessung des Schiebereglers. Der Wert kann in einer der von Tk_GetPixels akzeptierten Formen angegeben werden.

**-sliderrelief (sliderRelief/SliderRelief)**

Legt das Relief fest, das beim Zeichnen des Schiebereglers verwendet wird, z. B. **raised** oder

**sunken**.

**-state (state/State)**

Legt einen von drei Zuständen für die Scale fest: **normal**, **active** oder **disabled**. Wenn die Scale deaktiviert ist, kann der Wert nicht geändert werden und die Scale wird nicht aktiviert. Wenn die Scale aktiv ist, wird der Schieberegler mit der durch die Option **-activebackground** angegebenen Farbe angezeigt.

**-tickinterval (tickInterval/TickInterval)**

Muss ein reeller Wert sein. Bestimmt den Abstand zwischen numerischen Markierungen, die unterhalb oder links vom Schieberegler angezeigt werden. Die Werte werden alle mit derselben Anzahl Dezimalstellen angezeigt, die ausreicht, um sicherzustellen, dass sie alle innerhalb von 20 % eines Markierungsintervalls genau sind. Wenn 0, werden keine Markierungen angezeigt.

**-to (to/To)**

Legt einen reellen Wert fest, der dem rechten oder unteren Ende der Scale entspricht. Dieser Wert kann kleiner oder größer als die Option **-from** sein.

**-variable (variable/Variable)**

Gibt den Namen einer globalen Variable an, die mit der Scale verknüpft wird. Wenn sich der Wert der Variable ändert, aktualisiert sich die Scale, um diesen Wert widerzuspiegeln. Wenn die Scale interaktiv manipuliert wird, wird die Variable geändert, um den neuen Wert der Scale widerzuspiegeln.

**-width (width/Width)**

Legt die gewünschte schmale Abmessung der Scale in Bildschirmeinheiten fest (d.h. alle von Tk_GetPixels akzeptierten Formen). Bei vertikalen Scales ist dies die Breite der Scale; bei horizontalen Scales ist es die Höhe der Scale.

## Beschreibung

Der Befehl **scale** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Scale-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte der Scale wie Farben, Ausrichtung und Relief zu konfigurieren. Der Befehl **scale** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Eine Scale ist ein Widget, das eine rechteckige *Rinne* (*trough*) und einen kleinen *Schieberegler* (*slider*) anzeigt. Die Rinne entspricht einem Bereich reeller Werte (bestimmt durch die Optionen **-from**, **-to** und **-resolution**), und die Position des Schiebereglers wählt einen bestimmten reellen Wert aus. Die Position des Schiebereglers (und damit der Wert der Scale) kann mit Maus oder Tastatur angepasst werden, wie im Abschnitt Standard-Bindings unten beschrieben. Wenn sich der Wert der Scale ändert, wird ein Tcl-Befehl aufgerufen (über die Option **-command**), um andere interessierte Widgets über die Änderung zu informieren. Zusätzlich kann der Wert der Scale mit einer Tcl-Variable verknüpft werden (über die Option **-variable**), sodass Änderungen in der einen in der anderen widergespiegelt werden.

In einem Scale-Widget können drei Anmerkungen angezeigt werden: eine Beschriftung oben rechts im Widget (oben links bei horizontalen Scales), eine Zahl direkt links vom Schieberegler (direkt über dem Schieberegler bei horizontalen Scales) und eine Sammlung numerischer Markierungen direkt links vom aktuellen Wert (direkt unter der Rinne bei horizontalen Scales).

Jede dieser drei Anmerkungen kann über die Konfigurationsoptionen aktiviert oder deaktiviert werden.

## Widget-Befehl

Der Befehl **scale** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Scale-Widgets möglich:

***pathName* cget *option***

Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **scale** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **scale** akzeptierten Werte haben.

***pathName* coords ?*value*?**

Gibt eine Liste zurück, deren Elemente die x- und y-Koordinaten des Punkts entlang der Mittellinie der Rinne sind, der *value* entspricht. Wenn *value* weggelassen wird, wird der aktuelle Wert der Scale verwendet.

***pathName* get ?*x y*?**

Wenn *x* und *y* weggelassen werden, gibt der Befehl den aktuellen Wert der Scale zurück. Wenn *x* und *y* angegeben sind, geben sie Pixelkoordinaten innerhalb des Widgets an; der Befehl gibt den Scale-Wert zurück, der dem gegebenen Pixel entspricht. Nur eines von *x* oder *y* wird verwendet: Bei horizontalen Scales wird *y* ignoriert, bei vertikalen Scales wird *x* ignoriert.

***pathName* identify *x y***

Gibt einen String zurück, der angibt, welcher Teil der Scale unter den durch *x* und *y* gegebenen Koordinaten liegt. Ein Rückgabewert **slider** bedeutet, dass der Punkt über dem Schieberegler liegt; **trough1** bedeutet, dass der Punkt über dem Teil der Rinne oberhalb oder links vom Schieberegler liegt; und **trough2** bedeutet, dass der Punkt über dem Teil der Rinne unterhalb oder rechts vom Schieberegler liegt. Wenn der Punkt über keinem dieser Elemente liegt, wird ein leerer String zurückgegeben.

***pathName* set *value***

Dieser Befehl wird aufgerufen, um den aktuellen Wert der Scale zu ändern und damit die Position, an der der Schieberegler angezeigt wird. *Value* gibt den neuen Wert für die Scale an. Der Befehl hat keine Wirkung, wenn die Scale deaktiviert ist.

## Standard-Bindings

Tk erstellt automatisch Klassen-Bindings für Scales, die ihnen folgendes Standardverhalten geben. Wenn sich das Verhalten für vertikale und horizontale Scales unterscheidet, wird das horizontale Verhalten in Klammern beschrieben.

1. Wenn Taste 1 in der Rinne gedrückt wird, wird der Wert der Scale um den Wert der Option **-resolution** erhöht oder verringert, sodass sich der Schieberegler in Richtung des Mauszeigers bewegt. Wenn die Taste gedrückt gehalten wird, wiederholt sich die Aktion automatisch.
2. Wenn Taste 1 über dem Schieberegler gedrückt wird, kann der Schieberegler mit der Maus gezogen werden.
3. Wenn Taste 1 in der Rinne mit gedrückter Steuerungstaste gedrückt wird, bewegt sich der Schieberegler bis zum Ende seines Bereichs in Richtung des Mauszeigers.
4. Wenn Taste 2 gedrückt wird, wird der Wert der Scale auf die Mausposition gesetzt. Wenn die Maus mit gedrückter Taste 2 gezogen wird, ändert sich der Wert der Scale mit dem Ziehen.
5. Die Tasten Pfeil-hoch und Pfeil-links bewegen den Schieberegler nach oben (links) um den Wert der Option **-resolution**.
6. Die Tasten Pfeil-runter und Pfeil-rechts bewegen den Schieberegler nach unten (rechts) um den Wert der Option **-resolution**.
7. Strg-Pfeil-hoch und Strg-Pfeil-links bewegen den Schieberegler nach oben (links) um den Wert der Option **-bigincrement**.
8. Strg-Pfeil-runter und Strg-Pfeil-rechts bewegen den Schieberegler nach unten (rechts) um den Wert der Option **-bigincrement**.
9. Pos1 bewegt den Schieberegler zum oberen (linken) Ende seines Bereichs.
10. Ende bewegt den Schieberegler zum unteren (rechten) Ende seines Bereichs.

Wenn die Scale mit der Option **-state** deaktiviert ist, haben keine der obigen Bindings Wirkung.

Das Verhalten von Scales kann geändert werden, indem neue Bindings für einzelne Widgets definiert werden oder indem die Klassen-Bindings neu definiert werden.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# scrollbar

*Stichwörter:* scrollbar, widget

## Name

scrollbar - Erstellt und manipuliert 'Scrollbar'-Scroll-Steuerungs- und Anzeige-Widgets

## Übersicht

**scrollbar** *pathName* ?*options*?

## Standardoptionen

**-activebackground**, **-highlightcolor**, **-repeatdelay**, **-background**, **-highlightthickness**, **-repeatinterval**, **-borderwidth**, **-jump**, **-takefocus**, **-cursor**, **-orient**, **-troughcolor**, **-highlightbackground**, **-relief**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-activerelief (activeRelief/ActiveRelief)**

Legt das Relief fest, das beim Anzeigen des aktiven Elements verwendet wird, falls vorhanden. Elemente außer dem aktiven Element werden immer mit einem erhabenen Relief angezeigt.

**-command (command/Command)**

Legt das Präfix eines Tcl-Befehls fest, der aufgerufen wird, um die Ansicht im mit der Scrollbar verknüpften Widget zu ändern. Wenn ein Benutzer eine Ansichtsänderung durch Manipulation der Scrollbar anfordert, wird ein Tcl-Befehl aufgerufen. Der tatsächliche Befehl besteht aus dieser Option gefolgt von zusätzlichen Informationen, wie später beschrieben. Diese Option hat fast immer einen Wert wie **.t xview** oder **.t yview**, bestehend aus dem Namen eines Widgets und entweder **xview** (wenn die Scrollbar für horizontales Scrollen ist) oder **yview** (für vertikales Scrollen). Alle scrollbaren Widgets haben **xview**- und **yview**-Befehle, die genau die zusätzlichen Argumente annehmen, die von der Scrollbar angehängt werden, wie in Scroll-Befehle unten beschrieben.

**-elementborderwidth (elementBorderWidth/BorderWidth)**

Legt die Breite der Ränder fest, die um die internen Elemente der Scrollbar gezeichnet werden (die beiden Pfeile und der Schieberegler). Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben. Wenn dieser Wert die leere Zeichenkette ist (der Standardwert), wird stattdessen der Wert der Option **-borderwidth** verwendet.

**-width (width/Width)**

Legt die gewünschte schmale Dimension des Scrollbar-Fensters fest, ohne 3-D-Rand, falls vorhanden. Bei vertikalen Scrollbars ist dies die Breite und bei horizontalen Scrollbars die Höhe. Der Wert kann eine der von Tk_GetPixels akzeptierten Formen haben.

## Beschreibung

Der Befehl **scrollbar** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Scrollbar-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte der Scrollbar wie Farben, Ausrichtung und Relief zu konfigurieren. Der Befehl **scrollbar** gibt sein *pathName* -Argument zurück. Zum Zeitpunkt des Aufrufs dieses Befehls darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Eine Scrollbar ist ein Widget, das zwei Pfeile anzeigt, einen an jedem Ende der Scrollbar, und einen *Schieberegler* im mittleren Teil der Scrollbar. Sie liefert Informationen darüber, was in einem *verknüpften Fenster* sichtbar ist, das ein Dokument irgendeiner Art anzeigt (wie eine bearbeitete Datei oder eine Zeichnung). Position und Größe des Schiebereglers geben an, welcher Teil des Dokuments im verknüpften Fenster sichtbar ist. Wenn beispielsweise der Schieberegler in einer vertikalen Scrollbar das obere Drittel des Bereichs zwischen den beiden Pfeilen abdeckt, bedeutet dies, dass das verknüpfte Fenster das obere Drittel seines Dokuments anzeigt.

Scrollbars können verwendet werden, um die Ansicht im verknüpften Fenster durch Klicken oder Ziehen mit der Maus anzupassen. Siehe den Abschnitt Bindings unten für Details.

## Elemente

Eine Scrollbar zeigt fünf Elemente an, auf die in den Widget-Befehlen für die Scrollbar verwiesen wird:

**arrow1**
Der obere oder linke Pfeil in der Scrollbar.

**trough1**
Der Bereich zwischen dem Schieberegler und **arrow1**.

**slider**
Das Rechteck, das angibt, was im verknüpften Widget sichtbar ist.

**trough2**
Der Bereich zwischen dem Schieberegler und **arrow2**.

**arrow2**
Der untere oder rechte Pfeil in der Scrollbar.

## Widget-Befehl

Der Befehl **scrollbar** erstellt einen neuen Tcl-Befehl, dessen Name *pathName* ist. Dieser Befehl kann verwendet werden, um verschiedene Operationen auf dem Widget aufzurufen. Er hat folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls. Die folgenden Befehle sind für Scrollbar-Widgets möglich:

***pathName* activate ?*element*?**
Markiert das durch *element* angegebene Element als aktiv, wodurch es wie durch die Optionen **-activebackground** und **-activerelief** festgelegt angezeigt wird. Die einzigen Elementwerte, die dieser Befehl versteht, sind **arrow1**, **slider** oder **arrow2**. Wenn ein anderer Wert angegeben ist, ist kein Element der Scrollbar aktiv. Wenn *element* nicht angegeben ist, gibt der Befehl den Namen des derzeit aktiven Elements zurück oder eine leere Zeichenkette, wenn kein Element aktiv ist.

***pathName* cget *option***
Gibt den aktuellen Wert der durch *option* gegebenen Konfigurationsoption zurück. *Option* kann jeden der vom Befehl **scrollbar** akzeptierten Werte haben.

***pathName* configure ?*option*? ?*value option value* ...?**
Abfrage oder Änderung der Konfigurationsoptionen des Widgets. Wenn keine *option* angegeben ist, gibt eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (siehe Tk_ConfigureInfo für Informationen über das Format dieser Liste). Wenn *option* ohne *value* angegeben ist, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (diese Liste ist identisch mit der entsprechenden Unterliste des zurückgegebenen Wertes, wenn keine *option* angegeben ist). Wenn ein oder mehrere *option*-*value*-Paare angegeben sind, ändert der Befehl die gegebene(n) Widget-Option(en) auf die gegebenen Werte; in diesem Fall gibt der Befehl einen leeren String zurück. *Option* kann jeden der vom Befehl **scrollbar** akzeptierten Werte haben.

***pathName* delta *deltaX deltaY***
Gibt eine reelle Zahl zurück, die die Bruchteilsänderung in der Scrollbar-Einstellung angibt, die einer gegebenen Änderung der Schiebereglerposition entspricht. Wenn die Scrollbar beispielsweise horizontal ist, gibt das Ergebnis an, um wie viel sich die Scrollbar-Einstellung ändern muss, um den Schieberegler *deltaX* Pixel nach rechts zu bewegen (*deltaY* wird in

diesem Fall ignoriert). Wenn die Scrollbar vertikal ist, gibt das Ergebnis an, um wie viel sich die Scrollbar-Einstellung ändern muss, um den Schieberegler *deltaY* Pixel nach unten zu bewegen. Die Argumente und das Ergebnis können null oder negativ sein.

***pathName* fraction *x y***

Gibt eine reelle Zahl zwischen 0 und 1 zurück, die angibt, wo der durch *x* und *y* gegebene Punkt im Trog-Bereich der Scrollbar liegt. Der Wert 0 entspricht dem oberen oder linken Rand des Trogs, der Wert 1 dem unteren oder rechten Rand, 0,5 der Mitte und so weiter. *X* und *y* müssen Pixelkoordinaten relativ zum Scrollbar-Widget sein. Wenn *x* und *y* auf einen Punkt außerhalb des Trogs verweisen, wird der dem Trog am nächsten liegende Punkt verwendet.

***pathName* get**

Gibt die Scrollbar-Einstellungen in Form einer Liste zurück, deren Elemente die Argumente des zuletzt ausgeführten Widget-Befehls **set** sind.

***pathName* identify *x y***

Gibt den Namen des Elements unter dem durch *x* und *y* gegebenen Punkt zurück (wie **arrow1**) oder eine leere Zeichenkette, wenn der Punkt in keinem Element der Scrollbar liegt. *X* und *y* müssen Pixelkoordinaten relativ zum Scrollbar-Widget sein.

***pathName* set *first last***

Dieser Befehl wird vom verknüpften Widget der Scrollbar aufgerufen, um der Scrollbar die aktuelle Ansicht im Widget mitzuteilen. Der Befehl nimmt zwei Argumente entgegen, von denen jedes ein reeller Bruch zwischen 0 und 1 ist. Die Brüche beschreiben den Bereich des Dokuments, der im verknüpften Widget sichtbar ist. Wenn beispielsweise *first* 0,2 und *last* 0,4 ist, bedeutet dies, dass der erste sichtbare Teil des Dokuments im Fenster 20 % des Weges durch das Dokument ist und der letzte sichtbare Teil 40 % des Weges durch das Dokument.

## Scroll-Befehle

Wenn der Benutzer mit der Scrollbar interagiert, beispielsweise durch Ziehen des Schiebereglers, benachrichtigt die Scrollbar das verknüpfte Widget, dass es seine Ansicht ändern muss, indem sie einen aus der Option **-command** erzeugten Tcl-Befehl auswertet. Der Befehl kann eine der folgenden Formen annehmen. In jedem Fall ist *prefix* der Inhalt der Option **-command**, der üblicherweise eine Form wie "**.t yview**" hat.

***prefix* moveto *fraction***

*Fraction* ist eine reelle Zahl zwischen 0 und 1. Das Widget soll seine Ansicht so anpassen, dass der durch *fraction* angegebene Punkt am Anfang des Widgets erscheint. Ist *fraction* 0, bezeichnet dies den Anfang des Dokuments. 1.0 bezeichnet das Ende, 0.333 einen Punkt ein Drittel durch das Dokument usw.

***prefix* scroll *number* pages**

Das Widget soll seine Ansicht um *number* Seiten anpassen. Die Bedeutung einer Seite legt das Widget fest; typischerweise ist sie etwas kleiner als der sichtbare Fensterinhalt, sodass alte und neue Ansicht leicht überlappen. *Number* ist entweder 1 (nächste Seite sichtbar) oder -1 (vorherige Seite sichtbar). Bruchzahlen werden von 0 weg gerundet, sodass das Scrollen um 0,001 Seiten dieselbe Wirkung hat wie das Scrollen um 1 Seite.

***prefix* scroll *number* units**

Das Widget soll seine Ansicht um *number* Einheiten anpassen. Die Einheiten sind widget-spezifisch definiert, etwa Zeichen oder Zeilen in einem Text-Widget. *Number* ist entweder 1 (eine Einheit scrollt oben oder links aus dem Fenster) oder -1 (eine Einheit scrollt unten oder rechts aus dem Fenster). Bruchzahlen werden von 0 weg gerundet, sodass das Scrollen um 0,001 Einheiten dieselbe Wirkung hat wie das Scrollen um 1 Einheit.

## Bindings

Tk erstellt automatisch Klassen-Bindings für Scrollbars, die ihnen folgendes Standardverhalten geben. Wenn sich das Verhalten für vertikale und horizontale Scrollbars unterscheidet, ist das horizontale Verhalten in Klammern beschrieben.

1. Das Drücken von Taste 1 über **arrow1** bewirkt, dass die Ansicht im verknüpften Widget um eine Einheit nach oben (links) verschoben wird, sodass das Dokument um eine Einheit nach unten (rechts) zu wandern scheint. Wenn die Taste gedrückt gehalten wird, wiederholt sich die Aktion automatisch.
2. Das Drücken von Taste 1 über **trough1** bewirkt, dass die Ansicht im verknüpften Widget um eine Bildschirmfüllung nach oben (links) verschoben wird, sodass das Dokument um eine Bildschirmfüllung nach unten (rechts) zu wandern scheint. Wenn die Taste gedrückt gehalten wird, wiederholt sich die Aktion automatisch.
3. Das Drücken von Taste 1 über den Schieberegler und Ziehen bewirkt, dass die Ansicht mit dem Schieberegler mitgezogen wird. Wenn die Option **jump** wahr ist, zieht die Ansicht nicht mit dem Schieberegler mit; sie ändert sich erst, wenn die Maustaste losgelassen wird.
4. Das Drücken von Taste 1 über **trough2** bewirkt, dass die Ansicht im verknüpften Widget um eine Bildschirmfüllung nach unten (rechts) verschoben wird, sodass das Dokument um eine Bildschirmfüllung nach oben (links) zu wandern scheint. Wenn die Taste gedrückt gehalten wird, wiederholt sich die Aktion automatisch.
5. Das Drücken von Taste 1 über **arrow2** bewirkt, dass die Ansicht im verknüpften Widget um eine Einheit nach unten (rechts) verschoben wird, sodass das Dokument um eine Einheit nach oben (links) zu wandern scheint. Wenn die Taste gedrückt gehalten wird, wiederholt sich die Aktion automatisch.
6. Wenn Taste 2 über dem Trog oder dem Schieberegler gedrückt wird, setzt sie die Ansicht entsprechend der Mausposition; das Ziehen der Maus mit gedrückter Taste 2 bewirkt, dass die Ansicht mit der Maus mitgezogen wird. Wenn Taste 2 über einem der Pfeile gedrückt wird, bewirkt sie dasselbe Verhalten wie das Drücken von Taste 1.
7. Wenn Taste 1 mit gedrückter Steuerungstaste gedrückt wird, ändert sich die Ansicht zum oberen (linken) Rand des Dokuments, wenn die Maus über **arrow1** oder **trough1** ist; wenn die Maus über **arrow2** oder **trough2** ist, ändert sich die Ansicht zum unteren (rechten) Rand des Dokuments; wenn die Maus sonst irgendwo ist, hat der Tastendruck keine Wirkung.
8. Bei vertikalen Scrollbars haben die Auf- und Ab-Tasten dasselbe Verhalten wie Mausklicks über **arrow1** bzw. **arrow2**. Bei horizontalen Scrollbars haben diese Tasten keine Wirkung.
9. Bei vertikalen Scrollbars haben Steuerung-Auf und Steuerung-Ab dasselbe Verhalten wie Mausklicks über **trough1** bzw. **trough2**. Bei horizontalen Scrollbars haben diese Tasten keine Wirkung.
10. Bei horizontalen Scrollbars haben die Auf- und Ab-Tasten dasselbe Verhalten wie Mausklicks über **arrow1** bzw. **arrow2**. Bei vertikalen Scrollbars haben diese Tasten keine Wirkung.
11. Bei horizontalen Scrollbars haben Steuerung-Auf und Steuerung-Ab dasselbe Verhalten wie Mausklicks über **trough1** bzw. **trough2**. Bei vertikalen Scrollbars haben diese Tasten keine Wirkung.
12. Die Bild-Auf- und Bild-Ab-Tasten haben dasselbe Verhalten wie Mausklicks über **trough1** bzw. **trough2**.

13. Die Pos1-Taste passt die Ansicht an den oberen (linken) Rand des Dokuments an.
14. Die Ende-Taste passt die Ansicht an den unteren (rechten) Rand des Dokuments an.

### Beispiel

Erstellt ein Fenster mit einem scrollbaren text-Widget:

```
toplevel .tl
text .tl.t -yscrollcommand {.tl.s set}
scrollbar .tl.s -command {.tl.t yview}
grid .tl.t .tl.s -sticky nsew
grid columnconfigure .tl 0 -weight 1
grid rowconfigure .tl 0 -weight 1
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

## selection

*Stichwörter:* selection, clear, format, handler, ICCCM, own, target, type

### Name

selection - X-Auswahl steuern

### Übersicht

**selection** *option* ?*arg* ...?

### Beschreibung

Dieser Befehl bietet eine Tcl-Schnittstelle zum X-Selection-Mechanismus und implementiert die vollständige Selection-Funktionalität aus dem X Inter-Client Communication Conventions Manual (ICCCM).

Zur Verwaltung der **CLIPBOARD**-Selection (siehe unten) kann auch der Befehl clipboard verwendet werden.

Das erste Argument von **selection** legt das Format der weiteren Argumente und das Verhalten fest. Unterstützt werden:

**selection clear ?*-displayof window*? ?*-selection selection*?**

Existiert *selection* irgendwo auf dem Display von *window*, wird sie gelöscht, sodass kein Fenster die Selection mehr besitzt. *Selection* gibt die zu löschende X-Selection an und sollte ein Atom-Name wie **PRIMARY** oder **CLIPBOARD** sein; Details im Inter-Client Communication Conventions Manual. *Selection* standardmäßig **PRIMARY**, *window* standardmäßig ".". Gibt einen leeren String zurück.

**selection get ?*-displayof window*? ?*-selection selection*? ?*-type type*?**

Liest den Wert von *selection* vom Display von *window* und gibt ihn zurück. *Selection* standardmäßig **PRIMARY**, *window* standardmäßig ".". *Type* legt die Rückgabeform fest (gewünschtes ICCCM-"target" für die Konvertierung) und sollte ein Atom-Name wie **STRING** oder **FILE_NAME** sein; Details im ICCCM. *Type* standardmäßig **STRING**. Der Selection-Besitzer kann die Selection in verschiedenen Darstellungsformaten zurückgeben, z. B. **STRING**, **UTF8_STRING**, **ATOM**, **INTEGER** usw. (dieses Format unterscheidet sich vom Selection-Typ; Details im ICCCM). Wird die Selection in einem Nicht-String-Format wie

**INTEGER** oder **ATOM** geliefert, wandelt **selection** sie in einen String aus durch Leerzeichen getrennten Feldern um: Atome werden in ihre Textnamen, alles andere in hexadezimale Ganzzahlen. **selection get** liest die Selection nicht im Format **UTF8_STRING**, sofern nicht angefordert.

**selection handle ?*-selection s*? ?*-type t*? ?*-format f*? *window command***

Erzeugt einen Handler für Selection-Anfragen: *command* wird ausgeführt, wenn *window* die Selection *s* besitzt und jemand sie in der durch Typ *t* angegebenen Form abruft (z. B. *t* in **selection get**). *S* standardmäßig **PRIMARY**, *t* standardmäßig **STRING**, *f* standardmäßig **STRING**. Ist *command* ein leerer String, wird ein vorhandener Handler für *window*, *t* und *s* entfernt. Wird die Selection als Typ **STRING** behandelt, geschieht das automatisch auch für **UTF8_STRING**.

Bei einer Selection-Anfrage ist *window* der Besitzer, *type* der angeforderte Typ; *command* wird als Tcl-Befehl mit zwei zusätzlichen Zahlen (durch Leerzeichen getrennt) ausgeführt: *offset* und *maxChars*. *offset* ist die Startposition in der Selection, *maxChars* die maximale Zeichenanzahl. Der Befehl soll höchstens *maxChars* Zeichen ab Position *offset* zurückgeben. Bei sehr großen Selections (größer als *maxChars*) erfolgt mehrfacher Aufruf von *command* mit steigendem *offset*. Ist die Ergebnislänge kleiner als *maxChars*, gilt der Rest als geliefert; ist sie gleich *maxChars*, wird *command* erneut aufgerufen, bis ein kürzeres Ergebnis kommt. *maxChars* ist stets relativ groß (Tausende Zeichen).

Liefert *command* einen Fehler, wird die Abfrage abgelehnt, als ob die Selection nicht existierte.

*format* legt die Übertragungsdarstellung fest (ICCCM Tabelle 2, Spalte 2), Standard **STRING**. Bei **STRING**: 8-Bit-ASCII (wie von *command* zurückgegeben, in der System-encoding; **UTF8_STRING** nutzt stets UTF-8). Bei **ATOM**: Rückgabe von *command* in durch Leerzeichen getrennte Felder, jedes Feld zum Atom-Wert, 32-Bit-Atom-Wert statt Name. Bei anderem *format*: Felder zu 32-Bit-Ganzzahlen, Array an den Anforderer.

*format* ist nur für Kompatibilität mit Anfragern nötig, die kein Tk verwenden. Bei Tk-Abfrage wird am Ende wieder ein String geliefert — *format* ist irrelevant.

**selection own ?*-displayof window*? ?*-selection selection*?**

...siehe nächstes...

**selection own ?*-command command*? ?*-selection selection*? *window***

Die erste Form von **selection own** gibt den Pfadnamen des Fensters in dieser Anwendung zurück, das *selection* auf dem Display von *window* besitzt, oder einen leeren String, wenn kein Fenster in dieser Anwendung die Selection besitzt. *Selection* standardmäßig **PRIMARY**, *window* standardmäßig ".".

Die zweite Form lässt *window* neuer Besitzer von *selection* auf dem Display von *window* werden; Rückgabe ist ein leerer String. Der bisherige Besitzer wird benachrichtigt, dass er die Selection verloren hat. Ist *command* angegeben, ist es ein Tcl-Skript, das ausgeführt wird, wenn ein anderes Fenster die Selection von *window* übernimmt. *Selection* standardmäßig **PRIMARY**.

## Widget-Funktionen

Die Widgets text, entry, ttk::entry, listbox, spinbox und ttk::spinbox haben die Option **-exportselection**. Ist sie auf den booleschen Wert **true** gesetzt, wird (in einem unsicheren Interpreter) eine im Widget getroffene Auswahl automatisch in die **PRIMARY**-Selection geschrieben.

Ein GUI-Ereignis, z. B. **<<PasteSelection>>**, kann die **PRIMARY**-Selection in bestimmte Widgets kopieren. Die Kopie erfolgt über eine Widget-Bindung an das Ereignis; das Bindungs-Skript ruft **selection** auf.

## Portierbarkeit

Unter X11 ist die **PRIMARY**-Selection eine systemweite Funktion des X-Servers und ermöglicht Kommunikation zwischen verschiedenen X11-Client-Prozessen.

Unter Windows stellt das System keine **PRIMARY**-Selection bereit, sondern nur Tk; sie wird nur zwischen Fenstern eines Eltern-Interpreters und seiner Kind-Interpreter geteilt, nicht zwischen Interpretern in verschiedenen Prozessen oder Threads. Jeder Eltern-Interpreter hat eine eigene **PRIMARY**-Selection, die nur mit seinen Kind-Interpreter geteilt wird, die keine sicheren Interpreter sind.

## Sicherheit

Ein sicherer Interpreter kann die **PRIMARY**-Selection nicht lesen, weil sein **selection**-Befehl verborgen ist. Daher kann die **PRIMARY**-Selection nicht in die Tk-Widgets eines sicheren Interpreters geschrieben werden.

Ein Tk-Widget kann **-exportselection** auf **true** setzen; in einem sicheren Interpreter hat diese Option keine Wirkung: Schreiben vom Widget in die **PRIMARY**-Selection ist deaktiviert.

Das sind Sicherheitsmerkmale. In einem sicheren Interpreter können nicht vertrauenswürdige Skripte ausgeführt werden, und es wäre ein Sicherheitsrisiko, wenn dieser Code die **PRIMARY** -Selection anderer Interpreter lesen oder beschreiben könnte.

## Beispiele

Auf X11-Plattformen ist eine der Standard-Selections die **SECONDARY**-Selection. Sie wird kaum genutzt; so liest man sie mit Tk:

```
set selContents [selection get -selection SECONDARY]
```

Für eine Selection können viele Datentypen verfügbar sein; der spezielle Typ **TARGETS** liefert eine Liste verfügbarer Typen:

```
foreach type [selection get -type TARGETS] {
   puts "Selection PRIMARY supports type $type"
}
```

Um die Selection zu beanspruchen, richten Sie zuerst einen Handler ein, der die Daten liefert. Dann beanspruchen Sie die Selection …

```
# Daten-Handler für eingehende Anfragen einrichten
set foo "This is a string with some data in it... blah blah"
selection handle -selection SECONDARY . getData
proc getData {offset maxChars} {
   puts "Retrieving selection starting at $offset"
   return [string range $::foo $offset [expr {$offset+$maxChars-1}]]
}

# Selection selbst beanspruchen
puts "Claiming selection"
selection own -command lost -selection SECONDARY .
proc lost {} {
   puts "Lost selection"
}
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# send

*Stichwörter:* send, application, dde, name, remote execution, security

## Name

send - Führt einen Befehl in einer anderen Anwendung aus

## Übersicht

**send** ?*options*? *app cmd* ?*arg* ...?

## Beschreibung

Dieser Befehl sorgt dafür, dass *cmd* (und *arg*) in der Anwendung mit dem Namen *app* ausgeführt werden. Er gibt das Ergebnis oder den Fehler dieser Ausführung zurück. *App* kann der Name jeder Anwendung sein, deren Hauptfenster auf dem Display liegt, das auch das Hauptfenster des Senders enthält; sie muss nicht im selben Prozess laufen. Sind keine *arg*-Argumente vorhanden, steht der auszuführende Befehl vollständig im Argument *cmd*. Sind ein oder mehr *arg* vorhanden, werden sie wie beim Befehl eval zu einem auszuführenden Befehl zusammengefügt.

Beginnen die ersten Argumente des Befehls mit "-", werden sie als Optionen behandelt. Folgende Optionen sind derzeit definiert:

**-async**

Fordert asynchrone Ausführung an. In diesem Fall endet der Befehl **send** sofort, ohne auf den Abschluss von *cmd* in der Zielanwendung zu warten; es steht kein Ergebnis zur Verfügung, und Fehler im gesendeten Befehl werden ignoriert. Liegt die Zielanwendung im selben Prozess wie die sendende Anwendung, wird die Option **-async** ignoriert.

**-displayof *pathName***

Legt fest, dass das Hauptfenster der Zielanwendung auf dem Display des durch *pathName* angegebenen Fensters liegt, statt auf dem Display des Hauptfensters der Anwendung.

**--**

Dient nur dazu, die Optionsliste zu beenden. Diese Option ist nur nötig, wenn *app* mit "-"

beginnen könnte.

## Anwendungsnamen

Der Name einer Anwendung wird zunächst aus dem Namen des Programms oder Skripts gesetzt, das die Anwendung erzeugt hat. Name abfragen und ändern lässt sich mit dem Befehl tk appname.

## Deaktivierung von send

Wird der Befehl **send** aus einer Anwendung entfernt (z. B. mit rename **send {}**), antwortet die Anwendung nicht mehr auf eingehende Send-Anfragen und kann auch keine ausgehenden Anfragen mehr stellen. Die Kommunikation lässt sich durch Aufruf von tk appname wieder aktivieren.

## Sicherheit

Der Befehl **send** ist potenziell eine ernsthafte Sicherheitslücke. Unter Unix kann jede Anwendung, die sich mit Ihrem X-Server verbinden kann, Skripte an Ihre Anwendungen senden. Diese eingehenden Skripte können Tcl nutzen, um unter Ihrem Namen Dateien zu lesen und zu schreiben und Unterprozesse zu starten. Host-basierte Zugriffskontrolle wie die von **xhost** ist besonders unsicher, da jeder mit einem Konto auf bestimmten Rechnern sich mit Ihrem Server verbinden kann; ist sie deaktiviert, kann sich jeder überall verbinden. Um wenigstens etwas Sicherheit zu bieten, prüft Tk die vom Server verwendete Zugriffskontrolle und lehnt eingehende Send-Anfragen ab, es sei denn, (a) **xhost**-artige Zugriffskontrolle ist aktiv (nur bestimmte Rechner dürfen Verbindungen aufbauen) und (b) die Liste der freigeschalteten Rechner ist leer. Anwendungen können sich also nur verbinden, wenn sie eine andere Autorisierung wie die von **xauth** verwenden. Unter Windows ist **send** derzeit deaktiviert. Den Großteil der Funktionalität bietet stattdessen der Befehl dde.

## Einschränkungen

Unter macOS/aqua funktioniert send nur mit Interpretern im selben Prozess (das sind die, die "winfo interps" zurückgibt). Aufrufe an einen Interpreter in einem anderen Prozess bewirken nichts.

## Beispiel

Dieses Skriptfragment kann verwendet werden, damit eine Anwendung auf einem bestimmten Display nur einmal läuft.

```
if {[tk appname FoobarApp] ne "FoobarApp"} {
    send -async FoobarApp RemoteStart $argv
    exit
}
# Befehl, der remote aufgerufen wird, hebt das
# Hauptfenster an und öffnet die angeforderten Dateien
proc RemoteStart args {
    raise .
    foreach filename $args {
        OpenFile $filename
    }
}
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# spinbox

*Stichwörter:* spinbox, entry, widget

## Name

spinbox - Erstellt und manipuliert 'Spinbox'-Wert-Dreh-Widgets

## Übersicht

**spinbox** *pathName* ?*options*?

## Standardoptionen

**-activebackground**, **-highlightthickness**, **-repeatinterval**, **-background**, **-insertbackground**, **-selectbackground**, **-borderwidth**, **-insertborderwidth**, **-selectborderwidth**, **-cursor**, **-insertontime**, **-selectforeground**, **-exportselection**, **-insertwidth**, **-takefocus**, **-font**, **-insertofftime**, **-textvariable**, **-foreground**, **-justify**, **-xscrollcommand**, **-highlightbackground**, **-relief**, **-highlightcolor**, **-repeatdelay**, **-placeholder**, **-placeholderforeground**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-buttonbackground (buttonBackground/Background)**

Die Hintergrundfarbe für die Spin-Buttons.

**-buttoncursor (buttonCursor/Cursor)**

Der Cursor, der über den Spin-Buttons verwendet wird. Ist dies leer (Standard), wird ein Standard-Cursor verwendet.

**-buttondownrelief (buttonDownRelief/Relief)**

Das Relief für den oberen Spin-Button.

**-buttonuprelief (buttonUpRelief/Relief)**

Das Relief für den unteren Spin-Button.

**-command (command/Command)**

Legt einen Tcl-Befehl fest, der aufgerufen wird, wann immer ein Spin-Button betätigt wird. Der

Befehl erkennt mehrere Prozent-Substitutionen: **%W** für den Widget-Pfad, **%s** für den aktuellen Wert des Widgets und **%d** für die Richtung des gedrückten Buttons (**up** oder **down** ).

**-disabledbackground (disabledBackground/DisabledBackground)**

Legt die Hintergrundfarbe fest, die verwendet wird, wenn die Spinbox deaktiviert ist. Ist diese Option der leere String, wird die normale Hintergrundfarbe verwendet.

**-disabledforeground (disabledForeground/DisabledForeground)**

Legt die Vordergrundfarbe fest, die verwendet wird, wenn die Spinbox deaktiviert ist. Ist diese Option der leere String, wird die normale Vordergrundfarbe verwendet.

**-format (**format**/Format)**

Legt ein alternatives Format fest, das beim Setzen des String-Werts bei Verwendung des Bereichs **-from** und **-to** verwendet wird. Dies muss ein Format-Spezifizierer der Form **%.f** sein, da er eine Fließkommazahl formatiert.

**-from (from/From)**

Ein Fließkommawert entsprechend dem niedrigsten Wert einer Spinbox, verwendet zusammen mit **-to** und **-increment**. Sind alle korrekt angegeben, steuert die Spinbox ihren Inhalt damit. Ist dieser Wert größer als die Option **-to**, werden **-from** und **-to** automatisch vertauscht. Ist **-values** angegeben, hat diese Option Vorrang.

**-invalidcommand, -invcmd (invalidCommand/InvalidCommand)**

Legt ein Skript fest, das ausgewertet wird, wenn **-validatecommand** 0 zurückgibt. Setzen auf den leeren String deaktiviert diese Funktion (Standard). Die beste Verwendung ist *bell*. Siehe Validierung unten.

**-increment (increment/Increment)**

Ein Fließkommawert für die Schrittweite. Zusammen mit **-from** und **-to** wird der Wert im Widget bei Betätigung eines Spin-Buttons um **-increment** angepasst (nach oben addiert, nach unten subtrahiert).

**-readonlybackground (readonlyBackground/ReadonlyBackground)**

Legt die Hintergrundfarbe fest, die verwendet wird, wenn die Spinbox schreibgeschützt ist. Ist diese Option der leere String, wird die normale Hintergrundfarbe verwendet.

**-state (state/State)**

Legt einen von drei Zuständen für die Spinbox fest: **normal**, **disabled** oder **readonly**. Ist die Spinbox schreibgeschützt, kann der Wert nicht mit Widget-Befehlen geändert werden und es wird kein Einfügecursor angezeigt, auch wenn der Eingabefokus im Widget liegt; der Inhalt kann weiterhin ausgewählt werden. Ist die Spinbox deaktiviert, kann der Wert nicht geändert werden, es wird kein Einfügecursor angezeigt, der Inhalt ist nicht auswählbar, und die Spinbox kann je nach **-disabledforeground** und **-disabledbackground** in einer anderen Farbe dargestellt werden.

**-to (to/To)**

Ein Fließkommawert entsprechend dem höchsten Wert der Spinbox, verwendet zusammen mit **-from** und **-increment**. Sind alle korrekt angegeben, steuert die Spinbox ihren Inhalt damit. Ist dieser Wert kleiner als die Option **-from**, werden **-from** und **-to** automatisch vertauscht. Ist **-values** angegeben, hat diese Option Vorrang.

**-validate (validate/Validate)**

Legt den Modus fest, in dem die Validierung arbeiten soll: **none**, **focus**, **focusin**, **focusout**, **key** oder **all**. Der Standard ist **none**. Wenn Sie Validierung wünschen, müssen Sie explizit angeben, welchen Modus Sie verwenden möchten. Siehe Validierung unten für weitere Informationen.

**-validatecommand, -vcmd (validateCommand/ValidateCommand)**

Legt ein Skript fest, das ausgewertet wird, wenn die Eingabe im Widget validiert werden soll. Setzen auf den leeren String deaktiviert diese Funktion (Standard). Die Validierung erfolgt gemäß **-validate**. Dieser Befehl muss einen gültigen Tcl-Boolean-Wert zurückgeben. Gibt er 0 zurück (oder das gültige Tcl-Boolean-Äquivalent), ändert sich der Widget-Wert nicht und **-invalidcommand** wird ausgewertet, falls gesetzt. Gibt er 1 zurück, wird der Wert geändert. Siehe Validierung unten.

**-values (values/Values)**

Muss ein gültiger Listenwert sein. Ist dies angegeben, verwendet die Spinbox diese Werte zur Steuerung des Inhalts, beginnend mit dem ersten Wert. Diese Option hat Vorrang vor dem Bereich **-from** und **-to**.

**-width (width/Width)**

Gibt einen ganzzahligen Wert an, der die gewünschte Breite des Spinbox-Fensters in Zeichen durchschnittlicher Größe der Widget-Schriftart angibt. Ist der Wert negativ oder null, wählt das Widget eine Größe, die gerade groß genug für den aktuellen Text ist.

**-wrap (wrap/wrap)**

Muss ein gültiger boolescher Wert sein. Ist er an, springt die Spinbox an den Enden der Daten im Widget um.

## Beschreibung

Der Befehl **spinbox** erstellt ein neues Fenster (gegeben durch das *pathName*-Argument) und macht es zu einem Spinbox-Widget. Zusätzliche Optionen, wie oben beschrieben, können in der Befehlszeile oder in der Optionsdatenbank angegeben werden, um Aspekte der Spinbox wie Farben, Schriftart und Relief zu konfigurieren. Der Befehl **spinbox** gibt sein *pathName*-Argument zurück. Zum Zeitpunkt des Aufrufs darf kein Fenster mit dem Namen *pathName* existieren, aber das Eltern-Element von *pathName* muss existieren.

Eine **spinbox** ist ein erweitertes entry-Widget, mit dem der Benutzer durch eine feste Menge auf- oder absteigender Werte (z. B. Zeiten oder Daten) "drehen" kann, zusätzlich zur Bearbeitung des Werts wie in einem entry. Bei der Erstellung ist der String der Spinbox leer. Ein Teil der Spinbox kann wie unten beschrieben ausgewählt werden. Exportiert eine Spinbox ihre Auswahl (siehe die Option **-exportselection**), beachtet sie die Standard-Protokolle für die Behandlung der Selection; Spinbox-Auswahlen sind als Typ **STRING** verfügbar. Spinboxen beachten auch die Standard-Tk-Regeln für den Umgang mit dem Eingabefokus. Hat eine Spinbox den Eingabefokus, zeigt sie einen *Einfügecursor* an, um anzuzeigen, wo neue Zeichen eingefügt werden.

Spinboxen können Strings anzeigen, die zu lang sind, um vollständig in das Fenster des Widgets zu passen. In diesem Fall wird nur ein Teil des Strings angezeigt; unten beschriebene Befehle können verwendet werden, um die Ansicht im Fenster zu ändern. Spinboxen verwenden den Standard-Mechanismus **-xscrollcommand** für die Interaktion mit Scrollbars (siehe die Beschreibung der Option **-xscrollcommand** für Details). Sie unterstützen außerdem Scannen, wie unten beschrieben.

## Validierung

Die Validierung funktioniert, indem **-validatecommand** auf ein Skript gesetzt wird, das gemäß

**-validate** wie folgt ausgewertet wird:

**none**
Standard. Es findet keine Validierung statt.

focus
**-validatecommand** wird aufgerufen, wenn die Spinbox den Fokus erhält oder verliert.

**focusin**
**-validatecommand** wird aufgerufen, wenn die Spinbox den Fokus erhält.

**focusout**
**-validatecommand** wird aufgerufen, wenn die Spinbox den Fokus verliert.

**key**
**-validatecommand** wird aufgerufen, wenn die Spinbox bearbeitet wird.

**all**
**-validatecommand** wird bei allen obigen Bedingungen aufgerufen.

Auf **-validatecommand** und **-invalidcommand** sind Prozent-Substitutionen möglich, wie in einem bind-Skript. Folgende Substitutionen werden erkannt:

**%d**
Art der Aktion: 1 für **insert**, 0 für **delete**, oder -1 für Fokus-, erzwungene oder **textvariable** -Validierung.

**%i**
Index des einzufügenden/zu löschenden Zeichenstrings, falls vorhanden, sonst -1.

**%P**
Der Wert der Spinbox, wenn die Bearbeitung erfolgen soll. Konfigurieren Sie die Spinbox mit einer neuen **textvariable**, ist dies der Wert dieser **textvariable**.

**%s**
Der aktuelle Wert der Spinbox vor der Bearbeitung.

**%S**
Der einzufügende/zu löschende Text-String, falls vorhanden, sonst ein leerer String.

**%v**
Der Typ der aktuell gesetzten Validierung.

**%V**
Der Typ der Validierung, die die Rückruffunktion ausgelöst hat (key, focusin, focusout, forced).

**%W**
Der Name des Spinbox-Widgets.

Im Allgemeinen können **-textvariable** und **-validatecommand** problematisch kombiniert werden. Das wurde so gelöst, dass **-validatecommand** das traditionelle Verhalten der Spinbox nicht stört. **-textvariable** nur lesend zu verwenden, verursacht nie Probleme. Gefährlich wird es, wenn **-textvariable** auf etwas gesetzt wird, das **-validatecommand** nicht akzeptieren würde — dann wird **-validate none** (**-invalidcommand** wird nicht ausgelöst). Dasselbe passiert bei einem Fehler bei der Auswertung von **-validatecommand**.

Vor allem tritt ein Fehler auf, wenn **-validatecommand** oder **-invalidcommand** einen Fehler im Skript hat oder **-validatecommand** keinen gültigen Tcl-Boolean zurückgibt. **-validate** setzt sich auch auf **none**, wenn die Spinbox aus **-validatecommand** oder **-invalidcommand** bearbeitet wird. Solche Bearbeitungen überschreiben die gerade validierte. Soll der Wert während der

Validierung geändert und **-validate** beibehalten werden, gehört in **-validatecommand** oder **-invalidcommand** (je nachdem, von wo aus bearbeitet wurde):

```
     %W config -validate %v
```

Es wird empfohlen, während der Validierung keine verknüpfte **-textvariable** zu setzen, da die Spinbox sonst aus dem Takt mit **-textvariable** geraten kann.

**-validate** setzt sich auch auf **none**, wenn sich der Spinbox-Wert durch Anpassung von **-from** oder **-to** ändert und **-validatecommand** false zurückgibt. Beispielsweise

```
     spinbox pathName -from 1 -to 10 -validate all -validatecommand
    {return 0}
```

setzt **-validate** tatsächlich auf **none**, weil sich der Standardwert der Spinbox (durch **-from** und **-to**) auf einen von der Validierung nicht akzeptierten Wert ändert.

Außerdem wird bei Betätigung eines Spin-Buttons erzwungene Validierung durchgeführt. Gibt das Validierungsskript in dieser Situation false zurück, wird **-validate** automatisch **none**.

## Widget-Befehl

Der Befehl **spinbox** erstellt einen neuen Tcl-Befehl mit dem Namen *pathName*. Dieser Befehl kann verschiedene Operationen am Widget aufrufen. Er hat die folgende allgemeine Form:

```
pathName option ?arg ...?
```

*Option* und die *arg*s bestimmen das genaue Verhalten des Befehls.

### Indizes

Viele Widget-Befehle für Spinboxen nehmen einen oder mehrere Indizes als Argumente. Ein Index bezeichnet ein bestimmtes Zeichen im String der Spinbox, auf eine der folgenden Arten:

***number***
Bezeichnet das Zeichen als numerischen Index, wobei 0 dem ersten Zeichen im String entspricht.

**anchor**
Bezeichnet den Ankerpunkt für die Auswahl, der mit den Widget-Befehlen **select from** und **select adjust** gesetzt wird.

**end**
Bezeichnet das Zeichen unmittelbar nach dem letzten im String der Spinbox. Entspricht einem numerischen Index gleich der Länge des Spinbox-Strings.

**insert**
Bezeichnet das Zeichen neben und unmittelbar nach dem Einfügecursor.

**sel.first**
Bezeichnet das erste Zeichen in der Auswahl. Es ist ein Fehler, diese Form zu verwenden, wenn die Auswahl nicht im Spinbox-Fenster liegt.

**sel.last**
Bezeichnet das Zeichen unmittelbar nach dem letzten in der Auswahl. Es ist ein Fehler, diese Form zu verwenden, wenn die Auswahl nicht im Spinbox-Fenster liegt.

**@*number***
In dieser Form wird *number* als x-Koordinate im Fenster der Spinbox behandelt; das Zeichen,

das diese x-Koordinate überspannt, wird verwendet. Beispielsweise bezeichnet "**@0**" das linkeste Zeichen im Fenster.

Abkürzungen können für jede der obigen Formen verwendet werden, z. B. "**e**" oder "**sel.f**". Im Allgemeinen werden Indizes außerhalb des gültigen Bereichs automatisch auf den nächsten zulässigen Wert gerundet. Indizes unterstützen dieselbe einfache Interpretation wie beim Befehl string index, mit einfacher ganzzahliger Index-Arithmetik und Indizierung relativ zu **end**.

## Unterbefehle

Folgende Befehle sind für Spinbox-Widgets möglich:

***pathName* bbox *index***

Gibt eine Liste von vier Zahlen zurück, die die Begrenzungsbox des durch *index* angegebenen Zeichens beschreiben. Die ersten beiden Elemente der Liste geben die x- und y-Koordinaten der oberen linken Ecke des vom Zeichen bedeckten Bildschirmbereichs an (in Pixeln relativ zum Widget), die letzten beiden geben Breite und Höhe des Zeichens in Pixeln an. Die Begrenzungsbox kann sich auf einen Bereich außerhalb des sichtbaren Fensterbereichs beziehen.

***pathName* cget *option***

Gibt den aktuellen Wert der Konfigurationsoption *option* zurück. *Option* kann jeden von **spinbox** akzeptierten Wert haben.

***pathName* configure ?*option*? ?*value option value* ...?**

Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Ist keine *option* angegeben, gibt der Befehl eine Liste zurück, die alle verfügbaren Optionen für *pathName* beschreibt (Format siehe Tk_ConfigureInfo). Ist *option* ohne *value* angegeben, gibt der Befehl eine Liste zurück, die die eine benannte Option beschreibt (identisch mit dem entsprechenden Teilliste der Rückgabe ohne *option*). Sind ein oder mehr *option*-*value*-Paare angegeben, setzt der Befehl die angegebenen Widget-Optionen auf die angegebenen Werte; in diesem Fall gibt er den leeren String zurück. *Option* kann jeden von **spinbox** akzeptierten Wert haben.

***pathName* delete *first* ?*last*?**

Löscht ein oder mehrere Elemente der Spinbox. *First* ist der Index des ersten zu löschenden Zeichens, *last* der Index des Zeichens unmittelbar nach dem letzten zu löschenden. Ist *last* nicht angegeben, entspricht es *first*+1, d. h. es wird ein einzelnes Zeichen gelöscht. Dieser Befehl gibt den leeren String zurück.

***pathName* get**

Gibt den String der Spinbox zurück.

***pathName* icursor *index***

Sorgt dafür, dass der Einfügecursor unmittelbar vor dem durch *index* gegebenen Zeichen angezeigt wird. Gibt einen leeren String zurück.

***pathName* identify *x y***

Gibt den Namen des Fensterelements zurück, das den Koordinaten *x* und *y* in der Spinbox entspricht. Rückgabewert ist einer von: **none**, **buttondown**, **buttonup**, entry.

***pathName* index *index***

Gibt den numerischen Index zurück, der *index* entspricht.

***pathName* insert *index string***

Fügt die Zeichen von *string* unmittelbar vor dem durch *index* angegebenen Zeichen ein. Gibt einen leeren String zurück.

***pathName* invoke *element***

Löst das angegebene Element, entweder **buttondown** oder **buttonup**, aus und löst die damit verbundene Aktion aus.

***pathName* scan *option args***

Dieser Befehl implementiert Scannen auf Spinboxen. Er hat zwei Formen, abhängig von *option* :

***pathName* scan mark *x***

Merkt sich *x* und die aktuelle Ansicht im Spinbox-Fenster; wird zusammen mit späteren **scan dragto**-Befehlen verwendet. Typischerweise ist dieser Befehl mit einem Mausklick im Widget verknüpft. Gibt den leeren String zurück.

***pathName* scan dragto *x***

Dieser Befehl berechnet die Differenz zwischen seinem *x*-Argument und dem *x*-Argument des letzten **scan mark**-Befehls für das Widget. Anschließend passt er die Ansicht um das 10-fache der x-Koordinatendifferenz nach links oder rechts an. Dieser Befehl ist typischerweise mit Mausbewegungsereignissen im Widget verknüpft, um den Effekt zu erzeugen, die Spinbox mit hoher Geschwindigkeit durch das Fenster zu ziehen. Der Rückgabewert ist ein leerer String.

***pathName* selection *option arg***

Dieser Befehl passt die Auswahl innerhalb einer Spinbox an. Er hat mehrere Formen, abhängig von *option*:

***pathName* selection adjust *index***

Findet das Ende der Auswahl, das *index* am nächsten liegt, und passt dieses Ende so an, dass es bei *index* liegt (d. h. einschließlich, aber nicht über *index* hinaus). Das andere Ende der Auswahl wird zum Ankerpunkt für künftige **select to**-Befehle. Liegt die Auswahl derzeit nicht in der Spinbox, wird eine neue Auswahl erstellt, die die Zeichen zwischen *index* und dem zuletzt gesetzten Auswahl-Ankerpunkt einschließt. Gibt den leeren String zurück.

***pathName* selection clear**

Löscht die Auswahl, wenn sie derzeit in diesem Widget liegt. Liegt die Auswahl nicht in diesem Widget, hat der Befehl keine Wirkung. Gibt einen leeren String zurück.

***pathName* selection element ?*element*?**

Setzt oder fragt das aktuell ausgewählte Element ab. Ist ein Spin-Button-Element angegeben, wird es gedrückt dargestellt.

***pathName* selection from *index***

Setzt den Auswahl-Ankerpunkt unmittelbar vor das durch *index* gegebene Zeichen. Ändert die Auswahl nicht. Gibt einen leeren String zurück.

***pathName* selection present**

Gibt 1 zurück, wenn in der Spinbox Zeichen ausgewählt sind, 0 wenn nichts ausgewählt ist.

***pathName* selection range *start end***

Setzt die Auswahl so, dass sie die Zeichen ab dem durch *start* indizierten bis einschließlich dem unmittelbar vor *end* umfasst. Bezieht sich *end* auf dasselbe Zeichen wie *start* oder ein früheres, wird die Auswahl in der Spinbox gelöscht.

***pathName* selection to *index***

Liegt *index* vor dem Ankerpunkt, wird die Auswahl auf die Zeichen von *index* bis vor den Ankerpunkt gesetzt. Ist *index* gleich dem Ankerpunkt, geschieht nichts. Liegt *index* nach dem Ankerpunkt, wird die Auswahl auf die Zeichen vom Ankerpunkt bis vor *index* gesetzt. Der Ankerpunkt wird durch den zuletzt in diesem Widget ausgeführten Befehl **select from** oder **select adjust** bestimmt. Liegt die Auswahl nicht in diesem Widget, wird eine neue Auswahl mit dem zuletzt für das Widget angegebenen Ankerpunkt erstellt. Gibt einen leeren String zurück.

***pathName* set ?*string*?**

Ist *string* angegeben, versucht die Spinbox, diesen Wert zu setzen, andernfalls gibt sie einfach den String der Spinbox zurück. Ist die Validierung aktiv, erfolgt sie beim Setzen des Strings.

***pathName* validate**

Dieser Befehl erzwingt eine Auswertung von **-validatecommand** unabhängig von den durch die Option **-validate** festgelegten Bedingungen. Dazu wird **-validate** vorübergehend auf **all** gesetzt. Er gibt 0 oder 1 zurück.

***pathName* xview *args***

Dieser Befehl fragt die horizontale Position des Texts im Fenster des Widgets ab und ändert sie. Er kann eine der folgenden Formen haben:

***pathName* xview**

Gibt eine Liste mit zwei Elementen zurück. Jedes Element ist ein reeller Bruchteil zwischen 0 und 1; zusammen beschreiben sie den horizontal sichtbaren Bereich im Fenster. Beispielsweise bedeuten die Elemente .2 und .6, dass 20 % des Spinbox-Texts links außerhalb des Bildschirms liegen, die mittleren 40 % im Fenster sichtbar sind und 40 % rechts außerhalb. Dies sind dieselben Werte, die Scrollbars über die Option **-xscrollcommand** übergeben werden.

***pathName* xview *index***

Passt die Ansicht im Fenster so an, dass das durch *index* gegebene Zeichen am linken Rand des Fensters angezeigt wird.

***pathName* xview moveto *fraction***

Passt die Ansicht im Fenster so an, dass das Zeichen *fraction* des Weges durch den Text am linken Rand des Fensters erscheint. *Fraction* muss ein Bruchteil zwischen 0 und 1 sein.

***pathName* xview scroll *number what***

Dieser Befehl verschiebt die Ansicht im Fenster nach links oder rechts gemäß *number* und *what*. *Number* muss eine Ganzzahl oder eine Fließkommazahl sein; ist es eine Fließkommazahl, wird sie in eine Ganzzahl umgewandelt, von 0 weg gerundet. *What* muss **pages** oder **units** oder eine Abkürzung davon sein. Ist *what* **pages**, passt sich die Ansicht um *number* Bildschirmseiten an. Ist *number* negativ, werden Zeichen weiter links sichtbar; ist es positiv, werden Zeichen weiter rechts sichtbar. Ist *what* **units**, verschiebt sich die Ansicht um *number* Zeichen durchschnittlicher Breite auf der Anzeige nach links oder rechts.

## Bindings

Tk erstellt automatisch Klassen-Bindings für Spinboxen, die ihnen folgendes Standardverhalten geben. In den Beschreibungen unten bezeichnet "Wort" eine zusammenhängende Gruppe aus Buchstaben, Ziffern oder "_"-Zeichen oder ein einzelnes anderes Zeichen.

1. Ein Klick mit Maustaste 1 positioniert den Einfügecursor unmittelbar vor dem Zeichen unter dem Mauszeiger, setzt den Eingabefokus auf dieses Widget und löscht jede Auswahl im

Widget. Ziehen mit Maustaste 1 erzeugt eine Auswahl zwischen Einfügecursor und dem Zeichen unter der Maus.

2. Doppelklick mit Maustaste 1 wählt das Wort unter der Maus aus und positioniert den Einfügecursor am Anfang des Worts. Ziehen nach einem Doppelklick erzeugt eine Auswahl aus ganzen Wörtern.
3. Dreifachklick mit Maustaste 1 wählt den gesamten Text in der Spinbox aus und positioniert den Einfügecursor vor dem ersten Zeichen.
4. Die Enden der Auswahl können durch Ziehen mit Maustaste 1 bei gedrückter Umschalttaste angepasst werden; dabei wird das Ende der Auswahl angepasst, das dem Mauszeiger beim Drücken von Taste 1 am nächsten war. Wurde vor dem Ziehen doppelgeklickt, wird die Auswahl in Einheiten ganzer Wörter angepasst.
5. Klick mit Maustaste 1 bei gedrückter Steuerung positioniert den Einfügecursor in der Spinbox, ohne die Auswahl zu beeinflussen.
6. Werden normale druckbare Zeichen in einer Spinbox eingegeben, werden sie an der Position des Einfügecursors eingefügt.
7. Die Ansicht in der Spinbox kann durch Ziehen mit der mittleren Maustaste (Taste 2, oder Taste 3 unter TkAqua) angepasst werden. Wird die mittlere Maustaste ohne Mausbewegung geklickt, wird die Auswahl an der Mausposition in die Spinbox kopiert.
8. Wird die Maus beim Gedrückthalten von Taste 1 links oder rechts aus der Spinbox gezogen, scrollt die Spinbox automatisch, um mehr Text sichtbar zu machen (falls auf der Seite, auf der die Maus das Fenster verließ, noch Text außerhalb des Bildschirms liegt).
9. Die Tasten Links und Rechts bewegen den Einfügecursor um ein Zeichen nach links oder rechts; sie löschen außerdem jede Auswahl in der Spinbox und setzen den Auswahl-Anker. Wird Links oder Rechts mit gedrückter Umschalttaste eingegeben, bewegt sich der Einfügecursor und die Auswahl wird um das neue Zeichen erweitert. Steuerung-Links und Steuerung-Rechts bewegen den Einfügecursor wortweise; Steuerung-Umschalt-Links und Steuerung-Umschalt-Rechts bewegen ihn wortweise und erweitern die Auswahl. Steuerung-b und Steuerung-f verhalten sich wie Links bzw. Rechts. Meta-b und Meta-f verhalten sich wie Steuerung-Links bzw. Steuerung-Rechts.
10. Die Pos1-Taste oder Steuerung-a bewegt den Einfügecursor an den Anfang der Spinbox und löscht jede Auswahl. Umschalt-Pos1 bewegt den Einfügecursor an den Anfang und erweitert die Auswahl bis dorthin.
11. Die Ende-Taste oder Steuerung-e bewegt den Einfügecursor ans Ende der Spinbox und löscht jede Auswahl. Umschalt-Ende bewegt den Cursor ans Ende und erweitert die Auswahl bis dorthin.
12. Die Select-Taste und Steuerung-Leertaste setzen den Auswahl-Anker auf die Position des Einfügecursors. Sie beeinflussen die aktuelle Auswahl nicht. Umschalt-Select und Steuerung-Umschalt-Leertaste passen die Auswahl an die aktuelle Position des Einfügecursors an und wählen vom Anker bis zum Einfügecursor, falls zuvor keine Auswahl bestand.
13. Steuerung-/ wählt den gesamten Text in der Spinbox aus.
14. Steuerung-\ löscht jede Auswahl in der Spinbox.
15. Die F16-Taste (auf vielen Sun-Arbeitsplätzen mit Copy beschriftet) oder Meta-w kopiert die Auswahl im Widget in die Zwischenablage, falls eine Auswahl besteht.
16. Die F20-Taste (auf vielen Sun-Arbeitsplätzen mit Cut beschriftet) oder Steuerung-w kopiert die Auswahl in die Zwischenablage und löscht sie. Gibt es keine Auswahl im Widget, haben

diese Tasten keine Wirkung.

17. Die F18-Taste (auf vielen Sun-Arbeitsplätzen mit Paste beschriftet) oder Steuerung-y fügt den Inhalt der Zwischenablage an der Position des Einfügecursors ein.
18. Die Entf-Taste löscht die Auswahl, falls eine in der Spinbox besteht. Gibt es keine Auswahl, löscht sie das Zeichen rechts vom Einfügecursor.
19. Die Rücktaste und Steuerung-h löschen die Auswahl, falls eine in der Spinbox besteht. Gibt es keine Auswahl, löschen sie das Zeichen links vom Einfügecursor.
20. Steuerung-d löscht das Zeichen rechts vom Einfügecursor.
21. Meta-d löscht das Wort rechts vom Einfügecursor.
22. Steuerung-k löscht alle Zeichen rechts vom Einfügecursor.
23. Steuerung-t kehrt die Reihenfolge der beiden Zeichen rechts vom Einfügecursor um.

Ist die Spinbox mit der Option **-state** deaktiviert, kann die Ansicht der Spinbox weiterhin angepasst und Text in der Spinbox weiterhin ausgewählt werden, aber es wird kein Einfügecursor angezeigt und es finden keine Textänderungen statt.

Das Verhalten von Spinboxen kann geändert werden, indem neue Bindings für einzelne Widgets definiert oder die Klassen-Bindings neu definiert werden.

*Copyright (c) 2000 Jeffrey Hobbs.*
*Copyright (c) 2000 Ajuba Solutions.*

## sysnotify

*Stichwörter:* sysnotify, notify, alert

### Name

sysnotify - Erstellt ein Benachrichtigungsfenster mit Titel und Nachricht

### Übersicht

**tk sysnotify** *title message*

### Beschreibung

Der Befehl tk sysnotify erzeugt eine plattformspezifische Systembenachrichtigung. Er soll dem Benutzer eine kurze, unaufdringliche Meldung liefern, indem kurz ein Fenster in einer Bildschirmecke erscheint.

### Beispiel

Beispiel für tk sysnotify:

```
tk sysnotify "Alert" \
      "This is just a test of the Tk System Notification Code."
```

### Plattformhinweise

Die Versionen für macOS und Windows sind native Implementierungen mit System-APIs. Die X11-Version hängt optional von libnotify ab und fällt auf eine reine Tcl-Implementierung zurück, wenn libnotify nicht installiert ist. Auf jeder Plattform enthält die Benachrichtigung ein plattformspezifisches Standardbild zum Text.

**macOS**

Unter macOS wird der Benutzer um Berechtigung für Benachrichtigungen gebeten. Das muss in den Systemeinstellungen von Apple unter "Mitteilungen" aktiviert werden.

Beim Einsatz einer Anwendung mit der eigenständigen Wish.app-Version scheint es nötig zu sein, die Bundle-ID in der Info.plist der Anwendung mit "**com**" beginnen zu lassen, damit Benachrichtigungen funktionieren. Eine andere Präfix-Variante der Bundle-ID (z. B. "**tk.tcl.tkchat**") führt dazu, dass Benachrichtigungen still fehlschlagen.

**Windows**

Das Bild stammt aus der Systemleiste; **sysnotify** kann nur aufgerufen werden, wenn ein systray eingerichtet wurde.

*Copyright (c) 2020 Kevin Walzer*

# systray

*Stichwörter:* systray, image, callback

## Name

systray - Erzeugt ein Symbol in der plattformspezifischen Systemleiste

## Übersicht

**tk systray create -image** *image* ?*-text text*? ?*-button1 callback*? ?*-button3 callback*?
**tk systray configure** *?option?* ?value option value ...?
**tk systray exists**
**tk systray destroy**

## Beschreibung

Der Befehl **tk systray create** erzeugt ein Symbol in der plattformspezifischen Leiste. Das Widget erhält ein Tk-Bild für die Anzeige, optional eine Zeichenkette für einen Tooltip und optionale Rückrufe, die an und gebunden sind.

Der Befehl **tk systray configure** setzt eine oder mehrere Optionen des Leistensymbols. Die einstellbaren Optionen sind dieselben wie beim Unterbefehl **create**. Ist nur ein Optionsname angegeben, gibt der Befehl den aktuellen Wert dieser Option zurück. Ohne Option gibt er die Liste aller Optionen und ihrer aktuellen Werte zurück.

Der Befehl **tk systray exists** prüft, ob ein Leistensymbol erzeugt wurde. Er gibt einen booleschen Wert zurück.

Der Befehl **tk systray destroy** nimmt das Symbol aus der Anzeige und gibt es frei.

Aus Sicht der Oberfläche wird nur ein Symbol je Interpreter unterstützt; der Versuch, weitere Symbole zu erzeugen, gibt einen Fehler zurück. Das vorhandene Leistensymbol lässt sich mit anderen Bildern und Zeichenketten ändern, um den Zustand der Anwendung anzuzeigen. Weitere Interpreter in einer laufenden Wish-Instanz erlauben weitere Symbole.

## Beispiele

Ein Beispiel für tk systray:

```
image create photo book -data \
        R0lGODlhDwAPAKIAAP//////AP8AAMDAwICAgAAAAAAAAAAAAACwAAAAADwAPAAA
    DSQhA2u5ksPeKABKSCaya29d4WKgERFF0l1IMQCAKatvBJ0OTdzzXI1xMB3TBZAvATt
    B6NSLKleXi3OBoLqrVgc0yv+DVSEUuFxIAOw==
tk systray create -image book -text "tk systray sample" \
        -button1 {puts "Here is the tk systray output"} \
        -button3 {puts "here is alternate output"}
```

So lässt sich das Symbol tk systray ändern:

```
image create photo book_page -data \
        R0lGODlhCwAPAKIAAP//////AMDAwICAgAAA/wAAAAAAAAAAACwAAAAACwAPAAA
    DMzi6CzAugiAgDGE68aB0RXgRJBFVX0SNpQlUWfahQOvSsgrX7eZJMlQMWBEYj8iQch
    lKAAA7
tk systray configure -image book_page -text "Updated sample" \
        -button1 {puts "Different output from the tk systray"} \
        -button3 {puts "and more different output from the tk systray"}
```

## Plattformhinweise

Die X11-Implementierung wird nach bestem Bemühen unterstützt, weil sie vom Fenstermanager abhängt. Die Option "text", umgesetzt als Tooltip, erscheint nicht immer, wenn der Fenstermanager das nicht kann; bei manchen Fenstermanagern erscheint das Leistensymbol selbst nicht.

Unter Windows muss das in **-image** angegebene Tk-Bild ein Photo-Image sein. Auf anderen Plattformen ist ein Bitmap- oder ein Photo-Image zulässig.

*Copyright (c) 2020 Kevin Walzer*

# text

*Stichwörter:* text, widget, tkvars

## Name

text, tk_textCopy, tk_textCut, tk_textPaste - Erstellt und manipuliert 'text'-Hypertext-Bearbeitungswidgets

## Übersicht

**text** *pathName ?options?*
**tk_textCopy** *pathName*
**tk_textCut** *pathName*
**tk_textPaste** *pathName*

## Standardoptionen

**-background**, **-highlightthickness**, **-relief**, **-borderwidth**, **-insertbackground**, **-selectbackground**, **-cursor**, **-insertborderwidth**, **-selectborderwidth**, **-exportselection**, **-insertofftime**, **-selectforeground**, **-font**, **-insertontime**, **-setgrid**, **-foreground**, **-insertwidth**, **-takefocus**, **-highlightbackground**, **-padx**, **-xscrollcommand**, **-highlightcolor**, **-pady**, **-yscrollcommand**

Siehe die Manualseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-autoseparators (autoSeparators/AutoSeparators)**
Legt einen booleschen Wert fest, der angibt, ob automatisch Trennzeichen in den Rückgängig-Stapel eingefügt werden. Nur bedeutsam, wenn die Option **-undo** true ist.

**-blockcursor (blockCursor/BlockCursor)**
Legt einen booleschen Wert fest, der angibt, ob der blinkende Einfügecursor als rechteckiger Block in Zeichengröße gezeichnet werden soll. Ist der Wert false (Standard), wird eine dünne vertikale Linie als Einfügecursor verwendet.

**-endline (endLine/EndLine)**
Legt einen ganzzahligen Zeilenindex fest, der die Zeile des zugrunde liegenden textuellen Datenspeichers angibt, die unmittelbar nach der letzten im Widget enthaltenen Zeile liegen soll. Dadurch kann ein Text-Widget nur einen Teil eines größeren Textes wiedergeben. Statt einer Ganzzahl kann für diese Konfigurationsoption der leere String angegeben werden; das Widget endet dann in der allerletzten Zeile des textuellen Datenspeichers.

**-height (height/Height)**
Legt die gewünschte Höhe des Fensters in Zeichen der mit **-font** angegebenen Schriftart fest. Muss mindestens eins sein.

**-inactiveselectbackground (inactiveSelectBackground/Foreground)**
Legt die Farbe fest, die für die Auswahl (Tag **sel**) verwendet wird, wenn das Fenster nicht den Eingabefokus hat. Ist der Wert leer (**{}**), wird keine Auswahl angezeigt, wenn das Fenster nicht den Fokus hat.

**-insertunfocussed (insertUnfocussed/InsertUnfocussed)**
Legt fest, wie der Einfügecursor angezeigt wird, wenn das Widget nicht den Fokus hat. Muss none (Standard) sein, d. h. der Cursor wird nicht angezeigt, hollow bedeutet eine hohle Box, solid eine ausgefüllte Box. hollow und solid wirken sehr ähnlich, wenn -blockcursor false ist.

**-maxundo (maxUndo/MaxUndo)**
Legt die maximale Anzahl zusammengesetzter Rückgängig-Aktionen auf dem Rückgängig-Stapel fest. Null oder ein negativer Wert bedeuten einen unbegrenzten Stapel.

**-spacing1 (spacing1/Spacing1)**
Fordert zusätzlichen Abstand oberhalb jeder Textzeile im Widget an (beliebige Standardformen für Bildschirmabstände). Bei umbrochenen Zeilen gilt die Option nur für die erste Anzeigezeile. Kann durch **-spacing1** in Tags überschrieben werden.

**-spacing2 (spacing2/Spacing2)**
Gibt bei umbrochenen Zeilen den zusätzlichen Abstand zwischen den Anzeigezeilen an, die eine logische Textzeile darstellen. Der Wert kann in beliebigen Standardformen für Bildschirmabstände angegeben werden. Kann durch **-spacing2** in Tags überschrieben werden.

**-spacing3 (spacing3/Spacing3)**
Fordert zusätzlichen Abstand unterhalb jeder Textzeile an (beliebige Standardformen für Bildschirmabstände). Bei umbrochenen Zeilen gilt die Option nur für die letzte Anzeigezeile. Kann durch **-spacing3** in Tags überschrieben werden.

**-startline (startLine/StartLine)**
Legt einen ganzzahligen Zeilenindex für die erste Zeile des zugrunde liegenden textuellen

Datenspeichers fest, die im Widget enthalten sein soll. Dadurch kann ein Text-Widget nur einen Teil eines größeren Textes wiedergeben. Statt einer Ganzzahl kann der leere String angegeben werden; das Widget beginnt dann in der allerersten Zeile des Datenspeichers.

**-state (state/State)**

Legt einen von zwei Zuständen für den Text fest: **normal** oder **disabled**. Ist der Text deaktiviert, dürfen keine Zeichen eingefügt oder gelöscht werden und es wird kein Einfügecursor angezeigt, auch wenn der Eingabefokus im Widget liegt.

**-tabs (tabs/Tabs)**

Legt eine Menge von Tabstopps für das Fenster fest. Der Optionswert ist eine Liste von Bildschirmabständen mit den Positionen der Tabstopps, jeweils relativ zum linken Rand des Widgets (ohne Rahmen, Padding usw.). Jeder Position kann im nächsten Listenelement eines der Schlüsselwörter **left**, **right**, **center** oder **numeric** folgen, die die Ausrichtung des Textes am Tabstopp festlegen. **Left** ist Standard: Der Text nach dem Tabulator wird mit seiner linken Kante am Tabstopp positioniert. **Right** positioniert die rechte Kante, **center** zentriert den Text. **Numeric** positioniert den Dezimalpunkt am Tabstopp; fehlt ein Dezimalpunkt, wird die niederwertigste Ziffer knapp links vom Tabstopp gesetzt; fehlt eine Zahl, wird rechtsbündig am Tabstopp ausgerichtet. Beispiel: "**-tabs {2c left 4c 6c center}**" erzeugt drei Tabstopps im Abstand von zwei Zentimetern; die ersten beiden linksbündig, der dritte zentriert.

Reicht die Tabstopp-Liste nicht für alle Tabs einer Zeile, extrapoliert Tk neue Tabstopps anhand von Abstand und Ausrichtung des letzten Tabstopps. Tab-Abstände müssen strikt positiv und streng wachsend sein (sonst Fehler). Der Wert von **-tabs** kann durch **-tabs** in Tags überschrieben werden.

Ist **-tabs** nicht gesetzt oder eine leere Liste, verwendet Tk Standard-Tabs im Abstand von acht (durchschnittlich breiten) Zeichen. Für anderen Standardabstand, z. B. alle 4 Zeichen: "**-tabs "[expr {4 * [font measure $font 0]}] left" -tabstyle wordprocessor**".

**-tabstyle (tabStyle/TabStyle)**

Legt fest, wie der Zusammenhang zwischen Tabstopps einer Zeile und Tabulatoren im Text dieser Zeile interpretiert wird. Der Wert muss **tabular** (Standard) oder **wordprocessor** sein. Tabs werden beim Durchlaufen des Textes ausgewertet. Bei **tabular** wird der *n*. Tabulator der Zeile dem *n*. Tabstopp der Zeile zugeordnet. Liegt die x-Koordinate des Tabulators rechts vom *n*. Tabstopp, wird als Fallback ein einzelnes Leerzeichen eingefügt. Bei **wordprocessor** verwendet jeder Tabulator den ersten Tabstopp rechts der bereits gesetzten Zeichen. **-tabstyle** kann durch Tag-Optionen überschrieben werden.

**-undo (undo/Undo)**

Legt einen booleschen Wert fest, ob der Rückgängig-Mechanismus aktiv ist.

**-width (width/Width)**

Legt die gewünschte Fensterbreite in Zeichen der mit **-font** angegebenen Schriftart fest. Hat die Schrift keine einheitliche Breite, wird die Breite der Zeichen "0" für die Umrechnung verwendet.

**-wrap (wrap/Wrap)**

Legt fest, wie mit Zeilen umgegangen wird, die zu lang für eine Anzeigezeile sind. Der Wert muss **none**, **char** oder **word** sein. **none**: Jede Textzeile erscheint als genau eine Bildschirmzeile; überzählige Zeichen werden nicht angezeigt. In den anderen Modi wird bei

Bedarf umgebrochen. **char** erlaubt Umbrüche nach jedem Zeichen; **word** nur an Wortgrenzen.

## Beschreibung

Der Befehl **text** erstellt ein neues Fenster (Argument *pathName*) und macht es zu einem Text-Widget. Weitere Optionen können in der Befehlszeile oder der Optionsdatenbank gesetzt werden (Hintergrundfarbe, Relief usw.). Der Befehl **text** gibt den Pfadnamen des neuen Fensters zurück.

Ein Text-Widget zeigt eine oder mehrere Textzeilen an und erlaubt deren Bearbeitung. Es unterstützt vier Arten von Annotationen: Tags, Marken, eingebettete Fenster und eingebettete Bilder. Tags erlauben unterschiedliche Darstellung (Schrift, Farben). Tcl-Befehle können Tags zugeordnet werden, sodass Skripte bei Tastendrücken oder Mausklicks in bestimmten Bereichen ausgeführt werden. Siehe Tags unten.

Die zweite Annotationsform sind schwebende Markierungen im Text, "Marken". Marken merken sich interessante Positionen während der Bearbeitung. Siehe Marken unten.

Die dritte Form erlaubt beliebige eingebettete Fenster. Siehe Eingebettete Fenster unten.

Die vierte Form erlaubt eingebettete Tk-Bilder. Siehe Eingebettete Bilder unten.

Das Text-Widget hat einen eingebauten Rückgängig-/Wiederholen-Mechanismus. Siehe Der Rückgängig-Mechanismus unten.

Peer-Widgets sind weitere Text-Widgets mit denselben zugrunde liegenden Daten (Text, Marken, Tags, Bilder usw.). Siehe Peer-Widgets unten.

## Indizes

Viele Widget-Befehle für Text-Widgets nehmen einen oder mehrere Indizes als Argumente. Ein Index ist eine Zeichenkette, die eine bestimmte Stelle im Text bezeichnet, z. B. Einfügeposition oder Endpunkt eines zu löschenden Bereichs. Indizes haben die Syntax

```
base modifier modifier modifier ...
```

*wobei base* den Ausgangspunkt angibt und die *modifier* den Index davon aus justieren (z. B. ein Zeichen vor oder zurück). Jeder Index muss eine *base* enthalten; *modifier* sind optional. Die meisten Modifier (siehe unten) erlauben einen optionalen Submodifier. Gültige Submodifier sind **any** und **display**. Ist der Submodifier abgekürzt, muss Leerzeichen folgen; sonst kann Submodifier und *modifier* ohne Leerzeichen aneinander stehen. Typischerweise bewirkt **display**, dass der folgende *modifier* sichtbare bzw. nicht ausgeblendete Einheiten statt logischer Einheiten meint; Details je nach Fall unten. *count* in einem Modifier kann positiv oder negativ sein; "*base* - -3 lines" ist gültig (äquivalent zu "*base* +3lines").

Die *base* eines Index muss eine der folgenden Formen haben:

***line.char***

Bezeichnet das *char*. Zeichen in Zeile *line*. Zeilen werden ab 1 nummeriert (wie bei anderen UNIX-Programmen). Innerhalb einer Zeile beginnen Zeichen bei 0. Ist *char* **end**, bezeichnet das den Zeilenumbruch am Zeilenende.

**@*x,y***

Bezeichnet das Zeichen unter dem Pixel mit den Koordinaten *x* und *y* im Textfenster.

**end**

Bezeichnet das Textende (das Zeichen unmittelbar nach dem letzten Zeilenumbruch).

***mark***

Bezeichnet das Zeichen unmittelbar nach der Marke *mark* (siehe Marken).

***tag*.first**

Erstes mit *tag* markiertes Zeichen. Fehler, wenn derzeit kein Zeichen dieses Tags hat.

***tag*.last**

Zeichen unmittelbar nach dem letzten mit *tag* markierten Zeichen. Fehler, wenn derzeit kein Zeichen dieses Tags hat.

***pathName***

Position des eingebetteten Fensters *pathName*. Fehler, wenn kein solches Fenster existiert.

***imageName***

Position des eingebetteten Bildes *imageName*. Fehler, wenn kein solches Bild existiert.

Könnte die *base* mehreren Formen entsprechen (z. B. gleicher Wert für *mark* und *imageName*), hat die frühere Form in der obigen Liste Vorrang. Folgen Modifier der Basis, muss jeder eine der unten genannten Formen haben. Schlüsselwörter wie **chars** und **wordend** dürfen abgekürzt werden, solange die Abkürzung eindeutig ist.

**+ *count* ?*submodifier*? chars**

Verschiebt den Index um *count* Zeichen vorwärts, ggf. auf spätere Zeilen. Gibt es nach dem aktuellen Index weniger als *count* Zeichen, wird der letzte Index im Text gesetzt. Leerzeichen um *count* sind optional. Mit Submodifier **display** werden ausgeblendete Zeichen übersprungen. Mit **any** werden alle gezählt. Aus historischen Gründen zählt ohne Submodifier die Einheit "Indexposition" (siehe Indizes). Das kann sich in einer künftigen Hauptversion ändern; für Indexzählungen **indices** verwenden, wo möglich.

**- *count* ?*submodifier*? chars**

Verschiebt den Index um *count* Zeichen rückwärts, ggf. auf frühere Zeilen. Gibt es vor dem aktuellen Index weniger als *count* Zeichen, wird der erste Index (1.0) gesetzt. Leerzeichen um *count* sind optional. Mit **display** werden ausgeblendete Zeichen übersprungen. Mit **any** werden alle gezählt. Ohne Submodifier zählt aus historischen Gründen die Einheit "Indexposition" (siehe Indizes); künftig kann sich das ändern — **indices** bevorzugen.

**+ *count* ?*submodifier*? indices**

Verschiebt den Index um *count* Indexpositionen vorwärts. Eine Indexposition ist ein Zeichen oder ein eingebettetes Bild/Fenster. Gibt es weniger als *count* Positionen danach, wird die letzte gesetzt. Mit **display** werden ausgeblendete Positionen übersprungen. Mit **any** (oder ohne Submodifier) werden alle gezählt.

**- *count* ?*submodifier*? indices**

Verschiebt den Index um *count* Indexpositionen rückwärts. Gibt es weniger als *count* Positionen davor, wird 1.0 gesetzt. Mit **display** werden ausgeblendete Positionen übersprungen. Mit **any** (oder ohne Submodifier) werden alle gezählt.

**+ *count* ?*submodifier*? lines**

Verschiebt den Index um *count* Zeilen vorwärts bei gleicher Zeichenposition. Gibt es weniger Zeilen danach, wird dieselbe Position in der letzten Zeile verwendet; reicht die Zeile nicht, die letzte Zeichenposition (Zeilenumbruch). Mit **display** zählt jede sichtbare Anzeigezeile; mit **any** oder ohne Submodifier jede logische Zeile (unabhängig vom Umbruch). Ohne Umbruch sind beide gleichwertig.

**- *count* ?*submodifier*? lines**
Verschiebt den Index um *count* logische Zeilen rückwärts bei gleicher Zeichenposition. Gibt es weniger Zeilen davor, wird dieselbe Position in der ersten Zeile verwendet; reicht die Zeile nicht, die letzte Zeichenposition. Mit **display** zählt jede Anzeigezeile; mit **any** oder ohne Submodifier jede logische Zeile. Ohne Umbruch gleichwertig.

**?*submodifier*? linestart**
Setzt den Index auf den ersten Index der Zeile. Mit **display** der Anzeigezeile, sonst der logischen Zeile.

**?*submodifier*? lineend**
Setzt den Index auf den letzten Index der Zeile (Zeilenumbruch). Mit **display** der Anzeigezeile, sonst der logischen Zeile.

**?*submodifier*? wordstart**
Erstes Zeichen des Wortes am aktuellen Index. Ein Wort besteht aus Buchstaben, Ziffern oder Unterstrichen oder ist ein einzelnes anderes Zeichen. Mit **display** nur nicht ausgeblendete Zeichen, sonst alle.

**?*submodifier*? wordend**
Zeichen unmittelbar nach dem letzten des Wortes am aktuellen Index. Steht der Index bereits auf dem letzten Textzeichen, bleibt er unverändert. Mit **display** nur nicht ausgeblendete Zeichen, sonst alle.

Stehen mehrere Modifier aneinander, werden sie von links nach rechts angewendet. Beispiel: "**end - 1 chars**" ist das vorletzte Zeichen; "**insert wordstart - 1 c**" das Zeichen vor dem ersten des Worts am Einfügecursor. Nach jedem Schritt wird der Index auf Gültigkeit im Widget beschränkt; "**1.0 -1c +1c**" ergibt "**2.0**".

Führen Modifier zu Änderungen über Anzeigezeilen, Anzeigezeichen oder Anzeigeindizes und verweist die *base* auf einen Index innerhalb eines ausgeblendeten Tags, gilt die *base* als äquivalent zum ersten folgenden nicht ausgeblendeten Index.

## Tags

Die erste Annotationsform in Text-Widgets ist ein Tag. Ein Tag ist eine Zeichenkette, die mit einigen Textzeichen verknüpft ist. Tags dürfen beliebige Zeichen enthalten; am besten vermeidet man Leerzeichen, **+** und **-**, da diese in Indizes Sonderbedeutung haben und solche Tags nicht als Indizes verwendet werden können. Beliebig viele Tags können Zeichen zugeordnet sein. Ein Tag kann ein Zeichen, einen Bereich oder mehrere Bereiche betreffen. Ein Zeichen kann beliebig viele Tags haben.

Unter Tags gilt eine Prioritätsreihenfolge, die bei tag-bezogenen Funktionen unten verwendet wird. Beim Definieren (Zuordnung zu Zeichen, Anzeigeoptionen, Bindings) erhält ein Tag höhere Priorität als alle bestehenden. Die Reihenfolge kann mit "*pathName* **tag raise**" und "*pathName* **tag lower**" geändert werden.

Tags haben drei Zwecke. Erstens steuern sie die Darstellung. Standardmäßig gelten **-background**, **-font** und **-foreground** des Widgets. Über "*pathName* **tag configure**" können einzelnen Tags Anzeigeoptionen zugeordnet werden; diese überschreiben den Standard. Für Tags werden derzeit folgende Optionen unterstützt:

**-background *color***
*Color* legt die Hintergrundfarbe für mit dem Tag verknüpfte Zeichen fest (Formen von

Tk_GetColor).

**-bgstipple *bitmap***

*Bitmap* ist ein Stippelmuster für den Hintergrund (Tk_GetBitmap). Fehlt *bitmap* oder ist es leer, wird vollflächig gefüllt.

**-borderwidth *pixels***

*Pixels* ist die Rahmenbreite (Tk_GetPixels), zusammen mit **-relief** für das gewünschte Erscheinungsbild.

**-elide *boolean***

*Elide* gibt an, ob Daten ausgeblendet werden. Ausgeblendete Daten (Zeichen, Bilder, Fenster usw.) werden nicht angezeigt und nehmen keinen Platz ein, verhalten sich sonst normal.

**-fgstipple *bitmap***

*Bitmap* ist ein Stippelmuster für Vordergrund (Text, Unterstreichungen). Fehlt *bitmap* oder ist es leer, wird vollflächig gefüllt.

**-font *fontName***

*FontName* ist der Schriftname (Tk_GetFont).

**-foreground *color***

*Color* legt die Vordergrundfarbe fest (Tk_GetColor).

**-justify *justify***

Hat das erste nicht ausgeblendete Zeichen einer Anzeigezeile ein Tag mit dieser Option, bestimmt *justify* die Ausrichtung (**left**, **right**, **center**). Bei Umbruch gilt pro Anzeigezeile das erste nicht ausgeblendete Zeichen.

**-lmargin1 *pixels***

Hat das erste nicht ausgeblendete Zeichen einer Textzeile ein Tag mit dieser Option, gibt *pixels* den linken Einzug an (Standardformen für Bildschirmabstände). Bei Umbruch nur für die erste Anzeigezeile; **-lmargin2** für folgende.

**-lmargin2 *pixels***

Gilt für die erste nicht ausgeblendete Zeichen einer Anzeigezeile, die nicht die erste ihrer Textzeile ist (Umbruch), *pixels* als linker Einzug. Nur bei aktiviertem Umbruch für die zweite und weitere Anzeigezeilen.

**-lmargincolor *color***

*Color* ist die Hintergrundfarbe in Bereichen ohne Zeichen wegen **-lmargin1** oder **-lmargin2**. Fehlt *color* oder ist es leer, gilt **-background** des Tags oder des Widgets.

**-offset *pixels***

*Pixels* verschiebt die Text-Baseline vertikal (z. B. positiv für Hochstellung, negativ für Tiefstellung).

**-overstrike *boolean***

Gibt an, ob ein horizontaler Strich durch die Zeichenmitte gezeichnet wird (Tcl_GetBoolean).

**-overstrikefg *color***

*Color* für die Durchstreichung. Fehlt *color* oder ist es leer, gilt **-foreground** des Tags.

**-relief *relief***

*Relief* für den Rahmen (Tk_GetRelief), zusammen mit **-borderwidth**.

**-rmargin *pixels***

Hat das erste nicht ausgeblendete Zeichen einer Anzeigezeile ein Tag mit dieser Option, *pixels* als rechter Rand bis zum Fensterrand. Nur bei Umbruch; pro Anzeigezeile das erste nicht ausgeblendete Zeichen.

**-rmargincolor *color***
*Color* für Hintergrund in Bereichen ohne Zeichen wegen **-rmargin**. Fehlt *color* oder ist es leer, gilt **-background** des Tags oder des Widgets.

**-selectbackground *color***
*Color* für den Hintergrund ausgewählter Elemente. Fehlt *color* oder ist es leer, gilt **-background** des Tags.

**-selectforeground *color***
*Color* für den Vordergrund ausgewählter Elemente. Fehlt *color* oder ist es leer, gilt **-foreground** des Tags.

**-spacing1 *pixels***
Zusätzlicher Abstand oberhalb jeder Textzeile. Bei Umbruch nur für die erste Anzeigezeile.

**-spacing2 *pixels***
Bei Umbruch zusätzlicher Abstand zwischen Anzeigezeilen einer Textzeile.

**-spacing3 *pixels***
Zusätzlicher Abstand unterhalb jeder Textzeile. Bei Umbruch nur für die letzte Anzeigezeile.

**-tabs *tabList***
*TabList* wie **-tabs** des Widgets. Gilt nur, wenn sie für das erste nicht ausgeblendete Zeichen der Anzeigezeile gilt. Leerer String hebt die Option auf. Nicht-leerer String mit leerer Liste (z. B. **-tags\0{\0}**) fordert Standard-Tabs à 8 Zeichen wie bei **-tabs** des Widgets.

**-tabstyle *style***
*Style* ist *tabular* oder *wordprocessor*. Gilt nur für das erste nicht ausgeblendete Zeichen der Anzeigezeile. Leerer String hebt die Option auf.

**-underline *boolean***
Unterstreichung ja/nein (Tcl_GetBoolean).

**-underlinefg *color***
*Color* für die Unterstreichung. Fehlt *color* oder ist es leer, gilt **-foreground** des Tags.

**-wrap *mode***
*Mode* für zu breite Zeilen (**none**, **char**, **word** wie beim Widget). Gilt nur für das erste nicht ausgeblendete Zeichen der Anzeigezeile und überschreibt **-wrap** des Widgets.

Hat ein Zeichen mehrere Tags mit widersprüchlichen Anzeigeoptionen, gilt der Tag mit höchster Priorität. Ist eine Option für einen Tag nicht gesetzt oder leer, wird sie nicht verwendet; die nächsthöhere Priorität gilt. Spezifiziert kein Tag eine Option, gilt der Widget-Standard.

Zweitens dienen Tags Ereignis-Bindings. Bindings können einem Tag wie einer Widget-Klasse zugeordnet werden: Bei bestimmten X-Ereignissen auf markierten Zeichen wird ein Tcl-Befehl ausgeführt. So lassen sich Hypertext-ähnliche Verhalten realisieren; siehe "*pathName* **tag bind**" unten. Tag-Bindings werden von allen Peer-Widgets geteilt (einschließlich **sel**).

Drittens verwalten Tags die Auswahl. Siehe Die Auswahl unten. Außer **sel** teilen alle Tags alle Peer-Widgets und sind von jedem gleich bearbeitbar. **sel** existiert in jedem Peer separat (Bindings zu **sel** werden geteilt).

## Marken

Die zweite Annotationsform sind Marken. Marken merken sich bestimmte Stellen im Text. Sie ähneln Tags (Namen, Bezug auf Stellen), sind aber keinem Zeichen zugeordnet, sondern der

Lücke zwischen zwei Zeichen. Eine Marke hat jeweils nur eine Position. Werden benachbarte Zeichen gelöscht, bleibt die Marke mit neuen Nachbarn. Werden Zeichen mit einem Tag gelöscht, entfällt die Tag-Zuordnung. Marken werden mit "*pathName* **mark**" bearbeitet; ihre Position ist als Index mit dem Markennamen abfragbar.

Jede Marke hat eine "Schwerkraft" (**left** oder **right**), die bestimmt, was bei Einfügen am Markenpunkt passiert. Bei **left** bleibt die Marke links vom eingefügten Text; bei **right** erscheint neuer Text links von der Marke (Marke bleibt ganz rechts). Standard ist **right**.

Der Namensraum der Marken ist getrennt von dem der Tags: derselbe Name kann für Marke und Tag verwendet werden, bezeichnet aber Unterschiedliches.

Zwei Marken sind besonders: **insert** gehört zum Einfügecursor (siehe Der Einfügecursor); **current** zum der Maus nächsten Zeichen und folgt Maus und Textänderungen (Ausnahme: bei gedrückter Maustaste keine Aktualisierung bis alle Tasten losgelassen sind). Diese beiden dürfen nicht gelöscht werden. Alle anderen Marken teilen Peer-Widgets und sind von jedem Peer gleich bearbeitbar.

## Eingebettete Fenster

Die dritte Annotationsform sind eingebettete Fenster. Jedes eingebettete Fenster wird an einer Textstelle angezeigt. Beliebig viele sind möglich; jedes Widget kann eingebettet werden (Geometrie-Regeln: Textfenster ist Eltern- oder Vorfahren-Widget).

Die Position wird bei Textänderung und Scrollen aktualisiert; beim Betreten/Verlassen des sichtbaren Bereichs wird gemappt/unmapped. Jedes Fenster belegt eine Index-Einheit und ist über Fensternamen oder Indexposition ansprechbar. Wird der Textbereich gelöscht, wird das Fenster zerstört; dasselbe gilt beim Löschen des gesamten Text-Widgets.

Ausblenden unmittelbar nach Planung via *pathName* **window create** *index* **-create** verhindert die effektive Erstellung. Einblenden löst das Erstellungsskript aus. Nach Zerstörung eines ausgeblendeten Fensters erfolgt keine automatische Neuerstellung.

Beim Hinzufügen mit *pathName* **window create** können Optionen gesetzt und später mit *pathName* **window configure** geändert werden. Unterstützt werden:

**-align *where***

Ist das Fenster niedriger als die Zeile, bestimmt *where* die vertikale Position: **top** (Oberkante an Zeilenoberkante), **center** (zentriert), **bottom** (Unterkante an Zeilenunterkante), **baseline** (Unterkante an Zeilen-Baseline).

**-create *script***

Tcl-Skript zur Fenstererstellung. Ohne **-window** wird es vor der Anzeige ausgewertet. *Script* muss ein Fenster erzeugen und dessen Namen zurückgeben. Vor der Auswertung: *%W* → Name des Text-Widgets, *%%* → %. Wird das Fenster gelöscht, wird *script* bei der nächsten Anzeige erneut ausgewertet.

**-padx *pixels***

Zusätzlicher Abstand links und rechts (übliche Formen für Bildschirmabstände).

**-pady *pixels***

Zusätzlicher Abstand oben und unten.

**-stretch *boolean***

Soll ein zu niedriges Fenster vertikal auf Zeilenhöhe gestreckt werden? Mit **-pady** bleibt der gewünschte Abstand erhalten.

**-window *pathName***
Name des anzuzeigenden Fensters. Späterer leerer **-window**-Wert löscht das Widget nicht, sondern hebt nur die Verknüpfung auf. Bei mehreren Peers ist **-create** meist einfacher.

## Eingebettete Bilder

Die vierte Annotationsform sind eingebettete Bilder. Jedes Bild wird an einer Textstelle angezeigt. Beliebig viele sind möglich; dasselbe Bild kann mehrfach eingebettet sein.

Die Position wird bei Textänderung und Scrollen aktualisiert. Jedes Bild belegt eine Index-Einheit und ist über Indexposition oder den bei *pathName* **image create** vergebenen Namen ansprechbar. Wird der Textbereich gelöscht, verschwindet diese Bildkopie vom Bildschirm.

Ausblenden unmittelbar nach Planung via *pathName* **image create** *index* **-create** verhindert die Erstellung. Einblenden löst das Erstellungsskript aus. Nach Zerstörung kein automatisches Neuerstellen.

Bei *pathName* **image create** wird ein für diese Instanz eindeutiger Name zurückgegeben ( **-name**, siehe unten; sonst **-image**). Ist *imageName* bereits vergeben, wird **#***nn* angehängt (*nn* beliebig). Der Name bleibt nach Zuweisung unverändert, auch wenn **-image** oder **-name** per *pathName* **image configure** geändert werden.

Optionen können bei der Erstellung gesetzt und mit *pathName* **image configure** geändert werden:

**-align *where***
Ist das Bild niedriger als die Zeile: **top**, **center**, **bottom** oder **baseline** (wie bei eingebetteten Fenstern).

**-image *image***
Name des Tk-Bildes. Ist *image* ungültig, Fehler.

**-name *ImageName***
Referenzname im Text-Widget. Fehlt er, gilt der Tk-Bildname. Bei Kollision #*nn* wie oben.

**-padx *pixels***
Zusätzlicher Abstand links und rechts.

**-pady *pixels***
Zusätzlicher Abstand oben und unten.

## Die Auswahl

Die Auswahl wird über Tags realisiert. Ist **-exportselection** true, wird der **sel**-Tag mit der Auswahl verknüpft:

1. Werden Zeichen mit **sel** markiert, beansprucht das Text-Widget die Auswahl.
2. Abfragen der Auswahl liefert alle Zeichen mit **sel**.
3. Wird die Auswahl von einer anderen Anwendung oder einem anderen Fenster übernommen, wird **sel** von allen Zeichen entfernt.
4. Bei jeder Änderung des **sel**-Bereichs wird **<<Selection>>** erzeugt, ggf. auch wenn die Auswahl betroffen, aber nicht geändert ist. Mehrere Änderungen vor der Ereignisverarbeitung können mehrere Ereignisse mit gleicher sichtbarer Auswahl ergeben.

**sel** wird beim Erstellen automatisch definiert und darf nicht mit "*pathName* **tag delete**" gelöscht werden. **-selectbackground**, **-selectborderwidth** und **-selectforeground** des Widgets sind an **-background**, **-borderwidth** und **-foreground** von **sel** gekoppelt;

Änderungen spiegeln sich gegenseitig. Ohne Fokus gilt **-inactiveselectbackground** statt **-selectbackground**. So lässt sich **sel** für Vorder- und Hintergrundfenster steuern oder bei leerem **-inactiveselectbackground** ausblenden. Jedes Peer-Widget hat ein eigenes **sel**.

## Der Einfügecursor

Die Marke **insert** ist besonders. Sie wird automatisch erzeugt und darf nicht mit "*pathName* **mark unset**" entfernt werden. **insert** ist die Einfügeposition; der Cursor wird dort gezeichnet, wenn das Widget den Eingabefokus hat.

## Die Modified-Flagge

Das Widget kann Änderungen über die modified-Flagge verfolgen. Einfügen oder Löschen setzt sie. Abfrage, Setzen und Löschen sind programmatisch möglich. Bei Zustandsänderung wird **<<Modified>>** erzeugt. Siehe *pathName* **edit modified**.

## Der Rückgängig-Mechanismus

Bei **-undo** true zeichnet das Widget jede Einfüge- und Löschaktion auf einem Stapel auf (unbegrenzt).

Trennzeichen ("separators") gruppieren Aktionen zu zusammengesetzten Bearbeitungen. Rückgängig macht alles zwischen zwei Trennzeichen; die Aktionen wandern auf den Wiederholen-Stapel. Neuerungen auf dem Rückgängig-Stapel leeren den Wiederholen-Stapel. Beide Stapel können geleert werden.

Bei **-autoseparators** true werden Trennzeichen automatisch eingefügt; auch manuell möglich. Steht oben bereits eines, wird kein zweites eingefügt — zwei Trennzeichen sind stets durch mindestens eine Einfüge- oder Löschaktion getrennt.

**<<UndoStack>>** wird erzeugt, wenn Rückgängig- oder Wiederholen-Stapel leer oder nicht mehr leer wird.

Der Mechanismus ist mit der modified-Flagge gekoppelt: Rückgängig/Wiederholen kann zum unveränderten Zustand führen. Die automatische Kopplung gilt nicht, wenn der Benutzer die Flagge gesetzt hat, bis sie zurückgesetzt wurde.

Siehe *pathName* **edit** unten.

## Peer-Widgets

Das Text-Widget speichert Zeileninhalt, Marken, Tags, Bilder, Fenster und den Rückgängig-Stapel separat.

Der Speicher ist nicht direkt zugreifbar (nur über ein Text-Widget), aber mehrere Widgets können dieselben Daten unterschiedlich darstellen — Peer-Text-Widgets.

Änderungen (Text, Farben, Bilder, Marken, Tags) in einem Widget spiegeln sich in allen Peers.

Daten und Markup werden geteilt, mit Ausnahmen: **sel** pro Peer unterschiedlich setzbar/konfigurierbar; **insert** und **current** pro Peer, andere Marken geteilt; eingebettete Fenster nicht teilbar — **-window** pro Peer (**-create** empfohlen); Konfiguration (z. B. **-font**) pro Peer, außer **-undo**, **-maxundo**, **-autoseparators** (Rückgängig/Wiederholen und modified geteilt).

Ein Peer muss nicht alle Zeilen enthalten; beim Erstellen kann ein Zeilenbereich (z. B. 52–125) angegeben werden. Der Bereich wächst/schrumpft mit Einfügen/Löschen. Nur ganze Zeilen werden angezeigt. Schrumpft der Inhalt auf nichts, sind keine neuen Zeilen einfügbar — leere Hülle ohne Inhalt (ohne manuelle **-startline**/**-endline**-Anpassung). Indizes laufen dennoch von "1.0" bis "end", beziehen sich aber auf die Teilmenge; *peerName* **index end** und *peerName* **tag ranges** können zwischen Peers abweichen. **-startline** und **-endline** steuern den Umfang.

Peers sind gleichberechtigt: Löschen des "ursprünglichen" Widgets betrifft andere Peers nicht.

Siehe *pathName* **peer** unten.

## Asynchrone Aktualisierung der Zeilenhöhen

Für Reaktionsfähigkeit berechnet das Widget Zeilenmetriken (Höhen in Pixeln) asynchron. Manche Befehle liefern daher falsche Ergebnisse, wenn die Berechnung noch läuft — u. a. *pathName* **count -ypixels** und *pathName* **yview**.

Aus Performance-Gründen warten diese Befehle nicht immer auf Abschluss; in vielen Fällen genügt eine ungefähre Antwort.

Bei Bedarf an Genauigkeit (und Geometrie-Verwaltung): *pathName* **sync** und *pathName* **pendingsync**.

**<<WidgetViewSync>>** tritt auf, wenn Zeilenhöhen veraltet sind, und erneut bei Synchronität. Das Detail-Feld (%d) ist true (synchron) oder false.

*pathName* **sync**, *pathName* **pendingsync** und **<<WidgetViewSync>>** gelten pro Widget unabhängig von Peers.

Beispiele:

```
## Example 1:
# immediately complete line metrics at any cost (GUI unresponsive)
$w sync
$w yview moveto $fraction

## Example 2:
# synchronously wait for up-to-date line metrics (GUI responsive)
# before executing the scheduled command, but don't block execution flow
$w sync -command [list $w yview moveto $fraction]

## Example 3:
# init
set yud($w) 0
proc updateaction w {
set ::yud($w) 1
# any other update action here...
}
# runtime, synchronously wait for up-to-date line metrics (GUI responsive)
$w sync -command [list updateaction $w]
vwait yud($w)
$w yview moveto $fraction
```

```
## Example 4:
# init
set todo($w) {}
proc updateaction w {
foreach cmd $::todo($w) {uplevel #0 $cmd}
set todo($w) {}
}
# runtime
lappend todo($w) [list $w yview moveto $fraction]
$w sync -command [list updateaction $w]

## Example 5:
# init
set todo($w) {}
bind $w <<WidgetViewSync>> {
if {%d} {
foreach cmd $todo(%W) {eval $cmd}
set todo(%W) {}
}
}
# runtime
if {![$w pendingsync]} {
$w yview moveto $fraction
} else {
lappend todo($w) [list $w yview moveto $fraction]
}
```

## Widget-Befehl

Der Befehl **text** erstellt einen neuen Tcl-Befehl mit demselben Namen wie der Pfad des Textfensters. Er ruft verschiedene Widget-Operationen auf. Allgemeine Form:

```
pathName option ?arg ...?
```

*PathName* ist der Befehlsname, identisch mit dem Widget-Pfad. *Option* und die *arg*s bestimmen das Verhalten. Für Text-Widgets sind folgende Befehle möglich:

***pathName* bbox *index***

Gibt eine Liste mit vier Elementen zurück, die den Bildschirmbereich des Zeichens bei *index* beschreiben: x- und y-Koordinate der oberen linken Ecke sowie Breite und Höhe. Ist das Zeichen nur teilweise sichtbar, gilt nur der sichtbare Teil. Ist es unsichtbar, leere Liste.

***pathName* cget *option***

Aktueller Wert der Konfigurationsoption *option* (alle vom Befehl **text** akzeptierten Werte).

***pathName* compare *index1 op index2***

Vergleicht *index1* und *index2* mit dem Relationoperator *op*; 1 wenn erfüllt, sonst 0. *Op* ist <, <=, ==, >=, > oder!=. Bei == gleiches Zeichen; bei < steht *index1* vor *index2* usw.

***pathName* configure ?*option*? ?*value option value* ...?**

Abfrage oder Änderung der Konfiguration. Ohne *option* Liste aller Optionen (Tk_ConfigureInfo

). *option* ohne *value*: Beschreibung dieser Option. *option-value*-Paare: setzen und leeren String zurück. *Option* wie beim Befehl **text**.

***pathName* count ?*options*? *index1 index2***

Zählt relevante Einheiten zwischen zwei Indizes. Liegt *index1* nach *index2*, negatives Ergebnis (für jede Option). Die gezählten Einheiten hängen von den Optionen ab. Ergebnis: Liste von Ganzzahlen, je eine pro Zähloption. Gültig: **-chars**, **-displaychars**, **-displayindices**, **-displaylines**, **-indices**, **-lines**, **-xpixels**, **-ypixels**. Standard: **-indices**. Zusätzlich **-update** als Modifier: bei gesetztem Wert (und Geometrie-Verwaltung) werden veraltete Daten neu berechnet — derzeit vor allem für **-ypixels** (ohne **-update** Cache pro Zeile). **-update** ist veraltet zugunsten von *pathName* **sync**, *pathName* **pendingsync** und **<<WidgetViewSync>>**. Interpretation:

**-chars**

alle Zeichen, ausgeblendet oder nicht; keine eingebetteten Fenster/Bilder.

**-displaychars**

alle nicht ausgeblendeten Zeichen.

**-displayindices**

alle nicht ausgeblendeten Zeichen, Fenster und Bilder.

**-displaylines**

Anzeigezeilen (je Umbruch +1) von der Zeile des ersten Index bis vor die Anzeigezeile des zweiten; auf derselben Anzeigezeile: 0. Nur sichtbare Abschnitte.

**-indices**

Zeichen und eingebettete Fenster/Bilder (alles in Index-Raum), ausgeblendet oder nicht.

**-lines**

logische Zeilen (ohne Umbruch) von der Zeile des ersten Index bis vor die des zweiten; auf derselben Zeile: 0; auch ausgeblendete Zeilen.

**-xpixels**

horizontale Pixel vom ersten Pixel des ersten Index bis vor den des zweiten. Gesamtbreite ohne Umbruch: längste Zeile finden, dann ".text count -xpixels "${line}.0" "${line}.0 lineend"".

**-ypixels**

vertikale Pixel analog; auf derselben Anzeigezeile: 0. Gesamthöhe: ".text count -ypixels 1.0 end"; aktuell: ".text count -update -ypixels 1.0 end".

Rückgabe: positive oder negative Ganzzahl pro Option; mehrere Optionen → Liste. ".text count -xpixels -ypixels 1.3 4.5" ist gültig und liefert zwei Elemente.

***pathName* debug ?*boolean*?**

Ist *boolean* gesetzt, muss es ein von Tcl_GetBoolean akzeptierter Wahrheitswert sein. Bei true werden interne Konsistenzprüfungen im B-Baum-Code aktiviert, bei false deaktiviert. Rückgabe: leerer String. Ohne *boolean*: **on** oder **off**. Ein Schalter gilt für alle Text-Widgets. Bei großen Texten kann es spürbar langsamer werden.

Bei aktiviertem Debugging setzen Zeichenroutinen **tk_textRedraw** und **tk_textRelayout** auf die Listen neu gezeichneter Indizes (Tk-Testsuite).

***pathName* delete *index1* ?*index2* ...?**

Löscht einen Zeichenbereich. Mit *index1* und *index2*: von *index1* bis vor *index2* (Zeichen bei *index2* bleibt). Liegt *index2* nicht nach *index1*, nichts gelöscht. Ohne *index2*: ein Zeichen bei *index1*. Löschungen, die den abschließenden Zeilenumbruch entfernen würden, werden angepasst; beim Löschen bis Textende auch der Zeilenumbruch davor. Leerer String. Mehrere Indexpaare: mehrere Bereiche; zuerst Gültigkeitsprüfung, sortiert, Löschung von hinten nach vorne gegen Indexverschiebung. Gleicher Start: längster Bereich. Überlappung: Zusammenführung ohne Löschen außerhalb der Bereiche.

***pathName* dlineinfo *index***

Fünf Elemente für die Anzeigezeile mit *index*: x, y, Breite, Höhe, Baseline-Position (Pixel). Bei **none** und überstehender Zeile: gesamte Zeile inkl. unsichtbarer Teile. Kürzere Zeile: nur belegter Teil. Unsichtbare Anzeigezeile: leere Liste.

***pathName* dump ?*switches*? *index1* ?*index2*?**

Inhalt von *index1* bis vor *index2* inkl. Text sowie Marken, Tags und eingebettete Fenster. Ohne *index2*: ein Zeichen nach *index1*. Format:

*key1 value1 index1 key2 value2 index2* ...

Mögliche *key*: **text**, **mark**, **tagon**, **tagoff**, **image**, **window**; *value* entsprechend; *index* Startposition. Schalter:

**-all**

alles (Standard).

**-command *command***

statt Rückgabe *command* pro Element mit angehängten *key*, *value*, *index*.

**-image**

Bilder einbeziehen.

**-mark**

Marken einbeziehen.

**-tag**

Tag-Übergänge als **tagon**/**tagoff**.

**-text**

Text bis zum nächsten Element oder *index2*; kein Textelement über Zeilenumbrüche; mehrzeiliger Block ohne Marken/Tags als Segmente mit Zeilenumbruch im Wert.

**-window**

Fenster; Wert ist Tk-Pfadname, oder leer wenn noch nicht erstellt (**-create**); dann Abfrage per Index.

***pathName* edit *option* ?*arg* ...?**

Steuert Rückgängig-Mechanismus und modified-Flagge. Verhalten hängt von *option* ab:

***pathName* edit canredo**

true wenn Wiederholen möglich (Wiederholen-Stapel nicht leer), sonst false.

***pathName* edit canundo**

true wenn Rückgängig möglich (Rückgängig-Stapel nicht leer), sonst false.

***pathName* edit modified ?*boolean*?**

Ohne *boolean*: modified-Flagge. insert, delete, edit undo/redo oder der Benutzer setzen/löschen sie. Mit *boolean*: setzen auf *boolean*.

***pathName* edit redo**

Bei **-undo** true: letzte rückgängig gemachte Bearbeitungen erneut anwenden (wenn seitdem nichts Neues), Liste geänderter Indizes. Fehler bei leerem Stapel. Bei **-undo** false: nichts.

***pathName* edit reset**

Leert Rückgängig- und Wiederholen-Stapel.

***pathName* edit separator**

Trennzeichen auf Rückgängig-Stapel. Bei **-undo** false: nichts.

***pathName* edit undo**

Bei **-undo** true: letzte Bearbeitung rückgängig (alles zwischen zwei Trennzeichen), Liste geänderter Indizes. Fehler bei leerem Stapel. Bei **-undo** false: nichts.

***pathName* get ?-displaychars? ?--? *index1* ?*index2* ...?**

Zeichenbereich aus dem Text. Von *index1* bis vor *index2* (Zeichen bei *index2* nicht enthalten). Ohne *index2*: ein Zeichen. Leerer Bereich → leerer String. Eingebettete Fenster nicht im String. Mehrere Paare → Liste in übergebener Reihenfolge; ungültige Bereiche fehlen in der Liste. Mit **-displaychars** nur nicht ausgeblendete Zeichen — Bereiche können leer sein.

***pathName* image *option* ?*arg* ...?**

Eingebettete Bilder. Verhalten hängt von *option* ab:

***pathName* image cget *index option***

Konfigurationswert; *index* identifiziert das Bild; *option* aus Eingebettete Bilder.

***pathName* image configure *index* ?*option value* ...?**

Abfrage/Änderung wie **configure**; Optionen siehe Eingebettete Bilder.

***pathName* image create *index* ?*option value* ...?**

Neues Bild bei *index*; eindeutiger Bezeichner als Index; siehe Eingebettete Bilder.

***pathName* image names**

Liste aller eingebetteten Bildinstanzen in *window*.

***pathName* index *index***

Position als *line.char*; *index* gemäß Indizes.

***pathName* insert *index chars* ?*tagList chars tagList* ...?**

Fügt *chars* vor *index* ein. Bei Textende vor dem letzten Zeilenumbruch. Ein *chars* ohne *tagList*: Tags, die vor und nach der Einfügestelle gemeinsam sind; nur auf einer Seite → nicht auf neuen Text. Mit *tagList*: nur diese Tags. Mehrere Paare wie mehrfaches **insert**; letztes *tagList* optional.

***pathName* mark *option* ?*arg* ...?**

Marken bearbeiten:

***pathName* mark gravity *markName* ?*direction*?**

Ohne *direction*: **left** oder **right**. Mit *direction* (**left**/**right**): Schwerkraft setzen.

***pathName* mark names**

Liste aller gesetzten Marken.

***pathName* mark next *index***

Name der nächsten Marke ab *index* (numerisch ab dieser Position; bei Markenname direkt danach; mehrere Marken am gleichen Index möglich). Reihenfolge wie **dump**. Marke auf **end** wirkt *nach* **end**. Keine Marke danach: leerer String.

***pathName* mark previous *index***

Name der vorherigen Marke; numerisch ab Zeichen davor; bei Markenname davor.

Umgekehrte **dump**-Reihenfolge. Keine Marke davor: leerer String.

***pathName* mark set *markName index***

*markName* vor *index* setzen oder verschieben. Leerer String.

***pathName* mark unset *markName* ?*markName markName* ...?**

Marken entfernen; nicht mehr in Indizes oder "*pathName* **mark names**". Leerer String.

***pathName* peer *option args***

Peer-Widgets:

***pathName* peer create *newPathName* ?*options*?**

Peer mit *newPathName* und optionalen Standardoptionen (wie **text**). Standard: gleiche Start-/Endzeile wie Eltern-Widget.

***pathName* peer names**

Liste der Peers (ohne sich selbst); Reihenfolge undefiniert.

***pathName* pendingsync**

1 wenn Zeilenhöhen nicht aktuell, sonst 0.

***pathName* replace *index1 index2 chars* ?*tagList chars tagList* ...?**

Ersetzt Bereich *index1*–*index2* durch *chars* und Tags; *tagList* wie bei **insert**, Indizes wie bei **delete**. Liegt *index2* vor *index1*, Fehler.

Löschen und Einfügen ohne unnötiges Scrollen oder Cursor-Verschieben; Rückgängig-Stapel korrekt bei aktivem **-undo**. Leerer String.

***pathName* scan *option args***

Scanning:

***pathName* scan mark *x y***

Merkt *x*, *y* und aktuelle Ansicht für **scan dragto**. Leerer String.

***pathName* scan dragto *x y***

Verschiebt die Ansicht um das Zehnfache der Differenz zu **scan mark**. Leerer String.

***pathName* search ?*switches*? *pattern index* ?*stopIndex*?**

Sucht ab *index* nach *pattern*. Treffer: Index des ersten Zeichens; sonst leerer String. Schalter:

**-forwards**

vorwärts ab *index* (Standard).

**-backwards**

rückwärts; erstes Zeichen des Treffers vor *index* (nicht bei *index*). Rückwärtssuche oft langsamer (**-regexp**); bei Performance vorwärts suchen.

**-exact**

exakter Vergleich (Standard).

**-regexp**

*pattern* als regulärer Ausdruck (regexp, re_syntax). Standard übergibt **-lineanchor** und **-linestop** (außer **-nolinestop**): *^$* Zeilenanfang/-ende;. und *[^* nicht *\n*.

**-nolinestop**

. und *[^* dürfen *\n* treffen; nur mit **-regexp**, sonst Fehler. Gesamter Text: "*pathName* **search -nolinestop -regexp** ".*" 1.0".

**-nocase**

Groß-/Kleinschreibung ignorieren.

**-count *varName***

Variable für Anzahl Indexpositionen im Treffer; ohne Bilder/Fenster (und ohne **-elide** auch ohne ausgeblendete Zeichen) = Zeichenanzahl. *matchIdx* bis *matchIdx + $count chars* liefert den Treffer.

**-all**

alle Treffer als Indexliste; mit **-count** Liste in *varName*. Wie regexp -all: überlappende Treffer normalerweise nicht. "\w+" in "hello there": zweimal; "Z[a-z]+Z" in "ZooZooZoo": einmal.

**-overlap**

bei **-all**: auch Treffer, die einen bereits gefundenen überlappen, wenn sie nicht vollständig eingeschlossen sind. Nur mit **-all**, sonst Fehler.

**-strictlimits**

gesamter Treffer muss innerhalb Start/Stopp liegen (nicht nur Trefferanfang).

**-elide**

auch ausgeblendeten Text suchen.

**--**

beendet Schalterliste; nächstes Argument ist *pattern* auch bei führendem **-**.

Treffer kann eine oder mehrere Zeilen umfassen. Mit *stopIndex*: Suche endet dort (vorwärts: nicht ab *stopIndex*; rückwärts: nicht vor *stopIndex*). Ohne *stopIndex*: Wrap-around am Textende. **-forwards** mit *stopIndex* vor Start: kein Treffer. Siehe Bekannte Fehler zu **search**.

***pathName* see *index***

Ansicht so anpassen, dass *index* vollständig sichtbar ist. Bereits sichtbar: nichts. Knapp außerhalb: minimal an Rand bringen. Weit außerhalb: zentrieren.

***pathName* sync ?*-command command*?**

Synchronisation der Ansicht:

***pathName* sync**

Zeilenmetriken sofort aktualisieren; sofort zurück wenn nichts veraltet, sonst nach Berechnung. Leerer String.

***pathName* sync -command *command***

*command* einmal nach aktuellen Zeilenhöhen ausführen; bei keiner ausstehenden Berechnung sofort. Leerer String. Bei Fehler bgerror.

***pathName* tag *option* ?*arg* ...?**

Tags bearbeiten:

***pathName* tag add *tagName index1* ?*index2 index1 index2* ...?**

Verknüpft *tagName* mit Zeichen von *index1* bis vor *index2*. Mehrere Paare möglich. Ohne letztes *index2*: ein Zeichen. Leerer Bereich: keine Wirkung.

***pathName* tag bind *tagName* ?*sequence*? ?*script*?**

Ordnet *script* dem Tag zu; bei *sequence* auf markierten Zeichen wird es ausgeführt. Ähnlich **bind**, aber für Zeichen. Siehe **bind** zu *sequence* und Substitutionen. Alle Argumente: neues Binding (ersetzt gleiches *sequence*/*tagName*; führendes "+" in *script* ergänzt). Ohne *script*: zugehöriges *script*. Ohne beides: Liste aller Sequenzen.

Bindings nur für Maus/Tastatur (**Enter**, **Leave**, **Button**, **Motion**, **Key**) oder virtuelle Ereignisse. Maus: Marke **current**; Tastatur: **insert** (siehe Marken). **Enter**/**Leave** beim Betreten/Verlassen des Tags am aktuellen Zeichen (nicht wie Fenster-Enter/Leave).

Mehrere Tags: ein Binding pro Tag, niedrigste zu höchster Priorität; bei einem Tag die spezifischste Variante (**bind**). continue/break wie bei **bind**.

Widget-weite **bind**-Bindings ergänzen Tag-Bindings (Tags zuerst).

***pathName* tag cget *tagName option***

Aktueller Optionswert für *tagName*.

***pathName* tag configure *tagName* ?*option*? ?*value*? ?*option value* ...?**

Wie **configure**, aber für Tag-Optionen; siehe Tags.

***pathName* tag delete *tagName* ?*tagName* ...?**

Löscht Tag-Informationen inkl. Bindings und Anzeige. Leerer String.

***pathName* tag lower *tagName* ?*belowThis*?**

Priorität knapp unter *belowThis*; ohne *belowThis*: niedrigste Priorität.

***pathName* tag names ?*index*?**

Tags an *index* oder alle existierenden (auch ohne markierte Zeichen), sortiert nach Priorität.

***pathName* tag nextrange *tagName index1* ?*index2*?**

Nächster *tagName*-Bereich ab *index1* bis vor *index2*; zwei Indizes oder leer. Ohne *index2*: bis Textende.

***pathName* tag prevrange *tagName index1* ?*index2*?**

Vorheriger Bereich vor *index1*, nicht vor *index2*. Ohne *index2*: ab Textanfang.

***pathName* tag raise *tagName* ?*aboveThis*?**

Priorität knapp über *aboveThis*; ohne *aboveThis*: höchste Priorität.

***pathName* tag ranges *tagName***

Alle Bereiche als Indexpaare; ohne markierte Zeichen: leerer String.

***pathName* tag remove *tagName index1* ?*index2 index1 index2* ...?**

Entfernt *tagName* von *index1* bis vor *index2*; wie **tag add**. Leerer String.

***pathName* window *option* ?*arg* ...?**

Eingebettete Fenster:

***pathName* window cget *index option***

Konfigurationswert; Optionen aus Eingebettete Fenster.

***pathName* window configure *index* ?*option value* ...?**

Abfrage/Änderung; siehe Eingebettete Fenster.

***pathName* window create *index* ?*option value* ...?**

Neues Fenster bei *index*; siehe Eingebettete Fenster. Leerer String.

***pathName* window names**

Liste aller eingebetteten Fenster in *window*.

***pathName* xview *option args***

Horizontale Position des Textes:

***pathName* xview**

Liste mit zwei Brüchen 0–1: sichtbarer horizontaler Anteil. Beispiel .2 und .6: 20 % links außerhalb, 40 % sichtbar, 40 % rechts. Nur tatsächlich sichtbare Zeilen; kurze sichtbare Zeilen → 0 und 1. Werte für **-xscrollcommand**.

***pathName* xview moveto *fraction***

*fraction* des horizontalen Umfangs links außerhalb (0–1).

***pathName* xview scroll *number what***

Verschiebt die Ansicht; *what* ist **pages**, **pixels** oder **units**. Bei **pages**/**units** ganzzahliges *number*, sonst Tk_GetPixels (gerundet; ohne Einheit Pixel). **pages**: Bildschirmseiten; **pixels** : Pixel; **units**: durchschnittlich breite Zeichen. Negatives *number*: weiter links sichtbar; positiv: weiter rechts.

***pathName* yview ?*args*?**

Vertikale Position:

***pathName* yview**

Zwei Brüche 0–1: oberes sichtbares Pixel der obersten Zeile und Pixel nach dem letzten sichtbaren der untersten Zeile, relativ zum Gesamttext. Für **-yscrollcommand**.

***pathName* yview moveto *fraction***

Pixel bei *fraction* an den oberen Rand der obersten Zeile (0 = Textanfang; nahe 1 wird auf maximalen Scroll begrenzt).

***pathName* yview scroll *number what***

Verschiebt vertikal; *what* **pages**, **pixels** oder **units**; bei **units**/**pages** ganzzahlig, sonst Tk_GetPixels. **units**: Zeilen; **pages**: Bildschirmseiten; **pixels**: Pixel. Negativ: frühere Positionen; positiv: spätere.

***pathName* yview ?-pickplace? *index***

*index* sichtbar machen. Ohne **-pickplace**: oben im Fenster. Mit **-pickplace**:

1. Bereits sichtbar: nichts.
2. Knapp oberhalb: oben positionieren.
3. Knapp unterhalb: unten positionieren.
4. Sonst zentrieren.

**-pickplace** ist veraltet; *pathName* **see** übernimmt x- und y-Bewegung (**-pickplace** nur y).

***pathName* yview *number***

Erstes Zeichen der Zeile nach *number* oben sichtbar; *number* ganzzahlig. Veraltet.

## Bindings

Tk erstellt automatisch Klassen-Bindings für Text-Widgets mit folgendem Standardverhalten. "Wort" hängt von **tcl_wordchars** ab; siehe tclvars(n).

1. Maustaste 1 setzt den Einfügecursor an die nächste Kante des Zeichens unter dem Cursor, setzt den Eingabefokus und löscht die Auswahl. Ziehen mit Taste 1 erweitert die Auswahl zwischen Einfügecursor und Zeichen unter der Maus.
2. Doppelklick mit Taste 1 wählt das Wort unter der Maus und setzt den Cursor an den Wortanfang. Ziehen danach erweitert die Auswahl in ganzen Wörtern.
3. Dreifachklick mit Taste 1 wählt die Zeile und setzt den Cursor an den Zeilenanfang. Ziehen danach erweitert die Auswahl in ganzen Zeilen.
4. Mit gedrückter Umschalttaste und Taste 1 lässt sich das Auswahlende anpassen (das Ende,

das der Maus beim Drücken am nächsten war). Nach Doppelklick: in Wörtern; nach Dreifachklick: in Zeilen.

5. Taste 1 mit Strg verschiebt den Einfügecursor ohne die Auswahl zu ändern.
6. Normale druckbare Zeichen werden am Einfügecursor eingefügt.
7. Die Ansicht lässt sich mit der mittleren Maustaste ziehen (Taste 2, unter TkAqua Taste 3). Klick ohne Bewegung kopiert die Auswahl an die Mausposition. Die Einfüge-Taste fügt die Auswahl am Einfügecursor ein.
8. Wird die Maus mit gedrückter Taste 1 aus dem Widget gezogen, scrollt es automatisch, wenn dort noch Text außerhalb des Fensters liegt.
9. Links/Rechts bewegen den Cursor um ein Zeichen und löschen die Auswahl. Mit Umschalt: Cursor bewegen und Auswahl erweitern. Strg-Links/Rechts: wortweise; Strg-Umschalt-Links/Rechts: wortweise mit Auswahl. Strg-b/Strg-f wie Links/Rechts. Meta-b/Meta-f wie Strg-Links/Rechts.
10. Hoch/Runter bewegen den Cursor um eine Zeile und löschen die Auswahl. Mit Umschalt (im Original "Up or Right"; gemeint: Up oder Down): Auswahl erweitern. Strg-Hoch/Runter: Absätze (durch Leerzeilen getrennt); Strg-Umschalt-Hoch/Runter: Absätze mit Auswahl. Strg-p/Strg-n wie Hoch/Runter.
11. Bild-ab/Bild-auf bewegen den Cursor um einen Bildschirm und löschen die Auswahl. Mit Umschalt: Auswahl erweitern.
12. Strg-Bild-ab/Bild-auf scrollen die Ansicht um eine Seite nach rechts/links, ohne Cursor oder Auswahl zu ändern.
13. Pos1 und Strg-a: Anfang der Anzeigezeile, Auswahl löschen. Umschalt-Pos1: Anfang der Anzeigezeile mit Auswahl.
14. Ende und Strg-e: Ende der Anzeigezeile, Auswahl löschen. Umschalt-Ende: Ende der Anzeigezeile mit Auswahl.
15. Strg-Pos1 und Meta-<: Textanfang, Auswahl löschen. Strg-Umschalt-Pos1: Textanfang mit Auswahl.
16. Strg-Ende und Meta->: Textende, Auswahl löschen. Strg-Umschalt-Ende: Textende mit Auswahl.
17. Select und Strg-Leerzeichen setzen den Auswahlanker auf den Einfügecursor, ohne die aktuelle Auswahl zu ändern. Umschalt-Select und Strg-Umschalt-Leerzeichen passen die Auswahl vom Anker zum Cursor an.
18. Strg-/ wählt den gesamten Inhalt.
19. Strg-\ löscht die Auswahl.
20. F16 (auf vielen Sun-Workstations "Copy") oder Meta-w kopiert die Auswahl (**tk_textCopy**), falls vorhanden.
21. F20 ("Cut") oder Strg-w kopiert und löscht die Auswahl (**tk_textCut**). Ohne Auswahl keine Wirkung.
22. F18 ("Paste") oder Strg-y fügt die Zwischenablage am Einfügecursor ein (**tk_textPaste**).
23. Entf löscht die Auswahl oder, ohne Auswahl, das Zeichen rechts vom Cursor.
24. Rücktaste und Strg-h löschen die Auswahl oder das Zeichen links vom Cursor.
25. Strg-d löscht das Zeichen rechts vom Cursor.
26. Meta-d löscht das Wort rechts vom Cursor.
27. Strg-k löscht vom Cursor bis Zeilenende; am Zeilenende den Zeilenumbruch.
28. Strg-o fügt einen Zeilenumbruch vor dem Cursor ein, ohne den Cursor zu verschieben.
29. Meta-Rücktaste und Meta-Entf löschen das Wort links vom Cursor.

30. Strg-t vertauscht die beiden Zeichen rechts vom Cursor.
31. Strg-x kopiert die Auswahl in die Zwischenablage und löscht sie.
32. Strg-z macht die letzte Bearbeitung rückgängig, wenn **-undo** true ist.
33. Strg-Z (unter Windows Strg-y) stellt die letzte rückgängig gemachte Bearbeitung wieder her, wenn **-undo** true ist.

Ist das Widget mit **-state** deaktiviert, lässt sich die Ansicht noch anpassen und Text auswählen, aber es gibt keinen Einfügecursor und keine Textänderungen.

Das Verhalten lässt sich durch neue Widget-Bindings oder neu definierte Klassen-Bindings ändern.

## Bekannte Probleme

### Probleme bezüglich chars und Indizes

Vor Tk 8.5 bezog sich "chars" auf Indexpositionen (Zeichen, eingebettete Fenster und Bilder). Ab Tk 8.5 werden "chars" und "indices" getrennt behandelt. Aus Kompatibilität beziehen "+N chars" und "-N chars" weiterhin Indizes ein; für echte Zeichen "+N any chars", "-N any chars" usw. verwenden. In einer künftigen Version könnte "+N chars" synonym zu "+N any chars" werden.

### Performance-Probleme

Text-Widgets sollten unter verschiedenen Bedingungen effizient laufen. Etwa 2–3 Byte Hauptspeicher pro Textbyte; Megabyte-Texte sind auf den meisten Workstations machbar. Intern modifizierter B-Baum; Tags und Marken skalieren in vielen Fällen gut.

Problematisch können Hunderte oder Tausende Tags sein, deren erster und letzter Bereich nahe am Textanfang bzw. -ende liegen oder ein Bereich fast das ganze Widget abdeckt — Kosten proportional zu ähnlichen Tags. Viele Tags mit lokal verteilten Bereichen sind unkritisch.

Sehr lange Zeilen können teuer sein, besonders mit vielen Marken und Tags.

Die Anzeigezeile mit dem Einfügecursor wird bei jedem Blinken neu gezeichnet — setzen Sie **-insertofftime** auf 0, um das zu vermeiden.

### Bekannte Fehler

*pathName* **search -regexp** sucht seit Tk 8.5 zeilenweise und in kleinen Gruppen effizient über mehrere Zeilen; unter Umständen weicht das Ergebnis von einer regexp über den gesamten Text ab. Bei gierigen Mustern werden Zeilen ergänzt, solange Tcl einen längeren Treffer meldet (nicht immer korrekt) oder jede neue Zeile teilweise passt. Fehlt der Treffer in der ersten Zusatzzeile bei falschem Tcl-Code, obwohl die zweite passen würde, ist das Ergebnis falsch — selten, aber möglich, z. B.:

```
pack [text .t]
.t insert 1.0 "aaaa\nbbbb\ncccc\nbbbb\naaaa\n"
.t search -regexp -- {(a+|b+\nc+\nb+)+\na+} 1.0
```

findet keinen Treffer, obwohl ab dem ersten "b" ein 19-Zeichen-Treffer existiert.

Umschließt ein möglicher Treffer einen anderen vollständig, soll nur der größere zurückgegeben werden. Bei rückwärts regexp-Suche gelingt das nicht immer, wenn kurze nicht überlappende Treffer einem großen vorausgehen, der sie alle umfasst — der Algorithmus schaut nicht beliebig weit zurück. Beispiel:

```
pack [text .t]
.t insert 1.0 "aaaa\nbbbb\nbbbb\nbbbb\nbbbb\n"
.t search -regexp -backward -- {b+\n|a+\n(b+\n)+} end
```

trifft bei "5.0" zu, ein wirklich gieriger Treffer wäre "1.0". Mit **-all** trifft es "5.0", "4.0", "3.0" und "1.0" zu, korrekt wäre nur "1.0", da dieser alle anderen umschließt.

*Copyright (c) 1992 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# tk

*Stichwörter:* tk, application name, print, send, sysnotify, systray

## Name

tk - Internen Tk-Zustand manipulieren

## Übersicht

**tk** *option* ?*arg* ...?

## Beschreibung

Der Befehl **tk** bietet Zugriff auf verschiedene Elemente des internen Tk-Zustands. Die meisten durch diesen Befehl manipulierten Informationen betreffen die Anwendung als Ganzes oder einen Bildschirm bzw. ein Display, nicht ein bestimmtes Fenster. Je nach dem Argument *option* kann der Befehl verschiedene Formen annehmen. Die zulässigen Formen sind:

**tk appname ?*newName*?**

Wenn *newName* nicht angegeben ist, gibt dieser Befehl den Namen der Anwendung zurück (den Namen, der in send-Befehlen zur Kommunikation mit der Anwendung verwendet werden kann). Wenn *newName* angegeben ist, wird der Anwendungsname in *newName* geändert. Ist der angegebene Name bereits vergeben, wird ein Suffix der Form " **#2**" oder " **#3**" angehängt, um den Namen eindeutig zu machen. Das Ergebnis des Befehls ist der tatsächlich gewählte Name. *newName* sollte nicht mit einem Großbuchstaben beginnen. Das stört die Optionsverarbeitung, da Namen mit Großbuchstaben als Klassen gelten; dadurch kann Tk ggf. einige Optionen der Anwendung nicht finden. Wurden Sends durch Löschen des Befehls send deaktiviert, aktiviert dieser Befehl sie wieder und erstellt send neu.

**tk busy *subcommand* ...**

Dieser Befehl steuert die Markierung von Fensterhierarchien als "busy", wodurch sie während einer anderen Operation nicht interaktiv sind. Weitere Details siehe die Manpage busy.

**tk caret *window* ?-*x x*? ?-*y y*? ?-*height height*?**

Legt die Caret-Position für die Anzeige des angegebenen Tk-Fensters *window* fest und fragt sie ab. Der Caret ist die pro-Display-Cursorposition zur Anzeige des globalen Fokus (z. B. gemäß Microsoft-Barrierefreiheitsrichtlinien) sowie zur Position von Over-the-Spot-XIM- (X Input Methods) oder Windows-IME-Fenstern. Wenn keine Optionen angegeben sind, werden die zuletzt zum Setzen des Carets verwendeten Werte im Options-Wert-Paar-Format zurückgegeben. **-x** und **-y** sind fensterrelative Koordinaten; **-height** ist die Höhe der aktuellen Cursorposition oder die Höhe des angegebenen *window*, falls keine angegeben ist.

**tk inactive ?-*displayof window*? ?reset?**

Gibt eine positive Ganzzahl zurück: die Anzahl der Millisekunden seit der letzten

Benutzerinteraktion mit dem System. Wenn die Option **-displayof** angegeben ist, bezieht sich der Rückgabewert auf das Display von *window*; andernfalls auf das Display des Hauptfensters der Anwendung.

**tk inactive** gibt -1 zurück, wenn die Abfrage der Benutzer-Inaktivitätszeit vom System nicht unterstützt wird, sowie in sicheren Interpretern.

Wird die Literalzeichenkette **reset** als zusätzliches Argument übergeben, wird der Timer zurückgesetzt und eine leere Zeichenkette zurückgegeben. Das Zurücksetzen der Inaktivitätszeit ist in sicheren Interpretern verboten und löst bei einem Versuch einen Fehler aus.

**tk fontchooser *subcommand* ...**

Steuert den Tk-Schriftartauswahldialog. Weitere Details siehe die Manpage fontchooser.

**tk print *window***

Der Befehl **tk print** öffnet einen Dialog, mit dem Benutzer die Ausgabe von canvas- und text -Widgets drucken können. Der Druck erfolgt mit plattformnativen APIs und Dialogen, sofern verfügbar. Weitere Details siehe die Manpage print.

**tk scaling ?*-displayof window*? ?*number*?**

Legt den aktuellen Skalierungsfaktor fest und fragt ihn ab, mit dem Tk zwischen physikalischen Einheiten (z. B. Punkte, Zoll oder Millimeter) und Pixeln umrechnet. Das Argument *number* ist eine Gleitkommazahl und gibt die Anzahl der Pixel pro Punkt auf dem Display von *window* an. Wenn das Argument *window* weggelassen wird, gilt das Hauptfenster. Wenn *number* weggelassen wird, wird der aktuelle Wert des Skalierungsfaktors zurückgegeben.

Ein "Punkt" ist eine Maßeinheit von 1/72 Zoll. Ein Skalierungsfaktor von 1,0 entspricht 1 Pixel pro Punkt, was einem Standard-72-dpi-Monitor entspricht. Ein Skalierungsfaktor von 1,25 bedeutet 1,25 Pixel pro Punkt, die Einstellung für einen 90-dpi-Monitor; die Einstellung 1,25 auf einem 72-dpi-Monitor würde alles in der Anwendung 1,25-mal so groß wie normal darstellen. Der Anfangswert des Skalierungsfaktors wird beim Start der Anwendung anhand der Eigenschaften des installierten Monitors gesetzt, kann aber jederzeit geändert werden. Messungen nach einer Änderung des Skalierungsfaktors verwenden den neuen Faktor; ob bestehende Widgets sich dynamisch an den neuen Skalierungsfaktor anpassen, ist undefiniert.

**tk sysnotify *title? message?***

Der Befehl **tk sysnotify** erzeugt eine plattformspezifische Systembenachrichtigung. Ziel ist eine kurze, unaufdringliche Benachrichtigung durch ein Fenster, das kurz in einer Bildschirmecke erscheint. Weitere Details siehe die Manpage sysnotify.

**tk systray create *subcommand...***

Der Befehl **tk systray** legt ein Symbol in der plattformspezifischen Taskleiste an. Weitere Details siehe die Manpage systray.

**tk useinputmethods ?*-displayof window*? ?*boolean*?**

Legt fest und fragt ab, ob Tk XIM (X Input Methods) zum Filtern von Ereignissen verwenden soll. Der resultierende Zustand wird zurückgegeben. XIM wird in einigen Locales (z. B. Japanisch, Koreanisch) für spezielle Eingabegeräte verwendet. Diese Funktion ist nur unter X relevant. Wenn XIM-Unterstützung nicht verfügbar ist, wird immer 0 zurückgegeben. Wenn das Argument *window* weggelassen wird, gilt das Hauptfenster. Wenn *boolean* weggelassen wird, wird der aktuelle Zustand zurückgegeben. Für das Hauptdisplay ist dies standardmäßig eingeschaltet.

**tk windowingsystem**

Gibt das aktuelle Tk-Fenstersystem zurück, eines von **x11** (X11-basiert), **win32** (MS Windows) oder **aqua** (macOS Aqua).

*Copyright (c) 1992 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# tk_mac

*Stichwörter:* tk_mac, about dialog, antialiasing, Apple event, icon, NSImage

## Name

tk::mac - Mac-spezifische Funktionen unter macOS aus Tk nutzen

## Übersicht

**::tk::mac::DoScriptFile**
**::tk::mac::DoScriptText**
**::tk::mac::ShowPreferences**
**::tk::mac::OpenApplication**
**::tk::mac::ReopenApplication**
**::tk::mac::OpenDocument** *file...*
**::tk::mac::PrintDocument** *file...*
**::tk::mac::Quit**
**::tk::mac::OnHide**
**::tk::mac::OnShow**
**::tk::mac::ShowHelp**
**::tk::mac::PerformService**
**::tk::mac::LaunchURL** *URL...*
**::tk::mac::GetAppPath**

**::tk::mac::standardAboutPanel**

**::tk::mac::useCompatibilityMetrics** *boolean*
**::tk::mac::CGAntialiasLimit** *limit*
**::tk::mac::antialiasedtext** *number*
**::tk::mac::useThemedToplevel** *boolean*

**::tk::mac::iconBitmap** *name width height -kind value*

## Ereignisbehandler-Rückrufungen

Die Aqua/macOS-Anwendungsumgebung definiert zusätzliche Ereignisse, auf die Anwendungen reagieren sollten. Tk mappt diese Ereignisse auf Aufrufe von Befehlen im Namespace **::tk::mac**; sofern nicht anders angegeben, erfolgt keine Aktion, wenn der Befehl fehlt.

**::tk::mac::DoScriptFile**

Standard-Apple-Event-Handler für AEDoScriptHandler. Dieser Befehl führt eine Tcl-Datei aus, wenn ein AppleScript einen "do script"-Befehl an Wish mit einem Dateipfad als Parameter sendet.

**::tk::mac::DoScriptText**

Standard-Apple-Event-Handler für AEDoScriptHandler. Dieser Befehl führt Tcl-Code aus, wenn ein AppleScript einen "do script"-Befehl an Wish mit Tcl-Code oder einer Tcl-Prozedur als Parameter sendet.

**::tk::mac::ShowPreferences**

Standard-Apple-Event-Handler für kAEShowPreferences, "pref". Der Menüeintrag "Einstellungen" im Anwendungsmenü ist nur aktiv, wenn diese Prozedur definiert ist. Typischerweise umschließt der Befehl eine eigene Einstellungsfunktion, die ein Einstellungsfenster öffnet, etwa:

```
proc ::tk::mac::ShowPreferences {} {
    setPref
}
```

**::tk::mac::OpenApplication**

Ist eine Prozedur dieses Namens definiert, wird sie beim ersten Öffnen der Anwendung ausgeführt. Standard-Apple-Event-Handler für kAEOpenApplication, "oapp".

**::tk::mac::ReopenApplication**

Ist eine Prozedur dieses Namens definiert, ist sie der Standard-Apple-Event-Handler für kAEReopenApplication, "rapp", das Apple Event beim Öffnen einer bereits laufenden Anwendung (z. B. Klick auf das Dock-Symbol). Beispiel: minimiertes Fenster wiederherstellen beim Dock-Klick:

```
proc ::tk::mac::ReopenApplication {} {
    if {[wm state .] eq "withdrawn"} {
        wm state . normal
    } else {
        wm deiconify .
    }
    raise .
}
```

**::tk::mac::OpenDocument *file...***

Ist eine Prozedur dieses Namens definiert, ist sie der Standard-Apple-Event-Handler für kAEOpenDocuments, "odoc", wenn die Anwendung aufgefordert wird, ein oder mehrere Dokumente zu öffnen (z. B. per Drag & Drop oder durch Öffnen eines dem Programm zugeordneten Dokumenttyps). Die Prozedur soll Pfade der zu öffnenden Dateien als Argumente erhalten, z. B.:

```
proc ::tk::mac::OpenDocument {args} {
    foreach f $args {my_open_document $f}
}
```

**::tk::mac::PrintDocument *file...***

Ist eine Prozedur dieses Namens definiert, ist sie der Standard-Apple-Event-Handler für kAEPrintDocuments, "pdoc", wenn die Anwendung zum Drucken eines Dokuments aufgefordert wird. Sie erhält einen absoluten Dateipfad als Argument.

**::tk::mac::Quit**

Ist eine Prozedur dieses Namens definiert, ist sie der Standard-Apple-Event-Handler für kAEQuitApplication, "quit", z. B. über Beenden im Anwendungs- oder Dock-Menü oder beim Abmelden/Neustart/Herunterfahren. Ist sie nicht definiert, wird stattdessen exit aufgerufen.

**::tk::mac::OnHide**

Wenn definiert, wird sie bei kEventAppHidden aufgerufen, z. B. über "Ausblenden" im Anwendungs- oder Dock-Menü.

**::tk::mac::OnShow**

Wenn definiert, wird sie bei kEventAppShown aufgerufen, z. B. über "Alle einblenden" im Anwendungsmenü oder durch Klick auf das Dock-Symbol einer ausgeblendeten Anwendung.

**::tk::mac::ShowHelp**

Passt das Verhalten des Apple-Hilfe-Menüs an; ist die Prozedur nicht definiert, führt der plattformspezifische Hilfeeintrag "YourApp Help" die Standard-Cocoa-Aktion aus (Hilfebuch aus Info.plist oder Meldung, wenn keines gesetzt ist).

**::tk::mac::PerformService**

Führt eine aus dem macOS-Menü "Dienste" im Anwendungsmenü aufgerufene Tcl-Prozedur aus. "Dienste" ermöglichen Kommunikation zwischen Anwendungen; Daten (z. B. markierter Text) können an eine andere Anwendung gesendet werden, z. B. an Safari als Google-Suche oder an TextEdit zum Anhängen. Beispiel (in der Anwendung anpassen):

```
proc ::tk::mac::PerformService {} {
    set data [clipboard get]
    $w insert end $data
}
```

Die Daten werden über die Zwischenablage bezogen; es gibt keinen anderen unterstützten Weg. Sollen Dienste nicht genutzt werden, können die NSServices-Schlüssel aus der Info.plist entfernt werden. Der zugrunde liegende Code erlaubt text-, entry- und ttk::entry-Widgets den Zugriff auf Dienste anderer Anwendungen über das Dienste-Menü. Der NSPortName-Schlüssel in Wishs Info.plist ist derzeit "Wish"; ändert ein Entwickler den Namen der Wish-Ausführungsdatei, sollte dieser Schlüssel entsprechend angepasst werden.

**::tk::mac::LaunchURL *URL...***

Wenn definiert, startet eine URL innerhalb von Tk, z. B. wenn die Anwendung eine URL selbst verarbeiten soll (RSS-Anzeige) statt die Standardanwendung zu nutzen — kann aber auch so definiert werden. Wish enthält ein Platzhalter-URL-Schema "foo://" in CFBundleURLSchemes; das sollte für das gewünschte URL-Schema angepasst werden.

**::tk::mac::GetAppPath**

Gibt den Dateipfad der aktuellen Anwendung zurück.

## Zusätzliche Dialoge

Aqua/macOS definiert weitere Dialoge, die Anwendungen unterstützen sollten.

**::tk::mac::standardAboutPanel**

Bringt das Standard-Cocoa-"Über"-Panel nach vorn mit Informationen aus den Bundle-Dateien. Das Panel zeigt das Anwendungssymbol und Werte zu den Info.plist-Schlüsseln CFBundleName, CFBundleShortVersionString, NSAboutPanelOptionVersion und NSHumanReadableCopyright. Existiert *Credits.html* oder *Credits.rtf* im

Resources-Verzeichnis des Bundles, wird der Inhalt unten im Dialog in einer scrollbaren Textbox angezeigt. Siehe die Dokumentation zu -[NSApplication orderFrontStandardAboutPanelWithOptions:]. Ein Hook für einen eigenen "Über"-Dialog: Ist im Hauptinterpreter eine Tcl-Prozedur tkAboutDialog definiert, wird diese statt standardAboutPanel aufgerufen.

## Systemkonfiguration

Es gibt globale Konfigurationsoptionen, die steuern, wie Tk standardmäßig rendert.

**::tk::mac::useCompatibilityMetrics *boolean***

Erhält Kompatibilität mit älteren Tk/Aqua-Metriken; auf **false** für nativere Abstände setzen.

**::tk::mac::CGAntialiasLimit *limit***

Setzt die Antialiasing-Grenze; Linien dünner als *limit* Pixel werden nicht antialiased. Ganzzahl, Standard 0 (alle Linien antialiased).

**::tk::mac::antialiasedtext *number***

Steuert Text-Antialiasing. Mögliche Werte für *number*: -1 (Standard, Systemvorgabe), 0 (kein Text-AA), 1 (Text-AA).

**::tk::mac::useThemedToplevel *boolean***

Zeichnet Toplevel-Fenster mit dem modernen grauen Mac-Hintergrund. Entspricht **-background systemWindowHeaderBackground** am Toplevel oder der Verwendung eines ttk::frame.

## Hilfsbefehle

**::tk::mac::iconBitmap *name width height -kind value***

Rendert native Symbole und Bitmaps in Tk-Anwendungen (einschließlich jeder von NSImage lesbaren Bilddatei). Ein nativer Bitmap-Name wird (in dieser Reihenfolge) interpretiert als:
- vordefinierter eingebauter 32×32-Symbolname (**stop**, **caution**, **document** usw.)
- *name*, wie von **tk::mac::iconBitmap** definiert
- NSImage-benannter Bildname
- NSImage-URL-Zeichenkette
- 4-Zeichen-OSType eines IconServices-Symbols

Die Argumente *width* und *height* von **tk::mac::iconBitmap** legen die Bildabmessungen fest; -*kind* muss eines von Folgendem sein:

**-file**

Symbol der Datei am angegebenen Pfad

**-fileType**

Symbol des angegebenen Dateityps

**-osType**

Symbol des angegebenen 4-Zeichen-OSType-Dateityps

**-systemType**

Symbol für den angegebenen IconServices-4-Zeichen-OSType

**-namedImage**

benanntes NSImage für den angegebenen Namen

**-imageFile**

Bild am angegebenen Pfad

*Copyright (c) 2011 Kevin Walzer.*
*Copyright (c) 2011 Donal K. Fellows.*

# tkerror

*Stichwörter:* tkerror, background error, reporting

## Name

tkerror - Befehl zur Verarbeitung von Hintergrundfehlern

## Übersicht

**tkerror** *message*

## Beschreibung

Hinweis: Ab Tk 4.1 wurde der Befehl **tkerror** in bgerror umbenannt, weil die Ereignisschleife (die ihn normalerweise aufruft) nun Teil von Tcl ist. Die Verwendung des Namens **tkerror** ist veraltet. Zur Abwärtskompatibilität versucht die vom aktuellen Tk bereitgestellte bgerror weiterhin, **tkerror** aufzurufen, falls einer vorhanden ist (oder per Auto-Load geladen werden kann), sodass alte Skripte, die diesen Fehlerbehandler definieren, weiterhin funktionieren sollten. Sie sollten Ihre Skripte dennoch auf bgerror statt **tkerror** umstellen, da die Unterstützung für den alten Namen in Zukunft entfallen könnte. Schlägt dieser Aufruf fehl, zeigt bgerror einen Dialog mit der Fehlermeldung an und bietet dem Benutzer an, den Stack-Trace anzuzeigen. Wenn Sie eine eigene Fehlerbehandlung wünschen, sollten Sie bgerror direkt überschreiben statt **tkerror**. Die Dokumentation zu bgerror ist Teil der Tcl-Dokumentation.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# tkvars

*Stichwörter:* tkvars, environment, text, variables, version

## Name

geometry, tk_library, tk_patchLevel, tk::scalingPct, tk_strictMotif, tk::svgFmt, tk_version - Von Tk verwendete oder gesetzte Variablen

## Beschreibung

Die folgenden Tcl-Variablen werden von Tk zu verschiedenen Zeitpunkten während der Ausführung gesetzt oder verwendet:

**tk_library**

Diese Variable enthält den Dateinamen eines Verzeichnisses mit einer Bibliothek von Tcl-Skripten, die mit Tk zusammenhängen. Diese Skripte umfassen eine Initialisierungsdatei, die normalerweise bei jedem Start einer Tk-Anwendung verarbeitet wird, sowie weitere Dateien mit Prozeduren, die Standardverhalten für Widgets implementieren.

Der Anfangswert von **tk_library** wird gesetzt, wenn Tk einem Interpreter hinzugefügt wird; dazu werden mehrere Verzeichnisse durchsucht, bis eines gefunden wird, das ein passendes Tk-Startskript enthält. Existiert die Umgebungsvariable **TK_LIBRARY**, wird das von ihr benannte Verzeichnis zuerst geprüft. Ist **TK_LIBRARY** nicht gesetzt oder verweist nicht auf ein passendes Verzeichnis, prüft Tk weitere Verzeichnisse anhand eines kompilierten Standardorts, des Tcl-Bibliotheksverzeichnisses, des Speicherorts der Binärdatei der Anwendung und des aktuellen Arbeitsverzeichnisses.

Die Variable kann von einer Anwendung geändert werden, um zu einer anderen Bibliothek zu wechseln.

**tk_patchLevel**

Enthält eine durch Punkte getrennte Folge von Dezimalzahlen mit dem aktuellen Patch-Level von Tk. Der Patch-Level wird bei jeder neuen Veröffentlichung oder jedem Patch erhöht und identifiziert eindeutig eine offizielle Tk-Version.

Dieser Wert entspricht normalerweise dem Ergebnis von "package require **tk**".

**tk::scalingPct**

Tk setzt diese Variable bei der Initialisierung auf den Skalierungsprozentsatz, der dem DPI-Skalierungsniveau des Displays entspricht. Der Wert ist mindestens 100 und auf Vielfache von 25 beschränkt (100, 125, 150, 175, 200, 225, ...). Die Größen und verschiedenen Attribute der Tk-Kern- und Ttk-Widgets sowie ihrer Komponenten und die Größen der von Tk verwendeten Bilder werden nach diesem Skalierungsprozentsatz gewählt; dasselbe wird für Anwendungen und Bibliothekspakete empfohlen.

Jeder Zugriff auf diese Variable soll ausschließlich schreibgeschützt sein! Wird der Skalierungsfaktor zur Umrechnung zwischen physikalischen Einheiten und Pixeln über **tk scaling** geändert, wird der Wert von **tk::scalingPct** automatisch aktualisiert.

Unter den Fenstersystemen **win32** und **aqua** wird der Skalierungsprozentsatz berechnet, indem [**tk scaling**] * 75 auf das nächste Vielfache von 25 gerundet wird, das mindestens 100 ist. (Unter **aqua** ist das Ergebnis immer 100; die Desktop-Engine skaliert alles bei Bedarf automatisch.) Unter **x11** wird die Ableitung aus [**tk scaling**] nur als Fallback genutzt, weil die Display-Skalierung stark vom Desktop abhängt und meist Systemressourcen außerhalb von Xlib manipuliert, auf dem Tk basiert. Aus technischen Gründen kann der Wert von **tk::scalingPct** von der Systemeinstellung abweichen (z. B. 200 statt 125, 150 oder 175 unter GNOME auf Xorg oder Cinnamon). Unter **x11** wird der Prozentsatz überwiegend (nicht ausschließlich) aus der X-Ressource Xft.dpi ermittelt; Tk synchronisiert anschließend den Umrechnungsfaktor mit dem Prozentsatz mittels **tk scaling**.

**tk_strictMotif**

Diese Variable ist standardmäßig null. Setzt eine Anwendung sie auf eins, versucht Tk, den Motif-Look-and-Feel-Standards so weit wie möglich zu entsprechen. Aktive Elemente wie Buttons und Scrollbar-Schieberegler ändern z. B. ihre Farbe nicht, wenn der Mauszeiger darüber fährt. Moderne Anwendungen sollten diese Variable normalerweise nicht setzen.

**tk::svgFmt**

Diese Variable wird bei der Tk-Initialisierung gesetzt auf

[**list svg -scale** [expr {$**tk::scalingPct** / 100.0}]]

Typische Werte sind {**svg -scale** 1.0}, {**svg -scale** 1.25}, {**svg -scale** 2.0} usw. Es wird empfohlen, den Wert dieser Variable den Befehlen **image create photo**, *imageName*

**configure**, *imageName* **put** und *imageName* read als Wert ihrer Option **-format** zu übergeben, wenn SVG-Bilder erzeugt oder bearbeitet werden, damit ihre Größen dem DPI-Skalierungsniveau des Displays entsprechen.

Jeder Zugriff auf diese Variable soll ausschließlich schreibgeschützt sein! Wird der Skalierungsfaktor zur Umrechnung zwischen physikalischen Einheiten und Pixeln über **tk scaling** geändert, wird der Wert von **tk::svgFmt** automatisch aktualisiert.

**tk_version**

Tk setzt diese Variable im Interpreter jeder Anwendung. Sie enthält die aktuelle Versionsnummer der Tk-Bibliothek in der Form *major.minor*. *Major* und *minor* sind Ganzzahlen. Die Hauptversionsnummer steigt bei jeder Tk-Veröffentlichung mit nicht abwärtskompatiblen Änderungen (wenn bestehende Tk-Anwendungen und Skripte angepasst werden müssen). Die Nebenversionsnummer steigt bei jeder neuen Tk-Veröffentlichung und wird auf null zurückgesetzt, wenn sich die Hauptversionsnummer ändert.

### Interne und Debug-Variablen

Diese Variablen sollten normalerweise nicht vom Anwendungscode gesetzt werden.

**tk::Priv**

Diese Variable ist ein Array mit mehreren für Tk private Informationen. Die Elemente von **tk::Priv** werden von Tk-Bibliotheksprozeduren und Standard-Bindings verwendet. Sie dürfen von keinem Code außerhalb von Tk zugegriffen werden.

**tk_textRedraw**

siehe unten ...

**tk_textRelayout**

Diese Variablen werden von Text-Widgets gesetzt, wenn Debugging eingeschaltet ist. Die in diese Variablen geschriebenen Werte können zum Testen oder Debuggen von Text-Widget-Operationen verwendet werden. Sie werden überwiegend von Tks Test-Suite genutzt.

## Weitere globale Variablen

Die folgenden Variablen sind nur in wish-Programmen garantiert vorhanden; die Tk-Bibliothek definiert sie selbst nicht, aber viele Tk-Umgebungen tun es.

**geometry**

Falls gesetzt, enthält sie die vom Benutzer vorgegebene Geometriespezifikation für das Haupt-Tk-Fenster.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# tkwait

*Stichwörter:* tkwait, variable, visibility, wait, window

## Name

tkwait - Wartet auf Variablenänderung oder Fensterzerstörung

## Übersicht

**tkwait variable** *name*
**tkwait visibility** *name*
**tkwait window** *name*

## Beschreibung

Der Befehl **tkwait** wartet darauf, dass eines von mehreren Ereignissen eintritt, und kehrt dann zurück, ohne weitere Aktionen auszuführen. Der Rückgabewert ist stets eine leere Zeichenkette. Ist das erste Argument **variable** (oder eine Abkürzung davon), ist das zweite Argument der Name einer globalen Variable; der Befehl wartet, bis diese Variable geändert wird. Ist das erste Argument **visibility** (oder eine Abkürzung davon), ist das zweite Argument der Name eines Fensters; der Befehl **tkwait** wartet auf eine Änderung seines Sichtbarkeitszustands (angezeigt durch das Eintreffen eines VisibilityNotify-Ereignisses). Diese Form wird typischerweise verwendet, um zu warten, bis ein neu erstelltes Fenster auf dem Bildschirm erscheint, bevor eine Aktion ausgeführt wird. Ist das erste Argument **window** (oder eine Abkürzung davon), ist das zweite Argument der Name eines Fensters; der Befehl **tkwait** wartet, bis dieses Fenster zerstört wird. Diese Form wird typischerweise verwendet, um zu warten, bis der Benutzer die Interaktion mit einem Dialogfeld beendet hat, bevor das Ergebnis dieser Interaktion verwendet wird.

Während **tkwait** wartet, verarbeitet es Ereignisse auf übliche Weise, sodass die Anwendung weiterhin auf Benutzerinteraktionen reagiert. Ruft ein Ereignisbehandler **tkwait** erneut auf, muss der verschachtelte Aufruf von **tkwait** abgeschlossen sein, bevor der äußere Aufruf beendet werden kann.

*Copyright (c) 1992 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# toplevel

*Stichwörter:* toplevel, widget

## Name

toplevel - Erstellt und manipuliert Haupt- und Popup-Fenster vom Typ 'toplevel'

## Übersicht

**toplevel** *pathName* ?*options*?

## Standardoptionen

**-borderwidth**, **-highlightcolor**, **-pady**, **-cursor**, **-highlightthickness**, **-relief**, **-highlightbackground**, **-padx**, **-takefocus**

Siehe die Handbuchseite options für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-background (background/Background)**

Diese Option ist identisch mit der Standard-Option **-background**, außer dass ihr Wert auch als leerer String angegeben werden kann. In diesem Fall zeigt das Widget keinen Hintergrund oder Rahmen an, und es werden keine Farben aus seiner Farbkarte für Hintergrund und Rahmen verbraucht. Ein leerer Hintergrund deaktiviert das Zeichnen des Hintergrundbildes.

**-backgroundimage (backgroundImage/BackgroundImage)**

Bild, das im Hintergrund des Toplevel innerhalb des Rahmens angezeigt wird (das Bild wird durch Hervorhebungsring und Rahmen des Toplevel abgeschnitten, falls vorhanden); Unter-Widgets werden darüber gezeichnet. Das Bild muss mit image create erstellt worden sein. Als leerer String: kein Bild.

**-class (class/Class)**

Klasse des Fensters. Wird bei Abfragen der Optionsdatenbank und später z. B. für Bindings verwendet. Manche Window-Manager zeigen den Klassennamen im Dock, andere den Fenstertitel. **-class** kann nicht mit **configure** geändert werden.

**-colormap (colormap/Colormap)**

Farbkarte für das Fenster: **new** (neue Farbkarte für Fenster und Kinder) oder Name eines anderen Fensters (gleicher Bildschirm, gleiche Visualisierung wie *pathName*); dann nutzt das neue Fenster dessen Farbkarte. Ohne Angabe: Standard-Farbkarte des Bildschirms. Nicht mit **configure** änderbar.

**-container (container/Container)**

Wert muss ein Boolean sein. Ist er wahr, dient das Fenster als Container für eine eingebettete Anwendung (z. B. Tk-Toplevel mit **-use**). Das Fenster unterstützt die passenden Window-Manager-Protokolle (z. B. Geometrie). In dieser Anwendung sollte das Fenster keine eigenen Kinder haben. Nicht mit **configure** änderbar.

**-height (height/Height)**

Legt die gewünschte Höhe für das Fenster in einer der von Tk_GetPixels akzeptierten Formen fest. Wenn diese Option negativ oder null ist, fordert das Fenster überhaupt keine Größe an.

**-menu (menu/Menu)**

Menü-Widget als Menüleiste. Auf dem Macintosh über den gesamten Hauptmonitor; unter Windows und UNIX in der Fensterdekoration des Toplevel.

**-screen**

Bildschirm für das neue Fenster. Jeder gültige Bildschirmname, auch eines anderen Displays. Standard: gleicher Bildschirm wie das Eltern-Element. Speziell: nicht über die Optionsdatenbank setzbar, nicht mit **configure** änderbar.

**-tile (tile/Tile)**

Legt fest, wie das Hintergrundbild (siehe -backgroundimage) gezeichnet wird. Ist der Wert wahr (Tcl_GetBoolean), wird das Bild gekachelt, Ursprung der ersten Kopie oben links im Innenraum des Toplevel. Standard falsch: Bild zentriert.

**-use (use/Use)**

Für Einbettung. Ist der Wert nicht leer, muss er die Fenster-ID eines Container-Fensters sein (hexadezimal wie von winfo id). Das Toplevel wird Kind dieses Containers statt des Root-Fensters des Bildschirms. Liegt der Container in einer Tk-Anwendung, muss es ein frame- oder toplevel-Widget mit gesetzter Option **-container** sein. Nicht mit **configure** änderbar.

**-visual (visual/Visual)**

Visuelle Informationen in allen von Tk_GetVisual akzeptierten Formen. Ohne Angabe: Standard-Visual des Bildschirms. **-visual** nicht mit **configure** änderbar.

**-width (width/Width)**

Gewünschte Breite in allen von Tk_GetPixels akzeptierten Formen. Negativ oder null: keine Größenanforderung.

## Beschreibung

Der Befehl **toplevel** erstellt ein neues Toplevel-Widget (*pathName*). Weitere Optionen können in der Befehlszeile oder Optionsdatenbank gesetzt werden (Hintergrund, Relief usw.). **toplevel** liefert den Pfadnamen des neuen Fensters.

Ein Toplevel ähnelt einem frame, wird aber als Top-Level-Fenster erzeugt: sein X-Eltern-Element ist das Root-Fenster eines Bildschirms, nicht das logische Eltern-Element aus dem Tk-Pfadnamen. Hauptzweck: Container für Dialoge und Widget-Sammlungen. Sichtbar sind im Wesentlichen Hintergrund und optional ein 3D-Rahmen (erhaben/eingesenkt).

## Widget-Befehl

**toplevel** erzeugt einen neuen Tcl-Befehl mit dem Namen des Fensterpfads. Form:

```
pathName option ?arg ...?
```

*PathName* ist der Befehlsname (gleich dem Toplevel-Pfad). *Option* und *arg* bestimmen das Verhalten. Für Toplevel-Widgets möglich:

***pathName* cget *option***

Aktueller Wert der Konfigurationsoption *option* (alle vom Befehl **toplevel** akzeptierten Werte).

***pathName* configure ?*option*? ?*value option value* ...?**

Konfigurationsoptionen abfragen oder ändern. Ohne *option*: Liste aller Optionen (Format siehe Tk_ConfigureInfo). Nur *option*: Liste für diese eine Option. Mit *option*-*value*-Paaren: Optionen setzen, leerer String als Rückgabe. *option*: alle von **toplevel** akzeptierten Werte.

## Bindings

Beim Erzeugen eines neuen Toplevel gibt es keine Standard-Bindings; Toplevels sind nicht für Interaktion gedacht.

Bindings auf Toplevels können Ereignisse von Unter-Widgets erhalten.

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# winfo

*Stichwörter:* winfo, atom, children, class, geometry, height, identifier, information, interpreters, mapped, parent, path name, screen, virtual root, width, window

## Name

winfo - Gibt fensterbezogene Informationen zurück

## Übersicht

**winfo** *option* ?*arg* ...?

## Beschreibung

Der Befehl **winfo** dient dazu, Informationen über von Tk verwaltete Fenster abzurufen. Je nach dem Argument *option* kann er in vielen verschiedenen Formen aufgerufen werden. Die zulässigen Formen sind:

**winfo atom ?*-displayof window*? *name***

Gibt eine Dezimalzeichenkette zurück, die den ganzzahligen Bezeichner für das Atom mit dem Namen *name* angibt. Existiert kein Atom mit dem Namen *name*, wird ein neues erzeugt. Ist die Option **-displayof** angegeben, wird das Atom auf dem Display von *window* nachgeschlagen; andernfalls auf dem Display des Hauptfensters der Anwendung.

**winfo atomname ?*-displayof window*? *id***

Gibt den Textnamen für das Atom mit dem ganzzahligen Bezeichner *id* zurück. Ist die Option **-displayof** angegeben, wird der Bezeichner auf dem Display von *window* nachgeschlagen; andernfalls auf dem Display des Hauptfensters der Anwendung. Dieser Befehl ist die Umkehrung von **winfo atom**. Er erzeugt einen Fehler, wenn kein solches Atom existiert.

**winfo cells *window***

Gibt eine Dezimalzeichenkette zurück, die die Anzahl der Zellen in der Farbtabelle für *window* angibt.

**winfo children *window***

Gibt eine Liste zurück, die die Pfadnamen aller Kinder von *window* enthält. Toplevel-Fenster werden als Kinder ihrer logischen Eltern zurückgegeben. Die Liste ist in Stapelreihenfolge angeordnet, mit dem untersten Fenster zuerst, außer bei Toplevel-Fenstern, die nicht in Stapelreihenfolge zurückgegeben werden. Verwenden Sie den Befehl wm stackorder, um die Stapelreihenfolge von Toplevel-Fenstern abzufragen.

**winfo class *window***

Gibt den Klassennamen für *window* zurück.

**winfo colormapfull *window***

Gibt 1 zurück, wenn die Farbtabelle für *window* als voll bekannt ist, andernfalls 0. Die Farbtabelle eines Fensters gilt als "voll bekannt", wenn der letzte Versuch, auf diesem Fenster eine neue Farbe zu allozieren, fehlgeschlagen ist und diese Anwendung seit der fehlgeschlagenen Allozierung keine Farben in der Farbtabelle freigegeben hat.

**winfo containing ?*-displayof window*? *rootX rootY***

Gibt den Pfadnamen des Fensters zurück, das den durch *rootX* und *rootY* gegebenen Punkt enthält. *RootX* und *rootY* sind in Bildschirmeinheiten angegeben (d. h. in jeder für Tk_GetPixels zulässigen Form) im Koordinatensystem des Root-Fensters (bei Verwendung eines Virtual-Root-Window-Managers wird das Koordinatensystem des virtuellen Root-Fensters verwendet). Ist die Option **-displayof** angegeben, beziehen sich die Koordinaten auf den Bildschirm, der *window* enthält; andernfalls auf den Bildschirm des Hauptfensters der Anwendung. Enthält kein Fenster in dieser Anwendung den Punkt, wird ein leerer String zurückgegeben. Ein leerer String wird auch zurückgegeben, wenn der Punkt in der Titelleiste oder im Rahmen des höchsten enthaltenden Toplevel in dieser Anwendung liegt. (Beachten Sie, dass bei manchen Window-Managern die Rahmen unsichtbar sein können.) Bei der Auswahl des enthaltenden Fensters haben Kinder höhere Priorität als Eltern, und unter Geschwistern wird das höchste in der Stapelreihenfolge gewählt.

**winfo depth *window***

Gibt eine Dezimalzeichenkette zurück, die die Tiefe von *window* angibt (Anzahl der Bits pro Pixel).

**winfo exists *window***

Gibt 1 zurück, wenn ein Fenster mit dem Namen *window* existiert, 0 wenn kein solches Fenster existiert.

**winfo fpixels *window number***

Gibt einen Gleitkommawert zurück, der die Anzahl der Pixel in *window* angibt, die der durch *number* gegebenen Entfernung entsprechen. *Number* kann in jeder für Tk_GetScreenMM zulässigen Form angegeben werden, z. B. "2.0c" oder "1i". Der Rückgabewert kann gebrochen sein; für einen ganzzahligen Wert verwenden Sie **winfo pixels**.

**winfo geometry *window***

Gibt die Geometrie für *window* in der Form *width***x***height***+***x***+***y* zurück. Alle Abmessungen sind in Pixeln.

**winfo height *window***

Gibt eine Dezimalzeichenkette zurück, die die Höhe von *window* in Pixeln angibt. Wenn ein Fenster neu erzeugt wird, beträgt seine Höhe zunächst 1 Pixel; die Höhe wird später vom Geometrie-Manager geändert, um den Anforderungen des Fensters zu genügen. Benötigen Sie die tatsächliche Höhe unmittelbar nach dem Erzeugen eines Widgets, rufen Sie update auf, um den Geometrie-Manager zur Anordnung zu zwingen, oder verwenden Sie **winfo reqheight**, um die angeforderte statt der tatsächlichen Höhe des Fensters zu erhalten.

**winfo id *window***

Gibt eine hexadezimale Zeichenkette zurück, die einen plattformspezifischen Bezeichner auf niedriger Ebene für *window* angibt. Auf Unix-Plattformen ist dies der X-Fensterbezeichner. Unter Windows ist dies die Windows-HWND. Auf dem Macintosh hat der Wert außerhalb von Tk keine Bedeutung.

**winfo interps ?*-displayof window*?**

Gibt eine Liste zurück, deren Elemente die Namen aller Tcl-Interpreter (z. B. aller Tk-basierten Anwendungen) sind, die derzeit für ein bestimmtes Display registriert sind. Ist die Option **-displayof** angegeben, bezieht sich der Rückgabewert auf das Display von *window*; andernfalls auf das Display des Hauptfensters der Anwendung.

**winfo ismapped *window***

Gibt **1** zurück, wenn *window* derzeit gemappt ist, andernfalls **0**.

**winfo manager *window***

Gibt den Namen des Geometrie-Managers zurück, der derzeit für *window* verantwortlich ist, oder einen leeren String, wenn *window* von keinem Geometrie-Manager verwaltet wird. Der Name ist in der Regel der Name des Tcl-Befehls für den Geometrie-Manager, z. B. pack oder place. Ist der Geometrie-Manager ein Widget, z. B. Canvas oder Text, ist der Name der Klassenbefehl des Widgets, z. B. canvas.

**winfo name *window***

Gibt den Namen von *window* zurück (d. h. seinen Namen innerhalb des Eltern-Widgets, im Gegensatz zum vollständigen Pfadnamen). Der Befehl **winfo name.** gibt den Namen der Anwendung zurück.

**winfo parent *window***

Gibt den Pfadnamen des Eltern-Widgets von *window* zurück oder einen leeren String, wenn *window* das Hauptfenster der Anwendung ist.

**winfo pathname ?*-displayof window*? *id***

Gibt den Pfadnamen des Fensters zurück, dessen X-Bezeichner *id* ist. *Id* muss eine Dezimal-, Hexadezimal- oder Oktalzahl sein und einem Fenster in der aufrufenden Anwendung entsprechen. Ist die Option **-displayof** angegeben, wird der Bezeichner auf dem Display von *window* nachgeschlagen; andernfalls auf dem Display des Hauptfensters der Anwendung.

**winfo pixels *window number***

Gibt die Anzahl der Pixel in *window* zurück, die der durch *number* gegebenen Entfernung entsprechen. *Number* kann in jeder für Tk_GetPixels zulässigen Form angegeben werden, z. B.

"2.0c" oder "1i". Das Ergebnis wird auf den nächsten ganzzahligen Wert gerundet; für ein gebrochenes Ergebnis verwenden Sie **winfo fpixels**.

**winfo pointerx *window***

Befindet sich der Mauszeiger auf demselben Bildschirm wie *window*, gibt er die x-Koordinate des Zeigers zurück, gemessen in Pixeln im Root-Fenster des Bildschirms. Ist auf dem Bildschirm ein virtuelles Root-Fenster in Verwendung, wird die Position im virtuellen Root gemessen. Befindet sich der Mauszeiger nicht auf demselben Bildschirm wie *window*, wird -1 zurückgegeben.

**winfo pointerxy *window***

Befindet sich der Mauszeiger auf demselben Bildschirm wie *window*, gibt er eine Liste mit zwei Elementen zurück: die x- und y-Koordinaten des Zeigers, gemessen in Pixeln im Root-Fenster des Bildschirms. Ist auf dem Bildschirm ein virtuelles Root-Fenster in Verwendung, wird die Position im virtuellen Root berechnet. Befindet sich der Mauszeiger nicht auf demselben Bildschirm wie *window*, sind beide zurückgegebenen Koordinaten -1.

**winfo pointery *window***

Befindet sich der Mauszeiger auf demselben Bildschirm wie *window*, gibt er die y-Koordinate des Zeigers zurück, gemessen in Pixeln im Root-Fenster des Bildschirms. Ist auf dem Bildschirm ein virtuelles Root-Fenster in Verwendung, wird die Position im virtuellen Root berechnet. Befindet sich der Mauszeiger nicht auf demselben Bildschirm wie *window*, wird -1 zurückgegeben.

**winfo reqheight *window***

Gibt eine Dezimalzeichenkette zurück, die die angeforderte Höhe von *window* in Pixeln angibt. Dies ist der Wert, den der Geometrie-Manager von *window* zur Berechnung der Geometrie verwendet.

**winfo reqwidth *window***

Gibt eine Dezimalzeichenkette zurück, die die angeforderte Breite von *window* in Pixeln angibt. Dies ist der Wert, den der Geometrie-Manager von *window* zur Berechnung der Geometrie verwendet.

**winfo rgb *window color***

Gibt eine Liste mit drei Dezimalwerten im Bereich 0 bis 65535 zurück, die die Rot-, Grün- und Blau-Intensitäten darstellen, die *color* im durch *window* gegebenen Fenster entsprechen. *Color* kann in jeder für eine Farboption zulässigen Form angegeben werden.

**winfo rootx *window***

Gibt eine Dezimalzeichenkette zurück, die die x-Koordinate der oberen linken Ecke des Rahmens von *window* (oder von *window*, wenn es keinen Rahmen hat) im Root-Fenster des Bildschirms angibt.

**winfo rooty *window***

Gibt eine Dezimalzeichenkette zurück, die die y-Koordinate der oberen linken Ecke des Rahmens von *window* (oder von *window*, wenn es keinen Rahmen hat) im Root-Fenster des Bildschirms angibt.

**winfo screen *window***

Gibt den Namen des mit *window* verknüpften Bildschirms in der Form *displayName*.*screenIndex* zurück.

**winfo screencells *window***

Gibt eine Dezimalzeichenkette zurück, die die Anzahl der Zellen in der Standard-Farbtabelle für den Bildschirm von *window* angibt.

**winfo screendepth *window***

Gibt eine Dezimalzeichenkette zurück, die die Tiefe des Root-Fensters des Bildschirms von *window* angibt (Anzahl der Bits pro Pixel).

**winfo screenheight *window***

Gibt eine Dezimalzeichenkette zurück, die die Höhe des Bildschirms von *window* in Pixeln angibt.

**winfo screenmmheight *window***

Gibt eine Dezimalzeichenkette zurück, die die Höhe des Bildschirms von *window* in Millimetern angibt.

**winfo screenmmwidth *window***

Gibt eine Dezimalzeichenkette zurück, die die Breite des Bildschirms von *window* in Millimetern angibt.

**winfo screenvisual *window***

Gibt eine der folgenden Zeichenketten zurück, um die Standard-Visualklasse für den Bildschirm von *window* anzugeben: **directcolor**, **grayscale**, **pseudocolor**, **staticcolor**, **staticgray** oder **truecolor**.

**winfo screenwidth *window***

Gibt eine Dezimalzeichenkette zurück, die die Breite des Bildschirms von *window* in Pixeln angibt.

**winfo server *window***

Gibt eine Zeichenkette zurück, die Informationen über den Server für das Display von *window* enthält. Das genaue Format dieser Zeichenkette kann von Plattform zu Plattform variieren. Bei X-Servern hat die Zeichenkette die Form "**X***major***R***minor vendor vendorVersion*", wobei *major* und *minor* die vom Server bereitgestellten Versions- und Revisionsnummern sind (z. B. **X11R5**), *vendor* der Name des Server-Herstellers ist und *vendorRelease* eine vom Server bereitgestellte ganzzahlige Release-Nummer ist.

**winfo toplevel *window***

Gibt den Pfadnamen des Fensters an der Spitze der Hierarchie zurück, das *window* enthält. In Standard-Tk ist dies stets ein toplevel-Widget, Erweiterungen können jedoch andere Arten von Hierarchie-Spitzen-Widgets erzeugen.

**winfo viewable *window***

Gibt 1 zurück, wenn *window* und alle seine Vorfahren bis zum nächsten Toplevel-Fenster gemappt sind. Gibt 0 zurück, wenn eines dieser Fenster nicht gemappt ist.

**winfo visual *window***

Gibt eine der folgenden Zeichenketten zurück, um die Visualklasse für *window* anzugeben: **directcolor**, **grayscale**, **pseudocolor**, **staticcolor**, **staticgray** oder **truecolor**.

**winfo visualid *window***

Gibt den X-Bezeichner für das Visual von *window* zurück.

**winfo visualsavailable *window* ?includeids?**

Gibt eine Liste zurück, deren Elemente die für den Bildschirm von *window* verfügbaren Visuals beschreiben. Jedes Element besteht aus einer Visualklasse gefolgt von einer ganzzahligen Tiefe. Die Klasse hat dieselbe Form wie der Rückgabewert von **winfo visual**. Die Tiefe gibt die Anzahl der Bits pro Pixel im Visual an. Wird zusätzlich das Argument **includeids** angegeben, folgt auf die Tiefe der X-Bezeichner für das Visual.

**winfo vrootheight *window***

Gibt die Höhe des mit *window* verknüpften virtuellen Root-Fensters zurück, falls eines vorhanden ist; andernfalls die Höhe des Bildschirms von *window*.

**winfo vrootwidth *window***

Gibt die Breite des mit *window* verknüpften virtuellen Root-Fensters zurück, falls eines vorhanden ist; andernfalls die Breite des Bildschirms von *window*.

**winfo vrootx *window***

Gibt den x-Versatz des mit *window* verknüpften virtuellen Root-Fensters relativ zum Root-Fenster seines Bildschirms zurück. Dies ist normalerweise null oder negativ. Gibt 0 zurück, wenn es kein virtuelles Root-Fenster für *window* gibt.

**winfo vrooty *window***

Gibt den y-Versatz des mit *window* verknüpften virtuellen Root-Fensters relativ zum Root-Fenster seines Bildschirms zurück. Dies ist normalerweise null oder negativ. Gibt 0 zurück, wenn es kein virtuelles Root-Fenster für *window* gibt.

**winfo width *window***

Gibt eine Dezimalzeichenkette zurück, die die Breite von *window* in Pixeln angibt. Wenn ein Fenster neu erzeugt wird, beträgt seine Breite zunächst 1 Pixel; die Breite wird später vom Geometrie-Manager geändert, um den Anforderungen des Fensters zu genügen. Benötigen Sie die tatsächliche Breite unmittelbar nach dem Erzeugen eines Widgets, rufen Sie update auf, um den Geometrie-Manager zur Anordnung zu zwingen, oder verwenden Sie **winfo reqwidth**, um die angeforderte statt der tatsächlichen Breite des Fensters zu erhalten.

**winfo x *window***

Gibt eine Dezimalzeichenkette zurück, die die x-Koordinate der oberen linken Ecke des Rahmens von *window* (oder von *window*, wenn es keinen Rahmen hat) im Eltern-Widget von *window* angibt.

**winfo y *window***

Gibt eine Dezimalzeichenkette zurück, die die y-Koordinate der oberen linken Ecke des Rahmens von *window* (oder von *window*, wenn es keinen Rahmen hat) im Eltern-Widget von *window* angibt.

## Beispiele

Ausgabe der Position des Mauszeigers und des Fensters, über dem er sich befindet:

```
lassign [winfo pointerxy .] x y
puts -nonewline "Mouse pointer at ($x,$y) which is "
set win [winfo containing $x $y]
if {$win eq ""} {
    puts "over no window"
} else {
    puts "over $win"
}
```

*Copyright (c) 1990-1994 The Regents of the University of California.*
*Copyright (c) 1994-1997 Sun Microsystems, Inc.*

# wish

*Stichwörter:* wish, application, argument, interpreter, prompt, script file, shell, toolkit, toplevel

## Name

wish - Einfache Fenster-Shell

## Übersicht

**wish** ?**options**? ?*fileName arg* ...?

## Optionen

**-encoding *name***

Legt die Kodierung des in *fileName* gespeicherten Textes fest. Die Option wird nur vor dem Argument *fileName* erkannt.

**-colormap *new***

Legt fest, dass das Fenster eine eigene private Farbkarte erhält statt der Standardfarbkarte des Bildschirms.

**-display *display***

Display (und Bildschirm), auf dem das Fenster erscheint.

**-geometry *geometry***

Anfangsgeometrie des Fensters. Ist die Option gesetzt, landet ihr Wert in der globalen Variable geometry des Tcl-Interpreters der Anwendung.

**-name *name***

*name* wird der Fenstertitel und der Name des Interpreters für send-Befehle.

**-sync**

Alle Befehle an den X-Server laufen synchron, Fehler werden sofort gemeldet. Die Ausführung wird deutlich langsamer, das hilft beim Debuggen.

**-use *id***

Legt fest, dass das Hauptfenster der Anwendung in das Fenster mit der Kennung *id* eingebettet wird und nicht als eigenständiges Toplevel-Fenster entsteht. *Id* hat dieselbe Form wie der Wert der Option **-use** bei Toplevel-Widgets, also die Form, die winfo id liefert. Auf manchen Plattformen funktioniert das nur, wenn *id* einen Tk-frame oder ein toplevel mit gesetzter Option **-container** bezeichnet.

**-visual *visual***

Legt das Visual für das Fenster fest. *Visual* darf jede Form haben, die die Prozedur Tk_GetVisual unterstützt.

**--**

Alle übrigen Argumente gehen unbearbeitet in die Variable argv des Skripts. So lässt sich etwa **-name** an das Skript durchreichen, ohne dass **wish** es auswertet.

## Beschreibung

**Wish** besteht aus der Tcl-Befehlssprache, dem Tk-Toolkit und einem Hauptprogramm, das Befehle von der Standardeingabe oder aus einer Datei liest. Es erzeugt ein Hauptfenster und verarbeitet dann Tcl-Befehle. Wird **wish** mit Argumenten aufgerufen, nennen die ersten Argumente ?**-encoding** *name*? ?*fileName*? den Namen einer Skriptdatei und optional die Kodierung der darin gespeicherten Textdaten. Ein Wert für *fileName* wird erkannt, wenn das betreffende Argument nicht mit "-" beginnt.

Gibt es keine Argumente oder nennen sie kein *fileName*, liest wish Tcl-Befehle interaktiv von der Standardeingabe. Die Verarbeitung läuft, bis alle Fenster gelöscht sind oder die Standardeingabe das Dateiende erreicht. Liegt im Home-Verzeichnis des Benutzers eine Datei "**.wishrc**" (unter Windows "**wishrc.tcl**"), wertet **wish** sie als Tcl-Skript aus, unmittelbar bevor der erste Befehl von der Standardeingabe gelesen wird.

Nennen die Argumente von **wish** ein *fileName*, gilt *fileName* als Name einer Skriptdatei. **Wish** wertet das Skript in *fileName* aus (das in der Regel eine Oberfläche aufbaut) und reagiert danach auf Ereignisse, bis alle Fenster gelöscht sind. Von der Standardeingabe werden keine Befehle gelesen. Eine automatische Auswertung von "**.wishrc**" findet nicht statt, wenn auf der Befehlszeile von **wish** ein Skriptdateiname steht; die Skriptdatei kann die Datei bei Bedarf mit source laden.

Unter Windows unterscheidet sich das Programm **wish***version***.exe** in einem weiteren Punkt von tclsh*version***.exe**: es verbindet sich nicht mit einer Windows-Konsole, sondern ist ein Fensterprogramm. Deshalb stellt es zusätzlich seinen eigenen Befehl console bereit.

## Verarbeitung der Optionen

**Wish** verarbeitet alle Befehlszeilenoptionen aus der Übersicht Optionen selbst. Alle übrigen Befehlszeilenargumente gehen über die später beschriebenen Variablen argc und argv an die Anwendung.

## Anwendungsname und Klasse

Der Name der Anwendung, etwa für send-Befehle, kommt aus der Option **-name**, falls sie gesetzt ist. Sonst aus *fileName*, falls angegeben, oder aus dem Befehlsnamen, mit dem **wish** aufgerufen wurde. Enthält der Name in den letzten beiden Fällen ein "/", zählen nur die Zeichen nach dem letzten Schrägstrich als Anwendungsname.

Die Klasse der Anwendung, etwa für Optionen über eine **RESOURCE_MANAGER**-Eigenschaft oder eine .Xdefaults-Datei, ist derselbe Name mit großem Anfangsbuchstaben.

## Variablen

**Wish** setzt diese globalen Tcl-Variablen:

argc
Anzahl der *arg*-Argumente (0, wenn keine), ohne die oben beschriebenen Optionen.

argv
Tcl-Liste der *arg*-Argumente, die auf eine Option **--** folgen oder auf keine der unter Optionen beschriebenen Optionen passen, in derselben Reihenfolge, oder eine leere Zeichenkette, wenn es keine solchen Argumente gibt.

argv0
*fileName*, wenn er angegeben wurde. Sonst der Name, mit dem **wish** aufgerufen wurde.

geometry
Ist **-geometry** gesetzt, kopiert **wish** den Wert in diese Variable. Existiert die Variable noch, nachdem *fileName* ausgewertet wurde, setzt **wish** mit wm geometry die Geometrie des Hauptfensters auf diesen Wert.

tcl_interactive
1, wenn **wish** Befehle interaktiv liest (kein *fileName* und die Standardeingabe ist ein terminalartiges Gerät), sonst 0.

## Skriptdateien

Steht in der ersten Zeile eines Tcl-Skripts

```
#!/usr/local/bin/wish
```

lässt sich die Datei direkt aus der Shell aufrufen, sobald sie als ausführbar gekennzeichnet ist. Das setzt voraus, dass **wish** am üblichen Ort /usr/local/bin installiert ist; sonst muss die Zeile angepasst werden. Viele UNIX-Systeme lassen die Zeile **#!** nur etwa 30 Zeichen lang werden. Die ausführbare Datei **wish** muss daher über einen kurzen Dateinamen erreichbar sein.

Besser beginnen Skriptdateien mit diesen drei Zeilen:

```
#!/bin/sh
# the next line restarts using wish \
exec wish "$0" ${1+"$@"}
```

Das hat gegenüber dem vorigen Absatz drei Vorteile. Erstens muss der Ort der **wish**-Binärdatei nicht fest im Skript stehen: sie darf irgendwo im Suchpfad der Shell liegen. Zweitens entfällt die Grenze von etwa 30 Zeichen für den Dateinamen. Drittens funktioniert das auch, wenn **wish** selbst ein Shell-Skript ist (auf manchen Systemen wählt das **wish**-Skript eine von mehreren Binärdateien, um mehrere Architekturen oder Betriebssysteme zu bedienen). Die drei Zeilen lassen sowohl **sh** als auch **wish** das Skript verarbeiten, exec führt aber nur **sh** aus. **sh** verarbeitet das Skript zuerst, behandelt die zweite Zeile als Kommentar und führt die dritte aus. exec beendet die Verarbeitung durch die Shell und startet **wish**, das das ganze Skript erneut verarbeitet. Beim Start von **wish** gelten alle drei Zeilen als Kommentar, weil der Backslash am Ende der zweiten Zeile die dritte Zeile zum Kommentar der zweiten macht.

Das Ende einer Skriptdatei ist das physische Ende des Mediums oder das Zeichen "\032" ("\u001a", Strg-Z). Steht dieses Zeichen in der Datei, liest **wish** den Text bis unmittelbar vor dieses Zeichen. Eine Anwendung, die das Zeichen in der Datei braucht, kann es als "\032", "\x1a" oder "\u001a" kodieren oder mit Befehlen wie format oder binary erzeugen.

## Eingabeaufforderung

Interaktives **wish** fordert jeden Befehl mit "**%** " an. Die Variablen **tcl_prompt1** und **tcl_prompt2** ändern die Aufforderung. Existiert **tcl_prompt1**, muss sie ein Tcl-Skript sein, das die Aufforderung ausgibt; statt selbst eine Aufforderung zu schreiben, wertet **wish** das Skript in **tcl_prompt1** aus. **tcl_prompt2** gilt entsprechend, wenn ein Zeilenumbruch kommt, der aktuelle Befehl aber noch nicht vollständig ist. Ist **tcl_prompt2** nicht gesetzt, gibt es für unvollständige Befehle keine Aufforderung.

## Zipvfs

Wird eine Zip-Datei an das Ende von **wish** angehängt, wird ihr Inhalt beim Start unter dem virtuellen Dateisystem **//zipfs:/** eingehängt. Liegt im Archiv ein Verzeichnis tk_library auf oberster Ebene, wird es als env(TK_LIBRARY) geladen. Liegt im obersten Verzeichnis des Archivs eine Datei **main.tcl**, wird sie geladen statt der üblichen Befehlszeilenverarbeitung der Shell.

An das Ende eines Programmbilds (tclsh oder wish) kann nur eine Zip-Datei angehängt werden. Sind mehrere angehängt, gilt nur die letzte. Dieses Dateisystem ist nur lesbar. Dateien darin lassen sich nicht hinzufügen oder ändern. Einzelheiten stehen in zipfs(n).

*Copyright (c) 1991-1994 The Regents of the University of California.*

*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# wm

*Stichwörter:* wm, aspect ratio, deiconify, focus model, geometry, grid, group, icon, iconify, increments, position, size, title, top-level window, units, window manager

## Name

wm - Kommunikation mit dem Fenstermanager

## Übersicht

**wm** *option window* ?*args*?

## Beschreibung

Der **wm**-Befehl dient der Interaktion mit Fenstermanagern, um Eigenschaften wie den Titel eines Fensters, seine Geometrie oder die Schritte, in denen es skaliert werden darf, zu steuern. Der **wm**-Befehl kann je nach dem *option*-Argument viele verschiedene Formen annehmen. Alle Formen erwarten mindestens ein weiteres Argument, *window*, das der Pfadname eines Toplevel-Fensters sein muss.

Die gültigen Formen des **wm**-Befehls sind:

**wm aspect *window* ?*minNumer minDenom maxNumer maxDenom*?**

Wenn *minNumer*, *minDenom*, *maxNumer* und *maxDenom* alle angegeben sind, werden sie an den Fenstermanager übergeben; der Fenstermanager soll sie nutzen, um einen zulässigen Bereich von Seitenverhältnissen für *window* durchzusetzen. Das Seitenverhältnis von *window* (Breite/Länge) wird so eingeschränkt, dass es zwischen *minNumer*/*minDenom* und *maxNumer* /*maxDenom* liegt. Wenn *minNumer* usw. alle als leere Strings angegeben sind, werden bestehende Seitenverhältnis-Beschränkungen entfernt. Wenn *minNumer* usw. angegeben sind, gibt der Befehl einen leeren String zurück. Andernfalls gibt er eine Tcl-Liste mit vier Elementen zurück, die die aktuellen Werte von *minNumer*, *minDenom*, *maxNumer* und *maxDenom* sind (wenn keine Seitenverhältnis-Beschränkungen gelten, wird ein leerer String zurückgegeben).

**wm attributes *window***

...siehe nächstes...

**wm attributes *window* ?option?**

...siehe nächstes...

**wm attributes *window* ?option value option value...?**

Dieser Unterbefehl gibt plattformspezifische Attribute eines Fensters zurück oder setzt sie. Die erste Form gibt eine Liste der plattformspezifischen Flags und ihrer Werte zurück. Die zweite Form gibt den Wert der angegebenen Option zurück. Die dritte Form setzt einen oder mehrere Werte. Die Werte sind wie folgt:

Alle Plattformen unterstützen die folgenden Attribute (X11-Nutzer sollten jedoch die Hinweise unten beachten):

**-alpha**

Legt die Alpha-Transparenzstufe des Toplevel-Fensters fest. Es wird ein Wert von **0.0** (vollständig transparent) bis **1.0** (undurchsichtig) akzeptiert. Werte außerhalb dieses Bereichs werden eingeschränkt. Wo nicht unterstützt, bleibt der **-alpha**-Wert bei **1.0**.

**-fullscreen**
Platziert das Fenster in einem Modus, der den gesamten Bildschirm einnimmt, keine Rahmen hat und den allgemeinen Nutzungsbereich abdeckt (z. B. Startmenü und Taskleiste unter Windows, Dock und Menüleiste unter macOS, allgemeine Fensterdekorationen unter X11).

**-topmost**
Legt fest, ob dies ein Topmost-Fenster ist (wird über allen anderen Fenstern angezeigt).

Unter Windows können die folgenden Attribute gesetzt werden.

**-disabled**
Legt fest, ob das Fenster deaktiviert ist.

**-toolwindow**
Legt ein Toolwindow-Stil-Fenster fest (wie in der MSDN definiert).

**-transparentcolor**
Legt den transparenten Farbindex des Toplevel-Fensters fest. Es wird jeder von Tk_GetColor akzeptierte Farbwert genommen. Wenn der leere String angegeben wird (Standard), wird keine transparente Farbe verwendet. Unterstützt unter Windows 2000/XP+. Wo nicht unterstützt, bleibt der **-transparentcolor**-Wert bei **{}**.

Unter macOS können die folgenden Attribute gesetzt werden.

**-appearance**
Legt fest, ob das Fenster im "Dark Mode" dargestellt wird. Zulässige Werte sind **auto**, **aqua** und **darkaqua**. Bei der Einstellung **auto** wird das Erscheinungsbild des Fensters durch die Systemeinstellungen gesteuert.

**-class**
Legt fest, ob das zugrunde liegende Aqua-Fenster eines Toplevel-Fensters ein Objekt der Klasse NSWindow oder NSPanel ist. Die beiden zulässigen Werte sind **nswindow** und **nspanel**. Die Klasse des zugrunde liegenden Aqua-Fensters kann nicht mehr geändert werden, sobald das Fenster instanziiert wurde; der Versuch ist ein Fehler. Es ist jedoch erlaubt, diese Option für einen Pfadnamen zu setzen, der keinem bestehenden Fenster entspricht. Dann wird der Klassenname für die spätere Verwendung zwischengespeichert. Wenn später ein Toplevel-Fenster mit diesem Pfadnamen erstellt wird, bestimmt der zwischengespeicherte Klassenname, welche Klasse für das zugrunde liegende Aqua-Fenster verwendet wird.

**-isdark**
Gibt einen booleschen Wert zurück, der **true** ist, wenn das Fenster derzeit im Dark Mode ist.

**-modified**
Legt den Änderungszustand des Fensters fest (bestimmt, ob das Schließen-Widget den Änderungsindikator enthält und ob das Proxy-Icon ziehbar ist).

**-notify**
Legt den Prozess-Benachrichtigungszustand fest (Springen des Anwendungs-Dock-Icons).

**-stylemask**
Legt eine Ganzzahl fest, die als styleMask des zugrunde liegenden Aqua-Fensters zugewiesen wird. (Siehe die Apple-Dokumentation zur styleMask-Eigenschaft der NSWindow-Klasse.) Der Wert dieser Option sollte eine Liste von Bitnamen sein. Jedes in der

Liste genannte Bit wird auf 1 gesetzt, alle anderen auf 0. Zulässige Bitnamen sind: **titled**, **closable**, **miniaturizable**, **resizable**, **fullsizecontentview**, **docmodal**, **utility**, **nonactivatingpanel** und **HUDwindow**. Beachten Sie, dass das Setzen des Bits **fullsizecontentview** zur Folge hat, dass die Titelleiste des Fensters transparent wird.

**-tabbingid**

Steuert, wie tabbed Toplevel-Fenster gruppiert werden. Zwei Tabs in derselben Gruppe müssen Toplevel-Fenstern mit derselben tabbingid entsprechen, die eine beliebige UTF-8-Zeichenkette sein kann. In der Tk-Implementierung führt das Ändern der tabbingid eines Toplevel-Fensters in einer Tab-Gruppe dazu, dass es in eine andere Gruppe verschoben wird, in der alle Tabs die neue tabbingid haben, oder — wenn die neue tabbingid eindeutig ist — zu einem normalen, nicht getabten Toplevel-Fenster wird. Es ist erlaubt, die tabbingid zu setzen, bevor das Toplevel-Fenster erstellt wird. Entspricht der im Befehl angegebene Pfadname keinem Toplevel-Fenster, wird der Wert zwischengespeichert und später verwendet, wenn das Toplevel-Fenster tatsächlich erstellt wird.

**-tabbingmode**

Steuert, ob ein Toplevel-Fenster als Tab innerhalb einer Tab-Gruppe geöffnet werden kann. Zulässige Werte sind **auto**, **preferred** oder **disallowed**. Ein Toplevel-Fenster kann genau in zwei Situationen als Tab geöffnet werden: wenn sein tabbingmode **preferred** ist; oder wenn sein tabbingmode **auto** ist und der Benutzer in den Systemeinstellungen unter Desktop und Dock "Beim Öffnen von Dokumenten Tabs bevorzugen" gewählt hat. Es ist erlaubt, den tabbingmode zu setzen, bevor das Toplevel-Fenster erstellt wird. Entspricht der im Befehl angegebene Pfadname keinem Toplevel-Fenster, wird der Wert zwischengespeichert und später verwendet, wenn das Toplevel-Fenster tatsächlich erstellt wird.

**-titlepath**

Legt den Pfad der Datei fest, die als Proxy-Icon des Fensters referenziert wird (das anstelle des Finder-Icons der Datei gezogen und abgelegt werden kann).

**-transparent**

Macht den Inhaltsbereich des Fensters transparent und schaltet den Fensterschatten aus. Damit die Transparenz wirksam ist, muss der Toplevel-Hintergrund auf eine Farbe mit Alpha gesetzt werden, z. B. "systemTransparent".

Unter X11 können die folgenden Attribute gesetzt werden. Nicht alle Fenstermanager unterstützen sie; bei älteren WMs haben sie keine Wirkung.

**-type**

Fordert an, dass der Fenstermanager das Fenster als den angegebenen Typ bzw. die angegebenen Typen interpretiert. Das kann zu anderer Dekoration oder anderer Verwaltung führen; was genau passiert, liegt vollständig beim Fenstermanager. Eine Typenliste kann in Reihenfolge der Präferenz verwendet werden. Die folgenden Werte werden auf in der EWMH-Spezifikation definierte Konstanten abgebildet (andere sind möglich, aber nicht empfohlen):

**desktop**

kennzeichnet eine Desktop-Funktion,

**dock**

kennzeichnet eine Dock-/Panel-Funktion,

**toolbar**

kennzeichnet ein Toolbar-Fenster, das im Auftrag eines anderen Fensters handeln soll, wie mit wm transient angegeben,

**menu**

kennzeichnet ein abgetrenntes Menü, das im Auftrag eines anderen Fensters handeln soll, wie mit wm transient angegeben,

**utility**

kennzeichnet ein Hilfsfenster (z. B. Palette oder Werkzeugkasten), das im Auftrag eines anderen Fensters handeln soll, wie mit wm transient angegeben,

**splash**

kennzeichnet einen Splash-Screen beim Anwendungsstart,

**dialog**

kennzeichnet ein allgemeines Dialogfenster, das im Auftrag eines anderen Fensters handeln soll, wie mit wm transient angegeben,

**dropdown_menu**

kennzeichnet ein aus einer Menüleiste aufgerufenes Menü, das üblicherweise auch override-redirected gesetzt sein sollte (mit wm overrideredirect),

**popup_menu**

kennzeichnet ein Popup-Menü, das üblicherweise auch override-redirected gesetzt sein sollte (mit wm overrideredirect),

**tooltip**

kennzeichnet ein Tooltip-Fenster, das üblicherweise auch override-redirected gesetzt sein sollte (mit wm overrideredirect),

**notification**

kennzeichnet ein Fenster mit Hintergrundbenachrichtigung über ein Ereignis, das üblicherweise auch override-redirected gesetzt sein sollte (mit wm overrideredirect),

**combo**

kennzeichnet die Dropdown-Liste eines Combobox-Widgets, die üblicherweise auch override-redirected gesetzt sein sollte (mit wm overrideredirect),

**dnd**

kennzeichnet ein Fenster, das etwas Ziehbares darstellt, das üblicherweise auch override-redirected gesetzt sein sollte (mit wm overrideredirect),

**normal**

kennzeichnet ein Fenster ohne besondere Interpretation.

**-zoomed**

Fordert an, dass das Fenster maximiert wird. Entspricht unter Windows und macOS **wm state zoomed**.

Unter X11 werden Änderungen an Fensterattributen asynchron ausgeführt. Das Abfragen eines Attributwerts liefert den aktuellen Zustand, der nicht mit dem zuletzt gesetzten Wert übereinstimmen muss, wenn der Fenstermanager die Anfrage noch nicht verarbeitet hat oder das Attribut nicht unterstützt.

**wm client *window* ?*name*?**

Wenn *name* angegeben ist, speichert dieser Befehl *name* (der Name des Rechners, auf dem die Anwendung läuft) in der **WM_CLIENT_MACHINE**-Eigenschaft von *window* für den

Fenstermanager oder Sitzungsmanager und gibt in diesem Fall einen leeren String zurück. Wenn *name* nicht angegeben ist, gibt der Befehl den zuletzt in einem **wm client**-Befehl für *window* gesetzten Namen zurück. Wenn *name* als leerer String angegeben wird, löscht der Befehl die **WM_CLIENT_MACHINE**-Eigenschaft von *window*.

**wm colormapwindows *window* ?*windowList*?**

Dieser Befehl manipuliert die **WM_COLORMAP_WINDOWS**-Eigenschaft, die Fenstermanagern Informationen über Fenster mit privaten Farbtabellen liefert.

Wenn *windowList* nicht angegeben ist, gibt der Befehl eine Liste zurück, deren Elemente die Namen der Fenster in der **WM_COLORMAP_WINDOWS**-Eigenschaft sind. Wenn *windowList* angegeben ist, besteht sie aus einer Liste von Fensterpfadnamen; der Befehl überschreibt die **WM_COLORMAP_WINDOWS**-Eigenschaft mit den angegebenen Fenstern und gibt einen leeren String zurück. Die **WM_COLORMAP_WINDOWS**-Eigenschaft sollte normalerweise eine Liste der internen Fenster innerhalb von *window* enthalten, deren Farbtabellen sich von denen ihrer Eltern unterscheiden.

Die Reihenfolge der Fenster in der Eigenschaft gibt eine Prioritätsreihenfolge an: Der Fenstermanager versucht, so viele Farbtabellen wie möglich vom Anfang dieser Liste zu installieren, wenn *window* den Colormap-Fokus erhält. Wenn *window* nicht in *windowList* enthalten ist, fügt Tk es implizit am Ende der **WM_COLORMAP_WINDOWS**-Eigenschaft hinzu, sodass seine Farbtabelle die niedrigste Priorität hat. Wenn **wm colormapwindows** nicht aufgerufen wird, setzt Tk die Eigenschaft für jedes Toplevel-Fenster automatisch auf alle internen Fenster, deren Farbtabellen sich von denen ihrer Eltern unterscheiden, gefolgt vom Toplevel-Fenster selbst; die Reihenfolge der internen Fenster ist undefiniert. Siehe die ICCCM-Dokumentation zur **WM_COLORMAP_WINDOWS**-Eigenschaft.

**wm command *window* ?*value*?**

Wenn *value* angegeben ist, speichert dieser Befehl *value* in der **WM_COMMAND**-Eigenschaft von *window* für den Fenstermanager oder Sitzungsmanager und gibt einen leeren String zurück. *Value* muss eine gültige Listenstruktur haben; die Elemente sollten die Wörter des Befehls enthalten, mit dem die Anwendung gestartet wurde. Wenn *value* nicht angegeben ist, gibt der Befehl den zuletzt in einem **wm command**-Befehl für *window* gesetzten Wert zurück. Wenn *value* als leerer String angegeben wird, löscht der Befehl die **WM_COMMAND** -Eigenschaft von *window*.

**wm deiconify *window***

Sorgt dafür, dass *window* in normaler (nicht iconifizierter) Form angezeigt wird. Dies geschieht durch das Mapping des Fensters. Wenn das Fenster noch nie gemappt wurde, mappt dieser Befehl das Fenster nicht, stellt aber sicher, dass es beim ersten Mapping de-iconifiziert angezeigt wird. Unter Windows wird ein de-iconifiziertes Fenster außerdem in den Vordergrund geholt und erhält den Fokus (wird zum aktiven Fenster). Gibt einen leeren String zurück.

**wm focusmodel *window* ?active|passive?**

Wenn **active** oder **passive** als optionales Argument übergeben wird, legt es das Fokusmodell für *window* fest; der Befehl gibt dann einen leeren String zurück. Wenn kein weiteres Argument übergeben wird, gibt der Befehl das aktuelle Fokusmodell für *window* zurück.

Ein **active**-Fokusmodell bedeutet, dass *window* den Eingabefokus für sich oder seine

Nachkommen beansprucht, auch wenn der Fokus gerade in einer anderen Anwendung liegt. **Passive** bedeutet, dass *window* den Fokus nie für sich beansprucht: Der Fenstermanager soll *window* zu passenden Zeiten den Fokus geben. Sobald der Fokus *window* oder einem seiner Nachkommen gegeben wurde, kann die Anwendung den Fokus unter den Nachkommen von *window* neu zuweisen. Das Fokusmodell ist standardmäßig **passive**; Tks focus-Befehl setzt ein passives Fokusmodell voraus.

**wm forget *window***

*window* wird vom Bildschirm unmappt und nicht mehr von **wm** verwaltet. Mit dem toplevel -Befehl erstellte Fenster werden wie frame-Fenster behandelt, sobald sie nicht mehr von **wm** verwaltet werden; die **-menu**-Konfiguration wird jedoch gemerkt, und die Menüs kehren zurück, sobald das Widget wieder verwaltet wird.

**wm frame *window***

Wenn *window* vom Fenstermanager in einen dekorativen Rahmen umgeparentet wurde, gibt der Befehl die plattformspezifische Fensterkennung des äußersten Rahmens zurück, der *window* enthält (das Fenster, dessen Eltern das Root- oder virtuelle Root-Fenster ist). Wenn *window* nicht umgeparentet wurde, gibt der Befehl die plattformspezifische Fensterkennung für *window* zurück.

**wm geometry *window* ?*newGeometry*?**

Wenn *newGeometry* angegeben ist, wird die Geometrie von *window* geändert und ein leerer String zurückgegeben. Andernfalls wird die aktuelle Geometrie von *window* zurückgegeben (die zuletzt durch manuelles Skalieren oder einen **wm geometry**-Befehl gesetzte Geometrie). *NewGeometry* hat die Form **=***width***x***height***±***x***±***y*; **=**, *width***x***height* oder **±***x***±***y* können weggelassen werden. *Width* und *height* sind positive Ganzzahlen für die gewünschten Abmessungen von *window*. Ist *window* gitterbasiert (siehe Gitterbasierte Geometrieverwaltung unten), sind die Abmessungen in Gittereinheiten angegeben; sonst in Pixeln.

*X* und *y* legen die gewünschte Position von *window* auf dem Bildschirm in Pixeln fest. Steht *x* vor **+**, ist es der Abstand zwischen dem linken Bildschirmrand und dem linken Rand von *window*; bei **-** der Abstand zwischen dem rechten Bildschirmrand und dem rechten Rand von *window*. Steht *y* vor **+**, ist es der Abstand zwischen dem oberen Bildschirmrand und dem oberen Rand von *window*; bei **-** der Abstand zwischen dem unteren Rand von *window* und dem unteren Bildschirmrand.

Wird *newGeometry* als leerer String angegeben, werden benutzerdefinierte Geometrie-Vorgaben für *window* aufgehoben; das Fenster kehrt zur intern von seinen Widgets angeforderten Größe zurück.

Beachten Sie den Zusammenhang mit winfo geometry, der aber nicht identisch ist: **winfo geometry** kann nur abfragen und spiegelt stets Tks aktuelles Verständnis der tatsächlichen Größe und Position von *window* wider, während **wm geometry** sowohl setzen als auch die *Fenstermanager*-Sicht auf Größe und Position abfragen kann. Das kann deutlich abweichen, z. B. durch Titelleisten; Fenstermanager müssen Anfragen über diesen Befehl nicht exakt befolgen.

**wm grid *window* ?*baseWidth baseHeight widthInc heightInc*?**

Dieser Befehl legt fest, dass *window* als gitterbasiertes Fenster verwaltet wird, und gibt das

Verhältnis zwischen Gitter- und Pixeleinheiten an. *BaseWidth* und *baseHeight* sind die Gittereinheiten, die den intern per Tk_GeometryRequest angeforderten Pixelabmessungen von *window* entsprechen. *WidthInc* und *heightInc* sind die Pixel pro horizontaler bzw. vertikaler Gittereinheit. Diese vier Werte bestimmen einen zulässigen Größenbereich für *window* (nicht-negative ganzzahlige Gitterbreiten und -höhen). Tk übergibt diese Information an den Fenstermanager; beim manuellen Skalieren beschränkt der Fenstermanager die Größe auf zulässige Werte.

Beim manuellen Skalieren zeigt der Fenstermanager die aktuelle Größe in Gittereinheiten statt in Pixeln. Wenn *baseWidth* usw. alle als leere Strings angegeben sind, wird *window* nicht mehr gitterbasiert verwaltet. Wenn *baseWidth* usw. angegeben sind, ist der Rückgabewert ein leerer String.

Andernfalls ist der Rückgabewert eine Tcl-Liste mit vier Elementen für die aktuellen *baseWidth* , *baseHeight*, *widthInc* und *heightInc*; ist *window* derzeit nicht gitterbasiert, ein leerer String.

Dieser Befehl wird selten benötigt, da Tk_SetGrid und die **setGrid**-Option einfacheren Zugang bieten.

**wm group *window* ?*pathName*?**

Wenn *pathName* angegeben ist, ist es der Pfadname des Leaders einer Gruppe verwandter Fenster. Der Fenstermanager kann diese Information nutzen, z. B. um alle Fenster einer Gruppe zu unmapen, wenn der Leader iconifiziert wird. *PathName* kann als leerer String angegeben werden, um *window* aus jeder Gruppenzuordnung zu entfernen. Wenn *pathName* angegeben ist, gibt der Befehl einen leeren String zurück; sonst den Pfadnamen des aktuellen Gruppenleaders von *window* oder einen leeren String, wenn *window* zu keiner Gruppe gehört.

**wm iconbadge *window badge***

Setzt ein Badge für das Icon von *window*. Das Badge kann eine positive Ganzzahl sein, z. B. die Anzahl neuer oder ungelesener Nachrichten, oder ein Ausrufezeichen für erforderliche Aufmerksamkeit. Ist das Badge ein leerer String, wird das Badge-Bild vom Anwendungsicon entfernt. Die Verwaltung solcher Änderungen über Bindings wie liegt beim Entwickler.

Unter X11 muss für diesen Befehl die Variable **::tk::icons::base_icon($window)** auf das Bild gesetzt sein, das als Fenstericon von $window verwendet wird. Unter Windows und X11 funktionieren iconphoto-Bilder am besten bei 32×32 oder ähnlichen Abmessungen, da Tk die Badge-Bilder bereitstellt und per nativer API (Windows) oder Tk-Rendering über die Icons legt. Unter macOS rendert das System-API das Icon-Badge; Tk stellt es nicht bereit. Das Icon selbst sollte höher aufgelöst sein, vorzugsweise 512 Pixel, um Unschärfe zu vermeiden.

Das Icon-Badge ist für Dock (macOS), Taskleiste (Windows) oder App-Panel (X11) gedacht. Unter macOS wird das zuletzt gesetzte Badge im Dock angezeigt, unabhängig davon, wie viele verschiedene Icon-Badges verschiedenen Fenstern zugewiesen sind. Unter Windows hängt die Taskleistenanzeige davon ab, ob Taskleisten-Schaltflächen kombiniert sind (OS-Einstellung): kombiniert wie unter macOS, sonst zeigt jede Schaltfläche ihr zugewiesenes Badge. Die Badge-Anzeige unter macOS wird in den Systemeinstellungen konfiguriert. Unter X11 hängt das Verhalten vom Fenstermanager und/oder der Desktop-Umgebung ab.

**wm iconbitmap *window* ?*bitmap*?**

Wenn *bitmap* angegeben ist, benennt es ein Bitmap in den von Tk akzeptierten Standardformen (siehe Tk_GetBitmap). Dieses Bitmap wird dem Fenstermanager für das Icon von *window* übergeben; der Befehl gibt einen leeren String zurück. Bei leerem *bitmap* wird ein bestehendes Icon-Bitmap für *window* aufgehoben. Wenn *bitmap* angegeben ist, gibt der Befehl einen leeren String zurück; sonst den Namen des aktuellen Icon-Bitmaps von *window* oder einen leeren String, wenn keines gesetzt ist. Unter Windows wird zusätzlich unterstützt:

**wm iconbitmap *window* ?-default? ?*image*?**

Mit dem Flag **-default** gilt das Icon für alle Toplevel-Fenster (bestehende und künftige), denen noch kein anderes spezifisches Icon zugewiesen wurde. Neben Bitmap-Typen wird auch ein vollständiger Pfad zu einer Datei mit gültigem Windows-Icon akzeptiert (üblicherweise .ico oder .icr) oder zu einer Datei, der die Shell ein Icon zugewiesen hat. Tcl prüft zuerst, ob die Datei ein Icon enthält, dann ob ein Icon zugewiesen ist, und schließlich, ob es ein Bitmap ist.

**wm iconify *window***

Sorgt dafür, dass *window* iconifiziert wird. Wenn *window* noch nie zum ersten Mal gemappt wurde, wird es beim späteren Mapping im iconifizierten Zustand erscheinen.

**wm iconmask *window* ?*bitmap*?**

Wenn *bitmap* angegeben ist, benennt es ein Bitmap in den von Tk akzeptierten Standardformen (siehe Tk_GetBitmap). Dieses Bitmap wird dem Fenstermanager als Maske zur **iconbitmap**-Option übergeben: wo die Maske Nullen hat, wird kein Icon angezeigt; wo Einsen, die Bits des Icon-Bitmaps. Bei leerem *bitmap* wird eine bestehende Icon-Maske für *window* aufgehoben (entspricht einem Bitmap aus lauter Einsen). Wenn *bitmap* angegeben ist, gibt der Befehl einen leeren String zurück; sonst den Namen der aktuellen Icon-Maske von *window* oder einen leeren String, wenn keine Maske gilt.

**wm iconname *window* ?*newName*?**

Wenn *newName* angegeben ist, wird es an den Fenstermanager übergeben; der Fenstermanager soll *newName* im mit *window* verknüpften Icon anzeigen. In diesem Fall ist das Ergebnis ein leerer String. Wenn *newName* nicht angegeben ist, gibt der Befehl den aktuellen Icon-Namen von *window* zurück oder einen leeren String, wenn keiner gesetzt wurde (dann zeigt der Fenstermanager üblicherweise den mit **wm title** gesetzten Fenstertitel).

**wm iconphoto *window* ?-default? *image1* ?*image2* ...?**

Setzt das Titelleisten-Icon für *window* anhand der genannten Foto-Images. Mit **-default** gilt dies auch für alle künftig erstellten Toplevel-Fenster. Die Bilddaten werden zum Zeitpunkt des Aufrufs als Momentaufnahme genommen; spätere Änderungen an den Images wirken sich nicht auf die Titelleisten-Icons aus. Mehrere Images erlauben verschiedene Größen (z. B. 16×16 und 32×32). Der Fenstermanager kann Icons auf eine passende Größe skalieren.

Unter Windows werden die Images in eine Windows-Icon-Struktur gepackt. Das überschreibt ein per **wm iconbitmap** gesetztes .ico und umgekehrt. Dieser Befehl setzt auch das Taskleisten-Icon des Fensters.

Unter X werden die Images in die X-Eigenschaft _NET_WM_ICON gelegt, die die meisten modernen Fenstermanager unterstützen. Ein **wm iconbitmap** kann gleichzeitig existieren. Es wird empfohlen, nicht mehr als zwei Icons zu verwenden und das größere zuerst anzugeben.

Dieser Befehl setzt auch das Panel-Icon der Anwendung, wenn der Fenstermanager oder die Desktop-Umgebung das unterstützt.

Unter Macintosh wird das erste aufgerufene Image in ein natives OS-Icon-Format geladen und wird Anwendungsicon in Dialogen, im Dock und in anderen Kontexten. Auf Skriptebene akzeptiert der Befehl nur das erste übergebene Image, da Mehrfachgrößen/-auflösungen unter macOS außerhalb des Tk-Bereichs liegen. Entwickler sollten das größtmögliche unterstützte Icon verwenden (vorzugsweise 512 Pixel).

**wm iconposition *window* ?*x y*?**

Wenn *x* und *y* angegeben sind, werden sie dem Fenstermanager als Hinweis für die Icon-Position von *window* übergeben; der Befehl gibt einen leeren String zurück. Als leere Strings werden bestehende Icon-Positionshinweise aufgehoben. Wenn weder *x* noch *y* angegeben sind, gibt der Befehl eine Tcl-Liste mit zwei Werten zurück (aktuelle Icon-Positionshinweise; ohne Hinweise ein leerer String).

**wm iconwindow *window* ?*pathName*?**

Wenn *pathName* angegeben ist, ist es der Pfadname eines Fensters, das als Icon für *window* dient: bei Iconifizierung von *window* wird *pathName* gemappt; bei De-Iconifizierung wieder unmappt. Als leerer String wird eine bestehende Icon-Fenster-Zuordnung für *window* aufgehoben. Wenn *pathName* angegeben ist, gibt der Befehl einen leeren String zurück; sonst den Pfadnamen des aktuellen Icon-Fensters von *window* oder einen leeren String. Button-Press-Ereignisse sind für *window* deaktiviert, solange es ein Icon-Fenster ist, damit Fenstermanager diese Ereignisse "besitzen" können. Nicht alle Fenstermanager unterstützen Icon-Fenster; auf Nicht-X11-Plattformen ist das Konzept bedeutungslos.

**wm manage *widget***

Das angegebene *widget* wird ein eigenständiges Toplevel-Fenster. Es wird mit Titelleiste usw. des Fenstermanagers dekoriert. Nur *frame*-, *labelframe*- und *toplevel*-Widgets sind zulässig; andere Widget-Typen lösen einen Fehler aus. Ein *toplevel*-Widget zu verwalten ist harmlos und bewirkt nichts. Siehe auch Geometrieverwaltung.

**wm maxsize *window* ?*width height*?**

Wenn *width* und *height* angegeben sind, legen sie die maximal zulässigen Abmessungen von *window* fest. Bei gitterbasierten Fenstern in Gittereinheiten, sonst in Pixeln. Der Fenstermanager beschränkt die Abmessungen auf höchstens *width* und *height*. Bei Angabe gibt der Befehl einen leeren String zurück; sonst eine Tcl-Liste mit den aktuell geltenden Maximalbreite und -höhe. Standard ist die Bildschirmgröße. Siehe die Abschnitte zur Geometrieverwaltung unten.

**wm minsize *window* ?*width height*?**

Wenn *width* und *height* angegeben sind, legen sie die minimal zulässigen Abmessungen von *window* fest. Bei gitterbasierten Fenstern in Gittereinheiten, sonst in Pixeln. Der Fenstermanager beschränkt die Abmessungen auf mindestens *width* und *height*. Bei Angabe gibt der Befehl einen leeren String zurück; sonst eine Tcl-Liste mit Minimalbreite und -höhe. Standard ist ein Pixel pro Dimension. Siehe die Abschnitte zur Geometrieverwaltung unten.

**wm overrideredirect *window* ?*boolean*?**

Wenn *boolean* angegeben ist, muss es eine gültige boolesche Form haben; das Override-Redirect-Flag von *window* wird gesetzt. Ohne *boolean* wird **1** oder **0** zurückgegeben. Das Setzen des Override-Redirect-Flags bewirkt, dass der Fenstermanager das Fenster ignoriert; u. a. wird es nicht vom Root-Fenster in einen dekorativen Rahmen umgeparentet, und der Benutzer kann es nicht mit den üblichen Fenstermanager-Mechanismen manipulieren.

Der Fenstermanager gibt einem solchen Fenster auch nicht den Tastaturfokus. Unter X11 muss eine Anwendung, die darin Tastenereignisse braucht, den Fokus mit focus -force anfordern.

Das Override-Redirect-Flag wird garantiert beim ersten Mapping oder beim Mapping nach Wechsel von withdrawn zu normal beachtet. Einige, aber nicht alle Plattformen beachten es auch zu anderen Zeiten.

**wm positionfrom *window* ?*who*?**

Wenn *who* angegeben ist, muss es **program** oder **user** sein (oder eine Abkürzung). Es gibt an, ob die aktuelle Position von *window* vom Programm oder vom Benutzer angefordert wurde. Viele Fenstermanager ignorieren programmgesteuerte Anfangspositionen; bei **user** soll der Fenstermanager das Fenster an der angegebenen Stelle platzieren, ohne den Benutzer zu fragen. Als leerer String wird die aktuelle Positionsquelle aufgehoben. Bei Angabe von *who* gibt der Befehl einen leeren String zurück; sonst **user** oder **program** oder einen leeren String, wenn noch keine Quelle gesetzt wurde. Die meisten Fenstermanager werten "keine Quelle" wie **program**. Tk setzt die Positionsquelle beim Aufruf von **wm geometry** automatisch auf **user**, sofern sie nicht explizit **program** ist.

**wm protocol *window* ?*name*? ?*command*?**

Dieser Befehl verwaltet Fenstermanager-Protokolle. Das *name*-Argument in **wm protocol** ist der Name eines Atoms, das einem Fenstermanager-Protokoll entspricht. Beispiele: **WM_DELETE_WINDOW**, **WM_SAVE_YOURSELF** oder **WM_TAKE_FOCUS**.

Ein *Fenstermanager-Protokoll* ist eine Klasse von Nachrichten vom Fenstermanager an eine Tk-Anwendung außerhalb des normalen Ereignisverarbeitungssystems. Das Hauptbeispiel ist **WM_DELETE_WINDOW**; diese Nachrichten werden gesendet, wenn der Benutzer das Schließen-Widget in der Titelleiste anklickt. Handler werden mit **wm protocol** installiert. Regelmäßig werden alle Nachrichten eines Protokolls ignoriert, wenn kein Handler per **wm protocol** installiert wurde. **WM_DELETE_WINDOW** ist eine Ausnahme: Beim Start installiert Tk einen Handler, der das Fenster zerstört. **wm protocol** kann diesen Standard-Handler ersetzen.

Die verfügbaren Protokolle hängen vom Fenstermanager ab; alle von Tk unterstützten Fenstermanager bieten **WM_DELETE_WINDOW**. Unter Windows wird bei Abmeldung oder Neustart **WM_SAVE_YOURSELF** gesendet.

Wenn *name* und *command* angegeben sind, wird *command* zum Handler für das Protokoll *name*. Das Atom für *name* wird der **WM_PROTOCOLS**-Eigenschaft von *window* hinzugefügt; *command* wird aufgerufen, wenn der Fenstermanager eine Nachricht dieses Protokolls sendet. **wm protocol** gibt dann einen leeren String zurück. Wenn nur *name* angegeben ist, wird der aktuelle Handler für *name* zurückgegeben oder ein leerer String (der Standard-Handler für **WM_DELETE_WINDOW** wird nicht zurückgegeben). Ist *command* ein leerer String, wird das Atom für *name* aus **WM_PROTOCOLS** entfernt und der Handler zerstört; Rückgabe leerer String. Wenn weder *name* noch *command* angegeben sind, gibt **wm protocol** eine Liste aller

Protokolle zurück, für die Handler für *window* definiert sind.

**wm resizable *window* ?*width height*?**

Dieser Befehl steuert, ob der Benutzer ein Toplevel-Fenster interaktiv skalieren darf. Wenn *width* und *height* angegeben sind, sind es boolesche Werte, ob Breite und Höhe von *window* geändert werden dürfen; der Befehl gibt einen leeren String zurück. Ohne *width* und *height* eine Liste mit zwei 0/1-Elementen. Standardmäßig sind Fenster in beiden Dimensionen skalierbar. Ist Skalierung deaktiviert, gilt die Größe der letzten interaktiven Skalierung oder des letzten **wm geometry**-Befehls; sonst die natürliche Größe.

**wm sizefrom *window* ?*who*?**

Wenn *who* angegeben ist, muss es **program** oder **user** sein (oder eine Abkürzung). Es gibt an, ob die aktuelle Größe von *window* vom Programm oder vom Benutzer angefordert wurde. Einige Fenstermanager ignorieren programmgesteuerte Größen; bei **user** soll der Fenstermanager die angegebene Größe setzen, ohne den Benutzer zu fragen. Als leerer String wird die aktuelle Größenquelle aufgehoben. Bei Angabe von *who* gibt der Befehl einen leeren String zurück; sonst **user** oder **window** oder einen leeren String. Die meisten Fenstermanager werten "keine Quelle" wie **program**.

**wm stackorder *window* ?*isabove*|*isbelow window*?**

Der **stackorder**-Befehl gibt eine Liste von Toplevel-Fenstern in Stapelreihenfolge zurück, von unten nach oben. Bei einem einzelnen Toplevel-Fenster enthält die Liste rekursiv alle Toplevel-Nachkommen des Fensters. Nur derzeit auf dem Bildschirm gemappte Toplevels werden zurückgegeben. Mit zwei Fensterargumenten, getrennt durch **isabove** oder **isbelow**, liefert ein boolesches Ergebnis, ob das erste Fenster über oder unter dem zweiten in der Stapelreihenfolge liegt.

**wm state *window* ?*newstate*?**

Wenn *newstate* angegeben ist, wird das Fenster in den neuen Zustand versetzt; sonst gibt der Befehl den aktuellen Zustand von *window* zurück: **normal**, **iconic**, **withdrawn**, **icon** oder (nur Windows und macOS) **zoomed**. Der Unterschied zwischen **iconic** und **icon**: **iconic** bezeichnet ein iconifiziertes Fenster (z. B. per **wm iconify**), **icon** ein Fenster, das nur als Icon für ein anderes dient (per **wm iconwindow**). Der Zustand **icon** kann nicht gesetzt werden.

**wm title *window* ?*string*?**

Wenn *string* angegeben ist, wird er dem Fenstermanager als Titel für *window* übergeben (Anzeige in der Titelleiste). Der Befehl gibt einen leeren String zurück. Ohne *string* gibt er den aktuellen Titel von *window* zurück. Standard ist der Name des Fensters.

**wm transient *window* ?*container*?**

Wenn *container* angegeben ist, wird der Fenstermanager informiert, dass *window* ein transientes Fenster ist (z. B. Pull-down-Menü), das im Auftrag von *container* arbeitet ( *container* ist der Pfadname eines Toplevel-Fensters). Als leerer String wird *window* nicht mehr als transient markiert. Sonst gibt der Befehl den Pfadnamen des aktuellen Containers von *window* zurück oder einen leeren String. Ein transientes Fenster spiegelt Zustandsänderungen des Containers und erbt beim ersten Mapping dessen Zustand. Der gerichtete Graph mit einer Kante von jedem Transienten zu seinem Container muss azyklisch sein; ein Fenster darf nicht transient von sich selbst sein. Der Fenstermanager kann transiente Fenster anders dekorieren (z. B. ohne Minimieren/Maximieren), nach eigenem Ermessen.

**wm withdraw *window***

Sorgt dafür, dass *window* vom Bildschirm zurückgezogen wird. Das Fenster wird unmappt und vom Fenstermanager nicht mehr berücksichtigt. Wenn es noch nie gemappt wurde, wird es im withdrawn-Zustand gemappt. Nicht alle Fenstermanager scheinen zu wissen, wie mit Fenstern umzugehen ist, die im withdrawn-Zustand gemappt sind. Manchmal muss ein Fenster zurückgezogen und neu gemappt werden (z. B. mit **wm deiconify**), damit Fenstermanager auf Änderungen an Fensterattributen wie **group** reagieren.

## Geometrieverwaltung

Standardmäßig erscheint ein Toplevel-Fenster in seiner *natürlichen Größe*, die intern von seinen Widgets und Geometriemanagern bestimmt wird. Ändert sich die natürliche Größe, passt sich die Fenstergröße an. Eine andere Größe ist auf zwei Wegen möglich: der Benutzer skaliert manuell (z. B. Griffe des Fenstermanagers) oder die Anwendung fordert eine Größe per **wm geometry** an. Tk behandelt beide Fälle gleich; die angeforderte Größe setzt die natürliche Größe außer Kraft. Zurück zur natürlichen Größe: **wm geometry** mit leerer *geometry* -Zeichenkette.

Normalerweise kann ein Toplevel-Fenster von einem Pixel pro Dimension bis zur Bildschirmgröße skaliert werden. Mit **wm minsize** und **wm maxsize** lässt sich der zulässige Bereich begrenzen. Der Bereich gilt für alle Skalierungsarten, einschließlich natürlicher Größe, manueller Skalierung und **wm geometry**. Jeder von Tk_GetPixels akzeptierte Wert ist möglich. Mit **wm resizable** kann interaktive Skalierung in einer oder beiden Dimensionen vollständig deaktiviert werden.

Die Befehle **wm manage** und **wm forget** können zum Abdocken und Andocken von Fenstern verwendet werden. Nach **wm manage** können alle anderen **wm**-Unterbefehle mit dem Widget verwendet werden. Nur mit dem toplevel-Befehl erstellte Widgets können ein Menü über die **-menu**-Konfigurationsoption haben. Ein Toplevel-Widget kann nach **wm forget** wie ein Frame mit anderen Geometriemanagern verwaltet werden. Ein mit einem Toplevel verknüpftes Menü wird verborgen, wenn ein anderer Geometriemanager verwendet wird; es erscheint wieder, sobald das Fenster von **wm** verwaltet wird. Alle angepassten bindtags für Widgets in einem Teilbaum, deren Toplevel per **wm manage** oder **wm forget** geändert wurde, müssen neu gesetzt werden, um Toplevel-Pfade in den bindtags anzupassen. Nicht angepasste bindtags müssen nicht neu gesetzt werden.

## Gitterbasierte Geometrieverwaltung

Gitterbasierte Geometrieverwaltung tritt auf, wenn ein Widget der Anwendung einen Bereich sinnvoller Größen unterstützt, z. B. in einem Texteditor mit festen Scrollleisten und Menüs, während das Bearbeitungs-Widget beliebig viele Zeilen oder Zeichen pro Zeile erlaubt. Dann soll der Benutzer Zeilen- oder Zeichen-pro-Zeile-Anzahl per **wm geometry** oder interaktiver Skalierung wählen können. Oft sind nur diskrete Fenstergrößen sinnvoll (ganzzahlige Zeilen und Zeichen pro Zeile); beliebige Pixelgrößen sind nicht nützlich.

Gitterbasierte Geometrieverwaltung unterstützt solche Anwendungen. Tk (und der Fenstermanager) gehen von einem Gitter in der Anwendung aus und skalieren in *Gittereinheiten* statt in Pixeln. Typischerweise durch Einschalten der **setGrid**-Option eines Widgets; auch per **wm grid** oder Tk_SetGrid. Das Widget (oder Anwendungscode) legt das Verhältnis zwischen ganzzahligen Gittergrößen und Pixeln fest. Zurück zur nicht gitterbasierten Geometrieverwaltung: **wm grid** mit leeren Argumentstrings.

Ist gitterbasierte Geometrieverwaltung aktiv, gelten alle in **wm minsize**, **wm maxsize** und **wm geometry** angegebenen Abmessungen als Gittereinheiten. Interaktive Skalierung erfolgt in geraden Vielfachen von Gittereinheiten.

### Fehler

Die meisten bestehenden Fenstermanager scheinen Fehler zu haben, die den **wm**-Befehl betreffen. Einige Änderungen wirken nicht, wenn das Fenster bereits aktiv ist: Das Fenster muss zurückgezogen und de-iconifiziert werden, damit die Änderung greift.

### Beispiele

Ein Fenster fester Größe, das dies auch angibt:

```
toplevel .fixed
wm title     .fixed "Fixed-size Window"
wm resizable .fixed 0 0
```

Ein einfaches dialogähnliches Fenster, zentriert auf dem Bildschirm:

```
# Dialoginhalt erstellen und anordnen.
toplevel .msg
label  .msg.l  -text "This is a very simple dialog demo."
button .msg.ok -text OK -default active -command {destroy .msg}
pack .msg.ok -side bottom -fill x
pack .msg.l  -expand 1    -fill both

# Widget als zentrierten Dialog einrichten.

# Zuerst müssen die Geometriemanager den Dialog-Innenraum
# fertig einrichten; dazu die Ereignisschleife laufen lassen,
# während das Widget vollständig verborgen ist...
wm withdraw .msg
update
set x [expr {([winfo screenwidth .] - [winfo width .msg]) / 2}]
set y [expr {([winfo screenheight .] - [winfo height .msg]) / 2}]
wm geometry  .msg +$x+$y
wm transient .msg .
wm title     .msg "Dialog demo"
wm deiconify .msg
```

*Copyright (c) 1991-1994 The Regents of the University of California.*
*Copyright (c) 1994-1996 Sun Microsystems, Inc.*

# Ttk

## ttk_button

*Stichwörter:* ttk_button, widget, button, default, command

### Name

ttk::button - Widget, das beim Drücken einen Befehl ausführt

## Übersicht

**ttk::button** *pathName* ?*options*?

## Beschreibung

Ein **ttk::button**-Widget zeigt eine Textbeschriftung und/oder ein Bild an und wertet beim Drücken einen Befehl aus.

## Standardoptionen

**-class**, **-compound**, **-cursor**, **-image**, **-justify**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-command (command/Command)**

Legt ein Skript fest, das ausgewertet wird, wenn das Widget aufgerufen wird.

**-default (default/Default)**

Kann auf einen von **normal**, **active** oder **disabled** gesetzt werden. In einem Dialogfeld kann ein Button als "Standard"-Button bezeichnet werden (gemeint ist ungefähr "derjenige, der aufgerufen wird, wenn der Benutzer drückt"). **active** bedeutet, dass dies derzeit der Standard-Button ist; **normal** bedeutet, dass er der Standard-Button werden kann, und **disabled** bedeutet, dass er nicht als Standard festgelegt werden kann. Der Standardwert ist **normal**.

Je nach Theme kann der Standard-Button mit einem zusätzlichen Hervorhebungsring oder mit einer anderen Rahmenfarbe angezeigt werden.

## Widget-Befehl

Zusätzlich zu den Standardbefehlen **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget) unterstützen Button-Widgets folgende zusätzliche Befehle:

***pathName* invoke**

Ruft den mit dem Button verknüpften Befehl auf.

## Standard-Stile

**Ttk::button**-Widgets unterstützen den Stil **Toolbutton** in allen Standard-Themes, was nützlich ist, um Widgets für Symbolleisten zu erstellen.

Im Aqua-Theme gibt es mehrere weitere Stile, mit denen Nachbildungen vieler der verschiedenen Button-Typen erzeugt werden können, die in Apples Human Interface Guidelines beschrieben sind. Dazu gehören **DisclosureButton**, **DisclosureTriangle**, **HelpButton**, **ImageButton**, **InlineButton**, **GradientButton**, **RoundedRectButton** und **RecessedButton**.

## Styling-Optionen

Der Klassenname für ein **ttk::button** ist **TButton**.

Dynamische Zustände: **active**, **disabled**, **pressed**, **readonly**.

**TButton**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-anchor** *anchor*

**-background** *color*

**-bordercolor** *color*

**-compound** *compound*

**-darkcolor** *color*

**-foreground** *color*

**-font** *font*

**-highlightcolor** *color*

**-highlightthickness** *amount*

**-justify** *justification*

**-lightcolor** *color*

**-padding** *padding*

**-relief** *relief*

**-shiftrelief *amount***

**-shiftrelief** legt fest, wie weit der Button-Inhalt im Zustand *pressed* nach unten und rechts verschoben wird. Diese Aktion liefert zusätzliches skeuomorphes Feedback.

**-width** *amount*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2004 Joe English*

# ttk_checkbutton

*Stichwörter:* ttk_checkbutton, widget, button, toggle, check, option

## Name

ttk::checkbutton - Ein/Aus-Widget

## Übersicht

**ttk::checkbutton** *pathName* ?*options*?

## Beschreibung

Ein **ttk::checkbutton**-Widget zeigt eine Einstellung an oder ändert sie. Es hat zwei Zustände: ausgewählt und abgewählt. Der Zustand kann mit einer Tcl-Variable verknüpft sein.

## Standardoptionen

**-class**, **-compound**, **-cursor**, **-image**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-command (command/Command)**

Tcl-Skript bei jedem Aufruf des Widgets.

**-offvalue (offValue/OffValue)**

Wert in der zugehörigen **-variable**, wenn das Widget abgewählt ist. Standard **0**.

**-onvalue (onValue/OnValue)**

Wert in der zugehörigen **-variable**, wenn das Widget ausgewählt ist. Standard **1**.

**-variable (variable/Variable)**

Name einer globalen Variable, deren Wert mit dem Widget verknüpft ist. Standard ist der Widget-Pfadname, falls nicht angegeben.

## Widget-Befehl

Zusätzlich zu **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget) unterstützen Checkbutton-Widgets:

***pathname*** **invoke**

Wechselt zwischen ausgewählt und abgewählt und wertet **-command** aus. Ist das Widget ausgewählt, setzt **-variable** auf **-offvalue** und wählt ab; andernfalls **-onvalue**. Liefert das Ergebnis von **-command**.

## Widget-Zustände

Bei **disabled** reagiert das Widget nicht auf Benutzereingaben. **selected** wird gesetzt, wenn **-variable** dem **-onvalue** entspricht, sonst gelöscht. **alternate** wird gesetzt, wenn **-variable** nicht gesetzt ist (Drei-Zustand / unbestimmt).

## Standardstile

**Ttk::checkbutton**-Widgets unterstützen in allen Standard-Themes den Stil **Toolbutton** — nützlich für Symbolleisten.

## Styling-Optionen

Der Klassenname für ein **ttk::checkbutton** ist **TCheckbutton**.

Dynamische Zustände: **active**, **alternate**, **disabled**, **pressed**, **selected**, **readonly**.

**TCheckbutton**-Styling-Optionen, die mit ttk::style konfiguriert werden können:

**-background** *color*

**-compound** *compound*

**-foreground** *color*

**-indicatorbackground** *color*

**-indicatorcolor** *color*

**-indicatormargin** *padding*

**-indicatorrelief** *relief*

**-padding** *padding*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2004 Joe English*

# ttk_combobox

*Stichwörter:* ttk_combobox, choice, entry, list box, text box, widget

## Name

ttk::combobox - Textfeld mit ausklappbarer Auswahlliste

## Übersicht

**ttk::combobox** *pathName* ?*options*?

## Beschreibung

Ein **ttk::combobox** kombiniert ein Textfeld mit einer ausklappbaren Liste von Werten; der Benutzer kann den Wert des Textfelds aus den Einträgen der Liste wählen.

## Standardoptionen

**-class**, **-cursor**, **-takefocus**, **-style**, **-placeholder**, **-placeholderforeground**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-exportselection (exportSelection/ExportSelection)**

Boolescher Wert. Ist er gesetzt, ist die Widget-Auswahl mit der X-Auswahl verknüpft.

**-justify (justify/Justify)**

Legt fest, wie der Text im Widget ausgerichtet ist. Muss einer von **left**, **center** oder **right** sein.

**-height (height/Height)**

Legt die Höhe der ausklappbaren Listbox in Zeilen fest.

**-postcommand (postCommand/PostCommand)**

Tcl-Skript, das unmittelbar vor dem Anzeigen der Listbox ausgewertet wird. Das Skript **-postcommand** kann die anzuzeigenden **-values** festlegen.

**-state (state/State)**

Einer von **normal**, **readonly** oder **disabled**. Im Zustand **readonly** kann der Wert nicht direkt bearbeitet werden; der Benutzer kann nur einen der **-values** aus der Dropdown-Liste wählen. Im Zustand **normal** ist das Textfeld direkt editierbar. Im Zustand **disabled** ist keine Interaktion möglich.

**-textvariable (textVariable/TextVariable)**

Name einer globalen Variable, deren Wert mit dem Widget-Wert verknüpft ist. Ändert sich die Variable, wird der Widget-Wert aktualisiert und umgekehrt.

**-values (values/Values)**

Liste der in der Dropdown-Listbox anzuzeigenden Werte.

**-width (width/Width)**

Ganzzahl für die gewünschte Breite des Eingabefensters in Zeichen durchschnittlicher Größe der Widget-Schriftart.

## Widget-Befehl

Zusätzlich zu den Standardbefehlen **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget) unterstützen Combobox-Widgets folgende zusätzliche Befehle:

***pathName* current ?*newIndex*?**

Wird *newIndex* angegeben, setzt der Befehl den Combobox-Wert auf das Element an Position *newIndex* in der Liste **-values** (neben Ganzzahlen wird der Index **end** unterstützt und bezeichnet das letzte Element der Liste; außerdem gilt dieselbe einfache Interpretation wie beim Befehl string index, mit einfacher ganzzahliger Indexarithmetik und Indizierung relativ zu **end**). Andernfalls gibt er den Index des aktuellen Werts in der Liste **-values** zurück oder **{}**, wenn der aktuelle Wert nicht in der Liste vorkommt.

***pathName* get**

Gibt den aktuellen Wert der Combobox zurück.

***pathName* set *value***

Setzt den Wert der Combobox auf *value*.

Die Combobox unterstützt außerdem folgende Befehle des ttk::entry-Widgets:

**bbox delete icursor**
**index insert selection**
**xview**

## Virtuelle Ereignisse

Die Combobox erzeugt ein virtuelles Ereignis **<<ComboboxSelected>>**, wenn der Benutzer ein Element aus der Werteliste wählt. Schließt die Auswahlaktion die Listbox, wird dieses Ereignis nach dem Schließen der Listbox ausgeliefert.

## Styling-Optionen

Der Klassenname für ein **ttk::combobox** ist **TCombobox**. **ttk::combobox** verwendet intern die Widgets entry und listbox. Der Listbox-Rahmen hat den Klassennamen **ComboboxPopdownFrame**.

Dynamische Zustände: **disabled**, **focus**, **pressed**, **readonly**.

**TCombobox**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-arrowcolor** *color*

**-arrowsize** *amount*

**-background** *color*

**-bordercolor** *color*

**-darkcolor** *color*

**-focusfill** *color*

**-foreground** *color*

**-fieldbackground *color***

Kann nur bei nicht-nativen und nicht-grafischen Themes geändert werden.

**-insertcolor** *color*

**-insertwidth** *amount*

**-lightcolor** *color*

**-padding** *padding*

**-placeholderforeground** *color*

**-postoffset** *padding*

**-selectbackground *color***

Hintergrund der Textauswahl.

**-selectforeground *color***
  Vordergrund der Textauswahl.

Die ausklappbare Listbox von **ttk::combobox** lässt sich weder mit ttk::style noch über den Widget-Befehl **configure** konfigurieren. Die Listbox kann über die Optionsdatenbank konfiguriert werden.

option add *TCombobox*Listbox.background *color*

option add *TCombobox*Listbox.font *font*

option add *TCombobox*Listbox.foreground *color*

option add *TCombobox*Listbox.selectBackground *color*

option add *TCombobox*Listbox.selectForeground *color*

Zum Konfigurieren einer bestimmten Listbox (kann sich künftig ändern):

```
set popdown [ttk::combobox::PopdownWindow .mycombobox]
$popdown.f.l configure -font font
```

**ComboboxPopdownFrame**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-borderwidth** *amount*

**-relief** *relief*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2004 Joe English*

# ttk_entry

*Stichwörter:* ttk_entry, entry, widget, text field

## Name

ttk::entry - Bearbeitbares Einzeilen-Textfeld-Widget

## Übersicht

**ttk::entry** *pathName* ?*options*?

## Beschreibung

Ein **ttk::entry**-Widget zeigt einen einzeiligen Text-String an und erlaubt dem Benutzer, diesen String zu bearbeiten. Der Wert des Strings kann mit der Option **-textvariable** an eine Tcl-Variable gebunden werden. Entry-Widgets unterstützen horizontales Scrollen mit der Standardoption **-xscrollcommand** und dem Widget-Befehl **xview**.

## Standardoptionen

**-class**, **-cursor**, **-font**, **-foreground**, **-style**, **-takefocus**, **-xscrollcommand**, **-placeholder**, **-placeholderforeground**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-exportselection (exportSelection/ExportSelection)**

Ein boolescher Wert, der angibt, ob eine Auswahl im Widget mit der X-Selection verknüpft werden soll. Wird die Auswahl exportiert, hebt die Auswahl im Widget die aktuelle X-Selection auf, hebt die Auswahl außerhalb des Widgets jede Widget-Auswahl auf, und das Widget reagiert auf Anfragen zur Abfrage der Selection, wenn es eine Auswahl hat.

**-invalidcommand (invalidCommand/InvalidCommand)**

Eine Skript-Vorlage, die ausgewertet wird, wann immer **-validatecommand** 0 zurückgibt. Siehe Validierung unten für weitere Informationen.

**-justify (justify/Justify)**

Legt fest, wie der Text im Entry-Widget ausgerichtet ist. Einer von **left**, **center** oder **right**.

**-show (show/Show)**

Wenn diese Option angegeben ist, wird der wahre Inhalt des Entrys nicht im Fenster angezeigt. Stattdessen wird jedes Zeichen im Wert des Entrys als das erste Zeichen im Wert dieser Option angezeigt, z. B. "*" oder ein Aufzählungszeichen. Dies ist nützlich, wenn das Entry beispielsweise zur Eingabe eines Passworts verwendet werden soll. Werden Zeichen im Entry ausgewählt und anderswo kopiert, werden die angezeigten Informationen kopiert, nicht der wahre Inhalt des Entrys.

**-state (state/State)**

Kompatibilitätsoption; siehe *ttk::widget(n)* für Details. Legt einen von drei Zuständen für das Entry fest: **normal**, **disabled** oder **readonly**. Siehe Widget-Zustände unten.

**-textvariable (textVariable/Variable)**

Gibt den Namen einer globalen Variable an, deren Wert mit dem Inhalt des Entry-Widgets verknüpft ist. Wann immer sich der Variablenwert ändert, wird der Widget-Inhalt aktualisiert und umgekehrt.

**-validate (validate/Validate)**

Legt den Modus fest, in dem die Validierung arbeiten soll: **none**, **focus**, **focusin**, **focusout**, **key** oder **all**. Standard ist **none**, d. h. die Validierung ist deaktiviert. Siehe Validierung unten.

**-validatecommand (validateCommand/ValidateCommand)**

Eine Skript-Vorlage, die ausgewertet wird, wann immer eine Validierung ausgelöst wird. Ist sie auf den leeren String gesetzt (Standard), ist die Validierung deaktiviert. Das Skript muss einen booleschen Wert zurückgeben. Siehe Validierung unten.

**-width (width/Width)**

Ganzzahl für die gewünschte Breite des Eingabefensters in Zeichen durchschnittlicher Größe der Widget-Schriftart.

## Hinweise

Ein Teil des Entrys kann wie unten beschrieben ausgewählt werden. Exportiert ein Entry seine Auswahl (siehe die Option **-exportselection**), beachtet es die Standard-X11-Protokolle für die Behandlung der Selection; Entry-Auswahlen sind als Typ **STRING** verfügbar. Entrys beachten auch die Standard-Tk-Regeln für den Umgang mit dem Eingabefokus. Hat ein Entry den Eingabefokus, zeigt es einen *Einfügecursor* an, um anzuzeigen, wo neue Zeichen eingefügt werden.

Entrys können Strings anzeigen, die zu lang sind, um vollständig in das Fenster des Widgets zu passen. In diesem Fall wird nur ein Teil des Strings angezeigt; unten beschriebene Befehle können verwendet werden, um die Ansicht im Fenster zu ändern. Entrys verwenden den

Standard-Mechanismus **-xscrollcommand** für die Interaktion mit Scrollbars (siehe die Beschreibung der Option **-xscrollcommand** für Details).

## Indizes

Viele der entry-Widget-Befehle nehmen einen oder mehrere Indizes als Argumente. Ein Index bezeichnet ein bestimmtes Zeichen im String des Entrys, auf eine der folgenden Arten:

***number***
: Bezeichnet das Zeichen als numerischen Index, wobei 0 dem ersten Zeichen im String entspricht.

**@*number***
: In dieser Form wird *number* als x-Koordinate im Fenster des Entrys behandelt; das Zeichen, das diese x-Koordinate überspannt, wird verwendet. Beispielsweise bezeichnet "**@0**" das linkeste Zeichen im Fenster.

**end**
: Bezeichnet das Zeichen unmittelbar nach dem letzten im String des Entrys. Entspricht einem numerischen Index gleich der Länge des Strings des Entrys.

**insert**
: Bezeichnet das Zeichen neben und unmittelbar nach dem Einfügecursor.

**sel.first**
: Bezeichnet das erste Zeichen in der Auswahl. Die Verwendung dieser Form ist ein Fehler, wenn die Auswahl nicht im Entry-Fenster liegt.

**sel.last**
: Bezeichnet das Zeichen unmittelbar nach dem letzten in der Auswahl. Es ist ein Fehler, diese Form zu verwenden, wenn die Auswahl nicht im Entry-Fenster liegt.

Abkürzungen können für jede der obigen Formen verwendet werden, z. B. "**e**" oder "**sel.l**". Im Allgemeinen werden Indizes außerhalb des gültigen Bereichs automatisch auf den nächsten zulässigen Wert gerundet. Indizes unterstützen dieselbe einfache Interpretation wie beim Befehl string index, mit einfacher ganzzahliger Index-Arithmetik und Indizierung relativ zu **end**.

## Widget-Befehl

Zusätzlich zu den Standard-Befehlen **cget**, **configure**, **identify element**, **instate**, **state**, **style** und **xview** (siehe ttk::widget) unterstützen Entry-Widgets die folgenden weiteren Befehle:

***pathName* bbox *index***
: Gibt eine Liste von vier Zahlen zurück, die die Begrenzungsbox des durch *index* angegebenen Zeichens beschreiben. Die ersten beiden Elemente der Liste geben die x- und y-Koordinaten der oberen linken Ecke des vom Zeichen bedeckten Bildschirmbereichs an (in Pixeln relativ zum Widget), die letzten beiden geben Breite und Höhe des Zeichens in Pixeln an. Die Begrenzungsbox kann sich auf einen Bereich außerhalb des sichtbaren Fensterbereichs beziehen.

***pathName* delete *first* ?*last*?**
: Löscht ein oder mehrere Elemente des Entrys. *First* ist der Index des ersten zu löschenden Zeichens, *last* der Index des Zeichens unmittelbar nach dem letzten zu löschenden. Ist *last* nicht angegeben, entspricht es *first*+1, d. h. es wird ein einzelnes Zeichen gelöscht. Dieser Befehl gibt den leeren String zurück.

***pathName* get**

Gibt den String des Entrys zurück.

***pathName* icursor *index***

Sorgt dafür, dass der Einfügecursor unmittelbar vor dem durch *index* angegebenen Zeichen angezeigt wird. Gibt den leeren String zurück.

***pathName* index *index***

Gibt den numerischen Index zurück, der *index* entspricht.

***pathName* insert *index string***

Fügt *string* unmittelbar vor dem durch *index* angegebenen Zeichen ein. Gibt den leeren String zurück.

***pathName* selection *option arg***

Dieser Befehl passt die Auswahl innerhalb eines Entrys an. Er hat mehrere Formen, abhängig von *option*:

***pathName* selection clear**

Löscht die Auswahl, falls sie derzeit in diesem Widget liegt. Liegt die Auswahl nicht in diesem Widget, hat der Befehl keine Wirkung. Gibt den leeren String zurück.

***pathName* selection present**

Gibt 1 zurück, wenn im Entry Zeichen ausgewählt sind, 0 wenn nichts ausgewählt ist.

***pathName* selection range *start end***

Setzt die Auswahl so, dass sie die Zeichen ab dem durch *start* indizierten bis vor dem durch *end* indizierten einschließt. Verweist *end* auf dasselbe Zeichen wie *start* oder ein früheres, wird die Auswahl des Entrys gelöscht.

***pathName* validate**

Erzwingt eine erneute Validierung, unabhängig von den durch die Option **-validate** festgelegten Bedingungen. Gibt 0 zurück, wenn die Validierung fehlschlägt, 1 bei Erfolg. Setzt oder löscht den Zustand **invalid** entsprechend. Siehe Validierung unten für weitere Details.

## Validierung

Die Optionen **-validate**, **-validatecommand** und **-invalidcommand** aktivieren die Validierung des Entry-Widgets.

### Validierungsmodi

Es gibt zwei Haupt-Validierungsmodi: *Prävalidierung*, bei der **-validatecommand** vor jeder Bearbeitung ausgewertet wird und der Rückgabewert bestimmt, ob die Änderung angenommen oder abgelehnt wird; und *Revalidierung*, bei der **-validatecommand** ausgewertet wird, um zu prüfen, ob der aktuelle Wert gültig ist.

Die Option **-validate** legt fest, wann validiert wird; sie kann auf einen der folgenden Werte gesetzt werden:

**none**

Standard. Die Validierung erfolgt nur, wenn sie ausdrücklich durch den Widget-Befehl **validate** angefordert wird.

**key**

Das Entry wird vor jeder Bearbeitung prävalidiert (insbesondere wann immer die Widget-Befehle **insert** oder **delete** aufgerufen werden). Schlägt die Prävalidierung fehl, wird die Bearbeitung abgelehnt.

**focus**
Das Entry wird revalidiert, wenn es den Fokus erhält oder verliert.

**focusin**
Das Entry wird revalidiert, wenn es den Fokus erhält.

**focusout**
Das Entry wird revalidiert, wenn es den Fokus verliert.

**all**
Die Validierung wird bei allen obigen Bedingungen durchgeführt.

**-invalidcommand** wird ausgewertet, wann immer **-validatecommand** einen falschen Wert zurückgibt.

**-validatecommand** und **-invalidcommand** können den Wert des Entry-Widgets über die Widget-Befehle **insert** oder **delete** oder durch Setzen der verknüpften **-textvariable** ändern. Tut eines von beiden dies während der Prävalidierung, wird die Bearbeitung abgelehnt, unabhängig vom von **-validatecommand** zurückgegebenen Wert.

Ist **-validatecommand** leer (Standard), ist die Validierung immer erfolgreich.

## Prozent-Substitutionen in Validierungsskripten

Auf **-validatecommand** und **-invalidcommand** sind Prozent-Substitutionen möglich, wie in einem bind-Skript. Folgende Substitutionen werden erkannt:

**%d**
Art der Aktion: 1 für Prävalidierung bei **insert**, 0 für Prävalidierung bei **delete**, oder -1 für Revalidierung.

**%i**
Index des einzufügenden/zu löschenden Zeichenstrings, falls vorhanden, sonst -1.

**%P**
Bei Prävalidierung der neue Wert des Entrys, wenn die Bearbeitung angenommen wird. Bei Revalidierung der aktuelle Wert des Entrys.

**%s**
Der aktuelle Wert des Entrys vor der Bearbeitung.

**%S**
Der einzufügende/zu löschende Textstring, falls vorhanden, sonst {}.

**%v**
Der aktuelle Wert der Option **-validate**.

**%V**
Die Validierungsbedingung, die den Rückruf ausgelöst hat (**key**, **focusin**, **focusout** oder **forced**).

**%W**
Der Name des Entry-Widgets.

## Unterschiede zur Validierung des Tk-Entry-Widgets

Das Standard-Tk-Entry-Widget deaktiviert die Validierung automatisch (durch Setzen von **-validate** auf **none**), wenn **-validatecommand** oder **-invalidcommand** den Wert des Entrys ändert. Das Tk-Themen-Entry-Widget deaktiviert die Validierung nur, wenn eines der Validierungsskripte einen Fehler auslöst oder **-validatecommand** keinen gültigen booleschen

Wert zurückgibt. (Es ist also nicht nötig, die Validierung nach Änderung des Entry-Werts in einem Validierungsskript wieder zu aktivieren.)

Außerdem ruft das Standard-Entry-Widget die Validierung auf, wann immer die verknüpfte **-textvariable** geändert wird; das Tk-Themen-Entry-Widget tut dies nicht.

## Bindings

Die Standard-Bindings des Entry-Widgets ermöglichen folgendes Verhalten. In den Beschreibungen unten bezeichnet "Wort" eine zusammenhängende Gruppe aus Buchstaben, Ziffern oder "_"-Zeichen oder ein einzelnes anderes Zeichen.

- Ein Klick mit Maustaste 1 positioniert den Einfügecursor unmittelbar vor dem Zeichen unter dem Mauszeiger, setzt den Eingabefokus auf dieses Widget und löscht jede Auswahl im Widget. Ziehen mit gedrückter Maustaste 1 erzeugt eine Auswahl zwischen Einfügecursor und dem Zeichen unter der Maus.
- Doppelklick mit Maustaste 1 wählt das Wort unter der Maus aus und positioniert den Einfügecursor am Ende des Worts. Ziehen nach einem Doppelklick erzeugt eine Auswahl aus ganzen Wörtern.
- Dreifachklick mit Maustaste 1 wählt den gesamten Text im Entry aus und positioniert den Einfügecursor am Zeilenende.
- Die Enden der Auswahl können durch Ziehen mit Maustaste 1 bei gedrückter Umschalttaste angepasst werden. Wurde vor dem Ziehen doppelgeklickt, wird die Auswahl in Einheiten ganzer Wörter angepasst.
- Klick mit Maustaste 1 bei gedrückter Steuerung positioniert den Einfügecursor im Entry, ohne die Auswahl zu beeinflussen.
- Werden normale druckbare Zeichen in einem Entry eingegeben, werden sie an der Position des Einfügecursors eingefügt.
- Die Ansicht im Entry kann durch Ziehen mit der mittleren Maustaste (Taste 2, oder Taste 3 unter TkAqua) angepasst werden. Wird die mittlere Maustaste ohne Mausbewegung geklickt, wird die Auswahl an der Mausposition in das Entry kopiert.
- Wird die Maus beim Gedrückthalten von Taste 1 links oder rechts aus dem Entry gezogen, scrollt das Entry automatisch, um mehr Text sichtbar zu machen (falls auf der Seite, auf der die Maus das Fenster verließ, noch Text außerhalb des Bildschirms liegt).
- Die Tasten Links und Rechts bewegen den Einfügecursor um ein Zeichen nach links oder rechts; sie löschen außerdem jede Auswahl im Entry. Wird Links oder Rechts mit gedrückter Umschalttaste eingegeben, bewegt sich der Einfügecursor und die Auswahl wird um das neue Zeichen erweitert. Steuerung-Links und Steuerung-Rechts bewegen den Einfügecursor wortweise; Steuerung-Umschalt-Links und Steuerung-Umschalt-Rechts bewegen ihn wortweise und erweitern die Auswahl. Steuerung-b und Steuerung-f verhalten sich wie Links bzw. Rechts.
- Die Pos1-Taste und Steuerung-a bewegen den Einfügecursor an den Anfang des Entrys und löschen jede Auswahl. Umschalt-Pos1 bewegt den Einfügecursor an den Anfang und erweitert die Auswahl bis dorthin.
- Die Ende-Taste und Steuerung-e bewegen den Einfügecursor ans Ende des Entrys und löschen jede Auswahl. Umschalt-Ende bewegt den Cursor ans Ende und erweitert die Auswahl bis dorthin.
- Steuerung-/ wählt den gesamten Text im Entry aus.
- Steuerung-\ löscht jede Auswahl im Entry.
- Die virtuellen Ereignisse <>, <>, <> und <> von Tk wirken auf die Auswahl wie erwartet.

- Die Entf-Taste löscht die Auswahl, falls eine im Entry besteht. Gibt es keine Auswahl, löscht sie das Zeichen rechts vom Einfügecursor.
- Die Rücktaste und Steuerung-h löschen die Auswahl, falls eine im Entry besteht. Gibt es keine Auswahl, löschen sie das Zeichen links vom Einfügecursor.
- Steuerung-d löscht das Zeichen rechts vom Einfügecursor.
- Steuerung-k löscht alle Zeichen rechts vom Einfügecursor.

## Widget-Zustände

Im Zustand **disabled** kann das Entry nicht bearbeitet und der Text nicht ausgewählt werden. Im Zustand **readonly** wird kein Einfügecursor angezeigt und das Entry kann nicht bearbeitet werden (insbesondere: die Befehle **insert** und **delete** haben keine Wirkung). Der Zustand **disabled** entspricht **readonly**, zusätzlich kann der Text nicht ausgewählt werden.

Änderungen an der verknüpften **-textvariable** werden weiterhin im Entry angezeigt, auch wenn es disabled oder readonly ist.

Typischerweise wird der Text im Zustand **disabled** "ausgegraut" dargestellt, im Zustand **readonly** ein anderer Hintergrund verwendet.

Das Entry-Widget setzt den Zustand **invalid**, wenn die Revalidierung fehlschlägt, und löscht ihn, wann immer die Validierung erfolgreich ist.

## Stil-Optionen

Der Klassenname für **ttk::entry** ist **TEntry**.

Dynamische Zustände: **disabled**, **focus**, **readonly**.

Mit ttk::style konfigurierbare **TEntry**-Stiloptionen:

**-background *color***

Zur Abwärtskompatibilität verhält sich diese Option beim aqua-Theme (für macOS) als Alias für **-fieldbackground**, sofern für **-fieldbackground** kein Wert angegeben ist. Andernfalls wird sie ignoriert.

**-bordercolor** *color*

**-darkcolor** *color*

**-fieldbackground *color***

Einige Themes verwenden einen grafischen Hintergrund; deren Feld-Hintergrundfarben können nicht geändert werden.

**-foreground** *color*

**-insertcolor** *color*

**-insertwidth** *amount*

**-lightcolor** *color*

**-padding** *padding*

**-placeholderforeground** *color*

**-relief** *relief*

**-selectbackground** *color*

**-selectborderwidth** *amount*

**-selectforeground** *color*

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 1998-2000 Scriptics Corporation.*
*Copyright (c) 2004 Joe English*

# ttk_frame

*Stichwörter:* ttk_frame, widget, frame, container

## Name

ttk::frame - Einfaches Container-Widget

## Übersicht

**ttk::frame** *pathName* ?*options*?

## Beschreibung

Ein **ttk::frame**-Widget ist ein Container zum Gruppieren anderer Widgets.

## Standardoptionen

**-class**, **-cursor**, **-padding**, **-style**, **-takefocus**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-borderwidth (borderWidth/BorderWidth)**

Gewünschte Breite des Widget-Rahmens. Standard ist 0. Je nach Theme kann die Option ignoriert werden.

**-relief (relief/Relief)**

Einer der Standard-Tk-Rahmenstile: **flat**, **groove**, **raised**, **ridge**, **solid** oder **sunken**. Standard ist **flat**.

**-width (width/Width)**

Legt bei Angabe die gewünschte Widget-Breite in Pixeln fest.

**-height (height/Height)**

Ist angegeben, die gewünschte Widget-Höhe in Pixeln.

## Widget-Befehl

Frame-Widgets unterstützen die Standardbefehle **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget).

## Hinweise

Werden pack, grid oder andere Geometrie-Manager für die Kinder des frame verwendet, hat die vom Manager gewünschte Größe meist Vorrang vor den Optionen **-width** und **-height** des frame. Mit pack propagate und grid propagate lässt sich das ändern.

## Styling-Optionen

Der Klassenname für ein **ttk::frame** ist **TFrame**.

**TFrame**-Styling-Optionen, die mit ttk::style konfiguriert werden können:

**-background** *color*

**-relief** *relief*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

### Bindings

Beim Erzeugen eines neuen **ttk::frame** gibt es keine Standard-Ereignis-Bindings; **ttk::frame** -Widgets sind nicht für Interaktion gedacht.

*Copyright (c) 2005 Joe English*

## ttk_image

*Stichwörter:* ttk_image, style, theme, appearance, pixmap theme, image

### Name

ttk_image - Definiert ein Element auf Basis eines Bildes

### Übersicht

**ttk::style element create** *name* **image** *imageSpec* ?*options*?

### Beschreibung

Die *image*-Elementfabrik erzeugt ein neues Element im aktuellen Theme, dessen Erscheinungsbild durch Tk-Bilder bestimmt wird. *imageSpec* ist eine Liste mit einem oder mehreren Elementen. Das erste Element ist der Standard-Bildname. Der Rest der Liste ist eine Folge von *statespec / value*-Paaren, die andere Bilder angeben, wenn das Element in einem bestimmten Zustand oder einer Kombination von Zuständen ist.

### Optionen

Gültige *options* sind:

**-border** ***padding***

*padding* ist eine Liste mit bis zu vier Ganzzahlen für die Ränder links, oben, rechts und unten. Sind weniger als vier Elemente angegeben, gilt standardmäßig *bottom* = *top*, *right* = *left* und *top* = *left*. Eine Liste mit drei Zahlen legt den linken, vertikalen und rechten Rand fest; zwei Zahlen den horizontalen und vertikalen Rand; eine einzelne Zahl denselben Rand ringsum. Siehe Bild-Streckung unten.

**-height** ***height***

Legt eine Mindesthöhe für das Element fest. Ist der Wert negativ, wird die Höhe des Basisbildes als Standard verwendet.

**-padding** ***padding***

Legt den Innenabstand des Elements fest. Der Innenabstand ist eine Liste mit bis zu vier Längenangaben *left top right bottom*. Sind weniger als vier Elemente angegeben, gilt standardmäßig *bottom* = *top*, *right* = *left* und *top* = *left*. Eine Liste mit drei Zahlen legt den linken, vertikalen und rechten Innenabstand fest; zwei Zahlen den horizontalen und vertikalen Innenabstand; eine einzelne Zahl denselben Innenabstand ringsum. Standard ist **-border**, falls nicht angegeben.

**-sticky** ***spec***

Legt fest, wie das Bild im endgültigen Bereich platziert wird. *spec* enthält null oder mehr Zeichen "n", "s", "w" oder "e".

**-width *width***

Legt eine Mindestbreite für das Element fest. Ist der Wert negativ, wird die Breite des Basisbildes als Standard verwendet.

## Bild-Streckung

Ist der dem Element zugewiesene Bereich größer als das Bild, wird das Bild gemäß der Option **-sticky** im Bereich platziert. Muss das Bild horizontal gestreckt werden (d. h. **-sticky ew**) oder vertikal (**-sticky ns**), werden Teilbereiche des Bildes entsprechend der Option **-border** wiederholt, um den Bereich zu füllen. **-border** teilt das Bild in neun Regionen: vier feste Ecken, obere und linke Kanten (horizontal kachelbar), linke und rechte Kanten (vertikal kachelbar) und den zentralen Bereich (in beide Richtungen kachelbar).

Ein Bildelement, das keinen Platz beanspruchen soll (z. B. als Hintergrundbild), sollte **-width 0** und **-height 0** verwenden.

## Beispiel

```
set img1 [image create photo -file button.png]
set img2 [image create photo -file button-pressed.png]
set img3 [image create photo -file button-active.png]
ttk::style element create Button.button image \
    [list $img1  pressed $img2  active $img3] \
    -border {2 4} -sticky we
```

*Copyright (c) 2004 Joe English*

# ttk_intro

*Stichwörter:* ttk_intro

## Name

ttk::intro - Einführung in die Tk-Theme-Engine

## Einleitung

Das Tk-Widget-Set mit Themes basiert auf einer überarbeiteten und erweiterten Version der in TIP #48 (https://tip.tcl-lang.org/48) spezifizierten Style-Engine. Die wichtigsten Konzepte werden unten beschrieben. Die Grundidee ist, soweit möglich den Code für das Verhalten eines Widgets vom Code für sein Erscheinungsbild zu trennen. Widget-Klassen-Bindungen sind vor allem dafür zuständig, den Widget-Zustand zu pflegen und Rückruffunktionen aufzurufen; alle Aspekte des Erscheinungsbilds werden über den Style des Widgets gesteuert (d. h. den Style der Elemente des Widgets).

## Themes

Ein *Theme* ist eine Sammlung von Elementen und Styles, die Look-and-Feel des Widget-Sets bestimmen. Themes können genutzt werden, um:

- Plattformunterschiede auszugleichen (X11 vs. klassisches Windows vs. XP vs. Aqua ...)
- sich an Anzeigebeschränkungen anzupassen (wenige Farben, Graustufen, Monochrom, kleine

Bildschirme)
- Barrierefreiheit (hoher Kontrast, große Schrift)
- Corporate Design einer Anwendungssuite
- sich in die restliche Desktop-Umgebung einzufügen (Gnome, KDE, Java)
- und natürlich: optische Aufwertung.

## Elemente

Ein *Element* stellt einen einzelnen Teil eines Widgets dar. Ein vertikales Scrollbar-Widget enthält z. B. die Elemente **uparrow**, **downarrow**, **trough** und **slider**.

Elementnamen verwenden eine rekursive Punktnotation. **uparrow** bezeichnet z. B. ein generisches Pfeil-Element; **Scrollbar.uparrow** und **Combobox.uparrow** bezeichnen widgetspezifische Elemente. Bei der Elementsuche prüft die Style-Engine zuerst den spezifischen Namen; wird kein Element dieses Namens gefunden, sucht sie nach generischen Elementen, indem sie nacheinander führende Komponenten des Elementnamens entfernt.

Wie Widgets haben Elemente *Optionen*, die festlegen, was und wie angezeigt wird. Das **text** -Element (zeigt eine Textzeichenkette) hat z. B. die Optionen **-text**, **-font**, **-foreground**, **-background**, **-underline** und **-width**. Der Wert einer Elementoption wird übernommen (in dieser Reihenfolge):

- einer gleichnamigen Option gleichen Typs im Widget, das das Element enthält;
- einer dynamischen Einstellung über **style map** und den aktuellen Zustand;
- der Standardeinstellung über **style configure**; oder
- dem eingebauten Standardwert des Elements für die Option.

## Layouts

Ein *Layout* legt fest, welche Elemente ein Widget bilden und wie sie angeordnet sind. Die Layout-Engine nutzt eine vereinfachte Variante des pack-Algorithmus: Ausgehend von einer anfänglichen Höhlung in Widgetgröße werden Elemente entlang der mit **-side** angegebenen Seite ein Parzelle in der Höhlung zugewiesen und gemäß **-sticky** darin platziert. Das Layout einer horizontalen Scrollbar ist z. B.:

```
ttk::style layout Horizontal.TScrollbar {
    Scrollbar.trough -children {
    Scrollbar.leftarrow  -side left  -sticky w
    Scrollbar.rightarrow -side right -sticky e
    Scrollbar.thumb -sticky ew
    }
}
```

Standardmäßig entspricht das Layout eines Widgets seinem Klassennamen. Manche Widgets überschreiben das (z. B. wählt ttk::scrollbar je nach Option **-orient** unterschiedliche Layouts).

## Zustände

In Standard-Tk haben viele Widgets eine Option **-state**, die (meist) **normal** oder **disabled** ist. Manche Widgets unterstützen weitere Zustände, z. B. entry mit **readonly** oder die verschiedenen Button-Varianten mit **active**.

Die Tk-Widgets mit Themes verallgemeinern diese Idee: Jedes Widget hat eine Bitmap unabhängiger Zustandsflags. Dazu gehören **active**, **disabled**, **pressed**, **focus** usw. (vollständige Liste der Zustandsflags in *ttk::widget(n)*).

Statt einer Option **-state** hat jedes Widget nun den Widget-Befehl **state** zum Setzen oder Abfragen des Zustands. Eine *Zustandsspezifikation* ist eine Liste symbolischer Zustandsnamen, die angibt, welche Bits gesetzt sind; optional mit vorangestelltem Ausrufezeichen, um ein Bit zu löschen.

Die Klassen-Bindungen für ttk::button sind z. B.:

```
bind TButton <Enter>        { %W state active }
bind TButton <Leave>        { %W state !active }
bind TButton <Button-1>     { %W state pressed }
bind TButton <Button1-Leave>    { %W state !pressed }
bind TButton <Button1-Enter>    { %W state pressed }
bind TButton <ButtonRelease-1>  \
    { %W instate {pressed} { %W state !pressed ; %W invoke } }
```

Das Widget wird **active**, wenn der Mauszeiger es betritt, und inaktiv beim Verlassen. Beim Drücken der Maustaste wird es **pressed**, bei ButtonRelease**!pressed**. Wird die Taste bei gedrückter Button-1 außerhalb des Widgets gezogen, wird der Druck aufgehoben; beim Zurückziehen wieder **pressed**. Beim Loslassen wird **-command** aufgerufen, aber nur, wenn der Button gerade im Zustand **pressed** ist. (Die tatsächlichen Bindungen sind etwas komplexer, aber nicht viel.)

## Styles

Jedes Widget ist einem *Style* zugeordnet, der Werte für Elementoptionen festlegt. Stylenamen verwenden wie Layouts und Elemente eine rekursive Punktnotation; standardmäßig nutzen Widgets den Klassennamen, um einen Style im aktuellen Theme zu finden. Beispiel:

```
ttk::style configure TButton \
    -background #d9d9d9 \
    -foreground black \
    -relief raised \
    ;
```

Viele Elemente werden je nach Widget-Zustand unterschiedlich dargestellt. Buttons haben z. B. anderen Hintergrund, wenn sie aktiv sind, andere Vordergrundfarbe, wenn sie deaktiviert sind, und anderes Relief, wenn sie gedrückt sind. Der Befehl **style map** legt dynamische Optionseinstellungen für einen bestimmten Style fest:

```
ttk::style map TButton \
    -background [list disabled #d9d9d9  active #ececec] \
    -foreground [list disabled #a3a3a3] \
    -relief [list {pressed !disabled} sunken] \
    ;
```

*Copyright (c) 2004 Joe English*

# ttk_label

*Stichwörter:* ttk_label

## Name

ttk::label - Textzeichenkette und/oder Bild anzeigen

## Übersicht

**ttk::label** *pathName* ?*options*?

## Beschreibung

Ein **ttk::label**-Widget zeigt eine Textbeschriftung und/oder ein Bild an. Die Beschriftung kann mit einer Tcl-Variable verknüpft werden, damit sich der angezeigte Text automatisch ändert.

## Standardoptionen

**-anchor**, **-class**, **-compound**, **-cursor**, **-font**, **-foreground**, **-image**, **-justify**, **-padding**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**, **-wraplength**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-background (frameColor/FrameColor)**

Hintergrundfarbe des Widgets. Ist sie nicht angegeben, gilt der Theme-Standard.

**-relief (relief/Relief)**

Gewünschter 3D-Effekt für den Widget-Rahmen. Gültige Werte: **flat**, **groove**, **raised**, **ridge**, **solid** und **sunken**.

**-wraplength (wrapLength/WrapLength)**

Maximale Zeilenlänge in Pixeln. Ist die Option negativ oder null, erfolgt kein automatischer Umbruch; andernfalls wird der Text so geteilt, dass keine Zeile länger als der angegebene Wert ist.

## Widget-Befehl

Label-Widgets unterstützen die Standardbefehle **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget).

## Styling-Optionen

Der Klassenname für ein **ttk::label** ist **TLabel**.

Dynamische Zustände: **disabled**, **readonly**.

**TLabel**-Styling-Optionen, die mit ttk::style konfiguriert werden können:

**-background** *color*

**-compound** *compound*

**-foreground** *color*

**-font** *font*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2004 Joe English*

# ttk_labelframe

*Stichwörter:* ttk_labelframe, widget, frame, container, label, groupbox

## Name

ttk::labelframe - Container-Widget mit optionalem Label

## Übersicht

**ttk::labelframe** *pathName* ?*options*?

## Beschreibung

Ein **ttk::labelframe**-Widget ist ein Container, der andere Widgets gruppiert. Es hat ein optionales Label, das eine einfache Textzeichenkette oder ein anderes Widget sein kann.

## Standardoptionen

**-class**, **-cursor**, **-padding**, **-style**, **-takefocus**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-height (height/Height)**

Legt bei Angabe die gewünschte Widget-Höhe in Pixeln fest. (Siehe *ttk::frame(n)* für weitere Hinweise zu **-width** und **-height**).

**-labelanchor (labelAnchor/LabelAnchor)**

Legt fest, wo das Label platziert wird. Zulässige Werte sind (im Uhrzeigersinn von der oberen linken Ecke): **nw**, **n**, **ne**, **en**, **e**, **es**, **se**, **s**,**sw**, **ws**, **w** und **wn**. Der Standardwert ist theme-abhängig.

**-labelwidget (labelWidget/LabelWidget)**

Name eines Widgets, das als Label verwendet wird. Wenn gesetzt, überschreibt dies die Option **-text**. Das **-labelwidget** muss ein Kind des labelframe-Widgets oder eines seiner Vorfahren sein und zum selben Toplevel-Fenster wie das labelframe gehören.

**-text (text/Text)**

Legt den Text des Labels fest.

**-underline (underline/Underline)**

Wenn gesetzt, gibt dies den ganzzahligen Index (0-basiert) eines Zeichens an, das in der Textzeichenkette unterstrichen wird. Das unterstrichene Zeichen dient der Mnemonik-Aktivierung. Die Mnemonik-Aktivierung für ein **ttk::labelframe** setzt den Tastaturfokus auf das erste Kind des **ttk::labelframe**-Widgets.

**-width (width/Width)**

Legt bei Angabe die gewünschte Widget-Breite in Pixeln fest.

## Widget-Befehl

Labelframe-Widgets unterstützen die Standardbefehle **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget).

## Styling-Optionen

Der Klassenname für ein **ttk::labelframe** ist **TLabelframe**. Das Textlabel hat die Klasse **TLabelframe.Label**.

Dynamische Zustände: **disabled**, **readonly**.

**TLabelframe**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-background** *color*

**-bordercolor** *color*

**-borderwidth** *amount*

**-darkcolor** *color*

**-labelmargins** *amount*

**-labeloutside** *boolean*

**-lightcolor** *color*

**-relief** *relief*

**TLabelframe.Label**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-background** *color*

**-font** *font*

**-foreground** *color*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2005 Joe English*

# ttk_menubutton

*Stichwörter:* ttk_menubutton, widget, button, menu

## Name

ttk::menubutton - Widget, das beim Drücken ein Menü öffnet

## Übersicht

**ttk::menubutton** *pathName* ?*options*?

## Beschreibung

Ein **ttk::menubutton**-Widget zeigt eine Textbeschriftung und/oder ein Bild und öffnet beim Drücken ein Menü.

## Standardoptionen

**-class**, **-compound**, **-cursor**, **-image**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-direction (direction/Direction)**

Gibt an, wo das Menü relativ zum Menubutton geöffnet wird. Einer von: **above**, **below**, **left**, **right** oder **flush**. Standard ist **below**. **flush** öffnet das Menü direkt über dem Menubutton.

**-menu (menu/Menu)**

Pfadname des zugehörigen Menüs. Zur Sicherheit sollte das Menü ein direktes Kind des Menubuttons sein.

## Widget-Befehl

Menubutton-Widgets unterstützen die Standardbefehle **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget).

## Standard-Stile

**Ttk::menubutton**-Widgets unterstützen in allen Standard-Themes den Stil **Toolbutton** — nützlich für Symbolleisten.

## Styling-Optionen

Der Klassenname für ein **ttk::menubutton** ist **TMenubutton**.

Dynamische Zustände: **active**, **disabled**, **readonly**.

**TMenubutton**-Styling-Optionen, die mit ttk::style konfiguriert werden können:

**-arrowsize** *amount*

**-background** *color*

**-compound** *compound*

**-foreground** *color*

**-font** *font*

**-padding** *padding*

**-relief** *relief*

**-width** *amount*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2004 Joe English*

# ttk_notebook

*Stichwörter:* ttk_notebook, pane, tab

## Name

ttk::notebook - Container-Widget mit mehreren Bereichen

## Übersicht

**ttk::notebook** *pathname ?options...?*
*pathname* **add** *window ?options...?*
*pathname* **insert** *index window ?options...?*

## Beschreibung

Ein **ttk::notebook**-Widget verwaltet eine Sammlung von Fenstern und zeigt jeweils nur eines an. Jedes Inhaltsfenster ist einem *Tab* zugeordnet, den der Benutzer auswählen kann, um das aktuell angezeigte Fenster zu wechseln.

Ist ein Inhaltsfenster kein Kind des Notebooks, muss es in der Stapelreihenfolge über dem Notebook liegen, sonst verdeckt das Notebook es. Am einfachsten erzeugen Sie das Notebook zuerst: das zuletzt erzeugte Fenster liegt oben in der Stapelreihenfolge. Oder Sie verwenden die Befehle raise und lower.

## Standardoptionen

**-class**, **-cursor**, **-takefocus**, **-style**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-height (height/Height)**

Wenn vorhanden und größer als null, legt die gewünschte Höhe des Bereichs fest (ohne internen Innenabstand oder Tabs). Andernfalls wird die maximale Höhe aller Bereiche verwendet.

**-padding (padding/Padding)**

Legt fest, wie viel zusätzlicher Platz außen um das Notebook herum hinzugefügt wird. Der Innenabstand ist eine Liste mit bis zu vier Längenangaben *left top right bottom*. Sind weniger als vier Elemente angegeben, gilt standardmäßig *bottom* = *top*, *right* = *left* und *top* = *left*. Eine Liste mit drei Zahlen gibt den linken, vertikalen und rechten Innenabstand an; zwei Zahlen den horizontalen und vertikalen; eine einzelne Zahl denselben Innenabstand rund um das Widget.

**-width (width/Width)**

Wenn vorhanden und größer als null, legt die gewünschte Breite des Bereichs fest (ohne internen Innenabstand). Andernfalls wird die maximale Breite aller Bereiche verwendet.

## Tab-Optionen

Die folgenden Optionen können für einzelne Notebook-Bereiche angegeben werden:

**-state (state/State)**

Entweder **normal**, **disabled** oder **hidden**. Bei **disabled** ist der Tab nicht auswählbar. Bei **hidden** wird der Tab nicht angezeigt.

**-sticky (sticky/Sticky)**

Legt fest, wie das Inhaltsfenster innerhalb des Bereichs positioniert wird. Der Wert ist ein String mit null oder mehr der Zeichen **n**, **s**, **e** oder **w**. Jeder Buchstabe bezeichnet eine Seite (Norden, Süden, Osten oder Westen), an der das Inhaltsfenster "klebt", entsprechend dem Geometriemanager grid.

**-padding (padding/Padding)**

Legt fest, wie viel zusätzlicher Platz zwischen dem Notebook und diesem Bereich hinzugefügt wird. Die Syntax entspricht der Widget-Option **-padding**.

**-text (text/Text)**

Legt einen String fest, der im Tab angezeigt wird.

**-image (image/Image)**

Legt ein Bild fest, das im Tab angezeigt wird. Siehe *ttk_widget(n)* für Details.

**-compound (compound/Compound)**

Legt fest, wie das Bild relativ zum Text angezeigt wird, wenn sowohl **-text** als auch **-image** vorhanden sind. Siehe *label(n)* für zulässige Werte.

**-underline (underline/Underline)**

Legt den ganzzahligen Index (0-basiert) eines zu unterstreichenden Zeichens im Textstring fest. Das unterstrichene Zeichen wird für die Mnemonik-Aktivierung verwendet, wenn **ttk::notebook::enableTraversal** aufgerufen wird.

## Tab-Kennungen

Das Argument *tabid* der folgenden Befehle kann eine der folgenden Formen annehmen:

- Eine ganze Zahl zwischen null und der Anzahl der Tabs;
- Der Name eines Inhaltsfensters;
- Eine Positionsangabe der Form "@*x*,*y*", die den Tab identifiziert
- Der Literalstring "**current**", der den aktuell ausgewählten Tab bezeichnet; oder:
- Der Literalstring "**end**", der die Anzahl der Tabs zurückgibt (nur gültig für "*pathname* **index**").

Indizes unterstützen dieselbe einfache Interpretation wie beim Befehl string index, mit einfacher ganzzahliger Indexarithmetik und Indexierung relativ zu **end**.

## Widget-Befehl

Zusätzlich zu den Standardbefehlen **cget**, **configure**, **instate**, **state** und **style** (siehe ttk::widget) unterstützen Notebook-Widgets folgende zusätzliche Befehle:

***pathname* add *window* ?*options...*?**

Fügt dem Notebook einen neuen Tab hinzu. Siehe Tab-Optionen für die Liste der verfügbaren *options*. Wenn *window* bereits vom Notebook verwaltet, aber ausgeblendet ist, wird es an seine frühere Position zurückgesetzt.

***pathname* forget *tabid***

Entfernt den durch *tabid* angegebenen Tab, blendet das zugehörige Fenster aus und beendet die Verwaltung.

***pathname* hide *tabid***

Blendet den durch *tabid* angegebenen Tab aus. Der Tab wird nicht angezeigt, das zugehörige Fenster bleibt aber vom Notebook verwaltet und seine Konfiguration wird gespeichert. Ausgeblendete Tabs können mit dem Befehl **add** wiederhergestellt werden.

***pathname* identify *component x y***

Gibt den Namen des Elements unter dem Punkt *x* und *y* zurück oder den leeren String, wenn an dieser Position kein Element vorhanden ist. Folgende Unterbefehle werden unterstützt:

***pathname* identify element *x y***

Gibt den Namen des Elements an der angegebenen Position zurück.

***pathname* identify tab *x y***

Gibt den Index des Tabs an der angegebenen Position zurück.

***pathname* index *tabid***

Gibt den numerischen Index des durch *tabid* angegebenen Tabs zurück oder die Gesamtanzahl der Tabs, wenn *tabid* der String "**end**" ist.

***pathname* insert *pos subwindow options...***

Fügt einen Bereich an der angegebenen Position ein. *pos* ist entweder der String **end**, ein ganzzahliger Index oder der Name eines verwalteten Unterfensters. Wenn *subwindow* bereits vom Notebook verwaltet wird, wird es an die angegebene Position verschoben. Siehe Tab-Optionen für die Liste der verfügbaren Optionen.

***pathname* select ?*tabid*?**

Wählt den angegebenen Tab aus. Das zugehörige Inhaltsfenster wird angezeigt, das zuvor ausgewählte Fenster (falls abweichend) wird ausgeblendet. Wenn *tabid* weggelassen wird, gibt der Befehl den Widget-Namen des aktuell ausgewählten Bereichs zurück.

***pathname* tab *tabid* ?-*option* ?*value* ...?**

Fragt die Optionen des angegebenen Tabs ab oder ändert sie. Wenn keine -*option* angegeben ist, gibt ein Dictionary der Tab-Optionswerte zurück. Ist eine -*option* angegeben, gibt der Wert dieser *option* zurück. Andernfalls werden die -*option*en auf die entsprechenden *value*s gesetzt. Siehe Tab-Optionen für die verfügbaren Optionen.

***pathname* tabs**

Gibt die Liste der vom Notebook verwalteten Fenster in der Indexreihenfolge ihrer zugehörigen Tabs zurück.

## Tastatur-Navigation

Um die Tastaturnavigation für ein Toplevel-Fenster mit einem Notebook-Widget *$nb* zu aktivieren, rufen Sie auf:

```
ttk::notebook::enableTraversal $nb
```

Dies erweitert die Bindungen für das Toplevel-Fenster, das das Notebook enthält, wie folgt:

- **Control-Tab** wählt den Tab nach dem aktuell ausgewählten aus.
- **Control-Shift-Tab** wählt den Tab vor dem aktuell ausgewählten aus.
- **Alt-***K*, wobei *K* das mnemonische (unterstrichene) Zeichen eines Tabs ist, wählt diesen Tab aus.

Mehrere Notebooks in einem Toplevel können für die Navigation aktiviert werden, einschließlich verschachtelter Notebooks. Die Notebook-Navigation funktioniert jedoch nur korrekt, wenn alle Bereiche direkte Kinder des Notebooks sind.

## Virtuelle Ereignisse

Das Notebook-Widget erzeugt nach der Auswahl eines neuen Tabs ein virtuelles Ereignis **<>**.

## Beispiel

```
pack [ttk::notebook .nb]
.nb add [frame .nb.f1] -text "First tab"
.nb add [frame .nb.f2] -text "Second tab"
.nb select .nb.f2
ttk::notebook::enableTraversal .nb
```

## Styling-Optionen

Der Klassenname für ein **ttk::notebook** ist **TNotebook**. Der Tab hat den Klassennamen **TNotebook.Tab**

Dynamische Zustände: **active**, **disabled**, **selected**.

**TNotebook**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-background** *color*

**-bordercolor** *color*

**-darkcolor** *color*

**-foreground** *color*

**-lightcolor** *color*

**-padding** *padding*

**-tabmargins** *padding*

**-tabposition** ***position***

Legt die Position der Tab-Zeile oder -Spalte als String der Länge 1 oder 2 fest. Das erste Zeichen gibt die Seite als **n**, **s**, **w** oder **e** an, das zweite Zeichen (falls vorhanden) ist der Sticky-Bit (angegeben als **w**, **e**, **n** oder **s**) innerhalb der Tab-Position. Die Standardposition ist **n** für das Theme **aqua** und **nw** für alle anderen eingebauten Themes.

**-tabplacement** ***placement***

Legt die Platzierung (den Anker) der Tabs innerhalb der Tab-Zeile oder -Spalte als String der Länge 1 oder 2 fest. Das erste Zeichen gibt die Seite als **w**, **e**, **n** oder **s** an, das zweite Zeichen (falls vorhanden) ist der Sticky-Bit (angegeben als **s**, **n**, **e** oder **w**) innerhalb der Tab-Platzierung. Die Platzierung **ws** heißt zum Beispiel: die Westseite (links) jedes Tabs an den vorherigen hängen und die Südkanten (unten) aneinander ausrichten. Entsprechend hängt **se** die Südseite (unten) jedes Tabs an den vorherigen und richtet die Ostkanten (rechts) aneinander aus. Die Standardplatzierung hängt vom ersten Zeichen der Tab-Position ab: **ws** für **n***, **wn** für **s***, **ne** für **w*** und **nw** für **e***.

**TNotebook.Tab**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-background** *color*

**-bordercolor** *color*

**-compound** *compound*

**-expand** ***padding***

Legt fest, wie stark der Tab in der Größe wächst. Wird üblicherweise mit dem dynamischen Zustand **selected** verwendet. **-tabmargins** sollte entsprechend gesetzt werden, damit Platz für das Tab-Wachstum vorhanden ist. Die Ttk-Bibliotheksdatei **vistaTheme.tcl** enthält beispielsweise die Zeilen

```
ttk::style configure TNotebook -tabmargins {2 2 2 0}
ttk::style map TNotebook.Tab -expand {selected {2 2 2 2}}
```

die für den Standardwert **nw** der Stiloption **-tabposition** gültig sind. Für einen **ttk::notebook**-Stil **nbStyle**, definiert durch

```
set nbStyle SW.TNotebook
ttk::style configure $nbStyle -tabposition sw
```

müssen die obigen Einstellungen wie folgt angepasst werden:

```
ttk::style configure $nbStyle -tabmargins {2 0 2 2}
ttk::style map $nbStyle.Tab -expand {selected {2 2 2 2}}
```

Am einfachsten geschieht dies durch Aufruf der Bibliotheksprozedur **ttk::configureNotebookStyle** mit **$nbStyle** als Argument, nachdem die Stiloption **-tabposition** gesetzt wurde.

**-font** *font*

**-foreground** *color*

**-padding** ***padding***

Einige Themes verwenden einen anderen *padding* für den ausgewählten Tab. Die

Ttk-Bibliotheksdatei **clamTheme.tcl** enthält beispielsweise die Zeilen

```
ttk::style configure TNotebook.Tab \
    -padding {4.5p 1.5p 4.5p 1.5p}
ttk::style map TNotebook.Tab \
    -padding {selected {4.5p 3p 4.5p 1.5p}}
```

die für den Standardwert **nw** der Stiloption **-tabposition** gültig sind. Für einen **ttk::notebook**-Stil mit abweichender Tab-Position müssen die obigen Einstellungen entsprechend angepasst werden. Am einfachsten geschieht dies durch Aufruf der Bibliotheksprozedur **ttk::configureNotebookStyle** mit dem Stilnamen als Argument, nachdem die Stiloption **-tabposition** gesetzt wurde.

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2004 Joe English*

# ttk_panedwindow

*Stichwörter:* ttk_panedwindow

## Name

ttk::panedwindow - Container-Fenster mit mehreren Bereichen

## Übersicht

**ttk::panedwindow** *pathname* ?*options*?
*pathname* **add** *window* ?*options...*?
*pathname* **insert** *index window* ?*options...*?

## Beschreibung

Ein **ttk::panedwindow**-Widget zeigt eine Anzahl von Unterfenstern an, entweder vertikal oder horizontal gestapelt. Der Benutzer kann die relativen Größen der Unterfenster anpassen, indem er den Sash zwischen den Bereichen zieht.

Ist ein Bereich kein Kind des Panedwindows, muss er in der Stapelreihenfolge über dem Panedwindow liegen, sonst verdeckt das Panedwindow ihn. Am einfachsten erzeugen Sie das Panedwindow zuerst: das zuletzt erzeugte Fenster liegt oben in der Stapelreihenfolge. Oder Sie verwenden die Befehle raise und lower.

## Standardoptionen

**-class**, **-cursor**, **-takefocus**, **-style**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-orient (orient/Orient)**

Legt die Ausrichtung des Fensters fest. Bei **vertical** werden Unterbereiche von oben nach unten gestapelt; bei **horizontal** von links nach rechts.

**-width (width/Width)**

Wenn vorhanden und größer als null, legt die gewünschte Breite des Widgets in Pixeln fest.

Andernfalls wird die angeforderte Breite durch die Breite der verwalteten Fenster bestimmt.

**-height (height/Height)**

Wenn vorhanden und größer als null, legt die gewünschte Höhe des Widgets in Pixeln fest. Andernfalls wird die angeforderte Höhe durch die Höhe der verwalteten Fenster bestimmt.

## Bereichsoptionen

Die folgenden Optionen können für jeden Bereich angegeben werden:

**-weight (weight/Weight)**

Eine ganze Zahl, die die relative Dehnbarkeit des Bereichs angibt. Wenn das Paned-Window in der Größe geändert wird, wird der zusätzliche Platz jedem Bereich proportional zu seinem **-weight** hinzugefügt oder entzogen.

## Widget-Befehl

Zusätzlich zu den Standardbefehlen **cget**, **configure**, **instate**, **state** und **style** (siehe ttk::widget) unterstützen Panedwindow-Widgets folgende zusätzliche Befehle:

***pathname* add *subwindow options...***

Fügt dem Fenster einen neuen Bereich hinzu. Siehe Bereichsoptionen für die Liste der verfügbaren Optionen.

***pathname* forget *pane***

Entfernt den angegebenen Unterbereich aus dem Widget. *pane* ist entweder ein ganzzahliger Index oder der Name eines verwalteten Unterfensters.

***pathname* identify *component x y***

Gibt den Namen des Elements unter dem Punkt *x* und *y* zurück oder den leeren String, wenn an dieser Position kein Element vorhanden ist. Wenn *component* weggelassen wird, ist der Standardwert **sash**. Folgende Unterbefehle werden unterstützt:

***pathname* identify element *x y***

Gibt den Namen des Elements an der angegebenen Position zurück.

***pathname* identify sash *x y***

Gibt den Index des Sash an der angegebenen Position zurück.

***pathname* insert *pos subwindow options...***

Fügt einen Bereich an der angegebenen Position ein. *pos* ist entweder der String **end**, ein ganzzahliger Index oder der Name eines verwalteten Unterfensters. Wenn *subwindow* bereits vom Paned-Window verwaltet wird, wird es an die angegebene Position verschoben. Siehe Bereichsoptionen für die Liste der verfügbaren Optionen.

***pathname* pane *pane -option* ?*value* ?*-option value...*?**

Fragt die Optionen des angegebenen *pane* ab oder ändert sie, wobei *pane* entweder ein ganzzahliger Index oder der Name eines verwalteten Unterfensters ist. Wenn keine *-option* angegeben ist, gibt ein Dictionary der Bereichs-Optionswerte zurück. Ist eine *-option* angegeben, gibt der Wert dieser *option* zurück. Andernfalls werden die *-option*en auf die entsprechenden *value*s gesetzt.

***pathname* panes**

Gibt die Liste aller vom Widget verwalteten Fenster in der Indexreihenfolge ihrer zugehörigen Bereiche zurück.

***pathname* sashpos *index* ?*newpos*?**

Wenn *newpos* angegeben ist, setzt die Position des Sash mit der Nummer *index*. Kann die Positionen benachbarter Sashes anpassen, damit die Positionen monoton steigend bleiben. Sash-Positionen sind außerdem auf den Bereich zwischen 0 und der Gesamtgröße des Widgets beschränkt. Gibt die neue Position des Sash mit der Nummer *index* zurück.

## Virtuelle Ereignisse

Das Panedwindow-Widget erzeugt bei LeaveNotify/NotifyInferior-Ereignissen ein virtuelles Ereignis **<>**.

## Styling-Optionen

Der Klassenname für ein **ttk::panedwindow** ist **TPanedwindow**. Der Sash hat den Klassennamen **Sash**.

**TPanedwindow**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-background** *color*

**Sash**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-background** *color*

**-bordercolor** *color*

**-gripsize** *size* (Anzahl Bildschirmeinheiten)

**-handlepad** *amount*

**-handlesize** *amount*

**-lightcolor** *color*

**-sashpad** *amount*

**-sashrelief** *relief*

**-sashthickness** *amount*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2005 Joe English*

# ttk_progressbar

*Stichwörter:* ttk_progressbar

## Name

ttk::progressbar - Fortschrittsanzeige

## Übersicht

**ttk::progressbar** *pathName ?options?*

## Beschreibung

Ein **ttk::progressbar**-Widget zeigt den Status einer länger dauernden Operation. Es gibt zwei Modi: *determinate* zeigt den erledigten Anteil relativ zur Gesamtarbeit; *indeterminate* liefert eine Animation, damit der Benutzer sieht, dass etwas läuft.

Ist **-orient horizontal**, kann innerhalb der Fortschrittsleiste ein Text angezeigt werden —

konfigurierbar mit **-anchor**, **-font**, **-foreground**, **-justify**, **-text** und **-wraplength**. Bei **vertical** werden diese Optionen ignoriert. Standardoptionen (siehe **ttk_widget**): **-anchor**, **-class**, **-cursor**, **-font**, **-foreground**, **-justify**, **-style**, **-takefocus**, **-text**, **-wraplength**.

## Standardoptionen

**-anchor**, **-class**, **-cursor**, **-font**, **-foreground**, **-justify**, **-style**, **-takefocus**, **-text**, **-wraplength**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-length (length/Length)**

Länge der langen Achse der Fortschrittsleiste (Breite bei horizontal, Höhe bei vertikal). Wert in allen von Tk_GetPixels akzeptierten Formen.

**-maximum (maximum/Maximum)**

Gleitkommazahl für maximalen **-value**. Standard 100.

**-mode (mode/Mode)**

**determinate** oder **indeterminate**.

**-orient (orient/Orient)**

**horizontal** oder **vertical** — Ausrichtung der Fortschrittsleiste.

**-phase (phase/Phase)**

Nur-Lese-Option. Das Widget erhöht diesen Wert periodisch, wenn **-value** größer als 0 und im Modus *determinate* kleiner als **-maximum** ist. Das aktuelle Theme kann damit zusätzliche Animationen steuern.

**-value (value/Value)**

Aktueller Wert. Im Modus *determinate*: erledigter Anteil. Im Modus *indeterminate*: modulo **-maximum**; ein "Zyklus" ist abgeschlossen, wenn **-value** um **-maximum** steigt. Ist **-variable** gesetzt, hat **-value** keine Wirkung (Variable hat Vorrang).

**-variable (variable/Variable)**

Name einer globalen Tcl-Variable, verknüpft mit **-value**. Bei bestehender Variable wird **-value** automatisch bei Änderung der Variable gesetzt.

## Widget-Befehl

Zusätzlich zu **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget) unterstützen Progressbar-Widgets:

***pathName* start ?*interval*?**

Startet Auto-Inkrement: periodischer Timer ruft **step** alle *interval* Millisekunden auf. Fehlt *interval*, Standard 50 ms (20 Schritte/s).

***pathName* step ?*amount*?**

Erhöht **-value** um *amount*. Fehlt *amount*, Standard 1.0.

***pathName* stop**

Stoppt Auto-Inkrement und bricht den von *pathName* **start** gestarteten Timer ab.

## Styling-Optionen

Der Klassenname für ein **ttk::progressbar** ist **TProgressbar**.

**TProgressbar**-Styling-Optionen, die mit ttk::style konfiguriert werden können:

**-background** *color*

**-bordercolor** *color*

**-darkcolor** *color*

**-lightcolor** *color*

**-maxphase**
Für das Aqua-Theme.

**-period**
Für das Aqua-Theme.

**-troughcolor** *color*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2005 Joe English*

# ttk_radiobutton

*Stichwörter:* ttk_radiobutton, widget, button, option

## Name

ttk::radiobutton - Widget für sich gegenseitig ausschließende Optionen

## Übersicht

**ttk::radiobutton** *pathName ?options*?

## Beschreibung

**ttk::radiobutton**-Widgets werden in Gruppen verwendet, um eine Menge sich gegenseitig ausschließender Optionen anzuzeigen oder zu ändern. Radiobuttons sind mit einer Tcl-Variable verknüpft und haben einen zugehörigen Wert; beim Klick setzt ein Radiobutton die Variable auf diesen Wert.

## Standardoptionen

**-class**, **-compound**, **-cursor**, **-image**, **-state**, **-style**, **-takefocus**, **-text**, **-textvariable**, **-underline**, **-width**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-command (command/Command)**
Tcl-Skript, das bei jedem Aufruf des Widgets ausgewertet wird.

**-value (Value/Value)**
Wert, der in der zugehörigen **-variable** gespeichert wird, wenn das Widget ausgewählt ist.

**-variable (variable/Variable)**
Name einer globalen Variable, deren Wert mit dem Widget verknüpft ist. Standard ist **::selectedButton**.

## Widget-Befehl

Zusätzlich zu **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget) unterstützen Radiobutton-Widgets:

***pathname* invoke**

Setzt **-variable** auf **-value**, wählt das Widget aus und wertet **-command** aus. Liefert das Ergebnis von **-command** oder einen leeren String, wenn kein **-command** angegeben ist.

## Widget-Zustände

Bei gesetztem Zustand **disabled** reagiert das Widget nicht auf Benutzereingaben. Der Zustand **selected** wird gesetzt, wenn die verknüpfte **-variable** dem **-value** des Widgets entspricht, sonst gelöscht. Der Zustand **alternate** wird gesetzt, wenn die verknüpfte **-variable** nicht gesetzt ist (**alternate** kann eine "Drei-Zustand"- oder unbestimmte Auswahl anzeigen).

## Standardstile

**Ttk::radiobutton**-Widgets unterstützen in allen Standard-Themes den Stil **Toolbutton** — nützlich für Symbolleisten.

## Styling-Optionen

Der Klassenname für ein **ttk::radiobutton** ist **TRadiobutton**.

Dynamische Zustände: **active**, **alternate**, **disabled**, **pressed**, **readonly**, **selected**.

**TRadiobutton**-Styling-Optionen, die mit ttk::style konfiguriert werden können:

**-background** *color*

**-compound** *compound*

**-foreground** *color*

**-indicatorbackground** *color*

**-indicatorcolor** *color*

**-indicatormargin** *padding*

**-indicatorrelief** *relief*

**-padding** *padding*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2004 Joe English*

# ttk_scale

*Stichwörter:* ttk_scale, scale, slider, trough, widget

## Name

ttk::scale - Schieberegler-Widget erzeugen und steuern

## Übersicht

**ttk::scale** *pathName ?options...?*

## Beschreibung

Ein **ttk::scale**-Widget steuert typischerweise den numerischen Wert einer verknüpften Variable über einen Bereich. Es zeigt einen *Schieberegler* auf einer *Rinne*; die relative Position gibt den

Wert an.

## Standardoptionen

**-class**, **-cursor**, **-style**, **-takefocus**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-command (command/Command)**

Legt das Präfix eines Tcl-Befehls fest, der aufgerufen wird, wenn sich der Wert der Scale über einen Widget-Befehl ändert. Der tatsächliche Befehl besteht aus dieser Option gefolgt von einem Leerzeichen und einer reellen Zahl, die den neuen Wert der Scale angibt.

**-from (from/From)**

Reeller Wert für das linke bzw. obere Ende der Skala.

**-length (length/Length)**

Gewünschte Längenmaß der langen Dimension in Bildschirmeinheiten (Tk_GetPixels). Bei vertikaler Ausrichtung die Höhe, bei horizontaler die Breite.

**-orient (orient/Orient)**

**horizontal** oder **vertical** (oder Abkürzung) — Layout-Ausrichtung.

**-to (to/To)**

Reeller Wert für das rechte bzw. untere Ende; kann kleiner oder größer als **-from** sein.

**-value (value/Value)**

Aktueller Gleitkommawert. Ist **-variable** gesetzt, hat **-value** keine Wirkung (Variable hat Vorrang).

**-variable (variable/Variable)**

Name einer globalen Variable; bei Änderung aktualisiert sich die Skala; bei interaktiver Bedienung wird die Variable angepasst.

## Widget-Befehl

Zusätzlich zu **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget):

***pathName* get ?*x y*?**

Aktuellen **-value** oder den Wert zu den Pixelkoordinaten *x*, *y* (relativ zum Widget-Ursprung).

***pathName* set *value***

**-value** auf *value* setzen, begrenzt auf **-from**/**-to**. Setzen der verknüpften Variable führt nicht zu dieser Begrenzung.

## Interne Befehle

***pathName* coords ?*value*?**

Koordinaten zu *value* oder zum aktuellen **-value**, falls *value* fehlt.

## Styling-Optionen

Der Klassenname für ein **ttk::scale** ist **TScale**.

Dynamische Zustände: **active**.

**TScale**-Styling-Optionen mit ttk::style:

**-background** *color*

**-borderwidth** *amount*

**-darkcolor** *color*

**-groovewidth** *amount*

**-lightcolor** *color*

**-sliderwidth** *amount*

**-troughcolor** *color*

**-troughrelief** *relief*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe ttk::style zur Konfiguration von ttk-Stilen.

*Copyright (c) 2008 Donal Fellows*

# ttk_scrollbar

*Stichwörter:* ttk_scrollbar, scrollbar, widget

## Name

ttk::scrollbar - Steuert den Viewport eines scrollbaren Widgets

## Übersicht

**ttk::scrollbar** *pathName* ?*options...*?

## Beschreibung

**ttk::scrollbar**-Widgets sind typischerweise mit einem zugehörigen Fenster verknüpft, das ein Dokument anzeigt, etwa eine bearbeitete Datei oder eine Zeichnung. Eine Scrollbar zeigt einen *Thumb* im mittleren Bereich an, dessen Position und Größe Auskunft über den im zugehörigen Fenster sichtbaren Dokumentanteil geben. Der Benutzer kann den Thumb ziehen, um den sichtbaren Bereich zu steuern. Je nach Theme können zwei oder mehr Pfeilbuttons vorhanden sein; diese dienen zum Scrollen des sichtbaren Bereichs in diskreten Einheiten.
Standardoptionen (siehe **ttk_widget**): **-class**, **-cursor**, **-style**, **-takefocus**.

## Standardoptionen

**-class**, **-cursor**, **-style**, **-takefocus**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-command (command/Command)**

Ein Tcl-Skript-Präfix, das ausgewertet wird, um die Ansicht im mit der Scrollbar verknüpften Widget zu ändern. Zusätzliche Argumente werden an den Wert dieser Option angehängt, wie in Scroll-Befehle unten beschrieben, wann immer der Benutzer durch Manipulation der Scrollbar eine Ansichtsänderung anfordert.

Diese Option besteht typischerweise aus einer Liste mit zwei Elementen: dem Namen eines scrollbaren Widgets, gefolgt von entweder **xview** (für horizontale Scrollbars) oder **yview** (für vertikale Scrollbars).

**-orient (orient/Orient)**

Einer von **horizontal** oder **vertical**. Legt die Ausrichtung der Scrollbar fest.

## Widget-Befehl

Zusätzlich zu den Standardbefehlen **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget) unterstützen Scrollbar-Widgets folgende zusätzliche Befehle:

***pathName* get**

Gibt die Scrollbar-Einstellungen in Form einer Liste zurück, deren Elemente die Argumente des zuletzt ausgeführten Widget-Befehls **set** sind.

***pathName* set *first last***

Dieser Befehl wird normalerweise vom mit der Scrollbar verknüpften Widget aus einem **-xscrollcommand**- oder **-yscrollcommand**-Callback aufgerufen. Legt den anzuzeigenden sichtbaren Bereich fest. *first* und *last* sind reelle Bruchteile zwischen 0 und 1.

## Interne Befehle

Die folgenden Widget-Befehle werden intern von den Bindungen der Widget-Klasse **TScrollbar** verwendet.

***pathName* delta *deltaX deltaY***

Gibt eine reelle Zahl zurück, die die fraktionale Änderung der Scrollbar-Einstellung angibt, die einer gegebenen Änderung der Thumb-Position entspricht. Ist die Scrollbar horizontal, gibt das Ergebnis an, um wie viel sich die Scrollbar-Einstellung ändern muss, um den Thumb *deltaX* Pixel nach rechts zu bewegen (*deltaY* wird in diesem Fall ignoriert). Ist die Scrollbar vertikal, gibt das Ergebnis an, um wie viel sich die Einstellung ändern muss, um den Thumb *deltaY* Pixel nach unten zu bewegen. Die Argumente und das Ergebnis können null oder negativ sein.

***pathName* fraction *x y***

Gibt eine reelle Zahl zwischen 0 und 1 zurück, die angibt, wo der durch *x* und *y* gegebene Punkt im Trough-Bereich der Scrollbar liegt, wobei 0.0 dem oberen oder linken Rand des Troughs und 1.0 dem unteren oder rechten Rand entspricht. *X* und *y* sind Pixelkoordinaten relativ zum Scrollbar-Widget. Liegen *x* und *y* außerhalb des Troughs, wird der nächstgelegene Punkt im Trough verwendet.

## Scroll-Befehle

Wenn der Benutzer mit der Scrollbar interagiert, etwa durch Ziehen des Thumbs, benachrichtigt die Scrollbar das zugehörige Widget, dass es seine Ansicht ändern muss. Die Scrollbar erfolgt dies durch Auswertung eines Tcl-Befehls, der aus der **-command**-Option der Scrollbar erzeugt wird. Der Befehl kann eine der folgenden Formen annehmen. In jedem Fall ist *prefix* der Inhalt der **-command**-Option, der üblicherweise die Form **.t yview** hat

***prefix* moveto *fraction***

*Fraction* ist eine reelle Zahl zwischen 0 und 1. Das Widget soll seine Ansicht so anpassen, dass der durch *fraction* angegebene Punkt am Anfang des Widgets erscheint. Ist *fraction* 0, bezeichnet dies den Anfang des Dokuments. 1.0 bezeichnet das Ende, 0.333 einen Punkt ein Drittel durch das Dokument usw.

***prefix* scroll *number* pages**

Das Widget soll seine Ansicht um *number* Seiten anpassen. Die Bedeutung einer Seite legt das Widget fest; typischerweise ist sie etwas kleiner als der sichtbare Fensterinhalt, sodass alte und neue Ansicht leicht überlappen. *Number* ist entweder 1 (nächste Seite sichtbar) oder -1 (vorherige Seite sichtbar).

***prefix* scroll *number* units**

Das Widget soll seine Ansicht um *number* Einheiten anpassen. Die Einheiten sind widget-spezifisch definiert, etwa Zeichen oder Zeilen in einem Text-Widget. *Number* ist entweder 1 (eine Einheit scrollt oben oder links aus dem Fenster) oder -1 (eine Einheit scrollt unten oder rechts aus dem Fenster).

## Widget-Zustände

Die Scrollbar setzt automatisch das Zustandsbit **disabled**, wenn der gesamte Bereich sichtbar ist (Bereich 0.0 bis 1.0), und löscht es andernfalls. Sie setzt außerdem die Zustandsflags **active** und **pressed** einzelner Elemente basierend auf Position und Zustand des Mauszeigers.

## Beispiel

```
set f [frame .f]
ttk::scrollbar $f.hsb -orient horizontal -command [list $f.t xview]
ttk::scrollbar $f.vsb -orient vertical -command [list $f.t yview]
text $f.t -xscrollcommand [list $f.hsb set] -yscrollcommand [list
    $f.vsb set]
grid $f.t -row 0 -column 0 -sticky nsew
grid $f.vsb -row 0 -column 1 -sticky nsew
grid $f.hsb -row 1 -column 0 -sticky nsew
grid columnconfigure $f 0 -weight 1
grid rowconfigure $f 0 -weight 1
pack $f
```

## Styling-Optionen

Der Klassenname für eine **ttk::scrollbar** ist **TScrollbar**.

Dynamische Zustände: **active**, **disabled**.

**TScrollbar**-Styling-Optionen (genauer **Vertical.TScrollbar** und **Horizontal.TScrollbar**), die mit ttk::style konfiguriert werden können, sind:

**-arrowcolor** *color*

**-arrowsize** *amount*

**-background** *color*

**-bordercolor** *color*

**-darkcolor** *color* (Farbe des dunklen Teils des 3D-Reliefs)

**-foreground** *color*

**-gripsize** *size* (Anzahl Bildschirmeinheiten)

**-lightcolor** *color* (Farbe des hellen Teils des 3D-Reliefs)

**-troughcolor** *color*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 1994-1996 Sun Microsystems, Inc.*
*Copyright (c) 2004 Joe English*

# ttk_separator

*Stichwörter:* ttk_separator, widget, separator

## Name

ttk::separator - Trennlinie

## Übersicht

**ttk::separator** *pathName* ?*options*?

## Beschreibung

Ein **ttk::separator**-Widget zeigt einen horizontalen oder vertikalen Trennstrich an.

## Standardoptionen

**-class**, **-cursor**, **-style**, **-takefocus**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-orient (orient/Orient)**

Einer von **horizontal** oder **vertical**. Legt die Ausrichtung des Trennstrichs fest.

## Widget-Befehl

Separator-Widgets unterstützen die Standardbefehle **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget).

## Styling-Optionen

Der Klassenname für ein **ttk::separator** ist **TSeparator**.

**TSeparator**-Styling-Optionen, die mit ttk::style konfiguriert werden können:

**-background** *color*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2004 Joe English*

# ttk_sizegrip

*Stichwörter:* ttk_sizegrip, widget, sizegrip, grow box

## Name

ttk::sizegrip - Widget zum Vergrößern in der unteren rechten Ecke

## Übersicht

**ttk::sizegrip** *pathName* ?*options*?

## Beschreibung

Ein **ttk::sizegrip**-Widget (auch *grow box* genannt) ermöglicht dem Benutzer, das enthaltende

Toplevel-Fenster durch Drücken und Ziehen des Griffs zu vergrößern oder zu verkleinern.

## Standardoptionen

**-class**, **-cursor**, **-style**, **-takefocus**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-Befehl

Sizegrip-Widgets unterstützen die Standardbefehle **cget**, **configure**, **identify element**, **instate**, **state** und **style** (siehe ttk::widget).

## Plattform-Hinweise

Unter macOS enthalten Toplevel-Fenster standardmäßig einen eingebauten Sizegrip. Ein zusätzliches **ttk::sizegrip** ist dort unkritisch, da der eingebaute Griff das Widget einfach überdeckt.

## Beispiele

Mit **pack**:

```
pack [ttk::frame $top.statusbar] -side bottom -fill x
pack [ttk::sizegrip $top.statusbar.grip] -side right -anchor se
```

Mit **grid**:

```
grid [ttk::sizegrip $top.statusbar.grip] \
    -row $lastRow -column $lastColumn -sticky se
# ... optional: vertikale Scrollbar in $lastColumn,
# ... optional: horizontale Scrollbar in $lastRow
```

## Bugs

Wurde die Position des enthaltenden Toplevels relativ zur rechten oder unteren Bildschirmkante angegeben (z. B. "**wm geometry ...** *w***x***h***-***x***-***y*" statt "**wm geometry ...** *w***x***h***+***x***+***y*"), vergrößert das Sizegrip-Widget das Fenster nicht.

**ttk::sizegrip**-Widgets unterstützen nur die Vergrößerung in Richtung "southeast".

## Styling-Optionen

Der Klassenname für ein **ttk::sizegrip** ist **TSizegrip**.

**TSizegrip**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-background** *color*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2006 Joe English*

# ttk_spinbox

*Stichwörter:* ttk_spinbox, entry, spinbox, widget, text field

## Name

ttk::spinbox - Widget mit auswählbarem Textfeld

## Übersicht

**ttk::spinbox** *pathName* ?*options*?

## Beschreibung

Ein **ttk::spinbox**-Widget ist ein ttk::entry-Widget mit eingebauten Auf- und Ab-Schaltflächen, mit denen entweder ein numerischer Wert geändert oder aus einer Menge von Werten ausgewählt werden kann. Das Widget implementiert alle Funktionen des ttk::entry-Widgets, einschließlich der Unterstützung der Option **-textvariable**, um den angezeigten Wert mit einer Tcl-Variable zu verknüpfen.

## Standardoptionen

**-class**, **-cursor**, **-state**, **-style**, **-takefocus**, **-xscrollcommand**, **-placeholder**, **-placeholderforeground**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

**-validate**, **-validatecommand**

Siehe die Handbuchseite ttk_entry für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-command (command/Command)**

Legt ein Tcl-Skript fest, das ausgewertet wird, wenn eine Spin-Schaltfläche betätigt wird.

**-format (format/Format)**

Legt ein alternatives Format fest, das beim Setzen des Zeichenkettenwerts im Bereich **-from** und **-to** verwendet wird. Es muss ein Formatbezeichner der Form **%.f** sein, da damit eine Gleitkommazahl formatiert wird.

**-from (from/From)**

Gleitkommazahl für den kleinsten Wert der Spinbox. Wird zusammen mit **-to** und **-increment** verwendet, um einen numerischen Bereich festzulegen.

**-increment (increment/Increment)**

Gleitkommazahl für die Wertänderung bei jedem Drücken einer Spin-Schaltfläche. Die Auf-Schaltfläche wendet ein positives Inkrement an, die Ab-Schaltfläche ein negatives.

**-to (to/To)**

Gleitkommazahl für den höchsten zulässigen Wert des Widgets. Siehe auch **-from** und **-increment**.

**-values (values/Values)**

Muss eine Tcl-Liste von Werten sein. Ist diese Option gesetzt, überschreibt sie jeden mit **-from** , **-to** und **-increment** festgelegten Bereich; das Widget verwendet stattdessen die angegebenen Werte, beginnend mit dem ersten.

**-wrap (wrap/Wrap)**

Muss ein gültiger boolescher Wert sein. Ist er an, springt die Spinbox an den Enden der Daten im Widget um.

## Indizes

Siehe die Handbuch-Seite ttk::entry für Informationen zur Indizierung von Zeichen.

## Validierung

Siehe die Handbuch-Seite ttk::entry für Informationen zur Verwendung der Optionen **-validate** und **-validatecommand**.

## Widget-Befehl

Für Spinbox-Widgets sind zusätzlich zu den in der Handbuch-Seite ttk::entry beschriebenen Befehlen folgende Unterbefehle möglich:

***pathName*** **get**

Gibt den aktuellen Wert der Spinbox zurück.

***pathName*** **set** ***value***

Setzt die Zeichenkette der Spinbox auf *value*. Ist die Option **-format** konfiguriert, wird dieses Format angewendet. Schlägt die Formatierung fehl, ist keine Formatierung gesetzt oder wurde die Option **-values** verwendet, wird der Wert direkt gesetzt.

## Virtuelle Ereignisse

Das Spinbox-Widget erzeugt ein virtuelles Ereignis **<<Increment>>**, wenn der Benutzer drückt, und ein virtuelles Ereignis **<<Decrement>>**, wenn er drückt.

## Styling-Optionen

Der Klassenname für ein **ttk::spinbox** ist **TSpinbox**.

Dynamische Zustände: **active**, **disabled**, **focus**, **readonly**.

**TSpinbox**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-arrowcolor** *color*

**-arrowsize** *amount*

**-background** ***color***

Aus Kompatibilitätsgründen verhält sich diese Option beim Aqua-Theme (für macOS) als Alias für **-fieldbackground**, sofern für **-fieldbackground** kein Wert angegeben ist; andernfalls wird sie ignoriert.

**-bordercolor** *color*

**-darkcolor** *color*

**-fieldbackground** *color*

**-foreground** *color*

**-insertcolor** *color*

**-insertwidth** *amount*

**-lightcolor** *color*

**-padding** *padding*

**-placeholderforeground** *color*

**-selectbackground** *color*

**-selectforeground** *color*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2008 Pat Thoyts*

# ttk_style

*Stichwörter:* ttk_style, style, theme, appearance

## Name

ttk::style - Manipuliert die Style-Datenbank

## Übersicht

**ttk::style** *option* ?*args*?

## Hinweise

Siehe auch die Tcl'2004-Konferenzpräsentation unter https://tktable.sourceforge.net/tile/tile-tcl2004.pdf

## Definitionen

Jedem Widget wird ein *style* zugewiesen, der die Menge der Elemente bestimmt, aus denen das Widget besteht, und wie sie angeordnet sind, sowie dynamische und Standard-Einstellungen für Elementoptionen. Standardmäßig entspricht der Stilname der Widget-Klasse; dies kann mit der Option **-style** überschrieben werden.

Ein *theme* ist eine Sammlung von Elementen und Styles, die das Gesamterscheinungsbild einer Anwendung steuert. Der Style "." ist der Theme-Wurzelstil, auf dem abgeleitete Styles basieren.

## Beschreibung

Der Befehl **ttk::style** akzeptiert folgende Argumente:

**ttk::style configure *style* ?*-option* ?*value option value...*? ?**

Legt den Standardwert der angegebenen Option(en) in *style* fest. Existiert *style* nicht, wird er erstellt. Sind nur *style* und *-option* angegeben, wird der Standardwert der Option *-option* von Style *style* abgerufen. Ist nur *style* angegeben, werden die Standardwerte aller Optionen von Style *style* abgerufen.

**ttk::style element *args***

siehe unten ... **ttk::style element create** *elementName type* ?*args...*?

Erstellt ein neues Element im aktuellen Theme vom Typ *type*. Der einzige plattformübergreifende eingebaute Elementtyp ist *image* (siehe ttk_image(n)); Themes können weitere Elementtypen definieren (siehe **Ttk_RegisterElementFactory**). Auf geeigneten Windows-Versionen ist eine Elementfabrik registriert, die Windows-Theme-Elemente erzeugt (siehe ttk_vsapi(n)).

**ttk::style element names**

Gibt die Liste der im aktuellen Theme definierten Elemente zurück.

**ttk::style element options *element***

Gibt die Liste der Optionen von *element* zurück.

**ttk::style layout *style* ?*layoutSpec*?**

Definiert das Widget-Layout für Style *style*. Siehe Layouts unten für das Format von *layoutSpec*. Wird *layoutSpec* weggelassen, wird die Layout-Spezifikation für Style *style* zurückgegeben.

**ttk::style lookup *style -option ?state ?default*?**

Gibt den für *-option* in Style *style* im Zustand *state* festgelegten Wert zurück, unter Verwendung der Standard-Lookup-Regeln für Elementoptionen. *state* ist eine Liste von Zustandsnamen; fehlt sie, entspricht der Standard alle Bits aus ("normal"-Zustand). Ist das Argument *default* vorhanden, wird es als Fallback verwendet, falls keine Spezifikation für *-option* gefunden wird. Existiert *style* nicht, wird er erstellt.

**ttk::style map *style* ?*-option { statespec value... }*?**

Legt dynamische (zustandsabhängige) Werte der angegebenen Option(en) in *style* fest. Jedes Paar *statespec / value* wird der Reihe nach geprüft; der Wert des ersten passenden *statespec* wird verwendet. Existiert *style* nicht, wird er erstellt. Sind nur *style* und *-option* angegeben, werden die dynamischen Werte der Option *-option* von Style *style* abgerufen. Ist nur *style* angegeben, werden die dynamischen Werte aller Optionen von Style *style* abgerufen.

**ttk::style theme *args***

siehe unten ... **ttk::style theme create** *themeName* ?*-parent basedon*? ?*-settings script...*?

Erstellt ein neues Theme. Existiert *themeName* bereits, ist das ein Fehler. Ist **-parent** angegeben, erbt das neue Theme Styles, Elemente und Layouts vom Eltern-Theme *basedon*. Ist **-settings** vorhanden, wird *script* im Kontext des neuen Themes gemäß **ttk::style theme settings** ausgewertet.

**ttk::style theme names**

Gibt eine Liste aller bekannten Themes zurück.

**ttk::style theme settings *themeName script***

Setzt das aktuelle Theme vorübergehend auf *themeName*, wertet *script* aus und stellt das vorherige Theme wieder her. Typischerweise definiert *script* nur Styles und Elemente, beliebiger Tcl-Code ist jedoch möglich.

**ttk::style theme styles ?*themeName*?**

Gibt eine Liste aller Styles in *themeName* zurück. Wird *themeName* weggelassen, wird das aktuelle Theme verwendet.

**ttk::style theme use ?*themeName*?**

Ohne Argument ist das Ergebnis der Name des aktuellen Themes. Andernfalls setzt dieser Befehl das aktuelle Theme auf *themeName* und aktualisiert alle Widgets.

## Layouts

Ein *layout* legt eine Liste von Elementen fest, jedes gefolgt von einer oder mehreren Optionen zur Anordnung des Elements. Der Layout-Mechanismus verwendet eine vereinfachte Variante des Geometriemanagers pack: Ausgehend von einem anfänglichen Hohlraum (*cavity*) erhält jedes Element ein *parcel* (Zuweisungsbereich). Anschließend wird das vom Element tatsächlich genutzte *parcel* innerhalb des zugewiesenen Bereichs angepasst. Gültige Optionen sind:

**-children *{ sublayout... }***

Legt eine Liste von Elementen fest, die innerhalb des Elements platziert werden.

**-expand *boolean***

Legt fest, ob das zugewiesene *parcel* der gesamte Hohlraum ist. Ist das der Fall, wird eine gleichzeitige Angabe von **-side** ignoriert. Standard ist 0.

**-side *side***

Legt fest, an welcher Seite des Hohlraums das Element platziert wird; einer von **left**, **right**,

**top** oder **bottom**. Beispielsweise weist **-side top** das *parcel* entlang der Oberkante des Hohlraums zu, mit Breite und Höhe jeweils der Breite des Hohlraums bzw. der Höhe des Elements. Fehlt die Angabe, ist das zugewiesene *parcel* der gesamte Hohlraum (gleicher Effekt wie **-expand** 1).

**-sticky [nswe]**

Legt Position und Größe des tatsächlichen *parcel* innerhalb des zugewiesenen Bereichs fest. Bei leerer Zeichenkette wird das tatsächliche *parcel* im zugewiesenen Bereich zentriert. Standard ist **nswe**.

Beispiel:

```
ttk::style layout Horizontal.TScrollbar {
    Scrollbar.trough -children {
        Scrollbar.leftarrow -side left
        Scrollbar.rightarrow -side right
        Horizontal.Scrollbar.thumb -side left -sticky ew
    }
}
```

## Wurzel-Stil

Der Theme-Wurzelstil "." kann wie jeder andere Style konfiguriert werden und liefert Standardwerte für abgeleitete Styles. Zusätzlich zu den üblichen Optionen sind für "." mit **ttk::style** konfigurierbare Styling-Optionen:

**-insertofftime *amount***

Legt einen nicht negativen Ganzzahlwert fest: die Anzahl Millisekunden, die der Einfügecursor in jedem Blinkzyklus "aus" bleiben soll. Ist die Option null, blinkt der Cursor nicht: er ist durchgehend ein. Standard 300 ms, sofern nicht durch eine **RESOURCE_MANAGER**-Property oder eine **.Xdefaults**-Datei überschrieben.

**-insertontime *amount***

Legt einen nicht negativen Ganzzahlwert fest: die Anzahl Millisekunden, die der Einfügecursor in jedem Blinkzyklus "ein" bleiben soll. Standard 600 ms, sofern nicht durch eine **RESOURCE_MANAGER**-Property oder eine **.Xdefaults**-Datei überschrieben.

*Copyright (c) 2004 Joe English*

# ttk_treeview

*Stichwörter:* ttk_treeview

## Name

ttk::treeview - Hierarchisches mehrspaltiges Datenanzeige-Widget

## Übersicht

**ttk::treeview** *pathname* ?*options*?

## Beschreibung

Das **ttk::treeview**-Widget zeigt eine hierarchische Sammlung von Elementen an. Jedes Element hat eine Textbeschriftung, ein optionales Bild und eine optionale Liste von Datenwerten. Die

Datenwerte werden in aufeinanderfolgenden Spalten nach der Baumspaltenbeschriftung angezeigt.

Die Reihenfolge, in der Datenwerte angezeigt werden, kann über die Widget-Option **-displaycolumns** gesteuert werden. Das Treeview-Widget kann auch Spaltenüberschriften anzeigen. Auf Spalten kann per Nummer oder über die in der Widget-Option **-columns** aufgeführten symbolischen Namen zugegriffen werden; siehe Spaltenidentifikatoren.

Jedes Element wird durch einen eindeutigen Namen identifiziert. Fehlen Element-IDs vom Aufrufer, erzeugt das Widget sie selbst. Es gibt ein ausgezeichnetes Wurzelelement mit dem Namen **{}**. Das Wurzelelement selbst wird nicht angezeigt; seine Kinder erscheinen auf der obersten Ebene der Hierarchie.

Jedes Element besitzt außerdem eine Liste von *Tags*, mit denen Ereignisbindungen einzelnen Elementen zugeordnet und das Erscheinungsbild des Elements gesteuert werden können.

Treeview-Widgets unterstützen horizontales und vertikales Scrollen mit den üblichen Optionen **-**[**xy**]**scrollcommand** und den Widget-Befehlen [**xy**]**view**.

## Standardoptionen

**-class**, **-cursor**, **-takefocus**, **-style**, **-xscrollcommand**, **-yscrollcommand**, **-padding**

Siehe die Handbuchseite ttk_widget für Einzelheiten zu den Standardoptionen.

## Widget-spezifische Optionen

**-columns (columns/Columns)**

Liste von Spaltenkennungen; legt die Anzahl der Spalten und deren Namen fest.

**-displaycolumns (displayColumns/DisplayColumns)**

Liste von Spaltenkennungen (symbolische Namen oder ganzzahlige Indizes), die festlegt, welche Datenspalten angezeigt werden und in welcher Reihenfolge, oder die Zeichenkette **#all**. Ist **#all** gesetzt (Standard), werden alle Spalten in der angegebenen Reihenfolge angezeigt.

**-height (height/Height)**

Legt die Anzahl der sichtbaren Zeilen fest. Hinweis: Die angeforderte Breite ergibt sich aus der Summe der Spaltenbreiten.

**-selectmode (selectMode/SelectMode)**

Legt fest, wie die eingebauten Klassenbindungen die Auswahl verwalten. Einer von **extended** , **browse** oder **none**.

Ist **extended** gesetzt (Standard), können mehrere Elemente ausgewählt werden. Bei **browse** ist jeweils nur ein Element ausgewählt. Bei **none** wird die Auswahl nicht geändert.

Hinweis: Anwendungscode und Tag-Bindungen können die Auswahl unabhängig vom Wert von **-selectmode** beliebig setzen.

**-selecttype (selectType/SelectType)**

Legt fest, wie die eingebauten Klassenbindungen die Auswahl verwalten. Einer von **item** oder **cell**.

**-show (show/Show)**

Liste mit null oder mehr der folgenden Werte; legt fest, welche Teile des Baums angezeigt werden.

**tree**

Baumspaltenbeschriftungen in Spalte #0 anzeigen.

**headings**

Die Überschriftenzeile anzeigen.

Der Standardwert ist **tree headings**.

**HINWEIS:** Spalte #0 bezeichnet immer die Baumspalte, auch wenn **-show tree** nicht gesetzt ist.

**-striped (striped/Striped)**

Boolescher Wert für Zebra-Streifen-Färbung der Elemente. Gestreifte Elemente verwenden die Option **-stripedbackground**, falls vom Theme oder einem Tag gesetzt. Unterstützt das aktuelle Theme dies nicht, ist nichts sichtbar.

**-titlecolumns (titleColumns/TitleColumns)**

Anzahl der linken Anzeigespalten, die nicht horizontal gescrollt werden. Die Baumspalte zählt mit, auch wenn **-show tree** nicht gesetzt ist. Bei Wert N dieser Option ist Spalte #N die erste scrollbare Spalte. Standard ist 0.

**-titleitems (titleItems/TitleItems)**

Anzahl der oberen Elemente, die nicht vertikal gescrollt werden. Standard ist 0.

## Widget-Befehl

Zusätzlich zu den Standardbefehlen **cget**, **configure**, **instate**, **state**, **style**, **xview** und **yview** (siehe ttk::widget) unterstützen Treeview-Widgets folgende zusätzliche Befehle:

***pathname* bbox *item* ?*column*?**

Gibt die Begrenzungsbox (relativ zum Fenster des Treeview-Widgets) des angegebenen *item* in der Form *x y width height* zurück. Ist *item* nicht sichtbar (z. B. Nachkomme eines geschlossenen Elements oder vertikal aus dem sichtbaren Bereich gescrollt), wird die leere Liste zurückgegeben. Ist *column* angegeben und nicht ausgeblendet (durch **-displaycolumns** ), wird die Begrenzungsbox dieser Zelle innerhalb von *item* zurückgegeben (auch wenn die Zelle horizontal aus dem sichtbaren Bereich gescrollt ist).

***pathname* cellselection ?*selop arg* ...?**

Verwaltet die Zellauswahl. Die Zellauswahl ist unabhängig von der Elementauswahl, die der Befehl **selection** verwaltet. Eine Zelle wird durch eine Liste mit zwei Elementen, item und column, angegeben. Bei Rechteck-Varianten der Befehle müssen die Zellen in angezeigten Spalten liegen. Jede Änderung von **-columns** löscht die Zellauswahl. Ein Argument *cellList* kann eine einzelne Zelle oder eine Liste von Zellen sein. Ist *selop* nicht angegeben, wird die Liste der ausgewählten Zellen zurückgegeben. Andernfalls ist *selop* einer der folgenden:

***pathname* cellselection set *cellList***

*cellList* wird die neue Zellauswahl.

***pathname* cellselection set *firstCell lastCell***

Das definierte Rechteck wird die neue Zellauswahl.

***pathname* cellselection add *cellList***

*cellList* zur Zellauswahl hinzufügen.

***pathname* cellselection add *firstCell lastCell***

Das definierte Rechteck zur Zellauswahl hinzufügen.

***pathname* cellselection remove *cellList***

*cellList* aus der Zellauswahl entfernen.

***pathname* cellselection remove *firstCell lastCell***

Das definierte Rechteck aus der Zellauswahl entfernen.

***pathname* cellselection toggle *cellList***

Den Zellauswahlzustand jeder Zelle in *cellList* umschalten. Ist das Element teilweise sichtbar, umfasst das Ergebnis den vollen Bereich des Elements einschließlich nicht sichtbarer Teile.

***pathname* cellselection toggle *firstCell lastCell***

Den Zellauswahlzustand jeder Zelle im definierten Rechteck umschalten.

***pathname* children *item* ?*newchildren*?**

Ist *newchildren* nicht angegeben, gibt die Liste der Kinder von *item* zurück.

Ist *newchildren* angegeben, ersetzt die Kinderliste von *item* durch *newchildren*. Elemente der alten Kinderliste, die nicht in der neuen Liste stehen, werden vom Baum getrennt. Keines der Elemente in *newchildren* darf ein Vorfahre von *item* sein.

***pathname* column *column* ?*-option ?value -option value...*?**

Abfragen oder Ändern der Optionen der angegebenen *column*. Ist kein *-option* angegeben, wird ein Dictionary der Option/Wert-Paare zurückgegeben. Ist genau ein *-option* angegeben, wird dessen Wert zurückgegeben. Andernfalls werden die Optionen mit den angegebenen Werten aktualisiert. Für jede Spalte können folgende Optionen gesetzt werden:

**-id *name***

Der Spaltenname. Schreibgeschützte Option. Beispiel: [*$pathname* **column #***n* **-id**] liefert die Datenspalte zur Anzeigespalte *n*. Die Baumspalte hat **-id #0**.

**-anchor *anchor***

Legt fest, wie der Text in dieser Spalte relativ zur Zelle ausgerichtet wird. *Anchor* ist einer von **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw** oder **center**.

**-minwidth *minwidth***

Mindestbreite der Spalte in Pixeln. Das Treeview-Widget macht die Spalte beim Größenändern oder beim Ziehen eines Spaltentrenners nicht schmaler als **-minwidth**. Standard 20 Pixel.

**-separator *boolean***

Legt fest, ob rechts von der Spalte ein Spaltentrenner gezeichnet wird. Standard false.

**-stretch *boolean***

Legt fest, ob die Spaltenbreite beim Größenändern des Widgets oder beim Ziehen eines Spaltenüberschrift-Trenners angepasst wird. *Boolean* kann jede von Tcl_GetBoolean akzeptierte Form haben. Standardmäßig sind Spalten dehnbar.

**-width *width***

Breite der Spalte in Pixeln. Standard 200 Pixel. Die angegebene Breite kann von Tk geändert werden, um **-stretch** und/oder **-minwidth** zu berücksichtigen oder bei Größenänderung bzw. Ziehen eines Spaltentrenners.

Verwenden Sie *pathname* **column #0**, um die Baumspalte zu konfigurieren.

***pathname* delete *itemList***

Löscht jedes Element in *itemList* und alle Nachkommen. Das Wurzelelement darf nicht gelöscht werden. Siehe auch: **detach**.

***pathname* detach *itemList***

Trennt alle angegebenen Elemente in *itemList* vom Baum. Die Elemente und alle Nachkommen bleiben vorhanden und können mit **move** an anderer Stelle wieder eingefügt werden, werden aber erst dann angezeigt. Das Wurzelelement darf nicht getrennt werden. Siehe auch: **delete**.

***pathname* detached ?*item*?**

Ist *item* angegeben, gibt einen booleschen Wert zurück, ob es der Name eines getrennten Elements ist (siehe **detach**). Andernfalls eine Liste aller getrennten Elemente (in beliebiger Reihenfolge). Das Wurzelelement ist nie getrennt.

***pathname* exists *item***

Gibt 1 zurück, wenn *item* im Baum vorhanden ist, sonst 0.

***pathname* focus ?*item*?**

Ist *item* angegeben, wird das Fokuselement auf *item* gesetzt. Andernfalls das aktuelle Fokuselement oder **{}**, wenn keines gesetzt ist.

***pathname* heading *column* ?*-option ?value -option value...*?**

Abfragen oder Ändern der Überschriftenoptionen der angegebenen *column*. Gültige Optionen:

**-text *text***

Im Spaltenkopf anzuzeigender Text.

**-image *imageName***

Bild rechts neben dem Spaltenkopf.

**-anchor *anchor***

Ausrichtung des Überschriftentexts. Einer der üblichen Tk-Ankerwerte.

**-command *script***

Skript, das beim Drücken auf die Überschrift ausgewertet wird.

Verwenden Sie *pathname* **heading #0**, um die Baumspaltenüberschrift zu konfigurieren.

***pathname* identify *component x y***

Beschreibung des angegebenen *component* unter dem Punkt *x* und *y*, oder leere Zeichenkette, wenn dort kein solcher *component* liegt. *x* und *y* können jede von Tk_GetPixels akzeptierte Form haben. Folgende Unterbefehle werden unterstützt:

***pathname* identify region *x y***

Gibt einen der folgenden Werte zurück:

**heading**

Bereich der Baumüberschrift; [**pathname identify column** *x y*] liefert die Überschriftennummer.

**separator**

Zwischenraum zwischen zwei Spaltenüberschriften; [**pathname identify column** *x y*] liefert die Anzeigespaltenkennung der Überschrift links vom Trenner.

**tree**

Der Baumbereich.

**cell**

Eine Datenzelle.

***pathname* identify item *x y***

Element-ID des Elements an Position *y*.

***pathname* identify column *x y***

Anzeigespaltenkennung der Zelle an Position *x*. Die Baumspalte hat die ID **#0**.

***pathname* identify cell *x y***

Zellenkennung an Position *x y*. Eine Zellenkennung ist eine Liste aus Element-ID und Spalten-ID.

***pathname* identify element *x y***

Das Element an Position *x*,*y*.

***pathname* identify row *x y***

Veraltetes Synonym für *pathname* **identify item**.

Siehe Spaltenidentifikatoren zur Unterscheidung von Anzeige- und Datenspalten.

***pathname* index *item***

Ganzzahliger Index von *item* in der Kinderliste des Elternelements.

***pathname* insert *parent index* ?*-id id*? *options...***

Erstellt ein neues Element. *parent* ist die Element-ID des Elternelements oder die leere Zeichenkette **{}** für ein neues Element der obersten Ebene. *index* ist eine Ganzzahl oder **end** und legt die Einfügeposition in der Kinderliste von *parent* fest. Ist *index* negativ oder null, wird das neue Element am Anfang eingefügt; ist *index* größer oder gleich der aktuellen Kinderanzahl, am Ende. Ist **-id** angegeben, wird es als Elementkennung verwendet; *id* darf im Baum noch nicht existieren. Andernfalls wird eine neue eindeutige Kennung erzeugt.

*pathname* **insert** gibt die Elementkennung des neuen Elements zurück. Siehe Elementoptionen für verfügbare Optionen.

***pathname* item *item* ?*-option* ?*value -option value...*?**

Abfragen oder Ändern der Optionen von *item*. Ohne *-option* ein Dictionary der Option/Wert-Paare. Mit genau einem *-option* dessen Wert. Andernfalls Aktualisierung der Elementoptionen. Siehe Elementoptionen.

***pathname* move *item parent index***

Verschiebt *item* auf Position *index* in der Kinderliste von *parent*. Ein Element unter einen eigenen Nachkommen zu verschieben ist unzulässig.

Ist *index* negativ oder null, wird *item* an den Anfang verschoben; ist er größer oder gleich der Kinderanzahl, ans Ende.

***pathname* next *item***

Kennung des nächsten Geschwisterelements von *item* oder **{}**, wenn *item* das letzte Kind des Elternelements ist.

***pathname* parent *item***

ID des Elternelements von *item* oder **{}**, wenn *item* auf der obersten Hierarchieebene liegt.

***pathname* prev *item***

Kennung des vorherigen Geschwisterelements von *item* oder **{}**, wenn *item* das erste Kind ist.

***pathname* see *item***

Stellt sicher, dass *item* sichtbar ist: setzt alle Vorfahren von *item* auf **-open true** und scrollt bei Bedarf, sodass *item* im sichtbaren Baumbereich liegt.

***pathname* selection ?*selop itemList*?**

Verwaltet die Elementauswahl. Unabhängig von der Zellauswahl des Befehls **cellselection**. Ist *selop* nicht angegeben, Liste der ausgewählten Elemente. Andernfalls ist *selop* einer der folgenden:

***pathname* selection set *itemList***

*itemList* wird die neue Auswahl.

***pathname* selection add *itemList***

*itemList* zur Auswahl hinzufügen.

***pathname* selection remove *itemList***

*itemList* aus der Auswahl entfernen.

***pathname* selection toggle *itemList***

Auswahlzustand jedes Elements in *itemList* umschalten.

***pathname* set *item* ?*column*? ?*value*?**

Mit einem Argument Dictionary der Spalten/Wert-Paare für *item*. Mit zwei Argumenten aktueller Wert von *column*. Mit drei Argumenten wird der Wert von Spalte *column* in Element *item* auf *value* gesetzt. Siehe auch Spaltenidentifikatoren.

***pathName* tag *args...***

Verwaltet Tags. Tags können auf Elementen und Zellen gesetzt werden. Die Tag-Menge ist für Elemente und Zellen gemeinsam. Element-Tagging und Zellen-Tagging sind jedoch unabhängig (ein Tag auf einem Element gilt nicht automatisch für dessen Zellen). Zellen-Tags haben beim Zeichnen Vorrang vor Element-Tags. Folgende Unterbefehle werden unterstützt:

***pathName* tag add *tag items***

Fügt *tag* jedem der aufgeführten *items* hinzu. Ist *tag* für ein Element bereits vorhanden, bleiben die **-tags** unverändert.

***pathName* tag bind *tagName* ?*sequence*? ?*script*?**

Tk-Bindungsskript für die Ereignissequenz *sequence* auf Tag *tagName*. Bei X-Ereignissen an einem Element werden Bindungen für jedes **-tags** des Elements der Reihe nach wie in *bindtags(n)* ausgewertet. Lässt sich das Ereignis einer Zelle zuordnen (z. B. Mausereignisse), werden auch Bindungen der **-tags** der Zelle ausgewertet.

, und virtuelle Ereignisse gehen an das Fokuselement., und an das Element unter dem Mauszeiger. Andere Ereignistypen werden nicht unterstützt.

Das Bindungsskript *script* durchläuft **%-Ersetzungen** vor der Auswertung; siehe **bind**(n).

***pathName* tag cell *subcommand...***

Verwaltet Tags auf einzelnen Zellen. *cellList* kann eine Zelle oder eine Liste von Zellen sein.

***pathName* tag cell add *tag cellList***

Fügt *tag* jeder Zelle in *cellList* hinzu. Ist *tag* für eine Zelle bereits vorhanden, bleibt deren Tag-Liste unverändert.

***pathName* tag cell has *tagName* ?*cell*?**

Mit *cell*: 1 oder 0, ob die Zelle den Tag *tagName* hat. Ohne *cell*: Liste aller Zellen mit diesem Tag.

***pathName* tag cell remove *tag* ?*cellList*?**

Entfernt *tag* von jeder Zelle in *cellList*. Fehlt *cellList*, von jeder Zelle im Baum.

***pathName* tag configure *tagName* ?*option*? ?*value option value...*?**
Abfragen oder Ändern der Optionen von *tagName*. Mit *option/value*-Paaren werden Optionen gesetzt. Mit einer *option* deren Wert (oder leere Zeichenkette, wenn für *tagName* nicht gesetzt). Ohne weitere Argumente Dictionary der Optionen. Siehe Tag-Optionen.

***pathName* tag delete *tagName***
Löscht alle Tag-Informationen zu *tagName*. Entfernt den Tag von allen Elementen und Zellen und löscht zugehörige Daten wie Bindungen und Anzeigeinformationen. Gibt eine leere Zeichenkette zurück.

***pathName* tag has *tagName* ?*item*?**
Mit *item*: 1 oder 0, ob das Element den Tag *tagName* hat. Ohne *item*: Liste aller Elemente mit diesem Tag.

***pathName* tag names**
Liste aller vom Widget verwendeten Tags.

***pathName* tag remove *tag* ?*items*?**
Entfernt *tag* von jedem Element in *items*. Fehlt *items*, von jedem Element im Baum. Ist *tag* für ein Element nicht vorhanden, bleiben die **-tags** unverändert.

## Elementoptionen

Die folgenden Elementoptionen können bei **insert** und **item** angegeben werden.

**-text**
Textbeschriftung des Elements in der Baumspalte.

**-height**
Höhe des Elements als ganzzahliges Vielfaches von **-rowheight**. Standard 1.

**-image**
Tk-Bild neben der Beschriftung in der Baumspalte, positioniert gemäß **-imageanchor**.

**-imageanchor**
Anzeige von **-image** relativ zum Text. Standard **w**. Einer der üblichen Tk-Ankerwerte.

**-values**
Mit dem Element verknüpfte Werteliste.

Jedes Element sollte so viele Werte haben wie die Widget-Option **-columns** Spalten vorsieht. Gibt es weniger Werte als Spalten, gelten die restlichen als leer. Gibt es mehr Werte als Spalten, werden die überschüssigen ignoriert.

**-hidden**
Boolescher Wert, ob das Element angezeigt (**-hidden false**) oder ausgeblendet (**-hidden true**) wird. Ist ein Elternelement ausgeblendet, sind auch alle Nachkommen ausgeblendet.

**-open**
Boolescher Wert, ob die Kinder angezeigt (**-open true**) oder ausgeblendet (**-open false**) werden.

**-tags**
Liste der mit diesem Element verknüpften Tags.

## Tag-Optionen

Folgende Optionen können auf Tags gesetzt werden:

**-foreground**
Vordergrundfarbe des Textes.

**-background**
Hintergrundfarbe der Zelle oder des Elements.

**-font**
Schriftart beim Zeichnen des Textes.

**-image**
Bild der Zelle oder des Elements.

**-imageanchor**
Bildanker der Zelle oder des Elements.

**-padding**
Innenabstand der Zelle. Standard für Datenzellen {4 0}

**-stripedbackground**
Hintergrundfarbe für abwechselnde Zeilen, wenn **-striped** true ist.

Tags auf Zellen haben Vorrang vor Tags auf Elementen. Die Tag-Priorität folgt der Erstellungsreihenfolge: zuerst erstellte Tags haben höhere Priorität. Elementoptionen wie **-image** und **-imageanchor** haben Vorrang vor Tags.

## Bilder

Die Option **-image** auf einem Element und auf einem Element-Tag steuert das Bild neben der Beschriftung in der Baumspalte. Andere Zellen können Bilder über die Zellen-Tag-Option **-image** erhalten.

## Spaltenidentifikatoren

Spaltenkennungen können folgende Formen annehmen:

- Ein symbolischer Name aus der Liste **-columns**.
- Eine Ganzzahl *n* für die *n*-te Datenspalte.
- Eine Zeichenkette der Form **#***n*, wobei *n* eine Ganzzahl ist und die *n*-te Anzeigespalte bezeichnet.

Spaltenkennungen unterstützen dieselbe einfache Interpretation wie beim Befehl string index, mit einfacher ganzzahliger Indexarithmetik und Indexierung relativ zu **end**.

**HINWEIS:** Die **-values** eines Elements können in anderer Reihenfolge angezeigt werden als gespeichert.

**HINWEIS:** Spalte #0 bezeichnet immer die Baumspalte, auch wenn **-show tree** nicht gesetzt ist.

Eine *Datenspaltennummer* ist ein Index in die **-values**-Liste eines Elements; eine *Anzeigespaltennummer* ist die Spaltennummer im Baum, in der die Werte erscheinen. Baumbeschriftungen stehen in Spalte #0. Ist **-displaycolumns** nicht gesetzt, wird Datenspalte *n* in Anzeigespalte **#***n+1* angezeigt. Erneut: **Spalte #0 ist immer die Baumspalte**.

## Virtuelle Ereignisse

Das Treeview-Widget erzeugt folgende virtuelle Ereignisse.

**<>**
Wird ausgelöst, wenn sich Auswahl oder Zellauswahl ändert. Kann auch ausgelöst werden, wenn die Auswahl betroffen, aber nicht tatsächlich geändert wurde. Mehrere Auswahländerungen können vor der Ereignisverarbeitung erfolgen und mehrere Ereignisse mit gleicher sichtbarer Auswahl erzeugen.

**<>**
Kurz bevor das Fokuselement auf **-open true** gesetzt wird.

**<>**
Kurz nachdem das Fokuselement auf **-open false** gesetzt wurde.

Die Widget-Befehle **focus** und **selection** liefern das betroffene Element bzw. die betroffenen Elemente.

## Stiloptionen

Der Klassenname für **ttk::treeview** ist **Treeview**. Der Klassenname der Treeview-Überschrift ist **Heading**. Der Klassenname eines Treeview-Elements ist **Item**. Der Klassenname einer Treeview-Zelle ist **Cell**.

Dynamische Zustände: **disabled**, **selected**.

**Treeview**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-background** *color*

**-fieldbackground** *color*

**-font** *font*

**-foreground** *color*

**-indent** ***amount***
Legt fest, wie weit Elemente von ihren Eltern eingerückt sind. Standard 20 Pixel. Der Wert kann jede von Tk_GetPixels akzeptierte Form haben.

**-columnseparatorwidth** ***amount***
Breite der Spaltentrenner. Standard 1 Pixel. Der Wert kann jede von Tk_GetPixels akzeptierte Form haben.

**-rowheight** ***amount***
Standardhöhe eines Elements. Standard 20 Pixel. Der Wert kann jede von Tk_GetPixels akzeptierte Form haben. Ist **-rowheight** im Stil nicht gesetzt, wird sie durch Messen eines Elements und eines Zellenlayouts mit den Stileinstellungen bestimmt — damit werden Schriftart sowie Fokusring oder Innenabstand des Themes übernommen. **-rowheight** muss ggf. gesetzt werden, damit eine Zeile groß genug für Bilder ist.

Beispiel zum Setzen von **-rowheight**, angepasst an eine Schriftart analog zum Standardwert:

```
ttk::style configure Treeview \
    -rowheight [expr {[font metrics font -linespace] + 2}]
```

**-stripedbackground** *color*

**Heading**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-background** *color*

**-font** *font*

**-relief** *relief*

**Item**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-foreground** *color*

**-indicatormargins** *padding*

**-indicatorsize** *amount*

**-padding** *padding*

**Cell**-Styling-Optionen, die mit ttk::style konfiguriert werden können, sind:

**-padding** *padding*

Einige Optionen sind nur für bestimmte Themes verfügbar.

Siehe die Handbuch-Seite ttk::style für Informationen zur Konfiguration von ttk-Stilen.

*Copyright (c) 2004 Joe English*

# ttk_vsapi

*Stichwörter:* ttk_vsapi, style, theme, appearance, windows

## Name

ttk_vsapi - Microsoft-Visual-Styles-Element definieren

## Übersicht

**ttk::style element create** *name* **vsapi** *className partId* ?*stateMap*? ?*options*?

## Beschreibung

Die **vsapi**-Elementfabrik erstellt im aktuellen Theme ein neues Element, dessen visuelle Darstellung mit der Microsoft Visual Styles API gezeichnet wird, die ttk für die nativen Stile unter MS-Windows verwendet. Diese Fabrik erlaubt, beliebige Visual-Styles-Teile als Ttk-Elemente zu deklarieren, die dann in ein Style-Layout eingebunden werden können, um das Erscheinungsbild von Ttk-Widgets zu ändern.

Die Parameter *className* und *partId* sind erforderlich und legen die Visual-Styles-Klasse und den Teil gemäß der Microsoft-Dokumentation fest. *stateMap* kann angegeben werden, um Ttk-Zustände auf Visual-Styles-API-Zustände abzubilden (siehe Zustandszuordnung).

## Optionen

Gültige *options* sind:

**-padding** ***padding***

Legt den Innenabstand des Elements fest. *padding* ist eine Liste mit bis zu vier Ganzzahlen für die Abstände links, oben, rechts und unten. Sind weniger als vier Elemente angegeben, gilt *bottom* = *top*, *right* = *left* und *top* = *left*. Eine Liste mit drei Zahlen gibt links, vertikal und rechts an; zwei Zahlen horizontal und vertikal; eine Zahl denselben Abstand rundherum. Diese Option darf nicht mit anderen Optionen kombiniert werden.

**-margins** ***padding***

Legt den Außenabstand des Elements fest. *padding* ist eine Liste mit bis zu vier Ganzzahlen für die Abstände links, oben, rechts und unten. Diese Option darf nicht mit anderen Optionen kombiniert werden.

**-width *width***

Legt die Breite des Elements fest. Wenn diese Option gesetzt ist, wird die Visual Styles API nicht nach der empfohlenen Größe des Teils abgefragt. Wenn diese Option gesetzt ist, sollte auch **-height** gesetzt werden. Die Optionen **-width** und **-height** dürfen nicht mit **-padding** oder **-margins** kombiniert werden.

**-height *height***

Legt die Höhe des Elements fest. Siehe die Hinweise zu **-width**.

## Zustandszuordnung

Der Parameter *stateMap* ist eine Liste von ttk-Zuständen und dem zugehörigen Visual-Styles-API-Zustandswert. Damit kann sich das Erscheinungsbild des Elements bei Widget-Zustandsänderungen anpassen (z. B. aktiv oder gedrückt). Die Liste sollte wie beim Befehl ttk::style map beschrieben aufgebaut sein; das letzte Paar in der Liste sollte der Standardzustand sein und ist typischerweise eine leere Liste und 1. Leider haben alle Visual-Styles-Teile unterschiedliche Zustandswerte; diese müssen in der Microsoft-Dokumentation oder eher in den Header-Dateien nachgeschlagen werden. Ursprünglich war *tmschema.h* zu verwenden; in neueren Versionen des Windows Development Kit ist es *vssym32.h*.

Wenn kein Parameter *stateMap* angegeben wird, gilt implizit die Standardzuordnung `{{} 1}`

## Beispiele

Erstellen eines korrekt thematisierten Schließen-Buttons durch Änderung des Layouts eines ttk::button(n). Verwendet den WINDOW-Teil WP_SMALLCLOSEBUTTON; wie dokumentiert werden die Zustände CBS_DISABLED, CBS_HOT, CBS_NORMAL und CBS_PUSHED von ttk-Zuständen abgebildet.

```
ttk::style element create smallclose vsapi WINDOW 19 \
    {disabled 4 pressed 3 active 2 {} 1}
ttk::style layout CloseButton {CloseButton.smallclose -sticky news}
pack [ttk::button .close -style CloseButton]
```

Ändern des Erscheinungsbilds eines ttk::checkbutton(n), um den Explorer-Pin-Teil EBP_HEADERPIN zu verwenden.

```
ttk::style element create pin vsapi EXPLORERBAR 3 {
    {pressed !selected} 3
    {active !selected} 2
    {pressed selected} 6
    {active selected} 5
    {selected} 4
    {} 1
}
ttk::style layout Explorer.Pin {Explorer.Pin.pin -sticky news}
pack [ttk::checkbutton .pin -style Explorer.Pin]
```

*Copyright (c) 2008 Pat Thoyts*

# ttk_widget

*Stichwörter:* ttk_widget, state, configure, option

## Name

ttk::widget - Standardoptionen und -befehle der Tk-Themed-Widgets

## Beschreibung

Diese Handbuchseite beschreibt gemeinsame Widget-Optionen und -befehle.

## Standardoptionen

Die folgenden Optionen werden von allen Tk-Themed-Widgets unterstützt:

**-class (undefined/undefined)**

Legt die Fensterklasse fest. Die Klasse wird beim Abfragen der Optionendatenbank für die übrigen Optionen des Fensters, zur Bestimmung der Standard-Bindtags und zur Auswahl des Standard-Layouts und -Stils verwendet. Dies ist eine schreibgeschützte Option: sie darf nur bei der Fenstererzeugung angegeben werden und kann nicht mit dem Widget-Befehl **configure** geändert werden.

**-cursor (cursor/Cursor)**

Legt den für das Widget zu verwendenden Mauszeiger fest. Siehe Tk_GetCursor und *cursors(n)* im Tk-Referenzhandbuch für die zulässigen Werte. Ist er die leere Zeichenkette (Standard), wird der Zeiger vom Eltern-Widget geerbt.

**-takefocus (takeFocus/TakeFocus)**

Legt fest, ob das Fenster den Fokus bei Tastaturtraversierung annimmt. Entweder **0**, **1**, ein Befehlspräfix (dem der Widget-Pfad angehängt wird und der **0** oder **1** zurückgeben soll) oder die leere Zeichenkette. Siehe *options(n)* im Tk-Referenzhandbuch für die vollständige Beschreibung.

**-style (style/Style)**

Kann verwendet werden, um einen benutzerdefinierten Widget-Stil anzugeben.

## Optionen für scrollbare Widgets

Die folgenden Optionen werden von Widgets unterstützt, die mit einer Scrollbar gesteuert werden können. Siehe *scrollbar(n)* für Details.

**-xscrollcommand (xScrollCommand/ScrollCommand)**

Ein Befehlspräfix zur Kommunikation mit horizontalen Scrollbars. Ändert sich die Ansicht im Widget-Fenster, erzeugt das Widget einen Tcl-Befehl durch Anhängen des Scroll-Befehls und zweier Zahlen. Jede Zahl ist ein Bruchteil zwischen 0 und 1 und gibt eine Position im Dokument an; 0 ist der Anfang, 1 das Ende. Der erste Bruchteil ist die erste im Fenster sichtbare Information, der zweite die Information unmittelbar nach dem letzten sichtbaren Teil.

Typischerweise besteht **-xscrollcommand** aus dem Pfadnamen einer scrollbar gefolgt von "set", z. B. ".x.scrollbar set". Die Scrollbar wird dann bei jeder Ansichtsänderung aktualisiert.

Ist diese Option die leere Zeichenkette (Standard), wird kein Befehl ausgeführt.

**-yscrollcommand (yScrollCommand/ScrollCommand)**

Ein Befehlspräfix zur Kommunikation mit vertikalen Scrollbars. Siehe die Beschreibung von **-xscrollcommand** oben.

## Label-Optionen

Die folgenden Optionen werden von Labels, Buttons und anderen button-ähnlichen Widgets unterstützt:

**-anchor (anchor/Anchor)**

Legt fest, wie die Information im Widget relativ zu den inneren Rändern positioniert ist. Zulässige Werte: **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw** und **center**. Ist sie nicht angegeben, gilt der Theme-Standard. Siehe auch **-justify** (bei Widgets mit dieser Option).

**-compound (compound/Compound)**

Legt fest, wie Bild und Text zueinander stehen, wenn sowohl **-text** als auch **-image** gesetzt sind. Ist die Option die leere Zeichenkette (Standard), gelten die Regeln im Abschnitt "Elements" von *ttk::intro(n)*. Weitere gültige Werte:

**text**

Nur Text anzeigen.

**image**

Nur Bild anzeigen.

**center**

Text zentriert über dem Bild anzeigen.

**top**

Bild oberhalb des Textes anzeigen.

**bottom**

Bild unterhalb des Textes anzeigen.

**left**

Bild links vom Text anzeigen.

**right**

Bild rechts vom Text anzeigen.

**none**

Bild anzeigen, falls vorhanden, sonst Text.

**-font (**font**/Font)**

Schriftart für den vom Widget angezeigten Text.

**-foreground (textColor/TextColor)**

Vordergrundfarbe des Widgets. Ohne Angabe gilt der Theme-Standard.

**-image (image/Image)**

Legt ein anzuzeigendes Bild fest. Liste mit mindestens einem Element; das erste ist der Standard-Bildname. Der Rest ist eine Folge von *statespec / value*-Paaren wie bei **style map**, die verschiedene Bilder für bestimmte Widget-Zustände festlegen. Alle Bilder in der Liste sollten dieselbe Größe haben.

**-justify (justify/Justify)**

Gibt es mehrere Textzeilen, legt dies fest, wie die Zeilen zueinander angeordnet werden. Einer von **left**, **center** oder **right**. Ist die Option nicht angegeben, gilt der Theme-Standard. Siehe auch **-anchor** (bei Widgets mit dieser Option).

**-padding (padding/Padding)**

Legt den inneren Abstand des Widgets fest. Die Liste enthält bis zu vier Längenangaben *left top right bottom*. Bei weniger als vier Elementen gilt: *bottom* = *top*, *right* = *left*, *top* = *left*. Drei Zahlen: links, vertikal, rechts; zwei Zahlen: horizontal und vertikal; eine Zahl: gleicher Abstand ringsum.

**-text (text/Text)**

Zeichenkette im Widget (sofern nicht durch **-textvariable** überschrieben, bei Widgets mit dieser Option).

**-textvariable (textVariable/Variable)**

Name einer globalen Variable, deren Wert anstelle der **-text**-Resource verwendet wird.

**-underline (underline/Underline)**

Wenn gesetzt, Index (0-basiert) des zu unterstreichenden Zeichens in der Textzeichenkette. **end** entspricht dem letzten Zeichen, **end**-1 dem vorletzten usw. Das unterstrichene Zeichen dient der Mnemonik-Aktivierung.

**-width (width/Width)**

Ist der Wert größer als null, legt er den Platz in Zeichenbreiten für die Textbeschriftung fest. Ist er kleiner als null, eine Mindestbreite. Bei null oder ohne Angabe wird die natürliche Textbreite verwendet. Manche Themes setzen im Stil eine von Null verschiedene **-width**.

**-wraplength (wrapLength/WrapLength)**

Maximale Zeilenlänge. Der Wert kann jede von Tk_GetPixels akzeptierte Form haben. Ist die Option kleiner oder gleich null, erfolgt kein automatischer Umbruch; sonst wird der Text so geteilt, dass keine Zeile länger ist.

## Entry-Optionen

Die folgenden Optionen werden von entry, spinbox und combobox unterstützt:

**-placeholder (placeHolder/PlaceHolder)**

Hilfetext, der angezeigt wird, wenn sonst kein Text sichtbar ist, d. h. wenn das Widget leer ist. Der Platzhaltertext wird mit **-font**, **-justify** und **-placeholderforeground** dargestellt.

**-placeholderforeground (placeHolderForeground/PlaceHolderForeground)**

Vordergrundfarbe des Platzhaltertexts.

## Kompatibilitätsoptionen

Diese Option ist nur bei Themed-Widgets mit "entsprechenden" klassischen Tk-Widgets verfügbar.

**-state (state/State)**

Kann auf **normal** oder **disabled** gesetzt werden, um das **disabled**-Zustandsbit zu steuern. Schreiboption: das Setzen ändert den Widget-Zustand; der Widget-Befehl **state** beeinflusst **-state** nicht.

## Befehle

***pathName* cget *option***

Gibt den aktuellen Wert der Konfigurationsoption *option* zurück.

***pathName* configure ?*option*? ?*value option value* ...?**

Fragt die Konfigurationsoptionen des Widgets ab oder ändert sie. Bei *option*-*value*-Paaren werden die Optionen gesetzt; der Befehl gibt dann eine leere Zeichenkette zurück. Ist nur *option* ohne *value* angegeben, gibt er eine Liste zur benannten Option zurück: Optionsname, Datenbankname, Datenbankklasse, Standardwert und aktueller Wert. Ohne *option* eine Liste aller verfügbaren Optionen für *pathName*.

***pathName* identify element *x y***

Gibt den Namen des Elements unter dem Punkt *x*, *y* zurück, oder eine leere Zeichenkette, wenn kein Element getroffen wird. *x* und *y* sind Pixelkoordinaten relativ zum Widget. Manche Widgets akzeptieren weitere **identify**-Unterbefehle in ihrer eigenen Dokumentation.

***pathName* instate *statespec* ?*script*?**

Prüft den Widget-Zustand. Ohne *script*: 1, wenn der Zustand *statespec* entspricht, sonst 0. Mit *script* äquivalent zu

```
if {[pathName instate stateSpec]} script
```

***pathName* state ?*stateSpec*?**

Ändert oder fragt den Widget-Zustand ab. Mit *stateSpec* werden die Flags gesetzt; ein führendes Ausrufezeichen löscht das Flag. Rückgabe: neue Zustandsspezifikation der geänderten Flags:

```
set changes [pathName state spec]
pathName state $changes
```

stellt den ursprünglichen Zustand wieder her. Ohne *stateSpec*: Liste der aktuell aktiven Zustandsflags.

***pathName* style**

Gibt den vom Widget verwendeten Stil zurück.

***pathName* xview *args***

Fragt die horizontale Position des Inhalts im Widget-Fenster ab oder ändert sie. Mögliche Formen:

***pathName* xview**

Liste mit zwei Elementen, je ein Bruchteil zwischen 0 und 1; beschreibt den sichtbaren horizontalen Bereich. Z. B. .2 und .6: 20 % links außerhalb, 40 % sichtbar, 40 % rechts außerhalb — dieselben Werte wie an Scrollbars über **-xscrollcommand**.

***pathName* xview *index***

Passt die Ansicht so an, dass der Inhalt *index* am linken Fensterrand liegt.

***pathName* xview moveto *fraction***

Passt die Ansicht so an, dass der Inhalt an Position *fraction* am linken Rand erscheint. *fraction* zwischen 0 und 1.

***pathName* xview scroll *number what***

Verschiebt die Ansicht horizontal gemäß *number* und *what*. *number* muss Ganzzahl oder Gleitkommazahl sein; bei Gleitkommazahl wird auf Ganzzahl gerundet (weg von 0). *what* ist **pages** oder **units**. Bei **pages** um *number* Bildschirmseiten; negativ: weiter links sichtbar, positiv: weiter rechts. Bei **units** um *number* durchschnittliche Zeichenbreiten.

***pathName* yview *args***

Wie **xview**, aber vertikal. Mögliche Formen:

***pathName* yview**

Zwei Bruchteile für den vertikal sichtbaren Bereich; dieselben Werte wie über **-yscrollcommand**.

***pathName* yview *index***

Inhalt *index* am oberen Fensterrand.

***pathName* yview moveto *fraction***

Element an Position *fraction* am oberen Rand. *fraction* zwischen 0 und 1.

***pathName* yview scroll *number what***

Verschiebt vertikal. *number* Ganzzahl oder Gleitkommazahl (bei Gleitkomma gerundet). *what* **pages** oder **units**. Bei **pages** um *number* Bildschirmseiten; negativ: weiter oben, positiv: weiter unten. Bei **units** um *number* durchschnittliche Zeichenbreiten.

## Widget-Zustände

Der Widget-Zustand ist eine Bitmap-Menge unabhängiger Zustandsflags. Dazu gehören:

**active**

Mauszeiger über dem Widget; Drücken einer Maustaste löst eine Aktion aus (auch "prelight" (Gnome), "hot" (Windows), "hover").

**disabled**

Widget ist programmgesteuert deaktiviert (auch "unavailable", "inactive").

focus

Widget hat Tastaturfokus.

**pressed**

Widget wird gedrückt (Motif: "armed").

**selected**

"An", "true" oder "current" bei checkbutton und radiobutton.

**background**

Windows und Mac kennen ein "aktives" bzw. Vordergrundfenster. **background** ist für Widgets in Hintergrundfenstern gesetzt, in Vordergrundfenstern gelöscht.

**readonly**

Benutzer darf den Wert nicht ändern.

**alternate**

Widget-spezifisches alternatives Anzeigeformat, z. B. Tristate bei checkbutton/radiobutton oder Buttons mit **-default active**.

**invalid**

Widget-Wert ist ungültig (z. B. Scale außerhalb der Grenzen, Entry-Validierung fehlgeschlagen).

**hover**

Mauszeiger im Widget. Ähnlich **active**; in manchen Themes zusätzliches visuelles Feedback neben dem aktiven Element.

**user1-user6**

Frei für andere Zwecke nutzbar

Eine *Zustandsspezifikation* oder *stateSpec* ist eine Liste von Zustandsnamen, optional mit Ausrufezeichen (!) für "aus".

## Beispiele

```
set b [ttk::button .b]

# Widget deaktivieren:
$b state disabled

# Widget nur aufrufen, wenn gedrückt und nicht deaktiviert:
$b instate {pressed !disabled} { .b invoke }

# Widget wieder aktivieren:
$b state !disabled
```

*Copyright (c) 2004 Joe English*

# Lizenz

Deutsche Übersetzung © 2026 Gregor Ebbing.

Die Seiten wurden aus den Originalen nach Markdown umgewandelt und ins Deutsche übersetzt; Auszeichnung ohne Platz im Format (Verweise ohne Ziel, Layoutbilder) kann dabei wegfallen. Das sind die Änderungen; für Teile unter der Mozilla Public License 1.1 (tDOM) ist es die Beschreibung der Änderungen nach Abschnitt 3.3.

Die Übersetzung steht unter denselben Bedingungen wie das jeweilige englische Original. Diese Bedingungen folgen. Die Copyright-Angaben der Originalseiten stehen bei jeder Seite.

## Lizenzbedingungen Tcl

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, ActiveState Corporation and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7014 (b) (3) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

## Lizenzbedingungen Tk

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, ActiveState Corporation, Apple Inc. and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (b) (3) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

# Stichwortverzeichnis

## A

abort 34, 125
about dialog 722
absolute file name 144
abstract 12
abstract class 12
access mode 286
add 185
after 13
alert 696
alias 196, 340, 613
anonymous function 15
antialiasing 722
API 321, 409
appearance 766, 792, 804
append 15, 209, 286, 493
Apple event 722
application 93, 372, 520, 685, 737
application name 720
apply 15
architecture 302, 303
argument 15, 306, 372, 737
arithmetic 126, 390
array 17
aspect ratio 740
assign 209
asynchronous I/O 356, 418, 424
atom 731
attributes 136
auto-exec 211
auto-load 211, 298, 299
auto-loading 340, 613
auto_mkindex 340, 613

## B

background 307
background error 20, 726
backslash 8
backslash substitution 367
base class 284
beep 441
bell 441
bgerror 20
binary 21

binary code 225, 415
binary search 233
bind 356, 442, 565
binding 442, 454, 530, 565
bindtags 454
bisque 648
bitmap 455, 521
blocking 40, 133
boolean 126, 147, 184, 429
break 34, 337
build-info 35
buildinfo 35
busy 456
button 460, 752, 754, 772, 782
bytecode 390

**C**

call 280, 352, 371
callback 36, 697
cancel 13
canvas 464, 668
case conversion 362
catch 38, 337
cd 39
cell 557
chan 40
channel 40, 133, 321, 356, 409
character 362
check 754
checkbutton 487
child 307
child interpreter 196, 340, 613
children 731
choice 755
chooseColor 491
chooseDirectory 492
class 12, 59, 62, 77, 95, 254, 284, 355, 637, 731
class variable 62
classvariable 62
cleanup 412
clear 493, 682
clipboard 493
clock 63
clone 87
close 75
color 491, 495, 648, 654
color selection 491

colors 495
command 8, 186, 220, 248, 268, 336, 404, 752
command substitution 367
comment 8
compare 126, 362
compiler 390
compress 430, 435
concat 76
concatenate 76, 117
conditional 184
configurable 77
configuration 77
configure 806
connection 356
console 514
const 82
constant 82
container 664, 765, 770
content 664
context 419, 420
continue 83, 337
conversion specifier 149, 348
cookie 84
cookiejar 84
copy 87
copy files 136
coroutine 88
cpu architecture 302, 303
create 108, 286
ctype 362
current directory 144
cursor 517
cursors 517

**D**

database 634
date 63
dde 93, 685
debug 253
decompress 435
default 752
define 95, 530
definition 95
deflate 435
deiconify 740
delay 13
delete 336

delete files 136
destroy 520
detach 307
dialog 491, 492, 521, 547
dict 108
dictionary 108
directory 136, 492
domain name 356
duplication 87
dynamic library 225

**E**

element 208, 209, 210, 218, 222, 223, 228, 229, 230, 231, 233, 236, 238, 240
element names 17
else 184
encoding 113
end of file 40, 133
end of line 133
ensemble 268, 321, 409
entry 522, 687, 755, 758, 789
environment 390, 726
eof 116
equal 362
error 38, 116, 337, 390, 397, 412, 413
eval 117
evaluate 117
event 157, 418, 424, 442, 454, 530, 565
events 40, 539
exception 38, 116, 337, 397, 412
exec 119
execute 119
exist 154
exit 125
exported 268
expr 126
expression 126, 248

**F**

false 184
fblocked 132
fconfigure 132
fcopy 133
file 136, 154, 286, 299, 360
file selection dialog 552
fileevent 144
filename 144

filesystem 430
filter 108
final 412
floating point 153
flush 147, 418
focus 539, 541
focus model 740
focusNext 541
font 541, 547
font chooser 547
font panel 547
font selection 547
fontchooser 547
for 147
foreach 148, 224
format 21, 149, 493, 682
fpclassify 153
frame 420, 550, 765, 770
fuzzy comparison 126

**G**

generator 88
geometry 731, 740
geometry management 649
geometry manager 557, 643, 664
getOpenFile 552
gets 154
glob 154
global 157, 420, 422
grab 555
graphics 668
grid 557, 740
group 740
groupbox 770
grow box 788
gzip 435

**H**

handle 530
handler 418, 682
height 563, 632, 664, 731
history 157
host 356
http 159

**I**

i18n 254
ICCCM 682

icon 722, 740
iconify 740
identifier 731
idle 418
idle callback 13
idna 183
if 184
image 455, 563, 632, 654, 697, 766
incr 185
increment 185
increments 740
index 218, 228, 231, 236, 238, 298, 299, 362
inflate 435
info 186
information 186, 731
input 40
insert 222
integer value 126, 362
interactive 514
internal 268
internationalization 254
internet 84, 159, 183
interp 196
interpreter 186, 372, 514, 737
interpreters 731
introspection 186, 352
iterate 108
iteration 83, 147, 148, 224

**J**

join 76, 208

**K**

keyboard 539
keyboard events 456, 555
keyboard traversal 541
keysym 565
keysyms 565

**L**

l10n 254
label 600, 770
labelframe 602
lambda 15
lappend 209
lassign 209
ledit 210
length 223

level 186, 419, 420
library 211
lindex 218
linear search 233
link 220
linsert 222
list 76, 148, 208, 209, 210, 218, 222, 223, 224, 228, 229, 230, 231, 233, 236, 238, 240, 361
list box 755
listbox 605
llength 223
lmap 224
load 225, 340, 613
loadTk 613
localization 254
location 557, 643, 664
lookup 108
loop 34, 83, 147, 148, 224, 429
lower 614
lpop 228
lrange 229
lremove 230
lrepeat 231
lreplace 231
lreverse 233
lsearch 233
lseq 236
lset 238
lsort 240

**M**

map 108, 224
mapped 731
match 233, 310, 328, 333, 362, 369
mathfunc 243
mathop 248
memory 253
menu 615, 667, 772
menubutton 627
message 254, 629
message box 631
messageBox 631
metaclass 12, 59, 355
method 95, 220, 266, 280
method chain 280
method visibility 266
modal 521

modules 400
move files 136
msgcat 254
multiple 209
my 266

**N**

name 93, 136, 637, 685
namespace 82, 157, 186, 268, 336, 419, 420, 422
network address 356
next 280
non-blocking 40, 286
non-existent command 413
nonblocking 133
notify 696
NSImage 722
nsimage 632

**O**

object 12, 36, 59, 77, 87, 95, 186, 220, 254, 266, 284, 352, 355
obscure 614, 673
offset 40
open 286
operating system 302, 303
operator 248
option 495, 517, 634, 754, 782, 806
option menu 636
optionMenu 636
options 637
order 240
output 40, 668
output channels 514
own 682

**P**

pack 557, 643
package 294, 298, 299, 400
packagens 298
packer 643
palette 648
pane 773
panedwindow 649
parcel 643
parent 731
parent interpreter 196, 340, 613
parse 348
parsing 328
path name 731

pattern 154, 233, 328, 333, 362
performance measurement 398
permissions 286
photo 654
pid 299
pipeline 119, 286, 299
pixmap theme 766
pkgMkIndex 299
place 664
platform 302, 303
platform-specific 492
platform_shell 303
pointer events 456, 555
pop 228
popup 667
portability 144
position 740
POSIX 390
precision 390
prefix 304, 321, 409
print 668, 720
priority 634
private method 266
proc 306
procedure 15, 82, 157, 186, 306, 337, 420, 422
process 125, 286, 307
process identifier 299
prompt 372, 737
propagation 557, 643
properties 77
public method 266
puts 309
pwd 310

**Q**

queue 228
quoting 223, 333, 367

**R**

radiobutton 669
raise 673
range 229
re_syntax 310
read 133, 321, 354, 404
readable 40
record 157
recursion 371

redirection 119
refchan 321
reflection 321
regexp 328
registry 330
regsub 333
regular expression 233, 310, 328, 333, 369
relative file name 144
remote execution 93, 685
remove 228, 230, 417
rename 336, 404
rename files 136
replace 210, 231, 238
reporting 20, 726
resource management 412
result 337
retrieve 634
return 337
reverse 233, 362
ring 441
root class 284
rubber sheet 664

**S**

safe 340
safe interpreter 196, 225, 340, 415, 613
scale 673, 783
scan 21, 348
screen 731
script 8, 117, 360, 397, 398
script file 372, 737
scrollbar 677, 785
search 17, 233
security 685
security policy 84, 159
seek 40, 351
selection 492, 493, 682
self 352
send 685, 720
separator 208, 788
serial 286
set 209, 238, 354
shared library 225, 415
shell 372, 737
single instance 355
singleton 355
size 557, 643, 740

sizegrip 788
sleep 13
slider 673, 783
slot 95
socket 159, 356
sort 240
source 340, 360, 613
spinbox 687, 789
split 361
splitting 328
sprintf 149
stack 228
stack frame 419
stacking order 614, 673
standard option 637
stat 136
state 806
stdio 40
string 149, 233, 310, 328, 361, 362
style 766, 792, 804
sublist 229
subprocess 119, 390
subst 367
substitution 8, 149, 333, 367
switch 369, 637
sysnotify 696, 720
systray 697, 720

**T**

tab 773
table lookup 304
tag 454
tail recursion 371
tailcall 371
target 682
Tcl 8
tclsh 372
tcltest 374
tclvars 390
tcp 356
tell 40, 396
test 374, 429
test harness 374
test suite 374
text 254, 668, 698, 726
text box 755
text field 758, 789

theme 766, 792, 804
throw 397
time 13, 63, 397, 398
timerate 398
title 740
tk 720
tk_mac 722
tkerror 726
tkvars 698, 726
tkwait 728
tm 400
toggle 754
toolkit 737
top-level 539, 541
top-level window 740
toplevel 729, 737
trace 404
transchan 409
transformation 409
translation 133, 254
trough 673, 783
true 184
try 412
ttk_button 752
ttk_checkbutton 754
ttk_combobox 755
ttk_entry 758
ttk_frame 765
ttk_image 766
ttk_intro 767
ttk_label 769
ttk_labelframe 770
ttk_menubutton 772
ttk_notebook 773
ttk_panedwindow 778
ttk_progressbar 780
ttk_radiobutton 782
ttk_scale 783
ttk_scrollbar 785
ttk_separator 788
ttk_sizegrip 788
ttk_spinbox 789
ttk_style 792
ttk_treeview 794
ttk_vsapi 804
ttk_widget 806

type 493, 682
types of images 563, 632

**U**

unicode 113
units 740
unknown 211, 413
unload 415
unloading 415
unset 404, 417
update 108, 418
uplevel 419
upvar 420
user 136, 390

**V**

value 185
variable 8, 15, 62, 82, 157, 185, 186, 209, 268, 354, 404, 417, 419, 420, 422, 424, 728
variable substitution 367
variables 390, 726
version 294, 298, 299, 726
virtual event 530
virtual root 731
visibility 728
volume-relative file name 144
vwait 424

**W**

wait 307, 424, 728
while 429
whitespace 211
widget 460, 464, 487, 522, 550, 600, 602, 605, 615, 627, 629, 649, 669, 673, 677, 687, 698, 729, 752, 754, 755, 758, 765, 770, 772, 782, 783, 785, 788, 789
width 563, 632, 664, 731
window 456, 514, 520, 555, 728, 731
window manager 539, 740
windows 804
winfo 731
wish 737
wm 740
word 211, 362
working directory 39, 310
writable 40
write 354, 404
www 84, 159, 183

**Z**

zip 430
zipfs 372, 430
zlib 435